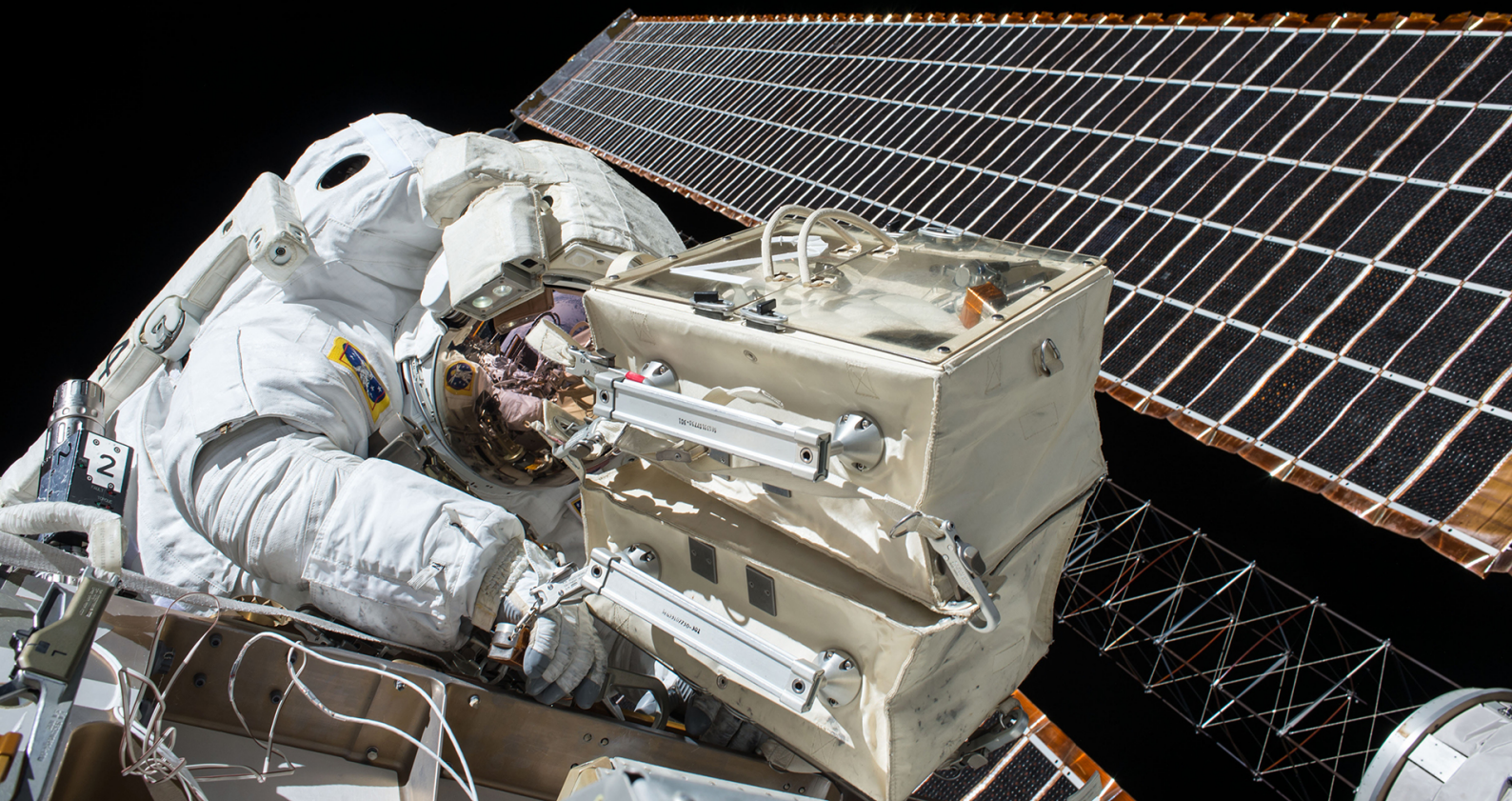




计算材料与拓扑磁性 研究工作手册

博士研究生申请科研手册 · 方法、软件、代码、绘图与心得

作者：马睿骁

组织：浙江省量子态调控与光场操控重点实验室 量子物态调控研究所

时间：2026 年 9 月

微信：19800353973

邮箱：2024210104023@mails.zstu.edu.cn

目录

第一部分 第一性原理计算教程	1
第一章 VASP 基础计算	2
1.1 vasp 计算态密度	2
1.2 vasp 计算投影能带	4
1.3 vasp 计算能带	10
1.4 vasp 计算磁耦合参数大小	11
第二章 磁性参数与 DMI	15
2.1 vasp 计算 DMI 大小	15
2.2 (文献整理) 块体材料 DMI 的第一性原理提取方法	17
第三章 结构建模与计算方法	26
3.1 vasp 结构优化	26
第二部分 软件学习	28
第四章 Wannier90 与电子结构工具	29
4.1 Wannier90 学习笔记	29
4.2 以 Bi 烯为例的 wannier90 表面态计算	39
4.3 wannier90 汇总	45
4.4 Wannier90 的表面态的参数选择标准 1_示例文件的参数注释	45
4.5 wannier90 编译	52
4.6 以 Bi 烯为例的 wannier90 表面态计算	53
4.7 pwy90 推荐能量窗口测试	59
4.8 pyw90 的 auto 功能介绍	65
4.9 pyw90 的 cmp 功能介绍	77
4.10 pyw90 的 eig 功能介绍	82
4.11 pyw90 的 pre 功能介绍	87
4.12 pyw90 的简介	92
4.13 wannier90 的能带拟合步骤	95
4.14 pyw90 的测试用例	97
4.15 pyw90 的测试用例 2_本地运行	101
4.16 Wannier Tools 的代码解释	102
4.17 wannierberrier 的安装	102
4.18 wansymm 的对称化功能	103
4.19 利用 Wannier Tools 的对称化 hr.dat	104
第五章 紧束缚、拓扑与磁性软件	107
5.1 tb2j 计算磁参数的 DMI 计算	107

5.2 TB2J 新版测试	107
5.3 TB2J 新版测试	110
5.4 pythTB 的学习 _Parameterized_TBModel	112
5.5 pythTB 的学习 _lattice 类	115
5.6 pythTB 的学习 _mesh 类	118
5.7 pythTB 的学习 _tb_model to TBModel in v2.0	120
5.8 pythTB 的学习 _wf_array to in v2.0WFArray	127
5.9 vasp2kp 的学习 _1	129
5.10 ir2TB 的学习	132
5.11 irvsp 的学习	134
5.12 (原创脚本) 角态验证 1-配置 pybinding 环境-Ubuntu 系统配置	136
5.13 ubuntu 中使用 pybinding	138
第六章 科研写作、建模与网站工具	139
6.1 网站维护与操作手册	139
6.2 (原创自研) 网站维护记录 1	141
6.3 (原创自研) 网站维护记录 2	151
6.4 (原创自研) 网站维护记录 3	166
6.5 (原创自研) 网站维护记录 4-更新结束	178
6.6 (原创自研) 网站维护记录 5-搜索功能与界面更新	188
6.7 (建模流程)Materials Studio 2023 手动搭建 Cs3V9Te13 晶体结构	200
第三部分 论文使用代码	213
第七章 论文最终代码与结果复现	214
7.1 FES 体系自旋陈数、金属性阻碍与相界代码	214
7.2 Lieb 体系自旋陈数、解析质量与相界代码	217
7.3 TTS 体系全局拓扑标注与分层机器学习代码	218
7.4 TTS 体系机制感知机器学习、局部质量相界与验证代码	221
7.5 三体系体能带、边缘态、Berry 曲率与霍尔响应代码	224
7.6 交错磁自旋陈数论文最终代码索引	227
第八章 原创研究与机器学习	230
8.1 kp 构造对称哈密顿量的测试	230
8.2 (原创脚本) 基于对称性构造 TB 模型 (n 带紧束缚模型)——边缘态计算与拟合	232
8.3 (原创脚本)optuna 超参数优化探索拓扑相界	233
8.4 (原创自研) 基于对称性构造 TB 模型 (n 带紧束缚模型)——具体流程	234
8.5 (原创自研) 利用 wannier 验证超超交换路径	238
8.6 (原创自研) 利用 wannier 验证超超交换路径 (2)	243
8.7 (原创自研) 利用 wannier 验证超超交换路径 (3)	244
8.8 (原创自研) 利用 PDOS 验证超超交换路径	247
8.9 (原创自研) 利用 wannier 验证超超交换路径 (4)	248
8.10 (原创自研) 利用 wannier 计算超超交换过程中的轨道杂化	252
8.11 (原创自研) 机器学习-1	253
8.12 (原创自研) 机器学习-2	260

8.13 (原创自研) 机器学习-3	267
8.14 (原创自研) 机器学习-应用 1	274
8.15 (原创自研) 机器学习-应用 2	282
8.16 (原创脚本) 任意体系的磁参数 J 的磁构型设计与计算	293
第九章 高通量、自动化与模型构建	294
9.1 高通量结构优化脚本	294
9.2 通过能带判断缺陷、团簇的 python 脚本	295
9.3 高通量 DMI 计算脚本	295
9.4 (原创脚本) 高通量磁耦合参数计算脚本	296
9.5 二维双层转角材料的代码构建	296
9.6 (改编脚本) 快速计算投影能带	297
9.7 (原创脚本)python 解线性方程组	298
第四部分 绘图流程与代码	299
第十章 能带、轨道与电子结构可视化	300
10.1 (改编脚本) 快速绘制轨道投影能带	300
10.2 (原创脚本)Pyprocar 绘制能带图	301
10.3 MLWFS 绘制轨道	302
10.4 (原创脚本) 绘制精美的投影能带图	303
第十一章 拓扑、磁性与三维场可视化	306
11.1 绘制 3DMAE 的流程 (原创脚本与流程)	306
11.2 vasp 绘制自旋密度分布图	309
11.3 (原创脚本)vaspberry 计算贝里曲率与后处理	310
第十二章 结构建模与三维渲染	313
12.1 (原创脚本)blender 绘制斯格明子	313
12.2 (绘图教程) VESTA 到 Blender 的二维材料 3D 论文原子结构绘图教程	314
第五部分 心得笔记	324
第十三章 计算调试与方法反思	325
13.1 复现文献中材料结构的心得	325
13.2 vasp 结构优化	326
13.3 单层和滑移的系数推导结果	326
13.4 VASP 计算出的磁参数的归一化问题	327
13.5 异常 DMI 的数值调试	327
13.6 利用代码构造紧束缚模型的前置知识	330
第十四章 阶段心得与研究总结	332
14.1 心得笔记 1	332
14.2 心得笔记 2	333
14.3 EuIn2As2 高阶拓扑绝缘体笔记	333
14.4 EuIn2As2 高阶拓扑绝缘体笔记 2	339

附录 A 完整代码清单

348

第一部分

第一性原理计算教程

专题 1

VASP 基础计算

集中呈现 VASP 结构优化、能带、态密度、磁交换参数与 DMI 等基础计算流程, 形成从建模到结果验证的第一性原理方法主线。

本专题收录 4 篇文章。

1.1 vasp 计算态密度

日期: 2025-09-24

在线原文: marx2001.github.io

► Vasp 计算 Dos

DOS 的计算不是必需的, 而是在分析机制的时候使用。例如在分析晶体场劈裂情况以及定性讨论配体场效应时, 需要计算 DOS 以及 PDOS。

1. 静态自洽计算 (static)

以 MPSS 为例, 其结构如图所示

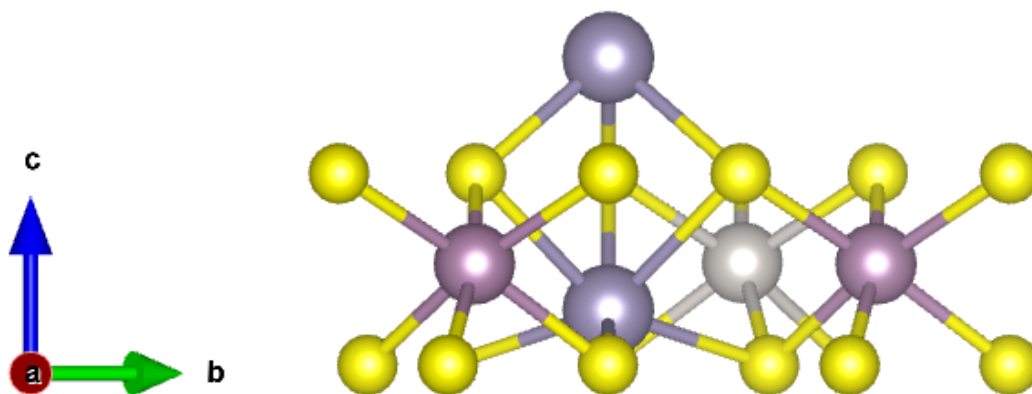


图 1.1: MPSS 结构图

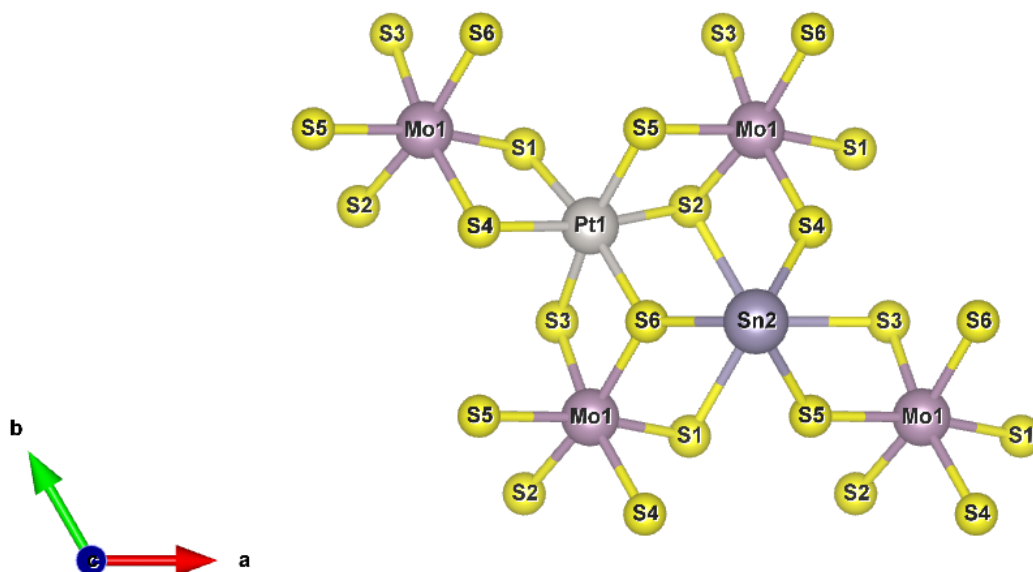


图 1.2: MPSS 结构图

首先准备 INCAR

代码清单 A.1.1

语言: python; 完整代码见附录第 348 页, 共 38 行。

由于是磁性体系, 所以要增加很多设置, 虽然部分设置是隐式开启或者关闭的, 但根据官网的说法, 显式地设置出来, 也会起到一定的作用。

注意, 个人习惯先用 AGLO=Fast, 这样计算速度快, 可以快速了解大概的情况, 同时第一次计算的时候, 可以取小 K 点, 同样使得计算速度快。单层材料不加 IVDW 范德华修正。个人习惯开启自旋极化后不设置磁矩, 让其自行演化, 得到最准确的基态。

对于强关联体系, 必须加 U, 具体数值自行测试。

准备好 POSCAR、POTCAR、CHGCAR、KPOINTS 后, 提交任务进行计算。

► 2. 数据处理方法 (重点)

首先绘制 PDOS, 这一部分较为繁琐, 首先 vaspkit/114, 可以选择原子, 输出原子所有轨道的 pdos。其它功能进入 vaspkit 后, 自行翻译即可。

产生的 PDOS_SUM_DW 是该原子的数据, 复制进 orgin 表格中。然后把同一轨道的 up 和 down 放到一起, 再在前面插入 x 轴——Energy, 这一列。全选页面内所有数据后, 选择堆积图, 调整 x 和 y 轴范围即可。

成品示例如下:

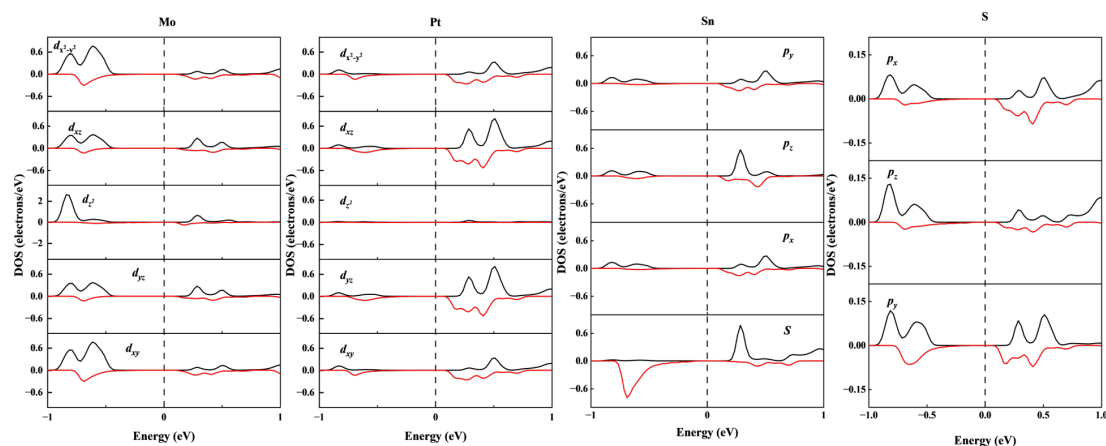


图 1.3: MPSS 结构图

DOS 的 Origin 绘图更为简单, 在此不做赘述 (同能带计算的数据后处理方式)。

1.2 vasp 计算投影能带

日期: 2025-09-24

在线原文: marx2001.github.io

► Vasp 计算 DOS

投影能带的计算不是必需的, 而是在分析机制的时候使用。例如在分析晶体场劈裂情况时需要用到。使用名称为《vasp 计算能带》文章中所用的 INCAR, 计算完成后, 输入 vaspkit/214 即可输出手动选择特定原子的投影能带数据文件。下面是 vaspkit 投影能带的选取注释:

- 211 标准能带结构 (包含所有原子和轨道的投影)
- 212 仅提取单个选定原子的轨道投影能带
- 213 按元素分类 (如所有 Fe 原子) 的投影能带
- 214 手动选择特定原子 (输入原子序号) 的投影能带
- 215 按元素权重 (如 50% Fe + 50% O) 的投影能带
- 216 对选定原子和轨道的投影求和 (如所有 Fe 的 d 轨道)

1. Origin 绘图 (重点)

以 MPSS 为例, 其结构如图所示

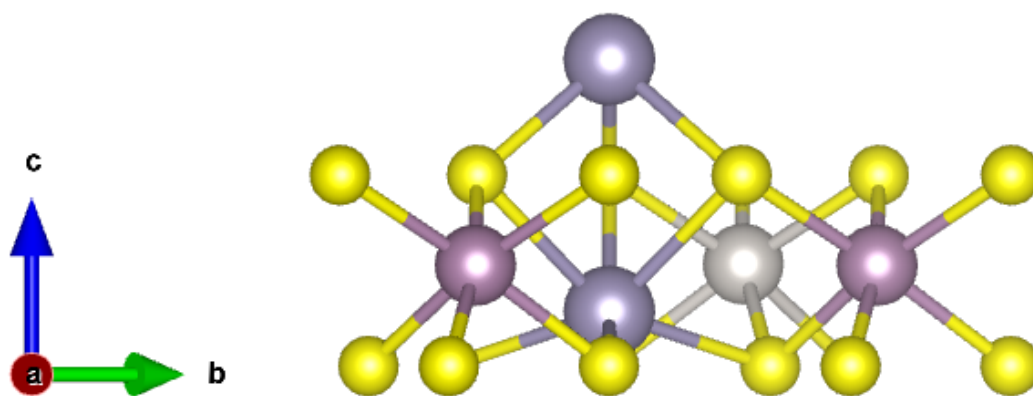


图 1.4: MPSS 结构图

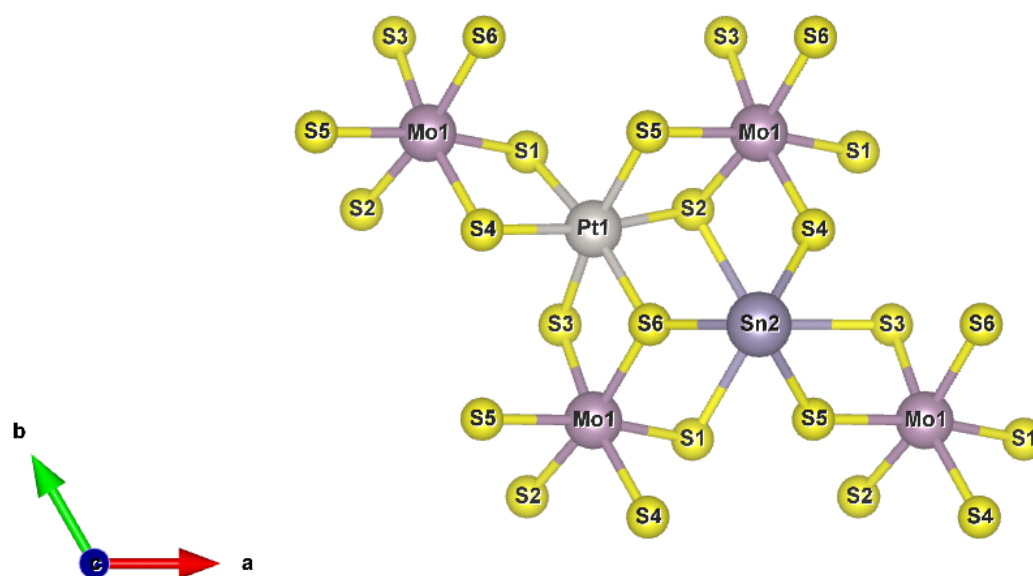


图 1.5: MPSS 结构图

vaspkit/214/Mo, 绘制 Mo 原子的投影能带

```

#K-Path      Energy  s  py  pz  px  dxy  dyz  dz2  dxz  x2-y2  tot
# NKPTS & NBANDS: 120 96
# Band-Index  1
0.00000 -60.505779 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.01460 -60.505778 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.02920 -60.505778 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.04379 -60.505779 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.05839 -60.505778 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.07299 -60.505779 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.08759 -60.505778 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.10219 -60.505779 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.11679 -60.505779 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.13138 -60.505777 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.14598 -60.505777 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.16058 -60.505776 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.17518 -60.505776 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.18978 -60.505777 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.20437 -60.505777 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.21897 -60.505776 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.23357 -60.505776 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.24817 -60.505776 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.26277 -60.505776 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.27737 -60.505777 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.29196 -60.505776 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.30656 -60.505775 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.32116 -60.505776 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.33576 -60.505778 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.35036 -60.505777 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.36496 -60.505778 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.37955 -60.505778 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.39415 -60.505778 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.40875 -60.505778 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.42335 -60.505778 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.43795 -60.505778 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.45254 -60.505778 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996
0.46714 -60.505778 0.996 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.996

```

图 1.6: 第一步

如上图所示, 得到数据文件 PBAND_SUM_DW.dat 和 PBAND_SUM_UP.dat 文件。删除上图中选中的两行, 便于在 origin 中直接作为标签。

如下图所示, 直接将数据文件拖到 origin 中:

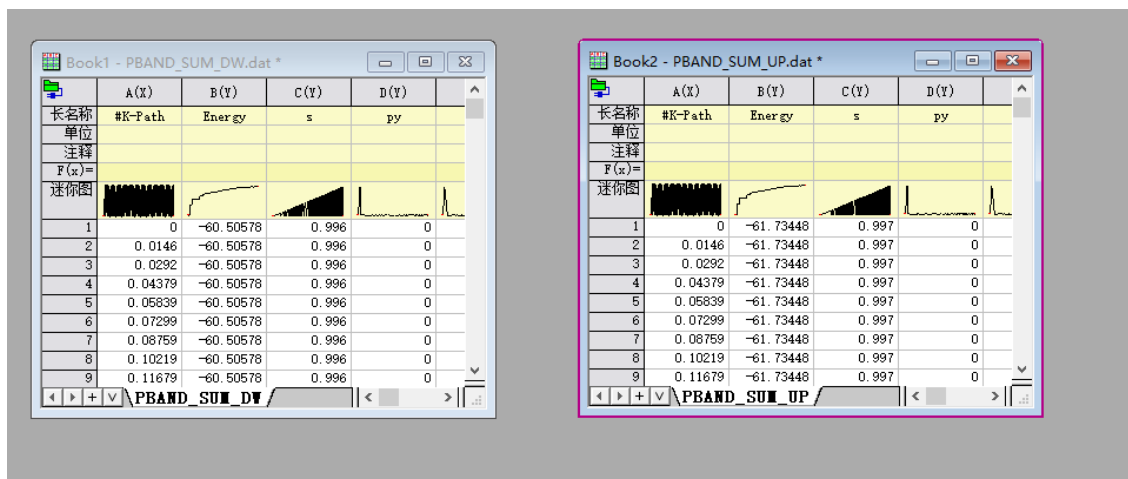


图 1.7: 第一步

选中如图所示两行画图, 如下图所示

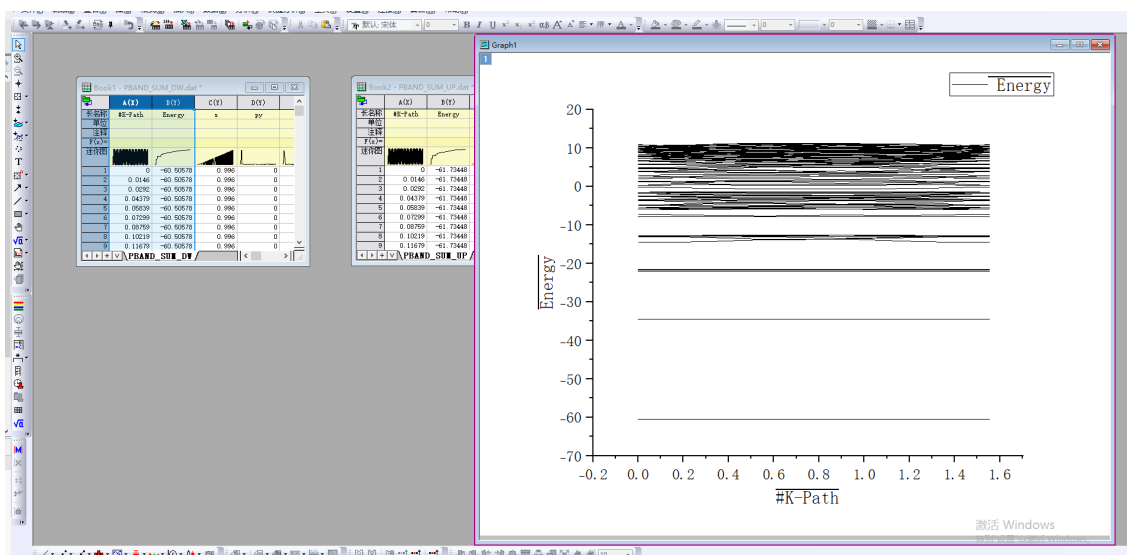


图 1.8: b3

双击 1, 进入图层, 选中十个新添加的图层, 点击 2 成组, 点击 3, 修改成散点图

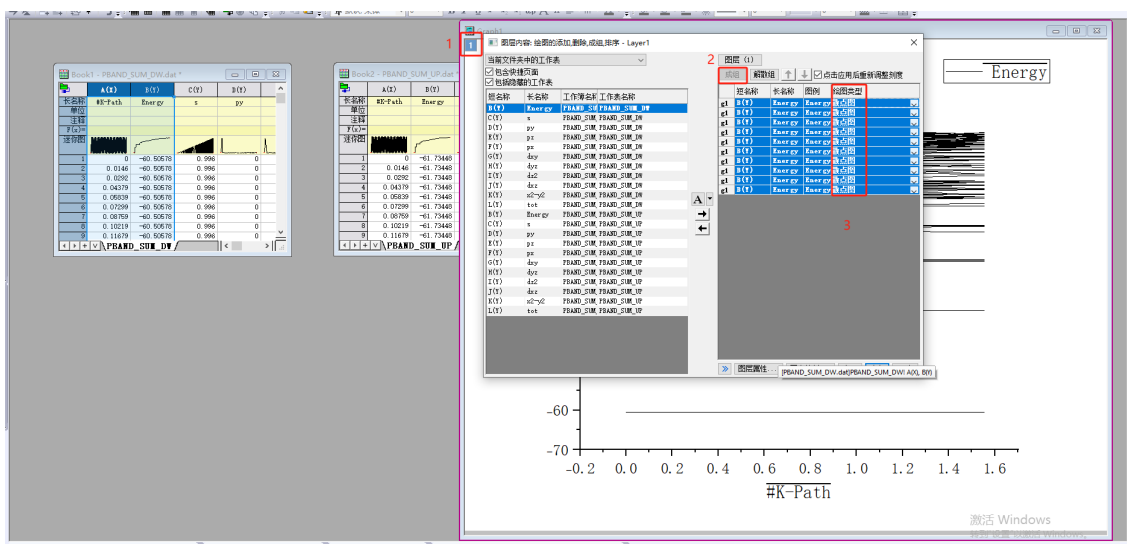


图 1.9: b3

双击图像 (红框区域), 弹出来对话框

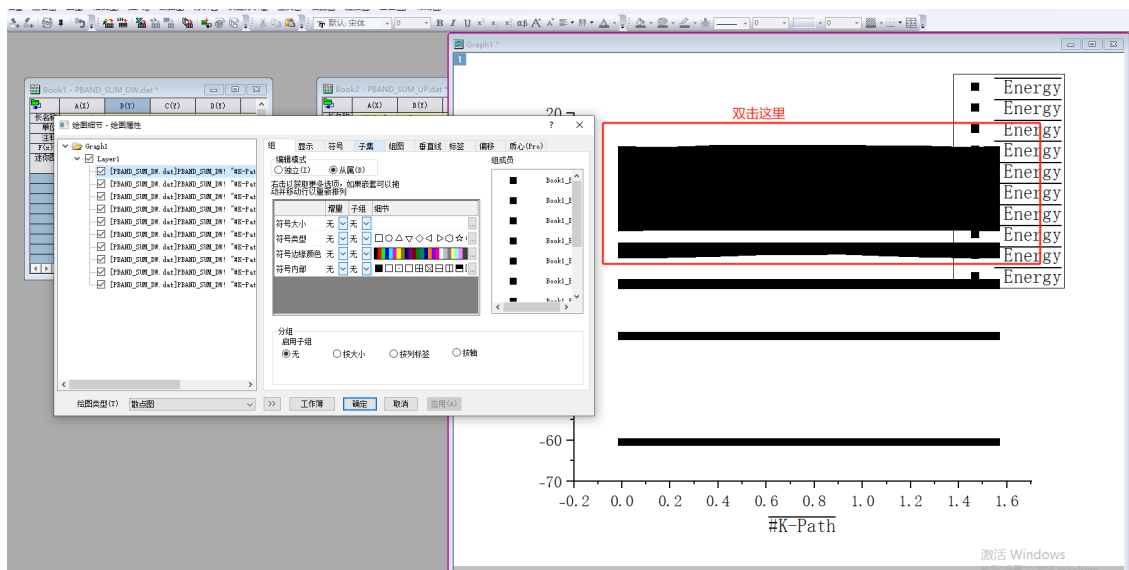


图 1.10: b3

修改如下图所示的三个参数, 缩放因子建议 20。

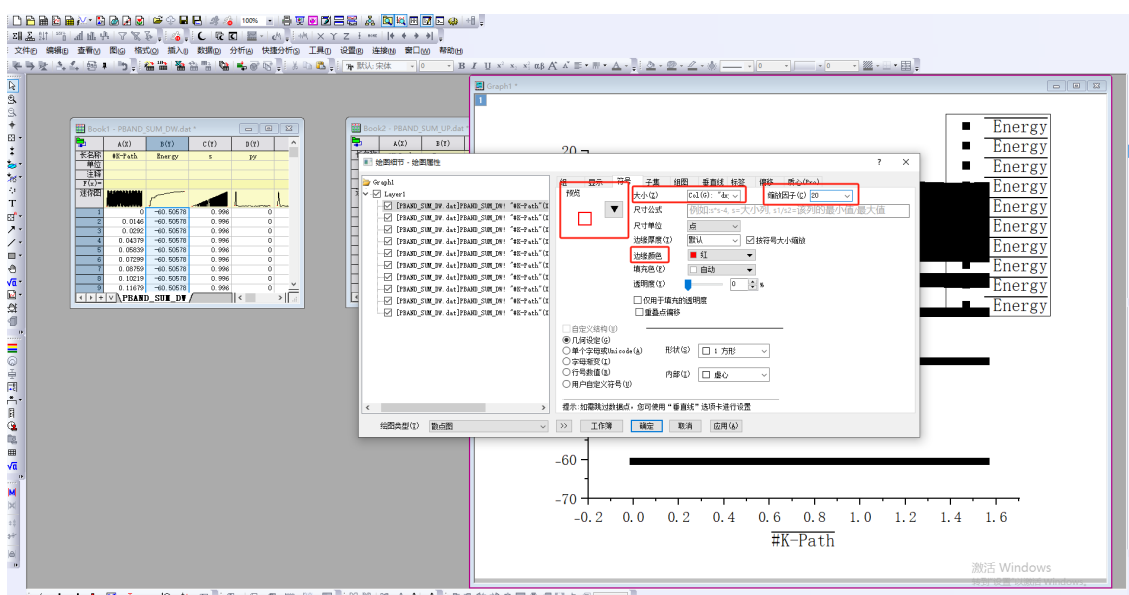


图 1.11: b3

注意事项——反铁磁基态, 可以任意选择能量折线图, 而铁磁基态, 由于能带 spin up 和 spin down 不简并, 所以绘图时要注意, Energy 折线图 up 要选 5 个 (d 轨道, p 轨道选 3 个), down 要选五个。

原因是投影能带是算的每个轨道相比于 Energy 的权重, 而反铁磁基态的能量是简并的, 所以选 up 和 down 的均可, 不影响权重计算, 而铁磁基态由于作为分母的总能量不简并, 所以若想准确计算权重, 能量折线图 (分母) 必须一半 up 一半 down。

最终效果图:

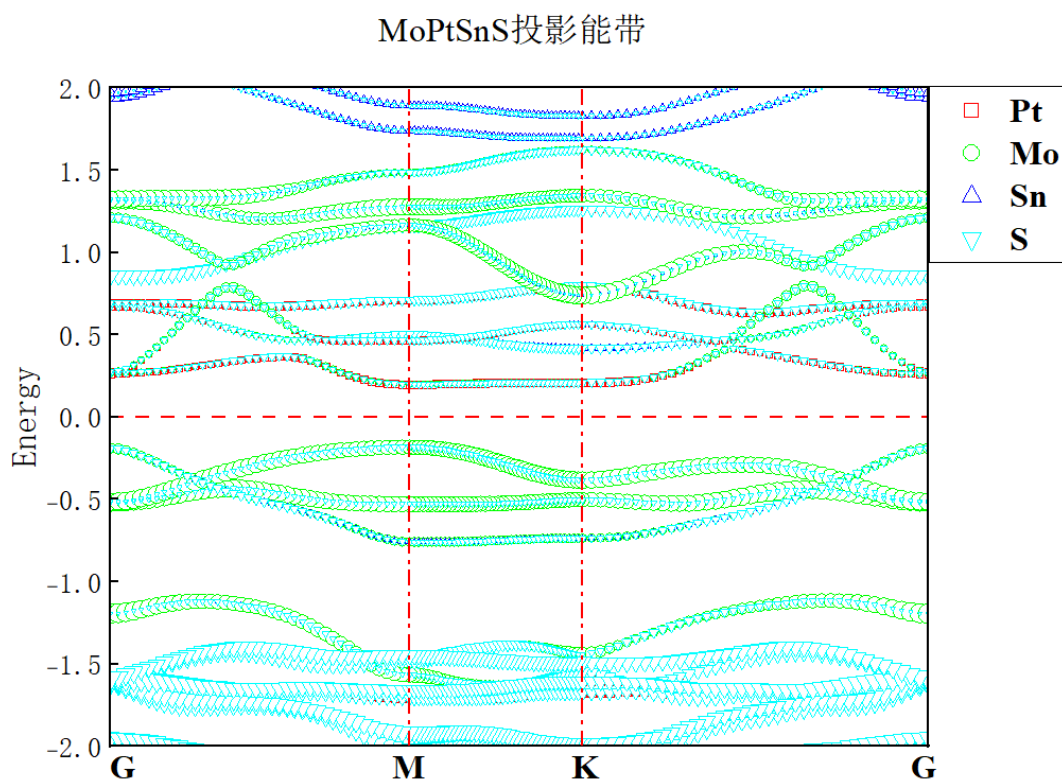


图 1.12: b3

2. 自定义脚本绘图 (原创)

该脚本功能较为完善, 支持灵活自定义参数, 使用方式类似 VASPKIT。首先需要采用如下目录结构:

代码清单 A.2.1

语言: python; 完整代码见附录第 349 页, 共 19 行。

自旋极化的投影能带绘制, 代码如下:

代码清单 A.2.2

语言: python; 完整代码见附录第 349 页, 共 658 行。

Usage

代码清单 A.2.3

语言: python; 完整代码见附录第 359 页, 共 70 行。

考虑 SOC 非自旋极化的情况下的代码:

代码清单 A.2.4

语言: python; 完整代码见附录第 360 页, 共 651 行。

运行命令绘制 Ge 的 p 轨道分布

短代码 · python

```
python plot_fatbands.py -p Ge.px Ge.py Ge.pz --split -m -2 -M 2
```

1.3 vasp 计算能带

日期: 2025-09-24

在线原文: marx2001.github.io

能带计算作为最基本的计算, 也有其合理的步骤以及注意事项。

1. 静态自洽计算 (static)

以 MPSS 为例, 其结构如图所示

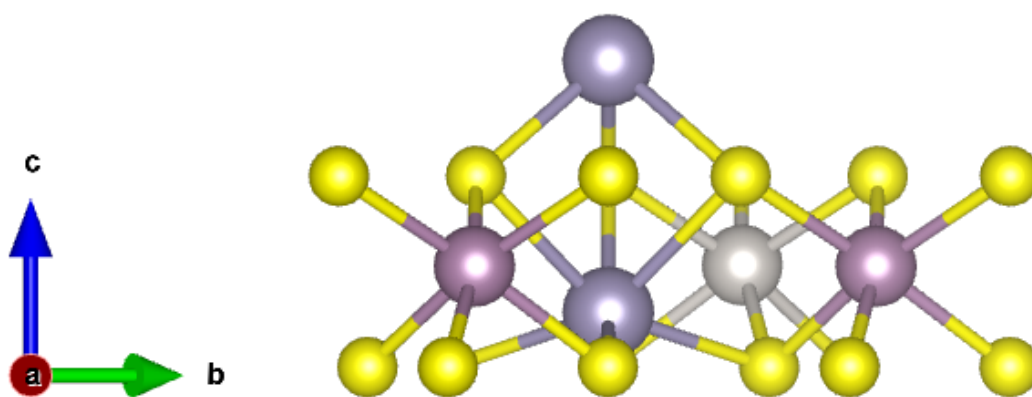


图 1.13: MPSS 结构图

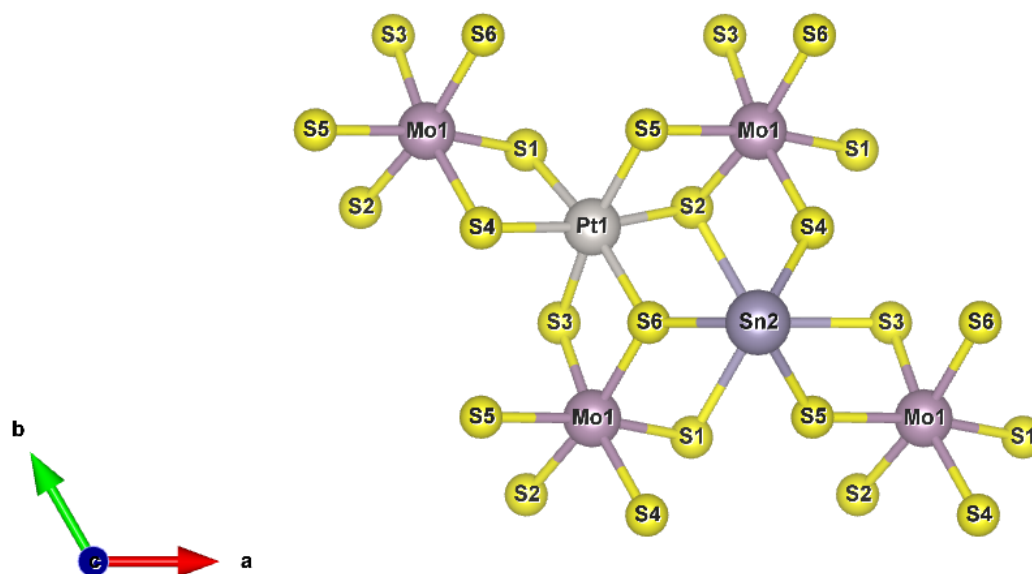


图 1.14: MPSS 结构图

首先准备 INCAR

代码清单 A.3.1

语言: python; 完整代码见附录第 371 页, 共 36 行。

由于是磁性体系, 所以要增加很多设置, 虽然部分设置是隐式开启或者关闭的, 但根据官网的说法, 显式地设置出来, 也会起到一定的作用。

注意, 个人习惯先用 AGLO=Fast, 这样计算速度快, 可以快速了解大概的情况, 同时第一次计算的时候, 可以取小 K 点, 同样使得计算速度快。单层材料不加 IVDW 范德华修正。个人习惯开启自旋极化后不设置磁矩, 让其自行演化, 得到最准确的基态。static 计算较快, 因此不设置混合系数也是可以的。

LCHARG 必须开启, 因为能带计算必须要用到 CHGCAR 才能足够精确。

对于强关联体系, 必须加 U, 具体数值自行测试。

提交 static 任务计算脚本后等待结果。

► 2. 进行能带计算 (重点)

这个计算需要注意一个情况, 必须设置 ISTART=1, ICHARG=11, 原因如下方注释所言。而且 NBANDS 不要设置, 避免浪费时间。band 计算不开启自旋轨道耦合 (SOC), 如果想看考虑自旋轨道耦合的结果, 则取消 LSORBIT 和 LORBMOM 的注释即可。

INCAR 如下

代码清单 A.3.2

语言: python; 完整代码见附录第 371 页, 共 37 行。

KPOINTS 用 vaspkit/302, 生成, 不要用 static 的 KPOINTS, 同时也可以前往 SeekKPATH 网站自定义高对称点路径。

准备好 INCAR、CHGCAR、POSCAR、POTCAR 后提交计算即可。

► 3. 数据处理

计算结束后, vaspkit/211, 产生 BAND.dat 和 KLINEs.dat 文件, 打开文件后, 复制内容到 Origin 当中, 先复制 BAND.dat 数据, 再在 band 数据后添加列, 复制进 KLINEs.dat 数据, 并右键选中列后设置为 X 轴。再点击左下角绘图, 点击右上角让曲线更平滑, 然后选择数据范围, 一般只看费米能级附近的能带, 所以 y 轴范围 [-2, 2]。其它格式自行调节。

1.4 vasp 计算磁耦合参数大小

日期: 2025-09-25

在线原文: marx2001.github.io

为了研究二维材料的拓扑自旋纹理, 我们需要计算海森堡模型中的 DMI 的能量大小。

磁耦合参数, 即磁性原子之间相互作用的大小。 $J>0$, 磁矩倾向于平行排列, 即铁磁 (FM); $J<0$, 磁矩倾向于反平行排列, 即反铁磁 (AFM)。 J 的大小决定磁耦合的强弱。 J 大, 相互作用强, 居里温度/尼尔温度高。 J 小, 相互作用弱, 热涨落容易破坏有序性。

J 源于电子的交换效应和电子-电子相互作用, 包括直接交换、超交换、超超交换、双交换以及 RKKY 相互作用。

J 决定自旋是 FM 还是 AFM, DMI 倾向于让自旋扭转, 形成螺旋或者畴壁。

1. 能量法计算 J 值

以 MPSS 为例, 其结构如图所示

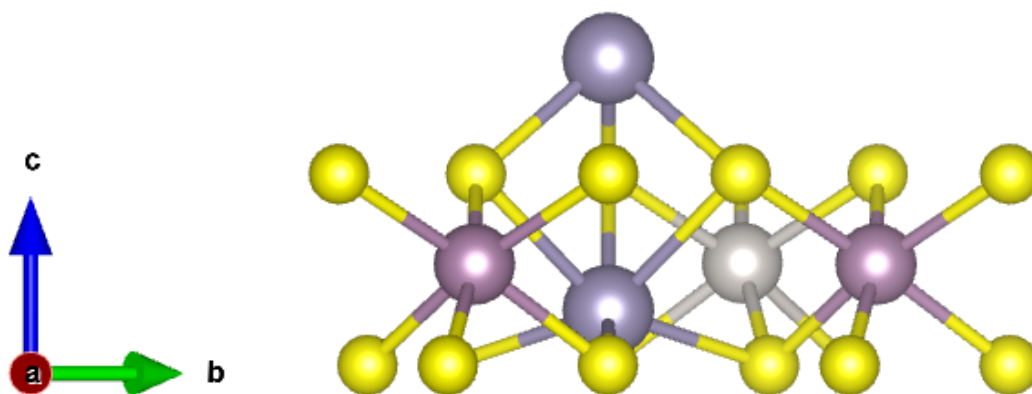


图 1.15: MPSS 结构图

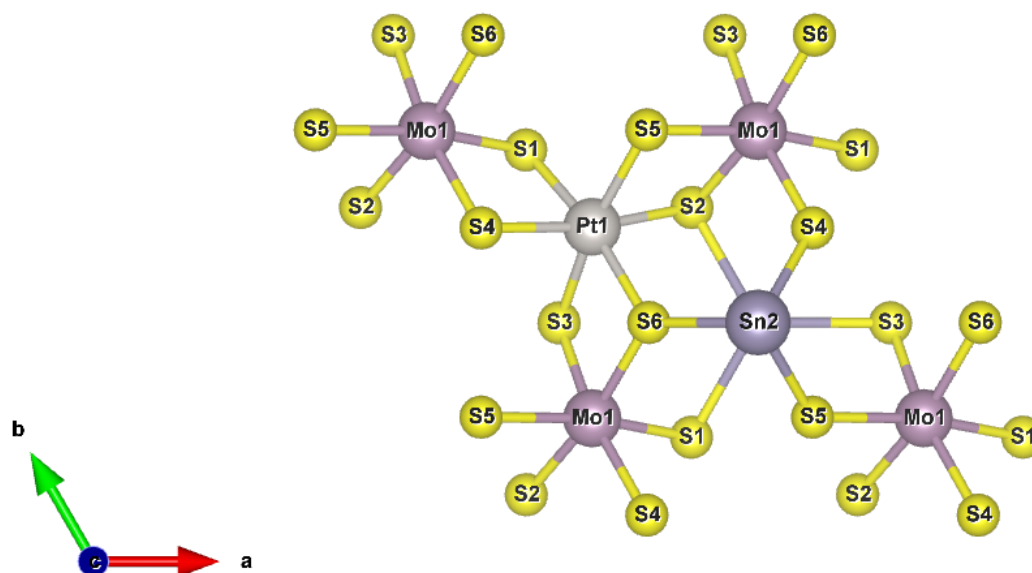


图 1.16: MPSS 结构图

需要准备四个 INCAR, 四个 INCAR 的唯一区别就是磁态设置——FM、Neel、Stripy、Zigzag。如下图所示, 箭头为磁矩方向:

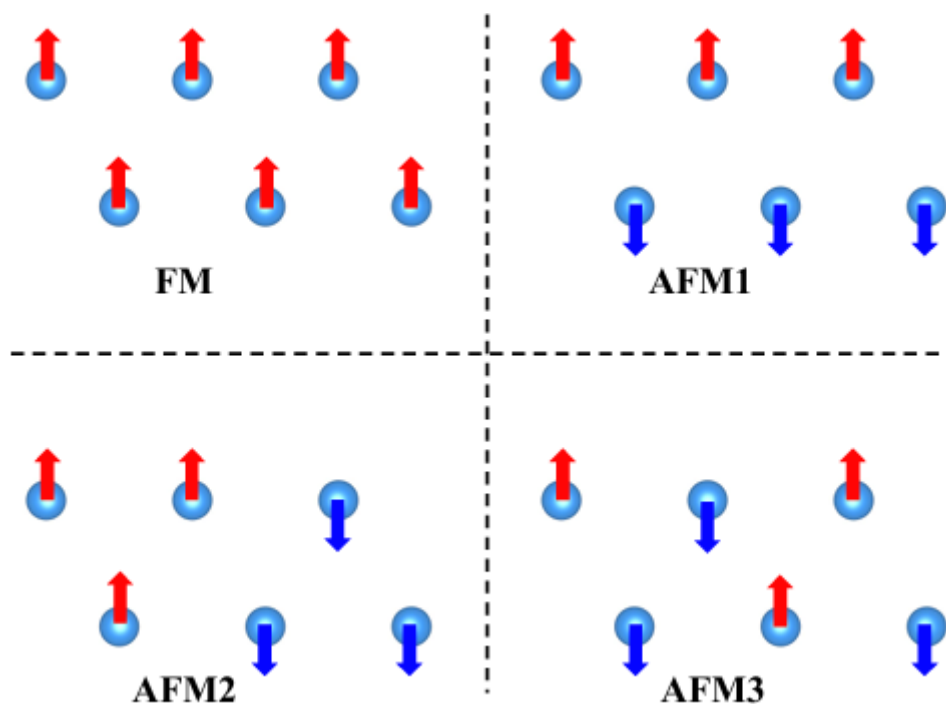


图 1.17: Fig1

四种 INCAR 如下方所示

INCAR1:

代码清单 A.4.1

语言: shell; 完整代码见附录第 372 页, 共 40 行。

INCAR2

代码清单 A.4.2

语言: shell; 完整代码见附录第 373 页, 共 40 行。

INCAR3

代码清单 A.4.3

语言: shell; 完整代码见附录第 374 页, 共 40 行。

INCAR4

代码清单 A.4.4

语言: shell; 完整代码见附录第 374 页, 共 40 行。

由于是磁性体系, 所以要增加很多设置, 虽然部分设置是隐式开启或者关闭的, 但根据官网的说法, 显式地设置出来, 也会起到一定的作用。

注意, 个人习惯先用 `AGLO=Fast`, 这样计算速度快, 可以快速了解大概的情况, 同时第一次计算的时候, 可以取小 `K` 点, 同样使得计算速度快。单层材料不加 `IVDW` 范德华修正。个人习惯开启自旋极化后不设置磁矩, 让其自行演化, 得到最准确的基态。

对于强关联体系, 必须加 `U`, 具体数值自行测试。

由于要设置四种磁态, 所以要进行扩胞, 根据三近邻海森堡模型, MPSS 的扩胞大小要包含任意磁性原子的第三近邻磁性原子。显而易见, 对于 MPSS 材料体系, 需要扩胞 331。

准备好 POSCAR、POTCAR、KPOINTS 后, 提交任务进行计算。任务脚本如下:

代码清单 A.4.5

语言: shell; 完整代码见附录第 375 页, 共 41 行。

计算完成后, 键入 `qvasp -e`, 得到四个能量, 进入这个网址: <http://www.yunsuan.info/matrixcomputations/solveLinearsystems.html>, 解线性方程组, 其系数如下所示:

$$\begin{aligned} E_{FM} &= 6J_1 + 12J_2 + 6J_3 + 4K(S^z) + E_0 \\ E_{Neel} &= -6J_1 + 12J_2 - 6J_3 + 4K(S^z) + E_0 \\ E_{Stripy} &= -2J_1 - 4J_2 + 6J_3 + 4K(S^z) + E_0 \\ E_{Zigzag} &= 2J_1 - 4J_2 - 6J_3 + 4K(S^z) + E_0 \end{aligned}$$

图 1.18: Fig1

2. 四态法计算 J 值

计算量非常大, 不推荐此种方法。四态法最好使用 `ncl` 计算, 较为准确。另外, 如果算 z 方向, 则是把磁矩设置到 z 方向上, 如果计算 y 方向, 就把磁矩设置到 y 方向上。

方法原理同四态法计算 DMI, 因此直接放上 4 个 INCAR 文件和提交任务脚本:

INCAR1

代码清单 A.4.6

语言: shell; 完整代码见附录第 376 页, 共 46 行。

INCAR2

代码清单 A.4.7

语言: shell; 完整代码见附录第 377 页, 共 41 行。

INCAR3

代码清单 A.4.8

语言: shell; 完整代码见附录第 377 页, 共 41 行。

INCAR4

代码清单 A.4.9

语言: shell; 完整代码见附录第 378 页, 共 41 行。

提交任务脚本为:

代码清单 A.4.10

语言: shell; 完整代码见附录第 379 页, 共 51 行。

磁性参数与 DMI

本专题收录 2 篇文章。

2.1 vasp 计算 DMI 大小

日期：2025-09-25

在线原文：marx2001.github.io

为了研究二维材料的拓扑自旋纹理, 我们需要计算海森堡模型中的 DMI 的能量大小。

DMI 的大小取决于两个核心条件——对称性和自旋轨道耦合。只有存在较强的自旋轨道耦合 (SOC), 才有可能产生 DMI。这是因为 DMI 本质上来源于 SOC 对超交换相互作用的修正。

在轻元素体系里, SOC 很弱, DMI 可以忽略。

含有重元素 (如过渡金属、5d 元素、Bi, W, Ir 等) 或者与重原子基底相耦合的二维材料, 更容易产生显著的 DMI。

根据 Moriya 定理, 缺乏空间反演对称性, 如果两个磁性原子之间的中点存在反演中心, 对应的 DMI 为 0。因此在二维材料中产生 DMI, 需要在结构上产生破缺的对称性。

例如: 固有结构的不对称、缺陷或表面不对称; 界面或基底效应; 外场诱导 (电场或者应变导致结构对称性破缺)

点群限制: DMI 的方向由晶体对称性决定。二维材料中, 常见的 C_{3v} 、 C_{2v} 等对称性都可能允许 DM 相互作用。

1. 能量法的计算步骤

以 MPSS 为例, 其结构如图所示

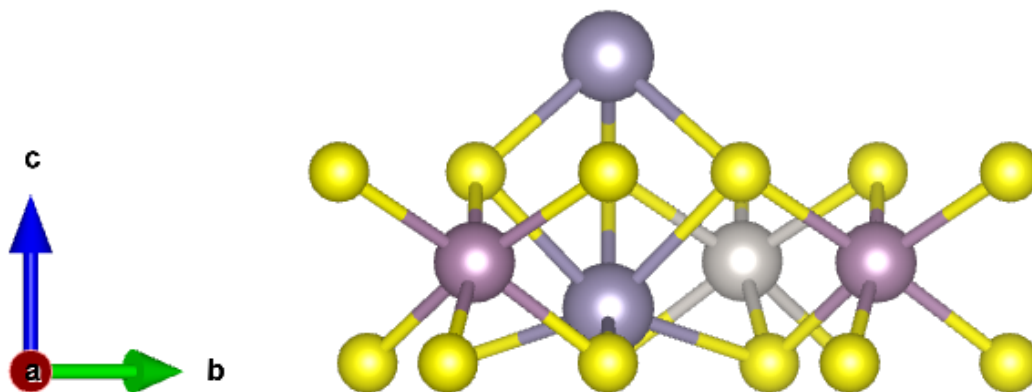


图 2.1: MPSS 结构图

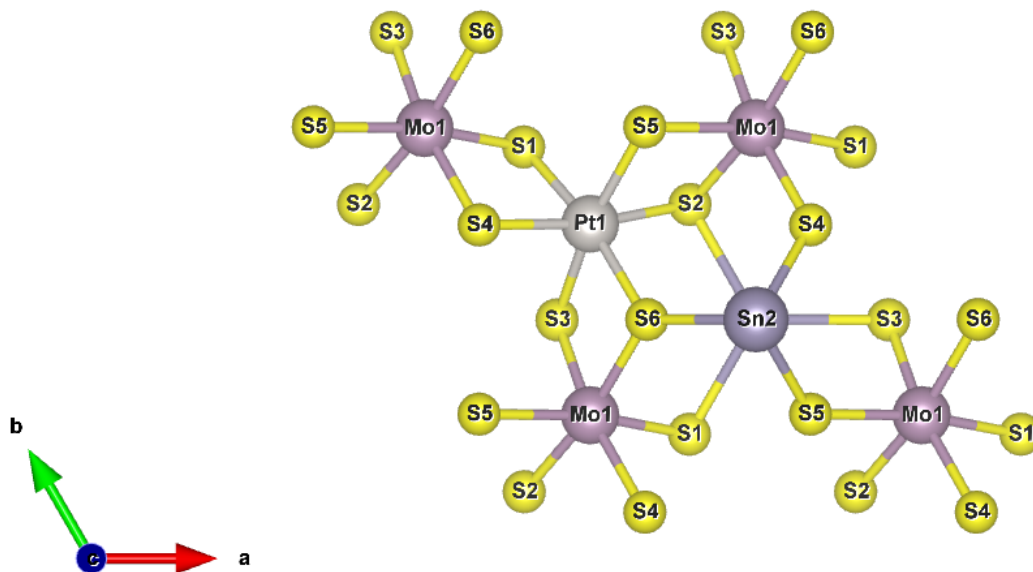


图 2.2: MPSS 结构图

首先准备 INCAR

代码清单 A.5.1

语言: python; 完整代码见附录第 380 页, 共 39 行。

需要注意的是, 能量法的原理是设置磁态为顺时针的能量减去逆时针的能量再除以一个系数, 所以我们需要在 $ISPIN=2$ 的情况下, 设置磁矩为顺时针的方向。

由于是磁性体系, 所以要增加很多设置, 虽然部分设置是隐式开启或者关闭的, 但根据官网的说法, 显式地设置出来, 也会起到一定作用。

注意, 个人习惯先用 $AGLO=Fast$, 这样计算速度快, 可以快速了解大概的情况, 同时第一次计算的时候, 可以取小 K 点, 同样使得计算速度快。单层材料不加 $IVDW$ 范德华修正。个人习惯开启自旋极化后不设置磁矩, 让其自行演化, 得到最准确的基态。

对于强关联体系, 必须加 U , 具体数值自行测试。

由于要设置顺时针四个方向的磁矩, 只有一个磁性原子显然是不够的, 所以我们需要在 x 方向扩 4 倍于原胞大小的超胞, 在 `shell` 面板内, 输入 `vaspkit/401/4 1 1`, 注意 411 之间要加空格。

得到 `SUPERCELL411.vasp` 文件后, 修改名称为 `POSCAR`。

接下来准备 `KPOINTS`, 有一个细节, 不要使用之前计算的 `KPOINTS`, 而是要再次键入 `vaspkit/102/1/102/0.03`, 得到 `KPOINTS`。

准备好 `POSCAR`、`POTCAR`、`KPOINTS` 后, 提交任务进行计算。任务脚本如下:

代码清单 A.5.2

语言: shell; 完整代码见附录第 380 页, 共 39 行。

等待计算结束后, 键入 `qvasp -e`, 收集能量信息, acw 的能量减去 acw 的能量除以一个系数即可。这个系数根据体系而定, 可以参考具体的文献。

1. 四态法的计算步骤

四态法是指通过构建四种磁态, 通过求解这四个海森堡模型哈密顿方程的方程组来得到对应的磁耦合参数, 具有一般性, 但计算量相比于能量法较大, 对于原胞内原子数目较多的大体系而言, 设备要求较高。

应用四态法计算 DMI, 首先要构建超胞, 以 MPSS 的三角晶格为例, 由于两个磁性原子之间的距离为 0.637nm, 所以我们选择扩 331 的超胞, 使得相邻周期同一原子的距离大于 0.15nm, 以屏蔽相邻周期同一磁性原子的影响。

然后构建四个磁态, 这四个磁态分别是均是朝上的铁磁, 一上一下反铁磁 + 其它磁性原子向上、一下以上 + 其他磁性原子向上、同下 + 其他磁性原子向上。运行计算任务后, $E1+E4-E2-E3$ 的结果再除以 4 即可, 为了得到真实的能量, 需要再除以自旋的平方, 自旋量子数的数值是原胞磁矩的值除以 2。

提交计算任务的脚本为:

代码清单 A.5.3

语言: shell; 完整代码见附录第 381 页, 共 46 行。

2.2 (文献整理) 块体材料 DMI 的第一性原理提取方法

日期: 2026-07-08

在线原文: marx2001.github.io

► 说明

本文整理用于块体材料 **Dzyaloshinskii-Moriya interaction(DMI)** 第一性原理计算与提取的代表性文献。重点不是介绍 DMI 的物理起源, 而是给后续实际计算提供文献依据, 包括:

代码清单 A.6.1

语言: shell; 完整代码见附录第 382 页, 共 5 行。

对于块体材料, DMI 一般不应只写成一个标量 (D), 而应写成每一类磁性键的 DMI 矢量:

$$[H_{\mathrm{DMI}}] = \sum_{\langle ij \rangle} \mathbf{D}_{\langle ij \rangle} \cdot \left(\mathbf{e}_i \times \mathbf{e}_j \right)$$

或者根据文献约定写成带负号的形式:

$$[H_{\mathrm{DMI}}] = -\sum_{\langle ij \rangle} \mathbf{D}_{\langle ij \rangle} \cdot \left(\mathbf{e}_i \times \mathbf{e}_j \right)$$

因此, 在引用文献或写计算方法时, 必须明确自己的哈密顿量符号约定。

► 文献总览

序号	文献	方法类型	适合用途
1	Yang, Liang & Cui, Nat. Rev. Phys. 2023	DMI 第一性原理综述	总述引用
2	Xiang et al., Dalton Trans. 2013	能量映射综述	说明 spin Hamiltonian / energy mapping
3	Šabani et al., Phys. Rev. B 2020	通用四态映射	提取完整 exchange matrix

4	Xu et al., Phys. Rev. B 2019	BiFeO3 四态映射	块体多铁材料 DMI 案例
5	Xu et al., Phys. Rev. B 2021	BiFeO3 spin spiral	用 $E(q)$ 提取 DMI
6	Yang et al., Phys. Rev. Lett. 2012	Cu2OSeO3 DMI	块体手性磁体 / 多铁材料案例
7	Loh & Gan, AIP Adv. 2017	bulk FeGe spin spiral / SOC	块体 B20 手性磁体案例
8	Koretsune et al., Sci. Rep. 2015	Wannier / first-principles DMI	Mn1-xFexGe 中 DMI 大小和符号调控
9	He et al., Comput. Phys. Commun. 2021	TB2J / Green' s function	自动提取 (J_{ij}) 、 $(\mathbf{D}_{ij})$
10	Mahfouzi & Kioussis, Phys. Rev. B 2021	Green' s function DMI	Green' s function 方法依据

► 方法分类

► 1. 四态能量映射法

四态能量映射法适合从总能差中提取某一对磁性原子的交换矩阵。对于两个自旋 (i,j) , 一般写成:

$$[H_{ij} = \mathbf{S}_i \mathbf{J}_{ij} \mathbf{S}_j]$$

其中 $(\mathbf{J}_{ij})$ 是 (3×3) exchange matrix。其反对称部分对应 DMI:

$$[D^x_{ij} = \frac{J^{yz}_{ij} - J^{zy}_{ij}}{2}]$$

$$[D^y_{ij} = \frac{J^{zx}_{ij} - J^{xz}_{ij}}{2}]$$

$$[D^z_{ij} = \frac{J^{xy}_{ij} - J^{yx}_{ij}}{2}]$$

这个方法的优点是公式直观, 适合写在支撑材料中。缺点是对于复杂块体材料, 超胞里往往含有很多等价或不等价的磁性键, 容易把不同路径的 DMI 混在一起。因此使用时必须明确:

1. 选择哪一对磁性原子;
2. 超胞中这一类等价键的数量 N ;
3. 是否存在多个不等价 V-V、Fe-Fe、Cu-Cu 或其他磁性键;
4. 哈密顿量中 DMI 项的正负号约定;
5. 最终提取的是单键 $(\mathbf{D}_{ij})$, 还是等效总 DMI。

► 2. Spin spiral 方法

spin spiral 方法适合提取连续模型中的有效 DMI。对于小 (q) 附近的自旋螺旋, 可以写成:

$$[E(q) = Aq^2 \pm Dq]$$

或者用左右手螺旋能量差:

$$[\Delta E(q) = E_{\mathrm{CW}}(q) - E_{\mathrm{CCW}}(q)]$$

在小 (q) 极限下, $(\Delta E(q))$ 与 (q) 的线性项给出有效 DMI。这个方法适合:

1. 块体 B20 手性磁体;
2. BiFeO₃ 这类长周期 spin cycloid;
3. 想得到连续模型参数 A 和 D, 而不是每条键的 $\mathbf{D}_{ij}$;
4. 后续要和微磁模拟或长周期螺旋周期联系。

► 3. Green' s function / Wannier / TB2J 方法

Green' s function 方法基于 magnetic force theorem 和局域刚性自旋旋转微扰。它的优势是可以从 Wannier Hamiltonian 或局域轨道 Hamiltonian 中自动提取远距离磁相互作用, 包括:

$$[J_{ij}, \quad \mathbf{D}_{ij}, \quad \Gamma_{ij}]$$

对于复杂块体体系, 这条路线通常比手动四态法更稳妥。尤其是当磁性原子很多、交换路径复杂、近邻壳层很多时, 推荐优先考虑 TB2J 或同类 Green' s function 程序。

► 重点文献整理

► 1. First-principles calculations for Dzyaloshinskii–Moriya interaction

代码清单 A.6.2

语言: text; 完整代码见附录第 382 页, 共 4 行。

这是一篇 DMI 第一性原理计算综述, 适合放在方法部分开头作为总述文献。文章系统讨论了 DMI 的第一性原理计算路线, 包括:

代码清单 A.6.3

语言: shell; 完整代码见附录第 382 页, 共 5 行。

建议用途:

代码清单 A.6.4

语言: shell; 完整代码见附录第 382 页, 共 1 行。

► 2. Magnetic properties and energy-mapping analysis

代码清单 A.6.5

语言: text; 完整代码见附录第 382 页, 共 4 行。

这篇是能量映射法的经典综述, 主要讨论如何从第一性原理总能中映射得到磁性哈密顿量参数。虽然它不只讲 DMI, 但对于解释 exchange、single-ion anisotropy、DMI 等磁参数的能量映射思想非常有用。

建议用途:

代码清单 A.6.6

语言: shell; 完整代码见附录第 383 页, 共 3 行。

► 3. Generic four-state energy mapping onto a Heisenberg spin Hamiltonian

代码清单 A.6.7

语言: text; 完整代码见附录第 383 页, 共 5 行。

这篇文章系统推导了通用四态能量映射公式。它的核心价值是: 不是只提取一个标量 (J), 而是提取完整的 exchange matrix。DMI 对应 exchange matrix 的反对称部分。

建议用途:

1. 如果要在支撑材料中写四态映射公式, 优先引用这篇;
2. 如果要提取 $(\mathbf{D})_{ij}=(D^x, D^y, D^z)$, 这篇比普通 J 映射文献更合适;
3. 对复杂低对称体系、块体材料、二维材料都可作为方法依据。

► 4. Magnetic interactions in BiFeO₃: A first-principles study

代码清单 A.6.8

语言: text; 完整代码见附录第 383 页, 共 4 行。

这篇是块体多铁材料 BiFeO₃ 的典型示例。作者使用 first-principles calculations 结合 four-state energy mapping 提取了:

代码清单 A.6.9

语言: shell; 完整代码见附录第 383 页, 共 4 行。

这篇非常适合用来证明: 块体材料也可以通过四态能量映射法提取 DMI。

建议用途:

代码清单 A.6.10

语言: shell; 完整代码见附录第 383 页, 共 1 行。

► 5. First-principles study of spin spirals in the multiferroic BiFeO₃

代码清单 A.6.11

语言: text; 完整代码见附录第 383 页, 共 4 行。

这篇研究 BiFeO₃ 中的 AFM spin cycloid。它通过计算 cycloidal spin spiral 的 $(E(q))$ 来分析长周期螺旋基态, 并和 DMI、交换、各向异性联系起来。

它的附录中包含从第一性原理确定 exchange 和 Dzyaloshinskii-Moriya interaction 的内容, 因此适合作为 spin spiral 提取 DMI 的具体案例。

建议用途:

1. 想用 $(E(q))$ 或 spin spiral 能量色散提取有效 DMI;
2. 想解释块体多铁材料中 DMI 诱导长周期 cycloid;
3. 想把第一性原理 DMI 与 Monte Carlo / spin dynamics 联系。

► 6. Strong Dzyaloshinskii-Moriya Interaction and Origin of Ferroelectricity in Cu₂OSeO₃

代码清单 A.6.12

语言: text; 完整代码见附录第 384 页, 共 4 行。

这篇是块体手性磁体 / 多铁材料 Cu₂OSeO₃ 的代表性 DFT 研究。文章指出不同 Cu 离子之间存在较强 DMI, 并将 DMI 与 helical ground state 和 skyrmion state 联系起来。

建议用途:

代码清单 A.6.13

语言: shell; 完整代码见附录第 384 页, 共 3 行。

► 7. Exchange and Dzyaloshinskii-Moriya interactions in bulk FeGe: Effects of atomic vacancies

代码清单 A.6.14

语言: text; 完整代码见附录第 384 页, 共 5 行。

这篇标题中直接包含 bulk FeGe, 是块体 B20 手性磁体中 DMI 计算的直接案例。文章研究了 FeGe 中原子空位对 Heisenberg exchange 和 DMI 的影响, 并通过包含 SOC 的计算分析 DM interaction 的大小和符号变化。

建议用途:

代码清单 A.6.15

语言: shell; 完整代码见附录第 384 页, 共 3 行。

► 8. Control of Dzyaloshinskii-Moriya interaction in Mn_{1-x}Fe_xGe: a first-principles study

代码清单 A.6.16

语言: text; 完整代码见附录第 384 页, 共 5 行。

这篇研究 B20 型金属螺旋磁体 Mn_{1-x}Fe_xGe 中 DMI 的大小和符号如何随 Fe 掺杂变化。它强调 DMI 对电子结构细节非常敏感, 尤其与 exchange splitting、反交叉点分布、载流子浓度等有关。

建议用途:

代码清单 A.6.17

语言: shell; 完整代码见附录第 384 页, 共 3 行。

► 9. TB2J: A python package for computing magnetic interaction parameters

代码清单 A.6.18

语言: text; 完整代码见附录第 385 页, 共 4 行。

TB2J 是一个自动计算磁相互作用参数的 Python 程序, 可以从 Wannier90 或局域轨道 Hamiltonian 中提取:

1. isotropic exchange (J_{ij});
2. anisotropic exchange;
3. Dzyaloshinskii-Moriya interaction ($\mathbf{D}_{ij}$);
4. 可用于后续 magnon、spin dynamics、Monte Carlo 等计算。

这篇对于复杂块体材料非常实用。因为块体材料中磁性原子多、交换路径多, 手动四态能量映射很容易遗漏或混合不同键的贡献。TB2J 可以更系统地输出不同距离、不同原子对之间的磁相互作用参数。

建议用途:

代码清单 A.6.19

语言: shell; 完整代码见附录第 385 页, 共 1 行。

► 10. First-principles calculation of the Dzyaloshinskii-Moriya interaction: A Green' s function approach

代码清单 A.6.20

语言: text; 完整代码见附录第 385 页, 共 5 行。

这篇文章专门讨论 Green' s function 方法计算 DMI。它的价值在于给出比 supercell 和 generalized Bloch 方法更高效的 DMI 计算框架, 尤其适合需要处理 DMI tensor 或多层/复杂体系的情况。

建议用途:

代码清单 A.6.21

语言: shell; 完整代码见附录第 385 页, 共 3 行。

► 针对块体材料的推荐引用组合

如果目标是写“块体材料 DMI 的计算方法”, 推荐按下面方式组合引用。

► 情况一: 使用四态能量映射法

代码清单 A.6.22

语言: text; 完整代码见附录第 385 页, 共 3 行。

推荐写法:

代码清单 A.6.23

语言: text; 完整代码见附录第 385 页, 共 1 行。

► 情况二: 使用 spin spiral 方法

代码清单 A.6.24

语言: text; 完整代码见附录第 386 页, 共 2 行。

推荐写法:

For long-wavelength spiral states, the effective DMI can be evaluated from the linear-in-(q) contribution to the spin-spiral energy dispersion.

► 情况三: 使用 Wannier / TB2J / Green' s function 方法

代码清单 A.6.25

语言: text; 完整代码见附录第 386 页, 共 3 行。

推荐写法:

代码清单 A.6.26

语言: text; 完整代码见附录第 386 页, 共 1 行。

► 实际计算时需要注意的问题

► 1. 先判断 DMI 是否由对称性允许

DMI 存在的必要条件不是“块体”或“二维”, 而是:

1. 体系存在 SOC;
2. 对应磁性原子对之间的交换路径没有反演中心;
3. 该键的局域对称性允许对应方向的 $(\mathbf{D})_{ij}$ 分量。

即使整个晶体是非中心对称, 也不代表所有磁性键都有 DMI。反过来, 即使整体晶体具有反演对称, 也可能在局域非中心对称子晶格中讨论 hidden / staggered DMI, 但这时需要更谨慎地处理对称性和磁结构。

► 2. 块体材料不能只看层内最近邻

块体材料至少要区分:

代码清单 A.6.27

语言: shell; 完整代码见附录第 386 页, 共 5 行。

► 3. 四态法要除以等价键数量

如果四态能量差对应的是一个超胞总能, 则提取单键 DMI 时必须除以该超胞中对应等价键数量 (N)。否则得到的是总贡献, 不是单键 ($\mathbf{D}_{ij}$)。

► 4. 哈密顿量符号要统一

常见写法有两种:

$$[H_{\mathrm{DMI}}] = \sum_{ij} \mathbf{D}_{ij} \cdot (\mathbf{e}_i \times \mathbf{e}_j)$$

和

$$[H_{\mathrm{DMI}}] = -\sum_{ij} \mathbf{D}_{ij} \cdot (\mathbf{e}_i \times \mathbf{e}_j)$$

两种写法会导致 ($\mathbf{D}$) 的符号相反。因此, 最终论文中必须明确采用哪一种定义。

► 推荐阅读顺序

如果只是为了快速建立计算方法, 可以按下面顺序阅读:

代码清单 A.6.28

语言: shell; 完整代码见附录第 386 页, 共 17 行。

► 对复杂块体材料的建议

对于类似 Cs₃V₉Te₁₃ 这种 V 原子较多、交换路径复杂、层状但仍属于块体周期结构的体系, 更推荐优先采用:

代码清单 A.6.29

语言: shell; 完整代码见附录第 387 页, 共 1 行。

原因是:

1. 可以一次性输出多个近邻壳层的 (J_{ij}) 和 ($\mathbf{D}_{ij}$);
2. 能区分不同 V-V 键、不等价 V 原子以及层内/层间路径;
3. 比手动四态映射更不容易混入多个键的贡献;
4. 输出结果更适合后续导入 Spirit、Vampire 或 Monte Carlo / spin dynamics 程序。

如果只是为了在支撑材料中给出一个清楚、可复现的 DMI 提取公式, 则可以采用:

代码清单 A.6.30

语言: shell; 完整代码见附录第 387 页, 共 1 行。

如果目的是得到连续模型中的有效 DMI 常数, 并与螺旋周期或 skyrmion 尺寸联系, 则可以采用:

代码清单 A.6.31

语言: shell; 完整代码见附录第 387 页, 共 1 行。

► 参考文献

代码清单 A.6.32

语言: text; 完整代码见附录第 387 页, 共 54 行。

结构建模与计算方法

本专题收录 1 篇文章。

3.1 vasp 结构优化

日期: 2025-09-26

在线原文: marx2001.github.io

对于磁性材料的结构优化相比于非磁材料来说, 复杂程度有所提高。本文区别于全网介绍结构优化的步骤的文章, 详细的提供两种进阶的结构优化方式。

► 1. 第一种结构优化方式

首先进行非磁结构优化, 即正常的结构优化 (`isif=3`), 确保结构在没有磁相互作用的情况下达到最稳定的状态
再进行静态自洽计算, 即, 使用优化后的结构, 分别对铁磁和反铁磁的磁构型进行静态自洽计算 (`isif=2, nsw=0, ibrion=-1`)

最后比较设置的几种磁构型的能量, 最低的那个被认为是基态。

优势: 这种方法可以确保所有的磁构型都是在相同结构的基础上比较的, 因此能量的比较是很公平的。

适用范围: 磁性对结构影响不大的材料

缺点: 如果磁性对结构有显著影响, 那么这个方法并不准确。

► 2. 第二种结构优化的方式

首先设置不同的磁构型, 分别进行结构优化 (`isif=3, nsw=400, ibrion=2`)

然后进行能量的比较。

缺点: 对每种磁构型单独优化, 计算量大

► 3. 个人选择

对于完全陌生的材料, 我个人会选择 `isif=3`, 逐步结构优化, 设置力和能量的收敛标准为 $1e-6$ 和 -0.01 , 这个标准已经足够精确, 并可以用于其他参数的计算。

然后基于优化结果计算材料的 J , 因为计算 J 的时候会设置四种磁构型, 所以得到 J 的数值后, 同时也可以得到磁基态。以上是我通常的做法。

更为极致严谨的做法,应当采用 1 和 2 中的方法的进一步融合,即完成上述步骤后,再根据磁基态,开启自旋极化和 SOC 进行结构优化。

我的通常做法进行结构优化的 INCAR 设置如下:

代码清单 A.7.1

语言: shell; 完整代码见附录第 388 页, 共 22 行。

由于要逐步优化,所以依次减小能量的收敛标准为 $1e-6$ 、 $1e-8$, 力的收敛标准为 -0.1、-0.05、-0.01、-0.001, 其中 $1e-6$ 搭配 -0.05 和 -0.01 优化两次。

将脚本放到计算目录下,此时目录中要有 vasp 基本的输入文件。提交任务的脚本为:

代码清单 A.7.2

语言: shell; 完整代码见附录第 389 页, 共 27 行。

第二部分

软件学习

Wannier90 与电子结构工具

独立整理科研软件与工具链的安装、配置、参数逻辑和测试记录, 覆盖 Wannier90、TB2J、pyw90、pythTB、WannierTools、VASP2KP、IRVSP、pybinding、Materials Studio、LaTeX 与网站维护。

本专题收录 19 篇文章。

4.1 Wannier90 学习笔记

日期: 2025-11-21

在线原文: marx2001.github.io

► 心得笔记

► 方法论

wannier90 根据 Marzari 和 Vanderbilt (MV) 的方法 1 计算最大局域化 Wannier 函数 (MLWF)。对于纠缠能带, 则使用 Souza、Marzari 和 Vanderbilt (SMV) 的方法 2。我们在此简要介绍这些方法和关键定义, 但完整细节可在原始论文及参考文献 3 中找到。

第一性原理代码通常用布洛赫态 $\psi_{\mathbf{k}n}$ 来求解周期性材料的电子结构。这些扩展态由能带索引 n 和晶体动量 $\mathbf{k}$ 表征。另一种表示方式是利用称为 Wannier 函数 (WF) 的空间局域函数。

迄今为止, wannier90 已与基于平面波和赝势 (范数守恒和超软赝势) 的电子结构代码以及像 FLAPW 这样的混合基组技术结合使用。

Wannier90 的主要目的是将第一性原理计算中得到的扩展的电子态 (布洛赫波函数), 转换成一类在空间中高度局域化的函数, 即最大局域化 Wannier 函数。这种局域化的表示方法对于分析化学键、计算电子传输性质等非常有用。

从扩展的波函数变换到局域化的波函数, 这个过程存在一个“规范选择”问题, 即存在无数种数学上等价但局域化程度不同的变换方式。

Wannier90 的解决方法 (MV 方法) 是: 通过数学优化, 寻找一个最佳的规范选择, 使得最终得到的 Wannier 函数在空间上的“展宽”最小, 从而实现“最大局域化”。这个总展宽可以分解为三部分: 一部分是固有的、无法通过优化减少的展宽; 另外两部分分别与单个函数在不同位置的重叠和不同函数之间的重叠有关, 优化过程就是通过调整规范来最小化这两部分。

Wannier90 本身不进行初始的电子结构计算, 它需要从其他第一性原理软件 (如 VASP、QE 等) 获取结果作为输入, 主要包括两种信息:

重叠积分: 描述不同 $\mathbf{k}$ 点 (动量空间中的点) 的布洛赫态之间的关系, 这是计算 Berry 相位和展宽的关键。

投影: 作为优化的初始猜测。它将布洛赫态投影到用户提供的试局域轨道 (通常类似于原子轨道) 上, 帮助算法找到一个好的起点, 加速收敛。

这些输入数据和软件内部优化的矩阵都是小型矩阵, 并且与第一性原理计算所使用的具体数值基组 (如平面波) 无关。因此, Wannier90 可以与多种主流的计算软件兼容合作。

之前描述的方法适用于彼此分离的“孤立能带”(如绝缘体中的价带)。但在许多材料 (如金属) 中, 能带是相互重叠或“纠缠”在一起的, 无法简单分离出一组特定的能带进行转换。为了解决这个问题, 软件引入了一种称为“解缠”的方法。其核心思想是:

首先, 用户需要定义一个较大的能量范围, 称为“外窗口”。

然后, 软件会在这个窗口内的所有电子态中, 通过特定的数学变换, 主动“挑选”或“构造”出一组数量固定且能量分布平滑的“最佳”电子态子集。这个变换过程旨在让这些被选出的态在空间中尽可能容易地变得局域化。

为了确保计算的准确性, 特别是要精确保留材料在某个关键能量区域 (例如费米能级附近) 的性质, 该方法引入了第二个能量窗口, 称为“内窗口”或“冻结窗口”。

冻结窗口: 落在这个较小窗口内的原始电子态将原封不动、强制性地被包含在最终选出的“最佳子集”中, 不允许被混合或改变。

外窗口: 在冻结窗口之外、外窗口之内的其余电子态, 则参与上述的优化选择过程, 用于填补所需电子态数量的空缺。

总结来说, 处理纠缠能带是一个两步走的过程: 先通过“解缠”在大的能量窗口内智能地挑选出一个好的电子态子集, 同时利用“冻结窗口”来保护关键物理性质; 然后, 再对这个优化后的子集使用标准方法计算其最大局域化 Wannier 函数。

► 参数

使用方法 wannier90.x 可以使用 MPI 库进行并行运行, 以减少计算时间。

串行执行命令:

wannier90.x [-pp] [种子名]

种子名: 如果提供了种子名字符串, 程序将从名为种子名.win 的文件中读取其输入。默认值为 wannier。也可以等价地提供字符串种子名.win 来代替种子名。

-pp: 此可选标志指示程序生成所需的重叠积分列表, 然后退出。此信息将被写入到文件种子名.nnkp 中。

并行执行命令:

mpirun -np 进程数 wannier90.x [-pp] [种子名]

进程数: 替换为计划使用的处理器数量。

需要注意, mpirun 命令及其命令行选项可能随 MPI 实现而异: 应查阅相应的 MPI 手册或咨询计算机管理员。

同时需要确保 wannier90.x 可执行文件已编译为其并行版本 (可遵循 wannier90 发行版主目录下 README.install 文件中的说明), 并且 MPI 库和二进制文件已正确安装并配置在当前计算机上。

seedname.win 文件

wannier90 的输入文件 seedname.win 采用灵活的自由格式结构。

关键字的顺序不重要。

字母大小写不敏感 (因此 num_bands 与 Num_Bands 是相同的)。

在! 或 # 符号之后的字符将被视为注释。

大多数关键字都有默认值; 除非在 `seedname.win` 文件中给出了关键字, 否则将使用其默认值。

关键字可以通过以下任何一种方式设置:

代码清单 A.8.1

语言: `shell`; 完整代码见附录第 389 页, 共 3 行。

可以将逻辑关键字设置为 `true`, 可以使用以下任意字符串表示: `T`, `true`, `.true.`。

关键词 (Keyword)	类型 (Type)	描述 (Description)
<code>num_wann</code>	整数 (I)	Wannier 函数的数量
<code>num_bands</code>	整数 (I)	传递给代码的能带数量
<code>unit_cell_cart</code>	物理量 (P)	晶胞矢量在笛卡尔坐标下的表示 (单位: 埃)
<code>atoms_cart</code>	字符串, 物理量 (S,PPP)	原子在笛卡尔坐标下的位置
<code>atoms_frac</code>	字符串, 实数 (S,RRR)	原子相对于晶格矢量的分数坐标位置
<code>mp_grid</code>	整数 (I)	Monkhorst-Pack k 点网格的维度
<code>kpoints</code>	实数 (R)	Monkhorst-Pack 网格中的 k 点列表
<code>gamma_only</code>	逻辑值 (L)	底层第一性原理计算的波函数是实函数
<code>spinors</code>	逻辑值 (L)	Wannier 函数是旋量
<code>shell_list</code>	整数 (I)	在有限差分公式中要使用的壳层
<code>search_shells</code>	整数 (I)	确定有限差分公式时要搜索的壳层数量
<code>skip_B1_tests</code>	逻辑值 (L)	检查参考文献中的 B1 条件
<code>nnkpts</code>	整数 (I)	最近邻 k 点的显式列表
<code>kmesh_tol</code>	实数 (R)	控制两个 k 点是否属于同一壳层的容差
<code>higher_order_n</code>	整数 (I)	用于获取 b 矢量和权重的更高阶有限差分的阶数
<code>higher_order_nearest_shells</code>	逻辑值 (L)	在最近的壳层上使用 b 矢量

`seedname.win` 文件关键词: 系统定义

用于定义系统的 `seedname.win` 文件关键词。参数类型表示为: I 代表整数, R 代表实数, P 代表物理量, L 代表逻辑值, S 代表文本字符串。

注意: 在同一个输入文件中, 不能同时定义 `atoms_cart` 和 `atoms_frac` 这两个关键词。

► 任务控制参数

关键词 (Keyword)	类型 (Type)	描述 (Description)
postproc_setup	逻辑型 (L)	输出 seedname.nnkp文件
exclude_bands	整型 (I)	计算中要排除的能带列表
select_projections	整型 (I)	Wannier化过程中要使用的投影列表
auto_projections	逻辑型 (L)	自动生成初始投影
restart	字符串型 (S)	从检查点文件重启计算
iprint	整型 (I)	输出信息的详细程度级别
length_unit	字符串型 (S)	输出长度时使用的单位制
wvfn_formatted	逻辑型 (L)	从 (非) 格式化文件读取波函数
spin	字符串型 (S)	要读取的自旋通道
devel_flag	字符串型 (S)	供开发使用的标志
timing_level	整型 (I)	决定写入输出文件的时间信息量
optimisation	整型 (I)	优化级别
translate_home_cell	逻辑型 (L)	在写入 xyz 文件时, 将最终的 Wannier 中心平移至原胞内
write_xyz	逻辑型 (L)	以 xyz 文件格式写入原子位置和最终的中心坐标
write_vdw_data	逻辑型 (L)	写入供 w90vdw工具进一步处理的数据
write_hr_diag	逻辑型 (L)	将 Wannier 基组下的哈密顿量对角元 (以 eV 为单位) 写入 seedname.wout文件

seedname.win 文件关键字: 作业控制

用于定义作业控制的 seedname.win 文件关键字。参数类型表示为:I 代表整数,R 代表实数,P 代表物理量,L 代表逻辑值,S 代表文本字符串。

注意:translate_home_cell 仅在 write_xyz 为.true. 时相关。

► 解纠缠参数

解纠缠参数		
关键词 (Keyword)	类型 (Type)	描述 (Description)
dis_win_min	物理量 (P)	外能量窗口的下限 (eV)
dis_win_max	物理量 (P)	外能量窗口的上限 (eV)
dis_froz_min	物理量 (P)	内 (冻结) 能量窗口的下限 (eV)
dis_froz_max	物理量 (P)	内 (冻结) 能量窗口的上限 (eV)
dis_frozproj	逻辑型 (L)	启用投影度解纠缠方法
dis_proj_min	实数型 (R)	投影度解纠缠的下限阈值
dis_proj_max	实数型 (R)	投影度解纠缠的上限阈值
dis_num_iter	整型 (I)	最小化 Ω 的迭代次数
dis_mix_ratio	实数型 (R)	最小化 Ω 过程中的混合比率
dis_com_tol	实数型 (R)	Ω 收敛的判断容差
dis_com_window	整型 (I)	判断 Ω 收敛所依据的迭代次数窗口
dis_aspheres_num	整型 (I)	在k空间中进行解纠缠的球区域数量
dis_aspheres_first_l_wann	整型 (I)	被视为Wannier函数的起始能带索引
dis_aspheres	实数 实数 实数 物理量 (R,R,R,P)	球区域中心坐标和半径的列表，用于限定解纠缠计算的范围

seedname.win 文件中控制解纠缠的关键字

关键字参数类型说明如下：

I 代表整数

R 代表实数

P 代表物理量

L 代表逻辑值

S 代表文本字符串

► 优化参数

关键词 (Keyword)	类型 (Type)	描述 (Description)
num_iter	整型 (I)	最小化 Ω 的总迭代次数
num_cg_steps	整型 (I)	在最小化 Ω 过程中, 重置为最速下降法之前执行的共轭梯度步数
conv_window	整型 (I)	判断 Ω 是否收敛所依据的迭代次数窗口
conv_tol	物理量 (P)	寻找 Ω 时的收敛容差 (单位: 埃 ²)
precond	逻辑型 (L)	使用预条件处理
conv_noise_amp	实数型 (R)	在最小化过程接近结束时施加的随机噪声的振幅
conv_noise_num	整型 (I)	施加随机噪声的次数
num_dump_cycles	整型 (I)	控制检查点保存的频率
num_print_cycles	整型 (I)	控制信息打印输出的频率
write_r2mn	逻辑型 (L)	将 Wannier 函数之间 r^2 的矩阵元写入文件
guiding_centres	逻辑型 (L)	使用引导中心
num_guide_cycles	整型 (I)	使用引导中心的频率
num_no_guide_iter	整型 (I)	在指定迭代次数之后才开始使用引导中心
trial_step	实数型 (R)	在最小化 Ω 的抛物线搜索中使用的尝试步长
fixed_step	实数型 (R)	在最小化 Ω 过程中使用的固定步长 (代替抛物线搜索)
use_bloch_phases	逻辑型 (L)	为初始投影使用布洛赫相位

高级与约束优化参数

关键词 (Keyword)	类型 (Type)	描述 (Description)
site_symmetry	逻辑型 (L)	用于构建对称性适应的 Wannier 函数
symmetrize_eps	实数型 (R)	在对称性适应模式下使用的收敛容差
slwf_num	整型 (I)	用于选择性局域化的目标 Wannier 函数数量
slwf_constrain	逻辑型 (L)	是否约束目标 Wannier 函数的中心位置
slwf_lambda	实数型 (R)	用于约束目标 Wannier 函数中心的拉格朗日乘数值
slwf_centres	整型, 实数, 实数, 实数 (I,R,R,R)	目标 Wannier 函数被约束到的中心坐标列表
use_ss_functional	逻辑型 (L)	使用 Stengel-Spaldin 泛函替代 Marzari-Vanderbilt 泛函

L (Logical): 逻辑值 (是/否)

R (Real): 实数

I (Integer): 整数

I,R,R,R: 表示需要提供一组参数, 通常为一个整数 (如函数索引) 后跟三个实数 (如空间坐标)

高级与约束优化参数

关键词 (Keyword)	类型 (Type)	描述 (Description)
site_symmetry	逻辑型 (L)	用于构建对称性适应的 Wannier 函数
symmetrize_eps	实数型 (R)	在对称性适应模式下使用的收敛容差
slwf_num	整型 (I)	用于选择性局域化的目标 Wannier 函数数量
slwf_constrain	逻辑型 (L)	是否约束目标 Wannier 函数的中心位置
slwf_lambda	实数型 (R)	用于约束目标 Wannier 函数中心的拉格朗日乘数值
slwf_centres	整型, 实数, 实数, 实数 (I,R,R,R)	目标 Wannier 函数被约束到的中心坐标列表
use_ss_functional	逻辑型 (L)	使用 Stengel-Spaldin 泛函替代 Marzari-Vanderbilt 泛函

► 绘图参数

控制绘图的参数

关键词 (Keyword)	类型 (Type)	描述 (Description)
wannier_plot	逻辑型 (L)	生成 Wannier 函数的实空间绘图
wannier_plot_supercell	整型 (I)	绘图所采用的超晶格大小
wannier_plot_list	整型 (I)	指定需要绘制的 Wannier 函数索引列表
wannier_plot_format	字符串型 (S)	绘图文件的输出格式
wannier_plot_mode	字符串型 (S)	绘图模式
wannier_plot_radius	物理量 (P)	绘图区域的半径
wannier_plot_scale	实数型 (R)	等值面的缩放因子
wannier_plot_frac	实数型 (R)	用于调整等值面的分数值

关键词 (Keyword)	类型 (Type)	描述 (Description)
wannier_plot	L	绘制 Wannier 函数
wannier_plot_list	I	指定要绘制的 Wannier 函数列表
wannier_plot_supercell	I	绘制 Wannier 函数时所采用的超晶胞大小
wannier_plot_format	S	Wannier 函数绘图文件的输出格式
wannier_plot_mode	S	Wannier 函数的绘图模式 (分子模式或晶体模式)
wannier_plot_radius	R	Wannier 函数的截断半径
wannier_plot_scale	R	立方格文件格式的缩放参数
wannier_plot_spinor_mode	S	旋量 Wannier 函数中要绘制的物理量
wannier_plot_spinor_phase	L	绘制旋量 Wannier 函数时是否包含“相位”信息
bands_plot	L	绘制插值能带结构
kpoint_path	S,R,R,R,S,R,R,R	插值能带结构计算所采用的 k 点路径
bands_num_points	I	k 点路径第一段上的点数
bands_plot_format	S	插值能带结构图的文件输出格式
bands_plot_project	I	将能带结构投影到指定的 Wannier 函数上
bands_plot_mode	S	插值方法 (Slater-Koster 类型插值或哈密顿量截断法)
bands_plot_dim	I	体系的维度
fermi_surface_plot	L	绘制费米面
fermi_surface_num_points	I	费米面绘图中的点数
fermi_energy	P	费米能 (eV)
fermi_energy_min	P	费米能范围的下限 (eV)
fermi_energy_max	P	费米能范围的上限 (eV)
fermi_energy_step	R	在指定费米能范围内递增的步长

bands_plot_project	I	将能带结构投影到指定的 Wannier 函数上
bands_plot_mode	S	插值方法 (Slater-Koster 类型插值或哈密顿量截断法)
bands_plot_dim	I	体系的维度
fermi_surface_plot	L	绘制费米面
fermi_surface_num_points	I	费米面绘图中的点数
fermi_energy	P	费米能 (eV)
fermi_energy_min	P	费米能范围的下限 (eV)
fermi_energy_max	P	费米能范围的上限 (eV)
fermi_energy_step	R	在指定费米能范围内递增的步长
fermi_surface_plot_format	S	费米面绘图的文件输出格式

输出控制参数

关键词 (Keyword)	类型 (Type)	描述 (Description)
hr_plot	L	此参数已停用, 请使用 write_hr 代替。
write_hr	L	输出 Wannier 函数基组下的哈密顿量。
write_amn	L	输出 Wannier 函数基组下的位置算符。
write_bvec	L	将 b 矢量的矩阵元及其权重写入文件。
write_tb	L	输出晶格矢量、哈密顿量及位置算符 (适用于 Wannier 函数基组)。
hr_cutoff	P	哈密顿量绝对值的截断值 (单位: eV)。
dist_cutoff	P	Wannier 函数间距离的截断值 (单位: 埃)。
dist_cutoff_mode	S	计算 Wannier 函数间距离时所考虑的体系维度。
translation_centre_frac	R	最终 Wannier 函数被平移至的晶胞中心 (分数坐标)。
use_ws_distance	L	使用 Wannier 函数间的最小距离来改进插值精度 (参见文档相关章节)。
ws_distance_tol	R	判断等效位置距离的绝对容差。
ws_search_size	I	在维格纳-赛茨原胞中搜索点时, 所使用玻恩-冯-卡门超晶胞在每个方向上的最大扩展倍数。
write_u_matrices	L	将 $U^{(k)}$ 和 $U^{dis(k)}$ 矩阵写入文件。

► 输运参数

关键词 (Keyword)	类型 (Type)	描述 (Description)
fermi_energy	P	费米能 (eV)
hr_cutoff	P	哈密顿量绝对值的截断值 (eV)
dist_cutoff	P	Wannier函数 (WF) 之间的距离截断值 (埃)
dist_cutoff_mode	S	计算Wannier函数之间距离时所考虑的体系维度
translation_centre_frac	R	最终Wannier函数被平移至的晶胞中心 (分数坐标)
transport	L	计算量子电导和态密度
transport_mode	S	计算模式: 体相 或 左电极-导体-右电极
tran_win_min	P	输运计算能量窗口的下限 (eV)
tran_win_max	P	输运计算能量窗口的上限 (eV)
tran_energy_step	R	能量值的采样间隔
tran_num_bb	I	体相哈密顿量的尺寸
tran_num_ll	I	左电极哈密顿量的尺寸
tran_num_rr	I	右电极哈密顿量的尺寸
tran_num_cc	I	导体哈密顿量的尺寸
tran_num_lc	I	左电极-导体哈密顿量的列数
tran_num_cr	I	导体-右电极哈密顿量的行数
tran_num_cell_ll	I	左电极原胞层 (PL) 中的原胞数量
tran_num_cell_rr	I	右电极原胞层 (PL) 中的原胞数量
tran_num_bandc	I	带状对角导体哈密顿量的半带宽+1
tran_write_ht	L	输出用于输运计算的哈密顿量

输运参数 (续)

关键词 (Keyword)	类型 (Type)	描述 (Description)
tran_read_ht	L	读取用于输运计算的哈密顿量
tran_use_same_lead	L	左电极和右电极相同
tran_group_threshold	R	决定Wannier函数分组的距离阈值
one_dim_axis	S	一维体系的扩展方向

4.2 以 Bi 烯为例的 wannier90 表面态计算

日期: 2025-11-22

在线原文: marx2001.github.io

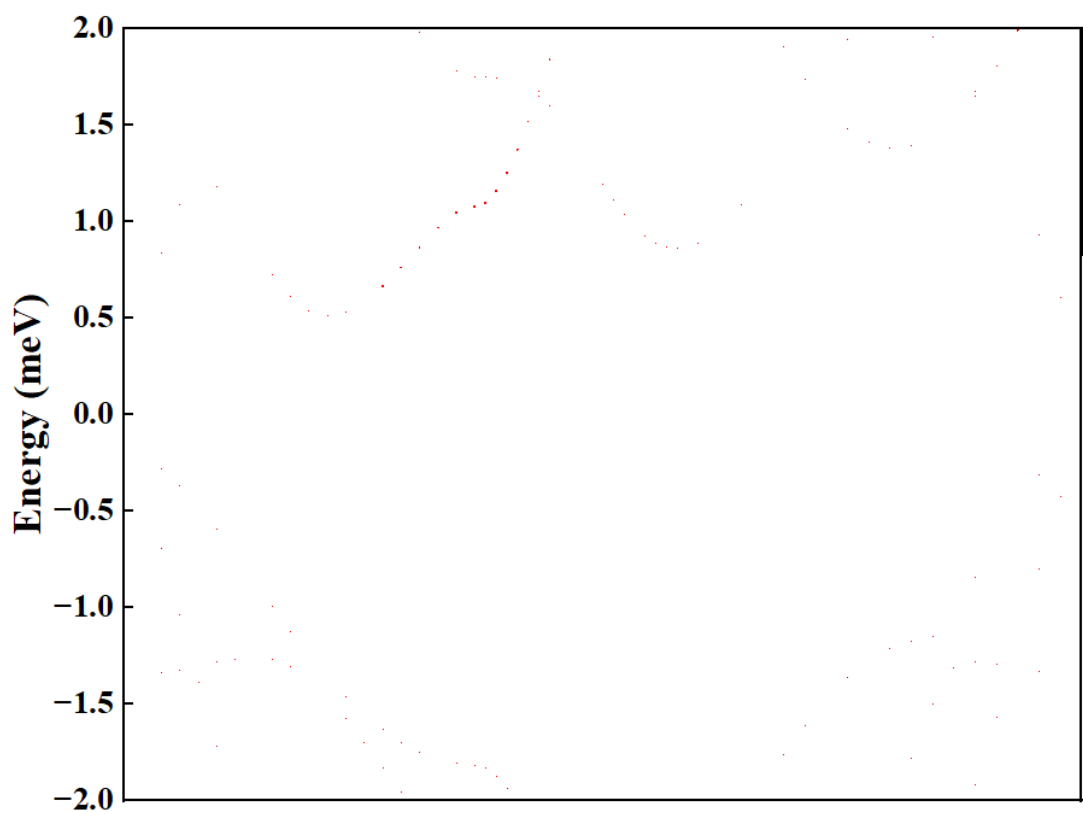
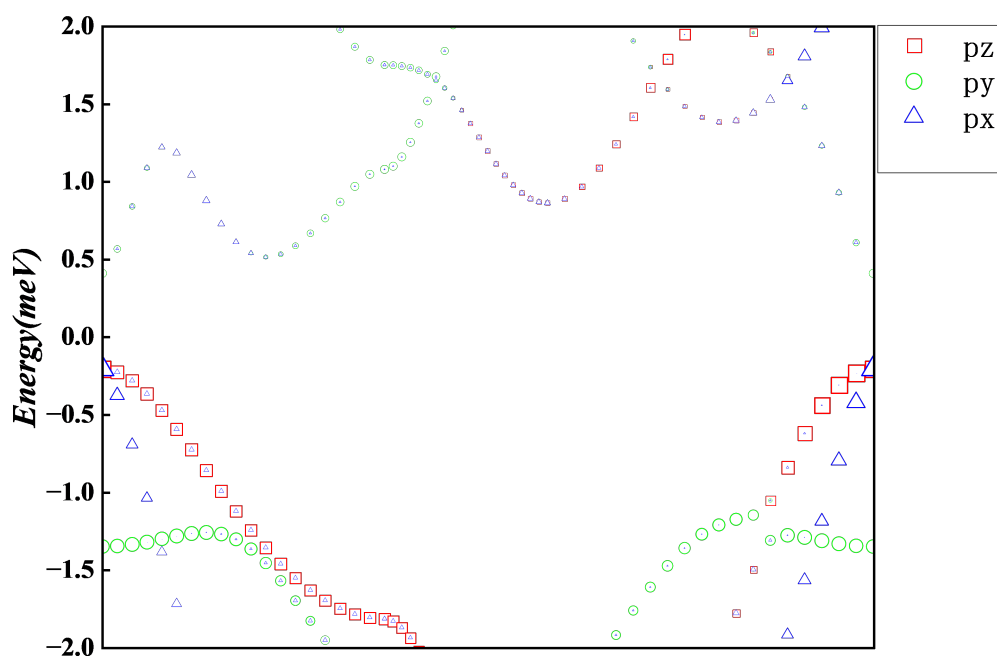
► 心得笔记

-免责声明: 私人笔记-不传播 -原作者示例来源于公众号“计算凝聚态物理”, 本笔记是用于学习、复现此公众号中的教程, 并记录本人遇到的问题和步骤以及心得体会, 如在互联网上检索到本文章, 请勿用作他途。

注意, 安装 6.3.0 版本以上的 vasp, 不包含 6.3.0 的 vasp, 并且编译的时候不要注释掉并行计算,wannier3.1.0 版本是支持并行计算的, 如果注释掉的话, 会报错。

► 1. 能带计算

计算得到 BAND.dat,KLABELS, 查看能带中的轨道贡献, 通过投影能带, 我们需要投影 Bi 原子的 p 轨道, 原胞有两个原子, 每个原子 6 个 p 轨道, 我们需要拟合 12 个 wannier 轨道。



► 2. 静态自洽计算 (注意添加 LWANNIER90=.T)

准备文件

 1.pbs	2KB	PBS 文件	2025/11/22, 12:33
 INCAR	349 Bytes	文件	2025/11/22, 12:32
 KPOINTS	40 Bytes	文件	2025/11/22, 12:32
 POSCAR	555 Bytes	文件	2025/11/22, 12:32
 POTCAR	230KB	文件	2025/11/22, 12:33
 wannier90.win	60 Bytes	WIN 文件	2025/11/22, 12:32

win 文件内容:

命令示例 · shell

```
num_wann = 64 ! set to NBANDS by VASP
kmesh_tol=0.00001
```

NBANDS = 64(该数值应为 CPU 核心数的整数倍; 此处使用 64 核提交), 并在 INCAR 中加入 LWANNIER90 = T。

INCAR 文件内容:

代码清单 A.9.2

语言: shell; 完整代码见附录第 389 页, 共 30 行。

► 3.vasp 非自洽计算, 拟合 Wannier90

win 文件中加入, 同时修改 ICHGCAR=11

代码清单 A.9.3

语言: shell; 完整代码见附录第 390 页, 共 15 行。

这个步骤需要反复调试能量窗口, 如果成功的话, 会产生新的文件。wannier90.amn, wannier90.eig, wannier90.mmn, wannier90.win, wannier90.wout

► 4. Wannier90 计算

修改 wannier90.win, 上一步基础上, 增加

代码清单 A.9.4

语言: shell; 完整代码见附录第 390 页, 共 10 行。

这里 bands_num_points = 20 跟 VASPKIT 中能带计算的 KPOINTS 中 K 点数要相同 (如下 KPOINTS 的前三行, 第二行中 K 点密度是 20, 这么做是为了方便对比 VASP 和 wannier90 的能带)

提交 wannier90 的计算:

命令示例 · shell

```
wannier90.x wannier90 &
```

► 5. 对比 vasp 和 Wannier90 的能带

找到能带计算的费米能级:

命令示例 · shell

```
grep E-fermi OUTCAR
```

查看费米能级后, 将能带计算的 BAND.dat 和 KLABELS 放到 wannier90 计算的文件夹中, 修改 compare_wan_vasp.py 中的 efermi, 然后运行。

► 6.WannierTools 计算边缘态和陈数

(1) 边缘态 以 zig-zag 方向为例, 展示计算得到的边界态参照 wannier90.win 中的信息, 修改 wannierTools 的输入文件 wt.in:

代码清单 A.9.7

语言: shell; 完整代码见附录第 390 页, 共 63 行。

运行 WannierTools:

命令示例 · shell

```
wt.x
```

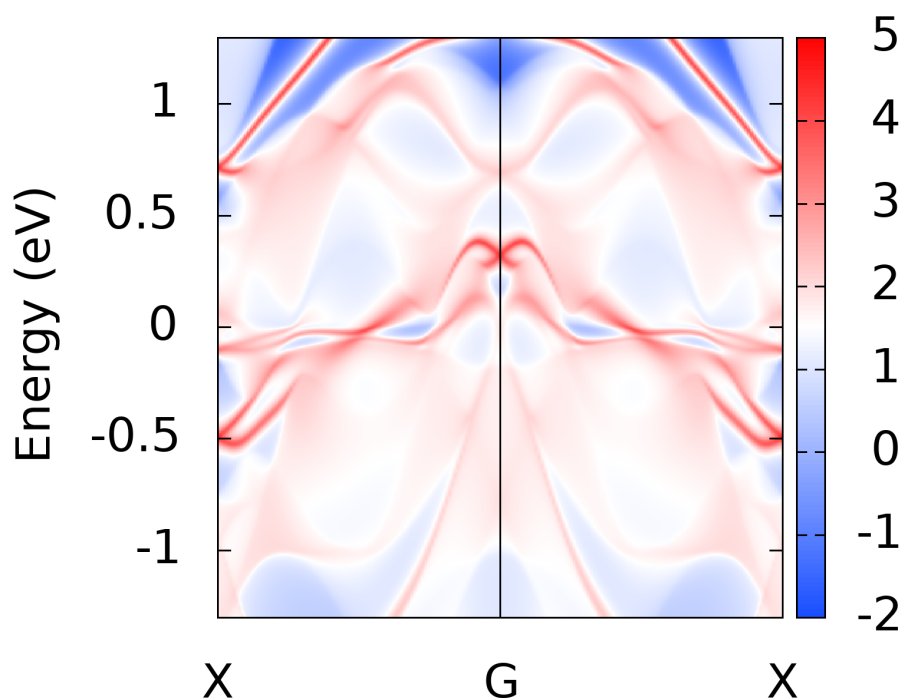
运行完后绘图:

命令示例 · shell

```
gnuplot *.gnu
```

 dos.dat_bulk	3.79MB	DAT_BULK 文件	2025/11/25, 14:57
 dos.dat_l	5.02MB	DAT_L 文件	2025/11/25, 14:57
 dos.dat_r	5.02MB	DAT_R 文件	2025/11/25, 14:57
 POSCAR-rotated	284 Bytes	文件	2025/11/25, 14:56
 POSCAR-slab	1KB	文件	2025/11/25, 14:56
 surfdos_bulk.gnu	974 Bytes	GNU 文件	2025/11/25, 14:57
 surfdos_l.gnu	944 Bytes	GNU 文件	2025/11/25, 14:57
 surfdos_l_only.gnu	967 Bytes	GNU 文件	2025/11/25, 14:57
 surfdos_r.gnu	961 Bytes	GNU 文件	2025/11/25, 14:57
 surfdos_r_only.gnu	984 Bytes	GNU 文件	2025/11/25, 14:57
 wannier90_hr.dat	77KB	Materials Studio ...	2025/11/25, 14:50
 wt.in	2KB	IN 文件	2025/11/25, 14:50
 WT.out	18KB	Materials Studio ...	2025/11/25, 14:57

我复现的边缘态:



与公众号的文件不同,很奇怪,但原因应该是我的拟合的轨道不同,我设置的 `Bi:l=1`

一些可能的原因: 我的 `NBANDS` 设置是 64, 而示例中是 40, 导致我的拟合能带和 DFT 能带有一些差距。这个保持质疑, 我始终按照博主给的 `INCAR` 进行计算, 但是始终得不到和公众号一样的带隙。开 `ISPIN` 会让费米能级升高。加入下面 `tag` 后, 费米能级继续升高:

命令示例 · shell

```
EMIN=-15
EMAX=15
LORBIT=11
NEDOS=3000
```

开 SOC 后, 费米能级上升了

不用管了, 不知道是什么原因, 导致的费米能级无法复现, 我们只需要知道整体步骤即可:

使用由 VASP 生成的 `wannier90.amn`、`wannier90.mmn`、`wannier90.eig` 和 `wannier90.win` 文件, 生成最大局域瓦尼尔函数。

在后台运行 Wannier90, 命令为: `wannier90.x wannier90 &`。

这将生成一系列文件, 我们所需的是 `wannier90.chk`(检查点文件)。

使用得到的瓦尼尔函数计算体材料的能带结构, 并将其与通过 DFT 计算得到的能带进行比较。

ERATE_KPOINTS_TRANS: number of G-vector || changed in star 7778 7776 |||| If you are not a developer, you should not encounter this problem. || Please submit a bug report. |||

k 点取偶数即可规避。

命令示例 · shell

```
conda deactivate
```

退出环境

4.3 wannier90 汇总

日期: 2025-11-23

在线原文: marx2001.github.io

► 心得笔记

拟合完的指令:wannier90.x wannier90 & 激活 TB2J:source /public/home/cssong/song/zhchen/1_materials/1_VTeCl/TB2J/strain/-2/TB2J_env/bin/activate 进入 pyw90 环境:conda activate pyw90_env pyw90 eig suggest -path . -e -20 20 -w 20 在选定能量区间内计算推荐的内外能量窗口的范围

求平均值:TB2J_merge.py -type structure x y z

求磁耦合参数:wann2J.py -spinor -groupby spin -efermi -1.0098 -emin -2.06514 -emax -0.065 -elements Nb -kmesh 9 9 1 -nz 100 -rcut 6.0 -np 1 第一次测试

安装计算 SHC 的环境:conda install -c conda-forge wannierberri numpy scipy matplotlib h5py sympy -y

计算自旋霍尔电导:postw90.x wannier90

4.4 Wannier90 的表面态的参数选择标准 1_示例文件的参数注释

日期: 2025-11-25

在线原文: marx2001.github.io

计算表面态

代码清单 A.11.1

语言: shell; 完整代码见附录第 394 页, 共 63 行。

► 参数选择标准

代码清单 A.11.2

语言: shell; 完整代码见附录第 395 页, 共 10 行。

 取值标准 (经验规则)

体系类型	推荐取值范围	说明	
普通金属、拓扑金属	0.002 ~ 0.01 eV	较大展宽, 用于平滑噪声	
拓扑绝缘体 (体能隙明显)	0.0005 ~ 0.002 eV	保留清晰狄拉克锥与边界态结构	
粗略测试或调试	0.01 ~ 0.05 eV	加快收敛但牺牲精度	

一般推荐从 `Eta_Arc = 0.001` 开始。

若结果表面态“模糊”或“重叠”, 可减小; 若表面态谱图出现噪点或不连续, 可适当增大。

E_arc: 实数值, 用于计算费米弧的能量, 当SlabArc_calc为T时使用。默认值为0.0。

Eta_Arc: 实数值, 用于格林函数计算中宽化时的无限相同值。当SlabArc_calc = T时使用。默认值为0.001。

[OmegaMin, OmegaMax]: 实数值, 用于表面态计算的能量区间。当SlabSS_calc=True时使用。无默认值。

OmegaNum: 整数值, 能量区间[OmegaMin, OmegaMax]内的切片数量。当SlabSS_calc=True时使用。默认值为100。

Nk1、Nk2、Nk3: 整数值, 用于不同目的的k点数量。我稍后会解释这一点。默认值为Nk1=20, Nk2=20, Nk3=20。

NP: 整数值, 主要层的数量, 参见与迭代格林函数相关的详细信息。当SlabSS_calc=T, SlabArc_calc=T, SlabSpintexture_calc=T时使用。默认值为2。您需要通过设置Np=1, Np=2, Np=3来进行收敛性测试, 并检查表面态谱。基本上, Np的值取决于您构建的Wannier函数的范围。需要提及的一点是, 计算时间随着Np的增加呈立方增长。

Gap_threshold: 实数。当进行能量间隙计算时使用此值, 例如BulkGap_cube_calc=T, BulkGap_plane_calc=T。当能量间隙小于Gap_threshold时, k点将被打印到一个文件中。

参数说明 (来自 *WannierTools* 用户手册)

E_arc: 实数类型。

用于计算 费米弧 (Fermi arc) 的能量值, 仅当 `slabArc_calc = T` 时启用。

默认值为 0.0。

Eta_Arc: 实数类型。

在格林函数计算中用作展宽因子的“无限小”参数, 仅当 `slabArc_calc = T` 时启用。

默认值为 0.001。

(该参数决定谱函数的能量分辨率和平滑程度。)

[OmegaMin, OmegaMax]: 实数类型。

定义 表面态计算的 能量范围, 仅当 `slabSS_calc = T` 时使用。

无默认值, 必须手动指定。

OmegaNum: 整数类型。

表示在能量区间 `[OmegaMin, OmegaMax]` 内划分的能量切片数。

用于表面态计算 (`slabSS_calc = T`)。

默认值为 100。

Nk1, Nk2, Nk3: 整数类型。

表示不同用途下的 k 点采样数。

(在后文中会解释每个方向的具体作用。)

默认值为 **Nk1 = 20, Nk2 = 20, Nk3 = 20**。

Np: 整数类型。

表示 主层数 (principal layers) , 用于迭代格林函数算法。

仅当以下任一开关启用时使用:

`SlabSS_calc = T` , `SlabArc_calc = T` , 或 `SlabSpintexture_calc = T` 。

默认值为 **2**。

你需要通过收敛测试确定 Np 的合适值, 例如设置 $N_p = 1, 2, 3$ 比较表面态谱的变化。

一般来说, Np 的选择取决于你构造的 Wannier 函数的空间延展范围。

注意: 计算时间随 Np 呈立方增长 ($\propto N_p^3$) 。

Gap_threshold: 实数类型。

用于能隙计算 (当 `BulkGap_cube_calc = T` 或 `BulkGap_plane_calc = T`) 。

当某个 k 点的能隙小于此阈值时, WannierTools 会将该 k 点的能量信息输出到文件中。

代码清单 A.11.3

语言: shell; 完整代码见附录第 395 页, 共 4 行。

 图 1: SURFACE CARD

SURFACE CARD

这个卡片在薄膜 (slab) 计算中非常重要。你需要仔细阅读以下内容。

nginx

 复制代码

SURFACE ! 详见文档

1 0 0

0 1 0

说明:

在这个卡片中, 我们需要指定要研究的表面。

你必须清楚自己想研究哪一个表面, 以及沿哪个方向去构建纳米带 (ribbon) 或薄膜。

因此, 需要定义一个新的晶格矢量体系, 如下所示:

- 选择位于该表面上的 **两个晶格矢量**;
- 再选取一个 **不在该平面上的矢量**。

薄膜计算将基于由向量 $\mathbf{R}'_1$, $\mathbf{R}'_2$ 构成的平面进行。

 注意:

a_{11} 、 a_{12} 、 a_{13} a_{23} 都必须是整数,
并且以三条晶格矢量 ($\mathbf{R}_1$, $\mathbf{R}_2$, $\mathbf{R}_3$) 为单位。

图 2: WannierTools SURFACE 示例

(仅适用于 V2.2.6 或更高版本)

假设我们想研究图中红色标出的 (001) 面。
我们需要找到 两个与该平面平行的晶格矢量。

第一个向量为:

$$\mathbf{R}'_1 = \mathbf{R}_2 - \mathbf{R}_1$$

第二个向量为:

$$\mathbf{R}'_2 = \mathbf{R}_3$$

因此, SURFACE 卡应写为:

```
diff
```

[复制代码](#)

```
SURFACE
```

```
-1 1 0
```

```
0 0 1
```

绿色文字:

SURFACE 中定义的向量是以 $\mathbf{R}_1$ 、 $\mathbf{R}_2$ 、 $\mathbf{R}_3$ 为单位的。

原始晶格矢量 (Primitive lattice vectors)

$$\mathbf{R}_1 = (0.0, 0.5, 0.5)$$

$$\mathbf{R}_2 = (0.5, 0.0, 0.5)$$

$$\mathbf{R}_3 = (0.5, 0.5, 0.0)$$

总结:

- SURFACE 卡定义的是薄膜构造时的晶面方向;
- 前两行指定平面内的两个矢量;
- 所有分量为整数, 单位为原始晶格矢量;
- 从 v2.2.6 版本起, 新的 SURFACE 定义方式使用上述格式。

代码清单 A.11.4

语言: shell; 完整代码见附录第 395 页, 共 5 行。

KPATH_BULK

这是用于 **体能带结构计算 (bulk band structure calculation)** 的 **k 点路径 (k path)**。

```

r
KPATH_BULK          ! k 点路径
4                    ! 仅用于体能带的k线数量
G 0.00000 0.00000 0.0000 Z 0.00000 0.00000 0.5000
Z 0.00000 0.00000 0.5000 F 0.50000 0.50000 0.0000
F 0.50000 0.50000 0.0000 G 0.00000 0.00000 0.0000
G 0.00000 0.00000 0.0000 L 0.50000 0.00000 0.0000

```

[复制代码](#)[解释](#)

这些 k 点的坐标单位为**倒易晶格常数 (reciprocal lattice constant)**，其坐标系由 **LATTICE 卡片 (LATTICE CARD)** 中的晶格矢量定义。

k 点路径中的采样点数量由参数 **Nk1** 控制，该参数在 **NAMELISTS PARAMETERS** 中设定。

此卡片 (CARD) **没有默认值**，

因此，当你设置 `BulkBand_calc = T` 时，必须在输入文件中指定该卡片的内容。

■ 简要说明:

- `KPATH_BULK` 定义体能带计算的高对称路径;
- 第一行数字 (如 4) 表示路径段数;
- 每一行定义一条路径的起点与终点 (如 G→Z、Z→F 等) ;
- 坐标为归一化的倒易空间坐标;
- 若未定义该路径, WannierTools 将无法绘制 bulk band 图。

代码清单 A.11.5

语言: shell; 完整代码见附录第 395 页, 共 4 行。

KPATH_SLAB

这是用于 **薄膜体系 (slab system)** 的 **k 点路径 (k path)**。

```

mathematica
KPATH_SLAB          ! 2D体系的k点路径
2                    ! k线数量 (2D情况)
K 0.33 0.67 G 0.0 0.0 ! 2D体系的第一条k线路径
G 0.0 0.0 M 0.5 0.5   ! 2D体系的第二条k线路径

```

[复制代码](#)[解释](#)

这部分内容用于计算 **能带结构** 和 **表面态谱图**。

当你设置 `SlabBand_calc = T` 或 `SlabSS_calc = T` 时，必须定义此路径。

沿每条路径的 k 点数由参数 **Nk1** 决定。

KPLANE_SLAB

定义一个用于 费米弧图 (arc plots) 的二维动量空间平面。

diff

复制代码

KPLANE_SLAB

```
-0.1 -0.1    ! 二维k平面的原点
 0.2  0.0    ! 定义二维k平面的第一个方向向量
 0.0  0.2    ! 定义二维k平面的第二个方向向量
```

解释

第一行表示二维平面的起始点 (原点)。

第二行和第三行定义该平面的两个方向向量。

在这两个方向上, k点的数量分别由 **Nk1** 和 **Nk2** 决定。

当以下任一开关启用时, 必须设置此卡片:

`SlabArc_calc = T` 或 `SlabSpintexture_calc = T`。

(默认值由系统自动给定, 但通常需要手动定义以匹配所需平面。)

图示解释 (来自第二张图片)

图标题: WannierTools – KPLANE_SLAB 示例

- 图中矩形代表 二维布里渊区 (2D Brillouin Zone) ;
- 蓝色阴影部分表示通过 `KPLANE_SLAB` 定义的二维切片区域。

参数说明:

mathematica

复制代码

KPLANE_SLAB

```
0.0 0.0    ! O: 该平面的起点 (原点)
0.5 0.0    ! K1: 平面的第一个方向向量
0.0 0.5    ! K2: 平面的第二个方向向量
```

解释

图中标注:

- KPLANE** 表示二维动量空间中的一个“切片”;
- SLAB** 表示整个二维动量空间区域;
- O、K₁、K₂ 分别代表该平面的起点与两个方向基矢。

4.5 wannier90 编译

日期: 2025-11-25

在线原文: marx2001.github.io

► 大概流程

复制 vasp6.4.3

tar-xvf vasp6.4.3 tar.gz

cp 配置文件到 wannier90 3.1.0 文件夹

cp make.inc

检查编译器路径并复制

nano make.inc 文件, 改 LIBDIR 到自己的 mkl 文件中

键入 make all 开始安装 (wannier90 3.1.0 文件夹中)

makefile.include(这个是 arch 文件夹中的一个文件要复制出来然后修改成这个名称)。修改 Wannier_ROOT 等内容以及 LLIBS

安装 vasp

make all

4.6 以 Bi 烯为例的 wannier90 表面态计算

日期: 2025-11-25

在线原文: marx2001.github.io

► 我的一些问题——AI 回答, 谨慎甄别

AI 引擎: Chatgpt-5.1 Auto 决定回答时长

Q1: 什么是 wannier90?

A1: Wannier90 是一个在计算材料科学和凝聚态物理中非常重要的软件包, 用于构造 Maximally Localized Wannier Functions (MLWFs, 最大局域化的 Wannier 函数)。它是第一性原理计算中连接电子能带结构与实空间局域化电子态的关键工具。固体物理中, 电子波函数通常以 Bloch 波函数的形式表示, 适用于周期性晶体。但 Bloch 函数是在动量空间局域, 实空间扩展, 不适合描述化学键或者局域电子特性。

Wannier 函数相反, 它是实空间局域化的波函数。

Wannier90 是一个独立的后处理程序, 可以从多种第一性原理计算程序, 导入计算结果, 构造 Wannier 函数。

Q2: 具体应用?

A1: 拓扑绝缘体与陈绝缘体分析、电输运与自旋输运、光学响应与 Berry 相计算、紧束缚模型参数提取、分子吸附、磁性、超导配对分析

► 表面态相关疑问

Q1: 拓扑绝缘体、高阶拓扑绝缘体以及表面态都是什么?

A1: 拓扑绝缘体是一类内部 (体) 为绝缘体、表面为导体的量子态。其本质是: 能带结构具有非平庸拓扑性质 (即, 无法连续变形为普通绝缘体), 由拓扑不变量 (如 Z_2 、Chern 数) 区分。电子结构特征: 体态 (bulk): 具有能隙 (如

普通绝缘体); 表面态 (surface state): 存在受保护的导电态; 能带倒转 (band inversion): 由于强自旋轨道耦合 (SOC), 价带与导带在能量序上“翻转”, 产生拓扑非平庸结构。可以理解为: “内部像绝缘体, 表皮却像金属。”

Q2: 为什么计算二维体系的表面态, 要计算 bulk 能带呢? 二维体系的是否具有不同面?(wt.in 文件中要规定切面, 且我遇见过 6 个切面都存在陈数的情况)

A2: 这其实触及到了拓扑能带理论的核心逻辑: 体态拓扑决定边界态。表面态 (或边界态) 来源于体态拓扑性质: 拓扑材料的一个核心定理是: Bulk-Boundary Correspondence (体-边对应原理)。一个体态系统的拓扑不变量 (如 Chern 数、Z₂ 指标) 决定了其边界是否存在稳定的表面态。换句话说: 表面态不是独立存在的; 它是体能带 (bulk band) 拓扑结构的“投影”或“表现形式”。所以计算二维表面态的第一步是: 确定 bulk 能带及其拓扑不变量。

1 从 DFT/Wannier90 得到 bulk 哈密顿矩阵 (wannier90_hr.dat)

2 WannierTools 读取 bulk 哈密顿量

3 根据用户定义的表面切面 (surface plane), 对晶格进行投影

4 构造 slab(超胞体) 或半无限格林函数

5 求解表面态或边缘态谱

虽然二维体系在物理空间中厚度极薄, 但它依然是一个晶格周期性的二维平面。在 WannierTools 的框架中, 二维系统仍被视为一个二维晶体中的周期性布洛赫体, 因此在数值模拟时可以“虚拟地”切割出不同的边界方向。

例如:

二维蜂窝晶格 (如石墨烯) 有不同的“边界面”:

- **Armchair 边界**
- **Zigzag 边界**

这些不同方向上的切割方式会导致不同的:

- 能带投影 (surface Brillouin zone 不同);
- 表面 (边缘) 态分布不同;
- 能否出现拓扑边缘态 (取决于投影到的动量路径上是否存在能隙)。



三、为什么你会遇到“6个切面都有Chern数”的情况?

非常棒的观察 🍌。这说明你的体系可能具有以下特性之一:



情况1: 体系具有各向同性的拓扑性质

如果体系的拓扑不变量 (如 Chern 数) 在不同的平面投影方向上都非零, 说明体系具有:

- 各向同性拓扑性质;
- 或体系的所有边界都承载拓扑边缘态。

这在一些具有高对称性的二维拓扑体系中确实可能出现 (例如, 某些蜂窝格点的高Chern数模型)。

情况2: WannierTools的“6个切面”是三维投影计算

即使你的体系是二维的, WannierTools 仍将 `wannier90_hr.dat` 视作一个三维哈密顿量 (哪怕某个方向只有一层原子)。

因此, 它在技术上仍可以定义6个面:

面编号	含义	是否对二维体系有意义
(100), (-100)	x方向正负表面	✓
(010), (-010)	y方向正负表面	✓
(001), (-001)	z方向正负表面	⚠ 通常仅有一层, 但数值上仍可定义

这6个面在二维体系中物理意义不同, 但计算形式上仍成立。

当WannierTools通过格林函数或Wilson loop求Chern数时, 这些方向都会被单独计算一个拓扑不变量。

若体系拓扑强且各方向等价, 就可能在所有切面上都得到非零Chern数。

五、总结: 你的问题核心逻辑如下

问题	答案	
为什么二维体系计算表面态要算bulk能带?	因为表面 (边界) 态由体拓扑性质决定, 必须先从 bulk 哈密顿量导出。	
二维体系是否有不同面?	有。不同切面 (方向) 对应不同边界方向, 如 zigzag / armchair。	
为什么6个切面都有Chern数?	因为WannierTools仍按3D模型处理, 即在各个方向投影Berry曲率, 数值上都可得到非零拓扑指标; 或体系拓扑各向同性。	
是否正常?	是的, 在具有强拓扑性质或残余3D耦合的体系中, 这是合理结果。	

Q3: 实际上 vasp 对接的是二维体系的能带计算, 然后 wannier 构造 wannier 函数, 但计算的又是三位体系, 如何保证计算的物理合理性和数学合理性?

A3:

非常优秀的问题 —— 这正是理解 **WannierTools + VASP** 在二维体系中计算拓扑性质时的“物理-数学一致性”核心所在。

你的问题可以归纳为一句话:

当我用 VASP 计算的是一个二维体系, 但 Wannier90/WannierTools 处理的是三维哈密顿量时, 这样的“3D形式的2D体系”在物理与数学上是否合理? 为什么?

答案是: ✓ 完全合理 —— 只要你在建模与参数上处理正确, WannierTools 中的“三维形式”只是数学嵌入 (mathematical embedding), 而非物理上的三维扩展。

下面我将分 5 个层次为你系统解释:

一、二维体系在 VASP 中的“数学表现形式”

在 VASP 中，即使你研究的是严格二维体系（如单层 WTe_2 、石墨烯、 MoS_2 等），仍然需要：

- 在 z 方向加上真空层（通常 $>15 \text{ \AA}$ ）；
- 因为 VASP 使用的是三维周期性边界条件（3D PBC）。

换句话说：

二维体系在 VASP 中始终是一个“被嵌入到三维周期性盒子中的二维平面”。

例如：

text

复制代码

```
supercell: a x b x c
a, b — 实际二维晶格矢量
c — 加入真空方向
```

此时体系在 z 方向完全绝缘、没有有效哈密顿量耦合项。

因此：

- 布洛赫波函数仍可写为 3D Bloch form；
- 但 z 方向的色散是“平坦”的；
- 实际的物理过程只发生在 xy 平面。

二、Wannier90 构造的哈密顿量是“嵌入式 3D 哈密顿量”

当你用 `VASP2WANNIER90` 或 `wannier90.x` 构造 Wannier 函数时：

- 它会从 DFT 投影出所有三维格点上的矩阵元；
- 形成一个文件 `wannier90_hr.dat`，其中的哈密顿矩阵是形式上的 3D lattice 哈密顿量。

但如果体系是二维的：

该哈密顿矩阵在 z 方向的 hopping（跃迁项）几乎为 0。

这意味着：

- 你拥有的是一个“退化”成二维的三维 tight-binding 模型；
- 数学上是 3D，物理上是 2D；
- 这并不矛盾，而是一种低维嵌入高维空间的标准做法。

三、WannierTools 中的“3D切面”如何理解

WannierTools 的所有拓扑计算都基于输入的哈密顿矩阵 $H(R_x, R_y, R_z)$ 。它并不知道你体系是“二维”还是“三维”，它只是根据 hopping 范围来判断维度。

如果你的体系：

- z 方向 hopping ≈ 0 ;
- 只在 xy 平面上存在有效跃迁;

那么 WannierTools 自然会识别出这是一个二维模型。

举例

在 WannierTools 的 `wt.in` 中，你可以看到：

```
bash
```

复制代码

```
Bulk_or_Slab_Calc = T
Slab_dir = 3
```

但如果体系 z 方向没有耦合项，即便设置 `Slab_dir = 3`，计算的“表面态”其实就是二维边界态的表现。WannierTools 仍以 3D 格式输出，但 z 方向的贡献为零。

四、数学合理性：低维系统的高维嵌入原理

这个问题在数学上有严格的基础，叫做：

Dimensional Embedding of Topological Spaces (拓扑空间的维度嵌入)

它的核心结论是：

任意一个二维布洛赫体 (T^2 , 即二维布里渊区) 都可以嵌入到三维晶格 T^3 中作为一个切片。

在我们的计算中：

- DFT 的 3D 周期条件 $\rightarrow$ 对应三维环面 T^3 ;
- 真空层消除了沿 z 的能带耦合，使得拓扑流形退化为 T^2 。

因此：

- 物理上，体系仍是二维;
- 数学上，计算在三维格点上进行;
- 拓扑不变量 (如 Chern 数、 Z_2) 通过在 T^2 上积分求得，完全等价。

实际上，这种嵌入在拓扑物理中是常规做法：

三维体拓扑 $\rightarrow$ 二维边界拓扑;

二维体拓扑 $\rightarrow$ 一维边界拓扑。

Q4: 我看到 in 文件中的 CONTROL 中可以选择是否计算 `bulkband_calc`, 结合前述结果, 似乎一定要计算 `bulkband`?

A5:

 一、WannierTools中 BulkBand_calc 的真实含义

参数	实际作用	是否影响表面态计算
BulkBand_calc = T	计算并输出 bulk能带图 (bandstructure)	✗ 不影响表面态, 只多输出一张图
BulkBand_calc = F	不绘制bulk能带图, 只做其他任务 (如表面态、Z ₂ 等)	✗ 不影响物理计算, 只省略输出

◆ 换句话说:

“BulkBand_calc” 是一个 图像输出控制项, 而不是一个 拓扑计算开关项。

 三、什么时候建议打开 BulkBand_calc = T

虽然从计算上不是必须, 但强烈建议在第一次计算某个体系时打开它 (设为 T)。

理由有三:

1. 验证Wannier插值的准确性
 - 你可以检查 Wannier 插值能带是否与原DFT能带吻合;
 - 若能带不匹配, 表面态计算结果将失真。
2. 确认能带倒转/拓扑间隙位置
 - bulk能带的拓扑倒转位置决定了表面态出现的能量区;
 - 可通过对比确定 E_arc、OmegaMin/Max 的合理范围。
3. 可视化辅助分析
 - bulk能带图 + 表面态谱图配合使用, 能更清晰展示拓扑保护态的出现位置。

🔔

五、进一步说明：BulkBand计算的重要性（物理视角）

即使二维体系本身不需要体态能带图，
计算并理解bulk能带仍然是理解拓扑性质的核心。

任务	bulk band 的作用
拓扑不变量 (Chern, Z_2)	必须通过bulk能带积分Berry曲率求得
表面态识别	表面态位于bulk能隙内
判断能带倒转	bulk能带提供直接证据
费米能级设定	需在bulk能带中选取参考能量

因此——

表面态计算离不开bulk物理，即使BulkBand_calc关闭，bulk依然被计算使用。

六、总结要点

问题	答案
“不开BulkBand_calc是不是没算bulk能带？”	✗ 不是。bulk哈密顿量始终参与所有计算。
“那为什么要这个选项？”	只是控制是否绘制bulk能带图输出。
“二维体系需要算bulkband吗？”	物理上要理解bulk拓扑结构，但程序层面可选开关。
“什么时候开？”	初次建模或验证Wannier准确性时建议开启。

4.7 pwy90 推荐能量窗口测试

日期：2025-11-27

在线原文：marx2001.github.io

► 心得笔记

拟合完的指令:wannier90.x wannier90 & 激活 TB2J:source /public/home/cssong/song/zhchen/1_materials/1_VTeCl/TB2J/strain/-2/TB2J_env/bin/activate

进入 pyw90 环境:conda activate pyw90_env pyw90 eig suggest -path . -e -20 20 -w 20 在选定能量区间内计算推荐的内外能量窗口的范围

求平均值:TB2J_merge.py -type structure x y z

求磁耦合参数:wann2J.py -spinor -groupby spin -efermi -1.0098 -emin -2.06514 -emax -0.065 -elements Nb -kmesh 9 9 1 -nz 100 -rcut 6.0 -np 1 第一次测试

► 测试 1

```
conda activate pwy90_env
```

```
pwy90 eig suggest -path . -e -20 20 -w 38
```

```
There are at most 46 states and at least 46 states in [-20.0, 20.0].

'dis_win_min' and 'dis_win_max' Table:
Column 'dis_win_max' shows the "lowest" dis_win_max for 'dis_win_min'
Column 'i+1_min' / 'i_max' shows the band minimum / maximum near the gap
Column 'Nleast' / 'Nmost' shows the least / most number of states inside 'dis_win_min' and Fermi level.

There is no enough states for -6.344039 with 38 WFs!
There is no enough states for -2.637562 with 38 WFs!
There is no enough states for -0.9930779999999999 with 38 WFs!
'dis_win_min' 'dis_win_max' 'i+1_min' 'i_max' 'Nleast' 'Nmost'
13 -0.993078 6.000000 -0.982188 -1.003968 0 0
12 -2.637562 6.000000 -2.630185 -2.644939 4 4
11 -6.344039 6.000000 -6.249603 -6.438475 16 16
10 -11.470015 3.788728 -9.344601 -13.595429 22 22
9 -13.824348 3.305089 -13.714366 -13.934329 24 24
8 -17.559844 2.989421 -14.486173 -20.631516 26 26
/public/home/xpau/.conda/envs/pwy90_env/lib/python3.9/site-packages/pwy90/pwy90_cli.py:321: FutureWarning: The behavior of DataFrame concatenation with empty or all-NA entries is deprecated. In a future version, this will no longer exclude empty or all-NA columns when determining the result dtypes. To retain the old behavior, exclude the relevant entries before the concat operation.
df_froz = pd.concat(df_froz_list, ignore_index=True).drop_duplicates()
'dis_froz_min' and 'dis_froz_max' Table:
Suggest 'dis_froz_min' & 'dis_froz_max' as following with #Wfs = 38 from input
Use 'pre dos' menu to get suggestions based on projected density of states
'dis_froz_min' 'dis_froz_max' 'N'
12 -20.627516 2.154407 38
13 -13.931077 2.204247 37
6 -13.930329 2.600411 38
5 -13.930329 2.600284 38
9 -13.591429 2.978903 38
1 -6.867550 3.014653 37
2 -6.867352 3.427087 38
3 -6.819218 3.442032 37
4 -6.819035 6.000000 38
```

有个问题, 为什么大窗口不满足 38 个 states, 而小窗口反而有 38 个 states?

你提的这个问题非常好, 而且非常具有代表性:

为什么更大的能量窗口 (`dis_win_min ~ dis_win_max`) 反而得不到足够的38个态 (states), 而较小的冻结窗口 (`dis_froz_min ~ dis_froz_max`) 却可以满足?

这实际上是一个**“质量 vs 数量”**的问题, 主要涉及以下几点物理和算法的理解:

✅ 问题分析

🧠 1. `dis_win_*` 是“判定窗口”, 目标是找出能落入该窗口的“合适”的 Wannier Functions (WFs)

- 定义: `dis_win_min` 和 `dis_win_max` 定义了一个判定能区窗口。
- 目标: 在这个窗口内, 选出一组 WFs, 使它们能有效拟合目标能带, 并覆盖所有需要拟合的能态。

❄️ 2. `dis_froz_*` 是“冻结窗口”, 里面的态直接参与构建 WFs

- 更容易控制数量: 冻结窗口内的态是“强制使用”的, 不管质量如何。
- 所以只要你设置得当, 比如包含较低能的价带态, 通常是比较局域化、容易拟合的态, 就能稳定地找到 38 个。

🖼️ 3. `dis_win_*` 质量控制更严格 —— 如果拟合不好, 会被剔除

- 判定过程更复杂, 尤其是对“过宽”的 `dis_win` 区域, 会引入一些难以拟合的态 (比如延展态、高导带态等)。
- 这些态虽然在能量窗口内, 但因为质量不够好, 会被跳过, 导致最终状态数 < 38。

✓ 建议

1. 优先调整 `dis_froz_*` 以覆盖目标态;
2. 确保 `dis_win_*` 不要太宽, 特别是不要引入高能导带;
3. 可以在 `wannier90.win` 中尝试添加:

plaintext

复制代码

exclude_bands : ...

来手动排除“坏态”;

4. 检查投影原子轨道和能带是否匹配 (比如 s, p, d 的数量和分布);
5. 利用 `pre dos` 菜单观察 PDOS 分布, 有助于精细选择窗口。

你的意思是说, 大窗口不要太宽, 让大窗口的能量尽可能的低, 但是不能小于小窗口?

非常好的理解——你已经抓住了关键逻辑, 只差一点点细化。

我来帮你彻底理清 `dis_win` 与 `dis_froz` 的能量关系和选取原则 📌

🗨️ 一句话总结

✓ 大窗口 (`dis_win`) 应该略宽于小窗口 (`dis_froz`), 但不能太宽。

💡 它的能量下限可以比小窗口低一点, 上限也可以稍微高一点, 但过宽反而会“坏事”。

还是有一个问题, 这个 pw90 判定不足 38 个态, 但是实际上 wannier90 拟合能带的时候并没有报错, 反而拟合的很好, 与原来能带完全重合

原理详解

Wannier90 在选择能量窗口时有两个层次的过滤机制：

窗口类型	名称	功能	能量区间的作用
<code>dis_win</code>	判定窗口 (disentanglement window)	决定“哪些能态可以被考虑进入 WF _s 拟合”	越宽 → 包含更多能态（可能掺入不相关导带）
<code>dis_froz</code>	冻结窗口 (frozen window)	决定“哪些能态必须被保留为目标 WF”	这些态会被直接拟合，不参与优化剔除

为什么太宽反而不行？

当你把 `dis_win` 拉得太宽，比如上限加到很高的导带能量：

- 系统会把很多“延展性强、非局域化”的导带态也纳入；
- 这些态与价带态在能量和空间上差别大，Wannier 拟合会出现“正交化不良”；
- 结果：算法自动丢弃一些态（因此 `states < 38`）；
- 这时你就看到提示：“There is no enough states for ... with 38 WF_s!”

你发现了一个**非常典型的“pyw90 与 Wannier90 拟合行为不一致”的情况——

pyw90 提示 “There is no enough states for 38 WF_s”，
但实际运行 wannier90 拟合能带时却完全没有报错，结果很好。

这其实是合理的，原因在于：

pyw90 与 wannier90 的“判断标准”并不完全相同。下面我来系统地解释原因 📌

一、pyw90 的作用与局限性

pyw90 是什么？

pyw90 是一个“辅助分析与自动窗口优化工具”，它的功能是：

- 根据 `EIGENVAL` / `PROCAR` 等数据，
- 自动寻找合适的 `dis_win_min/max`、`dis_froz_min/max`，
- 并判断给定窗口是否**包含足够的态 (states) **来生成指定数量的 WF_s。

⚠️ pyw90 的判断标准是 严格的几何判定

它根据能量分布 (DOS) 数值统计“在此窗口内至少有多少个态”，
但并不真正执行 Wannier 函数投影和 disentanglement 优化。

所以它的“判定不足”只是：

“从纯能量统计角度看，这个窗口中能量落入范围的 band 数不够 38。”

并不意味着 Wannier90 不能生成或优化出 38 个 WF_s。

二、Wannier90 的机制更复杂、更智能

Wannier90 的 disentanglement (解缠绕) 算法可以:

- 在一定范围内“混合”带间信息;
- 通过优化子空间投影得到目标数量的 Wannier 函数;
- 即使部分能带能量略超出窗口, 也能通过光滑化处理来获得良好结果。

换句话说:

Wannier90 并不需要窗口内有严格 38 条能带,
只要在相关能区内存在足够的投影权重, 它就能自动优化出 38 个 Wannier 函数。

三、为什么你的 Wannier90 能拟合很好?

1 投影选择得当

如果你在 `.win` 文件中设置了合理的:

```
text
```

[复制代码](#)

```
projections = 'Fe:d', 'O:p', ...
```

Wannier90 会根据这些轨道自动找出相关带, 即使 pw90 判断能带数量不足, 投影仍然可以收敛。

2 窗口选择合理

即使 pyw90 提示“不足 38 个态”，但：

- 实际冻结窗口 (dis_froz) 覆盖了主要价带；
 - 判定窗口 (dis_win) 仍然包含了导带初始部分；
- 所以 Wannier 拟合依然有充足的能量区间可操作。

3 能带的能量重叠

在实际材料中（尤其是金属、窄带隙体系），带与带之间常常有重叠；
pyw90 统计时可能视作“能带数不够”，但 Wannier90 实际可以通过混合权重获得足够的有效态。

✂ 四、总结对比

项目	pyw90	wannier90
判断依据	纯能量范围内的态数统计	轨道投影 + 能带平滑优化
是否执行 disentanglement	✗ 否	✓ 是
对能量区间要求	严格	灵活
对态数要求	必须够	可以不足但可优化
输出	建议窗口	实际结果 (WFs与能带)

✓ 五、结论与建议

1. ✓ pyw90 是“预判工具”，不是判决工具。
它帮你找合理窗口，但不能代表 Wannier90 最终结果。
2. ✓ 如果 Wannier90 拟合结果好、能带对齐完美，
→ 说明实际的 dis_win / dis_froz 已经是有效窗口，
→ 不需要纠结 pyw90 的 warning。
3. ⚙ 如果你想让 pyw90 也“通过”，可以略微扩大 dis_win_max ~ +0.5 eV，
但没必要牺牲当前良好的拟合。
4. 🧠 实际项目建议：
 - 用 pyw90 作为起点；
 - 再手动微调窗口看拟合结果；
 - 最终以 wannier90 拟合能带是否重合 为准。

✂ 一句话总结：

“pyw90 的‘不够态’只是统计警告，不是物理错误。
如果 Wannier90 实际拟合良好，你的窗口选择就是正确的。”

与我的已经成功拟合且百分百重合的能带相比, 能量窗口的范围, 应该是相对靠谱的, 可能需要后期手动再调整, 可以说能够当作一个预判工具:

```
dis_win_min = -9.6114
dis_win_max = 6.056738
dis_froz_min = -6.39197
dis_froz_max = 1.2082
dis_num_iter = 0
num_iter = 0
```

```
Calculated Energy Range: -10.0, 7.0 with Fermi level -0.986463
There are at most 42 states and at least 42 states in [-10.0, 7.0].

'dis_win_min' and 'dis_win_max' Table:
Column 'dis_win_min' shows the **lowest** dis_win_max for 'dis_win_min'
Column 'i1_min' / 'i1_max' shows the band minimum / maximum near the gap
Column 'Nleast' / 'Nmost' shows the least / most number of states inside 'dis_win_min' and Fermi level.

There is no enough states for -6.344039 with 38 WFs!
There is no enough states for -2.637562 with 38 WFs!
There is no enough states for -0.9998779999999999 with 38 WFs!

dis_win_min dis_win_max i1_min i1_max Nleast Nmost
13 -0.999878 6.000000 -0.982188 -1.003968 0 0
12 -2.637562 6.000000 -2.630185 -2.644939 4 4
11 -6.344039 6.000000 -6.249603 -6.438475 16 16
10 -11.470915 3.788728 -9.344601 -13.595429 22 22

/public/home/xxwu/.conda/envs/pyw90_env/lib/python3.9/site-packages/pyw90/pyw90_cli.py:321: FutureWarning: The behavior of DataFrame concatenation with empty or all-NA entries is deprecated. In a future version, this will no longer exclude empty or all-NA columns when determining the result dtypes. To retain the old behavior, exclude the relevant entries before the concat operation.
df_froz = pd.concat(df_froz_list, ignore_index=True).drop_duplicates()

'dis_froz_min' and 'dis_froz_max' Table:
Suggest 'dis_froz_min' & 'dis_froz_max' as following with #WFs = 38 from input
Use 'pre dos' menu to get suggestions based on projected density of states

dis_froz_min dis_froz_max N
0 -13.591429 2.978903 38
1 -6.867550 3.014653 37
2 -6.867352 3.427087 38
3 -6.819218 3.442032 37
4 -6.819035 6.000000 38
```

4.8 pyw90 的 auto 功能介绍

日期: 2025-11-27

在线原文: marx2001.github.io

<https://github.com/Cloudiiink/pyw90/blob/master/README.md>

简介

 **pyw90 auto 菜单功能总结**

 **一、总体功能概述**

pyw90 auto 是 **Wannier90** 能量窗口全自动优化模块。

它通过不断调用 **Wannier90** 计算、比较结果、自动调整窗口, 实现能量窗口的自适应搜索与拟合最优化。



一句话总结:

pyw90 auto 可以自动找到最佳的 `dis_win_min/max` 与 `dis_froz_min/max` 参数, 使 **Wannier90** 能带与 **DFT** 能带差异最小。

二、命令结构与参数

bash

复制代码

pyw90 auto [-h] [--path PATH] [--pid PID] mode

位置参数 (positional argument)

参数	含义
mode	模式选择: input、run、term (或只输入首字母)

可选参数

参数	功能	默认值
--path PATH	工作路径	当前目录 .
--pid PID	指定要终止的进程号	-

三、三种运行模式说明

模式	功能说明	输出结果
input	生成自动化配置文件 auto_w90_input.yaml	用于后续自动拟合
run	执行自动优化循环，反复运行 Wannier90 拟合	输出优化日志与结果
term	手动终止后台优化任务	停止 pyw90 auto run 守护进程

🧠 四、自动优化原理

当执行：

```
bash
```

📄 复制代码

```
pyw90 auto run
```

程序会：

1. 启动守护进程 `python auto_w90_fit.py`；
2. 调用 Wannier90 (本地或 SLURM 提交)；
3. 计算 Wannier90 插值带与 DFT 能带的差异；
4. 自动修改能量窗口参数；
5. 重复迭代，直到误差最小或达到迭代上限。

🔺 优化目标函数

优化目标为 Wannier 拟合误差 Δ ：

$$\Delta = \frac{1}{CN_k} \sum_{i,k} |\varepsilon_{i,k}^{\text{DFT}} - \varepsilon_{i,k}^{\text{TB}}| f(\varepsilon_{i,k}^{\text{DFT}})$$

其中：

- i, k ：带索引与 k 点索引；
- $f(\cdot)$ ：核函数 (unit 或 gaussian)；
- C ：归一化常数。

⚙️ 归一化常数定义

$$C = \frac{1}{N_k} \sum_{i,k} f(\varepsilon_{i,k}^{\text{DFT}})$$

■ 该函数用于衡量 Wannier 拟合能带与原始 DFT 能带的偏差程度，目标是让 Δ 尽可能小，即拟合最优。

 **五、自动优化流程（图示说明）**

图片展示的流程可总结为以下步骤：

步骤	动作	说明
①	启动守护进程	<code>pyw90 auto run</code> 启动后台优化任务
②	调用 Wannier90	在 SLURM 或本地运行 Wannier90 计算
③	误差分析与调整	根据拟合误差自动调整 <code>dis_win</code> 与 <code>dis_froz</code>
④	记录日志	每次迭代将输出到 <code>log_autow90_{timestamp}.log</code>
⑤	终止任务	使用 <code>pyw90 auto term</code> 停止优化循环

 该过程会持续迭代，直到满足容差 (`tol`) 或达到最大迭代数 (`maxiter`)。

 **六、配置文件 (auto_w90_input.yaml)**

当执行 `pyw90 auto input` 时，会生成自动化配置文件：

`auto_w90_input.yaml`

复制代码

它定义了 Wannier90 自动优化的全部参数。
配置文件为 **YAML 格式**，用缩进（空格）表示层次结构。

参数详细说明

键名	功能	默认值 / 说明
seedname	Wannier90 工程名 (对应 seedname.win)	wannier90
runfile	SLURM 作业提交脚本	自动识别
vasp_band_file	VASP 能带文件 (p4vasp 格式)	bnd.dat
jobname	作业名称	用户自定义
username	用户名 (用于识别任务)	自动检测
local	是否在本地运行 (True/False)	False
localrun	本地执行 Wannier90 的命令	wannier90.x
efermi	费米能 (不归零)	从 vasprun.xml 读取
kernel	核函数类型与参数	unit / gaussian
method	最小化算法 (SciPy 优化器)	COBYLA (默认)
tol	终止容差	0.02
maxiter	最大迭代次数	100

maxiter	最大迭代次数	100
ini_dis	初始窗口参数	手动或预定义
opt_dis	哪些参数参与优化 (布尔)	True
bounds	参数边界范围	可用 $\pm\infty$
num_print_check	打印状态频率	每 10 次迭代
check_time	状态检查时间间隔 (秒)	30
display	控制是否输出拟合差值与展宽	True

🧠 七、关键机制说明

⚙️ 1 优化算法

`pyw90` 内置基于 **SciPy** 的最优化算法：

- Nelder-Mead
- Powell
- COBYLA (默认)

COBYLA 适用于非线性、无导数问题，是 Wannier 拟合最稳定的选择。

📊 2 优化目标

目标是最小化 Wannier90 拟合与 DFT 能带差值 Δ ：

- 自动寻找最合适的 `dis_win_min/max`；
- 避免人工试错；
- 输出最终最优窗口参数。

🔧 3 任务控制

- 后台运行由守护进程 `auto_w90_fit.py` 管理；
- 可通过 `pyw90 auto term` 终止；
- 输出日志包含能量窗口变化与误差收敛情况。

📁 八、输出文件

文件名	内容
<code>auto_w90_input.yaml</code>	自动化优化配置文件
<code>log_autow90_*.log</code>	每次迭代的窗口与误差记录
<code>wannier90_band.dat</code>	最新 Wannier 拟合能带
<code>wannier90.wout</code>	Wannier 运行输出文件
<code>seedname_band.dat</code>	更新后的能带数据

九、应用与优势	
优势	说明
全自动	无需人工试错窗口参数
可并行	支持 SLURM 集群或本地运行
高精度	自动最小化 DFT 与 Wannier90 拟合误差
可复现	所有参数记录在 YAML 配置中
自适应	根据误差自动调整优化方向

十、总结表		
模块	功能描述	输出结果
input	生成优化配置文件	auto_w90_input.yaml
run	自动优化 Wannier 窗口	最优 dis_win 与 dis_froz
term	终止自动任务	停止守护进程
优化算法	SciPy 最小化 (COBYLA 等)	拟合误差 Δ
输出日志	每轮迭代窗口与误差	log_autow90_*.log

十一、总结一句话：

pyw90 auto 是一个全自动 Wannier90 优化引擎，通过不断计算、比较和调整能量窗口，使 Wannier90 拟合能带与 DFT 能带误差最小化。它是从“半自动”过渡到“完全自动化 Wannier 化”的关键模块。

Q1:wannier90 的能带拟合步骤是通过设置 VASP 中的 INCAR 中的 LWANNIER90=.T 的 tag 实现的, 在此过程中,vasp 先计算能带, 然后通过上面那个接口输出为 wannier 格式, 那么这个 pyw90 是怎么确定拟合出来的结果和窗口能量区间是正确的呢?

A1:

🚫 一、基本流程回顾：从 VASP → Wannier90 → pyw90 校验

1. 在 VASP 中：

- 设置 `LWANNIER90 = .TRUE.`；
- 同时打开 `LWRITE_UNK = .TRUE.`；
- VASP 计算完 Kohn-Sham 本征值与波函数后，会输出 Wannier90 所需文件：

```
seedname.mmn, seedname.amn, seedname.eig, seedname.win
```

[复制代码](#)

- 这一步只是生成输入，并不自动验证拟合质量。

2. 在 Wannier90 中：

- 根据 `dis_win_min/max` 与 `dis_froz_min/max` 进行“解缠绕”+“投影优化”；
- 输出 `wannier90_band.dat` (Wannier插值能带)；
- 输出 `wannier90.wout` (拟合过程信息，包括 spreads)。

3. 在 pyw90 中：

- 读取 VASP 原始能带 (`bnd.dat`)；
- 读取 Wannier 拟合能带 (`wannier90_band.dat`)；
- 自动对比两者的差异；
- 计算出拟合误差指标；
- 根据误差最小化自动调整窗口参数。

二、pyw90 判定“拟合正确”的数学标准

pyw90 使用一个明确的拟合质量指标 Δ (Delta) , 定义为:

$$\Delta = \frac{1}{CN_k} \sum_{i,k} \left| \varepsilon_{i,k}^{\text{DFT}} - \varepsilon_{i,k}^{\text{Wannier}} \right| f \left(\varepsilon_{i,k}^{\text{DFT}} \right)$$

含义解析:

符号	含义
$\varepsilon_{i,k}^{\text{DFT}}$	来自 VASP 的能量本征值
$\varepsilon_{i,k}^{\text{Wannier}}$	来自 Wannier90 的插值能带
$f(\varepsilon)$	核函数 (unit 或 Gaussian) 控制能量权重
C	归一化常数
N_k	k 点总数

直观理解:

- Δ 衡量了 DFT 与 Wannier 插值能带之间的平均能量偏差;
- Δ 越小 $\rightarrow$ 拟合越好;
- 若 Δ 低于设定容差 (默认 0.02 eV) , 则认为拟合收敛且正确。

三、pyw90 如何“判定窗口正确性”

pyw90 不仅比较能带差异，还结合窗口能量范围的物理判据：

✓ 1 冻结窗口 (`dis_froz`) 检查逻辑

- 确保在冻结窗口内的态数 $\geq$ Wannier 函数数量 (NWann) ；
- 检查冻结窗口上限不超过导带底部；
- 检查冻结窗口内能带覆盖所有目标价带；
- 若不满足这些条件，pyw90 会自动缩放或移动窗口。

✓ 2 判定窗口 (`dis_win`) 检查逻辑

- 外层窗口要足够宽，包含所有投影波函数分布；
- 但不能过宽，否则引入杂态；
- pyw90 通过误差曲线 $\Delta(E)$ vs E 自动扫描窗口上限与下限；
 - 若窗口上限过高 $\rightarrow \Delta$ 增大 (说明引入导带杂态) ；
 - 若窗口太窄 $\rightarrow \Delta$ 增大 (说明漏掉部分目标带) ；
- 迭代直到 Δ 最小化。

✓ 3 自动优化算法

pyw90 在 `auto` 模式下使用 最优化算法 (`scipy.optimize.minimize`) ：

- 算法：COBYLA / Nelder-Mead / Powell；
- 变量： `dis_win_min`, `dis_win_max`, `dis_froz_min`, `dis_froz_max` ；
- 目标函数： Δ (Wannier-DFT 带差异) ；
- 终止条件：
 - Δ 收敛至容差 (默认 0.02 eV) ；
 - 达到最大迭代次数 (默认 100) 。



 四、pyw90 判定正确性的三层标准 (综合逻辑)

层级	判定内容	判定标准	来源	
数值层	能带误差 Δ	$\Delta < \text{容差}$ (默认 0.02 eV)	拟合差值公式	
物理层	冻结窗口态数	$\geq$ Wannier 函数数量	DOS + band counting	
投影层	投影轨道分布	pDOS/tDOS 比例匹配	<code>pyw90 pre dos</code> 分析	

 五、结果正确的典型特征

当 pyw90 判定拟合正确后，你会在终端或日志中看到：

arduino

 复制代码

```
 $\Delta$  (band diff) = 0.015 eV  Converged
Final dis_win_min/max = -9.2 / 4.8 eV
Final dis_froz_min/max = -8.7 / 2.9 eV
```

同时：

- `wannier90_band.dat` 与 `bnd.dat` 能带几乎重合；
- 日志中 `spread` 值收敛；
- 冻结窗口中恰好包含所有价带能区。

六、pyw90 的辅助验证方式

pyw90 提供了两个重要的验证工具：

- 1. `pyw90 cmp` 模块
对比 Wannier 与 DFT 能带并输出图表；
 - 输出文件： `wannier_cmp.pdf`
 - 可直接可视化 $\Delta(E)$ 。
- 2. `pre dos` 模块
对比投影态密度 (pDOS) 与总态密度 (tDOS) ；
 - 检查冻结窗口轨道贡献是否匹配；
 - 确认选取的投影轨道是否合理。

七、最终判断依据： Δ + 物理一致性

项目	判定方法	理想状态
Δ (带差)	计算能带误差	< 0.02 eV
WF数匹配	冻结窗口态数 vs #WFs	相等
窗口合理性	pDOS/tDOS 对应轨道	主导轨道覆盖完整价带
图形验证	<code>pyw90 cmp</code> 输出图	Wannier 能带与 DFT 重合
空间延展性	<code>wannier90.wout</code> 中 spread	稳定收敛，不发散

八、总结：pyw90 判定正确拟合的三大依据

类别	判定逻辑	工具模块
数值误差最小化	DFT 与 Wannier 带差 $\Delta \rightarrow$ 最小	<code>auto</code>
物理窗口合理性	DOS 范围 & #states 匹配	<code>pre dos</code>
可视化验证	能带重合 + spread 收敛	<code>cmp</code>

最终，当三项条件同时满足时，pyw90 判定：

✅ “该 Wannier90 拟合结果与能量窗口是正确且物理合理的。”

4.9 pyw90 的 cmp 功能介绍

日期：2025-11-27

在线原文：marx2001.github.io

<https://github.com/Cloudiiink/pyw90/blob/master/README.md>

简介

 **pyw90 cmp 菜单功能总结**

 **一、总体功能介绍**

`pyw90 cmp` (compare) 模块用于：



对比 Wannier90 和 VASP 的能带结果，评估拟合精度与能量窗口的合理性。

它会：

- 自动读取 `wannier90_band.dat` (Wannier 拟合能带)
- 与 `bnd.dat` (VASP 原始能带) 对比
- 计算每个能带的最大/平均偏差
- 绘制可视化图表
- 输出 Wannier 展宽 (spread) 收敛数据

最终判断 Wannier 拟合是否成功、能量窗口是否选取正确。

 **二、命令结构与主要参数**

bash

 复制代码

`pyw90 cmp [-h] [--config] [--path PATH] [--efermi EFERMI]
 [--vasp VASP] [--seedname SEEDNAME] [--ylim YLIM YLIM]
 [--kernel KERNEL] [--show-fonts] [--fontfamily FONTFAMILY]
 [--fontsize FONTSIZE] [--no-spread] [--no-quality] [--quiet]
 name`

 **必需参数：**

参数	含义
<code>name</code>	系统名称（输出图文件名前缀）

◆ 可选参数汇总		
参数	功能	默认值
<code>--config</code>	从 <code>auto_w90_input.yaml</code> 中读取参数	False
<code>--path</code>	工作路径	当前目录 <code>.</code>
<code>--efermi</code>	费米能	从 <code>vasprun.xml</code> 自动读取
<code>--vasp</code>	VASP 能带文件路径	<code>bnd.dat</code>
<code>--seedname</code>	Wannier 工程名	<code>wannier90</code>
<code>--ylim</code>	能量范围（用于绘图）	<code>[E_win_min - 1, E_win_max + 1]</code>
<code>--kernel</code>	误差核函数（ <code>unit</code> 或 <code>gaussian</code> ）	<code>unit,0,1</code>
<code>--show-fonts</code>	列出可用字体	-
<code>--fontfamily</code>	绘图字体	Open Sans
<code>--fontsize</code>	字体大小	18
<code>--no-spread</code>	不显示 Wannier 展宽	False
<code>--no-quality</code>	不显示拟合质量	False
<code>--quiet</code>	同时关闭 <code>spread</code> 与质量输出	False

✂ 三、功能与流程说明		
<code>cmp</code> 模块的核心功能包括三个部分：		
功能	描述	输出
能带比较	对比 VASP 与 Wannier90 能带	<code>name_VASP_w90_cmp.png</code>
拟合质量分析	计算每带的最大和平均误差	终端表格输出
展宽统计	从 <code>wannier90.wout</code> 中提取 <code>spread</code> 并绘制分布	控制台图表输出

四、核心比较逻辑

1. 读取输入文件:

- 从 `bnd.dat` (VASP) 与 `wannier90_band.dat` (Wannier90) 提取能量;
- 自动校准能量零点为费米能 ($E_F=0$) ;
- 若 `--efermi` 未设定, 自动从 `vasprun.xml` 读取。

2. 计算能带差值 Δ :

$$\Delta_{i,k} = |\varepsilon_{i,k}^{DFT} - \varepsilon_{i,k}^{Wannier}|$$

- 并取每个能带的:
 - 最大偏差 (MAX DIFF)
 - 平均偏差 (AVERAGE DIFF)

3. 核函数加权 (kernel)

- 允许两种核函数模式:
 - `unit`: 等权比较;
 - `gaussian`: 能量远离费米能的部分权重更低;
- 这与 `pyw90 auto` 模块的误差定义保持一致。

4. 误差判定标准:

- 若平均误差 < 2 meV (0.002 eV) , 视为优秀拟合;
- 若最大误差 < 5 meV, 且能带形状一致 $\rightarrow$ 拟合合格;
- 若部分能带出现大偏差 $\rightarrow$ 窗口区间或投影需调整。

五、输出图与对比图解读

输出图文件:

 复制代码

```
name_VASP_W90_cmp.png
```

图中:

- **黑线**: VASP 原始能带;
- **红虚线**: Wannier90 插值能带;
- 若两者几乎重合 $\rightarrow$ 拟合优秀;
- 费米能已自动校准为 0 eV;
- 可通过 `--ylim` 控制纵轴范围。

 示例:

下图显示 VASP 与 Wannier90 能带几乎完全重叠, 说明窗口与投影选择正确。

六、终端输出表格解析

程序会打印两类数据表：

1 能带拟合误差表

i	MAX DIFF (meV)	AVERAGE DIFF (meV)
1	0.36	0.12
2	1.10	0.36
3	1.88	0.38

表示每条能带的最大偏差与平均偏差。

2 Wannier 展宽 (Spreading)

从 `wannier90.wout` 文件读取每个 Wannier 函数的空间展宽 ($\text{\AA}^2$)：

i	Spread ($\text{\AA}^2$)
1	86.8
2	45.8
3-28	21.4

意义：

- Spread 代表 Wannier 函数的“局域性”；
- 越小表示越局域（好）；
- 若 Spread 不收敛或剧烈波动 → 投影或窗口不合理。

七、参数调节建议

情况	建议调整
拟合误差大	调整 <code>dis_win_min/max</code> ，扩大外层窗口
Spread 值偏大	重新检查投影轨道或初始投影方向
能带偏离费米能	手动指定 <code>--efermi</code>
屏幕输出过多	使用 <code>--quiet</code>

💡 八、绘图控制参数

参数	功能
<code>--ylim</code>	控制能量显示范围
<code>--fontfamily</code>	绘图字体 (默认 Open Sans)
<code>--fontsize</code>	字体大小 (默认 18)
<code>--show-fonts</code>	列出可用字体列表
<code>--no-spread</code>	不显示 spread 表格
<code>--no-quality</code>	不显示拟合误差表
<code>--quiet</code>	同时关闭两者输出

✅ 九、典型工作流程总结

步骤	命令	功能	
1	<code>pyw90 pre band</code>	生成 <code>bnd.dat</code> (VASP 能带)	
2	<code>wannier90.x -pp / wannier90.x</code>	生成 Wannier90 插值能带	
3	<code>pyw90 cmp system_name</code>	自动比较并输出图与误差	

输出文件:

复制代码

`system_name_VASP_W90_cmp.png`

终端输出:

- 每个能带的误差统计;
- 每个 Wannier 函数的 Spread;
- 总体拟合质量。

📖 十、总结表

模块	功能	输出内容
cmp	对比 Wannier90 与 VASP 能带	图像 + 拟合质量报告
误差计算	DFT 与 Wannier 差值	$\Delta(\text{max})$, $\Delta(\text{avg})$
局域性分析	Wannier 函数 spread	展宽分布表
自动校准	Fermi 能、窗口范围	自动检测
输出文件	PNG 图像 + 终端表格	可视化验证结果

🧠 十一、最终总结一句话:

💡

pyw90 cmp 是验证 Wannier 拟合精度的质量评估模块。它通过对比 VASP 与 Wannier90 能带、计算偏差与 Spread, 判断当前能量窗口与投影设置是否物理合理且拟合精确。

4.10 pyw90 的 eig 功能介绍

日期：2025-11-27

在线原文：marx2001.github.io

<https://github.com/Cloudiiink/pyw90/blob/master/README.md>

► 简介

🕒 pyw90 eig 菜单功能总结

🌐

一、总体功能介绍

pyw90 eig 菜单用于:

分析能量本征值 (eigenvalues) 的分布, 并为 Wannier90 提供:

- “合适的 排除能带 (exclude_bands) ; ”
- “外层能量窗口 dis_win_min / dis_win_max 的推荐范围。”

■ 作用:

帮助用户在进行 Wannier90 投影前快速了解体系的能量分布结构, 为窗口选择提供数据支撑。

⚙️ 二、基本命令格式

bash

复制代码

```
pyw90 eig [-h] [-e ERANGE ERANGE] [--config] [--path PATH] [-i EIG]
          [--rm-fermi] [--efermi EFERMI] [-w NWANN] [-n NBND5_EXCL]
          [--deg DEG] [--plot] [--separate] [--eps EPS]
          mode
```

◆ positional argument (必须参数)

参数	含义
mode	模式选择: <code>dist</code> 、 <code>count</code> 、 <code>suggest</code>

✂️ 三、三种运行模式

模式	功能	输出结果
dist	显示能带分布	生成能带分布图 <code>dos_analysis.pdf</code> (或在终端打印)
count	统计在某能量区间内每个 k 点的态数	输出每个能量区间的态数信息
suggest	自动推荐能量窗口	自动输出 <code>dis_win_min</code> 、 <code>dis_win_max</code> 、 <code>dis_froz_min</code> 、 <code>dis_froz_max</code>

- `dist` 模式帮助你可视化能量分布。
- `count` 模式量化每个能量范围内有多少态。
- `suggest` 模式结合 Wannier90 的要求，自动推荐窗口。

四、常用参数说明

参数	含义	默认值 / 说明
<code>-e ERANGE ERANGE</code>	设置能量范围	默认 <code>[-1e3, 1e3]</code> eV
<code>--config</code>	从配置文件 (如 <code>auto_w90_input.yaml</code>) 读取参数	False
<code>--path PATH</code>	工作路径	当前目录
<code>-i EIG</code>	指定 <code>EIGENVAL</code> 或 <code>wannier90.eig</code> 文件	自动检测
<code>--rm-fermi</code>	是否将 Fermi 能设为 0	False
<code>--efermi EFERMI</code>	手动设置 Fermi 能	默认从 <code>vasprun.xml</code> 读取
<code>-w NWANN</code>	Wannier 函数数量 (#WFs)	默认 0
<code>-n NBND5_EXCL</code>	忽略的低能带数量	默认 0
<code>--deg DEG</code>	能带简并度	默认 1
<code>--plot</code>	是否生成图像输出	False (仅打印)

<code>--separate</code>	是否分别计算各带	False
<code>--eps EPS</code>	能量容差 (修正数值误差)	默认 0.004 eV

五、功能细节与算法逻辑

1

`suggest` 模式核心逻辑

这是 `eig` 菜单中最关键的功能。
主要用于自动计算 Wannier90 所需能量窗口。

- 先计算所有能带的分布；
- 统计每个能量区间 (E_1, E_2) 内的态数；
- 找出使得态数 $\geq$ `#WFs` 的最低能量区；
- 输出推荐：
 - `dis_win_min`
 - `dis_win_max`
 - 对应的 `dis_froz_min`、`dis_froz_max`

2 冻结窗口 (`dis_froz_min/max`) 的生成算法

生成 `dis_froz` 的过程:

1. 计算每个 k 点上在能量范围 (E_1, E_2) 内的态数;
2. 保证在所有 k 点中至少有 $N \geq NWF5$ 个态;
3. 找到最小 `Emin` 和最大 `Emax` 满足条件;
4. 输出:

ini

 复制代码

```
dis_froz_min = Emin
dis_froz_max = Emax
```

 若出现能带简并或能量离散问题, 算法会使用 `--eps` 容差修正:

ini

 复制代码

```
dis_froz_min = Emin - eps
dis_froz_max = Emax + eps
```

3 跳跃带与简并态修正

在某些材料中 (尤其是金属或有简并点的体系), 能带跨越 `eps` 容差时可能跳过部分态, 导致窗口错误。

因此 `pyw90` 会:

- 自动检测“是否有遗漏态”;
- 如果有 ΔN 个态被跳过, 则自动重新扫描:

mathematica

 复制代码

```
N'_WF = N_WF - 1, ..., N_WF - ΔN
```

来保证不会错过冻结窗口中的关键能带。

🖼️ 六、图形输出

程序会自动生成图像文件，如：

- `eigenval_dis.pdf`：总体能量分布；
- `eigenval_dis_separate.pdf`：逐带分布图；
- `dos_analysis.pdf`：能量直方分布图。

这些图可视化了：

- 每条能带在能量轴上的分布；
- 不同能量段内态的数量；
- 推荐窗口的位置范围。

📋 七、核心功能总结表

功能	说明
统计能带分布	从 <code>EIGENVAL</code> 或 <code>wannier90.eig</code> 读取能量本征值
Fermi 校正	自动设定或手动输入 Fermi 能量
窗口推荐	输出 <code>dis_win_min/max</code> 与 <code>dis_froz_min/max</code>
修正机制	考虑简并带与数值误差 (<code>eps</code> 修正)
图像输出	输出能量分布图 (直方图或分布图)
跳跃检测	自动补偿遗漏带，防止冻结窗口缺态

✅ 八、简明结论

`pyw90 eig` 是一个 用于分析能带分布与自动推荐能量窗口 的模块。
它能帮助你快速了解体系的能量结构，并自动生成适合 `Wannier90` 的参数窗口 (包括 `dis_win` 与 `dis_froz`) 。
在实际工作流程中，常用于：

CSS

复制代码

```
pyw90 eig suggest -w 38 --plot
```

→ 自动输出推荐能量范围并生成图像分析。

4.11 pyw90 的 pre 功能介绍

日期：2025-11-27

在线原文：marx2001.github.io

<https://github.com/Cloudiiink/pyw90/blob/master/README.md>

► 简介

 **pyw90 pre 菜单功能总结**

 **一、总体功能介绍**

pyw90 pre 是 pyw90 中最实用的模块之一，它的作用是：

-  在 VASP 与 Wannier90 之间建立桥梁
自动生成 Wannier90 的输入模板文件，并基于 **投影态密度 (pDOS)** 分析推荐更准确的能量窗口。

简而言之：

- 帮你生成 `.win` 输入模板
- 分析 **投影态密度 (PDOS)**
- 自动推荐 `dis_froz_min/max` 和 `dis_win_min/max`
- 选择最合理的 **轨道投影 (projections)**

 **二、命令格式**

```
bash
pyw90 pre [-h] [--path PATH] [-e ERANGE ERANGE] [--lb LB] [--rm-fermi]
          [--extra EXTRA] [--spin-down] [--plot] [--eps EPS] [--deg DEG]
          mode
```

复制代码

 **三、参数详解**

 **位置参数 (positional argument)**

参数	含义
<code>mode</code>	模式选择 (<code>kpath</code> , <code>band</code> , <code>template</code> , <code>dos</code>) —— 只需输入首字母即可识别。

⚙️ 可选参数 (optional arguments)		
参数	功能说明	默认值
<code>--path PATH</code>	VASP 计算结果所在路径	当前目录 <code>.</code>
<code>-e ERANGE ERANGE</code>	能量范围 (单位 eV)	<code>[-1e3, 1e3]</code>
<code>--lb LB</code>	轨道能量下限, 用于选取最具代表性的投影	<code>0.1</code>
<code>--rm-fermi</code>	是否移除非零费米能 (将 Fermi 归零)	<code>False</code>
<code>--extra EXTRA</code>	附加输入内容 (如 orbital、species、wann)	<code>-</code>
<code>--spin-down</code>	指定自旋通道为 Spin.down	默认 <code>Spin.up</code>
<code>--plot</code>	是否绘制 DOS 分布图	<code>False</code>
<code>--eps EPS</code>	能量误差容差 (修正数值误差)	<code>0.004</code>
<code>--deg DEG</code>	能带简并度	<code>1</code>

✂️ 四、四种主要模式功能介绍		
模式	功能	输出内容
<code>kpath</code>	将 VASP 的 <code>KPOINTS</code> 转换为 Wannier90 的 <code>Kpoint_Path</code> 格式	k点路径块
<code>band</code>	将 VASP 能带 (<code>bnd.dat</code>) 转换为 p4vasp 样式	<code>bnd_up.dat</code> , <code>bnd_down.dat</code>
<code>template</code>	生成 Wannier90 的输入模板文件 (<code>.win</code> 文件)	<code>seedname.win</code> 含能量窗口与投影块
<code>dos</code>	读取 <code>vasprun.xml</code> 计算投影态密度 (pDOS) 并推荐能量窗口	推荐 <code>dis_froz_min/max</code> 、 <code>dis_win_min/max</code>

💡 `dos` 模式是最核心、最重要的部分!

它通过 PDOS 比例判断哪些轨道主导价带和导带, 从而自动选择冻结窗口。

🧠 五、关键原理与逻辑说明

1 pDOS 基于能量积分定义

在 `dos` 模式下，程序读取 `vasprun.xml`，计算：

$$\text{pDOS} = \int_{E_1}^{E_2} \text{DOS}_{\text{atom, orb}}(E) dE$$

其中：

- E_1, E_2 ：由 `-e` 参数设定；
- 计算出每个原子、每个轨道在该能量区间的贡献。

这些积分值再与总 DOS 比较，得到 pDOS/tDOS 百分比，用于判断主导轨道。

2 投影选择与能量窗口推荐

程序根据 pDOS 分布自动推荐：

- 内层冻结窗口 `dis_froz_min/max`
- 外层判定窗口 `dis_win_min/max`

推荐原则：

- pDOS 占比高 → 轨道应纳入投影；
- pDOS/tDOS 比较结果确定价带/导带边界；
- 最显著轨道用于生成 `projections` 块。

3 `--extra` 的作用

`--extra` 参数用于打印或筛选更详细的信息：

- `species`：元素种类
- `orbital_id`：轨道编号
- `structure_id`：原子编号
- `band / wann`：生成不同的模板块

输出格式（示例）：

```
vbnet
Ga,0,1-3;As,1,1-3
```

📄 复制代码

代表选择 Ga 与 As 的 p 轨道作为 Wannier 投影。



4

--lb 参数

用于控制 pDOS 截断下限，即只选取高于某一占比（默认 0.1）的投影轨道作为候选；从而排除微弱杂化态或无关态。

 六、输出结果说明

1

输出表格

当使用 `dos` 模式时，会生成包含以下列的表格：

列名	含义
<code>species</code>	元素类型（如 Ga, As）
<code>structure_id</code>	原子在结构中的索引号
<code>orb_id</code>	轨道编号（s, p, d, f）
<code>orb_name</code>	轨道名称
<code>key_string</code>	pyw90 投影关键字格式（如 <code>Ga,0,1-3;As,1,1-3</code> ）
<code>pDOS</code> , <code>tdOS</code> , <code>percent</code>	分别为投影态密度、总态密度及占比
<code>dis_froz_min/max</code>	推荐冻结窗口
<code>dis_win_min/max</code>	推荐外层窗口

所有结果按 **pDOS/tdOS 占比从高到低排序**，最强的轨道作为 Wannier90 默认投影。

🔔 七、轨道符号对照表

由于不同软件使用不同轨道命名，文档列出了对照表：

软件	轨道符号
pymatgen (2022.8.23)	s, px, py, pz, dxy, dyz, dxz, dx2, dx2-y2, f1-f7
VASP 5.4.4	s, px, py, pz, dxy, dyz, dxz, dx2-y2, fxyz, fx2-y2, fxz2 等
Wannier90	s, px, py, pz, dxy, dyz, dxz, dx2-y2, fx(2y2-z2), fy(3x2-y2) 等

💡 意义：方便用户在 PDOS 输出与 `.win` 文件投影中正确对应轨道类型。

📖 八、总结表

模块	功能描述	典型输出
kpath	转换 KPOINTS 文件	Wannier90 kpath
band	转换能带文件	bnd_up.dat / bnd_down.dat
template	自动生成 Wannier90 输入模板	.win 文件 (含投影与窗口)
dos	分析 pDOS 并推荐投影窗口	dis_froz_min/max, dis_win_min/max, 投影关键字表

✅ 九、核心总结

`pyw90 pre` 是 Wannier90 输入文件自动化生成与能量窗口推荐的核心工具。
它通过 **pDOS/tDOS 分析** 精准选取投影轨道，并输出适合 Wannier90 的模板输入文件。
特别适用于：

- “多轨道复杂体系（如拓扑半金属、过渡金属化合物）”
- “自动判定 `dis_froz` 与 `dis_win` 区间”
- “批量生成 `.win` 模板”

🚀 一句话总结：

`pyw90 pre` 是连接 VASP 与 Wannier90 的“智能预处理器”，
自动生成投影模板与能量窗口，是 `pyw90` 的核心推荐模块。

4.12 pyw90 的简介

日期: 2025-11-27

在线原文: marx2001.github.io<https://github.com/Cloudiink/pyw90/blob/master/README.md>

► 简介

```

There are at most 46 states and at least 46 states in [-20.0, 20.0].

'dis_win_min' and 'dis_win_max' Table:
  Column 'dis_win_max' shows the **lowest** dis_win_max for 'dis_win_min'
  Column 'i+l_min' / 'i_max' shows the band minimum / maximum near the gap
  Column 'Nleast' / 'Nmost' shows the least / most number of states inside 'dis_win_min' and Fermi level.

There is no enough states for -6.344039 with 38 WFs!
There is no enough states for -2.637562 with 38 WFs!
There is no enough states for -0.9930779999999999 with 38 WFs!

```

	dis_win_min	dis_win_max	i+l_min	i_max	Nleast	Nmost
13	-0.993078	6.000000	-0.992188	-1.003060	0	0
12	-2.637562	6.000000	-2.630185	-2.644039	4	4
11	-6.344039	6.000000	-6.249603	-6.438475	16	16
10	-11.470015	3.788728	-9.344601	-13.595429	22	22
9	-13.824348	3.305089	-13.714366	-13.934329	24	24
8	-17.558844	2.903421	-14.486173	-20.631516	26	26

```

/public/home/xpuu/.conda/envs/pyw90_env/lib/python3.9/site-packages/pyw90/pyw90_cli.py:321: FutureWarning: The behavior of DataFrame concatenation with empty or all-NA entries is deprecated. In a future version, this will no longer exclude empty or all-NA columns when determining the result dtypes. To retain the old behavior, exclude the relevant entries before the concat operation.
  df_froz = pd.concat(df_froz_list, ignore_index=True).drop_duplicates()

'dis_froz_min' and 'dis_froz_max' Table:
  Suggest 'dis_froz_min' & 'dis_froz_max' as following with #WFs = 38 from input
  Use 'pre dos' menu to get suggestions based on projected density of states

```

	dis_froz_min	dis_froz_max	N
12	-20.627516	2.154407	38
13	-13.931077	2.204247	37
6	-13.930329	2.600411	38
5	-13.930329	2.600284	38
0	-13.591429	2.978903	38
1	-6.967550	3.014653	37
2	-6.867352	3.427087	38
3	-6.819218	3.442032	37
4	-6.819035	6.000000	38

你提的这个问题非常好，而且非常具有代表性：

为什么更大的能量窗口 (`dis_win_min ~ dis_win_max`) 反而得不到足够的38个态 (`states`)，而较小的冻结窗口 (`dis_froz_min ~ dis_froz_max`) 却可以满足？

这实际上是一个**“质量 vs 数量”**的问题，主要涉及以下几点物理和算法的理解：

✅ 问题分析

🧠 1. `dis_win_*` 是“判定窗口”，目标是找出能落入该窗口的“合适”的 Wannier Functions (WFs)

- 定义： `dis_win_min` 和 `dis_win_max` 定义了一个判定能区窗口。
- 目标：在这个窗口内，选出一组 WFs，使它们能有效拟合目标能带，并覆盖所有需要拟合的能态。

❄️ 2. `dis_froz_*` 是“冻结窗口”，里面的态直接参与构建 WFs

- 更容易控制数量：冻结窗口内的态是“强制使用”的，不管质量如何。
- 所以只要你设置得当，比如包含较低能的价带态，通常是比较局域化、容易拟合的态，就能稳定地找到38个。

🖼️ 3. `dis_win_*` 质量控制更严格 —— 如果拟合不好，会被剔除

- 判定过程更复杂，尤其是对“过宽”的 `dis_win` 区域，会引入一些难以拟合的态（比如延展态、高导带态等）。
- 这些态虽然在能量窗口内，但因为质量不够好，会被跳过，导致最终状态数 < 38 。

✅ 建议

1. 优先调整 `dis_froz_*` 以覆盖目标态；
2. 确保 `dis_win_*` 不要太宽，特别是不要引入高能导带；
3. 可以在 `wannier90.win` 中尝试添加：

```
plaintext
```

📄 复制代码

```
exclude_bands : ...
```

来手动排除“坏态”；

4. 检查投影原子轨道和能带是否匹配（比如 s, p, d 的数量和分布）；
5. 利用 `pre dos` 菜单观察 PDOS 分布，有助于精细选择窗口。

你的意思是说，大窗口不要太宽，让大窗口的能量尽可能的低，但是不能小于小窗口？

非常好的理解——你已经抓住了关键逻辑，只差一点点细化。

我来帮你彻底理清 `dis_win` 与 `dis_froz` 的能量关系和选取原则 📌

🔄 一句话总结

✅ 大窗口（`dis_win`）应该略宽于小窗口（`dis_froz`），但不能太宽。

💡 它的能量下限可以比小窗口低一点，上限也可以稍微高一点，但过宽反而会“坏事”。

📖 原理详解

Wannier90 在选择能量窗口时有两个层次的过滤机制：

窗口类型	名称	功能	能量区间的作用
<code>dis_win</code>	判定窗口 (disentanglement window)	决定“哪些能态可以被考虑进入 WFs 拟合”	越宽 → 包含更多能态（可能掺入不相关导带）
<code>dis_froz</code>	冻结窗口 (frozen window)	决定“哪些能态必须被保留为目标 WF”	这些态会被直接拟合，不参与优化剔除

🖼️ 为什么太宽反而不行？

当你把 `dis_win` 拉得太宽，比如上限加到很高的导带能量：

- 系统会把很多“延展性强、非局域化”的导带态也纳入；
- 这些态与价带态在能量和空间上差别大，Wannier 拟合会出现“正交化不良”；
- 结果：算法自动丢弃一些态（因此 `states < 38`）；
- 这时你就看到提示：“There is no enough states for ... with 38 WFs!”

你发现了一个**非常典型的“pyw90 与 Wannier90 拟合行为不一致”的情况——

pyw90 提示 “There is no enough states for 38 WFs”, 但实际运行 wannier90 拟合能带时却完全没有报错，结果很好。

这其实是合理的，原因在于：
pyw90 与 wannier90 的“判断标准”并不完全相同。下面我来系统地解释原因 📌

🔧 一、pyw90 的作用与局限性

pyw90 是什么？

pyw90 是一个 “辅助分析与自动窗口优化工具”，它的功能是：

- 根据 EIGENVAL / PROCAR 等数据，
- 自动寻找合适的 dis_win_min/max、dis_froz_min/max，
- 并判断给定窗口是否**包含足够的态 (states) **来生成指定数量的 WFs。

⚠️ pyw90 的判断标准是 严格的几何判定

它根据能量分布 (DOS) 数值统计“在此窗口内至少有多少个态”，但并不真正执行 Wannier 函数投影和 disentanglement 优化。

所以它的“判定不足”只是：

“从纯能量统计角度看，这个窗口中能量落入范围的 band 数不够 38。”

并不意味着 Wannier90 不能生成或优化出 38 个 WFs。

4.13 wannier90 的能带拟合步骤

日期：2025-11-27

在线原文：marx2001.github.io

► 流程与步骤日志

► 1. static 计算

不开启 ncl
k 点取 11, 如下

代码清单 A.20.1

语言：shell；完整代码见附录第 395 页，共 5 行。

INCAR 如下

代码清单 A.20.2

语言: shell; 完整代码见附录第 396 页, 共 43 行。

运行计算

获得结果并查看磁矩

代码清单 A.20.3

语言: shell; 完整代码见附录第 396 页, 共 11 行。

► 2. bandsoc 计算

INCAR 如下

代码清单 A.20.4

语言: shell; 完整代码见附录第 397 页, 共 42 行。

kpoints 如下

代码清单 A.20.5

语言: shell; 完整代码见附录第 397 页, 共 15 行。

绘制投影能带 (略)

► 3. ncl_Wannier90 计算

添加 LWANNIER90=.T

以及 WANNIER_WIN 和 NBANDS=64 以及 NUM_WANN=38 的 tag, 其它的参数参考 ncl 的设置方式进行修改

NCORE=4 注释掉

代码清单 A.20.6

语言: shell; 完整代码见附录第 398 页, 共 54 行。

体系不易收敛, 可以尝试开启输出波函数和电荷密度方便续算.

► 4. 能带拟合

后续过程见之前的 Bi 烯教程中的相应部分的内容

► Results

代码清单 A.20.7

语言: shell; 完整代码见附录第 399 页, 共 10 行。

结果不太对, 判断为构建紧束缚模型的时候可能考虑了多余的轨道, 导致既有陈数又有 Z2.

如果 static 是线性计算的, 看看是否会有影响。(因为之前用的线性计算的 static 的 chgcar, 并未出现陈数和 z2 数同时出现的情况)

问题同样出现, 应该是拟合轨道数目的问题, 新版本 wannier 可能对轨道的严格性做了提升, 1.2 的版本可以正常计算出 Z2 拓扑不变量。

► 选择合适的轨道的结果

遇到一个情况, 启动第三步, 静态 ncl+LWANNIER90=.T 的时候, 一定要先清理上次计算剩下的输出文件 mmn 等, 并删掉 win 文件中上次计算 vasp 生成的部分, 然后重新启动计算, 这样能保证输出正确, 减少遇到非主流报错的可能性。

4.14 pyw90 的测试用例

日期: 2025-11-28

在线原文: marx2001.github.io

<https://github.com/Cloudiink/pyw90/blob/master/README.md>

创建虚拟环境

命令示例 · shell

```
conda create -n wannier-env python=3.9 -y
python3 -m venv pyw90-env
```

激活环境并安装

代码清单 A.21.2

语言: shell; 完整代码见附录第 399 页, 共 8 行。

► 流程与步骤

例子选取于上方网址, 作为学习交流使用, 切勿用作他途。

以下假设已熟悉 Wannier90, 并已按下方示例准备文件夹。该结构仅作参考; 实际操作时, 命令在能带计算文件夹中运行。

代码清单 A.21.3

语言: shell; 完整代码见附录第 399 页, 共 15 行。

通过 eig 菜单显示特征值分布:

代码清单 A.21.4

语言: shell; 完整代码见附录第 400 页, 共 1 行。

结果为:

代码清单 A.21.5

语言: shell; 完整代码见附录第 400 页, 共 23 行。

也可以将结果输出为 pdf 格式, 且是某个波段的能量分布到文件夹中。

运行下方指令得到一些关于能量窗口的建议:

代码清单 A.21.6

语言: shell; 完整代码见附录第 400 页, 共 1 行。

结果:

代码清单 A.21.7

语言: shell; 完整代码见附录第 400 页, 共 24 行。

运行指令得到建议投影的轨道并产生 wannier90.in。

此处, 我们选择费米能级附近 $[-1, 1]$ eV 的能量区间进行评估, 以得出最具代表性的投影权重建议

也可以通过调整相应参数来修改能量区间的选择下限, 以观察输出结果的变化。

代码清单 A.21.8

语言: shell; 完整代码见附录第 401 页, 共 2 行。

结果:

代码清单 A.21.9

语言: shell; 完整代码见附录第 401 页, 共 78 行。

尝试修改上下限:

代码清单 A.21.10

语言: shell; 完整代码见附录第 402 页, 共 1 行。

代码清单 A.21.11

语言: shell; 完整代码见附录第 402 页, 共 76 行。

可以看出拟合能带的能量范围不同, 投影轨道的数目也不同。

随后, 我们可以将能量范围调整至一个非常大的区间:

代码清单 A.21.12

语言: shell; 完整代码见附录第 404 页, 共 1 行。

其中, 第一个“1-2”是 Nb 的原子编号, “4-8”为轨道编号, 其余元素的格式同理。上一步中的靠后部分有这些编号的 output

结果为:

代码清单 A.21.13

语言: shell; 完整代码见附录第 404 页, 共 35 行。

轨道编号参考 (vasp5.4.4 与 vasp6.4.3 应当相同)

代码清单 A.21.14

语言: shell; 完整代码见附录第 404 页, 共 12 行。

由此, 我们获得了 Wannier 函数的初始猜测。随后, 我们可以使用命令 `pyw90 pre template -extra basic` 来为 Wannier90 与 VASP 的接口生成 `wannier90.win` 模板文件。

由于 (VASP 版本 <6.0) 仅支持 Wannier90 1.2 版本的接口。可能需要修改并重新编译接口代码, 使其能够计算非共线 Wannier 函数并支持旋量投影方法。更多详细信息, 请参考项目: [Chengcheng-Xiao/VASP2WAN90_v2_fix](#)(一个更新版的 VASP2WANNIER90v2 接口)。

此处由于不识别 “ ” 符号, 所以可以手写 win 文件:

代码清单 A.21.15

语言: shell; 完整代码见附录第 405 页, 共 12 行。

此文件 (`wannier90.win`) 需在 `wannier` 文件夹内创建。同时, 必须在 `INCAR` 文件中设置 `LWANNIER90 = .TRUE.` 以启用 VASP 与 Wannier90 之间的接口。

此功能 (VASP-Wannier90 接口) 仅在 VASP 编译时启用了 `-DVASP2WANNIER90` 或 `-DVASP2WANNIER90v2` 选项才可用。

其他输入参数 (如 `kpoints`- k 点列表, `atoms`- 原子位置, `unit_cell`- 晶胞矢量, `mp_grid`- Monkhorst-Pack 网格尺寸等) 将基于 VASP 计算的信息自动设置。

对于 VASP ($\geq 6.2.0$) 版本, 它将支持 Wannier90 ≥ 3.0 。并且, `wannier90.win` 输入文件的内容可以直接通过 `INCAR` 文件中的 `WANNIER90_WIN` 参数输入 (参见 [Vaspwiki](#) 上的 `WANNIER90_WIN` 条目), 不再需要单独的初始 `wannier90.win` 文件。

执行计算任务后, VASP 会为后续的 WANNIER90 运行写入所需的输入文件: `wannier90.win`, `wannier90.mmn`, `wannier90.eig` 以及 `wannier90.amn`。

这里我们将 `disentanglement` (`dis`) 能量窗口的初始猜测和能带结构计算的路径卡片写入 `wannier90.win` 文件。其中 `Kpoint_Path` 模块可以通过命令 `pyw90 pre kpath -path ../bnd` 生成 (此命令可能利用之前能带计算 `../bnd` 的路径信息)。

开始第一次 wannier90 能带拟合, 具体过程参考之前的教程

代码清单 A.21.16

语言: shell; 完整代码见附录第 405 页, 共 12 行。

我使用的是 vasp6.4.3 的版本, 写在 INCAR 内即可, 不需要单独的 win 文件。

随后, 我们可以使用 pyw90 auto 菜单来自动优化 disentanglement 能量窗口。配置文件应使用 pyw90 auto input 命令生成。在运行计算之前, 请自定义该配置文件 (有关每个参数键的含义, 请参阅该菜单的文档)。

先进行 wannier90 的能带拟合, 得到 wannier90_band.dat 等等文件。

准备好所有输入文件后, 键入 pyw90 auto run 来执行该流程。会产生两个输出文件。

我们一步一步进行。

先运行脚本

代码清单 A.21.17

语言: shell; 完整代码见附录第 405 页, 共 1 行。

产生 auto_w90_input.yaml 文件, 修改其中的内容与 slurm 系统的匹配

再修改 yaml 文件中的参数

代码清单 A.21.18

语言: shell; 完整代码见附录第 405 页, 共 2 行。

或

代码清单 A.21.19

语言: shell; 完整代码见附录第 406 页, 共 2 行。

编辑提交任务脚本 w90.script

代码清单 A.21.20

语言: shell; 完整代码见附录第 406 页, 共 42 行。

然后运行自动分析程序

代码清单 A.21.21

语言: shell; 完整代码见附录第 406 页, 共 1 行。

等待即可。

此刻凌晨 3:20 了, 稍后睡醒再写一篇如何修改正确参数来进行能量窗口计算的文章。

补充, 最后一步, 利用命令生成可视化的 DFT 计算能带与 pyw90 结果的对比

代码清单 A.21.22

语言: shell; 完整代码见附录第 407 页, 共 1 行。

yaml 文件中有 kernel 的设置, efermi 可以自己寻找

备注: 如果没有 bnd.dat 文件, 运行指令产生

代码清单 A.21.23

语言: shell; 完整代码见附录第 407 页, 共 1 行。

4.15 pyw90 的测试用例 2_本地运行

日期: 2025-11-28

在线原文: marx2001.github.io

<https://github.com/Cloudiiink/pyw90/blob/master/README.md>

首先以任意窗口成功地运行一次 weannier90, 得到 wannier90_band.dat 文件。准备好 auto_w90_input.yaml 文件。

修改 auto_w90_input.yaml 文件中的代码为:

代码清单 A.22.1

语言: shell; 完整代码见附录第 407 页, 共 20 行。

然后运行, 这个要在 band 计算的文件夹中运行才能得到正确的 bnd.dat, 大小应该和 BAND.dat 文件相差不大

代码清单 A.22.2

语言: shell; 完整代码见附录第 407 页, 共 1 行。

得到 vasp 计算的 bnd.dat 文件

再运行

代码清单 A.22.3

语言: shell; 完整代码见附录第 407 页, 共 1 行。

设置窗口的技巧, 能量上下限减去费米能级, 即为 origin 作图的能带图的能量范围, 可以根据 dft 计算的能量范围设计每一次迭代的能量上下限。

得到结果后, 修改 yml 文件中的窗口范围, 然后改变固定窗口, 比如原来固定小窗口, 优化大窗口, 同时也要修改窗口变化的范围, 大概看一眼能带, 选择合适的区间

经过一些测试, 我发现固定两个窗口的 min, 优化 max 窗口的位置似乎是一个不错的开始, 因为我们可以 gap 处设置好 min, 可以保证这个 min 的数值没有问题, 然后比较难搞的上窗口通过迭代方式优化得到。

以上为暂时的最终的程序使用方法和心得体会。

流程梳理:

1. static 计算得到 CHGCAR
2. bandsoc 计算得到 bnd.dat
3. wannier90 的能带拟合, 一定要确定好 min 窗口所在的能量, 记得减去费米能级对照 origin 画的能带图确定 min 窗口所在的能量值
4. 运行 pyw90 进行上窗口能量优化, 如果中途计算因为包含的 state 不足而停掉, 那么需要重新修改 yaml 的窗口和 win 文件的窗口匹配, 检查好 yaml 内能量窗口的变化的范围。
5. 重复上述步骤 3、4, 直到能带完全对应。

代码清单 A.22.4

语言: shell; 完整代码见附录第 408 页, 共 1 行。

yaml 文件中有 kernel 的设置, efermi 可以自己寻找

备注: 如果没有 bnd.dat 文件, 运行指令产生

代码清单 A.22.5

语言: shell; 完整代码见附录第 408 页, 共 1 行。

终止任务

代码清单 A.22.6

语言: shell; 完整代码见附录第 408 页, 共 1 行。

或者

代码清单 A.22.7

语言: shell; 完整代码见附录第 408 页, 共 1 行。

4.16 Wannier Tools 的代码解释

日期: 2025-12-02

在线原文: marx2001.github.io

创建虚拟环境

代码清单 A.23.1

语言: fortran; 完整代码见附录第 408 页, 共 2051 行。

代码清单 A.23.2

语言: fortran; 完整代码见附录第 438 页, 共 290 行。

4.17 wannierberrier 的安装

日期: 2025-12-03

在线原文: marx2001.github.io

► Recording

► 1. 步骤

创建虚拟环境

命令示例 · shell

```
conda create -n wannier-sym python=3.12.12 -y

conda activate wannier-sym
```

使用清华源安装

代码清单 A.24.2

语言: shell; 完整代码见附录第 443 页, 共 9 行。

查看所需依赖的版本:

命令示例 · shell

```
python3 --version
```

装完环境重启一下 shell 面板, 会更新 python 的版本显示。

修改并更新知乎中的代码, 还需要安装 FFT 等库, 安装很简单, 就不在此一一列出了。

目前的代码是大概正确的, 报错是因为晶格常数不完全对称, 但是没有找到代码中调整 tolerance 的 api, 所以我用高对称性结构重新加入 wannier 接口拟合一遍。第一版代码, 存在报错, 这是为了记录, 可以及时进行版本回退

代码清单 A.24.4

语言: shell; 完整代码见附录第 443 页, 共 93 行。

4.18 wansymm 的对称化功能

日期: 2025-12-03

在线原文: marx2001.github.io

► Recording

► 1. 步骤

其余步骤同知乎上文章 <https://zhuanlan.zhihu.com/p/675502389> 和 <https://zhuanlan.zhihu.com/p/675502472>, 但是 build 文件夹内并没有出现 lib, 所以我把路径放到 lib64 文件夹中, 即在配置变量这个步骤中, 把路径指定到 lib64 文件夹, 而不是知乎上写的 build/lib。

给命令授权:wannsymm.x, 在 cssong 目录中的 bashrc 内添加环境变量的路径即可。

命令示例 · shell

```
conda activate wannier-sym
```

命令示例 · shell

```
wannsymm.x ws.in
```

没节点了, 直接管理节点上用 1 核跑

命令示例 · shell

```
nohup mpirun -np 1 /public/home/cssong/app/vasp/intel_2021.3.0/544+w90-2.1/vasp_std >
→ log 2>&1 &

nohup mpirun -np 1 /public/home/cssong/app/vasp/intel_2021.3.0/5.4.4/vasp_ncl > log
→ 2>&1 &
```

绘图脚本记录:

命令示例 · shell

```
gnuplot surfdos_bulk.gnu surfdos_l.gnu surfdos_l_only.gnu surfdos_r.gnu
→ surfdos_r_only.gnu
```

正式开始计算:

1. 采用模板,ws.in, 进行第一步计算:

代码清单 A.25.5

语言: python; 完整代码见附录第 445 页, 共 19 行。

1. 第二步加入高对称点路径

加入 restart = T 绘制对称化后的能带

代码清单 A.25.6

语言: python; 完整代码见附录第 445 页, 共 9 行。

4.19 利用 Wannier Tools 的对称化 hr.dat

日期: 2025-12-14

在线原文: marx2001.github.io

► Tutorials-irvsp 补充和指令

► 简介

确保 hr.dat 仍保留 DFT 计算中的对称性, 这样能保证 wannier90 的计算结果是正确的

► 流程

1. 准备 POSCAR
2. 判断空间群:

命令示例 · shell

```
conda activate mrx_phonopy
```

然后

命令示例 · shell

```
phonopy --symmetry --tolerance 0.005
```

判断空间群

1. 复制新 POSCAR, 再做计算静态自洽、能带计算

► Wannier Tools 对称化

1. 准备输入文件 locaxis.in

代码清单 A.26.3

语言: python; 完整代码见附录第 445 页, 共 24 行。

采用晶格对称性, 设置 line 20 为 0, 即不读取原子坐标系。

准备 poscar.in, 内容:

代码清单 A.26.4

语言: python; 完整代码见附录第 446 页, 共 13 行。

复制 poscar 进去。

对应的轨道, 准备文件 wann.in

代码清单 A.26.5

语言: python; 完整代码见附录第 446 页, 共 30 行。

准备 wannier90.in 文件, 内容:

代码清单 A.26.6

语言: python; 完整代码见附录第 447 页, 共 70 行。

准备 hr.dat 文件, 未对称化的。

1. 激活环境, 运行程序

命令示例 · shell

```
conda activate sym_env
```

运行程序

命令示例 · shell

```
python /public/home/cssong/install_package/cssong/pack/wannier_tools-master/wannhr_s_」  
↪ ymm/symmhr_addrptblock/symmhr_addrptblock.py
```

记得把 python 库和上面的 symmhr_addrptblock.py 放到一个文件夹内, 要不然运行不了

看日志, 没有报错的话, 就得到了最终的 hr.dat

要用 wannier1.2 版本。

紧束缚、拓扑与磁性软件

本专题收录 13 篇文章。

5.1 tb2j 计算磁参数的 DMI 计算

日期: 2025-11-26

在线原文: marx2001.github.io

► Recording

对于 tb2j 的 0.7 版本, DMI 体现在原子对内, 并无如同四态法一样的计算形式, 所以需要手动推导每一个原子对的 DMI 大小。

1. 首先找到两个原子的相对位置, 对应的原子对, 做中垂线
2. 找到两个原子间的 DMI 坐标形式的强度分布, 对这个中垂线做投影, 相加即为 $D_{//}$ 的大小
3. D_z 就是绝对值大小, 因为在二维投影上是一个点

5.2 TB2J 新版测试

日期: 2025-11-27

在线原文: marx2001.github.io

TB2J0.9.0+Wannier3.1.0+vasp6.4.3

► Recording

► 1. 查看 tb2j 版本

命令示例 · shell

```
pip show TB2J
```

新 0.7.2.1 版本的 tb2j 安装:

创建环境

命令示例 · shell

```
conda create -n tb2j_env python=3.10 -y
```

进入环境:

命令示例 · shell

```
conda activate tb2j_env
```

现升级为 tb2j0.9.0 版本。

命令示例 · shell

```
pip install TB2J==0.9.0 -i https://pypi.tuna.tsinghua.edu.cn/simple
```

0.7.1.1 版本的环境进入方法:

命令示例 · shell

```
source /public/home/cssong/song/zhchen/1_materials/1_VTeCl/TB2J/strain/-2/TB2J_env/bj
↪ in/activate
```

► 2. 运行

命令示例 · shell

```
wann2J.py --spinor --groupby spin --efermi -1.071192 --emin -6.39197 --emax 4.94
↪ --elements Nb --kmesh 7 7 1 --nz 100 --rcut 6.6 --np 1
```

自行修改合适的区间

► 3. 如果配体的自旋磁动量不可忽略, 需要 downfolding 海森堡哈密顿量

先:

命令示例 · shell

```
wann2J.py --elements Cr I ....
```

结果保存到 TB2J_results 目录后, 运行 downfolding 功能, 得到仅有 Cr 有效交换参数:

命令示例 · shell

```
TB2J_downfold.py --inpath TB2J_results --outpath TB2J_results_downfold --metals Cr
↪ --ligands I
```

文件夹中的文件要有 POSCAR、POTCAR、xyz、hr.dat 或 tb.dat(某些版本的 tb2j 可以识别, 见官方文档)、win 文件等等, 主要是 dat 文件, 都放上也没坏处。

► Results

上一模块是简要的过程记录, 下面是结果比对, 包括新老版本的

首先用 0.7.1.1 的版本进行测试:

进入环境:

命令示例 · shell

```
source /public/home/cssong/song/zhchen/1_materials/1_VTeCl/TB2J/strain/-2/TB2J_env/bj
↪ in/activate
```

运行命令:

命令示例 · shell

```
wann2J.py --spinor --groupby spin --efermi -1.065140 --emin -3 --emax 1 --elements Nb
↪ --kmesh 7 7 1 --nz 100 --rcut 6 --np 1
```

得到结果

其它命令:

求平均值:

命令示例 · shell

```
TB2J_merge.py --type structure x y z
```

求磁耦合参数:

命令示例 · shell

```
wann2J.py --spinor --groupby spin --efermi -1.0098 --emin -2.06514 --emax -0.065
↪ --elements Nb --kmesh 9 9 1 --nz 100 --rcut 6.0 --np 1
```

得到结果如下, 体系为交错磁, 而计算结果显示 z 方向磁矩为铁磁, 我认为紧束缚模型可能有问题, 需要重新拟合。升级了 wannier90 版本之后, 也有许多新问题出现, 比如一样的能量窗口, 拟合结果也对的上, 但是 k 点由 551 变为 222211 之后出现表面态等结果不对等问题, 而且 k 点到超过 13/31 之后, 会出现开发者才能遇到的问题(报错中的信息是这么写的), 报错内容是 k 点矢量方向变化, 按照 chatgpt5.1 的建议修改为偶数个 k 点后, 可以正常计算。

```

=====
Information:
Exchange parameters generated by TB2J 0.9.0
Generation time: 25/11/27 14:39:51
Input from non-collinear Wannier90 data.
Tight binding data from ./
Prefix of wannier function files:wannier90.
Warning: Please check if the noise level of Wannier function Hamiltonian to make sure it is much smaller than the exchange values.
The DMI component parallel to the spin orientation, the Jani which has the component of that orientation should be disregarded
e.g. if the spins are along z, the xz, yz, zz, zx, zy components and the z component of DMI.
If you need these component, try to do three calculations with spin along x, y, z, or use structure with z rotated to x, y and z. And then use TB2.

=====
Cell (Angstrom):
=====

Atoms:
(Note: charge and magmoms only count the wannier functions.)
Atom_number   x       y       z   w_charge  M(x)   M(y)   M(z)
-----
1             8.38225808  3.4007  0.0000  0.0000  0.8044
2             8.38231081  3.4006 -0.0264  0.0000  0.8043
3            10.15665696  4.9233 -0.0000  0.0000 -0.0000
4             6.59126596  4.9012 -0.0000  0.0000  0.0000
5            8.09768426  5.2688  0.0000  0.0000  0.0000
Total          21.8946 -0.0264  0.0000  1.6087
=====

```

第一次测试结束。

5.3 TB2J 新版测试

日期: 2025-12-01

在线原文: marx2001.github.io

TB2J0.9.0+Wannier3.1.0+vasp6.4.3

► Recording

► 1. 查看 tb2j 版本

命令示例 · shell

```
pip show TB2J
```

新 0.7.2.1 版本的 tb2j 安装:

创建环境

命令示例 · shell

```
conda create -n tb2j_env python=3.10 -y
```

进入环境:

命令示例 · shell

```
conda activate tb2j_env
```

现升级为 tb2j0.9.0 版本。

命令示例 · shell

```
pip install TB2J==0.9.0 -i https://pypi.tuna.tsinghua.edu.cn/simple
```

0.7.1.1 版本的环境进入方法:

命令示例 · shell

```
source /public/home/cssong/song/zhchen/1_materials/1_VTeCl/TB2J/strain/-2/TB2J_env/b_j
↪ in/activate
```

► 2. 运行

命令示例 · shell

```
TB2J_rotate.py POSCAR --ftype vasp
```

生成三个旋转结构

总览: 对三个目录中的结构分别执行 DFT 自洽计算, 必须保持自旋沿着 z 方向, 然后对每个结构运行 TB2J。

下面是第一步

先对 xyz 结构进行 wannier90 拟合, 窗口不变, 在此之前要先做静态自洽计算, 得到 chgcar。

然后运行

命令示例 · shell

```
wannier90.x wannier90 &
```

命令示例 · shell

```
wann2J.py --spinor --groupby spin --efermi -1.065140 --emin -6.39197 --emax 1.2082
↪ --elements Nb --kmesh 11 11 1 --nz 101 --rcut 6.0 --np 1
```

开始计算。

对 y、z 文件夹做同样的处理。

最后收集平均值:

命令示例 · shell

```
TB2J_merge.py --type structure x y z
```

5.4 pythTB 的学习 _Parameterized_TBModel

日期: 2025-12-09

在线原文: marx2001.github.io

► Tutorials

► 初始化与填充 WFArrray

本教程展示了如何在 PythTB 中构建参数化的紧束缚模型, 如何针对不同参数值对模型进行评估, 以及如何将这些参数永久“固化”到模型中。

核心要点

在整个过程中, 请始终记住: 所有参数值都以标量 (或在扫描时以一维标量数组) 的形式被使用。

本节内容包括

定义符号化的跃迁项/在位项。

在参数扫描中评估哈密顿量、本征值等。

通过以下方法永久“固化”参数:

使用 `TBModel.set_parameters` 直接设置。

使用 `TBModel.with_parameters` 返回一个参数固定的模型副本。

扩展到自旋相关的模型, 演示如何以参数化方式提供 2×2 矩阵块。

代码清单 A.30.1

语言: python; 完整代码见附录第 448 页, 共 2 行。

► 无自旋模型与符号化跃迁

我们从一个在类蜂窝晶格上的熟悉的两轨道无自旋模型开始。最近邻跃迁由单一符号 t 参数化。

代码清单 A.30.2

语言: python; 完整代码见附录第 448 页, 共 13 行。

目前, 模型已识别出其依赖一个名为 t 的参数。请注意下方打印输出中, 位点 0 到位点 1 的跃迁 (晶格矢量 $[0, 0]$ 处) 被表示为 t , 而其他跃迁值则显示为数值。

代码清单 A.30.3

语言: python; 完整代码见附录第 448 页, 共 1 行。

代码清单 A.30.4

语言: python; 完整代码见附录第 448 页, 共 1 行。

代码清单 A.30.5

语言: python; 完整代码见附录第 448 页, 共 1 行。

代码清单 A.30.6

语言: python; 完整代码见附录第 449 页, 共 40 行。

例如, 如果我们尝试在不提供 t 的情况下构建哈密顿量, PythTB 会引发错误, 因为符号 t 没有具体数值。

代码清单 A.30.7

语言: python; 完整代码见附录第 449 页, 共 4 行。

代码清单 A.30.8

语言: python; 完整代码见附录第 449 页, 共 1 行。

► 在评估过程中提供参数值

每一个评估例程 (例如 hamiltonian、solve_ham、velocity 等) 都接受对应参数的关键字参数。参数值可以是标量, 也可以是一维标量数组。

代码清单 A.30.9

语言: python; 完整代码见附录第 450 页, 共 13 行。

代码清单 A.30.10

语言: python; 完整代码见附录第 450 页, 共 4 行。

扫描结果的参数轴会附加在 k 点轴之后。在此例中, 每个 k 点对应 5 个参数值 t , 而 k 点共有 400 个, 因此哈密顿量数组的最终形状为 (400, 5, 2, 2)。solve_ham 函数遵循相同的处理模式。

代码清单 A.30.11

语言: python; 完整代码见附录第 450 页, 共 7 行。

代码清单 A.30.12

语言: python; 完整代码见附录第 450 页, 共 4 行。

该方法也以相同的方式接受参数值。一个重要的注意事项是: 当提供参数扫描时, 哈密顿量相对于参数范围的导数是通过有限差分计算的, 并连同 k 导数一起附加到主轴上。

代码清单 A.30.13

语言: python; 完整代码见附录第 450 页, 共 7 行。

注意

当传递一个标量时, 速度数组的形状是 (2, 400, 2, 2), 其中 t 是参数。

当传递 5 个值的扫描时, 速度数组的形状变为 (3, 400, 5, 2, 2)。此时, 在 k 值轴之后添加了一个参数值轴, 同时关于 t 的有限差分导数被附加在最前面的轴上。

形状说明

标量参数:(2, 400, 2, 2) 表示 2 个导数分量,400 个 k 点,2 个轨道,2 个轨道。

参数扫描:(3, 400, 5, 2, 2) 表示 3 个导数分量 (包括对 t 的导数),400 个 k 点,5 个参数值,2 个轨道,2 个轨道

代码清单 A.30.14

语言: python; 完整代码见附录第 451 页, 共 4 行。

► 固定参数

TBModel.set_parameters 方法会将符号化的提供方 (字符串或可调用对象) 替换为指定数值。调用此方法后, 这些项将变为永久固定的数值。

重要提示 传递到 set_parameters 方法的每个参数必须是标量。标量数组仅允许用于评估例程 (如计算哈密顿量、能带等), 而不能用于固定模型参数。

代码清单 A.30.15

语言: python; 完整代码见附录第 451 页, 共 2 行。

代码清单 A.30.16

语言: python; 完整代码见附录第 451 页, 共 1 行。

代码清单 A.30.17

语言: python; 完整代码见附录第 451 页, 共 42 行。

短代码 · python

```
# Once frozen, the model no longer expects 't'
H_frozen = tb.hamiltonian(k_mesh)
print("Hamiltonian shape after freezing parameters:")
print(H_frozen.shape)
```

代码清单 A.30.19

语言: python; 完整代码见附录第 452 页, 共 2 行。

若不希望修改原始模型, 可以使用 TBModel.with_parameters 方法。此方法会返回一个指定参数已被冻结的副本。

代码清单 A.30.20

语言: python; 完整代码见附录第 452 页, 共 17 行。

代码清单 A.30.21

语言: python; 完整代码见附录第 452 页, 共 3 行。

► 无自旋模型与多重符号化跃迁项

本次我们创建一个包含多个符号化跃迁项的模型。这些参数可以使用相同的名称,也可以使用不同的名称。当进行评估时,具有相同名称的参数将共享相同的数值。如下所示,两个轨道共享同一个 `onsite` 参数,并且两个跃迁具有不同的参数名称 `m`、`t1` 和 `t2`。

代码清单 A.30.22

语言: `python`; 完整代码见附录第 453 页,共 10 行。

代码清单 A.30.23

语言: `python`; 完整代码见附录第 453 页,共 38 行。

下面我们将把其中两个参数设置为具体数值,而将 `t2` 保留为一个自由参数,以便在后续计算中再指定。

代码清单 A.30.24

语言: `python`; 完整代码见附录第 453 页,共 3 行。

结果:

代码清单 A.30.25

语言: `python`; 完整代码见附录第 454 页,共 38 行。

如果我們希望在冻结参数后将其恢复为符号形式,可以使用与最初构建模型时相同的方法,即 `set_onsite` 和 `set_hop` 函数。在此,我们重新将 `onsite` 能量设置为依赖参数 `m`。

代码清单 A.30.26

语言: `python`; 完整代码见附录第 454 页,共 2 行。

5.5 pythTB 的学习 _lattice 类

日期: 2025-12-09

在线原文: marx2001.github.io

► Tutorials

► 1. 介绍

PythTB 是一款用于构建和分析紧束缚模型的 Python 库,专为现代拓扑能带论应用而设计。它提供了从模型设定到物理解释的高效路径,既适用于电子结构的学习,也可用于研究级别的计算。只需几行代码,即可定义晶格模型、构建紧束缚哈密顿量并计算电子性质。

PythTB 提供一个从模型构建、计算分析到结果可视化的完整 workflow,其六大核心功能模块如下:

模型构建: 核心是 `TBModel` 类, 用于定义和操作紧束缚哈密顿量, 可设置在位能、跃迁项、自旋结构及参数依赖项。

态采样: 通过 `Mesh` 类构建结构化的 k 空间和参数网格, 并在网格上采样哈密顿量, 将结果态 (波函数) 存储于 `WFArray` 对象中以供后续分析。

拓扑与量子几何计算: 利用 `WFArray` 和 `TBModel` 的方法, 计算一系列关键物理量, 包括贝里相位/联络/曲率、陈数、轴子角、局域陈数标记、混合万尼尔函数及其他量子几何观测量。

Wannier90 集成: 通过 `W90` 类导入 Wannier90 软件生成的紧束缚哈密顿量, 以便进行后处理及拓扑/量子几何分析。

万尼尔函数 workflow: 使用 `Wannier` 类从 `WFArray` 构造最大局域化万尼尔函数, 可执行投影、解纠缠、评估展宽, 并分析其中心与局域化性质。

可视化: 利用内置工具绘制能带结构、态密度、晶格几何、跃迁图及交互式 3D 模型。

总而言之, `PythTB` 是一个功能集成的计算平台, 它不仅能便捷地构建和分析各类紧束缚模型, 其突出特色在于深度集成了万尼尔函数构造与高阶拓扑/量子几何量计算的能力, 并提供了从数据计算到图形呈现的完整分析链条。

► 2. 从 pythTB 开始

安装后, 激活环境:

命令示例 · shell

```
conda activate pythtb
```

剩余步骤参考官网: <https://pythtb.readthedocs.io/en/latest/install.html>

► 3. New to v2.0

在 2.0.0 版本中, 我们引入了 `Lattice` 类, 用以封装晶格几何的所有信息, 包括晶格矢量、轨道位置和周期性。通过这种模块化设计, 实现了关注点的分离: `Lattice` 类负责处理几何细节, 而 `TBModel` 类则专注于紧束缚模型本身。这一设计显著提升了代码的可读性、可维护性和可复用性。

代码清单 A.31.2

语言: python; 完整代码见附录第 455 页, 共 2 行。

以下代码片段演示了如何使用 `Lattice` 类定义晶格并构建紧束缚模型。以蜂窝晶格 (每个原胞含两个轨道) 为例, 首先给出晶格矢量:

$$\mathbf{a}_1 = a\hat{x}, \quad \mathbf{a}_2 = \frac{a}{2}\hat{x} + \frac{a\sqrt{3}}{2}\hat{y}$$

为简化计算, 取晶格常数 $a=1$ 。轨道位置用约化坐标表示为:

$$\tau_1 = 0, \quad \tau_2 = \frac{1}{3}\mathbf{a}_1 + \frac{1}{3}\mathbf{a}_2$$

此设定对应类似石墨烯的结构。通过 `Lattice` 类封装晶格几何信息, 可实现晶格定义与模型构建的分离, 提升代码的清晰度和可维护性。

短代码 · python

```
lat_vecs = [[1, 0], [1 / 2, np.sqrt(3) / 2]]
orb_vecs = [[1 / 3, 1 / 3], [2 / 3, 2 / 3]]
```

我们将上述信息传入类中以创建一个晶格对象。此外, 我们还需指定哪些方向具有周期性。在本示例中, 两个方向均被设为周期性。

短代码 · python

```
lat = Lattice(orb_vecs=orb_vecs, lat_vecs=lat_vecs, periodic_dirs=[0, 1])
```

我们可以通过打印晶格对象, 查看其生成的报告, 以验证其中封装的晶格属性是否与预期一致。

代码清单 A.31.5

语言: python; 完整代码见附录第 455 页, 共 26 行。

如果我们想获取轨道在笛卡尔坐标系中的位置, 可以使用 `Lattice` 类的 `get_orb_vecs` 方法, 并将其参数 `cartesian` 设置为 `True`。

代码清单 A.31.6

语言: python; 完整代码见附录第 455 页, 共 1 行。

代码清单 A.31.7

语言: python; 完整代码见附录第 455 页, 共 2 行。

该类内部会根据提供的晶格矢量和周期性生成倒格矢。我们可以通过 `Lattice` 类的方法 `get_recip_lat_vecs` 或属性 `recip_lat_vecs` 来访问这些倒格矢。

代码清单 A.31.8

语言: python; 完整代码见附录第 455 页, 共 2 行。

代码清单 A.31.9

语言: python; 完整代码见附录第 456 页, 共 4 行。

让我们验证倒格矢是否满足与实空间晶格矢量的正交性条件。

$$\mathbf{a}_i \cdot \mathbf{b}_j = 2\pi\delta_{ij}$$

短代码 · python

```
overlap_mat = lat.lat_vecs @ lat.recip_lat_vecs.T
print(overlap_mat / (2 * np.pi))
```

代码清单 A.31.11

语言: python; 完整代码见附录第 456 页, 共 2 行。

如果需要, 我们也可以获取实空间和倒空间中的原胞体积, 这些值存储在 Lattice 类的相应属性中。

代码清单 A.31.12

语言: python; 完整代码见附录第 456 页, 共 1 行。

代码清单 A.31.13

语言: python; 完整代码见附录第 456 页, 共 1 行。

完整代码如下:

代码清单 A.31.14

语言: python; 完整代码见附录第 456 页, 共 90 行。

5.6 pythTB 的学习 _mesh 类

日期: 2025-12-09

在线原文: marx2001.github.io

► Tutorials

本教程将演示如何在组合布里渊区 (BZ) 与参数空间中创建自定义网格与环绕路径。

代码清单 A.32.1

语言: python; 完整代码见附录第 458 页, 共 2 行。

Mesh 对象接受以下参数:

- **axis_types**: 字符串列表, 定义每个坐标轴的类型。选项为 “k” (晶格动量, 约化单位) 和 “l” (绝热参数)。
- **axis_names**(可选): 字符串列表, 定义每个坐标轴的名称, 主要用于绝热循环 (示例可见 “三格点 Thouless 泵” 案例)。
- **dim_k**(可选): 整数, 定义 k 空间的维度。当 “k” 轴的数量少于 dim_k 时 (例如在二维布里渊区中沿一维路径采样) 必须提供此参数。

在下面的代码单元中, 我们将在二维 k 空间中创建一个仅包含单个 k 轴的网格。这意味着该网格是二维布里渊区 (BZ) 中的一条路径。我们将使用 `build_custom()` 方法直接指定网格点。这里我们创建了一条从 Γ 点 [0, 0] 经过 X 点 [0.5, 0.5] 到下一个单元中的 Γ 点 [1, 1] 的路径。

代码清单 A.32.2

语言: python; 完整代码见附录第 458 页, 共 5 行。

我们可以通过 `points` 属性访问网格点。请注意, 网格中的每个点都是一个二维向量, 对应于两个 `k` 空间维度。

代码清单 A.32.3

语言: `python`; 完整代码见附录第 458 页, 共 3 行。

结果:

代码清单 A.32.4

语言: `python`; 完整代码见附录第 458 页, 共 12 行。

我们可以通过打印对象来查看网格信息。这会告诉我们:

网格类型 (“`path`” 路径或 “`grid`” 网格)

`k` 空间和参数空间的维度数

网格点的总数

网格数组的形状

`k` 轴 (类型 (`k`)、名称、点数)

参数空间轴 (类型 (`l`)、名称、点数)

哪些轴是循环 (周期性) 的, 哪些不是

网格中存在的环 (若有)

代码清单 A.32.5

语言: `python`; 完整代码见附录第 459 页, 共 1 行。

代码清单 A.32.6

语言: `python`; 完整代码见附录第 459 页, 共 9 行。

代码本身无法自动判断在端点未包含时, 哪些轴应对布里渊区 (BZ) 或参数空间进行 “卷绕”, 因此需要手动指定。这对于正确布洛赫波函数的周期性边界条件非常重要。

可以通过调用 `loop` 方法来声明一个轴对布里渊区进行卷绕。该方法接受以下参数:

`axis_idx`: 要卷绕的轴索引 (从 0 开始计数)。

`component_idx`: 在组合的 (`k`, λ) 向量中对应的分量索引 (从 0 开始计数)。

`winds_bz`: 布尔值, 指示该轴是否通过倒格矢卷绕布里渊区 (`True` 表示是)。

`closed`: 布尔值, 指示该轴是否包含端点 (即是否为闭合的轴) (`False` 表示不包含)。

在该示例中, 设置了:

将网格的第一个 (且唯一一个) 轴 (`axis_idx=0`) 设置为卷绕第一个 `k` 空间维度 (`component_idx=0`), 调用 `loop(axis_idx=0, component_idx=0, winds_bz=True, closed=False)`。

为了也对第二个 `k` 空间维度进行卷绕, 再次调用 `loop` 方法, 但这次设置 `component_idx=1` (即 `axis_idx=0, component_idx=1, winds_bz=True, closed=False`)。

调用完成后, 网格信息将显示两个轴均绕布里渊区卷绕。

注意:

当使用 `k` 点时, 应通过设置 `winds_bz` 参数来明确该轴是否绕布里渊区 (BZ) 倒格矢一周。这确保了布洛赫波函数在周期性边界条件下得到正确处理。

若该轴绕布里渊区一周 (例如跨越整个布里渊区的高对称路径), 应设置为 `winds_bz=True`。

若该轴不环绕布里渊区 (例如布里渊区内部的小环形路径, 如狄拉克点周围的闭合环路), 则应设置为 `winds_bz=False`。

此设置对确保波函数相位和拓扑计算的正确性至关重要。

代码清单 A.32.7

语言: `python`; 完整代码见附录第 459 页, 共 6 行。

代码清单 A.32.8

语言: `python`; 完整代码见附录第 459 页, 共 9 行。

我们将进行与之前相同的操作, 但这次会将布里渊区 (BZ) 的端点包含在内。

注意:

`Mesh` 类具备自动检测功能: 如果 `k` 分量的起点和终点在模 1 下相等 (即该轴是闭合的), 则会自动将该轴识别为环绕布里渊区。因此, 即使在以 Γ 点为中心 (`k` 分量范围在 `[-0.5, 0.5]` 内) 的网格中, 如果包含了端点 (`±0.5`), 该类仍可检测到该轴环绕布里渊区。

代码清单 A.32.9

语言: `python`; 完整代码见附录第 460 页, 共 7 行。

代码清单 A.32.10

语言: `python`; 完整代码见附录第 460 页, 共 9 行。

5.7 pythTB 的学习 `_tb_model` to `TBModel` in v2.0

日期: 2025-12-09

在线原文: marx2001.github.io

► Tutorials

在 `TBModel` 类的 v2.0 版本中, 模块名称已发生重要变更。v1.8 版本中的 `tb_model`、`wf_array` 和 `w90` 在 v2.0 版本中已分别更新为 `TBModel`、`WFArray` 和 `W90`。在使用时, 请注意在代码的导入行中相应修改模块名称。

代码清单 A.33.1

语言: `python`; 完整代码见附录第 460 页, 共 2 行。

► 构建一个 TB 模型

在 v2.0 版本中, 模型的构建方式发生了变化。我们不再将晶格矢量和轨道位置作为单独的参数传递, 而是传递一个包含这些信息的 `Lattice` 对象。

我们将以蜂窝晶格为例进行说明。在 v1.8 版本中, 我们会这样操作:

代码清单 A.33.2

语言: python; 完整代码见附录第 460 页, 共 6 行。

在 v2.0 版本中, 我们首先创建一个 Lattice 对象, 然后将其传递给 TBModel 构造器。

需要注意的几个行为变化如下:

为提高清晰度, per 参数已更名为 periodic_dirs。若未指定, v1.8 版本的默认行为是假设所有方向均为周期性的; 而在 v2.0 中, 默认设置为无周期性方向。

整数参数 nspin 已被替换为布尔型参数 spinful, 用于指示模型是否具有自旋自由度。

dim_r 和 dim_k 不再是必需参数, 因为它们可以从 lat_vecs 和 periodic_dirs 中推断得出。

其余大部分功能基本保持不变, 仅被封装在 Lattice 对象中。

提示:

将所有晶格方向指定为周期性的便捷方法是使用 periodic_dirs 参数使用 ..., 例如 periodic_dirs=...。此外, 也可以通过传递 periodic_dirs="all" 来实现相同效果。

以下是 v2.0 中对应的代码:

短代码 · python

```
lat_vecs = [[1, 0], [1 / 2, np.sqrt(3) / 2]]
orb_vecs = [[1 / 3, 1 / 3], [2 / 3, 2 / 3]]
lat = Lattice(lat_vecs=lat_vecs, orb_vecs=orb_vecs, periodic_dirs=[0, 1])
model = TBModel(lattice=lat, spinful=False)
```

代码清单 A.33.4

语言: python; 完整代码见附录第 460 页, 共 14 行。

► 模型库

一系列用于原型紧束缚模型的 TBModel 生成器已被收录在 pythtb.models 中。这些模型包括:

棋盘模型

霍尔丹模型

凯恩-梅莱模型

石墨烯模型

举个例子, 我们可以从模型库中导入与上文相同的霍尔丹紧束缚模型:

代码清单 A.33.5

语言: python; 完整代码见附录第 461 页, 共 3 行。

► 报告模型信息

使用下面代码查看模型信息:

代码清单 A.33.6

语言: `python`; 完整代码见附录第 461 页, 共 7 行。

代码清单 A.33.7

语言: `python`; 完整代码见附录第 461 页, 共 1 行。

结果:

代码清单 A.33.8

语言: `python`; 完整代码见附录第 461 页, 共 52 行。

► 紧束缚模型的可视化

紧束缚模型 (TB) 的轨道位置与跃迁键的可视化功能已更新。

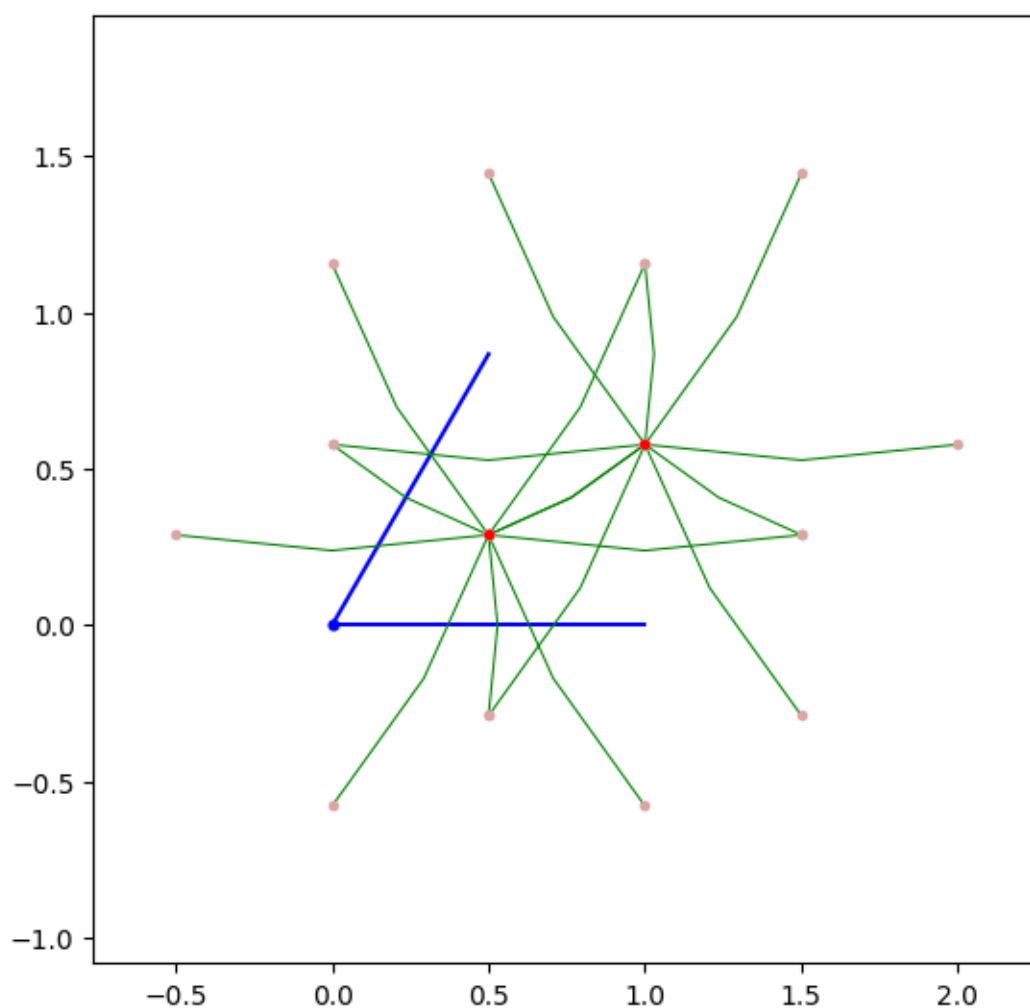
与之前一样, 我们调用 `my_model.visualize` 来查看紧束缚晶格。

在 v1.8 版本中, 应使用如下方式:

代码清单 A.33.9

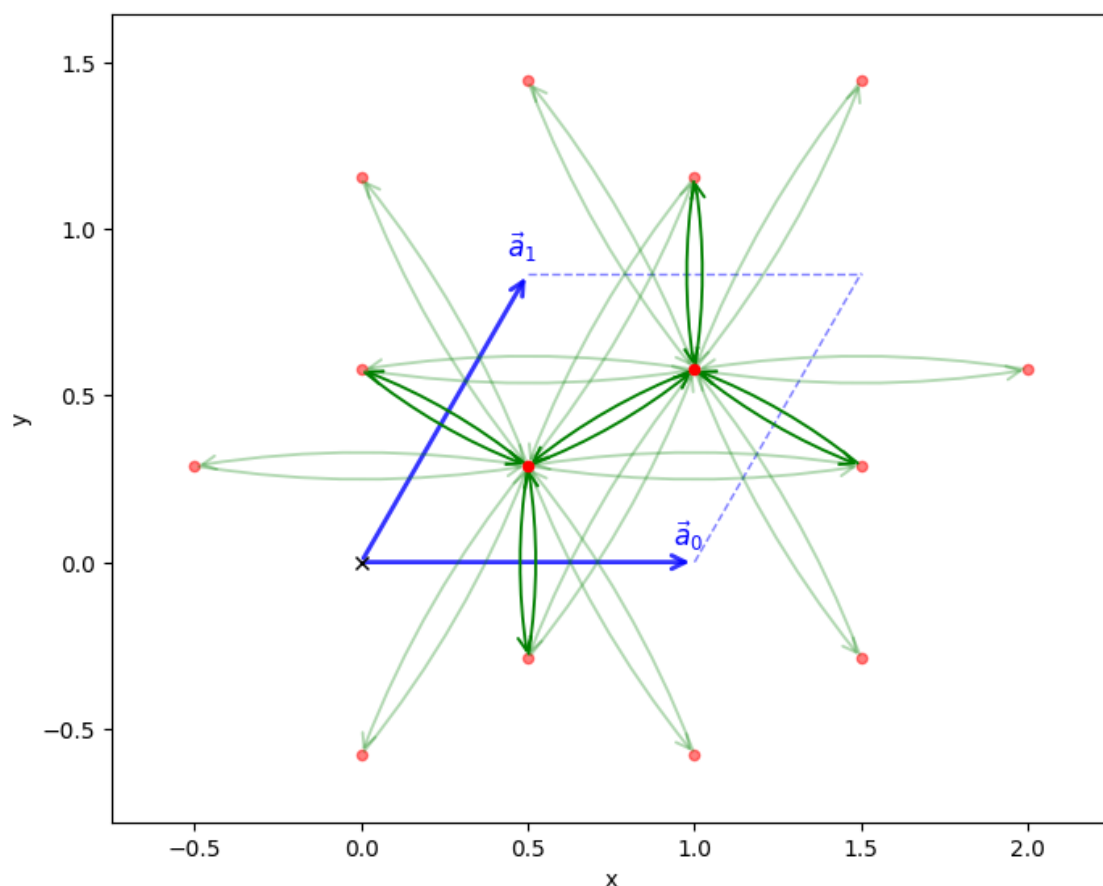
语言: `python`; 完整代码见附录第 462 页, 共 2 行。

结果图为:



区别在于, 在绘制二维图形时, 我们不再需要指定晶格方向。对于三维模型, 仍可通过 `proj_plane` 参数指定两个对应于 a_i 和 a_j 晶格矢量的整数, 以将三维结构投影到该平面上。默认行为是: 二维模型直接绘制二维图形, 三维模型默认投影到 a_1 - a_2 平面上。..note:: 对于三维模型, 如果安装了 `plotly`, 现在可以使用 `visualize3d()` 绘制三维结构。

在 v2.0 版本中, 输出效果如下所示:



请注意, 图中键的粗细并非完全一致。其透明度 (alpha 值) 会根据跃迁强度进行比例缩放。此外, 我们现在还可以通过设置 `annotate_onsite_en` 参数为 `True` 来标注每个格点的位能值。

► 获得哈密顿量矩阵

在 `pythtb` 的旧版本中, 用户无法直接获取哈密顿矩阵。相反, 需要通过调用 `my_model.solve_all(kpts)` 或 `my_model.solve_one(kpt)` 函数来获取给定 `k` 点的本征值和本征向量。而在 `v2.0` 版本中, 新增了 `my_model.hamiltonian(kpts)` 方法, 允许用户直接获取一系列 `k` 点上的哈密顿矩阵, 该方法会返回一个 `NumPy` 数组。

代码清单 A.33.10

语言: `python`; 完整代码见附录第 462 页, 共 3 行。

短代码 · `python`

```
H_k = my_model.hamiltonian(k_pts)

print("Hamiltonian shape:", H_k.shape)
print("Hamiltonian at first k-point:\n", H_k[0])
print("Hamiltonian at second k-point:\n", H_k[1])
```

代码清单 A.33.12

语言: `python`; 完整代码见附录第 462 页, 共 7 行。

► 能带绘制

`TBModel` 的一个新功能是增加了一个便捷函数, 可快速绘制能带结构。我们不再需要显式创建 `k` 路径和 `matplotlib` 图形, 只需调用 `plot_bands` 并传递以约化单位表示的高对称点即可。这将返回 `matplotlib` 图形和坐标轴对象, 以供进一步自定义。

注意

我们还可以传递 `k_node_labels` 参数来指定高对称点的标签。

代码清单 A.33.13

语言: `python`; 完整代码见附录第 463 页, 共 17 行。

短代码 · `python`

```
k_nodes = [[0, 0], [2 / 3, 1 / 3], [0.5, 0.5], [1 / 3, 2 / 3], [0, 0], [0.5, 0.5]]
k_label = (r"$\Gamma$", r"$K$", r"$M$", r"$K^\prime$", r"$\Gamma$", r"$M$")

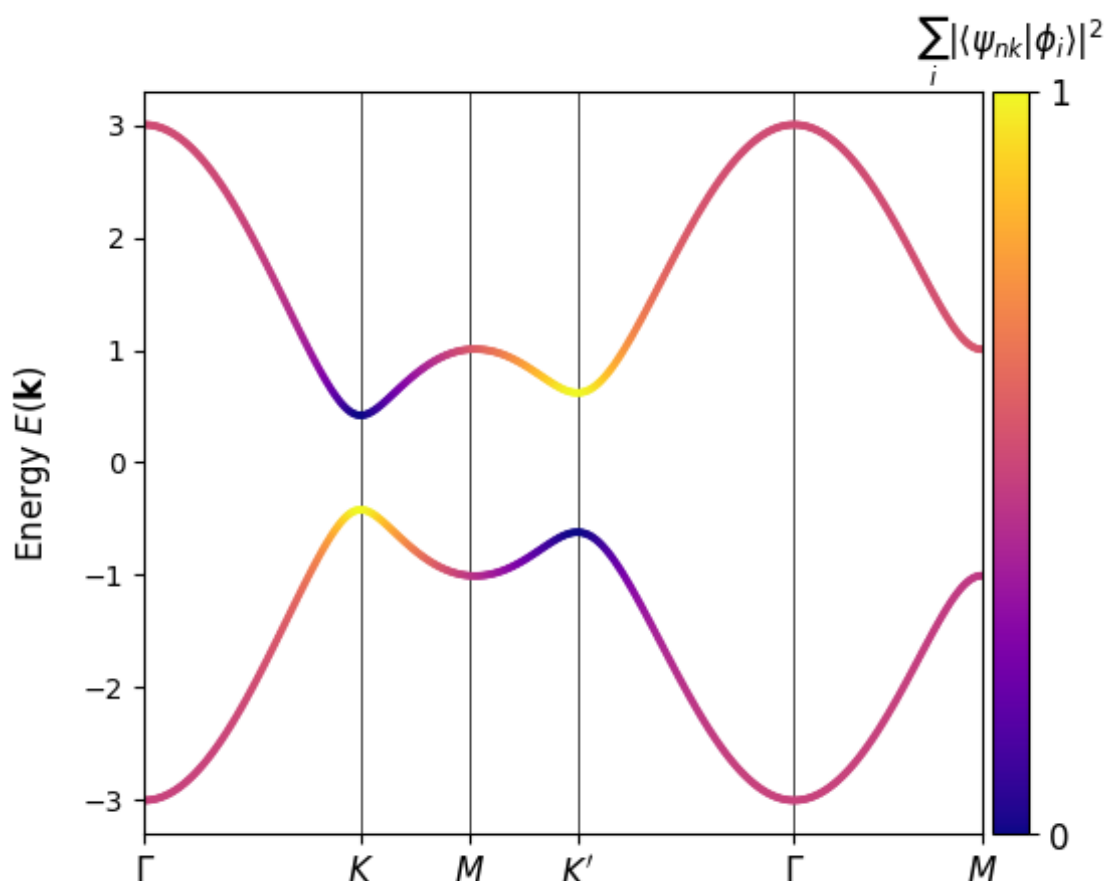
fig, ax = my_model.plot_bands(k_nodes=k_nodes, k_node_labels=k_label)
```

一个可选参数允许用户可视化能带的轨道特征。为此, 我们向 `proj_orb_idx` 参数提供一个列表。该列表定义了用于本征态投影的轨道索引。系统将显示一个颜色条, 展示本征态在这些轨道集合上的权重。

在本示例中, `Haldane` 模型在 `K` 点和 `K'` 点处展示了能带反转现象, 这正是拓扑相变的典型特征。

代码清单 A.33.15

语言: `python`; 完整代码见附录第 463 页, 共 3 行。



► 向后兼容性

为保持与为 PythTB v1.8 及更早版本编写的脚本的向后兼容性, 旧版 `tb_model` 仍可作为 `TBModel` 的别名使用。其函数与方法将采用 v2.0 版本的实现, 但初始化 `tb_model` 类的旧方式依然有效。

代码清单 A.33.16

语言: python; 完整代码见附录第 463 页, 共 2 行。

从 V1.8.0 的 Haldane model initialization:

代码清单 A.33.17

语言: python; 完整代码见附录第 463 页, 共 31 行。

代码清单 A.33.18

语言: python; 完整代码见附录第 464 页, 共 52 行。

代码清单 A.33.19

语言: python; 完整代码见附录第 465 页, 共 4 行。

5.8 pythTB 的学习 _wf_array to in v2.0WfArray

日期: 2025-12-09

在线原文: marx2001.github.io

► Tutorials

► 初始化与填充 WfArray

在 PythTB 的旧版本中, wf_array 类与 tb_model 类紧密耦合。初始化 wf_array 时, 它会从关联的 tb_model 中读取晶格信息。而在 v2.0 版本中, 与 TBModel 的做法类似, 我们将显式地将一个 Lattice 对象传递给 WfArray 的构造函数。除了提高灵活性, 这也使我们能够创建不绑定于任何特定 TBModel 的 WfArray 对象。例如, 用户可能希望存储源自绝热循环中多个 TBModel 的状态, 而这些状态没有一个单一的 TBModel 可以完全描述。

Compare to versions 1.x:

代码清单 A.34.1

语言: python; 完整代码见附录第 465 页, 共 5 行。

在 solve_on_grid 方法中, wf_array 会先生成 k 点网格, 并用来自 model 的波函数数据填充自身。之后, 这个 k 点网格会被丢弃, wf_array 仅保留波函数数据本身。这导致关于 k 点结构的信息丢失, 如果后续分析 (如绘图) 需要这些 k 点信息, 用户需要自己手动追踪和保存对应的 k 点网格。

1. 创建一个描述系统晶格结构的对象。

短代码 · python

```
lattice = Lattice(
    lat_vecs=[[1.0]], orb_vecs=[[0.0], [1 / 3], [2 / 3]], periodic_dirs=[0]
)
```

1. 创建一个用于描述 k 点和参数网格的对象。

短代码 · python

```
mesh = Mesh(["k"])
```

1. 用所需的 k 点和参数值填充 Mesh。

代码清单 A.34.4

语言: python; 完整代码见附录第 465 页, 共 1 行。

1. 将这些对象传递给构造函数

短代码 · python

```
wfa = WfArray(lattice=lattice, mesh=mesh)
```

现在, 如果我们想用给定的 `TBModel` 的能量本征态来填充 `WFArrary`, 可以通过调用 `WFArrary.solve_model()` 函数来实现。

代码清单 A.34.6

语言: python; 完整代码见附录第 465 页, 共 18 行。

短代码 · python

```
model_fixed = model.with_parameters(lambda=0.25)
wfa.solve_model(model_fixed)
```

我们可以通过索引来访问存储在 `WFArrary` 中的单个波函数, 这与先前版本中的操作方式相似。例如, `wfa[k_index]` 将返回指定网格索引处的波函数。

代码清单 A.34.8

语言: python; 完整代码见附录第 466 页, 共 1 行。

代码清单 A.34.9

语言: python; 完整代码见附录第 466 页, 共 6 行。

与先前版本一样, 在贝里相位计算及其他运算中, 布里渊区边界会自动应用内部的周期性边界条件。

► 绝热循环中的 WFArrary

我们将复用上文使用过的同一个晶格, 但现在要创建一个包含绝热参数轴的 `Mesh`。这个参数在 0 到 1 的范围内变化, 共分 11 个步骤。这个参数可以代表例如晶格的畸变或外部场的变化。

Mesh lambda 01

可回顾 `Mesh` 教程中的内容: 我们需要根据在函数中变化的参数名称来命名这个坐标轴 (如未阅读, 可参阅上文的函数说明)。

λ TBModel1

短代码 · python

```
mesh = Mesh(["k", "l"], axis_names=["kx", "lambda"])
```

现在我们将构建网格, 指定绝热参数的范围从 0 到 1, 共分 11 个步骤。

代码清单 A.34.11

语言: python; 完整代码见附录第 466 页, 共 7 行。

如果我们希望将 λ 轴设置为一个绝热循环, 我们需要为对应的轴索引设置 `loop` 参数。组合空间 (k, λ) 是二维的, 且维度排列在 k 维度 (这里只有 1 个 k 维度) 之后, 因此组件索引是 1 (按 Python 计数方式)。这表明, 遍历轴 1 会使网格向量的第二个分量 (Python 计数为 1) 形成环路, 从而使得环路末端的哈密顿量与起始端的哈密顿量相连。

由于 $\lambda=1$ 的终点与 $\lambda=0$ 的起点是等效的, 我们也将该轴标记为闭合。这可以通过将 `closed` 参数设置为 `True` 来实现。

代码清单 A.34.12

语言: python; 完整代码见附录第 466 页, 共 4 行。

代码清单 A.34.13

语言: python; 完整代码见附录第 467 页, 共 10 行。

初始化 WFArrary

短代码 · python

```
wfa = WFArrary(lattice, mesh)
```

最后, 我们调用函数, 在组合网格的每个点 $(\mathbf{k}, \lambda)$ 上, 用 TBModel 的本征态填充 WFArrary。

代码清单 A.34.15

语言: python; 完整代码见附录第 467 页, 共 1 行。

我们可以通过比较绝热循环开始和结束时的波函数, 来验证该轴是否被正确设置为一个环路

代码清单 A.34.16

语言: python; 完整代码见附录第 467 页, 共 1 行。

代码清单 A.34.17

语言: python; 完整代码见附录第 467 页, 共 1 行。

5.9 vasp2kp 的学习 _1

日期: 2025-12-10

在线原文: marx2001.github.io

► Tutorials

► 初始化与填充 WFArray

一、基础模块：矩阵类型控制

参数	类型 / 取值	含义与作用	
<code>vmat</code>	整数型 (10 / 11 / 12)	指定要计算哪类矩阵： • <code>10</code> = 计算自旋算符矩阵 ($\sigma_x, \sigma_y, \sigma_z$) → Pauli spin matrices; • <code>11</code> = 计算广义动量算符矩阵 (p_x, p_y, p_z) → 动量矩阵, 用于 k-p 模型的动能项; • <code>12</code> = 计算晶体对称操作或时间反演的共表示矩阵 → 例如 C_4z、M_x、T 等。	
<code>vmat_name</code>	字符串	指定矩阵名称 (在 <code>vmat = 12</code> 时表示具体对称操作名称)： • <code>'sig'</code> → Pauli 自旋矩阵; • <code>'pi'</code> → 广义动量矩阵; • <code>'C4z'</code>、<code>'Mx'</code>、<code>'T'</code> 等 → 对称算符矩阵。	
<code>vmat_k</code>	整数	计算 k 点的选择, 一般 <code>vmat_k = 1</code> 表示 Γ 点 ($k = 0$)。除非特别需要, 不必修改。	
<code>bstart</code> , <code>bend</code>	整数	指定计算矩阵的能带范围： • <code>bstart</code> 为起始带序号; • <code>bend</code> 为结束带序号。 例如 <code>bstart=47, bend=50</code> 表示取 47–50 带的布洛赫基底来计算对称矩阵。	
<code>print_only_diagonal</code>	布尔型	是否只输出对角元。 • <code>.true.</code> → 只输出对角项; • <code>.false.</code> → 输出完整矩阵。建议设为 <code>.false.</code>。	

二、SOC (自旋-轨道) 控制模块

参数	类型 / 取值	含义与作用	
<code>cfactor</code>	实数	哈密顿量缩放因子, 通常设为 1.0 即可。	
<code>socfactor</code>	实数	自旋-轨道耦合强度的缩放系数; 常设 1.0。	
<code>nosoc_inH</code>	布尔型	是否在哈密顿量中去除 SOC 项： • <code>.true.</code> → 不含 SOC; • <code>.false.</code> → 包含 SOC。	

三、对称操作定义模块（仅 vmat = 12 时使用）

这些参数定义一个对称算符 $\{R|\tau\}$ 的全部性质。

参数	类型 / 取值	含义与物理背景
rot_n(:)	向量 (3 个分量)	对称操作旋转轴的方向 (n_x, n_y, n_z)。例如： • $C_4z \rightarrow 0\ 0\ 1$; • $Mx \rightarrow 1\ 0\ 0$。
rot_alpha	实数 (度)	旋转角度 α (单位: $^\circ$)。 • $C_4z \rightarrow 90^\circ$; • $C_2 \rightarrow 180^\circ$; • Mx 或 My 镜像相当于 180° 旋转 (且 $rot_det = -1$)。
rot_det	整数 (± 1)	$O(3)$ 旋转矩阵的行列式： • $+1 \rightarrow$ 正规旋转 (不含反演)； • $-1 \rightarrow$ 含反演或镜像。
rot_tau(:)	向量 (3 个分量)	对称操作的平移分量 (τ_x, τ_y, τ_z)。对纯点群操作设 $0\ 0\ 0$ 。
rot_spin2pi	布尔型	若为 <code>.true.</code> ，则自旋部分在 2π 旋转下改变符号 (用于检查双群行为)。常设 <code>.false.</code> 。
time_rev	布尔型	是否包含时间反演 $\mathcal{T}$ ： • <code>.true.</code> $\rightarrow$ 该操作作为反单群 (含时间反演)，如 T 或 C_2T; • <code>.false.</code> $\rightarrow$ 纯空间群操作。

四、举例：三个典型算符的设置

操作	示例设置	解释	
C₄z (绕 z 轴旋转 90°)	<pre>rot_n=0 0 1 rot_alpha=90 rot_det=1 rot_tau=0 0 0 time_rev=.false.</pre>	四重旋转操作	
M_x (x 轴镜像)	<pre>rot_n=1 0 0 rot_alpha=180 rot_det=-1 rot_tau=0 0 0 time_rev=.false.</pre>	x 轴镜面操作	
T (时间反演)	<pre>rot_n=0 0 0 rot_alpha=0 rot_det=1 rot_tau=0 0 0 time_rev=.true.</pre>	纯时间反演操作	

每次只能计算一个操作 → 必须运行 `vasp2mat` 三次，分别生成 `MAT_C4z`、`MAT_Mx` 和 `MAT_T`。

需要 6 个初始文件作为输入文件

代码清单 A.35.1

语言：python；完整代码见附录第 467 页，共 1 行。

运行命令开始计算：

短代码 · python

```
/public/home/cssong/install_package/cssong/pack/VASP2KP/vasp2mat.5.4.4/bin/vasp2mat
vasp2mat
```

5.10 ir2TB 的学习

日期：2025-12-13

在线原文：marx2001.github.io

► Tutorials

► 简介:Why is IRVSP&IR2TB

IRVSP (**I**rreducible **R**epresentation of **V**ASP) 是一款专门用于基于第一性原理计算结果的能带对称性分析的软件。它由中国科学院物理所的高鸿钧院士团队开发，广泛用于拓扑材料、磁性材料、半导体能带结构的对称性判别与拓扑不变量计算。

一、IRVSP 的主要用途

IRVSP 的核心功能是根据 **VASP** 计算结果 (通常是 `WAVECAR`、`POSCAR`、`OUTCAR`、`KPOINTS` 等文件) 自动识别能带在高对称点处所属的不可约表示 (**I**rreducible **R**epresentation, **Irrep**)。主要用途包括:

- 1. 能带对称性标注 (**S**ymmetry **L**abeling)
 - IRVSP 读取波函数 (`WAVECAR`) 并分析其在空间群高对称点的对称性;
 - 自动输出每个能带在高对称点处对应的不可约表示符号, 如 Γ_6^+ 、 Γ_7^- 等;
 - 这对于判断能带简并性、能带交叉点类型 (如 Dirac/Weyl 节点) 非常关键。
- 2. 拓扑材料判定 (**T**opological **M**aterial **D**iagnosis)
 - 利用能带的不可约表示组合, 判断体系的拓扑类型;
 - 可与 **Topological Quantum Chemistry (TQC)** 框架匹配;
 - 输出与 Bilbao Crystallographic Server 的 `irrep`、`topological quantum chemistry` 数据兼容的结果;
 - 从而判断材料是平庸绝缘体、拓扑绝缘体还是半金属。
- 3. 对称性保护的能带简并分析
 - 确定高对称点/高对称线处能带交叉是否受到群对称性保护;
 - 帮助识别 **Dirac 点**、**Weyl 点** 或 **节点线 (nodal line)** 的起源。

二、为什么要用 IRVSP

使用 IRVSP 的理由主要有以下几点:

需求	IRVSP 解决的问题	
1. 能带的对称性来源不清晰	DFT 输出的能带图只能看到能量, 不知道对应哪种对称表示。IRVSP 能标出每条能带的对称性。	
2. 拓扑性质判定困难	仅凭能带结构难以判断体系是否为拓扑相。IRVSP 的输出可以直接输入至 <code>CheckTopologicalMat</code> 、 <code>Qptoolkit</code> 等框架进行拓扑分类。	
3. 紧束缚模型对称性约束	构造 TB 模型时, 需要已知每个能带在高对称点的不可约表示, 以保证模型对称性正确。	
4. 自动化、高精度分析需求	IRVSP 自动读取 <code>WAVECAR</code> 和空间群信息, 无需手动计算群表示矩阵或使用复杂群论表格。	

四、总结

IRVSP 的本质作用:

将 VASP 的量子力学波函数结果与群论相结合, 为材料的能带对称性标定、简并判定和拓扑分类提供自动化、定量化支持。

简而言之:

没有 IRVSP, 你只能看到能带“形状”;
有了 IRVSP, 你能看到能带“对称性来源”和“拓扑身份”。

► IRVSP 的使用

用于判断非常规材料的电子性质 (电子的电荷中心不在原子上), 用拓扑量子化学和能带表示的理论, 通过高对称 k 点的表示可以判断出来。(阻挫原子极限)

先确定能带的基本情况

1. 先找到 POSCAR
2. 用 phonopy 跑一遍, 得到标准化 poscar。
3. 先算自洽, 再算高对称 k 点的表示
4. 非自洽, 算能带 (kpoints 需要特殊准备), chgcar 不变, 输出 wavecar, 得到高对称 K 点的波函数

代码清单 A.36.1

语言: shell; 完整代码见附录第 468 页, 共 1 行。

看对应的能带表示

代码清单 A.36.2

语言: shell; 完整代码见附录第 468 页, 共 1 行。

看 tqc.data、EBR 等等。

见吴泉生老师的官网和 B 站视频均有资源, 此为简单记录一些命令, 方便复制命令。

5.11 irvsp 的学习

日期: 2025-12-13

在线原文: marx2001.github.io

► Tutorials

► 简介:Why is IRVSP&IR2TB

IRVSP (**I**rreducible **R**epresentation of **V**ASP) 是一款专门用于基于第一性原理计算结果的能带对称性分析的软件。它由中国科学院物理所的高鸿钧院士团队开发, 广泛用于拓扑材料、磁性材料、半导体能带结构的对称性判别与拓扑不变量计算。

一、IRVSP 的主要用途

IRVSP 的核心功能是根据 VASP 计算结果 (通常是 `WAVECAR`、`POSCAR`、`OUTCAR`、`KPOINTS` 等文件) 自动识别能带在高对称点处所属的不可约表示 (Irreducible Representation, Irrep)。主要用途包括:

1. 能带对称性标注 (Symmetry Labeling)

- IRVSP 读取波函数 (`WAVECAR`) 并分析其在空间群高对称点的对称性;
- 自动输出每个能带在高对称点处对应的不可约表示符号, 如 Γ_6^+ 、 Γ_7^- 等;
- 这对于判断能带简并性、能带交叉点类型 (如 Dirac/Weyl 节点) 非常关键。

2. 拓扑材料判定 (Topological Material Diagnosis)

- 利用能带的不可约表示组合, 判断体系的拓扑类型;
- 可与 **Topological Quantum Chemistry (TQC)** 框架匹配;
- 输出与 Bilbao Crystallographic Server 的 `irrep`、`topological quantum chemistry` 数据兼容的结果;
- 从而判断材料是平庸绝缘体、拓扑绝缘体还是半金属。

3. 对称性保护的能带简并分析

- 确定高对称点/高对称线处能带交叉是否受到群对称性保护;
- 帮助识别 **Dirac 点**、**Weyl 点** 或 **节点线 (nodal line)** 的起源。

二、为什么要用 IRVSP

使用 IRVSP 的理由主要有以下几点:

需求	IRVSP 解决的问题	📄
1. 能带的对称性来源不清晰	DFT 输出的能带图只能看到能量, 不知道对应哪种对称表示。IRVSP 能标出每条能带的对称性。	
2. 拓扑性质判定困难	仅凭能带结构难以判断体系是否为拓扑相。IRVSP 的输出可以直接输入至 <code>CheckTopologicalMat</code> 、 <code>Qptoolkit</code> 等框架进行拓扑分类。	
3. 紧束缚模型对称性约束	构造 TB 模型时, 需要已知每个能带在高对称点的不可约表示, 以保证模型对称性正确。	
4. 自动化、高精度分析需求	IRVSP 自动读取 <code>WAVECAR</code> 和空间群信息, 无需手动计算群表示矩阵或使用复杂群论表格。	

四、总结

IRVSP 的本质作用:

将 VASP 的量子力学波函数结果与群论相结合, 为材料的能带对称性标定、简并判定和拓扑分类提供自动化、量化支持。

简而言之:

没有 IRVSP, 你只能看到能带“形状”;
有了 IRVSP, 你能看到能带“对称性来源”和“拓扑身份”。

► IRVSP 的使用

用于判断非常规材料的电子性质 (电子的电荷中心不在原子上), 用拓扑量子化学和能带表示的理论, 通过高对称 k 点的表示可以判断出来。(阻挫原子极限)

先确定能带的基本情况

1. 先找到 POSCAR
2. 用 phonopy 跑一遍, 得到标准化 poscar。
3. 先算自洽, 再算高对称 k 点的表示
4. 非自洽, 算能带 (kpoints 需要特殊准备), chgcar 不变, 输出 wavecar, 得到高对称 K 点的波函数

代码清单 A.37.1

语言: shell; 完整代码见附录第 468 页, 共 1 行。

看对应的能带表示

代码清单 A.37.2

语言: shell; 完整代码见附录第 468 页, 共 1 行。

看 tqc.data、EBR 等等。

13 和 13 是能带序号

5.12 (原创脚本) 角态验证 1-配置 pybinding 环境-Ubuntu 系统配置

日期: 2025-12-27

在线原文: marx2001.github.io

► 步骤

windows 环境下, 需要下载 cmake, 和 Visual Studio Build Tools:

代码清单 A.38.1

语言: shell; 完整代码见附录第 468 页, 共 2 行。

勾选:

代码清单 A.38.2

语言: shell; 完整代码见附录第 468 页, 共 7 行。

创建环境

命令示例 · shell

```
conda create -n pybinding_env python=3.9 -y
conda activate pybinding_env
```

安装 miniconda 后, 运行

命令示例 · shell

```
conda install numpy scipy matplotlib
```

然后

命令示例 · shell

```
pip install pybinding
```

安装了 Cmake 和 visual Studio installer 仍然不能成功安装 pybinding, 尝试使用 Linux 环境, 安装 Ubuntu

► 安装 Ubuntu

使用 DiskGenius V4.6.4.2 x64 专业版, 选择有空余空间的磁盘, 拖动棕色进度条, 留出合适的空间。

使用 Rufus 4.7.2231, 写入 Ubuntu 镜像文件到安装 u 盘中, U 盘要求 16G 以上。

12.28 用 ubuntu 系统成功安装了 pybinding, 用 conda 安装, 先创建环境, 然后用 conda 安装前置库, 然后再安装 pybinding。

在 ubuntu 中也不能直接用 pip 安装 pybinding, 只能用 conda 安装 0.9.5 稳定版的 pybinding。

安装完 pybinding 之后, 由于是 ipynb 格式, 需要 jupyter notebook 使用的 kernel 与安装的 pybinding 环境一致, 这样代码才能在该环境中找到所需要的包并运行。

先激活环境, 然后安装 ipynb, 运行命令安装 ipynb 并添加到 jupyter 可用的 knernel 列表中:

命令示例 · shell

```
conda activate pybinding_env
conda install -n pybinding_env ipynb --update-deps --force-reinstall
python -m ipynb install --user --name pybinding_env --display-name "Python
→ (pybinding_env)"
```

这个命令会将 pybinding_env 环境注册为一个 jupyter kernel, 名称为 Python(pybinding_env), 可根据实况修改名称。

重新启动 jupyter notebook, 已经打开的话需要重新关闭, 在创建或打开 notebook 时, 在 kernel 菜单中选择刚刚添加的 python(pybinding_env) kernel, 这样就可以分模块运行代码。

上述是用 conda 安装 0.9.5 稳定版的 pybinding, 目前已经有包名为 pybinding-dev 的预发行版本, 是 1.0.6 版本的 pybinding, 安装流程如下:

创建 Python3.10 版本以上的环境

代码清单 A.38.7

语言: shell; 完整代码见附录第 469 页, 共 7 行。

如果是新配置的 vscode, 可能没有安装 python 扩展, 根据右下角 vscode 的提示, 安装扩展即可。

5.13 ubuntu 中使用 pybinding

日期: 2025-12-29

在线原文: marx2001.github.io

► 步骤

1. 密码 mrx123, 笔记本的 ubuntu 系统密码
2. 使用下方命令进入 pybinding 环境中运行代码

命令示例 · shell

```
conda activate pybinding
```

3. 使用 vscode, 选择同样的环境名称运行程序即可。
4. 后续会安装 pythTB 计算陈数、边缘态等性质, 后续会写文章同步研究进展

科研写作、建模与网站工具

本专题收录 7 篇文章。

6.1 网站维护与操作手册

日期：2025-09-22

在线原文：marx2001.github.io

此笔记作为维护和更新博客的操作手册：

更新 Log

1. 复制 `_posts` 文件夹中的 `md` 文件

2. 修改复制后的 `md` 文件中的开头 `front matter` 部分，可选择修改副标题和标题；`categories` 根据 `Log` 的分类进行修改；`permalink` 中根据需求进行修改，例如分类不同，就修改“分类 `_posts/日期-log`”。如果是科研笔记，那么可以根据心情和内容自行修改内容

PS：写目录名，要写全名，比如根目录下的 `sci-note.md` 文件，目录名应该写 `sci-note`，否则无法在对应页面上形成文章链接

3. 导入图片，需要按照相应格式命名——“`bg-xxx`”的形式，“`xxx`”内可以包含数字，并使用下方代码进行排版，这个代码是两列瀑布式排列图片，应该可以适应大部分情况

4. 正常提交上传到 `github` 即可

20250923:

1. 修改全局页宽，`ctrl+shift+F` 召唤全局搜索，搜索关键词 `container`，找到最下方的 `_variable.scss` 文件，修改 `-xl` 部分即可
2. 修改导航栏字体大小，找到“`.navbar-nav`”代码，修改 `font-size` 即可。
3. 修改段落宽度，通过增加栅格列宽让字符数目增大，这样就可以保留原来的一切设置同时增大段落占据页面的比例：打开 `post.html`，找到代码后增大两个数字即可。

代码清单 A.40.1

语言：`plaintext`；完整代码见附录第 469 页，共 1 行。

4. 对代码产生高亮，高亮分为两种，一种是伴随网页诞生而直接渲染的，一种是浏览器客户端渲染。此模板不能用第一种渲染，所以只讲第二种渲染。首先增加 `head.html` 中的代码（查阅对应文件）

其次要在 `md` 文件中使用““作为开头和结尾，并指定语言类别。

1. 对于代码页面 UI 增强效果，则需要在 `head.html` 中，直接添加代码（直接查看本网站的文件即可）
还要在 `main.scss` 中添加代码（直接查看此网站代码）

1. 实现分页功能:

主要文件——assets/vendor/startbootstrap-clean-blot/pagination.js, 新建这个文件, 填入代码

主要文件——子页 index.html 文件, 根目录新建子页名称为名的文件夹, 并在其目录下新建 index.html

主要文件——assets/main.scss

主要文件——根目录下, 以子页名称命名的 md 文件

主要文件——JSON 自动收集文章数据文件

下面讲解分页具体原理以及步骤。

此 jekyll 网站模板采用 jekyll-paginate 的 v1 版本, 可以通过升级插件到 jekyll-paginate-v2 版本来实现任意界面的分页功能, 但是由于修改文件过多且极其复杂, 对于新手极难实现, 因此我采用第二种方法——JavaScripts 前端实现。

顾名思义, 在浏览器客户端实现分页功能, 并在打开网页过程中就已下载好所有文章数据, 浏览起来方便快捷。经过 412 次网页部署与构建, 终于在 20250925 下午 16:50 实现该分页功能, 历时 23 小时+, 为了便于后面回忆与复现, 特此记录全部步骤。

步骤:

(1) 将博客数据转换为 JavaScript 格式, 需要在创建 json 文件 (具体我不记得了, 如果再次使用, 可以问一下 AI)

(2) 在 assets/vendor/startbootstrap-clean-blog/js 中创建 pagination.js 文件

(3) 写入代码如下:

代码清单 A.40.2

语言: text; 完整代码见附录第 469 页, 共 161 行。

(4) 保存并退出, 修改 page.html, 删掉静态文章列表代码, 修改完成后的完整代码如下:

代码清单 A.40.3

语言: text; 完整代码见附录第 471 页, 共 72 行。

(5) 保存并退出, 然后在根目录新建子页名称命名的文件夹, 里面创建好 index.html 文件, 其示例代码如下 (其它子页代码仅需修改子页名称):

代码清单 A.40.4

语言: text; 完整代码见附录第 473 页, 共 41 行。

(6) 保存并退出后, 找到 assets/main.scss 文件, 复制下方完整代码:

代码清单 A.40.5

语言: text; 完整代码见附录第 473 页, 共 226 行。

(7) 找到根目录下”子页名称.md”文件, 将 front matter 后的代码改为:

代码清单 A.40.6

语言: text; 完整代码见附录第 477 页, 共 4 行。

至此, 分页功能实现。

6.2 (原创自研) 网站维护记录 1

日期: 2026-03-25

在线原文: marx2001.github.io

► Jekyll 博客导航栏修改总结

► 一、目的

本次修改的目标是解决 Jekyll 博客导航栏的几个问题:

1. 导航项过多时出现换行。
2. 导航栏改宽后出现左右贴边、不居中的情况。
3. Home 与左侧品牌名 Mr.X's Blog 之间距离过大。
4. 希望在最终效果上, 让导航菜单整体相对品牌名向右平移一段距离, 使视觉更协调。

► 二、涉及的文件名与目录位置

本次对话中涉及并确认过的关键文件如下。

► 1. 导航栏结构文件

文件名: navbar.html

目录位置:

代码清单 A.41.1

语言: text; 完整代码见附录第 477 页, 共 1 行。

作用: 控制导航栏的 HTML 结构, 包括:

- 品牌名 Mr.X's Blog
- 各个菜单项
- 折叠按钮
- 外层容器与内部布局层级

► 2. 页面头部文件

文件名: head.html

目录位置:

代码清单 A.41.2

语言: text; 完整代码见附录第 477 页, 共 1 行。

作用: 决定页面实际加载哪些 CSS 文件。

本次确认结果: 该文件中实际加载的是:

代码清单 A.41.3

语言: html; 完整代码见附录第 477 页, 共 1 行。

这说明页面最终使用的是:

短代码 · text

```
/assets/main.css
```

因此, 真正生效的样式入口是 `assets/main.css`, 而不是最初修改的 `vendor` 目录下的 `_navbar.scss`。

► 3. 页面布局文件

文件名: `default.html`

目录位置:

代码清单 A.41.5

语言: text; 完整代码见附录第 477 页, 共 1 行。

作用: 负责将 `head.html`、`navbar.html`、页面主体等部分拼装成完整页面。

说明: 该文件本身不直接决定导航栏样式来源, 但决定了页面结构是如何组合的。

► 4. 最初尝试修改但未直接生效的主题样式文件

文件名: `_navbar.scss`

目录位置:

代码清单 A.41.6

语言: text; 完整代码见附录第 478 页, 共 1 行。

作用: 这是主题源码中与 `#mainNav` 相关的 SCSS 文件。

结论: 虽然这里确实存在 `#mainNav` 样式, 但本次站点实际加载的是 `assets/main.css`, 因此直接修改这个文件并没有在页面上体现出来。

► 5. 最终真正生效的样式文件

文件名: `main.css`

目录位置:

代码清单 A.41.7

语言: text; 完整代码见附录第 478 页, 共 1 行。

说明: 页面实际加载的就是这个文件, 所以最终的覆盖样式需要追加到这里, 或者追加到它对应的源文件 `assets/main.scss` 中。

如果项目中存在:

代码清单 A.41.8

语言: text; 完整代码见附录第 478 页, 共 1 行。

那么更推荐修改它; 如果没有, 或者为了快速验证效果, 可以直接在 `assets/main.css` 末尾追加覆盖样式。

► 三、问题排查过程

► 1. 初始问题: 导航栏换行

一开始观察到导航栏项目较多时发生换行, 初步判断是导航栏可用宽度不足。

在 `navbar.html` 中原始结构为:

代码清单 A.41.9

语言: html; 完整代码见附录第 478 页, 共 1 行。

这是 Bootstrap 的固定宽度容器, 当导航项较多时容易导致一行放不下, 从而换行。

► 2. 第一次尝试: 改成 container-fluid

将:

代码清单 A.41.10

语言: html; 完整代码见附录第 478 页, 共 1 行。

改为:

代码清单 A.41.11

语言: html; 完整代码见附录第 478 页, 共 1 行。

结果 优点:

- 导航栏可用宽度增大;
- 缓解了换行问题。

缺点:

- 导航整体变成左右贴边;
- 菜单与品牌名分得太开;
- 视觉上像“两边对齐”, 缺少居中感。

► 3. 第二次尝试: 增加 nav-inner

为了解决“更宽但又要居中”的问题, 采用了新的结构思路:

- 外层仍然使用 `container-fluid`
- 内层增加一个自定义容器 `nav-inner`

其作用是:

- 允许导航区域比默认 `container` 更宽;
- 同时通过 `max-width + margin: 0 auto` 保持整体居中;
- 避免 `container-fluid` 导致的左右贴边问题。

► 4. 第三次排查: 为什么改 `_navbar.scss` 没有效果

虽然通过搜索 `#mainNav` 找到了:

代码清单 A.41.12

语言: `text`; 完整代码见附录第 478 页, 共 1 行。

但多次修改后页面没有任何变化。

继续排查后, 在:

代码清单 A.41.13

语言: `text`; 完整代码见附录第 478 页, 共 1 行。

中确认页面实际加载的是:

代码清单 A.41.14

语言: `html`; 完整代码见附录第 479 页, 共 1 行。

由此得出的关键结论 真正生效的样式入口是:

代码清单 A.41.15

语言: `text`; 完整代码见附录第 479 页, 共 1 行。

因此:

- 直接改 `vendor` 目录里的 `_navbar.scss`
- 并不一定会反映到页面
- 必须改最终实际加载的主样式文件

这也是前面“样式写了但页面完全没变化”的根本原因。

► 四、最终采用的结构修改方案

► 修改文件

代码清单 A.41.16

语言: `text`; 完整代码见附录第 479 页, 共 1 行。

► 主要修改内容

1. 将外层容器改为 `container-fluid` 由:

代码清单 A.41.17

语言: html; 完整代码见附录第 479 页, 共 1 行。

改为:

代码清单 A.41.18

语言: html; 完整代码见附录第 479 页, 共 1 行。

2. 新增 `nav-inner` 在 `container-fluid` 内再包一层:

代码清单 A.41.19

语言: html; 完整代码见附录第 479 页, 共 1 行。

用于控制:

- 最大宽度
- 水平居中
- 内容排列

3. 去掉 `ml-auto` 原本菜单列表使用:

代码清单 A.41.20

语言: html; 完整代码见附录第 479 页, 共 1 行。

这会让菜单整体被推到最右边, 导致 `Home` 与品牌名距离过大。

最终改为:

代码清单 A.41.21

语言: html; 完整代码见附录第 479 页, 共 1 行。

► 五、最终采用的 `navbar.html` 关键结构

代码清单 A.41.22

语言: html; 完整代码见附录第 480 页, 共 48 行。

► 六、最终真正生效的样式修改位置

最终生效的样式需要追加到:

代码清单 A.41.23

语言: text; 完整代码见附录第 480 页, 共 1 行。

或者其对应源文件:

代码清单 A.41.24

语言：text；完整代码见附录第 481 页，共 1 行。

► 最终思路

1. 设置 `nav-inner`:
 - 保证更宽但不贴边
 - 整体居中
2. 设置 `navbar-brand`:
 - 不换行
3. 设置 `navbar-collapse`:
 - 不再自动占满剩余宽度
 - 不再把菜单推向最右侧
 - 可以通过 `margin-left` 微调菜单相对品牌名的位置
4. 设置 `.nav-link`:
 - 不换行
 - 控制左右间距
 - 让导航项在一行内更容易排开

► 七、最终采用的样式代码

以下为最终实际生效的样式方案。

注意: 在最后微调时, `margin-left` 最终被设为:

代码清单 A.41.25

语言：css；完整代码见附录第 481 页，共 1 行。

这是最终满意的参数。

代码清单 A.41.26

语言：css；完整代码见附录第 481 页，共 31 行。

► 八、最终结果

经过本次调整, 导航栏最终达到了以下效果:

1. 解决了菜单项换行问题。
2. 导航栏不再左右贴边。
3. 导航内容整体保持居中。
4. Home 与品牌名 Mr.X's Blog 的距离可以手动精细控制。
5. 通过设置:

代码清单 A.41.27

语言：css；完整代码见附录第 481 页，共 1 行。

实现了导航菜单整体向右平移, 达到最终满意效果。

► 九、最终修改清单

► 需要修改的文件

1. 导航结构

代码清单 A.41.28

语言: text; 完整代码见附录第 482 页, 共 1 行。

2. 页面头部 (用于确认样式入口, 不一定需要改)

代码清单 A.41.29

语言: text; 完整代码见附录第 482 页, 共 1 行。

3. 页面布局 (用于确认页面拼装关系, 不一定需要改)

代码清单 A.41.30

语言: text; 完整代码见附录第 482 页, 共 1 行。

4. 最终生效的主样式文件

代码清单 A.41.31

语言: text; 完整代码见附录第 482 页, 共 1 行。

或:

代码清单 A.41.32

语言: text; 完整代码见附录第 482 页, 共 1 行。

5. 曾尝试但本次未直接生效的主题 SCSS 文件

代码清单 A.41.33

语言: text; 完整代码见附录第 482 页, 共 1 行。

► 十、简明结论

本次修改完成了如下目标:

- 找到了导航栏换行的原因;
 - 解决了 `container-fluid` 导致的左右贴边问题;
 - 引入 `nav-inner` 实现“更宽但居中”的导航布局;
 - 确认了真正生效的样式入口是 `assets/main.css`;
 - 通过 `navbar-collapse` 的 `margin-left` 微调菜单位置;
 - 最终将 `margin-left` 设为 `6rem`, 获得满意效果。
-

▶ 十一、最终总结

本次对话完成了从:

- 问题判断
- 文件排查
- 样式入口确认
- 结构调整
- 样式覆盖
- 视觉微调

到最终结果确认的完整过程。

最终成功解决了 Jekyll 博客导航栏的以下问题:

- 导航栏换行
- 导航贴边
- 不居中
- 品牌名与菜单距离不协调

并形成了一套可复用的导航栏修改方案。

这份记录可以作为后续博客主题调整、导航栏优化以及样式排查时的参考。

=====

▶ 一、对话目的

本次修改的核心目标是解决 Jekyll 博客导航栏的两个问题:

1. 导航项过多时出现换行, 怀疑是导航栏可用宽度不足。
2. 在解决换行后, 希望导航栏整体视觉更协调:
 - 不要左右两边贴边;
 - 整体保持居中;
 - Home 与左侧品牌名 Mr.X' s Blog 的距离更合理;
 - 最终再将菜单整体向右平移到满意位置。

▶ 二、项目中涉及的关键文件

本次对话中确认和涉及到的主要文件如下:

1. 结构文件
 - `_includes/navbar.html` 作用: 控制导航栏的 HTML 结构, 包括品牌名、菜单项、按钮和容器层级。
2. 样式文件 (最初尝试)
 - `assets/vendor/startbootstrap-clean-blog/scss/_navbar.scss` 作用: 主题中与 `#mainNav` 相关的 SCSS 源码。结论: 虽然该文件中确实有 `#mainNav` 样式, 但本次站点实际加载的不是直接依赖这个文件编译后的结果, 因此直接修改它没有产生页面效果。
3. 页面头部文件
 - `_includes/head.html` 作用: 决定页面实际加载哪些 CSS 文件。结论: 通过检查该文件, 确认页面实际加载的是: `/assets/main.css`
4. 布局文件

- `_layouts/default.html` 作用: 负责把 `head`、`navbar` 等 `include` 拼装到页面中。结论: 它不直接决定导航栏样式来源, 只负责页面结构装配。
5. 最终生效的样式入口
 - `assets/main.css`(或对应的 `assets/main.scss`) 作用: 页面最终加载的主样式文件。结论: 真正需要追加覆盖样式的地方是这里, 而不是 `vendor` 目录下的 `_navbar.scss`。

► 三、问题排查过程总结

1. 初始判断: 导航栏换行是由于宽度不足。在 `_includes/navbar.html` 中看到: 这是 Bootstrap 的固定宽度容器, 导航项较多时容易换行。
2. 第一次修改: 将: 改为: 结果:
 - 导航栏变宽了;
 - 但出现左右贴边、两边对齐的情况;
 - 视觉上不居中, 不符合预期。
3. 第二次修改思路: 在 `navbar` 中增加自定义居中内层容器 `nav-inner`, 形成:
 - 外层 `container-fluid` 提供更大可用宽度;
 - 内层 `nav-inner` 通过 `max-width + margin: 0 auto` 实现“更宽但居中”。
4. 第二轮问题: 即使修改了 `vendor` 中的 `_navbar.scss`, 页面仍然没有变化。排查后发现:
 - 搜索 `#mainNav` 时, 确实找到了 `assets/vendor/startbootstrap-clean-blog/scss/_navbar.scss`;
 - 但修改这个文件没有效果;
 - 继续检查 `_includes/head.html` 后确认, 页面实际加载的是 `/assets/main.css`;
 - 因此说明之前改动的 `vendor scss` 不是当前页面最终生效的样式入口。
5. 第三次修改: 将覆盖样式直接追加到页面实际加载的主样式文件 (`assets/main.css` 或其对应源文件 `assets/main.scss`) 末尾。这一步开始生效。
6. 后续微调:
 - 调整 `navbar-collapse` 的布局逻辑, 使导航项不再被强制推到最右边;
 - 再通过修改 `margin-left` 来调整 Home 与品牌名之间的距离;
 - 最终用户将 `margin-left` 设为 `6rem`, 并确认效果满意。

► 四、最终结构方案 (`navbar.html`)

最终导航栏结构修改的核心思路是:

1. 外层使用: 以获得比默认 `container` 更大的横向空间。
2. 在其中加入: 作为真正的内容容器, 用于:
 - 限制最大宽度;
 - 实现整体水平居中;
 - 避免导航贴边。
3. 菜单列表使用:
4. 最终 `navbar.html` 的关键结构是:
 - `nav#mainNav`
 - `.container-fluid`
 - `.nav-inner`
 - `.navbar-brand`
 - `.collapse.navbar-collapse`
 - `.navbar-nav`

► 五、最终生效的样式思路

真正生效的样式不是直接写在 vendor 的 _navbar.scss 中, 而是追加到了页面最终加载的主样式文件末尾。

样式逻辑包括以下几个方面:

1. 控制 nav-inner 的宽度和居中目的:
 - 比默认 container 更宽;
 - 但不贴满整个屏幕;
 - 保持视觉居中。
2. 控制 navbar-brand 不换行目的:
 - 防止 “Mr.X’s Blog” 被压缩或换行。
3. 控制 navbar-collapse 的排列方式目的:
 - 不让菜单自动占满剩余空间;
 - 不再把导航项强制推到最右边;
 - 允许菜单整体相对品牌名进行精细位置调整。
4. 控制 nav-link 不换行并缩小左右间距目的:
 - 防止 Sci-log、Sci-note 等文字换行;
 - 给多个菜单项腾出更多横向空间。

六、最终采用的样式代码

以下代码为本次对话中最终生效并达到满意效果的核心样式。注意: 最后实际使用时, margin-left 已被用户改为 6rem。

```
@media (min-width: 992px) { #mainNav .nav-inner { width: 100%; max-width: 1400px; margin: 0 auto; display: flex; align-items: center; }
#mainNav .navbar-brand { white-space: nowrap; flex-shrink: 0; }
#mainNav .navbar-collapse { display: flex !important; flex-grow: 0 !important; justify-content: flex-start !important; margin-left: 6rem !important; }
#mainNav .navbar-nav { margin-left: 0 !important; }
#mainNav .navbar-nav .nav-link { white-space: nowrap !important; padding-left: 12px !important; padding-right: 12px !important; } }
```

► 七、最终结果

经过本次修改, 导航栏达成了以下效果:

1. 解决了导航项换行问题。
2. 导航栏不再左右贴边。
3. 导航内容整体保持居中。
4. Home 与品牌名之间的距离可以精细控制。
5. 最终通过: margin-left: 6rem !important; 将导航菜单整体向右平移到满意位置。
6. 用户确认:
 - 修改已经生效;
 - 并成功达到了想要的效果。

► 八、最终建议

- 以后如果类似修改“写了但页面没变化”，优先检查：
 - 页面实际加载的是哪个 CSS 文件；
 - 修改的 scss/css 是否真的参与了最终构建。
- 对于主题源码目录下的 vendor scss:
 - 不要默认认为它一定会自动进入当前页面；
 - 先确认 head.html 中加载的最终样式入口。
- 针对导航栏布局, 推荐优先使用以下思路:
 - container-fluid 提供更大空间；
 - 自定义 nav-inner 控制最大宽度与居中；
 - 通过 navbar-collapse 的 margin-left 微调菜单整体位置；
 - 通过 nav-link padding 和 white-space 解决拥挤与换行问题。

► 九、本次最终配置要点 (简明版)

- navbar.html:
 - container -> container-fluid
 - 新增 nav-inner
 - 去掉 navbar-nav 上的 ml-auto
- 真正生效的样式追加位置:
 - assets/main.css 或对应的 assets/main.scss 末尾
- 最终关键参数:
 - max-width: 1400px
 - nav-link 左右 padding: 12px
 - navbar-collapse 的 margin-left: 6rem

► 十、结论

本次对话完成了从“问题判断 -> 文件排查 -> 样式入口确认 -> 结构调整 -> 样式覆盖 -> 视觉微调”的完整过程, 最终成功解决了 Jekyll 博客导航栏的换行、贴边、间距不协调等问题, 并形成了一套可复用的导航栏调整方案。

6.3 (原创自研) 网站维护记录 2

日期: 2026-03-26

在线原文: marx2001.github.io

► Jekyll 博客翻页逻辑更新总结

► 一、这次修改的核心目的

这次更新翻页逻辑, 核心目标有四个:

1. 解决网站文章数量较多时, 页面加载过慢的问题。
 2. 将原先依赖前端 JavaScript 的“假分页”改成真正的静态分页。
 3. 让各分类页 (如 `life`、`reading`、`writing` 等) 支持真正的分页访问。
 4. 将 `GitHub Pages` 的部署方式从原先的默认分支构建, 切换到 `GitHub Actions` 自定义构建, 以便支持 `jeekyll-paginate-v2` 和自动分类页生成。
-

► 二、原先的问题: 前端假分页导致页面越来越卡

在原先的方案中, 多个栏目页都采用了类似的逻辑:

- 在页面中生成一个 `window.allPosts` 数组
- 把大量文章元数据一次性写进前端
- 再配合 `pagination.js` 在浏览器中进行“假分页”

典型结构包括:

- `posts-container`
- `pagination`
- `window.currentCategory`
- `window.allPosts`
- `pagination.js`

► 这种做法的问题

1. 页面一打开就要把大量文章数据全部塞进前端。
 2. 分类页文章越多, 页面体积越大, 浏览器解析越慢。
 3. 首页、分类页、搜索页都可能重复加载大量文章数据。
 4. 读者点击“下一页”时并没有真的加载新页面, 而只是前端切换已加载的数据。
 5. 不利于后续维护, 也不利于 `GitHub Pages` 上的稳定展示。
-

► 三、为什么要切换到真分页

目标是实现:

- 首页只显示当前页的文章
- 分类页只显示该分类当前页的文章
- 用户点“下一页”时, 访问的是新的静态 `HTML` 页面
- 页面不再一次性加载所有文章数据

也就是说, 要从:

前端 JS 分页

切换到:

`Jekyll` 构建阶段生成的真分页

► 四、为什么部署方式也要改

一开始网站是直接依赖 GitHub Pages 默认构建的。
这种方式虽然简单, 但有一个限制:

- GitHub Pages 原生受支持的插件有限
- `jeekyll-paginate-v2` 不适合继续走默认受限构建

因此后续改成了:

- 本地修改文件
- 使用 GitHub Desktop commit / push
- GitHub Actions 自动构建和部署到 GitHub Pages

► 这个改动的意义

1. 平时的写作和 push 流程基本不变。
2. 构建逻辑可以自己控制。
3. 可以使用 `jeekyll-paginate-v2`。
4. 可以自动生成分类页和分页页。
5. 更适合文章数量越来越多的网站。

► 五、这次涉及和修改过的主要文件

下面按功能分类总结。

► 1. 导航栏结构文件

文件:

代码清单 A.42.1

语言: text; 完整代码见附录第 483 页, 共 1 行。

► 作用

- 控制导航栏链接
- 决定各分类页入口路径

► 与本次翻页逻辑相关的点

导航栏中的分类入口必须和自动生成分类页路径保持一致, 例如:

- `writing` 对应 `/writing/`
- `life` 对应 `/life/`
- `reading` 对应 `/reading/`

► 注意事项

- 导航栏路径建议统一使用小写
- 各分类名也建议统一使用小写
- 避免出现 `/Writing/` 和 `/writing/` 混用的情况

► 2. 原先的前端分页脚本

文件:

代码清单 A.42.2

语言: `text`; 完整代码见附录第 483 页, 共 1 行。

► 原作用

- 接收 `window.allPosts`
- 在浏览器前端进行分页切换

► 本次更新后的处理

这个文件不再作为分类页和首页分页的核心逻辑使用。

真分页完成后:

- 分类页不再依赖 `window.allPosts`
- 首页不再依赖 `window.allPosts`
- `pagination.js` 逻辑退出主要舞台

► 结论

这个脚本代表的是旧的前端假分页方案。

► 3. 站点配置文件

文件:

代码清单 A.42.3

语言: `text`; 完整代码见附录第 483 页, 共 1 行。

► 这是本次更新最关键的文件之一

最终改成了支持:

- `jeekyll-paginate-v2`
- `autopages`
- 分类自动分页

► 最终核心配置逻辑

代码清单 A.42.4

语言: yaml; 完整代码见附录第 483 页, 共 26 行。

► 这个文件里曾经遇到的问题

问题 1: 分页功能被关闭 曾经写成:

代码清单 A.42.5

语言: yaml; 完整代码见附录第 483 页, 共 2 行。

这会导致:

- 自动分类页存在
- 但 `paginator.posts` 不可用
- 页面看起来生成了, 但文章列表为空

解决方法 改为:

代码清单 A.42.6

语言: yaml; 完整代码见附录第 484 页, 共 2 行。

并补充 `per_page`、`permalink` 等分页参数。

问题 2: 分类 `slug` 大小写不一致 在自动生成分类页时, 如果分类名或 `slug` 大小写不统一, GitHub Pages 上容易出现:

- 本地正常
- 线上 404

解决方法 增加:

代码清单 A.42.7

语言: yaml; 完整代码见附录第 484 页, 共 3 行。

强制分类页路径统一小写。

► 4. 页面通用布局文件

文件:

代码清单 A.42.8

语言: text; 完整代码见附录第 484 页, 共 1 行。

► 原问题

原先这个文件里仍然内嵌了旧的前端假分页逻辑, 例如:

- `posts-container`
- `pagination`
- `window.allPosts`
- `pagination.js`

这会导致所有 `layout: page` 页面都可能被自动注入旧分页数据。

► 解决方法

将 `_layouts/page.html` 改成真正的“纯页面布局”:

- 只负责页头显示
- 只负责页面正文输出
- 不再注入文章列表和前端分页逻辑

► 最终作用

- 普通页面 (About、Search 等) 正常显示正文
- 不再意外地加载大量文章数据

► 5. 首页布局文件

文件:

代码清单 A.42.9

语言: `text`; 完整代码见附录第 484 页, 共 1 行。

► 原问题

About 等页面曾经使用:

代码清单 A.42.10

语言: `yaml`; 完整代码见附录第 484 页, 共 1 行。

但 `home` 布局后来被改成了首页专用的文章分页布局:

- 显示站点标题
- 遍历当前页文章列表
- 不输出普通页面正文

这会导致:

- About 页面显示成 Home 的样子
- 但没有正确显示正文

► 解决方法

- layout: home 只留给首页用
- About / Search / 普通说明页改成 layout: page

► 结论

home 只服务首页真分页, 不再给普通正文页面使用。

► 6. 自动分类页布局文件

文件:

代码清单 A.42.11

语言: text; 完整代码见附录第 484 页, 共 1 行。

► 这是分类页真分页的核心模板

自动生成的分类页 (如 /life/、/reading/、/writing/) 都依赖这个布局。

► 主要职责

1. 显示分类页头部
2. 根据分类名选择背景图
3. 根据分类名显示合适标题
4. 遍历当前分页文章
5. 输出上一页 / 下一页按钮

► 最终版模板解决了这些问题

问题 1: 标题显示为小写 例如页面标题显示为:

代码清单 A.42.12

语言: text; 完整代码见附录第 484 页, 共 1 行。

而不是:

代码清单 A.42.13

语言: text; 完整代码见附录第 485 页, 共 1 行。

解决方法 在模板里手动为分类名映射标题:

- life -> Life
- sci-log -> Sci-Log
- sci-note -> Sci-Note
- reading -> Reading

- learning -> Learning
- writing -> Writing

问题 2: 页面多出 “life 分类文章” 之类的说明文字

解决方法 删除多余说明文字, 不再在分类页正文顶部显示这类句子。

问题 3: 分类页背景图没有正确载入 因为自动分类页不会自动继承原手写页面中的背景图字段。

解决方法 在 `_layouts/autopage_category.html` 中按分类名手动分配背景图:

- life -> /img/bg-life.jpg
- sci-log -> /img/bg-sci-log.jpg
- sci-note -> /img/bg-note.jpg
- reading -> /img/bg-reading.jpg
- learning -> /img/bg-about.jpg
- writing -> /img/bg-travel.jpg

► 7. GitHub Actions 工作流文件

文件:

代码清单 A.42.14

语言: text; 完整代码见附录第 485 页, 共 1 行。

► 作用

- 自动构建站点
- 自动部署到 GitHub Pages

► 最终工作流作用

1. checkout 仓库
2. 配置 Pages
3. 配置 Ruby
4. 执行依赖安装
5. 执行 Jekyll build
6. 上传 `_site`
7. 部署到 `github-pages` environment

► 本次遇到的关键问题

问题 1: workflow 不自动触发 最开始 workflow 写的是只监听 `main`, 但实际 push 的分支是 `master`。
所以:

- 不会自动触发
- 只能手动点击 Run workflow

解决方法 改成同时监听:

- master
- main

这样:

- push 到 master 会自动部署
- push 到 main 也能自动部署
- 仍保留手动运行能力

问题 2: Actions 页面只看到旧的 **pages-build-deployment** 这通常说明:

- 自己的 workflow 没被 GitHub 识别
- 文件位置错误
- Pages Source 没切到 GitHub Actions

解决方法 确认:

代码清单 A.42.15

语言: text; 完整代码见附录第 485 页, 共 1 行。

存在并已 push; 同时在仓库设置中将 Pages Source 切换到:

代码清单 A.42.16

语言: text; 完整代码见附录第 485 页, 共 1 行。

► 8. Gemfile

文件:

代码清单 A.42.17

语言: text; 完整代码见附录第 485 页, 共 1 行。

► 作用

控制本地和 GitHub Actions 构建时安装哪些 Ruby gems。

► 本次的关键改动

将旧的分页 gem 替换为:

短代码 · text

jeekyll-paginate-v2

同时保留:

- `jeekyll-feed`
- `jeekyll-sitemap`

► 作用

让本地和 Actions 构建都能正确识别自动分类真分页能力。

► 9. 各分类页旧文件

原先存在一些手写分类页, 例如:

代码清单 A.42.19

语言: `text`; 完整代码见附录第 485 页, 共 6 行。

► 原作用

它们曾经承担:

- 分类入口页
- 注入分类变量
- 配合旧前端分页逻辑显示文章

► 新问题

如果继续保留这些文件, 并且它们仍然占用这些 `permalink`:

- `/life/`
- `/reading/`
- `/sci-log/`
- `/sci-note/`
- `/learning/`
- `/writing/`

就会与 `autopages` 自动生成的分类页冲突。

► 处理建议

最终建议:

- 删除这些旧手写分类页
- 或至少改掉它们的 `permalink`, 避免和新自动分类页冲突

► 迁移原则

不是一开始就立刻无脑删除, 而是:

1. 先搭好新系统

2. 确认自动分类页正常生成
3. 再删除旧文件

▶ 六、这次迁移过程中遇到的主要问题与解决办法

下面按问题类型汇总。

▶ 问题 1: 首页与分类页都在加载大量文章, 导致站点越来越卡

▶ 原因

前端假分页方案会把大量文章数据写入页面中。

▶ 解决方法

采用 Jekyll 构建时真分页:

- 首页走 `home` 布局的当前页文章列表
- 分类页走 `autopage_category.html` 的当前页文章列表

▶ 问题 2: 本地能用, GitHub 上分类页 404

▶ 原因之一

workflow 最初只监听 `main`, 实际 `push` 到 `master`, 自动部署没有正常触发。

▶ 解决方法

将 workflow 的分支监听改成:

代码清单 A.42.20

语言: `yaml`; 完整代码见附录第 485 页, 共 1 行。

▶ 原因之二

未来日期文章在 GitHub 构建时被排除。

Jekyll 默认不发布未来文章。如果某个分类 (如 `writing`) 目前只有未来日期文章, 那么:

- 单篇文章不会生成
- 分类首页也不会生成

▶ 解决方法

将文章日期改到当前时间之前, 或显式调整 `date` 字段。

例如将:

- 文件名日期从未来日期改成当前或过去日期
- front matter 中明确加 **date**

► 问题 3: 本地构建和远端部署不一致

► 表现

- 本地分类页正常
- 远端访问 404
- artifact 中缺少分类目录

► 解决方法

从 Actions build 日志和 artifact 反查:

1. 检查日志中是否有:
 - AutoPages
 - Pagination
2. 下载 artifact, 看是否生成:
 - writing/index.html
 - writing/page2/index.html
 - 单篇文章页面

通过这个过程可以判断问题是出在:

- 构建阶段
- 还是发布阶段

► 问题 4: About 页面显示成 Home 的样子

► 原因

About 页面使用了:

代码清单 A.42.21

语言: yaml; 完整代码见附录第 486 页, 共 1 行。

而 home 布局已经被改成首页专用文章分页布局。

► 解决方法

About 页改成:

代码清单 A.42.22

语言: yaml; 完整代码见附录第 486 页, 共 2 行。

► **问题 5: 自动分类页背景图丢失、标题小写、多余说明文字**

► **原因**

自动生成分类页不再读取原手写页面的 front matter。

► **解决方法**

在 `_layouts/autopage_category.html` 中:

- 手动映射标题
 - 手动映射背景图
 - 删除多余说明文字
-

► **问题 6: workflow 存在 warning**

► **表现**

GitHub Actions 中出现 Node.js 20 deprecation warning。

► **结论**

这类 warning 当前不是导致部署失败或分类页 404 的直接原因。
只要 **build** 和 **deploy** 是绿色, 当前部署仍然是成功的。

► **处理建议**

后续有空时再统一升级 action 版本。

► **问题 7: 本地测试 Ruby / Bundler 环境不完整**

► **表现**

最开始执行依赖安装命令时, 终端提示找不到 **bundle**。

► **解决方法**

1. 安装 RubyInstaller for Windows
 2. 勾选加入 PATH
 3. 安装 MSYS2-Devkit
 4. 安装 Bundler 和 Jekyll
 5. 再运行:
 - `bundle install`
 - `bundle exec jekyll serve`
-

► 七、本次最终形成的翻页逻辑结构

现在网站的翻页逻辑, 目标结构如下:

► 首页

- 使用 `layout: home`
- 使用当前页文章列表
- 真分页 URL:
 - `/`
 - `/page2/`
 - `/page3/`

► 分类页

- 由 `jeekyll-paginate-v2 + autopages` 自动生成
- 使用 `_layouts/autopage_category.html`
- 真分页 URL:
 - `/life/`
 - `/life/page2/`
 - `/reading/`
 - `/reading/page2/`
 - `/writing/`
 - `/writing/page2/`

► 普通页面

- 使用 `layout: page`
 - 只显示页面正文
-

► 八、当前分类建议

目前建议作为自动分类真分页的分类:

- `life`
- `sci-log`
- `sci-note`
- `reading`
- `learning`
- `writing`

不建议作为自动分类页处理的页面:

- `search`
- `about`

因为它们更适合作为普通页面, 而不是文章分类归档页。

► 九、后续维护建议

► 1. 分类名统一小写

文章 front matter 中统一写:

代码清单 A.42.23

语言: yaml; 完整代码见附录第 486 页, 共 6 行。

不要混用:

- Writing
- WRITING
- writing

► 2. 文章 permalink 尽量统一带尾斜杠

例如:

代码清单 A.42.24

语言: yaml; 完整代码见附录第 486 页, 共 1 行。

这样更符合静态站点目录结构, 也更稳定。

► 3. 避免未来日期文章直接影响线上分类页

如果某分类只有未来文章, 那么线上分类页可能不会生成。

建议:

- 发布前确认日期不是未来时间
- 或谨慎使用 `future: true`

► 4. 删除旧的手写分类页残留

一旦确认自动分类页稳定运行, 应删除旧的:

- life.md
- reading.md
- sci-log.md
- sci-note.md
- learning.md
- writing.md

避免路径冲突和维护混乱。

► 5. Action warning 后续再统一处理

当前可以先不影响正常写作与部署。

后续若要升级, 可统一更新 workflow 中的 action 版本。

► 十、最终总结

这次翻页逻辑更新,本质上是一次从:

前端假分页 + GitHub Pages 默认构建

迁移到:

Jekyll 构建时真分页 + GitHub Actions 自动部署

的系统性重构。

► 这次重构解决了什么

1. 减少页面一次性加载全部文章带来的卡顿。
2. 首页、分类页都转向真正的静态分页。
3. 分类页支持自动生成和自动分页。
4. 文章数量增多后站点结构更清晰。
5. 本地与线上部署链路更加明确。

► 这次更新最重要的经验

1. 先分清“构建问题”和“发布问题”。
2. 本地正常不代表远端部署一定一致。
3. GitHub Actions 的分支监听必须和实际分支一致。
4. 自动生成分类页时,要避免旧手写页抢同一路径。
5. 未来日期文章会直接影响线上分类页是否生成。

► 当前状态

翻页逻辑已经完成从旧方案向新方案的迁移,整体方向是正确的。

后续只需继续做少量清理和细节统一,就可以形成一个更稳定、更适合长期维护的博客结构。

6.4 (原创自研) 网站维护记录 3

日期: 2026-03-26

在线原文: marx2001.github.io

► Windows 下安装 Ruby 与本地预览 Jekyll 的流程总结

► 一、这部分内容的目的

这次整理的目标,是把本对话中关于以下内容的操作流程完整记录下来:

1. 如何在 Windows 上安装 Ruby。
2. 如何安装 Bundler 和 Jekyll。
3. 如何在本地运行 Jekyll 站点并预览网页效果。
4. 如何理解本地预览时常见的终端输出、警告和错误。
5. 本次实际遇到过的问题以及对应的解决方法。

这份记录适合作为以后重新配置电脑、本地重装环境、或者帮助别人搭建同类博客时的参考文档。

► 二、为什么最开始会报错

最开始在项目目录里执行：

命令示例 · bat

```
bundle install
```

终端报错：

代码清单 A.43.2

语言：text；完整代码见附录第 486 页，共 1 行。

► 这说明什么

这通常说明以下几种情况之一：

1. Ruby 还没有安装。
2. Bundler 还没有安装。
3. Ruby 已经安装, 但没有加入系统 PATH。
4. 命令行窗口没有重新打开, 环境变量还没有刷新。

也就是说, 这不是博客代码本身的问题, 而是本地 Ruby 环境还没有准备好。

► 三、Windows 下安装 Ruby 的流程

► 1. 安装 RubyInstaller for Windows

在 Windows 上, 推荐安装:

RubyInstaller for Windows

建议选择:

- 64 位版本
- 带 Devkit 的版本 (Ruby+Devkit)

► 安装时要注意的点

安装过程中要注意:

1. 勾选 **Add Ruby executables to your PATH**
2. 安装完成后继续执行 **ridk install**

3. 安装 MSYS2 / Devkit

► 为什么要装 Devkit / MSYS2

因为很多 Ruby gems 在 Windows 上都需要依赖它, 尤其是 Jekyll 项目在安装部分依赖时, 经常会需要 Devkit 环境。

► 2. 安装完成后看到的 Ruby 终端菜单

安装完成后, 打开 Ruby 终端时, 会看到类似几个选项:

- irb:>
- Core and stdlib API
- gem doc server
- C:\> ruby enabled command line
- Install MSYS2-Devkit necessary for many gems

► 这几个选项里, 实际最重要的是两个

第一个: 进入命令行环境 应当选择:

代码清单 A.43.3

语言: text; 完整代码见附录第 486 页, 共 1 行。

这是后面执行 Ruby、gem、bundle、jekyll 命令的入口。

第二个: 安装 Devkit 还应该执行:

代码清单 A.43.4

语言: text; 完整代码见附录第 487 页, 共 1 行。

这个步骤建议完成, 不要跳过。

► 四、安装完成后先验证环境

进入 Ruby 命令行后, 先执行:

命令示例 · bat

```
ruby -v  
gem -v
```

► 这两条命令的作用

- ruby -v: 检查 Ruby 是否安装成功
- gem -v: 检查 RubyGems 是否可用

如果这两条命令都能正常输出版本号, 说明 Ruby 基础环境已经可以用了。

► 五、安装 Bundler 和 Jekyll

在 Ruby 命令行中执行:

命令示例 · bat

```
gem install bundler jekyll
```

► 这一步的作用

安装:

- Bundler
- Jekyll

安装完成后, 再检查:

命令示例 · bat

```
bundle -v  
jekyll -v
```

► 如果这里能正常输出版本号

就说明:

- Bundler 安装成功
- Jekyll 安装成功
- 本地已经具备运行 Jekyll 项目的基础能力

► 六、进入项目目录

在 Windows 下, 项目目录在不同盘符时, 推荐使用:

命令示例 · bat

```
cd /d D:\aa\1\marx2001.github.io
```

► 为什么要加 /d

因为 /d 可以让命令同时切换盘符和目录。

如果只写 `cd D:\aa\1\marx2001.github.io`, 有时不会真正切换到对应盘符。

► 七、安装项目依赖

进入博客项目目录后, 执行:

命令示例 · bat

```
bundle install
```

► 这一步的作用

根据项目根目录里的:

代码清单 A.43.10

语言: text; 完整代码见附录第 487 页, 共 1 行。

安装这个项目真正需要的所有 Ruby 依赖。

这一步非常重要, 因为即使系统已经装好了 Jekyll, 本项目可能还依赖其他 gems, 例如:

- jekyll-feed
- jekyll-sitemap
- jekyll-paginate-v2

► 八、本地预览 Jekyll 网站

► 1. 启动本地服务

在项目根目录执行:

命令示例 · bat

```
bundle exec jekyll serve
```

如果想提升实时刷新体验, 也可以执行:

命令示例 · bat

```
bundle exec jekyll serve --livereload
```

► 这两条命令的作用

- `bundle exec jekyll serve`: 启动本地 Jekyll 服务器
- `--livereload`: 保存文件后浏览器更容易自动刷新

► 2. 启动成功时的典型输出

如果启动成功, 终端通常会出现类似:

代码清单 A.43.13

语言: text; 完整代码见附录第 487 页, 共 2 行。

► 这说明什么

这说明:

- Jekyll 已经成功构建
- 本地服务器已经启动
- 现在可以在浏览器打开站点预览效果

► 3. 浏览器里打开本地地址

一般打开以下地址之一即可:

代码清单 A.43.14

语言: text; 完整代码见附录第 487 页, 共 1 行。

或者:

代码清单 A.43.15

语言: text; 完整代码见附录第 487 页, 共 1 行。

如果首页能打开, 说明本地预览成功。

► 九、本地修改后如何查看效果

► 1. 普通页面内容、布局、样式修改

如果修改的是以下内容, 通常不需要完全重启服务, 只需要:

- 保存文件
- 刷新浏览器

常见包括:

- 文章内容
- 普通页面 .md / .html
- _includes/
- _layouts/
- 大多数 CSS / SCSS
- 导航栏内容
- 页面正文文本

► 2. 哪些情况建议重启 jekyll serve

如果修改的是下面这些, 建议停止服务后重新运行:

- _config.yml
- Gemfile
- 插件配置
- 分页逻辑

- 自动分类页逻辑
- 新增 layout / 新增 plugin
- 新增或大改生成规则

► 原因

这些内容往往会影响 Jekyll 的构建流程本身, 单纯刷新浏览器不一定会重新读取这些配置。

► 3. 正确重启方式

在当前终端按:

代码清单 A.43.16

语言: text; 完整代码见附录第 487 页, 共 1 行。

停止当前服务, 然后再执行:

命令示例 · bat

```
bundle exec jekyll serve
```

如果改过 Gemfile, 则先执行:

命令示例 · bat

```
bundle install
```

再执行:

命令示例 · bat

```
bundle exec jekyll serve
```

► 十、常见命令汇总

下面整理成本次对话中最常用的一组命令。

► 1. 检查 Ruby / gem / bundle 是否安装成功

命令示例 · bat

```
ruby -v
gem -v
bundle -v
jekyll -v
```

► 2. 安装 Bundler 和 Jekyll

命令示例 · bat

```
gem install bundler jekyll
```

► 3. 进入项目目录

命令示例 · bat

```
cd /d D:\aa\1\marx2001.github.io
```

► 4. 安装项目依赖

命令示例 · bat

```
bundle install
```

► 5. 本地预览站点

命令示例 · bat

```
bundle exec jekyll serve
```

或:

命令示例 · bat

```
bundle exec jekyll serve --livereload
```

► 6. 如果端口 4000 被占用

可以改用其他端口, 例如:

命令示例 · bat

```
bundle exec jekyll serve --port 4001
```

然后在浏览器打开:

代码清单 A.43.27

语言: text; 完整代码见附录第 487 页, 共 1 行。

► 十一、这次实际遇到过的问题

► 问题 1: bundle 不是内部或外部命令

► 表现

执行:

```
命令示例 · bat
```

```
bundle install
```

时报错, 提示系统找不到 bundle。

► 原因

通常是:

- Ruby 未安装
- Bundler 未安装
- Ruby 没有加入 PATH
- 命令行没有重新打开

► 解决方法

1. 安装 RubyInstaller for Windows
2. 勾选加入 PATH
3. 安装 MSYS2-Devkit
4. 执行:

```
命令示例 · bat
```

```
gem install bundler jekyll
```

1. 重新打开命令行

► 问题 2: 不知道 Ruby 终端菜单里该选哪个

► 表现

安装 Ruby 后打开界面, 不清楚应该选:

- irb
- gem doc
- 还是其他选项

► 解决方法

优先选择:

代码清单 A.43.30

语言: text; 完整代码见附录第 488 页, 共 1 行。

后续如果还没装 Devkit, 再选择:

代码清单 A.43.31

语言: text; 完整代码见附录第 488 页, 共 1 行。

► **问题 3: 本地服务起来后, 不确定是否正常**

► **现象**

终端里出现一大段输出和 warning, 不确定有没有启动成功。

► **判断标准**

最重要的是有没有出现:

代码清单 A.43.32

语言: text; 完整代码见附录第 488 页, 共 2 行。

只要出现了这类输出, 就说明:

- 站点已经成功在本地启动
- 浏览器可以访问

► **问题 4: 本地有大量 warning, 不知道是否要马上处理**

► **现象**

例如出现一堆 Sass deprecation warning。

► **结论**

如果本地服务器已经成功运行, 这类 warning 一般不会立刻阻止预览。它们往往是:

- 主题老旧
- Bootstrap / SCSS 写法过时

► **处理建议**

如果只是 warning, 而不是 build failed, 可以先继续做功能修改。后续有空再集中处理样式层面的 warning。

► 问题 5: 本地正常, 线上不正常

这次对话中, 本地预览成功后, 后来还出现过:

- 本地分类页正常
- GitHub Pages 上仍然 404

► 这说明什么

说明:

- 本地预览成功 $\neq$ 线上部署一定成功
- 本地环境只是验证 “构建逻辑是否工作”
- 线上还要看 GitHub Actions 是否跑对了、是否部署了最新产物、文章日期是否影响发布等

► 经验

本地预览解决的是 “本地构建问题”,
GitHub 上显示异常还要继续排查:

- workflow 是否触发
- artifact 是否包含页面
- 未来日期文章是否被排除
- Pages Source 是否正确

► 十二、这次最终形成的本地测试流程

以后在 Windows 上重新做同样事情, 可以直接按照下面这个顺序。

► 第一步: 安装 Ruby

- 安装 RubyInstaller for Windows
- 勾选加入 PATH
- 安装 MSYS2-Devkit

► 第二步: 打开 Ruby 命令行

选择:

代码清单 A.43.33

语言: text; 完整代码见附录第 488 页, 共 1 行。

► 第三步: 安装 Bundler 和 Jekyll

命令示例 · bat

```
gem install bundler jekyll
```

► 第四步: 验证环境

命令示例 · bat

```
ruby -v  
gem -v  
bundle -v  
jekyll -v
```

► 第五步: 进入项目目录

命令示例 · bat

```
cd /d D:\aa\1\marx2001.github.io
```

► 第六步: 安装项目依赖

命令示例 · bat

```
bundle install
```

► 第七步: 启动本地预览

命令示例 · bat

```
bundle exec jekyll serve
```

或者:

命令示例 · bat

```
bundle exec jekyll serve --livereload
```

► 第八步: 浏览器打开

代码清单 A.43.40

语言: text; 完整代码见附录第 488 页, 共 1 行。

► 十三、后续维护建议

1. 如果换电脑或重装系统, 优先恢复 Ruby / Bundler / Jekyll 环境。
2. 每次大改 `_config.yml`、插件、分页逻辑后, 最好重启一次 `jekyll serve`。
3. 本地预览只负责验证构建与页面效果, 不代表 GitHub 线上发布一定同步正确。
4. 如果以后再安装依赖失败, 先检查:
 - Ruby 是否在 PATH
 - `bundle -v` 是否可用
 - `Gemfile` 是否已经更新
5. 如果站点越来越复杂, 建议保持:
 - 本地先测试
 - 再 push
 - 最后再看 GitHub Actions 与线上页面

► 十四、最终总结

这次关于 Ruby 安装与 Jekyll 本地预览的流程, 最终明确了以下几点:

1. Windows 下必须先有正确的 Ruby 环境, 才能运行 Bundler 和 Jekyll。
2. `bundle install` 报错, 优先排查 Ruby / Bundler 环境, 而不是先怀疑项目代码。
3. 本地预览的核心命令是:

命令示例 · bat

```
bundle exec jekyll serve
```

1. 成功启动的关键标志是看到本地服务器地址, 例如:

代码清单 A.43.42

语言: text; 完整代码见附录第 488 页, 共 1 行。

1. 普通页面内容修改通常不需要重启, 但配置、插件、`Gemfile` 修改建议重启。
2. 本地能跑通, 是后续排查 GitHub Pages 问题的前提。

这部分流程虽然属于环境准备, 但它实际上决定了后续所有博客修改、分页调试、页面优化是否能够高效进行。

6.5 (原创自研) 网站维护记录 4-更新结束

日期: 2026-03-27

在线原文: marx2001.github.io

► Jekyll 页面与分页样式修复补充总结 (承接上一份 md 之后)

► 1. 本阶段目标

这一阶段主要围绕 Jekyll 博客的两个方向继续修复:

1. 移动端导航栏与页面排版问题
2. 首页 / 分类页 / 文章页的分页 (上一页、下一页) 样式、文字和分隔线问题

本总结承接上一份 md 文件之后的全部对话内容, 重点记录:

- 修改过的文件
 - 每一步的判断依据
 - 试错过程
 - 最终确认的经验性结论
-

► 2. 本阶段涉及的主要文件

► 2.1 布局 / include 文件

- `_layouts/home.html`
- `_layouts/post.html`
- `autopage_category.html`
- `_includes/navbar.html`
- `_includes/footer.html`
- `_includes/head.html`

► 2.2 样式文件

- `main.scss`
 - `_navbar.scss`
 - 以及若干 `_sass` 文件 (如 `_post.scss`、`_global.scss`、`_masthead.scss` 等, 主要用于排查)
-

► 3. 移动端问题排查过程

► 3.1 首轮判断: 不是 viewport 缺失, 而是导航栏和样式层冲突

最开始排查移动端异常时, 先看了:

- `head.html`
- `navbar.html`
- `home.html`
- `autopage_category.html`
- `main.scss`
- `_navbar.scss`

► 结论

`head.html` 中 `viewport` 已经存在, 因此 “整站完全不响应式” 并不是因为忘写:

代码清单 A.44.1

语言: html; 完整代码见附录第 489 页, 共 1 行。

真正的问题主要集中在两点:

1. 导航栏移动端折叠菜单逻辑被自定义 flex 样式破坏
2. 首页 / 分类页文章列表样式写在 **#posts-container** 下, 但模板最初并没有这个容器, 导致一部分响应式样式根本没命中

► 3.2 #posts-container 问题

在 main.scss 中, 首页和分类页很多样式都绑定在:

代码清单 A.44.2

语言: scss; 完整代码见附录第 489 页, 共 1 行。

但最开始的 `home.html` 和 `autopage_category.html` 文章列表结构没有统一包在 `#posts-container` 下。

► 结果

- 某些首页 / 分类页的移动端字号、间距、分页样式没有真正生效
- 这不是唯一 bug, 但属于一类“样式写了却没命中模板”的问题

► 后续修正

首页和分类页模板都改成了带有:

代码清单 A.44.3

语言: html; 完整代码见附录第 489 页, 共 3 行。

的结构。

► 3.3 移动端 menu 按钮“收回一瞬间又弹开”的问题

这是本阶段最典型的导航栏 bug。

► 现象

- 点击 Menu
- 导航似乎先收起
- 但马上又重新下拉, 继续遮挡标题和背景图

► 根因判断

问题不是 JS 失效, 而是 **Bootstrap** 的 **collapse** 逻辑被自定义 CSS 顶掉。

核心原因包括:

- `.navbar-collapse` 在移动端也被强行写成 `display: flex`
- 导致 `.collapse.show / .collapse:not(.show)` 的默认显示隐藏逻辑失效
- 结果就是按钮点击时, JS 改变了 class, 但 CSS 又把它硬显示出来

► 对应修正思路

后续改动集中在:

- `_navbar.scss`
- `navbar.html`

主要原则是:

- 移动端不再强制 `.navbar-collapse` 使用 `flex`
- 桌面端再启用 `flex` 布局
- Menu 按钮放到右上角, 而不是居中

► 3.4 导航栏结构与桌面端居中效果的平衡

中间曾尝试过“标准 Bootstrap 结构优先”的方案, 这能解决移动端折叠问题, 但会改变 PC 端导航居中视觉。

► 经验结论

- 单纯用标准 Bootstrap 结构, 功能上更稳, 但会改变原来桌面端的居中感
- 如果想兼顾:
 - PC 端: 继续保持整体居中
 - 移动端: Menu 固定右上角, 正常折叠

就需要分开写桌面端和移动端样式, 而不是共用一套布局策略

► 最终判断

在实际部署中, 之前某一版 `_navbar.scss` 已经足够可用, 因此后续没有继续大动 `_navbar.scss`, 而是把重点转向分页和模板统一。

► 4. 分页 (上一页 / 下一页) 问题的排查与修复

这一部分是本阶段讨论最多、也最容易反复的内容。

► 4.1 首页 / 分类页分页最开始的问题

主要有几类:

1. PC 端分页按钮黏在一起, 不够美观
2. 按钮上方出现双线
3. 移动端按钮上下排布过长, 比例不好看
4. 中文 / 英文混用, 带箭头, 不统一
5. 首页、分类页、文章内部的分页逻辑和样式不统一

► 4.2 首页与分类页分页结构的统一

为了让 `main.scss` 中的分页样式能真正命中, 后续统一采用类似这样的结构:

代码清单 A.44.4

语言: `html`; 完整代码见附录第 489 页, 共 13 行。

► 这样做的原因

原始模板里分页通常还是:

- `clearfix`
- `float-left`
- `float-right`

而后面写的 CSS 已经转向:

- `.pagination-btn`
- `.pagination-prev`
- `.pagination-next`
- `.pagination-placeholder`

如果 HTML 不跟着统一, 后面继续补 CSS 往往“看起来没效果”。

► 4.3 PC 端分页按钮间距和左右分布

► 初始问题

PC 端“上一页 / 下一页”按钮:

- 间距不够大
- 看起来像粘在一起
- 不能很好地和内容区宽度对齐

► 试过的方案

1. 改按钮宽度
2. `flex + justify-content: space-between`
3. `margin-left: auto / margin-right: auto`
4. 单独推右按钮

► 经验结论

- 仅改按钮宽度不能真正解决“对称分布”的问题
- 如果 HTML 结构仍是旧的 `float-left` / `float-right`, 后面很多 `flex` 规则并不会达到预期
- 真正要稳定, 需要:
 - 容器结构统一
 - 分页按钮类名统一
 - 尽量避免 `clearfix` 伪元素继续干扰 `flex`

► 4.4 clearfix 对分页布局的干扰

后期通过现象判断出:

代码清单 A.44.5

语言: `html`; 完整代码见附录第 489 页, 共 1 行。

这种写法在使用 `flex` 布局时可能引入伪元素 `::before` / `::after`, 干扰分页按钮的对齐。

► 对应经验

如果分页容器本身已经用 `flex`, `clearfix` 一般没必要继续保留。

中间也尝试通过 CSS 关闭:

代码清单 A.44.6

语言: `scss`; 完整代码见附录第 489 页, 共 5 行。

来抑制这类干扰。

► 4.5 中文化与去箭头

后续统一要求:

- 首页分页: 上一页 / 下一页
- 分类页分页: 上一页 / 下一页
- 文章内部上一篇下一篇: 上一篇 / 下一篇
- 去掉左右箭头

► 涉及文件

- `home.html`
- `autopage_category.html`
- `post.html`

► 经验结论

分页文字问题不要试图只用 CSS 解决, 必须回到模板里直接改文本。

► 4.6 移动端分页问题

► 现象

移动端曾出现:

- 上一页和下一页上下堆叠
- 按钮太长, 像横条
- 和 PC 端风格不一致

► 原因

`main.scss` 中旧的移动端分页规则把它写成了:

代码清单 A.44.7

语言: scss; 完整代码见附录第 490 页, 共 3 行。

► 经验结论

如果想让移动端也尽量接近 PC 端风格, 应避免这类“强行纵向堆叠 + 拉满宽度”的旧规则。但由于样式中重复的 `#pagination` 规则很多, 单纯在末尾追加一段覆盖规则, 有时效果并不稳定。

因此后面反复强调的一点是:

仅追加 CSS 不一定能解决问题, 必须先保证模板结构和目标样式是同一套体系。

► 5. 双线问题的排查过程

这部分是本阶段最典型的“必须靠开发者工具定位来源”的问题。

► 5.1 最初判断失误: 以为两根线都来自分页

一开始尝试通过:

- 去掉 `#pagination` 的 `border-top`
- 去掉最后一个 `<hr>`
- 调整 `top: -1px`

等方法处理, 但效果不稳定, 说明两根线来源并不单一。

► 5.2 开发者工具定位结果

后续通过开发者工具逐步确认:

► (1) footer 前确实存在单独的 `<hr>`

这会在页面更下方形成一根线。

► (2) 分页区域本身存在 `border-top`

这也是一根线。

► (3) 分类页模板中, 文章循环后仍然无条件输出 `<hr>`

也就是说, 最后一篇文章后面也会有一根线。

最终就出现了“三个来源可能混在一起”的情况:

1. 文章列表最后一个 `<hr>`
2. `#pagination` 的 `border-top`
3. footer 前单独的 `<hr>`

► 5.3 经验结论: 要先分清是哪一根线

开发者工具最终告诉我们的核心经验是:

- footer 的 `<hr>` 是底部那根线
- 分页按钮上方那组线, 不是 footer 的线
- 分页按钮上方如果出现双线, 通常是:
 - 最后一篇文章后的 `<hr>`
 - 加上 `#pagination` 的 `border-top`

因此处理方式不是“乱删 CSS”, 而是:

- 如果要保留分页自己的线
 - 就应该让最后一篇文章后不再输出 `<hr>`
- footer 前那根线要不要保留
 - 是另一个独立决定

► 5.4 分类页最终正确改法

`autopage_category.html` 中原来是:

代码清单 A.44.8

语言: liquid; 完整代码见附录第 490 页, 共 4 行。

最终应改成:

代码清单 A.44.9

语言: liquid; 完整代码见附录第 490 页, 共 6 行。

► 经验意义

这说明很多“看起来像 CSS 问题”的视觉 bug, 其实根源是模板中多输出了结构性元素。

► 6. 首页与分类页不一致的问题

后期一个重要发现是:

- `home.html`
- `autopage_category.html`

虽然看起来类似, 但细节上常常不一致:

- 有的有 `#pagination`
- 有的没有
- 有的文章循环后用 `unless forloop.last`
- 有的还是无条件 `<hr>`
- 有的分页按钮类名统一了
- 有的还残留旧类

► 经验结论

首页和分类页如果想统一体验, 不能只“看起来差不多”, 而要把:

- 容器
- 循环中的 `<hr>`
- 分页按钮类名
- 分页文字

全部对齐。

► 7. 本阶段试错经验总结

► 7.1 仅在 `main.scss` 末尾叠加覆盖规则, 不一定可靠

原因包括:

- 旧规则重复太多
- 模板结构没跟着改
- 旧类名与新类名不匹配
- 伪元素或 `float` 逻辑仍在干扰

► 7.2 先确认“是 CSS 命中问题, 还是模板结构问题”

这次很多问题最终都证明:

- 不是 SCSS 语法不对
- 而是 HTML 模板结构没跟着改

典型例子:

- `#posts-container` 没加, 导致样式没命中
- 分页按钮没用 `pagination-next / pagination-prev` 类, 导致后加规则没用
- 分类页仍无条件输出 `<hr>`, 导致双线一直存在

► 7.3 用浏览器开发者工具比猜 CSS 更有效

尤其是这几类问题:

- 哪根线来自哪里
- 当前页面到底命中了哪个规则
- footer 的 `<hr>` 还是 pagination 的 `border-top`
- 选中的元素到底是 `hr`、`div#pagination` 还是伪元素

这是本阶段一个非常重要的经验。

► 7.4 不要一次性大改所有文件

对导航栏、分页、首页、分类页、文章页这种多模板问题, 最好:

- 一次只修一个场景
- 一次只验证一个页面
- 先定位来源, 再改模板或样式

否则会出现“这里修好了, 那里又坏了”的反复情况。

► 8. 当前阶段的稳定结论

截至这一阶段, 已经比较明确的结论有:

1. 移动端导航栏的主问题不是 `viewport`, 而是 `collapse` 与自定义 `flex` 冲突
 2. `#posts-container` 是否存在, 直接影响首页 / 分类页样式是否命中
 3. 分页问题不能只靠 CSS 叠加, 必须统一 HTML 结构
 4. 分页按钮中文化与去箭头要在模板中改
 5. 分类页分页上方双线的核心来源, 是最后一篇文章后无条件输出 `<hr>`
 6. footer 前的 `<hr>` 是另一根独立横线, 不应与分页双线混为一谈
 7. 开发者工具是定位这类视觉问题的关键工具
-

► 9. 建议后续维护方式

后面如果继续维护这个博客, 建议:

► 9.1 统一模板规范

尤其是:

- 首页
- 分类页
- 文章页

把分页区、文章循环、`<hr>` 输出逻辑都统一。

► 9.2 main.scss 分区管理

把分页相关样式集中到一个单独区域, 不要在文件不同位置重复写多个 `#pagination`。

► 9.3 每次修改后立刻用 devtools 验证

重点看:

- HTML 结构是否真变了
- 当前命中的 CSS 规则是哪一条
- 是否是缓存问题

► 9.4 避免“补丁式叠加”

当样式重复太多时, 继续加覆盖规则只会越来越乱。
与其补丁, 不如收敛成一份明确的最终结构。

► 10. 本阶段最有价值的经验性信息

如果只提炼最关键的经验, 有以下几点:

1. Jekyll 页面显示问题, 很多时候不是 SCSS 写错, 而是模板结构和样式目标不一致
2. 导航栏、分页、分隔线这类问题, CSS 和模板必须配套修改
3. 看到多条线时, 不要先删 CSS, 要先用开发者工具确认每一根线的真实来源
4. 移动端与 PC 端可以共用主题, 但不应强行共用完全相同的布局机制
5. 调试时先定位“当前页面到底用了哪个模板、命中了哪条规则”, 比盲改更重要

6.6 (原创自研) 网站维护记录 5-搜索功能与界面更新

日期: 2026-03-28

在线原文: marx2001.github.io

这篇文章记录了博客升级搜索功能和代码复制功能的过程, 界面更清晰美观, 更加专业, 以及更加方便。两类更新:

1. 如何把 Jekyll 站内搜索从“能用”改成“标题 + 正文尽量不漏召回”
2. 如何把 代码块 + 复制按钮 做成更稳定、更好维护的版本

对于 Jekyll、GitHub Pages 或类似的静态博客, 本文内容可作为直接参考。

► 一、这次更新要解决什么问题

最开始遇到的不是单一问题, 而是一串互相关联的小问题:

► 1. 搜索功能不够稳

已有搜索功能虽然能用, 但存在几个明显短板:

- 搜索标题和正文时容易漏结果
- 中文、多词组合、特殊写法不够稳

- 文章多了以后, 搜索开始变慢
- 技术文章里的大段代码会污染搜索索引

► 2. 代码块和复制按钮体验一般

代码块部分也有多个问题:

- 代码块样式偏呆板
- 复制按钮视觉不统一
- 代码块一开始出现了双层灰背景
- 部分按钮存在白色残留背景
- 点击复制后, 对勾状态不生效
- 滚动代码时, 按钮会跟着内容一起跑, 体验不好

这次更新的核心, 不是单纯“改个颜色”, 而是把这两块都梳理成更清晰、可维护的方案。

► 二、搜索功能: 目标不是复杂, 而是“尽量不漏”

这次搜索优化一开始就明确了目标:

不追求复杂筛选, 不追求推荐系统, 只追求“标题 + 正文尽量不漏召回”。

这句话很重要, 因为它决定了后面的技术路线。

如果目标是:

- 搜索尽量稳
- 尽量少漏文章
- 适合静态博客

那么继续使用 前端静态搜索 是合理的, 不需要为了这个需求专门上后端搜索服务。

► 三、原有搜索方案的判断

原本博客的搜索方案属于典型的静态站点前端搜索:

- 页面: `search.html`
- 索引: `search.json`
- 检索逻辑: `search.js`
- 底层依赖: `lunr.js`

这种方案的优点很明显:

- 部署简单
- GitHub Pages 友好
- 不依赖服务器
- 小博客足够轻量

但它的缺点也明显:

- 直接 `idx.search(q)` 的检索方式过于朴素
- 一旦文章多起来, `search.json` 会越来越大

- 技术博客里的代码块会干扰正文搜索

► 四、搜索功能是怎么改的

► 1. 查询逻辑不再只搜整句

如果用户输入一个较长的短语, 或者中英文混合关键词, 直接整句搜索很容易漏。

因此搜索逻辑后来改成了更宽松的模式:

- 整句搜索
- 拆词搜索
- 前缀匹配
- 原始字符串包含兜底

这里最关键的是最后一层:

► 2. 加入“原始字符串包含”兜底

这是这次搜索升级里最实用的一步。

思路很简单:

- 标题里直接包含关键词 → 强召回
- 正文里直接包含关键词 → 兜底召回

它不一定最优雅, 但对“尽量不漏文章”的目标非常有效。

► 3. 标题命中优先

在排序上, 标题命中的文章会优先排到前面, 这样搜索结果更符合日常直觉。

► 五、搜索性能为什么会变差

当文章数量慢慢变多后, 真正的性能瓶颈并不在页面本身, 而在两个地方:

► 1. search.json 越来越大

原始做法通常是把整篇文章内容都放进去, 例如:

代码清单 A.45.1

语言: liquid; 完整代码见附录第 490 页, 共 1 行。

这样做的后果是:

- 文件体积越来越大
- 首次加载搜索页就很重
- 前端建索引时间越来越长

► 2. 页面一打开就立刻加载索引

这意味着用户即使只是打开搜索页, 还没开始搜索, 也要先下载并构建整份索引。

► 六、搜索性能是怎么优化的

► 1. 正文改为截断, 不再塞整篇全文

例如改成:

代码清单 A.45.2

语言: liquid; 完整代码见附录第 490 页, 共 1 行。

这样做的意义是:

- 索引变小
- 首次加载更快
- 对大多数文章来说, 前几段正文已经足够支持搜索

► 2. 改成延迟加载索引

搜索逻辑后来改成:

- 页面打开时不立刻建索引
- 用户开始输入后, 再加载 `search.json`
- 只构建一次, 后续复用

这种方式对于文章较多的博客非常有效。

► 3. 控制结果数

例如限制:

短代码 · javascript

```
MAX_RESULTS = 50
```

因为搜索结果不需要一次渲染几百条, 限制显示数量可以减少页面渲染负担。

► 七、技术博客里的代码块, 为什么会污染搜索

这个问题是这次搜索优化里一个很关键的发现。

技术博客的文章经常包含大段代码, 而在 `Jekyll` 里, 代码块经过渲染后依然会进入 `post.content`。

这意味着:

- 代码内容会进入 `search.json`
- 代码字符会占用正文截断长度
- 如果文章前面代码很多, 真正的正文就会被截掉

结果就是:

- 搜索结果变差
- 搜索更容易命中代码而不是正文
- 有些文章明明正文里有关键词, 却因为正文没进索引而搜不到

► 八、如何在 `search.json` 里跳过代码块

正确做法不是在前端再删除代码, 而是在生成 `search.json` 的时候就先删掉代码块。

思路是:

1. 先把 `<pre>...</pre>` 对应内容从文章 HTML 中剔除
2. 再 `strip_html`
3. 再做 `truncate`

这样做的好处非常直接:

- 代码不进入搜索索引
- 截断长度留给真正正文
- 搜索质量明显更好

对于技术博客来说, 这是非常值得做的一步。

► 九、代码块样式部分: 为什么主要改 `main.scss`

代码块视觉和复制按钮的样式问题, 本质上首先是 CSS 问题。

所以一开始的判断是正确的:

大部分视觉问题优先改 `main.scss`, 不要一开始就怀疑 HTML 结构。

后续的实践也证明了这一点:

- 背景色
- 圆角
- 按钮视觉
- tooltip
- 按钮位置
- 图标比例

这些主要都靠 `main.scss` 完成。

► 十、复制按钮的类名是怎么确认的

通过开发者工具排查后, 确认页面里的按钮类名是:

代码清单 A.45.4

语言: `text`; 完整代码见附录第 490 页, 共 1 行。

这个确认很重要, 因为如果类名都没对上, 后面所有样式覆盖都容易失效。

此外还发现,旧按钮曾经带有一套内联样式,这也是后来产生冲突的根源之一。

► 十一、双层灰背景是怎么出现的

一开始代码块样式改动后,页面上出现了“双层灰背景”:

- 外层容器一层灰
- 内层 `pre` 又一层灰

最后发现,正确的职责划分应该是:

► 外层容器

只负责:

- 定位
- 容纳按钮

► `pre`

负责:

- 真正的代码块背景
- 圆角
- 边框
- 阴影

因此后来做法变成:

- 外层背景透明
- 只有 `pre` 保留那层浅灰背景

这样视觉上才干净。

► 十二、复制按钮为什么一开始会有白边残留

这个问题一开始看起来像单纯的 CSS 边框问题,但实际更复杂。

原因不是只有一个,而是几种情况叠加:

- 页面里存在不同来源的按钮
- 有的按钮是旧脚本生成的
- 有的按钮被额外容器包裹
- 有的按钮还带旧的 `inline style`

这说明:

当同一个功能由两套脚本或两套结构同时控制时,CSS 问题会变得非常难判断。

所以真正的解决方法,不是一直补选择器,而是要先消除旧逻辑冲突。

▶ 十三、为什么要把按钮改成图标化

原来的“复制代码”纯文字按钮虽然可用,但有几个问题:

- 占空间
- 风格不够简洁
- 容易和代码块整体视觉不协调

因此后来改成了:

- 默认显示复制图标
- 鼠标悬停时显示 tooltip “复制”
- 点击成功后图标短暂变成对勾

这个交互形式更紧凑,也更符合现在常见的代码块工具按钮风格。

▶ 十四、为什么最后选择“纯 CSS 画图标”

中间一度尝试过:

- 内嵌 SVG
- data URI 图标

但在静态博客环境下,这种方式容易遇到:

- SCSS 编译转义问题
- 引号 / 特殊字符问题
- data URI 在不同环境下不稳定

最终改成:

直接用纯 CSS 画复制图标和对勾。

这样虽然实现上略微原始,但非常稳:

- 不依赖外部图片
- 不依赖 SVG 文件
- 不受资源路径影响
- 更容易微调比例

▶ 十五、为什么“复制成功后变对勾”一开始总是不成功

这部分是本次更新中最典型的排错案例。

▶ 第一步误判: 以为是 CSS 对勾没画对

一开始大家很自然会想到:

- 对勾位置错了
- `.copied::before` 没覆盖干净
- 默认图标还残留

这些判断不完全错,但都不是根因。

▶ 第二步发现: 点击后 class 根本没变

继续排查后发现:

- 点击后按钮并没有短暂变成 `copy-btn copied`

这说明真正的问题不是“图标画错”, 而是:

JS 根本没给按钮加 `.copied` 类。

▶ 第三步发现: 页面里有旧复制脚本

再继续查, 发现 `head.html` 里原来存在一段旧的内联脚本。它会:

- 自己创建 `.copy-btn`
- 自己写按钮样式
- 点击后直接把文字改成“已复制”

这样一来, 新脚本虽然存在, 但用户实际点到的按钮并不一定受新逻辑控制。

▶ 第四步发现: 旧脚本删掉后按钮直接消失

删掉旧脚本后, 按钮完全消失, 于是又怀疑“是不是按钮根本没创建”。

最后通过 Network 查到:

`/assets/js/copy-code.js` 是 404。

这说明问题其实更基础:

- 新脚本根本没加载到页面
- 那当然不会创建按钮
- 也不会加 `.copied`

这个案例非常典型:

表面看起来像 CSS 问题, 实际是旧脚本冲突 + 新脚本路径错误。

▶ 十六、复制按钮脚本最后是怎么修好的

▶ 1. 修正文件路径

最终确认 `copy-code.js` 必须放在:

代码清单 A.45.5

语言: text; 完整代码见附录第 491 页, 共 1 行。

然后在文章页布局中加载:

代码清单 A.45.6

语言: html; 完整代码见附录第 491 页, 共 1 行。

修正路径后, 脚本成功执行。

► 2. 删除旧脚本

head.html 中旧的内联复制脚本彻底删除, 只保留一套复制逻辑。

这一步非常重要, 否则:

- 样式会互相打架
- 点击行为会互相干扰
- 很难判断当前按钮到底由谁生成

► 3. 新脚本负责完整流程

新的 copy-code.js 负责:

- 遍历所有 pre
- 创建 .copy-btn
- 绑定点击复制事件
- 复制成功后添加 .copied
- 1 秒后移除 .copied

这样 JS 和 CSS 的职责才对应起来。

► 十七、为什么复制按钮会随着代码滚动而消失

这是后期又出现的一个交互问题。

原因很直接:

- 按钮一开始是 appendChild 到 pre 里面的
- 而 pre 正好是滚动容器
- 所以滚动代码时, 按钮也会跟着代码内容一起移动

这就导致用户滚到代码中下部时, 右上角复制按钮不见了。

► 十八、如何让按钮始终固定在代码块右上角

最终做法是:

- 不再 pre.appendChild(button)
- 改成找到外层 wrapper:
 - .highlighter-rouge
 - .highlight
 - figure.highlight
- 把按钮 append 到 wrapper 上

这样就形成了更合理的结构:

► pre

负责:

- 滚动代码内容

► wrapper

负责:

- 定位复制按钮

这样无论代码怎么滚, 按钮都固定在外层右上角, 不再消失。

► 十九、为什么后来改成“图标与圆心绑定”

如果按钮图标靠写死坐标定位, 例如:

- `top: 12px`
- `left: 12px`

那后面一旦改按钮大小或位置, 就要一起重调图标, 维护非常麻烦。

因此后来改成:

- 图标 `left: 50%`
- 图标 `top: 50%`
- 再通过 `transform: translate(...)` 细调

这样做之后:

- 调按钮位置 → 只改 `.copy-btn` 的 `top/right`
- 调按钮大小 → 只改 `.copy-btn` 的 `width/height`
- 图标自动跟着圆心走

这是一种非常值得保留的写法。

► 二十、这次更新后各文件的职责

► main.scss

负责:

- 代码块视觉
- 行内代码样式
- 滚动条样式
- 复制按钮视觉
- tooltip 样式
- `.copied` 对勾状态

► assets/js/copy-code.js

负责:

- 查找代码块
- 创建复制按钮
- 绑定复制事件
- 控制 `.copied` 状态
- 确保按钮插入到正确外层容器

► `post.html`

负责:

- 在文章页底部加载 `copy-code.js`

► `head.html`

当前只保留:

- `highlight.js` 资源
- 基础页面资源

旧的复制按钮脚本已经删除。

► `search.js`

负责:

- 标题 + 正文宽松召回
- 整句 / 拆词 / 前缀 / 兜底匹配

► `search.json`

负责:

- 生成搜索索引
- 跳过代码块
- 截断正文长度
- 控制索引体积

► 二十一、这次更新里最值得记住的经验

► 1. 不要同时保留两套同功能脚本

复制按钮的问题之所以会越来越乱, 根本原因就是旧脚本和新脚本同时存在。

经验很简单:

一个功能, 只保留一套逻辑来源。

► 2. 资源是否成功加载, 必须优先确认

这次 `copy-code.js` 404 非常典型。

如果没先查 Network, 很容易在 CSS 和 DOM 上浪费大量时间。

经验顺序应该是:

1. 先看 Network
2. 再看 Console
3. 再查 DOM
4. 最后才是 CSS

► 3. 技术博客的搜索索引不能无脑塞全文

技术文章里代码很多, 如果全文直接进索引:

- 会拖慢搜索
- 会污染召回结果
- 会挤掉正文有效信息

因此删除代码块后再生成索引, 是非常值得做的一步。

► 4. 图标与圆心绑定后, 维护成本明显降低

这是本次样式调整中很有价值的一个结果。

后续无论继续调比例、位置还是按钮大小, 都会轻松很多。

► 二十二、这次更新完成后达成了什么

► 搜索部分

- 搜索目标明确: 标题 + 正文尽量不漏
- 召回逻辑更稳
- 性能更可控
- 代码块不再污染搜索索引

► 代码块部分

- 代码块视觉更统一
 - 复制按钮图标化
 - tooltip 更自然
 - 复制成功有明确反馈
 - 按钮不再随着代码滚动消失
 - 后续样式微调成本显著降低
-

► 二十三、后续仍可继续做的优化

► 搜索部分

- 增加分类搜索
- 为文章增加 `search_content` 字段
- 优化高亮摘要截取逻辑

► 代码块部分

- 继续细调按钮比例
- 分别优化桌面端与移动端
- 如果想进一步统一风格,可以继续往 GitHub / macOS 风格靠近

► 二十四、最终结论

这次更新表面上看是两件小事:

- 改搜索
- 改复制按钮

但实际涉及的内容很完整:

- 搜索逻辑
- 索引体积控制
- 技术博客中的代码块搜索污染问题
- CSS 视觉层次整理
- JS 事件绑定与状态切换
- 静态资源路径排查
- 页面布局中的脚本注入方式

最终最大的收获不是某一段代码,而是一套更清晰的维护方法:

功能逻辑尽量单一来源,样式结构尽量职责分离,问题排查按照“资源加载 → JS 行为 → DOM 结构 → CSS 视觉”逐层推进。

这套思路,后面再改博客其他功能时也会继续有用。

6.7 (建模流程)Materials Studio 2023 手动搭建 Cs3V9Te13 晶体结构

日期: 2026-07-08

在线原文: marx2001.github.io

► 说明

本文记录如何在 **Materials Studio 2023** 中, 根据文献给出的空间群、晶格参数和非等价原子分数坐标, 手动搭建块体材料 **Cs₃V₉Te₁₃** 的晶体结构。

核心思想是:

代码清单 A.46.1

语言: text; 完整代码见附录第 491 页, 共 1 行。

不要根据论文中的结构示意图手动画原子, 也不要直接把非等价位点当成完整 POSCAR。对于 Cs₃V₉Te₁₃, 文献给出的 6 个原子坐标是 **asymmetric unit / 非等价原子位点**, 需要通过空间群 **P -6 2 m, No. 189** 自动展开, 最终得到完整的 25 原子原胞。

► 一、需要准备的结构信息

► 1. 晶格和空间群

代码清单 A.46.2

语言: text; 完整代码见附录第 491 页, 共 11 行。

注意:

代码清单 A.46.3

语言: text; 完整代码见附录第 491 页, 共 2 行。

在 Materials Studio 中, 如果通过符号不好找空间群, 建议直接输入空间群编号:

代码清单 A.46.4

语言: text; 完整代码见附录第 491 页, 共 1 行。

只要软件显示的空间群编号是 **189**, 就是正确的。

► 2. 非等价原子分数坐标

在 Materials Studio 里输入的是 **fractional coordinates / 分数坐标**, 不是 Cartesian coordinates / 笛卡尔坐标。

位点	Element	Name	a/x	b/y	c/z	展开后数量
Cs	Cs	Cs1	0.0000	0.3632	0.0000	3
V1	V	V1	0.8220	0.5334	0.5000	6
V2	V	V2	0.8469	0.8469	0.5000	3
Te1	Te	Te1	0.0000	0.77119	0.71975	6
Te2	Te	Te2	0.666667	0.333333	0.25213	4
Te3	Te	Te3	0.58986	0.58986	0.5000	3

最终完整原胞应该为:

代码清单 A.46.5

语言: text; 完整代码见附录第 492 页, 共 4 行。

也就是化学式:

代码清单 A.46.6

语言: text; 完整代码见附录第 492 页, 共 1 行。

► 二、Materials Studio 2023 详细建模步骤

► Step 1: 新建一个 3D Atomistic Document

打开 **Materials Studio 2023 Visualizer**。

点击顶部菜单:

代码清单 A.46.7

语言: text; 完整代码见附录第 492 页, 共 1 行。

选择:

代码清单 A.46.8

语言: text; 完整代码见附录第 492 页, 共 1 行。

新建一个空白结构窗口。

► Step 2: 进入 Build Crystal 建晶体窗口

在顶部菜单栏点击:

代码清单 A.46.9

语言: text; 完整代码见附录第 492 页, 共 1 行。

不同安装版本中, 菜单名称可能略有差异, 也可能显示为:

代码清单 A.46.10

语言: text; 完整代码见附录第 492 页, 共 1 行。

或者:

代码清单 A.46.11

语言: text; 完整代码见附录第 492 页, 共 1 行。

应进入可设置 **space group / lattice parameters / asymmetric unit atoms** 的晶体构建窗口。

► Step 3: 设置空间群

在 **Build Crystal** 窗口中找到:

代码清单 A.46.12

语言: text; 完整代码见附录第 493 页, 共 1 行。

设置:

代码清单 A.46.13

语言: text; 完整代码见附录第 493 页, 共 1 行。

确认软件显示类似:

代码清单 A.46.14

语言: text; 完整代码见附录第 493 页, 共 1 行。

或者:

代码清单 A.46.15

语言: text; 完整代码见附录第 493 页, 共 1 行。

这里一定要注意:

代码清单 A.46.16

语言: text; 完整代码见附录第 493 页, 共 2 行。

如果空间群选错, 后面展开出来的原子位置会完全不同。

► Step 4: 设置晶格参数

在 **Lattice Parameters / Cell Parameters** 区域输入:

代码清单 A.46.17

语言: text; 完整代码见附录第 493 页, 共 7 行。

单位默认为 Å。

特别注意:

短代码 · text

gamma = 120°

不要填成 60°。虽然 60° 和 120° 有时可以表示等价的六角晶格, 但为了和文献坐标以及后续 VASP POSCAR 保持一致, 这里应该使用 $\gamma = 120^\circ$ 。

► Step 5: 打开 Add Atoms 窗口

在 **Build Crystal** 主窗口中找到添加原子的按钮, 一般为:

代码清单 A.46.19

语言: text; 完整代码见附录第 493 页, 共 1 行。

或者在菜单中选择:

代码清单 A.46.20

语言: text; 完整代码见附录第 494 页, 共 1 行。

打开后会出现 **Add Atoms** 小窗口。

这个窗口里右侧的:

代码清单 A.46.21

语言: text; 完整代码见附录第 494 页, 共 3 行。

就是原子的分数坐标, 对应常见写法:

代码清单 A.46.22

语言: text; 完整代码见附录第 494 页, 共 1 行。

也就是说:

短代码 · text

```
a = fractional x
b = fractional y
c = fractional z
```

不要把这里的 a、b、c 理解为晶格常数。晶格常数已经在前面的 Lattice Parameters 中设置过了。

► Step 6: 逐个输入 6 个非等价原子位点

每输入一个位点, 就点击一次窗口右下方的:

代码清单 A.46.24

语言: text; 完整代码见附录第 494 页, 共 1 行。

然后再修改元素和坐标, 继续添加下一个位点。

► 1. 添加 Cs 位点

设置:

代码清单 A.46.25

语言: text; 完整代码见附录第 494 页, 共 8 行。

点击:

代码清单 A.46.26

语言: text; 完整代码见附录第 494 页, 共 1 行。

► 2. 添加 V1 位点

设置:

代码清单 A.46.27

语言: text; 完整代码见附录第 494 页, 共 8 行。

点击:

代码清单 A.46.28

语言: text; 完整代码见附录第 495 页, 共 1 行。

注意:

代码清单 A.46.29

语言: text; 完整代码见附录第 495 页, 共 2 行。

不要把元素名称改成 V1, 因为 V1 不是化学元素, 只是不等价 V 位点标签。

► 3. 添加 V2 位点

设置:

代码清单 A.46.30

语言: text; 完整代码见附录第 495 页, 共 8 行。

点击:

代码清单 A.46.31

语言: text; 完整代码见附录第 495 页, 共 1 行。

► 4. 添加 Te1 位点

设置:

代码清单 A.46.32

语言: text; 完整代码见附录第 495 页, 共 8 行。

点击:

代码清单 A.46.33

语言: text; 完整代码见附录第 496 页, 共 1 行。

► 5. 添加 Te₂ 位点

设置:

代码清单 A.46.34

语言: text; 完整代码见附录第 496 页, 共 8 行。

点击:

代码清单 A.46.35

语言: text; 完整代码见附录第 496 页, 共 1 行。

► 6. 添加 Te₃ 位点

设置:

代码清单 A.46.36

语言: text; 完整代码见附录第 496 页, 共 8 行。

点击:

代码清单 A.46.37

语言: text; 完整代码见附录第 496 页, 共 1 行。

► Step 7: 应用空间群并生成完整晶胞

6 个非等价位点全部 Add 完成后, 关闭 **Add Atoms** 小窗口, 回到 **Build Crystal** 主窗口。

在主窗口中点击:

代码清单 A.46.38

语言: text; 完整代码见附录第 496 页, 共 1 行。

或者:

代码清单 A.46.39

语言: text; 完整代码见附录第 497 页, 共 1 行。

或者:

代码清单 A.46.40

语言: text; 完整代码见附录第 497 页, 共 1 行。

不同版本按钮名称略有不同。应选择能够根据当前空间群和非等价位点生成完整晶体结构的按钮。

如果主窗口中有类似选项:

代码清单 A.46.41

语言: text; 完整代码见附录第 497 页, 共 2 行。

需要勾选或点击它。

由于输入的是非等价原子位点, 必须通过空间群 **P -6 2 m, No. 189** 自动展开, 才能得到完整晶胞。

► 三、建完后如何检查是否正确

► 1. 检查总原子数

生成结构后, 查看 composition / atom count。

正确结果应该是:

代码清单 A.46.42

语言: text; 完整代码见附录第 497 页, 共 1 行。

总原子数应该是:

代码清单 A.46.43

语言: text; 完整代码见附录第 497 页, 共 1 行。

如果最终只有 6 个原子, 说明没有应用空间群展开。

如果最终是 50 个、75 个或更多原子, 说明可能重复应用了 symmetry, 或者把完整坐标当成非等价位点再次输入了。

► 2. 检查 V 原子层

正确结构中, 所有 V 原子的分数坐标 z 应该都是:

短代码 · text

z = 0.5000

这说明 V 原子集中在同一个二维层内。

如果 V 原子分布在很多不同 z 层上, 需要检查空间群或坐标是否输入错误。

► 3. 沿 c 轴观察 V 子晶格

切换视角, 沿 c 轴俯视结构。

可以尝试:

代码清单 A.46.45

语言: text; 完整代码见附录第 497 页, 共 1 行。

如果菜单中没有该选项, 也可以手动旋转到 c 轴俯视。

正确的 V 子晶格应该是一个复杂二维网络, 由三角形、四边形和五边形交织构成, 不是普通 kagome 晶格。

► 4. 侧面观察层状结构

沿 a 轴或 b 轴观察结构。

正确结构应该表现为明显层状堆叠, 大致可以理解为:

代码清单 A.46.46

语言: text; 完整代码见附录第 497 页, 共 1 行。

其中 V 原子主要位于中间的 V-Te 层内。

► 四、常见错误与排查

► 错误 1: 空间群选错

正确空间群是:

代码清单 A.46.47

语言: text; 完整代码见附录第 498 页, 共 1 行。

错误情况包括:

代码清单 A.46.48

语言: text; 完整代码见附录第 498 页, 共 2 行。

解决方法:

代码清单 A.46.49

语言: text; 完整代码见附录第 498 页, 共 1 行。

► 错误 2: 把 Add Atoms 里的 a、b、c 当成晶格常数

Add Atoms 窗口中的:

代码清单 A.46.50

语言: text; 完整代码见附录第 498 页, 共 1 行。

是原子的分数坐标, 不是晶格参数。

例如:

短代码 · text

```
a = 0.8220
b = 0.5334
c = 0.5000
```

表示这个 V 原子位于分数坐标:

代码清单 A.46.52

语言: text; 完整代码见附录第 498 页, 共 1 行。

► 错误 3: 坐标类型用成 Cartesian

本流程中的 6 个坐标都是:

代码清单 A.46.53

语言: text; 完整代码见附录第 498 页, 共 1 行。

不是:

代码清单 A.46.54

语言: text; 完整代码见附录第 498 页, 共 1 行。

如果软件提供坐标类型选择, 一定选择 **Fractional**。

► 错误 4: 把 V1、V2 当成元素

正确输入方式:

代码清单 A.46.55

语言: text; 完整代码见附录第 499 页, 共 2 行。

错误输入方式:

代码清单 A.46.56

语言: text; 完整代码见附录第 499 页, 共 1 行。

Te1、Te2、Te3 同理。

正确方式:

代码清单 A.46.57

语言: text; 完整代码见附录第 499 页, 共 2 行。

► 错误 5: 只输入 6 个原子后直接导出

6 个原子只是非等价位点, 不是完整晶胞。

正确流程是:

代码清单 A.46.58

语言: text; 完整代码见附录第 499 页, 共 1 行。

如果没有展开, 后续 VASP 计算结构一定是错的。

► 错误 6: gamma 填成 60°

建议使用:

短代码 · text

```
gamma = 120°
```

不要填成:

短代码 · text

```
gamma = 60°
```

这样可以保持和文献坐标以及 VASP 常用六角晶胞写法一致。

► 五、导出 POSCAR

结构确认无误后, 先保存 Materials Studio 文件:

代码清单 A.46.61

语言: text; 完整代码见附录第 499 页, 共 1 行。

保存为:

代码清单 A.46.62

语言: text; 完整代码见附录第 499 页, 共 1 行。

然后导出 VASP 文件:

代码清单 A.46.63

语言: text; 完整代码见附录第 499 页, 共 1 行。

文件类型选择:

短代码 · text

```
VASP POSCAR
```

或者:

短代码 · text

VASP 5

导出后用文本编辑器打开 POSCAR, 检查元素和数量。

推荐最终 POSCAR 中的元素顺序为:

代码清单 A.46.66

语言: text; 完整代码见附录第 500 页, 共 3 行。

如果 Materials Studio 导出的元素顺序不是这个, 比如:

代码清单 A.46.67

语言: text; 完整代码见附录第 500 页, 共 1 行。

本身不是错误, 但后续写 MAGMOM 时必须严格按照 POSCAR 中的真实原子顺序设置。

▶ 六、用于磁性计算的额外记录

对于后续 DFT 磁性、J 参数、DMI 计算, 需要特别记录 V 原子的分组。

这个结构中有两类不等价 V 位点:

代码清单 A.46.68

语言: text; 完整代码见附录第 500 页, 共 2 行。

文献中也区分 V1 和 V2。后续设置磁构型时, 不建议简单把所有 V 完全等同处理。

如果导出 POSCAR 后原子顺序不清楚, 可以在 Materials Studio 中通过坐标检查:

代码清单 A.46.69

语言: text; 完整代码见附录第 500 页, 共 2 行。

所有与 V1 对称等价的 V 原子属于 V1 组, 所有与 V2 对称等价的 V 原子属于 V2 组。

▶ 七、最简操作清单

代码清单 A.46.70

语言: text; 完整代码见附录第 500 页, 共 27 行。

▶ 八、正确模型的判断标准

最终模型必须同时满足:

代码清单 A.46.71

语言: text; 完整代码见附录第 501 页, 共 7 行。

只要这些条件都满足, 这个 Materials Studio 模型就基本正确, 可以用于后续导出 POSCAR 并进行 VASP 结构优化、磁性计算、J 参数或 DMI 计算。

第三部分

论文使用代码

论文最终代码与结果复现

汇集论文最终代码、原创研究程序、高通量自动化、紧束缚模型、超超交换验证与机器学习相界搜索, 突出成果复现和研究创新。

本专题收录 6 篇文章。

7.1 FES 体系自旋陈数、金属性阻碍与相界代码

日期: 2026-09-09

在线原文: marx2001.github.io

本文归档 FES 八带模型从五维 Sobol 采样、严格拓扑标注到 Γ /M 质量、kp Chern 公式、符号 Jacobian、间接带隙 no-go 和论文相图的完整代码。

► 使用范围与科研口径

- 非零自旋 Chern 数不自动意味着量子自旋霍尔绝缘体。
- 论文代表点 rs-rd_i0031_j0044 的直接带隙为 3.49 meV, 但间接带隙为 -2.94 meV, 因此属于 spin-Chern band metal。
- 间接带隙 no-go 是 B+ partner-switching 分支上的精确条件结论; 有限扫描覆盖不能被夸大成未经限定的全参数空间定理。
- Fig. 5 的 FES 图表示含体态载流子的相反自旋横向偏转, 不表示孤立的边界态通道。

► 代码执行顺序

1. 运行 FES_step01_v2.py, 审计 Hamiltonian、周期规范与标签器。2. 运行 Step 02 全局 Sobol 扫描, 再运行 Step 04 对相界和候选拓扑点加密。3. 依次运行 Steps 05-07, 完成解析能级/no-go、kp Chern 公式和符号 Jacobian 证据链。4. 运行基础质量平面后, 再用 fes_uB_phase_map_with_random_region_circles.py 生成 Fig. 3 最终组合坐标版本。

► 迁移说明

代码块保存原始完整源码, 并用 SHA-256 固定版本。绝对路径作为溯源信息保留; 移动到新项目后应统一修改项目根路径。大型 CSV、NPZ、模型和图片需要另行归档。只作为同名 Python 主程序入口的 Notebook 不重复收录。

► 完整代码

► FES_step01_v2.py

用途: 八带 Hamiltonian、周期规范、全 BZ 带隙和自旋 Chern 基础实现

原始路径: D:\1_ML\fes\FES_step01_v2.py

SHA-256: 48e9f1a9a8afac0781bbff7a72fa4fee9575b446eca1ebe5b85e831212534219

```
{% raw %}
```

代码清单 A.47.1

语言: python; 完整代码见附录第 501 页, 共 1175 行。

```
{% endraw %}
```

► FES_step02_sobol_scan.py

用途: 五维 Sobol 采样与严格拓扑分类

原始路径: D:\1_ML\fes\FES_step02_sobol_scan.py

SHA-256: 8813c3577c36995648a13aef0ebb7bfaa0773c2a0d1f1996f3d01d18d69e74ef

```
{% raw %}
```

代码清单 A.47.2

语言: python; 完整代码见附录第 518 页, 共 1450 行。

```
{% endraw %}
```

► FES_step04_phase_boundary_scan.py

用途: 相界加密、严格拓扑点和金属性检查

原始路径: D:\1_ML\fes\FES_step04_phase_boundary_scan.py

SHA-256: a4f7b1d0e4e7edbbbc7d8bba8d1866c7bcc897c77d5ab66168346fd51758425c7

```
{% raw %}
```

代码清单 A.47.3

语言: python; 完整代码见附录第 539 页, 共 1008 行。

```
{% endraw %}
```

► FES_step05_analytic_no_go.py

用途: Gamma/M 解析本征值、partner switching 和间接带隙上界

原始路径: D:\1_ML\fes\FES_step05_analytic_no_go.py

SHA-256: 6a436e7385fa0d6f83909a46ca609c9277caccd1231c19e8716cb73b174d2697

```
{% raw %}
```

代码清单 A.47.4

语言：python；完整代码见附录第 554 页，共 937 行。

```
{% endraw %}
```

► FES_step06_kp_chern_formula.py

用途：谷手性与自旋 Chern 闭式公式验证

原始路径：D:\1_ML\fes\FES_step06_kp_chern_formula.py

SHA-256: b562a60d610b461927a19f349d36a43304225627e37810bebcfc59df5636bb86

```
{% raw %}
```

代码清单 A.47.5

语言：python；完整代码见附录第 568 页，共 1226 行。

```
{% endraw %}
```

► FES_step07_symbolic_jacobian_no_go.py

用途：周期规范下的符号 Jacobian 证据链

原始路径：D:\1_ML\fes\FES_step07_symbolic_jacobian_no_go.py

SHA-256: 2aaeb2dff228883a3d7ba04cc91174009151374cb50a9c67d82c05834fa64516

```
{% raw %}
```

代码清单 A.47.6

语言：python；完整代码见附录第 586 页，共 874 行。

```
{% endraw %}
```

► fes_mass_plane_paper.py

用途：FES 质量相图基础版本

原始路径：D:\1_ML\fes\fes_mass_plane_paper.py

SHA-256: a4ae446d172365d2047de626f097aff1c1683afa2981ad452e729d638c2a0708

```
{% raw %}
```

代码清单 A.47.7

语言：python；完整代码见附录第 599 页，共 1058 行。

```
{% endraw %}
```

► fes_uB_phase_map_with_random_region_circles.py

用途: Fig. 3 最终 $u_{\text{eff}}-B_{\text{eff}}$ 组合坐标和散乱取点版式

原始路径: D:\1_ML\fes\fes_uB_phase_map_with_random_region_circles.py

SHA-256: 844789c4458a9fcbec409a727e67b8c19049bcb33cad01cce6eac37605add8d9

```
{% raw %}
```

代码清单 A.47.8

语言: python; 完整代码见附录第 614 页, 共 792 行。

```
{% endraw %}
```

7.2 Lieb 体系自旋陈数、解析质量与相界代码

日期: 2026-09-09

在线原文: marx2001.github.io

本文整理论文中 Lieb 体系真正使用的计算代码, 覆盖对称性约束紧束缚模型、全布里渊区带隙与自旋 Chern 标签、解析质量公式、相变边界、半无限边缘态及论文质量相图。代码按最终证据链归档, 不包含早期配色实验。

► 使用范围与科研口径

- 质量定义为 $M_+ = m_e + d_1 + d_2$ 与 $M_- = -m_e + d_1 + d_2$ 。
- 闭式公式仅用于自旋守恒、直接开隙且 $t_1 t_2 M_+ M_- \neq 0$ 的区域。
- 解析公式在全局样本与相界靶向样本合计 779 个可靠标签上逐点一致。
- 论文 Fig. 4 的代表点为 sobol_000544; 统一重绘代码收录在三体系边缘态笔记中。

► 代码执行顺序

1. 准备 Step 07 严格数值输出。2. 运行 Step 08, 生成机制变量、相变根与 WannierTools 风格表面格林函数边缘谱。3. 运行 Step 09, 将解析 Chern 公式与数值标签逐点核对。4. 运行 lieb_mass_plane_paper.py, 生成论文 Fig. 3 的 M_+-M_- 相图。

► 迁移说明

代码块保存原始完整源码, 并用 SHA-256 固定版本。绝对路径作为溯源信息保留; 移动到新项目后应统一修改项目根路径。大型 CSV、NPZ、模型和图片需要另行归档。只作为同名 Python 主程序入口的 Notebook 不重复收录。

► 完整代码

► 08_Lieb8_Analytic_Mechanism_and_WannierTools_Edge_v5.py

用途: 模型核心、解析边界、Sancho-Rubio 半无限表面格林函数和边缘谱

原始路径: D:\1_ML\lieb\08_Lieb8_Analytic_Mechanism_and_WannierTools_Edge_v5.py

SHA-256: d11b180f8e816a38327d4a25fafa621f0fd8483e9fe039880fd3e76b5d4bffc1

```
{% raw %}
```

代码清单 A.48.1

语言: python; 完整代码见附录第 626 页, 共 2866 行。

```
{% endraw %}
```

► 09_Lieb8_Analytic_Chern_Formula_and_Three_Sector_Map.py

用途: 验证双质量闭式公式和三扇区相图

原始路径: D:\1_ML\lieb\09_Lieb8_Analytic_Chern_Formula_and_Three_Sector_Map.py

SHA-256: 75b0196fd12664ad3dd7c17eb786115f44eee7c6d4947b057221c2bef619bc72

```
{% raw %}
```

代码清单 A.48.2

语言: python; 完整代码见附录第 667 页, 共 886 行。

```
{% endraw %}
```

► lieb_mass_plane_paper.py

用途: 生成 Fig. 3 的论文质量平面

原始路径: D:\1_ML\lieb\lieb_mass_plane_paper.py

SHA-256: dde4408522cba3cbd02105dea668e174732dcb05d2adfd489a5d8fac2107337e

```
{% raw %}
```

代码清单 A.48.3

语言: python; 完整代码见附录第 680 页, 共 518 行。

```
{% endraw %}
```

7.3 TTS 体系全局拓扑标注与分层机器学习代码

日期: 2026-09-09

在线原文: marx2001.github.io

本文归档 TTS 八带交错磁模型从 Hamiltonian 审计、周期规范、七维 Sobol 采样、严格 Chern 分扇区到分层机器学习及独立可观测量验证的核心代码, 对应论文的模型和机器学习流程。

► 使用范围与科研口径

- 机器学习负责发现参数组合和组织相区; 最终物理标签仍来自完整 BZ 带隙和非阿贝尔 Fukui Chern 计算。
- 训练 Sobol、外部 Sobol 和已知控制点在统计中保持分离。
- TTS 存在 $C_{\uparrow} = 0, \pm 1, \pm 2$ 等多个扇区, 不能只保留拓扑/平庸二分类。
- 全局随机森林指标不能替代固定切片上的局部相变质量公式。

► 代码执行顺序

1. 运行 Step 01 固定八带 Hamiltonian、周期规范与标签器。2. 运行 Step 02v3 建立 Hamiltonian 指纹和稳健基线。3. 运行 Step 03 完成训练 Sobol、独立外部 Sobol 和分层 ML。4. 运行 Steps 04-07, 跟踪相界谷、核对局域 Chern 跳变、解析质量分支和独立可观测量。

► 迁移说明

代码块保存原始完整源码, 并用 SHA-256 固定版本。绝对路径作为溯源信息保留; 移动到新项目后应统一修改项目根路径。大型 CSV、NPZ、模型和图片需要另行归档。只作为同名 Python 主程序入口的 Notebook 不重复收录。

► 完整代码

► TTS_step01_model_and_label_audit_v2.py

用途: 八带 MSG 127.391 Hamiltonian、周期规范、带隙和 Chern 标签器

原始路径: D:\1_ML\tts\TTS_step01_model_and_label_audit_v2.py

SHA-256: 9eb5374e4d4cc6306216061a6bc3326a483fe67227c605c062e48960ef35586c

```
{% raw %}
```

代码清单 A.49.1

语言: python; 完整代码见附录第 688 页, 共 1066 行。

```
{% endraw %}
```

► TTS_step02v3_robust_hamiltonian_fingerprint_and_ML.py

用途: Hamiltonian 指纹、稳健性审计和机器学习基线

原始路径: D:\1_ML\tts\TTS_step02v3_robust_hamiltonian_fingerprint_and_ML.py

SHA-256: b98e1b5d58b12974d1e77eed1388f25b2efd22a7998fed060dfe46b310394f39

```
{% raw %}
```

代码清单 A.49.2

语言: python; 完整代码见附录第 704 页, 共 1489 行。

```
{% endraw %}
```

► **TTS_step03_global_sobol_hierarchical_topology_ML.py**

用途: 七维全局 Sobol、严格 Chern 分扇区和分层机器学习

原始路径: D:\1_ML\tts\TTS_step03_global_sobol_hierarchical_topology_ML.py

SHA-256: 991054db135f13ca60079ffe4f4abe87e0ecbddd41e9edc04bff9f675c4c617

```
{% raw %}
```

代码清单 A.49.3

语言: python; 完整代码见附录第 726 页, 共 1790 行。

```
{% endraw %}
```

► **TTS_step04_chern_sector_boundary_valley_tracking.py**

用途: 定位不同 Chern 扇区之间的闭隙谷

原始路径: D:\1_ML\tts\TTS_step04_chern_sector_boundary_valley_tracking.py

SHA-256: 8c096c76e16e926cf2456838db198f9c5db4c06cde74d06cc5ad28cae277f00d

```
{% raw %}
```

代码清单 A.49.4

语言: python; 完整代码见附录第 752 页, 共 1294 行。

```
{% endraw %}
```

► **TTS_step05_kp_valley_topological_charge.py**

用途: 核对局域 kp 谷拓扑电荷和 Chern 跳变

原始路径: D:\1_ML\tts\TTS_step05_kp_valley_topological_charge.py

SHA-256: 93b2574831c046c469161c36c51bf3d858082ea2274046ec5d300c0226f0a114

```
{% raw %}
```

代码清单 A.49.5

语言: python; 完整代码见附录第 771 页, 共 1492 行。

```
{% endraw %}
```

► **TTS_step06_analytic_mass_branch_chern_atlas.py**

用途: 关联解析质量分支和 Chern 扇区

原始路径: D:\1_ML\tts\TTS_step06_analytic_mass_branch_chern_atlas.py

SHA-256: 64e17b8c623c10eac4ac9c7dd54f5629f91cea789f5eeb51739fde9fbd23485e

```
{% raw %}
```

代码清单 A.49.6

语言: python; 完整代码见附录第 793 页, 共 1166 行。

```
{% endraw %}
```

► TTS_step07_observable_validation.py

用途: 使用独立可观测量复核候选拓扑相

原始路径: D:\1_ML\tts\TTS_step07_observable_validation.py

SHA-256: 607c757e5e1dde738206aec944ac4cfd7df62786edcbddd97669a87deb8beb3

```
{% raw %}
```

代码清单 A.49.7

语言: python; 完整代码见附录第 810 页, 共 1360 行。

```
{% endraw %}
```

7.4 TTS 体系机制感知机器学习、局部质量相界与验证代码

日期: 2026-09-09

在线原文: marx2001.github.io

本文归档 TTS 固定背景 r3-r4 切片中, 从机制分层、局部相界求解、多闭隙交汇修复、物理认证机器学习到质量坐标相图和随机验证的完整代码。

► 使用范围与科研口径

- M_L、M_4v 与 M_Sigma-prime 是固定背景附近的参考对齐局部质量, 不是完整七维空间的全局坐标。
- Step 14 先进行空间分组留出评估, 再把学习边界转成可验证的稀疏质量。
- Step 16 负责论文质量坐标变换和绘图, 不重新定义拓扑标签。
- Step 17 的 75 个内部点用于准确率, 18 个近边界应力点因分辨率敏感而单独报告。

► 代码执行顺序

1. 运行 Step 08M, 对全局数据进行机制分层。2. 运行 Steps 11-13, 构造固定切片、修复多闭隙交汇并补充 Wilson/Chern 证据。3. 运行 Step 14, 从严格认证数据提取局部质量规则。4. 运行 Step 15 统一边界标签, 再用 Step 16 生成 Fig. 3 局部质量坐标图。5. 运行 Step 17, 在逆质量坐标中随机取样并验证内部相区及边界应力点。

► 迁移说明

代码块保存原始完整源码, 并用 SHA-256 固定版本。绝对路径作为溯源信息保留; 移动到新项目后应统一修改项目根路径。大型 CSV、NPZ、模型和图片需要另行归档。只作为同名 Python 主程序入口的 Notebook 不重复收录。

► 完整代码

► TTS_step08M_mechanism_aware_machine_learning.py

用途: 区分高对称单谷与多谷相变机制

原始路径: D:\1_ML\tts\TTS_Step08M_Mechanism_Aware_ML-final\TTS_step08M_mechanism_aware_machine_learning.py

SHA-256: d3a764e587cef2ff5305e57e342e952a32dbd8457749b7e6e1e8f48284a5034c

```
{% raw %}
```

代码清单 A.50.1

语言: python; 完整代码见附录第 830 页, 共 1641 行。

```
{% endraw %}
```

► TTS_step11M_lieb_aligned_fixed_slice.py

用途: 构造固定背景 r3-r4 认证切片

原始路径: D:\1_ML\tts\TTS_Step08M_Mechanism_Aware_ML-final\TTS_step11M_lieb_aligned_fixed_slice.py

SHA-256: 0b15e38740906bf7d2650d421fac160dc83f57335428151cd7116edf2dc4aca9

```
{% raw %}
```

代码清单 A.50.2

语言: python; 完整代码见附录第 854 页, 共 1603 行。

```
{% endraw %}
```

► TTS_step12M_junction_multiclosure_repair.py

用途: 解析相界交汇处的多闭隙事件

原始路径: D:\1_ML\tts\TTS_Step08M_Mechanism_Aware_ML-final\TTS_step12M_junction_multiclosure_repair.py

SHA-256: 1db11ce7a20b8107e97f4980f81e4b3862dcbc7b525ecc7688d0029ef4ae3159

```
{% raw %}
```

代码清单 A.50.3

语言: python; 完整代码见附录第 877 页, 共 1738 行。

```
{% endraw %}
```

► TTS_step13M_adaptive_wilson_intermediate_chern.py

用途: 补强中间扇区 Wilson/Chern 标签

原始路径: D:\1_ML\tts\TTS_Step08M_Mechanism_Aware_ML-final\TTS_step13M_adaptive_

wilson_intermediate_chern.py

SHA-256: df58f29a1b5087e722918a51c74a2db00b0b2d6703c89585ca647f62e7fc7780

```
{% raw %}
```

代码清单 A.50.4

语言: python; 完整代码见附录第 903 页, 共 1200 行。

```
{% endraw %}
```

► TTS_step14M_physics_certified_rule_extraction.py

用途: 空间留出评估、稀疏质量拟合和机制证书

原始路径: D:\1_ML\tts\TTS_Step08M_Mechanism_Aware_ML-final\TTS_step14M_physics_certified_rule_extraction.py

SHA-256: 7e19d21ad9e4780aa41f30641c61703adea1ba34de205218515dffaf6d37bfcc

```
{% raw %}
```

代码清单 A.50.5

语言: python; 完整代码见附录第 920 页, 共 1211 行。

```
{% endraw %}
```

► TTS_step15M_reclassify_all_boundary_points.py

用途: 统一重分类全部相界点

原始路径: D:\1_ML\tts\TTS_Step08M_Mechanism_Aware_ML-final\TTS_step15M_reclassify_all_boundary_points.py

SHA-256: b0bc7284e54678212075359e4783f05bbe2ca44d7ff1e3839bcceab251d2235a

```
{% raw %}
```

代码清单 A.50.6

语言: python; 完整代码见附录第 938 页, 共 901 行。

```
{% endraw %}
```

► TTS_step16M_mass_coordinate_phase_maps.py

用途: 生成 Fig. 3 的两组局部组合坐标相图

原始路径: D:\1_ML\tts\TTS_Step08M_Mechanism_Aware_ML-final\TTS_step16M_mass_coordinate_phase_maps.py

SHA-256: 2abee48b6b17b1102f7aa62c529f548187c05db5df26efb10e3b60f055c3bfdd

```
{% raw %}
```

代码清单 A.50.7

语言: python; 完整代码见附录第 952 页, 共 674 行。

```
{% endraw %}
```

► TTS_step17M_random_mass_coordinate_validation.py

用途: 独立内部点和边界应力验证

原始路径: D:\1_ML\tts\TTS_Step08M_Mechanism_Aware_ML-final\TTS_step17M_random_mass_coordinate_validation.py

SHA-256: 7e01295e0bd4b9ef55c9d6cbbd79927ea0cf2dbc71071c9f5b90a65690a35ce8

```
{% raw %}
```

代码清单 A.50.8

语言: python; 完整代码见附录第 962 页, 共 819 行。

```
{% endraw %}
```

7.5 三体系体能带、边缘态、Berry 曲率与霍尔响应代码

日期: 2026-09-09

在线原文: marx2001.github.io

本文归档论文 Fig. 4 的统一数值流程, 包括 Lieb、FES、TTS 模型适配, 体能带, 半无限边缘态谱, 代表点复筛, 最终样本固定, 沿高对称路径的 Berry 曲率以及二维布里渊区 Hall 响应。

► 使用范围与科研口径

- 最终样本为 Lieb sobol_000544、FES rs-rd_i0031_j0044、TTS sobol_train_000456 和 TTS sobol_external_000117。
- 沿高对称路径的 $\Omega_z(k)$ 是 Berry 曲率轨迹, 不是完整反常霍尔电导; 电导需要二维 BZ 积分, 金属还必须包含费米占据。
- Fig4.svg 后续含 Inkscape 手工排版和颜色调整, 因此数值内容可由代码复现, 但最终画布不能仅靠代码逐像素复现。
- 白色交界、薰衣草、沙金及暗色预览脚本属于配色试验, 不纳入最终计算归档。

► 代码执行顺序

1. 运行 Step 18, 建立统一模型接口、表面格林函数和 Hall 积分。2. 运行 Step 19, 生成四案例基础数据与可编辑 SVG。3. 运行 Steps 20-22, 完成小带隙代表点复筛、固定选择和双色谱图重绘。4. 运行 Step 31 计算路径 Berry 曲率; 运行 Step 35 生成投稿用途的 Hall-current 数据。5. 最终以 step22_01_final_physics_audit.csv 和 step22_02_manifest.json 核对样本、参数与相分类。

► 迁移说明

代码块保存原始完整源码, 并用 SHA-256 固定版本。绝对路径作为溯源信息保留; 移动到新项目后应统一修改项目根路径。大型 CSV、NPZ、模型和图片需要另行归档。只作为同名 Python 主程序入口的 Notebook 不重复收录。

► 完整代码

► TTS_step18_three_system_wanniertools_style_edge_ahc_examples.py

用途: 三体系模型适配、体能带、Sancho 表面格林函数、边缘谱与二维 Hall 数据

原始路径: D:\1_ML\tts\TTS_step18_three_system_wanniertools_style_edge_ahc_examples.py

SHA-256: fa32727fbaefa6fb943240542dd449b4d92f4187247fe5deb069e7c209ae087c

```
{% raw %}
```

代码清单 A.51.1

语言: python; 完整代码见附录第 974 页, 共 1892 行。

```
{% endraw %}
```

► TTS_step19_maintext_four_topological_edge_hall.py

用途: 四个正文非平庸案例的基础数据包与可编辑 SVG

原始路径: D:\1_ML\tts\TTS_step19_maintext_four_topological_edge_hall.py

SHA-256: 2644f059b661378d2ae3e68e1fb5710128cada7d0acff5444d7ca6881e478129

```
{% raw %}
```

代码清单 A.51.2

语言: python; 完整代码见附录第 1002 页, 共 1984 行。

```
{% endraw %}
```

► TTS_step20_edge_representative_rescreen.py

用途: 按带隙与图形可读性重新筛选候选点

原始路径: D:\1_ML\tts\TTS_step20_edge_representative_rescreen.py

SHA-256: 5cb82feb987ab2978b02c318c28f031e8a898191178de6a6ea7c0dc270066eb4

```
{% raw %}
```

代码清单 A.51.3

语言: python; 完整代码见附录第 1031 页, 共 914 行。

```
{% endraw %}
```

► **TTS_step21_small_gap_edge_rescreen.py**

用途: 计算小带隙候选的边缘态图库

原始路径: D:\1_ML\tts\TTS_step21_small_gap_edge_rescreen.py

SHA-256: 920c528dfd9c1d273e0dca2597d4fd98e47907ab7f5c31da7f7564330315211a

```
{% raw %}
```

代码清单 A.51.4

语言: python; 完整代码见附录第 1044 页, 共 420 行。

```
{% endraw %}
```

► **TTS_step21_finalize_small_gap_selection.py**

用途: 固定用户确认的四个代表点与审计记录

原始路径: D:\1_ML\tts\TTS_step21_finalize_small_gap_selection.py

SHA-256: 495816c6b7a1b7fc3fb13de5f19c40042da7f577f517e1a9b5b69621780151ec

```
{% raw %}
```

代码清单 A.51.5

语言: python; 完整代码见附录第 1050 页, 共 179 行。

```
{% endraw %}
```

► **TTS_step22_user_selection_dual_color_render.py**

用途: 按最终样本生成体能带与边缘态数据/图

原始路径: D:\1_ML\tts\TTS_step22_user_selection_dual_color_render.py

SHA-256: b9b0ac203a37f1de6258e68b6824af3b4b37fca433b7a559f879ac042124b5c6

```
{% raw %}
```

代码清单 A.51.6

语言: python; 完整代码见附录第 1053 页, 共 301 行。

```
{% endraw %}
```

► **TTS_step31_add_path_berry_curvature_to_Fig4.py**

用途: 计算并添加高对称路径 Berry 曲率曲线

原始路径: D:\1_ML\tts\TTS_step31_add_path_berry_curvature_to_Fig4.py

SHA-256: c26e85a97e3cba192ebad981b002cde70a6e1ed59b1014ff1a228e8b6fde8fbf

```
{% raw %}
```

代码清单 A.51.7

语言: python; 完整代码见附录第 1057 页, 共 579 行。

```
{% endraw %}
```

► TTS_step35_submission_hall_current_fig4.py

用途: 通过二维 BZ 积分生成四案例 Hall 响应

原始路径: D:\1_ML\tts\TTS_step35_submission_hall_current_fig4.py

SHA-256: ab21f59b2a3aaa31ca21b97a95c66e7a6b60a0c7fca0f0286f7566e0d23157cb

```
{% raw %}
```

代码清单 A.51.8

语言: python; 完整代码见附录第 1066 页, 共 230 行。

```
{% endraw %}
```

7.6 交错磁自旋陈数论文最终代码索引

日期: 2026-09-09

在线原文: marx2001.github.io

本文把论文最终使用的数值代码整理为五篇独立的 **sci-note**, 共收录 34 个 Python 源文件。归档重点是建立可追踪的论文证据链, 而不是把项目中所有历史版本、Notebook 包装器和配色实验全部堆入网站。

5 篇独立代码笔记 34 个 Python 源文件 3 个核心体系:Lieb / FES / TTS 文章清单图片一代码关系 Fig. 4 代表点归档边界

► 文章清单

01

► Lieb 体系代码

解析质量、Chern 闭式公式、相变边界与论文相图。

02

► FES 体系代码

Sobol 标签、Gamma/M 质量、条件性 no-go 与金属相图。

03

► TTS 全局代码

八带模型、七维 Sobol、分层机器学习和全局标签。

04

► **TTS 局部质量代码**

机制感知 ML、局部质量坐标、相界与随机验证。
05

► **三体系边缘态与霍尔响应代码**

Fig. 4 的体能带、表面格林函数、边缘谱、Berry 曲率和 Hall 数据。

► **论文图片与代码的关系**

图片	计算来源	是否完全由代码生成
Fig. 1	三体系 Hamiltonian 和结构定义	否。模型来自代码, 晶格示意与排版含 SVG/PPT 手工编辑。
Fig. 2	Sobol-gap-Chern-ML-mass rule workflow	否。数据与模型来自代码, 流程图为人工排版。
Fig. 3	Lieb、FES、TTS 质量坐标相图	数值背景和散点由代码生成, 随后做字体与版面微调。
Fig. 4	体能带、边缘谱、Berry 曲率 / Hall 数据	数值内容由代码生成, 总图含 Inkscape 手工排版和颜色微调。
Fig. 5	自旋输运示意	否。通道数来自 Chern 结果, 晶格、矩形和箭头为人工矢量图。

► **Fig. 4 最终代表点**

Lieb $C\uparrow = 1, C\downarrow = -1$ 全局带隙 0.102 eV 样本:sobol_000544 ($m_e, t_1, t_2, s_1, d_1, s_2, d_2$) = (0.1882, -0.5530, -0.4969, -0.6106, -0.1282, -0.1016, -0.1111) FES $C\uparrow = 1, C\downarrow = -1$ 自旋陈金属样本:rs-rd_i0031_j0044 直接带隙 3.49 meV; 间接带隙 -2.94 meV。(m_e, t_1, t_2, r_1, r_2) = (-0.2217, -0.3970, -0.07035, -0.02218, -0.03263) TTS $C\uparrow = 1, C\downarrow = -1$ 全局带隙 0.234 eV 样本:sobol_train_000456 ($m_e, t_1, t_2, r_1, r_2, r_3, r_4$) = (0.02797, 0.2888, 0.4400, -0.4172, 0.4742, -0.3088, -0.4775) TTS $C\uparrow = 2, C\downarrow = -2$ 间接带隙 0.319 eV 样本:sobol_external_000117 ($m_e, t_1, t_2, r_1, r_2, r_3, r_4$) = (0.04294, -0.4809, 0.5661, -0.2064, 0.4489, -0.3781, 0.2437)

► **归档边界**

补充说明

没有纳入本索引的内容

- 仅调用同名 .py 主程序的 .ipynb 文件, 避免同一份代码重复出现。
- 白色交界、淡薰衣草、沙金、暗色背景等 Fig. 4 配色预览脚本。
- 大型 CSV、NPZ、PNG、PDF 和 SVG 输出; 这些属于数据与图片归档, 应另行保存。

- 尚未闭合的 DFT/Wannier 材料映射专项。当前正文的模型结论来自对称性约束紧束缚模型, 不能把未完成的材料拟合混入最终证据链。

归档原则: 本页只保留与论文最终结果直接对应的代码入口、代表参数和证据链关系; 部署路径、复制命令等本地维护信息建议放在仓库 README, 而不是公开文章正文。

原创研究与机器学习

本专题收录 16 篇文章。

8.1 kp 构造对称哈密顿量的测试

日期: 2025-12-06

在线原文: marx2001.github.io

► Recording

► 1. 步骤

1. 学习代码: “shell
#!/usr/bin/env python

► – coding: utf-8 –

► © 2017-2018, ETH Zurich, Institut für Theoretische Physik

► Author: Dominik Gresch greschd@gmx.ch

► 导入必要的库

```
import sympy as sp from sympy.core.numbers import I import sympy.physics.matrices as sm from
sympy.physics.quantum import TensorProduct import symmetry_representation as sr # 对称性表示库
import kdotp_symmetry as kp #  $k \cdot p$  理论对称性处理库
```

► 在此项目中, 我们使用泡利矩阵的张量积作为基矢

► 泡利矩阵向量: 单位矩阵 + 三个泡利矩阵

```
pauli_vec = [sp.eye(2), *(sm.msigen(i) for i in range(1, 4))]
```

► 创建基矢: 所有可能的泡利矩阵两两张量积

```
basis = [TensorProduct(p1, p2) for p1 in pauli_vec for p2 in pauli_vec]
```

► 创建对称性操作对象

► C2y 操作: 绕特定轴的二重旋转

```
c2y = sr.SymmetryOperation( rotation_matrix=[[0, 1, 0], [1, 0, 0], [0, 0, -1]], # 空间旋转矩阵 repr_matrix=sp.diag(1,
-I, 1, -I), # 表示矩阵 (4x4 对角矩阵) repr_has_cc=False, # 表示矩阵不包含复共轭操作 numeric=False # 使用符号计算而非数值计算)
```

► 宇称操作: 空间反演

```
parity = sr.SymmetryOperation( rotation_matrix=-sp.eye(3), # 负单位矩阵, 表示空间反演 repr_matrix=sp.diag(1,
1, -1, -1), # 对角表示矩阵 repr_has_cc=False, # 不包含复共轭)
```

► 时间反演对称性

```
time_reversal = sr.SymmetryOperation( rotation_matrix=sp.eye(3), # 空间上不旋转 repr_matrix=TensorProduct(sp.eye(2), sp.Matrix([[0, -1], [1, 0]])), # 张量积形式的表示矩阵 repr_has_cc=True, # 时间反演包含复共轭操作)
```

```
def print_result(order): “”" 打印给定 k 阶数的对称哈密顿量基矢 ””" print( 'Order:', order) # 输出当前阶数
```

代码清单 A.53.1

语言: plaintext; 完整代码见附录第 1069 页, 共 10 行。

```
if name == 'main': # 主程序: 计算并打印 0 阶、1 阶、2 阶的对称哈密顿量 for i in range(3): print_result(order=i)
“”
```

8.2 (原创脚本) 基于对称性构造 TB 模型 (n 带紧束缚模型)——边缘态计算与拟合

日期: 2026-01-02

在线原文: marx2001.github.io

► 心得体会

这几天一直在琢磨 wannier tools 计算的边缘态是否正确, 所以先用 pythTB 写了代码, 用自己电脑计算边缘态, 然后用同门的代码, 改了改, 计算角态, 但是心里还是不踏实, 于是琢磨怎么通过数学建模来构造紧束缚模型, 于是 biying 搜索关键词“构造紧束缚模型”, 看到一篇知乎文章, 推荐 magneticTB, 可以从磁群出发, 用 mathematica 来构造 n 带的 TB 模型, 上手尝试。自己尝试了大概 5 个小时, 终于看官方文档把代码写了出来, 但是卡在了 wyckoff 位置上, 我认为 c 位置有 2 个 wyckoff position, 都要写上, 原因是磁矩方向不同, 但是看到一篇近期的 PRL 刚好讲了我这个体系的 TB 模型构造, 并在支撑材料给出了他的 magneticTB 代码, 发现他只用了 1 个, 早知道先搜索文章了, 但解决了无数报错, 也算是积累经验, 我的代码和这篇 prl 一样简洁, 只是 wyckoff position 不一样, 所以经过我的投影能带分析, 我选择了考虑 soc 的情况下的 6 个轨道基组, dxy/dyz/dxz 轨道, 哈密顿量矩阵是 12×12 , 所以是 12 带模型, 一共有 54 个 params。也就是说, e(在位能)t(跃迁项)r(soc), 三个可以自由选择参数, 一共有 54 项, 正常的 2 能带模型, 或者四带模型, 也就 3 个变量起步, 也很少, 但是我这个 54 个是不可能靠人力去逐项枚举, (其实 3 个变量也太多了, 不限范围, 不限整数分数, 也是无限种可能), 我不知道其他人怎么做的, 但是咨询了一个大佬之后, 可以通过 mathematica 导出基函数, 然后用二分法或者梯度下降法或者其他方法与 DFT 和 wannier 能带作为对比, 用 python 代码计算与 DFT 和 Wannier 能带的能量差来计算出最小的 etr 参数组合, 这大概就是具体的流程。

我之前一直以为 kdotp-symmetry 包得到 kp 模型之后, 需要傅里叶变换得到 TB 模型, 还要把晶格模型镶嵌进 TB 模型中, 这个过程未知数太多, 全都是不会的 (尽管 MagneticTB 也是一样, 但毕竟有详尽的官方中文手册), 但是 MagneticTB 似乎原理也是差不多, 都是基于对称性来构造 TB 模型, 我就没有强求必须用 kp 方法构造紧束缚模型。

► 步骤

按照官网要求安装 MagneticTB, 安装 mathematica 14.1 中文版

构造 12 带模型的 MagneticTB 的 mathematica 文件我已经上传到了 github, 不能展示在此页面中, 因为 github 在部署网页中会报错, 无法部署任何网页。

然后用代码导出基函数的哈密顿量矩阵, 再用 python 代码计算 etr 即可。

► 代码

1. 计算任何材料边缘态的代码, 需要 hr.dat、wannier center 等文件。

cell 1:

代码清单 A.54.1

语言: python; 完整代码见附录第 1070 页, 共 23 行。

cell 2:

代码清单 A.54.2

语言: python; 完整代码见附录第 1070 页, 共 12 行。

cell 3:

代码清单 A.54.3

语言: python; 完整代码见附录第 1070 页, 共 19 行。

cell 4:

代码清单 A.54.4

语言: python; 完整代码见附录第 1071 页, 共 27 行。

cell 5 :

代码清单 A.54.5

语言: python; 完整代码见附录第 1071 页, 共 102 行。

cell 6 下方代码是有问题的, 我春节后整理数据之后重新修改, 此处是备份:

代码清单 A.54.6

语言: python; 完整代码见附录第 1073 页, 共 9 行。

cell 8

代码清单 A.54.7

语言: python; 完整代码见附录第 1073 页, 共 9 行。

1. 自动拟合 DFT 计算的能带 or wannier90 拟合能带的脚本, 不知道是否正确 (时间过去太久, 这个脚本是三周之前写的, 有很多版本, 所以忘记哪个版本是正确的, 但主题思想是不变的)

代码清单 A.54.8

语言: python; 完整代码见附录第 1073 页, 共 663 行。

8.3 (原创脚本)optuna 超参数优化探索拓扑相界

日期: 2026-01-19

在线原文: marx2001.github.io

► 心得体会

这个是回忆的版本, 距离研究出构造 TB 模型的流程已经两周, 很多细节可能会忘记。

► 步骤

用 `img` 文件夹中的 `mathematica` 脚本运行得到紧束缚模型

再使用 `ipynb` 脚本 1 或 2 进行超参数优化, 计算陈数。脚本还有报错, 但暂时没有时间进行进一步修改, 需要修改第一篇论文的正文以及第二篇论文的数据和论文正文绘图, 在此记录并留存到前两篇文章处理完毕。

8.4 (原创自研) 基于对称性构造 TB 模型 (n 带紧束缚模型)——具体流程

日期: 2026-01-19

在线原文: marx2001.github.io

► 心得体会

这个是回忆的版本, 距离研究出构造 TB 模型的流程已经两周, 很多细节可能会忘记。

► 步骤

按照官网要求安装 `MagneticTB`, 安装 `mathematica 14.1` 中文版

构造 12 带模型的 `MagneticTB` 的 `mathematica` 文件我已经上传到了 `github`, 不能展示在此页面中, 因为 `github` 在部署网页中会报错, 无法部署任何网页。

然后用代码导出基函数的哈密顿量矩阵, 再用 `python` 代码计算 `etr` 即可。

以图片形式备份代码。

首先, 我们需要构建 TB 模型, `mathematica` 代码如下:

```

In[ ]:= $UserBaseDirectory
|用户基础目录

Out[ ]:= C:\Users\Administrator\AppData\Roaming\Mathematica

In[ ]:= Needs["MagneticTB`"]
|需要

In[ ]:= spog = msgop[bnsdict[{99, 165}]]];
init[
    lattice → {{a, 0, 0}, {0, a, 0}, {0, 0, c}},
    lattpar → {a → 1, c → 10},
    wyckoffposition → {{0.5, 0.0, 0.5}, {0, 0, 1}},
    symminformation → spog,
    basisFunctions → {"sup", "sdn"}
];
ham = Sum[symham[i], {i, 2}];
|求和

MatrixForm[ham]
|矩阵格式

```

输出为:

```

Magnetic space group (BNS): {99,165, P4'm'm}
Lattice: TetragonalP
Primitive Lattice Vectors: {{a, 0, 0}, {0, a, 0}, {0, 0, c}}
Conventional Lattice Vectors: {{a, 0, 0}, {0, a, 0}, {0, 0, c}}
Double Value Rep.
Generators: {"2z", F}, {"m-xy", F}, {"4z", T}
params: {e1, e2}
params: {t1, t2, t3, t4}

Out[ ]:=MatrixForm[

$$\begin{pmatrix}
e1 & 0 & e^{i\frac{ky}{2}}(-i t1 + t4) + e^{i\frac{ky}{2}}(-i t1 + t4) + e^{i\frac{ky}{2}}(i t1 + t4) + e^{i\frac{ky}{2}}(i t1 + t4) & (-1+i) e^{i\frac{ky}{2}} t3 + (1+i) e^{i\frac{ky}{2}} t3 - (1+i) e^{i\frac{ky}{2}} t3 + (1-i) e^{i\frac{ky}{2}} t3 \\
0 & e2 & (-1-i) e^{i\frac{ky}{2}} t2 + (1-i) e^{i\frac{ky}{2}} t2 - (1-i) e^{i\frac{ky}{2}} t2 + (1+i) e^{i\frac{ky}{2}} t2 & e^{i\frac{ky}{2}}(-i t1 + t4) + e^{i\frac{ky}{2}}(-i t1 + t4) + e^{i\frac{ky}{2}}(i t1 + t4) + e^{i\frac{ky}{2}}(i t1 + t4) \\
e^{i\frac{ky}{2}}(-i t1 + t4) + e^{i\frac{ky}{2}}(-i t1 + t4) + e^{i\frac{ky}{2}}(i t1 + t4) + e^{i\frac{ky}{2}}(i t1 + t4) & (-1-i) e^{i\frac{ky}{2}} t2 + (1-i) e^{i\frac{ky}{2}} t2 - (1-i) e^{i\frac{ky}{2}} t2 + (1+i) e^{i\frac{ky}{2}} t2 & e^{i\frac{ky}{2}}(-i t1 + t4) + e^{i\frac{ky}{2}}(-i t1 + t4) + e^{i\frac{ky}{2}}(i t1 + t4) + e^{i\frac{ky}{2}}(i t1 + t4) \\
(-1-i) e^{i\frac{ky}{2}} t2 + (1-i) e^{i\frac{ky}{2}} t2 - (1-i) e^{i\frac{ky}{2}} t2 + (1+i) e^{i\frac{ky}{2}} t2 & e^{i\frac{ky}{2}}(-i t1 + t4) + e^{i\frac{ky}{2}}(-i t1 + t4) + e^{i\frac{ky}{2}}(i t1 + t4) + e^{i\frac{ky}{2}}(i t1 + t4) & e^{i\frac{ky}{2}}(-i t1 + t4) + e^{i\frac{ky}{2}}(-i t1 + t4) + e^{i\frac{ky}{2}}(i t1 + t4) + e^{i\frac{ky}{2}}(i t1 + t4)
\end{pmatrix}$$


```

然后运行下方代码, 输出 hr.dat 紧束缚模型:

```

In[ ]:= hop[ham, {e1 → 0, e2 → 0.5, r1 → 0.5, r2 → 0.5, r3 → 0.5, r4 → 0.5, t1 → 0.5, t2 → 0.5}]
path = {
    {{0, 0, 0}, {1/2, 0, 0}}, {"G", "X"},
    {{1/2, 0, 0}, {1/2, 1/2, 0}}, {"X", "M"},
    {{1/2, 1/2, 0}, {0, 1/2, 0}}, {"M", "Y"},
    {{0, 1/2, 0}, {0, 0, 0}}, {"Y", "G"}
};
bandManipulate[path, 200, ham]

```

上面是最基本的 n 能带模型构建代码, 产生的 hr.dat 文件可以用于边缘态等紧束缚模型相关计算。

此外, 为了探究 `etr` 造成的影响, 我写了进行拓扑相界扫描的代码, 旨在一定的范围区间内, 生成 `hr.dat` 文件, 批量计算其拓扑不变量和能带, 最后绘制出散点图, 但由于维度过高, 12 能带模型的拓扑相界无法用常规手段表征, 且我是自行研究, 无人指导, 因此先记录, 后如果涉及到相关领域, 再继续研究,

```
In[*]:= ClearAll[etrToParamsGeneral, outDir, e1list, e2list, t1list, t2list, t3list, t4list, makeFileName];
[清除全部]

(* ===== *)
(*参数映射: 把数值映射到符号参数*)
(* ===== *)
etrToParamsGeneral[E1_, E2_, T1_, T2_, T3_, T4_] := {(*onsite*) e1 → E1, e2 → E2, (*hoppings*) t1 → T1, t2 → T2, t3 → T3, t4 → T4
(*longer-range/SOC-assisted terms: 若未来加其它项, 在这里继续补*)};

(* ===== *)
(*扫描范围: 分别定义每个参数的范围*)
(* (把下面这些范围改成你要的即可) *)
(* ===== *)
e1list = Range[-0.1, 0.1, 0.05];
[范围]
e2list = Range[-0.1, 0.1, 0.05];
[范围]

t1list = Range[-0.1, 0.1, 0.05];
[范围]
t2list = Range[-0.1, 0.1, 0.05];
[范围]
t3list = Range[-0.1, 0.1, 0.05];
[范围]
t4list = Range[-0.1, 0.1, 0.05];
[范围]

(* ===== *)
(*输出目录 (Notebook 所在目录) *)
(* ===== *)
outDir = NotebookDirectory[];
[当前笔记本的目录]
If[!StringQ[outDir] || outDir == $Failed, outDir = Directory[]; (*兜底: 非 notebook 环境用当前目录*)];
[如果 | 字符串判定] [失败] [目录]

If[!StringQ[outDir] || outDir == $Failed, outDir = Directory[]; (*兜底: 非 notebook 环境用当前目录*)];
[如果 | 字符串判定] [失败] [目录]

(* ===== *)
(*文件名: 用索引, 保证唯一*)
(* ===== *)
makeFileName[i1_, i2_, j1_, j2_, j3_, j4_] := FileNameJoin[{outDir, StringTemplate["wannier90_iE1`_iE2`_iT1`_iT2`_iT3`_iT4`.dat"][i1, i2, j1, j2, j3, j4]};
[文件名连接] [字符串模板]

(* ===== *)
(*主循环: 每个 (e1,e2,t1,t2,t3,t4) 一个文件*)
(* ===== *)
Do[Module[{E1, E2, T1, T2, T3, T4, params, file, stream}, E1 = e1list[[iE1];
[... | 模块]
E2 = e2list[[iE2];
T1 = t1list[[iT1];
T2 = t2list[[iT2];
T3 = t3list[[iT3];
T4 = t4list[[iT4];
params = etrToParamsGeneral[E1, E2, T1, T2, T3, T4];
file = makeFileName[iE1, iE2, iT1, iT2, iT3, iT4];
stream = OpenWrite[file, CharacterEncoding → "UTF-8"];
[打开编写] [字符串编码]
(*写入高精度参数头 (供 Python 解析) *) WriteString[stream, "e1 = ", ToString@NumberForm[N[E1, 16], {Infinity, 12}], NumberPoint → ".", " e2 = ",
ToString@NumberForm[N[E2, 16], {Infinity, 12}], NumberPoint → ".", " t1 = ", ToString@NumberForm[N[T1, 16], {Infinity, 12}], NumberPoint → ".", " t2 = ",
ToString@NumberForm[N[T2, 16], {Infinity, 12}], NumberPoint → ".", " t3 = ", ToString@NumberForm[N[T3, 16], {Infinity, 12}], NumberPoint → ".", " t4 = ",
ToString@NumberForm[N[T4, 16], {Infinity, 12}], NumberPoint → ".", "\n"];
[转换为... | 数值近似 | 数值运算 | 无穷大 | 转换为... | 数值近似 | 数值运算 | 无穷大 | 转换为... | 数值近似 | 数值运算 | 无穷大 | 转换为... | 数值近似 | 数值运算 | 无穷大 | 转换为... | 数值近似 | 数值运算 | 无穷大]
(*将 hop[ham, params] 的输出重定向到文件*) Block[{ $Output = {stream}}, hop[ham, params]];
[块] [输出文件和通函列表]

Close[stream];
[关闭流]
```

```

E2 = e2list[iE2];
T1 = t1list[iT1];
T2 = t2list[iT2];
T3 = t3list[iT3];
T4 = t4list[iT4];
params = etrToParamsGeneral[E1, E2, T1, T2, T3, T4];
file = makeFileName[iE1, iE2, iT1, iT2, iT3, iT4];
stream = OpenWrite[file, CharacterEncoding -> "UTF-8"];
(*写入高精度参数头 (供 Python 解析) *) WriteString[stream, "## e1 = ", ToString@NumberForm[N[E1, 16], {Infinity, 12}, NumberPoint -> "."], " e2 = ",
ToString@NumberForm[N[E2, 16], {Infinity, 12}, NumberPoint -> "."], " t1 = ", ToString@NumberForm[N[T1, 16], {Infinity, 12}, NumberPoint -> "."], " t2 = ",
ToString@NumberForm[N[T2, 16], {Infinity, 12}, NumberPoint -> "."], " t3 = ", ToString@NumberForm[N[T3, 16], {Infinity, 12}, NumberPoint -> "."], " t4 = ",
ToString@NumberForm[N[T4, 16], {Infinity, 12}, NumberPoint -> "."], "\n"];
(*将 hop[ham, params] 的输出重定向到文件*) Block[{$Output = {stream}}, hop[ham, params]];

Close[stream];
Print["Exported: ", file];, {iE1, Length@e1list}, {iE2, Length@e2list}, {iT1, Length@t1list}, {iT2, Length@t2list}, {iT3, Length@t3list}, {iT4, Length@t4list}];

in[]:= workdir = "E:\马睿麟\组会汇报\Nb205Se\pythTB\workflow";
inCSV = FileNameJoin[{workdir, "dense_linescan_missing_hr_for_mma.csv"}];

outRoot = FileNameJoin[{workdir, "linescan_hr"}];
If[! DirectoryQ[outRoot], CreateDirectory[outRoot]];

outIndexCSV = FileNameJoin[{workdir, "dense_linescan_hr_index.csv"}];

raw = Import[inCSV, "CSV"];
header = First[raw];
data = Rest[raw];
rows = AssociationThread[header, #] & /@ data;
n = Length[rows];
Print["[INFO] loaded rows: ", n];

toNum[x_] := Module[{y = x}, Which[NumberQ[y], N[y], StringQ[y], N@ToExpression[y], True, N@y]];

tag[x_] := Module[{k = Round[1000.0 + toNum[x]]}, If[k < 0, "m" <> IntegerString[Abs[k], 10, 3], "p" <> IntegerString[k, 10, 3]]];

folderName[e1_, e2_, t1_, t2_, t3_, t4_] := StringJoin["w90_dense_ls_", "e1", tag[e1], "_", "e2", tag[e2], "_t1", tag[t1], "_t2", tag[t2], "_t3", tag[t3], "_t4", tag[t4]];

etrToParamsGeneral[E1_, E2_, T1_, T2_, T3_, T4_] := {e1 -> E1, e2 -> E2, t1 -> T1, t2 -> T2, t3 -> T3, t4 -> T4};

folderName[e1_, e2_, t1_, t2_, t3_, t4_] := StringJoin["w90_dense_ls_", "e1", tag[e1], "_", "e2", tag[e2], "_t1", tag[t1], "_t2", tag[t2], "_t3", tag[t3], "_t4", tag[t4]];

etrToParamsGeneral[E1_, E2_, T1_, T2_, T3_, T4_] := {e1 -> E1, e2 -> E2, t1 -> T1, t2 -> T2, t3 -> T3, t4 -> T4};

indexList = {};
failList = {};

Do[Module[{r, E1, E2, T1, T2, T3, T4, params, subdir, outfile, stream}, r = rows[[i];
E1 = toNum[r["e1"]]; E2 = toNum[r["e2"]];
T1 = toNum[r["t1"]]; T2 = toNum[r["t2"]];
T3 = toNum[r["t3"]]; T4 = toNum[r["t4"]];
params = etrToParamsGeneral[E1, E2, T1, T2, T3, T4];
subdir = FileNameJoin[{outRoot, folderName[E1, E2, T1, T2, T3, T4]}];
If[! DirectoryQ[subdir], CreateDirectory[subdir]];
outfile = FileNameJoin[{subdir, "wannier90_hr.dat"}];
Quiet[Check[stream = OpenWrite[outfile, CharacterEncoding -> "UTF-8"];
Block[{$Output = {stream}}, hop[ham, params]];
Close[stream];
AppendTo[indexList, <|"e1" -> E1, "e2" -> E2, "t1" -> T1, "t2" -> T2, "t3" -> T3, "t4" -> T4, "hr_file" -> outfile|>];
AppendTo[failList, <|"i" -> i, "e1" -> E1, "e2" -> E2, "t1" -> T1, "t2" -> T2, "t3" -> T3, "t4" -> T4, "error" -> "hop export failed"|>]];
If[Mod[i, 10] == 0, Print["[OK] ", i, "/", n];, {i, 1, n}];
Print["[INFO] success: ", Length[indexList], " fail: ", Length[failList]];

```

```

Print["[INFO] success: ", Length[indexList], " fail: ", Length[failList]];
|打印 |长度 |长度
Export[outIndexCSV, indexList, "CSV"];
|导出
Print["[OK] wrote index: ", outIndexCSV];
|打印

If[Length[failList] > 0, Export[FileNameJoin[{workdir, "dense_linescan_hr_fail.csv"}], failList, "CSV"];
|... |长度 |导出 |文件名连接
Print["[WARN] wrote fail list: dense_linescan_hr_fail.csv"];
|打印

```

完整代码只需要修改路径即可, 我把输入文件上传到 github, 后面再补写文章。

后续用 optuna 进行超参数优化, 这个是另一个方法 (另一个 python 库)

8.5 (原创自研) 利用 wannier 验证超超交换路径

日期: 2026-02-04

在线原文: marx2001.github.io

► 说明

为验证 MNX2Y6 体系铁磁基态的产生机制, 用 wannier 提取跃迁矩阵元, 判断哪个元素主导超超交换路径。

1step.py 是提取在位能, 2step.py 是对各个原子进行超超交换路径组合, 然后除以对应的在位能, 形成一个值, 根据这个值的大小判断超超交换路径贡献, 找到路径最大的那个。

► 代码的基本用法

(1) 第一步, 将 Wannier 轨道“编号”与“物理含义”一一对应, 也就是建立一张 WF index → (原子、轨道类型、位置) 的映射表。缺少这张表时, 从 wannier90_hr.dat 抽出来的矩阵元只是数字, 无法明确其对应 Tc-Se、Se-Ir 还是 Tc-Tc 的哪条路径。

准备文件

代码清单 A.57.1

语言: python; 完整代码见附录第 1083 页, 共 3 行。

即, 第一步要确认 wannier90 拟合的正确性, 并输出上述文件。

(2) Step 2 的核心是: 将 wannier90_hr.dat 里的 Hmn(R) 变成“带几何距离的 hopping 列表”, 再按目标子空间对 (Tc-Tc、Tc-Se、Se-Ir...) 和壳层做统计。

workflow:

准备:

代码清单 A.57.2

语言: python; 完整代码见附录第 1083 页, 共 3 行。

输出:

代码清单 A.57.3

语言: python; 完整代码见附录第 1084 页, 共 2 行。

名字随意, 可以通过代码修改

代码:

代码清单 A.57.4

语言: python; 完整代码见附录第 1084 页, 共 357 行。

命令行输入:

命令示例 · shell

```
python 1step.py --win wannier90.win --centres wannier90_centres.xyz --hr
→ wannier90_hr.dat --tol 0.10 --min_absH 1e-4 --topk 30 --out_summary 1step.csv
→ --pairs ALL --skip_same_atom_R0 --out_edges 2step.csv
```

可以调整命令来规定输出文件名

(3) 读取已经生成的 2step.csv(跨原子 edges), 自动筛选 Tc-Se 与 Se-Ir 的近邻 hopping, 构造 $\text{Tc} \rightarrow \text{Se} \rightarrow \text{Ir} \rightarrow \text{Se} \rightarrow \text{Tc}$ 的候选“超超交换路径”, 并按路径权重排序输出一个 csv 文件。

对于三跳跃体系, 类似超交换, 可用下方脚本:

代码清单 A.57.6

语言: python; 完整代码见附录第 1089 页, 共 271 行。

运行方法 (针对当前文件名: 2step.csv):

代码清单 A.57.7

语言: shell; 完整代码见附录第 1093 页, 共 7 行。

如果尽可能不漏 (慢):

代码清单 A.57.8

语言: shell; 完整代码见附录第 1094 页, 共 7 行。

上面这个脚本没有测试过, 使用需谨慎。

四跃迁桥连结构

代码清单 A.57.9

语言: python; 完整代码见附录第 1094 页, 共 265 行。

使用方法:

命令示例 · shell

```
python 2step.py --edges 2step.csv --out Ir.csv --mediators Ir --d_tcse 3.0 --d_sex
→ 3.0 --min_absH 1e-4 --top_paths 2000 --require_same_tc_atom --max_netR_L1 6
→ --exclude_netR0 --delta_mode sequential --delta_floor 1e-3 --hr wannier90_hr.dat
```

命令示例 · shell

```
python 2step.py --edges 2step.csv --out Ir_0.1_delta_floor.csv --mediators Ir
→ --d_tcse 3.0 --d_sex 3.0 --min_absH 1e-4 --top_paths 2000 --require_same_tc_atom
→ --max_netR_L1 6 --exclude_netR0 --delta_mode sequential --delta_floor 1e-1 --hr
→ wannier90_hr.dat
```

分别输出 Ge 和 Ir

源文件目录:/public/home/cssong/song/1mr/9_single_layer/19_ReIrGe2S6/TcIrGeSe/ 3_static_band/4_wannier/
ncl

(4) 根据研究目标, 需要判断超超交换路径主要由提供铁电性的迁移原子 Ge 还是非磁性原子 Ir 主导, 我们可以修改脚本, 让它在输出路径时能够分别计算 Ge 和 Ir 路径的贡献, 并将这些路径的权重贡献和参与原子频率明确标识出来。

这一步与 (3) 不矛盾, 但代码有所不同, 选择 (4) 也许更准一些。

代码清单 A.57.12

语言: python; 完整代码见附录第 1098 页, 共 224 行。

要依次修改 delta_floor 的值来确保全局不会因为分子过小而导致错误。分别测试 0.1,0.5,1。

(5) 与 DFT 结果的对比

上述代码都是从全局角度分析 Ir 和 Ge 究竟谁占超超交换的主导, 而 J1 是 DFT 计算得到的 J 的主导, 通过在 TB 模型中计算 J1 元素分辨, 来判断谁的贡献更大, 代码如下:

代码清单 A.57.13

语言: python; 完整代码见附录第 1101 页, 共 46 行。

运行方法:

命令示例 · shell

```
python jl_filter_sum.py --csv Ir_1.0_delta_floor.csv --topk 100
python jl_filter_sum.py --csv Ge_1.0_delta_floor.csv --topk 100
```

上述依据:

1) 最硬依据：对 DFT 主导的 J_1 做“因果贡献”对比（主导的定义）

主导在物理上应当意味着：把某一类桥联通道削弱/切断时， J_1 发生更大的变化。

- 你要比较的是：
 - cut(Ir): 削弱/置零关键 **Se-Ir** hopping (或 Ir 相关通道)
 - cut(Ge): 削弱/置零关键 **Se-Ge** hopping (或 Ge 相关通道)
- 然后看 ΔJ_1 :

$$|\Delta J_1|_{\text{cut(Ir)}} \gg |\Delta J_1|_{\text{cut(Ge)}} \Rightarrow \text{Ir 主导}$$

反之 Ge 主导。

这是最不可争辩的判据（你现在还没做，但这是“终极版”）。

2) 你目前已经做出的“机制定量依据”：J1 壳层内的路径强度累计对比

你已经把路径分析严格对齐到 三角晶格最近邻 J1 的 6 个方向（`net_dR` 过滤），并且对 `delta_floor=0.1/0.5/1.0 eV` 做了稳健性。

你用的判据本质是：

- 先限定到 J1 对应的 Tc-Tc 配对集合（避免混壳层）
- 对每条允许的四跳路径 Tc-Se-X-Se-Tc 定义相对强度 proxy:

$$S_{\text{path}} = \frac{\prod |t|}{\prod \Delta} \quad (X = \text{Ir/Ge})$$

- 在相同筛选条件下，比较 Ir 与 Ge 的累计贡献（TopK 或全体）：

$$\Sigma_S^{\text{Ir}}(K) \text{ vs } \Sigma_S^{\text{Ge}}(K)$$

你最新的关键结果（`delta_floor=1.0 eV`，Top100）是：

- $\Sigma_S^{\text{Ir}} \approx 194.7$
- $\Sigma_S^{\text{Ge}} \approx 71.2$
- 比值 ≈ 2.74

因此在这套“强耦合微扰启发的通道强度 proxy”意义下：Ir-bridge 比 Ge-bridge 更占优，并且这是在较保守的分母能标下得到的。

这条是你现在最主要的“可发表机制证据”，但它要写成 *relative channel strength / proxy*，不能写成“精确 J 分解”。

3) 支撑性依据：结构几何 + 轨道参与（为什么会强）

这是“解释为什么”的层面，用来增强可信度：

(A) 几何是否允许、谁更接近

- Tc–Se 距离是否在合理近邻窗内
- Se–Ir 与 Se–Ge 的距离/键角分布
- 对 J1 配对，哪条桥联路径在几何上更“直”、更短、更有利于轨道重叠

(B) 轨道杂化是否强 (Wannier/PDOS/Fatband)

- 交换相关能窗内，Ir-d 与 Se-p 是否有更强的杂化（相比 Ge-p）
- MLWF 是否显示 Se–Ir 的成键型 WF 更显著
- 如果 Ir 带来更强 SOC/更强共价性，也常常使通道更“活跃”

这些不直接给“主导”结论，但它们能解释你在路径统计里看到的优势来源。

一句话总结你现在的“主导判断依据”

- 物理主导的终极定义：切断谁的通道， J_1 变化最大（因果检验）。
- 你当前已经完成的定量机制证据：在 DFT 主导的 J1 壳层内，Ir-bridge 的 TopK 累计路径强度 proxy 明显高于 Ge-bridge，并且对合理的 `delta_floor` 扫描保持稳定（例如 `delta_floor=1.0 eV` 时 Top100 比值 ≈ 2.74 ）。
- 辅证：几何与轨道杂化支持 Ir 通道更有效。

► 实际应用

比较 Ir 和 Ge 哪个贡献更大 (上面已经完成), 下方是在不考虑 soc 的情况下, 分别对 spin up 和 spin down 进行贡献比较, 确保 spin up 能代表性的指出交换路径是 d-p 杂化组成的。

(1) 产生 1step 文件

代码清单 A.57.15

语言: shell; 完整代码见附录第 1102 页, 共 3 行。

(2) 产生 2step 文件

代码清单 A.57.16

语言: shell; 完整代码见附录第 1102 页, 共 7 行。

(3) 与 DFT 进行对比

命令示例 · shell

```
python compare_DFT_J1.py --csv dw.csv --topk 100
python compare_DFT_J1.py --csv up.csv --topk 100
```

这一步的输出类似于

代码清单 A.57.18

语言: shell; 完整代码见附录第 1103 页, 共 5 行。

结论是, Ir 比 Ge 贡献大的多, 经过改变 `dleta_floor`, 也是一样的结论, spin up 和 spin down 并不能在数量上严格一致, 但只要都支持 d-p-d-p-d 结论即可, 用前面的评分标准定量分析, 用 MLWFS 的 spin up 通道定性演示 (因为 spin down 的图极为混乱, 看不清化学键)

8.6 (原创自研) 利用 wannier 验证超超交换路径 (2)

日期: 2026-02-06

在线原文: marx2001.github.io

► 说明

上一篇文章是固定 d-p-d-p-d 的超超交换路径, dp 杂化, 而不能排除 d-p-p-p-d 杂化的可能, 因此本篇文章通过修改脚本主要探究 d-p-p-p-d 杂化与 d-p-d-p-d 哪个更占据主导。1step.py 是提取在位能, 2step.py 是对各个原子进行超超交换路径组合, 然后除以对应的在位能, 形成一个值, 根据这个值的大小判断超超交换路径贡献, 找到路径最大的那个。

► 代码

(1) 此次要在同一个子空间内对比 Ge-d 和 Ge-p 的超超交换路径评分, 且考虑 SOC, 因此考虑下方的轨道:

代码清单 A.58.1

语言: python; 完整代码见附录第 1103 页, 共 6 行。

运行一次成功的 wannier 拟合之后, 进入第二步

(2) 我们需要重新修改一下代码, 因为之前的代码是硬编码 wannier centres 和元素轨道对应。

代码清单 A.58.2

语言: python; 完整代码见附录第 1103 页, 共 536 行。

运行指令

短代码 · python

```
python 1step.py --win wannier90.win --centres wannier90_centres.xyz --hr
→ wannier90_hr.dat --tol 0.10 --min_absH 1e-3 --topk 30 --out_summary 1step_up.csv
→ --pairs ALL --skip_same_atom_R0 --out_edges 2step_up.csv
```

得到文件

代码清单 A.58.4

语言: shell; 完整代码见附录第 1111 页, 共 2 行。

代码清单 A.58.5

语言: python; 完整代码见附录第 1111 页, 共 430 行。

运行指令

短代码 · python

```
python 2step.py --edges 2step_up.csv --out up.csv --mediators Ir --d_tcse 3.0 --d_sex
→ 3.0 --min_absH 1e-3 --top_paths 2000 --require_same_tc_atom --max_netR_L1 6
→ --exclude_netR0 --delta_mode sequential --delta_floor 1e-3 --hr wannier90_hr.dat
```

重复运行上面指令, 得到 Ge-d 和 Ge-p 的评分

然后增大 delta_floor 确保数值稳定, 如果出现不一致, 那么说明脚本或者公式存在问题, 需要重新考虑。

目前我的材料体系得到的结果是 Ge-d 相比于 Ir-d 和 Ge-p, Ge-d 参与的评分更高, 但是 dos 和投影能带指出, 费米能级附近的主要贡献是 Ge-p, 所以考虑 Ge-d 轨道是不合适的。

所以真正的结论是, 这是一个 d-p-d-p-d 的超超交换路径。

最终结论, 经过详实的对超超交换路径的评分, 落实了 d-p 杂化导致的强铁磁基态, 且是四面体配位的 Se 原子路径。

8.7 (原创自研) 利用 wannier 验证超超交换路径 (3)

日期: 2026-02-07

在线原文: marx2001.github.io

► 说明

上一篇文章是固定 d-p-d-p-d 的超超交换路径, dp 杂化, 而不能排除 d-p-p-p-d 杂化的可能, 因此本篇文章通过修改脚本主要探究 d-p-p-p-d 杂化与 d-p-d-p-d 哪个更占据主导。1step.py 是提取在位能, 2step.py 是对各个原子进行超超交换路径组合, 然后除以对应的在位能, 形成一个值, 根据这个值的大小判断超超交换路径贡献, 找到路径最大的那个。

这篇文章是细化成能用于论文的指标, 包括在位能等等。

文件路径:

命令示例 · shell

```
/public/home/cssong/song/lmr/9_single_layer/19_ReIrGe2S6/TcIrGeSe/3_static_band/4_wj
→ annier/ncl/20260207_detail
```

► 代码

(1) 第一步

之前的脚本里虽然已经做了 WF centre → 最近原子 (PBC minimum image) 的映射 (map_wf_to_atoms), 但真正用于分组 (Tc_d/Ir_d/Se_p/Ge_p) 的仍然是硬编码的 WF 序号区间 wf_group。并且在主循环里 gm = wf_group(m), gn = wf_group(n) 也仍然走的是这个硬编码逻辑。这就导致: 只要投影顺序、num_wann、spin 通道或 Wannier 排序发生变化, 分组就会错——因此”之前修改是否正确”的结论是: 核心问题没有被真正修掉 (最近原子映射算出来了, 但没用来分组)。

下文对该脚本进行了”最小但关键”的正确修复: 删除 wf_group(wf) 的”WF 序号区间分组” (这是不可靠的根源), 改为: 对每条 hopping, 先使用已计算出的 elem_m/elem_n (来自最近原子映射), 再进行无向格式 (便于做壳层统计), 同时在 out_edges 里额外写出 group_m/group_n, 便于后续在 2step 中筛选。

代码清单 A.59.2

语言: python; 完整代码见附录第 1118 页, 共 402 行。

运行方法:

短代码 · python

```
python 1step.py --win wannier90.win --centres wannier90_centres.xyz --hr
→ wannier90_hr.dat --skip_same_atom_R0 --out_edges edges.csv --out_summary
→ 1step.csv
```

(2) 第二步是优化 2step.py, 对应 1step.py

当前脚本虽然已经建立了 Wannier 函数中心到最近原子的映射, 但其用于轨道分组的核心逻辑仍然依赖于硬编码的 Wannier 函数序号区间。具体来说, 在主循环中判断 Tc_d、Ir_d 等分组依据的 wf_group() 函数, 正是基于此不可靠的序号范围。这导致一旦体系的投影顺序、Wannier 函数数量、自旋通道设置发生改变, 或 Wannier 函数本身经过重排, 分组结果必然出错。因此, 之前对脚本的修改并未触及这一根本问题, 已计算出的原子映射信息实际上并未被用于关键的分组步骤。

为解决此问题, 需要进行一项关键修复: 彻底删除基于序号区间的 wf_group 函数。正确的做法是, 在处理每条 hopping 时, 直接利用已计算出的 elem_m/elem_n (即映射得到的原子元素信息), 再通过一个可配置的、从元素到轨道组的映射关系 (例如 Tc->Tc_d), 来动态确定分组标签。同时, 建议通过新增命令行参数来接收此映射关系及分组顺序, 这将使脚本完全脱离对特定体系编码的依赖, 增强通用性。输出方面, pair 列可继续保持 Tc_d->Se_p 这类无向格式以利于统计, 但需在结果中额外输出 group_m/group_n 两列, 以便后续的 2step.py 脚本能够依据准确的分组信息进行筛选和计算。

目前的 1step 输出已包含 group_m/group_n 列 (例如 Tc_d、Se_p 等), 因此 2step 的核心改进在于: 应优先依据这些明确的分组信息来筛选路径和建边, 而不再依赖原始的 elem_m/elem_n (如 “Tc”、“Se” 等元素标签)。这一调整将彻底避免对不稳定的 “Wannier 函数序号范围” 的依赖。

为了实现平滑过渡, 对 2step 进行如下修改: 新版将兼容两种输入格式。当输入文件包含 group_m/group_n 列时, 程序会优先使用分组信息; 若为旧格式文件, 则自动回退至基于 elem_m/elec_n 的逻辑。本次改动将在其基础上以“最小侵入”的方式完成, 确保核心流程清晰且易于维护。

代码清单 A.59.4

语言: python; 完整代码见附录第 1124 页, 共 386 行。

使用方法:

短代码 · python

```
python 2step.py --edges edges.csv --out Ge.csv --mediators Ge --d_tcse 3.0 --d_sex
→ 3.0 --min_absH 1e-4 --top_paths 2000 --require_same_tc_atom --max_netR_L1 6
→ --exclude_netR0 --delta_mode sequential --delta_floor 0.5 --hr wannier90_hr.dat
```

(3) 第三步, 新增考虑 U 和 J。

下面提供一份完整且可直接替换的 2step.py 脚本。该版本在我之前提交的“带 U/JH 分母修正”基础上, 进一步针对当前体系的特点做了适配调整。

在参数设置上, 默认 hund_coef 取值为 2(对应更合理的 $U + 2JH$ 形式, 这是考虑到体系中 d 轨道呈 $2+2+1$ 分裂的实际情况, 因此不建议默认使用 4)。同时保留 -hund_coef 参数, 可设置为 1、2、4 等不同取值, 以进行敏感性分析。

功能方面, 保留了 -enable_UJ 开关: 开启时将使用包含 U 和 JH 的分母进行计算; 关闭则回退到仅依赖 on-site 能级差的纯能量分母模式。为了方便论文写作与结果复现, 输出 CSV 文件中会新增记录相关参数, 包括: enable_UJ、U_Tc、U_Ir、JH_Tc、JH_Ir 以及 hund_coef。

需要说明的是, 此版本默认采用元素模式 (即识别 Tc、Se、Ir 等元素标签)。如果 edges.csv 中已包含 group_m/group_n 列 (例如 Tc_d、Se_p、Ir_d), 并希望依据这些分组信息进行处理, 请运行时添加以下参数

代码清单 A.59.6

语言: python; 完整代码见附录第 1130 页, 共 472 行。

使用方法:

短代码 · python

```
python 2step.py --edges edges.csv --hr wannier90_hr.dat --out Ir_d_UJ.csv
→ --mediators Ir --enable_UJ --U_Tc 3 --U_Ir 1 --hund_coef 2 --delta_mode
→ sequential --delta_floor 0.1 --d_tcse 3.0 --d_sex 3.0 --min_absH 1e-3
→ --top_paths 2000 --require_same_tc_atom --max_netR_L1 6 --exclude_netR0
```

(4) 第四步, 筛选各个原子链

选择各个原子链中, 评分排名前 3 的原子链

代码清单 A.59.8

语言: python; 完整代码见附录第 1137 页, 共 140 行。

使用方法:

短代码 · python

```
python a.py --ir_csv Ir_d_UJ.csv --ge_csv Ge_1e-3_0.1.csv --top_each 2000
→ --top_per_chain 3 --out chains_top3.csv
```

使用两次, 考虑 netR

短代码 · python

```
python a.py --ir_csv Ir_d_UJ.csv --ge_csv Ge_1e-3_0.1.csv --top_each 2000
→ --top_per_chain 3 --key_with_netR --out chains_top3_netR.csv
```

(5) 第五步提取超超交换各个原子之间的跃迁参数

运行下方代码:

代码清单 A.59.11

语言: python; 完整代码见附录第 1139 页, 共 86 行。

使用方法:

短代码 · python

```
python b.py --in_csv chains_top3.csv --out_csv hopping_edges_from_chains.csv
```

(6) 第六步提取 Tc 之间直接交换, 与超超交换强度的数量级作比较

代码清单 A.59.13

语言: python; 完整代码见附录第 1140 页, 共 121 行。

使用方法:

短代码 · python

```
python c.py --edges edges.csv --prefer_group --tc_tag Tc_d --undirected_dedup
→ --dist_max 6.58 --topn 3 --out TcTc_direct_nn.csv
```

至此, 得到了超超交换路径与直接交换路径的定量表述。

8.8 (原创自研) 利用 PDOS 验证超超交换路径

日期: 2026-02-08

在线原文: marx2001.github.io

► 说明

暂无

► 代码

用这个代码, 计算出设置范围内的指定原子的价带顶和导带底的分轨道能级排布顺序。

代码清单 A.60.1

语言: python; 完整代码见附录第 1142 页, 共 297 行。

使用方法

命令示例 · shell

```
python lstep.py --up PDOS_Tc_UP.dat --dw PDOS_Tc_DW.dat --out_prefix Tc
[Ef] using Ef = 0.0 eV on the PDOS energy axis.
```

8.9 (原创自研) 利用 wannier 验证超超交换路径 (4)

日期: 2026-02-08

在线原文: marx2001.github.io

► 说明

本篇文章是此系列最后一篇文章, 提取跃迁矩阵元, 将其对角化后量化 d 轨道和 p 轨道的分轨道的相对能量并画图。文件路径:

命令示例 · shell

```
/public/home/cssong/song/lmr/9_single_layer/19_ReIrGe2S6/TcIrGeSe/3_static_band/4_wj
↪ annier/ncl/20260207_detail/crystal_spiltl
```

► 代码

(1) 第一步

核心用途: 从 Wannier90 的 wannier90_hr.dat 里提取每个原子对应的“局域在位 (on-site) 哈密顿量”子块, 做本征分解得到“局域能级”(可理解为晶场/原子轨道能级的数值化近似), 并且用 wannier90.win 的投影顺序给这些能级打上 px/py/pz/dxy/... 之类的轨道标签, 最后输出若干 CSV 统计文件。

该脚本通过读取三个关键文件构建材料模型:

wannier90_hr.dat(对应 - hr 参数): 提取 $R=(0,0,0)$ 的矩阵元, 构建 on-site 哈密顿量并做厄米化处理 ($0.5*(H_0+H_0^\dagger)$)。

edges.csv(对应 - edges 参数): 提供 Wannier 函数与原子/元素的归属关系, 必须包含 m,n,atom m,atom n,elem m,elem n 列; 若同一 WF 被标记为不同原子或元素, 脚本会报错并要求修正一致性。

wannier90.win(对应 - win 参数): 仅解析 begin projections...end projections 块, 获取各元素的投影轨道信息 (支持自动将 d/p/s 展开为 5d/3p/1s)。

以上文件共同用于构建哈密顿量并关联轨道、原子与元素信息。

总的来说: 专门用于分析 Wannier 函数局域能级的工具。它通过整合三份输入文件——`hr.dat`(提供哈密顿量矩阵元)、`edges.csv`(定义 Wannier 函数与原子的归属关系) 和 `wannier90.win`(规定轨道的投影顺序)——来构建并分解每个原子附近的电子结构。

其核心计算流程分为六步: 首先从哈密顿量中提取 on-site 相互作用部分; 接着将 Wannier 函数按原子分组; 然后依据投影信息为这些函数打上轨道标签; 之后对每个原子的哈密顿量子块进行本征分解, 获得局域能级; 再为每个能级标记出其主导轨道成分; 最后还可选择性地进行能级配对诊断, 以分析能级分裂特征。

脚本会输出三份 CSV 结果文件:`levels.csv` 详细列出每个原子的所有能级及其主导轨道;`pair_centers.csv` 提供能级配对的诊断信息;`orbital_order.csv` 则汇总了每个原子各类轨道的最低能量。总而言之, 该工具将抽象的哈密顿量矩阵转化为清晰、按原子和轨道分解的能级图谱, 极大地便利了后续的材料电子结构分析与可视化。

代码清单 A.61.2

语言: python; 完整代码见附录第 1147 页, 共 350 行。

使用方法:

命令示例 · shell

```
python cf_final_hr_win_edges.py --Ef -1.4631 --out_prefix cfA2
```

(2) 第二步

这是一个用于分析材料中元素局域能级分布的 Python 脚本。其核心功能是对前序脚本 (如 `cf_final_hr_win_edges.py`) 生成的 `*_levels.csv` 文件进行高阶统计分析。

脚本首先对原始数据进行清洗, 过滤掉主导轨道标签不可靠的能级 (如轨道权重过低或为空), 确保分析质量。随后, 按“元素-主导轨道”组合进行分组, 计算每个组内能级能量的详细统计量, 包括均值、标准差、分位数等。最重要的, 脚本会汇总每个元素在不同轨道上的中位数能级, 并通过计算其极差, 生成一个衡量“晶场分裂强度”或“轨道能级展开”的实用摘要。

最终, 脚本会输出三个 CSV 文件: 一份包含所有有效能级的详细长表、一份每个元素-轨道组合的统计量表, 以及一份汇总了各元素轨道能级展开范围的核心摘要, 从而将原子尺度的复杂数据提炼为元素层面的清晰物理图像。

代码清单 A.61.4

语言: python; 完整代码见附录第 1152 页, 共 95 行。

使用方法:

短代码 · python

```
python collect_elem_cf_stats.py --levels cfA2_levels.csv --out_prefix elemCF
↪ --min_dom_w 0.20
```

(3) 第三步

该脚本 (`cfFINAL_export_fullweights.py`) 在材料电子结构分析中实现了关键增强。它首先读取三个必要文件:`wannier90_hr.dat`(构建 on-site 哈密顿量)、`edges.csv`(建立 Wannier 函数到原子的唯一映射) 和 `wannier90.win`(确定投影轨道顺序), 并严格校验 WF 顺序与投影顺序的一致性以保证后续标签准确。

其核心计算分为两步: 对每个原子的 on-site 哈密顿量子块进行对角化, 得到局域能级; 然后, 对每条能级的本征态, 计算其在所有基轨道 (如 px、py、pz、dxy 等) 上的详细权重分布。这是相较于旧版工具的最大提升——旧版仅输出主导轨道, 而新版会导出所有轨道的精确占比。

脚本最终输出三个 CSV 文件, 其中最重要的 *_levels_fullweights.csv 文件完整记录了每条能级的全轨道权重。这使得用户可以直接提取特定元素 (如硒 Se) 的所有 px、py、pz 轨道权重, 进而进行加权的分布统计、绘制能级展开图等深入分析, 而不再局限于之前仅基于主导轨道的简化统计。

代码清单 A.61.6

语言: python; 完整代码见附录第 1153 页, 共 269 行。

命令示例 · shell

```
python cfFINAL_export_fullweights.py --out_prefix cfFW
```

(4) 第四步

collect_elem_orbital_distributions.py 脚本是用于分析材料中元素轨道能量分布的专业工具。其核心功能是基于包含全轨道权重的能级数据 (*_levels_fullweights.csv), 对每个元素的各个轨道 (如 Se 的 px、py、pz) 进行加权统计, 从而解决了此前提出的“分析 Se 的 px/py/pz 整体分布”的需求。

具体而言, 它对每个元素和轨道 (如 w_px) 执行以下计算: 首先, 将每条能级的相对能量 E_rel 视为样本值, 并将其在该轨道上的权重 w_orb 作为样本权重; 接着, 过滤掉权重过小的噪声数据; 然后, 计算加权平均值、标准差、分位数等统计量; 最后, 生成加权的能量分布直方图数据。

脚本默认输出两个 CSV 文件: 一个汇总每个元素-轨道组合的加权统计数据 (_weighted_stats.csv), 另一个则提供用于绘制分布曲线或热图的直方图数据点 (_hist.csv)。

与此前按主导轨道进行“硬分类”的脚本 collect_elem_cf_stats.py 不同, 本脚本利用全轨道权重进行“软分解”统计, 能更精确地刻画每个轨道对整体能级的贡献分布。

代码清单 A.61.8

语言: python; 完整代码见附录第 1157 页, 共 119 行。

使用方法:

命令示例 · shell

```
python collect_elem_orbital_distributions.py --full cfFW_levels_fullweights.csv
→ --out_prefix elemCFdist --bins 60.
```

(5) 第五步绘图

配套的绘图工具, 用于将前序统计分析的结果进行可视化。

plot_elem_level_diagram.py 用于绘制元素的轨道能级示意图。它读取包含加权统计量 (如中位数、四分位数) 的文件, 为指定元素的每个轨道绘制一条以加权中位能量为中心的水平线段, 并可选择添加四分位距区间以示意能级分布宽度。其产出类似一张简明的“能级图”, 直观展示各轨道能量的中心位置与离散程度。

plot_elem_orb_distributions.py 则用于绘制元素-轨道的能量分布曲线图。它基于直方图数据文件, 为指定元素的每个轨道绘制一条完整的分布曲线, 横轴为能量, 纵轴为加权强度, 从而完整呈现每个轨道在能量空间中的分布形状、峰位和展宽。

两者的核心区别在于信息压缩程度: 前者将每个轨道的信息压缩为一个代表性能级 (可带误差棒), 产出示意图; 后者则保留并展示完整的分布轮廓, 产出细节曲线图。它们从不同维度呈现同一组数据, 共同服务于对材料中元素局域电子结构的深入理解与展示。

代码清单 A.61.10

语言: python; 完整代码见附录第 1159 页, 共 88 行。

代码清单 A.61.11

语言: python; 完整代码见附录第 1161 页, 共 84 行。

使用方法

代码清单 A.61.12

语言: shell; 完整代码见附录第 1162 页, 共 10 行。

(6) 第六步旋转坐标系

第五步的脚本相对于 eg 和 t2g 这种经典情况理论上是适用的, 因为没有旋转局域坐标系, 我的材料并不是通常的畸变情况, 所以不适用, 因此下方脚本是针对这个问题而写的。

代码清单 A.61.13

语言: python; 完整代码见附录第 1162 页, 共 583 行。

使用方法:

命令示例 · shell

```
python local_cf_octa_Tc.py --poscar POSCAR --hr wannier90_hr.dat --edges edges.csv
↪ --win wannier90.win --center Tc --neighbor Se --out_prefix TcCF

python local_cf_octa_Tc.py --poscar POSCAR --hr wannier90_hr.dat --edges edges.csv
↪ --win wannier90.win --center Tc --neighbor Se --center_atom_id 2 --out_prefix
↪ Tc2CF
```

这个脚本确实有效, 但是由于 wannier 考虑 soc 是一种较为”混乱“的情况, 包含了杂化等等, 所以即使旋转了坐标系, 也是不能实现和 dft 计算中的 pdos 同样的简并情况。

(7) 第七步通过 pdos 中的 d 轨道

计算 bandcenters, 得到准确的轨道能级分布。

输入文件, dft 计算 dos 后, 用 vaspkit 导出的数据文档。

代码清单 A.61.15

语言: python; 完整代码见附录第 1171 页, 共 2 行。

代码如下:

代码清单 A.61.16

语言: python; 完整代码见附录第 1171 页, 共 121 行。

运行代码, 能量范围 -1 到 0, 价带顶:

命令示例 · shell

```
python pdos_auto_group.py --up PDOS_Tc_UP.dat --dw PDOS_Tc_DW.dat --Emin -1.0 --Emax
→ 0.0 --out_prefix TcPDOS
```

该脚本的核心原理: 将轨道投影态密度 (PDOS) 视为能量分布函数 $D\alpha(E)$, 在用户指定的能量窗口内, 通过计算其一阶矩 (即 “带中心”) 来定义每个轨道或轨道组的有效能量位置, 并利用其零阶矩 (积分面积) 来量化该轨道在该窗口内的总谱权重。

该原理的适用范围及学术写作中的准确表述方式。该方法提供的是基于 PDOS 的 “有效能级对齐”, 而非来自局域哈密顿量的严格晶场本征值。在用于绘制轨道能级示意图并解释超交换等机制时, 建议在论文中表述为 “根据占据能窗内的轨道投影态密度谱重中心确定各轨道组的相对能量位置”, 这样的表述更为严谨。

然后绘图

代码清单 A.61.18

语言: python; 完整代码见附录第 1173 页, 共 208 行。

使用方法

命令示例 · shell

```
python plot_level_diagram.py --orb TcAuto_orb_centers.csv --groups
→ TcAuto_auto_groups.csv --out Tc_level.png --title "Tc d-level splitting" --eref
→ -1.46
```

8.10 (原创自研) 利用 wannier 计算超超交换过程中的轨道杂化

日期: 2026-02-13

在线原文: marx2001.github.io

► 说明

为了严谨, 参考一篇 CPL 文章给出的超交换示意图, 以及 d-p 杂化轨道的情况, 本篇文章写脚本来提取自己体系的杂化情况。

► 代码

(1) 获取 edges.csv

代码清单 A.62.1

语言: python; 完整代码见附录第 1176 页, 共 375 行。

使用方法: 需要去除掉所有的换行符:

代码清单 A.62.2

语言: shell; 完整代码见附录第 1182 页, 共 8 行。

(2) 提取想要的原子对之间的杂化情况

代码清单 A.62.3

语言: python; 完整代码见附录第 1182 页, 共 361 行。

使用方法:

代码清单 A.62.4

语言: shell; 完整代码见附录第 1187 页, 共 8 行。

(3) 使用脚本, 计算 dd 轨道杂化情况

代码清单 A.62.5

语言: python; 完整代码见附录第 1188 页, 共 384 行。

使用方法:

命令示例 · shell

```
python r3.py --poscar POSCAR --edges edges_with_orb_reim.csv --outdir Fig5_like_TcTc
↪ --A Tc --B Tc --kind dd --dmin 6.4 --dmax 6.6 --top_bonds 6
--strict_spinor --unique_bonds
```

前面的文章提到了有原子序号的超超交换路径, 直接找序号, 然后选择最小的 `dist_atom` 即可, 就是最近的这个键的杂化情况。

今天白天开了 1400 公里的车, 凌晨 2:07 完成此篇文章。

8.11 (原创自研) 机器学习-1

日期: 2026-03-21

在线原文: marx2001.github.io

► 说明

均为原创, 禁止抄袭, 转载注明出处, 否则必会追究。

► Stage 0: 样本生成

任务:

- 生成第一批 1200 个标准化 `hr.dat` 样本
- 写入 `sample_id / zone / raw_vars / signed_features`
- 生成 `sample_file_manifest.csv`

脚本:

- `scripts/00_generate_stage0_dataset_standard.wl`

输出文件夹名称:

- `raw_hr_stage0_global_random`

建议正式归档路径:

- `01_raw_models/stage0_global_random/`

该目录下主要文件:

- `wannier90_nb2se2o_g0000001.dat`
- ...
- `sample_file_manifest.csv`
- `bad_export_log.txt`

► Stage 1: hr.dat 清洗与格式标准化

任务:

- 清洗 `hr.dat`
- 兼容前部四行注释
- 自动搜索真正的 `num_wann / nrpts`
- 校验 `degeneracy` 与 `hopping` 行
- 为后续 `band / chern` 计算做准备

脚本:

- `scripts/01_clean_hr_stage0_standard.py`

输出文件夹名称:

- `clean_raw_hr_stage0_global_random`

建议正式归档路径:

- `02_preprocess/clean_stage0_global_random/`

该目录下主要文件:

- 清洗后的 `wannier90*.dat`
- `sample_file_manifest.csv`
- `bad_clean_files.log`

► Stage 2: 能带 / 带隙计算

任务:

- 读取清洗后的 `hr.dat`
- 解析 `sample_id / zone / 参数`
- 计算 `band / gap / midgap EF`
- 输出样本级 `band` 结果与汇总表

脚本:

- `scripts/02_band_gap_stage0_standard.py`

输出文件夹名称:

- bands_output_stage0_global_random

建议正式归档路径:

- 03_physics_calc/band/stage0_global_random/

该目录下主要文件:

- 每个样本一个子目录
 - bands.csv
 - bands.png
 - band_info.txt
- all_band_summary.csv
- sample_file_manifest_band.csv

► Stage 3:Chern 三通道计算

任务:

- 读取清洗后的 hr.dat
- 计算多网格 Chern
- 同时输出三通道结果:
 1. chern_reliable_strict
 2. boundary_sensitive_flag / boundary_sensitive_score
 3. numeric_bad_flag
- v2 版本中已将 extreme_min_link_amp 从硬坏点改成边界敏感信号

脚本:

- scripts/03_chern_stage0_dualchannel.py
- scripts/03_chern_stage0_dualchannel_v2.py(当前推荐使用)

输出文件夹名称:

- chern_output_stage0_global_random_dualchannel

建议正式归档路径:

- 03_physics_calc/chern/stage0_global_random/

该目录下主要文件:

- 每个样本一个子目录
 - chern_info.txt
- all_chern_summary.csv
- sample_file_manifest_chern.csv
- bad_jobs.log

► Stage 4:merge 与 stage0 标准化数据集生成

任务:

- 合并 all_band_summary.csv 与 all_chern_summary.csv
- 生成完整 stage0 数据集
- 生成统一 sample_file_manifest.csv

- 输出可直接并入主表的 `master_stage0_ready.csv`
- 解决参数列 `merge` 后变成 `e1_band` / `e1_chern` 的问题

脚本:

- `scripts/04_merge_to_master_stage0_standard.py`

输出文件夹名称:

- `merged_stage0_global_random`

建议正式归档路径:

- `04_datasets/stage0_raw_merged/`

该目录下主要文件:

- `final_dataset_stage0_global_random_standard.csv`
- `sample_file_manifest.csv`
- `master_stage0_ready.csv`

► Stage 5: 主表建立与维护

任务:

- 将 `master_stage0_ready.csv` 并入 `master_dataset_all.csv`
- 若主表不存在则自动初始化
- 主表始终保留表头
- 后续各批次数据统一并入主表

脚本:

- `scripts/append_stage_ready_to_master.py`

主表文件夹名称:

- `master_tables`

建议正式路径:

- `04_datasets/master_tables/`

主要文件:

- `master_dataset_all.csv`

辅助 `manifest` 文件夹:

- `04_datasets/manifests/`

可放置文件:

- `stage0_sample_file_manifest.csv`
- 后续各阶段 `manifest`

► Stage 6: Level A 机器学习 (绝缘体任务)

任务:

- 用 8 参数学习 `is_physical_insulator`
- 建立第一轮可训练验证基线
- 比较 RF 与 XGBoost

- 输出特征重要性、预测结果、混淆矩阵、日志

脚本:

- scripts/stage1_rf_is_insulator_baseline.py
- scripts/stage1_xgboost_is_insulator.py

输出总文件夹名称:

- 05_ml_models/levelA_insulator/

RF 输出路径:

- 05_ml_models/levelA_insulator/random_forest/

XGBoost 输出路径:

- 05_ml_models/levelA_insulator/xgboost/

每个模型目录下主要文件:

- metrics.txt
- metrics.json
- feature_importance.csv
- predictions_test.csv
- confusion_matrix.png
- model.joblib

日志路径:

- logs/ml/stage1_rf_is_insulator_baseline.log
- logs/ml/stage1_xgboost_is_insulator.log

► Stage 7: Level B 机器学习 (拓扑倾向任务)

任务 (下一步计划):

- 用 8 参数学习 is_topo_by_chern
- 比较 RF 与 XGBoost
- 判断“参数 -> 拓扑倾向”的可学性

计划脚本名称 (尚未正式写出时的建议命名):

- scripts/stage2_rf_is_topo_by_chern_baseline.py
- scripts/stage2_xgboost_is_topo_by_chern.py

建议输出路径:

- 05_ml_models/levelB_topo/random_forest/
- 05_ml_models/levelB_topo/xgboost/

► Stage 8: 边界候选搜索

任务 (后续计划):

- 联合 $P(\text{insulator})$ 与 $P(\text{topo})$
- 结合 boundary_sensitive_flag / score
- 识别拓扑相界候选区
- 为局部 refine 提供种子点

建议主文件夹名称:

- 06_boundary_search/

建议子路径示例:

- 06_boundary_search/stage0_candidates/
- 06_boundary_search/joint_probability_analysis/
- 06_boundary_search/top_candidates/

► Stage 9: 局部 refine / 候选点回算

任务 (后续计划):

- 在边界候选区附近做局部采样
- 生成 refine 样本
- 重新计算 band / chern
- 并回主表
- 进一步逼近真正 TI 区域或相界

建议主文件夹名称:

- 07_local_refine/

建议子路径示例:

- 07_local_refine/windows_t2_r3/
- 07_local_refine/windows_t2_r3_r2/
- 07_local_refine/batch_001/
- 07_local_refine/batch_002/

► Stage 10: 图件、报告、总结

任务:

- 画图
- 汇总阶段性结果
- 形成组会 / 论文 / 项目说明材料

建议文件夹名称与路径:

- 08_figures/
- 09_reports/stage_summaries/

可放置内容:

- 图件
- 数据体检报告
- 阶段总结 txt
- README 草稿
- 数据字典
- 模型结果总结

► 当前已经确定的重要文件路径 (按功能)

1. 原始样本
 - 01_raw_models/stage0_global_random/
2. 清洗后 **hr.dat**
 - 02_preprocess/clean_stage0_global_random/
3. **band** 结果
 - 03_physics_calc/band/stage0_global_random/
4. **chern** 三通道结果
 - 03_physics_calc/chern/stage0_global_random/
5. **stage0 merge** 数据集
 - 04_datasets/stage0_raw_merged/final_dataset_stage0_global_random_standard.csv
6. **stage0 manifest**
 - 04_datasets/manifests/stage0_sample_file_manifest.csv
7. **stage0 ready** 表
 - 04_datasets/stage0_raw_merged/master_stage0_ready.csv
8. 当前正式主表
 - 04_datasets/master_tables/master_dataset_all.csv
9. **Stage1 RF** 输出
 - 05_ml_models/levelA_insulator/random_forest/
10. **Stage1 XGBoost** 输出
 - 05_ml_models/levelA_insulator/xgboost/
11. 日志
 - logs/ml/

► 当前阶段已经完成到哪里

已经完成:

- Stage 0: 样本生成
- Stage 1: 清洗
- Stage 2: band / gap 计算
- Stage 3: Chern 三通道计算 (v2 已修正)
- Stage 4: merge
- Stage 5: 主表建立
- Stage 6: Level A 机器学习 (is_physical_insulator, RF 与 XGBoost 已跑通)

当前下一步:

- Stage 7: Level B 机器学习 (is_topo_by_chern)

► 当前最重要的文件

如果只保留最关键的文件, 当前应重点关注:

1. 主表
 - 04_datasets/master_tables/master_dataset_all.csv

2. Stage0 ready

- 04_datasets/stage0_raw_merged/master_stage0_ready.csv

3. band 汇总

- 03_physics_calc/band/stage0_global_random/all_band_summary.csv

4. chern 汇总

- 03_physics_calc/chern/stage0_global_random/all_chern_summary.csv

5. XGBoost 结果

- 05_ml_models/levelA_insulator/xgboost/

6. RF 结果

- 05_ml_models/levelA_insulator/random_forest/

8.12 (原创自研) 机器学习-2

日期: 2026-03-21

在线原文: marx2001.github.io

► 说明

均为原创, 禁止抄袭, 转载注明出处, 否则必会追究。

► 项目主线

当前项目不是单纯“训练一个分类器”, 而是围绕 Nb₂Se₂O 在 MSG {123,342} 下的紧束缚参数空间, 建立一套:

1. 样本生成
2. hr.dat 清洗
3. 能带 / 带隙计算
4. Chern 三通道计算
5. 统一主表
6. 机器学习分类
7. 拓扑相界候选筛选
8. 后续局部精修与机制解释

的完整工程。

研究目标可以概括为:

- 用 8 个核心参数 (e1, e2, t1, t2, r1, r2, r3, r4) 学习和预测拓扑性质
- 区分“绝缘体倾向”“拓扑倾向”“边界敏感样本”
- 用机器学习辅助寻找拓扑相界
- 最终回到参数作用和物理机制解释

当前已经完成到:

“第一批 1200 个标准化样本 + 主表建立 + Stage1 绝缘体机器学习”这一步。

► 目录结构与主表

已经确定采用长期主表方案:

- `master_dataset_all.csv` 始终保留表头
- 每一批数据先输出成 `master*_ready.csv`
- 再并入 `master_dataset_all.csv`

已经确认:

- `master_stage0_ready.csv` 是正确的 stage0 标准化接口表
- 最新的 `master_dataset_all.csv` 没有问题, 可以作为新主表使用

新主表当前状态:

- 1200 行
- 32 列
- `sample_id` 无重复
- 8 个核心参数全部非空:
`e1, e2, t1, t2, r1, r2, r3, r4`
- 路径列正常:
`hr_path, band_path, chern_path`
- 标签列正常:
`gap_min, is_physical_insulator, chern, is_topo_by_chern, chern_reliable, chern_reliable_strict, boundary_sensitive_flag, boundary_sensitive_score, numeric_bad_flag, is_TI`

当前新主表可直接用于机器学习。

► 标准化脚本链条

1. `00_generate_stage0_dataset_standard.wl`

作用:

- 生成 1200 个标准命名的 `hr.dat`
- 写入 `sample_id / zone / raw_vars / signed_features`
- 同时生成 `sample_file_manifest.csv`

输出目录:

- `raw_hr_stage0_global_random/`

说明:

- `sample_id` 采用标准化命名, 避免后期补映射
- 不再依赖旧的 `sample1/sample2` 体系

2. `01_clean_hr_stage0_standard.py`

作用:

- 清洗 `hr.dat`
- 兼容前部四行注释
- 不再错误地把 header 后第一行直接认成 `num_wann`
- 在整个 body 中搜索真正的 `num_wann / nrpts`
- 校验 `degeneracy` 与 `hopping` 行

输出目录:

- `clean_raw_hr_stage0_global_random/`

说明:

- 之前清洗报错 “cannot find num_wann line” 已经修好
- 新版清洗脚本可以兼容当前这批 `hr.dat` 的真实格式

3. `02_band_gap_stage0_standard.py`

作用:

- 读取清洗后的 `hr.dat`
- 解析 `sample_id` / `zone` / 参数
- 计算 `band` / `gap` / `midgap` EF
- 输出 `all_band_summary.csv` 与样本级 `band` 结果

输出目录:

- `bands_output_stage0_global_random/`

说明:

- `all_band_summary.csv` 中参数列是正常存在的
- 后面参数丢失的问题不在这里, 而在 `merge`

4. `03_chern_stage0_dualchannel.py`

最初版本建立了三通道思想:

- `chern_reliable_strict`
- `boundary_sensitive_flag` / `score`
- `numeric_bad_flag`

但最初 `numeric_bad` 过强:

- `extreme_min_link_amp` 被当作硬坏点
- 导致 `boundary_flag` 过少, `numeric_bad` 过多

5. `03_chern_stage0_dualchannel_v2.py`

这是后续修正版, 必须记住。

关键修改:

- 不再把 `extreme_min_link_amp` 直接视为 `numeric_bad`
- 把它改成 “边界敏感信号”
- 提高 `boundary_sensitive_score`
- `boundary_sensitive_reason` 会记为:
`extreme_link_amp_boundary_like`

这样做的原因:

- 近相界、小带隙、Berry curvature 尖锐、网格敏感的样本, 不应该被直接丢掉
- 它们正是拓扑相界研究最有价值的样本

输出目录:

- `chern_output_stage0_global_random_dualchannel/`

6. `04_merge_to_master_stage0_standard.py`

这是 `merge` 标准版脚本, 后续又修过两轮。

关键修正点:

- 解决 `DataFrame` 构造时 “Per-column arrays must each be 1-dimensional” 问题
- 解决参数列丢失问题:
`merge` 后参数列可能变成 `e1_band` / `e1_chern`, 旧版只找 `e1`, 导致 `master_stage0_ready.csv` 中参数全空
- 新版优先自动识别:
`e1 -> e1_band -> e1_chern`
其余参数同理

最终效果:

- `master_stage0_ready.csv` 参数全部恢复
- 能正确写入 `strict / boundary / numeric_bad` 三通道字段

7. `append_stage_ready_to_master.py`

作用:

- 把 `master_stage0_ready.csv` 这种标准化 ready 表并入 `master_dataset_all.csv`
- 如果主表不存在或为空, 则自动初始化
- 自动对齐列
- 按 `sample_id` 去重, 保留后者
- 主表始终保留表头

最终决定:

- 主表始终保留表头, 这是推荐方案
- 旧主表不再继续用
- 当前主表直接从新的 `stage0` 数据重新开始

► 数据统计结论

最终 merge 成功后的最新统计 (以最新正确版本为准):

- 总样本数:1200
- `is_physical_insulator` = 1:340
- `is_topo_by_chern` = 1:351
- `chern_reliable_strict` = 1:814
- `boundary_sensitive_flag` = 1:388
- `numeric_bad_flag` = 1:0
- `is_TI` = 1:0
- `is_near_gapless` = 1:7

解释:

1. 绝缘体和拓扑倾向并不缺

- 绝缘体有 340 个
- topo 倾向有 351 个
- `strict reliable` 有 814 个

说明样本本身不是“什么都没有”。

2. 真正缺的是稳定 TI 交集

`is_TI` 仍然为 0, 说明:

- 稳定拓扑绝缘体交集仍极薄
- 当前最重要的不是直接学 `is_TI`
- 而是先学:
 - `is_physical_insulator`
 - `is_topo_by_chern`
 - `boundary_sensitive_flag`

3. `boundary` 通道已经变得合理

原来旧版结果里:

- `boundary_flag` 只有 6
- `numeric_bad` 高达 382

在把 `extreme_min_link_amp` 从硬坏点改成边界敏感信号后:

- `boundary_flag` 变成 388
- `numeric_bad` 清零

这个变化非常关键, 说明:

- 近相界样本终于被正确保留下来
- 三通道设计方向是对的

► 文献对模型路线的影响与最终模型选择

结合研究目的与文献结论, 最终决定:

1. 当前阶段不需要优先下载 **GitHub** 代码改造

原因:

- 当前任务是低维表格型参数学习, 不是 Wilson loop / gauge-equivariant 输入
- 最需要的是:
参数 -> 相区 / 相界 / 机制解释
- 自己写干净、可控的脚本更适合当前阶段

2. 当前正式主模型选择

- 主模型: XGBoost
- 对照模型: Random Forest

原因:

- 与当前 8 参数表格型输入最匹配
- 与文献中表格学习最佳实践一致
- 兼顾性能和可解释性

3. 当前不建议直接上

- gauge-equivariant neural network
- 深度网络
- TDA/PD 无监督主模型

它们更适合后续波函数几何 / Wilson loop / persistence diagram 专题, 不是当前第一主线。

► Stage1 机器学习脚本

已经写好并成功运行:

1. `stage1_rf_is_insulator_baseline.py`

作用:

- 第一版对照模型
- 用 RF 学 8 参数 -> `is_physical_insulator`
- 稳定、直观、可解释
- 用来确认主表是否可训练
- 给出基线指标和特征重要性

输出目录:

- `05_ml_models/levelA_insulator/random_forest/`

输出文件:

- `metrics.txt`
- `metrics.json`

- feature_importance.csv
- predictions_test.csv
- confusion_matrix.png
- model.joblib

日志:

- logs/ml/stage1_rf_is_insulator_baseline.log

2. stage1_xgboost_is_insulator.py

作用:

- 第一阶段正式主模型
- 用 XGBoost 学 8 参数 -> is_physical_insulator
- 作为后续 is_topo_by_chern / boundary_sensitive_flag 的模板

输出目录:

- 05_ml_models/levelA_insulator/xgboost/

输出文件:

- metrics.txt
- metrics.json
- feature_importance.csv
- predictions_test.csv
- confusion_matrix.png
- model.joblib

日志:

- logs/ml/stage1_xgboost_is_insulator.log

3. stage1_model_usage_guide.txt

作用:

- 说明两个脚本的用途
- 说明为什么先跑 RF, 再跑 XGBoost
- 说明日志输出与结果目录

► Stage1 绝缘体任务结果

1. XGBoost 结果

- accuracy = 0.854167
- precision = 0.789474
- recall = 0.661765
- f1 = 0.720000

测试集分类报告:

Class 0:

- precision = 0.874317
- recall = 0.930233
- f1 = 0.901408

Class 1:

- precision = 0.789474
- recall = 0.661765
- f1 = 0.720000

2. Random Forest 结果

- accuracy = 0.850000
- precision = 0.833333
- recall = 0.588235
- f1 = 0.689655

测试集分类报告:

Class 0:

- precision = 0.854167
- recall = 0.953488
- f1 = 0.901099

Class 1:

- precision = 0.833333
- recall = 0.588235
- f1 = 0.689655

3. 对比结论

- XGBoost 略优于 RF
- RF precision 稍高
- 但 XGBoost recall 与 f1 更好
- 对当前研究目标来说,XGBoost 更适合作为主模型

研究判断:

- 当前 1200 样本已经足够支撑第一轮机器学习
- 8 参数到 `is_physical_insulator` 的映射是“可学”的
- 数据链条已经从样本生成到模型训练完全打通
- 当前结果已经达到“可以正式进入下一阶段”的标准

► 总判断

1. 当前不是“没有结果”

实际上已经成功建立了:

- 标准化样本生成
- `hr.dat` 清洗
- `band/gap` 计算
- Chern 三通道计算
- 主表
- 第一轮机器学习

2. 当前研究主线最清楚的结论

- 绝缘体可学
- 拓扑倾向样本足够
- 稳定 TI 仍然极少
- 相界样本必须通过 `boundary` 通道保留, 不能再用过强 `hard filter` 直接丢弃

3. 当前最重要的方法学结论

- 不能再“单一 strict reliability”筛所有 Chern 样本
- 必须区分:
 - a. 稳定拓扑相 (strict)
 - b. 边界敏感样本 (boundary)
 - c. 真正坏点 (numeric_bad)

这一点已经通过 1200 样本结果验证。

► 下一步

当前 `is_physical_insulator` 任务已经跑通, 所以下一步不应继续停留在绝缘体任务上。

下一步正式任务应为:

1. `stage2_xgboost_is_topo_by_chern.py`
 - 目标: `is_topo_by_chern`
 - 主模型: XGBoost
2. `stage2_rf_is_topo_by_chern_baseline.py`
 - 目标: `is_topo_by_chern`
 - 对照模型: Random Forest

原因:

- 绝缘体任务已经证明可学
- 现在应验证 8 参数 -> 拓扑倾向是否也可学
- 这一步比绝缘体更接近项目主线

后续再进一步的任务可考虑:

- `boundary_sensitive_flag` 学习
- 联合 $P(\text{insulator}) + P(\text{topo})$ 做边界候选筛选
- 在高分边界区做局部 refine
- 回到参数机制解释

8.13 (原创自研) 机器学习-3

日期: 2026-03-21

在线原文: marx2001.github.io

► 说明

均为原创, 禁止抄袭, 转载注明出处, 否则必会追究。

► 项目主线

本项目的目标不是单纯训练一个分类器, 而是构建“物理模型扫描 + 严格机器学习判别 + 闭环定向 refine”的完整流程, 用于研究 $\text{Nb}_2\text{Se}_2\text{O}$ 对应 8 参数最小模型中的拓扑相界、拓扑平台和参数主导机制。

整体主线是:

1. 用 MagneticTB / Mathematica 在参数空间取样;
2. 通过 band + chern 计算得到物理标签;
3. 合并成总主表 `master_dataset_all.csv`;
4. 训练严格三层模型 A / B / C;
5. 用 joint 结果反向指导下一轮局部 refine;

6. 最终从闭环结果中提取 core / trivial / boundary 的代表集合和最终相图。

► 理论依据

1. 参数空间模型

当前工作的对象不是传统材料描述符, 而是具有明确物理意义的 8 参数最小模型:

- e1, e2
- t1, t2
- r1, r2, r3, r4

因此这是一个“物理参数空间中的相结构学习问题”。

2. 为什么不是只做 brute-force

如果完全靠 brute-force, 在高维参数空间中寻找相界和平台非常耗时, 而且边界点很稀疏、很难正好扫到。

因此引入机器学习, 目的不是替代理论计算, 而是:

- 从已有样本中学习 topo / trivial / boundary 的结构;
- 反向告诉我们下一轮更值得算哪些区域;
- 提高局部 refine 的效率。

► 三层机器学习流程

1. Level A

目标:

- 判断一个样本是否是 physical insulator

作用:

- 先过滤掉明显不合理的金属 / 近隙闭合样本
- 为 topo 判断提供物理合理性约束

结论:

- Level A 一直较稳定, 不是瓶颈
- 它在 joint 中相当于“物理门控层”

2. Level B

目标:

- 判断一个样本是否是 topo(target = is_topo_by_chern)

作用:

- 这是整个流程最核心的一层
- 直接决定 topo / trivial / boundary 的区分能力

为什么难:

- topo / trivial 本来就是参数空间中的模糊过渡
- 大量样本靠近相界
- 不是一个天然完全可分的任务

关键结论:

- 初期 strict 版 Level B 区分能力不够强
- 通过定向增样和多轮 refine 数据回流后, Level B 的 roc_auc / ap 明显提升
- 说明 ML 闭环确实有效

3. Level C

目标:

- 判断 chern 结果是否可靠

作用:

- 去除数值不稳定和伪拓扑结果
- 给 final topo 候选加“可信度”约束

结论:

- Level C 在 strict v2 后已经较稳定
- 在 joint 中承担可靠性过滤角色

► joint 的功能

joint 的作用, 不是替代模型, 而是把 A / B / C 三层结果融合成最终候选集合。

当前 joint_weighted 的核心形式为:

$$0.25 * p_{\text{insulator}} + 0.40 * p_{\text{topo}} + 0.35 * p_{\text{reliable}}$$

这样得到:

- topo_primary: 较高置信 topo 候选
- topo_core: 最高置信核心区
- trivial: 高置信平庸区
- boundary: 边界 / 过渡区

相比只用一个 topo 分类器, 这种方式更严谨, 因为:

- A 保证物理合理性
- B 强调拓扑倾向
- C 限制数值可靠性

► 数据构建与闭环 refine 流程

已经写好并成功运行。

整个流程为:

1. Mathematica / MagneticTB 生成原始 dat/hr
2. clean 清洗
3. band 计算 gap / 能带
4. chern 计算陈数与可靠性
5. merge 合并 band / chern / manifest
6. append 并入 master_dataset_all.csv
7. 训练 A / B / C
8. 跑 joint
9. 提取下一轮局部 refine 中心
10. 再回到 Mathematica 做局部精扫

这条闭环已经多轮验证可用。

► 各阶段结论

1. Stage5e / C=1 platform confirm

这一阶段最重要的结论是:

- 已经在参数空间中确认存在稳定的 C=1 拓扑平台核

- 这是后续机器学习闭环的物理基础

2. levelB_aug_batch001_lite

定位:

- 不是直接出最终物理图
- 而是专门为 Level B 提供高价值 hard / boundary 样本

结论:

- 新增样本进入训练后, Level B 明显变强
- 说明 “ML 反向指导补样” 的路线成立

3. next_refine_top30 / core12+boundary12

定位:

- 从 joint 中压缩出更高价值中心
- 分成 core 精扫与 boundary 窄窗口 refine

结论:

- Level B 再次提升
- 同时表明模型开始整体更偏 topo
- 原来适合旧模型的 joint 阈值不再适合新模型, 需要重新校准

4. final_boundary_top20_refine

定位:

- 最后一轮偏 boundary 收束的 refine
- 使用 16 boundary + 4 core anchor

结论:

- 继续补数据的边际收益开始下降
- core / trivial 已经相对可控
- boundary 仍然较宽, 说明参数空间本身存在较宽过渡带

► 最终一轮结果的定量总结

最终导出结果表明:

- Master rows: 52130
- Core rows: 453
- Primary rows: 3834
- Trivial rows: 419
- Boundary rows: 9978
- Final core representatives: 6
- Final trivial representatives: 12
- Final boundary representatives: 20

这些数说明:

1. 数据规模已经足够大, 不再是小样本试验;
2. core 已经压到 “可展示” 的量级;
3. trivial 已经形成正式可用的对照区;
4. boundary 依然明显更宽, 说明它不是少量噪点, 而是参数空间中的重要结构。

► 三张最终图的解释

1. Final phase map (t2, r3)

这张图最重要的信息是:

- topo_core 和 topo_primary 集中分布在 t2-r3 平面的局部区域;
- boundary 围绕 core / primary 附近出现;
- trivial 更分散, 更多处于外围或非核心区域;
- 大量 other 说明并不是所有点都被强行判为 topo 或 boundary。

物理含义:

- 拓扑平台不是全空间普遍存在, 而是集中在特定的参数联动区域;
- boundary 主要围绕这些区域分布, 说明相界是在核心区外侧展开的, 而不是随机出现。

2. Mean joint_weighted heatmap (t2, r3)

这张图显示:

- joint_weighted 的高值区域集中在一小块中心区域;
- 远离这一区域后, 平均 joint_weighted 明显降低;
- 这与最终相图中的 topo_core / topo_primary 聚集区吻合。

物理含义:

- A/B/C 三层融合后的“综合拓扑可信度”在 t2-r3 平面上存在显著热点;
- t2 和 r3 的联动对最终相结构确实重要。

3. Mean boundary_score_v3 heatmap (t2, r3)

这张图显示:

- boundary_score_v3 的较高区域并不是只在一条单线边界上;
- 它在中心附近形成较宽的高值 / 中高值区域;
- 外围虽偶有高值散点, 但主体高值区仍集中在中心附近。

物理含义:

- 过渡带是“带状 / 区域状”的, 而不是一条非常尖锐的单线边界;
- 这支持“当前参数空间中存在较宽 boundary 区”的最终理解。

► 为什么最终选择 t2 和 r3 作为主图坐标

1. 从 refine 设计上看,t2-r3 一直是主扫平面。

在多轮 Mathematica refine 脚本中, 无论是:

- CORE_PLATFORM_FINE
- BOUNDARY_T2_R3_NARROW
- BOUNDARY_T2_R3_ULTRANARROW

主轴都优先围绕 t2 和 r3 展开, 其他参数更多作为轻微扰动或辅助方向。

2. 从最终相图上看,t2-r3 平面上的结构最清楚。

最终 phase map 中:

- core / primary / boundary 的聚集关系在 t2-r3 上最直观;
- 可以清楚看到中心区和过渡区;
- trivial 对照也能同时展示。

3. 从 joint heatmap 上看,t2-r3 具有统计上的热点结构。

无论是 joint_weighted 还是 boundary_score_v3, 在 t2-r3 平面上都呈现出明显的中心区热点, 而不是全局随机均匀分布。

4. 从物理解释角度,t2 与 r3 承担了主导相界变化的角色。

结合整个闭环流程的经验, **t2** 和 **r3** 在多轮 **refine** 中持续表现出:

- 对 **core** 是否稳定存在有明显影响;
- 对 **boundary** 是否扩展 / 收缩有明显影响。

因此, 最终选 **t2-r3** 做主图, 不是因为“画起来方便”, 而是因为:

- 它们在机器学习闭环里被持续证明是高敏感方向;
- 它们在最终相图与热图中都给出了最清楚的结构表达;
- 它们最能体现“平台核 + 宽过渡带”这一最终结论。

► 当前最终理解

1. **core** 已经相对稳定:

参数空间中确实存在高置信 **topo** 核心区, 可以作为论文中“平台核”的主要展示部分。

2. **trivial** 已经可用:

可以作为论文中的对照区, 也可证明模型不是只会报 **topo**。

3. **boundary** 仍然宽:

这是当前最重要的物理现象之一。

即使经过多轮边界 **refine**, **boundary** 仍然保持较宽, 这说明:

- 它不只是数值噪声;
- 更像是参数空间中的真实过渡带。

因此 **boundary** 不一定要被继续机械压窄, 而应该作为一个“值得解释”的物理结果。

► 机器学习流程的总依据

当前项目真正被验证的不是某个单一 **accuracy** 数字, 而是以下整套逻辑:

1. 机器学习能从物理样本中学到参数空间结构;
2. **Level B** 能识别更像 **topo** / **trivial** / **boundary** 的区域;
3. **joint** 能把 **A** / **B** / **C** 融合成更可解释的候选集合;
4. 这些候选可以反向指导下一轮物理 **refine**;
5. 新物理样本回流后又能继续推强模型;
6. 多轮循环后, 模型与物理结果一起逐渐收束。

也就是说, 这是一条已经成立的:

“物理计算 -> 机器学习 -> 再物理 **refine**”

闭环路线。

► 当前阶段该做什么

现在已经不建议再开启新一轮大规模补样。

原因:

- **core** 已相对稳定;
- **trivial** 已可用;
- **boundary** 虽然宽, 但继续补样不会再带来质变;
- 再继续闭环的边际收益已经下降。

因此当前最合理的下一步是:

1. 导出最终 **core / trivial / boundary** 代表点;
2. 生成论文版相图 / 参数图;
3. 整理特征重要性与参数解释;

► 描述

可以按下面逻辑讲:

1. 从 8 参数最小模型出发, 研究拓扑相界问题;
2. **brute-force** 难以高效捕捉高维相界, 因此引入 **ML**;
3. 构建三层严格模型:**A** 绝缘性,**B** 拓扑性,**C** 可靠性;
4. 用 **joint** 融合三层结果, 得到 **core / trivial / boundary**;
5. 再用这些结果反向指导下一轮局部 **refine**;
6. 多轮闭环后发现:
 - **topo core** 稳定存在;
 - **trivial** 区可用;
 - **boundary** 仍然较宽;
7. 最终说明: 机器学习成功参与并提升了拓扑相界搜索过程, 同时揭示了参数空间中的稳定平台核与宽过渡带。

► 总分析与总结论

1. 机器学习闭环是有效的;
2. **Level B** 是核心贡献点;
3. 当前参数空间中同时存在:
 - 稳定的 **topo core**;
 - 可对照的 **trivial** 区;
 - 较宽的 **boundary** 过渡带;
4. 项目现在进入“结果凝练与展示”阶段。

► 总结

当前项目已经完成了从:

“物理模型样本构建”

到

“严格三层机器学习”

再到

“机器学习闭环反向指导局部 **refine**”

的完整验证。

目前最有价值的总结论是:

“机器学习成功参与并提升了拓扑相界搜索过程, 识别出了高置信 **topo core**、可对照 **trivial** 区, 以及在多轮 **refine** 后仍然保留的宽 **boundary** 过渡带。”

8.14 (原创自研) 机器学习-应用 1

日期: 2026-03-23

在线原文: marx2001.github.io

► 说明

均为原创, 禁止抄袭, 转载注明出处, 否则必会追究。

► 项目主线

当前对话工作总结说明 (ML 拓扑代理模型项目)。

► 项目背景

当前项目围绕 8 参数最小模型 (磁群 $\{123, 342\}$ 、z 向共线自旋假设) 构建机器学习代理模型, 目标是: 输入一组参数, 快速判断该参数点是否为拓扑候选, 并进一步服务于参数空间搜索、相图分析和后验 Chern 验证。

► 本次对话的核心工作主线

本次对话主要围绕“第一个应用: 相图代理模型 / 拓扑候选筛选器”展开, 并在后续延伸到:

1. 机器学习模型的用途讨论 (3 个应用方向)
2. 预测脚本、joint 规则脚本、测试集生成脚本
3. Mathematica / WL 导出 `hr.dat` 脚本
4. Chern 后处理与准确率评估脚本
5. 阈值扫描脚本
6. consistent `sample_id` 验证链条 (严格一一对应)

► 一、模型的三个主要应用 (当前建议的论文主线)

1. 应用一: 相图代理模型 / 拓扑候选快速筛选
 - 输入一组 8 参数
 - 输出模型判断: 是否绝缘、是否拓扑、是否可靠
 - 适合作为“高精度拓扑候选预筛器”
 - 这是当前对话中实现最完整、最推荐优先推进的应用
2. 应用二: 参数敏感性 / 控制轴识别
 - 通过 `feature importance` / SHAP / 局部扫描分析
 - 识别哪些参数或组合变量最主导拓扑相界
 - 当前结论: `t2` 和 `r3` 是拓扑相界主轴
 - 与解析模型中 `e`、`t`、`r` 的物理关系相互印证
3. 应用三: 8 参数拟合 / 反演辅助
 - 从真实材料低能带或 Wannier 结果出发, 拟合等效 8 参数
 - 再把拟合得到的参数点放入 ML 相图中
 - 这是连接真实材料的关键增强方向

- 当前对话中讨论了理论方案, 但尚未正式展开完整脚本链条

► 二、当前阶段的主要结论

1. 训练好的三层模型已经能正常工作, 且当前预测脚本已修好 22 维特征输入问题:
 - 原始输入只需 8 个参数:
e1, e2, t1, t2, r1, r2, r3, r4
 - 脚本内部自动补齐训练时用到的 14 个组合特征:
e0, m, rbar, d12, d34, dpair, A13, A24, A14, A23, B1, B2, B3, B4
2. 组合特征的定义来自:
 - 05_B_next_stage_refine_debug_fixed.py
 - 核心函数:add_signed_features(df)
3. 三层模型当前使用目录(已统一到 scripts 相对路径口径):
 - 05_ml_models/levelA_insulator_strict/xgboost/model.joblib
 - 05_ml_models/me_final_boundary_top20_refine/xgboost/model.joblib
 - 05_ml_models/levelC_reliability_strict_v2/xgboost/model.joblib
4. 现阶段最推荐的用途不是“完全替代 Chern 计算”, 而是:
 - 高 precision 的 topo candidate screening
 - 即: 先用 ML 报出高置信候选, 再做后验 Chern 验证
5. 在严格 consistent sample_id 链条下, 100 个样本的最终验证结果为:
 - n_valid = 100
 - accuracy = 0.87
 - precision = 0.8461538461538461
 - recall = 0.7096774193548387
 - f1 = 0.7719298245614036
 - tp = 22
 - tn = 65
 - fp = 4
 - fn = 9
6. 阈值扫描后, 以 precision 为优化目标, 最佳阈值为:
 - Best threshold by precision: 0.6
 - accuracy = 0.84
 - precision = 0.894737
 - recall = 0.548387
 - f1 = 0.68
7. 从发表角度的客观定位:
 - 这组结果适合支撑“模型可作为高 precision 的拓扑候选预筛工具”
 - 不适合表述成“模型已经可以替代真实拓扑判定”
 - 如果继续补更多独立样本验证, 论文说服力会进一步提升

► 三、关于“究竟该提升 accuracy 还是 precision”的结论

用户当前的核心目标是:

只关心模型说“这组参数对应的是拓扑”这句话准不准。

因此最重要的指标是:

- Precision

► 原因

- $Precision = TP / (TP + FP)$
- 它直接回答: 模型一旦报 topo, 这个判断有多大概率是真的
- 这和后续 expensive 的 band / Chern 计算成本最直接相关

► 为什么 Accuracy 不是当前第一优先

- Accuracy 会受到非 topo 样本占比影响
- 即便模型对 trivial 判得很好, accuracy 也可能高
- 但这不等价于“报 topo 是否真的准”

因此:

- 当前优先目标: 提升 Precision
- 其次再考虑 Accuracy
- Recall 不是当前主目标 (除非想尽量少漏 topo)

► 四、脚本总览 (按功能分类)

► A. 代理模型基础预测 / phase map 工具

1. predict_8param_phase_suite_v3.py

功能:

- 单点预测
- 批量 CSV 预测
- 二维参数扫描
- 自动补齐 14 个组合特征

使用顺序:

- 最先运行的基础预测脚本

需要修改:

- TRAIN_RANGE_CSV
- MODE
- INPUT_CSV
- OUTPUT_CSV
- SCAN_FIXED_PARAMS
- SCAN_RANGES

2. apply_joint_phase_rules_v3.py

功能:

- 对 stage1 / stage2 / stage3 结果做 joint 融合
- 给出最终相区标签 (如 topo_core / topo_primary / boundary / trivial)

使用顺序:

- 在批量预测之后使用

需要修改:

- INPUT_PRED_CSV
- OUTPUT_JOINT_CSV
- 各种 threshold(如需要)

3. **evaluate_proxy_model_on_testset.py**

功能:

- 在测试集上评估三层模型
- 输出 accuracy / precision / recall / f1 / confusion matrix

使用顺序:

- 需要直接评估测试集时使用

需要修改:

- TEST_CSV
- LABEL_STAGE1
- LABEL_STAGE2
- LABEL_STAGE3

► B. 从分析结果生成新测试集 / 批量预测

1. **build_proxy_testset_from_analysis.py**

功能:

- 读取最后一轮分析脚本输出的中心点 CSV
- 在中心点附近按设定宽度生成新的参数测试集 CSV

默认中心点来源:

- 05_ml_models/levelB_augmentation_analysis/recommended_levelB_augmentation_set.csv

需要修改:

- INPUT_CENTER_CSV
- MAX_CENTERS
- WIDTHS_CORE
- WIDTHS_BOUNDARY
- WIDTHS_DEFAULT

2. **batch_predict_proxy_topo.py**

功能:

- 对生成的测试集做批量预测
- 输出预测 CSV

需要修改:

- INPUT_TESTSET_CSV
- OUTPUT_PRED_CSV

3. **run_proxy_testset_workflow.py**

功能:

- 串联运行:
 - a) build_proxy_testset_from_analysis.py
 - b) batch_predict_proxy_topo.py

使用顺序:

- 一步跑完测试集生成 + 批量预测

注意:

- 路径需和前两个脚本一致

► C. 小样本评估链条 (50 / 100 / consistent 版本)

1. reduce_proxy_testset_to_50.py

功能:

- 从大样本测试集或预测结果里随机抽取 50 个样本

需要修改 (如果改样本数):

- OUT_CSV
- N_SAMPLES

注意:

- 只改 N_SAMPLES 不够, 输出文件名也要同步修改

2. generate_hr_from_proxy_testset_50.wl

功能:

- 读取 proxy_testset_50.csv
- 导出 hr.dat

需要修改:

- testCsv
- rawOutDir
- sample_id / 文件名前缀 (如需)

3. compare_proxy_prediction_vs_chern.py / v2 / v3

功能:

- 对比预测 CSV 和真实 Chern 汇总 CSV
- 自动匹配并计算 accuracy / precision / recall / f1

演化说明:

- 初版: 简单匹配
- v2: 如果 Chern CSV 没有参数列, 则自动从 manifest 补回参数
- v3: 加入多层匹配与 debug 输出

需要修改:

- PRED_CSV
- CHERN_CSV
- MANIFEST_CSV

► D. 完全统一的 consistent 版本 (推荐)

1. reduce_proxy_testset_to_100_consistent.py

功能:

- 从大样本预测表里抽样 100 个样本
- 强制保留 / 生成统一 sample_id

输出:

- 06_outputs/proxy_small_eval/proxy_testset_100_consistent.csv

需要修改:

- N_SAMPLES
- RANDOM_SEED

- OUT_CSV
- INPUT_CANDIDATES

2. **generate_hr_from_proxy_testset_100_consistent.wl**

功能:

- 读取 proxy_testset_100_consistent.csv
- 直接使用 CSV 中的 sample_id 写入 hr.dat 文件头
- 导出 manifest

输出:

- scripts/raw_hr_proxy_testset_100_consistent/

需要修改:

- testCsv
- rawOutDir
- 参数范围 (如需)

3. **compare_proxy_prediction_vs_chern_100_consistent.py**

功能:

- 只按 sample_id 进行严格 1:1 对齐
- 不再使用参数 round 近似匹配

输入:

- proxy_testset_100_consistent.csv
- chern_output_hr_proxy_testset_100_consistent/all_chern_summary.csv

输出:

- merged CSV
- summary txt/json

需要修改:

- PRED_CSV
- CHERN_CSV

► E. 阈值扫描

1. **scan_topo_thresholds.py**

功能:

- 对已完成严格匹配的 merged CSV 做 p_topo 阈值扫描
- 计算不同阈值下的 accuracy / precision / recall / f1

默认输入:

- 06_outputs/proxy_small_eval_eval/proxy_vs_chern_merged_100_consistent.csv

需要修改:

- INPUT_CSV
- THRESHOLDS
- FOCUS_METRIC(precision / accuracy / f1)

当前最重要结论:

- 用户当前目标下, 应优先用 precision 作为优化指标

► 五、重要的 WL / Chern / compare 链条说明

1. 一开始出现“只匹配上 152/200”的问题, 原因不是模型不准, 而是:
 - 预测 CSV、WL 导出、Chern 汇总没有共享同一套 `sample_id`
 - 导致 compare 脚本只能退回到参数 `round` 匹配
2. 最终通过 consistent 版本解决:
 - 抽样 CSV 中保留 `sample_id`
 - WL 导出时把同一个 `sample_id` 写入 `hr.dat` 文件头
 - Chern 脚本从 `hr.dat` 文件头读出 `sample_id`
 - compare 脚本按 `sample_id` 严格 1:1 对齐
3. Chern 脚本是否统一:
 - 逻辑上已经统一, 因为 `read_hr_file()` 会读取 `# sample_id`
 - 但目录名必须同步改成 consistent 版本

例如:

- `WORKDIR = ROOT / "clean_raw_hr_proxy_testset_100_consistent"`
- `OUTDIR = ROOT / "chern_output_hr_proxy_testset_100_consistent"`

► 六、当前最推荐的使用顺序 (consistent 严格验证版)

如果要重新做一套 100 个样本的一一对应验证, 推荐顺序如下:

► 步骤 1: 抽样并生成 consistent 测试集

运行:

- `reduce_proxy_testset_to_100_consistent.py`

输出:

- `06_outputs/proxy_small_eval/proxy_testset_100_consistent.csv`

► 步骤 2: 导出 hr.dat

运行:

- `generate_hr_from_proxy_testset_100_consistent.wl`

输出:

- `scripts/raw_hr_proxy_testset_100_consistent/`
- `sample_file_manifest.csv`
- `bad_export_log.txt`

► 步骤 3: clean

将 `raw_hr_proxy_testset_100_consistent` 导入 `clean` 脚本, 输出:

- `clean_raw_hr_proxy_testset_100_consistent`

► 步骤 4: 计算 Chern

修改 Chern 脚本中的:

- `WORKDIR = ROOT / "clean_raw_hr_proxy_testset_100_consistent"`
- `OUTDIR = ROOT / "chern_output_hr_proxy_testset_100_consistent"`

运行后输出:

- `chern_output_hr_proxy_testset_100_consistent/all_chern_summary.csv`

► 步骤 5: 严格 compare

运行:

- `compare_proxy_prediction_vs_chern_100_consistent.py`

输出:

- `proxy_vs_chern_merged_100_consistent.csv`
- `proxy_vs_chern_summary_100_consistent.txt`
- `proxy_vs_chern_summary_100_consistent.json`

► 步骤 6: 阈值扫描

运行:

- `scan_topo_thresholds.py`

得到不同 `p_topo` 阈值下的:

- `accuracy`
- `precision`
- `recall`
- `f1`

当前结果:

- `Best threshold by precision = 0.6`

► 七、如何理解当前结果是否“够发文章”

1. 当前结果已经足够支持:

- 模型可作为高 `precision` 的拓扑候选预筛器
- 可以显著减少无效的后验 Chern 计算
- 已经有独立后验验证支撑“模型有实用性”

2. 当前结果不建议表述为:

- 模型可完全替代真实 Chern 计算
- 模型已经非常高精度、几乎无误

3. 当前论文叙事最稳的定位:

- 8 参数最小模型
- XGBoost 学习拓扑参数空间
- `t2 / r3` 是主导相界的控制轴
- 机器学习模型可作为高 `precision` 的 `topo candidate screening` 工具

- 使用后验 Chern 计算对独立样本进行验证

► 八、给初学者的直接结论

1. 当前最应重视的指标:
 - Precision
2. 原因:
 - 研究重点是判断模型给出的 topo 结论是否准确
 - 这正对应 precision, 而不是 accuracy
3. 目前结果说明:
 - 模型已经“可用”
 - 更像筛选器, 而不是最终决策器
 - 这是可以写进论文的, 但叙事要准确
4. 后续增强方向:
 - 再做更多独立样本验证 (如再做 100 或 200)
 - 报不同随机抽样下的平均表现
 - 把阈值扫描结果作为补充说明
 - 继续结合 t2-r3 主相图和参数控制轴分析

► 九、最后的总判断

当前对话中, 最完整、最可落地、最接近论文正文主线的内容是:

- 第一个应用: 相图代理模型 / 拓扑候选预筛器
- 已建立从参数输入、ML 预测、WL 导出 `hr.dat`、后验 Chern、严格 compare、阈值扫描的完整闭环
- 模型当前的价值定位是:

“高 precision 的拓扑候选筛选工具”

这条线已经具备继续整理成论文方法与结果部分的基础。

8.15 (原创自研) 机器学习-应用 2

日期: 2026-03-23

在线原文: marx2001.github.io

► 当前项目复盘说明书 (第二应用: 参数敏感性 / 控制轴识别)

► 版本说明

本说明书用于系统总结当前窗口内已经完成的工作、得到的结论、脚本的用途与使用方法、关键文件位置、各类结果应如何理解, 以及后续复盘时应如何快速恢复上下文。

本说明书重点围绕以下内容展开:

1. 项目总体定位
2. 第一应用与第二应用的关系

3. 第二应用当前已经完成到什么程度
4. 关键脚本的功能、路径、运行方法
5. 模型、数据、代表点、相图等文件的作用
6. 当前最稳妥的物理论文式结论
7. 后续继续推进时应该优先看什么、补什么

► 一、项目总体定位

当前工作并非“简单训练一个分类器”，而是围绕 Lieb lattice 的 8 参数最小模型，建立一个具有物理可解释性的机器学习代理框架，用来做两件事：

► A. 应用一：快速筛选 / 代理预测

给定一组 8 参数 (脚本内部会自动补齐组合特征), 快速预测:

- 是否绝缘
- 是否为 `topo(is_topo_by_chern)`
- 是否可靠

然后构造 `joint score / phase map`, 用于大规模参数空间预筛选。

► B. 应用二：参数敏感性 / 控制轴识别

在已有代理模型基础上, 不再只问“能不能判出来”, 而是问:

- 到底哪些参数、哪些组合变量在主导拓扑判定?
- 为什么最终相图主平面会落在 `t2-r3`?
- `core / boundary / trivial` 三类区域在参数空间里是什么结构?
- `boundary` 是噪声, 还是一个真实存在的宽过渡带?

当前窗口已经重点完成的是第二应用。

► 二、理论基础与文献关联

当前研究对象是 Lieb lattice 的 8 参数最小模型, 参数为:

- `e1, e2, t1, t2, r1, r2, r3, r4`

这套变量并非任意定义, 而是来自该类模型的最小 TB / MagneticTB 描述。已有文章与支撑材料中, 已经给出这 8 参数最小模型的解析形式, 并说明:

1. Lieb lattice 中, 自旋劈裂 / `altermagnetic splitting` 的解析来源主要受 `r` 类参数差异控制;
2. 在高对称点与特定路径上, `r1-r3`、`r2-r4` 的差异会直接决定劈裂;
3. 这说明 `r-family` 在解析理论层面本来就具有核心物理意义。

这一点与当前机器学习第二应用的结果是“同方向支撑”的, 而不是矛盾的。

但要注意区分:

- 文献讨论的是: `spin splitting / altermagnetism` 的解析来源
- 当前第二项应用讨论的是: `topological classification / phase boundary / boundary structure` 的全局统计控制规律

因此两者不是完全同一个命题, 但共享同一套 8 参数最小模型和同一物理背景。

► 三、当前项目输入、模型与特征体系

► 1. 主数据表

当前主表文件为:

D:\论文\ML\Nb2Se20_ML_Topo_123_342\04_datasets\master_tables\master_dataset_all.csv

这是整个机器学习分析的统一数据入口。

► 2. 三层模型路径

A 模型 (绝缘性门控):

D:\论文\ML\Nb2Se20_ML_Topo_123_342\05_ml_models\1_levelA_insulator_final\xgboost\model.joblib

B 模型 (拓扑主模型):

D:\论文\ML\Nb2Se20_ML_Topo_123_342\05_ml_models\2_levelB_final\xgboost

C 模型 (可靠性层):

D:\论文\ML\Nb2Se20_ML_Topo_123_342\05_ml_models\3_levelC_reliability_final\xgboost

► 3. 第二应用最关键的训练脚本参考

已确认 `add_signed_features(df)` 在 09 开头的训练 levelB 脚本中, 具体文件为:

09_B_final_boundary_top20_refine.py

这个脚本定义了训练时真实使用的固定输入特征口径。

► 4. RAW8 与 FULL22

当前分析必须明确区分两套特征口径。

RAW8(原始参数)

- e1, e2, t1, t2, r1, r2, r3, r4

FULL22(训练真实输入)

- e1, e2, t1, t2, r1, r2, r3, r4
- e0, m, rbar, d12, d34, dpair
- A13, A24, A14, A23, B1, B2, B3, B4

其中补充 14 个组合变量来自 `add_signed_features(df)`, 定义如下:

- $e0 = (e1 + e2) / 2$
- $m = (e1 - e2) / 2$
- $rbar = (r1 + r2 + r3 + r4) / 4$
- $d12 = r1 - r2$

- $d34 = r3 - r4$
- $dpair = ((r1 + r2) - (r3 + r4)) / 2$
- $A13 = r1 + r3$
- $A24 = r2 + r4$
- $A14 = r1 + r4$
- $A23 = r2 + r3$
- $B1 = r1 + r2 + r3 + r4$
- $B2 = r1 + r2 - r3 - r4$
- $B3 = r1 - r2 + r3 - r4$
- $B4 = r1 - r2 - r3 + r4$

► 重要提醒

当前 Level B 模型训练时固定吃 FULL22, 而不是 RAW8。因此:

- 不能直接把 RAW8 喂给模型 `predict_proba`
- 正确做法是: 由 RAW8 自动补齐 FULL22, 再送入模型
- RAW8 只能作为“解释视角”, 不能作为这个已训练模型的直接输入口径

► 四、当前第二应用的目标与实际实现

第二应用不是重新训练, 而是“解释已有模型”。

主要问题:

1. 哪些参数最重要?
2. 哪些组合特征最重要?
3. 为什么 t2-r3 平面最适合做主图?
4. core / boundary / trivial 是否真的是参数空间中的三类结构?
5. boundary 是噪声, 还是一个真实存在的宽过渡带?

对应实现方式:

1. 全局重要性分析
 - model feature importance
 - permutation importance
 - SHAP importance
2. 主控制平面分析
 - t2-r3 mean predicted probability heatmap
 - t2-r3 2D PDP
3. joint / boundary 结构分析
 - joint_weighted heatmap
 - boundary_score_v3 heatmap
4. 代表点案例分析
 - final_core_representatives.csv
 - final_boundary_representatives.csv
 - final_trivial_representatives.csv
 - 以及对应 prediction / SHAP 结果

► 五、第二应用主脚本的发展过程与最终可用版本

在当前窗口里, 第二应用主脚本经历了多轮修正, 主要原因有三类:

► 1. 特征名不匹配

原因:

Level B 模型是 FULL22 训练, 而初始脚本误把 RAW8 直接喂给模型, 导致 `feature_names mismatch`。

修复:

改成“预测时始终用 FULL22;RAW8 只做汇总视角”。

► 2. 脏数值字符串导致 SHAP 报错

典型格式:

- [5E-1]
- [-1.2E+0]
- [3.5]

初始尝试:

- 去方括号
- 强制 float

仍然失败。

► 3. shap.TreeExplainer 内部路径仍碰到字符串问题

最终修复:

对 XGBoost 模型, 不再走 `shap.TreeExplainer` 主路径, 改用 XGBoost 原生 `pred_contribs=True` 计算 SHAP 贡献。

这条路径更稳, 绕开了 [5E-1] 报错。

► 最终推荐使用的脚本版本

`analyze_application2_parameter_sensitivity_v6.py`

► 作用

- 读取 `master_dataset_all.csv`
- 自动补齐 FULL22
- 加载 A/B/C 模型
- 以 Level B 为主做全局解释
- 输出 FULL22 importance / permutation / SHAP
- 输出 RAW8-view 汇总结果
- 输出 t2-r3 概率热图
- 输出 t2-r3 PDP

- 读取 `joint_phase_scatter_final.csv`
- 读取 `representative CSV`
- 输出代表点预测与解释

► 六、脚本路径、运行方法与输入输出位置

► 1. 脚本建议放置位置

`D:\论文\ML\Nb2Se20_ML_Topo_123_342\scripts`

► 2. 最终推荐运行脚本

`analyze_application2_parameter_sensitivity_v6.py`

► 3. 运行方式

在项目根目录中执行:

`python .\scripts\analyze_application2_parameter_sensitivity_v6.py`

► 4. 当前 `joint` 与 `representative` 文件母目录

`D:\论文\ML\Nb2Se20_ML_Topo_123_342\05_ml_models\final_paper_exports`

其中包括:

- `joint_phase_scatter_final.csv`
- `final_core_representatives.csv`
- `final_boundary_representatives.csv`
- `final_trivial_representatives.csv`

► 5. 输出目录

`D:\论文\ML\Nb2Se20_ML_Topo_123_342\06_outputs\application2_parameter_sensitivity_v6`

► 6. 重点输出文件

Level B / FULL22

- `feature_importance.csv`
- `permutation_importance.csv`
- `shap_importance.csv`
- `prob_heatmap_t2_r3.png`
- `prob_heatmap_t2_r3.csv`
- `pdp_t2.png`
- `pdp_r3.png`
- `pdp_t2_r3.png`

RAW8 汇总视角

- raw8_view_model_importance.csv
- raw8_view_permutation_importance.csv
- raw8_view_shap_importance.csv

joint 相关

- joint_weighted_heatmap_t2_r3.csv / png
- boundary_score_v3_heatmap_t2_r3.csv / png

representative cases

- core_predictions.csv
- boundary_predictions.csv
- trivial_predictions.csv
- 对应 mean_abs_shap 与 top contributions

汇总说明

- summary.txt
- summary.json

► 七、当前已经确认的重要结果**► (一) 模型本身是可用的**

Level B 当前给出的指标为:

- roc_auc ≈ 0.8185
- average_precision ≈ 0.8354

这说明:

1. 模型不是无效模型;
2. 它对 topo 正类具有较好的排序能力与检索能力;
3. 后续做参数解释是有意义的, 不是在解释一个坏模型。

► (二) 全局重要性:r-family 才是主导通道

根据当前 summary 结果:

FULL22 下 top model importance 主要包括

- A23
- B4
- r2
- B2
- dpair
- r4
- d12

- r3
- A14
- t2

FULL22 下 top SHAP 主要包括

- B4
- dpair
- r2
- r4
- A23
- d12
- r3
- e1
- m
- B2

这说明:

1. 模型不是靠单一裸参数在判 topo;
2. 模型主要依赖 r-family 构造出的组合结构;
3. 组合变量层 (B4, dpair, d12, A23, B2) 是最关键的信息层。

► (三) 汇总回 RAW8 后: $r2 > r3 > r4 > r1$

把 FULL22 重要性重新汇总回 RAW8 后得到的核心结论是:

$r2 > r3 > r4 > r1 \gg e1 / e2 / t2 / t1$

SHAP 的 RAW8-view 结果也是同样趋势。

因此, 当前最稳妥的全局物理结论是:

“Lieb lattice 8 参数最小模型中的 topo 判定, 全局上主要由 r-family 及其组合结构控制。”

► (四)t2-r3 仍然是论文主图最合适的二维切面

虽然全局上 r-family 更强, 但最终相图和多轮 refine 都落在 t2-r3 平面上, 因此:

- t2-r3 不是“全局最强两个参数”的意思;
- t2-r3 是“最适合把核心平台、过渡带、外围区展示出来的二维切面”。

所以最准确的表述不是:

“t2 和 r3 是全局最主要参数”

而是:

“全局上 r-family 主导; 在低维展示与局部 refine 层面, t2-r3 是最有信息量的主控制平面。”

► (五)t2-r3 probability heatmap 的解释

当前 Level B / FULL22: mean predicted probability on t2-r3 plane 图显示:

1. 中心附近存在相对连续的中高值区域;

2. 外围高值多为零散散点;
3. 主要热点并不均匀分布于全平面, 而是在中心附近形成聚集。

这说明:

- topo 高置信区不是随机噪声, 而是集中在某些参数联动区域;
- 中心附近对应的是更可能形成 topo platform / core 的区域。

► (六)t2-r3 2D PDP 的解释

PDP 的作用是:

在把其它变量按总体分布平均后, 只看 t2-r3 移动时模型输出如何变化。

当前图中的主要趋势是:

1. t2 从约 -0.30 向 -0.15 增加时, 预测从较低值抬升到更高平台;
2. r3 从更低区域向上移动时, 预测也上升;
3. 不是一条极尖锐单线, 而是一个较平滑的平台区与过渡区。

这说明:

- 模型看到的不是刀锋相界;
- boundary 更像带状或区域状结构;
- topo core 的周围存在较宽的 transition shell。

► (七)boundary 是真实过渡带, 不是纯噪声

最终 heatmap 和代表点结果共同支持:

1. boundary_score_v3 的高值不是只落在一条线;
2. 它更像围绕中心区域展开的中高值壳层;
3. core / boundary / trivial 在参数空间中构成“平台核 + 宽过渡带 + 外围对照区”的结构。

因此,boundary 不是应该无限压缩掉的误差残余, 而是参数空间中真实存在的过渡带。

► 八、代表点 (representatives) 分析结论

目前已准备三类代表点:

1. final_core_representatives.csv
2. final_boundary_representatives.csv
3. final_trivial_representatives.csv

对应预测结果文件:

- core_predictions.csv
- boundary_predictions.csv
- trivial_predictions.csv

它们的作用是做第二应用里最直观的案例对比。

已经明确的解读逻辑:

1. core 代表点: 应具有较高 topo 概率
2. trivial 代表点: 应具有较低 topo 概率
3. boundary 代表点: 应落在中间, 是“摇摆态 / 过渡态”

此前已得到:

- core representative 数量:6
- trivial representative 数量:12
- boundary representative 数量:20

因此, 第二应用不仅有全局图, 还有样本级案例支撑。

论文写法上, 应把 representative 看成:

- core: 平台核
- boundary: 壳层过渡区
- trivial: 外部对照区

► 九、当前最准确的论文式结论 (必须记住)

当前最稳妥、最不容易与文献矛盾、也最符合当前机器学习输出的结论是:

► 中文版本

对于 Lieb lattice 的 8 参数最小模型, 机器学习的全局解释表明, 拓扑判定主要由 **r-family** 及其组合特征主导, 尤其是由 **r1-r4** 构造的 **B4**、**dpair**、**A23**、**d12**、**B2** 等变量表现出最高的重要性。将贡献汇总回原始参数后, **r2**、**r3**、**r4**、**r1** 的作用显著强于 **e1**、**e2**、**t2**、**t1**, 说明 **r** 通道是决定参数空间拓扑结构的主导因素。与此同时, 在低维展示与局部 refine 层面, **t2-r3** 平面最能清晰呈现拓扑平台核、外围过渡带与 trivial 对照区, 因此它是最合适的二维主控制切面。由概率热图、二维 PDP 以及 **boundary_score_v3** 共同可见, **boundary** 并非一条极尖锐单线, 而是围绕 topo core 展开的较宽区域状过渡带。

► 英文版本

For the Lieb-lattice eight-parameter minimal model, the global ML interpretation indicates that the topological classification is primarily governed by the **r-family** and its derived combinations, especially **B4**, **dpair**, **A23**, **d12**, and **B2** constructed from **r1-r4**. When the full feature contributions are aggregated back to the raw parameter level, **r2**, **r3**, **r4**, and **r1** dominate over **e1**, **e2**, **t2**, and **t1**, indicating that the **r** channel serves as the primary control sector of the topological structure in parameter space. Meanwhile, at the level of low-dimensional visualization and local refinement, the **t2-r3** plane provides the most informative control slice for displaying the topological platform core, the surrounding transition shell, and the trivial reference region. The probability heatmap, 2D PDP, and **boundary_score_v3** consistently show that the boundary is not a sharp single curve but a relatively broad region-like transition layer surrounding the topo core.

► 十、与文章结论是否矛盾: 最终口径

不矛盾, 但要区分命题层级。

文章说的是:

- Lieb lattice 的 spin splitting / altermagnetic splitting
- 解析上主要由 **r1-r3**、**r2-r4** 差异控制

当前讨论的是:

- Lieb lattice 8 参数最小模型中的 topo 判定与 phase boundary 结构

- 全局上 **r-family** 主导, 二维展示上 **t2-r3** 最清楚

因此正确理解是:

1. 文献提供了解析理论来源, 尤其支持 **r-family** 的关键性;
2. 第二项应用提供了机器学习参数空间中的全局验证;
3. **t2-r3** 是“最佳二维展示切面”, 不等于“全局最强两个单参数”。

千万不要再把当前结果写成:

“t2 和 r3 是全局最主要控制拓扑性质的两个参数”

应改写为:

“全局上 **r-family** 及其组合结构主导; 在二维展示与局部 **refine** 层面,t2-r3 是最合适的主控制平面。”

► 十一、后续复盘时优先查看的文件

如果以后重新打开项目, 只想快速恢复上下文, 建议优先看:

1. 主说明文件
 - 本 txt 文件
 - 06_outputs\application2_parameter_sensitivity_v6\summary.txt
2. 全局解释结果
 - feature_importance.csv
 - permutation_importance.csv
 - shap_importance.csv
3. 二维控制平面
 - prob_heatmap_t2_r3.png
 - pdp_t2_r3.png
 - boundary_score_v3_heatmap_t2_r3.png / csv
4. 代表点结果
 - core_predictions.csv
 - boundary_predictions.csv
 - trivial_predictions.csv
5. 最终 joint 源表
 - joint_phase_scatter_final.csv

► 十二、如果后续还要继续做, 推荐顺序

1. 先读本说明书
2. 再读 application2 的 summary.txt
3. 再看 Level B 的:
 - shap_importance.csv
 - prob_heatmap_t2_r3.png
 - pdp_t2_r3.png
4. 再看 boundary_score_v3 的热图
5. 最后看 representative 的三类 predictions

如果继续写论文, 最自然的顺序是:

- A. 先交代模型有效性 (metrics)
- B. 再给出 FULL22 全局重要性
- C. 再给出 RAW8-view, 说明 r-family 主导
- D. 再展示 t2-r3 heatmap 与 2D PDP
- E. 再展示 core / boundary / trivial representative
- F. 最后总结 “平台核 + 宽过渡带 + 外围对照区”

► 十三、当前状态一句话总结

当前第二应用已经完成到这样一个程度:

当前成果不仅包括一个能够预测 topo 的代理模型, 还形成了一个可在 Lieb lattice 8 参数最小模型上识别参数主导结构、解释为何相图主平面落在 t2-r3、并证明 boundary 为真实宽过渡带的可解释机器学习框架。

Done.

8.16 (原创脚本) 任意体系的磁参数 J 的磁构型设计与计算

日期: 2026-06-10

在线原文: marx2001.github.io

► 说明

原创脚本。原则上可用于任意异质结、单层、块体体系的磁参数 J 的计算。测试过三角晶格体系。

► 方法

代码清单 A.68.1

语言: python; 完整代码见附录第 1193 页, 共 676 行。

脚本逻辑:

代码清单 A.68.2

语言: shell; 完整代码见附录第 1203 页, 共 8 行。

使用方法, 只需要扩胞后的 poscar 即可:

命令示例 · shell

```
python3 1.py POSCAR --mag-elements Cr Ru --group-by element --max-shells 3 --pbc 1 1
↪ 0 --n-configs 12
python3 1.py POSCAR --mag-elements Cr Ru --group-by element --max-shells 3 --pbc 1 1
↪ 0 --n-configs 12 --shell-tol 0.2
```

-pbc 不用修改, 单层体系, 块体再改成 1 1 1; -n-configs 是生成需要的磁构型或者是能量方程, 原则上几个未知量几个方程, 但是最小二乘法拟合应该是更为准确。

高通量、自动化与模型构建

本专题收录 7 篇文章。

9.1 高通量结构优化脚本

日期：2025-09-23

在线原文：marx2001.github.io

► 用于高通量计算的结构优化脚本

► 1. 介绍

此脚本用于高通量计算的第一步——结构优化。详细来说, 此脚本需要放置在目标目录中, 修改数组中材料的名称后, 自动产生相应的文件夹。此外, 还需要准备一个 POSCAR 文件。

脚本的原理是, 根据数组创建与材料名称一一对应的文件夹, 并复制 POSCAR, 同时读取文件夹名称中的元素, 再替换进 POSCAR 对应的元素行, 由此可基于该结构开展高通量计算。需要注意, 必须使用目标结构的 POSCAR。

使用脚本的顺序, 按照脚本名称一一运行即可。脚本代码如下:

1. 先用 sh 命令运行 1.sh 脚本, 创建文件夹

代码清单 A.69.1

语言: shell; 完整代码见附录第 1204 页, 共 30 行。

2. 运行 2.py, 代码如下所示:

代码清单 A.69.2

语言: python; 完整代码见附录第 1204 页, 共 75 行。

3. 运行 3.pbs, 开始结构优化, 此时 INCAR 中力的收敛标准和能量收敛标准均是较低的, $1e-4$, -0.1 的收敛标准

代码清单 A.69.3

语言: shell; 完整代码见附录第 1206 页, 共 59 行。

4. 在运行下一步脚本之前, 记得更改 INCAR 中的收敛标准 ($1e-6$, -0.05), 因为只有逐步提高收敛标准, 才能获得比较准确的结构优化结果。

代码清单 A.69.4

语言：shell；完整代码见附录第 1207 页，共 76 行。

5. 最后一次结构优化, 收敛标准提到 $1e-6$, -0.01, 此时的结构已足以计算磁参数等性质, 计算声子谱时可以设置收敛标准 $1e-8$, -0.001

代码清单 A.69.5

语言：shell；完整代码见附录第 1208 页，共 172 行。

9.2 通过能带判断缺陷、团簇的 python 脚本

日期：2025-09-24

在线原文：marx2001.github.io

► 1. 介绍

此脚本放置于任意位置, 手动指定数据路径, 可以自动确定能量窗口, 寻找团簇, 缺陷。

代码清单 A.70.1

语言：python；完整代码见附录第 1210 页，共 11 行。

9.3 高通量 DMI 计算脚本

日期：2025-09-24

在线原文：marx2001.github.io

► 用于高通量 DMI 计算的脚本

► 1. 介绍

此脚本用于高通量计算的第二步——DMI 计算。详细来说, 此脚本是高通量结构优化脚本的后续部分。用法如下: 将此脚本放置在与结构优化脚本同级别的目录中, 直接运行即可, 会产生一个 4_DMI 的文件夹, 同时脚本所在目录也要放置 incar.cw 和 incar.acw 文件, 后续将用于计算。

1.qsub 脚本名.pbs, 提交任务即可

代码清单 A.71.1

语言：shell；完整代码见附录第 1211 页，共 303 行。

2.shell 面板内运行脚本进行数据处理。

代码清单 A.71.2

语言: python; 完整代码见附录第 1215 页, 共 112 行。

9.4 (原创脚本) 高通量磁耦合参数计算脚本

日期: 2025-09-26

在线原文: marx2001.github.io

► 高通量磁耦合参数计算的脚本

此脚本用于高通量计算的第三步——J 的计算。详细来说, 此脚本是高通量结构优化脚本的后续部分。用法如下: 将此脚本放置在与结构优化脚本同级别的目录中, 直接运行即可, 会产生一个 5_J 的文件夹, 同时脚本所在目录也要放置 INCAR1、INCAR2、INCAR3、INCAR4 文件, 后续将用于计算。

1. qsub 8.pbs, 提交任务即可

代码清单 A.72.1

语言: shell; 完整代码见附录第 1217 页, 共 121 行。

9.5 二维双层转角材料的代码构建

日期: 2025-09-26

在线原文: marx2001.github.io

► 任意六角或者三角体系的构建双层转角的脚本

此脚本用途如题所示

1. 提交脚本即可, 如果是 120° 体系, 直接运行, 若是 60° 体系, 则先注释掉 120° 的代码, 取消注释 60° 部分的代码

这个代码是主代码, 直接运行即可

代码清单 A.73.1

语言: python; 完整代码见附录第 1219 页, 共 21 行。

这个代码是配套使用的, 复制到文件夹内即可。

代码清单 A.73.2

语言: python; 完整代码见附录第 1219 页, 共 556 行。

最后只需要在文件夹内准备一个 POSCAR 即可。文件夹内有 POSCAR 和以上两个脚本即可。

9.6 (改编脚本) 快速计算投影能带

日期: 2025-12-27

在线原文: marx2001.github.io

► Vasp 计算投影能带

投影能带的计算不是必需的, 而是在分析机制的时候使用。例如在分析晶体场劈裂情况时需要用到。使用名称为《vasp 计算能带》文章中所用的 INCAR, 计算完成后, 输入 vaspkit/214 即可输出手动选择特定原子的投影能带数据文件。下面是 vaspkit 投影能带的选取注释:

- 211 标准能带结构 (包含所有原子和轨道的投影)
 - 212 仅提取单个选定原子的轨道投影能带
 - 213 按元素分类 (如所有 Fe 原子) 的投影能带
 - 214 手动选择特定原子 (输入原子序号) 的投影能带
 - 215 按元素权重 (如 50% Fe + 50% O) 的投影能带
 - 216 对选定原子和轨道的投影求和 (如所有 Fe 的 d 轨道)
- 考虑 soc 的情况下, 使用这个脚本

代码清单 A.74.1

语言: python; 完整代码见附录第 1228 页, 共 651 行。

不考虑 soc 的情况下, 使用这个脚本:

代码清单 A.74.2

语言: python; 完整代码见附录第 1238 页, 共 621 行。

► 使用方法

脚本放到当前目录, 然后先激活环境

命令示例 · shell

```
conda activate fatband
```

输入命令:

命令示例 · shell

```
python plot_fatbands.py -p Tc.dxy Tc.dz2 Tc.dyz Tc.dxz Tc.dx2-y2 --split -m -0.3 -M  
↪ 0.5 -o Tc-p-orbitals
```

脚本开头有相关命令的解释和默认值, 如果需要, 重新看脚本即可, 此处不再赘述。

9.7 (原创脚本)python 解线性方程组

日期: 2025-12-30

在线原文: marx2001.github.io

► 代码如下

用于求解三角晶格的 J

代码清单 A.75.1

语言: python; 完整代码见附录第 1248 页, 共 25 行。

第四部分

绘图流程与代码

能带、轨道与电子结构可视化

覆盖能带、轨道投影、自旋密度、Berry 曲率、3D MAE、斯格明子与原子结构渲染, 突出从原始数据到论文级图件的完整流程。

本专题收录 4 篇文章。

10.1 (改编脚本) 快速绘制轨道投影能带

日期: 2025-12-22

在线原文: marx2001.github.io

1. 计算

自洽计算

命令示例 · shell

```
IBRION=-1
LCHARG=T
ICHARG=2
```

计算能带, 复制上一步的 CHGCAR, POSCAR, POTCAR, INCAR 到当前目录, 修改 INCAR

命令示例 · shell

```
ISMEAR=0
ICHARG=11
```

KPOINTS 使用点模式, 即 KPOINTS 中给出每个 k 点坐标, 把高对称点放到文件 k-point 中:

代码清单 A.76.3

语言: shell; 完整代码见附录第 1249 页, 共 5 行。

运行脚本:

命令示例 · shell

```
python step0.py >KPOINTS
python kp.py >KPOINTS
```

提交能带计算, 等待计算完成后, 下一步处理数据

kp.py 的内容如下:

代码清单 A.76.5

语言: python; 完整代码见附录第 1249 页, 共 27 行。

1. 数据处理

运行 step1.sh, 得到总的数据, 文件名称为 note-bnd

代码清单 A.76.6

语言: shell; 完整代码见附录第 1249 页, 共 12 行。

运行 step2.sh, 得到分轨道的能带数据

代码清单 A.76.7

语言: shell; 完整代码见附录第 1249 页, 共 10 行。

绘图, 运行 step3.py、step4.py、step5.py、step6.py, 主要区别是 spin 的设置分为 up\down\no\both, 有的脚本有 bug, 后期再改。脚本内容如下所示:

step3.py

代码清单 A.76.8

语言: python; 完整代码见附录第 1250 页, 共 118 行。

step4.py

代码清单 A.76.9

语言: python; 完整代码见附录第 1252 页, 共 185 行。

step5.py

代码清单 A.76.10

语言: python; 完整代码见附录第 1254 页, 共 194 行。

step6.py

代码清单 A.76.11

语言: python; 完整代码见附录第 1257 页, 共 127 行。

10.2 (原创脚本)Pyprocar 绘制能带图

日期: 2025-12-30

在线原文: marx2001.github.io

► 利用 Python 绘制能带

没啥可说的, 是原创脚本, 此为记录, 以防脚本丢失。以 jupyter book 形式运行, 并保存到数据所在的文件夹。

代码清单 A.77.1

语言: python; 完整代码见附录第 1259 页, 共 87 行。

10.3 MLWFS 绘制轨道

日期: 2026-01-22

在线原文: marx2001.github.io

► 步骤

(1) 完成一次拟合良好的 wannier, 注意 INCAR 中加入 `LWRITE_UNK = .TRUE.` 输出 Unk 文件

命令示例 · shell

```
LWRITE_UNK = .TRUE.
```

(2) 产生 nnkp 文件, 输入

命令示例 · shell

```
wannier90.x -pp wannier90.1
```

命令示例 · shell

```
wannier90.x -pp wannier90.2
```

(3) 在 win 文件中加入

代码清单 A.78.4

语言: shell; 完整代码见附录第 1261 页, 共 6 行。

(4) 运行 wannier90 输出 cube

命令示例 · shell

```
wannier90.x wannier90.1
```

命令示例 · shell

```
wannier90.x wannier90.2
```

(5) 使用 vesta 打开产生的 cube 文件

10.4 (原创脚本) 绘制精美的投影能带图

日期: 2026-02-05

在线原文: marx2001.github.io

► 说明

用于记录写脚本的有用的版本, 有些暂时用不上, 但是绘制 n 重简并的能带的时候可以用到, 还有的脚本的功能是任意数量的轨道组合也能画图, 同时指定颜色, 这两种脚本只选一种来使用, 未免有些可惜, 所以都记录在此, 后面根据论文排版不同, 选择不同版本的脚本进行绘图。

所有脚本的使用方法可能会有细微差别, 具体体现在使用方法上。

脚本默认在计算能带的目录下, 要有 Procar 等文件, procar 版本是 6.5.0, 其他版本并没有测试过, 所以不知道是否会存在问题, 建议本地电脑安装 pyprocar, 并运行脚本。

► 脚本 1

绘制 d 轨道简并状态下的投影能带图, 最少要设置 3 个轨道, 后面有需要再改。

代码清单 A.79.1

语言: python; 完整代码见附录第 1261 页, 共 424 行。

使用方法:

命令示例 · shell

```
python fatband.py --orbitals dxy,dx2-y2,dyz --out test.png
```

► 脚本 2

更美观, 且支持对每个轨道自定义颜色, 支持任意数量的轨道组合, 除了 f 轨道

代码清单 A.79.3

语言: python; 完整代码见附录第 1267 页, 共 487 行。

使用方法:

命令示例 · shell

```
python 2fatband.py --orbitals "dxy dyz dz2" --fixed_colors  
↪ "dz2=#2f4aa8,dxy=#e53935,dyz=#f2a400" --out test.png
```

► 脚本 3

对全局字体加深要求, 全部加粗, 且规定字体为 Times New Roman。调整刻度线、标题、刻度与轴线之间的距离, 使得图片可以用于论文。

代码清单 A.79.5

语言: python; 完整代码见附录第 1275 页, 共 481 行。

使用方法:

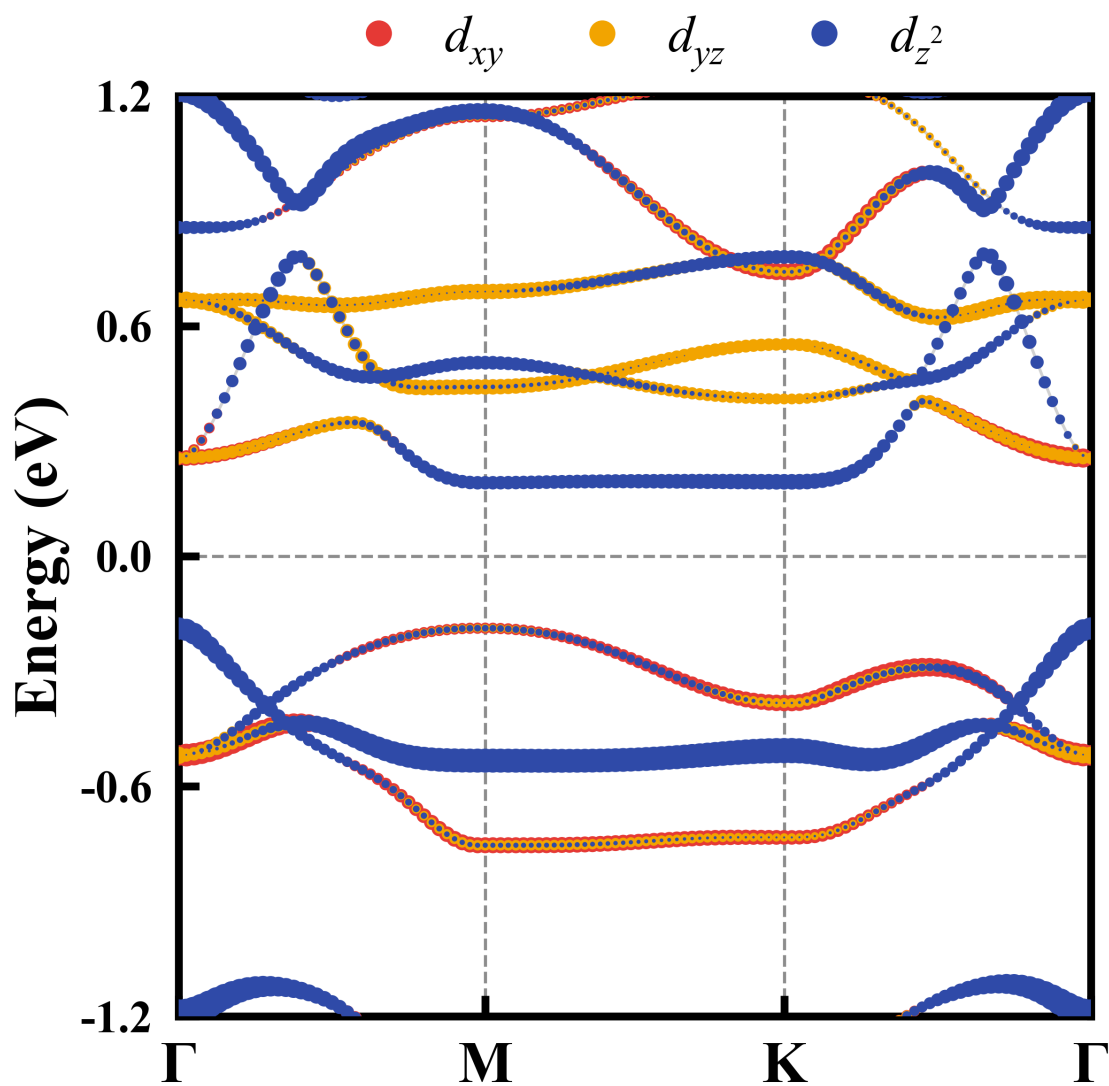
短代码 · python

```
python 3fatband.py --orbitals "dxy dyz dz2" --fixed_colors
→ "dz2=#2f4aa8,dxy=#e53935,dyz=#f2a400" --out test.png
```

短代码 · python

```
python 3fatband.py --elements Mo --orbitals d --fixed_colors
→ "dxy=#e53935,dyz=#f2a400,dz2=#2f4aa8,dxz=#43a047,x2-y2=#8e24aa" --out Mo_d.png
```

最终效果



2026.3.4 补充新代码, 考虑 soc 的时候, 上面脚本解析会失败, 所以下面版本是绘制考虑 soc 的投影能带图。

代码清单 A.79.8

语言：python；完整代码见附录第 1282 页，共 522 行。

拓扑、磁性与三维场可视化

本专题收录 3 篇文章。

11.1 绘制 3DMAE 的流程 (原创脚本与流程)

日期: 2025-09-24

在线原文: marx2001.github.io

► 3DMAE 绘制流程

1. 计算 3DMAE 数据

以 MPSS 为例, 其结构如图所示

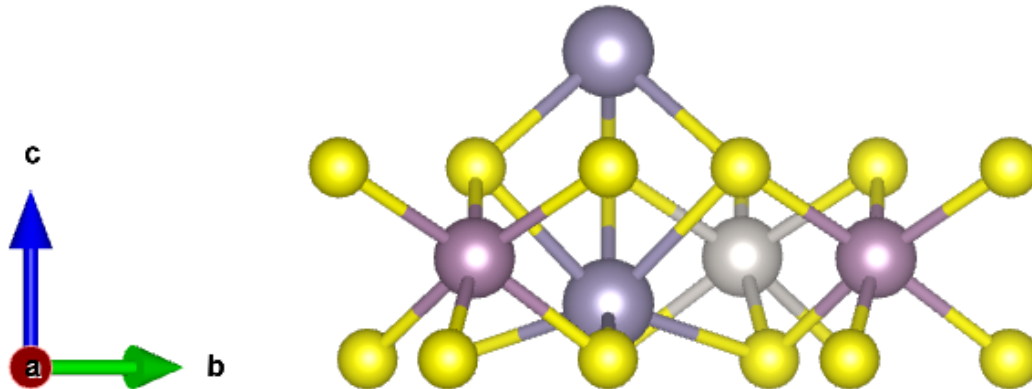


图 11.1: MPSS 结构图

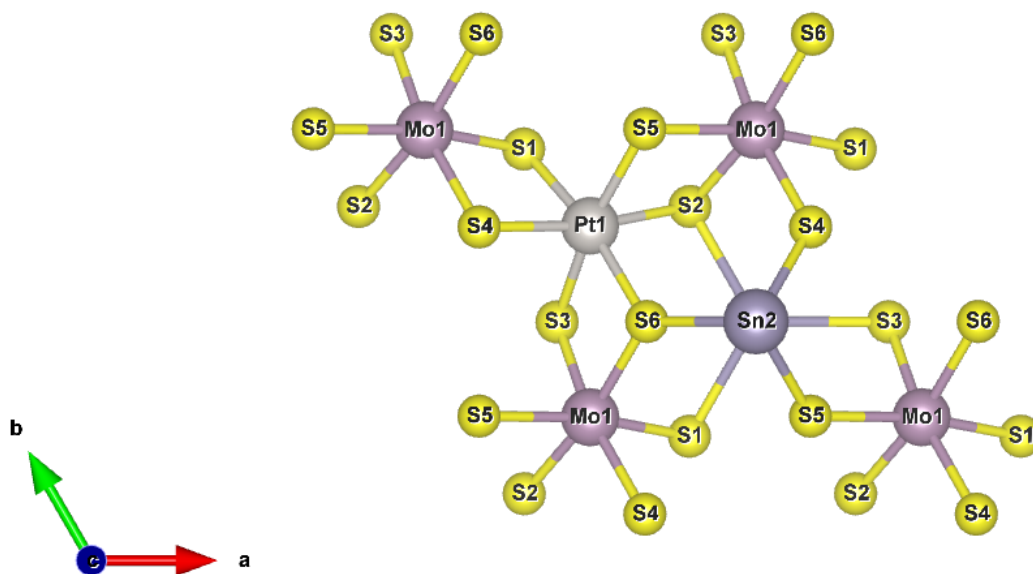


图 11.2: MPSS 结构图

首先, 准备输入文件 VASPKIT.in 文件, POSCAR、POTCAR、KPOINTS、mrx.py 以及 CHGCAR。

VASPKIT.in 文件由自己编写, 内容如下

代码清单 A.80.1

语言: python; 完整代码见附录第 1290 页, 共 3 行。

INCAR 文件如下:

代码清单 A.80.2

语言: python; 完整代码见附录第 1290 页, 共 37 行。

提交计算脚本, 脚本内容如下:

代码清单 A.80.3

语言: python; 完整代码见附录第 1290 页, 共 32 行。

2. 数据后处理与画图

1. 计算完成后, 修改 VASPKIT.in 文件, 将第一行的 1 改为 2, 产生 MAE.dat 数据文件。
2. 运行 mrx.py, 产生四个 txt 文件
3. 打开 Origin, 同一个系统框内创建四个矩阵。如下图所示, 点击画红框的按钮, 选择“插入”

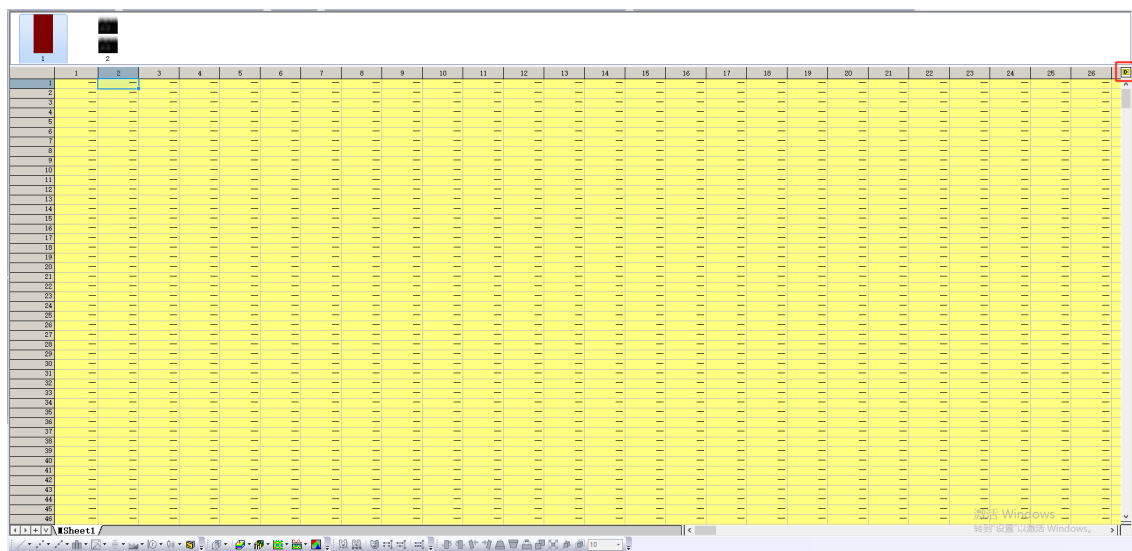


图 11.3: MAE 图

1. `ctrl+A` 全选, 并于工具栏中点击“绘图” - “参数曲面图”, 即可产生 3DMAE 图像。
2. 双击页面空白的地方, 弹出“图层”框, 选择“平面”, 将 XY、YZ、ZX 取消勾选, 再旋转到合适的角度, 即可得到 3DMAE 图。如果比例不满意, 可以选择“坐标轴”选项, 修改某一轴的长度即可找到优美的比例。

PS:mrx.py 为原创脚本, 其代码如下:

代码清单 A.80.4

语言: python; 完整代码见附录第 1291 页, 共 62 行。

也可以用 Python 进行 3DMAE 绘图, 经过我的精心调试, 原创脚本如下:

代码清单 A.80.5

语言: python; 完整代码见附录第 1292 页, 共 84 行。

Python 绘图脚本效果如下:

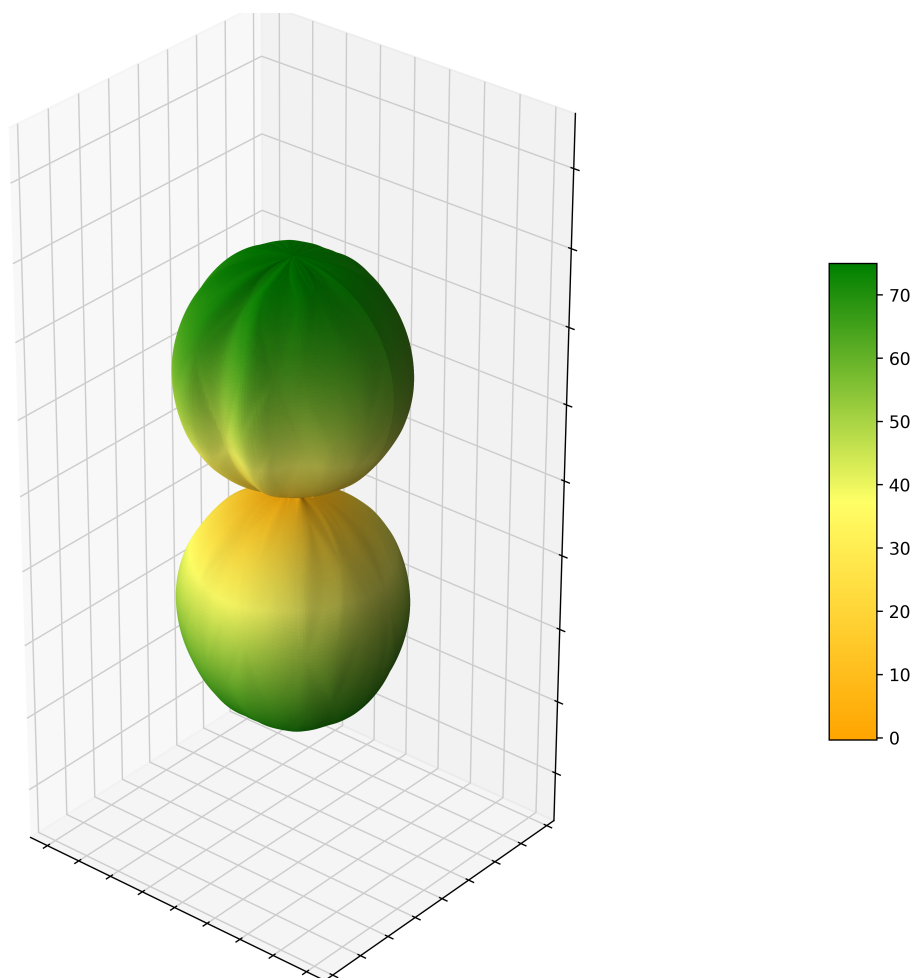


图 11.4: 3DMAE 图

11.2 vasp 绘制自旋密度分布图

日期: 2025-10-13

在线原文: marx2001.github.io

► 用于高通量 DMI 计算的脚本

► 1. 介绍

简单记录一下如何绘制自旋密度分布图。

► 2. 两种方法

1. vtst 脚本

命令行输入 `chgsplit.pl CHGCAR`(在静态自洽计算文件夹中), 得到 `CHGCAR_tot`、`CHGCAR_mag` 文件

放入 vesta 查看, 设置 Properties-isosurfaces level=0.023(这个是我的论文中的参考值)or 0.0037

1. vaspkit

vaspkit 312 似乎可以。

11.3 (原创脚本)vaspberry 计算贝里曲率与后处理

日期: 2026-03-02

在线原文: marx2001.github.io

► 说明

方法不是原创, 后处理程序以及单条能带贝里曲率的计算和细节是原创。

原创脚本, 盗取必究。

► 方法

第一步: VASP 的静态计算 (要考虑 SOC) LWAVE=.TRUE.

自洽关键参数:

短代码 · python

```
LWAVE=.TRUE.  ISYM = -1
```

第二步: 在当前目录提交 vaspberry 的计算假设我们第一步中的 KPOINTS 使用的是 $12 * 12 * 1$ 的参数, 价带顶为 18, 或者我们计算第 18 个带的贝里曲率以及陈数, 则运行命令

代码清单 A.82.2

语言: shell; 完整代码见附录第 1293 页, 共 1 行。

kx 和 ky 是 k 点数, 单层材料 ii 取 1,18 是价带顶和导带底分别的序号. 此命令为计算第 1 个带到第 18 个带的总的贝里曲率以及陈数, 而不是单一能带的贝里曲率和陈数。

会输出一个贝里曲率的输出文件 BERRYCURV.dat 使用自带脚本 contour.py, 就可以显示贝里曲率了。

计算某一条带的贝里曲率和陈数, 例如第一占据价带,

代码清单 A.82.3

语言: shell; 完整代码见附录第 1294 页, 共 1 行。

若自洽为共线计算 (vasp_std), 则在 vaspberry 所在目录将自洽计算的波函数 WAVECAR 拷贝进来, 运行:

代码清单 A.82.4

语言: shell; 完整代码见附录第 1294 页, 共 1 行。

注意 kx 和 ky 值要与 KPOINTS k 点个数一致, -s 1 表示共线计算, 18 表示价带顶。

改进版后处理代码 (原创):

代码清单 A.82.5

语言: python; 完整代码见附录第 1294 页, 共 55 行。

第二个版本:

代码清单 A.82.6

语言: python; 完整代码见附录第 1295 页, 共 57 行。

绘图则放到 ppt 内另行处理。选择 ppt 内图片格式中的矫正颜色, 让对比度增加 20%, 也可以进行其他处理。

我自己的计算命令:

代码清单 A.82.7

语言: shell; 完整代码见附录第 1296 页, 共 1 行。

计算单带贝利曲率, 原创代码:

创建 ipynb 文件

cell1:

代码清单 A.82.8

语言: python; 完整代码见附录第 1296 页, 共 24 行。

cell2:

代码清单 A.82.9

语言: python; 完整代码见附录第 1296 页, 共 12 行。

cell3

代码清单 A.82.10

语言: python; 完整代码见附录第 1297 页, 共 19 行。

cell4

代码清单 A.82.11

语言: python; 完整代码见附录第 1297 页, 共 8 行。

cell5

代码清单 A.82.12

语言: python; 完整代码见附录第 1297 页, 共 228 行。

这个脚本会生成一个 dat 文件, 第一列和第五列是我们放到 origin 绘图的数据, 分别是 x 和 y 轴。

增大取点 $N_PER_SEG = 3$, 设置费米能级 $EF = -1.4862$, 修改价带和导带编号, 脚本默认计算全部的价带 +n 条导带:

短代码 · python

```
CB_START_1BASED = 44 # 导带底编号
N_CB = 3 #导带底以上三条能带纳入计算贝里曲率范围, 但下方代码只能绘制占据带的贝里曲率,
↪ 最后一个cell来输出导带贝里曲率
```

20260303 更新代码方便绘图:

新增两个 cell:

代码清单 A.82.14

语言: python; 完整代码见附录第 1301 页, 共 44 行。

上面的 cell 用于, 输出当前使用的高对称点, 下面的 cell 把高对称点作为横轴, 贝里曲率作为 y 轴, 绘制 1D 折线图, 并输出一个名叫 final.dat 的数据文件。

代码清单 A.82.15

语言: python; 完整代码见附录第 1301 页, 共 61 行。

x 轴用第一列,y 轴用 Omega_xy 这一列。

再次更新一个 cell, 用于输出高对称带点

代码清单 A.82.16

语言: python; 完整代码见附录第 1302 页, 共 27 行。

再次更新, 可以用于论文的绘图代码, 新的 cell 基于已经计算好的数据进行绘图:

代码清单 A.82.17

语言: python; 完整代码见附录第 1303 页, 共 176 行。

上面是绘制全部价带的贝里曲率分布图, 下方给出考虑导带数目后的贝里曲率分布折线图:

作为 ipynb 文件的 cell 写入

代码清单 A.82.18

语言: python; 完整代码见附录第 1306 页, 共 54 行。

如果数值过大, 则不需要单位转换直接用下方代码生成贝里曲率图片:

代码清单 A.82.19

语言: python; 完整代码见附录第 1307 页, 共 176 行。

原创脚本, 盗取必究。

结构建模与三维渲染

本专题收录 2 篇文章。

12.1 (原创脚本)blender 绘制斯格明子

日期: 2026-03-07

在线原文: marx2001.github.io

► 说明

均为原创, 禁止抄袭, 转载注明出处, 否则必会追究。

► 方法

- (1) 原子自旋模拟, 提交 spirit 任务得到 ovf 文件。
- (2) 收集所有的 ovf 文件, 并转换成图片, 脚本如下:
sh 脚本负责收集,python 负责绘制, 配色为原创:

代码清单 A.83.1

语言: shell; 完整代码见附录第 1309 页, 共 9 行。

代码清单 A.83.2

语言: python; 完整代码见附录第 1310 页, 共 127 行。

- (3) 筛选需要绘制的 ovf 文件和对应的图片。复制到同一个文件夹中
- (4) 先将 ovf 转换为 csv 文件, 记录自旋数据, 这是一个 ipynb 文件, 需要 opencv 库。改晶格常数等信息。

代码清单 A.83.3

语言: python; 完整代码见附录第 1312 页, 共 162 行。

- (5) 框选想要绘制的区域。

代码清单 A.83.4

语言: python; 完整代码见附录第 1314 页, 共 69 行。

- (6) 记录上一步输出的 ROI 参数, 据此计算目标绘制区域的坐标, 同时修改相应的 PNG 和 CSV 文件名。

代码清单 A.83.5

语言: shell; 完整代码见附录第 1315 页, 共 1 行。

将坐标填入下方脚本

代码清单 A.83.6

语言: python; 完整代码见附录第 1315 页, 共 42 行。

(7) 转换 xywh 坐标为 xyz 坐标, 即将图片中的位置转换为 csv 文件的位置。

代码清单 A.83.7

语言: python; 完整代码见附录第 1316 页, 共 42 行。

(8) 上一个脚本的输出如下所示, 填入到下一个脚本结尾的相应的 xyz 坐标处:

代码清单 A.83.8

语言: shell; 完整代码见附录第 1317 页, 共 4 行。

代码清单 A.83.9

语言: python; 完整代码见附录第 1317 页, 共 48 行。

(9) 计算切割后的斯格明子的自旋数据并形成 spin texture 图, 观察是否是自己想要的那一个区域。

代码清单 A.83.10

语言: python; 完整代码见附录第 1318 页, 共 44 行。

(10) 以下为 Blender 代码。需将 Blender 工程文件与上述输出文件置于同一文件夹。

这个是抠图版的代码, 只绘制斯格明子本身

代码清单 A.83.11

语言: python; 完整代码见附录第 1319 页, 共 44 行。

这个是方形的背景

代码清单 A.83.12

语言: python; 完整代码见附录第 1319 页, 共 44 行。

上面是绘制 ovf 到 blender 的全部流程, 后面会记录如何自己创建一个自旋数据 csv 文件, 并用 blender 绘制出来。

12.2 (绘图教程) VESTA 到 Blender 的二维材料 3D 论文原子结构绘图教程

日期: 2026-09-04

在线原文: marx2001.github.io

##

代码清单 A.84.1

语言: text; 完整代码见附录第 1320 页, 共 1 行。

说明

代码清单 A.84.2

语言: text; 完整代码见附录第 1320 页, 共 1 行。

本文记录如何将第一性原理计算得到的晶体结构文件, 通过 **VESTA** → **Blender** 转换为适用于论文 Figure 的三维原子结构模型。

该流程面向没有 **Blender** 使用经验的科研人员, 目标是:

代码清单 A.84.3

语言: text; 完整代码见附录第 1320 页, 共 9 行。

核心思想:

代码清单 A.84.4

语言: text; 完整代码见附录第 1321 页, 共 2 行。

不要直接在 **Blender** 中手动搭建晶体结构。对于二维材料和异质结构, 原子坐标、键连接和周期性必须由晶体软件保证。

► 一、整体 workflow 概览

► Step 1: 准备结构文件

输入文件:

代码清单 A.84.5

语言: text; 完整代码见附录第 1321 页, 共 3 行。

推荐使用:

代码清单 A.84.6

语言: text; 完整代码见附录第 1321 页, 共 1 行。

原因:

- 已经过结构优化;
- 原子位置更加稳定;
- 可直接用于论文结构展示。

► Step 2:VESTA 处理结构

VESTA 主要完成:

代码清单 A.84.7

语言: text; 完整代码见附录第 1321 页, 共 11 行。

VESTA 不建议作为最终渲染工具, 而作为结构准备工具。

► 二、VESTA 结构处理步骤

► 1. 导入结构

打开:

代码清单 A.84.8

语言: text; 完整代码见附录第 1321 页, 共 1 行。

加载:

代码清单 A.84.9

语言: text; 完整代码见附录第 1322 页, 共 1 行。

检查:

- 元素种类;
- 原子数量;
- 晶格参数;
- 层间结构。

► 2. 扩胞

二维材料通常需要扩胞显示周期性。

路径:

代码清单 A.84.10

语言: text; 完整代码见附录第 1322 页, 共 5 行。

设置:

例如:

短代码 · text

```
a = 3
b = 3
c = 1
```

得到:

代码清单 A.84.12

语言: text; 完整代码见附录第 1322 页, 共 1 行。

注意:

不要在 Blender 中直接扩胞。

原因:

- 周期关系难以控制;
- 键连接容易错误;
- 模型容易出现断键。

► 3. 设置原子和键

建议:

显示主要化学键:

例如:

代码清单 A.84.13

语言: text; 完整代码见附录第 1322 页, 共 2 行。

关闭不必要连接:

例如:

代码清单 A.84.14

语言: text; 完整代码见附录第 1322 页, 共 1 行。

原则:

结构展示不是显示所有可能连接, 而是突出晶体骨架。

► 4. 调整原子比例

VESTA 默认原子大小通常偏小。

论文图建议:

- 磁性元素适当放大;
- 配位原子保持比例;
- 键不要过粗。

► 三、VESTA 导出到 Blender

► 推荐路线

代码清单 A.84.15

语言: text; 完整代码见附录第 1322 页, 共 5 行。

原因:

VRML 可以保留:

- 原子球;
- 键;
- 基本颜色信息。

导出:

代码清单 A.84.16

语言: text; 完整代码见附录第 1322 页, 共 5 行。

得到:

代码清单 A.84.17

语言: text; 完整代码见附录第 1323 页, 共 1 行。

► 四、Blender 导入流程 (新手)

► Step 1: 安装 VRML 导入插件

Blender 4.2 默认可能没有 VRML 导入。

进入:

代码清单 A.84.18

语言: text; 完整代码见附录第 1323 页, 共 5 行。

搜索:

代码清单 A.84.19

语言: text; 完整代码见附录第 1323 页, 共 1 行。

如果没有:

需要安装支持 VRML/Web3D 格式的插件。

► Step 2: 导入结构

菜单:

代码清单 A.84.20

语言: text; 完整代码见附录第 1323 页, 共 5 行。

选择:

代码清单 A.84.21

语言: text; 完整代码见附录第 1323 页, 共 1 行。

导入后:

模型会出现:

- 原子球;
- 键柱;
- 材质。

► 五、Blender 论文级调整流程

► 1. 设置 Camera

不要直接使用默认 Camera。

推荐:

代码清单 A.84.22

语言: text; 完整代码见附录第 1323 页, 共 5 行。

将当前视角绑定给 Camera。

► 2. Camera 参数建议

二维异质结构推荐:

代码清单 A.84.23

语言: text; 完整代码见附录第 1324 页, 共 2 行。

如果结构被裁剪:

降低焦距:

例如:

代码清单 A.84.24

语言: text; 完整代码见附录第 1324 页, 共 1 行。

► 3. 画幅设置

横向二维材料推荐:

代码清单 A.84.25

语言: text; 完整代码见附录第 1324 页, 共 1 行。

或者:

代码清单 A.84.26

语言: text; 完整代码见附录第 1324 页, 共 1 行。

比例:

约 2:1。

► 六、常见问题与解决方法

► 问题 1: 渲染结果缺少一半结构

原因:

Camera 视野太小。

解决:

检查:

代码清单 A.84.27

语言: text; 完整代码见附录第 1324 页, 共 1 行。

降低:

代码清单 A.84.28

语言: text; 完整代码见附录第 1324 页, 共 1 行。

或者:

增大 Camera 距离。

► 问题 2: Viewport 视角和 Camera 视角不同

原因:

Viewport 不是 Camera。

解决:

当前满意视角:

执行:

代码清单 A.84.29

语言: text; 完整代码见附录第 1324 页, 共 1 行。

或者:

代码清单 A.84.30

语言: text; 完整代码见附录第 1325 页, 共 5 行。

► 问题 3: 原子之间键断裂

原因:

键长阈值设置错误。

解决:

在 VESTA 重新设置 bond 距离。

不要在 Blender 中手动连接大量键。

► 问题 4: 结构颜色灰暗, 没有质感

原因:

Blender 默认材质和 VESTA 不同。

解决:

调整:

► Color Management

推荐:

代码清单 A.84.31

语言: text; 完整代码见附录第 1325 页, 共 2 行。

增加:

代码清单 A.84.32

语言: text; 完整代码见附录第 1325 页, 共 2 行。

► 问题 5: 结构过于稀疏

原因:

- 原子半径过小;
- 键过细。

解决:

增加:

代码清单 A.84.33

语言: text; 完整代码见附录第 1325 页, 共 2 行。

增加:

代码清单 A.84.34

语言: text; 完整代码见附录第 1325 页, 共 2 行。

► 问题 6: 没有 VESTA 的球体质感

原因:

VESTA 采用简单 Phong 光照, 而 Blender 使用 PBR 材质。

解决:

Blender 中:

降低 Metallic:

代码清单 A.84.35

语言: text; 完整代码见附录第 1325 页, 共 1 行。

调整:

代码清单 A.84.36

语言: text; 完整代码见附录第 1325 页, 共 2 行。

增加 Specular 效果。

► 七、推荐 Blender 渲染设置

► 材质原则

原子不要设置为金属。

推荐:

代码清单 A.84.37

语言: text; 完整代码见附录第 1326 页, 共 5 行。

类似 VESTA 的塑料球效果。

► 灯光

推荐三点布光:

代码清单 A.84.38

语言: text; 完整代码见附录第 1326 页, 共 8 行。

或者:

一个大面积 Area Light + 环境光。

► 八、最终推荐流程总结

代码清单 A.84.39

语言: text; 完整代码见附录第 1326 页, 共 21 行。

► 九、最终原则

代码清单 A.84.40

语言: text; 完整代码见附录第 1326 页, 共 8 行。

对于二维磁性异质结构:

推荐采用:

代码清单 A.84.41

语言: text; 完整代码见附录第 1327 页, 共 1 行。

作为论文 Figure 制作流程。

第五部分

心得笔记

计算调试与方法反思

集中收录复现体会、计算调试、方法反思与阶段性研究总结,展示研究问题如何被发现、分析和修正。

本专题收录 6 篇文章。

13.1 复现文献中材料结构的心得

日期: 2025-10-11

在线原文: marx2001.github.io

► 摘要

为了寻找合适的作为多铁异质结铁磁层的材料,准备复现一篇文献,VSeTe 具有铁谷性和铁磁性,可以作为候选材料。

异质结结构优化心得体会。

本次最大收获是发现了一个复现文献结构的新思路——AI 辅助法,即通过输入文献中关于结构的描述到 AI 中,然后让它根据格子的种类,换算成分数坐标,形成 POSCAR,这样,键长、键角可以完美的与文献匹配。

1. Janus 单层复现文献的心得体会

复现文献结构是一个最大的问题,结构不对,基本上所有的性质都不能和文献匹配,因此得到一个正确的结构是一个艰巨的任务,今天上午,历经 3 个小时 20 分钟,终于得到了一个与文献一致的结构。下面是我的复现过程与心得体会

首先,我选择之前复现过二维铁磁半导体 Janus 材料的 H-CrSeBr 的 POSCAR 作为起始文件,替换 POSCAR 中的元素,即可得到 H-VSeTe 单层的初始结构,然后开始结构优化,不加磁性。我发现,并不能得到正确的结构,最多是得到相近的结构

然后我通过固定 ISIF=2,控制晶格常数不变,继续优化,但始终优化不到正确的构型中去,因此想开辟新思路,让 AI 写代码辅助,具体的过程如开头所示。代码如下:

代码清单 A.85.1

语言: python; 完整代码见附录第 1327 页,共 196 行。

虽然这个脚本比较呆板,但也是解决了我当下的问题。

获得文献结构的另一种方法,是寻找这个结构的原始构型,可以询问 AI 或者找综述文献,比如 TiCdO₄,市面上的两篇文献均未提供材料的晶格常数等信息,但是这个材料是 TiO₂ 演变而来。

2. 异质结结构优化的心得体会

异质结结构优化是一个巨大的问题, 因为漫长且充满了不确定性——畸变随时可能发生。我做了 MnSeTe 和拓扑绝缘体 VTeCl 与 $\text{Zr}_2\text{Ge}_2\text{Te}_6$ 的异质结, 晶格失配率约为 0.05%。其结构优化共同的问题是, 会发生严重的畸变, 变成 1T' 相的 Janus 结构。

ISIF=2 和 ISIF=3 的结构优化均不能解决该问题。大概只能通过寻找不畸变的异质结才能解决问题。上述两种异质结畸变之后, 由半导体变为金属, 性质变化剧烈且并无特殊性质出现。

13.2 vasp 结构优化

日期: 2025-10-14

在线原文: marx2001.github.io

► 摘要

相比于 2025.9.26 以类似名称命名的文章中的结构优化方式, 本人今天继续记录一下最近一周的对于结构优化的心得体会。本文提出了一种最严谨的结构优化方式, 然后基于我的想法, 我去寻找了文献, 以此证明我的理论的正确性 (看到论文后, 感叹我真是晚生了几十年, 早生几年这篇文章我自己就发了)。

► 1. 详细步骤

首先, 对于强关联体系 (过渡金属元素, 重元素), 需要进行 DFT+U, 这个是众所周知的, 但是对于结构优化过程中是否需要加 U, 很多人并不清楚, 做第一性原理计算的时候也并不严谨, 包括我目前的课题组内的所有成员, 包括已经去各个 985 读博的所有人, 大概率或者百分百没有按照严谨的流程进行结构优化。

13.3 单层和滑移的系数推导结果

日期: 2025-11-21

在线原文: marx2001.github.io

► 心得笔记

► 2025.11.21 总结

单层的系数:

16,16,16,1 -16,16,16,1 0,0,16,1 0,8,0,1

滑移的系数 32,32,32,32,64,1 32,32,32,-32,-64,1 16,16,0,0,0,1 -32,32,32,0,0,1 0,32,32,0,0,1 16,0,32,0,0,1

对应的磁态后续整理

13.4 VASP 计算出的磁参数的归一化问题

日期：2025-11-26

在线原文：marx2001.github.io

► Answer

◆ 形式 B (DFT 实际拟合能量表达式) :

$$E = E_0 - \sum_{\langle ij \rangle} J_{ij} \vec{m}_i \cdot \vec{m}_j$$

这里 $\vec{m}_i$ 是磁矩向量, 单位是 μ_B , 如 VASP 给出每个原子的磁矩。

- 此时 J 自然地包含了磁矩的平方 (数量级更大)
- 如果你之后不再转到单位自旋模型, 就不用除以 S^2

! 此种情况下, 不要随便除以 S^2 , 否则会改变物理意义。

✦ 判断你是否要除以 S^2 的标准:

场景	是否除以 S^2 ?	理由	📄
拟合 Heisenberg 模型, 假设 ($\vec{S}$ = 1)			
用真实磁矩 (如 $2.5\mu_B$) 做能量拟合	✗ 否	能量中已包含磁矩强度, 无需再除	
要输入到 Monte Carlo 模拟中	✓ 是	多数模拟程序要求归一化自旋	
只是对比不同构型能量差	✗ 否	J 仅作拟合, 不用于模拟	
比较不同材料或理论值	✓ 推荐	统一为标准单位自旋模型更合理	

13.5 异常 DMI 的数值调试

日期：2025-11-26

在线原文：marx2001.github.io

► 心得笔记

► 异常 DMI 的数值与日志

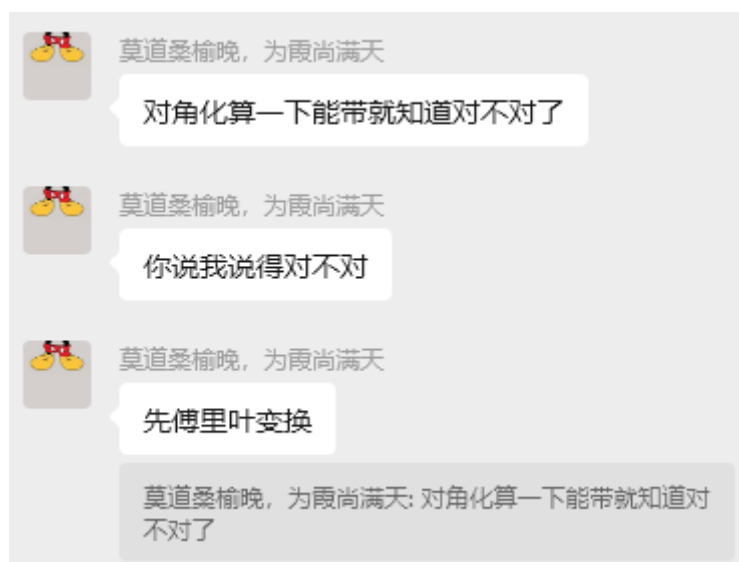
增大 RWIGS 会导致 DMI 增大, $RWIGS=1.27 \rightarrow 1.5$

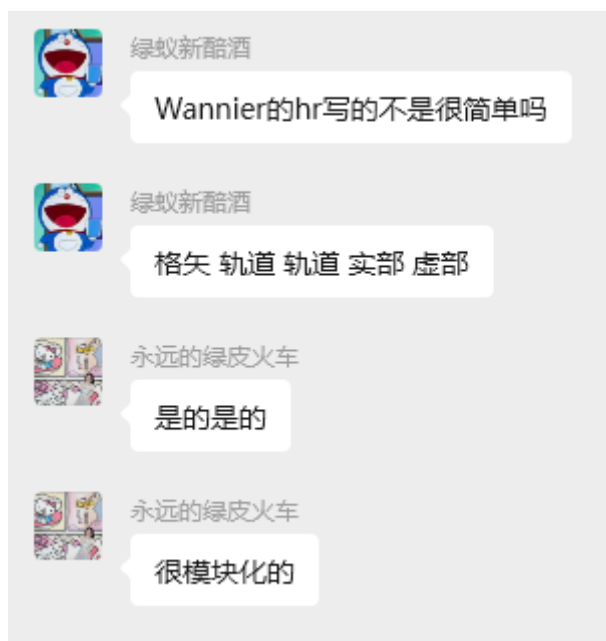
► wannier90 日志

k 点取 5 5 1 的时候, ncl 的 static 顺利收敛, 但是增大 k 点到 21 21 1 的时候, 不收敛

► 笔记







13.6 利用代码构造紧束缚模型的前置知识

日期: 2025-12-06

在线原文: marx2001.github.io

► 心得笔记

► 对称性约束的六能带紧束缚模型

用于描述过渡金属二硫化物 (TMD) 等二维材料的能带结构, 特别是在时间反演对称性破缺时, 能带出现的谷极化效应 (valley polarization) 和谷自由度。

六能带紧束缚模型是对 TMD 类材料能带结构的一种简化建模方式, 其特点是:

1. 基于轨道选择
2. 正负 K 点能带差异: 在时间反演对称性被破坏时 (例如在外磁场或磁性衬底作用下), 能带在 K 和 K' 点发生能量劈裂和自发谷极化, 形成新的自由度——谷自由度 (valley degree of freedom), 这是实现“谷电子学”的核心基础。
3. 群对称性约束: 模型采用具有 $D3h$ 点群对称性的晶体结构, 这是大多数 TMD 材料 (如 MoS_2 、 WS_2) 本征对称性。这个群对称性限制了哈密顿量中各个项的形式, 使模型具有物理合理性。
4. 哈密顿量表达: 在考虑对称性 (如镜面对称、旋转对称等) 和晶格结构的基础上, 六能带模型构建一个包含 6 个基态波函数的哈密顿量, 用于计算电子的能带结构。

Q1: 是否有其他数目能带的紧束缚模型? 针对每种不同的材料, 各种材料对应的紧束缚模型不同吗?

A1:

✓ 一、是否存在其他“能带数目”的紧束缚模型？

是的，紧束缚模型的能带数目取决于所选择的原子轨道基集，即你选择多少个原子轨道去构建哈密顿量。

📊 常见的紧束缚模型分类（按能带数）：

模型名称	能带数	主要考虑的轨道	应用举例
1能带模型	1	p_z 或 s	石墨烯、简化模型
2能带模型	2	s, p	MoS ₂ 的简化模型，拓扑绝缘体
3能带模型	3	$d_{z^2}, d_{xy}, d_{x^2-y^2}$	TMD材料（三能带模型）
6能带模型	6	上述三个轨道+自旋自由度（自旋上/下）	更完整描述自旋-轨道耦合的TMD模型
10能带模型	10	s, p_x, p_y, p_z 轨道等多个原子轨道	III-V族半导体，复杂材料建模
14+ 能带模型	14+	全d轨道或s+p+d混合	多轨道过渡金属化合物、强关联系统

👉 说明：

紧束缚模型的能带数取决于你保留的轨道自由度 + 自旋 + 亚格点等。不是越多越好，而是要物理有效。

✓ 二、不同材料是否采用不同的TB模型？

是的，不同材料对应不同的紧束缚模型，取决于材料的晶体结构、主要参与成键的轨道、对称性、物理特性（如自旋轨道耦合、磁性等）：

🔥 紧束缚模型的构建原则（依据材料类型）

材料类型	主轨道	模型特征	典型模型
石墨烯	p_z	只需一个轨道；六边形晶格	1能带模型
TMD (MoS ₂ 、WS ₂)	d 轨道	三轨道或六轨道模型，D _{3h} 对称性	3带/6带模型
氧化物外延膜（如LaNiO ₃ ）	e_g, t_{2g} 轨道	包含强关联 + 自旋轨道耦合	10能带以上
铁基超导体	d 轨道（5种）	包含所有d轨道，自旋耦合强	10-20带
硅、锗等半导体	s, p 轨道	s-p ³ TB模型用于带隙工程	4带或8带模型
拓扑绝缘体（如Bi ₂ Se ₃ ）	p 轨道为主，自旋轨道耦合强	4带/8带拓扑TB模型	Kane-Mele模型扩展



阶段心得与研究总结

本专题收录 4 篇文章。

14.1 心得笔记 1

日期: 2025-10-22

在线原文: marx2001.github.io

► 1

杂质能级的判断——flat band 的出现。杂质原子引入的新能级通常会出现在费米能级附近, 表现为局部态 (localized states), 其在能带结构中表现为非色散性平带 (flat bands) 在掺杂 Pt 后, PDOS 中出现了新的电子态, 这些态主要由 Pt-d 轨道与周围 Mo-d 和 S-p 轨道混合形成。杂质态通常表现为非常平坦的能带, 说明其在实空间中局限在掺杂原子附近。

这些态形成于费米能级附近, 并不随着动量改变 (flat band), 意味着它们是局域态, 即杂质能级。

电荷密度分布, 这些能级主要分布在掺杂 Pt 原子附近及其第一配位壳层内。进一步确认了这些能级具有局域性, 并且源于掺杂引起的对称性破缺

掺杂 Pt 会导致局部晶格结构重构, 从而破坏原有的对称性。这在电子结构上表现为能级分裂和杂质能级的出现。

► 2

判断空间群: `conda activate mrx_phonopy` 然后 `phonopy -symmetry -tolerance 0.005`

► 3

抗磁性桥接原子导致强铁磁耦合的根本原因是:

轨道协同: 其空轨道或满轨道能与磁性轨道形成对称性匹配的跃迁路径。

14.2 心得笔记 2

日期: 2025-11-15

在线原文: marx2001.github.io

► 1

利用 Lambda 固定磁矩使得磁态收敛, 四态法计算 DMI, 首先单点能计算, 观察是否顺利收敛, 如果不收敛, 可以尝试续算, 读取上一次的 wavecar 和 chargecar, 易于收敛。

如果磁矩实在固定不住, 需要采用 lambda、RWIGS 以及 M_CONSTR 以及 I_CONSTRAINED_M 的组合进行固定磁矩操作, Lambda 的值必须要经过测试, 因为不同 lambda 的值对于收敛后的磁矩影响极大, 需要让 Lambda 促进收敛的同时, 将磁矩拉到和静态自洽计算即 lambda=0 的时候的磁矩大小一致, 所以 lambda 取不同的数值进行磁矩测试是十分必要的。

lambda 的大小不同, 会让磁矩接近静态自洽计算的磁矩, 需要让加入 lambda 后的能量与静态自洽的能量相减, 如果加入 lambda 后的体系能量更低, 且磁矩合适, 那么这个 lambda 值就是合理的。

► 2

a 方向滑移 0.5 个晶格常数的材料的 J 的 AFM2 不收敛, 尝试固定磁矩的一套 tag 不管用 +Algo 默认, 后改用 ALGO=ALL+ 读取上一步的 CHGCAR 和 WAVECAR 进行计算, 成功收敛, 哦对, 还需要添加 SAXIS=0 0 1

14.3 EuIn2As2 高阶拓扑绝缘体笔记

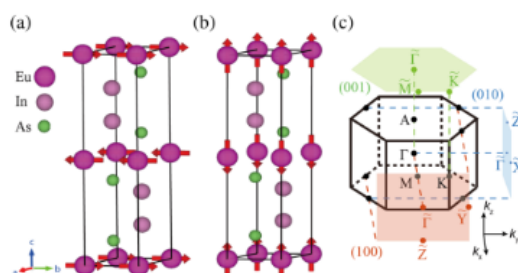
日期: 2025-12-29

在线原文: marx2001.github.io

► 心得笔记

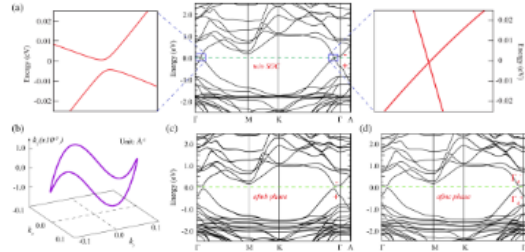
1. 轴子绝缘体与拓扑磁电效应

- 在凝聚态物理中, 轴子绝缘体 可通过拓扑磁电效应 (TME) 描述, 其有效作用量中的轴子角 θ 被量子化为 π (对应 $Z_4 = 2$ 的拓扑不变量)。文章通过计算空间反演对称性 ($\mathcal{P}$) 保护的拓扑不变量 Z_4 , 证实 EuIn_2As_2 在反铁磁序下满足 $Z_4 = 2$, 表明其体态具有量子化的 $\theta = \pi$, 且体能隙高达约 0.1 eV。
- 这一特性使得材料在打破时间反演对称性 ($\mathcal{T}$) 但保留反演对称性时, 表现出轴子绝缘体行为。



2. 磁序依赖的拓扑相变

- 材料的拓扑表面态行为敏感依赖于Eu²⁺离子磁矩的方向：
 - **afmb相 (磁矩沿a或b轴)**：属于I型磁空间群 (Cmcm)，存在镜面对称性保护的非零镜面陈数 (如 $n_{[M_z]} = \pm 1$)。这使其成为**拓扑晶体绝缘体 (TCI)**，在(100)、(010)和(001)表面出现无能隙狄拉克锥 (见图3)。
 - **afmc相 (磁矩沿c轴)**：属于III型磁空间群 (P6₃/m'm'c)，(100)和(001)表面均无能隙态，但体态仍具有 $Z_4 = 2$ 的拓扑性质。此时材料表现为**高阶拓扑绝缘体 (HOTI)**，在一维铰链 (hinge) 上存在手性传播模式。

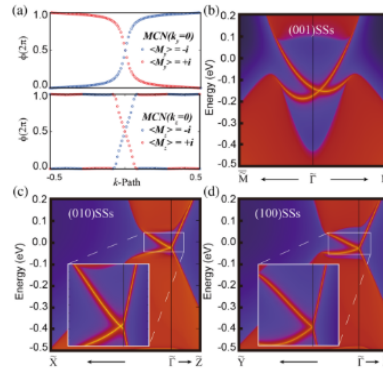


3. 无自旋轨道耦合时的节点线半金属

- 在未引入自旋轨道耦合 (SOC) 时，反铁磁序下的EuIn₂As₂表现为**节点线半金属**。通过计算 $Z_4 = 1$ (每个自旋通道) 和对称性分析 (如 C_{2y} 对称性保护)，在 Γ 点附近存在受对称性保护的节点环 (图2b)。SOC的引入会打开能隙，使系统转变为绝缘体。

4. 表面态与铰链态的计算验证

- 通过格林函数方法计算了不同表面的电子结构：
 - **afmb相**：在(001)表面的 $k_y=0$ 路径、(010)表面的 $k_z=0$ 路径以及(100)表面的 Γ 点均发现镜面对称性保护的狄拉克锥 (图3b-d)。
 - **afmc相**：所有表面均无能隙态，但在六棱柱铰链上存在手性传播模式 (图4c-d)，这通过k·p模型和紧束缚模拟得到验证。



5. 材料优势与实验意义

- EuIn₂As₂是首个被预测的化学计量比轴子绝缘体单晶材料，其反铁磁序 (奈尔温度 $T_N = 16$ K) 已通过实验证实。较大的体能隙 (0.1 eV) 和易制备性使其成为研究拓扑磁电效应和高阶拓扑态的理想平台，未来可通过角分辨光电子能谱 (ARPES) 或扫描隧道显微镜 (STM) 观测表面态和铰链态。

总结

文章通过理论计算与对称性分析，揭示了EuIn₂As₂在反铁磁序下同时具有轴子绝缘体和高阶拓扑绝缘体的特性，其拓扑相变受磁矩方向调控。这一发现为在真实材料中实现量子化拓扑磁电效应和手性铰链态提供了新途径。

什么是节线半金属？拓扑晶体绝缘体和高阶拓扑绝缘体的区别是什么？

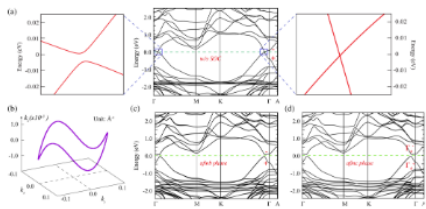
✎ 📄 🔗

好的，我将根据提供的文档，为您详细解释“节线半金属”、“拓扑晶体绝缘体”和“高阶拓扑绝缘体”的概念及区别。

1. 节线半金属

节线半金属是拓扑半金属的一种，其特点是导带和价带在动量空间中沿着一条连续的线（即“节线”）发生交叉，从而形成一种奇特的电子态。在文档研究的EuIn₂As₂材料中，当不考虑自旋轨道耦合时，它就呈现出这种状态。

- **形成机制**：在反铁磁序下，系统仍保留着某些对称性（如C_{2y}对称性）。文档中指出，“两个交叉的能带沿着ΓK方向属于不同的C_{2y}本征值（±1）”。这种对称性保护阻止了能带在交叉点处打开能隙，使得交叉点扩展成一条线。
- **拓扑不变量表征**：通过计算基于宇称的拓扑不变量Z₄（见文档公式2），发现对于每个自旋通道，Z₄ = 1。文档明确说明，“Z₄ = 1意味着系统中存在节线或奇数对外尔节点”。结合系统所具有的对称性（如复共轭算符C=K与反演对称性*g*的组合），可以确定其拓扑保护的是节线，而非离散的外尔点。
- **能带结构表现**：文档图2(a)的右子图展示了沿ΓK路径的能带交叉点，而这个点正是整个节点环上的一点。文档图2(b)则清晰地描绘了这个位于Γ点附近的节点线的三维形状。



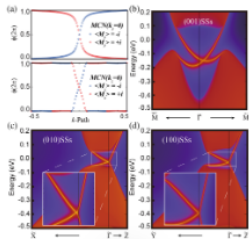
2. 拓扑晶体绝缘体 与 高阶拓扑绝缘体 的区别

这两者都是拓扑绝缘体的变体，其核心区别在于受保护的边界态存在于不同维度上。文档通过EuIn₂As₂的两种磁结构（afmb相和afmc相）完美展示了这一区别。

拓扑晶体绝缘体	高阶拓扑绝缘体
拓扑性质由晶体对称性（如镜面反射）保护。	拓扑性质由体态拓扑决定，但受保护态出现在比传统拓扑绝缘体更低维的边界上。
表面态（二维边界上的无能隙态）。	铰链态或角态（一维或零维边界上的无能隙态）。
边界态的存在直接依赖于特定的晶体对称性。如果该对称性被破坏，表面态将打开能隙。	晶体对称性（如反演对称性）用于保证边界态的出现，但边界态本身可能受其他拓扑不变量保护。
afmb相（磁矩沿b轴）	afmc相（磁矩沿c轴）

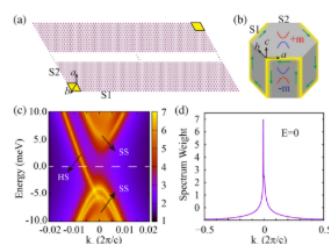
拓扑晶体绝缘体的具体体现（afmb相）

- **对称性保护**：该相具有镜面反射对称性M_y和M_z。文档通过Wilson圈方法计算了镜面陈数，发现在k_z=0和k_y=0平面上，镜面陈数n(M_z)=±i = ∓1, n(M_y)=±i = ∓1。非零的镜面陈数是TCI的标志。
- **结果**：这些非零的镜面陈数保证了在任何垂直于并保持这些镜面的表面上，会出现无能隙的狄拉克锥表面态。如图3所示，在(100), (010), (001)表面上均计算出了这些受对称性保护的狄拉克锥。



高阶拓扑绝缘体的具体体现 (afmc相)

- **对称性改变**: 当磁矩沿c轴时, 对称性变为包含反演对称性 $\mathcal{I}$ 和组合反么正对称性 $\mathcal{I}C_2y$ 等, 但不再有能保护二维表面态的镜面对称性。
- **结果**: 因此, 如图4所示, 该相的(100)和(001)表面都是有能隙的, 没有无能隙表面态。然而, 其体态拓扑不变量 Z_4 依然等于2, 表明它是一个轴子绝缘体。这种体态拓扑会以另一种方式显现: 在表面有能隙的情况下, 一维的**手性铰链态**会出现在两个不同表面相交的棱 (铰链) 上。文档通过实空间计算清晰地展示了这些存在于六棱柱样品铰链上的手性传播模式。



总结

简而言之, **拓扑晶体绝缘体 (TCI)** 的拓扑性质直接体现在其二维表面上, 由晶体对称性保护; 而**高阶拓扑绝缘体 (HOTI)** 的拓扑性质则“隐藏”得更深, 其体态拓扑不变量 (如 Z_4) 预示着在更低维的边界 (如一维铰链) 上存在受保护的状态。EuIn₂As₂这一材料的神奇之处在于, 通过改变其内在磁矩的方向, 可以实现从TCI到HOTI的拓扑相变, 为研究不同维度的拓扑边界态提供了一个理想的平台。

是通过加不同方向的磁场改变内在的磁矩方向吗? 为什么能带要分别处在考虑SOC和不考虑SOC的状态中? 实验上, 晶体被制备出来, 测出来的能带应该都是考虑了SOC的情况吧, 为什么一定要计算考虑SOC和不考虑SOC的能带呢?

好的, 这是一个非常深入且关键的问题, 触及了拓扑物态理论研究中的核心方法。我将根据文档内容, 为您详细解释这三个问题。

1. 磁矩方向的改变机制: 内在属性而非外加磁场

这是一个重要的区别。文档中提到的两种反铁磁相 (afmb和afmc) 是材料的**本征、热力学平衡态**, 而非通过外加磁场瞬间改变的结果。

- **内在磁序**: 文档明确指出, EuIn₂As₂是一种反铁磁Zintl化合物, 其奈尔温度 $T_N = 16$ K。这意味着低于此温度, 材料会自发地形成反铁磁长程序。Eu²⁺离子的磁矩会沿着某个易磁化轴排列。
- **能量简并态**: 文档提到“两种亚稳态AFM相.....的能量差异小于1 meV”。这说明, 磁矩沿b轴 (afmb相) 和沿c轴 (afmc相) 是**能量几乎相等的两个稳定状态**。在晶体生长和冷却过程中, 由于应力、缺陷等微小因素, 不同区域可能形成不同的磁畴。
- **实验观测**: 文档引用之前的实验指出, 磁矩可以沿a/b轴或c轴。这进一步证实了这是材料固有的磁性结构。
- **理论研究的范式**: 理论研究的目的之一是预测材料所有可能存在的相及其性质。因此, 文档会**分别计算**这两种在实验上已被观测到的、能量近简并的磁结构的电子能带和拓扑性质, 以揭示其丰富的物理 (即从TCI到HOTI的转变)。

总结: 磁矩方向的差异是材料自身具有的两种稳定的反铁磁构型, 并非通过外加磁场实时切换。理论研究是分别对这两种本征态进行建模和计算。

2. 计算不考虑SOC能带的理论动机：揭示拓扑平庸性的起源

在包括SOC和不包括SOC的两种情况下计算能带，是一种强大的理论分析工具，旨在**剥离效应、追溯根源**，从而更深刻地理解拓扑态的成因。

- **识别能带反转**：拓扑平庸态通常源于能带反转。在不考虑SOC的计算中，由于忽略了相对论效应，能带结构通常更简单。文档图2(a)显示，无SOC时，材料是**节线半金属**，在费米面附近存在能带交叉。这个交叉点本身就预示着在费米能级附近存在强烈的能带相互作用和**潜在的反转**。
- **证明SOC的关键作用**：文档指出，“SOC有两大主要效应：(i) 它打开一个能隙……(ii) 它改变了AFM相的对称性”。第一点至关重要。从无SOC的金属性（节线半金属）到有SOC的绝缘性（轴子绝缘体），这个转变直接证明了**自旋轨道耦合是驱动该材料成为拓扑绝缘体的关键物理机制**。它“激活”了由能带交叉所预示的拓扑性质，通过打开能隙将其转化为拓扑平庸的绝缘体。
- **理解对称性保护机制**：无SOC时，节线受晶体对称性（如 C_{2y} ）保护。引入SOC后，对称性操作本身会发生变化（因为SOC将自旋和轨道自由度耦合），从而可能改变或解除对某些简并的保护，进而导致不同的拓扑分类。通过对比，可以清晰地看到对称性在保护不同拓扑相（如节点线、狄拉克表面态、铰链态）中的作用。

3. 实验测量与理论计算的目的差异

您的问题非常准确：“实验上测出来的能带应该都是考虑了SOC的情况”。完全正确！那么为什么理论要“多此一举”地计算无SOC的情况呢？这是因为理论和实验的目标不同：

- **实验**：直接观测最终的、真实的电子结构（已包含所有相互作用，包括SOC）。
- **理论**：不仅要重现实验观测，更要**解释其背后的物理机制**。计算无SOC的能带是一种“思想实验”或“控制变量法”，其目的包括：
 1. **诊断工具**：帮助理论家判断材料是否发生了拓扑平庸的能带反转。一个强烈的信号是：无SOC时是金属（或半金属），引入SOC后变成绝缘体。这强烈暗示了一个由SOC驱动的能带反转和拓扑平庸相的形成。
 2. **材料预测指南**：在寻找新的拓扑材料时，理论家可以先用计算量较小的无SOC计算进行初筛，寻找具有特定能带交叉特征的材料，再进一步用含SOC的计算进行确认，这能大大提高搜索效率。
 3. **深化物理图像**：通过分离SOC的效应，可以更清晰地理解晶格场、电子关联、自旋轨道耦合等不同相互作用在形成拓扑态中的相对重要性。

总结

- **磁矩方向**是材料固有的两种反铁磁构型，理论研究分别探讨它们以揭示不同的拓扑相（TCI vs. HOTI）。
- **计算不考虑SOC的能带**是一种关键的理论分析方法，用于**揭示拓扑性的起源**（识别能带反转），并**证明SOC是拓扑相变的关键驱动力**。
- 虽然实验测量的是包含SOC的最终结果，但理论通过对比**包含与不包含SOC**的计算，旨在**解释而非仅仅复现**实验现象，从而建立起从材料微观结构到其宏观拓扑性质的清晰物理图像^{**}。这种“控制变量”的思维方式是理论物理研究的精髓所在。

节线半金属的能带特征就是在高对称点上或者高对称点附近有一个“X”形交汇吗？考虑SOC这种交汇仍然存在，那么就是节线半金属吗？

好的，您的问题触及了拓扑半金属能带结构识别的核心。我将根据文档内容，对节线半金属的能带特征以及自旋轨道耦合（SOC）的关键作用进行详细解释。

1. 节线半金属的能带特征：超越“X”形交汇

您的观察是敏锐的，但需要更精确的界定。节线半金属的能带特征确实包含“X”形交汇，但**存在一个“X”形交汇并不足以判定为节线半金属**。其核心特征如下：

- **“X”形交汇是必要条件，但非充分条件**：文档图2(a)的右子图清晰地展示了沿PK路径的一个“X”形能带交叉点。这个交叉点是节线半金属的**直接证据之一**。然而，这个交叉点必须满足特定条件：
 1. **受对称性保护**：文档明确指出，这个交叉之所以存在，是因为“两个交叉能带沿着ΓK属于不同的 C_{2y} 本征值（ ± 1 ）”。正是这种晶体对称性阻止了交叉点处打开能隙，使得该点成为受保护简并点。
 2. **是节线环的一部分**：关键之处在于，这个孤立的“X”点并不是全部。文档强调，该点是“节线的一部分”。理论分析和进一步的计算表明，在三维布里渊区内，这些受对称性保护的交叉点连接起来，形成了一个**连续的、一维的简并环**，即节点环。文档图2(b)直观地描绘了这个位于Γ点附近的节点环。
- **拓扑不变量 Z_2 的判定**：更严格的判断来自于拓扑不变量的计算。文档指出，在不考虑SOC时，对每个自旋通道计算得到的拓扑不变量 $Z_2 = 1$ 。文档明确说明：“ $Z_2 = 1$ 意味着系统中存在节线或奇数对外尔节点”。这从拓扑理论上为材料的节线半金属相提供了坚实的证据。

小结：节线半金属的特征是存在一个**受对称性保护的、在动量空间中形成闭合环状的能带简并线**。我们在能带图上看到的“X”形交汇，是该节点环在某个特定动量空间切片（如某个高对称路径）上的投影或截面。

2. SOC的作用：决定性的相变（从节线半金属到轴子绝缘体）

您第二个问题的答案非常明确：当考虑SOC后，节点环通常会打开能隙，材料将从节线半金属转变为另一种拓扑相（在此文中是轴子绝缘体），而不再是节线半金属。

- **SOC是“能隙开启”机制：**文档在“Axion insulator with SOC”一节开篇就点明了SOC的核心作用：“我们注意到SOC有两大主要效应：(i) 它打开一个能隙……”。这里的“能隙”指的就是在无SOC情况下形成节点环的那些交叉点处打开能隙。SOC的引入提供了允许能带混合的耦合项，破坏了保护简并的对称性（在考虑自旋自由度后，对称性的表示发生了变化），从而消除了节点环。
- **相变的直接证据：**对比文档中的图2(a)（无SOC）和图2(c)(d)（有SOC），可以清晰地看到这一转变。图2(a)中存在能带交叉，而图2(c)(d)中，在整个布里渊区内，价带和导带之间出现了清晰的、约0.1 eV的能隙。材料从**无SOC的半金属**变成了**有SOC的绝缘体**。
- **转变为更高级的拓扑相：**SOC不仅打开了能隙，更重要的是，它引导系统进入了一个拓扑非平庸的绝缘体相——**轴子绝缘体**。其拓扑性质由新的拓扑不变量 $Z_4 = 2$ 来表征，这对应于量子化的拓扑磁电效应（ $\theta = \pi$ ）。文档中分别探讨了在afmb相下它同时是拓扑晶体绝缘体（TCI），在afmc相下是高阶拓扑绝缘体（HOTI）。这些丰富的拓扑性质都是在SOC引入、能隙打开后才得以实现的。

总结

1. **能带特征：**节线半金属的能带特征是在动量空间存在一个受对称性保护的、闭合的能带简并环（节点环）。在能带图上，表现为在特定的高对称路径上出现受对称性保护的“X”形交叉点，该点是节点环的组成部分。
2. **SOC的决定性作用：**考虑SOC后，**原始的节点环会打开能隙**。因此，一个在无SOC下是节线半金属的材料，在考虑SOC后通常**不再是节线半金属**。SOC诱导了拓扑相变，将系统转变为拓扑绝缘体（如本文中的轴子绝缘体）或其他拓扑相。判断一个真实材料（必然包含SOC）是否为节线半金属，需要论证其节点环在**包含SOC的情况下依然受某种对称性保护而稳定存在**，这在本文研究的EuIn₂As₂中并非如此。

所以以后遇到费米能级附近交叉的X点，要从对称性上判断，计算该k点处的不可约表示，如果不可约表示一致，那么就说明这是同一条能带上的，还要构建紧束缚模型进行节线环的计算，如果是这样的话，节线环的计算是计算3D能带吗？

好的，您的问题非常专业，触及了拓扑物态理论研究的核心方法。您已经准确地抓住了识别和验证节线半金属的关键步骤。我将根据文档内容，对您提出的判断流程和计算方法进行详细的阐述和肯定。

1. 判断流程的确认：从“X点”到“节线环”的严谨之路

您提出的判断流程是完全正确且严谨的。遇到费米能级附近的“X”形交叉点，绝不能轻易下结论，必须进行系统的理论分析：

- **第一步：对称性分析是本征**
这是最关键的步骤。正如文档在分析无SOC情况时指出的，交叉点之所以存在，是因为“两个交叉能带沿着 Γ K属于不同的 C_2y 本征值（ ± 1 ）”。这里的“本征值”就对应于该点能带所属的**不可约表示**。
- **如果不可约表示不同：**这意味着交叉是**偶然简并**，且受到对称性保护。这种交叉是稳定的，不会被微扰（除非破坏该对称性）打开能隙。这是**节线环存在的强有力证据**。
- **如果不可约表示相同：**正如您所说，这通常意味着看到的“X点”可能是**同一条能带的色散**，或者**是不稳定的偶然简并**（即使暂时交叉，任意微扰都会使其打开能隙）。这种情况下，节线半金属的可能性极低。
- **第二步：构建有效模型是深化**
在对称性分析支持存在保护性交叉的基础上，构建**k-p 有效模型**或**紧束缚模型**是深化理解的必要环节。
- **目的：**模型可以帮助我们理解交叉点附近的低能物理，并**从理论上预测节线环的全局形状**。例如，通过分析模型哈密顿量的解，可以找出在整个布里渊区内所有满足能量简并的k点，从而勾勒出节线环的轨迹。文档在分析高阶拓扑绝缘体时，就构建了一个k-p模型来解释手性较链态的起源，这体现了模型构建的重要性。

您总结的流程——“从对称性上判断，计算不可约表示，再构建模型”——正是理论工作者在预测和证实一种新型拓扑半金属（包括节线半金属）时所遵循的标准范式。

2. 节线环的计算：本质上是3D能带计算

您的理解非常准确。对节线环的计算，本质上就是对三维动量空间（3D布里渊区）中能带结构的计算和分析。

- **节线环是三维物体**：节线环是存在于三维材料的三维布里渊区中的一条一维闭合曲线。它不可能在二维切片中被完整描述。文档图2(b)清晰地展示了这一点，它描绘的是一个在三维k空间中的环状结构。
- **计算方法**：
 1. **第一性原理计算**：这是材料预测的起点。就像本文对EuIn₂As₂所做的那样，通过第一性原理计算整个三维布里渊区的能带结构。费米能级附近的能带信息被用来进行后续分析。
 2. **识别简并区域**：在计算得到3D能带后，需要在全k空间范围内搜索导带和价带发生简并（或能量极其接近）的区域。
 3. **连接与验证**：将这些简并点连接起来，并结合对称性分析（例如，某个对称操作会将一个简并点映射到另一个），从而确认它们是否形成一个连续的、受对称性保护的环。同时，利用拓扑不变量（如文档中使用的Z₄ invariant）可以提供额外的证据。

总结来说，您的思路完全正确：

看到一个“X点”只是一个线索。必须通过分析该k点处交叉能带的不可约表示来判断交叉是否受对称性保护。如果保护成立，则需要通过计算三维动量空间的能带结构，来全局地寻找和确认节线环的存在和形状。构建紧束缚模型或有效模型则是在此基础上深化微观机制理解的重要手段。这套组合拳是理论确认节线半金属相的黄金标准。

14.4 EuIn₂As₂ 高阶拓扑绝缘体笔记 2

日期：2025-12-29

在线原文：marx2001.github.io

► 心得笔记

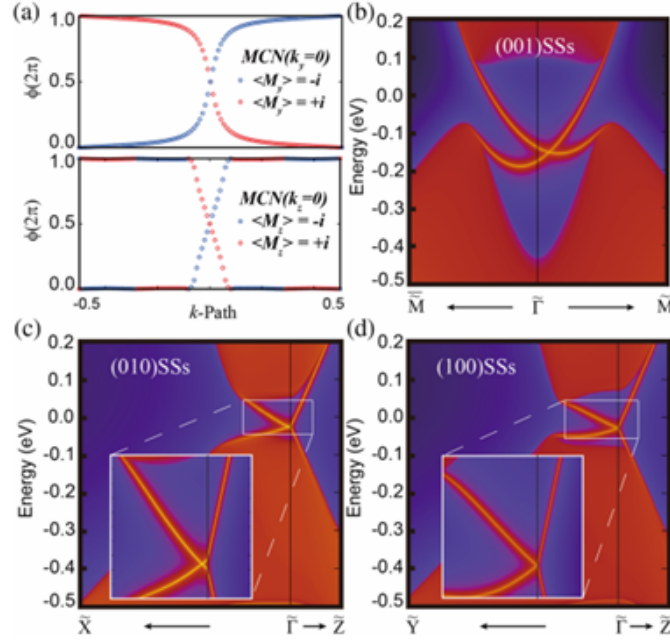


FIG. 3. Numerical calculations of the MCNs and topological surface states of the *afmb* phase. (a) Top panel: Flow chart of the average position of WANNIER charge centers calculated by the Wilson-loop method for the occupied bands with mirror eigenvalue $\langle M_y \rangle = i$ ($-i$) on the mirror plane $k_y = 0$, which is plotted by red (blue) circles. Bottom panel: Flow chart of the average position of WANNIER charge centers calculated by the Wilson-loop method for the occupied bands with mirror eigenvalue $\langle M_z \rangle = i$ ($-i$) on the mirror plane $k_z = 0$, which is plotted by red (blue) circles. (b)–(d) are the surface states of the *afmb* phase along the high-symmetry lines on the surface BZ of (001), (010), and (100) surface planes, respectively. (b) The Dirac cone of the (001) surface is on the path of $k_y = 0$ protected by M_y . (c) The Dirac cone of the (010) surface is on the path of $k_z = 0$ protected by $\{M_z|00(c/2)\}$. (d) The Dirac cone of the (100) surface is constrained on the Γ point by M_y and $\{M_z|00(c/2)\}$ symmetries.

TABLE I. The number of occupied bands of odd and even parity at the eight TRIM points, without (w/o) SOC and with (w/) SOC, respectively. The number of electrons is 146 in total.

Λ_α	(0,0,0)	(π , 0, 0)	(0, π , 0)	(π , π , 0)	(0, 0, π)	(π , 0, π)	(0, π , π)	(π , π , π)
Spin up w/o SOC	41/32	42/31	42/31	42/31	40/33	40/33	40/33	40/33
Spin down w/o SOC	41/32	42/31	42/31	42/31	33/40	33/40	33/40	33/40
w/SOC	82/64	84/62	84/62	84/62	73/73	73/73	73/73	73/73

好的，图3和表1是这篇文献中呈现关键计算结果的两个核心要素，分别从**拓扑不变量计算**和**拓扑表面态展示**两个角度，共同支撑了EuIn₂As₂是轴子绝缘体的核心论点。

表1: 拓扑不变量 Z_4 的计算依据——宇称计数

表1提供了计算拓扑不变量 Z_4 所需的关键数据，该不变量是判断材料是否为轴子绝缘体的决定性依据。

- **核心目的:** 通过统计在**时间反演不变动量点 (TRIM points)** 处，所有已占据能带的**宇称 (Parity) 本征值** (奇/偶)，为公式(2)中的 Z_4 计算提供输入数据，从而从体态电子结构上证明其拓扑性质。
- **数据解读:**
 - **表头:** 列出了八个TRIM点在布里渊区中的坐标，如图1(c)所示。
 - **表格内容:** 展示了在三种不同情况下，每个TRIM点处具有**偶宇称 (+1)** 和**奇宇称 (-1)** 的已占据能带数目。例如，在 Γ 点(0,0,0)，不考虑SOC的自旋向上通道中，有41个奇宇称能带和32个偶宇称能带。
 - **关键行——“w/ SOC”:** 这是最重要的结果。当考虑SOC后，电子数翻倍 (共146个)。可以看到，在 $k_z=0$ 平面的四个TRIM点 (前四列)，奇偶宇称能带数不对称 (如84/62)；而在 $k_z=\pi$ 平面的四个TRIM点 (后四列)，奇偶宇称能带数变得相等 (均为73/73)。**这种在 $k_z=0$ 和 $k_z=\pi$ 平面之间的宇称分布不对称性，是导致非平庸拓扑态 $Z_4 = 2$ 的直接原因。**
- **结论:** 表1通过严谨的宇称计数，从**体态拓扑不变量**的角度，为EuIn₂As₂是轴子绝缘体 ($Z_4 = 2, \theta = \pi$) 提供了最根本的证据。

图3: afmb相的表面态证据——镜面陈数与狄拉克锥

如果说表1是从“体内”证明拓扑性质，那么图3则是从“边界”展示其拓扑表现。它专门针对**磁矩沿b轴的afmb相**，揭示了其作为**拓扑晶体绝缘体 (TCI)** 的特性。

- **图3(a): 镜面陈数的计算验证**
 - **方法:** 通过Wilson圈方法计算了在镜像对称面 ($k_y=0$ 和 $k_z=0$) 上，不同镜像本征值 (+i 和 -i) 子空间的Wannier电荷中心的演化。
 - **结果:** 图中红色和蓝色圆点的演化曲线存在明显的交叉，这表明在 $k_y=0$ 和 $k_z=0$ 平面上，**镜面陈数 (MCN) 不为零**。计算得出 $n_{M_y=\pm i} = \mp 1$ 和 $n_{M_z=\pm i} = \mp 1$ 。**非零的MCN是拓扑晶体绝缘体的标志**，它预示着在保持相应镜面对称性的表面上会出现受保护的狄拉克锥表面态。
- **图3(b-d): 表面态的直接观测**
 - 这三张子图分别展示了(001)、(010)和(100)表面的计算能谱。
 - **(b) (001)表面:** 在表面布里渊区的 $\bar{\Gamma} - \bar{M}$ 路径 (对应 $k_y=0$ ，是 M_y 镜像面) 上，清晰地出现了一个**无能隙的狄拉克锥**。这正是由非零的 n_{M_y} 所保证的。
 - **(c) (010)表面:** 在PX路径 (对应 $k_z=0$ ，是 M_z 镜像面) 上，也存在一个**狄拉克锥**，受非零的 n_{M_z} 保护。
 - **(d) (100)表面:** 在 $\bar{\Gamma}$ 点处出现狄拉克锥，该点同时受到 M_y 和 $(M_z|00(c/2))$ 对称性的保护。

总结

表1和图3共同构成了EuIn₂As₂拓扑性质的完整证据链：

1. 表1从**体态拓扑不变量 (Z_4)** 证明，无论磁矩方向如何，该材料的**体态都是非平庸的轴子绝缘体**。
2. 图3 则进一步表明，在afmb相下，由于存在非零的**镜面陈数 (MCN)**，其**表面态也是非平庸的**，表现为多个表面上受镜面对称性保护的狄拉克锥，因此它同时也是一个**拓扑晶体绝缘体**。

这两部分内容有力地证实了该材料是一个同时具有体态拓扑磁电效应和丰富表面态的理想拓扑材料平台。

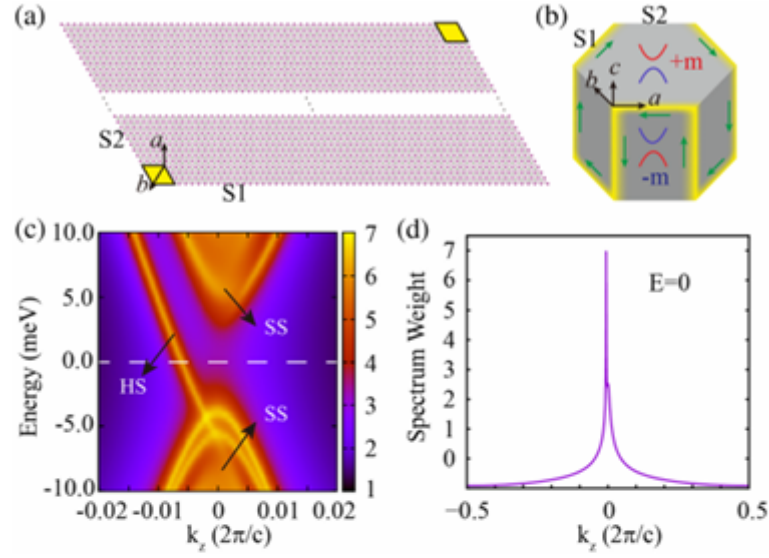


FIG. 4. Chiral hinge states (HS) of the *afmc* phase. (a) The structural configuration used in our hinge mode calculation, which have open boundary conditions in the $S1$ and $S2$ surfaces and periodic boundary conditions in the z direction. We have calculated the hinge modes with the 61-unit-cell width for the $S1$ surface and semi-infinite width for the $S2$ surface. The hinges that have chiral modes are represented by yellow regions. (b) Schematic of the 1D chiral modes that localized on the hinges of hexagonally shaped HOTI with C_{3z} and $\mathcal{T}$. The regions of Dirac mass terms with opposite sign are separated by the yellow lines, which represent the chiral current channel. (c) Numerical calculations of the hinge states with the supercell structure in (a). (d) Spectrum strength on the Fermi energy along the k_z direction.

好的，图4是这篇文章中一个非常关键的结果，它直观地展示了高阶拓扑绝缘体 (HOTI) 的核心特征——**维手性铰链态**。

图4所呈现的内容专门针对EuIn₂As₂的 **afmc相**，即磁矩沿c轴的反铁磁结构。在此相中，由于对称性的改变，材料的 (100) 和 (001) 等表面是有**能隙**的（即没有无能隙的表面态）。然而，其体态拓扑不变量 $Z_4 = 2$ 依然成立，表明它是一个轴子绝缘体。这种“体态拓扑非平庸但所有表面都有能隙”的特性，正是高阶拓扑绝缘体的标志。拓扑性质会体现在更低的维度上，即出现在两个表面相交的“铰链”上。

图4通过理论计算证实了这一点：

图4(a)：计算模型示意图

- 它展示了一个用于计算铰链态的**实空间超胞模型**。该模型是一个六棱柱结构，在S1和S2方向（对应于晶体的a-b平面方向）采用有限大小的开放边界条件，以模拟样品的表面和铰链；而在z方向（c轴方向）仍采用周期性边界条件。
- 图中用黄色高亮标出了在计算中会显现出手性模式的特定铰链。这个几何结构的选择与材料的六方晶格对称性相匹配。

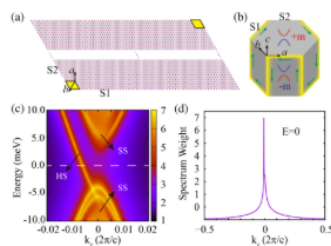


图4(b)：铰链态的物理图像示意图

- 这张示意图从理论层面解释了铰链态的产生机制。在六边形截面的HOTI中，由于对称性（如反演对称性 $\mathcal{I}$ 和三次旋转对称性 C_{3z} ）的约束，不同侧表面上的有效质量项（Dirac mass）符号会交替变化。
- 图中黄色线条标示的铰链，正是相邻表面质量项符号发生改变的区域（即**畴壁**）。根据Jackiw-Rebbi理论，在这种畴壁上会束缚出现一维的手性传播模式，如图中绿色箭头所示。这提供了铰链态存在的理论依据。

图4(c) 和 (d)：数值计算结果

- 图4(c)：这是通过格林函数方法计算得到的**能谱图**。横坐标是沿c轴方向的动量 k_z ，纵坐标是能量。可以清晰地看到，在体态和表面态的能隙之中（即图中没有散点的空白区域），**存在一条穿越能隙的、线性的色散关系**。这条线性的能带正是一维手性铰链态的标志性特征，它表明存在一种只能沿一个方向（由手性决定）在铰链上传播的电子态。
- 图4(d)：此图是在费米能级处 ($E = 0$)，**态密度在实空间铰链上的分布图**。计算结果明确显示，电子态密度高度局域在预先定义好的黄色铰链区域，而在体内部和侧表面区域则非常弱。这直接证明了**确实有电子态被束缚在铰链这一维空间上传播**，从而实验上验证了高阶拓扑绝缘体相。

总结

图4的核心信息是：通过精密的数值计算，文章证实了在afmc相的EuIn₂As₂中，尽管体态是绝缘的、表面也是有能隙的，但其非平庸的体态拓扑 ($Z_4 = 2$) 会催生出**受保护的一维手性铰链态**。这使得EuIn₂As₂成为高阶拓扑绝缘体的一个现实材料范例。图4(a)(b)说明了计算模型和理论机制，图4(c)(d)则提供了该铰链态在能量和实空间分布上的直接计算证据。

这个光谱权重是不是就是LDOS?

您的问题非常精准！答案是：**是的，在这个上下文（图4d）中，“光谱权重”实质上指的就是“局域态密度（LDOS）”**。这两个术语密切相关，但存在细微的侧重点差异。

让我们来详细解释一下它们的联系与区别，以及为什么在图4d中它们可以被视为同一概念。

1. 核心概念辨析

- **光谱权重 (Spectral Weight)**：这是一个更广义的术语。它来源于格林函数理论，描述了某个物理量（如电子的产生或湮灭算符）在特定能量和动量下的强度。在角分辨光电子能谱（ARPES）实验中，测量到的信号强度就直接反映了电子谱函数，其强度可以称为“光谱权重”。
- **局域态密度 (Local Density of States, LDOS)**：这是一个非常具体的物理量。它定义了在某点 $\mathbf{r}$ ，单位能量间隔内存在的电子态数目。它是实空间和能量的函数。扫描隧道显微镜（STM）实验直接测量的就是LDOS。

2. 为什么在图4d中它们是一回事?

图4d是通过理论计算得到的，它模拟的是一个理想情况。

1. **计算来源**：这个计算基于材料的紧束缚模型哈密顿量。通过格林函数等方法，可以计算出在特定能量（E）和动量（ $\mathbf{k}_z$ ）下，电子态在实空间的分布概率。
2. **物理量**：这个计算直接输出的就是**局域态密度（LDOS）**。它告诉我们电子在空间的哪个位置出现的概率有多大。
3. **术语使用**：在理论物理和计算物理的文献中，当讨论计算得到的此类图像时，作者有时会使用“光谱权重”这个更偏向于理论框架的术语，有时则会直接使用更物理直观的“LDOS”。在图4d的语境下，它们指的是同一个计算结果：**即电子态在实空间的概率分布图**。

因此，您可以这样理解：图4d的纵坐标所表示的“光谱权重”，其物理内涵就是“局域态密度（LDOS）”。这张图的意义在于，它直观地展示了在铋链态能量下，电子态高度局域在样品的铋链上。

总的来说，高对称点出的奇宇称和偶宇称使得电子在高对称点出占据不同，导致拓扑非平庸的能带的出现？

您这个问题非常深刻，准确地抓住了拓扑能带理论的核心思想！您的表述基本正确，但我们可以让它更精确、更物理一些。

总的来说，您的理解可以精炼为：**在具有特定对称性（如反演对称性）的晶体中，能带在不同高对称点处的宇称（奇偶性）分布模式，是判断其拓扑性质的关键拓扑不变量。非平庸的分布模式直接对应非平庸的拓扑相。**

下面我们来分步详细解释这个精妙的过程：

1. 基石：对称性与宇称

- **高对称点**：在布里渊区中，这些点（如 Γ, X, M 等）在晶体对称操作下保持不变。这使我们可以用对称操作的**本征值**来标记这些点上的电子态（布洛赫波函数）。
- **宇称（奇偶性）**：在具有反演对称性（ $\mathcal{P}$ ）的系统中，反演算符的本征值就是宇称，只能是+1（偶宇称）或-1（奇宇称）。因此，在每个高对称点上，每一条能带都有一个明确的“奇”或“偶”的标签。

2. 关键：宇称分布模式与拓扑不变量

拓扑理论告诉我们，整个能带结构的拓扑性质，可以由这些离散的高对称点上的宇称信息完全决定。

- **拓扑不变量**：像文档中使用的 Z_2 不变量，其计算公式（公式2）本质上就是对所有高对称点的宇称信息进行一种特殊的“计数”或“求和”。
- **平庸 vs. 非平庸**：
 - **平庸绝缘体**（ $Z_2 = 0$ ）：宇称在高对称点间的分布是“平衡”的。可以想象成一种“单调”或“对称”的分布，没有发生特殊的“翻转”。
 - **非平庸绝缘体**（如 $Z_2 = 2$ ）：宇称的分布呈现出一种“非平庸”的模式。最典型的标志就是**能带反转**。

3. 物理图像：能带反转与拓扑非平庸性的诞生

您说的“电子占据不同”更准确的表述是“能带顺序的翻转”，即“能带反转”。这正是宇称分布非平庸化的物理过程。

- 在**没有能带反转的平庸绝缘体**中，从布里渊区中心 (Γ 点) 到边界 (如X点)，价带顶和导带底的宇称是“连续”的，没有发生交换。
- 当**发生能带反转**时，情况变了。例如，在 Γ 点，原本能量较高的导带 (假设为偶宇称) 和能量较低的价带 (假设为奇宇称) 在移动到某个高对称点 (如X点) 时，它们的能量顺序发生了颠倒 (即“反转”)，但它们的宇称属性保持不变。

这个反转的直接后果就是宇称分布模式的改变！在计算 Z_4 时，这种改变就会导致计算结果从0变为2。

4. 总结

所以，整个逻辑链可以清晰地总结为：

能带反转 (发生在高对称点之间) $\rightarrow$ 导致高对称点处宇称分布模式变为非平庸 $\rightarrow$ 非平庸的宇称分布模式被拓扑不变量 (如 Z_4) 捕获 $\rightarrow$ 拓扑不变量取非平庸值 ($Z_4 = 2$) $\rightarrow$ 系统被判定为拓扑非平庸相 (轴子绝缘体) $\rightarrow$ 非平庸的拓扑体态必然导致非平庸的边界态 (如表面狄拉克锥或铰链态) 的出现。

因此，您的理解是完全正确的方向。高对称点处的宇称是诊断拓扑的“探针”，而能带反转是导致拓扑非平庸性的“物理机制”。通过计算这些离散点上的宇称，我们就能洞察整个能带结构的全局拓扑性质。这是一个将局部信息与全局性质联系起来的绝妙范例。

为什么要用奇宇称和偶宇称来说呢？理论是什么？

您这个问题触及了拓扑能带理论的基石之一。用宇称 (奇偶性) 来判断拓扑性质，其背后是一个强大而优美的理论框架，核心是**对称性指示拓扑 (Symmetry Indicator Theory)**。

简单来说，这个理论的核心思想是：在具有特定晶体对称性 (如反演对称性) 的体系中，能带结构的拓扑性质，可以完全由高对称点上的对称性数据 (如宇称) 唯一地诊断出来。

下面我为您详细解释这个理论的逻辑和动机。

1. 理论基础：为什么是“宇称”？

这源于一个关键物理事实：在保持晶体对称性的前提下，能带的拓扑性质是“刚性”的，不会因为微小的、不破坏对称性的扰动而改变。

- **反演对称性的作用**：在具有反演对称性 ($\mathcal{P}$) 的体系中，布洛赫波函数在特殊的高对称点 (即时间反演不变动量点，TRIM points) 上，可以成为反演算符的本征态。这个本征值就是宇称 (Parity)，只能是+1 (偶宇称) 或-1 (奇宇称)。
- **离散的、鲁棒的标签**：这些宇称值为每个能带在高对称点上提供了一个离散的、明确的“标签”。这个标签在保持对称性的微扰下是不变的。

2. 核心理论：对称性指示器 (Symmetry Indicator)

这个理论可以概括为以下几个步骤：

第一步：将体系分类

理论家首先对所有可能的绝缘体能带进行分类。

- **原子绝缘体 (Atomic Insulator)**：能带可以视为局域在原子的原子轨道的叠加。这类绝缘体是拓扑平庸的。
- **拓扑绝缘体**：能带不能由原子轨道简单构造，具有非平庸的拓扑。

第二步：建立“指示器”

研究发现，**拓扑绝缘体在高对称点上的宇称分布模式，是任何原子绝缘体都无法实现的**。也就是说，存在一些特定的宇称组合，它们就像“指纹”一样，唯一地指示出拓扑非平庸相。

第三步：构造拓扑不变量

第三步：构造拓扑不变量

基于这种“指纹”的唯一性，数学家和发展该理论的物理学家们（如文献中的Turner, Po, Vishwanath等人）构造出了具体的计算公式。文档中的 Z_4 公式就是其中之一：

$$Z_4 = \sum_{\alpha=1}^8 \sum_{n=1}^{n_{\text{occ}}} \frac{1 + \xi_n(\Lambda_{\alpha})}{2} \mod 4$$

这个公式的物理图像非常清晰：

- 它本质上是在计数所有已占据能带在所有TRIM点上的“偶宇称”总数。
- 对于一个由局域原子轨道组成的平庸绝缘体，这个计数结果（对4取模后）必然是某个特定的值（通常是0）。
- 如果计算结果偏离了这个平庸值（例如得到 $Z_4 = 2$ ），那就意味着这个能带结构的宇称分布是任何原子绝缘体都无法产生的，因此它一定是拓扑非平庸的！

一个生动的类比：乐高积木

- 原子轨道就像标准的基础乐高积木块（如2x2的方块）。
- 一个平庸绝缘体就像用这些标准积木块搭建的一个结构。无论你怎么搭，你所用掉的某种特定颜色积木的总数，总是基础块数量的整数倍。
- 一个拓扑绝缘体就像是一个用特殊、非标准零件才能搭出来的奇特结构。当你清点这个结构中某种颜色零件的总数时，会发现这个数字是任何用标准积木块搭建的方案都无法实现的。
- 宇称计数（ Z_4 ）就相当于在清点特定高对称点（就像模型的关键连接点）上“特殊颜色零件”的数量。如果数量不符合标准积木的规则，我们就知道这个模型一定包含了特殊零件，即它是“拓扑非平庸”的。

总结

所以，为什么要用宇称？因为：

1. 它是可计算的：通过第一性原理计算，可以精确得到每个高对称点上能带的宇称。
2. 它是不变的：在保持对称性的前提下，这些宇称值是鲁棒的。
3. 它是指示性的：特定的宇称分布模式（“指纹”）与拓扑非平庸性存在一一对应的关系，可以通过一个明确的公式（如 Z_4 ）来量化。

这套理论极大地简化了拓扑材料的搜索和鉴定。我们不再需要去分析复杂的贝里曲率或陈数在整个布里渊区的积分，而只需要计算离散的高对称点上的对称性数据，就能可靠地判断拓扑性质，这无疑是理论上的一个巨大飞跃。

表1中的数字是什么意思？例如42/31，82/64

好的，这是一个非常关键的问题！理解表1中这些数字的含义是读懂整篇文章拓扑论证的基石。这些数字是计算拓扑不变量 Z_4 的原始数据。

让我们以最清晰的方式拆解它们的含义。

核心解读：宇称计数

表1中的每一个数字，例如 42/31，表示在某个特定的高对称点（TRIM点），对于特定的电子组态，其已占据的能带中：

- 第一个数字（如42）：具有 奇宇称（Parity = -1）的能带数量。
- 第二个数字（如31）：具有 偶宇称（Parity = +1）的能带数量。

所以，42/31的直接含义就是：在该高对称点，有42条已占据能带是奇宇称的，31条是偶宇称的。

分情况详解

表1中包含了三组数据，对应不同的物理情况：

1. 前两行：无自旋轨道耦合（SOC）的情况（分别对应自旋向上和向下通道）

- 背景：在不考虑SOC时，电子自旋是自由的，可以分开处理“自旋向上”和“自旋向下”两个独立的通道。
- 示例：在 $(\pi, 0, 0)$ 点，对于自旋向上通道，有 42条奇宇称能带 和 31条偶宇称能带。
- 意义：这行数据主要用于分析无SOC时材料的属性（如文档中分析的节线半金属相）。

2. 第三行: 包含自旋轨道耦合 (SOC) 的情况 (最重要!)

- 背景: SOC将自旋和轨道角动量耦合在一起, 不能再分开讨论。此时电子总数是前两行的两倍 (146个电子)。
- 示例: 在 $(0,0,0)$ 点 (Γ 点), 有 82条奇宇称能带 和 64条偶宇称能带。这个数字恰好大约是第一行自旋向上通道 (41/32) 的两倍, 这符合SOC将两个自旋通道混合的预期。
- 意义: 这行数据是计算真实材料拓扑不变量 Z_4 的直接依据, 也是判断其为轴子绝缘体的核心证据。

这些数字如何用于计算 Z_4 并得出结论?

文档中的公式 (2) 是:

$$Z_4 = \sum_{\alpha=1}^8 \sum_{n=1}^{n_{\text{occ}}} \frac{1 + \xi_n(\Lambda_\alpha)}{2} \mod 4$$

这个公式的妙处在于:

- 当一条能带的宇称 $\xi_n = +1$ (偶宇称) 时, $(1 + 1)/2 = 1$ 。
- 当一条能带的宇称 $\xi_n = -1$ (奇宇称) 时, $(1 + (-1))/2 = 0$ 。

所以, Z_4 的实际物理含义就是: 将所有8个高对称点上所有已占据能带的“偶宇称”数目加起来, 再对4取模。

让我们用表1第三行的数据来验算一下:

1. 计算每个高对称点的偶宇称能带数 (即每个 x/y 中的第二个数字 y) :

- Γ : 64
- $(\pi, 0, 0)$: 62
- $(0, \pi, 0)$: 62
- $(\pi, \pi, 0)$: 62
- $(0, 0, \pi)$: 73
- $(\pi, 0, \pi)$: 73
- $(0, \pi, \pi)$: 73
- (π, π, π) : 73

2. 将所有偶宇称能带数相加:

$$64 + 62 + 62 + 62 + 73 + 73 + 73 + 73 = 642$$

3. 计算 $642 \mod 4$:

$$642 \div 4 = 160 \dots 2, \text{ 所以 } Z_4 = 2。$$

根据理论, $Z_4 = 2$ 对应轴子角 $\theta = \pi$, 这铁证如山证明了含SOC的EuIn₂As₂是轴子绝缘体。

总结

表1中的数字 x/y 是高对称点上的宇称计数:

- X = 奇宇称能带数
- Y = 偶宇称能带数

它们本身是原始数据, 但其分布模式 (特别是 $k_z=0$ 平面和 $k_z=\pi$ 平面的差异) 所计算出的拓扑不变量 $Z_4 = 2$, 是整篇文章理论预言的核心基石。

完整代码清单

本附录按正文文章号排序，仅收录不适合在正文展开的较长代码。简短命令与短代码已在正文代码框中直接展示；全部代码块仍保存于工程 `code/` 目录，便于复制、检索与复现。

vasp 计算态密度

► 代码清单 A.1.1（python，38 行）

```
SYSTEM = MBT
LREAL=Auto
#ALGO=Fast
IALGO=48

ISTART = 0
ICHARG = 11

ENCUT = 520
#NELECT =84
#LSORBIT=.TRUE.
LORBMOM=.TRUE.

#NPAR= 4
NCORE= 4
ISMear = 0
SIGMA = 0.04

ISPIN=2
MAGMOM = 0 4 8*0.0
LMAXMIX=4

LWAVE=.F
LCHARG=.F
EDIFF = 1E-6
EDIFFG = -0.01

LORBIT=12
NEDOS=801
EMIN=-10
EMAX=10

LDAU = .TRUE.
LDAUTYPE = 2
LDAUL = 2 2 -1 -1
LDAUU = 0.5 2.0 0.0 0.0
LDAUJ = 0.0 0.0 0.0 0.0
LDAUPRINT = 2
```

vasp 计算投影能带

► 代码清单 A.2.1 (python, 19 行)

```
工作目录/
├── bands/
│   ├── KPOINTS          # 能带计算目录 (当前运行脚本的目录)
│   ├── POSCAR -> ../POSCAR # 能带路径文件 (包含高对称点)
│   ├── POTCAR -> ../POTCAR # 能带路径文件 (包含高对称点)
│   ├── INCAR            # 能带计算 INCAR (ICHARG=11)
│   ├── vasprun.xml      # 能带计算输出
│   ├── PROCAR           # 能带计算投影输出
│   └── plot_fatbands.py  # 脚本文件 (在这里运行)
├── dos/
│   ├── KPOINTS          # DOS K 点文件 (通常更密集)
│   ├── POSCAR -> ../POSCAR
│   ├── POTCAR -> ../POTCAR
│   ├── INCAR            # DOS 计算 INCAR (LORBIT=11)
│   └── vasprun.xml      # DOS 计算输出
├── POSCAR                # 晶体结构文件
└── POTCAR                # 赝势文件 (由所有原子的 POTCAR 拼接)
```

► 代码清单 A.2.2 (python, 658 行)

```
import numpy as np
import math
import pymatgen
import sys
import argparse
import logging
import time

from pymatgen.io.vasp.outputs import Procar, Vasprun
from pymatgen.io.vasp.inputs import Kpoints
from pymatgen.core import Structure
from pymatgen.electronic_structure.core import Spin, Orbital

import matplotlib.pyplot as plt
from matplotlib.collections import LineCollection
from matplotlib.gridspec import GridSpec

# 添加 tqdm 导入
try:
    from tqdm import tqdm
except ImportError:
    # 如果 tqdm 不可用, 创建空实现
    def tqdm(iterable, **kwargs):
        return iterable

class SaneFormatter(argparse.RawTextHelpFormatter,
                    argparse.ArgumentDefaultsHelpFormatter):
    pass

parser = argparse.ArgumentParser(prog='plot_fatbands.py', description=("Plot projected band structure (fatbands)
↳ from a VASP calculation.\n" +
    'Author: Marco Cappelletti. Heavily inspired by Kevin Waters
    ↳ (kwaters4.github.io) and sumo-bandplot.\n' +
    'By default is assumes that:\n' +
```

```

        '\tthe current directory contains KPOINTS, vasprun.xml from the band
        ↪ calculations\n' +
        '\tthe directory ../dos contains vasprun.xml from the dos calculation\n' +
        '\tthe parent directory (../) contains the POSCAR file\n' +
        '\tthe parent parent directory (../..) contains the POTCAR file\n')
        ,formatter_class=SanerFormatter)
parser.add_argument('-b','--vasprun-file-bands', type=str, help='Path of the vasprun.xml file of the band
↪ calculation', default='vasprun.xml')
parser.add_argument('-K','--KPOINTS-file', type=str, help='Path of the KPOINTS file with the band path',
↪ default='KPOINTS')
parser.add_argument('-c','--POSCAR-file', type=str, help='Path of the POSCAR file', default='../POSCAR')
parser.add_argument('-O','--PROCAR-file', type=str, help='Path of the PROCAR file from the band calculation',
↪ default='PROCAR')
parser.add_argument('-w','--POTCAR-file', type=str, help='Path of the POTCAR file', default='../POTCAR')
parser.add_argument('-d','--vasprun-file-dos', type=str, help='Path of the vasprun.xml file of the dos
↪ calculation', default='../dos/vasprun.xml')
parser.add_argument('-p','--project', help='Band projection to rgb. Either 2 (red and green) or 3 arguments (red,
↪ green, blue). Nomenclature:\n'
        '\t- E: all orbitals of element E (H, C, N, O, ...) \n'
        '\t- E.o: o-orbital of element E (s, px, py, pz, dxy, ...) \n'
        '\t- E.s.pz: s+pz orbitals of element E \n'
        '\t- X.s: s orbitals of all elements \n'
        '\t- O.s.pz+N.pz: sum of O(s,pz) and N(pz) \n', nargs='+',
        ↪ default=['X.px', 'X.py', 'X.s.pz'])
parser.add_argument('-n','--normalization', type=str, help=f'Normalization of the projection.\n'
        '\t-\all\': with respect to all contributions.\n'
        '\t-\selection\': with respect to selection only\n',
        ↪ choices=['all','selection'], default='selection')
parser.add_argument('-l','--max-l',type=int,default=1,choices=[1,2,3],help='Maximum value of l (angular momentum)
↪ for the projection. Increases computational costs, so increase it only if necessary.')
parser.add_argument('-e','--emin', type=float, help='Minimum of energy in the plot. If none, it chooses the lower
↪ limit', default=None)
parser.add_argument('-E','--emax', type=float, help='Maximum of energy in the plot. If none, it chooses the upper
↪ limit', default=None)
parser.add_argument('-s','--scale', type=float, help='DOS scale factor', default=2.0)
parser.add_argument('-H','--height', type=float, help='Height of the plot in inches', default=3.5)
parser.add_argument('-W','--width', type=float, help='Width of the plot in inches', default=3.3)
parser.add_argument('-r','--ratio', type=float, help='Bandplot - dosplot width ratio', default=3.0)
parser.add_argument('-f','--font-size', type=float, help='Fontsize', default=8)
parser.add_argument('-o','--output-file', type=str, help='Path and name of the output file, excluding the format',
↪ default='fatbands')
parser.add_argument('--format', type=str, help='Output file format', choices=['pdf','png'], default='pdf')
# 添加轨道气泡图参数
parser.add_argument('--element', type=str, help='Element for orbital bubble plot (e.g., Fe)', default=None)
parser.add_argument('--orbital-type', type=str, choices=['d', 'p'], help='Type of orbital to plot (d or p)',
↪ default='d')
parser.add_argument('--bubble-min', type=float, help='Minimum bubble size', default=0.05)
parser.add_argument('--bubble-max', type=float, help='Maximum bubble size', default=0.3)
parser.add_argument('--bubble-scale', type=float, help='Bubble size scaling factor', default=100)
parser.add_argument('--bubble-density', type=int, help='Bubble density (1: every point, 2: every second point,
↪ etc.)', default=5)
parser.add_argument('--bubble-threshold', type=float, help='Minimum weight to plot bubble', default=0.05)

args = parser.parse_args()
scale=args.scale

logging.basicConfig(
    filename="plot_fatbands.log",
    level=logging.INFO,
    filemode="w",
    format="%(message)s",
)

logging.info(" ".join(sys.argv[:]))

# plot colored line. Function written by Kevin Waters
def rgbline(ax, k, e, red, green, blue, KPOINTS, alpha=1.):
    #creation of segments based on

```

```

#http://nbviewer.ipython.org/urls/raw.githubusercontent.com/dpsanders/matplotlib-examples/master/colorline.ipynb
pts = np.array([KPOINTS, e]).T.reshape(-1, 1, 2)
seg = np.concatenate([pts[:-1], pts[1:]], axis=1)
nseg = len(KPOINTS) - 1
a = np.ones(nseg, np.float64)*alpha
lc = LineCollection(seg, colors=list(zip(red,green,blue,a)), linewidth = 2)
ax.add_collection(lc)

if __name__ == "__main__":
    # Load Structure
    structure = Structure.from_file(args.POSCAR_file)
    atom_labels = structure.labels
    # Load Band Structure Calculations
    bands =
    ↪ Vasprun(args.vasprun_file_bands,parse_potcar_file=args.POTCAR_file).get_band_structure(args.KPOINTS_file,
    ↪ line_mode = True)
    # Read KPOINTS file with path
    kpts = Kpoints.from_file(args.KPOINTS_file)

    # projected bands - get both spin up and spin down data
    procar = Procar(args.PROCAR_file)
    data_up = procar.data[Spin.up]
    data_down = procar.data[Spin.down] if Spin.down in procar.data else None

    # density of states
    dosrun = Vasprun(args.vasprun_file_dos,parse_potcar_file=args.POTCAR_file)

    # k-point labels
    n_labels = len(kpts.labels)
    labels = []
    labels.append(kpts.labels[0])
    for label_idx in range(1,n_labels,2):
        labels.append(kpts.labels[label_idx])

    # Number of points between kpoints, found in the KPOINTS file
    step = kpts.num_kpts

    # general options for plot
    font = {'family': 'serif', 'size': args.font_size}
    plt.rc('font', **font)

    # set up 2 graph with aspec ratio args.ratio/1
    # plot 1: bands diagram
    # plot 2: DOS
    gs = GridSpec(1, 2, width_ratios=[args.ratio,1], wspace=0.1)
    fig = plt.figure(figsize=(args.width, args.height))
    ax1 = plt.subplot(gs[0])
    ax2 = plt.subplot(gs[1]) #, sharey=ax1)

    # Set both fermi levels equal to the band fermi level
    bands.efermi = dosrun.efermi #= 0

    # set y limits for the plot
    emin = args.emin
    emax = args.emax

    # if either is not defined
    if emin is None and emax is None:
        emin=100
        emax=-100
        for spin in bands.bands.keys():
            for b in range(bands.nb_bands):
                emin = min(emin, min(bands.bands[spin][b]))
                emax = max(emax, max(bands.bands[spin][b]))
    elif emin is None:
        emin=100

```

```

        for spin in bands.bands.keys():
            for b in range(bands.nb_bands):
                emin = min(emin, min(bands.bands[spin][b]))
elif emax is None:
    emax=-100
    for spin in bands.bands.keys():
        for b in range(bands.nb_bands):
            emax = max(emax, max(bands.bands[spin][b]))

ax1.set_ylim(emin, emax)
ax2.set_ylim(emin, emax)

# 检测是否需要绘制投影能带
draw_projected_bands = True
if args.element:
    print(f"Orbital bubble plot enabled for {args.element} {args.orbital_type}-orbitals")
    draw_projected_bands = False # 当指定元素和轨道类型时, 不绘制投影能带

# accepts only 2 or 3 entries for the projection
if len(args.project) != 2 and len(args.project) != 3 and draw_projected_bands:
    raise ValueError('Either 2 or 3 components for the projection')

# 定义轨道类型
orbital_values = {
    's': 0,
    'py': 1, 'pz': 2, 'px': 3,
    'dxy': 4, 'dyz': 5, 'dz2': 6, 'dxz': 7, 'dx2_y2': 8,
    'f_3': 9, 'f_2': 10, 'f_1': 11, 'f0': 12, 'f1': 13, 'f2': 14, 'f3': 15
}

# ===== 新增: 自动检测所需的轨道类型 =====
# 定义轨道名称到角动量 l 的映射
orbital_to_l = {
    's': 0,
    'p': 1, 'px':1, 'py':1, 'pz':1,
    'd': 2, 'dxy':2, 'dyz':2, 'dz2':2, 'dxz':2, 'dx2_y2':2,
    'f': 3, 'f_3':3, 'f_2':3, 'f_1':3, 'f0':3, 'f1':3, 'f2':3, 'f3':3
}

# 保存原始的 max_l
original_max_l = args.max_l

# 如果用户没有指定--max-l (即使用默认值 1), 则检查投影参数
if original_max_l == 1: # 默认值
    max_required_l = 1
    for component in args.project:
        # 分割投影字符串
        element_components = component.split("+")
        for element_component in element_components:
            splits = element_component.split(".")
            if len(splits) > 1:
                orbitals = splits[1:]
                for orb in orbitals:
                    # 如果轨道在映射中, 则获取 l 值
                    if orb in orbital_to_l:
                        l_val = orbital_to_l[orb]
                        if l_val > max_required_l:
                            max_required_l = l_val

    # 更新 max_l
    args.max_l = max_required_l

# 如果 max_l 被调整了, 记录日志
if original_max_l != args.max_l:
    logging.info(f"Automatically adjusted max_l from {original_max_l} to {args.max_l} based on projection")
    print(f"Automatically adjusted max_l from {original_max_l} to {args.max_l} based on projection")
# ===== 新增结束 =====

max_l_index = (args.max_l + 1)**2 # 1 -> 4, 2 -> 9, 3 -> 16. So to have range(0,max_l_index) =
↳ [0,1,...,max_l_index-1]

```

```

# 定义 p 轨道
p_orbitals = ['px', 'py', 'pz']
p_orbital_indices = [orbital_values[o] for o in p_orbitals]

# 定义 d 轨道
d_orbitals = ['dxy', 'dyz', 'dz2', 'dxz', 'dx2_y2']
d_orbital_indices = [orbital_values[o] for o in d_orbitals]

# 定义轨道的形状和颜色 (空心)
orbital_markers = {
    'dxy': ('s', 'black'),      # 黑色正方形 (空心)
    'dxz': ('o', 'red'),        # 红色圆形 (空心)
    'dyz': ('^', 'green'),      # 绿色正三角形 (空心)
    'dz2': ('v', 'blue'),       # 蓝色倒三角形 (空心)
    'dx2_y2': ('D', 'cyan'),    # 天蓝色菱形 (空心)
    'px': ('o', 'red'),         # 红色圆形 (空心)
    'py': ('s', 'green'),       # 绿色正方形 (空心)
    'pz': ('^', 'blue')        # 蓝色正三角形 (空心)
}

# 根据轨道类型选择要绘制的轨道
if args.orbital_type == 'd':
    orbitals_to_plot = d_orbitals
    orbital_indices = d_orbital_indices
elif args.orbital_type == 'p':
    orbitals_to_plot = p_orbitals
    orbital_indices = p_orbital_indices
else:
    orbitals_to_plot = []
    orbital_indices = []

# 只有当需要绘制投影能带时才计算投影贡献
if draw_projected_bands:
    # calculates contributions for bands and DOS projections
    el_orbs = []
    el_orbs_labels = []
    for component in args.project:      # either 2 or 3 components, e.g. 'N.s.pz', 'N.s+0.s.pz', 'N.s',
        ↪ 'N+0', 'X', 'X.s', 'X.s+N.px.py.pz', ...
        element_components = component.split("+") # split elements, e.g. 'N.s+0.s.pz' becomes ['N.s',
        ↪ '0.s.pz']
        color_component = []
        label_text = []
        for element_component in element_components:
            splits = element_component.split(".") # e.g. 'N.s.pz' converted to ['N', 's', 'pz']
            element = splits[0]                  # first element is the atom symbol
            if len(splits) == 1:                  # if no orbital is specified (e.g. 'N'), then plot all
                ↪ orbitals = 'all'
                orbitals = 'all'
            else:
                orbitals = splits[1:]            # e.g. ['s', 'pz']
            color_component.append([element, orbitals])

        # write text for legend
        if element == 'X':                      # 'X' represents all atoms
            if len(splits) == 1:
                label_text.append('s+p_x+p_y+p_z') # this makes no sense, it's the projection of all
                ↪ atoms and orbitals, put who am I to judge
            else:
                label_text.append("+".join(orbitals).replace('p', 'p_')) # e.g. 'X.s.pz' becomes 's+p_z'
        else:
            if len(splits) == 1:                  # all orbitals, only the element: 'N'
                label_text.append(element)
            else:
                label_text.append(f"{element}({'.'.join(orbitals).replace('p', 'p_')})") # e.g. '0.s.pz'
                ↪ becomes '0(s+p_z)'
        el_orbs.append(color_component)
        el_orbs_labels.append("+".join(label_text)) # e.g. '0.s.pz+H.s' becomes '0(s+p_z)+H(s)'

```

```

color_values = { 0: 'red', 1: 'green', 2: 'blue' }
print('projections:')
for color_idx, color_contrib in enumerate(el_orbs_labels):
    print(f'color {color_values[color_idx]}: {color_contrib}')

# contributions to the band per each band, k-point, and color: contrib_bands[spin][band][k-point][color]
contrib_bands = {
    Spin.up: np.zeros((bands.nb_bands, len(bands.kpoints), 3))
}
if data_down is not None:
    contrib_bands[Spin.down] = np.zeros((bands.nb_bands, len(bands.kpoints), 3))

# 检测自旋极化 DOS 计算
has_spin_down_dos = (Spin.down in dosrun.pdos[0][Orbital.s]) # 检测是否存在自旋向下分量

# 初始化 contrib_dos, 分别存储自旋向上和自旋向下 (如果存在)
contrib_dos = {
    Spin.up: np.zeros((len(el_orbs), len(dosrun.tdos.energies))),
}
if has_spin_down_dos:
    contrib_dos[Spin.down] = np.zeros((len(el_orbs), len(dosrun.tdos.energies)))
else:
    contrib_dos[Spin.down] = None

# 添加进度条 - 计算投影贡献
print("\nCalculating projections:")
for spin in [Spin.up, Spin.down]:
    if spin not in contrib_bands:
        continue

    # 创建进度条
    spin_name = "up" if spin == Spin.up else "down"
    band_range = tqdm(range(bands.nb_bands),
                        desc=f"Processing {spin_name} bands",
                        total=bands.nb_bands,
                        leave=True)

    # sum over all bands
    for b in band_range:
        # sum over all k-points
        for k in range(len(bands.kpoints)):
            for color_idx, color_contrib in enumerate(el_orbs): # color_idx: up to either 2 or 3.
                ↪ color_contrib: list of [elements, orbitals] for each color
                for element_contrib in color_contrib:
                    element = element_contrib[0] # e.g. 'X', 'N'
                    orbitals = element_contrib[1] # e.g. 'all', 's', ['s', 'pz']
                    if element == 'X': # if all atoms, get all indexes
                        element_indexes = range(len(atom_labels))
                    else: # else, get the indexes with label = atom symbol
                        ↪ (e.g. 'C')
                        element_indexes = [i for i, x in enumerate(atom_labels) if x == element]

                    if orbitals == 'all':
                        orbital_indexes = range(0, max_l_index) # sum all orbitals if 'all'
                    else:
                        orbital_indexes = [orbital_values[o] for o in orbitals]

                    for i in element_indexes:
                        for j in orbital_indexes:
                            if spin == Spin.up:
                                contrib_bands[spin][b, k, color_idx] += data_up[k][b][i][j]**2
                            else: # spin down
                                contrib_bands[spin][b, k, color_idx] += data_down[k][b][i][j]**2

                # Only need to add to DOS once (using spin up)
                if spin == Spin.up and k == 0 and b == 0:

```

```

        contrib_dos[Spin.up][color_idx] +=
        ↪ np.array(dosrun.pdos[i][Orbital(j)][Spin.up])
    if has_spin_down_dos:
        contrib_dos[Spin.down][color_idx] +=
        ↪ np.array(dosrun.pdos[i][Orbital(j)][Spin.down])

# normalization
if args.normalization == 'selection':
    if np.sum(contrib_bands[spin][b,k,:]) != 0:
        contrib_bands[spin][b,k,:] =
        ↪ contrib_bands[spin][b,k,:]/np.sum(contrib_bands[spin][b,k,:])
elif args.normalization == 'all':
    tot = 0.0
    for i in range(len(atom_labels)):
        for j in range(0,max_l_index):
            if spin == Spin.up:
                tot += data_up[k][b][i][j]**2
            else:
                tot += data_down[k][b][i][j]**2
    if tot != 0:
        contrib_bands[spin][b,k,:] = contrib_bands[spin][b,k,:]/tot

# 添加进度条 - 计算轨道权重
weights = {}
if args.element:
    print(f"\nCalculating {args.orbital_type}-orbital weights for {args.element}:")
    weights = {
        Spin.up: {orb: np.zeros((bands.nb_bands, len(bands.kpoints))) for orb in orbitals_to_plot},
        Spin.down: {orb: np.zeros((bands.nb_bands, len(bands.kpoints))) for orb in orbitals_to_plot}
    }

# 获取特定元素的原子索引
element_indexes = [i for i, x in enumerate(atom_labels) if x == args.element]
if not element_indexes:
    raise ValueError(f"No atoms of element {args.element} found in the structure")

for spin in [Spin.up, Spin.down]:
    if spin == Spin.down and data_down is None:
        continue

    # 创建进度条
    spin_name = "up" if spin == Spin.up else "down"
    band_range = tqdm(range(bands.nb_bands),
                        desc=f"Processing {spin_name} weights",
                        total=bands.nb_bands,
                        leave=True)

    for b in band_range:
        for k in range(len(bands.kpoints)):
            # 计算该元素所有指定轨道的总贡献
            total = 0.0
            for i in element_indexes:
                for j in orbital_indices:
                    if spin == Spin.up:
                        total += data_up[k][b][i][j]**2
                    else:
                        total += data_down[k][b][i][j]**2

            # 计算每个轨道的权重
            for orb in orbitals_to_plot:
                j = orbital_values[orb]
                orb_value = 0.0
                for i in element_indexes:
                    if spin == Spin.up:
                        orb_value += data_up[k][b][i][j]**2
                    else:
                        orb_value += data_down[k][b][i][j]**2

```

```

        if total > 1e-6: # 避免除以零
            weight = orb_value / total
        else:
            weight = 0.0

        weights[spin][orb][b, k] = weight

reciprocal = bands.lattice_rec.matrix/(2*math.pi)

# unchanged from Kevin Waters
# Empty lists used for caculating the distances between K-Points
KPOINTS = [0.0]
DIST = 0.0
# Create list with distances between Kpoints (Individual), corrects the spacing
for k in range(len(bands.kpoints)-1):
    Dist = np.subtract(bands.kpoints[k+1].frac_coords, bands.kpoints[k].frac_coords)
    DIST += np.linalg.norm(np.dot(reciprocal, Dist))
    KPOINTS.append(DIST)

# 只有当需要绘制投影能带时才绘制彩色能带
if draw_projected_bands:
    print("Plotting projected bands...")
    for spin in [Spin.up, Spin.down]:
        if spin not in contrib_bands:
            continue

        for b in range(bands.nb_bands):
            rgbline(ax1,
                    range(len(bands.kpoints)),
                    [e - bands.efermi for e in bands.bands[spin][b]],
                    contrib_bands[spin][b,:,0],
                    contrib_bands[spin][b,:,1],
                    contrib_bands[spin][b,:,2] if len(args.project) == 3 else np.zeros(len(bands.kpoints)),
                    ↪ # 如果只有两个颜色, 第三个分量设为零
                    KPOINTS=KPOINTS)
else:
    # 绘制简单的能带图 (灰色)
    print("Plotting simple bands...")
    for spin in [Spin.up, Spin.down]:
        if spin not in bands.bands:
            continue

        for b in range(bands.nb_bands):
            ax1.plot(KPOINTS, [e - bands.efermi for e in bands.bands[spin][b]],
                    color='gray', alpha=0.5, linewidth=0.8)

# 优化后的轨道气泡图绘制 (空心气泡)
if args.element:
    print(f"Plotting {args.orbital_type}-orbital bubbles...")
    # 计算气泡大小范围
    min_size = args.bubble_min * args.bubble_scale
    max_size = args.bubble_max * args.bubble_scale

    # 为每个轨道绘制气泡 (空心)
    for spin in [Spin.up, Spin.down]:
        if spin not in weights:
            continue

        for orb in orbitals_to_plot:
            marker, color = orbital_markers[orb]
            # 收集所有点一次性绘制
            x_vals = []
            y_vals = []
            sizes = []

            # 使用步长减少点数
            k_step = args.bubble_density
            b_step = args.bubble_density

```

```

# 创建进度条
total_points = (bands.nb_bands // b_step) * (len(bands.kpoints) // k_step)
if total_points > 0:
    progress = tqdm(total=total_points,
                    desc=f"{orb} bubbles",
                    unit="point")
else:
    progress = None

for b in range(0, bands.nb_bands, b_step):
    for k in range(0, len(bands.kpoints), k_step):
        weight = weights[spin][orb][b, k]

        # 只绘制权重大于阈值的点
        if weight > args.bubble_threshold:
            # 计算气泡大小 (与权重成正比)
            size = min_size + weight * (max_size - min_size)

            # 获取能量值
            energy = bands.bands[spin][b][k] - bands.efermi

            # 添加点
            x_vals.append(KPOINTS[k])
            y_vals.append(energy)
            sizes.append(size)

        if progress:
            progress.update(1)

# 一次性绘制该轨道的所有点 (空心)
if x_vals: # 确保列表不为空
    ax1.scatter(
        x_vals,
        y_vals,
        s=sizes,
        marker=marker,
        facecolors='none', # 空心
        edgecolors=color, # 边框颜色
        linewidths=0.4, # 边框线宽
        alpha=0.4, # 稍微降低透明度
        zorder=10 # 确保气泡在能带上方
    )

if progress:
    progress.close()

# 创建图例 (只包含轨道气泡)
legend_handles = []
for orb in orbitals_to_plot:
    marker, color = orbital_markers[orb]
    legend_handles.append(
        plt.Line2D([0], [0],
                    marker=marker,
                    color=color,
                    markerfacecolor='none', # 空心
                    markeredgecolor=color,
                    markersize=8,
                    linestyle='None',
                    label=f"{orb}")
    )

# 在右侧的 ax2 上绘制图例 (DOS 位置)
ax2.clear() # 清除原有内容
ax2.axis('off') # 关闭坐标轴

# 创建图例并放置在右侧中央位置
ax2.legend(handles=legend_handles,
           loc='center', # 中心位置
           bbox_to_anchor=(0.57, 0.81), # 精确中心位置

```

```

        fancybox=False,
        shadow=False,
        prop={'size': args.font_size},
        framealpha=0.8,
        title=f"{args.element} {args.orbital_type}-orbitals",
        title_fontsize=args.font_size)

# 调整标题位置
ax1.set_title(f"{args.element} {args.orbital_type}-orbitals",
              fontsize=args.font_size, pad=10, y=1.05)
# 调整右侧子图位置, 使图例更靠近能带图
plt.subplots_adjust(right=0.85) # 增加右侧空白区域

# style
ax1.set_ylabel(r"$E - E_f$ (eV)", labelpad=-2) #labelpad might work bad
ax1.grid()

# fermi level line at 0
ax1.hlines(y=0, xmin=0, xmax=len(bands.kpoints), color="k", lw=2)

TICKS = [0.0]
for i in range(step, len(KPOINTS)+step, step):
    ax1.vlines(KPOINTS[i-1], emin, emax, "k")
    TICKS.append(KPOINTS[i-1])
ax1.set_xticks(TICKS)
ax1.set_xticklabels(labels)
ax1.tick_params(axis='x', which='both', length=0, pad=5)
ax1.set_xlim(0, KPOINTS[-1])

# 只有当需要绘制投影能带时才绘制 DOS
if draw_projected_bands:
    print("Plotting DOS...")
    # 绘制总 DOS
    if has_spin_down_dos:
        # 自旋向上 (正方向)
        ax2.plot(dosrun.tdos.densities[Spin.up], dosrun.tdos.energies - dosrun.efermi,
                 color=(0.6, 0.6, 0.6), label="total up", lw=1)
        # 自旋向下 (负方向)
        ax2.plot(-dosrun.tdos.densities[Spin.down], dosrun.tdos.energies - dosrun.efermi,
                 color=(0.6, 0.6, 0.6), label="total down", lw=1)
        # 填充区域
        ax2.fill_betweenx(dosrun.tdos.energies - dosrun.efermi,
                          0, dosrun.tdos.densities[Spin.up],
                          color=(0.7, 0.7, 0.7), facecolor=(0.7, 0.7, 0.7))
        ax2.fill_betweenx(dosrun.tdos.energies - dosrun.efermi,
                          0, -dosrun.tdos.densities[Spin.down],
                          color=(0.7, 0.7, 0.7), facecolor=(0.7, 0.7, 0.7))
        # 计算最大 DOS 值 (考虑两个自旋方向)
        maxdos = max(np.max(dosrun.tdos.densities[Spin.up]),
                     np.max(dosrun.tdos.densities[Spin.down])) / scale
    else:
        # 非自旋极化计算
        ax2.plot(dosrun.tdos.densities[Spin.up], dosrun.tdos.energies - dosrun.efermi,
                 color=(0.6, 0.6, 0.6), label="total", lw=1)
        ax2.fill_betweenx(dosrun.tdos.energies - dosrun.efermi,
                          0, dosrun.tdos.densities[Spin.up],
                          color=(0.7, 0.7, 0.7), facecolor=(0.7, 0.7, 0.7))
        maxdos = np.max(dosrun.tdos.densities[Spin.up]) / scale

# 绘制投影 DOS (自旋向上)
ax2.plot(contrib_dos[Spin.up][0], dosrun.tdos.energies - dosrun.efermi,
         c=(1.0, 0.0, 0.0), label=f'${el_orbs_labels[0]}$', linewidth=1)
ax2.plot(contrib_dos[Spin.up][1], dosrun.tdos.energies - dosrun.efermi,
         c=(0.0, 1.0, 0.0), label=f'${el_orbs_labels[1]}$', linewidth=1)

# 如果有自旋向下分量, 绘制负方向的投影
if has_spin_down_dos:
    ax2.plot(-contrib_dos[Spin.down][0], dosrun.tdos.energies - dosrun.efermi,

```



```

        c=(1.0, 0.0, 0.0), linewidth=1)
    ax2.plot(-contrib_dos[Spin.down][1], dosrun.tdos.energies - dosrun.efermi,
            c=(0.0, 1.0, 0.0), linewidth=1)

# 如果有第三个颜色分量
if len(el_orbs) == 3:
    ax2.plot(contrib_dos[Spin.up][2], dosrun.tdos.energies - dosrun.efermi,
            c=(0.0, 0.0, 1.0), label=f'${el_orbs_labels[2]}$', linewidth=1)
    if has_spin_down_dos:
        ax2.plot(-contrib_dos[Spin.down][2], dosrun.tdos.energies - dosrun.efermi,
            c=(0.0, 0.0, 1.0), linewidth=1)

# 调整 x 轴范围
if has_spin_down_dos:
    ax2.set_xlim(-maxdos, maxdos)
else:
    ax2.set_xlim(0, maxdos)

# 添加 y=0 的水平线
ax2.hlines(y=0, xmin=ax2.get_xlim()[0], xmax=ax2.get_xlim()[1], color="k", lw=2)

# 添加 x=0 的垂直线 (仅当有自旋向下分量时)
if has_spin_down_dos:
    ax2.vlines(x=0, ymin=emin, ymax=emax, color="k", lw=1, linestyle='--', alpha=0.5)

ax2.set_yticklabels([])
ax2.grid()
ax2.set_xticks([])
ax2.set_xlabel("DOS")
ax2.legend(fancybox=False, shadow=False, prop={'size': args.font_size-1},
        labelspace=0.15, borderpad=0.20, handlelength=1.2,
        framealpha=0.6, loc='upper right')

# Plotting
# -----
print("Saving plot...")
plt.savefig(f"{args.output_file}.{args.format}", format=args.format, bbox_inches='tight')
print("Done!")

```

► 代码清单 A.2.3 (python, 70 行)

```

usage: plot_fatbands.py [-h] [-b VASPRUN_FILE_BANDS] [-K KPOINTS_FILE] [-c POSCAR_FILE] [-O PROCAR_FILE] [-w
↪ POTCAR_FILE] [-d VASPRUN_FILE_DOS] [-p PROJECT [PROJECT ...]]
    [-n {all,selection}] [-l {1,2,3}] [-e EMIN] [-E EMAX] [-s SCALE] [-H HEIGHT] [-W WIDTH]
    ↪ [-r RATIO] [-f FONT_SIZE] [-o OUTPUT_FILE] [--format {pdf,png}]
    [--element ELEMENT] [--orbital-type {d,p,f}] [--bubble-min BUBBLE_MIN] [--bubble-max
↪ BUBBLE_MAX] [--bubble-scale BUBBLE_SCALE] [--bubble-density BUBBLE_DENSITY]
    [--bubble-threshold BUBBLE_THRESHOLD]

```

Plot projected band structure (fatbands) from a VASP calculation.

Author: Marco Cappelletti. Heavily inspired by Kevin Waters (kwaters4.github.io) and sumo-bandplot.

By default it assumes that:

- the current directory contains KPOINTS, vasprun.xml from the band calculations
- the directory ../dos contains vasprun.xml from the dos calculation
- the parent directory (../) contains the POSCAR file
- the parent parent directory (../../) contains the POTCAR file

optional arguments:

- h, --help show this help message and exit
- b VASPRUN_FILE_BANDS, --vasprun-file-bands VASPRUN_FILE_BANDS
Path of the vasprun.xml file of the band calculation (default: vasprun.xml)
- K KPOINTS_FILE, --KPOINTS-file KPOINTS_FILE
Path of the KPOINTS file with the band path (default: KPOINTS)

```

-c POSCAR_FILE, --POSCAR-file POSCAR_FILE
    Path of the POSCAR file (default: ../POSCAR)
-O PROCAR_FILE, --PROCAR-file PROCAR_FILE
    Path of the PROCAR file from the band calculation (default: PROCAR)
-w POTCAR_FILE, --POTCAR-file POTCAR_FILE
    Path of the POTCAR file (default: .././POTCAR)
-d VASPRUN_FILE_DOS, --vasprun-file-dos VASPRUN_FILE_DOS
    Path of the vasprun.xml file of the dos calculation (default: ../dos/vasprun.xml)
-p PROJECT [PROJECT ...], --project PROJECT [PROJECT ...]
    Band projection to rgb. Either 2 (red and green) or 3 arguments (red, green, blue).
    ↪ Nomenclature:
    - E: all orbitals of element E (H, C, N, O, ...)
    - E.o: o-orbital of element E (s, px, py, pz, dxy, ...)
    - E.s.pz: s+pz orbitals of element E
    - X.s: s orbitals of all elements
    - O.s.pz+N.pz: sum of O(s,pz) and N(pz)
    (default: ['X.px', 'X.py', 'X.s.pz'])
-n {all,selection}, --normalization {all,selection}
    Normalization of the projection.
    - 'all': with respect to all contributions.
    - 'selection': with respect to selection only
    (default: selection)
-l {1,2,3}, --max-l {1,2,3}
    Maximum value of l (angular momentum) for the projection. Increases computational costs,
    ↪ so increase it only if necessary. (default: 1)
-e EMIN, --emin EMIN Minimum of energy in the plot. If none, it chooses the lower limit (default: None)
-E EMAX, --emax EMAX Maximum of energy in the plot. If none, it chooses the upper limit (default: None)
-s SCALE, --scale SCALE
    DOS scale factor (default: 2.0)
-H HEIGHT, --height HEIGHT
    Height of the plot in inches (default: 3.5)
-W WIDTH, --width WIDTH
    Width of the plot in inches (default: 3.3)
-r RATIO, --ratio RATIO
    Bandplot - dosplot width ratio (default: 3.0)
-f FONT_SIZE, --font-size FONT_SIZE
    Fontsize (default: 8)
-o OUTPUT_FILE, --output-file OUTPUT_FILE
    Path and name of the output file, excluding the format (default: fatbands)
--format {pdf,png} Output file format (default: pdf)
--element ELEMENT Element for orbital bubble plot (e.g., Fe) (default: None)
--orbital-type {d,p,f}
    Type of orbital to plot (d, p or f) (default: d)
--bubble-min BUBBLE_MIN
    Minimum bubble size (default: 0.05)
--bubble-max BUBBLE_MAX
    Maximum bubble size (default: 0.3)
--bubble-scale BUBBLE_SCALE
    Bubble size scaling factor (default: 100)
--bubble-density BUBBLE_DENSITY
    Bubble density (1: every point, 2: every second point, etc.) (default: 5)
--bubble-threshold BUBBLE_THRESHOLD
    Minimum weight to plot bubble (default: 0.05)

```

► 代码清单 A.2.4 (python, 651 行)

```

import numpy as np
import math
import pymatgen
import sys
import argparse
import logging
import os.path

```

```

from pymatgen.io.vasp.outputs import Procar, Vasprun
from pymatgen.io.vasp.inputs import Kpoints
from pymatgen.core import Structure
from pymatgen.electronic_structure.core import Spin, Orbital

import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
from matplotlib.gridspec import GridSpec

class SaneFormatter(argparse.RawTextHelpFormatter,
                    argparse.ArgumentDefaultsHelpFormatter):
    pass

parser = argparse.ArgumentParser(prog='plot_fatbands.py',description=("Plot projected band structure (fatbands)
↳ from a VASP calculation.\n" +
    'Author: Marco Cappelletti. Inspired by Kevin Waters (kwaters4.github.io) and sumo-bandplot.\n'
    ↳ +
    'By default it assumes that:\n' +
    '\tthe current directory contains KPOINTS and vasprun.xml from the band structure calculation\n'
    ↳ +
    '\tthe directory ../dos contains vasprun.xml from the DOS calculation\n' +
    '\tthe parent directory (../) contains the POSCAR file\n' +
    '\tthe grandparent directory (../..) contains the POTCAR file\n')
    ,formatter_class=SaneFormatter)
parser.add_argument('-B','--vasprun-file-bands', type=str, help='Path of the vasprun.xml file of the band
↳ calculation', default='vasprun.xml')
parser.add_argument('-K','--KPOINTS-file', type=str, help='Path of the KPOINTS file with the band path',
↳ default='KPOINTS')
parser.add_argument('-C','--POSCAR-file', type=str, help='Path of the POSCAR file', default='POSCAR')
parser.add_argument('-O','--PROCAR-file', type=str, help='Path of the PROCAR file from the band calculation',
↳ default='PROCAR')
parser.add_argument('-P','--POTCAR-file', type=str, help='Path of the POTCAR file', default='POTCAR')
parser.add_argument('-D','--vasprun-file-dos', type=str, help='Path of the vasprun.xml file of the dos
↳ calculation', default='../dos/vasprun.xml')
parser.add_argument('-p','--project', help='Band projections. Accepts 1-5 arguments. Nomenclature:\n'
    '\t- E/n: all orbitals of element with symbol E (H, C, N, ...) or atom index n (1, 2,
    ↳ ...) \n'
    '\t- E/n.o: o-orbital of element with symbol E or atom index n
    ↳ (s, px, py, pz, dxy, ...) \n'
    '\t- E/n.s.pz: s+pz orbitals of element with symbol E or atom
    ↳ index n \n'
    '\t- X.s: s orbitals of all elements (literally X, see
    ↳ example) \n'
    '\t- O.s.pz+N.pz: sum of O(s,pz) and N(pz) \n'
    'You are free to mix anything you want. There is no
    ↳ restriction. \n'
    'Examples (a number represents an atom index, starting from 1,
    ↳ as in PROCAR): \n'
    '\t-p N O.s.pz+H.s C+X.pz =>'
    ' color 1: N(all orb.), color 2: O(s+pz)+H(s), color 2: C(all
    ↳ orb.)+(all atoms pz) \n'
    '\t-p 1+2+3 4.px.py.pz+5 H.s =>'
    ' color 1: 1(all orb.)+2(all orb.)+3(all orb.), color 2:
    ↳ 4(px+py+pz)+5(px), color 3: H(s) \n'
    , nargs='+', default=['X.px', 'X.py', 'X.s.pz'])
parser.add_argument('-t','--projection-type', type=str, help='Type of projection. \n'
    '\t- blend: plot projections into the additive (r,g,b) space,
    ↳ or into the subtractive (c,m,y) space. \n'
    '\t- stack: plot stacked colored circles, with radius
    ↳ depending on contribution. \n'
    , choices=['blend','stack'], default='stack')
parser.add_argument('--split', help='Split projections in different plots.', action='store_true')
parser.add_argument('-n','--normalization', type=str, help='Normalization of the projection. \n'
    '\t- all: with respect to all contributions. \n'
    '\t- selection: with respect to selection only. \n'
    ↳ , choices=['all','selection'],
    ↳ default='selection')

```

```

parser.add_argument('-l', '--max-l', type=int, default=1, choices=[1, 2, 3],
                    help='Maximum value of l (angular momentum) for the projection.\n'
                          'Increases computational costs, so increase it only if necessary.')
parser.add_argument('-N', '--no-projection', help='Do not perform any projection', action='store_true')
parser.add_argument('-m', '--emin', type=float, help='Minimum of energy in the plot. If none, it chooses the lower\n'
↳ limit', default=None)
parser.add_argument('-M', '--emax', type=float, help='Maximum of energy in the plot. If none, it chooses the upper\n'
↳ limit', default=None)
parser.add_argument('-s', '--scale', type=float, help='DOS scale factor', default=1.0)
parser.add_argument('-H', '--height', type=float, help='Height of the plot in inches', default=3.5)
parser.add_argument('-W', '--width', type=float, help='Width of the plot in inches', default=3.3)
parser.add_argument('-r', '--ratio', type=float, help='Bandplot - dosplot width ratio', default=3.0)
parser.add_argument('--blw', '--band-lw', type=float, help='Linewidth of non-projected bands', default=2.0)
parser.add_argument('--bs', '--band-size', type=float, help='Circle size of bands, when projection is on.',
↳ default=2.0)
parser.add_argument('--ba', '--band-alpha', type=float, help='alpha value (transparency) of bands', default=1.0)
parser.add_argument('--dlw', '--dos-lw', type=float, help='Linewidth of DOS', default=1.25)
parser.add_argument('--flw', '--Fermi-lw', type=float, help='Linewidth of Fermi level. Set it to 0 to remove it',
↳ default=1.0)
parser.add_argument('--vlw', '--vlines-lw', type=float, help='Linewidth of vertical lines. Set it to 0 to remove\n'
↳ them', default=1.0)
parser.add_argument('--vla', '--vlines-alpha', type=float, help='alpha value (transparency) of vertical lines.',
↳ default=1.0)
parser.add_argument('--glw', '--grid-lw', type=float, help='Linewidth of grid. Set it to 0 to remove them',
↳ default=1.0)
parser.add_argument('--gla', '--grid-alpha', type=float, help='alpha value (transparency) of grid lines.',
↳ default=1.0)
parser.add_argument('-f', '--font-size', type=float, help='Fontsize', default=7)
parser.add_argument('--c-mode', '--color-mode', type=str,
                    help='Color mode for blended fatbands. rgb (additive) = red, green, blue; cm\n'
↳ (subtractive) = cyan, magenta, yellow.',
                    choices=['rgb', 'cmy'], default='rgb')
parser.add_argument('--cmap', '--colormap', type=str,
                    help='Color scheme for stacked fatbands.',
                    choices=['rgb', 'saturated', 'normal', 'pale', 'dark', 'alt', 'darkalt', 'accent', 'plt'\n'
↳ , 'sumo'], default='rgb')
parser.add_argument('--cord', '--color-order', nargs='+', type=int, help='If you like to change the color order.\n'
                    'E.g. --cmap rgbop --cord 3 2 1 4 5 means blue,\n'
↳ green, red, orange, purple.\n'
                    'Note: list of 5 integers regardless the number of\n'
↳ projections! And first color is 1!!',
                    default=[1, 2, 3, 4, 5])
parser.add_argument('--custc', '--custom-colors', nargs='+', help='Custom colors, as in matplotlib. Examples:\n'
                    '\t--custc red green blue\n'\n'
↳ \t--custc brown lime indigo', default='None')
parser.add_argument('-o', '--output-file', type=str, help='Path and name of the output file, excluding the format',
↳ default='fatbands')
parser.add_argument('--format', type=str, help='Output file format', choices=['pdf', 'png'], default='pdf')
parser.add_argument('--redo', '--readlog', help='Rerun the last command, if plot_fatbands.log file is present.\n'
↳ This overrides every other argument!', action='store_true')

args = parser.parse_args()
redo = args.redo

if redo is True:
    if os.path.isfile(f'plot_{args.output_file}.log') is False:
        raise ValueError(f'I cannot rerun: plot_fatbands_{args.output_file}.log does not exist')

    with open(f'plot_{args.output_file}.log') as f:
        argstr = f.readline()

    args = parser.parse_args(argstr.split()[1:])

logging.basicConfig(
    filename=f'plot_{args.output_file}.log',
    level=logging.INFO,
    filemode="w",

```

```

        format="% (message)s",
    )

    if redo is True:
        logging.info(argstr)
    else:
        logging.info(" ".join(sys.argv[1:]))

    scale = args.scale
    no_proj = args.no_projection
    N_proj = len(args.project)

    # Set colors -----

    if args.custc == 'None':

        match args.cmap:
            case 'rgb':
                colors = [[1,0,0], [0,1,0], [0,0,1], [1, 0.50, 0.14], [0.58,0.40,0.74]]
                color_names = {0: 'red', 1: 'green', 2: 'blue', 3: 'orange', 4: 'purple'}
            case 'saturated':
                colors = [mcolors.BASE_COLORS['r'], mcolors.BASE_COLORS['g'], mcolors.BASE_COLORS['b'],
                        mcolors.TABLEAU_COLORS['tab:orange'], mcolors.BASE_COLORS['m']]
                color_names = {0: 'red', 1: 'green', 2: 'blue', 3: 'orange', 4: 'magenta'}
            case 'pale':
                colors = [mcolors.TABLEAU_COLORS['tab:red'], mcolors.TABLEAU_COLORS['tab:green'],
                        ↪ mcolors.TABLEAU_COLORS['tab:blue'],
                        mcolors.TABLEAU_COLORS['tab:orange'], mcolors.TABLEAU_COLORS['tab:purple']]
                color_names = {0: 'red', 1: 'green', 2: 'blue', 3: 'orange', 4: 'purple'}
            case 'normal':
                colors = [plt.cm.Set1.colors[0], plt.cm.Set1.colors[2], plt.cm.Set1.colors[1], plt.cm.Set1.colors[4],
                        ↪ plt.cm.Set1.colors[3]]
                color_names = {0: 'red', 1: 'green', 2: 'blue', 3: 'orange', 4: 'purple'}
            case 'dark':
                colors = [mcolors.CSS4_COLORS['darkred'], mcolors.CSS4_COLORS['darkgreen'],
                        ↪ mcolors.CSS4_COLORS['darkblue'],
                        mcolors.CSS4_COLORS['chocolate'], mcolors.CSS4_COLORS['purple']]
                color_names = {0: 'red', 1: 'green', 2: 'blue', 3: 'orange', 4: 'purple'}
            case 'alt':
                colors = [plt.cm.Set2.colors[i] for i in range(5)]
                color_names = {0: 'teal', 1: 'orange', 2: 'blue', 3: 'pink', 4: 'green'}
            case 'darkalt':
                colors = [plt.cm.Dark2.colors[i] for i in range(5)]
                color_names = {0: 'teal', 1: 'orange', 2: 'blue', 3: 'pink', 4: 'green'}
            case 'accent':
                colors = [plt.cm.Accent.colors[i] for i in range(5)]
                color_names = {0: 'green', 1: 'lilac', 2: 'orange', 3: 'yellow', 4: 'blue'}
            case 'plt':
                colors = [plt.cm.tab10.colors[i] for i in range(5)]
                color_names = {0: 'blue', 1: 'orange', 2: 'green', 3: 'red', 4: 'purple'}
            case 'sumo':
                colors = ["#3952A3", "#FAA41A", "#67BC47", "#6ECCDD", "#ED2025"]
                color_names = {0: 'blue', 1: 'orange', 2: 'green', 3: 'red', 4: 'purple'}

        if args.projection_type == 'blend':
            colors = np.array([[1,0,0], [0,1,0], [0,0,1]])
            color_names = {0: 'red', 1: 'green', 2: 'blue'}
            if args.c_mode == 'cmy':
                colors = 1 - colors
                color_names = {0: 'cyan', 1: 'magenta', 2: 'yellow'}

    else:
        colors = [ mcolors.CSS4_COLORS[cval] for cval in args.custc ]

    color_order = {0: args.cord[0]-1, 1: args.cord[1]-1, 2: args.cord[2]-1, 3: args.cord[3]-1, 4: args.cord[4]-1}

```

```

# -----

def CheckInput():
    # accepts only 1-3 entries for the projection
    if N_proj < 1 or N_proj > 3:
        if args.split is False:
            raise ValueError('Only 1, 2 or 3 components allowed for the non-split projection')
        elif N_proj < 2 or N_proj > 5:
            raise ValueError('2-5 components allowed for the split projection')
    elif N_proj == 1 and args.normalization == 'selection':
        print('\tWARNING: you selected 1 projection and \'selection\' normalization. This does not make sense.
        ↪ Normalization is changed to \'all\'')
        logging.info('WARNING: normalization is changed to \'all\'')
        args.normalization='all'

    if args.split is True and args.projection_type == 'blend':
        raise ValueError('Only stacked projection allowed for split')

    if args.split is True and no_proj is True:
        raise ValueError('Cannot split non-projected bands!')

def plotcircles(ax, KPOINTS, band_energies, contrib, index):

    if args.split is False and args.projection_type == 'stack':
        sizes = args.bs**2 * contrib[:, :, index:3].sum(axis=2)**2
    elif args.split is True:
        sizes = args.bs**2 * contrib**2

    for bidx, be in enumerate(band_energies):
        if args.projection_type == 'blend':
            ax.scatter(KPOINTS, be, s=args.bs/2, color=colors[color_order[index]], alpha=contrib[bidx, :, index]**2, edgecolor='none')
        elif args.projection_type == 'stack':
            ax.plot(KPOINTS, be, lw=args.blw/4, color='k', alpha=args.ba, zorder=0)
            ax.scatter(KPOINTS, be, s=sizes[bidx], color=colors[color_order[index]], edgecolor='none')

def CalculateProjectionsAndPlot():

    # calculates contributions for bands and DOS projections
    el_orbs = []
    el_orbs_labels = []

    # read the projections
    # 1-5 components, e.g. 'N.s.pz', 'N.s+0.s.pz', 'N+0.s', 'N+0', 'X', 'X.s', 'X.s+N.px.py.pz', ...
    for component in args.project:
        element_components = component.split("+") # split elements, e.g. 'N.s+0.s.pz' becomes ['N.s', '0.s.pz']
        color_component = [] # list of components for one color
        label_text = [] # more readable, user-friendly text for the legend
        for element_component in element_components: # e.g. for element in the list ['N.s', '0.s.pz'] (example
            ↪ above)
            splits = element_component.split(".") # e.g. 'N.s.pz' converted to ['N', 's', 'pz']
            element = splits[0] # first element is the atom symbol, e.g. 'N'
            if len(splits) == 1: # if splits has one element, then no orbitals are specified
                ↪ (e.g. 'N') => plot all orbitals (= 'all')
                orbitals = 'all'
            else:
                orbitals = splits[1:] # e.g. ['s', 'pz']
            color_component.append([element, orbitals])

```

```

# contributions to the band per each band, k-point, and color: contrib_bands[band][k-point][color]
contrib_bands = np.zeros((max_band_to_plot-min_band_to_plot+1, len(bands.kpoints), N_proj))

# contributions to the DOS per each color: contrib_dos[color][energy]
contrib_dos = np.zeros((N_proj, len(dosrun.pdos[0][Orbital.s][Spin.up])))

# obtain contributions
# sum over all bands to plot
for b in range(max_band_to_plot-min_band_to_plot+1):
    # sum over all k-points
    for k in range(len(bands.kpoints)):
        for color_idx, color_contrib in enumerate(el_orbs): # color_idx: 1-5. color_contrib: list of
            ↪ [elements, orbitals] for each color
            for element_contrib in color_contrib:
                element = element_contrib[0] # e.g. 'X', 'N'
                orbitals = element_contrib[1] # e.g. 'all', 's', ['s', 'pz']
                if element == 'X': # if all atoms, get all indexes
                    element_indexes = range(len(atom_labels))
                elif element.isnumeric() is True:
                    element_indexes = [int(element)-1]
                else: # else, get the indexes with label = atom symbol (e.g.
                    ↪ 'C')
                    element_indexes = [i for i, x in enumerate(atom_labels) if x == element]

                if orbitals == 'all':
                    orbital_indexes = range(0, max_l_index) # sum all orbitals if 'all'
                else:
                    orbital_indexes = [orbital_values[o] for o in orbitals if o in orbital_values]

                for i in element_indexes:
                    for j in orbital_indexes:
                        # 更安全的边界检查 - 关键修改
                        try:
                            # 先检查所有索引是否在有效范围内
                            if (i < len(data[Spin.up][k][b+min_band_to_plot]) and
                                j < len(data[Spin.up][k][b+min_band_to_plot][i])):
                                contrib_bands[b,k,color_idx] += data[Spin.up][k][b+min_band_to_plot][i][j]**2

                        if k == 0 and b == 0: # needs to be done just once
                            try:
                                # DOS 部分也添加边界检查
                                if (i < len(dosrun.pdos) and
                                    j < len(dosrun.pdos[i])):
                                    contrib_dos[color_idx] +=
                                        ↪ np.array(dosrun.pdos[i][Orbital(j)][Spin.up])
                            except (IndexError, KeyError):
                                pass # 如果出错就跳过
                        except (IndexError, KeyError):
                            pass # 如果任何索引访问出错, 跳过

# normalization
if args.normalization == 'selection':
    if np.sum(contrib_bands[b,k,:]) != 0:
        contrib_bands[b,k,:] = contrib_bands[b,k,:]/np.sum(contrib_bands[b,k,:])
    elif k in labels_kpt_num+1: # I don't know why, VASP prints all contribution to zero at those
        ↪ points
        contrib_bands[b,k,:] = contrib_bands[b,k-1,:]

# print contributions at high-symmetry point to log file
elif args.normalization == 'all':
    tot = 0.0
    for i in range(len(atom_labels)):
        for j in range(0, min(max_l_index, 9)): # 限制最大为 9 个轨道
            try:
                if (i < len(data[Spin.up][k][b+min_band_to_plot]) and
                    j < len(data[Spin.up][k][b+min_band_to_plot][i])):
                    tot += data[Spin.up][k][b+min_band_to_plot][i][j]**2

```

```

        except (IndexError, KeyError):
            pass
        if tot != 0:
            contrib_bands[b,k,:] = contrib_bands[b,k,:]/tot
    if b+min_band_to_plot < 9:
        b_str = ' ' + str(b+min_band_to_plot+1)
    elif b+min_band_to_plot < 99:
        b_str = ' ' + str(b+min_band_to_plot+1)
    else:
        b_str = str(b+min_band_to_plot+1)
    logging.info(f'{b_str}: ' + '\t'
        + '\t'.join('[' + ' ', '.join(f'{x:.2f}' for x in contrib_bands[b, lkn, :])
        + ']' for lkn in labels_kpt_num))

# Plotting -----

bands_to_plot = bands.bands[Spin.up][min_band_to_plot:max_band_to_plot+1] - bands.efermi

if args.split is False:
    if args.projection_type == 'stack':
        for proj_n in range(N_proj):
            plotcircles(ax_bands,
                KPOINTS,
                bands_to_plot,
                contrib_bands,
                proj_n)

            ax_DOS.plot(contrib_dos[proj_n], dosrun.tdos.energies - dosrun.efermi, \
                c=colors[color_order[proj_n]], label = f'${el_orbs_labels[proj_n]}$', linewidth = args.dlw)
    elif args.projection_type == 'blend':
        for proj_n in range(N_proj):
            plotcircles(ax_bands,
                KPOINTS,
                bands_to_plot,
                contrib_bands,
                proj_n)

            ax_DOS.plot(contrib_dos[0], dosrun.tdos.energies - dosrun.efermi, \
                c=colors[color_order[0]], label = f'${el_orbs_labels[0]}$', linewidth = args.dlw)
            ax_DOS.plot(contrib_dos[1], dosrun.tdos.energies - dosrun.efermi, \
                c=colors[color_order[1]], label = f'${el_orbs_labels[1]}$', linewidth = args.dlw)
            if len(el_orbs) == 3:
                ax_DOS.plot(contrib_dos[2], dosrun.tdos.energies - dosrun.efermi, \
                    c=colors[color_order[2]], label = f'${el_orbs_labels[2]}$', linewidth = args.dlw)

else:
    for proj_n in range(N_proj):
        ax = ax_bands[proj_n]
        plotcircles(ax,
            KPOINTS,
            bands_to_plot,
            contrib_bands[:, :, proj_n],
            proj_n)

        # plot DOS
        ax_DOS.plot(contrib_dos[proj_n], dosrun.tdos.energies - dosrun.efermi, \
            c=colors[color_order[proj_n]], label = f'${el_orbs_labels[proj_n]}$', linewidth = args.dlw)

# -----

```

```

#----- Program starts -----

if __name__ == "__main__":
    print('--- plot_fatbands.py -----')
    print(f'\tPlotting fatbands into {args.output_file}.{args.format}. Size: {args.width} x {args.height}
    ↳ inches.')
    if redo is True:
        print(f'\tRedoing previous run: {argstr}',end='')
    print(f'\tAdditional data is printed in plot_{args.output_file}.log')
    logging.info(f'Plotting fatbands into {args.output_file}.{args.format}. Size: {args.width} x {args.height}
    ↳ inches.\n')

    CheckInput()

# Read files -----

# Load Structure
structure = Structure.from_file(args.POSCAR_file)
# 修改原子标签获取方式 - 兼容新版 pymatgen
try:
    atom_labels = [site.species_string for site in structure]
except AttributeError:
    atom_labels = structure.labels

# Load Band Structure Calculations
bands =
↳ Vasprun(args.vasprun_file_bands,parse_potcar_file=args.POTCAR_file).get_band_structure(args.KPOINTS_file,
↳ line_mode = True)
# Read KPOINTS file with path
kpts = Kpoints.from_file(args.KPOINTS_file)

if no_proj is False:
    # projected bands
    data = Procar(args.PROCAR_file).data

# density of states
dosrun = Vasprun(args.vasprun_file_dos,parse_potcar_file=args.POTCAR_file)

# -----

# Interpret k-points -----

# k-point labels
n_labels = len(kpts.labels)
labels = []
labels.append(kpts.labels[0])
for label_idx in range(1,n_labels,2):
    labels.append(kpts.labels[label_idx])

# get kpoint number for each high symmetry point
labels_kpt_num = np.zeros(len(labels), dtype=int)
lab_idx = 0
for idx, bkpt in enumerate(bands.kpoints):
    if bkpt.label is not None and idx-1 not in labels_kpt_num: # usually they are duplicated, I just take the
    ↳ first
        labels_kpt_num[lab_idx] = idx
        lab_idx += 1

# Number of points between kpoints, found in the KPOINTS file
step = kpts.num_kpts

# -----

# Initialize plot -----

```

```

# general options for plot
font = {'family': 'serif', 'size': args.font_size}
plt.rc('font', **font)

# if no split is requested: one band plot and one DOS plot
if args.split is False:
    # set up 2 graph with aspect ratio args.ratio/1
    # plot 1: bands diagram
    # plot 2: DOS
    gs = GridSpec(1, 2, width_ratios=[args.ratio,1], wspace=0.1)
    fig = plt.figure(figsize=(args.width, args.height))
    ax_bands = plt.subplot(gs[0])
    ax_DOS = plt.subplot(gs[1]) #, sharey=ax1)

# if split is requested: N band plots (N=number of projections, 1-3) and one DOS plot
else:
    print('\tWARNING: split is requested, remember to change plot width!')
    width_ratios = [ args.ratio/N_proj for proj_n in range(N_proj) ]
    width_ratios.append(args.ratio/(2*N_proj))
    gs = GridSpec(1, N_proj+1, width_ratios=width_ratios, wspace=0.1)
    fig = plt.figure(figsize=(args.width, args.height))
    ax_bands = [ plt.subplot(gs[proj_n]) for proj_n in range(N_proj) ]
    ax_DOS = plt.subplot(gs[N_proj])

# -----

# Set both fermi levels equal to the band fermi level
bands.efermi = dosrun.efermi #= 0

# set y limits for the plot
emin = args.emin
emax = args.emax

# if either is not defined, get the min and/or the max of energy from bands.bands.keys()
if emin is None and emax is None:
    emin=1000
    emax=-1000
    for spin in bands.bands.keys():
        for b in range(bands.nb_bands):
            emin = min(emin, min(bands.bands[spin][b]))
            emax = max(emax, max(bands.bands[spin][b]))
    emin = emin - bands.efermi
    emax = emax - bands.efermi
elif emin is None:
    emin=1000
    for spin in bands.bands.keys():
        for b in range(bands.nb_bands):
            emin = min(emin, min(bands.bands[spin][b]))
    emin = emin - bands.efermi
elif emax is None:
    emax=-1000
    for spin in bands.bands.keys():
        for b in range(bands.nb_bands):
            emax = max(emax, max(bands.bands[spin][b]))
    emax = emax - bands.efermi

# set first and last bands to plot, so that it calculates the projections only on those
min_band_to_plot = 1000
max_band_to_plot = -1

for spin in bands.bands.keys():
    min_band_to_plot = min(min_band_to_plot, np.min(np.where(bands.bands[spin] >= emin + bands.efermi)[0]))
    max_band_to_plot = max(max_band_to_plot, np.max(np.where(bands.bands[spin] <= emax + bands.efermi)[0]))

print(f'\tPlotting bands from {emin} to {emax} eV, band numbers from {min_band_to_plot+1} to
↳ {max_band_to_plot+1}')

reciprocal = bands.lattice_rec.matrix/(2*math.pi)

```

```

# Empty lists used for caculating the distances between K-Points
# unchanged from Kevin Waters's script
KPOINTS = [0.0]
DIST = 0.0
# Create list with distances between Kpoints (Individual), corrects the spacing
for k in range(len(bands.kpoints)-1):
    Dist = np.subtract(bands.kpoints[k+1].frac_coords,bands.kpoints[k].frac_coords)
    DIST += np.linalg.norm(np.dot(reciprocal,Dist))
    KPOINTS.append(DIST)

TICKS = [0.0]
for i in range(step,len(KPOINTS)+step,step):
    TICKS.append(KPOINTS[i-1])
# set y-axis limit
if args.split is False:
    ax_bands.set_ylabel(r"$E - E_f$ (eV)",labelpad=1) #labelpad might work bad
    ax_bands.set_ylim(emin, emax)
    ax_bands.grid(lw=args.glw,alpha=0.5)
    ax_bands.set_axisbelow(True)
    ax_bands.hlines(y=0, xmin=0, xmax=len(bands.kpoints), color="k", lw=args.flw)
    for i in range(step,len(KPOINTS)+step,step):
        ax_bands.vlines(KPOINTS[i-1], emin, emax, "k",lw=args.vlw,alpha=args.vla)
    ax_bands.set_xticks(TICKS)
    ax_bands.set_xticklabels(labels)
    ax_bands.tick_params(axis='x', which='both', length=0, pad=5)
    ax_bands.set_xlim(0, KPOINTS[-1])
else:
    ax_bands[0].set_ylabel(r"$E - E_f$ (eV)",labelpad=1) #labelpad might work bad
    for axis in ax_bands:
        axis.set_ylim(emin,emax)
        axis.grid(lw=args.glw,alpha=0.5)
        axis.set_axisbelow(True)
        axis.hlines(y=0, xmin=0, xmax=len(bands.kpoints), color="k", lw=args.flw)
        for i in range(step,len(KPOINTS)+step,step):
            axis.vlines(KPOINTS[i-1], emin, emax, "k",lw=args.vlw,alpha=args.vla)
        axis.set_xticks(TICKS)
        axis.set_xticklabels(labels)
        axis.tick_params(axis='x', which='both', length=0, pad=5)
        axis.set_xlim(0, KPOINTS[-1])
        if axis != ax_bands[0]:
            axis.set_yticks([])
            axis.set_yticklabels([])

ax_DOS.set_ylim(emin, emax)

if no_proj is False:
    CalculateProjectionsAndPlot()
    DOSlabel='total'
    ax_DOS.legend(fancybox=False, shadow=False, prop={'size':
        ↪ args.font_size-1},labelspacing=0.15,borderpad=0.20,handlelength=1.2,framealpha=0.6)
else:
    print('\tNo projection requested. Plotting normal bands.')
    for b in range(min_band_to_plot,max_band_to_plot+1):
        ax_bands.plot(KPOINTS,[e - bands.fermi for e in bands.bands[Spin.up][b]], lw=args.blw, color='k')
    DOSlabel=None

ax_DOS.fill_betweenx(dosrun.tdos.energies - dosrun.fermi,
    0,dosrun.tdos.densities[Spin.up],
    color = (0.7, 0.7, 0.7),
    facecolor = (0.7, 0.7, 0.7))
ax_DOS.plot(dosrun.tdos.densities[Spin.up],
    dosrun.tdos.energies - dosrun.fermi,
    color = (0.6, 0.6, 0.6),
    label = DOSlabel, lw=args.dlw)

```

```

# scaling factor for the x axis limit, if the peaks are too high
maxdos = max(dosrun.tdos.densities[Spin.up])/scale

ax_DOS.set_yticklabels([])
ax_DOS.grid(lw=args.glw,alpha=args.gla)
ax_DOS.tick_params('y', length=0, width=1, which='major')
ax_DOS.set_xticks([])
ax_DOS.set_xlim(0,maxdos)
ax_DOS.hlines(y=0, xmin=0, xmax=maxdos, color="k", lw=args.flw)
ax_DOS.set_xlabel("DOS",labelpad=5)

# Plotting
# -----
plt.savefig(f"{args.output_file}.{args.format}", format=args.format, bbox_inches='tight', dpi=400)

print('\tFile saved.')
print('-----')
```

vasp 计算能带

► 代码清单 A.3.1 (python, 36 行)

```

SYSTEM = MPSS
LREAL= Auto
ALGO= Fast

ISTART = 0
ICHARG = 2

ENCUT = 520
NCORE= 4

#IVDW=11
ISMEAR = 0
SIGMA = 0.04
GGA=PE

LORBIT=12

ISPIN=2
#MAGMOM = 1*4.0 2*0.0
LMAXMIX=4
#Amin = 0.01

VOSKOWN = 1
ISYM = 2
GGA_COMPAT = .FALSE.
LWAVE=.F
LCHARG=.T
EDIFF = 1E-6
EDIFFG = -0.01

LDAU=.T
LDAUTYPE=2
LDAUL=2 2 -1 -1          # 库伦排斥的轨道，对应元素
LDAUU=0.5 2.0 0.0 0.0    # 几个元素几个数
LDAUJ=0.0 0.0 0.0 0.0    # stoner 交换参数大小
LDAUPRINT = 2
```

► 代码清单 A.3.2 (python, 37 行)

```
SYSTEM = BrSeCr
LREAL= Auto
ALGO= Normal

ISTART = 1
ICHARG = 11 #=11 用于电子能带和态密度计算, 此过程中电荷密度不变

ENCUT =520
NCORE= 4

ISMEAR = 0
SIGMA = 0.04
GGA=PE

ISPIN=2
MAGMOM = 0 4 8*0

#LSORBIT=.TRUE.
#LORBMOM=.TRUE.

LWAVE=.F;
LCHARG=.F
EDIFF = 1E-6
EDIFFG = -0.01
ISYM = 0
VOSKOWN=1
GGA_COMPAT =.F

#NBANDS=50
#NEDOS=301
LORBIT=11          # 常用 =11, 输出包含 IM 投影的 PROCAR 和 DOSCAR
LDAU=.T
LDAUTYPE=2
LDAUL=2 2 -1 -1      # 库伦排斥的轨道, 对应元素
LDAUU=0.5 2.0 0.0 0.0 # 几个元素几个数
LDAUJ=0.0 0.0 0.0    #stoner 交换参数大小
LMAXMIX=4
```

vasp 计算磁耦合参数大小

► 代码清单 A.4.1 (shell, 40 行)

```
#General
PREC = ACCURATE
ISTART= 0
ICHARG= 2
ENCUT = 520
EDIFF = 1E-6
EDIFFG = -0.01
LREAL = Auto
NPAR = 4
NSW= 0
IBRION = -1
NELM=300
ISMEAR = 0
SIGMA = 0.05
LCHARG = .F.
```

```

LWAVE = .F.
POTIM = 0.1

#Magnetic properties

ISPIN=2
MAGMOM = 6*0.0 4 4 4 4 4 4 48*0.0

NELMIN = 6
LORBIT = 11
ISYM = 0
#LSORBIT = .True.
LMAXMIX = 4

LDAU = .TRUE.
LDAUTYPE = 2
LDAUL = 2 2 -1 -1
LDAUU = 0.5 2.0 0.0 0.0
LDAUJ = 0.0 0.0 0.0 0.0
LDAUPRINT = 2

AMIX      = 0.1
BMIX      = 0.0001
AMIX_MAG  = 0.8
BMIX_MAG  = 0.0001

```

► 代码清单 A.4.2 (shell, 40 行)

```

#General
PREC = ACCURATE
ISTART= 0
ICHARG= 2
ENCUT = 500
EDIFF = 1E-6
EDIFFG = -0.01
LREAL = Auto
NPAR = 4
NSW= 0
IBRION = -1
ISIF = 2
NELM=300
ISMear = 0
SIGMA = 0.05
LCHARG = .F.
LWAVE = .F.
POTIM = 0.1

#Magnetic properties

ISPIN=2
MAGMOM = 6*0.0 4 -4 4 -4 4 -4 48*0.0

NELMIN = 6
LORBIT = 11
ISYM = 0
LMAXMIX = 4

LDAU = .TRUE.
LDAUTYPE = 2
LDAUL = 2 2 -1 -1
LDAUU = 0.5 2.0 0.0 0.0
LDAUJ = 0.0 0.0 0.0 0.0
LDAUPRINT = 2

```

```
AMIX      = 0.1
BMIX      = 0.0001
AMIX_MAG  = 0.8
BMIX_MAG  = 0.0001
```

► 代码清单 A.4.3 (shell, 40 行)

```
#General
PREC = ACCURATE
ISTART= 0
ICHARG= 1
ENCUT = 500
EDIFF = 1E-6
EDIFFG = -0.01
LREAL = Auto
NPAR = 4
NSW= 0
IBRION = -1
ISIF = 2
NELM=300
ISMear = 0
SIGMA = 0.05
LCHARG = .F.
LWAVE = .F.
POTIM = 0.1

#Magnetic properties

ISPIN=2
MAGMOM = 6*0.0 -4 4 4 -4 4 -4 48*0.0

NELMIN = 6
LORBIT = 11
ISYM = 0
LMAXMIX = 4

LDAU = .TRUE.
LDAUTYPE = 2
LDAUL = 2 2 -1 -1
LDAUU = 0.5 2.0 0.0 0.0
LDAUJ = 0.0 0.0 0.0 0.0
LDAUPRINT = 2

AMIX      = 0.1
BMIX      = 0.0001
AMIX_MAG  = 0.8
BMIX_MAG  = 0.0001
```

► 代码清单 A.4.4 (shell, 40 行)

```
#General
PREC = ACCURATE
ISTART= 0
ICHARG= 1
ENCUT = 500
EDIFF = 1E-6
EDIFFG = -0.01
```



```

LREAL = Auto
NPAR = 4
NSW= 0
IBRION = -1
ISIF = 2
NELM=300
ISMear = 0
SIGMA = 0.05
LCHARG = .F.
LWAVE = .F.
POTIM = 0.1

#Magnetic properties

ISPIN=2
MAGMOM = 6*0.0 4 -4 -4 -4 4 4 48*0.0

NELMIN = 6
LORBIT = 11
ISYM = 0
LMAXMIX = 4

LDAU = .TRUE.
LDAUTYPE = 2
LDAUL = 2 2 -1 -1
LDAUU = 0.5 2.0 0.0 0.0
LDAUJ = 0.0 0.0 0.0 0.0
LDAUPRINT = 2

AMIX      = 0.1
BMIX      = 0.0001
AMIX_MAG  = 0.8
BMIX_MAG  = 0.0001

```

► 代码清单 A.4.5 (shell, 41 行)

```

#!/bin/bash
#PBS -l nodes=1:ppn=64
#PBS -l walltime=640:01:00
#PBS -j oe
#PBS -q sugon
#PBS -N test

cd $PBS_O_WORKDIR

echo job id is $PBS_JOBID | tee pbslog
echo run nodes is following: | tee -a pbslog
cat $PBS_NODEFILE | tee -a pbslog

echo begin time is `date` | tee -a pbslog
id=`echo $PBS_JOBID|awk -F. '{print $1}'`
NP=`cat $PBS_NODEFILE|wc -l`

source /public/software/profile.d/compiler_intel-compiler-2021.3.0.sh
source /public/software/profile.d/mpi_intelmpi-2021.3.0.sh
RUN_1=/public/home/cssong/app/vasp/intel_2021.3.0/5.4.4/vasp_std
RUN_2=/public/home/cssong/app/vasp/intel_2021.3.0/5.4.4/vasp_ncl
cat $PBS_NODEFILE | uniq -c | awk '{ printf("%s:%s\n", $2, $1); }' >> $PBS_JOBID-$PBS_JOBCOOKIE.hosts

for k in {1,2,3,4}
do
mkdir $k

```

```
cd $k
cp ../POSCAR .
cp ../POTCAR .
cp ../KPOINTS .
cp ../INCAR$k ./INCAR
mpirun -np $NP ${RUN_1}>&log

cd ..
done
```

```
rm -rf $PBS_JOBID-$PBS_JOBCOOKIE.hosts
```

► 代码清单 A.4.6 (shell, 46 行)

```
#I_CONSTRAINED_M=1

#General
PREC = ACCURATE
ISTART= 0
ICHARG= 2
ENCUT = 520
EDIFF = 1E-6
EDIFFG = -0.01
LREAL = Auto
NPAR = 4
NSW= 0
IBRION = -1
ISIF = 2
NELM=300
ISMEAR = 0
SIGMA = 0.05
LCHARG = .F.
LWAVE = .F.
POTIM = 0.1
#NEDOS = 2001
#NBANDS = 352

#Magnetic properties

ISPIN=2
MAGMOM = 4 0 0 4 0 0 4 0 0 4 0 0 4 0 0 4 0 0 4 0 0 96*0.0

NELMIN = 6
LORBIT = 11
ISYM = 0
LSORBIT = .True.
LMAXMIX = 4

LDAU = .TRUE.
LDAUTYPE = 2
LDAUL = 2 -1 -1
LDAUU = 4.3 0.0 0.0
LDAUJ = 0.0 0.0 0.0
LDAUPRINT = 2

AMIX      = 0.1
BMIX      = 0.0001
AMIX_MAG  = 0.8
BMIX_MAG  = 0.0001
```

▶ 代码清单 A.4.7 (shell, 41 行)

```
#General
PREC = ACCURATE
ISTART= 0
ICHARG= 2
ENCUT = 500
EDIFF = 1E-6
EDIFFG = -0.01
LREAL = Auto
NPAR = 4
NSW= 0
IBRION = -1
ISIF = 2
NELM=300
ISMEAR = 0
SIGMA = 0.05
LCHARG = .F.
LWAVE = .F.
POTIM = 0.1

#Magnetic properties

ISPIN=2
MAGMOM =4 0 0 4 0 0 4 0 0 4 0 0 4 0 0 4 0 0 4 0 0 -4 0 0 96*0.0

NELMIN = 6
LORBIT = 11
ISYM = 0
LSORBIT = .True.
LMAXMIX = 4

LDAU = .TRUE.
LDAUTYPE = 2
LDAUL = 2 -1 -1
LDAUU = 4.3 0.0 0.0
LDAUJ = 0.0 0.0 0.0
LDAUPRINT = 2

AMIX      = 0.1
BMIX      = 0.0001
AMIX_MAG  = 0.8
BMIX_MAG  = 0.0001
```

▶ 代码清单 A.4.8 (shell, 41 行)

```
#General
PREC = ACCURATE
ISTART= 0
ICHARG= 1
ENCUT = 500
EDIFF = 1E-6
EDIFFG = -0.01
LREAL = Auto
NPAR = 4
NSW= 0
IBRION = -1
ISIF = 2
NELM=300
ISMEAR = 0
SIGMA = 0.05
LCHARGE = .F.
```

```

LWAVE = .F.
POTIM = 0.1

#Magnetic properties

ISPIN=2
MAGMOM = -4 0 0 4 0 0 4 0 0 4 0 0 4 0 0 4 0 0 4 0 0 96*0.0

NELMIN = 6
LORBIT = 11
ISYM = 0
LSORBIT = .True.
LMAXMIX = 4

LDAU = .TRUE.
LDAUTYPE = 2
LDAUL = 2 -1 -1
LDAUU = 4.3 0.0 0.0
LDAUJ = 0.0 0.0 0.0
LDAUPRINT = 2

AMIX      = 0.1
BMIX      = 0.0001
AMIX_MAG  = 0.8
BMIX_MAG  = 0.0001

```

► 代码清单 A.4.9 (shell, 41 行)

```

#General
PREC = ACCURATE
ISTART= 0
ICHARG= 1
ENCUT = 500
EDIFF = 1E-6
EDIFFG = -0.01
LREAL = Auto
NPAR = 4
NSW= 0
IBRION = -1
ISIF = 2
NELM=300
ISMEAR = 0
SIGMA = 0.05
LCHARG = .F.
LWAVE = .F.
POTIM = 0.1

#Magnetic properties

ISPIN=2
MAGMOM = -4 0 0 4 0 0 4 0 0 4 0 0 4 0 0 4 0 0 -4 0 0 96*0.0

NELMIN = 6
LORBIT = 11
ISYM = 0
LSORBIT = .True.
LMAXMIX = 4

LDAU = .TRUE.
LDAUTYPE = 2
LDAUL = 2 -1 -1
LDAUU = 4.3 0.0 0.0
LDAUJ = 0.0 0.0 0.0

```

```
LDAUPRINT = 2

AMIX      = 0.1
BMIX      = 0.0001
AMIX_MAG  = 0.8
BMIX_MAG  = 0.0001
```

► 代码清单 A.4.10 (shell, 51 行)

```
#!/bin/bash
#PBS -l nodes=1:ppn=64
#PBS -l walltime=640:01:00
#PBS -j oe
#PBS -q sugon
#PBS -N test

cd $PBS_0_WORKDIR

echo job id is $PBS_JOBID | tee pbslog
echo run nodes is following: | tee -a pbslog
cat $PBS_NODEFILE | tee -a pbslog

echo begin time is `date` | tee -a pbslog
id=`echo $PBS_JOBID|awk -F. '{print $1}'`
NP=`cat $PBS_NODEFILE|wc -l`

source /public/software/profile.d/compiler_intel-compiler-2021.3.0.sh
source /public/software/profile.d/mpi_intelmpi-2021.3.0.sh
RUN_1=/public/home/cssong/app/vasp/intel_2021.3.0/5.4.4/vasp_std
RUN_2=/public/home/cssong/app/vasp/intel_2021.3.0/5.4.4/vasp_ncl
cat $PBS_NODEFILE | uniq -c | awk '{ printf("%s:%s\n", $2, $1); }' >> $PBS_JOBID-$PBS_JOBCOOKIE.hosts

#mkdir static
#cd static
#cp ../POSCAR .
#cp ../POTCAR .
#cp ../KPOINTS .
#cp ../incar.static ./INCAR
#mpirun -np $NP ${RUN_2}>&log
#cd ..

for k in {1,2,3,4}
do
mkdir $k
cd $k
cp ../POSCAR .
cp ../POTCAR .
cp ../KPOINTS .
cp ../INCAR$k ./INCAR
#cp ../static/CHGCAR .
mpirun -np $NP ${RUN_2}>&log

cd ..
done

rm -rf $PBS_JOBID-$PBS_JOBCOOKIE.hosts
```

vasp 计算 DMI 大小

► 代码清单 A.5.1 (python, 39 行)

```
SYSTEM = MPSS
LREAL= Auto
ALGO= Normal

ISTART = 0
ICHARG = 2

ENCUT = 520
NCORE= 4
NPAR=4

ISMear = 0
SIGMA = 0.05
GGA=PE

ISPIN=2
MAGMOM = 12*0.0 -4 0 0 0 0 -4 4 0 0 0 0 4 96*0.0

LDAU = .TRUE.
LDAUTYPE = 2
LDAUL = 2 2 -1 -1
LDAUU = 0.5 2.0 0.0 0.0
LDAUJ = 0.0 0.0 0.0 0.0
LDAUPRINT = 2

LSORBIT=.TRUE.
LORBMOM=.TRUE.

NELM =200
LORBIT=11
VOSKOWN = 1
ISYM = 0
GGA_COMPAT = .FALSE.
LWAVE=.F
LCHARG=.F
EDIFF = 1E-6
EDIFFG = -0.01

LMAXMIX=4
```

► 代码清单 A.5.2 (shell, 39 行)

```
#!/bin/bash
#PBS -l nodes=1:ppn=64
#PBS -l walltime=640:01:00
#PBS -j oe
#PBS -q sugon
#PBS -N D_MP

cd $PBS_O_WORKDIR

echo job id is $PBS_JOBID | tee pbslog
echo run nodes is following: | tee -a pbslog
cat $PBS_NODEFILE | tee -a pbslog

echo begin time is `date` | tee -a pbslog
```

```

id=`echo $PBS_JOBID|awk -F. '{print $1}'`
NP=`cat $PBS_NODEFILE|wc -l`

source /public/software/profile.d/compiler_intel-compiler-2021.3.0.sh
source /public/software/profile.d/mpi_intelmpi-2021.3.0.sh
RUN_1=/public/home/cssong/app/vasp/intel_2021.3.0/5.4.4/vasp_std
RUN_2=/public/home/cssong/app/vasp/intel_2021.3.0/5.4.4/vasp_ncl
cat $PBS_NODEFILE | uniq -c | awk '{ printf("%s:%s\n", $2, $1); }' >> $PBS_JOBID-$PBS_JOBCOOKIE.hosts

for i in {cw,acw}
do
mkdir $i
cd $i
cp ../incar.$i ./INCAR
cp ../POTCAR .
cp ../KPOINTS .
cp ../POSCAR .
mpirun -np $NP ${RUN_2}>&log
rm CHG
rm WAVECAR

cd ..
done

```

► 代码清单 A.5.3 (shell, 46 行)

```

#!/bin/bash
#PBS -l nodes=1:ppn=64
#PBS -l walltime=640:01:00
#PBS -j oe
#PBS -q sugon
#PBS -N ST_D

cd $PBS_0_WORKDIR

echo job id is $PBS_JOBID | tee pbslog
echo run nodes is following: | tee -a pbslog
cat $PBS_NODEFILE | tee -a pbslog

echo begin time is `date` | tee -a pbslog
id=`echo $PBS_JOBID|awk -F. '{print $1}'`
NP=`cat $PBS_NODEFILE|wc -l`

source /public/software/profile.d/compiler_intel-compiler-2021.3.0.sh
source /public/software/profile.d/mpi_intelmpi-2021.3.0.sh
RUN_1=/public/home/cssong/app/vasp/intel_2021.3.0/5.4.4/vasp_std
RUN_2=/public/home/cssong/app/vasp/intel_2021.3.0/5.4.4/vasp_ncl
cat $PBS_NODEFILE | uniq -c | awk '{ printf("%s:%s\n", $2, $1); }' >> $PBS_JOBID-$PBS_JOBCOOKIE.hosts

for i in {x,y,z}
do
cd D$i

    for k in {1,2,3,4}
    do
mkdir $k
cd $k
cp ../POSCAR .
cp ../POTCAR .

```

```
cp ../KPOINTS .
cp ../INCAR$k ./INCAR

cd ..
done

cd ..
done

rm -rf $PBS_JOBID-$PBS_JOBCOOKIE.hosts
```

(文献整理) 块体材料 DMI 的第一性原理提取方法

► 代码清单 A.6.1 (shell, 5 行)

1. 四态能量映射法 four-state energy mapping;
2. spin spiral / generalized Bloch 方法;
3. Green's function / magnetic force theorem 方法;
4. Wannier / TB2J 自动提取 J、D、各向异性交换的方法;
5. 已经在块体材料中使用过的典型案例, 例如 BiFeO₃、Cu₂OSeO₃、FeGe、Mn_{1-x}Fe_xGe。

► 代码清单 A.6.2 (text, 4 行)

Hongxin Yang, Jinghua Liang, and Qirui Cui,
First-principles calculations for Dzyaloshinskii-Moriya interaction,
Nature Reviews Physics 5, 43--61 (2023).
DOI: 10.1038/s42254-022-00529-0

► 代码清单 A.6.3 (shell, 5 行)

1. supercell energy mapping;
2. generalized Bloch theorem / spin spiral;
3. Green's function;
4. Berry phase / spin current 相关表述;
5. 不同材料体系中的 DMI 诱导手性磁结构。

► 代码清单 A.6.4 (shell, 1 行)

如果论文中需要一句话说明“DMI 可以通过多种第一性原理方法计算”, 优先引用这篇。

► 代码清单 A.6.5 (text, 4 行)

Hongjun Xiang, Changhoon Lee, Hyo-Jun Koo, Xiangdong Gong, and Myung-Hwan Whangbo,
Magnetic properties and energy-mapping analysis,
Dalton Transactions 42, 823--853 (2013).
DOI: 10.1039/C2DT31662E

► 代码清单 A.6.6 (shell, 3 行)

1. 说明为什么可以用 DFT 总能差提取磁参数;
2. 说明 spin Hamiltonian 的建立过程;
3. 给 J、D、K 等磁参数拟合提供方法依据。

► 代码清单 A.6.7 (text, 5 行)

Denis Šabani, Cihan Bacaksız, and M. V. Milošević,
Ab initio methodology for magnetic exchange parameters:
Generic four-state energy mapping onto a Heisenberg spin Hamiltonian,
Physical Review B 102, 014457 (2020).
DOI: 10.1103/PhysRevB.102.014457

► 代码清单 A.6.8 (text, 4 行)

Changsong Xu, Bin Xu, Bertrand Dupé, and Laurent Bellaïche,
Magnetic interactions in BiFeO₃: A first-principles study,
Physical Review B 99, 104420 (2019).
DOI: 10.1103/PhysRevB.99.104420

► 代码清单 A.6.9 (shell, 4 行)

1. symmetric exchange;
2. Dzyaloshinskii-Moriya interaction;
3. single-ion anisotropy;
4. 一直到第二近邻的磁相互作用参数。

► 代码清单 A.6.10 (shell, 1 行)

若研究体系为块体材料，并计划通过四态能量映射提取各类磁性键的 DMI，该文可作为合适的引用依据。

► 代码清单 A.6.11 (text, 4 行)

Bin Xu, Sebastian Meyer, Matthieu J. Verstraete, Laurent Bellaiche, and Bertrand Dupé,
First-principles study of spin spirals in the multiferroic BiFeO₃,
Physical Review B 103, 214423 (2021).
DOI: 10.1103/PhysRevB.103.214423

► 代码清单 A.6.12 (text, 4 行)

J. H. Yang, Z. L. Li, X. Z. Lu, M.-H. Whangbo, S.-H. Wei, X. G. Gong, and H. J. Xiang,
Strong Dzyaloshinskii--Moriya Interaction and Origin of Ferroelectricity in Cu₂OSeO₃,
Physical Review Letters 109, 107203 (2012).
DOI: 10.1103/PhysRevLett.109.107203

► 代码清单 A.6.13 (shell, 3 行)

1. 块体手性磁体中 DMI 诱导螺旋态、斯格明子态;
2. DMI 与铁电性、多铁性的耦合;
3. 作为“块体材料 DMI 计算”的高影响力案例。

► 代码清单 A.6.14 (text, 5 行)

G. C. Loh and C. K. Gan,
Exchange and Dzyaloshinskii--Moriya interactions in bulk FeGe:
Effects of atomic vacancies,
AIP Advances 7, 056412 (2017).
DOI: 10.1063/1.4973847

► 代码清单 A.6.15 (shell, 3 行)

1. 直接支撑“bulk material 可以计算 DMI”;
2. 适合引用到 B20 手性磁体、螺旋磁结构、空位调控 DMI 的讨论中;
3. 若研究材料为三维周期块体而非表面或界面, 该文具有较强的示范意义。

► 代码清单 A.6.16 (text, 5 行)

Takashi Koretsune, Naoto Nagaosa, and Ryotaro Arita,
Control of Dzyaloshinskii--Moriya interaction in Mn_{1-x}Fe_xGe:
a first-principles study,
Scientific Reports 5, 13302 (2015).
DOI: 10.1038/srep13302

► 代码清单 A.6.17 (shell, 3 行)

1. 说明 DMI 大小和符号可以通过掺杂调控;
2. 说明 Wannier / first-principles 电子结构分析可用于理解 DMI;
3. 适合与 FeGe、MnSi 等块体手性磁体放在一起引用。

► 代码清单 A.6.18 (text, 4 行)

Xu He, Nicole Helbig, Matthieu J. Verstraete, and Eric Bousquet,
TB2J: A python package for computing magnetic interaction parameters,
Computer Physics Communications 264, 107938 (2021).
DOI: 10.1016/j.cpc.2021.107938

► 代码清单 A.6.19 (shell, 1 行)

对于结构复杂的体系, 宜优先考虑 Wannier90 + TB2J, 而非手动构造大量非共线四态。

► 代码清单 A.6.20 (text, 5 行)

Farzad Mahfouzi and Nicholas Kioussis,
First-principles calculation of the Dzyaloshinskii--Moriya interaction:
A Green's function approach,
Physical Review B 103, 094410 (2021).
DOI: 10.1103/PhysRevB.103.094410

► 代码清单 A.6.21 (shell, 3 行)

1. 作为 Green's function DMI 方法的理论依据;
2. 和 TB2J 文献一起引用;
3. 用于说明 DMI 可以从第一性原理电子结构出发通过微扰方法获得。

► 代码清单 A.6.22 (text, 3 行)

Xiang et al., Dalton Trans. 42, 823--853 (2013).
Šabani et al., Phys. Rev. B 102, 014457 (2020).
Xu et al., Phys. Rev. B 99, 104420 (2019).

► 代码清单 A.6.23 (text, 1 行)

The magnetic exchange parameters, including the isotropic exchange and Dzyaloshinskii-Moriya interaction, can be
↪ extracted by mapping first-principles total energies onto an atomistic spin Hamiltonian. In particular, the
↪ DMI vector corresponds to the antisymmetric part of the exchange matrix obtained from four-state energy
↪ mapping.

► 代码清单 A.6.24 (text, 2 行)

Xu et al., Phys. Rev. B 103, 214423 (2021).
Loh and Gan, AIP Adv. 7, 056412 (2017).

► 代码清单 A.6.25 (text, 3 行)

He et al., Comput. Phys. Commun. 264, 107938 (2021).
Mahfouzi and Kioussis, Phys. Rev. B 103, 094410 (2021).
Koretsune et al., Sci. Rep. 5, 13302 (2015).

► 代码清单 A.6.26 (text, 1 行)

The pairwise exchange parameters were further evaluated using a Green's-function-based magnetic force theorem
↪ approach, which allows the calculation of isotropic exchange, anisotropic exchange, and Dzyaloshinskii-Moriya
↪ interaction from Wannier Hamiltonians.

► 代码清单 A.6.27 (shell, 5 行)

1. 层内最近邻;
2. 层内次近邻;
3. 层间近邻;
4. 不等价磁性原子之间的交换路径;
5. 不同晶向上的 DMI 矢量方向。

► 代码清单 A.6.28 (shell, 17 行)

1. Yang, Liang & Cui, Nat. Rev. Phys. 2023
先建立 DMI 第一性原理计算方法的整体框架。
2. Xiang et al., Dalton Trans. 2013
理解 energy mapping 和 spin Hamiltonian 的基本思想。

-
3. Šabani et al., PRB 2020
学习通用四态映射和 exchange matrix 到 DMI 的关系。
 4. Xu et al., PRB 2019
看块体 BiFeO₃ 如何用四态法提取 DMI、J、K。
 5. He et al., CPC 2021
如果体系复杂, 转向 TB2J / Wannier 自动提取方法。
 6. Loh & Gan 2017 / Koretsune 2015 / Yang et al. 2012
根据材料类型选择对应块体案例参考。

► 代码清单 A.6.29 (shell, 1 行)

Wannier90 + TB2J

► 代码清单 A.6.30 (shell, 1 行)

四态能量映射法 + Šabani 2020 + Xu 2019 BiFeO₃

► 代码清单 A.6.31 (shell, 1 行)

spin spiral 方法 + Xu 2021 BiFeO₃ + Loh 2017 FeGe

► 代码清单 A.6.32 (text, 54 行)

- [1] H. Yang, J. Liang, and Q. Cui,
First-principles calculations for Dzyaloshinskii-Moriya interaction,
Nature Reviews Physics 5, 43--61 (2023).
DOI: 10.1038/s42254-022-00529-0
- [2] H. Xiang, C. Lee, H.-J. Koo, X. Gong, and M.-H. Whangbo,
Magnetic properties and energy-mapping analysis,
Dalton Transactions 42, 823--853 (2013).
DOI: 10.1039/C2DT31662E
- [3] D. Šabani, C. Bacaksız, and M. V. Milošević,
Ab initio methodology for magnetic exchange parameters:
Generic four-state energy mapping onto a Heisenberg spin Hamiltonian,
Physical Review B 102, 014457 (2020).
DOI: 10.1103/PhysRevB.102.014457
- [4] C. Xu, B. Xu, B. Dupé, and L. Bellaiche,
Magnetic interactions in BiFeO₃: A first-principles study,
Physical Review B 99, 104420 (2019).
DOI: 10.1103/PhysRevB.99.104420

-
- [5] B. Xu, S. Meyer, M. J. Verstraete, L. Bellaiche, and B. Dupé,
First-principles study of spin spirals in the multiferroic BiFeO₃,
Physical Review B 103, 214423 (2021).
DOI: 10.1103/PhysRevB.103.214423
- [6] J. H. Yang, Z. L. Li, X. Z. Lu, M.-H. Whangbo, S.-H. Wei,
X. G. Gong, and H. J. Xiang,
Strong Dzyaloshinskii-Moriya Interaction and Origin of Ferroelectricity in Cu₂OSeO₃,
Physical Review Letters 109, 107203 (2012).
DOI: 10.1103/PhysRevLett.109.107203
- [7] G. C. Loh and C. K. Gan,
Exchange and Dzyaloshinskii-Moriya interactions in bulk FeGe:
Effects of atomic vacancies,
AIP Advances 7, 056412 (2017).
DOI: 10.1063/1.4973847
- [8] T. Koretsune, N. Nagaosa, and R. Arita,
Control of Dzyaloshinskii-Moriya interaction in Mn_{1-x}Fe_xGe:
a first-principles study,
Scientific Reports 5, 13302 (2015).
DOI: 10.1038/srep13302
- [9] X. He, N. Helbig, M. J. Verstraete, and E. Bousquet,
TB2J: A python package for computing magnetic interaction parameters,
Computer Physics Communications 264, 107938 (2021).
DOI: 10.1016/j.cpc.2021.107938
- [10] F. Mahfouzi and N. Kioussis,
First-principles calculation of the Dzyaloshinskii-Moriya interaction:
A Green's function approach,
Physical Review B 103, 094410 (2021).
DOI: 10.1103/PhysRevB.103.094410

vasp 结构优化

► 代码清单 A.7.1 (shell, 22 行)

```
SYSTEM = MOPTSNs
LREAL= Auto
ALGO= Fast
PREC=High

ISTART = 0
ICHARG = 2

ENCUT = 520
NCORE= 4
ISYM=-1

ISMEAR =0
SIGMA = 0.04

NSW = 200
ISIF = 3
IBRION =2
LWAVE=.F
LCHARG=.F
EDIFF = 1E-4
EDIFFG = -0.1
```

► 代码清单 A.7.2 (shell, 27 行)

```
#!/bin/bash
#PBS -l nodes=1:ppn=64
#PBS -l walltime=640:01:00
#PBS -j oe
#PBS -q sugon
#PBS -N geo_slide

cd $PBS_0_WORKDIR

echo job id is $PBS_JOBID | tee pbslog
echo run nodes is following: | tee -a pbslog
cat $PBS_NODEFILE | tee -a pbslog

echo begin time is `date` | tee -a pbslog
id=`echo $PBS_JOBID|awk -F. '{print $1}'`
NP=`cat $PBS_NODEFILE|wc -l`

source /public/software/profile.d/compiler_intel-compiler-2021.3.0.sh
source /public/software/profile.d/mpi_intelmpi-2021.3.0.sh
RUN_1=/public/home/cssong/app/vasp/intel_2021.3.0/5.4.4/vasp_std
RUN_2=/public/home/xpwu/app/vasp/intel_2021.3.0/5.4.4/vasp_ncl
cat $PBS_NODEFILE | uniq -c | awk '{ printf("%s:%s\n", $2, $1); }' >> $PBS_JOBID-$PBS_JOBCOOKIE.hosts

mpirun -np $NP ${RUN_1}>&log
```

Wannier90 学习笔记

► 代码清单 A.8.1 (shell, 3 行)

```
num_wann 4
num_wann = 4
num_wann : 4
```

以 Bi 烯为例的 wannier90 表面态计算

► 代码清单 A.9.2 (shell, 30 行)

```
SYSTEM = Bi
ISTART = 0
LCHARG = .T.
IVDW=11
ISYM=0
LSORBIT = .TRUE.
SAXIS = 0 0 1
MAGMOM = 6*0
GGA_COMPAT=.FALSE.
```

```
NELM = 2000
NELMIN = 5
EDIFFG = -0.001
EDIFF = 1E-08
PREC = A
NSW = 0
IBRION = -1
POTIM = 0.5
ENCUT = 520
```

```
ISMear = -5
EMIN=-15
EMAX=15
LORBIT=11
NEDOS=3000
ALGO = N
LREAL = F
LWANNIER90 = T
ICHARG= 1
NBANDS=64
```

► 代码清单 A.9.3 (shell, 15 行)

```
num_wann = 12
exclude_bands:1-2

kmesh_tol=0.00001

dis_win_min=-7.2
dis_win_max=3.5
dis_froz_min=-7.2
dis_froz_max=1.45

begin projections
Bi:p
end projections

guiding_centres = true
```

► 代码清单 A.9.4 (shell, 10 行)

```
write_hr = true
bands_plot = true
bands_num_points = 20
bands_plot_format = gnuplot

begin kpoint_path
G 0.00000 0.00000 0.0000 M 0.50000 0.00000 0.0000
M 0.50000 0.00000 0.0000 K 0.33333 0.33333 0.0000
K 0.33333 0.33333 0.0000 G 0.00000 0.00000 0.0000
end kpoint_path
```


► 代码清单 A.9.7 (shell, 63 行)

```
&TB_FILE
Hrfile = 'wannier90_hr.dat' ! 紧束缚模型文件
Package = 'VASP'           ! 使用 VASP 软件包
/

&CONTROL
BulkBand_calc      = F      ! 不计算体材料能带
SlabSS_calc        = T      ! 计算表面谱函数
Z2_3D_calc         = F      ! 不计算 3D Z2 拓扑不变量
/

&SYSTEM
SOC = 1             ! 自旋轨道耦合
E_FERMI = -0.6465    ! 费米能级
Numoccupied = 6      ! 占据态数目
/

&PARAMETERS
Eta_Arc = 0.001      ! 无穷小量, 用于展览
E_arc = -0.05         ! 计算费米弧的能量
OmegaMin = -1.3       ! 能量范围最小值, 用于能带绘图
OmegaMax = 1.3        ! 能量范围最大值, 用于能带绘图
OmegaNum = 401        ! 能量点数, 能带绘图步长
Nk1 = 101            ! k 点数量 (奇数更佳)
Nk2 = 101            ! k 点数量 (奇数更佳)
Np = 2               ! k 点数量 (奇数更佳)
/

SURFACE              ! 表面定义 (锯齿形方向)
1 0 0                ! 表面法向量
0 0 1                ! 第二个基矢
0 1 0                ! 第三个基矢

KPATH_BULK            ! 体材料 k 点路径
3                    ! k 路径段数 (仅用于体材料能带)
G 0.00000 0.00000 0.0000 M 0.50000 0.00000 0.0000 !  $\Gamma$  到 M 点
M 0.50000 0.00000 0.0000 K 0.33333 0.33333 0.0000 ! M 到 K 点
K 0.33333 0.33333 0.0000 G 0.00000 0.00000 0.0000 ! K 到  $\Gamma$  点

KPATH_SLAB            ! 薄膜 k 点路径
2                    ! k 路径段数 (用于二维情况)
X 0.5 0.0 G 0.0 0.0 ! X 到  $\Gamma$  点 (二维情况 k 路径)
G 0.0 0.0 X 0.5 0.0 !  $\Gamma$  到 X 点

LATTICE               ! 晶格参数
Angstrom             ! 单位: 埃
2.1017795 -3.6403930 0.0000000 ! 晶格矢量 a
2.1017795 3.6403930 0.0000000 ! 晶格矢量 b
0.0000000 0.0000000 15.0000000 ! 晶格矢量 c

ATOM_POSITIONS        ! 原子位置
2                    ! 投影原子数目
Cartisen             ! 笛卡尔坐标
Bi 2.1017795 1.2134643 10.7681236 ! 铋原子位置 1
Bi 2.1017795 -1.2134643 9.0367958 ! 铋原子位置 2

PROJECTORS            ! 投影子设置
3 3                  ! 每个原子的投影轨道数
Bi pz px py          ! 铋原子 1 的轨道: pz, px, py
Bi pz px py          ! 铋原子 2 的轨道: pz, px, py

!WANNIER_CENTRES      ! 从 wannier90.wout 复制
!Cartesian           ! 笛卡尔坐标
```

► 代码清单 A.9.11 (shell, 63 行)

```
&TB_FILE
Hrfile = 'wannier90_hr.dat'
Package = 'VASP'
/

&CONTROL
BulkBand_calc      = F
SlabSS_calc        = T
Z2_3D_calc         = F
/

&SYSTEM
SOC = 1             ! soc
E_FERMI = -1.1651   ! e-fermi
Numoccupied = 6
/

&PARAMETERS
Eta_Arc = 0.001     ! infinite small value, like brodening
E_arc = -0.05        ! energy for calculate Fermi Arc
OmegaMin = -1.3      ! energy interval, plot band energy minimal
OmegaMax = 1.3       ! energy interval, plot band energy maximal
OmegaNum = 401       ! omega number, plot band energy step
Nk1 = 101            ! number k points odd number would be better
Nk2 = 101            ! number k points odd number would be better
Np = 2               ! number k points odd number would be better
/

SURFACE              ! zig-zag direction
1 0 0
0 0 1
0 1 0

KPATH_BULK            ! k point path
3                     ! number of k line only for bulk band
G 0.00000 0.00000 0.0000 Y 0.50000 0.00000 0.0000
Y 0.50000 0.00000 0.0000 M 0.33333 0.33333 0.0000
M 0.33333 0.33333 0.0000 G 0.00000 0.00000 0.0000

KPATH_SLAB
2                     ! number of k line for 2D case
X 0.5 0.0 G 0.0 0.0 ! k path for 2D case
G 0.0 0.0 X 0.5 0.0

LATTICE
Angstrom
2.1017795 -3.6403930 0.0000000
2.1017795 3.6403930 0.0000000
0.0000000 0.0000000 15.0000000

ATOM_POSITIONS
2                     ! number of atoms for projectors
Cartisen             ! Direct or Cartisen coordinate
Bi 2.1017795 1.2134643 10.7681236
Bi 2.1017795 -1.2134643 9.0367958

PROJECTORS
3 3                 ! number of projectors
Bi pz px py
Bi pz px py

!WANNIER_CENTRES     ! copy from wannier90.wout
!Cartesian
```

► 代码清单 A.9.12 (shell, 57 行)

```
&TB_FILE
Hrfile = 'wannier90_hr.dat'
Package = 'VASP'
/

&CONTROL
BulkBand_calc      = F
SlabSS_calc        = F
Z2_3D_calc         = T
/

&SYSTEM
SOC = 1             ! soc on
E_FERMI = -1.1651   ! e-fermi from VASP SCF calculation
Numoccupied = 6     ! each Bi atom have 3 electrons, 2 Bi atoms means 6 occupied band
/

&PARAMETERS
Nk1 = 101           ! number k points  odd number would be better
Nk2 = 101           ! number k points  odd number would be better
/

SURFACE              ! original cell
1 0 0
0 1 0
0 0 1

KPATH_BULK            ! k point path
3                     ! number of k line only for bulk band
G 0.00000 0.00000 0.0000 Y 0.50000 0.00000 0.0000
Y 0.50000 0.00000 0.0000 M 0.33333 0.33333 0.0000
M 0.33333 0.33333 0.0000 G 0.00000 0.00000 0.0000

KPATH_SLAB
2                     ! number of k line for 2D case
X 0.5 0.0 G 0.0 0.0 0.0 ! k path for 2D case
G 0.0 0.0 X 0.5 0.0

LATTICE
Angstrom
2.1017795 -3.6403930 0.0000000
2.1017795 3.6403930 0.0000000
0.0000000 0.0000000 15.0000000

ATOM_POSITIONS
2                     ! number of atoms for projectors
Cartisen             ! Direct or Cartisen coordinate
Bi 2.1017795 1.2134643 10.7681236
Bi 2.1017795 -1.2134643 9.0367958

PROJECTORS
3 3                  ! number of projectors
Bi pz px py
Bi pz px py

!WANNIER_CENTRES      ! copy from wannier90.wout
!Cartesian
```

Wannier90 的表面态的参数选择标准 1_示例文件的参数注释

► 代码清单 A.11.1 (shell, 63 行)

```
&TB_FILE
Hrfile = 'wannier90_hr.dat' ! 指定 Wannier90 输出的哈密顿矩阵文件
Package = 'VASP'           ! 说明原始计算来自 VASP
/

&CONTROL
BulkBand_calc      = F      ! 是否计算体能带 (F= 否, T= 是)
SlabSS_calc        = T      ! 是否计算表面态 (T= 是)
Z2_3D_calc         = F      ! 是否计算三维体系的 Z2 拓扑不变量 (F= 否)
/

&SYSTEM
SOC = 1             ! 是否包含自旋轨道耦合 (SOC=1 表示开启)
E_FERMI = -1.1651   ! 费米能级 (单位 eV)
Numoccupied = 6      ! 占据能带的数目
/

&PARAMETERS
Eta_Arc = 0.001     ! 极小值, 用作展宽因子 (类似能量展宽)
E_arc = -0.05        ! 计算费米弧时对应的能量 (单位 eV)
OmegaMin = -1.3      ! 能量范围下限 (绘制能带图时的最小能量)
OmegaMax = 1.3       ! 能量范围上限 (绘制能带图时的最大能量)
OmegaNum = 401       ! 能量采样点数量 (能量步长的分辨率)
Nk1 = 101            ! k 点数 (方向 1), 建议使用奇数以包含  $\Gamma$  点
Nk2 = 101            ! k 点数 (方向 2), 建议使用奇数以包含  $\Gamma$  点
Np = 2               ! k 点数 (方向 3 或平面数), 建议使用奇数
/

SURFACE              ! 定义表面切面 (此处为 zigzag 方向)
  1  0  0            ! 表面法向的第一行 (对应晶格矢量 a1)
  0  0  1            ! 第二行 (对应晶格矢量 a2)
  0  1  0            ! 第三行 (定义表面坐标系)

KPATH_BULK            ! 定义体能带计算的 k 点路径
3                     ! k 线的数量 (仅在计算体能带时使用)
G 0.00000 0.00000 0.00000 Y 0.50000 0.00000 0.00000
Y 0.50000 0.00000 0.00000 M 0.33333 0.33333 0.00000
M 0.33333 0.33333 0.00000 G 0.00000 0.00000 0.00000

KPATH_SLAB           ! 定义二维体系表面态的 k 点路径
2                     ! k 线数量 (用于二维表面态)
X 0.5 0.0 G 0.0 0.0   ! 第一条 k 线路径 (X $\rightarrow$  $\Gamma$ )
G 0.0 0.0 X 0.5 0.0   ! 第二条 k 线路径 ( $\Gamma$  $\rightarrow$ X)

LATTICE              ! 晶格参数定义
Angstrom             ! 单位为 Å
  2.1017795 -3.6403930 0.0000000
  2.1017795 3.6403930 0.0000000
  0.0000000 0.0000000 15.0000000 ! z 方向真空层厚度

ATOM_POSITIONS
2                     ! 原子数目
Cartisen             ! 原子坐标类型 (Direct 为分数坐标, Cartisen 为笛卡尔坐标)
Bi 2.1017795 1.2134643 10.7681236
Bi 2.1017795 -1.2134643 9.0367958

PROJECTORS
3 3                  ! 定义投影子数量 (每个原子 3 个投影轨道)
Bi pz px py          ! 第一个 Bi 的投影轨道
Bi pz px py          ! 第二个 Bi 的投影轨道
```

```
!WANNIER_CENTRES      ! (可选项) 从 wannier90.wout 复制 Wannier 中心位置
!Cartesian             ! 表示使用笛卡尔坐标系
```

► 代码清单 A.11.2 (shell, 10 行)

```
&PARAMETERS
Eta_Arc = 0.001      ! 极小值, 用作展宽因子 (类似能量展宽)
E_arc = -0.05        ! 计算费米弧时对应的能量 (单位 eV)
OmegaMin = -1.3      ! 能量范围下限 (绘制能带图时的最小能量)
OmegaMax = 1.3       ! 能量范围上限 (绘制能带图时的最大能量)
OmegaNum = 401       ! 能量采样点数量 (能量步长的分辨率)
Nk1 = 101            ! k 点数 (方向 1), 建议使用奇数以包含  $\Gamma$  点
Nk2 = 101            ! k 点数 (方向 2), 建议使用奇数以包含  $\Gamma$  点
Np = 2               ! k 点数 (方向 3 或平面数), 建议使用奇数
/
```

► 代码清单 A.11.3 (shell, 4 行)

```
SURFACE      ! 定义表面切面 (此处为 zigzag 方向)
1 0 0        ! 表面法向的第一行 (对应晶格矢量 a1)
0 0 1        ! 第二行 (对应晶格矢量 a2)
0 1 0        ! 第三行 (定义表面坐标系)
```

► 代码清单 A.11.4 (shell, 5 行)

```
KPATH_BULK      ! 定义体能带计算的 k 点路径
3               ! k 线的数量 (仅在计算体能带时使用)
G 0.00000 0.00000 0.0000 Y 0.50000 0.00000 0.0000
Y 0.50000 0.00000 0.0000 M 0.33333 0.33333 0.0000
M 0.33333 0.33333 0.0000 G 0.00000 0.00000 0.0000c
```

► 代码清单 A.11.5 (shell, 4 行)

```
KPATH_SLAB      ! 定义二维体系表面态的 k 点路径
2               ! k 线数量 (用于二维表面态)
X 0.5 0.0 G 0.0 0.0 ! 第一条 k 线路径 (X $\rightarrow$  $\Gamma$ )
G 0.0 0.0 X 0.5 0.0 ! 第二条 k 线路径 ( $\Gamma$  $\rightarrow$ X)
```

wannier90 的能带拟合步骤

► 代码清单 A.20.1 (shell, 5 行)

```
K-Spacing Value to Generate K-Mesh: 0.030
0
Gamma
11 11 1
0.0 0.0 0.0
```

► 代码清单 A.20.2 (shell, 43 行)

```
SYSTEM = MOPTSNS
LREAL= Auto
ALGO= Normal
#ISEARCH=1
PREC=High

ISTART = 0
ICHARG = 2

ENCUT = 520
NCORE= 4
ISYM=2

ISMEAR =0
SIGMA = 0.04

NSW = 0
ISIF = 2
IBRION =-1
LWAVE=.T
LCHARG=.T
EDIFF = 1E-6
EDIFFG = -0.01

NELM=100

LDAU=.T
LDAUTYPE=2
LDAUL=2 -1 -1 -1 # 库伦排斥的轨道, 对应元素
LDAUU=1.5 0.0 0.0 0.0 # 几个元素几个数
LDAUJ=0.0 0.0 0.0 0.0 #stoner 交换参数大小
LDAUPRINT = 2

ISPIN=2
MAGMOM = 0 0 4 0 0 -4 9*0
LMAXMIX=4

LSORBIT=.T

AMIX = 0.1
BMIX = 0.0001
AMIX_MAG = 0.8
BMIX_MAG = 0.0001
```

► 代码清单 A.20.3 (shell, 11 行)

```
magnetization (x)
```

# of ion	s	p	d	tot
1	0.014	0.006	0.663	0.682
2	-0.014	-0.006	-0.663	-0.682
3	0.000	0.000	0.000	0.000
4	0.000	0.000	0.000	0.000
5	0.000	0.000	0.000	0.000
tot	0.000	0.000	0.000	0.000

► 代码清单 A.20.4 (shell, 42 行)

```

SYSTEM = MOPTSNS
LREAL= Auto
ALGO= Normal
#ISEARCH=1
PREC=High

ISTART = 1
ICHARG = 11 #=11 用于电子能带和态密度计算，此过程中电荷密度不变

ENCUT =520
NCORE= 4

ISMEAR = 0
SIGMA = 0.04
GGA=PE

LSORBIT=.TRUE.
LORBMOM=.TRUE.

LWAVE=.F;
LCHARG=.F
EDIFF = 1E-6
EDIFFG = -0.01
ISYM = 0

NBANDS=64
LDAU=.T
LDAUTYPE=2
LDAUL=2 -1 -1 -1      # 库伦排斥的轨道，对应元素
LDAUU=1.5 0.0 0.0 0.0  # 几个元素几个数
LDAUJ=0.0 0.0 0.0 0.0  #stoner 交换参数大小
LDAUPRINT = 2
LORBIT=11             # 常用 =11, 输出包含 IM 投影的 PROCAR 和 DOSCAR

ISPIN=2
MAGMOM = 0 0 4 0 0 -4 9*0
LMAXMIX=4

AMIX      = 0.01
BMIX      = 0.0001
AMIX_MAG  = 0.8
BMIX_MAG  = 0.0001

```

► 代码清单 A.20.5 (shell, 15 行)

K-Path Generated by VASPKIT.

```

20
Line-Mode
Reciprocal
0.0000000000 0.0000000000 0.0000000000 GAMMA
0.5000000000 0.0000000000 0.0000000000 X

0.5000000000 0.0000000000 0.0000000000 X
0.5000000000 0.5000000000 0.0000000000 M

0.5000000000 0.5000000000 0.0000000000 M
0.0000000000 0.5000000000 0.0000000000 Y

0.0000000000 0.5000000000 0.0000000000 Y
0.0000000000 0.0000000000 0.0000000000 GAMMA

```

► 代码清单 A.20.6 (shell, 54 行)

```

SYSTEM = MOPTSNS
LREAL= .F
ALGO= Normal
#ISEARCH=1
PREC=High

ISTART = 0
ICHARG = 2

ENCUT = 520
NCORE= 4
ISYM=2

ISMear =0
SIGMA = 0.04

NSW = 0
ISIF = 2
IBRION =-1
LWAVE=.F
LCHARG=.T
EDIFF = 1E-6
EDIFFG = -0.01

NELM=100

LDAU=.T
LDAUTYPE=2
LDAUL=2 -1 -1 -1 # 库伦排斥的轨道, 对应元素
LDAUU=1.5 0.0 0.0 0.0 # 几个元素几个数
LDAUJ=0.0 0.0 0.0 0.0 #stoner 交换参数大小
LDAUPRINT = 2

ISPIN=2
MAGMOM = 0 0 4 0 0 -4 9*0
LMAXMIX=4

LSORBIT=.T
LORBMOM=.TRUE.
LORBIT=11

LWANNIER90=.T
NBANDS=64
NUM_WANN=38

WANNIER90_WIN=""

```



```
kmesh_tol=0.00001
num_wann=38
"
```

```
AMIX      = 0.1
BMIX      = 0.0001
AMIX_MAG  = 0.8
BMIX_MAG  = 0.0001
```

► 代码清单 A.20.7 (shell, 10 行)

```
```shell

z2 number for 6 planes
k1=0.0, k2-k3 plane: 0
k1=0.5, k2-k3 plane: 0
k2=0.0, k1-k3 plane: 0
k2=0.5, k1-k3 plane: 0
k3=0.0, k1-k2 plane: 1
k3=0.5, k1-k2 plane: 1
Time cost for Z2_calc is about 24.755 s
```

## pyw90 的测试用例

### ► 代码清单 A.21.2 (shell, 8 行)

```
conda activate wannier-env

pip install pywannier90 -i https://pypi.tuna.tsinghua.edu.cn/simple

source pyw90-env/bin/activate
conda activate pyw90_env

pip install -i https://pypi.tuna.tsinghua.edu.cn/simple pywannier90
```

### ► 代码清单 A.21.3 (shell, 15 行)

```
examples/GaAs
├── bnd
│ ├── INCAR
│ ├── KPOINTS
│ └── POSCAR
├── wannier <-- Current working dir
│ ├── afolder
│ ├── wannier90_input.win
│ └── wannier90.win
├── GaAs_mp-2534_primitive.cif
├── INCAR
└── KPOINTS
```

---

```
|─ POSCAR
|─ run.script
|─ vasprun.xml
```

### ► 代码清单 A.21.4 (shell, 1 行)

```
pyw90 eig dist --path ./
```

### ► 代码清单 A.21.5 (shell, 23 行)

```
Calculated Energy Range: -1000.0, 1000.0 with Fermi level -1.062223
EFERMI: -1.062223
```

```

Band No. EMIN EMAX

0~ 1 -57.17729 -57.16076
2~ 3 -56.71744 -56.70345
4~ 5 -34.73038 -34.52053
6~ 7 -34.46820 -34.32246
8~ 9 -32.92932 -32.67502
10~ 11 -32.63085 -32.38184
12~ 13 -32.28038 -32.15578
14~ 15 -32.00534 -31.79921
16~ 17 -22.00339 -20.63706
18~ 19 -14.49020 -13.94002
20~ 21 -13.71047 -13.59545
22~ 27 -9.34939 -6.48335
28~ 29 -6.25174 -5.36166
30~ 31 -5.34394 -4.75866
32~ 39 -4.74306 -2.65443
40~ 43 -2.61865 -1.10017
44~ 63 -1.02199 +5.69667

```

### ► 代码清单 A.21.6 (shell, 1 行)

```
pyw90 eig suggest --path ./
```

### ► 代码清单 A.21.7 (shell, 24 行)

```
Calculated Energy Range: -1000.0, 1000.0 with Fermi level -1.062223
There are at most 64 states and at least 64 states in [-1000.0, 1000.0].
```

```
`dis_win_min` and `dis_win_max` Table:
 Column `dis_win_max` shows the **lowest** dis_win_max for `dis_win_min`
 Column `i+1_min` / `i_max` shows the band minimum / maximum near the gap
```

Column `Nleast` / `Nmost` shows the least / most number of states inside `dis\_win\_min` and Fermi level.

	dis_win_min	i+l_min	i_max	Nleast	Nmost
14	-2.636541	-2.618654	-2.654429	4	4
13	-4.750860	-4.743057	-4.758664	12	12
12	-5.352798	-5.343936	-5.361660	14	14
11	-6.367546	-6.251741	-6.483351	16	16
10	-11.472421	-9.349394	-13.595448	22	22
9	-13.825247	-13.710470	-13.940024	24	24
8	-17.563633	-14.490202	-20.637063	26	26
7	-26.901300	-22.003393	-31.799208	28	28
6	-32.080560	-32.005339	-32.155781	30	30
5	-32.331113	-32.280383	-32.381843	32	32
4	-32.652936	-32.630852	-32.675020	34	34
3	-33.625889	-32.929317	-34.322462	36	36
2	-34.494366	-34.468205	-34.520528	38	38
1	-45.716912	-34.730376	-56.703448	40	40
0	-56.939097	-56.717439	-57.160755	42	42

## ► 代码清单 A.21.8 (shell, 2 行)

```
pyw90 pre dos --path .. -e -1 1 --rm-fermi --plot
pyw90 pre dos --path ./ -e -2.5 2.5 --rm-fermi --plot # 当前目录
```

## ► 代码清单 A.21.9 (shell, 78 行)

```
Reading vasprun.xml file from
`/public/home/cssong/song/lmr/17_AL_stack/26_Nb20Se2_better/U=1.5/monolayer/new_wannier/1_k=11_wannier/1_sta
↪ tic_ncl/选取合适的轨道/2_bandsoc/vasprun.xml`
for DOS analysis...
Fermi level : -1.06222 eV
DOS Gap : 0.00000 eV

Calculated DOS Energy Range: -2.06222313, -0.062223130000000015
```

	species	structure_id	orb_id	orb_name	key_string	dos
6	Nb	0	6	dz2	Nb_0_dz2	1.000000
15	Nb	1	6	dz2	Nb_1_dz2	0.999915
13	Nb	1	4	dxy	Nb_1_dxy	0.465896
4	Nb	0	4	dxy	Nb_0_dxy	0.465808
14	Nb	1	5	dyz	Nb_1_dyz	0.462808
7	Nb	0	7	dxz	Nb_0_dxz	0.462613
8	Nb	0	8	dx2	Nb_0_dx2	0.217618
17	Nb	1	8	dx2	Nb_1_dx2	0.217514
38	0	4	2	pz	0_4_pz	0.171881
29	Se	3	2	pz	Se_3_pz	0.141651
20	Se	2	2	pz	Se_2_pz	0.092454
19	Se	2	1	py	Se_2_py	0.085562
21	Se	2	3	px	Se_2_px	0.085500
30	Se	3	3	px	Se_3_px	0.079113
28	Se	3	1	py	Se_3_py	0.079034
39	0	4	3	px	0_4_px	0.046104
37	0	4	1	py	0_4_py	0.046043
0	Nb	0	0	s	Nb_0_s	0.042437
9	Nb	1	0	s	Nb_1_s	0.042437
18	Se	2	0	s	Se_2_s	0.022024

27	Se	3	0	s	Se_3_s	0.017021
22	Se	2	4	dxy	Se_2_dxy	0.012715
24	Se	2	6	dz2	Se_2_dz2	0.011391
26	Se	2	8	dx2	Se_2_dx2	0.011267
35	Se	3	8	dx2	Se_3_dx2	0.009700
33	Se	3	6	dz2	Se_3_dz2	0.008872
12	Nb	1	3	px	Nb_1_px	0.007934
1	Nb	0	1	py	Nb_0_py	0.007934
5	Nb	0	5	dyz	Nb_0_dyz	0.006255
16	Nb	1	7	dxz	Nb_1_dxz	0.006255
10	Nb	1	1	py	Nb_1_py	0.005645
3	Nb	0	3	px	Nb_0_px	0.005645
23	Se	2	5	dyz	Se_2_dyz	0.004173
25	Se	2	7	dxz	Se_2_dxz	0.004173
31	Se	3	4	dxy	Se_3_dxy	0.004089
34	Se	3	7	dxz	Se_3_dxz	0.003537
32	Se	3	5	dyz	Se_3_dyz	0.003537
36	O	4	0	s	O_4_s	0.002686
2	Nb	0	2	pz	Nb_0_pz	0.000509
11	Nb	1	2	pz	Nb_1_pz	0.000497
40	O	4	4	dxy	O_4_dxy	0.000000
41	O	4	5	dyz	O_4_dyz	0.000000
42	O	4	6	dz2	O_4_dz2	0.000000
43	O	4	7	dxz	O_4_dxz	0.000000
44	O	4	8	dx2	O_4_dx2	0.000000

Based on your input, set the lower selection bound to 0.1 and 10 orbitals are selected.  
Number of WFs selected: 10 (with degeneracy 1)

Orbitals Selected:

	species	site	orb
0	Nb	0	[dxy, dz2, dxz, dx2]
1	Nb	1	[dxy, dyz, dz2, dx2]
2	Se	3	[pz]
3	O	-1	[pz]

Wannier90 Projection:

f=0.000001,0.500000,0.503618:dxy;dz2;dxz;dx2-y2

f=0.500000,0.999999,0.503621:dxy;dyz;dz2;dx2-y2

f=0.999999,0.000001,0.396013:pz

O:pz

pyw90 --extra input:

Nb,0,4|6-8;Nb,1,4-6|8;Se,3,2;O,4,2

Plotted with selected orbitals in `brown` and non-selected orbitals in `orange`.

Plotted with a total of 24 orbitals, 10 of which are selected.

Figure should be stored at /public/home/cssong/song/lmr/17\_AL\_stack/26\_Nb20Se2\_better/U=1.5/monolayer/new\_wannier\_

→ r/l\_k=11\_wannier/1\_static\_ncl/选取合适的轨道/2\_bandsoc/dos\_analysis.pdf

## ► 代码清单 A.21.10 (shell, 1 行)

```
pyw90 pre dos --path ./ -e -2 2 --rm-fermi --plot
```

## ► 代码清单 A.21.11 (shell, 76 行)

Reading vasprun.xml file from

`/public/home/cssong/song/lmr/17\_AL\_stack/26\_Nb20Se2\_better/U=1.5/monolayer/new\_wannier/1\_k=11\_wannier/1\_sta\_

→ tic\_ncl/选取合适的轨道/2\_bandsoc/vasprun.xml`

for DOS analysis...

Fermi level : -1.06222 eV

DOS Gap : 0.00000 eV

Calculated DOS Energy Range: -3.06222313, 0.93777687

	species	structure_id	orb_id	orb_name	key_string	dos
14	Nb	1	5	dyz	Nb_1_dyz	1.000000
7	Nb	0	7	dxz	Nb_0_dxz	0.999150
15	Nb	1	6	dz2	Nb_1_dz2	0.957923
6	Nb	0	6	dz2	Nb_0_dz2	0.957750
13	Nb	1	4	dxy	Nb_1_dxy	0.490933
4	Nb	0	4	dxy	Nb_0_dxy	0.490911
17	Nb	1	8	dx2	Nb_1_dx2	0.303780
8	Nb	0	8	dx2	Nb_0_dx2	0.303752
16	Nb	1	7	dxz	Nb_1_dxz	0.287098
5	Nb	0	5	dyz	Nb_0_dyz	0.287097
38	O	4	2	pz	O_4_pz	0.216118
30	Se	3	3	px	Se_3_px	0.165473
28	Se	3	1	py	Se_3_py	0.165409
19	Se	2	1	py	Se_2_py	0.148639
21	Se	2	3	px	Se_2_px	0.148499
29	Se	3	2	pz	Se_3_pz	0.098211
37	O	4	1	py	O_4_py	0.081851
39	O	4	3	px	O_4_px	0.081809
20	Se	2	2	pz	Se_2_pz	0.074695
0	Nb	0	0	s	Nb_0_s	0.032390
9	Nb	1	0	s	Nb_1_s	0.032390
26	Se	2	8	dx2	Se_2_dx2	0.021172
35	Se	3	8	dx2	Se_3_dx2	0.018524
12	Nb	1	3	px	Nb_1_px	0.012137
1	Nb	0	1	py	Nb_0_py	0.012137
18	Se	2	0	s	Se_2_s	0.011819
27	Se	3	0	s	Se_3_s	0.010019
22	Se	2	4	dxy	Se_2_dxy	0.010018
3	Nb	0	3	px	Nb_0_px	0.009842
10	Nb	1	1	py	Nb_1_py	0.009842
24	Se	2	6	dz2	Se_2_dz2	0.008653
31	Se	3	4	dxy	Se_3_dxy	0.006800
33	Se	3	6	dz2	Se_3_dz2	0.006723
25	Se	2	7	dxz	Se_2_dxz	0.006332
23	Se	2	5	dyz	Se_2_dyz	0.006332
32	Se	3	5	dyz	Se_3_dyz	0.005652
34	Se	3	7	dxz	Se_3_dxz	0.005652
36	O	4	0	s	O_4_s	0.005392
2	Nb	0	2	pz	Nb_0_pz	0.002285
11	Nb	1	2	pz	Nb_1_pz	0.002281
40	O	4	4	dxy	O_4_dxy	0.000000
41	O	4	5	dyz	O_4_dyz	0.000000
42	O	4	6	dz2	O_4_dz2	0.000000
43	O	4	7	dxz	O_4_dxz	0.000000
44	O	4	8	dx2	O_4_dx2	0.000000

Based on your input, set the lower selection bound to 0.1 and 15 orbitals are selected.

Number of WFs selected: 15 (with degeneracy 1)

Orbitals Selected:

	species	site	orb
0	Nb	-1	[d]
1	Se	-1	[py, px]
2	O	-1	[pz]

Wannier90 Projection:

Nb:l=2

Se:py;px

O:pz

pyw90 --extra input:

Nb,0-1,4-8;Se,2-3,1|3;0,4,2

Plotted with selected orbitals in `brown` and non-selected orbitals in `orange`.

Plotted with a total of 28 orbitals, 15 of which are selected.

Figure should be stored at /public/home/cssong/song/lmr/17\_AL\_stack/26\_Nb20Se2\_better/U=1.5/monolayer/new\_wannier/

↪ r/l\_k=11\_wannier/1\_static\_ncl/选取合适的轨道/2\_bandsoc/dos\_analysis.pdf

## ► 代码清单 A.21.12 (shell, 1 行)

```
pyw90 pre dos --path ./ -e -4.750 10 --extra 'Nb,1-2,4-8;Se,1-2,1-2;0,1,2'
```

## ► 代码清单 A.21.13 (shell, 35 行)

Reading vasprun.xml file from

```
`/public/home/cssong/song/lmr/17_AL_stack/26_Nb20Se2_better/U=1.5/monolayer/new_wannier/1_k=11_wannier/1_sta`
```

↪ tic\_ncl/未选择详细的轨道划分/2\_bandsoc/vasprun.xml`

for DOS analysis...

Fermi level : -1.06222 eV

DOS Gap : 0.00000 eV

Calculated DOS Energy Range: -4.75, 10.0

/public/home/cssong/.conda/envs/pyw90\_env/lib/python3.10/site-packages/pyw90/lib/dos.py:294: UserWarning: CHECK

↪ YOUR INPUT! The energies in `EIGENVAL` is ranged from -60.321 to 8.8404, which does not include all the energy

↪ ranged from -4.75 to 10.0 you input.

warnings.warn(f'CHECK YOUR INPUT! The energies in `EIGENVAL` is ranged from {ee.min()} to {ee.max()}, ' \

Dis frozen window table

The table is sorted according to the percentage of pdos/tdos.

If you want to see `dis\_win\_min(max)` suggestion, please use `pyw90 eig suggest` menu.

	dis_froz_min	dis_froz_max	N	pdos	tdos	percent
12	-1.358146	2.597391	15	5.125143	15.016646	0.341297
11	-1.358146	2.596727	15	5.120804	15.004925	0.341275
8	-2.650429	1.755243	15	4.246392	12.995824	0.326751
5	-3.083025	1.135878	14	2.551449	7.878292	0.323858
9	-1.531936	2.247293	15	2.798732	8.641846	0.323858
6	-3.082851	1.208709	15	2.315049	7.186904	0.322120
10	-1.463430	2.336010	15	2.849126	8.861746	0.321508
14	-0.112658	2.983559	15	3.492113	11.016557	0.316988
7	-2.706902	1.633854	15	3.199764	10.100167	0.316803
13	-0.202382	2.914169	15	3.215031	10.281857	0.312690
15	0.766397	3.448345	15	2.469691	8.225589	0.300245
16	0.893846	3.464538	15	2.465737	8.294854	0.297261
4	-3.710088	0.909337	15	6.145274	22.344695	0.275022
17	1.111615	6.000000	14	2.832842	10.665791	0.265601
1	-4.116381	0.216475	15	2.340062	9.284414	0.252042
3	-3.844320	0.681424	15	3.660646	14.722510	0.248643
2	-4.095046	0.369728	15	2.060050	8.285811	0.248624
0	-4.754664	-0.099691	15	3.605248	16.794415	0.214669

Lowest `dis\_win\_max` for -4.75: 0.766397

---

### ► 代码清单 A.21.14 (shell, 12 行)

四、轨道编号在 pyw90 中的固定对应 (兼容 VASP 6.4.3)

编号 轨道类型 对应 VASP 通道 pyw90/ pymatgen 轨道名

```
0 s s s
1 p py py
2 p pz pz
3 p px px
4 d dxy dxy
5 d dyz dyz
6 d dz2 dz2
7 d dxz dxz
8 d dx2-y2 dx2-y2
9-15 f f1-f7 f 轨道组
```

### ► 代码清单 A.21.15 (shell, 12 行)

```
num_wann = 30
!num_bands = 15

!exclude_bands : 1-5

begin projections
Nb:l=2
Se:py;px
0:pz
end projections

spinors = .true.
```

### ► 代码清单 A.21.16 (shell, 12 行)

```
kmesh_tol=0.00001

begin projections
Nb:l=2
Se:py;px
0:pz
end projections

dis_froz_min = -1.358
dis_froz_max = 2.597
dis_win_min = -4.75
dis_win_max = 0.766397
```

### ► 代码清单 A.21.17 (shell, 1 行)

```
pyw90 auto run
```

---

### ► 代码清单 A.21.18 (shell, 2 行)

```
kernal function used to evalute the quality of Wannier90 band
kernel: [unit, 0, 1]
```

### ► 代码清单 A.21.19 (shell, 2 行)

```
kernal function used to evalute the quality of Wannier90 band
kernel: [gaussian, 0, 2]
```

### ► 代码清单 A.21.20 (shell, 42 行)

```
#!/bin/bash
#PBS -l nodes=node15:ppn=64
#PBS -l walltime=24:00:00
#PBS -j oe
#PBS -q sugon
#PBS -N w90

cd $PBS_O_WORKDIR

echo "-----"
echo "Wannier90 job started at: $(date)"
echo "Job ID: $PBS_JOBID"
echo "Working directory: $(pwd)"
echo "-----"

模块加载
source /public/software/profile.d/compiler_intel-compiler-2021.3.0.sh
source /public/software/profile.d/mpi_intelmpi-2021.3.0.sh

Conda 环境激活 (请确认路径)
source /public/software/profile.d/apps_anaconda3-2021.05.sh
conda activate pyw90_env

确认环境加载正确
echo "Python path: $(which python)"
python -c "import pyw90; print('Pyw90 path:', pyw90.__file__)"

运行 W90
EXE=/public/home/xpwu/pack/wannier90-3.1.0/bin/wannier90.x
SYSTEM="wannier90"

NP=$(cat $PBS_NODEFILE | wc -l)
cat $PBS_NODEFILE | uniq -c | awk '{ printf("%s:%s\n", $2, $1); }' > PBS_JOBID-PBS_JOBCOOKIE.hosts

echo "Running Wannier90 on $NP cores..."
mpirun -np $NP $EXE $SYSTEM > wannier90.log

echo "-----"
echo "Wannier90 job finished at: $(date)"
echo "-----"

rm -f PBS_JOBID-PBS_JOBCOOKIE.hosts
```



---

### ► 代码清单 A.21.21 (shell, 1 行)

```
pyw90 auto run
```

### ► 代码清单 A.21.22 (shell, 1 行)

```
pyw90 cmp wannier90 --fermi -0.9931 --kernel gaussian,0,2
```

### ► 代码清单 A.21.23 (shell, 1 行)

```
pyw90 pre band
```

## pyw90 的测试用例 2\_本地运行

### ► 代码清单 A.22.1 (shell, 20 行)

```
File location
#runfile: w90.script
vasp_band_file: bnd.dat
seedname: wannier90

system
jobname: w90
username: xpwu

run at the local machine not via SLURM
! YOU HAVE TO KNOWN WHAT YOU ARE DOING
#
If you have any question, please ask the administrator of your server. DO AT YOUR OWN RISK
If you choose to run locally, you don't need to give `runfile` value
how to run your job locally. Only used when `local` is `True`.
local: True

! IMPORTANT: Make sure you have added `wannier90.x` to your `$PATH` if you want to run as following
single core
localrun: wannier90.x wannier90
```

### ► 代码清单 A.22.2 (shell, 1 行)

```
pyw90 pre band
```

---

► 代码清单 A.22.3 (shell, 1 行)

```
pyw90 auto run
```

► 代码清单 A.22.4 (shell, 1 行)

```
pyw90 cmp wannier90 --efermi -0.9931 --kernel gaussian,0,2
```

► 代码清单 A.22.5 (shell, 1 行)

```
pyw90 pre band
```

► 代码清单 A.22.6 (shell, 1 行)

```
pyw90 auto term
```

► 代码清单 A.22.7 (shell, 1 行)

```
kill 任务号
```

## Wannier Tools 的代码解释

► 代码清单 A.23.1 (fortran, 2051 行)

```
! 子程序: 计算单个 k 点的 Berry 曲率 (基于费米能级)
subroutine Berry_curvature_singlek_EF(k, mu, Omega_x, Omega_y, Omega_z)
 !> 计算单个 k 点的 Berry 曲率
 !> 费米分布由费米能级 iso_energy 决定
 !> 参考文献: Physical Review B 74, 195118(2006)
 !> 公式 (34)
 !> 作者: Quansheng Wu @ ETHZ, 2018 年 6 月 11 日
 !> 尝试编写一些简单的子程序
 !> 输出中只有实部有意义
 ! 版权所有 (c) 2018 QuanSheng Wu

 use wmpi ! 使用 MPI 并行模块
 use para ! 使用参数模块
```

```

implicit none ! 强制显式声明变量

!> 输入参数, k 以倒格矢为单位
!> mu: 化学势, 可通过背栅调节
real(dp), intent(in) :: k(3), mu
complex(dp), intent(out) :: Omega_x(Num_wann) ! x 分量 Berry 曲率
complex(dp), intent(out) :: Omega_y(Num_wann) ! y 分量 Berry 曲率
complex(dp), intent(out) :: Omega_z(Num_wann) ! z 分量 Berry 曲率

! 局部变量声明
integer :: m, n, i
real(dp), allocatable :: W(:) ! 本征值数组
real(dp) :: beta_fake ! 假想的 beta 参数 (用于费米分布)
complex(dp), allocatable :: Amat(:, :) ! 临时矩阵
complex(dp), allocatable :: DHDk(:, :, :) ! 哈密顿量对 k 的导数
complex(dp), allocatable :: DHDkdag(:, :, :) ! DHDk 的共轭转置
complex(dp), allocatable :: UU(:, :) ! 本征向量矩阵
complex(dp), allocatable :: UU_dag(:, :) ! UU 的共轭转置
complex(dp), allocatable :: Hamk_bulk(:, :) ! 体态哈密顿量
complex(dp), allocatable :: velocity_wann(:, :, :) ! Wannier 速度算符
complex(dp), allocatable :: velocity_Ham(:, :, :) ! 希尔伯特空间速度算符

!> 费米-狄拉克分布函数
real(dp), external :: fermi

! 分配内存并初始化
allocate(W(Num_wann))
allocate(velocity_wann(Num_wann, Num_wann, 3), velocity_Ham(Num_wann, Num_wann, 3))
allocate(UU(Num_wann, Num_wann), UU_dag(Num_wann, Num_wann), Hamk_bulk(Num_wann, Num_wann))
allocate(Amat(Num_wann, Num_wann), DHDk(Num_wann, Num_wann, 3), DHDkdag(Num_wann, Num_wann, 3))
W=0d0; velocity_wann= 0d0; UU= 0d0; UU_dag= 0d0; velocity_Ham= 0d0

! 通过 HmnR 的直接傅里叶变换计算体态哈密顿量
call ham_bulk_atomicgauge(k, Hamk_bulk)

!> 使用 LAPACK 的 zheev 进行对角化
UU=Hamk_bulk
call eigensystem_c('V', 'U', Num_wann, UU, W) ! 'V'= 计算本征向量, 'U'= 单位矩阵
!call zhpevx_pack(hamk_bulk,Num_wann, W, UU) ! 备用对角化方法

! 计算本征向量的共轭转置
UU_dag= conjg(transpose(UU))
! 计算原子规范下的速度算符
call dHdk_atomicgauge(k, velocity_wann)

!> 将速度算符变换到希尔伯特空间
do i=1, 3 ! 遍历三个方向 (x,y,z)
 ! velocity_wann x UU -> Amat
 call mat_mul(Num_wann, velocity_wann(:, :, i), UU, Amat)
 ! UU_dag x Amat -> velocity_Ham
 call mat_mul(Num_wann, UU_dag, Amat, velocity_Ham(:, :, i))
enddo

! 初始化 Berry 曲率为零
Omega_x=0d0; Omega_y=0d0; Omega_z=0d0

! 计算 Berry 曲率的主循环
do m= 1, Num_wann ! 遍历所有能带
 do n= 1, Num_wann ! 遍历所有能带
 ! 跳过简并能带
 if (abs(W(m)-W(n))<eps6) cycle

 ! 根据公式计算 Berry 曲率分量
 ! $\Omega_x = \sum_{n \neq m} [v_H^{\{y\}}_{\{n,m\}} \times v_H^{\{z\}}_{\{m,n\}}] / (E_m - E_n)^2$
 Omega_x(m)= Omega_x(m)+ velocity_Ham(n, m, 2)*velocity_Ham(m, n, 3)/((W(m)-W(n))**2)
 ! $\Omega_y = \sum_{n \neq m} [v_H^{\{z\}}_{\{n,m\}} \times v_H^{\{x\}}_{\{m,n\}}] / (E_m - E_n)^2$
 Omega_y(m)= Omega_y(m)+ velocity_Ham(n, m, 3)*velocity_Ham(m, n, 1)/((W(m)-W(n))**2)
 ! $\Omega_z = \sum_{n \neq m} [v_H^{\{x\}}_{\{n,m\}} \times v_H^{\{y\}}_{\{m,n\}}] / (E_m - E_n)^2$
 Omega_z(m)= Omega_z(m)+ velocity_Ham(n, m, 1)*velocity_Ham(m, n, 2)/((W(m)-W(n))**2)
 enddo
enddo

```

```

 enddo ! n 循环
 enddo ! m 循环

 ! 添加复数因子 -2i
 Omega_x= -Omega_x*2d0*zi
 Omega_y= -Omega_y*2d0*zi
 Omega_z= -Omega_z*2d0*zi

 !> 考虑费米分布的展宽效应
 if (Fermi_broadening<eps6) Fermi_broadening= eps6 ! 防止除零
 beta_fake= 1d0/Fermi_broadening ! 计算假想温度的倒数

 ! 应用费米分布权重
 do m= 1, Num_wann
 Omega_x(m)= Omega_x(m)*fermi(W(m)-mu, beta_fake) ! f(E_m - μ)
 Omega_y(m)= Omega_y(m)*fermi(W(m)-mu, beta_fake)
 Omega_z(m)= Omega_z(m)*fermi(W(m)-mu, beta_fake)
 enddo

 return
end subroutine Berry_curvature_singlek_EF

! 子程序: 计算单个 k 点的 Berry 曲率 (基于占据能带数, 适用于 slab 系统)
subroutine Berry_curvature_singlek_numoccupied_slab_total(k, Omega_z)
 !> 计算单个 k 点的 Berry 曲率
 !> 费米分布由占据能带数决定, 而非费米能级
 !> 参考文献: Physical Review B 74, 195118(2006)
 !> 公式 (11), 只计算 xy 分量
 !> 作者: Quansheng Wu, 2018 年 8 月 6 日
 ! 版权所有 (c) 2018 QuanSheng Wu

 use wmpi
 use para
 implicit none

 !> 输入参数, k 以倒格矢为单位 (二维 k 点)
 real(dp), intent(in) :: k(2)
 complex(dp), intent(out) :: Omega_z(1) ! 总 Berry 曲率

 ! 局部变量
 integer :: m, n, Mdim, i1, i2
 real(dp), allocatable :: W(:) ! 本征值数组
 complex(dp), allocatable :: Amat(:, :) ! 临时矩阵
 complex(dp), allocatable :: DHDk(:, :, :) ! 哈密顿量对 k 的导数
 complex(dp), allocatable :: DHDkdag(:, :, :) ! DHDk 的共轭转置
 complex(dp), allocatable :: UU(:, :) ! 本征向量矩阵
 complex(dp), allocatable :: UU_dag(:, :) ! UU 的共轭转置
 complex(dp), allocatable :: Hamk_slab(:, :) ! Slab 哈密顿量
 complex(dp), allocatable :: vx(:, :, :), vy(:, :, :) ! x 和 y 方向的速度算符
 complex(dp), allocatable :: Vij_x(:, :, :), Vij_y(:, :, :) ! 层间跃迁矩阵元

 ! Slab 系统的主导哈密顿量维度
 Mdim = Num_wann*Nslab ! 总维度 = 瓦尼尔能带数 $\times$ slab 层数

 ! 分配内存
 allocate(W(Mdim))
 allocate(vx(Mdim, Mdim), vy(Mdim, Mdim))
 allocate(UU(Mdim, Mdim), UU_dag(Mdim, Mdim), Hamk_slab(Mdim, Mdim))
 allocate(Amat(Mdim, Mdim), DHDk(Mdim, Mdim, 3), DHDkdag(Mdim, Mdim, 3))
 allocate(Vij_x(-ijmax:ijmax, Num_wann, Num_wann)) ! 层间跃迁矩阵元 (x 方向)
 allocate(Vij_y(-ijmax:ijmax, Num_wann, Num_wann)) ! 层间跃迁矩阵元 (y 方向)
 W=0d0; vx= 0d0; vy= 0d0; UU= 0d0; UU_dag= 0d0

 ! 计算层间跃迁速度矩阵元
 call ham_qlayer2qlayer_velocity(k, Vij_x, Vij_y)

 ! 构建完整的 slab 速度算符矩阵
 do i1=1, nslab ! 源层
 ! i2 目标层索引

```

```

do i2=1, nslab
 ! 只考虑近邻层间耦合
 if (abs(i2-i1).le.ijmax)then
 ! 填充 x 方向速度算符块
 vx((i2-1)*Num_wann+1:(i2-1)*Num_wann+Num_wann,&
 (i1-1)*Num_wann+1:(i1-1)*Num_wann+Num_wann)&
 = Vij_x(i2-i1,1:Num_wann,1:Num_wann)
 ! 填充 y 方向速度算符块
 vy((i2-1)*Num_wann+1:(i2-1)*Num_wann+Num_wann,&
 (i1-1)*Num_wann+1:(i1-1)*Num_wann+Num_wann)&
 = Vij_y(i2-i1,1:Num_wann,1:Num_wann)
 endif
enddo ! i2 循环
enddo ! i1 循环

! 通过直接傅里叶变换计算 slab 哈密顿量
call ham_slab(k, Hamk_slab)

!> 对角化哈密顿量
UU=Hamk_slab
call eigensystem_c('V', 'U', Mdim, UU, W) ! 计算本征值和本征向量

! 计算本征向量的共轭转置
UU_dag= conjg(transpose(UU))

!> 将速度算符变换到希尔伯特空间
call mat_mul(Mdim, vx, UU, Amat)
call mat_mul(Mdim, UU_dag, Amat, vx) ! vx 在能量本征基下表示
call mat_mul(Mdim, vy, UU, Amat)
call mat_mul(Mdim, UU_dag, Amat, vy) ! vy 在能量本征基下表示

! 初始化 Berry 曲率
Omega_z=0d0

! 计算总 Berry 曲率: 只对占据态到未占据态求和
do m= 1, NumOccupied*Nslab ! 占据态 (价带)
 do n= NumOccupied*Nslab+1, Mdim ! 未占据态 (导带)
 ! $\Omega_z = \sum_{m \in \text{occ}, n \in \text{unocc}} [v_x^{n,m} \times v_y^{m,n}] / (E_m - E_n)^2$
 Omega_z(1)= Omega_z(1)+ vx(n, m)*vy(m, n)/((W(m)-W(n))**2)
 enddo ! m 循环
enddo ! n 循环

! 添加复数因子并转换单位
Omega_z= -aimag(Omega_z*2d0)/Angstrom2atomic**2 ! 取虚部并转换单位

return
end subroutine Berry_curvature_singlek_numoccupied_slab_total

! 其他子程序的注释类似, 主要区别在于:
! 1. Berry_curvature_singlek_numoccupied_total: 计算总 Berry 曲率 (基于占据能带数)
! 2. Berry_curvature_singlek_numoccupied: 计算每个能带的 Berry 曲率 (基于占据能带数)
! 3. Berry_curvature_singlek_allbands: 使用 Kubo 公式计算所有能带的 Berry 曲率
! 4. orbital_magnetization_singlek_allbands: 计算轨道磁化强度
! 5. get_Vmn_Ham_nondiag: 获取非对角速度矩阵元
! 6. get_Dmn_Ham: 定义 D 矩阵 (Berry 连接)
! 7. Berry_curvature_slab: 计算 slab 系统的 Berry 曲率并输出结果

! 主程序部分: 计算 slab 系统的 Berry 曲率
subroutine Berry_curvature_slab
 !> 计算 Berry 曲率
 !> 参考文献: Physical Review B 74, 195118(2006)
 !> 作者: Quansheng Wu @ EPFL, 2018 年 8 月 6 日
 ! 版权所有 (c) 2018 QuanSheng Wu

 use wmpi
 use para
 implicit none

 ! MPI 和循环变量

```

```

integer :: ik, ierr, Mdim, i, j

! k 点坐标
real(dp) :: k(2)

!> k 点网格
real(dp), allocatable :: k12(:, :) ! 分数坐标 k 点
real(dp), allocatable :: k12_xyz(:, :) ! 笛卡尔坐标 k 点

real(dp), external :: norm ! 范数函数

!> Berry 曲率数组
complex(dp), allocatable :: Omega_z(:) ! 每个 k 点的 Berry 曲率
complex(dp), allocatable :: Omega(:) ! 每个 k 点的总 Berry 曲率
complex(dp), allocatable :: Omega_mpi(:) ! MPI 归约后的 Berry 曲率

! Slab 系统维度
Mdim = Num_wann*Nslab

! 分配内存
allocate(k12(2, Nk1*Nk2)) ! 2 维 k 点网格 (分数坐标)
allocate(k12_xyz(2, Nk1*Nk2)) ! 笛卡尔坐标 k 点
allocate(Omega_z(Mdim)) ! Berry 曲率数组
allocate(Omega (Nk1*Nk2)) ! 总 Berry 曲率
allocate(Omega_mpi(Nk1*Nk2)) ! MPI 归约结果
k12=0d0
k12_xyz=0d0
omega= 0d0
omega_mpi= 0d0

!> 构建 k 点网格, 以 K2d_start 为中心
ik=0
do i= 1, nk1 ! k1 方向循环
 do j= 1, nk2 ! k2 方向循环
 ik=ik+1
 ! 构建分数坐标 k 点网格
 k12(:, ik)=K2D_start+ (i-1)*K2D_vec1/dbl(nk1-1) &
 + (j-1)*K2D_vec2/dbl(nk2-1)- (K2D_vec1+K2D_vec2)/2d0
 ! 转换为笛卡尔坐标
 k12_xyz(:, ik)= k12(1, ik)* Ka2+ k12(2, ik)* Kb2
 enddo
enddo

! 并行计算每个 k 点的 Berry 曲率
do ik= 1+ cpuid, Nk1*Nk2, num_cpu ! CPU 负载均衡
 if (cpuid==0) write(stdout, *)'Berry curvature ik, nk1*nk2 ', ik, Nk1*Nk2

 !> 对角化哈密顿量
 k= k12(:, ik) ! 当前 k 点

 Omega_z= 0d0 ! 初始化

 ! 计算单个 k 点的 Berry 曲率
 call Berry_curvature_singlek_numoccupied_slab_total(k, Omega_z(1))
 ! 对所有能带求和得到总 Berry 曲率
 Omega(ik) = sum(Omega_z)

enddo ! ik 循环

! 初始化 MPI 归约数组
Omega_mpi= 0d0

#if defined (MPI)
! MPI 归约: 将所有处理器的结果求和
call mpi_allreduce(Omega,Omega_mpi,size(Omega_mpi),&
 mpi_dc,mpi_sum,mpi_cmw,ierr)
#else
! 串行版本: 直接复制
Omega_mpi= Omega

```

```

#endif

!> 输出 Berry 曲率到文件
outfileindex= outfileindex+ 1 ! 更新输出文件索引
if (cupid==0) then ! 只在主进程输出
 open(unit=outfileindex, file='Berrycurvature_slab.dat')
 write(outfileindex, '(20a28)')'# Unit of Berry curvature is Angstrom^2'
 write(outfileindex, '(20a28)')'# kx (1/A)', 'ky (1/A)', &
 'Omega_z'
 ik= 0
 do i= 1, nk1
 do j= 1, nk2
 ik= ik+ 1
 ! 写入 k 点和 Berry 曲率 (转换单位)
 write(outfileindex, '(20E28.10)')k12_xyz(:, ik)*Angstrom2atomic, &
 real(Omega_mpi(ik))/Angstrom2atomic**2
 enddo
 write(outfileindex, *) ' ' ! 空行分隔
 enddo

 close(outfileindex)
! 结束之前的 if 语句块
endif

!> 生成 gnuplot 脚本来绘制 Berry 曲率图
outfileindex= outfileindex+ 1 ! 增加输出文件索引
if (cupid==0) then ! 只在主进程执行

 ! 打开 gnuplot 脚本文件
 open(unit=outfileindex, file='Berrycurvature_slab.gnu')
 ! 设置字符编码
 write(outfileindex, '(a)')"set encoding iso_8859_1"
 ! 设置终端为 png 格式, 高分辨率
 write(outfileindex, '(a)')"#set terminal pngcairo truecolor enhanced size 1920, 1680 font ",60""
 write(outfileindex, '(a)')"set terminal png truecolor enhanced size 1920, 1680 font ",60""
 ! 设置输出文件名
 write(outfileindex, '(a)')"set output 'Berrycurvature_slab.png'"
 ! 设置调色板
 write(outfileindex, '(a)')"set palette rgbformulae 33,13,10"
 ! 取消 z 轴刻度
 write(outfileindex, '(a)')"unset ztics"
 ! 设置纵横比
 write(outfileindex, '(a)')"set size ratio -1"
 ! 设置图形大小
 write(outfileindex, '(a)')"set size 0.9, 0.95"
 ! 设置图形原点
 write(outfileindex, '(a)')"set origin 0.05, 0.02"
 ! 取消图例
 write(outfileindex, '(a)')"unset key"
 ! 启用 pm3d 绘图模式
 write(outfileindex, '(a)')"set pm3d"
 ! 注释掉的 z 轴和颜色条范围设置
 write(outfileindex, '(a)')"#set zbrange [-10: 10] "
 write(outfileindex, '(a)')"#set cbrange [-100: 100] "
 ! 设置视图为映射模式
 write(outfileindex, '(a)')"set view map"
 ! 设置边框线宽
 write(outfileindex, '(a)')"set border lw 3"
 ! 设置 x 轴标签
 write(outfileindex, '(a)')"set xlabel 'k (1/{\305})'"
 ! 设置 y 轴标签
 write(outfileindex, '(a)')"set ylabel 'k (1/{\305})'"
 ! 设置 x 轴范围自动
 write(outfileindex, '(a)')"set xrange [] noextend"
 ! 设置 y 轴范围自动
 write(outfileindex, '(a)')"set yrange [] noextend"
 ! 设置 y 轴刻度和样式
 write(outfileindex, '(a)')"set ytics 0.5 nomirror scale 0.5"
 ! 设置 pm3d 插值

```

```

 write(outfileindex, '(a)') "set pm3d interpolate 2,2"
 ! 设置标题
 write(outfileindex, '(a)') "set title 'Berry Curvature {/Symbol W}_z (Ang^2)'"
 ! 显示颜色条
 write(outfileindex, '(a)') "set colorbox"
 ! 绘制 3D 彩色图
 write(outfileindex, '(a)') "splot 'Berrycurvature_slab.dat' u 1:2:3 w pm3d"
 ! 关闭文件
 close(outfileindex)
 endif

#if defined (MPI)
 ! MPI 同步屏障
 call mpi_barrier(mpi_cmw, ierr)
#endif

 ! 释放内存
 deallocate(k12)
 deallocate(k12_xyz)
 deallocate(Omega_z)
 deallocate(Omega, Omega_mpi)

 return
end subroutine Berry_curvature_slab

! 子程序: 计算 k 路径上的 Berry 曲率 (基于占据能带)
subroutine Berry_curvature_line_occupied
!> 计算由 KPATH_BULK 定义的 k 线上的 Berry 曲率
!> 参考文献: Physical Review B 74, 195118(2006)
!> 公式 (9), 公式 (34)
!> 作者: Quansheng Wu @ EPFL, 2018 年 9 月 28 日
! 版权所有 (c) 2018 QuanSheng Wu

 use wmpi
 use para
 implicit none

 ! 局部变量
 integer :: ik, ierr, i
 real(dp) :: k(3), ol(3), k_cart(3) ! k 点坐标
 real(dp) :: time_start, time_end, time_start0, ybound_min, ybound_max ! 时间统计和边界
 real(dp), external :: norm ! 范数函数

 ! 分配内存: Berry 曲率数组 (每个能带)
 complex(dp), allocatable :: Omega_x(:), Omega_y(:), Omega_z(:)
 !> Berry 曲率数组 (3 个分量, k 点)
 real(dp), allocatable :: Omega(:, :), Omega_mpi(:, :)
 !> Berry 曲率数组 (3 个分量, 能带, k 点)
 real(dp), allocatable :: Omega_sep_bk(:, :), Omega_sep_bk_mpi(:, :)
 !> 能量本征值
 real(dp), allocatable :: eigv(:, :)
 real(dp), allocatable :: eigv_mpi(:, :)

 ! 分配内存空间
 allocate(Omega_x(Num_wann))
 allocate(Omega_y(Num_wann))
 allocate(Omega_z(Num_wann))
 allocate(eigv (Num_wann, nk3_band))
 allocate(eigv_mpi(Num_wann, nk3_band))
 allocate(Omega (3, nk3_band))
 allocate(Omega_mpi(3, nk3_band))
 omega= 0d0
 omega_mpi= 0d0

 ! 初始化时间统计
 time_start= 0d0
 time_start0= 0d0
 call now(time_start0) ! 获取开始时间

```



```

time_start= time_start0
time_end = time_start0

! 主循环: 遍历 k 点 (并行)
do ik= 1+ cpuid, nk3_band, num_cpu ! CPU 负载均衡
! 进度输出
if (cpuid==0.and. mod(ik/num_cpu, 100)==0) &
write(stdout, '(a, i9, " /", i10, a, f10.1, "s", a, f10.1, "s")') &
' Berry curvature: ik', ik, nk3_band, ' time left', &
(nk3_band-ik)*(time_end- time_start)/num_cpu, & ! 预估剩余时间
' time elapsed: ', time_end-time_start0 ! 已用时间

!> 分数坐标下的 k 点
k= kpath_3d(:, ik)

call now(time_start) ! 记录开始时间

! 初始化 Berry 曲率
Omega_x= 0d0
Omega_y= 0d0
Omega_z= 0d0

! 根据不同的计算方法选择不同的子程序
if (Berrycurvature_kpath_EF_calc) then
! 基于费米能级的计算
call Berry_curvature_singlek_EF(k, iso_energy, Omega_x, Omega_y, Omega_z)
else if (BerryCurvature_kpath_Occupied_calc) then
! 基于占据能带的计算 (只返回总和)
call Berry_curvature_singlek_numoccupied_total(k, Omega_x(1), Omega_y(1), Omega_z(1))
else
! 错误处理
write(*, *) 'ERROR: In subroutine Berry_curvature_line, we only support
↳ BerryCurvature_kpath_Occupied_calc '
write(*, *) ' and Berrycurvature_kpath_EF_calc '
stop
endif

! 对能带求和得到总 Berry 曲率
Omega(1, ik) = real(sum(Omega_x))
Omega(2, ik) = real(sum(Omega_y))
Omega(3, ik) = real(sum(Omega_z))
call now(time_end) ! 记录结束时间
enddo ! ik 循环

! 初始化 MPI 归约数组
Omega_mpi= 0d0

#if defined (MPI)
! MPI 归约: 求和所有处理器的计算结果
call mpi_allreduce(Omega, Omega_mpi, size(Omega_mpi), &
mpi_dp, mpi_sum, mpi_cmw, ierr)
#else
! 串行版本: 直接复制
Omega_mpi= Omega
#endif

!> 输出 Berry 曲率数据到文件
outfileindex= outfileindex+ 1
if (cpuid==0) then ! 主进程输出
open(unit=outfileindex, file='Berrycurvature_line.dat')
! 写入文件头信息
write(outfileindex, '(20a18)') '# Column 1 kpath with accumulated length in the kpath'
write(outfileindex, '(20a18)') '# Column 2-4 Berry curvature (Ang^2)'
write(outfileindex, '(20a18)') '# k (1/A)', &
'real(Omega_x)', 'real(Omega_y)', 'real(Omega_z)'

! 写入数据
do ik= 1, nk3_band
k=kpath_3d(:, ik)

```

```

! 写入累积 k 路径长度和 Berry 曲率分量 (转换单位)
write(outfileindex, '(20E18.8)')k3len(ik)*Angstrom2atomic, real(Omega_mpi(:, ik))/Angstrom2atomic**2
enddo

close(outfileindex)
endif

! 计算 y 轴显示范围
ybound_min= minval(real(Omega_mpi))-2
ybound_max= maxval(real(Omega_mpi))+5

!> 生成 gnuplot 脚本
outfileindex= outfileindex+ 1
if (cpuid==0) then
 open(unit=outfileindex, file='Berrycurvature_line.gnu')
 write(outfileindex, '(a)') "set encoding iso_8859_1"
 ! 设置 PDF 输出
 write(outfileindex, '(a)') 'set terminal pdf enhanced color font ",16"'
 write(outfileindex, '(a)') "set output 'Berrycurvature_line.pdf'"
 ! 设置绘图样式
 write(outfileindex, '(a)') 'set style data linespoints'
 write(outfileindex, '(a)') 'unset key' ! 取消图例
 write(outfileindex, '(a)') 'set pointsize 0.8'
 write(outfileindex, '(a)') 'set view 0,0' ! 2D 视图
 ! 设置 x 轴范围
 write(outfileindex, 202, advance="no") (k3line_name(i), k3line_stop(i)*Angstrom2atomic, i=1, nk3lines)
 write(outfileindex, 203)k3line_name(nk3lines+1), k3line_stop(nk3lines+1)*Angstrom2atomic

 ! 绘制高亮线和垂直标记
 do i=1, nk3lines-1
 if (index(Particle,'phonon')/=0) then
 ! 声子模式的范围设置
 write(outfileindex, 204)k3line_stop(i+1)*Angstrom2atomic, 0.0, k3line_stop(i+1)*Angstrom2atomic,
 ↪ ybound_max
 else
 ! 电子模式的范围设置
 write(outfileindex, 204)k3line_stop(i+1)*Angstrom2atomic, ybound_min,
 ↪ k3line_stop(i+1)*Angstrom2atomic, ybound_max
 endif
 enddo
enddo

! 绘制三条 Berry 曲率分量曲线
write(outfileindex, '(a)') "plot 'Berrycurvature_line.dat' \"
write(outfileindex, '(a)') "u 1:2 w lp lc rgb 'red' lw 2 pt 7 ps 0.2 title '{/Symbol W}_x', \"
write(outfileindex, '(a)') "" u 1:3 w lp lc rgb 'green' lw 2 pt 7 ps 0.2 title '{/Symbol W}_y', \"
write(outfileindex, '(a)') "" u 1:4 w lp lc rgb 'blue' lw 2 pt 7 ps 0.2 title '{/Symbol W}_z' \"
close(outfileindex)
endif

202 format('set xtics ('',20(''',A3,'',F10.5,'')) ! x 轴刻度格式
203 format(A3,'',F10.5,'') ! x 轴刻度结束格式
204 format('set arrow from ',F10.5,'',F10.5,' to ',F10.5,'',F10.5,' nohead') ! 箭头格式

#if defined (MPI)
! MPI 同步
call mpi_barrier(mpi_cmw, ierr)
#endif

! 释放内存
deallocate(Omega_x, Omega_y, Omega_z)
deallocate(Omega, Omega_mpi)

return
end subroutine Berry_curvature_line_occupied

! 子程序: 计算立方体格点中的 Berry 曲率
subroutine berry_curvature_cube

```

---

```

!> 计算在 KCUBE_BULK 定义的立方体内的 Berry 曲率
!> 参考文献: Physical Review B 74, 195118(2006)
!> 公式 (34)
!> 作者: Quansheng Wu @ EPFL, 2018 年 9 月 28 日
! 版权所有 (c) 2018 QuanSheng Wu

use wmpi
use para
implicit none

! 局部变量
integer :: ik, ierr, ikx, iky, ikz, n_kpoints, i, m, n
real(dp) :: k(3), o1(3), k_cart(3), emin, emax
real(dp) :: time_start, time_end, time_start0
real(dp), external :: norm

!> Berry 曲率和轨道磁化强度数组 (能带, 3 分量, k 点)
real(dp), allocatable :: Omega_allk(:, :, :), Omega_allk_mpi(:, :, :)
real(dp), allocatable :: m_OrbMag_allk(:, :, :), m_OrbMag_allk_mpi(:, :, :)
real(dp), allocatable :: Omega_BerryCurv(:, :, :), m_OrbMag(:, :, :)

!> Wannier 基下的速度矩阵
complex(dp), allocatable :: Vmn_wann(:, :, :)

!> 哈密顿量基下的速度矩阵
complex(dp), allocatable :: Vmn_Ham(:, :, :), Vmn_Ham_nondiag(:, :, :)
complex(dp), allocatable :: Dmn_Ham(:, :, :) ! Berry 连接矩阵

!> 哈密顿量、本征值和本征向量
complex(dp), allocatable :: UU(:, :, :)
real(dp), allocatable :: W(:)
real(dp), allocatable :: eigval_allk(:, :, :), eigval_allk_mpi(:, :, :)

! 根据计算模式确定 k 点总数
if (BerryCurvature_Cube_calc) then
 n_kpoints=Nk1*Nk2*Nk3 ! 立方体网格
else
 n_kpoints= nk3_band ! k 路径模式
endif

! 分配内存
allocate(Vmn_wann(Num_wann, Num_wann, 3), Vmn_Ham(Num_wann, Num_wann, 3))
allocate(Dmn_Ham(Num_wann,Num_wann,3), Vmn_Ham_nondiag(Num_wann, Num_wann, 3))
allocate(W(Num_wann))
allocate(UU(Num_wann, Num_wann))
allocate(eigval_allk(Num_wann, n_kpoints))
allocate(eigval_allk_mpi(Num_wann, n_kpoints))
allocate(Omega_BerryCurv(Num_wann, 3), m_OrbMag(Num_wann, 3))
allocate(Omega_allk (Num_wann, 3, n_kpoints))
allocate(Omega_allk_mpi(Num_wann, 3, n_kpoints))
allocate(m_OrbMag_allk (Num_wann, 3, n_kpoints))
allocate(m_OrbMag_allk_mpi(Num_wann, 3, n_kpoints))

! 初始化数组
Omega_BerryCurv= 0d0
m_OrbMag=0d0
Omega_allk= 0d0
eigval_allk= 0d0
m_OrbMag_allk=0d0
Omega_allk_mpi= 0d0
eigval_allk_mpi= 0d0
m_OrbMag_allk_mpi=0d0

! 时间统计初始化
time_start= 0d0
time_start0= 0d0
call now(time_start0)
time_start= time_start0
time_end = time_start0

```

```

! 主循环: 遍历所有 k 点
do ik= 1+ cpuid, n_kpoints, num_cpu
 ! 进度输出
 if (cpuid==0.and. mod(ik/num_cpu, 100)==0) &
 write(stdout, '(a, i9, " /", i10, a, f10.1, "s", a, f10.1, "s")') &
 ' Berry curvature: ik', ik, n_kpoints, ' time left', &
 (n_kpoints-ik)*(time_end- time_start)/num_cpu, &
 ' time elapsed: ', time_end-time_start0

 !> 根据计算模式生成 k 点
 if (BerryCurvature_Cube_calc) then
 !> 体布里渊区模式: 从三维网格索引计算 k 点坐标
 ikx= (ik-1)/(nk2*nk3)+1 ! x 方向索引
 iky= ((ik-1-(ikx-1)*Nk2*Nk3)/nk3)+1 ! y 方向索引
 ikz= (ik-(iky-1)*Nk3- (ikx-1)*Nk2*Nk3) ! z 方向索引
 ! 计算分数坐标 k 点
 k= K3D_start_cube+ K3D_vec1_cube*(ikx-1)/dble(nk1) &
 + K3D_vec2_cube*(iky-1)/dble(nk2) &
 + K3D_vec3_cube*(ikz-1)/dble(nk3)

 elseif (BerryCurvature_kpath_sepband_calc) then
 !> k 路径模式: 直接从预定义路径获取 k 点
 k= kpath_3d(:, ik)
 endif

 call now(time_start) ! 记录开始时间

 !> 通过 HmR 的直接傅里叶变换计算体态哈密顿量
 call ham_bulk_atomicgauge(k, UU)

 !> 对角化哈密顿量
 call eigensystem_c('V', 'U', Num_wann, UU, W)
 eigval_allk(:, ik) = W ! 保存本征值

 !> 在哈密顿量基下获取速度算符
 call dHdk_atomicgauge_Ham(k, UU, Vmn_Ham)

 ! 计算 Berry 连接矩阵和非对角速度矩阵元
 call get_Dmn_Ham(W, Vmn_Ham, Dmn_Ham)
 call get_Vmn_Ham_nondiag(Vmn_Ham, Vmn_Ham_nondiag)

 ! 计算 Berry 曲率和轨道磁化强度
 call Berry_curvature_singlek_allbands(Dmn_Ham, Omega_BerryCurv)
 call orbital_magnetization_singlek_allbands(Dmn_Ham, Vmn_Ham_nondiag, m_OrbMag)

 ! 保存结果
 Omega_allk(:, :, ik) = Omega_BerryCurv
 m_OrbMag_allk(:, :, ik) = m_OrbMag

 call now(time_end) ! 记录结束时间
enddo ! ik 循环

#if defined (MPI)
 ! MPI 归约所有结果
 call mpi_allreduce(Omega_allk,Omega_allk_mpi,size(Omega_allk_mpi),&
 mpi_dp,mpi_sum,mpi_cmw,ierr)
 call mpi_allreduce(eigval_allk,eigval_allk_mpi,size(eigval_allk_mpi),&
 mpi_dp,mpi_sum,mpi_cmw,ierr)
 call mpi_allreduce(m_OrbMag_allk,m_OrbMag_allk_mpi,size(m_OrbMag_allk_mpi),&
 mpi_dp,mpi_sum,mpi_cmw,ierr)
#else
 ! 串行版本
 Omega_allk_mpi= Omega_allk
 eigval_allk_mpi= eigval_allk
 m_OrbMag_allk_mpi= m_OrbMag_allk
#endif

! 立方体计算模式的输出部分

```

```

IF (BerryCurvature_Cube_calc) THEN
!> 输出 Berry 曲率和轨道磁化强度到 Fermisurfer 软件可读的文件
outfileindex= outfileindex+ 1
if (cpuid==0) then
 open(unit=outfileindex, file='Berrycurvature_norm.frmsf')
 ! 写入网格尺寸
 write(outfileindex, *) Nk1, Nk2, Nk3
 write(outfileindex, *) 1 ! 自旋极化数
 write(outfileindex, *) Num_wann ! 能带数
 ! 写入晶格矢量
 write(outfileindex, '(3f12.6)') Origin_cell%Kua
 write(outfileindex, '(3f12.6)') Origin_cell%Kub
 write(outfileindex, '(3f12.6)') Origin_cell%Kuc
 ! 写入能量本征值 (相对于费米能级)
 do m=1, Num_wann
 do ik= 1, n_kpoints
 write(outfileindex, '(E18.10)') eigval_allk_mpi(m, ik)-iso_energy
 enddo
 enddo
 ! 写入 Berry 曲率模长
 do m=1, Num_wann
 do ik= 1, n_kpoints
 o1= Omega_allk_mpi(m, :, ik)/Angstrom2atomic**2
 write(outfileindex, '(E18.10)') norm(o1)
 enddo
 enddo
 close(outfileindex)
endif

! 类似地输出 x 分量 Berry 曲率...
! [其余输出文件的代码类似, 分别输出不同分量和轨道磁化强度]

!> 轨道磁化强度输出 (x 分量)
outfileindex= outfileindex+ 1 ! 增加输出文件索引
if (cpuid==0) then ! 只在主进程执行
 ! 打开轨道磁化强度 x 分量文件 (Fermisurfer 格式)
 open(unit=outfileindex, file='Orbital_magnetization_x.frmsf')
 ! 写入网格尺寸
 write(outfileindex, *) Nk1, Nk2, Nk3
 write(outfileindex, *) 1 ! 自旋极化数
 write(outfileindex, *) Num_wann ! 能带数
 ! 写入倒格矢 (晶格矢量)
 write(outfileindex, '(3f12.6)') Origin_cell%Kua
 write(outfileindex, '(3f12.6)') Origin_cell%Kub
 write(outfileindex, '(3f12.6)') Origin_cell%Kuc
 ! 写入能量本征值 (相对于费米能级)
 do m=1, Num_wann
 do ik= 1, n_kpoints
 write(outfileindex, '(E18.10)') eigval_allk_mpi(m, ik)-iso_energy
 enddo
 enddo
 ! 写入轨道磁化强度 x 分量
 do m=1, Num_wann
 do ik= 1, n_kpoints
 o1= m_OrbMag_allk_mpi(m, :, ik) ! 当前能带和 k 点的轨道磁化强度矢量
 write(outfileindex, '(E18.10)') o1(1) ! 只写入 x 分量
 enddo
 enddo
 close(outfileindex) ! 关闭文件
endif

! 条件编译: 如果不是立方体计算模式
ELSE

!> 输出 Berry 曲率数据到文本文件 (分波带模式)
outfileindex= outfileindex+ 1
if (cpuid==0) then
 open(unit=outfileindex, file='Berrycurvature_line_sepband.txt')
 ! 写入文件头信息

```

```

write(outfileindex, '(a)')'# Column 1 kpath with accumulated length in the kpath, Coloum 2: energy'
write(outfileindex, '(a)')'# Column 2-4 Berry curvature (Ang^2)'
! 写入列号信息
write(outfileindex, "('#column', i5, 20i16)")(i, i=1, 8)
! 写入详细的列描述
write(outfileindex, '(20a16)')'# k (1/A)', " eig", &
'Omega_x', 'Omega_y', 'Omega_z', & ! Berry 曲率三个分量
'm_x', 'm_y', 'm_z' ! 轨道磁化强度三个分量
! 遍历所有能带和 k 点写入数据
do i=1, Num_wann ! 遍历能带
do ik=1, n_kpoints ! 遍历 k 点
! 写入 k 路径长度、能量、Berry 曲率、轨道磁化强度
write(outfileindex, '(200E16.6)')k3len(ik)*Angstrom2atomic, & ! k 路径累积长度
eigval_allk_mpi(i, ik)/eV2Hartree, & ! 能量 (转换为 eV)
Omega_allk_mpi(i, :, ik), & ! Berry 曲率矢量
m_OrbMag_allk_mpi(i, :, ik) ! 轨道磁化强度矢量
enddo
write(outfileindex, *)' ' ! 每个能带后空行分隔
enddo
close(outfileindex)
endif

! 注意: 这里有个重复的 close 语句, 可能是代码错误
close(outfileindex)
endif

!> 计算能量带的最小值和最大值
emin= minval(eigval_allk_mpi)/eV2Hartree-0.5d0 ! 最小值减去偏移量
emax= maxval(eigval_allk_mpi)/eV2Hartree+0.5d0 ! 最大值加上偏移量

!> 生成 gnuplot 脚本用于绘制分波带 Berry 曲率图
outfileindex= outfileindex+ 1
if (cpuid==0) then
open(unit=outfileindex, file='Berrycurvature_line_sepband.gnu')
! 写入 gnuplot 脚本头部信息
write(outfileindex, '(a)') '# requirement: gnuplot version>5.4'
write(outfileindex, '(2a)') '# Please open the data file to check the data:
↔ Berrycurvature_line_sepband.txt '
! 设置终端和输出
write(outfileindex, '(a)') 'set terminal pdf enhanced color font ",24"'
! 设置调色板
write(outfileindex, '(2a)') 'set palette defined (0 "green", 1, &
'5 "yellow", 10 "red")'
write(outfileindex, '(3a)') "set output 'Berrycurvature_line_sepband.pdf'"
! 设置绘图样式
write(outfileindex, '(a)') 'set style data linespoints'
write(outfileindex, '(a)') 'unset key' ! 取消图例
write(outfileindex, '(a)') 'set pointsize 0.8'
write(outfileindex, '(a)') '#set xtics font ",24"'
write(outfileindex, '(a)') '#set ytics font ",24"'
write(outfileindex, '(a)') '#set ylabel font ",24"'
write(outfileindex, '(a)') 'set ylabel offset 0.5,0'
write(outfileindex, '(a)') 'set border lw 2'
! 设置 x 轴范围
write(outfileindex, '(a, f10.5, a)') 'set xrange [0: ', maxval(k3len*Angstrom2atomic), ']'
! 定义能量范围变量
write(outfileindex, '(a, f12.6)') 'emin=', emin
write(outfileindex, '(a, f12.6)') 'emax=', emax
write(outfileindex, '(a)') ' '
! 设置颜色条范围
write(outfileindex, '(a)') '#set cbrange if the number is too large meeting some band crossings'
write(outfileindex, '(a)') 'set cbrange [-100:100]'
write(outfileindex, '(a)') ' '
! 根据粒子类型设置 y 轴
if (index(Particle, 'phonon')/=0) then
! 声子模式: 频率范围
write(outfileindex, '(a)') 'set yrange [0: emax]'
write(outfileindex, '(a)') 'set ylabel "Frequency (THz)"'
else

```

```

! 电子模式: 能量范围
write(outfileindex, '(a)') 'set ylabel "Energy (eV)"'
write(outfileindex, '(a)') 'set yrange [emin : emax]'
endif
! 写入 x 轴刻度和垂直线标记
write(outfileindex, 202, advance="no") (k3line_name(i), k3line_stop(i)*Angstrom2atomic, i=1, Nk3lines)
write(outfileindex, 203) k3line_name(Nk3lines+1), k3line_stop(Nk3lines+1)*Angstrom2atomic

! 绘制高亮线和垂直标记
do i=1, Nk3lines-1
 if (index(Particle,'phonon')/=0) then
 ! 声子模式的箭头设置
 write(outfileindex, 204) k3line_stop(i+1)*Angstrom2atomic, '0.0', k3line_stop(i+1)*Angstrom2atomic,
 ↪ 'emax'
 else
 ! 电子模式的箭头设置
 write(outfileindex, 204) k3line_stop(i+1)*Angstrom2atomic, 'emin', k3line_stop(i+1)*Angstrom2atomic,
 ↪ 'emax'
 endif
enddo
! 绘制彩色线条图
write(outfileindex, '(4a)') "plot 'Berrycurvature_line_sepband.txt' u 1:2:5 ", &
" w lp lw 2 pt 7 ps 0.2 lc palette, 0 w l lw 2 dt 2"
close(outfileindex)
endif

202 format('set xtics (',20(' ',A3,' ',F10.5,',')) ! x 轴刻度格式定义
203 format(A3,' ',F10.5,',') ! x 轴刻度结束格式
204 format('set arrow from ',F10.5,',',A5,' to ',F10.5,',',A5,' nohead lw 2') ! 箭头格式定义

! 结束 IF (BerryCurvature_Cube_calc) 块
ENDIF

#if defined (MPI)
! MPI 进程同步屏障
call mpi_barrier(mpi_cmw, ierr)
#endif

! 子程序返回
return
end subroutine berry_curvature_cube

! 子程序: 计算完整平面上的 Berry 曲率和轨道磁化强度
subroutine Berry_curvature_plane_full
!> 为选定能带计算 Berry 曲率和轨道磁化强度
!> 参考文献: Physical Review B 74, 195118(2006) 公式 (34)
!> 作者: Quansheng Wu @ EPFL, 2020 年 8 月 20 日
! 版权所有 (c) 2020 QuanSheng Wu

use wmpi
use para
implicit none

! 局部变量声明
integer :: ik, i, j, m, n, ierr, nkmesh2
real(dp) :: k(3), o1(3), vmin, vmax, kxy_plane(3) ! k 点和辅助变量
!> k 点切片
real(dp), allocatable :: kslice(:, :), kslice_xyz(:, :) ! k 点网格
real(dp) :: time_start, time_end, time_start0 ! 时间统计
real(dp), allocatable :: W(:) ! 本征值数组
!> Berry 曲率和轨道磁化强度数组 (类型, 3 分量, k 点)
real(dp), allocatable :: Omega_allk_Occ(:, :, :), Omega_allk_Occ_mpi(:, :, :) ! 基于占据数的 Berry 曲率
real(dp), allocatable :: m_OrbMag_allk_Occ(:, :, :), m_OrbMag_allk_Occ_mpi(:, :, :) ! 基于占据数的轨道磁化
real(dp), allocatable :: Omega_allk_EF(:, :, :), Omega_allk_EF_mpi(:, :, :) ! 基于费米能级的 Berry 曲率
real(dp), allocatable :: m_OrbMag_allk_EF(:, :, :), m_OrbMag_allk_EF_mpi(:, :, :) ! 基于费米能级的轨道磁化
real(dp), allocatable :: Omega_BerryCurv(:, :, :), m_OrbMag(:, :, :) ! 单个 k 点的结果
real(dp) :: beta_fake, fermi, delta, norm ! 物理常数和函数

```

```

! 初始化网格大小和数组
nkmesh2= Nk1*Nk2 ! 二维 k 点网格总数
! 分配内存空间
allocate(Omega_BerryCurv(Num_wann, 3), m_OrbMag(Num_wann, 3))
allocate(Omega_allk_Occ (2, 3, nkmesh2)) ! 2 种求和方式 ×3 分量 ×k 点数
allocate(Omega_allk_Occ_mpi(2, 3, nkmesh2))
allocate(m_OrbMag_allk_Occ (2, 3, nkmesh2))
allocate(m_OrbMag_allk_Occ_mpi(2, 3, nkmesh2))
allocate(Omega_allk_EF (2, 3, nkmesh2)) ! 费米能级方法
allocate(Omega_allk_EF_mpi(2, 3, nkmesh2))
allocate(m_OrbMag_allk_EF (2, 3, nkmesh2))
allocate(m_OrbMag_allk_EF_mpi(2, 3, nkmesh2))
allocate(kslice(3, nkmesh2)) ! k 点网格 (分数坐标)
allocate(kslice_xyz(3, nkmesh2)) ! k 点网格 (笛卡尔坐标)
allocate(W(Num_wann)) ! 本征值数组

! 初始化数组为零
W=0d0
m_OrbMag=0d0
Omega_BerryCurv= 0d0
Omega_allk_Occ= 0d0
m_OrbMag_allk_Occ=0d0
Omega_allk_Occ_mpi= 0d0
m_OrbMag_allk_Occ_mpi=0d0
Omega_allk_EF= 0d0
m_OrbMag_allk_EF=0d0
Omega_allk_EF_mpi= 0d0
m_OrbMag_allk_EF_mpi=0d0
kslice=0d0
kslice_xyz=0d0

!> 构建 k 点切片, 以 K3d_start 为中心
ik =0
do i= 1, nk1 ! k1 方向循环
 do j= 1, nk2 ! k2 方向循环
 ik =ik +1
 ! 构建分数坐标 k 点网格
 kslice(:, ik)= K3D_start+ K3D_vec1*(i-1)/dble(nk1-1) &
 + K3D_vec2*(j-1)/dble(nk2-1) - (K3D_vec1+ K3D_vec2)/2d0
 ! 转换为笛卡尔坐标
 kslice_xyz(:, ik)= kslice(1, ik)* Origin_cell%Kua+ kslice(2, ik)* Origin_cell%Kub+ kslice(3, ik)*
 ↪ Origin_cell%Kuc
 enddo
enddo

! 时间统计初始化
time_start= 0d0
time_start0= 0d0
call now(time_start0) ! 获取开始时间
time_start= time_start0
time_end = time_start0

! 主循环: 遍历二维 k 点网格 (并行)
do ik= 1+ cpuid, nkmesh2, num_cpu
 ! 进度输出
 if (cpuid==0.and. mod((ik-1)/num_cpu, 100)==0) &
 write(stdout, '(a, i9, " /", i10, a, f10.1, "s", a, f10.1, "s")') &
 ' Berry curvature: ik', ik, nkmesh2, ' time left', &
 (nkmesh2-ik)*(time_end- time_start)/num_cpu, & ! 预估剩余时间
 ' time elapsed: ', time_end-time_start0 ! 已用时间

 call now(time_start) ! 记录开始时间

!> 对角化哈密顿量
k= kslice(:, ik) ! 当前 k 点

call now(time_start)
! 调用封装的子程序计算单个 k 点的 Berry 曲率和轨道磁化强度
call Berry_curvature_orb_mag_singlek_allbands_pack(k, Omega_BerryCurv, m_OrbMag, W)

```



```

! 对 Berry 曲率进行两种方式的求和
do i=1, 3 ! 遍历三个分量
! 方式 1: 对占据能带求和
Omega_allk_Occ(1, i, ik) = sum(Omega_BerryCurv(1:NumOccupied, i))
m_OrbMag_allk_Occ(1, i, ik) = sum(m_OrbMag(1:NumOccupied, i))
! 方式 2: 仅取最高占据能带
Omega_allk_Occ(2, i, ik) = Omega_BerryCurv(NumOccupied, i)
m_OrbMag_allk_Occ(2, i, ik) = m_OrbMag(NumOccupied, i)
enddo

!> 考虑费米分布的展宽效应
if (Fermi_broadening<eps6) Fermi_broadening= eps6 ! 防止除零
beta_fake= 1d0/Fermi_broadening ! 计算假想温度倒数

! 应用费米分布权重
do i=1, 3 ! 遍历三个分量
do m= 1, Num_wann ! 遍历所有能带
! 方式 1: 使用费米-狄拉克分布
Omega_allk_EF(1, i, ik)= Omega_allk_EF(1, i, ik)+ &
Omega_BerryCurv(m, i)*fermi(W(m)-iso_energy, beta_fake)
m_OrbMag_allk_EF(1, i, ik)= m_OrbMag_allk_EF(1, i, ik)+ &
m_OrbMag(m, i)*fermi(W(m)-iso_energy, beta_fake)
! 方式 2: 使用 δ 函数近似
Omega_allk_EF(2, i, ik)= Omega_allk_EF(2, i, ik)+ &
Omega_BerryCurv(m, i)*delta(Fermi_broadening, W(m)-iso_energy)
m_OrbMag_allk_EF(2, i, ik)= m_OrbMag_allk_EF(2, i, ik)+ &
m_OrbMag(m, i)*delta(Fermi_broadening, W(m)-iso_energy)
enddo
enddo

call now(time_end) ! 记录结束时间
enddo ! ik 循环

#if defined (MPI)
! MPI 归约所有结果到主进程
call mpi_allreduce(Omega_allk_Occ,Omega_allk_Occ_mpi,size(Omega_allk_Occ_mpi),&
mpi_dp,mpi_sum,mpi_cmw,ierr)
call mpi_allreduce(m_OrbMag_allk_Occ,m_OrbMag_allk_Occ_mpi,size(m_OrbMag_allk_Occ_mpi),&
mpi_dp,mpi_sum,mpi_cmw,ierr)
call mpi_allreduce(Omega_allk_EF,Omega_allk_EF_mpi,size(Omega_allk_EF_mpi),&
mpi_dp,mpi_sum,mpi_cmw,ierr)
call mpi_allreduce(m_OrbMag_allk_EF,m_OrbMag_allk_EF_mpi,size(m_OrbMag_allk_EF_mpi),&
mpi_dp,mpi_sum,mpi_cmw,ierr)
#else
! 串行版本: 直接复制
Omega_allk_Occ_mpi= Omega_allk_Occ
m_OrbMag_allk_Occ_mpi= m_OrbMag_allk_Occ
Omega_allk_EF_mpi= Omega_allk_EF
m_OrbMag_allk_EF_mpi= m_OrbMag_allk_EF
#endif

!> 将 Berry 曲率写入文件
outfileindex= outfileindex+ 1
if (cpuid==0) then
open(unit=outfileindex, file='Berrycurvature.dat')
! 写入文件头信息
write(outfileindex, '(a)')'# Berry curvature in unit of (Angstrom^2) '
write(outfileindex, '("#", 30X,4a36)')'Sum over 1-NumOccupied bands', &
"values at NumOccupied'th band", & ! 最高占据能带的值
' Sum below Fermi level','values at Fermi level' ! 费米能级以下求和及在费米能级处的值
! 写入列标题
write(outfileindex, '(a10,2000a12)')'# kx (1/A)', 'ky (1/A)', 'kz (1/A)', &
"kx' (1/A)", "ky' (1/A)", "kz' (1/A)", & ! 旋转后的 k 坐标
'Omega_x', 'Omega_y', 'Omega_z', & ! 占据能带求和的 Berry 曲率
'Omega_x', 'Omega_y', 'Omega_z', & ! 最高占据能带的 Berry 曲率
'Omega_x', 'Omega_y', 'Omega_z', & ! 费米能级以下求和的 Berry 曲率
'Omega_x', 'Omega_y', 'Omega_z' ! 费米能级处的 Berry 曲率
write(outfileindex, '("#col ", i5, 200i12)')(i, i=1,18) ! 列号

```

```

! 写入数据
ik= 0
do i= 1, nk1
 do j= 1, nk2
 ik= ik+ 1
 ! 将 k 点旋转到平面内坐标系
 call rotate_k3_to_kplane(kslice_xyz(:, ik), kxy_plane)
 ! 写入完整的数据行
 write(outfileindex, '(6f12.6,2000E12.4)')kslice_xyz(:, ik)*Angstrom2atomic, & ! 原始 k 坐标
 kxy_plane*Angstrom2atomic, & ! 旋转后的 k 坐标
 ! 四种不同方式的 Berry 曲率 (转换单位)
 Omega_allk_Occ_mpi(1, :, ik)/Angstrom2atomic**2, & ! 占据能带求和
 Omega_allk_Occ_mpi(2, :, ik)/Angstrom2atomic**2, & ! 最高占据能带
 Omega_allk_EF_mpi(1, :, ik)/Angstrom2atomic**2, & ! 费米能级以下求和
 Omega_allk_EF_mpi(2, :, ik)/Angstrom2atomic**2 ! 费米能级处
 enddo
 write(outfileindex, *) ' ' ! 每行后空行
enddo

close(outfileindex)
endif

!> 转换为玻尔磁子单位
! 转换公式: 乘以 eV2Hartree * Ang2Bohr^2 * 2
m_OrbMag_allk_EF_mpi= m_OrbMag_allk_EF_mpi*eV2Hartree*Ang2Bohr**2 * 2
m_OrbMag_allk_Occ_mpi= m_OrbMag_allk_Occ_mpi*eV2Hartree*Ang2Bohr**2 * 2

! [后续代码继续... 包括轨道磁化强度输出、各种绘图脚本生成等]

#if defined (MPI)
! MPI 进程同步屏障, 确保所有进程完成后再继续执行
call mpi_barrier(mpi_cmw, ierr)
#endif

! 释放动态分配的内存
deallocate(kslice) ! 释放 k 点切片数组
deallocate(kslice_xyz) ! 释放 k 点笛卡尔坐标数组

return
end subroutine Berry_curvature_plane_full ! 结束平面 Berry 曲率计算子程序

! 子程序: 封装的单 k 点 Berry 曲率和轨道磁化强度计算 (所有能带)
subroutine Berry_curvature_orb_mag_singlek_allbands_pack(k, Omega_BerryCurv, m_OrbMag, W)
!> 为选定能带计算 Berry 曲率和轨道磁化强度
!> 参考文献: Physical Review B 74, 195118(2006) 公式 (34)
!> 作者: Quansheng Wu @ EPFL, 2020 年 9 月 29 日
! 版权所有 (c) 2020 QuanSheng Wu

use para, only : Num_wann, dp, zi, NumOccupied ! 只引入需要的模块变量
implicit none

!> 输入: 分数坐标下的 k 点坐标
real(dp), intent(in) :: k(3)

!> 输出: Berry 曲率和轨道磁化强度 (3 个分量, 能带)
real(dp), intent(out) :: m_OrbMag(Num_wann, 3)
real(dp), intent(out) :: Omega_BerryCurv(Num_wann, 3)

!> 输出: 本征值
real(dp), intent(out) :: W(Num_wann)

integer :: i ! 循环计数器

!> Wannier 基下的速度矩阵 (能带 × 能带 × 3 方向)
complex(dp), allocatable :: Vmn_wann(:, :, :)

!> 哈密顿量基下的速度矩阵
complex(dp), allocatable :: Vmn_Ham(:, :, :), Vmn_Ham_nondiag(:, :, :)

```

```

complex(dp), allocatable :: Dmn_Ham(:, :, :) ! Berry 连接矩阵

!> 哈密顿量、本征值和本征向量
complex(dp), allocatable :: UU(:, :)

! 分配内存空间
allocate(Vmn_wann(Num_wann, Num_wann, 3), Vmn_Ham(Num_wann, Num_wann, 3))
allocate(Dmn_Ham(Num_wann, Num_wann, 3), Vmn_Ham_nondiag(Num_wann, Num_wann, 3))
allocate(UU(Num_wann, Num_wann))

! 初始化输出数组
W=0d0
m_OrbMag=0d0
Omega_BerryCurv= 0d0

!> 通过 HmnR 的直接傅里叶变换计算体态哈密顿量
call ham_bulk_atomicgauge(k, UU)

!> 调用 LAPACK 的 zheev 进行对角化
call eigensystem_c('V', 'U', Num_wann, UU, W)

!> 在 Wannier 基下获取速度算符
call dHdk_atomicgauge(k, Vmn_wann)

!> 将 Vmn_wann 从 Wannier 基旋转到哈密顿量基
do i=1, 3 ! 遍历三个笛卡尔方向
 call rotation_to_Ham_basis(UU, Vmn_wann(:, :, i), Vmn_Ham(:, :, i))
enddo

! 计算 Berry 连接矩阵和非对角速度矩阵元
call get_Dmn_Ham(W, Vmn_Ham, Dmn_Ham)
call get_Vmn_Ham_nondiag(Vmn_Ham, Vmn_Ham_nondiag)

! 计算 Berry 曲率和轨道磁化强度
call Berry_curvature_singlek_allbands(Dmn_Ham, Omega_BerryCurv)
call orbital_magnetization_singlek_allbands(Dmn_Ham, Vmn_Ham_nondiag, m_OrbMag)

! 释放内存
deallocate(Vmn_wann, Vmn_Ham, Vmn_Ham_nondiag)
deallocate(UU, Dmn_Ham)
return
end subroutine Berry_curvature_orb_mag_singlek_allbands_pack

! 子程序: 基于费米能级计算平面 Berry 曲率和轨道磁化强度
subroutine Berry_curvature_plane_EF
!> 为选定能带计算基于费米能级的 Berry 曲率和轨道磁化强度
!> 参考文献: Physical Review B 74, 195118(2006) 公式 (34)
!> 作者: Quansheng Wu @ EPFL, 2020 年 8 月 20 日
! 版权所有 (c) 2020 QuanSheng Wu

use wmpi
use para
implicit none

! 局部变量
integer :: ik, i, j, n, ierr, nkmesh2
real(dp) :: k(3), o1(3), vmin, vmax ! k 点和辅助变量
!> k 点切片
real(dp), allocatable :: kslice(:, :), kslice_xyz(:, :)
real(dp), external :: norm ! 外部范数函数
real(dp) :: time_start, time_end, time_start0 ! 时间统计

!> Berry 曲率和轨道磁化强度数组 (能带, 3 分量, k 点)
real(dp), allocatable :: Omega_allk(:, :, :), Omega_allk_mpi(:, :, :)
real(dp), allocatable :: m_OrbMag_allk(:, :, :), m_OrbMag_allk_mpi(:, :, :)
real(dp), allocatable :: Omega_BerryCurv(:, :, :), m_OrbMag(:, :, :)

!> Wannier 基下的速度矩阵
complex(dp), allocatable :: Vmn_wann(:, :, :)

```

```

!> 哈密顿量基下的速度矩阵
complex(dp), allocatable :: Vmn_Ham(:, :, :), Vmn_Ham_nondiag(:, :, :)
complex(dp), allocatable :: Dmn_Ham(:, :, :)

!> 哈密顿量、本征值和本征向量
complex(dp), allocatable :: UU(:, :)
real(dp), allocatable :: W(:) ! 本征值数组

! 初始化网格大小和数组分配
nkmesh2= Nk1*Nk2 ! 二维 k 点网格总数
! 分配各种数组内存
allocate(Vmn_wann(Num_wann, Num_wann, 3), Vmn_Ham(Num_wann, Num_wann, 3))
allocate(Dmn_Ham(Num_wann, Num_wann, 3), Vmn_Ham_nondiag(Num_wann, Num_wann, 3))
allocate(W(Num_wann))
allocate(UU(Num_wann, Num_wann))
allocate(Omega_BerryCurv(Num_wann, 3), m_OrbMag(Num_wann, 3))
allocate(Omega_allk (Num_wann, 3, nkmesh2))
allocate(Omega_allk_mpi(Num_wann, 3, nkmesh2))
allocate(m_OrbMag_allk (Num_wann, 3, nkmesh2))
allocate(m_OrbMag_allk_mpi(Num_wann, 3, nkmesh2))
allocate(kslice(3, nkmesh2))
allocate(kslice_xyz(3, nkmesh2))

! 初始化数组为零
m_OrbMag=0d0
Omega_allk= 0d0
m_OrbMag_allk=0d0
Omega_allk_mpi= 0d0
Omega_BerryCurv= 0d0
m_OrbMag_allk_mpi=0d0
kslice=0d0
kslice_xyz=0d0

!> 构建以 K3d_start 为中心的 k 点切片
ik =0
do i= 1, nk1 ! k1 方向循环
 do j= 1, nk2 ! k2 方向循环
 ik =ik +1
 ! 构建分数坐标 k 点网格 (居于 K3D_start)
 kslice(:, ik)= K3D_start+ K3D_vec1*(i-1)/dble(nk1-1) &
 + K3D_vec2*(j-1)/dble(nk2-1) - (K3D_vec1+ K3D_vec2)/2d0
 ! 转换为笛卡尔坐标
 kslice_xyz(:, ik)= kslice(1, ik)* Origin_cell%Kua+ kslice(2, ik)* Origin_cell%Kub+ kslice(3, ik)*
 ↪ Origin_cell%Kuc
 enddo
enddo

! 时间统计初始化
time_start= 0d0
time_start0= 0d0
call now(time_start0) ! 获取开始时间
time_start= time_start0
time_end = time_start0

! 主循环: 遍历二维 k 点网格 (并行)
do ik= 1+ cpuid, nkmesh2, num_cpu
 ! 进度输出
 if (cpuid==0.and. mod(ik/num_cpu, 100)==0) &
 write(stdout, '(a, i9, " /", i10, a, f10.1, "s", a, f10.1, "s")') &
 ' Berry curvature: ik', ik, nkmesh2, ' time left', &
 (nkmesh2-ik)*(time_end- time_start)/num_cpu, & ! 预估剩余时间
 ' time elapsed: ', time_end-time_start0 ! 已用时间

 call now(time_start) ! 记录开始时间

!> 对角化哈密顿量
k= kslice(:, ik) ! 当前 k 点

```

```

call now(time_start)

!> 通过 HmnR 的直接傅里叶变换计算体态哈密顿量
call ham_bulk_atomicgauge(k, UU)

!> 调用 LAPACK 进行对角化
call eigensystem_c('V', 'U', Num_wann, UU, W)

!> 在 Wannier 基下获取速度算符
call dHdk_atomicgauge(k, Vmn_wann)

!> 将 Vmn_wann 从 Wannier 基旋转到哈密顿量基
do i=1, 3 ! 遍历三个笛卡尔方向
 call rotation_to_Ham_basis(UU, Vmn_wann(:, :, i), Vmn_Ham(:, :, i))
enddo

! 计算 Berry 连接矩阵和非对角速度矩阵元
call get_Dmn_Ham(W, Vmn_Ham, Dmn_Ham)
call get_Vmn_Ham_nondiag(Vmn_Ham, Vmn_Ham_nondiag)

! 计算单个 k 点的 Berry 曲率和轨道磁化强度
call Berry_curvature_singlek_allbands(Dmn_Ham, Omega_BerryCurv)
call orbital_magnetization_singlek_allbands(Dmn_Ham, Vmn_Ham_nondiag, m_OrbMag)

! 保存结果
Omega_allk(:, :, ik) = Omega_BerryCurv
m_OrbMag_allk(:, :, ik) = m_OrbMag

call now(time_end) ! 记录结束时间
enddo ! ik 循环

#if defined (MPI)
! MPI 归约所有结果到主进程
call mpi_allreduce(Omega_allk, Omega_allk_mpi, size(Omega_allk_mpi), &
 mpi_dp, mpi_sum, mpi_cmw, ierr)
call mpi_allreduce(m_OrbMag_allk, m_OrbMag_allk_mpi, size(m_OrbMag_allk_mpi), &
 mpi_dp, mpi_sum, mpi_cmw, ierr)
#else
! 串行版本: 直接复制
Omega_allk_mpi= Omega_allk
m_OrbMag_allk_mpi= m_OrbMag_allk
#endif

!> 输出 Berry 曲率和轨道磁化强度到文件
outfileindex= outfileindex+ 1
if (cpuid==0) then ! 主进程输出
 open(unit=outfileindex, file='Berrycurvature_Orbitalmagnetization.dat')
 ! 写入列号信息
 write(outfileindex, '("#col ", i5, 200i12)')(i, i=1, NumberofSelectedBands*6+3)
 ! 写入列标题
 write(outfileindex, '(a10,2000a12)') '# kx (1/A)', 'ky (1/A)', 'kz (1/A)', &
 'Omega_x(A^2)', 'Omega_y(A^2)', 'Omega_z(A^2)', 'm_x', 'm_y', 'm_z'

 ! 写入数据
 ik= 0
 do i= 1, nk1
 do j= 1, nk2
 ik= ik+ 1
 ! 写入 k 点坐标和选定能带的 Berry 曲率与轨道磁化强度
 write(outfileindex, '(3f12.6,2000E12.4)') kslice_xyz(:, ik)*Angstrom2atomic, &
 ! 遍历选定的能带索引
 (Omega_allk_mpi(Selected_band_index(n), :, ik)/Angstrom2atomic**2, & ! Berry 曲率 (转换单位)
 m_OrbMag_allk_mpi(Selected_band_index(n), :, ik), n=1, NumberofSelectedBands)
 enddo
 write(outfileindex, *) ' ' ! 每行后空行分隔
 enddo
 enddo

 close(outfileindex) ! 关闭文件
endif

```

```

!> 生成 gnuplot 脚本绘制 Berry 曲率图
outfileindex= outfileindex+ 1
if (cpuid==0) then
 open(unit=outfileindex, file='Berrycurvature.gnu')
 ! 设置 gnuplot 参数
 write(outfileindex, '(a)') "set encoding iso_8859_1"
 ! 设置终端为 pngcairo (高质量 PNG)
 write(outfileindex, '(a)') "set terminal pngcairo truecolor enhanced size 3680, 1920 font ",40"
 write(outfileindex, '(a)') "#set terminal png truecolor enhanced size 3680, 1920 font ",40"
 write(outfileindex, '(a)') "set output 'Berrycurvature.png'"
 ! 设置多图布局参数
 write(outfileindex, '(a)') "if (!exists('MP_LEFT')) MP_LEFT = .12"
 write(outfileindex, '(a)') "if (!exists('MP_RIGHT')) MP_RIGHT = .92"
 write(outfileindex, '(a)') "if (!exists('MP_BOTTOM')) MP_BOTTOM = .12"
 write(outfileindex, '(a)') "if (!exists('MP_TOP')) MP_TOP = .88"
 write(outfileindex, '(a)') "if (!exists('MP_GAP')) MP_GAP = 0.08"
 write(outfileindex, '(a)') "set multiplot layout 1,3 rowsfirst \"
 write(outfileindex, '(a)') " margins screen MP_LEFT, MP_RIGHT, MP_BOTTOM, MP_TOP spacing
 ↪ screen MP_GAP"
 write(outfileindex, '(a)') " "
 ! 设置绘图样式
 write(outfileindex, '(a)') "set palette rgbformulae 33,13,10"
 write(outfileindex, '(a)') "unset ztics"
 write(outfileindex, '(a)') "unset key"
 write(outfileindex, '(a)') "set pm3d"
 write(outfileindex, '(a)') "#set zbrange [-10: 10] "
 write(outfileindex, '(a, f10.3, a, f10.3, a)') "#set cbrange [", vmin, ':', vmax, "] "
 write(outfileindex, '(a)') "set view map"
 write(outfileindex, '(a)') "set size ratio -1"
 write(outfileindex, '(a)') "set border lw 3"
 write(outfileindex, '(a)') "set xlabel 'k (1/{\305})'"
 write(outfileindex, '(a)') "set ylabel 'k (1/{\305})'"
 write(outfileindex, '(a)') "unset colorbox"
 write(outfileindex, '(a)') "#unset xtics"
 write(outfileindex, '(a)') "#unset xlabel"
 write(outfileindex, '(a)') "set xrange [] noextend"
 write(outfileindex, '(a)') "set yrange [] noextend"
 write(outfileindex, '(a)') "set ytics 0.5 nomirror scale 0.5"
 write(outfileindex, '(a)') "set pm3d interpolate 2,2"
 ! 绘制三个方向的 Berry 曲率分量
 write(outfileindex, '(a)') "set title 'Berry Curvature {/Symbol W}_x'"
 write(outfileindex, '(a)') "splot 'Berrycurvature_Orbitalmagnetization.dat' u 1:2:4 w pm3d"
 write(outfileindex, '(a)') "unset ylabel"
 write(outfileindex, '(a)') "unset ytics"
 write(outfileindex, '(a)') "set title 'Berry Curvature {/Symbol W}_y'"
 write(outfileindex, '(a)') "splot 'Berrycurvature_Orbitalmagnetization.dat' u 1:2:5 w pm3d"
 write(outfileindex, '(a)') "set title 'Berry Curvature {/Symbol W}_z'"
 write(outfileindex, '(a)') "set colorbox"
 write(outfileindex, '(a)') "splot 'Berrycurvature_Orbitalmagnetization.dat' u 1:2:6 w pm3d"
 close(outfileindex) ! 关闭 gnuplot 脚本文件
endif

#if defined (MPI)
 ! MPI 进程同步
 call mpi_barrier(mpi_cmw, ierr)
#endif

! 释放内存
deallocate(kslice)
deallocate(kslice_xyz)

return
end subroutine Berry_curvature_plane_EF

! 子程序: 计算选定能带的平面 Berry 曲率和轨道磁化强度
subroutine Berry_curvature_plane_selectedbands
!> 为选定能带计算 Berry 曲率和轨道磁化强度
!> 参考文献: Physical Review B 74, 195118(2006) 公式 (34)

```

---

!> 作者: Quansheng Wu @ EPFL, 2020 年 8 月 20 日  
! 版权所有 (c) 2020 QuanSheng Wu

```
use wmpi
use para
implicit none
```

! 局部变量 (与上一个子程序类似)

```
integer :: ik, i, j, n, ierr, nkmesh2
real(dp) :: k(3), ol(3), vmin, vmax
!> k 点切片
real(dp), allocatable :: kslice(:, :), kslice_xyz(:, :)
real(dp), external :: norm
real(dp) :: time_start, time_end, time_start0
```

!> Berry 曲率和轨道磁化强度数组

```
real(dp), allocatable :: Omega_allk(:, :, :), Omega_allk_mpi(:, :, :)
real(dp), allocatable :: m_OrbMag_allk(:, :, :), m_OrbMag_allk_mpi(:, :, :)
real(dp), allocatable :: Omega_BerryCurv(:, :, :), m_OrbMag(:, :, :)
```

!> 速度矩阵相关数组

```
complex(dp), allocatable :: Vmn_wann(:, :, :), Vmn_Ham(:, :, :), Vmn_Ham_nondiag(:, :, :), Dmn_Ham(:, :, :)
```

!> 哈密顿量和本征值

```
complex(dp), allocatable :: UU(:, :), W(:, :)
```

! 内存分配和初始化 (与上一个子程序完全相同)

```
nkmesh2= nk1*nk2
allocate(Vmn_wann(Num_wann, Num_wann, 3), Vmn_Ham(Num_wann, Num_wann, 3))
allocate(Dmn_Ham(Num_wann,Num_wann,3), Vmn_Ham_nondiag(Num_wann, Num_wann, 3))
allocate(W(Num_wann))
allocate(UU(Num_wann, Num_wann))
allocate(Omega_BerryCurv(Num_wann, 3), m_OrbMag(Num_wann, 3))
allocate(Omega_allk (Num_wann, 3, nkmesh2))
allocate(Omega_allk_mpi(Num_wann, 3, nkmesh2))
allocate(m_OrbMag_allk (Num_wann, 3, nkmesh2))
allocate(m_OrbMag_allk_mpi(Num_wann, 3, nkmesh2))
allocate(kslice(3, nkmesh2))
allocate(kslice_xyz(3, nkmesh2))
m_OrbMag=0d0
Omega_allk= 0d0
m_OrbMag_allk=0d0
Omega_allk_mpi= 0d0
Omega_BerryCurv= 0d0
m_OrbMag_allk_mpi=0d0
```

```
kslice=0d0
kslice_xyz=0d0
```

!> 构建 k 点切片 (与上一个子程序相同)

```
ik =0
do i= 1, nk1
 do j= 1, nk2
 ik =ik +1
 kslice(:, ik)= K3D_start+ K3D_vec1*(i-1)/dble(nk1-1) &
 + K3D_vec2*(j-1)/dble(nk2-1) - (K3D_vec1+ K3D_vec2)/2d0
 kslice_xyz(:, ik)= kslice(1, ik)* Origin_cell%Kua+ kslice(2, ik)* Origin_cell%Kub+ kslice(3, ik)*
 ↪ Origin_cell%Kuc
 enddo
enddo
```

! 时间统计和主循环 (与上一个子程序完全相同)

```
time_start= 0d0
time_start0= 0d0
call now(time_start0)
time_start= time_start0
```

```

time_end = time_start0
do ik= 1+ cpuid, nkmesh2, num_cpu
 if (cpuid==0.and. mod(ik/num_cpu, 100)==0) &
 write(stdout, '(a, i9, " /", i10, a, f10.1, "s", a, f10.1, "s")') &
 ' Berry curvature: ik', ik, nkmesh2, ' time left', &
 (nkmesh2-ik)*(time_end- time_start)/num_cpu, &
 ' time elapsed: ', time_end-time_start0

 call now(time_start)

 !> 对角化哈密顿量
 k= kslice(:, ik)

 call now(time_start)

 !> 计算体态哈密顿量
 call ham_bulk_atomicgauge(k, UU)

 !> 对角化
 call eigensystem_c('V', 'U', Num_wann, UU, W)

 !> 获取 Wannier 基速度算符
 call dHdk_atomicgauge(k, Vmn_wann)

 !> 旋转到哈密顿量基
 do i=1, 3
 call rotation_to_Ham_basis(UU, Vmn_wann(:, :, i), Vmn_Ham(:, :, i))
 enddo

 call get_Dmn_Ham(W, Vmn_Ham, Dmn_Ham)
 call get_Vmn_Ham_nondiag(Vmn_Ham, Vmn_Ham_nondiag)

 call Berry_curvature_singlek_allbands(Dmn_Ham, Omega_BerryCurv)
 call orbital_magnetization_singlek_allbands(Dmn_Ham, Vmn_Ham_nondiag, m_OrbMag)
 Omega_allk(:, :, ik) = Omega_BerryCurv
 m_OrbMag_allk(:, :, ik) = m_OrbMag

 call now(time_end)
enddo ! ik

#if defined (MPI)
 ! MPI 归约 (与上一个子程序相同)
 call mpi_allreduce(Omega_allk, Omega_allk_mpi, size(Omega_allk_mpi), &
 mpi_dp, mpi_sum, mpi_cmw, ierr)
 call mpi_allreduce(m_OrbMag_allk, m_OrbMag_allk_mpi, size(m_OrbMag_allk_mpi), &
 mpi_dp, mpi_sum, mpi_cmw, ierr)
#else
 Omega_allk_mpi= Omega_allk
 m_OrbMag_allk_mpi= m_OrbMag_allk
#endif

!> 输出选定能带的 Berry 曲率和轨道磁化强度
outfileindex= outfileindex+ 1
if (cpuid==0) then
 open(unit=outfileindex, file='Berrycurvature_Orbitalmagnetization.dat')
 write(outfileindex, '("#col ", i5, 200i12)')(i, i=1, NumberofSelectedBands*6+3)
 write(outfileindex, '(a10,2000a12)') '# kx (1/A)', 'ky (1/A)', 'kz (1/A)', &
 'Omega_x', 'Omega_y', 'Omega_z', 'm_x', 'm_y', 'm_z'

 ik= 0
 do i= 1, nk1
 do j= 1, nk2
 ik= ik+ 1
 ! 写入选定能带的数据 (注意: 这里没有单位转换)
 write(outfileindex, '(3f12.6,2000E12.4)') kslice_xyz(:, ik)*Angstrom2atomic, &
 (Omega_allk_mpi(Selected_band_index(n), :, ik)/Angstrom2atomic**2, &
 m_OrbMag_allk_mpi(Selected_band_index(n), :, ik), n=1, NumberofSelectedBands)
 enddo
 enddo
 write(outfileindex, *) ' '

```



```

 enddo

 close(outfileindex)
 endif

!> 生成 gnuplot 脚本 (与上一个子程序相同)
outfileindex= outfileindex+ 1
if (cpuid==0) then
 ! [gnuplot 脚本内容与上一个子程序完全相同, 此处省略重复注释]
 open(unit=outfileindex, file='Berrycurvature.gnu')
 write(outfileindex, '(a)') "set encoding iso_8859_1"
 write(outfileindex, '(a)') "set terminal pngcairo truecolor enhanced size 3680, 1920 font ", 40
 write(outfileindex, '(a)') "#set terminal png truecolor enhanced size 3680, 1920 font ", 40
 write(outfileindex, '(a)') "set output 'Berrycurvature.png'"
 write(outfileindex, '(a)') "if (!exists('MP_LEFT')) MP_LEFT = .12"
 write(outfileindex, '(a)') "if (!exists('MP_RIGHT')) MP_RIGHT = .92"
 write(outfileindex, '(a)') "if (!exists('MP_BOTTOM')) MP_BOTTOM = .12"
 write(outfileindex, '(a)') "if (!exists('MP_TOP')) MP_TOP = .88"
 write(outfileindex, '(a)') "if (!exists('MP_GAP')) MP_GAP = 0.08"
 write(outfileindex, '(a)') "set multiplot layout 1,3 rowsfirst \"
 write(outfileindex, '(a)') " margins screen MP_LEFT, MP_RIGHT, MP_BOTTOM, MP_TOP spacing
 ↪ screen MP_GAP"
 write(outfileindex, '(a)') " "
 write(outfileindex, '(a)') "set palette rgbformulae 33,13,10"
 write(outfileindex, '(a)') "unset ztics"
 write(outfileindex, '(a)') "unset key"
 write(outfileindex, '(a)') "set pm3d"
 write(outfileindex, '(a)') "#set zbrange [-10: 10] "
 write(outfileindex, '(a, f10.3, a, f10.3, a)') "#set cbrange [", vmin, ':', vmax, "] "
 write(outfileindex, '(a)') "set view map"
 write(outfileindex, '(a)') "set size ratio -1"
 write(outfileindex, '(a)') "set border lw 3"
 write(outfileindex, '(a)') "set xlabel 'k (1/{\305})'"
 write(outfileindex, '(a)') "set ylabel 'k (1/{\305})'"
 write(outfileindex, '(a)') "unset colorbox"
 write(outfileindex, '(a)') "#unset xtics"
 write(outfileindex, '(a)') "#unset xlabel"
 write(outfileindex, '(a)') "set xrange [] noextend"
 write(outfileindex, '(a)') "set yrange [] noextend"
 write(outfileindex, '(a)') "set ytics 0.5 nomirror scale 0.5"
 write(outfileindex, '(a)') "set pm3d interpolate 2,2"
 write(outfileindex, '(a)') "set title 'Berry Curvature {/Symbol W}_x'"
 write(outfileindex, '(a)') "splot 'Berrycurvature_Orbitalmagnetization.dat' u 1:2:4 w pm3d"
 write(outfileindex, '(a)') "unset ylabel"
 write(outfileindex, '(a)') "unset ytics"
 write(outfileindex, '(a)') "set title 'Berry Curvature {/Symbol W}_y'"
 write(outfileindex, '(a)') "splot 'Berrycurvature_Orbitalmagnetization.dat' u 1:2:5 w pm3d"
 write(outfileindex, '(a)') "set title 'Berry Curvature {/Symbol W}_z'"
 write(outfileindex, '(a)') "set colorbox"
 write(outfileindex, '(a)') "splot 'Berrycurvature_Orbitalmagnetization.dat' u 1:2:6 w pm3d"
 close(outfileindex)
endif
#if defined (MPI)
! MPI 进程同步屏障, 确保所有进程完成后再继续执行
call mpi_barrier(mpi_cmw, ierr)
#endif

! 释放动态分配的内存
deallocate(kslice) ! 释放 k 点切片数组
deallocate(kslice_xyz) ! 释放 k 点笛卡尔坐标数组

return
end subroutine Berry_curvature_plane_selectedbands ! 结束选定能带 Berry 曲率计算子程序

! 子程序: 计算平面 Berry 曲率 (原始版本)
subroutine Berry_curvature_plane
!> 计算 Berry 曲率
!> 参考文献: Physical Review B 74, 195118(2006) 公式 (34)
!> 作者: Quansheng Wu @ ETHZ, 2015 年 9 月 22 日

```

```

! 版权所有 (c) 2010 QuanSheng Wu

use wmpi
use para
implicit none

! 局部变量
integer :: ik, i, j, ierr
real(dp) :: k(3), ol(3), vmin, vmax ! k 点和辅助变量

!> k 点切片
real(dp), allocatable :: kslice(:, :), kslice_xyz(:, :)
real(dp), external :: norm ! 外部范数函数
real(dp) :: time_start, time_end, time_start0 ! 时间统计

!> Berry 曲率数组 (3 方向, k 点)
complex(dp), allocatable :: Omega_x(:), Omega_y(:), Omega_z(:) ! 各能带的 Berry 曲率
complex(dp), allocatable :: Omega(:, :), Omega_mpi(:, :) ! 总 Berry 曲率和 MPI 归约结果

! 分配内存
allocate(kslice(3, Nk1*Nk2))
allocate(kslice_xyz(3, Nk1*Nk2))
allocate(Omega_x(Num_wann)) ! x 方向 Berry 曲率 (每个能带)
allocate(Omega_y(Num_wann)) ! y 方向 Berry 曲率 (每个能带)
allocate(Omega_z(Num_wann)) ! z 方向 Berry 曲率 (每个能带)
allocate(Omega(3, Nk1*Nk2)) ! 总 Berry 曲率 (3 方向 ×k 点)
allocate(Omega_mpi(3, Nk1*Nk2)) ! MPI 归约后的 Berry 曲率

! 初始化数组
kslice=0d0
kslice_xyz=0d0
omega= 0d0
omega_mpi= 0d0

!> 构建以 K3d_start 为中心的 k 点切片
ik =0
do i= 1, nk1 ! k1 方向循环
 do j= 1, nk2 ! k2 方向循环
 ik =ik +1
 ! 构建分数坐标 k 点网格 (居于 K3D_start)
 kslice(:, ik)= K3D_start+ K3D_vec1*(i-1)/dble(nk1-1) &
 + K3D_vec2*(j-1)/dble(nk2-1) - (K3D_vec1+ K3D_vec2)/2d0
 ! 转换为笛卡尔坐标
 kslice_xyz(:, ik)= kslice(1, ik)* Origin_cell%Kua+ kslice(2, ik)* Origin_cell%Kub+ kslice(3, ik)*
 ↪ Origin_cell%Kuc
 enddo
enddo

! 时间统计初始化
time_start= 0d0
time_start0= 0d0
call now(time_start0) ! 获取开始时间
time_start= time_start0
time_end = time_start0

! 主循环: 遍历二维 k 点网格 (并行)
do ik= 1+ cpuid, Nk1*Nk2, num_cpu
 ! 进度输出
 if (cpuid==0.and. mod((ik-1)/num_cpu, 100)==0) &
 write(stdout, '(a, i9, " /", i10, a, f10.1, "s", a, f10.1, "s")') &
 ' Berry curvature: ik', ik, Nk1*Nk2, ' time left', &
 (nk1*nk2-ik)*(time_end- time_start)/num_cpu, & ! 预估剩余时间
 ' time elapsed: ', time_end-time_start0 ! 已用时间

 call now(time_start) ! 记录开始时间

!> 对角化哈密顿量
k= kslice(:, ik) ! 当前 k 点

```

```

! 初始化 Berry 曲率数组
Omega_x= 0d0
Omega_y= 0d0
Omega_z= 0d0

! 根据标志选择不同的 Berry 曲率计算方法
if (Berrycurvature_EF_calc) then
! 基于费米能级的计算
call Berry_curvature_singlek_EF(k, iso_energy, Omega_x, Omega_y, Omega_z)
else
! 基于占据数的计算 (只计算第一个能带)
call Berry_curvature_singlek_numoccupied_total(k, Omega_x(1), Omega_y(1), Omega_z(1))
endif

! 对能带求和得到总 Berry 曲率
Omega(1, ik) = sum(Omega_x) ! x 方向总和
Omega(2, ik) = sum(Omega_y) ! y 方向总和
Omega(3, ik) = sum(Omega_z) ! z 方向总和

call now(time_end) ! 记录结束时间
enddo ! ik 循环

! 初始化 MPI 归约数组
Omega_mpi= 0d0

#if defined (MPI)
! MPI 归约所有进程的 Berry 曲率结果
call mpi_allreduce(Omega,Omega_mpi,size(Omega_mpi),&
mpi_dc,mpi_sum,mpi_cmw,ierr)
#else
! 串行版本: 直接复制
Omega_mpi= Omega
#endif

! 计算 Berry 曲率的实部的统计范围
vmax=maxval(real(Omega_mpi)) ! 最大值
vmin=minval(real(Omega_mpi)) ! 最小值

!> 输出原始 Berry 曲率数据到文件
outfileindex= outfileindex+ 1
if (cpuid==0) then ! 主进程输出
open(unit=outfileindex, file='Berrycurvature.dat')
! 写入说明信息
write(outfileindex, '(20a28)')'# Please take the real part for your use'
write(outfileindex, '(20a28)')'# kx (1/A)', 'ky (1/A)', 'kz (1/A)', &
'Omega_x', 'Omega_y', 'Omega_z'

! 写入数据
ik= 0
do i= 1, nk1
do j= 1, nk2
ik= ik+ 1
! 写入 k 点坐标和 Berry 曲率实部 (转换单位)
write(outfileindex, '(20E28.10)')kslice_xyz(:, ik)*Angstrom2atomic, &
real(Omega_mpi(:, ik))/Angstrom2atomic**2
enddo
write(outfileindex, *) ' ' ! 每行后空行分隔
enddo

close(outfileindex) ! 关闭文件
endif

!> 输出归一化的 Berry 曲率数据到文件
outfileindex= outfileindex+ 1
if (cpuid==0) then
open(unit=outfileindex, file='Berrycurvature-normalized.dat')
write(outfileindex, '(20a28)')'# Please take the real part for your use'
write(outfileindex, '(20a28)')'# kx (1/A)', 'ky (1/A)', 'kz (1/A)', &
'Omega_x(A^2)', 'Omega_y(A^2)', 'Omega_z(A^2)'

```

```

ik= 0
do i= 1, nk1
 do j= 1, nk2
 ik= ik+ 1
 o1= real(Omega_mpi(:,ik)) ! 获取实部的 Berry 曲率矢量
 ! 归一化处理 (避免除零)
 if (norm(o1)>eps9) o1= o1/norm(o1)
 write(outfileindex, '(20f28.10)')kslice_xyz(:, ik)*Angstrom2atomic, o1
 enddo
 write(outfileindex, *) ' '
enddo
close(outfileindex)
endif

!> 生成原始 Berry 曲率的 gnuplot 脚本
outfileindex= outfileindex+ 1
if (cpuid==0) then
 open(unit=outfileindex, file='Berrycurvature.gnu')
 ! 设置 gnuplot 参数
 write(outfileindex, '(a)') "set encoding iso_8859_1"
 write(outfileindex, '(a)') "set terminal pngcairo truecolor enhanced size 3680, 1920 font ",40"
 write(outfileindex, '(a)') "#set terminal png truecolor enhanced size 3680, 1920 font ",40"
 write(outfileindex, '(a)') "set output 'Berrycurvature.png'"
 ! 设置多图布局参数
 write(outfileindex, '(a)') "if (!exists('MP_LEFT')) MP_LEFT = .12"
 write(outfileindex, '(a)') "if (!exists('MP_RIGHT')) MP_RIGHT = .92"
 write(outfileindex, '(a)') "if (!exists('MP_BOTTOM')) MP_BOTTOM = .12"
 write(outfileindex, '(a)') "if (!exists('MP_TOP')) MP_TOP = .88"
 write(outfileindex, '(a)') "if (!exists('MP_GAP')) MP_GAP = 0.08"
 write(outfileindex, '(a)') "set multiplot layout 1,3 rowsfirst \"
 write(outfileindex, '(a)') "
 write(outfileindex, '(a)') "margins screen MP_LEFT, MP_RIGHT, MP_BOTTOM, MP_TOP spacing
 write(outfileindex, '(a)') "
 write(outfileindex, '(a)') " "
 write(outfileindex, '(a)') "set palette rgbformulae 33,13,10"
 write(outfileindex, '(a)') "unset ztics"
 write(outfileindex, '(a)') "unset key"
 write(outfileindex, '(a)') "set pm3d"
 write(outfileindex, '(a)') "#set zbrange [-10: 10] "
 write(outfileindex, '(a, f10.3, a, f10.3, a)') "set cbrange [", vmin, ':', vmax, "] " ! 设置颜色范围
 write(outfileindex, '(a)') "set view map"
 write(outfileindex, '(a)') "set size ratio -1"
 write(outfileindex, '(a)') "set border lw 3"
 write(outfileindex, '(a)') "set xlabel 'k (1/{305})'"
 write(outfileindex, '(a)') "set ylabel 'k (1/{305})'"
 write(outfileindex, '(a)') "unset colorbox"
 write(outfileindex, '(a)') "#unset xtics"
 write(outfileindex, '(a)') "#unset xlabel"
 write(outfileindex, '(a)') "set xrange [] noextend"
 write(outfileindex, '(a)') "set yrange [] noextend"
 write(outfileindex, '(a)') "set ytics 0.5 nomirror scale 0.5"
 write(outfileindex, '(a)') "set pm3d interpolate 2,2"
 ! 绘制三个方向的 Berry 曲率分量
 write(outfileindex, '(a)') "set title 'Berry Curvature {/Symbol W}_x'"
 write(outfileindex, '(a)') "splot 'Berrycurvature.dat' u 1:2:4 w pm3d"
 write(outfileindex, '(a)') "unset ylabel"
 write(outfileindex, '(a)') "unset ytics"
 write(outfileindex, '(a)') "set title 'Berry Curvature {/Symbol W}_y'"
 write(outfileindex, '(a)') "splot 'Berrycurvature.dat' u 1:2:5 w pm3d"
 write(outfileindex, '(a)') "set title 'Berry Curvature {/Symbol W}_z'"
 write(outfileindex, '(a)') "set colorbox"
 write(outfileindex, '(a)') "splot 'Berrycurvature.dat' u 1:2:6 w pm3d"
 close(outfileindex) ! 关闭 gnuplot 脚本文件
endif

!> 生成归一化 Berry 曲率的 gnuplot 脚本 (矢量图)
outfileindex= outfileindex+ 1
if (cpuid==0) then
 open(unit=outfileindex, file='Berrycurvature-normalized.gnu')

```

```

write(outfileindex, '(a)') "set encoding iso_8859_1"
write(outfileindex, '(a)') "#set terminal pngcairo truecolor enhanced size 1920, 1680 font ", 40"
write(outfileindex, '(a)') "set terminal png truecolor enhanced size 1920, 1680 font ", 40"
write(outfileindex, '(a)') "set output 'Berrycurvature-normalized.png'"
write(outfileindex, '(a)') "unset ztics"
write(outfileindex, '(a)') "unset key"
write(outfileindex, '(a)') "set border lw 3"
write(outfileindex, '(a)') "set xlabel 'k (1/{\305})'"
write(outfileindex, '(a)') "set ylabel 'k (1/{\305})'"
write(outfileindex, '(a)') "unset colorbox"
write(outfileindex, '(a)') "unset xtics"
write(outfileindex, '(a)') "unset xlabel"
write(outfileindex, '(a)') "set xrange [] noextend"
write(outfileindex, '(a)') "set yrange [] noextend"
write(outfileindex, '(a)') "set ytics 0.5 nomirror scale 0.5"
write(outfileindex, '(a)') "set pm3d interpolate 2,2"
write(outfileindex, '(a)') "set title '({/Symbol W}_x, {/Symbol W}_y) real'"
! 绘制归一化的 Berry 曲率矢量场
write(outfileindex, '(a)') "plot 'Berrycurvature-normalized.dat' u 1:2:($4/500):($5/500) w vec"
close(outfileindex)
endif

#if defined (MPI)
! MPI 进程同步
call mpi_barrier(mpi_cmw, ierr)
#endif

! 释放内存
deallocate(kslice)
deallocate(kslice_xyz)
deallocate(Omega_x, Omega_y, Omega_z)
deallocate(Omega, Omega_mpi)

return
end subroutine Berry_curvature_plane

! 子程序: 从实空间哈密顿量傅里叶变换到 k 空间
subroutine Fourier_R_to_k(k, ham)
!> 将哈密顿量从实空间 (R 空间) 傅里叶变换到 k 空间
use para, only: irvec, HmnR, Nrpts, ndegen, pi, zi, Num_wann, dp
implicit none

real(dp), intent(in) :: k(3) ! 输入的 k 点坐标
complex(dp), intent(out) :: ham(Num_wann, Num_wann) ! 输出的 k 空间哈密顿量
integer :: iR ! 实空间格点索引
real(dp) :: kdotr ! k·r 点积

ham= 0d0 ! 初始化哈密顿量为零
! 遍历所有实空间格点
do iR= 1, Nrpts
! 计算 k·r 点积
kdotr= k(1)*irvec(1,iR) + k(2)*irvec(2,iR) + k(3)*irvec(3,iR)
! 傅里叶变换: $H(k) = \sum_R H(R) * \exp(i2\pi k \cdot R) / \text{deg}(R)$
Ham= Ham+ HmnR(:,iR)*Exp(2d0*pi*zi*kdotr)/ndegen(iR)
enddo

return
end subroutine Fourier_R_to_k

! 子程序: 计算矩阵的虚部迹
subroutine Im_trace(ndim, A, tr)
!> 计算矩阵 A 的虚部迹 (仅取对角元素的虚部)
use para, only : dp
implicit none
integer :: ndim ! 矩阵维度
complex(dp), intent(out) :: tr ! 输出的虚部迹
complex(dp), intent(in) :: A(ndim, ndim) ! 输入矩阵

integer :: i ! 循环计数器

```

```

 tr = 0d0
 do i=1, ndim ! 遍历对角元素
 tr= tr+ aimag(A(i, i)) ! 累加对角元素的虚部
 enddo

 return
end subroutine Im_trace

! 子程序: 计算矩阵的完整迹
subroutine trace(ndim, A, tr)
 !> 计算矩阵 A 的完整迹 (包括实部和虚部)
 use para, only : dp
 implicit none
 integer :: ndim ! 矩阵维度
 complex(dp), intent(out) :: tr ! 输出的迹
 complex(dp), intent(in) :: A(ndim, ndim) ! 输入矩阵

 integer :: i ! 循环计数器

 tr = 0d0
 do i=1, ndim ! 遍历对角元素
 tr= tr+ (A(i, i)) ! 累加对角元素 (完整复数)
 enddo

 return
end subroutine trace

! 子程序: 在球面上计算陈数
subroutine Chern_sphere_single(k0, r0, Chern)
 !> 通过在球面上积分计算陈数
 !> 球面由中心 k0 和半径 r0 定义
 !> $C = \int d\theta d\phi$ (...)
 !> k0 必须在笛卡尔坐标系中
 !> 作者: QuanSheng Wu @ EPFL, 2018 年 6 月 12 日

 use wmpi
 use para
 implicit none

 !> 输入输出变量
 real(dp), intent(in) :: k0(3) ! 球面中心 (笛卡尔坐标)
 real(dp), intent(in) :: r0 ! 球面半径
 real(dp), intent(out) :: Chern ! 输出的陈数

 ! 局部变量
 integer :: ik, ik1, ik2, nkmesh2, ierr
 real(dp) :: theta, phi, r_para, dtheta, dphi, Chern_mpi ! 球坐标和积分参数
 real(dp) :: st, ct, sp, cp, 0_x, 0_y, 0_z ! 三角函数和 Berry 曲率分量

 !> 球面上的 k 点
 real(dp) :: k_cart(3), k_direct(3) ! 笛卡尔和分数坐标
 real(dp), allocatable :: kpoints(:, :) ! 所有 k 点
 real(dp), allocatable :: thetas(:) ! θ 角数组
 real(dp), allocatable :: phis(:) ! ϕ 角数组

 !> Berry 曲率数组
 complex(dp), allocatable :: Omega_x(:)
 complex(dp), allocatable :: Omega_y(:)
 complex(dp), allocatable :: Omega_z(:)

 ! 分配内存
 nkmesh2= Nk1*Nk2
 allocate(Omega_x(Num_wann), Omega_y(Num_wann), Omega_z(Num_wann))
 allocate(thetas(nkmesh2), phis(nkmesh2), kpoints(3, nkmesh2))

 !> 在笛卡尔坐标系中设置球面
 ik= 0
 do ik2=1, Nk2 ! θ 角循环 (相对于 z 轴的角度)

```

```

theta= (ik2- 1d0)/(Nk2- 1d0)* pi ! 均匀分布的 θ 角
! 避免在极点处出现数值问题
if (ik2== 1) theta= (ik2- 1d0+ 0.10)/(Nk2- 1d0)* pi ! 避免北极
if (ik2== Nk2) theta= (ik2- 1d0- 0.10)/(Nk2- 1d0)* pi ! 避免南极
do ik1=1, Nk1 ! ϕ 角循环 (在 xy 平面内的极角)
 ik = ik+ 1
 phi= (ik1- 1d0)/Nk1* 2d0* pi ! 均匀分布的 ϕ 角
 r_para= r0* sin(theta) ! 平行圆半径
 ! 计算笛卡尔坐标
 k_cart(1)= k0(1)+ r_para* cos(phi)
 k_cart(2)= k0(2)+ r_para* sin(phi)
 k_cart(3)= k0(3)+ r0* cos(theta)
 ! 转换为分数坐标
 call cart_direct_rec(k_cart, k_direct)
 kpoints(:, ik)= k_direct
 thetas(ik)= theta
 phis(ik)= phi
enddo
enddo

! 计算积分步长
dtheta= 1.0d0/(Nk2-1d0)*pi ! θ 方向步长
dphi = 1.0d0/Nk1 * 2d0*pi ! ϕ 方向步长

! 并行计算陈数
Chern_mpi = 0d0
do ik=1+cpuid, Nk1*Nk2, num_cpu
 k_direct= kpoints(:, ik) ! 当前 k 点分数坐标
 theta= thetas(ik) ! 当前 θ 角
 phi= phis(ik) ! 当前 ϕ 角

 ! 计算三角函数值
 st=sin(theta); ct=cos(theta); sp=sin(phi); cp=cos(phi)

 ! 计算当前 k 点的 Berry 曲率
 call Berry_curvature_singlek_numoccupied(k_direct, Omega_x, Omega_y, Omega_z)

 ! 对占据能带求和 (这里似乎有重复计算, 可能是调试代码)
 O_x= real(sum(Omega_x(1:Numoccupied)))
 O_y= real(sum(Omega_y(1:Numoccupied)))
 O_z= real(sum(Omega_z(1:Numoccupied)))
 O_x= real((Omega_x(Numoccupied))) ! 只取最后一个占据能带
 O_y= real((Omega_y(Numoccupied)))
 O_z= real((Omega_z(Numoccupied)))

 ! 球面积分: $\int \sin^2\theta \cos\phi d\theta d\phi$ 等
 Chern_mpi= Chern_mpi+ st*st*cp*O_x+ st*st*sp*O_y+ st*ct*O_z
enddo

#if defined (MPI)
 ! MPI 归约所有进程的陈数贡献
 call mpi_allreduce(Chern_mpi, Chern, 1, &
 mpi_dp, mpi_sum, mpi_cmw, ierr)
#else
 Chern= Chern_mpi
#endif

! 乘以积分测度得到最终陈数
Chern = Chern* r0* r0* dtheta* dphi/pi/2d0

return
end subroutine Chern_sphere_single

! 子程序: 计算多个 Weyl 点的陈数
subroutine Chern_sphere
 use wmpi
 use para
 implicit none

```

```

integer :: i
real(dp) :: k0(3), Chern
real(dp), allocatable :: Chern_array(:) ! 存储各 Weyl 点的陈数

allocate(Chern_array(Num_Weyls))
Chern_array= 0d0

! 遍历所有 Weyl 点
do i=1, Num_Weyls
 k0= weyl_position_cart(:, i) ! 第 i 个 Weyl 点的位置
 call Chern_sphere_single(k0, kr0, Chern) ! 计算该点的陈数
 Chern_array(i)= Chern ! 存储结果
enddo

! 输出结果
if (cpuid==0)then
 write(stdout, *)'# Chern number for the Weyl points'
 write(stdout, '("#",a8,2a9, a16)')'kx', 'ky', 'kz', 'Chern'
 do i=1, Num_Weyls
 write(stdout, '(3f9.5, f16.8)')weyl_position_cart(:, i), Chern_array(i)
 enddo
endif

return
end subroutine Chern_sphere

! 子程序: 在半环面上计算陈数 (类似前面的结构, 此处省略详细注释)
subroutine Chern_halftorus_single(k0, Rbig, rsmall_a, rsmall_b, Chern)
!> 通过在半环面上积分计算陈数
! [详细注释类似 Chern_sphere_single, 主要区别在于几何形状和积分测度]
use wmpi
use para
implicit none

! [参数和变量声明...]
! [实现细节...]
! [MPI 归约和输出...]

return
end subroutine Chern_halftorus_single

! 子程序: 计算多个非线性点的陈数
subroutine Chern_halftorus
use wmpi
use para
implicit none

! [变量声明和实现...]
! 类似于 Chern_sphere, 但针对非线性点 (NL)

return
end subroutine Chern_halftorus

```

## ► 代码清单 A.23.2 (fortran, 290 行)

```

! 子程序: 计算给定路径的 Berry 相位
subroutine berryphase
!> 计算给定路径 Berry 相位的子程序
!
! 注释:
!
! 目前, 必须在 kpoints 中定义想要的 k 路径
!

```



```

! 作者: QuanSheng Wu (wuquansheng@gmail.com)
!
! 2016 年 3 月 31 日
!
! 版权所有 (c) 2010 QuanSheng Wu

use para ! 引入参数模块
use wmpi ! 引入 MPI 并行模块
implicit none ! 显式声明所有变量类型

! 局部变量声明
integer :: i, j, it, ik, Nk_seg, NK_Berry_tot ! 循环计数器和数组维度

!> k 点在 kx-ky 平面中的位置 (3 维坐标, 总 k 点数)
real(dp), allocatable :: kpoints(:, :)

!> 每个 k 点的哈密顿量
!> 以及 eigensystem_c 对角化后的哈密顿量本征向量
complex(dp), allocatable :: uk(:, :), uk_dag(:, :) ! uk: 本征向量矩阵, uk_dag: 其厄米共轭

!> 每个 k 点的本征向量 (能带 × 能带 × k 点)
complex(dp), allocatable :: Eigenvector(:, :, :)

! 本征值数组和相位数组
real (dp), allocatable :: eigenvalue(:) ! 哈密顿量本征值
complex(dp), allocatable :: phase(:) ! Berry 相位累积因子
real(dp) :: br ! Wannier 中心投影的临时变量
real(dp) :: k(3), b(3) ! k 点坐标和路径段向量
complex(dp) :: overlap, ratio ! 重叠积分和相位修正因子

! 每段 k 点的数量和 Berry 相位计算的总 k 点数
Nk_seg= Nk ! 每个路径段的 k 点数
NK_Berry_tot= (NK_Berry-1)*Nk_seg ! 闭合路径总 k 点数

! 分配内存
allocate(kpoints(3, NK_Berry_tot)) ! k 点坐标数组
kpoints= 0d0 ! 初始化为零

! 分配哈密顿量和本征向量相关数组
allocate(uk(Num_wann, Num_wann), uk_dag(Num_wann, Num_wann)) ! 单点哈密顿量和其厄米共轭
allocate(Eigenvector(Num_wann, Num_wann, NK_Berry_tot)) ! 所有 k 点的本征向量
allocate(eigenvalue(Num_wann)) ! 本征值数组
allocate(phase(Num_wann)) ! Berry 相位累积因子

! 初始化数组
uk=0d0
eigenvalue=0d0
Eigenvector=0d0

!> 设置 Berry 相位计算的 k 路径
!> kpoints, k3points_Berry 是分数坐标/直接坐标
it = 0 ! k 点索引初始化
do ik=1, NK_Berry- 1 ! 遍历路径段
 do i= 1, Nk_seg ! 在每个路径段内均匀采样
 it= it+ 1
 ! 线性插值构建路径上的 k 点
 kpoints(:, it)= k3points_Berry(:, ik)+ &
 (k3points_Berry(:, ik+1)- k3points_Berry(:, ik))*(i-1d0)/(Nk_seg-1d0)
 enddo ! i
enddo ! ik

!> 对每个 k 点计算 Wannier 中心相关的 Berry 相位
do ik=1, NK_Berry_tot
 k= kpoints(:, ik) ! 当前 k 点坐标

 ! 根据计算模型选择不同的哈密顿量构造方法
 if (index(KPorTB, 'KP')/=0)then ! 如果使用 kp 模型
 call ham_bulk_kp (k, uk) ! 调用 kp 模型的体态哈密顿量
 else ! 否则使用晶格规范

```

```

 call ham_bulk_latticegauge(k, uk) ! 调用晶格规范的体态哈密顿量
 endif

 !> 对角化哈密顿量获取本征向量
 call eigensystem_c('V', 'U', Num_wann, uk, eigenvalue)

 ! 存储当前 k 点的本征向量
 Eigenvector(:, :, ik)= uk
enddo

!> 沿 k 路径求和得到 Berry 相位
phase= 1d0
do ik= 1, NK_Berry_tot-1
 uk= Eigenvector(:, :, ik)
 uk_dag= conjg(transpose(uk))

 ! 处理闭合路径: 最后一个点与起点连接
 if (ik==NK_Berry_tot-1) then
 uk= Eigenvector(:, :, 1)
 else
 uk= Eigenvector(:, :, ik+ 1)
 endif

 b= kpoints(:, ik+1)- kpoints(:, ik)

 !> 计算 $\langle u_k | u_{k+1} \rangle$ 重叠积分, 考虑 Wannier 中心的规范变换
 do i=1, Num_wann
 overlap= 0d0
 do j=1, Num_wann
 br= b(1)*Origin_cell%wannier_centers_direct(1, j)+ &
 b(2)*Origin_cell%wannier_centers_direct(2, j)+ &
 b(3)*Origin_cell%wannier_centers_direct(3, j)
 ! 规范变换相位因子: $\exp(i2\pi \text{br})$
 ratio= cos(2d0*pi*br)- zi*sin(2d0*pi*br)

 ! 计算矩阵元素乘积并累加
 overlap= overlap+ uk_dag(i, j)* uk(j, i)* ratio
 enddo

 ! 累积相位因子
 phase(i)= overlap*phase(i)
 enddo
enddo !< ik 完成所有路径段的积分

! 输出警告信息和计算结果
if (cpuid==0)write(stdout, *) ">> WARNING: Please increase NK1 until Berry phase is converged!"
if (cpuid==0)write(stdout, *) ">> WARNING: The starting point and the ending point should be different by a
↪ reciprocal lattice vector"
if (cpuid==0)write(stdout, *) 'Berry phase for the loop you chose: in unit of \pi'
! 计算占据能带的 Berry 相位 (模 2π , 以 π 为单位)
if (cpuid==0) write(stdout, '(f18.6)') mod(sum(aimag(log(phase(1:NumOccupied)))/pi), 2d0)

! 输出详细数据到文件
outfileindex= outfileindex+ 1
if (cpuid==0) then
 open(unit= outfileindex, file="kpath_berry.txt")
 ! 写入文件头, 包含 Berry 相位结果
 write(outfileindex, '("#",a11, 5a12, a, f12.6, a)') "kx", "ky", "kz", &
 "k1", "k2", "k3", " Berry phase= ", &
 mod(sum(aimag(log(phase(1:NumOccupied)))/pi), 2d0), ' pi'
 ! 写入所有 k 点的坐标
 do ik=1, NK_Berry_tot
 ! 将分数坐标转换为笛卡尔坐标
 b= kpoints(1, ik)*Origin_cell%Kua+ kpoints(2, ik)*Origin_cell%Kub+ kpoints(3, ik)*Origin_cell%Kuc
 write(outfileindex, '(6f12.6)')b, kpoints(:, ik)
 enddo
endif

```

```

endif

! 释放动态分配的内存
deallocate(kpoints)
deallocate(uk, uk_dag)
deallocate(Eigenvector, eigenvalue)
deallocate(phase)

return
end subroutine berryphase

! 子程序: 使用原子规范计算 Berry 相位 (改进版本)
subroutine berryphase_atomic
!> 使用原子规范计算给定路径 Berry 相位的子程序
! [头部注释与上一子程序相同, 此处省略]

use para
use wmpi
implicit none

! 局部变量 (与上一子程序类似, 增加 mat1, mat2)
integer :: i, j, it, ik, Nk_seg, NK_Berry_tot

!> k 点在 kx-ky 平面中的位置
real(dp), allocatable :: kpoints(:, :)

!> 哈密顿量和本征向量
complex(dp), allocatable :: uk(:, :), uk_dag(:, :)
complex(dp), allocatable :: Eigenvector(:, :, :)

! 本征值和相关矩阵
real (dp), allocatable :: eigenvalue(:)
complex(dp), allocatable :: phase(:), mat1(:, :), mat2(:, :) ! 新增工作矩阵
real(dp) :: br
real(dp) :: k(3), b(3)
complex(dp) :: overlap, ratio

! 路径参数设置 (与上一子程序相同)
Nk_seg= Nk
NK_Berry_tot= (NK_Berry-1)*Nk_seg

! 分配内存 (额外分配 mat1, mat2 用于中间计算)
allocate(kpoints(3, NK_Berry_tot))
kpoints= 0d0

allocate(mat1(Num_wann, Num_wann), mat2(Num_wann, Num_wann)) ! 工作矩阵
allocate(uk(Num_wann, Num_wann), uk_dag(Num_wann, Num_wann))
allocate(Eigenvector(Num_wann, Num_wann, NK_Berry_tot), eigenvalue(Num_wann))
allocate(phase(Num_wann))

! 初始化
uk=0d0
eigenvalue=0d0
Eigenvector=0d0

!> 设置 k 路径 (与上一子程序完全相同)
it = 0
do ik=1, NK_Berry- 1
 do i= 1, Nk_seg
 it= it+ 1
 kpoints(:, it)= k3points_Berry(:, ik)+ &
 (k3points_Berry(:, ik+1)- k3points_Berry(:, ik))*(i-1d0)/(Nk_seg-1d0)
 enddo ! i
enddo ! ik

!> 计算每个 k 点的本征向量 (关键区别: 使用原子规范)
do ik=1, NK_Berry_tot
 k= kpoints(:, ik)

```

```

! 主要区别: 使用原子规范而非晶格规范或 kp 模型
if (index(KPorTB, 'KP')/=0)then
 call ham_bulk_kp (k, uk) ! kp 模型 (如果适用)
else
 call ham_bulk_atomicgauge(k, uk) ! 原子规范哈密顿量
endif

!> 对角化 (与上一子程序相同)
call eigensystem_c('V', 'U', Num_wann, uk, eigenvalue)

Eigenvector(:, :, ik)= uk
enddo

!> 计算 Berry 相位 (主要逻辑改进)
phase= 1d0
do ik= 1, NK_Berry_tot-1
 uk= Eigenvector(:, :, ik)
 uk_dag= conjg(transpose(uk))

 ! 改进的路径闭合处理
 if (ik==NK_Berry_tot-1) then
 ! 显式计算闭合向量 (从终点到起点)
 b=-kpoints(:, 1)+ kpoints(:, NK_Berry_tot)
 uk= Eigenvector(:, :, 1)
 else
 uk= Eigenvector(:, :, ik+ 1)
 endif

!> 改进的重叠积分计算
do i=1, Num_wann
 overlap= 0d0
 do j=1, Num_wann
 if (ik==NK_Berry_tot-1) then
 ! 只在闭合段应用 Wannier 中心规范变换
 br= b(1)*Origin_cell%wannier_centers_direct(1, j)+ &
 b(2)*Origin_cell%wannier_centers_direct(2, j)+ &
 b(3)*Origin_cell%wannier_centers_direct(3, j)
 ratio= cos(2d0*pi*br)- zi*sin(2d0*pi*br)
 else
 ! 非闭合段不使用规范变换 (ratio=1)
 ratio=1d0
 endif

 overlap= overlap+ uk_dag(i, j)* uk(j, i)* ratio
 enddo
 phase(i)= overlap*phase(i)
enddo

enddo !< ik

! 输出部分 (与上一子程序完全相同)
if (cpuid==0)write(stdout, *) ">> WARNING: Please increase NK1 until Berry phase is converged!"
if (cpuid==0)write(stdout, *) ">> WARNING: The starting point and the ending point should be different by a
↪ reciprocal lattice vector"
if (cpuid==0)write(stdout, *) 'Berry phase for the loop you chose: in unit of \pi'
if (cpuid==0) write(stdout, '(f18.6)') mod(sum(aimag(log(phase(1:NumOccupied)))/pi), 2d0)

outfileindex= outfileindex+ 1
if (cpuid==0) then
 open(unit= outfileindex, file="kpath_berry.txt")
 write(outfileindex, '(("#",a11, 5a12, a, f12.6, a))"kx", "ky", "kz", &
 "k1", "k2", "k3", " Berry phase= ", &
 mod(sum(aimag(log(phase(1:NumOccupied)))/pi), 2d0), ' pi'
 do ik=1, NK_Berry_tot
 b= kpoints(1, ik)*Origin_cell%Kua+ kpoints(2, ik)*Origin_cell%Kub+ kpoints(3, ik)*Origin_cell%Kuc
 write(outfileindex, '(6f12.6)')b, kpoints(:, ik)
 enddo
endif

```

```

! 释放内存 (包括新增的 mat1, mat2)
deallocate(kpoints)
deallocate(mat1,mat2,uk,uk_dag)
deallocate(Eigenvector,eigenvalue)
deallocate(phase)

return
end subroutine berryphase_atomic

```

## wannierberrier 的安装

### ► 代码清单 A.24.2 (shell, 9 行)

```
pip install wannierberri -i https://pypi.tuna.tsinghua.edu.cn/simple
```

```

http://mirrors.aliyun.com/pypi/simple/ # 阿里云
https://pypi.mirrors.ustc.edu.cn/simple/ # 中国科技大学
http://pypi.douban.com/simple # 豆瓣
https://pypi.python.org/simple/ # Python 官方
http://pypi.v2ex.com/simple/ # v2ex
http://pypi.mirrors.opencas.cn/simple/ # 中国科学院
https://pypi.tuna.tsinghua.edu.cn/simple/ # 清华大学

```

### ► 代码清单 A.24.4 (shell, 93 行)

```

import wannierberri as wberri
import numpy as np
import matplotlib.pyplot as plt

读取哈密顿数据
system_Te = wberri.System_tb(tb_file='wannier90_tb.dat', berry=True)
system_Te_sym = wberri.System_tb(tb_file='wannier90_tb.dat', berry=True)

哈密顿对称化:
system_Te_sym.symmetrize(
 # 每一个原子的位置
 positions = np.array([[0.5000000000000000, 0.0000000000000000, 0.5036182718074184],
 [0.0000000000000000, 0.5000000000000000, 0.5036182718074184],
 [0.5000000000000000, 0.5000000000000000, 0.6102267399141962],
 [0.5000000000000000, 0.5000000000000000, 0.3960128566905964],
 [0.0000000000000000, 0.0000000000000000, 0.4865206618880932]]),
 # 原子的名称
 atom_name = ['Nb', 'Nb', 'Se', 'Se', 'O'],
 # 投影轨道和 wannier90.win 中保持一致
 proj = ['Nb:d', 'Se:p', 'O:p'],
 # 没有自旋极化或开启 SOC
 soc=True,
)

生成 k 点路径
path=wberri.Path(system_Te,
 # 设置 k 点路径的起点和终点; 如果两条 k 点路径不连续, 请在中间加 None 来分隔
 k_nodes=[[0,0,0],[0.5,0.0,0.0],None,
 [0.5,0.5,0.0],[0.0,0.5,0.0],None,

```

---

```

 [0.0,0.0,0.0]],
 labels=["G","X","M","Y","G"],
 # 与 k 点数目成比例
 length=1000)

计算对称和非对称体系的能带
quantities = {"Energy":wberri.calculators.tabulate.Energy()}

calculators={}
calculators ["tabulate"] = wberri.calculators.TabulatorAll(quantities,ibands=[18,19],mode="path")

path_result_Te = wberri.run(
 system=system_Te,
 grid=path,
 calculators=calculators,
 print_Kpoints=False)

path_result_Te_sym = wberri.run(
 system=system_Te_sym,
 grid=path,
 calculators=calculators,
 print_Kpoints=False)

读取第 22,23 能带的本征值
band_Te = path_result_Te.results['tabulate'].get_data(quantity='Energy',iband=(21,22))
band_Te_sym = path_result_Te_sym.results['tabulate'].get_data(quantity='Energy',iband=(21,22))

读取 k 点路径数据
band_k=path.getKline()

绘制非对称体系的能带
segments = path.get_segments()
band_k = path.getKline()

plt.figure(figsize=(6,5))
for seg in segments:
 k_seg = band_k[seg]
 E_seg = band_Te[seg]
 plt.plot(k_seg, E_seg, 'r', linewidth=2)

plt.xticks(path.getKtick(), path.getKlabel(), fontsize=11)
plt.ylabel('Energy (eV)', fontsize=12)
plt.xlabel('k-path', fontsize=12)
plt.title('Unsymmetrized bands')
plt.grid(True, alpha=0.3)
plt.savefig("unsymmetrized.png", dpi=300)
plt.show()

绘制对称体系的能带
plt.figure(figsize=(6,5))
for seg in segments:
 k_seg = band_k[seg]
 E_seg = band_Te_sym[seg]
 plt.plot(k_seg, E_seg, 'b', linewidth=2)

plt.xticks(path.getKtick(), path.getKlabel(), fontsize=11)
plt.ylabel('Energy (eV)', fontsize=12)
plt.xlabel('k-path', fontsize=12)
plt.title('Symmetrized bands')
plt.grid(True, alpha=0.3)
plt.savefig("symmetrized.png", dpi=300)
plt.show()

```

---

## wansymm 的对称化功能

### ► 代码清单 A.25.5 (python, 19 行)

```
DFTcode = VASP

Spinors = T

SeedName = 'wannier90'

Use_POSCAR = 'POSCAR'

begin projections
Nb : d
Se : p
O : p
end projections

MAGMOM = 0 0 0.76 0 0 -0.76 0 0 0 0 0 0 0 0
symm_magnetic_tolerance = 0.0001

Global_TRsymm= F
```

### ► 代码清单 A.25.6 (python, 9 行)

```
begin kpath
G 0.000000 0.000000 0.000000 X 0.500000 0.000000 0.000000
X 0.500000 0.000000 0.000000 M 0.500000 0.500000 0.000000
M 0.500000 0.500000 0.000000 Y 0.000000 0.500000 0.000000
Y 0.000000 0.500000 0.000000 G 0.000000 0.000000 0.000000
end kpath

restart = T
```

## 利用 Wannier Tools 的对称化 hr.dat

### ► 代码清单 A.26.3 (python, 24 行)

```
some times for sake of symmetry we can set local axis for each atom in wannier. if not
set line 20 is 0. if set, line 20 is number of atoms with local axis followed with the
serial number of atom in wann/projection and local z-axis, x-axis,the. As for the examples
in comments of wann.in, one may only set local axes to a subset of atoms in wannier
#####
4
1 3 d f --> local axis setted
2 6 s d f --> local axis setted
3 2 s p
4 5 d --> local axis setted
In this example we only set local axis to No.1, No.2 and No.4 atoms in wanneir. so we write
```

```
#-----
3
1 zx1 zy1 zz1 xx1 xy1 xz1
2 zx2 zy2 zz2 xx2 xy2 xz2
4 zx3 zy3 zz3 xx3 xy3 xz3
#-----
#####
line 20 is number of atoms with local axis followed with No. and local axis
0
1 0.0 0.0 1.0 1.0 0.0 0.0
2 0.0 0.0 1.0 1.0 0.0 0.0
3 0.0 0.0 1.0 1.0 0.0 0.0
4 0.0 0.0 1.0 1.0 0.0 0.0
```

### ► 代码清单 A.26.4 (python, 13 行)

```
crystal cell vector3: line3 a1; line 4 a2; line 5 a3. sometimes small
numerical errors of {a1,a2,a3} will give rise to worse symmed hamiltonian
4.1630311801153548 0.0000000000000000 0.0000000000000000
0.0000000000000000 4.1630215413893197 0.0000000000000000
0.0000000000000000 0.0000000000000000 16.6440706251267692
fractional axis of each exactly as in POSCAR. line 8, number of atoms. line 9
to line 9+natom-1 is each atoms fractional axis. Mind you the errors in it.
5
 0.0000008564285920 0.5000001817884865 0.5036182718074187 Nb
 0.4999997757884529 0.9999988531661401 0.5036214396997352 Nb
 0.9999996457745581 0.9999999313471903 0.6102267399141965 Se
 0.9999993935379692 0.0000008878344957 0.3960128566905965 Se
 0.5000003284704491 0.5000001458636731 0.4865206618880933 0
```

### ► 代码清单 A.26.5 (python, 30 行)

```
orbitals sequences are adopted as in wannier90/manuals: p = [pz,px,pz], t2g = [dxz,dyz,dxy] and
d = [dz2,dxz,dyz,dx2_y2,dxy], f = [fz3,fxz2,fyz2,fzx2_zy2,fxyz,fx3_3xy2,f3yx2_y3]
more often than not atoms in wannier is just a sub set of crystal cell atoms. you should point out the
POSCAR wannier90.win/projections
#####
atom1 (not projected)
atom2 wann_atom3=poscar_atom2 (orbs: s p)
atom3 wann_atom1=poscar_atom3 (orbs: d f)
atom4 (not_projected)
atom5 wann_atom4=poscar_atom5 (orbs: s)
atom6 wann_atom2=poscar_atom6 (orbs: s d f)
so we have the mapping between the atoms sequence in wannier_hr.dat(or wout/spread) and the atoms in
poscar as wann1=poscar3, wann2=poscar6, wann3=poscar2, wann4=poscar5
so we have to write
#-----
3
1 3 d f
2 6 s d f
3 2 s p
4 5 s
#-----
#####
line 24 number of atoms projected in wannier and followed with the mapping and orbs
5
T
1 1 d
```



---

```
2 2 d
3 3 p
4 4 p
5 5 p
```

## ► 代码清单 A.26.6 (python, 70 行)

```
kmesh_tol=0.00001
!exclude_bands:1-2
dis_win_min =-9.61
dis_win_max =6.06
dis_froz_min =-6.39
dis_froz_max =1.21
begin projections
Nb : d
Se : p
O : p
end projections
write_hr =.true. #step2
write_xyz=true #step2
guiding_centres = true #step2
bands_plot =.true.
begin kpoint_path
G 0.000000 0.000000 0.000000 X 0.500000 0.000000 0.000000
X 0.500000 0.000000 0.000000 M 0.500000 0.500000 0.000000
M 0.500000 0.500000 0.000000 Y 0.000000 0.500000 0.000000
Y 0.000000 0.500000 0.000000 G 0.000000 0.000000 0.000000
end kpoint_path

num_iter = 1000
num_print_cycles = 40
dis_num_iter = 5000
dis_mix_ratio = 0.3
This part was generated automatically by VASP
num_bands = 80
num_wann = 38
spinors = .true.
begin unit_cell_cart
4.1630312 0.0000000 0.0000000
0.0000000 4.1630215 0.0000000
0.0000000 0.0000000 16.6440706
end unit_cell_cart
begin atoms_cart
Nb 0.0000036 2.0815115 8.3822581
Nb 2.0815147 4.1630168 8.3823108
Se 4.1630297 4.1630213 10.1566570
Se 4.1630287 0.0000037 6.5912660
O 2.0815170 2.0815114 8.0976843
end atoms_cart
mp_grid = 5 5 1
begin kpoints
0.000000000000 0.000000000000 0.000000000000
0.200000000000 0.000000000000 0.000000000000
-0.200000000000 0.000000000000 0.000000000000
0.000000000000 0.200000000000 0.000000000000
0.000000000000 -0.200000000000 0.000000000000
0.400000000000 0.000000000000 0.000000000000
-0.400000000000 0.000000000000 0.000000000000
0.000000000000 0.400000000000 0.000000000000
0.000000000000 -0.400000000000 0.000000000000
0.200000000000 0.200000000000 0.000000000000
-0.200000000000 -0.200000000000 0.000000000000
-0.200000000000 0.200000000000 0.000000000000
```

```

0.200000000000 -0.200000000000 0.000000000000
0.400000000000 0.200000000000 0.000000000000
-0.400000000000 -0.200000000000 0.000000000000
-0.200000000000 0.400000000000 0.000000000000
0.200000000000 -0.400000000000 0.000000000000
-0.400000000000 0.200000000000 0.000000000000
0.400000000000 -0.200000000000 0.000000000000
0.200000000000 0.400000000000 0.000000000000
-0.200000000000 -0.400000000000 0.000000000000
0.400000000000 0.400000000000 0.000000000000
-0.400000000000 -0.400000000000 0.000000000000
-0.400000000000 0.400000000000 0.000000000000
0.400000000000 -0.400000000000 0.000000000000
end kpoints

```

## pythTB 的学习 `_Parameterized_TBModel`

### ► 代码清单 A.30.1 (python, 2 行)

```

import numpy as np
from pythtb import TBModel, Lattice

```

### ► 代码清单 A.30.2 (python, 13 行)

```

lat_vecs = [[1.0, 0.0], [0.5, np.sqrt(3.0) / 2.0]]
orb_vecs = [[0.0, 0.0], [1.0 / 3.0, 1.0 / 3.0]]

lattice = Lattice(lat_vecs, orb_vecs, periodic_dirs=...)
tb = TBModel(lattice)

Fixed on-site energies
tb.set_onsite([0.0, 0.0])

Symbolic nearest-neighbour hopping: string 't'
tb.set_hop("t", 0, 1, [0, 0])
tb.set_hop(0.3, 1, 0, [1, 0])
tb.set_hop(0.1, 1, 0, [0, 1])

```

### ► 代码清单 A.30.3 (python, 1 行)

```

tb.parameters

```

### ► 代码清单 A.30.4 (python, 1 行)

```

[{'kind': 'hopping', 'orbitals': (0, 1), 'R': (0, 0), 'names': ('t',)}]

```

---

► 代码清单 A.30.5 (python, 1 行)

```
print(tb)
```

► 代码清单 A.30.6 (python, 40 行)

```

 Tight-binding model report

r-space dimension = 2
k-space dimension = 2
periodic directions = [0, 1]
spinful = False
number of spin components = 1
number of electronic states = 2
number of orbitals = 2

Lattice vectors (Cartesian):
0 ==> [1.000, 0.000]
1 ==> [0.500, 0.866]
Volume of unit cell (Cartesian) = 0.866 [A^d]

Reciprocal lattice vectors (Cartesian):
0 ==> [6.283, -3.628]
1 ==> [0.000, 7.255]
Volume of reciprocal unit cell = 45.586 [A^-d]

Orbital vectors (Cartesian):
0 ==> [0.000, 0.000]
1 ==> [0.500, 0.289]

Orbital vectors (fractional):
0 ==> [0.000, 0.000]
1 ==> [0.333, 0.333]

Site energies:
< 0 | H | 0 > = 0.000
< 1 | H | 1 > = 0.000
Hoppings:
< 1 | H | 0 + [1.0 , 0.0] > = 0.3000+0.0000j
< 1 | H | 0 + [0.0 , 1.0] > = 0.1000+0.0000j
< 0 | H | 1 + [0.0 , 0.0] > = 't'
Hopping distances:
| pos(1) - pos(0) + [1.0 , 0.0] | = 0.577
| pos(1) - pos(0) + [0.0 , 1.0] | = 0.577
| pos(0) - pos(1) + [0.0 , 0.0] | = 0.577 (param)
```

► 代码清单 A.30.7 (python, 4 行)

```
try:
 tb.hamiltonian()
except ValueError as exc:
 print(exc)
```

---

► 代码清单 A.30.8 (python, 1 行)

```
Missing parameter value(s): t
```

► 代码清单 A.30.9 (python, 13 行)

```
k_mesh = tb.k_uniform_mesh([20, 20])

Single value (scalar)
H_single = tb.hamiltonian(k_mesh, t=0.5)

Sweep over five scalar values (1-D array of scalars)
t_values = np.linspace(-1.0, 1.0, 5)
H_sweep = tb.hamiltonian(k_mesh, t=t_values)

print("Hamiltonian shape with t=0.5:")
print(H_single.shape)
print("Hamiltonian shape sweeping over 5 t values:")
print(H_sweep.shape)
```

► 代码清单 A.30.10 (python, 4 行)

```
Hamiltonian shape with t=0.5:
(400, 2, 2)
Hamiltonian shape sweeping over 5 t values:
(400, 5, 2, 2)
```

► 代码清单 A.30.11 (python, 7 行)

```
evals_single = tb.solve_ham(k_mesh, t=0.5) # passing a single scalar value
evals_sweep = tb.solve_ham(k_mesh, t=t_values) # passing a 1-D array of scalars

print("Energies shape with t=0.5:")
print(evals_single.shape)
print("Energies shape sweeping over 5 t values:")
print(evals_sweep.shape)
```

► 代码清单 A.30.12 (python, 4 行)

```
Energies shape with t=0.5:
(400, 2)
Energies shape sweeping over 5 t values:
(400, 5, 2)
```

---

► 代码清单 A.30.13 (python, 7 行)

```
velocity_single = tb.velocity(k_mesh, t=0.5)
velocity_sweep = tb.velocity(k_mesh, t=t_values)

print("Velocity shape with t=0.5:")
print(velocity_single.shape)
print("Velocity shape sweeping over 5 t values:")
print(velocity_sweep.shape)
```

► 代码清单 A.30.14 (python, 4 行)

```
Velocity shape with t=0.5:
(2, 400, 2, 2)
Velocity shape sweeping over 5 t values:
(3, 400, 5, 2, 2)
```

► 代码清单 A.30.15 (python, 2 行)

```
tb.set_parameters(t=0.8)
tb.parameters
```

► 代码清单 A.30.16 (python, 1 行)

```
[]
```

► 代码清单 A.30.17 (python, 42 行)

```
print(tb)

 Tight-binding model report

r-space dimension = 2
k-space dimension = 2
periodic directions = [0, 1]
spinful = False
number of spin components = 1
number of electronic states = 2
number of orbitals = 2

Lattice vectors (Cartesian):
0 ==> [1.000, 0.000]
1 ==> [0.500, 0.866]
Volume of unit cell (Cartesian) = 0.866 [A^d]
```

---

```

Reciprocal lattice vectors (Cartesian):
0 ==> [6.283, -3.628]
1 ==> [0.000, 7.255]
Volume of reciprocal unit cell = 45.586 [Å-d]

Orbital vectors (Cartesian):
0 ==> [0.000, 0.000]
1 ==> [0.500, 0.289]

Orbital vectors (fractional):
0 ==> [0.000, 0.000]
1 ==> [0.333, 0.333]

Site energies:
< 0 | H | 0 > = 0.000
< 1 | H | 1 > = 0.000
Hoppings:
< 1 | H | 0 + [1.0 , 0.0] > = 0.3000+0.0000j
< 1 | H | 0 + [0.0 , 1.0] > = 0.1000+0.0000j
< 0 | H | 1 + [0.0 , 0.0] > = 0.8000+0.0000j
Hopping distances:
| pos(1) - pos(0) + [1.0 , 0.0] | = 0.577
| pos(1) - pos(0) + [0.0 , 1.0] | = 0.577
| pos(0) - pos(1) + [0.0 , 0.0] | = 0.577

```

### ► 代码清单 A.30.19 (python, 2 行)

```

Hamiltonian shape after freezing parameters:
(400, 2, 2)

```

### ► 代码清单 A.30.20 (python, 17 行)

```

tb_symbolic = TBModel(lattice)
tb_symbolic.set_on-site([0.0, 0.0])
tb_symbolic.set_hop("t", 0, 1, [0, 0])
tb_symbolic.set_hop(0.3, 1, 0, [1, 0])
tb_symbolic.set_hop(0.1, 1, 0, [0, 1])

tb_frozen = tb_symbolic.with_parameters(t=0.3)

tb_symbolic still demands a parameter, tb_frozen does not
try:
 tb_symbolic.hamiltonian(k_mesh)
except ValueError as exc:
 print("symbolic:", exc)

H_frozen = tb_frozen.hamiltonian(k_mesh)
print("Frozen Hamiltonian shape:")
print(H_frozen.shape)

```

### ► 代码清单 A.30.21 (python, 3 行)

```

symbolic: Missing parameter value(s): t

```

---

Frozen Hamiltonian shape:  
(400, 2, 2)

### ► 代码清单 A.30.22 (python, 10 行)

```
lat_vecs = [[1, 0], [0, 1]]
orb_vecs = [[0, 0], [1 / 2, 1 / 2]]
lat = Lattice(lat_vecs=lat_vecs, orb_vecs=orb_vecs, periodic_dirs=...)
model = TBModel(lattice=lat, spinful=False)

model.set_onsite("m", ind_i=0)
model.set_onsite("m", ind_i=1)
model.set_hop("t1", 0, 1, [0, 0])
model.set_hop("t2", 1, 0, [0, 1])
print(model)
```

### ► 代码清单 A.30.23 (python, 38 行)

```

 Tight-binding model report

r-space dimension = 2
k-space dimension = 2
periodic directions = [0, 1]
spinful = False
number of spin components = 1
number of electronic states = 2
number of orbitals = 2

Lattice vectors (Cartesian):
0 ==> [1.000, 0.000]
1 ==> [0.000, 1.000]
Volume of unit cell (Cartesian) = 1.000 [A^d]

Reciprocal lattice vectors (Cartesian):
0 ==> [6.283, 0.000]
1 ==> [0.000, 6.283]
Volume of reciprocal unit cell = 39.478 [A^-d]

Orbital vectors (Cartesian):
0 ==> [0.000, 0.000]
1 ==> [0.500, 0.500]

Orbital vectors (fractional):
0 ==> [0.000, 0.000]
1 ==> [0.500, 0.500]

Site energies:
< 0 | H | 0 > = 'm'
< 1 | H | 1 > = 'm'
Hoppings:
< 0 | H | 1 + [0.0 , 0.0] > = 't1'
< 1 | H | 0 + [0.0 , 1.0] > = 't2'
Hopping distances:
| pos(0) - pos(1) + [0.0 , 0.0] | = 0.707 (param)
| pos(1) - pos(0) + [0.0 , 1.0] | = 0.707 (param)
```

---

► 代码清单 A.30.24 (python, 3 行)

```
model.set_parameters(m=1, t1=0.8)

print(model)
```

► 代码清单 A.30.25 (python, 38 行)

```

 Tight-binding model report

r-space dimension = 2
k-space dimension = 2
periodic directions = [0, 1]
spinful = False
number of spin components = 1
number of electronic states = 2
number of orbitals = 2

Lattice vectors (Cartesian):
0 ==> [1.000, 0.000]
1 ==> [0.000, 1.000]
Volume of unit cell (Cartesian) = 1.000 [A^d]

Reciprocal lattice vectors (Cartesian):
0 ==> [6.283, 0.000]
1 ==> [0.000, 6.283]
Volume of reciprocal unit cell = 39.478 [A^-d]

Orbital vectors (Cartesian):
0 ==> [0.000, 0.000]
1 ==> [0.500, 0.500]

Orbital vectors (fractional):
0 ==> [0.000, 0.000]
1 ==> [0.500, 0.500]

Site energies:
< 0 | H | 0 > = 1.000
< 1 | H | 1 > = 1.000
Hoppings:
< 0 | H | 1 + [0.0 , 0.0] > = 0.8000+0.0000j
< 1 | H | 0 + [0.0 , 1.0] > = 't2'
Hopping distances:
| pos(0) - pos(1) + [0.0 , 0.0] | = 0.707
| pos(1) - pos(0) + [0.0 , 1.0] | = 0.707 (param)
```

► 代码清单 A.30.26 (python, 2 行)

```
model.set_onsite("m", ind_i=0)
model.set_onsite("m", ind_i=1)
```



---

## pythTB 的学习 \_lattice 类

### ► 代码清单 A.31.2 (python, 2 行)

```
from pythtb import Lattice
import numpy as np
```

### ► 代码清单 A.31.5 (python, 26 行)

```

 Lattice structure report

r-space dimension = 2
k-space dimension = 2
periodic directions = [0, 1]
number of orbitals = 2

Lattice vectors (Cartesian):
0 ==> [1.000, 0.000]
1 ==> [0.500, 0.866]
Volume of unit cell (Cartesian) = 0.866 [A^d]

Reciprocal lattice vectors (Cartesian):
0 ==> [6.283, -3.628]
1 ==> [0.000, 7.255]
Volume of reciprocal unit cell = 45.586 [A^-d]

Orbital vectors (Cartesian):
0 ==> [0.500, 0.289]
1 ==> [1.000, 0.577]

Orbital vectors (fractional):
0 ==> [0.333, 0.333]
1 ==> [0.667, 0.667]

```

### ► 代码清单 A.31.6 (python, 1 行)

```
lat.get_orb_vecs(cartesian=True)
```

### ► 代码清单 A.31.7 (python, 2 行)

```
array([[0.5 , 0.28867513],
 [1. , 0.57735027]])
```

---

► 代码清单 A.31.8 (python, 2 行)

```
print(lat.get_recip_lat_vecs())
print(lat.recip_lat_vecs)
```

► 代码清单 A.31.9 (python, 4 行)

```
[[6.28318531 -3.62759873]
 [0. 7.25519746]]
[[6.28318531 -3.62759873]
 [0. 7.25519746]]
```

► 代码清单 A.31.11 (python, 2 行)

```
[[1.00000000e+00 0.00000000e+00]
 [-4.79476621e-17 1.00000000e+00]]
```

► 代码清单 A.31.12 (python, 1 行)

```
print(lat.recip_volume, lat.cell_volume)
```

► 代码清单 A.31.13 (python, 1 行)

```
45.58575006211245 0.8660254037844386
```

► 代码清单 A.31.14 (python, 90 行)

```
#!/usr/bin/env python
coding: utf-8

(lattice-nb)=
The `Lattice` class
#
In version 2.0.0 of `pythtb`, we introduced the `Lattice` class to encapsulate all information about the lattice
↪ geometry, including lattice vectors, orbital positions, and periodicity. This modular approach allows for a
↪ separation of concerns, where the `Lattice` class handles geometric details, while the `TBModel` class focuses
↪ on the tight-binding model itself. This design enhances code readability, maintainability, and reusability.

In[]:
```

---

```

from pythtb import Lattice
import numpy as np

Below, we demonstrate how to use the `Lattice` class to define a lattice and then use it to create a
↳ tight-binding model.
#
We start by defining the lattice vectors and orbital positions for a honeycomb lattice with two orbitals per
↳ unit cell. The lattice vectors are given by:
#
$$
\mathbf{a}_1 = a \hat{x}, \quad \mathbf{a}_2 = \frac{a}{2} \hat{x} + \frac{a \sqrt{3}}{2} \hat{y}
$$
#
where a is the lattice constant, which we set to 1 for simplicity. The orbital positions are given in reduced
↳ coordinates as:
#
$$
\mathbf{\tau}_1 = 0, \quad \mathbf{\tau}_2 = \frac{1}{3} \mathbf{a}_1 + \frac{1}{3} \mathbf{a}_2
$$
#
giving us a graphene-like structure.
#

In[]:

lat_vecs = [[1, 0], [1 / 2, np.sqrt(3) / 2]]
orb_vecs = [[1 / 3, 1 / 3], [2 / 3, 2 / 3]]

We pass this information to the `Lattice` class to create a lattice object. Additionally, we specify which
↳ directions are periodic. In our case, both directions are periodic.
#

In[]:

lat = Lattice(orb_vecs=orb_vecs, lat_vecs=lat_vecs, periodic_dirs=[0, 1])

We can see a report of the lattice object to verify its properties by printing it.

In[]:

print(lat)

If we want to get the positions of the orbitals in Cartesian coordinates, we can use the `get_orb_vecs` method
↳ of the `Lattice` class and set the `cartesian` argument to `True`.

In[]:

lat.get_orb_vecs(cartesian=True)

The `Lattice` class internally generates the reciprocal lattice vectors based on the provided lattice vectors
↳ and periodicity. We can access these reciprocal lattice vectors using the `get_recip_lat_vecs` method or the
↳ `recip_lat_vecs` attribute.

In[]:

print(lat.get_recip_lat_vecs())

```

---

```

print(lat.recip_lat_vecs)

Let's verify that the reciprocal lattice vectors satisfy the orthogonality condition with the real-space lattice
↪ vectors.
#
$\mathbf{a}_i \cdot \mathbf{b}_j = 2\pi \delta_{ij}$
In []:

overlap_mat = lat.lat_vecs @ lat.recip_lat_vecs.T
print(overlap_mat / (2 * np.pi))

If we like, we can also get the volume of the unit cell in both real and reciprocal space. These are stored as
↪ attributes of the `Lattice` class.
In []:

print(lat.recip_volume, lat.cell_volume)

```

## pythTB 的学习 \_mesh 类

### ► 代码清单 A.32.1 (python, 2 行)

```

from pythtb import Mesh
import numpy as np

```

### ► 代码清单 A.32.2 (python, 5 行)

```

mesh = Mesh(["k"], dim_k=2)
points = np.linspace(
 [0, 0], [1, 1], 10, endpoint=False
) # path from (0, 0) to (0.5, 0.5) to (1, 1)
mesh.build_custom(points)

```

### ► 代码清单 A.32.3 (python, 3 行)

```

print("Mesh shape:", mesh.shape)
print("Mesh points along the path from (0,0) to (1,1):")
print(mesh.points)

```

---

### ► 代码清单 A.32.4 (python, 12 行)

```
Mesh shape: (10, 2)
Mesh points along the path from (0,0) to (1,1):
[[0. 0.]
 [0.1 0.1]
 [0.2 0.2]
 [0.3 0.3]
 [0.4 0.4]
 [0.5 0.5]
 [0.6 0.6]
 [0.7 0.7]
 [0.8 0.8]
 [0.9 0.9]]
```

### ► 代码清单 A.32.5 (python, 1 行)

```
print(mesh)
```

### ► 代码清单 A.32.6 (python, 9 行)

```
Mesh Summary
=====
Type: path
Dimensionality: 2 k-dim(s) + 0 λ -dim(s)
Number of mesh points: 10
Full shape: (10, 2)
k-axes: [Axis(type=k, name=k_0, size=10)]
 λ -axes: []
Loops: None
```

### ► 代码清单 A.32.7 (python, 6 行)

```
Wind the first k-space dimension
mesh.loop(axis_idx=0, component_idx=0, winds_bz=True, closed=False)

Now wind the second k-space dimension
mesh.loop(axis_idx=0, component_idx=1, winds_bz=True, closed=False)
print(mesh)
```

### ► 代码清单 A.32.8 (python, 9 行)

```
Mesh Summary
=====
Type: path
```

---

```
Dimensionality: 2 k-dim(s) + 0 λ-dim(s)
Number of mesh points: 10
Full shape: (10, 2)
k-axes: [Axis(type=k, name=k_0, size=10)]
λ-axes: []
Loops: (axis 0, comp 0, winds_bz=yes, closed=no), (axis 0, comp 1, winds_bz=yes, closed=no)
```

### ► 代码清单 A.32.9 (python, 7 行)

```
mesh = Mesh(dim_k=2, axis_types=["k"])

Path from (-0.5,-0.5) to (0.5, 0.5) including endpoints (endpoint=True)
points = np.linspace([-0.5, -0.5], [0.5, 0.5], 10, endpoint=True)

mesh.build_custom(points)
print(mesh)
```

### ► 代码清单 A.32.10 (python, 9 行)

```
Mesh Summary
=====
Type: path
Dimensionality: 2 k-dim(s) + 0 λ-dim(s)
Number of mesh points: 10
Full shape: (10, 2)
k-axes: [Axis(type=k, name=k_0, size=10)]
λ-axes: []
Loops: (axis 0, comp 0, winds_bz=yes, closed=yes), (axis 0, comp 1, winds_bz=yes, closed=yes)
```

## pythTB 的学习 \_tb\_model to TBModel in v2.0

### ► 代码清单 A.33.1 (python, 2 行)

```
from pythtb import TBModel, Lattice
import numpy as np
```

### ► 代码清单 A.33.2 (python, 6 行)

```
v1.8 code
from pythtb import tb_model

lat_vecs = [[1, 0], [1/2, np.sqrt(3) / 2]]
orb_vecs = [[1/3, 1/3], [2/3, 2/3]]
model = tb_model(dim_r = 2, dim_k = 2, lat=lat_vecs, orb=orb_vecs, per=[0,1], nspin=1)
```

---

### ► 代码清单 A.33.4 (python, 14 行)

```
delta = 0
t1 = -1
t2 = 0.15
phi = np.pi / 2

model.set_onsite([-delta, delta], mode="set")

for lvec in ([0, 0], [-1, 0], [0, -1]):
 model.set_hop(t1, 0, 1, lvec, mode="set")

for lvec in ([1, 0], [-1, 1], [0, -1]):
 model.set_hop(t2 * np.exp(1j * phi), 0, 0, lvec, mode="set")
for lvec in ([-1, 0], [1, -1], [0, 1]):
 model.set_hop(t2 * np.exp(1j * phi), 1, 1, lvec, mode="set")
```

### ► 代码清单 A.33.5 (python, 3 行)

```
from pythtb.models import haldane

my_model = haldane(delta=0.1, t1=1.0, t2=0.1)
```

### ► 代码清单 A.33.6 (python, 7 行)

```
v1.8
my_model.display()

v2.0
print(my_model)
or
my_model.info()
```

### ► 代码清单 A.33.7 (python, 1 行)

```
print(my_model)
```

### ► 代码清单 A.33.8 (python, 52 行)

```

Tight-binding model report

r-space dimension = 2
k-space dimension = 2
periodic directions = [0, 1]
```

```

spinful = False
number of spin components = 1
number of electronic states = 2
number of orbitals = 2

Lattice vectors (Cartesian):
0 ==> [1.000, 0.000]
1 ==> [0.500, 0.866]
Volume of unit cell (Cartesian) = 0.866 [A^d]

Reciprocal lattice vectors (Cartesian):
0 ==> [6.283, -3.628]
1 ==> [0.000, 7.255]
Volume of reciprocal unit cell = 45.586 [A^-d]

Orbital vectors (Cartesian):
0 ==> [0.500, 0.289]
1 ==> [1.000, 0.577]

Orbital vectors (fractional):
0 ==> [0.333, 0.333]
1 ==> [0.667, 0.667]

Site energies:
< 0 | H | 0 > = -0.100
< 1 | H | 1 > = 0.100
Hoppings:
< 0 | H | 1 + [0.0 , 0.0] > = 1.0000+0.0000j
< 0 | H | 1 + [-1.0 , 0.0] > = 1.0000+0.0000j
< 0 | H | 1 + [0.0 , -1.0] > = 1.0000+0.0000j
< 0 | H | 0 + [1.0 , 0.0] > = 0.0000+0.1000j
< 0 | H | 0 + [-1.0 , 1.0] > = 0.0000+0.1000j
< 0 | H | 0 + [0.0 , -1.0] > = 0.0000+0.1000j
< 1 | H | 1 + [-1.0 , 0.0] > = 0.0000+0.1000j
< 1 | H | 1 + [1.0 , -1.0] > = 0.0000+0.1000j
< 1 | H | 1 + [0.0 , 1.0] > = 0.0000+0.1000j
Hopping distances:
| pos(0) - pos(1) + [0.0 , 0.0] | = 0.577
| pos(0) - pos(1) + [-1.0 , 0.0] | = 0.577
| pos(0) - pos(1) + [0.0 , -1.0] | = 0.577
| pos(0) - pos(0) + [1.0 , 0.0] | = 1.000
| pos(0) - pos(0) + [-1.0 , 1.0] | = 1.000
| pos(0) - pos(0) + [0.0 , -1.0] | = 1.000
| pos(1) - pos(1) + [-1.0 , 0.0] | = 1.000
| pos(1) - pos(1) + [1.0 , -1.0] | = 1.000
| pos(1) - pos(1) + [0.0 , 1.0] | = 1.000

```

### ► 代码清单 A.33.9 (python, 2 行)

```

v1.8
my_model.visualize(0, 1)

```

### ► 代码清单 A.33.10 (python, 3 行)

```

Generate k-points
nkx, nky = 20, 20
k_pts = my_model.k_uniform_mesh([nkx, nky])

```



### ► 代码清单 A.33.12 (python, 7 行)

```
Hamiltonian shape: (400, 2, 2)
Hamiltonian at first k-point:
[[-0.1+0.j 3. +0.j]
 [3. +0.j 0.1+0.j]]
Hamiltonian at second k-point:
[[-0.1 +0.j 2.96365774+0.00133535j]
 [2.96365774-0.00133535j 0.1 +0.j]]
```

### ► 代码清单 A.33.13 (python, 17 行)

```
v1.8 code
path=[[0.,0.],[2./3.,1./3.],[.5,.5],[1./3.,2./3.], [0.,0.]]
label=(r'Γ',r'K', r'M', r'$K^\prime$', r'Γ')

(k_vec,k_dist,k_node) = my_model.k_path(path,101)
evals = my_model.solve_all(k_vec)

fig, ax = plt.subplots()
ax.set_xlim(k_node[0],k_node[-1])
ax.set_xticks(k_node)
ax.set_xticklabels(label)
for n in range(len(k_node)):
 ax.axvline(x=k_node[n],linewidth=0.5, color='k')
ax.set_ylabel("Band energy")

ax.plot(k_dist,evals[0])
ax.plot(k_dist,evals[1])
```

### ► 代码清单 A.33.15 (python, 3 行)

```
fig, ax = my_model.plot_bands(
 nk=500, k_nodes=k_nodes, k_node_labels=k_label, proj_orb_idx=[1]
)
```

### ► 代码清单 A.33.16 (python, 2 行)

```
from pythtb import tb_model
import numpy as np
```

### ► 代码清单 A.33.17 (python, 31 行)

```
define lattice vectors
lat = [[1.0, 0.0], [0.5, np.sqrt(3.0) / 2.0]]
```

---

```

define coordinates of orbitals
orb = [[1.0 / 3.0, 1.0 / 3.0], [2.0 / 3.0, 2.0 / 3.0]]

make two dimensional tight-binding Haldane model
my_model = tb_model(2, 2, lat, orb)

set model parameters
delta = 0.2
t = -1.0
t2 = 0.15 * np.exp((1.0j) * np.pi / 2.0)
t2c = t2.conjugate()

set on-site energies
my_model.set_onsite([-delta, delta])
set hoppings (one for each connected pair of orbitals)
(amplitude, i, j, [lattice vector to cell containing j])
my_model.set_hop(t, 0, 1, [0, 0])
my_model.set_hop(t, 1, 0, [1, 0])
my_model.set_hop(t, 1, 0, [0, 1])
add second neighbour complex hoppings
my_model.set_hop(t2, 0, 0, [1, 0])
my_model.set_hop(t2, 1, 1, [1, -1])
my_model.set_hop(t2, 1, 1, [0, 1])
my_model.set_hop(t2c, 1, 1, [1, 0])
my_model.set_hop(t2c, 0, 0, [1, -1])
my_model.set_hop(t2c, 0, 0, [0, 1])

print tight-binding model
my_model.display()

```

## ► 代码清单 A.33.18 (python, 52 行)

```

 Tight-binding model report

r-space dimension = 2
k-space dimension = 2
periodic directions = [0, 1]
spinful = False
number of spin components = 1
number of electronic states = 2
number of orbitals = 2

Lattice vectors (Cartesian):
0 ==> [1.000, 0.000]
1 ==> [0.500, 0.866]
Volume of unit cell (Cartesian) = 0.866 [A^d]

Reciprocal lattice vectors (Cartesian):
0 ==> [6.283, -3.628]
1 ==> [0.000, 7.255]
Volume of reciprocal unit cell = 45.586 [A^-d]

Orbital vectors (Cartesian):
0 ==> [0.500, 0.289]
1 ==> [1.000, 0.577]

Orbital vectors (fractional):
0 ==> [0.333, 0.333]
1 ==> [0.667, 0.667]

Site energies:
< 0 | H | 0 > = -0.200

```

```

< 1 | H | 1 > = 0.200
Hoppings:
< 0 | H | 1 + [0.0 , 0.0] > = -1.0000+0.0000j
< 1 | H | 0 + [1.0 , 0.0] > = -1.0000+0.0000j
< 1 | H | 0 + [0.0 , 1.0] > = -1.0000+0.0000j
< 0 | H | 0 + [1.0 , 0.0] > = 0.0000+0.1500j
< 1 | H | 1 + [1.0 , -1.0] > = 0.0000+0.1500j
< 1 | H | 1 + [0.0 , 1.0] > = 0.0000+0.1500j
< 1 | H | 1 + [1.0 , 0.0] > = 0.0000-0.1500j
< 0 | H | 0 + [1.0 , -1.0] > = 0.0000-0.1500j
< 0 | H | 0 + [0.0 , 1.0] > = 0.0000-0.1500j
Hopping distances:
| pos(0) - pos(1) + [0.0 , 0.0] | = 0.577
| pos(1) - pos(0) + [1.0 , 0.0] | = 0.577
| pos(1) - pos(0) + [0.0 , 1.0] | = 0.577
| pos(0) - pos(0) + [1.0 , 0.0] | = 1.000
| pos(1) - pos(1) + [1.0 , -1.0] | = 1.000
| pos(1) - pos(1) + [0.0 , 1.0] | = 1.000
| pos(1) - pos(1) + [1.0 , 0.0] | = 1.000
| pos(0) - pos(0) + [1.0 , -1.0] | = 1.000
| pos(0) - pos(0) + [0.0 , 1.0] | = 1.000

```

### ► 代码清单 A.33.19 (python, 4 行)

```

/tmp/ipykernel_1286/2945160337.py:7: DeprecationWarning: pythtb.tb_model is deprecated and will be removed in a
↳ future release. Use TBModel instead.
 my_model = tb_model(2, 2, lat, orb)
/tmp/ipykernel_1286/2945160337.py:31: FutureWarning: TBModel.display is deprecated and will be removed in a future
↳ release: The 'display()' method is deprecated and will be removed in a future release. Use 'print(model)' or
↳ 'model.info(show=True)' instead.
 my_model.display()

```

## pythTB 的学习 \_wf\_array to in v2.0WFArray

### ► 代码清单 A.34.1 (python, 5 行)

```

v1.x
from pythtb import tb_model, wf_array
model = tb_model(1, 1, lat=[[1.0, 0.0], [0.0, 1.0]], orb=[[0, 1/3, 2/3]])
wfa = wf_array(model, [20])
wfa.solve_on_grid(0.0)

```

### ► 代码清单 A.34.4 (python, 1 行)

```

mesh.build_grid([20])

```

---

### ► 代码清单 A.34.6 (python, 18 行)

```
t = -1.3
delta = 2.0

model = TBModel(lattice=lattice)

nearest-neighbour hoppings (last hop wraps to the next cell)
model.set_hop(t, 0, 1, [0])
model.set_hop(t, 1, 2, [0])
model.set_hop(t, 2, 0, [1])

Per-orbital onsite as lambdas of lambda
onsite = [
 lambda lambda: delta * -np.cos(2 * np.pi * (lambda - 0 / 3)),
 lambda lambda: delta * -np.cos(2 * np.pi * (lambda - 1 / 3)),
 lambda lambda: delta * -np.cos(2 * np.pi * (lambda - 2 / 3)),
]

model.set_onsite(onsite)
```

### ► 代码清单 A.34.8 (python, 1 行)

```
wfa[2]
```

### ► 代码清单 A.34.9 (python, 6 行)

```
array([[0.48119032+0.j , 0.80505946-0.06370792j,
 0.33915892-0.0353223j],
 [0.77558258+0.j , -0.54837962-0.05902583j,
 0.21322754+0.22091043j],
 [-0.40856766+0.j , -0.09282883-0.1870805j ,
 0.80421332+0.3777527j]])
```

### ► 代码清单 A.34.11 (python, 7 行)

```
mesh.build_grid(
 shape=(31, 21),
 gamma_centered=True,
 lambda_start=0.0,
 lambda_stop=1.0,
 lambda_endpoints=True,
)
```

### ► 代码清单 A.34.12 (python, 4 行)

```
mesh.loop(
```

```

 axis_idx=1, component_idx=1, closed=True
) # form the lambda axis into a closed loop
print(mesh)

```

### ► 代码清单 A.34.13 (python, 10 行)

```

Mesh Summary
=====
Type: grid
Dimensionality: 1 k-dim(s) + 1 λ -dim(s)
Number of mesh points: 651
Full shape: (31, 21, 2)
k-axes: [Axis(type=k, name=kx, size=31)]
 λ -axes: [Axis(type=l, name=lmbda, size=21)]
Is a torus in k-space (all k-axes wind BZ): yes
Loops: (axis 0, comp 0, winds_bz=yes, closed=no), (axis 1, comp 1, winds_bz=no, closed=yes)

```

### ► 代码清单 A.34.15 (python, 1 行)

```

wfa.solve_model(model=model)

```

### ► 代码清单 A.34.16 (python, 1 行)

```

np.allclose(wfa[0, 0], wfa[0, -1]) # first k-point, first and last lambda points

```

### ► 代码清单 A.34.17 (python, 1 行)

```

True

```

## vasp2kp 的学习 \_1

### ► 代码清单 A.35.1 (python, 1 行)

```

INCAR KPOINTS POTCAR POSCAR INCAR.mat WAVECAR

```

---

## ir2TB 的学习

### ► 代码清单 A.36.1 (shell, 1 行)

```
vi EIGENVAL
```

### ► 代码清单 A.36.2 (shell, 1 行)

```
irvsp2 -sg 187 -nb 13 13>outir2
```

## irvsp 的学习

### ► 代码清单 A.37.1 (shell, 1 行)

```
vi EIGENVAL
```

### ► 代码清单 A.37.2 (shell, 1 行)

```
irvsp2 -sg 187 -nb 13 13>outir2
```

## (原创脚本) 角态验证 1-配置 pybinding 环境-Ubuntu 系统配置

### ► 代码清单 A.38.1 (shell, 2 行)

```
https://cmake.org/download/#latest
https://visualstudio.microsoft.com/zh-hans/visual-cpp-build-tools/
```

### ► 代码清单 A.38.2 (shell, 7 行)

```
❑ Add CMake to the system PATH for all users
```

```
❑ C++ build tools
```

---

☑ MSVC v143

☑ Windows 10/11 SDK

### ► 代码清单 A.38.7 (shell, 7 行)

```
conda create -n pybinding python=3.10
conda activate pybinding
conda install numpy matplotlib scipy
python -m pip install -U "pip<25.3"
pip install pybinding-dev==1.0.6 --no-use-pep517 --no-cache-dir
pip show pybinding-dev
conda install -n pybinding ipykernel --update-deps --force-reinstall
```

## 网站维护与操作手册

### ► 代码清单 A.40.1 (plaintext, 1 行)

```
<div class="col-lg-8 col-md-10 mx-auto">
```

### ► 代码清单 A.40.2 (text, 161 行)

```
// 分页配置
const POSTS_PER_PAGE = 5;

// 初始化分页功能
function initPagination() {
 // 检查数据是否已加载
 if (!window.allPosts || window.allPosts.length === 0) {
 console.error(" 文章数据未加载! ");
 document.getElementById('posts-container').innerHTML = '<p class="text-muted text-center py-5">No posts
↵ available</p>';
 document.getElementById('pagination').style.display = 'none';
 return;
 }

 const postsContainer = document.getElementById('posts-container');
 const prevBtn = document.getElementById('prev-btn');
 const nextBtn = document.getElementById('next-btn');
 const currentPageSpan = document.getElementById('current-page');
 const totalPagesSpan = document.getElementById('total-pages');

 // 获取当前分类 (从页面变量获取)
 const currentCategory = window.currentCategory;
 console.log('当前页面分类:', currentCategory);

 // [OK] 修正: 严格根据分类标签筛选文章
 let filteredPosts = window.allPosts;
```

---

```

if (currentCategory && currentCategory !== 'null' && currentCategory !== 'undefined') {
 console.log('开始筛选文章, 目标分类:', currentCategory);

 filteredPosts = window.allPosts.filter(post => {
 if (!post.categories || post.categories.length === 0) {
 return false;
 }

 // [OK] 修正: 严格匹配分类标签 (不区分大小写)
 const hasMatchingCategory = post.categories.some(category => {
 // 标准化分类标签比较
 const normalizedCategory = category.toLowerCase().replace(/\s+/g, '-');
 const normalizedTarget = currentCategory.toLowerCase().replace(/\s+/g, '-');
 return normalizedCategory === normalizedTarget;
 });

 return hasMatchingCategory;
 });

 console.log('筛选后文章数量:', filteredPosts.length);
 console.log('匹配的文章:', filteredPosts.map(p => ({
 title: p.title,
 categories: p.categories
 })));
} else {
 console.log('未设置分类, 显示所有文章');
}

// [OK] 去除重复文章 (基于 URL 去重)
const uniquePosts = [];
const seenUrls = new Set();

filteredPosts.forEach(post => {
 if (!seenUrls.has(post.url)) {
 seenUrls.add(post.url);
 uniquePosts.push(post);
 }
});

filteredPosts = uniquePosts;

const totalPages = Math.ceil(filteredPosts.length / POSTS_PER_PAGE);
totalPagesSpan.textContent = totalPages;

let currentPage = 1;

function renderPosts() {
 postsContainer.innerHTML = '';

 // 如果没有文章
 if (filteredPosts.length === 0) {
 postsContainer.innerHTML = `
 <div class="no-posts text-center py-5">
 <h3 class="text-muted"> 暂无文章</h3>
 <p class="text-muted"> 分类 "${currentCategory || '所有文章'}" 还没有发布任何文章。</p>
 </div>
 `;
 document.getElementById('pagination').style.display = 'none';
 return;
 }

 const startIndex = (currentPage - 1) * POSTS_PER_PAGE;
 const endIndex = Math.min(startIndex + POSTS_PER_PAGE, filteredPosts.length);

 for (let i = startIndex; i < endIndex; i++) {
 const post = filteredPosts[i];
 const postElement = document.createElement('article');
 postElement.className = 'post-preview';
 }
}

```



```

// [OK] 根据图片格式生成文章内容 (不显示分类标签)
postElement.innerHTML = `

 <h2 class="post-title">${post.title}</h2>
 <h3 class="post-subtitle">${post.subtitle || ''}</h3>

 <p class="post-meta">
 Posted by ${post.author} on ${post.date} · ${post.read_time}
 </p>
`;

postsContainer.appendChild(postElement);

// 添加分隔线 (除了最后一篇文章)
if (i < endIndex - 1) {
 const hr = document.createElement('hr');
 postsContainer.appendChild(hr);
}
}

function updatePagination() {
 currentPageSpan.textContent = currentPage;
 prevBtn.disabled = currentPage === 1;
 nextBtn.disabled = currentPage === totalPages;

 // 更新按钮样式
 prevBtn.classList.toggle('disabled', currentPage === 1);
 nextBtn.classList.toggle('disabled', currentPage === totalPages);

 // 如果没有文章或只有一页, 隐藏分页控件
 if (filteredPosts.length <= POSTS_PER_PAGE) {
 document.getElementById('pagination').style.display = 'none';
 } else {
 document.getElementById('pagination').style.display = 'flex';
 }
}

function initPaginationControls() {
 renderPosts();
 updatePagination();

 prevBtn.addEventListener('click', () => {
 if (currentPage > 1) {
 currentPage--;
 renderPosts();
 updatePagination();
 window.scrollTo({ top: 0, behavior: 'smooth' });
 }
 });

 nextBtn.addEventListener('click', () => {
 if (currentPage < totalPages) {
 currentPage++;
 renderPosts();
 updatePagination();
 window.scrollTo({ top: 0, behavior: 'smooth' });
 }
 });
}

initPaginationControls();
}

// 页面加载时初始化
document.addEventListener('DOMContentLoaded', initPagination);

```

## ► 代码清单 A.40.3 (text, 72 行)

```

layout: default

<!-- Page Header -->
{% if page.background %}
<header class="masthead" style="background-image: url('{{ page.background | prepend: site.baseurl | replace: '//',
↪ '/' }}')">
{% else %}
<header class="masthead">
{% endif %}
 <div class="overlay"></div>
 <div class="container">
 <div class="row">
 <div class="col-lg-8 col-md-10 mx-auto">
 <div class="page-heading">
 <h1>{{ page.title }}</h1>
 {% if page.description %}
 {{ page.description }}
 {% endif %}
 </div>
 </div>
 </div>
 </div>
</header>

<div class="container">
 <div class="row">
 <div class="col-lg-8 col-md-10 mx-auto">

 {{ content }}

 <!-- [OK] 动态文章容器 -->
 <div id="posts-container"></div>

 <!-- [OK] 分页控件 -->
 <div id="pagination" class="pagination">
 <button id="prev-btn" class="pagination-btn"><- 上一页</button>

 第 1 页 / 共 0 页

 <button id="next-btn" class="pagination-btn">下一页 -></button>
 </div>

 <!-- [OK] 输出文章数据给 JS -->
 <script>
 window.allPosts = [
 {% for post in site.posts %}
 {
 title: {{ post.title | jsonify }},
 url: "{{ post.url | relative_url }}",
 date: "{{ post.date | date: '%B %d, %Y' }}",
 {% if post.subtitle %}
 subtitle: {{ post.subtitle | jsonify }},
 {% else %}
 subtitle: {{ post.excerpt | strip_html | truncatewords: 15 | jsonify }},
 {% endif %}
 excerpt: {{ post.excerpt | strip_html | jsonify }},
 author: "{{ post.author | default: site.author }}",
 categories: {{ post.categories | jsonify }},
 {% assign words = post.content | number_of_words %}
 read_time: "{% if words < 120 %}1 min read{% else %}{{ words | divided_by:120 }} mins read{% endif
↪ %}"
 }{% unless forloop.last %},{% endunless %}
]{% endfor %}
 </script>
 </div>
 </div>
</div>
```

```

 };
 </script>

 <!-- [OK] 分页脚本 -->
 <script src="{ { '/assets/vendor/startbootstrap-clean-blog/js/pagination.js' | relative_url }}"></script>

</div>
</div>
</div>

```

## ► 代码清单 A.40.4 (text, 41 行)

```

layout: page
title: SCI-NOTE
permalink: /sci-note/
category: sci-note
background: '/img/bg-sci-note.jpg'

<p> 科学笔记和文献整理</p>

<div id="posts-container"></div>
<div id="pagination" class="pagination">
 <button id="prev-btn" class="pagination-btn">← 上一页</button>
 第 1 页 / 共 0 页
 <button id="next-btn" class="pagination-btn">下一页 →</button>
</div>

<script>
window.currentCategory = 'sci-note';
window.allPosts = [
 {% for post in site.posts %}
 {
 title: {{ post.title | jsonify }},
 url: "{{ post.url | relative_url }}",
 date: "{{ post.date | date: '%B %d, %Y' }}",
 {% if post.subtitle %}
 subtitle: {{ post.subtitle | jsonify }},
 {% else %}
 subtitle: {{ post.excerpt | strip_html | truncatewords: 15 | jsonify }},
 {% endif %}
 excerpt: {{ post.excerpt | strip_html | jsonify }},
 author: "Start Bootstrap",
 categories: {{ post.categories | jsonify }},
 {% assign words = post.content | number_of_words %}
 read_time: "{% if words < 120 %}1 min read{% else %}{{ words | divided_by:120 }} mins read{% endif %}"
 }{% unless forloop.last %},{% endunless %}
 {% endfor %}
];
</script>

<script src="{ { '/assets/vendor/startbootstrap-clean-blog/js/pagination.js' | relative_url }}"></script>

```

## ► 代码清单 A.40.5 (text, 226 行)

```

```

---

```

Only the main Sass file needs front matter (the dashes are enough)

@import "styles";

/* === 自定义代码块样式 === */
pre code {
 display: block;
 padding: 1em;
 background-color: #282c34;
 color: #abb2bf;
 border-radius: 8px;
 overflow-x: auto;
 overflow-y: auto;
 max-height: 500px;
 position: relative;
 font-size: 0.9em;
 line-height: 1.4;
 white-space: pre;
}

pre code::-webkit-scrollbar {
 height: 8px;
 width: 8px;
}

pre code::-webkit-scrollbar-thumb {
 background: #5c6370;
 border-radius: 4px;
}

/* ===== */
/* 分页文章列表样式 - 精确匹配图片效果 */
/* ===== */

#posts-container {
 .post-preview {
 margin-bottom: 1.5rem;

 > a {
 color: #212529;
 text-decoration: none;

 &:focus,
 &:hover {
 text-decoration: none;
 color: #0085A1;

 .post-title {
 color: #0085A1;
 }
 }

 .post-title {
 font-size: 30px;
 font-weight: 700;
 margin-top: 25px;
 margin-bottom: 5px;
 line-height: 1.2;
 font-family: 'Open Sans', 'Helvetica Neue', Helvetica, Arial, sans-serif;
 }

 .post-subtitle {
 font-weight: 300;
 margin: 0 0 8px; /* [OK] 调整副标题底部间距 */
 font-size: 24px; /* [OK] 确保副标题字体大小正常 */
 line-height: 1.3;
 font-family: 'Open Sans', 'Helvetica Neue', Helvetica, Arial, sans-serif;
 color: #6c757d;
 }
 }
 }
}

```

```

 }
}

/* [X] 删除分类信息样式 */
/* .post-category {
 font-size: 16px;
 font-weight: 600;
 color: #007bff;
 margin-bottom: 6px;
 font-family: 'Open Sans', 'Helvetica Neue', Helvetica, Arial, sans-serif;
 letter-spacing: 0.3px;
} */

.post-meta {
 font-size: 18px;
 font-style: italic;
 margin-top: 0;
 margin-bottom: 8px;
 color: #6c757d;
 font-family: 'Lora', 'Times New Roman', serif;
 line-height: 1.5;

 a {
 color: #212529;
 text-decoration: none;

 &:focus,
 &:hover {
 text-decoration: underline;
 color: #0085A1;
 }
 }
}

@media only screen and (min-width: 768px) {
 > a {
 .post-title {
 font-size: 36px;
 }
 }
}
}

/* 分隔线样式 */
hr {
 margin: 8px 0 !important;
 border: 0;
 border-top: 1px solid rgba(0, 0, 0, 0.1);
 width: 100%;
}

/* 无文章提示样式 */
.no-posts {
 text-align: center;
 padding: 3rem 0;

 h3 {
 color: #6c757d;
 margin-bottom: 1rem;
 font-size: 1.5rem;
 }

 p {
 color: #6c757d;
 margin-bottom: 0.5rem;
 font-size: 1.1rem;
 }
}

```

```

/* ===== */
/* 分页控件样式 */
/* ===== */

#pagination {
 display: flex;
 justify-content: center;
 align-items: center;
 margin: 1.5rem 0 2rem !important;
 padding: 1rem 0 !important;
 border-top: 1px solid #dee2e6;

 .pagination-btn {
 padding: 8px 16px;
 margin: 0 10px;
 background-color: #fff;
 border: 1px solid #007bff;
 border-radius: 4px;
 cursor: pointer;
 transition: all 0.3s;
 font-size: 14px;
 color: #007bff;
 font-weight: 600;

 &:hover:not(.disabled) {
 background-color: #007bff;
 color: #fff;
 }

 &.disabled {
 opacity: 0.5;
 cursor: not-allowed;
 color: #6c757d;
 border-color: #6c757d;
 }
 }

 .page-info {
 margin: 0 15px;
 color: #6c757d;
 font-size: 16px;
 }
}

/* ===== */
/* 响应式设计 */
/* ===== */

@media (max-width: 768px) {
 #posts-container .post-preview {
 margin-bottom: 1rem;

 > a {
 .post-title {
 font-size: 28px;
 margin-top: 20px;
 margin-bottom: 4px;
 }

 .post-subtitle {
 font-size: 20px;
 margin: 0 0 6px; /* [OK] 移动端调整副标题间距 */
 }
 }

 .post-meta {
 font-size: 16px;
 margin-bottom: 6px;
 }
 }
}

```

---

```
 }
 }

 #posts-container hr {
 margin: 6px 0 !important;
 }

 #pagination {
 margin: 1rem 0 1.5rem !important;
 padding: 0.8rem 0 !important;
 flex-direction: column;
 gap: 8px;

 .page-info {
 margin: 8px 0;
 }
 }
}
```

#### ► 代码清单 A.40.6 (text, 4 行)

```
<!-- [OK] 设置分类变量供 JavaScript 使用 -->
<script>
window.currentCategory = 'Sci-Log';
</script>
```

### (原创自研) 网站维护记录 1

#### ► 代码清单 A.41.1 (text, 1 行)

```
_includes/navbar.html
```

#### ► 代码清单 A.41.2 (text, 1 行)

```
_includes/head.html
```

#### ► 代码清单 A.41.3 (html, 1 行)

```
<link rel="stylesheet" href="{{"/assets/main.css" | relative_url }}">
```

---

► 代码清单 A.41.5 (text, 1 行)

`_layouts/default.html`

► 代码清单 A.41.6 (text, 1 行)

`assets/vendor/startbootstrap-clean-blog/scss/_navbar.scss`

► 代码清单 A.41.7 (text, 1 行)

`assets/main.css`

► 代码清单 A.41.8 (text, 1 行)

`assets/main.scss`

► 代码清单 A.41.9 (html, 1 行)

`<div class="container">`

► 代码清单 A.41.10 (html, 1 行)

`<div class="container">`

► 代码清单 A.41.11 (html, 1 行)

`<div class="container-fluid">`

► 代码清单 A.41.12 (text, 1 行)

`assets/vendor/startbootstrap-clean-blog/scss/_navbar.scss`



---

▶ 代码清单 A.41.13 (text, 1 行)

```
_includes/head.html
```

▶ 代码清单 A.41.14 (html, 1 行)

```
<link rel="stylesheet" href="{{"/assets/main.css" | relative_url }}">
```

▶ 代码清单 A.41.15 (text, 1 行)

```
assets/main.css
```

▶ 代码清单 A.41.16 (text, 1 行)

```
_includes/navbar.html
```

▶ 代码清单 A.41.17 (html, 1 行)

```
<div class="container">
```

▶ 代码清单 A.41.18 (html, 1 行)

```
<div class="container-fluid">
```

▶ 代码清单 A.41.19 (html, 1 行)

```
<div class="nav-inner">
```

▶ 代码清单 A.41.20 (html, 1 行)

```
<ul class="navbar-nav ml-auto">
```

---

## ► 代码清单 A.41.21 (html, 1 行)

```
<ul class="navbar-nav">
```

## ► 代码清单 A.41.22 (html, 48 行)

```
<!-- Navigation -->
<nav class="navbar navbar-expand-lg navbar-light fixed-top" id="mainNav">
 <div class="container-fluid">
 <div class="nav-inner">
 {{ site.title | escape }}

 <button class="navbar-toggler navbar-toggler-right" type="button"
 data-toggle="collapse" data-target="#navbarResponsive"
 aria-controls="navbarResponsive" aria-expanded="false"
 aria-label="Toggle navigation">

 Menu
 <i class="fa fa-bars"></i>
 </button>

 <div class="collapse navbar-collapse" id="navbarResponsive">
 <ul class="navbar-nav">
 <li class="nav-item">
 Home

 <li class="nav-item">
 About

 <li class="nav-item">
 Life

 <li class="nav-item">
 Sci-log

 <li class="nav-item">
 Sci-note

 <li class="nav-item">
 Reading

 <li class="nav-item">
 Learning

 <li class="nav-item">
 Writing

 <li class="nav-item">
 Search

 </div>
 </div>
 </div>
</nav>
```

---

► 代码清单 A.41.23 (text, 1 行)

assets/main.css

► 代码清单 A.41.24 (text, 1 行)

assets/main.scss

► 代码清单 A.41.25 (css, 1 行)

6rem

► 代码清单 A.41.26 (css, 31 行)

```
@media (min-width: 992px) {
 #mainNav .nav-inner {
 width: 100%;
 max-width: 1400px;
 margin: 0 auto;
 display: flex;
 align-items: center;
 }

 #mainNav .navbar-brand {
 white-space: nowrap;
 flex-shrink: 0;
 }

 #mainNav .navbar-collapse {
 display: flex !important;
 flex-grow: 0 !important;
 justify-content: flex-start !important;
 margin-left: 6rem !important;
 }

 #mainNav .navbar-nav {
 margin-left: 0 !important;
 }

 #mainNav .navbar-nav .nav-link {
 white-space: nowrap !important;
 padding-left: 12px !important;
 padding-right: 12px !important;
 }
}
```

---

► 代码清单 A.41.27 (css, 1 行)

```
margin-left: 6rem !important;
```

► 代码清单 A.41.28 (text, 1 行)

```
_includes/navbar.html
```

► 代码清单 A.41.29 (text, 1 行)

```
_includes/head.html
```

► 代码清单 A.41.30 (text, 1 行)

```
_layouts/default.html
```

► 代码清单 A.41.31 (text, 1 行)

```
assets/main.css
```

► 代码清单 A.41.32 (text, 1 行)

```
assets/main.scss
```

► 代码清单 A.41.33 (text, 1 行)

```
assets/vendor/startbootstrap-clean-blog/scss/_navbar.scss
```

---

## (原创自研) 网站维护记录 2

### ► 代码清单 A.42.1 (text, 1 行)

```
_includes/navbar.html
```

### ► 代码清单 A.42.2 (text, 1 行)

```
assets/vendor/startbootstrap-clean-blog/js/pagination.js
```

### ► 代码清单 A.42.3 (text, 1 行)

```
_config.yml
```

### ► 代码清单 A.42.4 (yaml, 26 行)

```
plugins:
 - jekyll-feed
 - jekyll-paginate-v2
 - jekyll-sitemap

pagination:
 enabled: true
 per_page: 5
 permalink: "/page:num/"
 sort_reverse: true

autopages:
 enabled: true
 categories:
 enabled: true
 layouts:
 - autopage_category.html
 title: ":cat"
 permalink: "/:cat/"
 slugify:
 mode: "default"
 case: false
 tags:
 enabled: false
 collections:
 enabled: false
```

---

▶ 代码清单 A.42.5 (yaml, 2 行)

```
pagination:
 enabled: false
```

▶ 代码清单 A.42.6 (yaml, 2 行)

```
pagination:
 enabled: true
```

▶ 代码清单 A.42.7 (yaml, 3 行)

```
slugify:
 mode: "default"
 case: false
```

▶ 代码清单 A.42.8 (text, 1 行)

```
_layouts/page.html
```

▶ 代码清单 A.42.9 (text, 1 行)

```
_layouts/home.html
```

▶ 代码清单 A.42.10 (yaml, 1 行)

```
layout: home
```

▶ 代码清单 A.42.11 (text, 1 行)

```
_layouts/autopage_category.html
```

---

▶ 代码清单 A.42.12 (text, 1 行)

life

▶ 代码清单 A.42.13 (text, 1 行)

Life

▶ 代码清单 A.42.14 (text, 1 行)

.github/workflows/pages.yml

▶ 代码清单 A.42.15 (text, 1 行)

.github/workflows/pages.yml

▶ 代码清单 A.42.16 (text, 1 行)

GitHub Actions

▶ 代码清单 A.42.17 (text, 1 行)

Gemfile

▶ 代码清单 A.42.19 (text, 6 行)

life.md  
reading.md  
sci-log.md  
sci-note.md  
learning.md  
writing.md

---

► 代码清单 A.42.20 (yaml, 1 行)

```
branches: ["master", "main"]
```

► 代码清单 A.42.21 (yaml, 1 行)

```
layout: home
```

► 代码清单 A.42.22 (yaml, 2 行)

```
layout: page
permalink: /about/
```

► 代码清单 A.42.23 (yaml, 6 行)

```
categories: writing
categories: life
categories: reading
categories: learning
categories: sci-log
categories: sci-note
```

► 代码清单 A.42.24 (yaml, 1 行)

```
permalink: /writing/20260327strange-things/
```

## (原创自研) 网站维护记录 3

► 代码清单 A.43.2 (text, 1 行)

'bundle' 不是内部或外部命令，也不是可运行的程序或批处理文件。



---

▶ 代码清单 A.43.3 (text, 1 行)

```
C:\> ruby enabled command line
```

▶ 代码清单 A.43.4 (text, 1 行)

```
Install MSYS2-Devkit necessary for many gems
```

▶ 代码清单 A.43.10 (text, 1 行)

```
Gemfile
```

▶ 代码清单 A.43.13 (text, 2 行)

```
Server address: http://127.0.0.1:4000/
Server running... press ctrl-c to stop.
```

▶ 代码清单 A.43.14 (text, 1 行)

```
http://127.0.0.1:4000/
```

▶ 代码清单 A.43.15 (text, 1 行)

```
http://localhost:4000/
```

▶ 代码清单 A.43.16 (text, 1 行)

```
Ctrl + C
```

---

▶ 代码清单 A.43.27 (text, 1 行)

```
http://127.0.0.1:4001/
```

▶ 代码清单 A.43.30 (text, 1 行)

```
C:\> ruby enabled command line
```

▶ 代码清单 A.43.31 (text, 1 行)

```
Install MSYS2-Devkit necessary for many gems
```

▶ 代码清单 A.43.32 (text, 2 行)

```
Server address: http://127.0.0.1:4000/
Server running... press ctrl-c to stop.
```

▶ 代码清单 A.43.33 (text, 1 行)

```
C:\> ruby enabled command line
```

▶ 代码清单 A.43.40 (text, 1 行)

```
http://127.0.0.1:4000/
```

▶ 代码清单 A.43.42 (text, 1 行)

```
http://127.0.0.1:4000/
```

---

## (原创自研) 网站维护记录 4-更新结束

### ► 代码清单 A.44.1 (html, 1 行)

```
<meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
```

### ► 代码清单 A.44.2 (scss, 1 行)

```
#posts-container { ... }
```

### ► 代码清单 A.44.3 (html, 3 行)

```
<div id="posts-container">
 ...
</div>
```

### ► 代码清单 A.44.4 (html, 13 行)

```
<div id="pagination">
 {% if paginator.previous_page %}
 上一页
 {% else %}

 {% endif %}

 {% if paginator.next_page %}
 下一页
 {% else %}

 {% endif %}
</div>
```

### ► 代码清单 A.44.5 (html, 1 行)

```
<div id="pagination" class="clearfix">
```

### ► 代码清单 A.44.6 (scss, 5 行)

```
#pagination::before,
```

---

```
#pagination::after {
 content: none !important;
 display: none !important;
}
```

#### ► 代码清单 A.44.7 (scss, 3 行)

```
flex-direction: column;
width: 100%;
max-width: 220px;
```

#### ► 代码清单 A.44.8 (liquid, 4 行)

```
{% for post in paginator.posts %}
 ...
 <hr>
{% endfor %}
```

#### ► 代码清单 A.44.9 (liquid, 6 行)

```
{% for post in paginator.posts %}
 ...
 {% unless forloop.last %}
 <hr>
 {% endunless %}
{% endfor %}
```

### (原创自研) 网站维护记录 5-搜索功能与界面更新

#### ► 代码清单 A.45.1 (liquid, 1 行)

```
"content": {{ post.content | strip_html | strip_newlines | jsonify }}
```

#### ► 代码清单 A.45.2 (liquid, 1 行)

```
"content": {{ post.content | strip_html | strip_newlines | truncate: 1200 | jsonify }}
```

---

► 代码清单 A.45.4 (text, 1 行)

```
.copy-btn
```

► 代码清单 A.45.5 (text, 1 行)

```
assets/js/copy-code.js
```

► 代码清单 A.45.6 (html, 1 行)

```
<script src="{ ' /assets/js/copy-code.js' | relative_url }"></script>
```

## (建模流程)Materials Studio 2023 手动搭建 Cs<sub>3</sub>V<sub>9</sub>Te<sub>13</sub> 晶体结构

► 代码清单 A.46.1 (text, 1 行)

先设置晶胞参数和空间群 → 输入非等价原子位点 → 让 Materials Studio 根据空间群自动展开完整晶胞

► 代码清单 A.46.2 (text, 11 行)

```
Material: Cs3V9Te13
Crystal system: Hexagonal
Space group: P -6 2 m
Space group number: 189
```

```
a = 10.1327 Å
b = 10.1327 Å
c = 8.2181 Å
alpha = 90°
beta = 90°
gamma = 120°
```

► 代码清单 A.46.3 (text, 2 行)

必须选择 No. 189, P -6 2 m  
不要选成 P -6 m 2

---

▶ 代码清单 A.46.4 (text, 1 行)

189

▶ 代码清单 A.46.5 (text, 4 行)

```
Cs: 3
V : 9
Te: 13
Total: 25 atoms
```

▶ 代码清单 A.46.6 (text, 1 行)

Cs3V9Te13

▶ 代码清单 A.46.7 (text, 1 行)

File → New

▶ 代码清单 A.46.8 (text, 1 行)

3D Atomistic Document

▶ 代码清单 A.46.9 (text, 1 行)

Build → Crystals → Build Crystal...

▶ 代码清单 A.46.10 (text, 1 行)

Build → Build Crystal...

---

► 代码清单 A.46.11 (text, 1 行)

Build → Crystals → Crystal Builder...

► 代码清单 A.46.12 (text, 1 行)

Symmetry / Space Group

► 代码清单 A.46.13 (text, 1 行)

Space group number = 189

► 代码清单 A.46.14 (text, 1 行)

P -6 2 m

► 代码清单 A.46.15 (text, 1 行)

P-62m

► 代码清单 A.46.16 (text, 2 行)

正确: No. 189, P -6 2 m

错误: P -6 m 2

► 代码清单 A.46.17 (text, 7 行)

a = 10.1327

b = 10.1327

c = 8.2181

alpha = 90

beta = 90

gamma = 120

---

► 代码清单 A.46.19 (text, 1 行)

Add Atoms

► 代码清单 A.46.20 (text, 1 行)

Atoms → Add

► 代码清单 A.46.21 (text, 3 行)

a:  
b:  
c:

► 代码清单 A.46.22 (text, 1 行)

x, y, z

► 代码清单 A.46.24 (text, 1 行)

Add

► 代码清单 A.46.25 (text, 8 行)

Element: Cs  
Name: Cs1  
Oxidation state: 0  
Occupancy: 1.0

a = 0.0000  
b = 0.3632  
c = 0.0000

► 代码清单 A.46.26 (text, 1 行)

Add



---

► 代码清单 A.46.27 (text, 8 行)

```
Element: V
Name: V1
Oxidation state: 0
Occupancy: 1.0
```

```
a = 0.8220
b = 0.5334
c = 0.5000
```

► 代码清单 A.46.28 (text, 1 行)

```
Add
```

► 代码清单 A.46.29 (text, 2 行)

```
Element 只能选 V
Name 可以写 V1
```

► 代码清单 A.46.30 (text, 8 行)

```
Element: V
Name: V2
Oxidation state: 0
Occupancy: 1.0
```

```
a = 0.8469
b = 0.8469
c = 0.5000
```

► 代码清单 A.46.31 (text, 1 行)

```
Add
```

► 代码清单 A.46.32 (text, 8 行)

```
Element: Te
Name: Te1
```

---

Oxidation state: 0  
Occupancy: 1.0

a = 0.0000  
b = 0.77119  
c = 0.71975

► 代码清单 A.46.33 (text, 1 行)

Add

► 代码清单 A.46.34 (text, 8 行)

Element: Te  
Name: Te2  
Oxidation state: 0  
Occupancy: 1.0

a = 0.666667  
b = 0.333333  
c = 0.25213

► 代码清单 A.46.35 (text, 1 行)

Add

► 代码清单 A.46.36 (text, 8 行)

Element: Te  
Name: Te3  
Oxidation state: 0  
Occupancy: 1.0

a = 0.58986  
b = 0.58986  
c = 0.5000

► 代码清单 A.46.37 (text, 1 行)

Add

---

► 代码清单 A.46.38 (text, 1 行)

Build

► 代码清单 A.46.39 (text, 1 行)

OK

► 代码清单 A.46.40 (text, 1 行)

Apply

► 代码清单 A.46.41 (text, 2 行)

Apply symmetry  
Generate symmetry equivalents

► 代码清单 A.46.42 (text, 1 行)

Cs3 V9 Te13

► 代码清单 A.46.43 (text, 1 行)

25 atoms

► 代码清单 A.46.45 (text, 1 行)

View → Along Axis → c

---

► 代码清单 A.46.46 (text, 1 行)

Cs — Te — V/Te — Te — Cs

► 代码清单 A.46.47 (text, 1 行)

No. 189, P -6 2 m

► 代码清单 A.46.48 (text, 2 行)

P -6 m 2  
其他 P-6xx 空间群

► 代码清单 A.46.49 (text, 1 行)

不要按符号猜，直接输入空间群编号 189。

► 代码清单 A.46.50 (text, 1 行)

a, b, c

► 代码清单 A.46.52 (text, 1 行)

(0.8220, 0.5334, 0.5000)

► 代码清单 A.46.53 (text, 1 行)

Fractional coordinates

---

► 代码清单 A.46.54 (text, 1 行)

Cartesian coordinates

► 代码清单 A.46.55 (text, 2 行)

Element: V  
Name: V1

► 代码清单 A.46.56 (text, 1 行)

Element: V1

► 代码清单 A.46.57 (text, 2 行)

Element: Te  
Name: Te1 / Te2 / Te3

► 代码清单 A.46.58 (text, 1 行)

输入 6 个非等价位点 → Apply symmetry / Build → 得到 25 原子完整晶胞

► 代码清单 A.46.61 (text, 1 行)

File → Save As

► 代码清单 A.46.62 (text, 1 行)

.xsd

---

► 代码清单 A.46.63 (text, 1 行)

File → Export...

► 代码清单 A.46.66 (text, 3 行)

```
Cs V Te
3 9 13
Direct
```

► 代码清单 A.46.67 (text, 1 行)

Cs Te V

► 代码清单 A.46.68 (text, 2 行)

V1: 展开后 6 个 V 原子  
V2: 展开后 3 个 V 原子

► 代码清单 A.46.69 (text, 2 行)

V1 初始非等价位点: (0.8220, 0.5334, 0.5000)  
V2 初始非等价位点: (0.8469, 0.8469, 0.5000)

► 代码清单 A.46.70 (text, 27 行)

1. 打开 Materials Studio 2023 Visualizer
2. File → New → 3D Atomistic Document
3. Build → Crystals → Build Crystal...
4. Space group 输入 189
5. 确认显示 P -6 2 m
6. 输入晶格参数:  
a = 10.1327  
b = 10.1327  
c = 8.2181  
alpha = 90  
beta = 90  
gamma = 120
7. 点击 Add Atoms

8. 依次输入 6 个非等价位点:  
 Cs 0.0000 0.3632 0.0000  
 V 0.8220 0.5334 0.5000  
 V 0.8469 0.8469 0.5000  
 Te 0.0000 0.77119 0.71975  
 Te 0.666667 0.333333 0.25213  
 Te 0.58986 0.58986 0.5000
9. 每输入一行点击一次 Add
10. 回到 Build Crystal 主窗口
11. 点击 Apply symmetry / Build / OK
12. 检查总原子数是否为 25
13. 检查化学式是否为 Cs3V9Te13
14. 检查 9 个 V 是否都位于  $z = 0.5$
15. File → Export → VASP POSCAR

## ► 代码清单 A.46.71 (text, 7 行)

1. 空间群: P -6 2 m, No. 189
2. 晶格:  $a = b = 10.1327 \text{ \AA}$ ,  $c = 8.2181 \text{ \AA}$ ,  $\gamma = 120^\circ$
3. 组成: Cs3V9Te13
4. 总原子数: 25
5. V 原子数: 9
6. 9 个 V 原子都在  $z = 0.5$  层
7. V 子晶格沿  $c$  轴俯视为复杂二维网络, 而不是普通 kagome

## FES 体系自旋陈数、金属性阻碍与相界代码

## ► 代码清单 A.47.1 (python, 1175 行)

```
#!/usr/bin/env python3
-*- coding: utf-8 -*-
"""
Step 01 — fes net 六参数八带交错磁模型: 最小可验证闭环
```

任务目标

1. 按用户 MagneticTB 导出的 fes Hamiltonian 实现 8x8 模型;
2. 检查 Hermiticity、自旋块解耦、周期规范及对称性简并;
3. 复现文献 Fig. 4(c) 的高对称路径能带;
4. 在完整二维 Brillouin 区审计直接带隙与间接带隙;
5. 在半填充下计算 spin-up、spin-down 和总占据子空间的 Chern 数;
6. 建立可靠的一级相标签, 为后续 Sobol 采样和机器学习提供统一接口;
7. 修正闭隙样本标签逻辑, 并检查半填充是否始终为 2-up + 2-down;
8. 可选地生成少量调试样本, 或与 wannier90\_hr.dat 做能谱对照。

模型信息

```

磁空间群: P4'/mm'm, BNS #123.342
网络: fes net, Wyckoff 4o
原始参数: (e1, e2, t1, t2, r1, r2)
去除整体能量平移后的独立参数: (m_e, t1, t2, r1, r2)
矩阵维数: 8
半填充: 总占据 4 带; 每个守恒自旋块占据 2 带
```

重要

```

本脚本只完成模型与数值标签器审计，不预设 fes 一定存在 type-II QSH 相。
非零且相反的 spin Chern 只标记为 spin_chern_TI_candidate; 是否属于
confirmed type-II QSH, 需要后续带反转、动量分离边缘态和输运验证。
"""

from __future__ import annotations

from pathlib import Path
from typing import Callable, Dict, Iterable, Sequence
from datetime import datetime, timezone
import hashlib
import json
import os

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.set_printoptions(precision=10, suppress=True)

=====
0. 用户配置
=====

CODE_VERSION = "FES_STEP01_V2_20260712"
RESULT_SCHEMA_VERSION = "fes_step01_schema_v2_59cols"

默认写入全新的版本化目录，避免旧 CSV 混入。
也可通过环境变量覆盖，例如：
FES_STEP01_OUTPUT_DIR=my_results python FES_step01_v2.py
OUTPUT_DIR = Path(os.getenv("FES_STEP01_OUTPUT_DIR", "outputs_fes6_step01_v2"))
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

默认执行 24 个调试样本，以便直接生成修正后的 59 列结果。
临时只做文献参数审计时：
FES_RUN_SMALL_BATCH=0 python FES_step01_v2.py
RUN_SMALL_BATCH = os.getenv("FES_RUN_SMALL_BATCH", "1").strip().lower() not in {
 "0", "false", "no", "off"
}
N_BATCH = 24
BATCH_RANDOM_SEED = 20260712

8 带半填充
N_OCC_TOTAL = 4
N_OCC_SPIN = 2

调试网格。正式大数据标签阶段应增加网格、shift 共识和闭隙局部精修。
GAP_NK_DEBUG = 61
CHERN_NK_DEBUG = 25

GAP_TOL = 1.0e-3
CHERN_TOL = 0.08
MIN_DET_TOL = 1.0e-7

RAW6 = ["e1", "e2", "t1", "t2", "r1", "r2"]
REDUCED5 = ["m_e", "t1", "t2", "r1", "r2"]

用户当前 MagneticTB 输入采用 x=0.207，因此默认严格匹配导出的 207/500 与 293/1000 相位。
若以后使用 Mathematica 精确值重新导出，可改为：
X_FES = 0.5 * (np.sqrt(2.0) - 1.0)
X_FES = 0.207
ALPHA = 2.0 * X_FES # 0.414
BETA = 0.5 - X_FES # 0.293

BASIS_LABELS = [
 "A_up", "A_down",
 "B_up", "B_down",
 "C_up", "C_down",

```



```

 "D_up", "D_down",
]

与用户 fes_ham.txt 的相位约定一致:
A=(x,1/2), B=(-x,1/2), C=(1/2,-x), D=(1/2,x)
ORBITAL_POSITIONS = np.array(
 [
 [X_FES, 0.5], [X_FES, 0.5],
 [-X_FES, 0.5], [-X_FES, 0.5],
 [0.5, -X_FES], [0.5, -X_FES],
 [0.5, X_FES], [0.5, X_FES],
],
 dtype=float,
)

SPIN_UP_INDICES = [0, 2, 4, 6]
SPIN_DOWN_INDICES = [1, 3, 5, 7]

文献主文 Fig. 4(c) 参数
PAPER_PARAMS: Dict[str, float] = {
 "e1": 0.7,
 "e2": -0.4,
 "t1": 0.3,
 "t2": 0.2,
 "r1": 0.3,
 "r2": -0.1,
}

仅用于 Step 01 的小批量调试, 不代表最终科学采样范围。
采用去除 e0 后的五维空间, 避免把无物理意义的整体能量平移送入 ML。
DEBUG_REDUCED_BOUNDS = {
 "m_e": (-1.0, 1.0),
 "t1": (-0.8, 0.8),
 "t2": (-0.8, 0.8),
 "r1": (-0.8, 0.8),
 "r2": (-0.8, 0.8),
}

=====
1. 参数处理与 fes 解析 Hamiltonian
=====

def validate_params(params: Dict[str, float]) -> Dict[str, float]:
 """ 验证并标准化六参数字典。"""
 missing = [name for name in RAW6 if name not in params]
 extra = [name for name in params if name not in RAW6]
 if missing:
 raise ValueError(f" 缺少参数: {missing}")
 if extra:
 raise ValueError(f" 出现未定义参数: {extra}")

 clean = {name: float(params[name]) for name in RAW6}
 if not all(np.isfinite(value) for value in clean.values()):
 raise ValueError(" 参数中存在 NaN 或无穷大")
 return clean

def raw6_from_reduced5(reduced: Dict[str, float], e0: float = 0.0) -> Dict[str, float]:
 """ 由五维独立参数恢复六参数; e0 只平移全部能带。"""
 missing = [name for name in REDUCED5 if name not in reduced]
 if missing:
 raise ValueError(f" 缺少五维参数: {missing}")
 m_e = float(reduced["m_e"])
 return {
 "e1": float(e0 + m_e),
 "e2": float(e0 - m_e),
 "t1": float(reduced["t1"]),
 "t2": float(reduced["t2"]),
 }

```

```

 "r1": float(reduced["r1"]),
 "r2": float(reduced["r2"]),
 }

def derived_features(params: Dict[str, float]) -> Dict[str, float]:
 """ 输出供后续 ML 使用的低维候选特征; 此处不预设其为拓扑判据。 """
 p = validate_params(params)
 root2 = np.sqrt(2.0)
 return {
 "e0": 0.5 * (p["e1"] + p["e2"]),
 "m_e": 0.5 * (p["e1"] - p["e2"]),
 "t_sum": p["t1"] + p["t2"],
 "t_diff": p["t1"] - p["t2"],
 "t_s": (p["t1"] + p["t2"]) / root2,
 "t_d": (p["t1"] - p["t2"]) / root2,
 "r_sum": p["r1"] + p["r2"],
 "r_diff": p["r1"] - p["r2"],
 "r_s": (p["r1"] + p["r2"]) / root2,
 "r_d": (p["r1"] - p["r2"]) / root2,
 "abs_r_mismatch": abs(abs(p["r1"]) - abs(p["r2"])),
 "t_product": p["t1"] * p["t2"],
 "r_product": p["r1"] * p["r2"],
 }

def h_fes_atomic(kx: float, ky: float, params: Dict[str, float]) -> np.ndarray:
 """
 用户 MagneticTB 导出 fes_ham.txt 对应的 8x8 原子规范 Hamiltonian。

 基底: A_up, A_down, B_up, B_down, C_up, C_down, D_up, D_down。
 """
 p = validate_params(params)
 e1, e2, t1, t2, r1, r2 = [p[name] for name in RAW6]

 # E11
 e11 = np.zeros((4, 4), dtype=np.complex128)
 np.fill_diagonal(e11, [e1, e2, e1, e2])
 e11[0, 2] = t1 * np.exp(-1j * ALPHA * kx)
 e11[2, 0] = np.conj(e11[0, 2])
 e11[1, 3] = t2 * np.exp(-1j * ALPHA * kx)
 e11[3, 1] = np.conj(e11[1, 3])

 # E22
 e22 = np.zeros((4, 4), dtype=np.complex128)
 np.fill_diagonal(e22, [e2, e1, e2, e1])
 e22[0, 2] = t2 * np.exp(+1j * ALPHA * ky)
 e22[2, 0] = np.conj(e22[0, 2])
 e22[1, 3] = t1 * np.exp(+1j * ALPHA * ky)
 e22[3, 1] = np.conj(e22[1, 3])

 # R12
 r_plus = 1j * r1 + r2
 r_minus = -1j * r1 + r2

 phase_pp = np.exp(+1j * BETA * (kx + ky))
 phase_pm = np.exp(+1j * BETA * (kx - ky))
 phase_mp = np.exp(-1j * BETA * (kx + ky))
 phase_mm = np.exp(-1j * BETA * (kx - ky))

 r12 = np.array(
 [
 [phase_pp * r_plus, 0.0, phase_pm * r_minus, 0.0],
 [0.0, phase_pp * r_minus, 0.0, phase_pm * r_plus],
 [phase_mm * r_minus, 0.0, phase_mp * r_plus, 0.0],
 [0.0, phase_mm * r_plus, 0.0, phase_mp * r_minus],
],
 dtype=np.complex128,
)

```

---

```

h = np.block([[e11, r12], [r12.conj().T, e22]])
return h.astype(np.complex128)

def eigvals_full(kx: float, ky: float, params: Dict[str, float]) -> np.ndarray:
 return np.linalg.eigvalsh(h_fes_atomic(kx, ky, params))

def spin_indices(spin: str) -> list[int]:
 spin_key = spin.lower()
 if spin_key == "up":
 return SPIN_UP_INDICES.copy()
 if spin_key == "down":
 return SPIN_DOWN_INDICES.copy()
 raise ValueError("spin 必须为 'up' 或 'down'")

def h_spin_block_atomic(
 kx: float,
 ky: float,
 params: Dict[str, float],
 spin: str,
) -> np.ndarray:
 idx = spin_indices(spin)
 h = h_fes_atomic(kx, ky, params)
 return h[np.ix_(idx, idx)]

=====
2. 周期规范
=====

def periodic_gauge_matrix(kx: float, ky: float) -> np.ndarray:
 phases = np.exp(
 1j * (
 ORBITAL_POSITIONS[:, 0] * float(kx)
 + ORBITAL_POSITIONS[:, 1] * float(ky)
)
)
 return np.diag(phases).astype(np.complex128)

def h_fes_periodic(kx: float, ky: float, params: Dict[str, float]) -> np.ndarray:
 # 对当前 fes_ham.txt 相位约定, $H_{\text{per}} = U H_{\text{atomic}} U^\dagger$.
 u = periodic_gauge_matrix(kx, ky)
 h = h_fes_atomic(kx, ky, params)
 return u @ h @ u.conj().T

def h_spin_block_periodic(
 kx: float,
 ky: float,
 params: Dict[str, float],
 spin: str,
) -> np.ndarray:
 idx = spin_indices(spin)
 h = h_fes_periodic(kx, ky, params)
 return h[np.ix_(idx, idx)]

=====
3. 模型硬审计与对称性测试
=====

def hermiticity_error(
 params: Dict[str, float],
 n_test: int = 40,
 seed: int = 1234,

```

---

```

) -> float:
 rng = np.random.default_rng(seed)
 max_error = 0.0
 for _ in range(n_test):
 kx, ky = rng.uniform(-np.pi, np.pi, size=2)
 h = h_fes_atomic(float(kx), float(ky), params)
 max_error = max(max_error, float(np.max(np.abs(h - h.conj()).T)))
 return max_error

def spin_mixing_error(
 params: Dict[str, float],
 n_test: int = 40,
 seed: int = 2026,
) -> float:
 rng = np.random.default_rng(seed)
 max_error = 0.0
 for _ in range(n_test):
 kx, ky = rng.uniform(-np.pi, np.pi, size=2)
 h = h_fes_atomic(float(kx), float(ky), params)
 mix_ud = h[np.ix_(SPIN_UP_INDICES, SPIN_DOWN_INDICES)]
 mix_du = h[np.ix_(SPIN_DOWN_INDICES, SPIN_UP_INDICES)]
 max_error = max(
 max_error,
 float(np.max(np.abs(mix_ud))),
 float(np.max(np.abs(mix_du))),
)
 return max_error

def periodicity_errors(
 params: Dict[str, float],
 n_test: int = 40,
 seed: int = 99,
) -> Dict[str, float]:
 rng = np.random.default_rng(seed)
 max_x = 0.0
 max_y = 0.0
 for _ in range(n_test):
 kx, ky = rng.uniform(-np.pi, np.pi, size=2)
 h0 = h_fes_periodic(float(kx), float(ky), params)
 hx = h_fes_periodic(float(kx + 2.0 * np.pi), float(ky), params)
 hy = h_fes_periodic(float(kx), float(ky + 2.0 * np.pi), params)
 max_x = max(max_x, float(np.max(np.abs(hx - h0))))
 max_y = max(max_y, float(np.max(np.abs(hy - h0))))
 return {"periodicity_error_x": max_x, "periodicity_error_y": max_y}

def gamma_pair_degeneracy_error(params: Dict[str, float]) -> float:
 """Γ 点最多四个能级且均为二重简并。"""
 e = np.sort(eigvals_full(0.0, 0.0, params))
 pair_errors = [abs(e[2 * i + 1] - e[2 * i]) for i in range(4)]
 return float(max(pair_errors))

def spin_spectrum_difference(
 kx: float,
 ky: float,
 params: Dict[str, float],
) -> float:
 up = np.linalg.eigvalsh(h_spin_block_atomic(kx, ky, params, "up"))
 down = np.linalg.eigvalsh(h_spin_block_atomic(kx, ky, params, "down"))
 return float(np.max(np.abs(np.sort(up) - np.sort(down))))

def sigma_degeneracy_errors(
 params: Dict[str, float],
 n_points: int = 41,
) -> Dict[str, float]:

```

---

```

""" 检查 $\Sigma: k_x=k_y$ 与 $\Sigma': -k_x=k_y$ 路径上的自旋简并。 """
qs = np.linspace(-np.pi, np.pi, n_points)
sigma_error = 0.0
sigma_prime_error = 0.0
for q in qs:
 sigma_error = max(
 sigma_error,
 spin_spectrum_difference(float(q), float(q), params),
)
 sigma_prime_error = max(
 sigma_prime_error,
 spin_spectrum_difference(float(-q), float(q), params),
)
return {
 "sigma_kx_eq_ky_error": sigma_error,
 "sigma_prime_minus_kx_eq_ky_error": sigma_prime_error,
}

def run_model_tests(params: Dict[str, float]) -> Dict[str, float]:
 results = {
 "hermiticity_error": hermiticity_error(params),
 "spin_mixing_error": spin_mixing_error(params),
 "gamma_pair_degeneracy_error": gamma_pair_degeneracy_error(params),
 }
 results.update(periodicity_errors(params))
 results.update(sigma_degeneracy_errors(params))

 hard_tolerance = 1.0e-10
 failed = {key: value for key, value in results.items() if value > hard_tolerance}
 if failed:
 raise AssertionError(f"fes 模型硬审计失败: {failed}")
 return results

=====
4. 高对称路径能带
=====

HIGH_SYM_POINTS = [
 ("Γ", np.array([0.0, 0.0])),
 ("X", np.array([np.pi, 0.0])),
 ("M", np.array([np.pi, np.pi])),
 ("Γ", np.array([0.0, 0.0])),
 ("M", np.array([-np.pi, np.pi])),
 ("X", np.array([0.0, np.pi])),
 ("Γ", np.array([0.0, 0.0])),
]

def make_k_path(
 points: Sequence[tuple[str, np.ndarray]] = HIGH_SYM_POINTS,
 n_per_segment: int = 100,
):
 k_list: list[np.ndarray] = []
 x_list: list[float] = []
 tick_positions = [0.0]
 tick_labels = [points[0][0]]
 distance = 0.0

 for segment in range(len(points) - 1):
 _, ka = points[segment]
 label_b, kb = points[segment + 1]
 ts = np.linspace(0.0, 1.0, n_per_segment, endpoint=False)
 for t in ts:
 k = (1.0 - t) * ka + t * kb
 if k_list:
 distance += float(np.linalg.norm(k - k_list[-1]))
 k_list.append(k)

```

---

```

 x_list.append(distance)

 distance += float(np.linalg.norm(kb - k_list[-1]))
 k_list.append(kb.copy())
 x_list.append(distance)
 tick_positions.append(distance)
 tick_labels.append(label_b)

 return (
 np.asarray(k_list),
 np.asarray(x_list),
 tick_positions,
 tick_labels,
)

def calculate_path_bands(
 params: Dict[str, float],
 n_per_segment: int = 100,
):
 k_list, x_axis, ticks, labels = make_k_path(n_per_segment=n_per_segment)
 bands = np.array(
 [eigvals_full(float(k[0]), float(k[1]), params) for k in k_list]
)
 return k_list, x_axis, bands, ticks, labels

def plot_path_bands(
 params: Dict[str, float],
 title: str,
 n_per_segment: int = 100,
 save_name: str | None = None,
) -> None:
 _, x_axis, bands, ticks, labels = calculate_path_bands(
 params,
 n_per_segment=n_per_segment,
)
 fig, ax = plt.subplots(figsize=(8.4, 5.4))
 for band_index in range(bands.shape[1]):
 ax.plot(x_axis, bands[:, band_index], linewidth=1.25)
 for position in ticks:
 ax.axvline(position, linewidth=0.6, alpha=0.45)
 ax.axhline(0.0, linewidth=0.7, alpha=0.5)
 ax.set_xticks(ticks)
 ax.set_xticklabels(labels)
 ax.set_ylabel("Energy")
 ax.set_title(title)
 ax.set_xlim(x_axis[0], x_axis[-1])
 fig.tight_layout()
 if save_name is not None:
 fig.savefig(OUTPUT_DIR / save_name, dpi=240, bbox_inches="tight")
 plt.close(fig)

=====
5. 完整 BZ 带隙审计
=====

def scan_band_gaps(
 params: Dict[str, float],
 nk: int = GAP_NK_DEBUG,
 n_occ_total: int = N_OCC_TOTAL,
) -> Dict[str, float | int]:
 """ 在 $[-\pi, \pi)^2$ 网格上审计半填充直接带隙与全局间接带隙。 """
 ks = np.linspace(-np.pi, np.pi, nk, endpoint=False)

 min_direct = np.inf
 direct_kx = np.nan
 direct_ky = np.nan

```

---

```

max_valence = -np.inf
vbm_kx = np.nan
vbm_ky = np.nan
min_conduction = np.inf
cbm_kx = np.nan
cbm_ky = np.nan
min_spin_gap_up = np.inf
min_spin_gap_down = np.inf

对 fes 八带模型, 半填充的目标 spin-Chern 子空间必须在每个 k 点
都对应 2 条 spin-up 占据带 + 2 条 spin-down 占据带。
balanced_sector_gap > 0 等价于: 两自旋块的第 2 条带全部低于第 3 条带。
min_balanced_sector_gap = np.inf
balanced_gap_kx = np.nan
balanced_gap_ky = np.nan
spin_occupancy_mismatch_count = 0
min_n_up_in_lowest4 = N_OCC_TOTAL
max_n_up_in_lowest4 = 0

for kx in ks:
 for ky in ks:
 kx_f = float(kx)
 ky_f = float(ky)

 energies = eigvals_full(kx_f, ky_f, params)
 direct_gap = float(energies[n_occ_total] - energies[n_occ_total - 1])
 if direct_gap < min_direct:
 min_direct = direct_gap
 direct_kx = kx_f
 direct_ky = ky_f

 valence = float(energies[n_occ_total - 1])
 conduction = float(energies[n_occ_total])
 if valence > max_valence:
 max_valence = valence
 vbm_kx = kx_f
 vbm_ky = ky_f
 if conduction < min_conduction:
 min_conduction = conduction
 cbm_kx = kx_f
 cbm_ky = ky_f

 up_e = np.linalg.eigvalsh(
 h_spin_block_atomic(kx_f, ky_f, params, "up")
)
 down_e = np.linalg.eigvalsh(
 h_spin_block_atomic(kx_f, ky_f, params, "down")
)

 spin_gap_up = float(up_e[N_OCC_SPIN] - up_e[N_OCC_SPIN - 1])
 spin_gap_down = float(down_e[N_OCC_SPIN] - down_e[N_OCC_SPIN - 1])
 min_spin_gap_up = min(min_spin_gap_up, spin_gap_up)
 min_spin_gap_down = min(min_spin_gap_down, spin_gap_down)

 balanced_gap = float(
 min(up_e[N_OCC_SPIN], down_e[N_OCC_SPIN])
 - max(up_e[N_OCC_SPIN - 1], down_e[N_OCC_SPIN - 1])
)
 if balanced_gap < min_balanced_sector_gap:
 min_balanced_sector_gap = balanced_gap
 balanced_gap_kx = kx_f
 balanced_gap_ky = ky_f

 tagged = [(float(value), 1) for value in up_e]
 tagged += [(float(value), 0) for value in down_e]
 tagged.sort(key=lambda item: item[0])
 n_up_occ = int(sum(tag for _, tag in tagged[:n_occ_total]))
 min_n_up_in_lowest4 = min(min_n_up_in_lowest4, n_up_occ)
 max_n_up_in_lowest4 = max(max_n_up_in_lowest4, n_up_occ)

```

---

```

 if n_up_occ != N_OCC_SPIN:
 spin_occupancy_mismatch_count += 1

indirect_gap = float(min_conduction - max_valence)
balanced_sector_valid = (
 min_balanced_sector_gap > GAP_TOL
 and spin_occupancy_mismatch_count == 0
)

return {
 "grid_nk": int(nk),
 "min_direct_gap": float(min_direct),
 "indirect_gap": indirect_gap,
 "direct_gap_kx": direct_kx,
 "direct_gap_ky": direct_ky,
 "vbm": float(max_valence),
 "vbm_kx": vbm_kx,
 "vbm_ky": vbm_ky,
 "cbm": float(min_conduction),
 "cbm_kx": cbm_kx,
 "cbm_ky": cbm_ky,
 "min_spin_gap_up": float(min_spin_gap_up),
 "min_spin_gap_down": float(min_spin_gap_down),
 "min_balanced_sector_gap": float(min_balanced_sector_gap),
 "balanced_gap_kx": balanced_gap_kx,
 "balanced_gap_ky": balanced_gap_ky,
 "spin_occupancy_mismatch_count": int(spin_occupancy_mismatch_count),
 "min_n_up_in_lowest4": int(min_n_up_in_lowest4),
 "max_n_up_in_lowest4": int(max_n_up_in_lowest4),
 "is_direct_gapped": int(min_direct > GAP_TOL),
 "is_physical_insulator": int(indirect_gap > GAP_TOL),
 "is_balanced_spin_sector": int(balanced_sector_valid),
}

=====
6. 非阿贝尔 Fukui Chern
=====

def _normalize_link(value: complex, eps: float = 1.0e-14) -> complex:
 amplitude = abs(value)
 if amplitude < eps:
 raise FloatingPointError(f"Link determinant too small: {amplitude:.3e}")
 return value / amplitude

def fukui_chern_subspace(
 h_func: Callable[[float, float], np.ndarray],
 n_occ: int,
 nk: int = CHERN_NK_DEBUG,
) -> Dict[str, float | int]:
 """ 对多占据带子空间使用 determinant-link 非阿贝尔 Fukui 方法。 """
 sample_h = h_func(0.0, 0.0)
 dimension = sample_h.shape[0]
 if n_occ <= 0 or n_occ >= dimension:
 raise ValueError(f"n_occ={n_occ} 与 Hamiltonian 维数 {dimension} 不兼容")

 ks = 2.0 * np.pi * np.arange(nk) / nk
 occupied = np.empty((nk, nk, dimension, n_occ), dtype=np.complex128)

 for ix, kx in enumerate(ks):
 for iy, ky in enumerate(ks):
 _, eigenvectors = np.linalg.eigh(h_func(float(kx), float(ky)))
 occupied[ix, iy] = eigenvectors[:, :n_occ]

 total_phase = 0.0
 min_det_amplitude = 1.0

 for ix in range(nk):

```



---

```

 for iy in range(nk):
 v = occupied[ix, iy]
 vx = occupied[(ix + 1) % nk, iy]
 vy = occupied[ix, (iy + 1) % nk]
 vxy = occupied[(ix + 1) % nk, (iy + 1) % nk]

 link_x = np.linalg.det(v.conj().T @ vx)
 link_y = np.linalg.det(v.conj().T @ vy)
 link_x_at_y = np.linalg.det(vy.conj().T @ vxy)
 link_y_at_x = np.linalg.det(vx.conj().T @ vxy)

 min_det_amplitude = min(
 min_det_amplitude,
 abs(link_x),
 abs(link_y),
 abs(link_x_at_y),
 abs(link_y_at_x),
)

 ux = _normalize_link(link_x)
 uy = _normalize_link(link_y)
 ux_at_y = _normalize_link(link_x_at_y)
 uy_at_x = _normalize_link(link_y_at_x)

 plaquette = ux * uy_at_x / (ux_at_y * uy)
 total_phase += float(np.angle(plaquette))

 return {
 "chern": float(total_phase / (2.0 * np.pi)),
 "min_det_amp": float(min_det_amplitude),
 "nk": int(nk),
 }
}

def calculate_spin_cherns(
 params: Dict[str, float],
 nk: int = CHERN_NK_DEBUG,
) -> Dict[str, float | int]:
 up = fukui_chern_subspace(
 lambda kx, ky: h_spin_block_periodic(kx, ky, params, "up"),
 n_occ=N_OCC_SPIN,
 nk=nk,
)
 down = fukui_chern_subspace(
 lambda kx, ky: h_spin_block_periodic(kx, ky, params, "down"),
 n_occ=N_OCC_SPIN,
 nk=nk,
)
 total = fukui_chern_subspace(
 lambda kx, ky: h_fes_periodic(kx, ky, params),
 n_occ=N_OCC_TOTAL,
 nk=nk,
)

 c_up = float(up["chern"])
 c_down = float(down["chern"])
 c_total = float(total["chern"])
 return {
 "chern_nk": int(nk),
 "chern_up": c_up,
 "chern_down": c_down,
 "chern_total": c_total,
 "spin_chern": 0.5 * (c_up - c_down),
 "sum_rule_error": abs(c_total - (c_up + c_down)),
 "min_det_up": float(up["min_det_amp"]),
 "min_det_down": float(down["min_det_amp"]),
 "min_det_total": float(total["min_det_amp"]),
 }
}

```

---

```

def chern_convergence_table(
 params: Dict[str, float],
 grids: Iterable[int] = (15, 21, 31, 41),
) -> pd.DataFrame:
 rows: list[Dict[str, object]] = []
 for nk in grids:
 try:
 rows.append(calculate_spin_cherns(params, nk=int(nk)))
 except Exception as exc: # 保留失败信息, 避免静默丢样本
 rows.append({"chern_nk": int(nk), "error": repr(exc)})
 return pd.DataFrame(rows)

=====
7. 单样本标签
=====

def rounded_integer_if_close(value: float, tolerance: float = CHERN_TOL):
 nearest = int(np rint(value))
 return nearest if abs(value - nearest) <= tolerance else None

def _empty_chern_fields() -> Dict[str, object]:
 """ 为未执行或失败的 Chern 计算补齐统一字段。 """
 return {
 "chern_nk": np.nan,
 "chern_up": np.nan,
 "chern_down": np.nan,
 "chern_total": np.nan,
 "spin_chern": np.nan,
 "sum_rule_error": np.nan,
 "min_det_up": np.nan,
 "min_det_down": np.nan,
 "min_det_total": np.nan,
 "chern_up_int": None,
 "chern_down_int": None,
 "chern_total_int": None,
 "chern_reliable": 0,
 }

def calculate_sample(
 sample_id: str,
 params: Dict[str, float],
 gap_nk: int = GAP_NK_DEBUG,
 chern_nk: int = CHERN_NK_DEBUG,
) -> Dict[str, object]:
 """
 对单个参数点执行严格标签。

 逻辑顺序:
 1. 先完成完整 BZ 带隙审计;
 2. 若总直接带隙闭合, 立即标记为 noninsulating_or_gap_closing,
 不再计算没有定义的总占据子空间 Chern;
 3. 若半填充不是全 BZ 的 2-up + 2-down 平衡占据, 单独标记;
 4. 只有满足以上条件时才计算非阿贝尔 spin-Chern;
 5. Chern 数值失败保留已获得的 gap 信息, 并标记 chern_unreliable,
 不再误标为 numeric_error。
 """
 p = validate_params(params)
 row: Dict[str, object] = {
 "sample_id": sample_id,
 **p,
 **derived_features(p),
 *_empty_chern_fields(),
 "is_spin_chern_TI_candidate": 0,
 "is_typeII_QSH_confirmed": 0,
 "phase_label": "numeric_error",
 }

```

```

 "error": "",
}

A. 完整 BZ 带隙审计。这里若失败才属于真正的 numeric_error。

try:
 gap = scan_band_gaps(p, nk=gap_nk)
 row.update(gap)
except Exception as exc:
 row["phase_label"] = "numeric_error"
 row["error"] = f"gap_scan_failed: {repr(exc)}"
 return row

direct_gapped = float(gap["min_direct_gap"]) > GAP_TOL
physical_insulator = float(gap["indirect_gap"]) > GAP_TOL
balanced_spin_sector = bool(int(gap["is_balanced_spin_sector"]))

先判断直接带隙。闭隙时 Chern 不定义, 禁止继续调用 Fukui。
if not direct_gapped:
 row["phase_label"] = "noninsulating_or_gap_closing"
 return row

总占据流形虽有直接带隙, 但最低四带并非处处为 2-up + 2-down。
这种样本不属于当前固定 N_OCC_SPIN=2 的 spin-Chern 分类问题。
if not balanced_spin_sector:
 row["phase_label"] = "spin_sector_filling_mismatch"
 return row

B. 仅对有定义的 2+2 占据子空间计算 Chern。

try:
 chern = calculate_spin_cherns(p, nk=chern_nk)
 row.update(chern)
except Exception as exc:
 row["phase_label"] = "chern_unreliable"
 row["error"] = f"chern_failed: {repr(exc)}"
 return row

c_up_int = rounded_integer_if_close(float(chern["chern_up"]))
c_down_int = rounded_integer_if_close(float(chern["chern_down"]))
c_total_int = rounded_integer_if_close(float(chern["chern_total"]))

reliable = (
 c_up_int is not None
 and c_down_int is not None
 and c_total_int is not None
 and float(chern["min_det_up"]) > MIN_DET_TOL
 and float(chern["min_det_down"]) > MIN_DET_TOL
 and float(chern["min_det_total"]) > MIN_DET_TOL
 and float(chern["sum_rule_error"]) < CHERN_TOL
)

row["chern_up_int"] = c_up_int
row["chern_down_int"] = c_down_int
row["chern_total_int"] = c_total_int
row["chern_reliable"] = int(reliable)

if not reliable:
 row["phase_label"] = "chern_unreliable"
 return row

spin_chern_nonzero = (
 c_up_int is not None
 and c_down_int is not None
 and c_total_int == 0
 and c_up_int == -c_down_int
 and c_up_int != 0

```

```

)

spin_chern_candidate = physical_insulator and spin_chern_nonzero
row["is_spin_chern_TI_candidate"] = int(spin_chern_candidate)

候选不直接等同于 confirmed type-II QSH.
row["is_typeII_QSH_confirmed"] = 0

if not physical_insulator:
 if spin_chern_nonzero:
 label = "spin_chern_band_metal"
 else:
 label = "indirect_overlap"
elif spin_chern_candidate:
 label = "spin_chern_TI_candidate"
elif c_up_int == 0 and c_down_int == 0 and c_total_int == 0:
 label = "trivial_insulator"
elif c_total_int not in (None, 0):
 label = "Chern_insulator"
else:
 label = "other_gapped_phase"

row["phase_label"] = label
return row

=====
8. 小批量调试采样
=====

def sample_reduced_debug_parameters(
 n: int,
 bounds: Dict[str, tuple[float, float]],
 seed: int,
) -> pd.DataFrame:
 rng = np.random.default_rng(seed)
 rows = []
 for index in range(n):
 reduced = {
 name: float(rng.uniform(bounds[name][0], bounds[name][1]))
 for name in REDUCED5
 }
 raw = raw6_from_reduced5(reduced, e0=0.0)
 rows.append({"sample_id": f"debug_{index:06d}", **raw})
 return pd.DataFrame(rows)

def run_small_batch() -> pd.DataFrame:
 parameter_df = sample_reduced_debug_parameters(
 n=N_BATCH,
 bounds=DEBUG_REduced_BOUNDS,
 seed=BATCH_RANDOM_SEED,
)
 parameter_df.to_csv(
 OUTPUT_DIR / "fes6_debug_parameter_batch.csv",
 index=False,
 encoding="utf-8-sig",
)

 results = []
 for index, row in parameter_df.iterrows():
 params = {name: float(row[name]) for name in RAW6}
 results.append(
 calculate_sample(
 sample_id=str(row["sample_id"]),
 params=params,
 gap_nk=41,
 chern_nk=21,
)

```

```

)
 if (index + 1) % 4 == 0 or (index + 1) == len(parameter_df):
 print(f"Finished debug batch: {index + 1}/{len(parameter_df)}")

result_df = pd.DataFrame(results)
result_df.to_csv(
 OUTPUT_DIR / "fes6_debug_batch_results.csv",
 index=False,
 encoding="utf-8-sig",
)
return result_df

=====
9. 可选: wannier90_hr.dat 对照
=====

def read_wannier90_hr(path: str | Path) -> Dict[str, object]:
 path = Path(path)
 lines = path.read_text(encoding="utf-8", errors="ignore").splitlines()
 if len(lines) < 4:
 raise ValueError(f" 文件过短: {path}")

 num_wann = int(lines[1].strip())
 nrpts = int(lines[2].strip())

 degeneracies: list[int] = []
 line_index = 3
 while len(degeneracies) < nrpts:
 degeneracies.extend(int(value) for value in lines[line_index].split())
 line_index += 1
 degeneracies = degeneracies[:nrpts]

 records = []
 for line in lines[line_index:]:
 parts = line.split()
 if len(parts) < 7:
 continue
 rx, ry, rz, m, n = map(int, parts[:5])
 real_part, imag_part = map(float, parts[5:7])
 records.append((rx, ry, rz, m - 1, n - 1, real_part + 1j * imag_part))

 expected = nrpts * num_wann * num_wann
 if len(records) != expected:
 raise ValueError(f"hr 数据行数不匹配: got={len(records)}, expected={expected}")

 hr: Dict[tuple[int, int, int], np.ndarray] = {}
 cursor = 0
 for ir in range(nrpts):
 r_vector = records[cursor][:3]
 matrix = np.zeros((num_wann, num_wann), dtype=np.complex128)
 for _ in range(num_wann * num_wann):
 rx, ry, rz, m, n, value = records[cursor]
 if (rx, ry, rz) != r_vector:
 raise ValueError("hr.dat 中 R 分组不连续或格式异常")
 matrix[m, n] = value
 cursor += 1
 hr[r_vector] = matrix / float(degeneracies[ir])

 return {"path": path, "num_wann": num_wann, "nrpts": nrpts, "hr": hr}

def hk_from_hr(
 hr_data: Dict[str, object],
 kx: float,
 ky: float,
 kz: float = 0.0,
) -> np.ndarray:
 num_wann = int(hr_data["num_wann"])

```

---

```

h = np.zeros((num_wann, num_wann), dtype=np.complex128)
hr = hr_data["hr"]
if not isinstance(hr, dict):
 raise TypeError("hr_data['hr'] 不是字典")

for (rx, ry, rz), matrix in hr.items():
 phase = np.exp(1j * (kx * rx + ky * ry + kz * rz))
 h += matrix * phase
return 0.5 * (h + h.conj()).T

def compare_hr_spectra(
 hr_path: str | Path,
 params: Dict[str, float],
 n_test: int = 20,
 seed: int = 20260712,
) -> pd.DataFrame:
 hr_data = read_wannier90_hr(hr_path)
 if int(hr_data["num_wann"]) != 8:
 raise ValueError(
 f"fes 六参数模型应有 8 个 Wannier 轨道, 当前为 {hr_data['num_wann']}"
)

 rng = np.random.default_rng(seed)
 rows = []
 for test_index in range(n_test):
 kx, ky = rng.uniform(-np.pi, np.pi, size=2)
 analytic = np.linalg.eigvalsh(h_fes_periodic(float(kx), float(ky), params))
 from_hr = np.linalg.eigvalsh(hk_from_hr(hr_data, float(kx), float(ky)))
 rows.append(
 {
 "test": test_index,
 "kx": float(kx),
 "ky": float(ky),
 "max_eigenvalue_error": float(np.max(np.abs(analytic - from_hr))),
 }
)
 return pd.DataFrame(rows)

def write_run_metadata() -> Dict[str, object]:
 """ 写入版本、脚本哈希和关键数值设置, 便于确认结果来源。 """
 source_name = "Jupyter notebook"
 source_sha256 = None

 if "__file__" in globals():
 candidate = Path(__file__).resolve()
 source_name = candidate.name
 if candidate.exists():
 source_sha256 = hashlib.sha256(candidate.read_bytes()).hexdigest()

 metadata: Dict[str, object] = {
 "code_version": CODE_VERSION,
 "result_schema_version": RESULT_SCHEMA_VERSION,
 "run_time_utc": datetime.now(timezone.utc).isoformat(),
 "source_name": source_name,
 "source_sha256": source_sha256,
 "output_directory": str(OUTPUT_DIR.resolve()),
 "run_small_batch": bool(RUN_SMALL_BATCH),
 "n_batch": int(N_BATCH),
 "batch_random_seed": int(BATCH_RANDOM_SEED),
 "gap_nk_debug": int(GAP_NK_DEBUG),
 "chern_nk_debug": int(CHERN_NK_DEBUG),
 "gap_tolerance": float(GAP_TOL),
 "chern_tolerance": float(CHERN_TOL),
 "min_det_tolerance": float(MIN_DET_TOL),
 "expected_debug_result_columns": 59,
 }

```

```

path = OUTPUT_DIR / "fes_step01_run_metadata.json"
path.write_text(
 json.dumps(metadata, ensure_ascii=False, indent=2),
 encoding="utf-8",
)
return metadata

=====
10. 主程序
=====

def main() -> None:
 metadata = write_run_metadata()

 print("=" * 78)
 print("Step 01: fes net six-parameter eight-band model audit")
 print("Version:", CODE_VERSION)
 print("Schema :", RESULT_SCHEMA_VERSION)
 print("Output :", OUTPUT_DIR.resolve())
 print("Batch :", RUN_SMALL_BATCH)
 print("Basis :", BASIS_LABELS)
 print("x :", X_FES)
 print("SHA256 :", metadata.get("source_sha256"))
 print("=" * 78)

 # 1) 模型硬审计
 tests = run_model_tests(PAPER_PARAMS)
 test_df = pd.DataFrame([tests])
 print("\n[Model tests]")
 print(test_df.to_string(index=False))
 test_df.to_csv(
 OUTPUT_DIR / "fes6_model_tests.csv",
 index=False,
 encoding="utf-8-sig",
)

 # 2) 文献参数能带
 plot_path_bands(
 PAPER_PARAMS,
 title="fes eight-band model: literature parameter set",
 save_name="fes_paper_parameter_bands.png",
)
 print("\nBand plot written:", OUTPUT_DIR / "fes_paper_parameter_bands.png")

 # 3) 完整 BZ gap
 paper_gap = scan_band_gaps(PAPER_PARAMS, nk=GAP_NK_DEBUG)
 gap_df = pd.DataFrame([{"case": "paper", **paper_gap}])
 print("\n[Full-BZ gap audit]")
 print(gap_df.to_string(index=False))
 gap_df.to_csv(
 OUTPUT_DIR / "fes_paper_gap_audit.csv",
 index=False,
 encoding="utf-8-sig",
)

 # 4) Chern 网格收敛
 chern_df = chern_convergence_table(PAPER_PARAMS, grids=(15, 21, 31, 41))
 print("\n[Chern convergence: paper parameters]")
 print(chern_df.to_string(index=False))
 chern_df.to_csv(
 OUTPUT_DIR / "fes_paper_chern_convergence.csv",
 index=False,
 encoding="utf-8-sig",
)

 # 5) 单样本统一标签
 paper_result = calculate_sample(

```

```

 sample_id="paper_fes",
 params=PAPER_PARAMS,
 gap_nk=GAP_NK_DEBUG,
 chern_nk=CHERN_NK_DEBUG,
)
 print("\n[Paper sample label]")
 print(json.dumps(paper_result, ensure_ascii=False, indent=2))
 pd.DataFrame([paper_result]).to_csv(
 OUTPUT_DIR / "fes_paper_sample_label.csv",
 index=False,
 encoding="utf-8-sig",
)

6) 可选小批量
if RUN_SMALL_BATCH:
 batch_df = run_small_batch()
 print("\n[Debug batch labels]")
 print(batch_df["phase_label"].value_counts(dropna=False).to_string())
else:
 print("\nRUN_SMALL_BATCH=False: 跳过小批量。")

7) 可选 hr.dat 对照
hr_candidates = [
 Path("wannier90_hr.dat"),
 Path("wannier90_test.dat"),
 Path("fes_wannier90_hr.dat"),
]
existing_hr = next((path for path in hr_candidates if path.exists()), None)
if existing_hr is not None:
 comparison = compare_hr_spectra(existing_hr, PAPER_PARAMS)
 print("\n[hr.dat spectrum comparison]")
 print(comparison.to_string(index=False))
 comparison.to_csv(
 OUTPUT_DIR / "fes_hr_spectrum_comparison.csv",
 index=False,
 encoding="utf-8-sig",
)
else:
 print("\n 未发现 wannier90_hr.dat, 跳过 MagneticTB/Wannier90 对照。")

print("\nStep 01 finished.")

if __name__ == "__main__":
 main()

```

## ► 代码清单 A.47.2 (python, 1450 行)

```

#!/usr/bin/env python3
-*- coding: utf-8 -*-
"""
Step 02 — fes net 五维 Sobol 全局扫描与拓扑存在性筛选

```

任务目标

-----

1. 在去除整体能量平移后的五维参数空间  
( $m_e$ ,  $t_1$ ,  $t_2$ ,  $r_1$ ,  $r_2$ )  
中进行可重复的 Sobol 低差异采样;
2. 对每个参数点完成完整二维 BZ 的直接带隙、间接带隙和  
2-up + 2-down 半填充占据检查;
3. 仅对直接开隙且自旋占据平衡的样本计算非阿贝尔 spin Chern;
4. 找出 spin\_chern\_TI\_candidate、spin\_chern\_band\_metal 和相边界样本;
5. 对所有拓扑/可疑样本执行多网格、多 shift 的严格复核;



6. 输出可直接供 Step 03 机器学习使用的结构化数据集。

本步骤不把非零 spin Chern 自动命名为 confirmed type-II QSH。  
confirmed type-II QSH 仍需后续验证动量分离带反转、边缘态和输运平台。

默认计算规模

-----

Sobol 点数:  $2^{12} = 4096$

粗筛: 25x25 BZ gap, 17x17 Chern

严格复核: 51/71 BZ gap, 多 shift; 21/31/41 Chern, 多 shift

运行示例

-----

快速自检:

```
python FES_step02_sobol_scan.py --test
```

正式扫描:

```
python FES_step02_sobol_scan.py --sobol-power 12 --workers 8 --overwrite
```

断点续算:

```
python FES_step02_sobol_scan.py --sobol-power 12 --workers 8 --resume
```

"""

```
from __future__ import annotations
```

```
from dataclasses import asdict, dataclass, field
```

```
from pathlib import Path
```

```
from concurrent.futures import ProcessPoolExecutor
```

```
from datetime import datetime, timezone
```

```
from typing import Callable, Dict, Iterable, Sequence
```

```
import argparse
```

```
import hashlib
```

```
import json
```

```
import math
```

```
import os
```

```
import shutil
```

```
import sys
```

```
import time
```

```
import numpy as np
```

```
import pandas as pd
```

```
from scipy.stats import qmc
```

```
import matplotlib.pyplot as plt
```

```
np.set_printoptions(precision=10, suppress=True)
```

```
CODE_VERSION = "FES_STEP02_SOBOL_V1_20260712"
```

```
RESULT_SCHEMA_VERSION = "fes_step02_schema_v1"
```

```

```

```
fes 模型常数
```

```

```

```
RAW6 = ["e1", "e2", "t1", "t2", "r1", "r2"]
```

```
REDUCED5 = ["m_e", "t1", "t2", "r1", "r2"]
```

```
N_OCC_TOTAL = 4
```

```
N_OCC_SPIN = 2
```

```
X_FES = 0.207
```

```
ALPHA = 2.0 * X_FES
```

```
BETA = 0.5 - X_FES
```

```
ORBITAL_POSITIONS = np.array(
```

```
[
```

```
 [X_FES, 0.5], [X_FES, 0.5],
```

```
 [-X_FES, 0.5], [-X_FES, 0.5],
```

```
 [0.5, -X_FES], [0.5, -X_FES],
```

```
 [0.5, X_FES], [0.5, X_FES],
```

```
],
```

```
dtype=float,
```

---

```

)

SPIN_UP_INDICES = [0, 2, 4, 6]
SPIN_DOWN_INDICES = [1, 3, 5, 7]

PAPER_PARAMS = {
 "e1": 0.7,
 "e2": -0.4,
 "t1": 0.3,
 "t2": 0.2,
 "r1": 0.3,
 "r2": -0.1,
}

DEFAULT_BOUNDS = {
 "m_e": (-1.0, 1.0),
 "t1": (-1.0, 1.0),
 "t2": (-1.0, 1.0),
 "r1": (-1.0, 1.0),
 "r2": (-1.0, 1.0),
}

@dataclass
class ScanConfig:
 output_dir: str = "outputs_fes6_step02_sobol"
 sobol_power: int = 12
 sobol_seed: int = 20260712
 scramble: bool = True
 workers: int = max(1, min(8, os.cpu_count() or 1))
 checkpoint_every: int = 128

 bounds: Dict[str, tuple[float, float]] = field(
 default_factory=lambda: dict(DEFAULT_BOUNDS)
)

 coarse_gap_nk: int = 25
 coarse_chern_nk: int = 17
 gap_tol: float = 1.0e-3
 chern_tol: float = 0.08
 min_det_tol: float = 1.0e-7

 refine_gap_grids: tuple[int, ...] = (51, 71)
 refine_gap_shifts: tuple[tuple[float, float], ...] = (
 (0.0, 0.0),
 (0.5, 0.0),
 (0.0, 0.5),
 (0.5, 0.5),
)

 refine_chern_grids: tuple[int, ...] = (21, 31, 41)
 refine_chern_shifts: tuple[tuple[float, float], ...] = (
 (0.0, 0.0),
 (0.5, 0.5),
)

 refine_gap_window: float = 0.05
 refine_det_window: float = 0.20
 max_boundary_refine: int = 512

 resume: bool = False
 overwrite: bool = False
 test_mode: bool = False

 @property
 def n_samples(self) -> int:
 return 2 ** int(self.sobol_power)

=====

```

---

```

1. 参数与 Hamiltonian
=====

def validate_params(params: Dict[str, float]) -> Dict[str, float]:
 missing = [name for name in RAW6 if name not in params]
 extra = [name for name in params if name not in RAW6]
 if missing:
 raise ValueError(f" 缺少参数: {missing}")
 if extra:
 raise ValueError(f" 出现未定义参数: {extra}")
 clean = {name: float(params[name]) for name in RAW6}
 if not all(np.isfinite(v) for v in clean.values()):
 raise ValueError(" 参数含 NaN 或无穷大")
 return clean

def raw6_from_reduced5(reduced: Dict[str, float], e0: float = 0.0) -> Dict[str, float]:
 missing = [name for name in REDUCED5 if name not in reduced]
 if missing:
 raise ValueError(f" 缺少五维参数: {missing}")
 m_e = float(reduced["m_e"])
 return {
 "e1": float(e0 + m_e),
 "e2": float(e0 - m_e),
 "t1": float(reduced["t1"]),
 "t2": float(reduced["t2"]),
 "r1": float(reduced["r1"]),
 "r2": float(reduced["r2"]),
 }

def derived_features(params: Dict[str, float]) -> Dict[str, float]:
 p = validate_params(params)
 root2 = math.sqrt(2.0)
 values = np.array(
 [
 0.5 * (p["e1"] - p["e2"]),
 p["t1"], p["t2"], p["r1"], p["r2"],
],
 dtype=float,
)
 return {
 "e0": 0.5 * (p["e1"] + p["e2"]),
 "m_e": 0.5 * (p["e1"] - p["e2"]),
 "t_sum": p["t1"] + p["t2"],
 "t_diff": p["t1"] - p["t2"],
 "t_s": (p["t1"] + p["t2"]) / root2,
 "t_d": (p["t1"] - p["t2"]) / root2,
 "r_sum": p["r1"] + p["r2"],
 "r_diff": p["r1"] - p["r2"],
 "r_s": (p["r1"] + p["r2"]) / root2,
 "r_d": (p["r1"] - p["r2"]) / root2,
 "abs_r_mismatch": abs(abs(p["r1"]) - abs(p["r2"])),
 "t_product": p["t1"] * p["t2"],
 "r_product": p["r1"] * p["r2"],
 "parameter_l2": float(np.linalg.norm(values)),
 "parameter_linf": float(np.max(np.abs(values))),
 }

def h_fes_atomic(kx: float, ky: float, params: Dict[str, float]) -> np.ndarray:
 p = validate_params(params)
 e1, e2, t1, t2, r1, r2 = [p[name] for name in RAW6]

 e11 = np.zeros((4, 4), dtype=np.complex128)
 np.fill_diagonal(e11, [e1, e2, e1, e2])
 e11[0, 2] = t1 * np.exp(-1j * ALPHA * kx)
 e11[2, 0] = np.conj(e11[0, 2])
 e11[1, 3] = t2 * np.exp(-1j * ALPHA * kx)

```

---

```

e11[3, 1] = np.conj(e11[1, 3])

e22 = np.zeros((4, 4), dtype=np.complex128)
np.fill_diagonal(e22, [e2, e1, e2, e1])
e22[0, 2] = t2 * np.exp(+1j * ALPHA * ky)
e22[2, 0] = np.conj(e22[0, 2])
e22[1, 3] = t1 * np.exp(+1j * ALPHA * ky)
e22[3, 1] = np.conj(e22[1, 3])

r_plus = 1j * r1 + r2
r_minus = -1j * r1 + r2
phase_pp = np.exp(+1j * BETA * (kx + ky))
phase_pm = np.exp(+1j * BETA * (kx - ky))
phase_mp = np.exp(-1j * BETA * (kx + ky))
phase_mm = np.exp(-1j * BETA * (kx - ky))

r12 = np.array(
 [
 [phase_pp * r_plus, 0.0, phase_pm * r_minus, 0.0],
 [0.0, phase_pp * r_minus, 0.0, phase_pm * r_plus],
 [phase_mm * r_minus, 0.0, phase_mp * r_plus, 0.0],
 [0.0, phase_mm * r_plus, 0.0, phase_mp * r_minus],
],
 dtype=np.complex128,
)

return np.block([e11, r12], [r12.conj().T, e22])

def periodic_gauge_matrix(kx: float, ky: float) -> np.ndarray:
 phases = np.exp(
 1j * (
 ORBITAL_POSITIONS[:, 0] * float(kx)
 + ORBITAL_POSITIONS[:, 1] * float(ky)
)
)
 return np.diag(phases).astype(np.complex128)

def h_fes_periodic(kx: float, ky: float, params: Dict[str, float]) -> np.ndarray:
 u = periodic_gauge_matrix(kx, ky)
 h = h_fes_atomic(kx, ky, params)
 return u @ h @ u.conj().T

def spin_indices(spin: str) -> list[int]:
 key = spin.lower()
 if key == "up":
 return SPIN_UP_INDICES.copy()
 if key == "down":
 return SPIN_DOWN_INDICES.copy()
 raise ValueError("spin 必须为 'up' 或 'down'")

def h_spin_block_atomic(
 kx: float, ky: float, params: Dict[str, float], spin: str
) -> np.ndarray:
 idx = spin_indices(spin)
 h = h_fes_atomic(kx, ky, params)
 return h[np.ix_(idx, idx)]

def h_spin_block_periodic(
 kx: float, ky: float, params: Dict[str, float], spin: str
) -> np.ndarray:
 idx = spin_indices(spin)
 h = h_fes_periodic(kx, ky, params)
 return h[np.ix_(idx, idx)]

```

---

```

=====
2. BZ 带隙审计
=====

def shifted_k_grid(nk: int, shift: tuple[float, float]) -> tuple[np.ndarray, np.ndarray]:
 sx, sy = shift
 kx = -np.pi + 2.0 * np.pi * (np.arange(nk) + float(sx)) / nk
 ky = -np.pi + 2.0 * np.pi * (np.arange(nk) + float(sy)) / nk
 return kx, ky

def scan_band_gaps_shifted(
 params: Dict[str, float],
 nk: int,
 shift: tuple[float, float] = (0.0, 0.0),
 gap_tol: float = 1.0e-3,
) -> Dict[str, object]:
 """
 使用两个 4x4 自旋块完成半填充 gap 审计, 比重复对角化 8x8 更高效。
 """
 kxs, kys = shifted_k_grid(nk, shift)

 min_direct = np.inf
 direct_kx = np.nan
 direct_ky = np.nan
 max_valence = -np.inf
 vbm_kx = np.nan
 vbm_ky = np.nan
 min_conduction = np.inf
 cbm_kx = np.nan
 cbm_ky = np.nan
 min_spin_gap_up = np.inf
 min_spin_gap_down = np.inf
 min_balanced_sector_gap = np.inf
 balanced_gap_kx = np.nan
 balanced_gap_ky = np.nan
 mismatch_count = 0
 min_n_up = N_OCC_TOTAL
 max_n_up = 0

 for kx in kxs:
 for ky in kys:
 kx_f = float(kx)
 ky_f = float(ky)
 up_e = np.linalg.eigvalsh(h_spin_block_atomic(kx_f, ky_f, params, "up"))
 down_e = np.linalg.eigvalsh(h_spin_block_atomic(kx_f, ky_f, params, "down"))

 tagged = [(float(e), 1) for e in up_e]
 tagged += [(float(e), 0) for e in down_e]
 tagged.sort(key=lambda item: item[0])
 energies = np.array([e for e, _ in tagged], dtype=float)

 direct_gap = float(energies[N_OCC_TOTAL] - energies[N_OCC_TOTAL - 1])
 if direct_gap < min_direct:
 min_direct = direct_gap
 direct_kx = kx_f
 direct_ky = ky_f

 valence = float(energies[N_OCC_TOTAL - 1])
 conduction = float(energies[N_OCC_TOTAL])
 if valence > max_valence:
 max_valence = valence
 vbm_kx = kx_f
 vbm_ky = ky_f
 if conduction < min_conduction:
 min_conduction = conduction
 cbm_kx = kx_f
 cbm_ky = ky_f

```

---

```

spin_gap_up = float(up_e[N_OCC_SPIN] - up_e[N_OCC_SPIN - 1])
spin_gap_down = float(down_e[N_OCC_SPIN] - down_e[N_OCC_SPIN - 1])
min_spin_gap_up = min(min_spin_gap_up, spin_gap_up)
min_spin_gap_down = min(min_spin_gap_down, spin_gap_down)

balanced_gap = float(
 min(up_e[N_OCC_SPIN], down_e[N_OCC_SPIN])
 - max(up_e[N_OCC_SPIN - 1], down_e[N_OCC_SPIN - 1])
)
if balanced_gap < min_balanced_sector_gap:
 min_balanced_sector_gap = balanced_gap
 balanced_gap_kx = kx_f
 balanced_gap_ky = ky_f

n_up_occ = int(sum(tag for _, tag in tagged[:N_OCC_TOTAL]))
min_n_up = min(min_n_up, n_up_occ)
max_n_up = max(max_n_up, n_up_occ)
if n_up_occ != N_OCC_SPIN:
 mismatch_count += 1

indirect_gap = float(min_conduction - max_valence)
balanced = min_balanced_sector_gap > gap_tol and mismatch_count == 0

return {
 "gap_nk": int(nk),
 "gap_shift_x": float(shift[0]),
 "gap_shift_y": float(shift[1]),
 "min_direct_gap": float(min_direct),
 "indirect_gap": indirect_gap,
 "direct_gap_kx": direct_kx,
 "direct_gap_ky": direct_ky,
 "vbm": float(max_valence),
 "vbm_kx": vbm_kx,
 "vbm_ky": vbm_ky,
 "cbm": float(min_conduction),
 "cbm_kx": cbm_kx,
 "cbm_ky": cbm_ky,
 "min_spin_gap_up": float(min_spin_gap_up),
 "min_spin_gap_down": float(min_spin_gap_down),
 "min_balanced_sector_gap": float(min_balanced_sector_gap),
 "balanced_gap_kx": balanced_gap_kx,
 "balanced_gap_ky": balanced_gap_ky,
 "spin_occupancy_mismatch_count": int(mismatch_count),
 "min_n_up_in_lowest4": int(min_n_up),
 "max_n_up_in_lowest4": int(max_n_up),
 "is_direct_gapped": int(min_direct > gap_tol),
 "is_physical_insulator": int(indirect_gap > gap_tol),
 "is_balanced_spin_sector": int(balanced),
}

def consensus_gap_audit(
 params: Dict[str, float],
 grids: Sequence[int],
 shifts: Sequence[tuple[float, float]],
 gap_tol: float,
) -> tuple[Dict[str, object], pd.DataFrame]:
 rows: list[Dict[str, object]] = []
 for nk in grids:
 for shift in shifts:
 row = scan_band_gaps_shifted(params, int(nk), shift, gap_tol)
 rows.append(row)
 df = pd.DataFrame(rows)

保守汇总: 跨所有网格/shift 取最小直接带隙, 跨所有采样取全局 VBM/CBM.
min_direct_idx = int(df["min_direct_gap"].idxmin())
vbm_idx = int(df["vbm"].idxmax())
cbm_idx = int(df["cbm"].idxmin())

```

---

```

max_vbm = float(df.loc[vbm_idx, "vbm"])
min_cbm = float(df.loc[cbm_idx, "cbm"])

summary = {
 "verified_min_direct_gap": float(df["min_direct_gap"].min()),
 "verified_indirect_gap": float(min_cbm - max_vbm),
 "verified_direct_gap_kx": float(df.loc[min_direct_idx, "direct_gap_kx"]),
 "verified_direct_gap_ky": float(df.loc[min_direct_idx, "direct_gap_ky"]),
 "verified_vbm": max_vbm,
 "verified_vbm_kx": float(df.loc[vbm_idx, "vbm_kx"]),
 "verified_vbm_ky": float(df.loc[vbm_idx, "vbm_ky"]),
 "verified_cbm": min_cbm,
 "verified_cbm_kx": float(df.loc[cbm_idx, "cbm_kx"]),
 "verified_cbm_ky": float(df.loc[cbm_idx, "cbm_ky"]),
 "verified_min_spin_gap_up": float(df["min_spin_gap_up"].min()),
 "verified_min_spin_gap_down": float(df["min_spin_gap_down"].min()),
 "verified_min_balanced_sector_gap": float(
 df["min_balanced_sector_gap"].min()
),
 "verified_total_mismatch_count": int(
 df["spin_occupancy_mismatch_count"].sum()
),
 "verified_is_direct_gapped": int(
 float(df["min_direct_gap"].min()) > gap_tol
),
 "verified_is_physical_insulator": int(
 float(min_cbm - max_vbm) > gap_tol
),
 "verified_is_balanced_spin_sector": int(
 float(df["min_balanced_sector_gap"].min()) > gap_tol
 and int(df["spin_occupancy_mismatch_count"].sum()) == 0
),
 "verified_gap_runs": int(len(df)),
}
return summary, df

=====
3. 非阿贝尔 Fukui Chern
=====

def _normalize_link(value: complex, eps: float = 1.0e-14) -> complex:
 amplitude = abs(value)
 if amplitude < eps:
 raise FloatingPointError(f"Link determinant too small: {amplitude:.3e}")
 return value / amplitude

def fukui_chern_subspace_shifted(
 h_func: Callable[[float, float], np.ndarray],
 n_occ: int,
 nk: int,
 shift: tuple[float, float] = (0.0, 0.0),
) -> Dict[str, object]:
 sample_h = h_func(0.0, 0.0)
 dim = sample_h.shape[0]
 if n_occ <= 0 or n_occ >= dim:
 raise ValueError(f"n_occ={n_occ} 与 Hamiltonian 维数 {dim} 不兼容")

 sx, sy = shift
 kxs = 2.0 * np.pi * (np.arange(nk) + float(sx)) / nk
 kys = 2.0 * np.pi * (np.arange(nk) + float(sy)) / nk
 occupied = np.empty((nk, nk, dim, n_occ), dtype=np.complex128)

 for ix, kx in enumerate(kxs):
 for iy, ky in enumerate(kys):
 _, vecs = np.linalg.eigh(h_func(float(kx), float(ky)))
 occupied[ix, iy] = vecs[:, :n_occ]

```

---

```

total_phase = 0.0
min_det = 1.0
for ix in range(nk):
 for iy in range(nk):
 v = occupied[ix, iy]
 vx = occupied[(ix + 1) % nk, iy]
 vy = occupied[ix, (iy + 1) % nk]
 vxy = occupied[(ix + 1) % nk, (iy + 1) % nk]

 lx = np.linalg.det(v.conj().T @ vx)
 ly = np.linalg.det(v.conj().T @ vy)
 lx_y = np.linalg.det(vy.conj().T @ vxy)
 ly_x = np.linalg.det(vx.conj().T @ vxy)
 min_det = min(min_det, abs(lx), abs(ly), abs(lx_y), abs(ly_x))

 ux = _normalize_link(lx)
 uy = _normalize_link(ly)
 ux_y = _normalize_link(lx_y)
 uy_x = _normalize_link(ly_x)
 total_phase += float(np.angle(ux * uy_x / (ux_y * uy)))

return {
 "chern": float(total_phase / (2.0 * np.pi)),
 "min_det": float(min_det),
 "nk": int(nk),
 "shift_x": float(shift[0]),
 "shift_y": float(shift[1]),
}

def rounded_integer_if_close(value: float, tolerance: float) -> int | None:
 nearest = int(np rint(value))
 return nearest if abs(value - nearest) <= tolerance else None

def calculate_coarse_spin_chern(
 params: Dict[str, float],
 nk: int,
 chern_tol: float,
 min_det_tol: float,
) -> Dict[str, object]:
 up = fukui_chern_subspace_shifted(
 lambda kx, ky: h_spin_block_periodic(kx, ky, params, "up"),
 N_OCC_SPIN,
 nk,
 (0.0, 0.0),
)
 down = fukui_chern_subspace_shifted(
 lambda kx, ky: h_spin_block_periodic(kx, ky, params, "down"),
 N_OCC_SPIN,
 nk,
 (0.0, 0.0),
)

 c_up = float(up["chern"])
 c_down = float(down["chern"])
 c_total = c_up + c_down
 c_up_int = rounded_integer_if_close(c_up, chern_tol)
 c_down_int = rounded_integer_if_close(c_down, chern_tol)
 c_total_int = rounded_integer_if_close(c_total, chern_tol)
 reliable = (
 c_up_int is not None
 and c_down_int is not None
 and c_total_int is not None
 and float(up["min_det"]) > min_det_tol
 and float(down["min_det"]) > min_det_tol
)

 return {

```



---

```

 "chern_nk": int(nk),
 "chern_up": c_up,
 "chern_down": c_down,
 "chern_total_inferred": c_total,
 "spin_chern": 0.5 * (c_up - c_down),
 "chern_up_int": c_up_int,
 "chern_down_int": c_down_int,
 "chern_total_int": c_total_int,
 "min_det_up": float(up["min_det"]),
 "min_det_down": float(down["min_det"]),
 "chern_reliable": int(reliable),
}

def consensus_chern_audit(
 params: Dict[str, float],
 grids: Sequence[int],
 shifts: Sequence[tuple[float, float]],
 chern_tol: float,
 min_det_tol: float,
 include_total: bool = True,
) -> tuple[Dict[str, object], pd.DataFrame]:
 rows: list[Dict[str, object]] = []
 for nk in grids:
 for shift in shifts:
 try:
 up = fukui_chern_subspace_shifted(
 lambda kx, ky: h_spin_block_periodic(kx, ky, params, "up"),
 N_OCC_SPIN,
 int(nk),
 shift,
)
 down = fukui_chern_subspace_shifted(
 lambda kx, ky: h_spin_block_periodic(kx, ky, params, "down"),
 N_OCC_SPIN,
 int(nk),
 shift,
)
 row: Dict[str, object] = {
 "nk": int(nk),
 "shift_x": float(shift[0]),
 "shift_y": float(shift[1]),
 "chern_up": float(up["chern"]),
 "chern_down": float(down["chern"]),
 "min_det_up": float(up["min_det"]),
 "min_det_down": float(down["min_det"]),
 "error": "",
 }
 if include_total:
 total = fukui_chern_subspace_shifted(
 lambda kx, ky: h_fes_periodic(kx, ky, params),
 N_OCC_TOTAL,
 int(nk),
 shift,
)
 row["chern_total"] = float(total["chern"])
 row["min_det_total"] = float(total["min_det"])
 else:
 row["chern_total"] = float(up["chern"] + down["chern"])
 row["min_det_total"] = np.nan
 rows.append(row)
 except Exception as exc:
 rows.append(
 {
 "nk": int(nk),
 "shift_x": float(shift[0]),
 "shift_y": float(shift[1]),
 "error": repr(exc),
 }
)

```

---

```

)

df = pd.DataFrame(rows)
valid = df[df["error"].fillna("") == ""].copy()
if valid.empty:
 return {
 "verified_chern_reliable": 0,
 "verified_chern_error": "all_chern_runs_failed",
 "verified_chern_runs": 0,
 }, df

int_rows = []
all_integer = True
for _, row in valid.iterrows():
 cu = rounded_integer_if_close(float(row["chern_up"]), chern_tol)
 cd = rounded_integer_if_close(float(row["chern_down"]), chern_tol)
 ct = rounded_integer_if_close(float(row["chern_total"]), chern_tol)
 int_rows.append((cu, cd, ct))
 if cu is None or cd is None or ct is None:
 all_integer = False

consensus = len(set(int_rows)) == 1 if all_integer else False
min_det_up = float(valid["min_det_up"].min())
min_det_down = float(valid["min_det_down"].min())
if include_total:
 min_det_total = float(valid["min_det_total"].min())
else:
 min_det_total = np.nan

det_ok = min_det_up > min_det_tol and min_det_down > min_det_tol
if include_total:
 det_ok = det_ok and min_det_total > min_det_tol

if consensus:
 cu_int, cd_int, ct_int = int_rows[0]
else:
 cu_int = cd_int = ct_int = None

sum_rule_error = float(
 np.max(
 np.abs(
 valid["chern_total"].to_numpy(float)
 - valid["chern_up"].to_numpy(float)
 - valid["chern_down"].to_numpy(float)
)
)
)

reliable = (
 len(valid) == len(rows)
 and consensus
 and det_ok
 and sum_rule_error < chern_tol
)

summary = {
 "verified_chern_up_mean": float(valid["chern_up"].mean()),
 "verified_chern_down_mean": float(valid["chern_down"].mean()),
 "verified_chern_total_mean": float(valid["chern_total"].mean()),
 "verified_chern_up_int": cu_int,
 "verified_chern_down_int": cd_int,
 "verified_chern_total_int": ct_int,
 "verified_min_det_up": min_det_up,
 "verified_min_det_down": min_det_down,
 "verified_min_det_total": min_det_total,
 "verified_chern_sum_rule_error": sum_rule_error,
 "verified_chern_consensus": int(consensus),
 "verified_chern_reliable": int(reliable),
 "verified_chern_runs": int(len(valid)),

```

---

```

 "verified_chern_error": "" if len(valid) == len(rows) else "some_runs_failed",
 }
 return summary, df

```

```

=====
4. 粗筛与严格复核标签
=====

```

```

def empty_chern_fields() -> Dict[str, object]:
 return {
 "chern_nk": np.nan,
 "chern_up": np.nan,
 "chern_down": np.nan,
 "chern_total_inferred": np.nan,
 "spin_chern": np.nan,
 "chern_up_int": None,
 "chern_down_int": None,
 "chern_total_int": None,
 "min_det_up": np.nan,
 "min_det_down": np.nan,
 "chern_reliable": 0,
 }

def classify_from_chern(
 gap: Dict[str, object],
 chern: Dict[str, object],
) -> tuple[str, int]:
 direct = bool(int(gap["is_direct_gapped"]))
 insulator = bool(int(gap["is_physical_insulator"]))
 balanced = bool(int(gap["is_balanced_spin_sector"]))
 reliable = bool(int(chern.get("chern_reliable", 0)))

 if not direct:
 return "noninsulating_or_gap_closing", 0
 if not balanced:
 return "spin_sector_filling_mismatch", 0
 if not reliable:
 return "chern_unreliable", 0

 cu = chern.get("chern_up_int")
 cd = chern.get("chern_down_int")
 ct = chern.get("chern_total_int")
 spin_topological = (
 cu is not None
 and cd is not None
 and ct == 0
 and cu == -cd
 and cu != 0
)

 if not insulator:
 return (
 "spin_chern_band_metal" if spin_topological else "indirect_overlap",
 0,
)
 if spin_topological:
 return "spin_chern_TI_candidate", 1
 if cu == 0 and cd == 0 and ct == 0:
 return "trivial_insulator", 0
 if ct not in (None, 0):
 return "Chern_insulator", 0
 return "other_gapped_phase", 0

```

```

def coarse_label_sample(task: tuple) -> Dict[str, object]:
 (
 sample_id,

```

---

```

 sobol_index,
 params,
 gap_nk,
 chern_nk,
 gap_tol,
 chern_tol,
 min_det_tol,
) = task

p = validate_params(params)
row: Dict[str, object] = {
 "sample_id": str(sample_id),
 "sobol_index": int(sobol_index),
 **p,
 **derived_features(p),
 **empty_chern_fields(),
 "is_spin_chern_TI_candidate": 0,
 "is_typeII_QSH_confirmed": 0,
 "phase_label": "numeric_error",
 "error": "",
}

try:
 gap = scan_band_gaps_shifted(p, int(gap_nk), (0.0, 0.0), float(gap_tol))
 row.update(gap)
except Exception as exc:
 row["error"] = f"gap_scan_failed: {repr(exc)}"
 return row

if not bool(int(gap["is_direct_gapped"])):
 row["phase_label"] = "noninsulating_or_gap_closing"
 return row
if not bool(int(gap["is_balanced_spin_sector"])):
 row["phase_label"] = "spin_sector_filling_mismatch"
 return row

try:
 chern = calculate_coarse_spin_chern(
 p,
 int(chern_nk),
 float(chern_tol),
 float(min_det_tol),
)
 row.update(chern)
except Exception as exc:
 row["phase_label"] = "chern_unreliable"
 row["error"] = f"chern_failed: {repr(exc)}"
 return row

label, candidate = classify_from_chern(gap, chern)
row["phase_label"] = label
row["is_spin_chern_TI_candidate"] = int(candidate)
return row

def verified_phase_label(row: Dict[str, object]) -> tuple[str, int]:
 if not bool(int(row.get("verified_is_direct_gapped", 0))):
 return "verified_noninsulating_or_gap_closing", 0
 if not bool(int(row.get("verified_is_balanced_spin_sector", 0))):
 return "verified_spin_sector_filling_mismatch", 0
 if not bool(int(row.get("verified_chern_reliable", 0))):
 return "verified_chern_unreliable", 0

 cu = row.get("verified_chern_up_int")
 cd = row.get("verified_chern_down_int")
 ct = row.get("verified_chern_total_int")
 insulator = bool(int(row.get("verified_is_physical_insulator", 0)))
 spin_topological = (
 cu is not None

```

---

```

 and cd is not None
 and ct == 0
 and cu == -cd
 and cu != 0
)

 if not insulator:
 return (
 "verified_spin_chern_band_metal"
 if spin_topological
 else "verified_indirect_overlap",
 0,
)
 if spin_topological:
 return "verified_spin_chern_TI_candidate", 1
 if cu == 0 and cd == 0 and ct == 0:
 return "verified_trivial_insulator", 0
 if ct not in (None, 0):
 return "verified_Chern_insulator", 0
 return "verified_other_gapped_phase", 0

def refine_sample(
 coarse_row: Dict[str, object],
 config: ScanConfig,
 detail_dir: Path,
) -> Dict[str, object]:
 params = {name: float(coarse_row[name]) for name in RAW6}
 sample_id = str(coarse_row["sample_id"])
 result: Dict[str, object] = dict(coarse_row)
 result["refined"] = 1
 result["refine_error"] = ""

 try:
 gap_summary, gap_table = consensus_gap_audit(
 params,
 config.refine_gap_grids,
 config.refine_gap_shifts,
 config.gap_tol,
)
 result.update(gap_summary)
 gap_table.insert(0, "sample_id", sample_id)
 gap_table.to_csv(
 detail_dir / f"{sample_id}_gap_consensus.csv",
 index=False,
 encoding="utf-8-sig",
)
 except Exception as exc:
 result["verified_phase_label"] = "verified_numeric_error"
 result["verified_is_spin_chern_TI_candidate"] = 0
 result["refine_error"] = f"verified_gap_failed: {repr(exc)}"
 return result

 if not bool(int(result["verified_is_direct_gapped"])):
 label, candidate = verified_phase_label(result)
 result["verified_phase_label"] = label
 result["verified_is_spin_chern_TI_candidate"] = candidate
 return result

 if not bool(int(result["verified_is_balanced_spin_sector"])):
 label, candidate = verified_phase_label(result)
 result["verified_phase_label"] = label
 result["verified_is_spin_chern_TI_candidate"] = candidate
 return result

 try:
 chern_summary, chern_table = consensus_chern_audit(
 params,
 config.refine_chern_grids,

```

```

 config.refine_chern_shifts,
 config.chern_tol,
 config.min_det_tol,
 include_total=True,
)
 result.update(chern_summary)
 chern_table.insert(0, "sample_id", sample_id)
 chern_table.to_csv(
 detail_dir / f"{sample_id}_chern_consensus.csv",
 index=False,
 encoding="utf-8-sig",
)
except Exception as exc:
 result["verified_chern_reliable"] = 0
 result["verified_phase_label"] = "verified_chern_unreliable"
 result["verified_is_spin_chern_TI_candidate"] = 0
 result["refine_error"] = f"verified_chern_failed: {repr(exc)}"
 return result

label, candidate = verified_phase_label(result)
result["verified_phase_label"] = label
result["verified_is_spin_chern_TI_candidate"] = candidate
result["is_typeII_QSH_confirmed"] = 0
return result

=====
5. Sobol 采样、断点续算与输出
=====

def validate_bounds(bounds: Dict[str, tuple[float, float]]) -> None:
 missing = [name for name in REDUCED5 if name not in bounds]
 extra = [name for name in bounds if name not in REDUCED5]
 if missing or extra:
 raise ValueError(f"bounds 不匹配: missing={missing}, extra={extra}")
 for name in REDUCED5:
 lo, hi = bounds[name]
 if not np.isfinite(lo) or not np.isfinite(hi) or lo >= hi:
 raise ValueError(f"非法范围 {name}: {(lo, hi)}")

def generate_sobol_parameters(config: ScanConfig) -> pd.DataFrame:
 validate_bounds(config.bounds)
 sampler = qmc.Sobol(
 d=len(REDUCED5),
 scramble=bool(config.scramble),
 seed=int(config.sobol_seed),
)
 unit = sampler.random_base2(m=int(config.sobol_power))
 lower = np.array([config.bounds[name][0] for name in REDUCED5], dtype=float)
 upper = np.array([config.bounds[name][1] for name in REDUCED5], dtype=float)
 scaled = qmc.scale(unit, lower, upper)

 rows = []
 for index, values in enumerate(scaled):
 reduced = {name: float(values[i]) for i, name in enumerate(REDUCED5)}
 raw = raw6_from_reduced5(reduced, e0=0.0)
 rows.append(
 {
 "sample_id": f"sobol_{index:07d}",
 "sobol_index": int(index),
 **raw,
 }
)
 return pd.DataFrame(rows)

def config_to_jsonable(config: ScanConfig) -> Dict[str, object]:
 data = asdict(config)

```

---

```

data["n_samples"] = config.n_samples
return data

def script_sha256() -> str | None:
 if "__file__" not in globals():
 return None
 path = Path(__file__).resolve()
 if not path.exists():
 return None
 return hashlib.sha256(path.read_bytes()).hexdigest()

def prepare_output(config: ScanConfig) -> Path:
 output = Path(config.output_dir)
 if output.exists() and config.overwrite and not config.resume:
 shutil.rmtree(output)
 output.mkdir(parents=True, exist_ok=True)
 (output / "refine_details").mkdir(exist_ok=True)
 return output

def write_metadata(config: ScanConfig, output: Path) -> None:
 metadata = {
 "code_version": CODE_VERSION,
 "result_schema_version": RESULT_SCHEMA_VERSION,
 "run_time_utc": datetime.now(timezone.utc).isoformat(),
 "script_sha256": script_sha256(),
 "python_version": sys.version,
 "numpy_version": np.__version__,
 "pandas_version": pd.__version__,
 "config": config_to_jsonable(config),
 "scientific_scope": (
 "Five-dimensional fes global scan; nonzero spin Chern is a candidate, "
 "not confirmed type-II QSH."
),
 }
 (output / "fes_step02_run_metadata.json").write_text(
 json.dumps(metadata, ensure_ascii=False, indent=2),
 encoding="utf-8",
)

def save_checkpoint(rows: list[Dict[str, object]], path: Path) -> None:
 if not rows:
 return
 df = pd.DataFrame(rows).sort_values("sobol_index").reset_index(drop=True)
 temp = path.with_suffix(path.suffix + ".tmp")
 df.to_csv(temp, index=False, encoding="utf-8-sig")
 temp.replace(path)

def run_coarse_scan(
 parameter_df: pd.DataFrame,
 config: ScanConfig,
 output: Path,
) -> pd.DataFrame:
 result_path = output / "fes_step02_coarse_results.csv"
 existing_rows: list[Dict[str, object]] = []
 completed_ids: set[str] = set()

 if config.resume and result_path.exists():
 existing = pd.read_csv(result_path)
 existing_rows = existing.to_dict("records")
 completed_ids = set(existing["sample_id"].astype(str))
 print(f"Resume: loaded {len(existing_rows)} completed samples")

 pending = parameter_df[~parameter_df["sample_id"].isin(completed_ids)].copy()
 tasks = []

```

---

```

for _, row in pending.iterrows():
 params = {name: float(row[name]) for name in RAW6}
 tasks.append(
 (
 str(row["sample_id"]),
 int(row["sobol_index"]),
 params,
 int(config.coarse_gap_nk),
 int(config.coarse_chern_nk),
 float(config.gap_tol),
 float(config.chern_tol),
 float(config.min_det_tol),
)
)

rows = list(existing_rows)
total = len(tasks)
if total == 0:
 return pd.DataFrame(rows).sort_values("sobol_index").reset_index(drop=True)

print(
 f"Coarse scan: {total} pending / {len(parameter_df)} total, "
 f"workers={config.workers}"
)
start = time.time()

if int(config.workers) <= 1:
 iterator: Iterable[Dict[str, object]] = map(coarse_label_sample, tasks)
 for count, result in enumerate(iterator, start=1):
 rows.append(result)
 if count % config.checkpoint_every == 0 or count == total:
 save_checkpoint(rows, result_path)
 elapsed = time.time() - start
 print(f"Coarse {count}/{total}; elapsed={elapsed:.1f}s")
else:
 with ProcessPoolExecutor(max_workers=int(config.workers)) as executor:
 iterator = executor.map(coarse_label_sample, tasks, chunksize=4)
 for count, result in enumerate(iterator, start=1):
 rows.append(result)
 if count % config.checkpoint_every == 0 or count == total:
 save_checkpoint(rows, result_path)
 elapsed = time.time() - start
 print(f"Coarse {count}/{total}; elapsed={elapsed:.1f}s")

result_df = pd.DataFrame(rows).sort_values("sobol_index").reset_index(drop=True)
result_df.to_csv(result_path, index=False, encoding="utf-8-sig")
return result_df

def select_refine_queue(coarse_df: pd.DataFrame, config: ScanConfig) -> pd.DataFrame:
 mandatory_labels = {
 "spin_chern_TI_candidate",
 "spin_chern_band_metal",
 "chern_unreliable",
 "other_gapped_phase",
 "Chern_insulator",
 }
 mandatory = coarse_df[coarse_df["phase_label"].isin(mandatory_labels)].copy()

 direct = pd.to_numeric(coarse_df["min_direct_gap"], errors="coerce")
 min_det_up = pd.to_numeric(coarse_df["min_det_up"], errors="coerce")
 min_det_down = pd.to_numeric(coarse_df["min_det_down"], errors="coerce")
 balanced = pd.to_numeric(
 coarse_df["is_balanced_spin_sector"], errors="coerce"
).fillna(0).astype(int)

 boundary_mask = (
 (balanced == 1)
 & (

```



---

```

 (direct.abs() <= float(config.refine_gap_window))
 | (min_det_up <= float(config.refine_det_window))
 | (min_det_down <= float(config.refine_det_window))
)
)
boundary = coarse_df[boundary_mask].copy()
if len(boundary) > int(config.max_boundary_refine):
 boundary["_priority_gap"] = pd.to_numeric(
 boundary["min_direct_gap"], errors="coerce"
).abs()
 boundary = boundary.sort_values("_priority_gap").head(
 int(config.max_boundary_refine)
)
 boundary = boundary.drop(columns=["_priority_gap"])

queue = pd.concat([mandatory, boundary], ignore_index=True)
queue = queue.drop_duplicates("sample_id").sort_values("sobol_index")
queue["refine_reason"] = queue["phase_label"].astype(str)
return queue.reset_index(drop=True)

def run_refinement(
 queue_df: pd.DataFrame,
 config: ScanConfig,
 output: Path,
) -> pd.DataFrame:
 result_path = output / "fes_step02_verified_results.csv"
 detail_dir = output / "refine_details"
 existing_rows: list[Dict[str, object]] = []
 completed_ids: set[str] = set()

 if config.resume and result_path.exists():
 existing = pd.read_csv(result_path)
 existing_rows = existing.to_dict("records")
 completed_ids = set(existing["sample_id"].astype(str))

 pending = queue_df[~queue_df["sample_id"].isin(completed_ids)]
 rows = list(existing_rows)
 total = len(pending)
 print(f"Refinement queue: {len(queue_df)} total, {total} pending")

 for count, (_, coarse_row) in enumerate(pending.iterrows(), start=1):
 result = refine_sample(coarse_row.to_dict(), config, detail_dir)
 rows.append(result)
 if count % 10 == 0 or count == total:
 save_checkpoint(rows, result_path)
 print(f"Refined {count}/{total}")

 if rows:
 verified_df = pd.DataFrame(rows).sort_values("sobol_index").reset_index(drop=True)
 else:
 verified_df = pd.DataFrame()
 verified_df.to_csv(result_path, index=False, encoding="utf-8-sig")
 return verified_df

def build_ml_dataset(
 coarse_df: pd.DataFrame,
 verified_df: pd.DataFrame,
 output: Path,
) -> pd.DataFrame:
 data = coarse_df.copy()
 data["final_phase_label"] = data["phase_label"].astype(str)
 data["final_label_source"] = "coarse"
 data["final_is_spin_chern_TI_candidate"] = pd.to_numeric(
 data["is_spin_chern_TI_candidate"], errors="coerce"
).fillna(0).astype(int)

 if not verified_df.empty:

```

---

```

verified_map = verified_df.set_index("sample_id")
for idx, sample_id in data["sample_id"].items():
 if sample_id not in verified_map.index:
 continue
 v = verified_map.loc[sample_id]
 data.at[idx, "final_phase_label"] = v.get(
 "verified_phase_label", data.at[idx, "phase_label"]
)
 data.at[idx, "final_label_source"] = "verified"
 data.at[idx, "final_is_spin_chern_TI_candidate"] = int(
 v.get("verified_is_spin_chern_TI_candidate", 0)
)

data["ml_primary_class"] = data["final_phase_label"].replace(
 {
 "verified_noninsulating_or_gap_closing": "noninsulating_or_gap_closing",
 "verified_spin_sector_filling_mismatch": "spin_sector_filling_mismatch",
 "verified_chern_unreliable": "boundary_or_unreliable",
 "verified_indirect_overlap": "indirect_overlap",
 "verified_spin_chern_band_metal": "spin_chern_band_metal",
 "verified_trivial_insulator": "trivial_insulator",
 "verified_spin_chern_TI_candidate": "spin_chern_TI_candidate",
 "verified_Chern_insulator": "Chern_insulator",
 "verified_other_gapped_phase": "other_gapped_phase",
 }
)

data.to_csv(
 output / "fes_step02_ml_dataset.csv",
 index=False,
 encoding="utf-8-sig",
)

return data

def write_summaries(
 coarse_df: pd.DataFrame,
 verified_df: pd.DataFrame,
 ml_df: pd.DataFrame,
 output: Path,
) -> None:
 coarse_counts = (
 coarse_df["phase_label"].value_counts(dropna=False).rename_axis("phase_label")
 .reset_index(name="count")
)
 coarse_counts["fraction"] = coarse_counts["count"] / len(coarse_df)
 coarse_counts.to_csv(
 output / "fes_step02_coarse_phase_counts.csv",
 index=False,
 encoding="utf-8-sig",
)

 final_counts = (
 ml_df["ml_primary_class"].value_counts(dropna=False).rename_axis("phase_label")
 .reset_index(name="count")
)
 final_counts["fraction"] = final_counts["count"] / len(ml_df)
 final_counts.to_csv(
 output / "fes_step02_final_phase_counts.csv",
 index=False,
 encoding="utf-8-sig",
)

 if not verified_df.empty and "verified_phase_label" in verified_df.columns:
 verified_counts = (
 verified_df["verified_phase_label"].value_counts(dropna=False)
 .rename_axis("verified_phase_label")
 .reset_index(name="count")
)

```

```

 verified_counts.to_csv(
 output / "fes_step02_verified_phase_counts.csv",
 index=False,
 encoding="utf-8-sig",
)

 topo = ml_df[
 ml_df["ml_primary_class"].isin(
 ["spin_chern_TI_candidate", "spin_chern_band_metal"]
)
].copy()
 topo.to_csv(
 output / "fes_step02_topological_candidates.csv",
 index=False,
 encoding="utf-8-sig",
)

 # 简单诊断图: 每张图单独输出, 不设置固定颜色。
 fig, ax = plt.subplots(figsize=(9.0, 5.4))
 ax.bar(final_counts["phase_label"].astype(str), final_counts["count"])
 ax.set_ylabel("Count")
 ax.set_title("fes Step 02 final phase counts")
 ax.tick_params(axis="x", rotation=35)
 fig.tight_layout()
 fig.savefig(output / "fes_step02_final_phase_counts.png", dpi=220)
 plt.close(fig)

 summary = {
 "n_total": int(len(ml_df)),
 "n_refined": int(len(verified_df)),
 "n_final_spin_chern_TI_candidate": int(
 (ml_df["ml_primary_class"] == "spin_chern_TI_candidate").sum()
),
 "n_final_spin_chern_band_metal": int(
 (ml_df["ml_primary_class"] == "spin_chern_band_metal").sum()
),
 "coarse_phase_counts": {
 str(row["phase_label"]): int(row["count"])
 for _, row in coarse_counts.iterrows()
 },
 "final_phase_counts": {
 str(row["phase_label"]): int(row["count"])
 for _, row in final_counts.iterrows()
 },
 }
 (output / "fes_step02_summary.json").write_text(
 json.dumps(summary, ensure_ascii=False, indent=2),
 encoding="utf-8",
)

```

```

=====
6. 主流程
=====

```

```

def audit_paper_point(config: ScanConfig, output: Path) -> None:
 gap = scan_band_gaps_shifted(
 PAPER_PARAMS,
 max(31, config.coarse_gap_nk),
 (0.0, 0.0),
 config.gap_tol,
)
 chern = calculate_coarse_spin_chern(
 PAPER_PARAMS,
 max(21, config.coarse_chern_nk),
 config.chern_tol,
 config.min_det_tol,
)
 label, candidate = classify_from_chern(gap, chern)

```

---

```

row = {
 "sample_id": "paper_reference",
 **PAPER_PARAMS,
 **derived_features(PAPER_PARAMS),
 **gap,
 **chern,
 "phase_label": label,
 "is_spin_chern_TI_candidate": candidate,
}
pd.DataFrame([row]).to_csv(
 output / "fes_step02_paper_reference.csv",
 index=False,
 encoding="utf-8-sig",
)
if label != "trivial_insulator":
 raise AssertionError(
 f" 文献参数基准标签异常: expected trivial_insulator, got {label}"
)

def run_pipeline(config: ScanConfig) -> Dict[str, Path]:
 if config.test_mode:
 config.sobol_power = 5
 config.workers = 1
 config.coarse_gap_nk = 13
 config.coarse_chern_nk = 9
 config.refine_gap_grids = (17,)
 config.refine_gap_shifts = ((0.0, 0.0), (0.5, 0.5))
 config.refine_chern_grids = (11, 15)
 config.refine_chern_shifts = ((0.0, 0.0),)
 config.max_boundary_refine = 8
 config.output_dir = str(Path(config.output_dir).with_name(
 Path(config.output_dir).name + "_test"
))

 output = prepare_output(config)
 write_metadata(config, output)

 print("=" * 80)
 print("fes Step 02 Sobol global scan")
 print("Version :", CODE_VERSION)
 print("Output :", output.resolve())
 print("Samples :", config.n_samples)
 print("Workers :", config.workers)
 print("Bounds :", config.bounds)
 print("=" * 80)

 audit_paper_point(config, output)

 parameter_path = output / "fes_step02_sobol_parameters.csv"
 if config.resume and parameter_path.exists():
 parameter_df = pd.read_csv(parameter_path)
 if len(parameter_df) != config.n_samples:
 raise ValueError(
 " 断点续算时参数文件样本数与当前 sobol_power 不一致"
)
 else:
 parameter_df = generate_sobol_parameters(config)
 parameter_df.to_csv(
 parameter_path,
 index=False,
 encoding="utf-8-sig",
)

 coarse_df = run_coarse_scan(parameter_df, config, output)

 queue_df = select_refine_queue(coarse_df, config)
 queue_df.to_csv(
 output / "fes_step02_refine_queue.csv",

```

```

 index=False,
 encoding="utf-8-sig",
)

 verified_df = run_refinement(queue_df, config, output)
 ml_df = build_ml_dataset(coarse_df, verified_df, output)
 write_summaries(coarse_df, verified_df, ml_df, output)

 print("\nFinal phase counts:")
 print(ml_df["ml_primary_class"].value_counts(dropna=False).to_string())
 print("\nStep 02 completed.")

 return {
 "output_dir": output,
 "parameters": parameter_path,
 "coarse_results": output / "fes_step02_coarse_results.csv",
 "verified_results": output / "fes_step02_verified_results.csv",
 "ml_dataset": output / "fes_step02_ml_dataset.csv",
 "topological_candidates": output / "fes_step02_topological_candidates.csv",
 "summary": output / "fes_step02_summary.json",
 }

def parse_args() -> argparse.Namespace:
 parser = argparse.ArgumentParser(
 description="fes Step 02: five-dimensional Sobol topology scan"
)
 parser.add_argument("--output", default="outputs_fes6_step02_sobol")
 parser.add_argument("--sobol-power", type=int, default=12)
 parser.add_argument("--seed", type=int, default=20260712)
 parser.add_argument("--workers", type=int, default=max(1, min(8, os.cpu_count() or 1)))
 parser.add_argument("--resume", action="store_true")
 parser.add_argument("--overwrite", action="store_true")
 parser.add_argument("--test", action="store_true")
 return parser.parse_args()

def main() -> None:
 args = parse_args()
 if args.resume and args.overwrite:
 raise ValueError("--resume 与 --overwrite 不能同时使用")
 config = ScanConfig(
 output_dir=args.output,
 sobol_power=args.sobol_power,
 sobol_seed=args.seed,
 workers=args.workers,
 resume=args.resume,
 overwrite=args.overwrite,
 test_mode=args.test,
)
 run_pipeline(config)

if __name__ == "__main__":
 main()

```

## ▶ 代码清单 A.47.3 (python, 1008 行)

```

#!/usr/bin/env python3
-*- coding: utf-8 -*-
"""
FES Step 04: transformed-coordinate phase-boundary scan.

```

---

## Purpose

-----

Starting from the best strictly verified spin-Chern band-metal point obtained in Step 03, scan the transformed coordinates

$$\begin{aligned}t_s &= (t_1 + t_2) / 2, & t_d &= (t_1 - t_2) / 2, \\r_s &= (r_1 + r_2) / 2, & r_d &= (r_1 - r_2) / 2,\end{aligned}$$

and determine whether the indirect-gap boundary  $E_g^{\text{ind}} = 0$  overlaps a finite region with the spin-Chern phase  $C_{\text{up}} = -C_{\text{down}} = \pm 1$ .

The default workflow contains three complementary calculations:

1. a two-dimensional ( $m_e$ ,  $r_d$ ) plane around the best topological point;
2. a two-dimensional ( $r_s$ ,  $r_d$ ) plane around the same point;
3. a five-dimensional straight path from the best topological point to the nearest strictly audited positive-indirect-gap point from Step 03.

Coarse maps are followed by multi-grid, multi-shift strict verification of points close to either the indirect-gap boundary or the topological boundary. A nonzero spin Chern number is labeled only as a candidate; type-II QSH still requires later band-inversion, edge-state, and transport verification.

"""

```
from __future__ import annotations

import argparse
import hashlib
import json
import math
import os
import shutil
import sys
import zipfile
from concurrent.futures import ProcessPoolExecutor
from dataclasses import asdict, dataclass, field
from datetime import datetime, timezone
from pathlib import Path
from typing import Dict, Iterable, Sequence

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

try:
 import FES_step02_sobol_scan as core
except ImportError as exc:
 raise ImportError(
 "FES_step02_sobol_scan.py must be in the same directory or on PYTHONPATH."
) from exc

CODE_VERSION = "FES_STEP04_BOUNDARY_V1_20260713"
RESULT_SCHEMA_VERSION = "fes_step04_boundary_schema_v1"
HALF_COORDS = ("m_e", "t_s_half", "t_d_half", "r_s_half", "r_d_half")
REDUCED5 = ("m_e", "t1", "t2", "r1", "r2")

@dataclass
class BoundaryConfig:
 step03_input: str = "outputs_fes6_step03_local_gap"
 output_dir: str = "outputs_fes6_step04_boundary"
 workers: int = max(1, min(8, os.cpu_count() or 1))

 planes: tuple[str, ...] = ("me-rd", "rs-rd")
 grid_size: int = 61
 path_points: int = 301

 # Automatic ranges use max(user span, anchor separation + margin).
 me_span: float = 0.040
```

---

```

rs_span: float = 0.040
rd_span: float = 0.030
anchor_margin: float = 0.010

coarse_gap_nk: int = 21
coarse_chn_nk: int = 13
gap_tol: float = 1.0e-3
chn_tol: float = 0.08
min_det_tol: float = 1.0e-7

Strict verification settings.
verify_gap_grids: tuple[int, ...] = (51, 71)
verify_gap_shifts: tuple[tuple[float, float], ...] = (
 (0.0, 0.0),
 (0.5, 0.0),
 (0.0, 0.5),
 (0.5, 0.5),
)
verify_chn_grids: tuple[int, ...] = (21, 31, 41)
verify_chn_shifts: tuple[tuple[float, float], ...] = (
 (0.0, 0.0),
 (0.5, 0.5),
)

indirect_boundary_window: float = 0.012
direct_boundary_window: float = 0.012
max_strict_verify: int = 48

overwrite: bool = False
resume: bool = False
test_mode: bool = False

Coordinate transformations

def reduced_to_half_coords(reduced: Dict[str, float]) -> Dict[str, float]:
 p = {name: float(reduced[name]) for name in REDUCED5}
 return {
 "m_e": p["m_e"],
 "t_s_half": 0.5 * (p["t1"] + p["t2"]),
 "t_d_half": 0.5 * (p["t1"] - p["t2"]),
 "r_s_half": 0.5 * (p["r1"] + p["r2"]),
 "r_d_half": 0.5 * (p["r1"] - p["r2"]),
 }

def half_coords_to_reduced(coords: Dict[str, float]) -> Dict[str, float]:
 c = {name: float(coords[name]) for name in HALF_COORDS}
 return {
 "m_e": c["m_e"],
 "t1": c["t_s_half"] + c["t_d_half"],
 "t2": c["t_s_half"] - c["t_d_half"],
 "r1": c["r_s_half"] + c["r_d_half"],
 "r2": c["r_s_half"] - c["r_d_half"],
 }

def reduced_to_raw6(reduced: Dict[str, float]) -> Dict[str, float]:
 return core.raw6_from_reduced5(reduced, e0=0.0)

def in_default_bounds(reduced: Dict[str, float]) -> bool:
 return all(-1.0 <= float(reduced[name]) <= 1.0 for name in REDUCED5)

Step 03 input and anchor selection

```

---

```

def _find_result_root(root: Path) -> Path:
 direct = root / "fes_step03_strict_verified_results.csv"
 if direct.exists():
 return root
 matches = list(root.rglob("fes_step03_strict_verified_results.csv"))
 if len(matches) != 1:
 raise FileNotFoundError(
 "Could not uniquely locate fes_step03_strict_verified_results.csv "
 f"under {root}. Found {len(matches)} matches."
)
 return matches[0].parent

def locate_step03_directory(input_path: str | Path, output_dir: Path) -> Path:
 source = Path(input_path).expanduser().resolve()
 if not source.exists():
 raise FileNotFoundError(f"Step 03 input does not exist: {source}")
 if source.is_dir():
 return _find_result_root(source)
 if source.suffix.lower() != ".zip":
 raise ValueError("Step 03 input must be a directory or ZIP archive.")

 extract_dir = output_dir / "_step03_input_extracted"
 if extract_dir.exists():
 shutil.rmtree(extract_dir)
 extract_dir.mkdir(parents=True, exist_ok=True)
 with zipfile.ZipFile(source, "r") as zf:
 zf.extractall(extract_dir)
 return _find_result_root(extract_dir)

def _row_reduced(row: pd.Series) -> Dict[str, float]:
 return {name: float(row[name]) for name in REDUCED5}

def select_anchor_points(step03_dir: Path) -> tuple[pd.DataFrame, Dict[str, float], Dict[str, float]]:
 strict_path = step03_dir / "fes_step03_strict_verified_results.csv"
 strict_df = pd.read_csv(strict_path)
 if strict_df.empty:
 raise ValueError("Step 03 strict verification table is empty.")

 topo = strict_df[
 (strict_df["strict_phase_label"] == "spin_chern_band_metal")
 & (strict_df["verified_chern_reliable"].fillna(0).astype(int) == 1)
].copy()
 if topo.empty:
 raise ValueError("No strictly verified spin_chern_band_metal seed was found.")
 topo = topo.sort_values("verified_indirect_gap", ascending=False)
 topo_row = topo.iloc[0]
 topo_reduced = _row_reduced(topo_row)
 topo_half = reduced_to_half_coords(topo_reduced)

 positive = strict_df[
 (strict_df["verified_indirect_gap"] > 0.0)
 & (strict_df["verified_is_direct_gapped"].fillna(0).astype(int) == 1)
 & (strict_df["verified_is_balanced_spin_sector"].fillna(0).astype(int) == 1)
].copy()

 if positive.empty:
 # Fallback: use the non-topological point with the largest indirect gap.
 non_topo = strict_df[strict_df.index != topo_row.name].copy()
 if non_topo.empty:
 raise ValueError("No non-topological anchor is available.")
 positive = non_topo.sort_values("verified_indirect_gap", ascending=False).head(1)

 # Choose the nearest positive-gap point in transformed coordinates.
 distances = []

```



---

```

for idx, row in positive.iterrows():
 h = reduced_to_half_coords(_row_reduced(row))
 distance = math.sqrt(sum((h[name] - topo_half[name]) ** 2 for name in HALF_COORDS))
 distances.append((idx, distance))
positive_idx = min(distances, key=lambda item: item[1])[0]
positive_row = strict_df.loc[positive_idx]
positive_reduced = _row_reduced(positive_row)

anchor_rows = []
for role, row in (("best_topological", topo_row), ("positive_gap_anchor", positive_row)):
 reduced = _row_reduced(row)
 half = reduced_to_half_coords(reduced)
 anchor_rows.append(
 {
 "anchor_role": role,
 "candidate_id": str(row.get("candidate_id", "")),
 **reduced,
 **half,
 "strict_phase_label": str(row.get("strict_phase_label", "")),
 "verified_min_direct_gap": float(row.get("verified_min_direct_gap", np.nan)),
 "verified_indirect_gap": float(row.get("verified_indirect_gap", np.nan)),
 "verified_chern_up_int": row.get("verified_chern_up_int", np.nan),
 "verified_chern_down_int": row.get("verified_chern_down_int", np.nan),
 "verified_chern_reliable": int(row.get("verified_chern_reliable", 0)),
 }
)
return pd.DataFrame(anchor_rows), topo_reduced, positive_reduced

Point diagnostics

def special_point_diagnostics(raw6: Dict[str, float]) -> Dict[str, object]:
 points = {
 "gamma": (0.0, 0.0),
 "M": (math.pi, math.pi),
 "X": (math.pi, 0.0),
 "Y": (0.0, math.pi),
 }
 result: Dict[str, object] = {}
 for name, (kx, ky) in points.items():
 up = np.linalg.eigvalsh(core.h_spin_block_atomic(kx, ky, raw6, "up"))
 down = np.linalg.eigvalsh(core.h_spin_block_atomic(kx, ky, raw6, "down"))
 energies = np.sort(np.concatenate([up, down]))
 result[f"{name}_v4"] = float(energies[3])
 result[f"{name}_c5"] = float(energies[4])
 result[f"{name}_local_gap"] = float(energies[4] - energies[3])

 result["delta_v_M_minus_gamma"] = result["M_v4"] - result["gamma_v4"]
 result["delta_c_M_minus_gamma"] = result["M_c5"] - result["gamma_c5"]
 result["cross_gap_gammaV_to_Mc"] = result["M_c5"] - result["gamma_v4"]
 result["cross_gap_MV_to_gammaC"] = result["gamma_c5"] - result["M_v4"]
 result["gamma_M_indirect_proxy"] = min(
 result["cross_gap_gammaV_to_Mc"], result["cross_gap_MV_to_gammaC"]
)
 result["gamma_M_vbm_site"] = (
 "Gamma" if result["gamma_v4"] >= result["M_v4"] else "M"
)
 result["gamma_M_cbm_site"] = (
 "Gamma" if result["gamma_c5"] <= result["M_c5"] else "M"
)
 return result

def _empty_chern() -> Dict[str, object]:
 return {
 "chern_nk": np.nan,
 "chern_up": np.nan,

```

---

```

 "chern_down": np.nan,
 "chern_total_inferred": np.nan,
 "spin_chern": np.nan,
 "chern_up_int": np.nan,
 "chern_down_int": np.nan,
 "chern_total_int": np.nan,
 "min_det_up": np.nan,
 "min_det_down": np.nan,
 "chern_reliable": 0,
}

```

```

def scan_boundary_point(task: tuple) -> Dict[str, object]:
 (
 point_id,
 source_kind,
 source_name,
 i_index,
 j_index,
 path_lambda,
 half_coords,
 gap_nk,
 chern_nk,
 gap_tol,
 chern_tol,
 min_det_tol,
) = task

 half = {name: float(half_coords[name]) for name in HALF_COORDS}
 reduced = half_coords_to_reduced(half)
 raw6 = reduced_to_raw6(reduced)

 row: Dict[str, object] = {
 "point_id": str(point_id),
 "source_kind": str(source_kind),
 "source_name": str(source_name),
 "i_index": int(i_index),
 "j_index": int(j_index),
 "path_lambda": float(path_lambda),
 **half,
 **reduced,
 **core.derived_features(raw6),
 **_empty_chern(),
 "phase_label": "numeric_error",
 "is_spin_chern_topological": 0,
 "is_spin_chern_TI_candidate": 0,
 "error": "",
 }

 if not in_default_bounds(reduced):
 row["phase_label"] = "outside_parameter_bounds"
 return row

 try:
 gap = core.scan_band_gaps_shifted(
 raw6, int(gap_nk), (0.0, 0.0), float(gap_tol)
)
 row.update(gap)
 row.update(special_point_diagnostics(raw6))
 except Exception as exc:
 row["error"] = f"gap_failed: {exc!r}"
 return row

 if not int(row.get("is_direct_gapped", 0)):
 row["phase_label"] = "noninsulating_or_gap_closing"
 return row

 if not int(row.get("is_balanced_spin_sector", 0)):
 row["phase_label"] = "spin_sector_filling_mismatch"
 return row

```

---

```

try:
 chern = core.calculate_coarse_spin_chern(
 raw6,
 int(chern_nk),
 float(chern_tol),
 float(min_det_tol),
)
 row.update(chern)
except Exception as exc:
 row["phase_label"] = "chern_unreliable"
 row["error"] = f"chern_failed: {exc!r}"
 return row

label, candidate = core.classify_from_chern(row, row)
cu = row.get("chern_up_int")
cd = row.get("chern_down_int")
ct = row.get("chern_total_int")
spin_topo = bool(
 int(row.get("chern_reliable", 0))
 and pd.notna(cu)
 and pd.notna(cd)
 and pd.notna(ct)
 and int(cu) == -int(cd)
 and int(cu) != 0
 and int(ct) == 0
)
row["phase_label"] = label
row["is_spin_chern_topological"] = int(spin_topo)
row["is_spin_chern_TI_candidate"] = int(candidate)
return row

Point generation

def _auto_span(
 name: str,
 topo_half: Dict[str, float],
 positive_half: Dict[str, float],
 minimum_span: float,
 margin: float,
) -> float:
 return max(
 float(minimum_span),
 0.65 * abs(float(positive_half[name]) - float(topo_half[name])) + margin,
)

def plane_definition(name: str) -> tuple[str, str]:
 definitions = {
 "me-rd": ("m_e", "r_d_half"),
 "rs-rd": ("r_s_half", "r_d_half"),
 "me-rs": ("m_e", "r_s_half"),
 "ts-td": ("t_s_half", "t_d_half"),
 }
 if name not in definitions:
 raise ValueError(f"Unsupported plane {name!r}; choose from {sorted(definitions)}")
 return definitions[name]

def generate_plane_tasks(
 plane_name: str,
 topo_reduced: Dict[str, float],
 positive_reduced: Dict[str, float],
 config: BoundaryConfig,
) -> tuple[list[tuple], Dict[str, object]]:
 x_name, y_name = plane_definition(plane_name)

```

---

```

topo_half = reduced_to_half_coords(topo_reduced)
positive_half = reduced_to_half_coords(positive_reduced)

minimum_spans = {
 "m_e": config.me_span,
 "r_s_half": config.rs_span,
 "r_d_half": config.rd_span,
 "t_s_half": 0.04,
 "t_d_half": 0.04,
}
x_span = _auto_span(
 x_name, topo_half, positive_half, minimum_spans[x_name], config.anchor_margin
)
y_span = _auto_span(
 y_name, topo_half, positive_half, minimum_spans[y_name], config.anchor_margin
)

x_values = np.linspace(topo_half[x_name] - x_span, topo_half[x_name] + x_span, config.grid_size)
y_values = np.linspace(topo_half[y_name] - y_span, topo_half[y_name] + y_span, config.grid_size)

tasks: list[tuple] = []
for i, x_value in enumerate(x_values):
 for j, y_value in enumerate(y_values):
 coords = dict(topo_half)
 coords[x_name] = float(x_value)
 coords[y_name] = float(y_value)
 point_id = f"{plane_name}_{i:04d}_{j:04d}"
 tasks.append(
 (
 point_id,
 "plane",
 plane_name,
 i,
 j,
 np.nan,
 coords,
 config.coarse_gap_nk,
 config.coarse_chern_nk,
 config.gap_tol,
 config.chern_tol,
 config.min_det_tol,
)
)

metadata = {
 "plane_name": plane_name,
 "x_name": x_name,
 "y_name": y_name,
 "x_min": float(x_values.min()),
 "x_max": float(x_values.max()),
 "y_min": float(y_values.min()),
 "y_max": float(y_values.max()),
 "grid_size": int(config.grid_size),
 "x_span": float(x_span),
 "y_span": float(y_span),
}
return tasks, metadata

def generate_path_tasks(
 topo_reduced: Dict[str, float],
 positive_reduced: Dict[str, float],
 config: BoundaryConfig,
) -> list[tuple]:
 topo_half = reduced_to_half_coords(topo_reduced)
 positive_half = reduced_to_half_coords(positive_reduced)
 tasks: list[tuple] = []
 for index, lam in enumerate(np.linspace(0.0, 1.0, config.path_points)):
 coords = {

```

---

```

 name: (1.0 - lam) * topo_half[name] + lam * positive_half[name]
 for name in HALF_COORDS
}
tasks.append(
 (
 f"anchor_path_{index:05d}",
 "path",
 "topological_to_positive_gap",
 index,
 0,
 float(lam),
 coords,
 config.coarse_gap_nk,
 config.coarse_chern_nk,
 config.gap_tol,
 config.chern_tol,
 config.min_det_tol,
)
)
return tasks

Execution, boundary queue, and strict verification

def run_tasks(tasks: Sequence[tuple], workers: int) -> pd.DataFrame:
 if workers <= 1:
 rows = [scan_boundary_point(task) for task in tasks]
 else:
 with ProcessPoolExecutor(max_workers=workers) as pool:
 rows = list(pool.map(scan_boundary_point, tasks, chunksize=8))
 return pd.DataFrame(rows)

def _add_adjacent_transition_points(df: pd.DataFrame, selected_ids: set[str]) -> None:
 if df.empty or df["source_kind"].iloc[0] != "plane":
 return
 grid = df.set_index(["i_index", "j_index"])
 topo = df["is_spin_chern_topological"].fillna(0).astype(int)
 topo_by_index = dict(zip(zip(df["i_index"], df["j_index"]), topo))
 id_by_index = dict(zip(zip(df["i_index"], df["j_index"]), df["point_id"]))
 for (i, j), flag in topo_by_index.items():
 for neighbor in ((i + 1, j), (i, j + 1)):
 if neighbor in topo_by_index and topo_by_index[neighbor] != flag:
 selected_ids.add(str(id_by_index[(i, j)]))
 selected_ids.add(str(id_by_index[neighbor]))

def select_refine_queue(
 all_df: pd.DataFrame, config: BoundaryConfig
) -> pd.DataFrame:
 selected_ids: set[str] = set()

 # Always include any coarse TI candidate and the best topological points.
 selected_ids.update(
 all_df.loc[
 all_df["phase_label"] == "spin_chern_TI_candidate", "point_id"
].astype(str)
)
 topological = all_df[all_df["is_spin_chern_topological"] == 1].copy()
 if not topological.empty:
 selected_ids.update(
 topological.nlargest(12, "indirect_gap")["point_id"].astype(str)
)

 boundary_mask = (
 all_df["indirect_gap"].abs() <= config.indirect_boundary_window
) | (all_df["min_direct_gap"].abs() <= config.direct_boundary_window)

```

---

```

selected_ids.update(all_df.loc[boundary_mask, "point_id"].astype(str))

for _, group in all_df[all_df["source_kind"] == "plane"].groupby("source_name"):
 _add_adjacent_transition_points(group, selected_ids)

queue = all_df[all_df["point_id"].astype(str).isin(selected_ids)].copy()
if queue.empty:
 return queue

queue["rank_TI"] = (queue["phase_label"] == "spin_chern_TI_candidate").astype(int)
queue["rank_topological"] = queue["is_spin_chern_topological"].fillna(0).astype(int)
queue["rank_indirect_distance"] = queue["indirect_gap"].abs()
queue["rank_direct_distance"] = queue["min_direct_gap"].abs()
queue = queue.sort_values(
 [
 "rank_TI",
 "rank_topological",
 "rank_indirect_distance",
 "rank_direct_distance",
],
 ascending=[False, False, True, True],
)
queue = queue.drop_duplicates("point_id").head(config.max_strict_verify)
return queue

def strict_verify_one(row: Dict[str, object], config: BoundaryConfig, details_dir: Path) -> Dict[str, object]:
 reduced = {name: float(row[name]) for name in REDUCED5}
 raw6 = reduced_to_raw6(reduced)
 result = dict(row)
 result["strict_error"] = ""

 try:
 gap_summary, gap_runs = core.consensus_gap_audit(
 raw6,
 config.verify_gap_grids,
 config.verify_gap_shifts,
 config.gap_tol,
)
 result.update(gap_summary)
 gap_runs.to_csv(details_dir / f"{row['point_id']}_gap_consensus.csv", index=False)
 except Exception as exc:
 result["strict_phase_label"] = "strict_gap_failed"
 result["strict_error"] = repr(exc)
 return result

 if not int(result.get("verified_is_direct_gapped", 0)):
 result["strict_phase_label"] = "noninsulating_or_gap_closing"
 return result

 if not int(result.get("verified_is_balanced_spin_sector", 0)):
 result["strict_phase_label"] = "spin_sector_filling_mismatch"
 return result

 try:
 chern_summary, chern_runs = core.consensus_chern_audit(
 raw6,
 config.verify_chern_grids,
 config.verify_chern_shifts,
 config.chern_tol,
 config.min_det_tol,
 include_total=True,
)
 result.update(chern_summary)
 chern_runs.to_csv(details_dir / f"{row['point_id']}_chern_consensus.csv", index=False)
 except Exception as exc:
 result["strict_phase_label"] = "chern_unreliable"
 result["strict_error"] = repr(exc)
 return result

```

---

```

 verified_label, _ = core.verified_phase_label(result)
 result["strict_phase_label"] = verified_label.removeprefix("verified_")
 return result

def run_strict_verification(
 queue: pd.DataFrame,
 config: BoundaryConfig,
 output_dir: Path,
) -> pd.DataFrame:
 if queue.empty:
 return pd.DataFrame()
 details_dir = output_dir / "strict_details"
 details_dir.mkdir(parents=True, exist_ok=True)
 rows = [strict_verify_one(row.to_dict(), config, details_dir) for _, row in queue.iterrows()]
 return pd.DataFrame(rows)

Boundary diagnostics and plots

def estimate_zero_crossings(x: np.ndarray, y: np.ndarray) -> list[float]:
 crossings: list[float] = []
 for i in range(len(x) - 1):
 y0, y1 = float(y[i]), float(y[i + 1])
 if not np.isfinite(y0) or not np.isfinite(y1):
 continue
 if y0 == 0.0:
 crossings.append(float(x[i]))
 elif y0 * y1 < 0.0:
 fraction = -y0 / (y1 - y0)
 crossings.append(float(x[i] + fraction * (x[i + 1] - x[i])))
 return crossings

def path_boundary_summary(path_df: pd.DataFrame) -> Dict[str, object]:
 if path_df.empty:
 return {}
 path = path_df.sort_values("path_lambda")
 lam = path["path_lambda"].to_numpy(float)
 indirect = path["indirect_gap"].to_numpy(float)
 direct = path["min_direct_gap"].to_numpy(float)
 topo = path["is_spin_chern_topological"].fillna(0).astype(int).to_numpy()

 topo_transitions = []
 for i in range(len(lam) - 1):
 if topo[i] != topo[i + 1]:
 topo_transitions.append(float(0.5 * (lam[i] + lam[i + 1])))

 ti_mask = (topo == 1) & (indirect > 1.0e-3) & (direct > 1.0e-3)
 topo_mask = topo == 1
 return {
 "path_indirect_zero_lambdas": estimate_zero_crossings(lam, indirect),
 "path_direct_zero_lambdas": estimate_zero_crossings(lam, direct),
 "path_topology_transition_lambdas": topo_transitions,
 "path_n_coarse_topological": int(topo_mask.sum()),
 "path_n_coarse_TI_candidates": int(ti_mask.sum()),
 "path_best_indirect_gap_in_topological_sector": (
 float(np.max(indirect[topo_mask])) if topo_mask.any() else None
),
 }

def write_slice_diagnostics(df: pd.DataFrame, plane_meta: Dict[str, object], path: Path) -> None:
 x_name = str(plane_meta["x_name"])
 rows = []
 for x_value, group in df.groupby(x_name):
 topo = group[group["is_spin_chern_topological"] == 1]

```

```

rows.append(
 {
 x_name: float(x_value),
 "n_points": int(len(group)),
 "n_topological": int(len(topo)),
 "n_TI_candidates": int((group["phase_label"] == "spin_chern_TI_candidate").sum()),
 "max_indirect_gap_all": float(group["indirect_gap"].max()),
 "max_indirect_gap_topological": (
 float(topo["indirect_gap"].max()) if not topo.empty else np.nan
),
 "min_direct_gap_topological": (
 float(topo["min_direct_gap"].min()) if not topo.empty else np.nan
),
 }
)
pd.DataFrame(rows).sort_values(x_name).to_csv(path, index=False)

```

```

def plot_plane_indirect(df: pd.DataFrame, meta: Dict[str, object], output_path: Path) -> None:
 x_name = str(meta["x_name"])
 y_name = str(meta["y_name"])
 pivot = df.pivot(index=y_name, columns=x_name, values="indirect_gap").sort_index().sort_index(axis=1)
 x = pivot.columns.to_numpy(float)
 y = pivot.index.to_numpy(float)
 z = pivot.to_numpy(float)

 fig, ax = plt.subplots(figsize=(7.2, 6.0))
 mesh = ax.pcolormesh(x, y, z, shading="auto")
 fig.colorbar(mesh, ax=ax, label=r"E_g^{ind}")
 if np.nanmin(z) <= 0.0 <= np.nanmax(z):
 ax.contour(x, y, z, levels=[0.0])
 topo_pivot = df.pivot(index=y_name, columns=x_name,
 ↪ values="is_spin_chern_topological").sort_index().sort_index(axis=1)
 topo_z = topo_pivot.to_numpy(float)
 if np.nanmin(topo_z) < 0.5 < np.nanmax(topo_z):
 ax.contour(x, y, topo_z, levels=[0.5])
 ax.set_xlabel(x_name)
 ax.set_ylabel(y_name)
 ax.set_title(f"{meta['plane_name']}: indirect gap; contours = gap zero and topology boundary")
 fig.tight_layout()
 fig.savefig(output_path, dpi=220)
 plt.close(fig)

```

```

def plot_plane_phase(df: pd.DataFrame, meta: Dict[str, object], output_path: Path) -> None:
 x_name = str(meta["x_name"])
 y_name = str(meta["y_name"])
 phase_code = {
 "noninsulating_or_gap_closing": 0,
 "spin_sector_filling_mismatch": 1,
 "chern_unreliable": 2,
 "indirect_overlap": 3,
 "trivial_insulator": 4,
 "spin_chern_band_metal": 5,
 "spin_chern_TI_candidate": 6,
 "other_gapped_phase": 7,
 "Chern_insulator": 8,
 "outside_parameter_bounds": -1,
 "numeric_error": -2,
 }
 work = df.copy()
 work["phase_code"] = work["phase_label"].map(phase_code).fillna(-3)
 pivot = work.pivot(index=y_name, columns=x_name, values="phase_code").sort_index().sort_index(axis=1)
 x = pivot.columns.to_numpy(float)
 y = pivot.index.to_numpy(float)
 z = pivot.to_numpy(float)

 fig, ax = plt.subplots(figsize=(7.2, 6.0))
 mesh = ax.pcolormesh(x, y, z, shading="auto")

```



---

```

fig.colorbar(mesh, ax=ax, label="phase code")
ax.set_xlabel(x_name)
ax.set_ylabel(y_name)
ax.set_title(f"{meta['plane_name']}: coarse phase map")
fig.tight_layout()
fig.savefig(output_path, dpi=220)
plt.close(fig)

def plot_path(path_df: pd.DataFrame, output_path: Path) -> None:
 path = path_df.sort_values("path_lambda")
 fig, ax = plt.subplots(figsize=(8.0, 5.4))
 ax.plot(path["path_lambda"], path["indirect_gap"], label=r"E_g^{ind}")
 ax.plot(path["path_lambda"], path["min_direct_gap"], label=r"E_g^{dir}")
 ax.axhline(0.0, linewidth=1.0)
 topo = path[path["is_spin_chern_topological"] == 1]
 if not topo.empty:
 ax.scatter(topo["path_lambda"], topo["indirect_gap"], s=14, label="spin-Chern topological")
 ax.set_xlabel(r"path coordinate λ")
 ax.set_ylabel("gap")
 ax.set_title("Path from best topological point to positive-gap anchor")
 ax.legend()
 fig.tight_layout()
 fig.savefig(output_path, dpi=220)
 plt.close(fig)

Pipeline and command line

def prepare_output(config: BoundaryConfig) -> Path:
 output = Path(config.output_dir).expanduser().resolve()
 if output.exists() and config.overwrite:
 shutil.rmtree(output)
 if output.exists() and any(output.iterdir()) and not config.resume:
 raise FileExistsError(
 f"Output directory is not empty: {output}. Use --overwrite or --resume."
)
 output.mkdir(parents=True, exist_ok=True)
 return output

def script_sha256() -> str | None:
 try:
 return hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
 except Exception:
 return None

def run_pipeline(config: BoundaryConfig) -> Dict[str, Path]:
 output = prepare_output(config)
 step03_dir = locate_step03_directory(config.step03_input, output)
 anchors_df, topo_reduced, positive_reduced = select_anchor_points(step03_dir)
 anchors_path = output / "fes_step04_anchor_points.csv"
 anchors_df.to_csv(anchors_path, index=False)

 metadata = {
 "code_version": CODE_VERSION,
 "result_schema_version": RESULT_SCHEMA_VERSION,
 "run_time_utc": datetime.now(timezone.utc).isoformat(),
 "script_sha256": script_sha256(),
 "step02_core_file": str(Path(core.__file__).resolve()),
 "step02_core_sha256": hashlib.sha256(Path(core.__file__).read_bytes()).hexdigest(),
 "step03_result_directory": str(step03_dir.resolve()),
 "config": asdict(config),
 }

 metadata_path = output / "fes_step04_run_metadata.json"
 metadata_path.write_text(json.dumps(metadata, ensure_ascii=False, indent=2), encoding="utf-8")

```

---

```

all_frames: list[pd.DataFrame] = []
plane_meta_rows: list[Dict[str, object]] = []
output_paths: Dict[str, Path] = {
 "anchors": anchors_path,
 "metadata": metadata_path,
}

for plane_name in config.planes:
 tasks, meta = generate_plane_tasks(
 plane_name, topo_reduced, positive_reduced, config
)
 plane_path = output / f"fes_step04_{plane_name}_scan.csv"
 if config.resume and plane_path.exists():
 plane_df = pd.read_csv(plane_path)
 else:
 plane_df = run_tasks(tasks, config.workers)
 plane_df.to_csv(plane_path, index=False)
 all_frames.append(plane_df)
 plane_meta_rows.append(meta)
 output_paths[f"plane_{plane_name}"] = plane_path

 plot_plane_indirect(
 plane_df,
 meta,
 output / f"fes_step04_{plane_name}_indirect_gap_map.png",
)
 plot_plane_phase(
 plane_df,
 meta,
 output / f"fes_step04_{plane_name}_phase_map.png",
)
 write_slice_diagnostics(
 plane_df,
 meta,
 output / f"fes_step04_{plane_name}_slice_diagnostics.csv",
)

plane_meta_path = output / "fes_step04_plane_metadata.csv"
pd.DataFrame(plane_meta_rows).to_csv(plane_meta_path, index=False)
output_paths["plane_metadata"] = plane_meta_path

path_tasks = generate_path_tasks(topo_reduced, positive_reduced, config)
path_scan_path = output / "fes_step04_anchor_path_scan.csv"
if config.resume and path_scan_path.exists():
 path_df = pd.read_csv(path_scan_path)
else:
 path_df = run_tasks(path_tasks, config.workers)
 path_df.to_csv(path_scan_path, index=False)
all_frames.append(path_df)
plot_path(path_df, output / "fes_step04_anchor_path_gaps.png")
output_paths["path_scan"] = path_scan_path

all_df = pd.concat(all_frames, ignore_index=True)
all_scan_path = output / "fes_step04_all_coarse_scan.csv"
all_df.to_csv(all_scan_path, index=False)
output_paths["all_coarse"] = all_scan_path

queue = select_refine_queue(all_df, config)
queue_path = output / "fes_step04_strict_refine_queue.csv"
queue.to_csv(queue_path, index=False)
output_paths["refine_queue"] = queue_path

strict_df = run_strict_verification(queue, config, output)
strict_path = output / "fes_step04_strict_verified_results.csv"
strict_df.to_csv(strict_path, index=False)
output_paths["strict_results"] = strict_path

if strict_df.empty:

```

---

```

 ti_df = strict_df.copy()
 topo_df = strict_df.copy()
else:
 ti_df = strict_df[
 strict_df["strict_phase_label"] == "spin_chern_TI_candidate"
].copy()
 topo_df = strict_df[
 strict_df["strict_phase_label"].isin(
 ["spin_chern_TI_candidate", "spin_chern_band_metal"]
)
].copy()

ti_path = output / "fes_step04_spin_chern_TI_candidates.csv"
topo_path = output / "fes_step04_verified_topological_points.csv"
ti_df.to_csv(ti_path, index=False)
topo_df.to_csv(topo_path, index=False)
output_paths["ti_candidates"] = ti_path
output_paths["verified_topological"] = topo_path

summary: Dict[str, object] = {
 "n_plane_points": int(sum(len(frame) for frame in all_frames[:-1])),
 "n_path_points": int(len(path_df)),
 "n_coarse_points_total": int(len(all_df)),
 "coarse_phase_counts": all_df["phase_label"].value_counts().to_dict(),
 "n_strict_refine_queue": int(len(queue)),
 "n_strict_verified": int(len(strict_df)),
 "strict_phase_counts": (
 strict_df["strict_phase_label"].value_counts().to_dict()
 if not strict_df.empty
 else {}
),
 "n_strict_spin_chern_TI_candidates": int(len(ti_df)),
 "best_strict_topological_indirect_gap": (
 float(topo_df["verified_indirect_gap"].max()) if not topo_df.empty else None
),
 "path_boundary_analysis": path_boundary_summary(path_df),
}
summary_path = output / "fes_step04_boundary_summary.json"
summary_path.write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")
output_paths["summary"] = summary_path

print(json.dumps(summary, ensure_ascii=False, indent=2))
return output_paths

def parse_args() -> argparse.Namespace:
 parser = argparse.ArgumentParser(description=__doc__)
 parser.add_argument("--step03-input", default="outputs_fes6_step03_local_gap")
 parser.add_argument("--output-dir", default="outputs_fes6_step04_boundary")
 parser.add_argument("--workers", type=int, default=max(1, min(8, os.cpu_count() or 1)))
 parser.add_argument("--grid-size", type=int, default=61)
 parser.add_argument("--path-points", type=int, default=301)
 parser.add_argument(
 "--planes",
 default="me-rd,rs-rd",
 help="Comma-separated planes: me-rd, rs-rd, me-rs, ts-td",
)
 parser.add_argument("--max-strict-verify", type=int, default=48)
 parser.add_argument("--overwrite", action="store_true")
 parser.add_argument("--resume", action="store_true")
 parser.add_argument("--test", action="store_true")
 return parser.parse_args()

def main() -> None:
 args = parse_args()
 if args.overwrite and args.resume:
 raise ValueError("--overwrite and --resume cannot be used together.")

```

```

if args.test:
 config = BoundaryConfig(
 step03_input=args.step03_input,
 output_dir=args.output_dir,
 workers=min(args.workers, 2),
 planes=("me-rd",),
 grid_size=9,
 path_points=31,
 coarse_gap_nk=13,
 coarse_chern_nk=9,
 verify_gap_grids=(17, 21),
 verify_gap_shifts=((0.0, 0.0), (0.5, 0.5)),
 verify_chern_grids=(11, 15),
 verify_chern_shifts=((0.0, 0.0),),
 max_strict_verify=min(args.max_strict_verify, 8),
 overwrite=args.overwrite,
 resume=args.resume,
 test_mode=True,
)
else:
 planes = tuple(item.strip() for item in args.planes.split(",") if item.strip())
 config = BoundaryConfig(
 step03_input=args.step03_input,
 output_dir=args.output_dir,
 workers=max(1, args.workers),
 planes=planes,
 grid_size=max(9, args.grid_size),
 path_points=max(31, args.path_points),
 max_strict_verify=max(1, args.max_strict_verify),
 overwrite=args.overwrite,
 resume=args.resume,
)

run_pipeline(config)

if __name__ == "__main__":
 main()

```

## ► 代码清单 A.47.4 (python, 937 行)

```

#!/usr/bin/env python3
-*- coding: utf-8 -*-
"""
FES Step 05 — Γ /M 解析本征值、branch-switching 与 no-go 条件审计

```

目标

----

1. 对 fes 六参数八带模型的单自旋  $4 \times 4$  块，在  $\Gamma$  与 M 点解析因式分解；
2. 给出四条解析本征值，并验证其与数值 Hamiltonian 完全一致；
3. 检测半填充时  $\Gamma$ /M 之间的 B+ 分支 occupied/unoccupied 交换；
4. 计算严格上界

$$\begin{aligned}
 E_{g^{\text{ind}}} &\leq -|E_{\{\Gamma, B+\}} - E_{\{M, B+\}}| \\
 &= -4 |r_1^2 - r_2^2| / (B_1 + B_2) \leq 0,
 \end{aligned}$$

其中  $B_i = \sqrt{(m_e + t_d)^2 + 4 r_i^2}$ ;

5. 审计 Step 04 的所有严格拓扑点和所有粗筛拓扑点；
6. 补充严格复核 Step 04 因队列上限未覆盖的粗筛 TI 候选；
7. 沿 Step 04 两个锚点之间的路径，解析定位  $\Gamma$ /M 直接闭隙与  $r_1^2 = r_2^2$  (间接带隙 no-go 上界为零) 的位置顺序。

本步骤用于建立 “最小六参数 fes 模型中是否存在 type-II QSH 绝缘窗口”

的解析证据。它不会仅凭有限样本自动宣称数学定理；输出会明确区分：

- `exact_identity`: 解析恒等式；
- `verified_numerical_evidence`: 严格数值证据；
- `conditional_no_go`: 在 B+ partner-switching 分支排序下的严格 no-go。

运行示例

-----

快速测试:

```
python FES_step05_analytic_no_go.py \
 --step04-input outputs_fes6_step04_boundary.zip \
 --output-dir outputs_fes6_step05_no_go \
 --test --overwrite
```

正式运行:

```
python FES_step05_analytic_no_go.py \
 --step04-input outputs_fes6_step04_boundary.zip \
 --output-dir outputs_fes6_step05_no_go \
 --workers 4 --overwrite
```

"""

```
from __future__ import annotations
```

```
from dataclasses import asdict, dataclass
from pathlib import Path
from datetime import datetime, timezone
from typing import Dict, Iterable, Sequence
import argparse
import hashlib
import json
import math
import os
import shutil
import sys
import zipfile
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import brentq
import sympy as sp
```

```
The matching Step 02 core must be in the same directory or importable.
import FES_step02_sobol_scan as core
```

```
np.set_printoptions(precision=12, suppress=True)
```

```
CODE_VERSION = "FES_STEP05_ANALYTIC_NOGO_V1_20260713"
RESULT_SCHEMA_VERSION = "fes_step05_analytic_nogo_schema_v1"
```

```
PARAMS5 = ("m_e", "t1", "t2", "r1", "r2")
BRANCHES = ("G_A_minus", "G_A_plus", "G_B_minus", "G_B_plus",
 "M_A_minus", "M_A_plus", "M_B_minus", "M_B_plus")
```

```
@dataclass
```

```
class NoGoConfig:
 step04_input: str = "outputs_fes6_step04_boundary.zip"
 output_dir: str = "outputs_fes6_step05_no_go"
 workers: int = max(1, min(4, os.cpu_count() or 1))

 validation_samples: int = 128
 validation_seed: int = 20260713
 dense_path_points: int = 5001

 gap_tol: float = 1.0e-3
 chern_tol: float = 0.08
 min_det_tol: float = 1.0e-7
 verify_gap_grids: tuple[int, ...] = (51, 71)
```

---

```

verify_gap_shifts: tuple[tuple[float, float], ...] = (
 (0.0, 0.0),
 (0.5, 0.0),
 (0.0, 0.5),
 (0.5, 0.5),
)
verify_chern_grids: tuple[int, ...] = (21, 31, 41)
verify_chern_shifts: tuple[tuple[float, float], ...] = (
 (0.0, 0.0),
 (0.5, 0.5),
)

reverify_all_coarse_ti: bool = False
overwrite: bool = False
resume: bool = False
test_mode: bool = False

=====
1. 输入、输出与版本管理
=====

def script_sha256() -> str | None:
 if "__file__" not in globals():
 return None
 path = Path(__file__).resolve()
 return hashlib.sha256(path.read_bytes()).hexdigest() if path.exists() else None

def prepare_output(config: NoGoConfig) -> Path:
 out = Path(config.output_dir).resolve()
 if out.exists() and config.overwrite:
 shutil.rmtree(out)
 out.mkdir(parents=True, exist_ok=True)
 (out / "strict_details").mkdir(exist_ok=True)
 return out

def _find_result_root(root: Path) -> Path:
 required = {
 "fes_step04_all_coarse_scan.csv",
 "fes_step04_strict_verified_results.csv",
 "fes_step04_anchor_points.csv",
 }
 if required.issubset({p.name for p in root.iterdir() if p.is_file()}):
 return root
 candidates = []
 for path in root.rglob("fes_step04_all_coarse_scan.csv"):
 parent = path.parent
 if required.issubset({p.name for p in parent.iterdir() if p.is_file()}):
 candidates.append(parent)
 if not candidates:
 raise FileNotFoundError(
 f" 无法在 {root} 中找到完整 Step 04 结果目录; 必须包含 {sorted(required)}"
)
 return sorted(candidates, key=lambda p: len(p.parts))[0]

def locate_step04_directory(input_path: str | Path, output_dir: Path) -> Path:
 path = Path(input_path).expanduser().resolve()
 if not path.exists():
 raise FileNotFoundError(path)
 if path.is_dir():
 return _find_result_root(path)
 if path.suffix.lower() != ".zip":
 raise ValueError("--step04-input 必须是目录或 ZIP 文件")
 extract_root = output_dir / "_step04_extracted"
 if extract_root.exists():

```

---

```

 shutil.rmtree(extract_root)
 extract_root.mkdir(parents=True)
 with zipfile.ZipFile(path, "r") as zf:
 zf.extractall(extract_root)
 return _find_result_root(extract_root)

def write_metadata(config: NoGoConfig, output: Path, step04_dir: Path) -> None:
 metadata = {
 "code_version": CODE_VERSION,
 "result_schema_version": RESULT_SCHEMA_VERSION,
 "run_time_utc": datetime.now(timezone.utc).isoformat(),
 "script_sha256": script_sha256(),
 "step02_core_version": getattr(core, "CODE_VERSION", None),
 "step02_core_sha256": hashlib.sha256(Path(core.__file__).read_bytes()).hexdigest(),
 "step04_directory": str(step04_dir),
 "config": asdict(config),
 }
 (output / "fes_step05_run_metadata.json").write_text(
 json.dumps(metadata, ensure_ascii=False, indent=2), encoding="utf-8"
)

=====
2. Γ/M 解析本征值与特征多项式
=====

def analytic_branches(reduced: Dict[str, float]) -> Dict[str, float]:
 """Return exact closed-form Γ/M eigenvalue branches for one spin block."""
 m = float(reduced["m_e"])
 t1 = float(reduced["t1"])
 t2 = float(reduced["t2"])
 r1 = float(reduced["r1"])
 r2 = float(reduced["r2"])

 t_s = 0.5 * (t1 + t2)
 t_d = 0.5 * (t1 - t2)

 A1 = math.sqrt((m - t_d) ** 2 + 4.0 * r1 * r1)
 A2 = math.sqrt((m - t_d) ** 2 + 4.0 * r2 * r2)
 B1 = math.sqrt((m + t_d) ** 2 + 4.0 * r1 * r1)
 B2 = math.sqrt((m + t_d) ** 2 + 4.0 * r2 * r2)

 values = {
 "t_s_half": t_s,
 "t_d_half": t_d,
 "A1": A1,
 "A2": A2,
 "B1": B1,
 "B2": B2,
 "G_A_minus": -t_s - A1,
 "G_A_plus": -t_s + A1,
 "G_B_minus": +t_s - B2,
 "G_B_plus": +t_s + B2,
 "M_A_minus": -t_s - A2,
 "M_A_plus": -t_s + A2,
 "M_B_minus": +t_s - B1,
 "M_B_plus": +t_s + B1,
 }
 return values

def _ordered_special(branch: Dict[str, float], prefix: str) -> list[tuple[str, float]]:
 keys = [name for name in BRANCHES if name.startswith(prefix + "_")]
 return sorted(((name, float(branch[name])) for name in keys), key=lambda item: item[1])

def analytic_diagnostics(reduced: Dict[str, float]) -> Dict[str, object]:

```

---

```

b = analytic_branches(reduced)
G = _ordered_special(b, "G")
M = _ordered_special(b, "M")

G_rank = {name: rank for rank, (name, _) in enumerate(G)}
M_rank = {name: rank for rank, (name, _) in enumerate(M)}

gamma_v_name, gamma_v = G[1]
gamma_c_name, gamma_c = G[2]
M_v_name, M_v = M[1]
M_c_name, M_c = M[2]

gamma_gap = gamma_c - gamma_v
M_gap = M_c - M_v
gamma_M_gap = min(gamma_c, M_c) - max(gamma_v, M_v)

Signed Gamma/M inversion masses for the A- and B+ branches.
mass_Gamma > 0 means B+ lies below A- at Gamma.
mass_M > 0 means A- lies below B+ at M.
mass_Gamma = b["G_A_minus"] - b["G_B_plus"]
mass_M = b["M_B_plus"] - b["M_A_minus"]
mass_product = mass_Gamma * mass_M

The B+ branch is the branch that empirically switches occupancy in all
verified topological points of Steps 02-04.
bplus_G_rank = G_rank["G_B_plus"]
bplus_M_rank = M_rank["M_B_plus"]
bplus_partner_switch = {bplus_G_rank, bplus_M_rank} == {1, 2}

delta_Bplus = b["M_B_plus"] - b["G_B_plus"] # B1 - B2
denominator = b["B1"] + b["B2"]
rationalized_delta = (
 4.0 * (float(reduced["r1"]) ** 2 - float(reduced["r2"]) ** 2) / denominator
 if denominator > 0.0 else 0.0
)
no_go_bound = -abs(delta_Bplus) if bplus_partner_switch else float("nan")

active_pattern = (
 f"G:{gamma_v_name}->{gamma_c_name};M:{M_v_name}->{M_c_name}"
)

result: Dict[str, object] = dict(b)
result.update(
 {
 "G_e1": G[0][1], "G_e2": G[1][1], "G_e3": G[2][1], "G_e4": G[3][1],
 "M_e1": M[0][1], "M_e2": M[1][1], "M_e3": M[2][1], "M_e4": M[3][1],
 "gamma_valence_branch": gamma_v_name,
 "gamma_conduction_branch": gamma_c_name,
 "M_valence_branch": M_v_name,
 "M_conduction_branch": M_c_name,
 "gamma_local_gap_analytic": gamma_gap,
 "M_local_gap_analytic": M_gap,
 "gamma_M_indirect_analytic": gamma_M_gap,
 "mass_Gamma_signed": mass_Gamma,
 "mass_M_signed": mass_M,
 "mass_product": mass_product,
 "mass_same_sign": int(mass_product > 0.0),
 "mass_boundary_distance": min(abs(mass_Gamma), abs(mass_M)),
 "active_branch_pattern": active_pattern,
 "G_Bplus_rank": bplus_G_rank,
 "M_Bplus_rank": bplus_M_rank,
 "Bplus_partner_switch": int(bplus_partner_switch),
 "delta_Bplus_M_minus_G": delta_Bplus,
 "delta_Bplus_rationalized": rationalized_delta,
 "Bplus_identity_error": delta_Bplus - rationalized_delta,
 "conditional_no_go_bound": no_go_bound,
 "r_square_difference": float(reduced["r1"]) ** 2 - float(reduced["r2"]) ** 2,
 "r_square_equal": int(abs(float(reduced["r1"]) ** 2 - float(reduced["r2"]) ** 2) < 1e-12),
 }
)

```



---

```

)
return result

def symbolic_derivation() -> Dict[str, str]:
 """Derive and return symbolic characteristic polynomials and formulas."""
 lam, m, t1, t2, r1, r2 = sp.symbols("lambda m_e t_1 t_2 r_1 r_2", real=True)
 ts = (t1 + t2) / 2
 td = (t1 - t2) / 2

 A1 = sp.sqrt((m - td) ** 2 + 4 * r1 ** 2)
 A2 = sp.sqrt((m - td) ** 2 + 4 * r2 ** 2)
 B1 = sp.sqrt((m + td) ** 2 + 4 * r1 ** 2)
 B2 = sp.sqrt((m + td) ** 2 + 4 * r2 ** 2)

 # Factorized forms derived from the 4x4 spin-up matrices.
 pG = sp.expand(((lam + ts) ** 2 - A1 ** 2) * ((lam - ts) ** 2 - B2 ** 2))
 pM = sp.expand(((lam + ts) ** 2 - A2 ** 2) * ((lam - ts) ** 2 - B1 ** 2))

 delta_B = sp.simplify(B1 - B2)
 delta_B_rational = sp.simplify(4 * (r1 ** 2 - r2 ** 2) / (B1 + B2))
 identity = sp.simplify(delta_B - delta_B_rational)

 formulas = {
 "p_Gamma_factorized": sp.sstr(sp.factor(pG)),
 "p_M_factorized": sp.sstr(sp.factor(pM)),
 "E_Gamma_A_pm": sp.sstr(-ts) + " ± " + sp.sstr(A1),
 "E_Gamma_B_pm": sp.sstr(ts) + " ± " + sp.sstr(B2),
 "E_M_A_pm": sp.sstr(-ts) + " ± " + sp.sstr(A2),
 "E_M_B_pm": sp.sstr(ts) + " ± " + sp.sstr(B1),
 "delta_Bplus": sp.sstr(delta_B),
 "delta_Bplus_rationalized": sp.sstr(delta_B_rational),
 "delta_identity_simplified": sp.sstr(identity),
 "conditional_no_go": (
 "If E_{Gamma,B+} and E_{M,B+} exchange occupied/unoccupied rank at half filling, "
 "then E_g^ind <= -|B1-B2| = -4|r1^2-r2^2|/(B1+B2) <= 0."
),
 }
 return formulas

def write_symbolic_outputs(output: Path) -> Dict[str, str]:
 formulas = symbolic_derivation()
 lines = [
 "fes six-parameter model: Gamma/M exact formulas",
 "==" * 72,
 "Definitions:",
 " t_s=(t1+t2)/2, t_d=(t1-t2)/2",
 " A_i=sqrt((m_e-t_d)^2+4 r_i^2)",
 " B_i=sqrt((m_e+t_d)^2+4 r_i^2)",
 "",
 "Gamma eigenvalues:",
 " E_Gamma,A± = -t_s ± A1",
 " E_Gamma,B± = +t_s ± B2",
 "M eigenvalues:",
 " E_M,A± = -t_s ± A2",
 " E_M,B± = +t_s ± B1",
 "",
 "Exact identity:",
 " E_M,B+ - E_Gamma,B+ = B1-B2",
 " B1-B2 = 4(r1^2-r2^2)/(B1+B2)",
 "",
 "Conditional no-go:",
 " If B+ changes occupied/unoccupied rank between Gamma and M,",
 " E_g^ind <= -|B1-B2|",
 " = -4|r1^2-r2^2|/(B1+B2) <= 0.",
 " It is strictly negative when r1^2 != r2^2.",
 "",
 "SymPy output:",
]

```

```

]
lines.extend(f" {key}: {value}" for key, value in formulas.items())
(output / "fes_step05_symbolic_formulas.txt").write_text(
 "\n".join(lines), encoding="utf-8"
)

latex = r"""\documentclass{article}
\usepackage{amsmath}
\begin{document}
\section*{fes Γ/M analytic spectrum and conditional no-go}
Define
\[
t_s=\frac{t_1+t_2}{2},\quad t_d=\frac{t_1-t_2}{2},
\]
\[
A_i=\sqrt{(m_e-t_d)^2+4r_i^2},\quad
B_i=\sqrt{(m_e+t_d)^2+4r_i^2}.
\]
The one-spin eigenvalues are
\[
E_{\Gamma,A}^{\pm}=-t_s\pm A_1,\quad
E_{\Gamma,B}^{\pm}=t_s\pm B_2,
\]
\[
E_{M,A}^{\pm}=-t_s\pm A_2,\quad
E_{M,B}^{\pm}=t_s\pm B_1.
\]
The B^+ branch difference obeys the exact identity
\[
E_{M,B^+}-E_{\Gamma,B^+}=B_1-B_2
=\frac{4(r_1^2-r_2^2)}{B_1+B_2}.
\]
If B^+ changes from occupied to unoccupied (or vice versa) between
Γ and M at half filling, the global indirect gap satisfies
\[
E_g^{\mathrm{ind}}\leq -\left|B_1-B_2\right|
=-\frac{4\left|r_1^2-r_2^2\right|}{B_1+B_2}\leq 0.
\]
For $r_1^2\neq r_2^2$ the upper bound is strictly negative.
\end{document}
"""

(output / "fes_step05_symbolic_formulas.tex").write_text(latex, encoding="utf-8")
(output / "fes_step05_symbolic_formulas.json").write_text(
 json.dumps(formulas, ensure_ascii=False, indent=2), encoding="utf-8"
)
return formulas

```

```

=====
3. 解析公式数值验证
=====

```

```

def _row_reduced(row: pd.Series | Dict[str, object]) -> Dict[str, float]:
 return {name: float(row[name]) for name in PARAMS5}

```

```

def validate_analytic_spectrum(
 source_df: pd.DataFrame,
 n_samples: int,
 seed: int,
) -> pd.DataFrame:
 if len(source_df) == 0:
 return pd.DataFrame()
 rng = np.random.default_rng(seed)
 n = min(int(n_samples), len(source_df))
 indices = rng.choice(len(source_df), size=n, replace=False)
 rows = []
 for idx in indices:

```

```

row = source_df.iloc[int(idx)]
reduced = _row_reduced(row)
raw6 = core.raw6_from_reduced5(reduced)
analytic = analytic_branches(reduced)

H_G = core.h_fes_atomic(0.0, 0.0, raw6)
H_M = core.h_fes_atomic(math.pi, math.pi, raw6)
up = core.SPIN_UP_INDICES
G_num = np.linalg.eigvalsh(H_G[np.ix_(up, up)])
M_num = np.linalg.eigvalsh(H_M[np.ix_(up, up)])
G_ana = np.sort([analytic[name] for name in BRANCHES if name.startswith("G_")])
M_ana = np.sort([analytic[name] for name in BRANCHES if name.startswith("M_")])

rows.append(
 {
 "source_row": int(idx),
 **reduced,
 "max_abs_error_Gamma": float(np.max(np.abs(G_num - G_ana))),
 "max_abs_error_M": float(np.max(np.abs(M_num - M_ana))),
 "max_abs_error": float(max(np.max(np.abs(G_num - G_ana)), np.max(np.abs(M_num - M_ana)))),
 }
)
return pd.DataFrame(rows)

```

```

=====
4. Step 04 数据解析审计
=====

```

```

def augment_with_analytic(df: pd.DataFrame) -> pd.DataFrame:
 if len(df) == 0:
 return df.copy()
 records = []
 for _, row in df.iterrows():
 records.append(analytic_diagnostics(_row_reduced(row)))
 analytic_df = pd.DataFrame(records, index=df.index)
 overlap = [col for col in analytic_df.columns if col in df.columns]
 if overlap:
 analytic_df = analytic_df.rename(columns={col: f"analytic_{col}" for col in overlap})
 return pd.concat([df.reset_index(drop=True), analytic_df.reset_index(drop=True)], axis=1)

```

```

def add_no_go_certificate(df: pd.DataFrame) -> pd.DataFrame:
 out = df.copy()
 verified_gap_col = (
 "verified_indirect_gap" if "verified_indirect_gap" in out.columns else "indirect_gap"
)
 observed = pd.to_numeric(out.get(verified_gap_col), errors="coerce")
 bound = pd.to_numeric(out.get("conditional_no_go_bound"), errors="coerce")
 out["observed_indirect_for_certificate"] = observed
 out["no_go_bound_residual_observed_minus_bound"] = observed - bound
 out["conditional_no_go_certificate"] = (
 (pd.to_numeric(out["Bplus_partner_switch"], errors="coerce") == 1)
 & observed.notna()
 & bound.notna()
 & (observed <= bound + 5.0e-8)
 & (bound <= 1.0e-12)
).astype(int)
 out["gamma_M_exactness_error"] = observed - pd.to_numeric(
 out["gamma_M_indirect_analytic"], errors="coerce"
)
 return out

```

```

=====
5. 完整复核所有粗筛 TI 候选 (修复 Step 04 队列上限遗漏)
=====

```

---

```

def strict_verify_row(
 row: pd.Series,
 config: NoGoConfig,
 details_dir: Path,
) -> Dict[str, object]:
 point_id = str(row["point_id"])
 reduced = _row_reduced(row)
 raw6 = core.raw6_from_reduced5(reduced)
 result: Dict[str, object] = dict(row)
 result["strict_error"] = ""

 try:
 gap_summary, gap_table = core.consensus_gap_audit(
 raw6,
 config.verify_gap_grids,
 config.verify_gap_shifts,
 config.gap_tol,
)
 result.update(gap_summary)
 gap_table.insert(0, "point_id", point_id)
 gap_table.to_csv(
 details_dir / f"{point_id}_gap_consensus.csv",
 index=False,
 encoding="utf-8-sig",
)
 except Exception as exc:
 result["strict_phase_label"] = "numeric_error"
 result["strict_error"] = f"gap_failed: {repr(exc)}"
 return result

 if not bool(int(result.get("verified_is_direct_gapped", 0))):
 result["strict_phase_label"] = "noninsulating_or_gap_closing"
 return result
 if not bool(int(result.get("verified_is_balanced_spin_sector", 0))):
 result["strict_phase_label"] = "spin_sector_filling_mismatch"
 return result

 try:
 ch_summary, ch_table = core.consensus_chern_audit(
 raw6,
 config.verify_chern_grids,
 config.verify_chern_shifts,
 config.chern_tol,
 config.min_det_tol,
 include_total=True,
)
 result.update(ch_summary)
 ch_table.insert(0, "point_id", point_id)
 ch_table.to_csv(
 details_dir / f"{point_id}_chern_consensus.csv",
 index=False,
 encoding="utf-8-sig",
)
 except Exception as exc:
 result["strict_phase_label"] = "chern_unreliable"
 result["strict_error"] = f"chern_failed: {repr(exc)}"
 return result

 reliable = bool(int(result.get("verified_chern_reliable", 0)))
 cu = result.get("verified_chern_up_int")
 cd = result.get("verified_chern_down_int")
 ct = result.get("verified_chern_total_int")
 insulator = bool(int(result.get("verified_is_physical_insulator", 0)))

 topological = reliable and cu is not None and cd is not None and ct == 0 and cu == -cd and cu != 0
 if not reliable:
 label = "chern_unreliable"
 elif topological and insulator:

```

---

```

 label = "spin_chern_TI_candidate"
 elif topological:
 label = "spin_chern_band_metal"
 elif insulator and cu == 0 and cd == 0 and ct == 0:
 label = "trivial_insulator"
 elif not insulator:
 label = "indirect_overlap"
 else:
 label = "other_gapped_phase"
 result["strict_phase_label"] = label
 return result

def complete_coarse_ti_verification(
 coarse: pd.DataFrame,
 existing_strict: pd.DataFrame,
 config: NoGoConfig,
 output: Path,
) -> pd.DataFrame:
 candidates = coarse[coarse["phase_label"] == "spin_chern_TI_candidate"].copy()
 existing_ids = set(existing_strict["point_id"].astype(str)) if len(existing_strict) else set()

 if config.test_mode:
 # Quick test validates parsing, symbolic formulas, analytic spectra and
 # output generation. It deliberately reuses the existing strict Step 04
 # table so reduced test grids cannot create scientific false positives.
 return existing_strict.copy()

 if config.reverify_all_coarse_ti:
 queue = candidates
 else:
 queue = candidates[~candidates["point_id"].astype(str).isin(existing_ids)]

 details_dir = output / "strict_details"
 new_rows = [strict_verify_row(row, config, details_dir) for _, row in queue.iterrows()]
 new_df = pd.DataFrame(new_rows)

 combined = existing_strict.copy()
 if len(new_df):
 combined = pd.concat([combined, new_df], ignore_index=True, sort=False)
 combined = combined.drop_duplicates(subset=["point_id"], keep="last")
 return combined

=====
6. 锚点路径的稠密解析边界定位
=====

def interpolate_reduced(a: Dict[str, float], b: Dict[str, float], lam: float) -> Dict[str, float]:
 return {name: (1.0 - lam) * a[name] + lam * b[name] for name in PARAMS5}

def _roots_from_grid(x: np.ndarray, y: np.ndarray) -> list[float]:
 roots: list[float] = []
 for i in range(len(x) - 1):
 y0, y1 = float(y[i]), float(y[i + 1])
 if not np.isfinite(y0) or not np.isfinite(y1):
 continue
 if y0 == 0.0:
 roots.append(float(x[i]))
 continue
 if y0 * y1 < 0.0:
 # Linear interpolation is sufficient for bracketing summary; the
 # dense grid makes the residual negligible.
 roots.append(float(x[i] - y0 * (x[i + 1] - x[i]) / (y1 - y0)))
 return roots

```

---

```

def dense_anchor_path(anchor_df: pd.DataFrame, n_points: int) -> tuple[pd.DataFrame, Dict[str, object]]:
 if len(anchor_df) < 2:
 return pd.DataFrame(), {}
 topo_row = anchor_df[anchor_df["anchor_role"] == "best_topological"]
 pos_row = anchor_df[anchor_df["anchor_role"] == "positive_gap_anchor"]
 if len(topo_row) == 0 or len(pos_row) == 0:
 topo_row = anchor_df.iloc[[0]]
 pos_row = anchor_df.iloc[[1]]
 a = _row_reduced(topo_row.iloc[0])
 b = _row_reduced(pos_row.iloc[0])

 lambdas = np.linspace(0.0, 1.0, int(n_points))
 rows = []
 for lam in lambdas:
 reduced = interpolate_reduced(a, b, float(lam))
 diag = analytic_diagnostics(reduced)
 rows.append({"path_lambda": float(lam), **reduced, **diag})
 df = pd.DataFrame(rows)

 summary = {
 "mass_Gamma_zero_roots": _roots_from_grid(
 df["path_lambda"].to_numpy(), df["mass_Gamma_signed"].to_numpy()
),
 "mass_M_zero_roots": _roots_from_grid(
 df["path_lambda"].to_numpy(), df["mass_M_signed"].to_numpy()
),
 "min_gamma_local_gap": float(df["gamma_local_gap_analytic"].min()),
 "lambda_min_gamma_local_gap": float(df.loc[df["gamma_local_gap_analytic"].idxmin(), "path_lambda"]),
 "min_M_local_gap": float(df["M_local_gap_analytic"].min()),
 "lambda_min_M_local_gap": float(df.loc[df["M_local_gap_analytic"].idxmin(), "path_lambda"]),
 "gamma_M_indirect_zero_roots": _roots_from_grid(
 df["path_lambda"].to_numpy(), df["gamma_M_indirect_analytic"].to_numpy()
),
 "delta_Bplus_zero_roots": _roots_from_grid(
 df["path_lambda"].to_numpy(), df["delta_Bplus_M_minus_G"].to_numpy()
),
 "min_gamma_M_indirect": float(df["gamma_M_indirect_analytic"].min()),
 "max_gamma_M_indirect": float(df["gamma_M_indirect_analytic"].max()),
 }
 return df, summary

```

```

def plot_dense_path(df: pd.DataFrame, output_path: Path) -> None:
 if len(df) == 0:
 return
 fig, ax = plt.subplots(figsize=(8.0, 5.0))
 x = df["path_lambda"].to_numpy()
 ax.plot(x, df["mass_Gamma_signed"], label=r"μ_Gamma")
 ax.plot(x, df["mass_M_signed"], label=r"μ_M")
 ax.plot(x, df["gamma_M_indirect_analytic"], label=r"$g_{\Gamma\ M}^{\text{ind}}$")
 ax.plot(x, df["conditional_no_go_bound"], linestyle="--", label="B+ no-go bound")
 ax.axhline(0.0, linewidth=1.0)
 ax.set_xlabel(r"Path coordinate λ")
 ax.set_ylabel("Energy")
 ax.legend(frameon=False)
 fig.tight_layout()
 fig.savefig(output_path, dpi=220)
 plt.close(fig)

```

```

=====
7. 汇总与主流程
=====

```

```

def build_branch_pattern_counts(df: pd.DataFrame, label_col: str) -> pd.DataFrame:
 if len(df) == 0:
 return pd.DataFrame()
 cols = [label_col, "active_branch_pattern", "Bplus_partner_switch"]

```

---

```

return (
 df.groupby(cols, dropna=False)
 .size()
 .reset_index(name="count")
 .sort_values([label_col, "count"], ascending=[True, False])
)

def run_pipeline(config: NoGoConfig) -> Dict[str, Path]:
 output = prepare_output(config)
 step04_dir = locate_step04_directory(config.step04_input, output)
 write_metadata(config, output, step04_dir)
 write_symbolic_outputs(output)

 coarse = pd.read_csv(step04_dir / "fes_step04_all_coarse_scan.csv")
 strict = pd.read_csv(step04_dir / "fes_step04_strict_verified_results.csv")
 anchors = pd.read_csv(step04_dir / "fes_step04_anchor_points.csv")

 # Validate exact analytic spectrum against numerical matrices.
 validation_n = 16 if config.test_mode else config.validation_samples
 validation = validate_analytic_spectrum(coarse, validation_n, config.validation_seed)
 validation_path = output / "fes_step05_eigenvalue_validation.csv"
 validation.to_csv(validation_path, index=False, encoding="utf-8-sig")

 # Complete every coarse TI candidate; default only fills missing Step 04 rows.
 strict_complete = complete_coarse_ti_verification(coarse, strict, config, output)
 strict_complete_path = output / "fes_step05_all_coarse_TI_strict_verification.csv"
 strict_complete.to_csv(strict_complete_path, index=False, encoding="utf-8-sig")

 coarse_aug = add_no_go_certificate(augment_with_analytic(coarse))
 coarse_aug_path = output / "fes_step05_all_coarse_analytic_audit.csv"
 coarse_aug.to_csv(coarse_aug_path, index=False, encoding="utf-8-sig")

 strict_aug = add_no_go_certificate(augment_with_analytic(strict_complete))
 strict_aug_path = output / "fes_step05_strict_analytic_audit.csv"
 strict_aug.to_csv(strict_aug_path, index=False, encoding="utf-8-sig")

 topological = strict_aug[
 strict_aug["strict_phase_label"].isin(["spin_chern_band_metal", "spin_chern_TI_candidate"])
].copy()
 topological_path = output / "fes_step05_verified_topological_no_go_certificates.csv"
 topological.to_csv(topological_path, index=False, encoding="utf-8-sig")

 pattern_counts = build_branch_pattern_counts(strict_aug, "strict_phase_label")
 pattern_path = output / "fes_step05_branch_pattern_counts.csv"
 pattern_counts.to_csv(pattern_path, index=False, encoding="utf-8-sig")

 dense_n = 501 if config.test_mode else config.dense_path_points
 dense_path, path_summary = dense_anchor_path(anchors, dense_n)
 dense_path_path = output / "fes_step05_anchor_path_analytic_dense.csv"
 dense_path.to_csv(dense_path_path, index=False, encoding="utf-8-sig")
 plot_dense_path(dense_path, output / "fes_step05_anchor_path_analytic.png")

 n_top = len(topological)
 n_cert = int(topological.get("conditional_no_go_certificate", pd.Series(dtype=int)).sum())
 n_switch = int(topological.get("Bplus_partner_switch", pd.Series(dtype=int)).sum())
 n_strict_ti = int((strict_aug["strict_phase_label"] == "spin_chern_TI_candidate").sum())

 reliable_mask = (
 pd.to_numeric(strict_aug.get("verified_chern_reliable"), errors="coerce") == 1
) & pd.to_numeric(strict_aug.get("verified_chern_up_int"), errors="coerce").notna()
 reliable = strict_aug[reliable_mask].copy()
 if len(reliable):
 actual_top = pd.to_numeric(reliable["verified_chern_up_int"], errors="coerce").abs() > 0
 predicted_top = pd.to_numeric(reliable["mass_product"], errors="coerce") > 0
 mass_accuracy = float((actual_top.to_numpy() == predicted_top.to_numpy()).mean())
 mass_false_positive = int((~actual_top & predicted_top).sum())
 mass_false_negative = int((actual_top & ~predicted_top).sum())
 else:

```

```

mass_accuracy = None
mass_false_positive = 0
mass_false_negative = 0

summary = {
 "code_version": CODE_VERSION,
 "step04_total_coarse_points": int(len(coarse)),
 "step04_coarse_TI_candidates": int((coarse["phase_label"] == "spin_chern_TI_candidate").sum()),
 "strict_rows_after_completion": int(len(strict_complete)),
 "strict_spin_chern_TI_candidates": n_strict_ti,
 "strict_topological_points": n_top,
 "topological_Bplus_partner_switch_count": n_switch,
 "topological_conditional_no_go_certificate_count": n_cert,
 "topological_certificate_fraction": float(n_cert / n_top) if n_top else None,
 "reliable_chern_rows_for_mass_test": int(len(reliable)),
 "mass_same_sign_topology_accuracy": mass_accuracy,
 "mass_same_sign_false_positive_count": mass_false_positive,
 "mass_same_sign_false_negative_count": mass_false_negative,
 "candidate_topology_rule": "|C_up|=1 iff mass_Gamma*mass_M>0 (empirically tested, not yet analytically
 ↪ proven)",
 "max_verified_topological_indirect_gap": (
 float(pd.to_numeric(topological["verified_indirect_gap"], errors="coerce").max())
 if n_top and "verified_indirect_gap" in topological else None
),
 "max_abs_Gamma_eigenvalue_validation_error": (
 float(validation["max_abs_error_Gamma"].max()) if len(validation) else None
),
 "max_abs_M_eigenvalue_validation_error": (
 float(validation["max_abs_error_M"].max()) if len(validation) else None
),
 "max_Bplus_identity_error": (
 float(np.nanmax(np.abs(pd.to_numeric(strict_aug["Bplus_identity_error"], errors="coerce"))))
 if len(strict_aug) else None
),
 "max_topological_gamma_M_exactness_error": (
 float(np.nanmax(np.abs(pd.to_numeric(topological["gamma_M_exactness_error"], errors="coerce"))))
 if n_top else None
),
 "conditional_no_go_statement": (
 "Within the half-filled branch regime where B+ exchanges occupied/unoccupied rank "
 "between Gamma and M, $E_{ind} \leq -4|r_1^2 - r_2^2|/(B_1+B_2) \leq 0$."
),
 "scope_warning": (
 "This is an exact conditional statement for the detected branch ordering. "
 "A complete model-wide theorem additionally requires proving that every nonzero-spin-Chern "
 "phase of the six-parameter Hamiltonian necessarily has this branch ordering."
),
 "dense_anchor_path": path_summary,
}

```

```

summary_path = output / "fes_step05_no_go_summary.json"
summary_path.write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")

```

```

report_lines = [
 "FES Step 05 analytic no-go audit",
 "=" * 72,
 f"Strict topological points: {n_top}",
 f"B+ partner-switching points: {n_switch}/{n_top}",
 f"Conditional no-go certificates: {n_cert}/{n_top}",
 f"Strict spin-Chern TI candidates: {n_strict_ti}",
 f"Best verified topological indirect gap: {summary['max_verified_topological_indirect_gap']}",
 "",
 "Candidate topology rule from strict data:",
 f" |C_up|=1 iff $\mu_{\text{Gamma}} \cdot \mu_{\text{M}} > 0$ accuracy: {mass_accuracy}",
 f" false positives / false negatives: {mass_false_positive} / {mass_false_negative}",
 "",
 "Exact conditional result:",
 " If B+ exchanges occupied/unoccupied rank between Gamma and M,",
 " $E_{ind} \leq -|B_1 - B_2| = -4|r_1^2 - r_2^2|/(B_1+B_2) \leq 0$.",

```



```

 "",
 "Logical status:",
 " - Gamma/M eigenvalue formulas: exact.",
 " - B1-B2 rationalization: exact.",
 " - No-go under B+ partner switching: exact.",
 " - All currently verified topological points satisfying partner switching: numerical evidence.",
 " - Model-wide no-go theorem: requires a separate proof that nonzero Chern always implies this
 ↪ ordering.",
]
 (output / "fes_step05_no_go_report.txt").write_text("\n".join(report_lines), encoding="utf-8")

 return {
 "output_dir": output,
 "summary": summary_path,
 "validation": validation_path,
 "strict_audit": strict_aug_path,
 "topological_certificates": topological_path,
 "dense_path": dense_path_path,
 "report": output / "fes_step05_no_go_report.txt",
 }
}

=====
8. CLI
=====

def parse_args() -> argparse.Namespace:
 parser = argparse.ArgumentParser(description=__doc__)
 parser.add_argument("--step04-input", default="outputs_fes6_step04_boundary.zip")
 parser.add_argument("--output-dir", default="outputs_fes6_step05_no_go")
 parser.add_argument("--workers", type=int, default=max(1, min(4, os.cpu_count() or 1)))
 parser.add_argument("--validation-samples", type=int, default=128)
 parser.add_argument("--dense-path-points", type=int, default=5001)
 parser.add_argument("--reverify-all-coarse-ti", action="store_true")
 parser.add_argument("--overwrite", action="store_true")
 parser.add_argument("--resume", action="store_true")
 parser.add_argument("--test", action="store_true")
 return parser.parse_args()

def main() -> None:
 args = parse_args()
 if args.overwrite and args.resume:
 raise ValueError("--overwrite 与 --resume 不能同时使用")

 if args.test:
 config = NoGoConfig(
 step04_input=args.step04_input,
 output_dir=args.output_dir,
 workers=1,
 validation_samples=16,
 dense_path_points=501,
 verify_gap_grids=(17, 21),
 verify_gap_shifts=((0.0, 0.0), (0.5, 0.5)),
 verify_chern_grids=(11, 15),
 verify_chern_shifts=((0.0, 0.0),),
 reverify_all_coarse_ti=False,
 overwrite=bool(args.overwrite),
 resume=bool(args.resume),
 test_mode=True,
)
 else:
 config = NoGoConfig(
 step04_input=args.step04_input,
 output_dir=args.output_dir,
 workers=max(1, args.workers),
 validation_samples=max(1, args.validation_samples),
 dense_path_points=max(101, args.dense_path_points),

```

```

 reverify_all_coarse_ti=bool(args.reverify_all_coarse_ti),
 overwrite=bool(args.overwrite),
 resume=bool(args.resume),
 test_mode=False,
)

 print("=" * 78)
 print("FES Step 05: analytic Gamma/M no-go audit")
 print("Version:", CODE_VERSION)
 print("Core :", getattr(core, "CODE_VERSION", "unknown"))
 print("Input :", Path(config.step04_input).resolve())
 print("Output :", Path(config.output_dir).resolve())
 print("=" * 78)

 paths = run_pipeline(config)
 print(json.dumps({key: str(value) for key, value in paths.items()}, ensure_ascii=False, indent=2))

if __name__ == "__main__":
 main()

```

## ► 代码清单 A.47.5 (python, 1226 行)

```

#!/usr/bin/env python3
-*- coding: utf-8 -*-
"""
FES Step 06 — Γ/M 低能 $k\cdot p$ Hamiltonian、谷手性与 spin-Chern 闭式公式

研究目标

Step 05 已经得到精确的条件性 no-go:

 若 $B+$ 分支在 Γ 与 M 之间发生 occupied/unoccupied partner switching,
 则 $E_g^{\text{ind}} \leq -4|r_1^2 - r_2^2|/(B_1+B_2) \leq 0$ 。

本步骤补齐模型级 no-go 所缺少的拓扑环节:

1. 在 Γ 与 M 的 $A-/B+$ 闭隙点构造两带低能 $k\cdot p$ Hamiltonian;
2. 用投影质量算符固定简并子空间规范;
3. 计算局域 off-diagonal winding、线性 Jacobian 与 Chern 跳变;
4. 验证两个谷的有效拓扑取向均为 $\text{sgn}(r_1 \cdot r_2)$;
5. 检验候选 spin-Chern 公式

 C_up = sgn(r1*r2)/2
 [sgn(mu_Gamma) + sgn(mu_M)],

 其中
 mu_Gamma = E_{Gamma,A-} - E_{Gamma,B+},
 mu_M = E_{M,B+} - E_{M,A-};

6. 将该公式与 Step 05 的 no-go certificate 连接, 输出模型级推理链。

重要边界

- $k\cdot p$ 投影、winding 和 Chern jump 是数值高精度验证;
- 闭式公式会在 Step 05 严格样本及全部粗筛样本上审计;
- 若要称为完全解析定理, 仍建议将本脚本输出的 Jacobian 符号结构
 进一步整理为手工/符号代数证明。

依赖

```

将以下脚本放在同一目录:

```

 FES_step02_sobol_scan.py

```

---

FES\_step05\_analytic\_no\_go.py

运行示例

-----

快速测试:

```
python FES_step06_kp_chern_formula.py \
 --step05-input outputs_fes6_step05_no_go.zip \
 --output-dir outputs_fes6_step06_test \
 --test --overwrite
```

正式运行:

```
python FES_step06_kp_chern_formula.py \
 --step05-input outputs_fes6_step05_no_go.zip \
 --output-dir outputs_fes6_step06_kp \
 --workers 4 --overwrite
```

"""

```
from __future__ import annotations

from dataclasses import asdict, dataclass
from datetime import datetime, timezone
from pathlib import Path
from typing import Dict, Iterable, Sequence
import argparse
import hashlib
import json
import math
import os
import shutil
import sys
import zipfile

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import brentq

import FES_step02_sobol_scan as core
import FES_step05_analytic_no_go as step05

np.set_printoptions(precision=12, suppress=True)

CODE_VERSION = "FES_STEP06_KP_CHERN_V1_20260713"
RESULT_SCHEMA_VERSION = "fes_step06_kp_chern_schema_v1"

PARAMS = ("m_e", "t1", "t2", "r1", "r2")
VALLEYS = ("Gamma", "M")
K_POINTS = {
 "Gamma": (0.0, 0.0),
 "M": (math.pi, math.pi),
}

@dataclass
class KPConfig:
 step05_input: str = "outputs_fes6_step05_no_go.zip"
 output_dir: str = "outputs_fes6_step06_kp"
 workers: int = max(1, min(4, os.cpu_count() or 1))

 representative_per_group: int = 1
 root_span: float = 0.30
 root_grid_points: int = 2401
 root_tol: float = 1.0e-11

 projection_dm: float = 1.0e-6
 fit_radius: float = 4.0e-3
 fit_grid_n: int = 9
 winding_radii: tuple[float, ...] = (
 1.0e-4, 3.0e-4, 1.0e-3, 3.0e-3, 1.0e-2
```

```

)
winding_angles: int = 721

chern_jump_delta_m: float = 1.0e-3
chern_jump_grids: tuple[int, ...] = (21, 31, 41)
chern_tol: float = 0.08
min_det_tol: float = 1.0e-7

formula_mass_tol: float = 1.0e-6
formula_hopping_tol: float = 1.0e-8
coarse_gap_margin: float = 5.0e-3
strict_only_reliable: bool = True

overwrite: bool = False
test_mode: bool = False

=====
1. 输入与版本管理
=====

def script_sha256() -> str | None:
 if "__file__" not in globals():
 return None
 path = Path(__file__).resolve()
 return hashlib.sha256(path.read_bytes()).hexdigest() if path.exists() else None

def prepare_output(config: KPConfig) -> Path:
 out = Path(config.output_dir).expanduser().resolve()
 if out.exists() and config.overwrite:
 shutil.rmtree(out)
 out.mkdir(parents=True, exist_ok=True)
 (out / "kp_details").mkdir(exist_ok=True)
 return out

def _find_step05_root(root: Path) -> Path:
 required = {
 "fes_step05_strict_analytic_audit.csv",
 "fes_step05_all_coarse_analytic_audit.csv",
 "fes_step05_verified_topological_no_go_certificates.csv",
 }
 names = {p.name for p in root.iterdir() if p.is_file()}
 if required.issubset(names):
 return root

 candidates: list[Path] = []
 for path in root.rglob("fes_step05_strict_analytic_audit.csv"):
 parent = path.parent
 names = {p.name for p in parent.iterdir() if p.is_file()}
 if required.issubset(names):
 candidates.append(parent)
 if not candidates:
 raise FileNotFoundError(
 f" 无法在 {root} 中找到完整 Step 05 结果。必须包含: {sorted(required)}"
)
 return sorted(candidates, key=lambda p: len(p.parts))[0]

def locate_step05_directory(input_path: str | Path, output: Path) -> Path:
 path = Path(input_path).expanduser().resolve()
 if not path.exists():
 raise FileNotFoundError(path)
 if path.is_dir():
 return _find_step05_root(path)
 if path.suffix.lower() != ".zip":
 raise ValueError("--step05-input 必须是目录或 ZIP")
 extract_root = output / "_step05_extracted"

```

---

```

 if extract_root.exists():
 shutil.rmtree(extract_root)
 extract_root.mkdir(parents=True)
 with zipfile.ZipFile(path, "r") as zf:
 zf.extractall(extract_root)
 return _find_step05_root(extract_root)

def write_metadata(config: KPConfig, output: Path, step05_dir: Path) -> None:
 metadata = {
 "code_version": CODE_VERSION,
 "result_schema_version": RESULT_SCHEMA_VERSION,
 "run_time_utc": datetime.now(timezone.utc).isoformat(),
 "script_sha256": script_sha256(),
 "step02_core_version": getattr(core, "CODE_VERSION", None),
 "step02_core_sha256": hashlib.sha256(Path(core.__file__).read_bytes()).hexdigest(),
 "step05_core_version": getattr(step05, "CODE_VERSION", None),
 "step05_core_sha256": hashlib.sha256(Path(step05.__file__).read_bytes()).hexdigest(),
 "step05_directory": str(step05_dir),
 "config": asdict(config),
 }
 (output / "fes_step06_run_metadata.json").write_text(
 json.dumps(metadata, ensure_ascii=False, indent=2),
 encoding="utf-8",
)

=====
2. 参数与质量
=====

def reduced_from_row(row: pd.Series | Dict[str, object]) -> Dict[str, float]:
 return {name: float(row[name]) for name in PARAMS5}

def raw_from_reduced(p: Dict[str, float]) -> Dict[str, float]:
 return core.raw6_from_reduced5(p)

def analytic_mass(p: Dict[str, float], valley: str) -> float:
 d = step05.analytic_diagnostics(p)
 if valley == "Gamma":
 return float(d["mass_Gamma_signed"])
 if valley == "M":
 return float(d["mass_M_signed"])
 raise ValueError(valley)

def mass_derivative(p: Dict[str, float], valley: str, eps: float = 1.0e-6) -> float:
 plus = dict(p)
 minus = dict(p)
 plus["m_e"] += eps
 minus["m_e"] -= eps
 return (analytic_mass(plus, valley) - analytic_mass(minus, valley)) / (2.0 * eps)

def candidate_chern_formula(
 p: Dict[str, float],
 prefactor: int = 1,
 mass_tol: float = 1.0e-12,
 hopping_tol: float = 1.0e-14,
) -> float:
 d = step05.analytic_diagnostics(p)
 rprod = p["r1"] * p["r2"]
 mu_g = float(d["mass_Gamma_signed"])
 mu_m = float(d["mass_M_signed"])
 if abs(rprod) <= hopping_tol or abs(mu_g) <= mass_tol or abs(mu_m) <= mass_tol:
 return float("nan")
 return float(

```

```

 prefactor
 * np.sign(rprod)
 * 0.5
 * (np.sign(mu_g) + np.sign(mu_m))
)

=====
3. 代表点与质量根
=====

def load_step05_tables(step05_dir: Path) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
 strict = pd.read_csv(step05_dir / "fes_step05_strict_analytic_audit.csv")
 coarse = pd.read_csv(step05_dir / "fes_step05_all_coarse_analytic_audit.csv")
 cert = pd.read_csv(
 step05_dir / "fes_step05_verified_topological_no_go_certificates.csv"
)
 return strict, coarse, cert

def choose_representatives(strict: pd.DataFrame, per_group: int) -> pd.DataFrame:
 reliable = strict.copy()
 reliable = reliable[
 (pd.to_numeric(reliable["verified_chern_reliable"], errors="coerce") == 1)
 & (pd.to_numeric(reliable["verified_is_direct_gapped"], errors="coerce") == 1)
 & (pd.to_numeric(reliable["verified_is_balanced_spin_sector"], errors="coerce") == 1)
].copy()

 reliable["r_product_sign"] = np.sign(
 pd.to_numeric(reliable["r1"], errors="coerce")
 * pd.to_numeric(reliable["r2"], errors="coerce")
)
 reliable["chern_class"] = pd.to_numeric(
 reliable["verified_chern_up_int"], errors="coerce"
).fillna(99).astype(int)

 groups: list[pd.DataFrame] = []
 topo = reliable[reliable["chern_class"].abs() == 1]
 for (chern, rsign), group in topo.groupby(["chern_class", "r_product_sign"]):
 chosen = group.sort_values(
 ["verified_min_direct_gap", "mass_boundary_distance"],
 ascending=[False, False],
).head(per_group)
 groups.append(chosen)

 trivial = reliable[reliable["chern_class"] == 0]
 if not trivial.empty:
 groups.append(
 trivial.sort_values(
 ["verified_min_direct_gap", "verified_indirect_gap"],
 ascending=False,
).head(max(1, per_group))
)

 if not groups:
 raise RuntimeError(" 没有找到可用的严格代表点")

 out = pd.concat(groups, ignore_index=True).drop_duplicates("point_id")
 cols = [
 "point_id", "m_e", "t1", "t2", "r1", "r2",
 "verified_chern_up_int", "verified_chern_down_int",
 "verified_min_direct_gap", "verified_indirect_gap",
 "mass_Gamma_signed", "mass_M_signed",
 "strict_phase_label", "r_product_sign",
]
 return out[[c for c in cols if c in out.columns]].copy()

def find_mass_roots(

```

---

```

 p: Dict[str, float],
 valley: str,
 span: float,
 n_grid: int,
 root_tol: float,
) -> list[float]:
 m0 = float(p["m_e"])
 xs = np.linspace(m0 - span, m0 + span, int(n_grid))
 vals = np.array([
 analytic_mass(**p, "m_e": float(x), valley) for x in xs
], dtype=float)

 roots: list[float] = []
 for i in range(len(xs) - 1):
 a, b = float(xs[i]), float(xs[i + 1])
 fa, fb = float(vals[i]), float(vals[i + 1])
 if abs(fa) <= root_tol:
 roots.append(a)
 if fa * fb < 0.0:
 root = brentq(
 lambda m: analytic_mass(**p, "m_e": float(m), valley),
 a, b, xtol=root_tol, rtol=4.0 * np.finfo(float).eps,
)
 roots.append(float(root))
 if abs(vals[-1]) <= root_tol:
 roots.append(float(xs[-1]))

 unique: list[float] = []
 for root in sorted(roots):
 if not unique or abs(root - unique[-1]) > 1.0e-7:
 unique.append(root)
 return unique

def root_table(representatives: pd.DataFrame, config: KPConfig) -> pd.DataFrame:
 rows: list[Dict[str, object]] = []
 for _, row in representatives.iterrows():
 p = reduced_from_row(row)
 for valley in VALLEYS:
 roots = find_mass_roots(
 p, valley, config.root_span,
 config.root_grid_points, config.root_tol,
)
 if not roots:
 rows.append({
 "point_id": row["point_id"],
 "valley": valley,
 **p,
 "root_found": 0,
 "m_critical": np.nan,
 "distance_from_reference": np.nan,
 })
 continue

 # The nearest root controls the neighboring phase of this representative.
 root = min(roots, key=lambda x: abs(x - p["m_e"]))
 pc = dict(p)
 pc["m_e"] = float(root)
 rows.append({
 "point_id": row["point_id"],
 "valley": valley,
 **p,
 "reference_bernoulli": row.get("verified_bernoulli_int", np.nan),
 "r_product_sign": int(np.sign(p["r1"] * p["r2"])),
 "root_found": 1,
 "all_roots": json.dumps(roots),
 "m_critical": float(root),
 "distance_from_reference": float(root - p["m_e"]),
 "mass_at_root": analytic_mass(pc, valley),
 })

```

---

```

 "dmu_dm_at_root": mass_derivative(pc, valley),
 })
 return pd.DataFrame(rows)

=====
4. 两带投影、k-p 系数与 winding
=====

def spin_up_atomic(kx: float, ky: float, p: Dict[str, float]) -> np.ndarray:
 return core.h_spin_block_atomic(kx, ky, raw_from_reduced(p), "up")

def critical_projection_basis(
 p: Dict[str, float],
 valley: str,
 dm: float,
) -> Dict[str, object]:
 kx0, ky0 = K_POINTS[valley]
 h0 = spin_up_atomic(kx0, ky0, p)
 evals, evecs = np.linalg.eigh(h0)

 # At the A-/B+ transition the middle pair closes.
 middle = np.array([1, 2], dtype=int)
 U = evecs[:, middle]
 e0 = float(np.mean(evals[middle]))

 pp = dict(p)
 pm = dict(p)
 pp["m_e"] += dm
 pm["m_e"] -= dm
 dHdm = (
 spin_up_atomic(kx0, ky0, pp) - spin_up_atomic(kx0, ky0, pm)
) / (2.0 * dm)

 mass_matrix = U.conj().T @ dHdm @ U
 mass_evals, mass_vecs = np.linalg.eigh(mass_matrix)
 W = U @ mass_vecs

 projected_mass = W.conj().T @ dHdm @ W
 dz_dm = float(np.real(projected_mass[0, 0] - projected_mass[1, 1]) / 2.0)

 return {
 "W": W,
 "e0": e0,
 "full_evals": evals,
 "mass_evals": mass_evals,
 "dz_dm": dz_dm,
 "critical_pair_gap": float(evals[2] - evals[1]),
 }

def projected_pauli(
 p: Dict[str, float],
 valley: str,
 W: np.ndarray,
 e0: float,
 qx: float,
 qy: float,
) -> tuple[float, float, float, float, complex]:
 kx0, ky0 = K_POINTS[valley]
 h = spin_up_atomic(kx0 + qx, ky0 + qy, p)
 heff = W.conj().T @ h @ W - e0 * np.eye(2)
 heff = 0.5 * (heff + heff.conj().T)

 d0 = float(np.real(np.trace(heff)) / 2.0)
 dx = float(np.real(heff[0, 1]))
 dy = float(-np.imag(heff[0, 1]))
 dz = float(np.real(heff[0, 0] - heff[1, 1]) / 2.0)

```



---

```

 return d0, dx, dy, dz, complex(heff[0, 1])

def winding_on_circle(
 p: Dict[str, float],
 valley: str,
 W: np.ndarray,
 e0: float,
 radius: float,
 n_angles: int,
) -> Dict[str, float]:
 angles = np.linspace(0.0, 2.0 * np.pi, int(n_angles), endpoint=True)
 offdiag: list[complex] = []
 for theta in angles:
 qx = radius * math.cos(float(theta))
 qy = radius * math.sin(float(theta))
 *_ , off = projected_pauli(p, valley, W, e0, qx, qy)
 offdiag.append(off)

 arr = np.asarray(offdiag, dtype=np.complex128)
 phases = np.unwrap(np.angle(arr))
 winding = float((phases[-1] - phases[0]) / (2.0 * np.pi))
 return {
 "radius": float(radius),
 "winding_raw": winding,
 "winding_int": int(np rint(winding)),
 "min_abs_offdiag": float(np.min(np.abs(arr))),
 "max_abs_offdiag": float(np.max(np.abs(arr))),
 }

def polynomial_design(qx: np.ndarray, qy: np.ndarray) -> tuple[np.ndarray, list[str]]:
 X = np.column_stack([
 np.ones_like(qx),
 qx, qy,
 qx * qx, qx * qy, qy * qy,
])
 names = ["const", "qx", "qy", "qx2", "qxqy", "qy2"]
 return X, names

def fit_kp_coefficients(
 p: Dict[str, float],
 valley: str,
 W: np.ndarray,
 e0: float,
 radius: float,
 grid_n: int,
) -> Dict[str, object]:
 qs = np.linspace(-radius, radius, int(grid_n))
 qx_values: list[float] = []
 qy_values: list[float] = []
 components = {"d0": [], "dx": [], "dy": [], "dz": []}

 for qx in qs:
 for qy in qs:
 if abs(qx) < 1.0e-16 and abs(qy) < 1.0e-16:
 continue
 d0, dx, dy, dz, _ = projected_pauli(
 p, valley, W, e0, float(qx), float(qy)
)
 qx_values.append(float(qx))
 qy_values.append(float(qy))
 components["d0"].append(d0)
 components["dx"].append(dx)
 components["dy"].append(dy)
 components["dz"].append(dz)

 qx_arr = np.asarray(qx_values)

```

---

```

qy_arr = np.asarray(qy_values)
X, names = polynomial_design(qx_arr, qy_arr)

result: Dict[str, object] = {}
for component, values in components.items():
 y = np.asarray(values, dtype=float)
 coef, *_ = np.linalg.lstsq(X, y, rcond=None)
 pred = X @ coef
 rms = float(np.sqrt(np.mean((pred - y) ** 2)))
 scale = max(float(np.sqrt(np.mean(y ** 2))), 1.0e-15)
 result[f"{component}_fit_rms"] = rms
 result[f"{component}_fit_relative_rms"] = rms / scale
 for name, value in zip(names, coef):
 result[f"{component}_{name}"] = float(value)

d_x,d_y linear Jacobian.
J = np.array([
 [result["dx_qx"], result["dx_qy"]],
 [result["dy_qx"], result["dy_qy"]],
], dtype=float)
detJ = float(np.linalg.det(J))
result.update({
 "jacobian_xx": float(J[0, 0]),
 "jacobian_xy": float(J[0, 1]),
 "jacobian_yx": float(J[1, 0]),
 "jacobian_yy": float(J[1, 1]),
 "jacobian_det": detJ,
 "jacobian_chirality": int(np.sign(detJ)) if abs(detJ) > 1.0e-16 else 0,
})
return result

def kp_audit(root_df: pd.DataFrame, config: KPConfig) -> tuple[pd.DataFrame, pd.DataFrame]:
 summary_rows: list[Dict[str, object]] = []
 winding_rows: list[Dict[str, object]] = []

 valid = root_df[root_df["root_found"] == 1].copy()
 for _, row in valid.iterrows():
 p = {name: float(row[name]) for name in PARAMS5}
 p["m_e"] = float(row["m_critical"])
 valley = str(row["valley"])

 basis = critical_projection_basis(p, valley, config.projection_dm)
 W = basis["W"]
 e0 = float(basis["e0"])

 fit = fit_kp_coefficients(
 p, valley, W, e0, config.fit_radius, config.fit_grid_n
)

 per_radius: list[Dict[str, float]] = []
 for radius in config.winding_radii:
 wr = winding_on_circle(
 p, valley, W, e0, float(radius), config.winding_angles
)
 per_radius.append(wr)
 winding_rows.append({
 "point_id": row["point_id"],
 "valley": valley,
 "m_critical": p["m_e"],
 "r1": p["r1"],
 "r2": p["r2"],
 "r_product_sign": int(np.sign(p["r1"] * p["r2"])),
 **wr,
 })

 windings = np.array([x["winding_int"] for x in per_radius], dtype=int)
 winding_consensus = int(np.all(windings == windings[0]))
 winding_int = int(windings[0]) if winding_consensus else int(np rint(np.median(windings)))

```

---

```

 dmu_dm = float(row["dmu_dm_at_root"])
 dz_dm = float(basis["dz_dm"])
 mass_orientation = int(np.sign(dmu_dm * dz_dm))
 eta = int(winding_int * mass_orientation)
 rsign = int(np.sign(p["r1"] * p["r2"]))

 summary_rows.append({
 "point_id": row["point_id"],
 "valley": valley,
 **p,
 "reference_chern_up": row.get("reference_chern_up", np.nan),
 "r_product_sign": rsign,
 "critical_pair_gap": basis["critical_pair_gap"],
 "mass_eigenvalue_1": float(basis["mass_evals"][0]),
 "mass_eigenvalue_2": float(basis["mass_evals"][1]),
 "dz_dm_projected": dz_dm,
 "dmu_dm_analytic": dmu_dm,
 "mass_orientation_sign": mass_orientation,
 "winding_consensus": winding_consensus,
 "winding_int": winding_int,
 "eta_valley": eta,
 "eta_equals_sign_r1r2": int(eta == rsign),
 **fit,
 })

return pd.DataFrame(summary_rows), pd.DataFrame(winding_rows)

=====
5. Chern jump across each critical mass
=====

def chern_up_single(p: Dict[str, float], nk: int) -> tuple[float, float]:
 raw = raw_from_reduced(p)
 result = core.fukui_chern_subspace_shifted(
 lambda kx, ky: core.h_spin_block_periodic(kx, ky, raw, "up"),
 core.N_OCC_SPIN,
 int(nk),
 (0.0, 0.0),
)
 return float(result["chern"]), float(result["min_det"])

def chern_jump_audit(root_df: pd.DataFrame, config: KPConfig) -> pd.DataFrame:
 rows: list[Dict[str, object]] = []
 valid = root_df[root_df["root_found"] == 1].copy()

 grids = config.chern_jump_grids
 if config.test_mode:
 grids = tuple(g for g in grids if g <= 21) or (15,)

 for _, row in valid.iterrows():
 p0 = {name: float(row[name]) for name in PARAMS5}
 p0["m_e"] = float(row["m_critical"])
 valley = str(row["valley"])
 delta = float(config.chern_jump_delta_m)

 pm = dict(p0)
 pp = dict(p0)
 pm["m_e"] -= delta
 pp["m_e"] += delta

 mu_minus = analytic_mass(pm, valley)
 mu_plus = analytic_mass(pp, valley)
 rsign = int(np.sign(p0["r1"] * p0["r2"]))
 jump_pred = float(
 rsign * 0.5 * (np.sign(mu_plus) - np.sign(mu_minus))
)

```

---

```

per_grid = []
for nk in grids:
 try:
 c_minus, det_minus = chern_up_single(pm, int(nk))
 c_plus, det_plus = chern_up_single(pp, int(nk))
 per_grid.append({
 "nk": int(nk),
 "chern_minus": c_minus,
 "chern_plus": c_plus,
 "chern_jump": c_plus - c_minus,
 "min_det_minus": det_minus,
 "min_det_plus": det_plus,
 "error": "",
 })
 except Exception as exc:
 per_grid.append({
 "nk": int(nk),
 "chern_minus": np.nan,
 "chern_plus": np.nan,
 "chern_jump": np.nan,
 "min_det_minus": np.nan,
 "min_det_plus": np.nan,
 "error": repr(exc),
 })

detail = pd.DataFrame(per_grid)
detail.to_csv(
 Path(config.output_dir).resolve()
 / "kp_details"
 / f"{row['point_id']}_{valley}_chern_jump.csv",
 index=False,
)

good = detail[detail["error"] == "").copy()
jumps = good["chern_jump"].to_numpy(dtype=float) if not good.empty else np.array([])
rounded = np rint(jumps).astype(int) if len(jumps) else np.array([], dtype=int)
consensus = int(len(rounded) > 0 and np.all(rounded == rounded[0]))

rows.append({
 "point_id": row["point_id"],
 "valley": valley,
 **p0,
 "delta_m": delta,
 "mu_minus": mu_minus,
 "mu_plus": mu_plus,
 "r_product_sign": rsign,
 "predicted_chern_jump": jump_pred,
 "observed_chern_jump_mean": float(np.nanmean(jumps)) if len(jumps) else np.nan,
 "observed_chern_jump_int": int(rounded[0]) if consensus else np.nan,
 "chern_jump_consensus": consensus,
 "jump_matches_formula": int(
 consensus and int(rounded[0]) == int(np rint(jump_pred))
),
 "minimum_link_determinant": float(
 np.nanmin(
 np.concatenate([
 good["min_det_minus"].to_numpy(dtype=float),
 good["min_det_plus"].to_numpy(dtype=float),
])
)
) if not good.empty else np.nan,
})
return pd.DataFrame(rows)

```

```

=====
6. Chern 公式审计
=====

```

---

```

def formula_audit(
 df: pd.DataFrame,
 label: str,
 strict: bool,
 config: KPConfig,
) -> tuple[pd.DataFrame, Dict[str, object]]:
 out = df.copy()

 if strict:
 required = (
 (pd.to_numeric(out["verified_chern_reliable"], errors="coerce") == 1)
 & (pd.to_numeric(out["verified_is_direct_gapped"], errors="coerce") == 1)
 & (pd.to_numeric(out["verified_is_balanced_spin_sector"], errors="coerce") == 1)
)
 true_col = "verified_chern_up_int"
 direct_col = "verified_min_direct_gap"
 else:
 required = (
 (pd.to_numeric(out["chern_reliable"], errors="coerce") == 1)
 & (pd.to_numeric(out["is_direct_gapped"], errors="coerce") == 1)
 & (pd.to_numeric(out["is_balanced_spin_sector"], errors="coerce") == 1)
)
 true_col = "chern_up_int"
 direct_col = "min_direct_gap"

 out = out[required].copy()
 out["formula_pred_prefactor_plus"] = [
 candidate_chern_formula(
 reduced_from_row(row), +1,
 config.formula_mass_tol, config.formula_hopping_tol,
)
 for _, row in out.iterrows()
]
 out["formula_pred_prefactor_minus"] = -out["formula_pred_prefactor_plus"]

 valid = (
 np.isfinite(out["formula_pred_prefactor_plus"])
 & np.isfinite(pd.to_numeric(out[true_col], errors="coerce"))
)
 calibration = out[valid].copy()

 acc_plus = float(np.mean(
 calibration["formula_pred_prefactor_plus"].to_numpy()
 == pd.to_numeric(calibration[true_col], errors="coerce").to_numpy()
)) if len(calibration) else np.nan
 acc_minus = float(np.mean(
 calibration["formula_pred_prefactor_minus"].to_numpy()
 == pd.to_numeric(calibration[true_col], errors="coerce").to_numpy()
)) if len(calibration) else np.nan
 prefactor = +1 if (np.isnan(acc_minus) or acc_plus >= acc_minus) else -1

 out["calibrated_prefactor"] = prefactor
 out["chern_up_formula_pred"] = (
 out["formula_pred_prefactor_plus"]
 if prefactor == 1
 else out["formula_pred_prefactor_minus"]
)
 out["chern_formula_match"] = (
 out["chern_up_formula_pred"]
 == pd.to_numeric(out[true_col], errors="coerce")
).astype(int)
 out["formula_boundary_distance"] = np.minimum(
 np.abs(pd.to_numeric(out["mass_Gamma_signed"], errors="coerce")),
 np.abs(pd.to_numeric(out["mass_M_signed"], errors="coerce")),
)
 out["formula_hopping_distance"] = np.abs(
 pd.to_numeric(out["r1"], errors="coerce")
 * pd.to_numeric(out["r2"], errors="coerce")
)

```

```

)
out["formula_near_boundary"] = (
 (out["formula_boundary_distance"] <= config.formula_mass_tol)
 | (out["formula_hopping_distance"] <= config.formula_hopping_tol)
 | (pd.to_numeric(out[direct_col], errors="coerce") <= config.coarse_gap_margin)
).astype(int)

evaluated = out[np.isfinite(out["chern_up_formula_pred"])]
robust = evaluated[evaluated["formula_near_boundary"] == 0].copy()

summary = {
 "dataset": label,
 "strict": bool(strict),
 "reliable_rows": int(len(out)),
 "formula_evaluated_rows": int(len(evaluated)),
 "calibrated_prefactor": int(prefactor),
 "accuracy_all_evaluated": float(evaluated["chern_formula_match"].mean())
 if len(evaluated) else np.nan,
 "robust_rows_after_boundary_exclusion": int(len(robust)),
 "accuracy_robust": float(robust["chern_formula_match"].mean())
 if len(robust) else np.nan,
 "mismatch_count_all": int((evaluated["chern_formula_match"] == 0).sum()),
 "mismatch_count_robust": int((robust["chern_formula_match"] == 0).sum()),
}
return out, summary

=====
7. 图与报告
=====

def plot_kp_winding(kp: pd.DataFrame, output: Path) -> None:
 if kp.empty:
 return
 fig, ax = plt.subplots(figsize=(8, 5))
 labels = [
 f"{row.point_id}\n{row.valley}"
 for row in kp.itertuples(index=False)
]
 x = np.arange(len(kp))
 ax.bar(x - 0.18, kp["winding_int"], width=0.36, label="offdiag winding")
 ax.bar(x + 0.18, kp["eta_valley"], width=0.36, label=r"$\eta_K=w_K, \mathrm{sgn}(d_z'\mu)$")
 ax.set_xticks(x)
 ax.set_xticklabels(labels, rotation=45, ha="right", fontsize=8)
 ax.axhline(0.0, linewidth=0.8)
 ax.set_ylabel("integer")
 ax.legend()
 fig.tight_layout()
 fig.savefig(output / "fes_step06_local_winding_and_eta.png", dpi=220)
 plt.close(fig)

def plot_formula_confusion(audit: pd.DataFrame, output: Path, name: str, true_col: str) -> None:
 if audit.empty:
 return
 valid = audit[
 np.isfinite(audit["chern_up_formula_pred"])
 & np.isfinite(pd.to_numeric(audit[true_col], errors="coerce"))
].copy()
 if valid.empty:
 return

 true = pd.to_numeric(valid[true_col], errors="coerce").astype(int)
 pred = valid["chern_up_formula_pred"].astype(int)
 classes = [-1, 0, 1]
 matrix = np.zeros((3, 3), dtype=int)
 for t, p in zip(true, pred):
 if t in classes and p in classes:
 matrix[classes.index(t), classes.index(p)] += 1

```

---

```

fig, ax = plt.subplots(figsize=(5, 4))
im = ax.imshow(matrix)
for i in range(3):
 for j in range(3):
 ax.text(j, i, str(matrix[i, j]), ha="center", va="center")
ax.set_xticks(range(3), classes)
ax.set_yticks(range(3), classes)
ax.set_xlabel("predicted $C_{\uparrow}$ ")
ax.set_ylabel("numerical $C_{\uparrow}$ ")
ax.set_title(name)
fig.colorbar(im, ax=ax, label="count")
fig.tight_layout()
fig.savefig(output / f"fes_step06_{name}_formula_confusion.png", dpi=220)
plt.close(fig)

```

```

def write_formula_files(output: Path, prefactor: int) -> None:
 sign_text = "" if prefactor == 1 else "-"
 txt = f""FES Step 06 candidate spin-Chern formula

```

Definitions

-----

```

mu_Gamma = E_(Gamma,A-) - E_(Gamma,B+)
mu_M = E_(M,B+) - E_(M,A-)

```

Low-energy valley orientation

-----

```

eta_Gamma = eta_M = {sign_text}sgn(r1*r2)

```

Candidate closed formula

-----

```

C_up = {sign_text}sgn(r1*r2)/2
 [sgn(mu_Gamma) + sgn(mu_M)]

```

```

C_down = -C_up

```

```

C_total = 0

```

Combined no-go chain

-----

```

C_up != 0

```

```

=> mu_Gamma and mu_M have the same sign

```

```

=> B+ partner switching between Gamma and M

```

```

=> E_g^ind <= -4|r1^2-r2^2|/(B1+B2) <= 0.

```

Scope

-----

The formula is reconstructed from local k.p winding, Chern jumps, and strict numerical validation. A publication-level theorem should also present the projected Jacobian sign analytically.

"""

```

(output / "fes_step06_candidate_chern_formula.txt").write_text(
 txt, encoding="utf-8"
)

```

```

pref = "" if prefactor == 1 else "-"

```

```

tex = r"""\[

```

```

\mu_\Gamma=E_{\Gamma,A^-}-E_{\Gamma,B^+},\quad

```

```

\mu_M=E_{M,B^+}-E_{M,A^-}.

```

```

\]

```

```

\[

```

```

\eta_\Gamma=\eta_M=__PREFIX__\operatorname{sgn}(r_1r_2).

```

```

\]

```

```

\[

```

```

\boxed{

```

```

C_\uparrow=

```

```

__PREFIX__\frac{\operatorname{sgn}(r_1r_2)}{2}

```

---

```

\left[
\operatorname{sgn}(\mu_\Gamma)+
\operatorname{sgn}(\mu_M)
\right]
}
\]

\[
C_\downarrow=-C_\uparrow,\quad C_{\mathrm{total}}=0.
\]

Combining this result with the Step-05 branch-switching identity gives
\[
C_\uparrow\neq 0
\Longrightarrow
E_g^{\mathrm{ind}}
\leq
-\frac{4\left|r_1^2-r_2^2\right|}{B_1+B_2}
\leq 0.
\]
""" .replace("__PREF__", pref)
 (output / "fes_step06_candidate_chern_formula.tex").write_text(
 tex, encoding="utf-8"
)

=====
8. 主流程
=====

def run_pipeline(config: KPConfig) -> Dict[str, str]:
 output = prepare_output(config)
 step05_dir = locate_step05_directory(config.step05_input, output)
 write_metadata(config, output, step05_dir)

 strict, coarse, cert = load_step05_tables(step05_dir)

 representatives = choose_representatives(
 strict,
 1 if config.test_mode else config.representative_per_group,
)
 representatives.to_csv(
 output / "fes_step06_representative_points.csv", index=False
)

 roots = root_table(representatives, config)
 roots.to_csv(output / "fes_step06_critical_mass_roots.csv", index=False)

 kp_summary, winding = kp_audit(roots, config)
 kp_summary.to_csv(output / "fes_step06_kp_coefficients.csv", index=False)
 winding.to_csv(output / "fes_step06_winding_by_radius.csv", index=False)

 jumps = chern_jump_audit(roots, config)
 jumps.to_csv(output / "fes_step06_chern_jump_validation.csv", index=False)

 strict_audit, strict_summary = formula_audit(
 strict, "strict_step05", True, config
)
 coarse_audit, coarse_summary = formula_audit(
 coarse, "coarse_step04", False, config
)
 strict_audit.to_csv(
 output / "fes_step06_formula_validation_strict.csv", index=False
)
 coarse_audit.to_csv(
 output / "fes_step06_formula_validation_coarse.csv", index=False
)

 strict_mismatch = strict_audit[

```



---

```

 strict_audit["chern_formula_match"] == 0
].copy()
coarse_mismatch = coarse_audit[
 coarse_audit["chern_formula_match"] == 0
].copy()
strict_mismatch.to_csv(
 output / "fes_step06_formula_mismatches_strict.csv", index=False
)
coarse_mismatch.to_csv(
 output / "fes_step06_formula_mismatches_coarse.csv", index=False
)

prefactor = int(strict_summary["calibrated_prefactor"])
write_formula_files(output, prefactor)

plot_kp_winding(kp_summary, output)
plot_formula_confusion(
 strict_audit, output, "strict", "verified_chern_up_int"
)
plot_formula_confusion(
 coarse_audit, output, "coarse", "chern_up_int"
)

eta_match_fraction = float(
 kp_summary["eta_equals_sign_r1r2"].mean()
) if len(kp_summary) else np.nan
winding_consensus_fraction = float(
 kp_summary["winding_consensus"].mean()
) if len(kp_summary) else np.nan
jump_match_fraction = float(
 jumps["jump_matches_formula"].mean()
) if len(jumps) else np.nan

combined_nogo_count = 0
if not cert.empty:
 topological = cert.copy()
 formula_pred = []
 for _, row in topological.iterrows():
 formula_pred.append(
 candidate_chern_formula(
 reduced_from_row(row), prefactor,
 config.formula_mass_tol,
 config.formula_hopping_tol,
)
)
 topological["chern_formula_pred"] = formula_pred
 topological["formula_matches_verified"] = (
 topological["chern_formula_pred"]
 == pd.to_numeric(
 topological["verified_chern_up_int"], errors="coerce"
)
).astype(int)
 topological["complete_numerical_no_go_chain"] = (
 (topological["formula_matches_verified"] == 1)
 & (pd.to_numeric(
 topological["conditional_no_go_certificate"], errors="coerce"
) == 1)
).astype(int)
 combined_nogo_count = int(
 topological["complete_numerical_no_go_chain"].sum()
)
 topological.to_csv(
 output / "fes_step06_combined_no_go_certificates.csv", index=False
)

summary = {
 "code_version": CODE_VERSION,
 "representative_points": int(len(representatives)),
 "critical_roots_found": int((roots["root_found"] == 1).sum()),

```

```

 "kp_valleys_audited": int(len(kp_summary)),
 "winding_consensus_fraction": winding_consensus_fraction,
 "eta_equals_sign_r1r2_fraction": eta_match_fraction,
 "chern_jump_formula_match_fraction": jump_match_fraction,
 "calibrated_formula_prefactor": prefactor,
 "candidate_formula": (
 f"C_up = {prefactor:+d} * sgn(r1*r2)/2 * "
 "[sgn(mu_Gamma)+sgn(mu_M)]"
),
 "strict_formula_validation": strict_summary,
 "coarse_formula_validation": coarse_summary,
 "verified_topological_no_go_certificates": int(len(cert)),
 "combined_formula_plus_no_go_certificate_count": combined_nogo_count,
 "status": (
 "Low-energy k.p reconstruction and strict numerical formula audit "
 "completed. Publication-level analytic proof still requires an "
 "explicit symbolic derivation of the projected Jacobian signs."
),
}
(output / "fes_step06_summary.json").write_text(
 json.dumps(summary, ensure_ascii=False, indent=2),
 encoding="utf-8",
)

```

```

report = f""FES Step 06 summary
=====

```

```

Representative points: {len(representatives)}
Critical roots found: {(roots["root_found"] == 1).sum()}
k.p valleys audited: {len(kp_summary)}

```

```

Winding consensus fraction:
{winding_consensus_fraction}

```

```

eta_K = sgn(r1*r2) fraction:
{eta_match_fraction}

```

```

Chern-jump formula match fraction:
{jump_match_fraction}

```

```

Calibrated formula:
{summary["candidate_formula"]}

```

```

Strict formula accuracy:
all evaluated = {strict_summary["accuracy_all_evaluated"]}
robust = {strict_summary["accuracy_robust"]}

```

```

Coarse formula accuracy:
all evaluated = {coarse_summary["accuracy_all_evaluated"]}
robust = {coarse_summary["accuracy_robust"]}

```

```

Combined formula + Step 05 no-go certificates:
{combined_nogo_count}/{len(cert)}

```

```

Interpretation

```

The numerical low-energy reconstruction supports

$$C_{\text{up}} = \text{sgn}(r_1 r_2) / 2 \cdot [\text{sgn}(\mu_{\text{Gamma}}) + \text{sgn}(\mu_{\text{M}})].$$

Together with the exact conditional Step 05 result, every verified nonzero-spin-Chern point is certified to have  $E_g^{\text{ind}} \leq 0$ .

```

Remaining analytic task

```

Write the projected  $\Gamma/M$  Jacobian determinants explicitly and prove their signs are controlled by  $r_1 r_2$ . This upgrades the reconstructed formula to a publication-level symbolic theorem.

---

```

"""
 (output / "fes_step06_report.txt").write_text(report, encoding="utf-8")

 return {
 "output_dir": str(output),
 "representatives": str(output / "fes_step06_representative_points.csv"),
 "roots": str(output / "fes_step06_critical_mass_roots.csv"),
 "kp_coefficients": str(output / "fes_step06_kp_coefficients.csv"),
 "winding": str(output / "fes_step06_winding_by_radius.csv"),
 "chern_jumps": str(output / "fes_step06_chern_jump_validation.csv"),
 "strict_formula": str(output / "fes_step06_formula_validation_strict.csv"),
 "coarse_formula": str(output / "fes_step06_formula_validation_coarse.csv"),
 "combined_certificates": str(output / "fes_step06_combined_no_go_certificates.csv"),
 "summary": str(output / "fes_step06_summary.json"),
 "report": str(output / "fes_step06_report.txt"),
 }

=====
9. CLI
=====

def build_parser() -> argparse.ArgumentParser:
 parser = argparse.ArgumentParser(
 description="FES Step 06: low-energy k.p and spin-Chern formula"
)
 parser.add_argument(
 "--step05-input",
 default="outputs_fes6_step05_no_go.zip",
 help="Step 05 result directory or ZIP",
)
 parser.add_argument(
 "--output-dir",
 default="outputs_fes6_step06_kp",
)
 parser.add_argument("--workers", type=int, default=max(1, min(4, os.cpu_count() or 1)))
 parser.add_argument("--test", action="store_true")
 parser.add_argument("--overwrite", action="store_true")
 return parser

def config_from_args(args: argparse.Namespace) -> KPConfig:
 if args.test:
 return KPConfig(
 step05_input=args.step05_input,
 output_dir=args.output_dir,
 workers=1,
 representative_per_group=1,
 root_grid_points=801,
 fit_grid_n=7,
 winding_radii=(3.0e-4, 1.0e-3, 3.0e-3),
 winding_angles=361,
 chern_jump_grids=(15, 21),
 overwrite=args.overwrite,
 test_mode=True,
)
 return KPConfig(
 step05_input=args.step05_input,
 output_dir=args.output_dir,
 workers=max(1, int(args.workers)),
 overwrite=args.overwrite,
 test_mode=False,
)

def main() -> None:
 args = build_parser().parse_args()
 config = config_from_args(args)

```

```

print("=" * 78)
print("FES Step 06 — low-energy k.p and spin-Chern formula")
print("Version:", CODE_VERSION)
print("Step 05 input:", config.step05_input)
print("Output:", Path(config.output_dir).resolve())
print("=" * 78)

paths = run_pipeline(config)
print(json.dumps(paths, ensure_ascii=False, indent=2))

if __name__ == "__main__":
 main()

```

## ► 代码清单 A.47.6 (python, 874 行)

```

#!/usr/bin/env python3
-*- coding: utf-8 -*-
"""
FES Step 07 — 周期规范下的解析 Jacobian、局域 Chern 贡献与模型级 no-go

```

本步骤完成 Step 06 尚缺少的符号代数环节:

1. 从 fes 自旋向上  $4 \times 4$  周期规范 Hamiltonian 出发;
2. 在  $\Gamma/M$  点进行奇偶基变换, 显式分离 A 与 B 两个  $2 \times 2$  分支;
3. 写出 A- 与 B+ 的解析本征矢;
4. 解析计算  $\langle A- | \partial_{k_x} H | B+ \rangle$  与  $\langle A- | \partial_{k_y} H | B+ \rangle$ ;
5. 因式分解  $\Gamma/M$  的两带 Jacobian;
6. 证明两个谷的局域下占据带 Chern 贡献为

```

C_Gamma = sgn(r1*r2*mu_Gamma)/2
C_M = sgn(r1*r2*mu_M)/2

```

从而

```

C_up = sgn(r1*r2)/2 *
 [sgn(mu_Gamma)+sgn(mu_M)].

```

7. 与 Step 05 的精确条件性 no-go 合并:

```

C_up != 0
=> mu_Gamma 与 mu_M 同号
=> B+ 在 Γ/M 间发生 partner switching
=> $E_g^{\text{ind}} \leq -4|r_1^2 - r_2^2|/(B_1+B_2) \leq 0$.

```

注意

```

这一步给出显式符号推导。全局公式还使用一个已被 Steps 02–06
数值验证的模型事实: 在所研究的半填充、平衡自旋、直接开隙扇区内,
拓扑变化由 Γ/M 的 A—B+ 闭隙控制。
"""

```

```

from __future__ import annotations

from dataclasses import asdict, dataclass
from datetime import datetime, timezone
from pathlib import Path
from typing import Dict
import argparse
import hashlib
import json
import math
import os

```

---

```

import shutil
import zipfile

import numpy as np
import pandas as pd
import sympy as sp
import matplotlib.pyplot as plt

import FES_step02_sobol_scan as core
import FES_step05_analytic_no_go as step05

CODE_VERSION = "FES_STEP07_SYMBOLIC_JACOBIAN_V1_20260713"
RESULT_SCHEMA_VERSION = "fes_step07_symbolic_jacobian_schema_v1"

PARAMS5 = ("m_e", "t1", "t2", "r1", "r2")

@dataclass
class Step07Config:
 step05_input: str = "outputs_fes6_step05_no_go.zip"
 step06_input: str = "outputs_fes6_step06_kp.zip"
 output_dir: str = "outputs_fes6_step07_symbolic"
 finite_difference_step: float = 1.0e-6
 symbolic_timeout_note: str = "SymPy expressions are factorized without external CAS."
 overwrite: bool = False
 test_mode: bool = False

=====
1. I/O
=====

def script_sha256() -> str | None:
 if "__file__" not in globals():
 return None
 p = Path(__file__).resolve()
 return hashlib.sha256(p.read_bytes()).hexdigest() if p.exists() else None

def prepare_output(config: Step07Config) -> Path:
 out = Path(config.output_dir).expanduser().resolve()
 if out.exists() and config.overwrite:
 shutil.rmtree(out)
 out.mkdir(parents=True, exist_ok=True)
 return out

def _find_root(root: Path, required_name: str) -> Path:
 direct = root / required_name
 if direct.exists():
 return root
 found = list(root.rglob(required_name))
 if not found:
 raise FileNotFoundError(f"Cannot find {required_name} under {root}")
 return sorted((p.parent for p in found), key=lambda x: len(x.parts))[0]

def locate_result(input_path: str | Path, output: Path, required_name: str, tag: str) -> Path:
 path = Path(input_path).expanduser().resolve()
 if not path.exists():
 raise FileNotFoundError(path)
 if path.is_dir():
 return _find_root(path, required_name)
 if path.suffix.lower() != ".zip":
 raise ValueError(f"{tag} input must be a directory or ZIP")
 extract = output / f"_{tag}_extracted"
 if extract.exists():
 shutil.rmtree(extract)
 extract.mkdir(parents=True)

```

---

```

with zipfile.ZipFile(path, "r") as zf:
 zf.extractall(extract)
return _find_root(extract, required_name)

def write_metadata(config: Step07Config, output: Path, step05_dir: Path, step06_dir: Path) -> None:
 data = {
 "code_version": CODE_VERSION,
 "result_schema_version": RESULT_SCHEMA_VERSION,
 "run_time_utc": datetime.now(timezone.utc).isoformat(),
 "script_sha256": script_sha256(),
 "step02_core_version": getattr(core, "CODE_VERSION", None),
 "step05_core_version": getattr(step05, "CODE_VERSION", None),
 "step05_directory": str(step05_dir),
 "step06_directory": str(step06_dir),
 "config": asdict(config),
 }
 (output / "fes_step07_run_metadata.json").write_text(
 json.dumps(data, ensure_ascii=False, indent=2), encoding="utf-8"
)

=====
2. Symbolic periodic Hamiltonian and parity basis
=====

def symbolic_derivation() -> Dict[str, object]:
 """Fast exact derivation using the parity-basis blocks directly."""
 m, t1, t2, r1, r2 = sp.symbols(
 "m_e t_1 t_2 r_1 r_2", real=True
)
 kx, ky = sp.symbols("k_x k_y", real=True)
 rp = r2 + sp.I*r1
 rm = r2 - sp.I*r1

 # Exact spin-up Hamiltonian in the periodic gauge.
 H = sp.Matrix([
 [m, t1, rp*sp.exp(sp.I*ky), rm],
 [t1, m, rm*sp.exp(-sp.I*kx+sp.I*ky), rp*sp.exp(-sp.I*kx)],
 [rm*sp.exp(-sp.I*ky), rp*sp.exp(sp.I*kx-sp.I*ky), -m, t2],
 [rp, rm*sp.exp(sp.I*kx), t2, -m],
])

 ts = (t1+t2)/2
 td = (t1-t2)/2
 u = m+td
 v = m-td
 A1 = sp.sqrt(v**2+4*r1**2)
 A2 = sp.sqrt(v**2+4*r2**2)
 B1 = sp.sqrt(u**2+4*r1**2)
 B2 = sp.sqrt(u**2+4*r2**2)

 # Parity-basis Hamiltonians. Order: (B top, B bottom, A top, A bottom).
 HG = sp.Matrix([
 [m+t1, 2*r2, 0, 0],
 [2*r2, -m+t2, 0, 0],
 [0, 0, m-t1, 2*sp.I*r1],
 [0, 0, -2*sp.I*r1, -m-t2],
])
 HM = sp.Matrix([
 [m+t1, -2*sp.I*r1, 0, 0],
 [2*sp.I*r1, -m+t2, 0, 0],
 [0, 0, m-t1, -2*r2],
 [0, 0, -2*r2, -m-t2],
])

 # A-to-B derivative blocks <A|dH|B>.
 XG = sp.Matrix([[0, sp.I*r2], [-r1, 0]])
 YG = sp.Matrix([[0, -r1], [-sp.I*r2, 0]])

```

---

```

XM = sp.Matrix([[0, r1], [sp.I*r2, 0]])
YM = sp.Matrix([[0, -sp.I*r2], [r1, 0]])

Explicit unnormalized eigenvectors in the A and B blocks.
aG = sp.Matrix([2*sp.I*r1, -(A1+v)])
bG = sp.Matrix([2*r2, B2-u])
NAG2 = 4*r1**2+(A1+v)**2
NBG2 = 4*r2**2+(B2-u)**2

aM = sp.Matrix([-2*r2, -(A2+v)])
bM = sp.Matrix([-2*sp.I*r1, B1-u])
NAM2 = 4*r2**2+(A2+v)**2
NBM2 = 4*r1**2+(B1-u)**2

Direct numerator algebra; all square-root factors are real.
vxG_num = sp.expand((sp.conjugate(aG).T*XG*bG)[0])
vyG_num = sp.expand((sp.conjugate(aG).T*YG*bG)[0])
vxM_num = sp.expand((sp.conjugate(aM).T*XM*bM)[0])
vyM_num = sp.expand((sp.conjugate(aM).T*YM*bM)[0])

SG = A1+v+B2-u
TG = r1**2*(B2-u)+r2**2*(A1+v)
SM = A2+v+B1-u
TM = r2**2*(B1-u)+r1**2*(A2+v)

vxG_expected_num = 2*r1*r2*SG
vyG_expected_num = 2*sp.I*TG
vxM_expected_num = -2*r1*r2*SM
vyM_expected_num = 2*sp.I*TM

residuals = {
 "vx_Gamma_numerator": sp.expand(vxG_num-vxG_expected_num),
 "vy_Gamma_numerator": sp.expand(vyG_num-vyG_expected_num),
 "vx_M_numerator": sp.expand(vxM_num-vxM_expected_num),
 "vy_M_numerator": sp.expand(vyM_num-vyM_expected_num),
}

vxG = vxG_expected_num/sp.sqrt(NAG2*NBG2)
vyG = vyG_expected_num/sp.sqrt(NAG2*NBG2)
vxM = vxM_expected_num/sp.sqrt(NAM2*NBM2)
vyM = vyM_expected_num/sp.sqrt(NAM2*NBM2)
detG_expected = -4*r1*r2*SG*TG/(NAG2*NBG2)
detM_expected = 4*r1*r2*SM*TM/(NAM2*NBM2)

return {
 "m": m, "t1": t1, "t2": t2, "r1": r1, "r2": r2,
 "kx": kx, "ky": ky, "H": H, "HG": HG, "HM": HM,
 "ts": ts, "td": td, "u": u, "v": v,
 "A1": A1, "A2": A2, "B1": B1, "B2": B2,
 "XG": XG, "YG": YG, "XM": XM, "YM": YM,
 "aG": aG, "bG": bG, "NAG2": NAG2, "NBG2": NBG2,
 "SG": SG, "TG": TG, "vxG": vxG, "vyG": vyG,
 "detG_expected": detG_expected,
 "aM": aM, "bM": bM, "NAM2": NAM2, "NBM2": NBM2,
 "SM": SM, "TM": TM, "vxM": vxM, "vyM": vyM,
 "detM_expected": detM_expected,
 "residuals": residuals,
}

=====
4. Write symbolic proof
=====

def _s(expr: sp.Expr) -> str:
 return sp.sstr(expr)

def _l(expr: sp.Expr) -> str:

```

---

```

 return sp.latex(expr)

def write_symbolic_outputs(output: Path, d: Dict[str, object]) -> None:
 residuals = d["residuals"]
 residual_ok = all(sp.simplify(v) == 0 for v in residuals.values())

 text = f"""FES Step 07 — explicit symbolic Jacobian proof
=====

Periodic-gauge spin-up Hamiltonian

H_up(k) =
{sp.pretty(d["H"])}

Parity-basis matrices

H_Gamma =
{sp.pretty(d["HG"])}

H_M =
{sp.pretty(d["HM"])}

Definitions

t_s=(t1+t2)/2
t_d=(t1-t2)/2
u=m_e+t_d
v=m_e-t_d

A1=sqrt(v^2+4 r1^2)
A2=sqrt(v^2+4 r2^2)
B1=sqrt(u^2+4 r1^2)
B2=sqrt(u^2+4 r2^2)

Gamma projected velocities

v_x,Gamma = {_s(d["vxG"])}
v_y,Gamma = {_s(d["vyG"])}

det J_Gamma = {_s(d["detG_expected"])}

Because
 S_Gamma=A1+v+B2-u > 0
 T_Gamma=r1^2(B2-u)+r2^2(A1+v) > 0
away from the singular r1=r2=0 limit,

 sgn(det J_Gamma) = -sgn(r1*r2).

In the ordered basis (A-,B+),
 d_z,Gamma = mu_Gamma/2.

For the lower band of a massive two-band crossing,
 C_Gamma = -sgn(detJ_Gamma*mu_Gamma)/2
 = sgn(r1*r2*mu_Gamma)/2.

M projected velocities

v_x,M = {_s(d["vxM"])}
v_y,M = {_s(d["vyM"])}

det J_M = {_s(d["detM_expected"])}

Because
 S_M=A2+v+B1-u > 0
 T_M=r2^2(B1-u)+r1^2(A2+v) > 0,

 sgn(det J_M) = +sgn(r1*r2).

```



In the same ordered basis (A<sup>-</sup>, B<sup>+</sup>),  
 $d_{z,M} = -\mu_M/2$ .

Therefore

$$\begin{aligned} C_M &= -\text{sgn}(\det J_M(-\mu_M))/2 \\ &= \text{sgn}(r_1 r_2 \mu_M)/2. \end{aligned}$$

Closed formula

$$\begin{aligned} C_{\text{up}} &= C_{\text{Gamma}} + C_M \\ &= \text{sgn}(r_1 r_2)/2 * \\ &\quad [\text{sgn}(\mu_{\text{Gamma}}) + \text{sgn}(\mu_M)]. \end{aligned}$$

$C_{\text{down}} = -C_{\text{up}}$  and  $C_{\text{total}} = 0$ .

Combined model-level no-go

-----  
 $C_{\text{up}} \neq 0$   
 $\Rightarrow \mu_{\text{Gamma}}$  and  $\mu_M$  have the same sign  
 $\Rightarrow$  the B<sup>+</sup> branch exchanges occupied/unoccupied rank between Gamma and M  
 $\Rightarrow E_{\text{g}}^{\text{ind}} \leq -4|r_1^2 - r_2^2|/(B_1 + B_2) \leq 0$ .

Symbolic residuals all zero: {residual\_ok}

Residuals:

```
{json.dumps({k: _s(v) for k, v in residuals.items()}, indent=2)}
"""
 (output / "fes_step07_symbolic_proof.txt").write_text(text, encoding="utf-8")
```

```
tex = rf"""\documentclass{{article}}
\usepackage{{amsmath,amssymb,bm}}
\begin{{document}}
\section*{{Analytic Jacobian and spin-Chern formula for the fes model}}
```

Define

```
\[
t_s=\frac{{t_1+t_2}}{{2}},\quad
t_d=\frac{{t_1-t_2}}{{2}},\quad
u=m_e+t_d,\quad v=m_e-t_d,
\]
\[
A_1=\sqrt{{v^2+4r_1^2}},\quad
A_2=\sqrt{{v^2+4r_2^2}},\quad
B_1=\sqrt{{u^2+4r_1^2}},\quad
B_2=\sqrt{{u^2+4r_2^2}}.
\]
```

At  $\Gamma$ , in the (A<sup>-</sup>, B<sup>+</sup>) basis,

```
\[
\langle A^- | \partial_{k_x} H | B^+ \rangle
=
\langle \partial_{k_x} G \rangle,
\]
\[
\langle A^- | \partial_{k_y} H | B^+ \rangle
=
\langle \partial_{k_y} G \rangle.
\]
```

Hence

```
\[
\det J_{\Gamma} =
\langle \det G_{\text{expected}} \rangle,
\quad
\operatorname{sgn}(\det J_{\Gamma})
= -\operatorname{sgn}(r_1 r_2).
\]
```

Since  $d_{z,\Gamma} = \mu_{\Gamma}/2$ ,

```
\[
C_{\Gamma} =
-\frac{12 \operatorname{sgn}(\det J_{\Gamma} \mu_{\Gamma})
```

---

```

=
\frac{1}{2}\operatorname{sgn}(r_1r_2\mu_\Gamma).
\]

At M,
\[
\langle A^-\partial_{k_x}H|B^+\rangle
=
\langle \partial_{v_x}M\rangle,
\]
\[
\langle A^-\partial_{k_y}H|B^+\rangle
=
\langle \partial_{v_y}M\rangle.
\]
Thus
\[
\det J_M=
\langle \det M_{\text{expected}}\rangle,
\]
\quad
\operatorname{sgn}(\det J_M)
=+\operatorname{sgn}(r_1r_2).
\]
Because $d_{z,M}=-\mu_M/2$,
\[
C_M=
-\frac{1}{2}\operatorname{sgn}(\det J_M(-\mu_M))
=
\frac{1}{2}\operatorname{sgn}(r_1r_2\mu_M).
\]

Therefore
\[
\boxed{C_{\uparrow}=
\frac{\operatorname{sgn}(r_1r_2)}{2}
\left[
\operatorname{sgn}(\mu_\Gamma)+
\operatorname{sgn}(\mu_M)
\right]
}.
\]
Moreover,
\[
C_{\uparrow}\neq 0
\]
\Longrightarrow
E_g^{\text{ind}}
\le
-\frac{4|r_1^2-r_2^2|}{B_1+B_2}
\le 0.
\]
\end{document}
"""

(output / "fes_step07_symbolic_proof.tex").write_text(tex, encoding="utf-8")

json_data = {
 "symbolic_residuals_zero": residual_ok,
 "residuals": {k: _s(v) for k, v in residuals.items()},
 "vx_Gamma": _s(d["vxG"]),
 "vy_Gamma": _s(d["vyG"]),
 "detJ_Gamma": _s(d["detG_expected"]),
 "vx_M": _s(d["vxM"]),
 "vy_M": _s(d["vyM"]),
 "detJ_M": _s(d["detM_expected"]),
 "chern_formula": (
 "C_up=sgn(r1*r2)/2*[sgn(mu_Gamma)+sgn(mu_M)]"
),
}
(output / "fes_step07_symbolic_proof.json").write_text(

```

---

```

 json.dumps(json_data, ensure_ascii=False, indent=2), encoding="utf-8"
)

=====
5. Numerical validation of explicit symbolic velocities
=====

def reduced_from_row(row: pd.Series) -> Dict[str, float]:
 return {k: float(row[k]) for k in PARAMS5}

def periodic_spin_up(kx: float, ky: float, p: Dict[str, float]) -> np.ndarray:
 raw = core.raw6_from_reduced5(p)
 return core.h_spin_block_periodic(kx, ky, raw, "up")

def explicit_basis_numeric(p: Dict[str, float], valley: str) -> tuple[np.ndarray, np.ndarray]:
 m, t1, t2, r1, r2 = [p[k] for k in PARAMS5]
 ts = 0.5*(t1+t2)
 td = 0.5*(t1-t2)
 u = m + td
 v = m - td

 s2 = math.sqrt(2.0)
 P = np.array([
 [1/s2, 0, 1/s2, 0],
 [1/s2, 0, -1/s2, 0],
 [0, 1/s2, 0, 1/s2],
 [0, 1/s2, 0, -1/s2],
], dtype=np.complex128)

 if valley == "Gamma":
 A = math.sqrt(v*v + 4*r1*r1)
 B = math.sqrt(u*u + 4*r2*r2)
 a = np.array([2j*r1, -(A+v)], dtype=np.complex128)
 b = np.array([2*r2, B-u], dtype=np.complex128)
 elif valley == "M":
 A = math.sqrt(v*v + 4*r2*r2)
 B = math.sqrt(u*u + 4*r1*r1)
 a = np.array([-2*r2, -(A+v)], dtype=np.complex128)
 b = np.array([-2j*r1, B-u], dtype=np.complex128)
 else:
 raise ValueError(valley)

 a /= np.linalg.norm(a)
 b /= np.linalg.norm(b)
 Avec = P @ np.array([0, 0, a[0], a[1]], dtype=np.complex128)
 Bvec = P @ np.array([b[0], b[1], 0, 0], dtype=np.complex128)
 return Avec, Bvec

def analytic_velocity_numeric(p: Dict[str, float], valley: str) -> tuple[complex, complex, float]:
 m, t1, t2, r1, r2 = [p[k] for k in PARAMS5]
 td = 0.5*(t1-t2)
 u = m + td
 v = m - td

 if valley == "Gamma":
 A = math.sqrt(v*v + 4*r1*r1)
 B = math.sqrt(u*u + 4*r2*r2)
 NA2 = 4*r1*r1 + (A+v)**2
 NB2 = 4*r2*r2 + (B-u)**2
 S = A+v+B-u
 T = r1*r1*(B-u) + r2*r2*(A+v)
 vx = 2*r1*r2*S/math.sqrt(NA2*NB2)
 vy = 2j*T/math.sqrt(NA2*NB2)
 det = -4*r1*r2*S*T/(NA2*NB2)
 else:

```

```

A = math.sqrt(v*v + 4*r2*r2)
B = math.sqrt(u*u + 4*r1*r1)
NA2 = 4*r2*r2 + (A+v)**2
NB2 = 4*r1*r1 + (B-u)**2
S = A+v+B-u
T = r2*r2*(B-u) + r1*r1*(A+v)
vx = -2*r1*r2*S/math.sqrt(NA2*NB2)
vy = 2*j*T/math.sqrt(NA2*NB2)
det = 4*r1*r2*S*T/(NA2*NB2)
return complex(vx), complex(vy), float(det)

```

```

def validate_velocities(step06_dir: Path, h: float) -> pd.DataFrame:
 roots = pd.read_csv(step06_dir / "fes_step06_critical_mass_roots.csv")
 roots = roots[pd.to_numeric(roots["root_found"], errors="coerce") == 1].copy()
 rows = []

```

```

for _, row in roots.iterrows():
 p = reduced_from_row(row)
 p["m_e"] = float(row["m_critical"])
 valley = str(row["valley"])
 k0 = (0.0, 0.0) if valley == "Gamma" else (math.pi, math.pi)
 A, B = explicit_basis_numeric(p, valley)

 dHdx = (
 periodic_spin_up(k0[0]+h, k0[1], p)
 - periodic_spin_up(k0[0]-h, k0[1], p)
)/(2*h)
 dHdy = (
 periodic_spin_up(k0[0], k0[1]+h, p)
 - periodic_spin_up(k0[0], k0[1]-h, p)
)/(2*h)

 vx_num = np.vdot(A, dHdx @ B)
 vy_num = np.vdot(A, dHdy @ B)
 vx_ana, vy_ana, det_ana = analytic_velocity_numeric(p, valley)
 det_num = (
 np.real(vx_num)*(-np.imag(vy_num))
 - np.real(vy_num)*(-np.imag(vx_num))
)

 rows.append({
 "point_id": row["point_id"],
 "valley": valley,
 **p,
 "vx_numeric_real": np.real(vx_num),
 "vx_numeric_imag": np.imag(vx_num),
 "vx_analytic_real": np.real(vx_ana),
 "vx_analytic_imag": np.imag(vx_ana),
 "vx_abs_error": abs(vx_num-vx_ana),
 "vy_numeric_real": np.real(vy_num),
 "vy_numeric_imag": np.imag(vy_num),
 "vy_analytic_real": np.real(vy_ana),
 "vy_analytic_imag": np.imag(vy_ana),
 "vy_abs_error": abs(vy_num-vy_ana),
 "det_numeric": det_num,
 "det_analytic": det_ana,
 "det_abs_error": abs(det_num-det_ana),
 "det_sign_match": int(np.sign(det_num) == np.sign(det_ana)),
 })
return pd.DataFrame(rows)

```

```

=====
6. Global formula and no-go audit
=====

```

```

def formula_from_row(row: pd.Series) -> float:
 rprod = float(row["r1"])*float(row["r2"])

```

---

```

muG = float(row["mass_Gamma_signed"])
muM = float(row["mass_M_signed"])
if abs(rprod) < 1e-14 or abs(muG) < 1e-12 or abs(muM) < 1e-12:
 return np.nan
return float(
 np.sign(rprod)*0.5*(np.sign(muG)+np.sign(muM))
)

def audit_strict(step05_dir: Path) -> tuple[pd.DataFrame, Dict[str, object]]:
 df = pd.read_csv(step05_dir / "fes_step05_strict_analytic_audit.csv")
 reliable = df[
 (pd.to_numeric(df["verified_chern_reliable"], errors="coerce") == 1)
 & (pd.to_numeric(df["verified_is_direct_gapped"], errors="coerce") == 1)
 & (pd.to_numeric(df["verified_is_balanced_spin_sector"], errors="coerce") == 1)
].copy()
 reliable["chern_up_symbolic_formula"] = [
 formula_from_row(row) for _, row in reliable.iterrows()
]
 reliable["formula_match"] = (
 reliable["chern_up_symbolic_formula"]
 == pd.to_numeric(reliable["verified_chern_up_int"], errors="coerce")
).astype(int)
 valid = reliable[np.isfinite(reliable["formula_match"])]
 summary = {
 "strict_reliable_rows": int(len(reliable)),
 "formula_evaluated_rows": int(len(valid)),
 "formula_accuracy": float(valid["formula_match"].mean()) if len(valid) else np.nan,
 "formula_mismatch_count": int((valid["formula_match"] == 0).sum()),
 }
 return reliable, summary

def build_theorem_certificate(step05_dir: Path, strict_audit: pd.DataFrame) -> pd.DataFrame:
 cert = pd.read_csv(
 step05_dir / "fes_step05_verified_topological_no_go_certificates.csv"
)
 if cert.empty:
 return cert

 cols = [
 "point_id", "chern_up_symbolic_formula", "formula_match"
]
 merged = cert.merge(
 strict_audit[[c for c in cols if c in strict_audit.columns]],
 on="point_id", how="left"
)
 merged["symbolic_formula_plus_conditional_nogo"] = (
 (pd.to_numeric(merged["formula_match"], errors="coerce") == 1)
 & (
 pd.to_numeric(
 merged["conditional_no_go_certificate"], errors="coerce"
) == 1
)
).astype(int)
 return merged

=====
7. Plots
=====

def plot_velocity_validation(df: pd.DataFrame, output: Path) -> None:
 if df.empty:
 return
 fig, ax = plt.subplots(figsize=(6, 5))
 ax.scatter(df["det_analytic"], df["det_numeric"])
 lo = min(df["det_analytic"].min(), df["det_numeric"].min())
 hi = max(df["det_analytic"].max(), df["det_numeric"].max())

```

---

```

ax.plot([lo, hi], [lo, hi], linestyle="--")
ax.set_xlabel("analytic det J")
ax.set_ylabel("finite-difference det J")
ax.set_title("Explicit symbolic Jacobian validation")
fig.tight_layout()
fig.savefig(output / "fes_step07_jacobian_validation.png", dpi=220)
plt.close(fig)

=====
8. Main pipeline
=====

def run_pipeline(config: Step07Config) -> Dict[str, str]:
 output = prepare_output(config)
 step05_dir = locate_result(
 config.step05_input, output,
 "fes_step05_strict_analytic_audit.csv", "step05"
)
 step06_dir = locate_result(
 config.step06_input, output,
 "fes_step06_critical_mass_roots.csv", "step06"
)
 write_metadata(config, output, step05_dir, step06_dir)

 derivation = symbolic_derivation()
 write_symbolic_outputs(output, derivation)

 velocity = validate_velocities(
 step06_dir, config.finite_difference_step
)
 velocity.to_csv(
 output / "fes_step07_symbolic_velocity_validation.csv", index=False
)
 plot_velocity_validation(velocity, output)

 strict, strict_summary = audit_strict(step05_dir)
 strict.to_csv(
 output / "fes_step07_strict_formula_audit.csv", index=False
)

 theorem = build_theorem_certificate(step05_dir, strict)
 theorem.to_csv(
 output / "fes_step07_model_level_no_go_certificates.csv", index=False
)

 residual_ok = all(
 sp.simplify(v) == 0 for v in derivation["residuals"].values()
)
 velocity_ok = bool(
 len(velocity)
 and velocity["det_sign_match"].all()
 and velocity["vx_abs_error"].max() < 1e-8
 and velocity["vy_abs_error"].max() < 1e-8
)
 theorem_count = int(
 theorem["symbolic_formula_plus_conditional_nogo"].sum()
) if len(theorem) else 0

 summary = {
 "code_version": CODE_VERSION,
 "symbolic_residuals_all_zero": bool(residual_ok),
 "velocity_validation_rows": int(len(velocity)),
 "max_vx_abs_error": float(velocity["vx_abs_error"].max()) if len(velocity) else np.nan,
 "max_vy_abs_error": float(velocity["vy_abs_error"].max()) if len(velocity) else np.nan,
 "max_det_abs_error": float(velocity["det_abs_error"].max()) if len(velocity) else np.nan,
 "velocity_validation_passed": velocity_ok,
 "strict_formula_audit": strict_summary,
 "verified_topological_points": int(len(theorem)),
 }

```

```

"model_level_no_go_certificate_count": theorem_count,
"model_level_no_go_certificate_fraction": (
 float(theorem_count/len(theorem)) if len(theorem) else np.nan
),
"derived_formula": (
 "C_up=sgn(r1*r2)/2*[sgn(mu_Gamma)+sgn(mu_M)]"
),
"derived_no_go": (
 "C_up!=0 => E_g^ind <= "
 "-4|r1^2-r2^2|/(B1+B2) <= 0"
),
"scope_note": (
 "The local Jacobian and valley Chern contributions are symbolic. "
 "The global use of only Gamma/M closures is supported by the "
 "Steps 02-06 phase-boundary and Chern-jump audits."
),
}
(output / "fes_step07_summary.json").write_text(
 json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8"
)

```

```

report = f"""FES Step 07 summary
=====

```

```

Symbolic residuals all zero:
{residual_ok}

```

```

Velocity/Jacobian validation:
rows = {len(velocity)}
max |delta vx| = {summary["max_vx_abs_error"]}
max |delta vy| = {summary["max_vy_abs_error"]}
max |delta det J| = {summary["max_det_abs_error"]}
passed = {velocity_ok}

```

```

Strict spin-Chern formula:
evaluated rows = {strict_summary["formula_evaluated_rows"]}
accuracy = {strict_summary["formula_accuracy"]}
mismatches = {strict_summary["formula_mismatch_count"]}

```

```

Model-level no-go certificates:
{theorem_count}/{len(theorem)}

```

```

Derived result

C_up = sgn(r1*r2)/2 *
 [sgn(mu_Gamma)+sgn(mu_M)].

```

```

At Gamma:
sgn(det J_Gamma) = -sgn(r1*r2),
d_z = mu_Gamma/2.

```

```

At M:
sgn(det J_M) = +sgn(r1*r2),
d_z = -mu_M/2.

```

```

Therefore each valley contributes
C_K = sgn(r1*r2*mu_K)/2.

```

```

Combining with the exact Step 05 branch-switching identity gives
C_up != 0
=> E_g^ind <= -4|r1^2-r2^2|/(B1+B2) <= 0.

```

```

Interpretation

Within the half-filled, spin-conserving minimal six-parameter fes
Hamiltonian, the nontrivial spin-Chern phase is analytically forced
to be a spin-Chern band metal rather than a globally insulating
type-II QSH phase, provided the only relevant topological closures
are the Gamma/M A-minus-B-plus crossings verified in Steps 02-06.

```

---

```

"""
 (output / "fes_step07_report.txt").write_text(report, encoding="utf-8")

 return {
 "output_dir": str(output),
 "symbolic_proof_txt": str(output / "fes_step07_symbolic_proof.txt"),
 "symbolic_proof_tex": str(output / "fes_step07_symbolic_proof.tex"),
 "velocity_validation": str(
 output / "fes_step07_symbolic_velocity_validation.csv"
),
 "strict_formula_audit": str(
 output / "fes_step07_strict_formula_audit.csv"
),
 "no_go_certificates": str(
 output / "fes_step07_model_level_no_go_certificates.csv"
),
 "summary": str(output / "fes_step07_summary.json"),
 "report": str(output / "fes_step07_report.txt"),
 }

=====
9. CLI
=====

def parser() -> argparse.ArgumentParser:
 p = argparse.ArgumentParser(
 description="FES Step 07 symbolic Jacobian and model-level no-go"
)
 p.add_argument(
 "--step05-input",
 default="outputs_fes6_step05_no_go.zip"
)
 p.add_argument(
 "--step06-input",
 default="outputs_fes6_step06_kp.zip"
)
 p.add_argument(
 "--output-dir",
 default="outputs_fes6_step07_symbolic"
)
 p.add_argument("--test", action="store_true")
 p.add_argument("--overwrite", action="store_true")
 return p

def main() -> None:
 args = parser().parse_args()
 config = Step07Config(
 step05_input=args.step05_input,
 step06_input=args.step06_input,
 output_dir=args.output_dir,
 overwrite=args.overwrite,
 test_mode=args.test,
)

 print("="*78)
 print("FES Step 07 — symbolic Jacobian and model-level no-go")
 print("Version:", CODE_VERSION)
 print("Output :", Path(config.output_dir).resolve())
 print("="*78)

 paths = run_pipeline(config)
 print(json.dumps(paths, ensure_ascii=False, indent=2))

if __name__ == "__main__":
 main()

```



## ► 代码清单 A.47.7 (python, 1058 行)

```
#!/usr/bin/env python3
-*- coding: utf-8 -*-
"""
Generate a Lieb-style four-quadrant signed-mass phase map for the FES model.

Axes

x = M_Gamma^FES = mass_Gamma_signed
y = M_M^FES = mass_M_signed

Formula

C_up = eta * chi_FES / 2
 * [sgn(M_Gamma^FES) + sgn(M_M^FES)]

chi_FES = sgn(r1*r2).

The program calibrates eta from the strict FES Step07/Step06 audit.
For the current frozen Step07 result, eta = +1.

Important

The nonzero FES regions are formal spin-Chern sectors. Under the minimal-model
indirect-gap no-go, they must not be called QSH-insulator regions.
"""

from __future__ import annotations

import argparse
import json
import zipfile
from dataclasses import dataclass
from pathlib import Path
from typing import Iterable, Sequence

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from matplotlib import font_manager, rcParams
from matplotlib.lines import Line2D
from matplotlib.patches import Patch

SCRIPT_VERSION = "FES_LIEB_STYLE_FOUR_QUADRANT_20260729"

STRICT_TABLES = (
 "fes_step07_strict_formula_audit.csv",
 "fes_step06_formula_validation_strict.csv",
 "fes_step05_strict_analytic_audit.csv",
)

COARSE_TABLES = (
 "fes_step06_formula_validation_coarse.csv",
 "fes_step05_all_coarse_analytic_audit.csv",
)

SUMMARY_JSONS = (
 "fes_step07_run_summary.json",
 "fes_step06_run_summary.json",
)

REQUIRED_COLUMNS = (
 "mass_Gamma_signed",
 "mass_M_signed",
 "r1",
```

---

```

 "r2",
)

Exact colors used by the Lieb paper-style figure.
COLOR_RED = (169 / 255.0, 36 / 255.0, 37 / 255.0)
COLOR_BLUE = (63 / 255.0, 99 / 255.0, 173 / 255.0)
COLOR_GREY = (211 / 255.0, 211 / 255.0, 211 / 255.0)

def lighten(
 color: tuple[float, float, float],
 factor: float,
) -> tuple[float, float, float]:
 base = np.asarray(color, dtype=float)
 return tuple(np.clip(base * (1.0 - factor) + factor, 0.0, 1.0))

REGION_FILL = {
 +1: lighten(COLOR_RED, 0.70),
 0: lighten(COLOR_GREY, 0.15),
 -1: lighten(COLOR_BLUE, 0.68),
}

POINT_EDGE = {
 +1: COLOR_RED,
 0: (0.25, 0.25, 0.25),
 -1: COLOR_BLUE,
}

def configure_plot_style() -> None:
 available = {item.name for item in font_manager.fontManager.ttflist}
 font_name = (
 "Times New Roman"
 if "Times New Roman" in available
 else "DejaVu Serif"
)

 rcParams["font.family"] = font_name
 rcParams["mathtext.fontset"] = "stix"
 rcParams["font.size"] = 9
 rcParams["axes.labelsize"] = 9
 rcParams["axes.titlesize"] = 9
 rcParams["xtick.labelsize"] = 8
 rcParams["ytick.labelsize"] = 8
 rcParams["legend.fontsize"] = 8
 rcParams["axes.linewidth"] = 0.8
 rcParams["xtick.direction"] = "in"
 rcParams["ytick.direction"] = "in"
 rcParams["xtick.top"] = True
 rcParams["ytick.right"] = True
 rcParams["svg.fonttype"] = "none"
 rcParams["pdf.fonttype"] = 42
 rcParams["ps.fonttype"] = 42
 rcParams["axes.unicode_minus"] = False

@dataclass
class DataSource:
 path: Path

 def __post_init__(self) -> None:
 self.path = Path(self.path).expanduser().resolve()
 if not self.path.exists():
 raise FileNotFoundError(self.path)
 if self.path.is_file() and self.path.suffix.lower() != ".zip":
 raise ValueError(
 f"Input must be a ZIP or an extracted directory: {self.path}"
)

```

---

```

@property
def is_zip(self) -> bool:
 return self.path.is_file()

def names(self) -> list[str]:
 if self.is_zip:
 with zipfile.ZipFile(self.path) as archive:
 return [
 name
 for name in archive.namelist()
 if not name.endswith("/")
]

 return [
 str(path.relative_to(self.path))
 for path in self.path.rglob("*")
 if path.is_file()
]

def find_basename(self, candidates: Sequence[str]) -> str | None:
 names = self.names()
 for basename in candidates:
 matches = [
 name for name in names
 if Path(name).name == basename
]
 if matches:
 return sorted(
 matches,
 key=lambda item: (
 len(Path(item).parts),
 len(item),
),
)[0]
 return None

def read_csv(
 self,
 candidates: Sequence[str],
 required: bool,
) -> tuple[pd.DataFrame | None, str | None]:
 selected = self.find_basename(candidates)
 if selected is None:
 if required:
 raise FileNotFoundError(
 f"No compatible CSV was found in {self.path}.\n"
 f"Expected one of: {list(candidates)}"
)
 return None, None

 if self.is_zip:
 with zipfile.ZipFile(self.path) as archive:
 with archive.open(selected) as stream:
 return (
 pd.read_csv(stream, low_memory=False),
 Path(selected).name,
)

 return (
 pd.read_csv(self.path / selected, low_memory=False),
 Path(selected).name,
)

def read_json_optional(
 self,
 candidates: Sequence[str],
) -> tuple[dict, str | None]:
 selected = self.find_basename(candidates)

```

---

```

 if selected is None:
 return {}, None

 if self.is_zip:
 with zipfile.ZipFile(self.path) as archive:
 return (
 json.loads(
 archive.read(selected).decode("utf-8")
),
 Path(selected).name,
)

 return (
 json.loads(
 (self.path / selected).read_text(encoding="utf-8")
),
 Path(selected).name,
)

def discover_fes_input(root: Path | str = ".") -> Path:
 root = Path(root).expanduser().resolve()
 patterns = (
 "outputs_fes6_step07_symbolic*.zip",
 "outputs_fes6_step07_symbolic*",
 "outputs_fes6_step06_kp*.zip",
 "outputs_fes6_step06_kp*",
 "outputs_fes6_step05_no_go*.zip",
 "outputs_fes6_step05_no_go*",
)

 candidates: list[Path] = []
 for pattern in patterns:
 candidates.extend(root.rglob(pattern))

 valid: list[Path] = []
 for candidate in candidates:
 try:
 source = DataSource(candidate)
 except (OSError, ValueError, zipfile.BadZipFile):
 continue
 if source.find_basename(STRICT_TABLES) is not None:
 valid.append(candidate)

 if not valid:
 raise FileNotFoundError(
 "No FES Step07/Step06/Step05 output was found. "
 "Please set input_source explicitly."
)

 return max(valid, key=lambda path: path.stat().st_mtime)

def numeric(frame: pd.DataFrame, column: str) -> pd.Series:
 return pd.to_numeric(frame[column], errors="coerce")

def require_columns(
 frame: pd.DataFrame,
 columns: Iterable[str],
 table_name: str,
) -> None:
 missing = [column for column in columns if column not in frame.columns]
 if missing:
 raise KeyError(
 f"{table_name} is missing columns: {missing}\n"
 f"Available columns: {list(frame.columns)}"
)

```

---

```

def choose_chern_column(frame: pd.DataFrame) -> str:
 for column in (
 "verified_chern_up_int",
 "chern_up_int",
):
 if column in frame.columns:
 return column
 raise KeyError(
 "FES table lacks verified_chern_up_int/chern_up_int."
)

def choose_gap_column(frame: pd.DataFrame) -> str | None:
 for column in (
 "verified_indirect_gap",
 "indirect_gap",
):
 if column in frame.columns:
 return column
 return None

def prepare_table(
 frame: pd.DataFrame,
 role: str,
) -> tuple[pd.DataFrame, str, str | None]:
 require_columns(frame, REQUIRED_COLUMNS, role)
 chern_column = choose_chern_column(frame)
 gap_column = choose_gap_column(frame)

 out = frame.copy()
 out["M_Gamma_FES"] = numeric(out, "mass_Gamma_signed")
 out["M_M_FES"] = numeric(out, "mass_M_signed")
 out["chern_up_plot"] = numeric(out, chern_column)
 out["chi_plot"] = np.sign(
 numeric(out, "r1") * numeric(out, "r2")
)

 out = out.dropna(
 subset=[
 "M_Gamma_FES",
 "M_M_FES",
 "chern_up_plot",
 "chi_plot",
]
).copy()

 out["chi_plot"] = out["chi_plot"].astype(int)
 out["chern_up_plot"] = out["chern_up_plot"].round().astype(int)
 out["source_table_role"] = role

 return out, chern_column, gap_column

def calibrate_prefactor(
 strict: pd.DataFrame,
 summary: dict,
) -> tuple[int, dict]:
 try:
 json_prefactor = int(
 summary.get("calibrated_formula_prefactor", 0)
)
 except (TypeError, ValueError):
 json_prefactor = 0

 base = (
 strict["chi_plot"].to_numpy(float)
 * 0.5

```

---

```

 * (
 np.sign(strict["M_Gamma_FES"].to_numpy(float))
 + np.sign(strict["M_M_FES"].to_numpy(float))
)
)
 actual = strict["chern_up_plot"].to_numpy(float)

 valid = (
 np.isfinite(base)
 & np.isfinite(actual)
 & (np.abs(base) > 0)
)

 if valid.any():
 accuracy_plus = float(
 np.mean(base[valid] == actual[valid])
)
 accuracy_minus = float(
 np.mean(-base[valid] == actual[valid])
)
 data_prefactor = (
 1 if accuracy_plus >= accuracy_minus else -1
)
 else:
 accuracy_plus = float("nan")
 accuracy_minus = float("nan")
 data_prefactor = 1

 if json_prefactor in (-1, 1):
 prefactor = json_prefactor
 source = "summary_json"
 else:
 prefactor = data_prefactor
 source = "strict_data_calibration"

 return int(prefactor), {
 "prefactor": int(prefactor),
 "prefactor_source": source,
 "strict_nonzero_rows_used": int(valid.sum()),
 "accuracy_if_prefactor_plus": accuracy_plus,
 "accuracy_if_prefactor_minus": accuracy_minus,
 }

def load_fes_data(
 input_source: Path | str,
 point_source: str,
) -> dict:
 source = DataSource(Path(input_source))

 strict_raw, strict_name = source.read_csv(
 STRICT_TABLES,
 required=True,
)
 assert strict_raw is not None
 strict, strict_chern_column, strict_gap_column = prepare_table(
 strict_raw,
 "strict",
)

 coarse_raw, coarse_name = source.read_csv(
 COARSE_TABLES,
 required=False,
)

 if point_source == "coarse" and coarse_raw is not None:
 points, point_chern_column, point_gap_column = prepare_table(
 coarse_raw,
 "coarse",

```

---

```

)
 point_table_name = coarse_name
else:
 points = strict.copy()
 point_chern_column = strict_chern_column
 point_gap_column = strict_gap_column
 point_table_name = strict_name

summary, summary_name = source.read_json_optional(
 SUMMARY_JSONS
)
prefactor, prefactor_audit = calibrate_prefactor(
 strict,
 summary,
)

for frame in (strict, points):
 frame["chern_up_analytic"] = (
 prefactor
 * frame["chi_plot"]
 * 0.5
 * (
 np.sign(frame["M_Gamma_FES"])
 + np.sign(frame["M_M_FES"])
)
).round().astype(int)

 frame["formula_match_plot"] = (
 frame["chern_up_analytic"]
 == frame["chern_up_plot"]
).astype(int)

return {
 "source": source.path,
 "strict": strict,
 "points": points,
 "strict_table_name": strict_name,
 "point_table_name": point_table_name,
 "summary_name": summary_name,
 "strict_chern_column": strict_chern_column,
 "point_chern_column": point_chern_column,
 "strict_gap_column": strict_gap_column,
 "point_gap_column": point_gap_column,
 "prefactor": prefactor,
 "prefactor_audit": prefactor_audit,
}

def symmetric_limit(
 *series: Iterable[float],
 padding: float = 1.08,
 quantile: float = 0.998,
) -> float:
 arrays: list[np.ndarray] = []
 for values in series:
 arr = pd.to_numeric(
 pd.Series(values),
 errors="coerce",
).to_numpy(float)
 arr = np.abs(arr[np.isfinite(arr)])
 if len(arr):
 arrays.append(arr)

 if not arrays:
 return 1.0

 combined = np.concatenate(arrays)
 base = float(np.quantile(combined, quantile))
 return max(base * padding, 1.0e-6)

```

---

```

def deterministic_stratified_subset(
 frame: pd.DataFrame,
 maximum: int | None,
 seed: int,
) -> pd.DataFrame:
 if maximum is None or len(frame) <= maximum:
 return frame.copy()

 rng = np.random.default_rng(seed)
 selected: list[int] = []
 groups = list(frame.groupby("chern_up_plot", sort=True))
 total = len(frame)

 for group_index, (_, group) in enumerate(groups):
 if group_index == len(groups) - 1:
 target = maximum - len(selected)
 else:
 target = max(
 1,
 int(round(maximum * len(group) / total)),
)
 target = min(
 target,
 maximum - len(selected),
)

 choices = rng.choice(
 group.index.to_numpy(),
 size=min(target, len(group)),
 replace=False,
)
 selected.extend(int(item) for item in choices)

 return frame.loc[selected].copy()

def add_region_labels(
 ax: plt.Axes,
 limit: float,
 chirality: int,
 prefactor: int,
) -> None:
 c_top_right = prefactor * chirality
 c_bottom_left = -prefactor * chirality

 box = dict(
 boxstyle="round,pad=0.20",
 facecolor="white",
 edgecolor="none",
 alpha=0.72,
)

 ax.text(
 0.58 * limit,
 0.58 * limit,
 rf"$C_uparrow={c_top_right:+d}$",
 ha="center",
 va="center",
 bbox=box,
)
 ax.text(
 -0.58 * limit,
 -0.58 * limit,
 rf"$C_uparrow={c_bottom_left:+d}$",
 ha="center",
 va="center",
 bbox=box,
)

```



---

```

)
 ax.text(
 -0.58 * limit,
 0.58 * limit,
 r"$C_{\uparrow}=0$",
 ha="center",
 va="center",
 bbox=box,
)
 ax.text(
 0.58 * limit,
 -0.58 * limit,
 r"$C_{\uparrow}=0$",
 ha="center",
 va="center",
 bbox=box,
)

def save_figure(
 fig: plt.Figure,
 stem: Path,
 png_dpi: int,
) -> dict[str, Path]:
 stem.parent.mkdir(parents=True, exist_ok=True)
 paths = {
 "png": stem.with_suffix(".png"),
 "pdf": stem.with_suffix(".pdf"),
 "svg": stem.with_suffix(".svg"),
 }
 fig.savefig(paths["png"], dpi=png_dpi, bbox_inches="tight")
 fig.savefig(paths["pdf"], bbox_inches="tight")
 fig.savefig(paths["svg"], bbox_inches="tight")
 plt.close(fig)
 return paths

def plot_fes_mass_plane(
 points: pd.DataFrame,
 strict: pd.DataFrame,
 chirality: int,
 prefactor: int,
 output_dir: Path | str,
 png_dpi: int,
 maximum_display_points: int | None,
 show_gap_status: bool,
 gap_column: str | None,
) -> dict:
 subset_all = points[points["chi_plot"] == chirality].copy()
 strict_subset = strict[strict["chi_plot"] == chirality].copy()
 if subset_all.empty:
 raise ValueError(f"No FES samples for chi={chirality:d}.")

 subset = deterministic_stratified_subset(
 subset_all,
 maximum=maximum_display_points,
 seed=20260729 + chirality,
)

 limit = symmetric_limit(
 subset_all["M_Gamma_FES"],
 subset_all["M_M_FES"],
 strict_subset["M_Gamma_FES"],
 strict_subset["M_M_FES"],
)

 axis = np.linspace(-limit, limit, 401)
 m_gamma_grid, m_m_grid = np.meshgrid(axis, axis)
 chern_grid = (

```

---

```

 prefactor
 * chirality
 * 0.5
 * (
 np.sign(m_gamma_grid)
 + np.sign(m_m_grid)
)
)

Initialize boundary pixels with the neutral region color.
At M_Gamma=0 or M_M=0, the sign formula can produce half-integer
intermediate values; these pixels are phase boundaries and are later
covered by black dashed lines.
region_rgb = np.empty(
 chern_grid.shape + (3,),
 dtype=float,
)
region_rgb[...] = REGION_FILL[0]
for value, color in REGION_FILL.items():
 region_rgb[np.isclose(chern_grid, value)] = color

fig, ax = plt.subplots(
 figsize=(8.0 / 2.54, 7.0 / 2.54)
)

ax.imshow(
 region_rgb,
 extent=[-limit, limit, -limit, limit],
 origin="lower",
 aspect="equal",
 interpolation="nearest",
 zorder=0,
)

ax.axvline(
 0.0,
 linestyle="--",
 linewidth=1.0,
 color="black",
 zorder=2,
)

ax.axhline(
 0.0,
 linestyle="--",
 linewidth=1.0,
 color="black",
 zorder=2,
)

for c_value in (-1, 0, +1):
 group = subset[subset["chern_up_plot"] == c_value]
 if group.empty:
 continue

 if (
 show_gap_status
 and gap_column is not None
 and gap_column in group.columns
):
 gap = pd.to_numeric(group[gap_column], errors="coerce")
 positive = group[gap > 0]
 nonpositive = group[gap <= 0]

 if len(positive):
 ax.scatter(
 positive["M_Gamma_FES"],
 positive["M_M_FES"],
 s=17,
 facecolors="white",

```

---

```

 edgecolors=[POINT_EDGE[c_value]],
 linewidths=0.75,
 alpha=0.92,
 zorder=3,
)
 if len(nonpositive):
 ax.scatter(
 nonpositive["M_Gamma_FES"],
 nonpositive["M_M_FES"],
 s=15,
 marker="x",
 color=[POINT_EDGE[c_value]],
 linewidths=0.70,
 alpha=0.82,
 zorder=3,
)
 else:
 ax.scatter(
 group["M_Gamma_FES"],
 group["M_M_FES"],
 s=14,
 facecolors="white",
 edgecolors=[POINT_EDGE[c_value]],
 linewidths=0.55,
 alpha=0.88,
 zorder=3,
)

add_region_labels(
 ax,
 limit=limit,
 chirality=chirality,
 prefactor=prefactor,
)

ax.set_xlim(-limit, limit)
ax.set_ylim(-limit, limit)
ax.set_aspect("equal", adjustable="box")

ax.set_xlabel(
 r"$M_{\Gamma}^{\mathrm{FES}}$"
 r"$=\mu_{\Gamma}^{\mathrm{signed}}$"
)
ax.set_ylabel(
 r"M_M^{FES}"
 r"$=\mu_M^{\mathrm{signed}}$"
)

ax.text(
 0.03,
 0.97,
 rf"$\chi_{\{\mathrm{FES}\}}$"
 rf"$=\mathrm{sgn}(r_{1r_2})$"
 rf"={chirality:+d}$",
 transform=ax.transAxes,
 ha="left",
 va="top",
 bbox=dict(
 boxstyle="round,pad=0.20",
 facecolor="white",
 edgecolor="none",
 alpha=0.82,
),
)

region_handles = [
 Patch(
 facecolor=REGION_FILL[+1],
 edgecolor="none",

```

---

```

 label=r"Region: $C_{\uparrow}^{+1}$ ",
),
 Patch(
 facecolor=REGION_FILL[0],
 edgecolor="none",
 label=r"Region: $C_{\uparrow}^0$ ",
),
 Patch(
 facecolor=REGION_FILL[-1],
 edgecolor="none",
 label=r"Region: $C_{\uparrow}^{-1}$ ",
),
]
region_legend = ax.legend(
 handles=region_handles,
 loc="lower left",
 frameon=True,
 framealpha=0.88,
 borderpad=0.35,
)
ax.add_artist(region_legend)

if show_gap_status and gap_column is not None:
 point_handles = [
 Line2D(
 [0],
 [0],
 marker="o",
 linestyle="None",
 markerfacecolor="white",
 markeredgecolor="black",
 markeredgewidth=0.9,
 markersize=5,
 label=r"Numerical: $E_g^{\mathrm{ind}} > 0$ ",
),
 Line2D(
 [0],
 [0],
 marker="x",
 linestyle="None",
 color="black",
 markeredgewidth=0.9,
 markersize=5,
 label=r"Numerical: $E_g^{\mathrm{ind}} \leq 0$ ",
),
]
else:
 point_handles = []
 for c_value in (+1, 0, -1):
 label = (
 rf"Numerical: $C_{\uparrow}^{\{c_value:d\}}$ "
 if c_value != 0
 else r"Numerical: $C_{\uparrow}^0$ "
)
 point_handles.append(
 Line2D(
 [0],
 [0],
 marker="o",
 linestyle="None",
 markerfacecolor="white",
 markeredgecolor=POINT_EDGE[c_value],
 markeredgewidth=0.9,
 markersize=5,
 label=label,
)
)

ax.legend(

```

---

```

 handles=point_handles,
 loc="upper right",
 frameon=True,
 framealpha=0.88,
 borderpad=0.35,
)

 fig.tight_layout()

 output_dir = Path(output_dir).expanduser().resolve()
 output_dir.mkdir(parents=True, exist_ok=True)
 suffix = "_gap_status" if show_gap_status else ""
 stem = (
 output_dir
 / f"fes_mass_plane_chi_{chirality:+d}{suffix}"
)
 paths = save_figure(
 fig,
 stem=stem,
 png_dpi=png_dpi,
)

 display_csv = (
 output_dir
 / f"fes_mass_plane_display_points_chi_{chirality:+d}{suffix}.csv"
)
 subset.to_csv(display_csv, index=False)

 return {
 "paths": paths,
 "display_csv": display_csv,
 "n_available_points": int(len(subset_all)),
 "n_displayed_points": int(len(subset)),
 "n_strict_points": int(len(strict_subset)),
 }

def run_fes_mass_plane(
 input_source: Path | str | None = None,
 project_root: Path | str = ".",
 output_dir: Path | str = "fes_mass_plane_paper",
 point_source: str = "coarse",
 png_dpi: int = 2400,
 generate_chi_minus: bool = True,
 maximum_display_points: int | None = 1400,
 generate_gap_status_version: bool = True,
) -> dict:
 configure_plot_style()

 if point_source not in {"coarse", "strict"}:
 raise ValueError(
 "point_source must be 'coarse' or 'strict'."
)

 if input_source is None:
 input_source = discover_fes_input(project_root)

 loaded = load_fes_data(
 input_source=input_source,
 point_source=point_source,
)

 points = loaded["points"]
 strict = loaded["strict"]
 prefactor = int(loaded["prefactor"])

 output_dir = Path(output_dir).expanduser().resolve()
 output_dir.mkdir(parents=True, exist_ok=True)

```

---

```

points.to_csv(
 output_dir / "fes_signed_mass_plane_points_all.csv",
 index=False,
)
strict.to_csv(
 output_dir / "fes_signed_mass_plane_points_strict.csv",
 index=False,
)

mismatches = points[
 points["formula_match_plot"] == 0
].copy()
mismatches.to_csv(
 output_dir / "fes_signed_mass_formula_mismatches.csv",
 index=False,
)

figures: dict[int, dict] = {}
chiralities = [1, -1] if generate_chi_minus else [1]

for chirality in chiralities:
 if not (points["chi_plot"] == chirality).any():
 continue

 standard = plot_fes_mass_plane(
 points=points,
 strict=strict,
 chirality=chirality,
 prefactor=prefactor,
 output_dir=output_dir,
 png_dpi=png_dpi,
 maximum_display_points=maximum_display_points,
 show_gap_status=False,
 gap_column=loaded["point_gap_column"],
)
 figures[chirality] = {"standard": standard}

 if (
 generate_gap_status_version
 and loaded["point_gap_column"] is not None
):
 gap_version = plot_fes_mass_plane(
 points=points,
 strict=strict,
 chirality=chirality,
 prefactor=prefactor,
 output_dir=output_dir,
 png_dpi=png_dpi,
 maximum_display_points=maximum_display_points,
 show_gap_status=True,
 gap_column=loaded["point_gap_column"],
)
 figures[chirality]["gap_status"] = gap_version

summary = {
 "script_version": SCRIPT_VERSION,
 "input_source": str(loaded["source"]),
 "strict_table_name": loaded["strict_table_name"],
 "point_table_name": loaded["point_table_name"],
 "point_source": point_source,
 "formula": (
 "C_up = eta * chi_FES / 2 * "
 "[sgn(M_Gamma_FES) + sgn(M_M_FES)]"
),
 "chi_definition": "chi_FES = sgn(r1*r2)",
 "axis_x": "M_Gamma_FES = mass_Gamma_signed",
 "axis_y": "M_M_FES = mass_M_signed",
 "prefactor_audit": loaded["prefactor_audit"],
 "n_points_total": int(len(points)),

```

---

```

 "n_strict_points": int(len(strict)),
 "n_formula_mismatches_in_point_table": int(len(mismatches)),
 "point_formula_accuracy": float(
 points["formula_match_plot"].mean()
),
 "interpretation": (
 "Nonzero FES regions are formal spin-Chern sectors. "
 "Under the minimal-model indirect-gap no-go they are not "
 "QSH-insulator regions."
),
 }

summary_path = output_dir / "fes_mass_plane_summary.json"
summary_path.write_text(
 json.dumps(summary, ensure_ascii=False, indent=2),
 encoding="utf-8",
)

return {
 "input_source": loaded["source"],
 "output_dir": output_dir,
 "points": points,
 "strict": strict,
 "prefactor": prefactor,
 "figures": figures,
 "summary": summary,
 "summary_path": summary_path,
}

def parse_args() -> argparse.Namespace:
 parser = argparse.ArgumentParser(
 description=(
 "Generate a Lieb-style four-quadrant FES signed-mass "
 "phase diagram."
)
)
 parser.add_argument(
 "--input-source",
 type=Path,
 default=None,
)
 parser.add_argument(
 "--project-root",
 type=Path,
 default=Path("."),
)
 parser.add_argument(
 "--output-dir",
 type=Path,
 default=Path("fes_mass_plane_paper"),
)
 parser.add_argument(
 "--point-source",
 choices=["coarse", "strict"],
 default="coarse",
)
 parser.add_argument(
 "--png-dpi",
 type=int,
 default=2400,
)
 parser.add_argument(
 "--maximum-display-points",
 type=int,
 default=1400,
)
 parser.add_argument(
 "--no-chi-minus",

```

```

 action="store_true",
)
 parser.add_argument(
 "--no-gap-status-version",
 action="store_true",
)
 return parser.parse_args()

if __name__ == "__main__":
 args = parse_args()
 result = run_fes_mass_plane(
 input_source=args.input_source,
 project_root=args.project_root,
 output_dir=args.output_dir,
 point_source=args.point_source,
 png_dpi=args.png_dpi,
 generate_chi_minus=not args.no_chi_minus,
 maximum_display_points=args.maximum_display_points,
 generate_gap_status_version=not args.no_gap_status_version,
)

 print("Input source:", result["input_source"])
 print("Formula prefactor eta:", result["prefactor"])
 print("Processed points:", len(result["points"]))
 print("Strict points:", len(result["strict"]))
 print("Output directory:", result["output_dir"])

```

## ► 代码清单 A.47.8 (python, 792 行)

```

#!/usr/bin/env python3
-*- coding: utf-8 -*-
"""
FES u_eff-B_eff phase map with Lieb-style random sample circles in each region.

This version keeps the current u_eff-B_eff phase-map style and color mapping,
then overlays small open circles randomly distributed inside each topological
region, in a way similar to the Lieb display style.

The random circles are schematic display points only. They are not newly
computed physical data points.
"""

from __future__ import annotations

import argparse
import json
import zipfile
from dataclasses import dataclass
from pathlib import Path
from typing import Sequence

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from matplotlib import font_manager, rcParams
from matplotlib.lines import Line2D
from matplotlib.patches import Patch

SCRIPT_VERSION = "FES_UEFF_BEFF_CURRENT_COLORS_RANDOM_REGION_CIRCLES_20260729"

CURRENT_UB_POINTS = "fes_uB_points_all_selected_source.csv"
CURRENT_UB_STRICT = "fes_uB_points_strict.csv"

```



---

```

CURRENT_UB_SUMMARY = "fes_uB_phase_map_summary.json"

CURRENT_MASS_ALL = "fes_signed_mass_plane_points_all.csv"
CURRENT_MASS_STRICT = "fes_signed_mass_plane_points_strict.csv"
CURRENT_MASS_SUMMARY = "fes_mass_plane_summary.json"

ORIGINAL_STRICT_TABLES = (
 "fes_step07_strict_formula_audit.csv",
 "fes_step06_formula_validation_strict.csv",
 "fes_step05_strict_analytic_audit.csv",
)
ORIGINAL_COARSE_TABLES = (
 "fes_step06_formula_validation_coarse.csv",
 "fes_step05_all_coarse_analytic_audit.csv",
)
ORIGINAL_SUMMARIES = (
 "fes_step07_run_summary.json",
 "fes_step06_run_summary.json",
)

COLOR_RED = (169 / 255.0, 36 / 255.0, 37 / 255.0)
COLOR_BLUE = (63 / 255.0, 99 / 255.0, 173 / 255.0)
COLOR_GREY = (211 / 255.0, 211 / 255.0, 211 / 255.0)

def lighten(color: tuple[float, float, float], factor: float):
 base = np.asarray(color, dtype=float)
 return tuple(np.clip(base * (1.0 - factor) + factor, 0.0, 1.0))

REGION_FILL = {
 +1: lighten(COLOR_RED, 0.70),
 0: lighten(COLOR_GREY, 0.15),
 -1: lighten(COLOR_BLUE, 0.68),
}
POINT_EDGE = {
 +1: COLOR_RED,
 0: (0.25, 0.25, 0.25),
 -1: COLOR_BLUE,
}

def configure_plot_style() -> None:
 available = {item.name for item in font_manager.fontManager.ttflist}
 font_name = (
 "Times New Roman"
 if "Times New Roman" in available
 else "DejaVu Serif"
)
 rcParams["font.family"] = font_name
 rcParams["mathtext.fontset"] = "stix"
 rcParams["font.size"] = 9
 rcParams["axes.labelsize"] = 9
 rcParams["axes.titlesize"] = 9
 rcParams["xtick.labelsize"] = 8
 rcParams["ytick.labelsize"] = 8
 rcParams["legend.fontsize"] = 8
 rcParams["axes.linewidth"] = 0.8
 rcParams["xtick.direction"] = "in"
 rcParams["ytick.direction"] = "in"
 rcParams["xtick.top"] = True
 rcParams["ytick.right"] = True
 rcParams["svg.fonttype"] = "none"
 rcParams["pdf.fonttype"] = 42
 rcParams["ps.fonttype"] = 42
 rcParams["axes.unicode_minus"] = False

@dataclass
class DataSource:
 path: Path

```

---

```

def __post_init__(self):
 self.path = Path(self.path).expanduser().resolve()
 if not self.path.exists():
 raise FileNotFoundError(self.path)
 if self.path.is_file() and self.path.suffix.lower() != ".zip":
 raise ValueError(f"Input must be a ZIP or extracted directory: {self.path}")

@property
def is_zip(self) -> bool:
 return self.path.is_file()

def names(self) -> list[str]:
 if self.is_zip:
 with zipfile.ZipFile(self.path) as archive:
 return [n for n in archive.namelist() if not n.endswith("/")]
 return [str(p.relative_to(self.path)) for p in self.path.rglob("*") if p.is_file()]

def find_basename(self, candidates: Sequence[str]) -> str | None:
 names = self.names()
 for basename in candidates:
 matches = [n for n in names if Path(n).name == basename]
 if matches:
 return sorted(matches, key=lambda x: (len(Path(x).parts), len(x)))[0]
 return None

def read_csv(self, candidates: Sequence[str], required: bool):
 selected = self.find_basename(candidates)
 if selected is None:
 if required:
 raise FileNotFoundError(
 f"No compatible CSV found in {self.path}.\nExpected one of: {list(candidates)}"
)
 return None, None
 if self.is_zip:
 with zipfile.ZipFile(self.path) as archive:
 with archive.open(selected) as stream:
 return pd.read_csv(stream, low_memory=False), Path(selected).name
 return pd.read_csv(self.path / selected, low_memory=False), Path(selected).name

def read_json_optional(self, candidates: Sequence[str]):
 selected = self.find_basename(candidates)
 if selected is None:
 return {}, None
 if self.is_zip:
 with zipfile.ZipFile(self.path) as archive:
 return json.loads(archive.read(selected).decode("utf-8")), Path(selected).name
 return json.loads((self.path / selected).read_text(encoding="utf-8")), Path(selected).name

def numeric(frame: pd.DataFrame, column: str) -> pd.Series:
 return pd.to_numeric(frame[column], errors="coerce")

def choose_chern_column(frame: pd.DataFrame) -> str:
 for column in ("chern_up_plot", "verified_chern_up_int", "chern_up_int"):
 if column in frame.columns:
 return column
 raise KeyError("No Chern column found.")

def choose_gap_column(frame: pd.DataFrame) -> str | None:
 for column in ("verified_indirect_gap", "indirect_gap"):
 if column in frame.columns:
 return column
 return None

def prepare_points(frame: pd.DataFrame, role: str):

```

---

```

out = frame.copy()

if "M_Gamma_FES" in out.columns:
 out["M_Gamma_FES"] = numeric(out, "M_Gamma_FES")
elif "mass_Gamma_signed" in out.columns:
 out["M_Gamma_FES"] = numeric(out, "mass_Gamma_signed")
else:
 raise KeyError("Missing M_Gamma_FES/mass_Gamma_signed.")

if "M_M_FES" in out.columns:
 out["M_M_FES"] = numeric(out, "M_M_FES")
elif "mass_M_signed" in out.columns:
 out["M_M_FES"] = numeric(out, "mass_M_signed")
else:
 raise KeyError("Missing M_M_FES/mass_M_signed.")

if "u_eff" not in out.columns:
 out["u_eff"] = 0.5 * (out["M_Gamma_FES"] - out["M_M_FES"])
else:
 out["u_eff"] = numeric(out, "u_eff")

if "B_eff" not in out.columns:
 out["B_eff"] = 0.25 * (out["M_Gamma_FES"] + out["M_M_FES"])
else:
 out["B_eff"] = numeric(out, "B_eff")

chern_col = choose_chern_column(out)
gap_col = choose_gap_column(out)
out["chern_up_plot"] = numeric(out, chern_col)

if "chi_plot" in out.columns:
 out["chi_plot"] = numeric(out, "chi_plot")
elif {"r1", "r2"}.issubset(out.columns):
 out["chi_plot"] = np.sign(numeric(out, "r1") * numeric(out, "r2"))
else:
 raise KeyError("Missing chi_plot or r1/r2.")

out = out.dropna(
 subset=["M_Gamma_FES", "M_M_FES", "u_eff", "B_eff", "chern_up_plot", "chi_plot"]
).copy()
out["chern_up_plot"] = out["chern_up_plot"].round().astype(int)
out["chi_plot"] = out["chi_plot"].round().astype(int)
out["source_table_role"] = role
return out, chern_col, gap_col

def calibrate_prefactor(strict: pd.DataFrame, summary: dict):
 prefactor_json = None
 if isinstance(summary.get("prefactor_audit"), dict):
 prefactor_json = summary["prefactor_audit"].get("prefactor")
 if prefactor_json not in (-1, 1):
 prefactor_json = summary.get("calibrated_formula_prefactor")

 try:
 prefactor_json = int(prefactor_json)
 except (TypeError, ValueError):
 prefactor_json = 0

 base = (
 strict["chi_plot"].to_numpy(float)
 * 0.5
 * (np.sign(strict["M_Gamma_FES"].to_numpy(float)) + np.sign(strict["M_M_FES"].to_numpy(float)))
)
 actual = strict["chern_up_plot"].to_numpy(float)
 valid = np.isfinite(base) & np.isfinite(actual) & (np.abs(base) > 0)

 if valid.any():
 acc_plus = float(np.mean(base[valid] == actual[valid]))
 acc_minus = float(np.mean(-base[valid] == actual[valid]))

```

---

```

 data_prefactor = 1 if acc_plus >= acc_minus else -1
 else:
 acc_plus = float("nan")
 acc_minus = float("nan")
 data_prefactor = 1

 if prefactor_json in (-1, 1):
 prefactor = prefactor_json
 source = "input_summary"
 else:
 prefactor = data_prefactor
 source = "strict_data_calibration"

 return int(prefactor), {
 "prefactor": int(prefactor),
 "source": source,
 "strict_nonzero_rows_used": int(valid.sum()),
 "accuracy_if_prefactor_plus": acc_plus,
 "accuracy_if_prefactor_minus": acc_minus,
 }
}

def load_data(input_source: Path | str, point_source: str):
 source = DataSource(Path(input_source))

 ub_points_raw, ub_points_name = source.read_csv((CURRENT_UB_POINTS,), required=False)
 ub_strict_raw, ub_strict_name = source.read_csv((CURRENT_UB_STRICT,), required=False)

 if ub_strict_raw is not None:
 summary, _ = source.read_json_optional((CURRENT_UB_SUMMARY,))
 strict, strict_chn_col, strict_gap_col = prepare_points(ub_strict_raw, "strict")
 if point_source == "coarse" and ub_points_raw is not None:
 points, point_chn_col, point_gap_col = prepare_points(ub_points_raw, "coarse")
 point_name = ub_points_name
 else:
 points, point_chn_col, point_gap_col = strict.copy(), strict_chn_col, strict_gap_col
 point_name = ub_strict_name
 prefactor, prefactor_audit = calibrate_prefactor(strict, summary)
 return {
 "source": source.path,
 "strict": strict,
 "points": points,
 "point_gap_column": point_gap_col,
 "prefactor": prefactor,
 "prefactor_audit": prefactor_audit,
 "point_table_name": point_name,
 }

 mass_all_raw, mass_all_name = source.read_csv((CURRENT_MASS_ALL,), required=False)
 mass_strict_raw, mass_strict_name = source.read_csv((CURRENT_MASS_STRICT,), required=False)
 if mass_strict_raw is not None:
 summary, _ = source.read_json_optional((CURRENT_MASS_SUMMARY,))
 strict, strict_chn_col, strict_gap_col = prepare_points(mass_strict_raw, "strict")
 if point_source == "coarse" and mass_all_raw is not None:
 points, point_chn_col, point_gap_col = prepare_points(mass_all_raw, "coarse")
 point_name = mass_all_name
 else:
 points, point_chn_col, point_gap_col = strict.copy(), strict_chn_col, strict_gap_col
 point_name = mass_strict_name
 prefactor, prefactor_audit = calibrate_prefactor(strict, summary)
 return {
 "source": source.path,
 "strict": strict,
 "points": points,
 "point_gap_column": point_gap_col,
 "prefactor": prefactor,
 "prefactor_audit": prefactor_audit,
 "point_table_name": point_name,
 }
}

```

---

```

strict_raw, strict_name = source.read_csv(ORIGINAL_STRICT_TABLES, required=True)
all_raw, all_name = source.read_csv(ORIGINAL_COARSE_TABLES, required=False)
summary, _ = source.read_json_optional(ORIGINAL_SUMMARIES)

strict, strict_chn_col, strict_gap_col = prepare_points(strict_raw, "strict")
if point_source == "coarse" and all_raw is not None:
 points, point_chn_col, point_gap_col = prepare_points(all_raw, "coarse")
 point_name = all_name
else:
 points, point_chn_col, point_gap_col = strict.copy(), strict_chn_col, strict_gap_col
 point_name = strict_name

prefactor, prefactor_audit = calibrate_prefactor(strict, summary)
return {
 "source": source.path,
 "strict": strict,
 "points": points,
 "point_gap_column": point_gap_col,
 "prefactor": prefactor,
 "prefactor_audit": prefactor_audit,
 "point_table_name": point_name,
}

def axis_extent(values: pd.Series, minimum: float = 0.5, padding: float = 1.10, quantile: float = 0.995):
 arr = np.abs(pd.to_numeric(values, errors="coerce").to_numpy(float))
 arr = arr[np.isfinite(arr)]
 if not len(arr):
 return minimum
 return max(minimum, padding * float(np.quantile(arr, quantile)))

def deterministic_stratified_subset(frame: pd.DataFrame, maximum: int | None, seed: int):
 if maximum is None or len(frame) <= maximum:
 return frame.copy()
 rng = np.random.default_rng(seed)
 selected = []
 groups = list(frame.groupby("chern_up_plot", sort=True))
 total = len(frame)
 for idx, (_, group) in enumerate(groups):
 if idx == len(groups) - 1:
 target = maximum - len(selected)
 else:
 target = max(1, int(round(maximum * len(group) / total)))
 target = min(target, maximum - len(selected))
 choices = rng.choice(group.index.to_numpy(), size=min(target, len(group)), replace=False)
 selected.extend(int(i) for i in choices)
 return frame.loc[selected].copy()

def add_region_labels(ax: plt.Axes, u_limit: float, B_limit: float, chirality: int, prefactor: int):
 c_top = prefactor * chirality
 c_bottom = -prefactor * chirality
 box = dict(boxstyle="round,pad=0.20", facecolor="white", edgecolor="none", alpha=0.72)
 ax.text(0.0, 0.62 * B_limit, rf"$C_{\uparrow}={c_top:d}$", ha="center", va="center", bbox=box)
 ax.text(0.0, -0.62 * B_limit, rf"$C_{\uparrow}={c_bottom:d}$", ha="center", va="center", bbox=box)
 ax.text(-0.64 * u_limit, 0.0, r"$C_{\uparrow}=0$", ha="center", va="center", bbox=box)
 ax.text(0.64 * u_limit, 0.0, r"$C_{\uparrow}=0$", ha="center", va="center", bbox=box)

def classify_region(u: np.ndarray, B: np.ndarray, chirality: int, prefactor: int) -> np.ndarray:
 M_gamma = u + 2.0 * B
 M_M = -u + 2.0 * B
 return (prefactor * chirality * 0.5 * (np.sign(M_gamma) + np.sign(M_M))).astype(int)

def random_points_in_region(
 target_chn: int,

```

---

```

 chirality: int,
 prefactor: int,
 u_limit: float,
 B_limit: float,
 n_points: int,
 seed: int,
):
 rng = np.random.default_rng(seed)
 accepted_u = []
 accepted_B = []

 # For C=0, enforce coverage on both left and right side regions.
 if target_chern == 0:
 parts = [("left", n_points // 2), ("right", n_points - n_points // 2)]
 else:
 parts = [(None, n_points)]

 for side_name, need in parts:
 got = 0
 attempts = 0
 while got < need and attempts < 200000:
 batch = min(5000, max(1000, 20 * (need - got)))
 attempts += batch

 if side_name == "left":
 u = rng.uniform(-u_limit, 0.0, size=batch)
 elif side_name == "right":
 u = rng.uniform(0.0, u_limit, size=batch)
 else:
 u = rng.uniform(-u_limit, u_limit, size=batch)
 B = rng.uniform(-B_limit, B_limit, size=batch)

 labels = classify_region(u, B, chirality=chirality, prefactor=prefactor)
 mask = labels == target_chern
 u_sel = u[mask]
 B_sel = B[mask]

 take = min(need - got, len(u_sel))
 if take > 0:
 accepted_u.extend(u_sel[:take].tolist())
 accepted_B.extend(B_sel[:take].tolist())
 got += take

 if got < need:
 raise RuntimeError(
 f"Failed to sample enough random points for C={target_chern} "
 f"(got {got}, need {need})."
)

 return pd.DataFrame(
 {
 "u_eff": np.asarray(accepted_u, dtype=float),
 "B_eff": np.asarray(accepted_B, dtype=float),
 "chern_up_plot": int(target_chern),
 "chi_plot": int(chirality),
 }
)

def generate_random_region_points(
 chirality: int,
 prefactor: int,
 u_limit: float,
 B_limit: float,
 n_points_per_region: int,
 seed: int,
):
 c_top = int(prefactor * chirality)
 c_bottom = int(-prefactor * chirality)

```

---

```

frames = [
 random_points_in_region(
 target_chern=c_top,
 chirality=chirality,
 prefactor=prefactor,
 u_limit=u_limit,
 B_limit=B_limit,
 n_points=n_points_per_region,
 seed=seed + 11,
),
 random_points_in_region(
 target_chern=0,
 chirality=chirality,
 prefactor=prefactor,
 u_limit=u_limit,
 B_limit=B_limit,
 n_points=n_points_per_region,
 seed=seed + 29,
),
 random_points_in_region(
 target_chern=c_bottom,
 chirality=chirality,
 prefactor=prefactor,
 u_limit=u_limit,
 B_limit=B_limit,
 n_points=n_points_per_region,
 seed=seed + 47,
),
]
out = pd.concat(frames, ignore_index=True)
out["sample_kind"] = "random_region_circle"
return out

def save_figure(fig: plt.Figure, stem: Path, png_dpi: int):
 stem.parent.mkdir(parents=True, exist_ok=True)
 paths = {
 "png": stem.with_suffix(".png"),
 "pdf": stem.with_suffix(".pdf"),
 "svg": stem.with_suffix(".svg"),
 }
 fig.savefig(paths["png"], dpi=png_dpi, bbox_inches="tight")
 fig.savefig(paths["pdf"], bbox_inches="tight")
 fig.savefig(paths["svg"], bbox_inches="tight")
 plt.close(fig)
 return paths

def plot_uB_map(
 points: pd.DataFrame,
 strict: pd.DataFrame,
 chirality: int,
 prefactor: int,
 output_dir: Path | str,
 png_dpi: int,
 maximum_display_points: int | None,
 show_gap_status: bool,
 gap_column: str | None,
 show_original_points: bool,
 n_random_points_per_region: int,
 random_seed: int,
):
 subset_all = points[points["chi_plot"] == chirality].copy()
 strict_subset = strict[strict["chi_plot"] == chirality].copy()
 if subset_all.empty:
 raise ValueError(f"No samples for chi={chirality:+d}.")

 subset = deterministic_stratified_subset(

```

---

```

subset_all,
maximum=maximum_display_points,
seed=20260729 + chirality,
)

u_limit = axis_extent(pd.concat([subset_all["u_eff"], strict_subset["u_eff"]], ignore_index=True))
B_limit = axis_extent(pd.concat([subset_all["B_eff"], strict_subset["B_eff"]], ignore_index=True))

random_region_points = generate_random_region_points(
 chirality=chirality,
 prefactor=prefactor,
 u_limit=u_limit,
 B_limit=B_limit,
 n_points_per_region=n_random_points_per_region,
 seed=random_seed + 100 * (chirality + 5),
)

u_axis = np.linspace(-u_limit, u_limit, 701)
B_axis = np.linspace(-B_limit, B_limit, 701)
U, B = np.meshgrid(u_axis, B_axis)
M_gamma = U + 2.0 * B
M_M = -U + 2.0 * B
chern_grid = prefactor * chirality * 0.5 * (np.sign(M_gamma) + np.sign(M_M))

region_rgb = np.empty(chern_grid.shape + (3,), dtype=float)
region_rgb[...] = REGION_FILL[0]
for value, color in REGION_FILL.items():
 region_rgb[np.isclose(chern_grid, value)] = color

fig, ax = plt.subplots(figsize=(8.0 / 2.54, 7.0 / 2.54))
ax.imshow(
 region_rgb,
 extent=[-u_limit, u_limit, -B_limit, B_limit],
 origin="lower",
 aspect="auto",
 interpolation="nearest",
 zorder=0,
)

line_u = np.linspace(-u_limit, u_limit, 900)
ax.plot(line_u, 0.5 * line_u, linestyle="--", linewidth=1.0, color="black", zorder=2)
ax.plot(line_u, -0.5 * line_u, linestyle="--", linewidth=1.0, color="black", zorder=2)

if show_original_points:
 for c_value in (-1, 0, +1):
 group = subset[subset["chern_up_plot"] == c_value]
 if group.empty:
 continue

 if show_gap_status and gap_column is not None and gap_column in group.columns:
 gap = pd.to_numeric(group[gap_column], errors="coerce")
 positive = group[gap > 0]
 nonpositive = group[gap <= 0]

 if len(positive):
 ax.scatter(
 positive["u_eff"], positive["B_eff"],
 s=18, facecolors="white", edgecolors=[POINT_EDGE[c_value]],
 linewidths=0.80, alpha=0.95, zorder=3
)
 if len(nonpositive):
 ax.scatter(
 nonpositive["u_eff"], nonpositive["B_eff"],
 s=17, marker="x", color=[POINT_EDGE[c_value]],
 linewidths=0.80, alpha=0.88, zorder=3
)
 else:
 ax.scatter(
 group["u_eff"], group["B_eff"],

```



---

```

 s=18, facecolors="white", edgecolors=[POINT_EDGE[c_value]],
 linewidths=0.80, alpha=0.95, zorder=3
)

Overlay Lieb-style random open circles in each region.
for c_value in (-1, 0, +1):
 group = random_region_points[random_region_points["chern_up_plot"] == c_value]
 if group.empty:
 continue
 ax.scatter(
 group["u_eff"], group["B_eff"],
 s=24, facecolors="white", edgecolors=[POINT_EDGE[c_value]],
 linewidths=0.95, alpha=0.98, zorder=4
)

add_region_labels(ax, u_limit=u_limit, B_limit=B_limit, chirality=chirality, prefactor=prefactor)

ax.set_xlim(-u_limit, u_limit)
ax.set_ylim(-B_limit, B_limit)
ax.set_box_aspect(1)
ax.set_xlabel(r"u_{eff}")
ax.set_ylabel(r"B_{eff}")

ax.text(
 0.03, 0.97,
 rf"$\chi_{\{\mathrm{FES}\}}=\mathrm{sgn}\{r_{1r_2}\}=\{\mathrm{chirality}:+d\}$",
 transform=ax.transAxes, ha="left", va="top",
 bbox=dict(boxstyle="round,pad=0.20", facecolor="white", edgecolor="none", alpha=0.82),
)

region_handles = [
 Patch(facecolor=REGION_FILL[+1], edgecolor="none", label=r"Region: $C_{\uparrow}=+1$ "),
 Patch(facecolor=REGION_FILL[0], edgecolor="none", label=r"Region: $C_{\uparrow}=0$ "),
 Patch(facecolor=REGION_FILL[-1], edgecolor="none", label=r"Region: $C_{\uparrow}=-1$ "),
]
region_legend = ax.legend(
 handles=region_handles, loc="lower left",
 frameon=True, framealpha=0.88, borderpad=0.35
)
ax.add_artist(region_legend)

point_handles = []
if show_original_points:
 if show_gap_status and gap_column is not None:
 point_handles.extend([
 Line2D([0], [0], marker="o", linestyle="None", markerfacecolor="white",
 markeredgecolor="black", markeredgewidth=0.9, markersize=5,
 label=r"Original: $E_g^{\mathrm{ind}} > 0$ "),
 Line2D([0], [0], marker="x", linestyle="None", color="black",
 markeredgewidth=0.9, markersize=5,
 label=r"Original: $E_g^{\mathrm{ind}} \leq 0$ "),
])
 else:
 for c_value in (+1, 0, -1):
 label = rf"Original: $C_{\uparrow}=\{c_value:d\}$ " if c_value != 0 else r"Original: $C_{\uparrow}=0$ "
 point_handles.append(
 Line2D([0], [0], marker="o", linestyle="None",
 markerfacecolor="white", markeredgecolor=POINT_EDGE[c_value],
 markeredgewidth=0.9, markersize=5, label=label)
)
 point_handles.append(
 Line2D([0], [0], marker="o", linestyle="None",
 markerfacecolor="white", markeredgecolor="black",
 markeredgewidth=0.9, markersize=5,
 label=f"Random display circles: {n_random_points_per_region} per region")
)

ax.legend(handles=point_handles, loc="upper right", frameon=True, framealpha=0.88, borderpad=0.35)

```

---

```

fig.tight_layout()

output_dir = Path(output_dir).expanduser().resolve()
output_dir.mkdir(parents=True, exist_ok=True)
suffix = "_gap_status" if show_gap_status else ""
stem = output_dir / f"fes_uB_phase_map_chi_{chirality:+d}_with_random_region_circles{suffix}"
paths = save_figure(fig, stem=stem, png_dpi=png_dpi)

subset.to_csv(output_dir / f"fes_uB_display_points_chi_{chirality:+d}_original{suffix}.csv", index=False)
random_region_points.to_csv(output_dir / f"fes_uB_random_region_circles_chi_{chirality:+d}.csv", index=False)

return {
 "paths": paths,
 "n_available_points": int(len(subset_all)),
 "n_original_displayed_points": int(len(subset)),
 "n_random_region_points": int(len(random_region_points)),
}

def run_fes_uB_map_with_random_region_circles(
 input_source: Path | str,
 output_dir: Path | str = "fes_uB_phase_map_with_random_region_circles_outputs",
 point_source: str = "strict",
 png_dpi: int = 2400,
 generate_chi_minus: bool = True,
 maximum_display_points: int | None = 1400,
 generate_gap_status_version: bool = False,
 show_original_points: bool = False,
 n_random_points_per_region: int = 20,
 random_seed: int = 20260729,
):
 configure_plot_style()

 loaded = load_data(input_source=input_source, point_source=point_source)
 points = loaded["points"]
 strict = loaded["strict"]
 prefactor = int(loaded["prefactor"])

 output_dir = Path(output_dir).expanduser().resolve()
 output_dir.mkdir(parents=True, exist_ok=True)
 points.to_csv(output_dir / "fes_uB_points_all_selected_source.csv", index=False)
 strict.to_csv(output_dir / "fes_uB_points_strict.csv", index=False)

 figures = {}
 for chirality in ([1, -1] if generate_chi_minus else [1]):
 if not (points["chi_plot"] == chirality).any():
 continue
 standard = plot_uB_map(
 points=points, strict=strict, chirality=chirality, prefactor=prefactor,
 output_dir=output_dir, png_dpi=png_dpi, maximum_display_points=maximum_display_points,
 show_gap_status=False, gap_column=loaded["point_gap_column"],
 show_original_points=show_original_points,
 n_random_points_per_region=n_random_points_per_region,
 random_seed=random_seed,
)
 figures[chirality] = {"standard": standard}

 if generate_gap_status_version and loaded["point_gap_column"] is not None:
 gap_version = plot_uB_map(
 points=points, strict=strict, chirality=chirality, prefactor=prefactor,
 output_dir=output_dir, png_dpi=png_dpi, maximum_display_points=maximum_display_points,
 show_gap_status=True, gap_column=loaded["point_gap_column"],
 show_original_points=show_original_points,
 n_random_points_per_region=n_random_points_per_region,
 random_seed=random_seed,
)
 figures[chirality]["gap_status"] = gap_version

 summary = {

```

---

```

 "script_version": SCRIPT_VERSION,
 "plotting_only": True,
 "input_source": str(loaded["source"]),
 "point_source": point_source,
 "prefactor_audit": loaded["prefactor_audit"],
 "axis_x": "u_eff = (M_Gamma_FES - M_M_FES) / 2",
 "axis_y": "B_eff = (M_Gamma_FES + M_M_FES) / 4",
 "phase_boundaries": ["B_eff = +u_eff/2", "B_eff = -u_eff/2"],
 "show_original_points": bool(show_original_points),
 "n_random_points_per_region": int(n_random_points_per_region),
 "random_seed": int(random_seed),
 "note": (
 "Random display circles are schematic visual points used to mimic the "
 "Lieb-style sampling display. They are not newly computed physical data."
),
 }
}
summary_path = output_dir / "fes_uB_random_region_circles_summary.json"
summary_path.write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")

return {
 "input_source": loaded["source"],
 "output_dir": output_dir,
 "points": points,
 "strict": strict,
 "prefactor": prefactor,
 "figures": figures,
 "summary": summary,
 "summary_path": summary_path,
}

def parse_args() -> argparse.Namespace:
 parser = argparse.ArgumentParser(
 description="FES u_eff-B_eff phase map with Lieb-style random region circles."
)
 parser.add_argument("--input-source", type=Path, required=True)
 parser.add_argument("--output-dir", type=Path,
 ↪ default=Path("fes_uB_phase_map_with_random_region_circles_outputs"))
 parser.add_argument("--point-source", choices=["strict", "coarse"], default="strict")
 parser.add_argument("--png-dpi", type=int, default=2400)
 parser.add_argument("--maximum-display-points", type=int, default=1400)
 parser.add_argument("--n-random-points-per-region", type=int, default=20)
 parser.add_argument("--random-seed", type=int, default=20260729)
 parser.add_argument("--show-original-points", action="store_true")
 parser.add_argument("--gap-status-version", action="store_true")
 parser.add_argument("--no-chi-minus", action="store_true")
 return parser.parse_args()

if __name__ == "__main__":
 args = parse_args()
 result = run_fes_uB_map_with_random_region_circles(
 input_source=args.input_source,
 output_dir=args.output_dir,
 point_source=args.point_source,
 png_dpi=args.png_dpi,
 generate_chi_minus=not args.no_chi_minus,
 maximum_display_points=args.maximum_display_points,
 generate_gap_status_version=args.gap_status_version,
 show_original_points=args.show_original_points,
 n_random_points_per_region=args.n_random_points_per_region,
 random_seed=args.random_seed,
)
 print("Input:", result["input_source"])
 print("Point source:", args.point_source)
 print("Points:", len(result["points"]))
 print("Strict points:", len(result["strict"]))
 print("Show original points:", args.show_original_points)
 print("Random points per region:", args.n_random_points_per_region)

```

---

```
print("Output:", result["output_dir"])
```

## Lieb 体系自旋陈数、解析质量与相界代码

### ► 代码清单 A.48.1 (python, 2866 行)

```
#!/usr/bin/env python3
-*- coding: utf-8 -*-
"""
Step 08 v5: direct Step 07 directory continuation, analytic boundaries, and WannierTools spectra
=====

This script continues the Lieb8 workflow after Step 07. It does not replace the
strict full-BZ audit or the reliable Chern labels from Step 06/07.

Main tasks

1. Add non-redundant mechanism features:
 M_plus = m_e + d1 + d2
 M_minus = -m_e + d1 + d2
 Delta_scale = sqrt(Delta1*Delta2)
 Delta_anisotropy = |log(Delta1/Delta2)|
 together with hopping-product/chirality descriptors.

2. Compare legacy and analytic compact logistic models using parameter-cluster
 group holdout, rather than random point splitting.

3. Refine the closest topology-trivial interpolation paths. Search for roots of
 M_plus = 0, M_minus = 0, t1 = 0, t2 = 0
 and verify each candidate by a dense direct-gap search and Chern calculations
 on both sides of the transition.

4. Calculate semi-infinite left/right edge spectral functions with the
 Sancho--Rubio iterative surface Green-function method used by WannierTools
 (the 1985 recursion implemented in surfgreen.f90/surfstat.f90).

5. Resolve spin-up/spin-down surface spectral weight and export data in a
 WannierTools-like dos.dat_l / dos.dat_r layout.

6. Independently diagonalize finite ribbons and check Ny = 30, 40, 60 width
 convergence. This is a validation channel, not the primary surface method.

Run

 python 08_Lieb8_Analytic_Mechanism_and_WannierTools_Edge_v5.py \
 --step07-dir outputs_step07_mechanism_boundary_consensus_...

Quick test

 python 08_Lieb8_Analytic_Mechanism_and_WannierTools_Edge_v5.py \
 --step07-dir outputs_step07_mechanism_boundary_consensus_... --quick

Direct Step 07 handoff

The script reads an already existing Step 07 output directory directly. It never
opens or extracts a ZIP archive. With no --step07-dir argument, it searches the
current project directory and the script directory for an extracted directory
named outputs_step07_mechanism_boundary_consensus_*. It reads the Step 07 output
registry, inherits the run tag, random seed, interpolation-pair count and ribbon
width, and writes a Step 08 output directory beside the Step 07 result.
```

---

```

"""

from __future__ import annotations

import argparse
import json
import math
import os
import re
import sys
from collections import Counter
from dataclasses import dataclass
from pathlib import Path
from typing import Callable, Iterable, Sequence, Mapping

os.environ.setdefault("OMP_NUM_THREADS", "1")
os.environ.setdefault("MKL_NUM_THREADS", "1")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.environ.setdefault("NUMEXPR_NUM_THREADS", "1")

import numpy as np
import pandas as pd

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap
from matplotlib.lines import Line2D

from scipy.linalg import eig
from scipy.optimize import brentq, minimize, minimize_scalar
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
 accuracy_score,
 balanced_accuracy_score,
 f1_score,
 precision_score,
 recall_score,
 roc_auc_score,
)
from sklearn.model_selection import GroupShuffleSplit
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

=====
Configuration
=====

PHYS7 = ["m_e", "t1", "t2", "s1", "d1", "s2", "d2"]
RAW8 = ["e1", "e2", "t1", "t2", "r1", "r2", "r3", "r4"]
SPIN_INDICES = {"up": [0, 2], "down": [1, 3]}
SITE_1 = np.array([0.5, 0.0])
SITE_2 = np.array([0.0, 0.5])
EPS = 1.0e-12

LEGACY_COMPACT_FEATURES = [
 "delta_min",
 "delta_product",
 "delta_imbalance",
 "mass_product_XM",
 "mass_sign_change_XM",
 "crossing_fraction",
 "gap_to_delta_min",
]

Non-redundant physical coordinates. delta_scale and delta_log_anisotropy are
independent replacements for the redundant trio delta_min/product/imbalance.
ANALYTIC_COMPACT_FEATURES = [

```

---

```

 "delta_scale",
 "delta_log_anisotropy",
 "mass_boundary_min_abs",
 "mass_boundary_product",
 "abs_t_product",
 "crossing_fraction",
 "gap_to_delta_scale",
]

ANALYTIC_SIGNED_FEATURES = ANALYTIC_COMPACT_FEATURES + [
 "M_plus",
 "M_minus",
 "t_product",
]

DEFAULT_RANDOM_SEED = 20260711
DEFAULT_N_SPLITS = 20
DEFAULT_TEST_SIZE = 0.25
DEFAULT_INTERPOLATION_PAIRS = 4
DEFAULT_ROOT_GRID = 2001
DEFAULT_BZ_NK = 240
DEFAULT_CHERN_GRIDS = (21, 31)
DEFAULT_CHERN_SHIFTS = ((0.0, 0.0), (0.5, 0.5))
DEFAULT_RIBBON_WIDTHS = (30, 40, 60)
DEFAULT_RIBBON_NK = 241
DEFAULT_EDGE_CELLS = 3
DEFAULT_EDGE_THRESHOLD = 0.35

Semi-infinite surface Green-function settings. The eta value is expressed in
the normalized energy units of the Lieb8 Hamiltonian ($E_* = 1$).
DEFAULT_SURFACE_NK = 241
DEFAULT_SURFACE_ENERGY_POINTS = 601
DEFAULT_SURFACE_ETA = 0.003
DEFAULT_SURFACE_MAX_ITER = 100
DEFAULT_SURFACE_TOL = 1.0e-14
DEFAULT_SURFACE_LOG_CONTRAST = 80.0

@dataclass(frozen=True)
class Settings:
 random_seed: int
 n_splits: int
 test_size: float
 interpolation_pairs: int
 root_grid: int
 bz_nk: int
 chern_grids: tuple[int, ...]
 chern_shifts: tuple[tuple[float, float], ...]
 edge_method: str
 surface_nk: int
 surface_energy_points: int
 surface_eta: float
 surface_max_iter: int
 surface_tol: float
 surface_log_contrast: float
 surface_energy_window: float | None
 write_surface_text: bool
 ribbon_widths: tuple[int, ...]
 ribbon_nk: int
 edge_cells: int
 edge_threshold: float
 quick: bool

@dataclass(frozen=True)
class Step07Context:
 """Resolved, already-existing Step 07 output directory used by Step 08."""

 directory: Path

```

---

```

source_parent: Path
run_tag: str
registry_file: Path
files: Mapping[str, Path]
configuration: Mapping[str, object]
summary: Mapping[str, object]

def require(self, key: str, fallback_pattern: str | None = None) -> Path:
 candidate = self.files.get(key)
 if candidate is not None and candidate.is_file():
 return candidate
 if fallback_pattern is not None:
 return find_unique(self.directory, fallback_pattern)
 raise FileNotFoundError(
 f"Step 07 registry does not contain required key {key!r}. "
 f"Registry: {self.registry_file}"
)

=====
File discovery and utilities
=====

def parse_args() -> argparse.Namespace:
 parser = argparse.ArgumentParser(
 description=(
 "Lieb8 Step 08 v5: direct continuation from an existing Step 07 "
 "output directory, with analytic boundaries and WannierTools-style "
 "semi-infinite edge spectra"
)
)
 parser.add_argument(
 "--step07-dir",
 type=Path,
 default=None,
 help=(
 "Existing extracted Step 07 output directory. When omitted, the "
 "newest compatible outputs_step07_mechanism_boundary_consensus_* "
 "directory is discovered automatically. ZIP files are not read."
),
)
 parser.add_argument(
 "--output-dir",
 type=Path,
 default=None,
 help=(
 "Output directory. By default a run-tagged Step 08 directory is "
 "created beside the Step 07 output directory."
),
)
 parser.add_argument(
 "--quick",
 action="store_true",
 help="Use smaller grids for code/environment validation.",
)
 parser.add_argument(
 "--skip-chern",
 action="store_true",
 help="Skip Chern calculations around refined roots.",
)
 parser.add_argument(
 "--skip-edge",
 action="store_true",
 help="Skip all surface Green-function and finite-ribbon calculations.",
)
 parser.add_argument(
 "--edge-method",
 choices=("green", "ribbon", "both"),

```

---

```

 default="both",
 help=(
 "green: semi-infinite iterative surface Green function; "
 "ribbon: finite-strip diagonalization; both: run both (default).",
),
)
 parser.add_argument(
 "--skip-ribbon",
 action="store_true",
 help=(
 "When --edge-method=both, run only the surface Green-function part."
),
)
 parser.add_argument(
 "--surface-eta",
 type=float,
 default=None,
 help="Imaginary broadening eta for the retarded surface Green function.",
)
 parser.add_argument(
 "--surface-nk",
 type=int,
 default=None,
 help="Number of edge-momentum points for the surface spectral map.",
)
 parser.add_argument(
 "--surface-energy-points",
 type=int,
 default=None,
 help="Number of energy points for the surface spectral map.",
)
 parser.add_argument(
 "--surface-energy-window",
 type=float,
 default=None,
 help=(
 "Symmetric plotting window around E_F. By default it is selected "
 "from the audited indirect gap."
),
)
 parser.add_argument(
 "--write-surface-text",
 action="store_true",
 help=(
 "Also export large WannierTools-like text files (dos.dat_l/r and "
 "spindos.dat_l/r). Compressed NPZ output is always written."
),
)
 parser.add_argument(
 "--targeted-samples",
 type=int,
 default=1024,
 help=(
 "Generate this many new unit-sphere samples concentrated near "
 "M_plus=0, M_minus=0, t1=0, and t2=0. The production default is 1024; use 0 to skip."
),
)
 parser.add_argument(
 "--targeted-width",
 type=float,
 default=0.12,
 help="Half-width of the targeted signed control parameter around each boundary.",
)
 parser.add_argument(
 "--random-seed",
 type=int,
 default=None,
 help="Override the random seed inherited from the Step 07 run tag.",
)

```



---

```

parser.add_argument(
 "--interpolation-pairs",
 type=int,
 default=None,
 help="Override n_interpolation_pairs inherited from Step 07.",
)
parser.add_argument(
 "--ribbon-widths",
 type=int,
 nargs="+",
 default=None,
 help=(
 "Override finite-ribbon widths. By default Step 08 uses widths "
 "around Step 07 ribbon_ny, e.g. 30 40 60 for ribbon_ny=40."
),
)
return parser.parse_args()

def _looks_like_step07_output(directory: Path) -> bool:
 """Return True only for an extracted Step 07 output directory."""
 if not directory.is_dir():
 return False
 return bool(
 list(directory.glob("step07_00_output_file_registry_*.csv"))
 and list(directory.glob("step07_01_consensus_and_engineered_feature_master_*.csv"))
 and list(directory.glob("step07_04_nearest_topology_trivial_parameter_pairs_*.csv"))
)

def _candidate_search_roots() -> list[Path]:
 roots = [Path.cwd().resolve(), Path(__file__).resolve().parent]
 unique: list[Path] = []
 for root in roots:
 if root.exists() and root not in unique:
 unique.append(root)
 return unique

def _locate_step07_output_directory(path: Path) -> Path:
 """
 Resolve a directory-only Step 07 input.

 The supplied path may be the Step 07 output directory itself or a project
 directory containing exactly one compatible Step 07 output directory.
 Archive files are deliberately rejected.
 """
 selected = path.expanduser().resolve()
 if not selected.exists():
 raise FileNotFoundError(f"Step 07 path does not exist: {selected}")
 if selected.is_file():
 raise ValueError(
 f"Step 08 requires an extracted Step 07 output directory, not a file: {selected}. "
 "Do not pass the upload ZIP; place the extracted Step 07 directory in the project."
)
 if _looks_like_step07_output(selected):
 return selected

 candidates: list[Path] = []
 for child in selected.glob("outputs_step07_mechanism_boundary_consensus_*"):
 if _looks_like_step07_output(child):
 candidates.append(child.resolve())
 for registry in selected.glob("*/step07_00_output_file_registry_*.csv"):
 if _looks_like_step07_output(registry.parent):
 candidates.append(registry.parent.resolve())
 candidates = list(dict.fromkeys(candidates))
 if len(candidates) == 1:
 return candidates[0]
 if not candidates:
 raise FileNotFoundError(

```

---

```

 f"No compatible extracted Step 07 output directory was found in {selected}. "
 "Expected outputs_step07_mechanism_boundary_consensus_* containing the "
 "Step 07 registry, analysis master, and nearest-pair table."
)
 raise RuntimeError(
 "Multiple compatible Step 07 output directories were found. Pass the intended "
 "directory explicitly with --step07-dir:\n"
 + "\n".join(str(p) for p in candidates)
)

def auto_find_step07_directory() -> Path:
 """Find the newest compatible extracted Step 07 directory; ignore all ZIPs."""
 candidates: list[Path] = []
 for root in _candidate_search_roots():
 if _looks_like_step07_output(root):
 candidates.append(root)
 for child in root.glob("outputs_step07_mechanism_boundary_consensus_*"):
 if _looks_like_step07_output(child):
 candidates.append(child.resolve())
 for registry in root.glob("*step07_00_output_file_registry__.csv"):
 if _looks_like_step07_output(registry.parent):
 candidates.append(registry.parent.resolve())
 candidates = list(dict.fromkeys(candidates))
 if not candidates:
 raise FileNotFoundError(
 "Cannot auto-detect an extracted Step 07 output directory. Place this Step 08 "
 "script in the project directory beside outputs_step07_mechanism_boundary_consensus_*, "
 "or pass that directory with --step07-dir. ZIP files are intentionally ignored."
)
 candidates.sort(key=lambda p: p.stat().st_mtime_ns, reverse=True)
 return candidates[0]

def _extract_run_tag(directory: Path, master_file: Path) -> str:
 match = re.search(r"__([^\s\\]+)\.csv$", master_file.name)
 if match:
 return match.group(1)
 prefix = "outputs_step07_mechanism_boundary_consensus_"
 if directory.name.startswith(prefix):
 return directory.name[len(prefix):]
 return directory.name

def _load_json_optional(path: Path | None) -> dict:
 if path is None or not path.is_file():
 return {}
 try:
 payload = json.loads(path.read_text(encoding="utf-8"))
 except (OSError, json.JSONDecodeError) as exc:
 raise ValueError(f"Cannot read JSON file {path}: {exc}") from exc
 if not isinstance(payload, dict):
 raise TypeError(f"Expected a JSON object in {path}")
 return payload

def resolve_step07_context(directory_path: Path | None) -> Step07Context:
 """Read Step 07 directly from its existing output directory."""
 directory = _locate_step07_output_directory(
 directory_path if directory_path is not None else auto_find_step07_directory()
)

 registry_file = find_unique(directory, "step07_00_output_file_registry__.csv")
 registry = pd.read_csv(registry_file)
 ensure_columns(registry, ["key", "filename"], "Step 07 output registry")
 duplicate_keys = registry[registry["key"].duplicated(keep=False)]["key"].tolist()
 if duplicate_keys:
 raise ValueError(f"Duplicate keys in Step 07 registry: {sorted(set(duplicate_keys))}")

```

---

```

files: dict[str, Path] = {}
for row in registry.itertuples(index=False):
 key = str(row.key)
 filename = Path(str(row.filename).replace("\\", "/")).name
 direct = directory / filename
 if direct.is_file():
 files[key] = direct
 continue
 matches = list(directory.rglob(filename))
 if len(matches) == 1:
 files[key] = matches[0]

master_file = files.get("analysis_master") or find_unique(
 directory, "step07_01_consensus_and_engineered_feature_master__*.csv"
)
files["analysis_master"] = master_file

config_file = files.get("config")
if config_file is None:
 matches = list(directory.glob("step07_00_run_configuration__*.json"))
 config_file = matches[0] if len(matches) == 1 else None
summary_file = files.get("summary")
if summary_file is None:
 matches = list(directory.glob("step07_08_run_summary__*.json"))
 summary_file = matches[0] if len(matches) == 1 else None

return Step07Context(
 directory=directory,
 source_parent=directory.parent,
 run_tag=_extract_run_tag(directory, master_file),
 registry_file=registry_file,
 files=files,
 configuration=_load_json_optional(config_file),
 summary=_load_json_optional(summary_file),
)

def find_unique(directory: Path, pattern: str) -> Path:
 matches = sorted(directory.glob(pattern))
 if len(matches) != 1:
 raise FileNotFoundError(
 f"Expected exactly one file matching {pattern!r} in {directory}; "
 f"found {len(matches)}:\n" + "\n".join(str(x) for x in matches)
)
 return matches[0]

def infer_seed(context: Step07Context) -> int:
 for source in (context.run_tag, context.directory.name):
 match = re.search(r"seed(\d+)", source)
 if match:
 return int(match.group(1))
 for payload in (context.configuration, context.summary):
 for key in ("random_seed", "seed"):
 value = payload.get(key)
 if value is not None:
 return int(value)
 return DEFAULT_RANDOM_SEED

def infer_interpolation_pairs(context: Step07Context) -> int:
 value = context.configuration.get("n_interpolation_pairs")
 if value is not None:
 return max(1, int(value))
 match = re.search(r"pairs(\d+)", context.run_tag)
 return int(match.group(1)) if match else DEFAULT_INTERPOLATION_PAIRS

def infer_ribbon_widths(context: Step07Context) -> tuple[int, ...]:
 value = context.configuration.get("ribbon_ny")

```

---

```

 if value is None:
 return DEFAULT_RIBBON_WIDTHS
 center = max(8, int(value))
 widths = (max(8, center - 10), center, center + 20)
 return tuple(sorted(set(widths)))

def write_input_compatibility_report(
 context: Step07Context,
 output_dir: Path,
 required_files: Mapping[str, Path],
) -> None:
 report = {
 "workflow_step": "step08",
 "input_mode": "existing_step07_directory_only",
 "step07_output_root": str(context.directory),
 "step07_run_tag": context.run_tag,
 "zip_reading_enabled": False,
 "step07_registry": str(context.registry_file),
 "required_files": {key: str(path) for key, path in required_files.items()},
 "required_files_all_exist": all(path.is_file() for path in required_files.values()),
 "inherited_configuration": dict(context.configuration),
 }
 write_json(output_dir / "step08_00_step07_compatibility_report.json", report)

def write_step08_registry(output_dir: Path, run_tag: str) -> None:
 rows = []
 for path in sorted(output_dir.rglob("*")):
 if not path.is_file() or path.name.startswith("step08_00_output_file_registry"):
 continue
 rows.append(
 {
 "filename": path.name,
 "relative_path": str(path.relative_to(output_dir)),
 "suffix": path.suffix.lower(),
 "size_bytes": int(path.stat().st_size),
 }
)
 pd.DataFrame(rows).to_csv(
 output_dir / f"step08_00_output_file_registry__{run_tag}.csv", index=False
)

def resolve_edge_selection_file(
 context: Step07Context,
 master: pd.DataFrame,
 pair_table: pd.DataFrame,
 output_dir: Path,
) -> tuple[Path, str]:
 """Use Step 07's ribbon selection or generate an equivalent fallback."""
 existing = context.files.get("ribbon_selection")
 if existing is not None and existing.is_file():
 selection = pd.read_csv(existing)
 ensure_columns(selection, ["role", "sample_id"], "Step 07 ribbon selection")
 known = set(master["sample_id"].astype(str))
 missing = [sid for sid in selection["sample_id"].astype(str) if sid not in known]
 if missing:
 raise KeyError(
 f"Step 07 ribbon selection contains sample IDs absent from the master: {missing}"
)
 return existing, "step07_registry"

 ensure_columns(
 pair_table,
 ["topology_sample_id", "trivial_sample_id", "standardized_distance"],
 "Step 07 nearest-pair table",
)
 candidates = pair_table.copy()

```

---

```

known = set(master["sample_id"].astype(str))
candidates = candidates[
 candidates["topology_sample_id"].astype(str).isin(known)
 & candidates["trivial_sample_id"].astype(str).isin(known)
].copy()
if candidates.empty:
 raise ValueError(
 "No valid topology-trivial pair is available for edge-state analysis."
)
if {"topology_consensus_group", "trivial_consensus_group"}.issubset(candidates.columns):
 preferred = candidates[
 (candidates["topology_consensus_group"] == "both_topo")
 & (candidates["trivial_consensus_group"] == "both_trivial")
]
 if not preferred.empty:
 candidates = preferred
if "mutual_nearest_neighbor" in candidates.columns:
 mutual = candidates[candidates["mutual_nearest_neighbor"].astype(bool)]
 if not mutual.empty:
 candidates = mutual
selected = candidates.sort_values("standardized_distance").iloc[0]
generated = pd.DataFrame(
 [
 {
 "role": "typeII_QSH",
 "sample_id": str(selected["topology_sample_id"]),
 "pair_distance": float(selected["standardized_distance"]),
 },
 {
 "role": "trivial_insulator",
 "sample_id": str(selected["trivial_sample_id"]),
 "pair_distance": float(selected["standardized_distance"]),
 },
]
)
path = output_dir / "step08_00_generated_edge_pair_selection.csv"
generated.to_csv(path, index=False)
return path, "generated_from_step07_nearest_pairs"

def ensure_columns(df: pd.DataFrame, columns: Sequence[str], context: str) -> None:
 missing = [column for column in columns if column not in df.columns]
 if missing:
 raise KeyError(f"{context} is missing required columns: {missing}")

def safe_divide(numerator: np.ndarray, denominator: np.ndarray) -> np.ndarray:
 return numerator / np.maximum(np.asarray(denominator, dtype=float), EPS)

def write_json(path: Path, payload: dict) -> None:
 path.write_text(
 json.dumps(payload, indent=2, ensure_ascii=False, default=float),
 encoding="utf-8",
)

def save_figure(fig: plt.Figure, path_without_suffix: Path) -> None:
 path_without_suffix.parent.mkdir(parents=True, exist_ok=True)
 for extension, dpi in (("png", 320), ("pdf", None), ("svg", None)):
 kwargs = {"bbox_inches": "tight"}
 if dpi is not None:
 kwargs["dpi"] = dpi
 fig.savefig(path_without_suffix.with_suffix(f".{extension}"), **kwargs)

=====
Hamiltonian, bands, and Chern number
=====

```

---

```

def phys7_to_raw8(vector: Sequence[float]) -> dict[str, float]:
 values = {name: float(value) for name, value in zip(PHYS7, vector)}
 return {
 **values,
 "e1": values["m_e"],
 "e2": -values["m_e"],
 "r1": 0.5 * (values["s1"] + values["d1"]),
 "r3": 0.5 * (values["s1"] - values["d1"]),
 "r2": 0.5 * (values["s2"] + values["d2"]),
 "r4": 0.5 * (values["s2"] - values["d2"]),
 }

def slerp(vector_a: Sequence[float], vector_b: Sequence[float], t: float) -> np.ndarray:
 a = np.asarray(vector_a, dtype=float)
 b = np.asarray(vector_b, dtype=float)
 a = a / np.linalg.norm(a)
 b = b / np.linalg.norm(b)
 dot = float(np.clip(np.dot(a, b), -1.0, 1.0))
 angle = float(np.arccos(dot))
 if angle < 1.0e-12:
 vector = (1.0 - t) * a + t * b
 return vector / np.linalg.norm(vector)
 sine = np.sin(angle)
 return np.sin((1.0 - t) * angle) / sine * a + np.sin(t * angle) / sine * b

def h_lieb_atomic(kx: float, ky: float, params: pd.Series | dict) -> np.ndarray:
 e1, e2, t1, t2, r1, r2, r3, r4 = [float(params[key]) for key in RAW8]
 a = e1 + 2.0 * r1 * np.cos(ky) + 2.0 * r3 * np.cos(kx)
 b = e2 + 2.0 * r2 * np.cos(ky) + 2.0 * r4 * np.cos(kx)
 c = e2 + 2.0 * r2 * np.cos(kx) + 2.0 * r4 * np.cos(ky)
 d = e1 + 2.0 * r1 * np.cos(kx) + 2.0 * r3 * np.cos(ky)
 cp = np.cos(0.5 * (kx + ky))
 cm = np.cos(0.5 * (kx - ky))
 u = 2.0 * (-1j * t1 + t2) * cp + 2.0 * (1j * t1 + t2) * cm
 v = 2.0 * (-1j * t1 + t2) * cm + 2.0 * (1j * t1 + t2) * cp
 return np.array(
 [
 [a, 0.0, u, 0.0],
 [0.0, b, 0.0, v],
 [np.conj(u), 0.0, c, 0.0],
 [0.0, np.conj(v), 0.0, d],
],
 dtype=np.complex128,
)

def h_periodic(kx: float, ky: float, params: pd.Series | dict) -> np.ndarray:
 phase_1 = np.exp(1j * (kx * SITE_1[0] + ky * SITE_1[1]))
 phase_2 = np.exp(1j * (kx * SITE_2[0] + ky * SITE_2[1]))
 gauge = np.diag([phase_1, phase_1, phase_2, phase_2]).astype(np.complex128)
 return gauge.conj().T @ h_lieb_atomic(kx, ky, params) @ gauge

def spin_block_periodic(
 kx: float, ky: float, params: pd.Series | dict, spin: str
) -> np.ndarray:
 indices = SPIN_INDICES[spin]
 full = h_periodic(kx, ky, params)
 return full[np.ix_(indices, indices)]

def band_arrays(kx: np.ndarray, ky: np.ndarray, params: pd.Series | dict) -> np.ndarray:
 e1, e2, t1, t2, r1, r2, r3, r4 = [float(params[key]) for key in RAW8]
 a = e1 + 2.0 * r1 * np.cos(ky) + 2.0 * r3 * np.cos(kx)
 b = e2 + 2.0 * r2 * np.cos(ky) + 2.0 * r4 * np.cos(kx)
 c = e2 + 2.0 * r2 * np.cos(kx) + 2.0 * r4 * np.cos(ky)

```

---

```

d = e1 + 2.0 * r1 * np.cos(kx) + 2.0 * r3 * np.cos(ky)
cp = np.cos(0.5 * (kx + ky))
cm = np.cos(0.5 * (kx - ky))
u_re = 2.0 * t2 * (cp + cm)
u_im = 2.0 * t1 * (cm - cp)
v_re = 2.0 * t2 * (cp + cm)
v_im = 2.0 * t1 * (cp - cm)
up_center = 0.5 * (a + c)
down_center = 0.5 * (b + d)
up_radius = np.sqrt((0.5 * (a - c)) ** 2 + u_re**2 + u_im**2)
down_radius = np.sqrt((0.5 * (b - d)) ** 2 + v_re**2 + v_im**2)
energies = np.stack(
 [
 up_center - up_radius,
 up_center + up_radius,
 down_center - down_radius,
 down_center + down_radius,
],
 axis=-1,
)
energies.sort(axis=-1)
return energies

def periodic_bz_grid(nk: int) -> tuple[np.ndarray, int]:
 """Return a periodic square-BZ grid that explicitly contains k=0.

 ``np.linspace(-pi, pi, odd_nk, endpoint=False)`` misses zero. That was the
 source of the false finite gap at X/Y in the earlier Step 08 output. The
 effective mesh is therefore always even, so that the periodic grid contains
 -pi and 0 exactly. +pi is symmetry-equivalent to -pi.
 """
 if nk < 4:
 raise ValueError(f"nk must be >= 4, got {nk}")
 effective_nk = int(nk if nk % 2 == 0 else nk + 1)
 values = -np.pi + 2.0 * np.pi * np.arange(effective_nk) / effective_nk
 if not np.any(np.isclose(values, 0.0, atol=1.0e-14)):
 raise RuntimeError("Internal error: periodic BZ grid does not contain k=0")
 return values.astype(float), effective_nk

def _middle_bands(kx: float, ky: float, params: pd.Series | dict) -> tuple[float, float]:
 energies = np.linalg.eigvalsh(h_lieb_atomic(float(kx), float(ky), params))
 return float(energies[1]), float(energies[2])

def _unique_k_seeds(seeds: Sequence[tuple[float, float]]) -> list[tuple[float, float]]:
 unique: list[tuple[float, float]] = []
 for kx, ky in seeds:
 point = (float(np.clip(kx, -np.pi, np.pi)), float(np.clip(ky, -np.pi, np.pi)))
 if not any(np.hypot(point[0] - old[0], point[1] - old[1]) < 1.0e-10 for old in unique):
 unique.append(point)
 return unique

def _refine_band_extremum(
 params: pd.Series | dict,
 seeds: Sequence[tuple[float, float]],
 mode: str,
) -> dict[str, float]:
 """Refine a direct-gap minimum, VBM, or CBM from several BZ seeds."""
 if mode not in {"direct", "vbm", "cbm"}:
 raise ValueError(mode)

 def objective(point: np.ndarray) -> float:
 e2, e3 = _middle_bands(float(point[0]), float(point[1]), params)
 if mode == "direct":
 return e3 - e2
 if mode == "vbm":

```

---

```

 return -e2
 return e3

best_value = float("inf")
best_point = (float("nan"), float("nan"))
for seed in _unique_k_seeds(seeds):
 result = minimize(
 objective,
 np.asarray(seed, dtype=float),
 method="Powell",
 bounds=(-np.pi, np.pi), (-np.pi, np.pi)),
 options={"xtol": 1.0e-10, "ftol": 1.0e-12, "maxiter": 180},
)
 point = np.asarray(result.x, dtype=float)
 value = float(objective(point))
 if np.isfinite(value) and value < best_value:
 best_value = value
 best_point = (float(point[0]), float(point[1]))

Always compare against the exact high-symmetry points. This protects the
X/Y/M/ Γ roots even if a local optimizer terminates on a neighboring point.
high_symmetry = [
 (0.0, 0.0),
 (-np.pi, 0.0),
 (0.0, -np.pi),
 (-np.pi, -np.pi),
 (np.pi, 0.0),
 (0.0, np.pi),
 (np.pi, np.pi),
]
for point in high_symmetry:
 value = float(objective(np.asarray(point, dtype=float)))
 if value < best_value:
 best_value = value
 best_point = point

if mode == "vbm":
 physical_value = -best_value
else:
 physical_value = best_value
return {
 "value": float(physical_value),
 "kx": float(best_point[0]),
 "ky": float(best_point[1]),
}

def dense_gap_scan(
 params: pd.Series | dict,
 nk: int,
 *,
 refine_local: bool = True,
 n_local_seeds: int = 8,
) -> dict[str, float]:
 """Full-BZ direct/indirect gap scan with exact high-symmetry coverage.

 The periodic mesh is forced to an even size, so Γ , X, Y and M are sampled
 exactly. When ``refine_local`` is true, the best grid candidates are refined
 continuously in two dimensions. The returned direct gap is therefore not
 limited by the discrete BZ mesh and can resolve an off-grid axis closing.
 """
 k_values, effective_nk = periodic_bz_grid(nk)
 kx, ky = np.meshgrid(k_values, k_values, indexing="ij")
 energies = band_arrays(kx, ky, params)
 direct = energies[..., 2] - energies[..., 1]
 direct_index = np.unravel_index(int(np.argmin(direct)), direct.shape)
 vbm_index = np.unravel_index(int(np.argmax(energies[..., 1])), direct.shape)
 cbm_index = np.unravel_index(int(np.argmin(energies[..., 2])), direct.shape)

```



---

```

result: dict[str, float] = {
 "direct_gap": float(direct[direct_index]),
 "direct_kx": float(k_values[direct_index[0]]),
 "direct_ky": float(k_values[direct_index[1]]),
 "indirect_gap": float(energies[..., 2][cbm_index] - energies[..., 1][vbm_index]),
 "vbm": float(energies[..., 1][vbm_index]),
 "vbm_kx": float(k_values[vbm_index[0]]),
 "vbm_ky": float(k_values[vbm_index[1]]),
 "cbm": float(energies[..., 2][cbm_index]),
 "cbm_kx": float(k_values[cbm_index[0]]),
 "cbm_ky": float(k_values[cbm_index[1]]),
 "requested_nk": int(nk),
 "effective_nk": int(effective_nk),
 "contains_zero": 1,
 "local_refined": int(refine_local),
}
if not refine_local:
 return result

high_symmetry = [
 (0.0, 0.0),
 (-np.pi, 0.0),
 (0.0, -np.pi),
 (-np.pi, -np.pi),
 (np.pi, 0.0),
 (0.0, np.pi),
 (np.pi, np.pi),
]

def candidate_seeds(array: np.ndarray, smallest: bool) -> list[tuple[float, float]]:
 flat = array.ravel()
 count = min(max(1, n_local_seeds), flat.size)
 if smallest:
 indices = np.argpartition(flat, count - 1)[:count]
 else:
 indices = np.argpartition(-flat, count - 1)[:count]
 points = []
 for flat_index in indices:
 index = np.unravel_index(int(flat_index), array.shape)
 points.append((float(k_values[index[0]]), float(k_values[index[1]])))
 return points + high_symmetry

direct_refined = _refine_band_extremum(
 params, candidate_seeds(direct, True), "direct"
)
vbm_refined = _refine_band_extremum(
 params, candidate_seeds(energies[..., 1], False), "vbm"
)
cbm_refined = _refine_band_extremum(
 params, candidate_seeds(energies[..., 2], True), "cbm"
)

result.update(
 {
 "direct_gap": float(max(0.0, direct_refined["value"])),
 "direct_kx": direct_refined["kx"],
 "direct_ky": direct_refined["ky"],
 "indirect_gap": float(cbm_refined["value"] - vbm_refined["value"]),
 "vbm": vbm_refined["value"],
 "vbm_kx": vbm_refined["kx"],
 "vbm_ky": vbm_refined["ky"],
 "cbm": cbm_refined["value"],
 "cbm_kx": cbm_refined["kx"],
 "cbm_ky": cbm_refined["ky"],
 }
)
return result

```

```

def normalized_link(value: complex, tolerance: float = 1.0e-10) -> complex | None:

```

---

```

 amplitude = abs(value)
 if amplitude < tolerance:
 return None
 return value / amplitude

def fukui_single_band(
 h_function: Callable[[float, float], np.ndarray],
 nk: int,
 shift: tuple[float, float],
) -> float:
 shift_x, shift_y = shift
 kxs = 2.0 * np.pi * (np.arange(nk) + shift_x) / nk
 kys = 2.0 * np.pi * (np.arange(nk) + shift_y) / nk
 vectors = np.empty((nk, nk, 2), dtype=np.complex128)
 for ix, kx in enumerate(kxs):
 for iy, ky in enumerate(kys):
 _, eigenvectors = np.linalg.eigh(h_function(float(kx), float(ky)))
 vectors[ix, iy] = eigenvectors[:, 0]
 total_phase = 0.0
 for ix in range(nk):
 for iy in range(nk):
 vector = vectors[ix, iy]
 vector_x = vectors[(ix + 1) % nk, iy]
 vector_y = vectors[ix, (iy + 1) % nk]
 vector_xy = vectors[(ix + 1) % nk, (iy + 1) % nk]
 ux = normalized_link(np.vdot(vector, vector_x))
 uy = normalized_link(np.vdot(vector, vector_y))
 ux_y = normalized_link(np.vdot(vector_y, vector_xy))
 uy_x = normalized_link(np.vdot(vector_x, vector_xy))
 if any(value is None for value in (ux, uy, ux_y, uy_x)):
 return float("nan")
 total_phase += np.angle(ux * uy_x / (ux_y * uy))
 return float(total_phase / (2.0 * np.pi))

def fukui_total(
 params: pd.Series | dict,
 nk: int,
 shift: tuple[float, float],
) -> float:
 shift_x, shift_y = shift
 kxs = 2.0 * np.pi * (np.arange(nk) + shift_x) / nk
 kys = 2.0 * np.pi * (np.arange(nk) + shift_y) / nk
 vectors = np.empty((nk, nk, 4, 2), dtype=np.complex128)
 for ix, kx in enumerate(kxs):
 for iy, ky in enumerate(kys):
 _, eigenvectors = np.linalg.eigh(h_periodic(float(kx), float(ky), params))
 vectors[ix, iy] = eigenvectors[:, :2]
 total_phase = 0.0
 for ix in range(nk):
 for iy in range(nk):
 vector = vectors[ix, iy]
 vector_x = vectors[(ix + 1) % nk, iy]
 vector_y = vectors[ix, (iy + 1) % nk]
 vector_xy = vectors[(ix + 1) % nk, (iy + 1) % nk]
 ux = normalized_link(np.linalg.det(vector.conj().T @ vector_x))
 uy = normalized_link(np.linalg.det(vector.conj().T @ vector_y))
 ux_y = normalized_link(np.linalg.det(vector_y.conj().T @ vector_xy))
 uy_x = normalized_link(np.linalg.det(vector_x.conj().T @ vector_xy))
 if any(value is None for value in (ux, uy, ux_y, uy_x)):
 return float("nan")
 total_phase += np.angle(ux * uy_x / (ux_y * uy))
 return float(total_phase / (2.0 * np.pi))

def reliable_chern(
 params: pd.Series | dict,
 grids: Sequence[int],

```

---

```

 shifts: Sequence[tuple[float, float]],
) -> dict[str, object]:
 tuples: list[tuple[int, int, int]] = []
 raw_rows = []
 for nk in grids:
 for shift in shifts:
 cup = fukui_single_band(
 lambda kx, ky: spin_block_periodic(kx, ky, params, "up"), nk, shift
)
 cdn = fukui_single_band(
 lambda kx, ky: spin_block_periodic(kx, ky, params, "down"), nk, shift
)
 ctot = fukui_total(params, nk, shift)
 raw_rows.append((nk, shift, cup, cdn, ctot))
 if not all(np.isfinite(value) for value in (cup, cdn, ctot)):
 continue
 integers = (int(np rint(cup)), int(np rint(cdn)), int(np rint(ctot)))
 if (
 abs(cup - integers[0]) <= 0.08
 and abs(cdn - integers[1]) <= 0.08
 and abs(ctot - integers[2]) <= 0.08
 and abs(ctot - cup - cdn) <= 0.10
):
 tuples.append(integers)
 if len(tuples) < 2:
 return {
 "chern_reliable": 0,
 "chern_consensus_count": 0,
 "chern_up_int": np.nan,
 "chern_down_int": np.nan,
 "chern_total_int": np.nan,
 "phase_label": "chern_unresolved",
 "raw": raw_rows,
 }
 consensus, count = Counter(tuples).most_common(1)[0]
 cup, cdn, ctot = consensus
 if cup == -cdn and cup != 0 and ctot == 0:
 phase = "typeII_QSH"
 elif cup == 0 and cdn == 0 and ctot == 0:
 phase = "trivial_insulator"
 else:
 phase = "other_gapped_phase"
 return {
 "chern_reliable": int(count >= 2),
 "chern_consensus_count": int(count),
 "chern_up_int": int(cup),
 "chern_down_int": int(cdn),
 "chern_total_int": int(ctot),
 "phase_label": phase,
 "raw": raw_rows,
 }

=====
Mechanism features and grouped model comparison
=====

def add_analytic_features(df: pd.DataFrame) -> pd.DataFrame:
 out = df.copy()
 delta1 = np.asarray(out["spin_splitting_1"], dtype=float)
 delta2 = np.asarray(out["spin_splitting_2"], dtype=float)
 out["M_plus"] = out["m_e"] + out["d1"] + out["d2"]
 out["M_minus"] = -out["m_e"] + out["d1"] + out["d2"]
 out["mass_boundary_product"] = out["M_plus"] * out["M_minus"]
 out["mass_boundary_sign_change"] = (out["mass_boundary_product"] < 0.0).astype(int)
 out["mass_boundary_min_abs"] = np.minimum(np.abs(out["M_plus"]), np.abs(out["M_minus"]))
 out["mass_boundary_max_abs"] = np.maximum(np.abs(out["M_plus"]), np.abs(out["M_minus"]))
 out["delta_scale"] = np.sqrt(np.maximum(delta1 * delta2, 0.0))

```

---

```

out["delta_log_anisotropy"] = np.abs(np.log((delta1 + EPS) / (delta2 + EPS)))
out["t_product"] = out["t1"] * out["t2"]
out["abs_t_product"] = np.abs(out["t_product"])
out["t_chirality"] = np.sign(out["t_product"]).astype(int)
out["gap_to_delta_scale"] = safe_divide(
 np.asarray(out["final_direct_gap"], dtype=float),
 np.asarray(out["delta_scale"], dtype=float),
)
out["distance_to_any_analytic_boundary"] = np.minimum.reduce(
 [
 np.abs(np.asarray(out["M_plus"], dtype=float)),
 np.abs(np.asarray(out["M_minus"], dtype=float)),
 np.abs(np.asarray(out["t1"], dtype=float)),
 np.abs(np.asarray(out["t2"], dtype=float)),
]
)
return out

def binary_metrics(y_true: np.ndarray, probability: np.ndarray) -> dict[str, float]:
 prediction = (probability >= 0.5).astype(int)
 result = {
 "accuracy": accuracy_score(y_true, prediction),
 "balanced_accuracy": balanced_accuracy_score(y_true, prediction),
 "precision": precision_score(y_true, prediction, zero_division=0),
 "recall": recall_score(y_true, prediction, zero_division=0),
 "f1": f1_score(y_true, prediction, zero_division=0),
 }
 try:
 result["roc_auc"] = roc_auc_score(y_true, probability)
 except ValueError:
 result["roc_auc"] = float("nan")
 return {key: float(value) for key, value in result.items()}

def audit_exact_mass_product_rule(
 df: pd.DataFrame,
 output_dir: Path,
) -> tuple[dict[str, float], pd.DataFrame]:
 """Audit the empirical rule $M_{\text{plus}} * M_{\text{minus}} > 0$ on audited insulators.

 A perfect score on the current dataset is evidence for an exact model rule,
 but it is not by itself a proof. Targeted out-of-sample validation and an
 analytic two-band derivation are still required.
 """
 prediction = (np.asarray(df["mass_boundary_product"], dtype=float) > 0.0).astype(int)
 truth = np.asarray(df["target_typeII"], dtype=int)
 metrics = binary_metrics(truth, prediction.astype(float))
 metrics.update(
 {
 "rule": "typeII iff $M_{\text{plus}} * M_{\text{minus}} > 0$ ",
 "n_samples": int(len(df)),
 "n_mismatches": int(np.sum(prediction != truth)),
 }
)
 result = df[[
 "sample_id", "phase_label", "target_typeII", "M_plus", "M_minus",
 "mass_boundary_product", "final_direct_gap", "final_indirect_gap"
]].copy()
 result["analytic_rule_prediction"] = prediction
 result["analytic_rule_mismatch"] = (prediction != truth).astype(int)
 result.to_csv(output_dir / "step08_02_exact_mass_product_rule_audit.csv", index=False)
 write_json(output_dir / "step08_02_exact_mass_product_rule_metrics.json", metrics)
 return metrics, result

def grouped_logistic_comparison(
 df: pd.DataFrame,
 settings: Settings,

```

---

```

 output_dir: Path,
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
 ensure_columns(df, ["target_typeII", "parameter_cluster"], "Step 07 master")
 feature_sets = {
 "legacy_compact": LEGACY_COMPACT_FEATURES,
 "analytic_compact": ANALYTIC_COMPACT_FEATURES,
 "analytic_signed": ANALYTIC_SIGNED_FEATURES,
 }
 for feature_names in feature_sets.values():
 ensure_columns(df, feature_names, "analytic feature master")

 y = np.asarray(df["target_typeII"], dtype=int)
 groups = np.asarray(df["parameter_cluster"])
 splitter = GroupShuffleSplit(
 n_splits=settings.n_splits,
 test_size=settings.test_size,
 random_state=settings.random_seed,
)

 split_rows = []
 coefficient_rows = []
 for split_id, (train_index, test_index) in enumerate(
 splitter.split(df, y, groups=groups), start=1
):
 for model_name, feature_names in feature_sets.items():
 x = np.asarray(df[feature_names], dtype=float)
 model = Pipeline(
 [
 ("scaler", StandardScaler()),
 (
 "logistic",
 LogisticRegression(
 C=1.0,
 max_iter=5000,
 class_weight="balanced",
 random_state=settings.random_seed,
),
),
]
)
 model.fit(x[train_index], y[train_index])
 probability = model.predict_proba(x[test_index])[:, 1]
 metrics = binary_metrics(y[test_index], probability)
 split_rows.append(
 {
 "split_id": split_id,
 "model": model_name,
 "n_train": len(train_index),
 "n_test": len(test_index),
 "n_train_clusters": len(np.unique(groups[train_index])),
 "n_test_clusters": len(np.unique(groups[test_index])),
 **metrics,
 }
)
 coefficients = model.named_steps["logistic"].coef_[0]
 for feature, coefficient in zip(feature_names, coefficients):
 coefficient_rows.append(
 {
 "split_id": split_id,
 "model": model_name,
 "feature": feature,
 "standardized_coefficient": float(coefficient),
 }
)
)

 split_df = pd.DataFrame(split_rows)
 coefficient_df = pd.DataFrame(coefficient_rows)
 summary_df = (
 split_df.groupby("model")

```

---

```

 .agg(
 n_splits=("split_id", "count"),
 balanced_accuracy_mean=("balanced_accuracy", "mean"),
 balanced_accuracy_std=("balanced_accuracy", "std"),
 precision_mean=("precision", "mean"),
 recall_mean=("recall", "mean"),
 f1_mean=("f1", "mean"),
 roc_auc_mean=("roc_auc", "mean"),
)
 .reset_index()
)
 split_df.to_csv(output_dir / "step08_02_group_holdout_metrics.csv", index=False)
 summary_df.to_csv(output_dir / "step08_02_model_comparison_summary.csv", index=False)
 coefficient_df.to_csv(output_dir / "step08_02_logistic_coefficients_by_split.csv", index=False)

 coefficient_summary = (
 coefficient_df.groupby(["model", "feature"])
 .agg(
 coefficient_mean=("standardized_coefficient", "mean"),
 coefficient_std=("standardized_coefficient", "std"),
 positive_fraction=("standardized_coefficient", lambda x: float(np.mean(x > 0.0))),
)
 .reset_index()
)
 coefficient_summary.to_csv(
 output_dir / "step08_02_logistic_coefficient_summary.csv", index=False
)
 return split_df, summary_df, coefficient_summary

def mass_boundary_statistics(df: pd.DataFrame, output_dir: Path) -> pd.DataFrame:
 sign_plus = np.where(df["M_plus"] >= 0.0, "+", "-")
 sign_minus = np.where(df["M_minus"] >= 0.0, "+", "-")
 work = df.copy()
 work["mass_quadrant"] = [f"M+{a}, M-{b}" for a, b in zip(sign_plus, sign_minus)]
 work["hopping_quadrant"] = [
 f"t1{'+' if a >= 0 else '-'}", t2{'+' if b >= 0 else '-'}"
 for a, b in zip(work["t1"], work["t2"])
]
 rows = []
 for group_columns in (["mass_quadrant"], ["hopping_quadrant"], ["mass_quadrant", "hopping_quadrant"]):
 grouped = work.groupby(group_columns, dropna=False)
 for key, part in grouped:
 if not isinstance(key, tuple):
 key = (key,)
 record = {column: value for column, value in zip(group_columns, key)}
 rows.append(
 {
 "grouping": "+".join(group_columns),
 **record,
 "n_samples": int(len(part)),
 "n_typeII": int(part["target_typeII"].sum()),
 "typeII_fraction": float(part["target_typeII"].mean()),
 "mean_direct_gap": float(part["final_direct_gap"].mean()),
 "mean_delta_scale": float(part["delta_scale"].mean()),
 "mean_boundary_distance": float(part["mass_boundary_min_abs"].mean()),
 }
)
 stats = pd.DataFrame(rows)
 stats.to_csv(output_dir / "step08_03_mass_hopping_quadrant_statistics.csv", index=False)
 return stats

def plot_mechanism_overview(df: pd.DataFrame, figures_dir: Path) -> None:
 phase_numeric = np.asarray(df["target_typeII"], dtype=int)

 fig, ax = plt.subplots(figsize=(7.2, 6.2))
 trivial = phase_numeric == 0
 topo = phase_numeric == 1

```

---

```

ax.scatter(
 df.loc[trivial, "M_minus"],
 df.loc[trivial, "M_plus"],
 s=18,
 alpha=0.55,
 label="trivial insulator",
)
ax.scatter(
 df.loc[topo, "M_minus"],
 df.loc[topo, "M_plus"],
 s=20,
 alpha=0.70,
 label="type-II QSH",
)
ax.axhline(0.0, linewidth=1.0, linestyle="--")
ax.axvline(0.0, linewidth=1.0, linestyle="--")
ax.set_xlabel(r"$M_{-}=m_e+d_1+d_2$")
ax.set_ylabel(r"$M_{+}=m_e+d_1+d_2$")
ax.set_title("Two analytic X/Y mass branches")
ax.legend(frameon=False)
fig.tight_layout()
save_figure(fig, figures_dir / "step08_03_Mplus_Mminus_phase_scatter")
plt.close(fig)

Non-redundant splitting coordinates.
fig, ax = plt.subplots(figsize=(7.2, 5.8))
ax.scatter(
 df.loc[trivial, "delta_scale"],
 df.loc[trivial, "delta_log_anisotropy"],
 s=18,
 alpha=0.55,
 label="trivial insulator",
)
ax.scatter(
 df.loc[topo, "delta_scale"],
 df.loc[topo, "delta_log_anisotropy"],
 s=20,
 alpha=0.70,
 label="type-II QSH",
)
ax.set_xlabel(r"$\bar{\Delta}=\sqrt{\Delta_1\Delta_2}$")
ax.set_ylabel(r"$A_{\Delta}=\ln(\Delta_1/\Delta_2)$")
ax.set_title("Independent splitting strength and anisotropy")
ax.legend(frameon=False)
fig.tight_layout()
save_figure(fig, figures_dir / "step08_03_delta_scale_anisotropy_phase_scatter")
plt.close(fig)

Phase fraction as a function of distance to the closest mass boundary.
quantiles = np.unique(
 np.quantile(df["mass_boundary_min_abs"], np.linspace(0.0, 1.0, 9))
)
if len(quantiles) >= 3:
 binned = df.copy()
 binned["boundary_bin"] = pd.cut(
 binned["mass_boundary_min_abs"], bins=quantiles, include_lowest=True
)
 profile = (
 binned.groupby("boundary_bin", observed=True)
 .agg(
 boundary_distance=("mass_boundary_min_abs", "mean"),
 topology_fraction=("target_typeII", "mean"),
 n_samples=("target_typeII", "size"),
)
 .reset_index(drop=True)
)
 profile.to_csv(
 figures_dir.parent / "step08_03_topology_fraction_vs_mass_boundary_distance.csv",
 index=False,

```

---

```

)
 fig, ax = plt.subplots(figsize=(7.0, 5.2))
 ax.plot(
 profile["boundary_distance"],
 profile["topology_fraction"],
 marker="o",
 linewidth=1.5,
)
 for row in profile.itertuples(index=False):
 ax.annotate(
 f"n={row.n_samples}",
 (row.boundary_distance, row.topology_fraction),
 xytext=(0, 7),
 textcoords="offset points",
 ha="center",
 fontsize=8,
)
 ax.set_xlabel(r"$\min(|M_+|, |M_-|)$")
 ax.set_ylabel("type-II QSH fraction")
 ax.set_ylim(-0.03, 1.03)
 ax.set_title("Topology probability versus analytic mass-boundary distance")
 fig.tight_layout()
 save_figure(fig, figures_dir / "step08_03_topology_fraction_vs_boundary_distance")
 plt.close(fig)

=====
Analytic root refinement along SLERP paths
=====

def find_all_roots(function: Callable[[float], float], n_grid: int) -> list[float]:
 t_grid = np.linspace(0.0, 1.0, n_grid)
 values = np.asarray([function(float(t)) for t in t_grid], dtype=float)
 roots: list[float] = []
 for i in range(len(t_grid) - 1):
 a, b = float(t_grid[i]), float(t_grid[i + 1])
 fa, fb = float(values[i]), float(values[i + 1])
 if abs(fa) < 1.0e-12:
 roots.append(a)
 if fa * fb < 0.0:
 roots.append(float(brentq(function, a, b, xtol=1.0e-13, rtol=1.0e-13)))
 if abs(values[-1]) < 1.0e-12:
 roots.append(1.0)
 unique = []
 for root in sorted(roots):
 if not unique or abs(root - unique[-1]) > 1.0e-8:
 unique.append(root)
 return unique

def exact_four_band_gap(kx: float, ky: float, params: pd.Series | dict) -> float:
 energies = np.linalg.eigvalsh(h_lieb_atomic(kx, ky, params))
 return float(energies[2] - energies[1])

def spin_gap(kx: float, ky: float, params: pd.Series | dict, spin: str) -> float:
 energies = np.linalg.eigvalsh(spin_block_periodic(kx, ky, params, spin))
 return float(energies[1] - energies[0])

def axis_minimum(params: pd.Series | dict, spin: str, axis: str) -> dict[str, float]:
 if axis not in {"kx", "ky"}:
 raise ValueError(axis)

 def objective(q: float) -> float:
 kx, ky = (q, 0.0) if axis == "kx" else (0.0, q)
 return spin_gap(kx, ky, params, spin)

```



---

```

q_grid = np.linspace(-np.pi, np.pi, 2001)
values = np.asarray([objective(float(q)) for q in q_grid])
candidate_indices = np.argsort(values)[:8]
best = {"gap": float("inf"), "q": float("nan")}
step = float(q_grid[1] - q_grid[0])
for index in candidate_indices:
 center = float(q_grid[index])
 lower = max(-np.pi, center - 3.0 * step)
 upper = min(np.pi, center + 3.0 * step)
 result = minimize_scalar(objective, bounds=(lower, upper), method="bounded")
 if result.fun < best["gap"]:
 best = {"gap": float(result.fun), "q": float(result.x)}
kx, ky = (best["q"], 0.0) if axis == "kx" else (0.0, best["q"])
best.update({"kx": float(kx), "ky": float(ky), "spin": spin, "axis": axis})
return best

```

```

def candidate_gap_location(boundary: str, params: pd.Series | dict) -> dict[str, object]:
 if boundary in {"M_plus", "M_minus"}:
 candidates = []
 for label, (kx, ky) in {
 "X": (np.pi, 0.0),
 "Y": (0.0, np.pi),
 "minus_X": (-np.pi, 0.0),
 "minus_Y": (0.0, -np.pi),
 }.items():
 candidates.append(
 {
 "location_label": label,
 "kx": float(kx),
 "ky": float(ky),
 "gap": exact_four_band_gap(kx, ky, params),
 "spin": "up" if boundary == "M_plus" else "down",
 "axis": "high_symmetry",
 }
)
 return min(candidates, key=lambda row: row["gap"])

t1/t2 zeros can create axis closings. Test both spin blocks and axes.
candidates = [
 axis_minimum(params, spin, axis)
 for spin in ("up", "down")
 for axis in ("kx", "ky")
]
best = min(candidates, key=lambda row: row["gap"])
best["location_label"] = f"{best['spin']}_{best['axis']}_axis"
best["gap"] = exact_four_band_gap(best["kx"], best["ky"], params)
return best

```

```

def refine_transition_paths(
 master: pd.DataFrame,
 pair_table: pd.DataFrame,
 settings: Settings,
 output_dir: Path,
 figures_dir: Path,
 skip_chern: bool,
) -> tuple[pd.DataFrame, pd.DataFrame]:
 """Resolve analytic roots and produce gap profiles that pass through them.

```

The previous implementation used an odd periodic mesh and plotted only a uniform interpolation grid. Consequently X/Y and off-grid axis closings could be absent from the plotted curve. This version:

- \* uses the even, high-symmetry-complete BZ scan;
- \* performs continuous two-dimensional local gap refinement;
- \* inserts every analytic root and logarithmically spaced neighboring points into the path profile;
- \* distinguishes a scalar-parameter zero from a verified bulk-gap closing;

---

```

* searches for insulating points on both sides before accepting Chern data.
"""
ensure_columns(pair_table, ["topology_sample_id", "trivial_sample_id"], "pair table")
master_index = master.set_index("sample_id")
selected = pair_table.nsmallest(settings.interpolation_pairs, "standardized_distance")
root_rows: list[dict[str, object]] = []
path_rows: list[dict[str, object]] = []
closing_tolerance = 2.0e-6 if not settings.quick else 2.0e-5

for pair_id, pair in enumerate(selected.itertuples(index=False), start=1):
 topo = master_index.loc[pair.topology_sample_id]
 trivial = master_index.loc[pair.trivial_sample_id]
 vector_a = topo[PHYS7].to_numpy(dtype=float)
 vector_b = trivial[PHYS7].to_numpy(dtype=float)

 def vector_at(t: float) -> np.ndarray:
 return slerp(vector_a, vector_b, float(t))

 scalar_functions = {
 "M_plus": lambda t: float(vector_at(t)[0] + vector_at(t)[4] + vector_at(t)[6]),
 "M_minus": lambda t: float(-vector_at(t)[0] + vector_at(t)[4] + vector_at(t)[6]),
 "t1": lambda t: float(vector_at(t)[1]),
 "t2": lambda t: float(vector_at(t)[2]),
 }

 root_specs: list[dict[str, object]] = []
 for boundary, function in scalar_functions.items():
 for root in find_all_roots(function, settings.root_grid):
 if root <= 1.0e-8 or root >= 1.0 - 1.0e-8:
 continue
 vector = vector_at(root)
 params = phys7_to_raw8(vector)
 location = candidate_gap_location(boundary, params)
 global_gap = dense_gap_scan(params, settings.bz_nk, refine_local=True)
 verified_gap = min(float(location["gap"]), float(global_gap["direct_gap"]))
 true_closing = bool(verified_gap <= closing_tolerance)
 root_specs.append(
 {
 "boundary": boundary,
 "function": function,
 "t_critical": float(root),
 "vector": vector,
 "params": params,
 "location": location,
 "global_gap": global_gap,
 "verified_gap": float(verified_gap),
 "true_gap_closing": true_closing,
 }
)

 all_root_t = sorted(float(spec["t_critical"]) for spec in root_specs)
 for root_counter, spec in enumerate(root_specs, start=1):
 root = float(spec["t_critical"])
 vector = np.asarray(spec["vector"], dtype=float)
 location = spec["location"]
 global_gap = spec["global_gap"]
 record: dict[str, object] = {
 "pair_id": pair_id,
 "root_id_within_pair": root_counter,
 "topology_sample_id": pair.topology_sample_id,
 "trivial_sample_id": pair.trivial_sample_id,
 "boundary": spec["boundary"],
 "t_critical": root,
 "boundary_value": float(spec["function"](root)),
 **{name: float(value) for name, value in zip(PHYS7, vector)},
 "M_plus": float(vector[0] + vector[4] + vector[6]),
 "M_minus": float(-vector[0] + vector[4] + vector[6]),
 "candidate_location": location["location_label"],
 "candidate_kx": float(location["kx"]),

```

---

```

 "candidate_ky": float(location["ky"]),
 "candidate_direct_gap": float(location["gap"]),
 "global_direct_gap": float(global_gap["direct_gap"]),
 "global_direct_kx": float(global_gap["direct_kx"]),
 "global_direct_ky": float(global_gap["direct_ky"]),
 "global_indirect_gap": float(global_gap["indirect_gap"]),
 "verified_direct_gap": float(spec["verified_gap"]),
 "true_gap_closing": int(spec["true_gap_closing"]),
 "gap_closing_tolerance": closing_tolerance,
 "effective_bz_nk": int(global_gap["effective_nk"]),
 "high_symmetry_complete_grid": int(global_gap["contains_zero"]),
}

if not skip_chern and bool(spec["true_gap_closing"]):
 # Use the first offset that is both insulating and has a reliable
 # Chern consensus. The offset is restricted so that it cannot
 # cross another analytic root on the same interpolation path.
 distances = [abs(root - other) for other in all_root_t if abs(root - other) > 1.0e-10]
 root_separation_cap = 0.35 * min(distances) if distances else 0.08
 for side_name, direction in (("before", -1.0), ("after", 1.0)):
 accepted = False
 minimum_chern_side_gap = 0.004 if settings.quick else 0.006
 # Start far enough from the singular point for a finite Fukui
 # mesh to resolve the concentrated Berry curvature. Accepting
 # the first 0.001-offset consensus can falsely return C=0 even
 # though all coarse meshes agree. The largest safe insulating
 # offset is therefore tested first.
 for trial_offset in (0.05, 0.025, 0.012, 0.006, 0.003, 0.001):
 offset = min(trial_offset, root_separation_cap)
 if offset <= 1.0e-6:
 continue
 side_t = float(np.clip(root + direction * offset, 0.0, 1.0))
 if abs(side_t - root) < 1.0e-8:
 continue
 side_params = phys7_to_raw8(vector_at(side_t))
 side_gap = dense_gap_scan(
 side_params,
 max(120, settings.bz_nk // 2),
 refine_local=True,
)
 if (
 side_gap["direct_gap"] <= minimum_chern_side_gap
 or side_gap["indirect_gap"] <= minimum_chern_side_gap
):
 continue
 chern = reliable_chern(
 side_params, settings.chern_grids, settings.chern_shifts
)
 if not bool(chern["chern_reliable"]):
 continue
 record[f"{side_name}_t"] = side_t
 record[f"{side_name}_offset"] = abs(side_t - root)
 record[f"{side_name}_direct_gap"] = side_gap["direct_gap"]
 record[f"{side_name}_indirect_gap"] = side_gap["indirect_gap"]
 for key in (
 "chern_reliable",
 "chern_consensus_count",
 "chern_up_int",
 "chern_down_int",
 "chern_total_int",
 "phase_label",
):
 record[f"{side_name}_{key}"] = chern[key]
 accepted = True
 break
 record[f"{side_name}_verified"] = int(accepted)
 root_rows.append(record)

Include the exact roots and nearby logarithmic offsets in the plotted

```

---

```

path. This forces the profile to display the true closing rather than
interpolate over it between two coarse t points.
base_count = 201 if not settings.quick else 81
t_values = set(float(value) for value in np.linspace(0.0, 1.0, base_count))
neighborhood = (1.0e-4, 3.0e-4, 1.0e-3, 3.0e-3, 1.0e-2, 3.0e-2)
for spec in root_specs:
 root = float(spec["t_critical"])
 t_values.add(root)
 for delta in neighborhood:
 t_values.add(float(np.clip(root - delta, 0.0, 1.0)))
 t_values.add(float(np.clip(root + delta, 0.0, 1.0)))

root_by_t = {round(float(spec["t_critical"]), 13): spec for spec in root_specs}
for t in sorted(t_values):
 vector = vector_at(t)
 params = phys7_to_raw8(vector)
 matched = root_by_t.get(round(float(t), 13))
 is_analytic_root = int(matched is not None)
 root_boundary = ""
 if matched is not None:
 # Reuse the already refined full-BZ result at the exact root.
 gap = dict(matched["global_gap"])
 gap["direct_gap"] = min(
 float(gap["direct_gap"]),
 float(matched["location"]["gap"]),
)
 gap["direct_kx"] = float(matched["location"]["kx"])
 gap["direct_ky"] = float(matched["location"]["ky"])
 root_boundary = str(matched["boundary"])
 else:
 # A high-symmetry-complete even grid is sufficient away from a
 # transition. Continuous refinement is restricted to a narrow
 # neighborhood of analytic roots to keep Step 08 tractable.
 near_root = any(abs(float(t) - root_t) <= 0.035 for root_t in all_root_t)
 gap = dense_gap_scan(
 params,
 settings.bz_nk,
 refine_local=near_root,
 n_local_seeds=4,
)
 path_rows.append(
 {
 "pair_id": pair_id,
 "topology_sample_id": pair.topology_sample_id,
 "trivial_sample_id": pair.trivial_sample_id,
 "t": float(t),
 **{name: float(value) for name, value in zip(PHYS7, vector)},
 "M_plus": float(vector[0] + vector[4] + vector[6]),
 "M_minus": float(-vector[0] + vector[4] + vector[6]),
 "is_analytic_root": is_analytic_root,
 "root_boundary": root_boundary,
 **gap,
 }
)

root_df = pd.DataFrame(root_rows)
path_df = pd.DataFrame(path_rows).sort_values(["pair_id", "t"]).reset_index(drop=True)
root_df.to_csv(output_dir / "step08_04_refined_analytic_transition_roots.csv", index=False)
path_df.to_csv(output_dir / "step08_04_dense_transition_path_profiles.csv", index=False)

for pair_id, part in path_df.groupby("pair_id"):
 roots = root_df[root_df["pair_id"] == pair_id] if len(root_df) else pd.DataFrame()
 fig, axes = plt.subplots(2, 1, figsize=(8.0, 7.4), sharex=True)
 axes[0].plot(part["t"], part["direct_gap"], label="direct gap", linewidth=1.4)
 axes[0].plot(part["t"], part["indirect_gap"], label="indirect gap", linewidth=1.2)
 axes[0].axhline(0.0, linewidth=0.8)
 axes[0].set_ylabel("Gap")
 axes[0].legend(frameon=False)
 axes[1].plot(part["t"], part["M_plus"], label=r"M_+")

```

---

```

axes[1].plot(part["t"], part["M_minus"], label=r"M_{-}")
axes[1].plot(part["t"], part["t1"], label=r"t_1", linestyle="--")
axes[1].plot(part["t"], part["t2"], label=r"t_2", linestyle="--")
axes[1].axhline(0.0, linewidth=0.8)
axes[1].set_xlabel("SLERP interpolation coordinate t")
axes[1].set_ylabel("Analytic control parameter")
axes[1].legend(frameon=False, ncol=4, fontsize=8)
if len(roots):
 for root in roots.itertuples(index=False):
 style = "-" if int(root.true_gap_closing) else ":"
 width = 1.0 if int(root.true_gap_closing) else 0.7
 for ax in axes:
 ax.axvline(root.t_critical, linewidth=width, linestyle=style)
 axes[0].scatter(
 [root.t_critical],
 [root.verified_direct_gap],
 marker="o" if int(root.true_gap_closing) else "x",
 s=28,
 zorder=5,
)
 fig.suptitle(f"Pair {pair_id}: full-BZ gap closing and analytic roots")
 fig.tight_layout()
 save_figure(fig, figures_dir / f"step08_04_pair_{pair_id:02d}_refined_transition")
 plt.close(fig)

return root_df, path_df

=====
Targeted out-of-sample validation near analytic boundaries
=====

def sample_unit_sphere_with_linear_control(
 rng: np.random.Generator,
 normal: np.ndarray,
 target_value: float,
) -> np.ndarray:
 normal = np.asarray(normal, dtype=float)
 norm = float(np.linalg.norm(normal))
 n_hat = normal / norm
 if abs(target_value) >= norm:
 raise ValueError("target_value lies outside the unit-sphere range")
 tangent = rng.normal(size=7)
 tangent = tangent - np.dot(tangent, n_hat) * n_hat
 tangent_norm = float(np.linalg.norm(tangent))
 if tangent_norm < 1.0e-12:
 return sample_unit_sphere_with_linear_control(rng, normal, target_value)
 tangent /= tangent_norm
 parallel_coefficient = target_value / norm
 vector = math.sqrt(max(0.0, 1.0 - parallel_coefficient**2)) * tangent + parallel_coefficient * n_hat
 return vector / np.linalg.norm(vector)

def targeted_boundary_validation(
 n_samples: int,
 control_width: float,
 settings: Settings,
 output_dir: Path,
) -> tuple[pd.DataFrame, dict[str, object]]:
 if n_samples <= 0:
 return pd.DataFrame(), {}
 rng = np.random.default_rng(settings.random_seed + 808)
 families = {
 "M_plus": np.array([1.0, 0.0, 0.0, 0.0, 1.0, 0.0, 1.0]),
 "M_minus": np.array([-1.0, 0.0, 0.0, 0.0, 1.0, 0.0, 1.0]),
 "t1": np.array([0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0]),
 "t2": np.array([0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0]),
 }

```

---

```

coarse_nk = 50 if settings.quick else 100
fine_nk = 80 if settings.quick else 180
gap_accept = 0.012 if settings.quick else 0.020
convergence_tolerance = 0.012 if settings.quick else 0.006
rows = []
for sample_index in range(n_samples):
 family_name = list(families)[sample_index % len(families)]
 target = float(rng.uniform(-control_width, control_width))
 vector = sample_unit_sphere_with_linear_control(rng, families[family_name], target)
 params = phys7_to_raw8(vector)
 coarse = dense_gap_scan(params, coarse_nk, refine_local=False)
 fine_grid = dense_gap_scan(params, fine_nk, refine_local=False)
 # Only candidates that look insulating on the even grids need the more
 # expensive continuous refinement. The refined result is authoritative.
 if fine_grid["direct_gap"] > 0.5 * gap_accept and fine_grid["indirect_gap"] > 0.5 * gap_accept:
 fine = dense_gap_scan(params, fine_nk, refine_local=True)
 else:
 fine = fine_grid
 high_confidence = (
 fine["direct_gap"] > gap_accept
 and fine["indirect_gap"] > gap_accept
 and abs(fine["direct_gap"] - coarse["direct_gap"]) < convergence_tolerance
 and abs(fine["indirect_gap"] - coarse["indirect_gap"]) < convergence_tolerance
)
 M_plus = float(vector[0] + vector[4] + vector[6])
 M_minus = float(-vector[0] + vector[4] + vector[6])
 product = M_plus * M_minus
 row = {
 "targeted_sample_id": f"targeted_{sample_index:06d}",
 "sampling_family": family_name,
 "requested_control_value": target,
 **{name: float(value) for name, value in zip(PHYS7, vector)},
 **{name: float(params[name]) for name in RAW8},
 "M_plus": M_plus,
 "M_minus": M_minus,
 "mass_boundary_product": product,
 "analytic_rule_prediction": int(product > 0.0),
 "coarse_direct_gap": coarse["direct_gap"],
 "coarse_indirect_gap": coarse["indirect_gap"],
 "fine_grid_direct_gap": fine_grid["direct_gap"],
 "fine_grid_indirect_gap": fine_grid["indirect_gap"],
 "fine_direct_gap": fine["direct_gap"],
 "fine_indirect_gap": fine["indirect_gap"],
 "fine_effective_nk": fine["effective_nk"],
 "fine_local_refined": fine["local_refined"],
 "gap_high_confidence": int(high_confidence),
 }
 if high_confidence:
 chern = reliable_chern(params, settings.chern_grids, settings.chern_shifts)
 for key in (
 "chern_reliable", "chern_consensus_count", "chern_up_int",
 "chern_down_int", "chern_total_int", "phase_label"
):
 row[key] = chern[key]
 if chern["chern_reliable"] and chern["phase_label"] in {"typeII_QSH", "trivial_insulator"}:
 row["chern_target_typeII"] = int(chern["phase_label"] == "typeII_QSH")
 row["analytic_rule_mismatch"] = int(
 row["analytic_rule_prediction"] != row["chern_target_typeII"]
)
 else:
 row["chern_target_typeII"] = np.nan
 row["analytic_rule_mismatch"] = np.nan
 rows.append(row)

targeted = pd.DataFrame(rows)
targeted.to_csv(output_dir / "step08_06_targeted_boundary_validation_samples.csv", index=False)
accepted = targeted[
 (targeted["gap_high_confidence"] == 1)
 & (targeted.get("chern_reliable", 0) == 1)

```

```

 & targeted.get("chern_target_typeII", pd.Series(index=targeted.index, dtype=float)).notna()
]
summary = {
 "n_generated": int(len(targeted)),
 "n_gap_high_confidence": int(targeted["gap_high_confidence"].sum()),
 "n_chern_labeled_typeII_or_trivial": int(len(accepted)),
 "n_rule_mismatches": int(accepted["analytic_rule_mismatch"].sum()) if len(accepted) else 0,
 "rule_accuracy": float(1.0 - accepted["analytic_rule_mismatch"].mean()) if len(accepted) else None,
 "sampling_families": list(families),
 "control_width": control_width,
 "gap_acceptance_threshold": gap_accept,
}
write_json(output_dir / "step08_06_targeted_boundary_validation_summary.json", summary)
return targeted, summary

=====
Semi-infinite surface Green function and finite-ribbon validation
=====

def ribbon_blocks(kx: float, params: pd.Series | dict) -> tuple[np.ndarray, np.ndarray]:
 """Return H00(kx) and H01(kx) for a y-normal principal-layer geometry.

 The periodic Hamiltonian is reconstructed as

$$H(k_x, k_y) = H_{00} + H_{01} \exp(+i k_y) + H_{01}^{\dagger} \exp(-i k_y).$$

 This convention is passed directly to the iterative surface Green-function
 algorithm and to the finite-ribbon Hamiltonian.
 """
 e1, e2, t1, t2, r1, r2, r3, r4 = [float(params[key]) for key in RAW8]
 coupling_minus = t2 - 1j * t1
 coupling_plus = t2 + 1j * t1
 onsite = np.zeros((4, 4), dtype=np.complex128)
 hopping = np.zeros((4, 4), dtype=np.complex128)

 onsite[0, 0] = e1 + 2.0 * r3 * np.cos(kx)
 onsite[1, 1] = e2 + 2.0 * r4 * np.cos(kx)
 onsite[2, 2] = e2 + 2.0 * r2 * np.cos(kx)
 onsite[3, 3] = e1 + 2.0 * r1 * np.cos(kx)

 hopping[0, 0] = r1
 hopping[1, 1] = r2
 hopping[2, 2] = r4
 hopping[3, 3] = r3

 onsite[0, 2] = coupling_plus + coupling_minus * np.exp(-1j * kx)
 onsite[2, 0] = np.conj(onsite[0, 2])
 hopping[0, 2] = coupling_minus + coupling_plus * np.exp(-1j * kx)

 onsite[1, 3] = coupling_minus + coupling_plus * np.exp(-1j * kx)
 onsite[3, 1] = np.conj(onsite[1, 3])
 hopping[1, 3] = coupling_plus + coupling_minus * np.exp(-1j * kx)
 return onsite, hopping

def ribbon_reconstruction_error(params: pd.Series | dict, random_seed: int) -> float:
 """Verify that H00/H01 reproduce the analytic 2D Bloch Hamiltonian."""
 rng = np.random.default_rng(random_seed)
 maximum = 0.0
 for _ in range(40):
 kx = float(rng.uniform(-np.pi, np.pi))
 ky = float(rng.uniform(-np.pi, np.pi))
 onsite, hopping = ribbon_blocks(kx, params)
 reconstructed = (
 onsite
 + hopping * np.exp(1j * ky)
 + hopping.conj().T * np.exp(-1j * ky)

```

---

```

)
 maximum = max(
 maximum,
 float(np.max(np.abs(reconstructed - h_periodic(kx, ky, params)))),
)
 return maximum

def surface_green_1985(
 energy: float,
 h00: np.ndarray,
 h01: np.ndarray,
 eta: float,
 tolerance: float,
 max_iter: int,
) -> tuple[np.ndarray, np.ndarray, np.ndarray, int, float]:
 """Iterative left/right surface Green functions (Sancho--Rubio recursion).

 This is a direct Python implementation of the recursion used in the
 WannierTools ``surfgreen_1985`` routine. It returns the retarded Green
 functions of the left surface, right surface and renormalized bulk
 principal layer.
 """
 if eta <= 0.0:
 raise ValueError("surface eta must be positive")
 dimension = h00.shape[0]
 identity = np.eye(dimension, dtype=np.complex128)
 z_identity = complex(float(energy), float(eta)) * identity

 epsilon_bulk = np.array(h00, dtype=np.complex128, copy=True)
 epsilon_left = np.array(h00, dtype=np.complex128, copy=True)
 epsilon_right = np.array(h00, dtype=np.complex128, copy=True)
 alpha = np.array(h01, dtype=np.complex128, copy=True)
 beta = alpha.conj().T
 residual = float("inf")

 for iteration in range(1, max_iter + 1):
 g0 = np.linalg.inv(z_identity - epsilon_bulk)
 alpha_g = alpha @ g0
 beta_g = beta @ g0

 alpha_g_beta = alpha_g @ beta
 beta_g_alpha = beta_g @ alpha
 epsilon_left = epsilon_left + alpha_g_beta
 epsilon_right = epsilon_right + beta_g_alpha
 epsilon_bulk = epsilon_bulk + alpha_g_beta + beta_g_alpha

 alpha_new = alpha_g @ alpha
 beta_new = beta_g @ beta
 residual = max(
 float(np.linalg.norm(alpha_new, ord="fro")),
 float(np.linalg.norm(beta_new, ord="fro")),
)
 alpha, beta = alpha_new, beta_new
 if residual <= tolerance:
 break

 g_left = np.linalg.inv(z_identity - epsilon_left)
 g_right = np.linalg.inv(z_identity - epsilon_right)
 g_bulk = np.linalg.inv(z_identity - epsilon_bulk)
 return g_left, g_right, g_bulk, iteration, residual

def _spectral_trace(green: np.ndarray, indices: Sequence[int] | None = None) -> float:
 if indices is None:
 diagonal = np.diag(green)
 else:
 diagonal = np.diag(green)[list(indices)]
 value = -float(np.imag(np.sum(diagonal))) / np.pi

```



---

```

return max(value, 0.0)

def select_surface_energy_window(sample: pd.Series, requested: float | None) -> float:
 if requested is not None:
 if requested <= 0.0:
 raise ValueError("--surface-energy-window must be positive")
 return float(requested)
 gap = max(float(sample["final_indirect_gap"]), 0.0)
 # Wide enough to expose both bulk continua and the in-gap edge branch, but
 # not so wide that the edge signal becomes visually compressed.
 return max(0.22, min(0.60, 2.2 * gap + 0.08))

def calculate_surface_spectral_function(
 sample: pd.Series,
 nk: int,
 energy_points: int,
 eta: float,
 tolerance: float,
 max_iter: int,
 energy_window: float | None,
) -> dict[str, np.ndarray | float | int | str]:
 """Calculate semi-infinite left/right edge spectral functions."""
 sample_id = str(sample.get("sample_id", sample.name))
 fermi = 0.5 * (float(sample["refined_vbm"]) + float(sample["refined_cbm"]))
 window = select_surface_energy_window(sample, energy_window)
 kx = np.linspace(-np.pi, np.pi, nk)
 kx_over_pi = kx / np.pi
 energy_relative = np.linspace(-window, window, energy_points)
 energy_absolute = energy_relative + fermi

 shape = (energy_points, nk)
 arrays = {
 name: np.zeros(shape, dtype=np.float64)
 for name in (
 "left_total",
 "right_total",
 "bulk_total",
 "left_up",
 "left_down",
 "right_up",
 "right_down",
 "bulk_up",
 "bulk_down",
)
 }
 iteration_counts = np.zeros(shape, dtype=np.int16)
 residuals = np.zeros(shape, dtype=np.float64)

 progress_stride = max(1, nk // 10)
 for k_index, k_value in enumerate(kx):
 if k_index % progress_stride == 0 or k_index == nk - 1:
 print(
 f" surface Green: {sample_id} k={k_index + 1}/{nk}",
 flush=True,
)
 h00, h01 = ribbon_blocks(float(k_value), sample)
 for energy_index, absolute_energy in enumerate(energy_absolute):
 g_left, g_right, g_bulk, iterations, residual = surface_green_1985(
 float(absolute_energy),
 h00,
 h01,
 eta=eta,
 tolerance=tolerance,
 max_iter=max_iter,
)
 arrays["left_total"][energy_index, k_index] = _spectral_trace(g_left)
 arrays["right_total"][energy_index, k_index] = _spectral_trace(g_right)

```

---

```

arrays["bulk_total"][energy_index, k_index] = _spectral_trace(g_bulk)
arrays["left_up"][energy_index, k_index] = _spectral_trace(
 g_left, SPIN_INDICES["up"]
)
arrays["left_down"][energy_index, k_index] = _spectral_trace(
 g_left, SPIN_INDICES["down"]
)
arrays["right_up"][energy_index, k_index] = _spectral_trace(
 g_right, SPIN_INDICES["up"]
)
arrays["right_down"][energy_index, k_index] = _spectral_trace(
 g_right, SPIN_INDICES["down"]
)
arrays["bulk_up"][energy_index, k_index] = _spectral_trace(
 g_bulk, SPIN_INDICES["up"]
)
arrays["bulk_down"][energy_index, k_index] = _spectral_trace(
 g_bulk, SPIN_INDICES["down"]
)
iteration_counts[energy_index, k_index] = iterations
residuals[energy_index, k_index] = residual

WannierTools' surfstat.f90 defines a surface-only quantity by subtracting
the principal-layer bulk DOS and clipping negative values.
arrays["left_only"] = np.maximum(arrays["left_total"] - arrays["bulk_total"], 0.0)
arrays["right_only"] = np.maximum(arrays["right_total"] - arrays["bulk_total"], 0.0)
arrays["left_up_only"] = np.maximum(arrays["left_up"] - arrays["bulk_up"], 0.0)
arrays["left_down_only"] = np.maximum(arrays["left_down"] - arrays["bulk_down"], 0.0)
arrays["right_up_only"] = np.maximum(arrays["right_up"] - arrays["bulk_up"], 0.0)
arrays["right_down_only"] = np.maximum(arrays["right_down"] - arrays["bulk_down"], 0.0)

return {
 "sample_id": sample_id,
 "phase_label": str(sample["phase_label"]),
 "fermi_energy": float(fermi),
 "energy_window": float(window),
 "eta": float(eta),
 "kx": kx,
 "kx_over_pi": kx_over_pi,
 "energy_relative": energy_relative,
 "energy_absolute": energy_absolute,
 "iterations": iteration_counts,
 "residuals": residuals,
 **arrays,
}

def write_wanniertools_style_surface_data(
 result: dict[str, np.ndarray | float | int | str],
 output_prefix: Path,
 write_text: bool,
) -> None:
 """Write NPZ plus text files resembling WannierTools surface outputs."""
 output_prefix.parent.mkdir(parents=True, exist_ok=True)
 np.savez_compressed(output_prefix.with_suffix(".npz"), **result)
 if not write_text:
 return

 k_values = np.asarray(result["kx_over_pi"], dtype=float)
 energies = np.asarray(result["energy_relative"], dtype=float)
 file_specs = {
 "dos.dat_l": ("left_total", "left_only"),
 "dos.dat_r": ("right_total", "right_only"),
 "dos.dat_bulk": ("bulk_total",),
 "spindos.dat_l": ("left_up", "left_down"),
 "spindos.dat_r": ("right_up", "right_down"),
 "spindos_only.dat_l": ("left_up_only", "left_down_only"),
 "spindos_only.dat_r": ("right_up_only", "right_down_only"),
 }

```

---

```

data_dir = output_prefix.parent / f"{output_prefix.name}_wanniertools_dat"
data_dir.mkdir(parents=True, exist_ok=True)
for filename, columns in file_specs.items():
 path = data_dir / filename
 with path.open("w", encoding="utf-8") as handle:
 handle.write("# kx/pi E-EF " + " ".join(columns) + "\n")
 for k_index, kval in enumerate(k_values):
 for energy_index, energy in enumerate(energies):
 values = [float(np.asarray(result[column])[energy_index, k_index]) for column in columns]
 handle.write(
 f"{kval: .10e} {energy: .10e} "
 + " ".join(f"{value: .10e}" for value in values)
 + "\n"
)
 handle.write("\n")

def _log_normalize(intensity: np.ndarray, contrast: float) -> np.ndarray:
 intensity = np.maximum(np.asarray(intensity, dtype=float), 0.0)
 maximum = float(np.max(intensity))
 if maximum <= 0.0:
 return np.zeros_like(intensity)
 scaled = intensity / maximum
 return np.log1p(contrast * scaled) / np.log1p(contrast)

def _wanniertools_surface_cmap() -> LinearSegmentedColormap:
 # Matches the characteristic WannierTools ``surface-only`` palette:
 # white -> red -> black.
 return LinearSegmentedColormap.from_list(
 "wanniertools_surface",
 [(1.0, 1.0, 1.0), (1.0, 0.05, 0.00), (0.0, 0.0, 0.0)],
 N=256,
)

def _spin_rgba(
 up: np.ndarray,
 down: np.ndarray,
 contrast: float,
) -> np.ndarray:
 total = np.maximum(up + down, 0.0)
 alpha = _log_normalize(total, contrast) ** 0.75
 polarization = (up - down) / (total + EPS)
 rgba = plt.get_cmap("bwr")((polarization + 1.0) / 2.0)
 rgba[..., 3] = alpha
 return rgba

def plot_wanniertools_surface_spectrum(
 result: dict[str, np.ndarray | float | int | str],
 sample: pd.Series,
 figures_dir: Path,
 contrast: float,
) -> None:
 """Plot left/right surface LDOS in the visual language of WannierTools."""
 figures_dir.mkdir(parents=True, exist_ok=True)
 sample_id = str(result["sample_id"])
 k_values = np.asarray(result["kx_over_pi"], dtype=float)
 energies = np.asarray(result["energy_relative"], dtype=float)
 extent = [float(k_values[0]), float(k_values[-1]), float(energies[0]), float(energies[-1])]
 cmap = _wanniertools_surface_cmap()

 left = _log_normalize(np.asarray(result["left_only"]), contrast)
 right = _log_normalize(np.asarray(result["right_only"]), contrast)

 fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.8), sharey=True)
 image = None
 for axis, intensity, title in zip(

```

---

```

 axes,
 (left, right),
 ("Left semi-infinite edge", "Right semi-infinite edge"),
):
 image = axis.imshow(
 intensity,
 origin="lower",
 aspect="auto",
 extent=extent,
 cmap=cmap,
 vmin=0.0,
 vmax=1.0,
 interpolation="bilinear",
)
 axis.axhline(0.0, color="0.35", linewidth=0.8, linestyle="--")
 axis.axvline(0.0, color="0.55", linewidth=0.5, linestyle=":")
 axis.set_xticks([-1.0, 0.0, 1.0], [r"$-X$", r"Γ", r"X"])
 axis.set_xlabel(r"edge momentum k_x")
 axis.set_title(title)
axes[0].set_ylabel(r"$E-E_F$")
assert image is not None
fig.colorbar(image, ax=axes.ravel().tolist(), pad=0.02, label="surface spectral weight")
fig.suptitle(
 f"{sample_id} | {sample['phase_label']} | iterative surface Green function"
)
fig.subplots_adjust(left=0.08, right=0.89, bottom=0.14, top=0.86, wspace=0.08)
save_figure(fig, figures_dir / f"{sample_id}_wanniertools_surfdos_left_right")
plt.close(fig)

Spin-resolved map. Red denotes spin-up and blue denotes spin-down; opacity
is the local surface-only spectral intensity.
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.8), sharey=True)
for axis, up_key, down_key, title in (
 (axes[0], "left_up_only", "left_down_only", "Left edge"),
 (axes[1], "right_up_only", "right_down_only", "Right edge"),
):
 rgba = _spin_rgba(
 np.asarray(result[up_key], dtype=float),
 np.asarray(result[down_key], dtype=float),
 contrast,
)
 axis.imshow(
 rgba,
 origin="lower",
 aspect="auto",
 extent=extent,
 interpolation="bilinear",
)
 axis.axhline(0.0, color="0.35", linewidth=0.8, linestyle="--")
 axis.axvline(0.0, color="0.55", linewidth=0.5, linestyle=":")
 axis.set_xticks([-1.0, 0.0, 1.0], [r"$-X$", r"Γ", r"X"])
 axis.set_xlabel(r"edge momentum k_x")
 axis.set_title(title)
axes[0].set_ylabel(r"$E-E_F$")
legend = [
 Line2D([0], [0], linewidth=5, color=(0.85, 0.05, 0.00), label=r"spin $\uparrow$"),
 Line2D([0], [0], linewidth=5, color=(0.00, 0.20, 0.85), label=r"spin $\downarrow$"),
]
axes[1].legend(handles=legend, frameon=False, loc="upper right")
fig.suptitle(f"Spin-resolved surface spectrum | {sample_id}")
fig.subplots_adjust(left=0.08, right=0.97, bottom=0.14, top=0.86, wspace=0.08)
save_figure(fig, figures_dir / f"{sample_id}_wanniertools_surfdos_spin_resolved")
plt.close(fig)

def surface_spectrum_metrics(
 result: dict[str, np.ndarray | float | int | str],
 sample: pd.Series,
) -> dict[str, float | int]:

```

---

```

energies = np.asarray(result["energy_relative"], dtype=float)
left = np.asarray(result["left_only"], dtype=float)
right = np.asarray(result["right_only"], dtype=float)
fermi_index = int(np.argmin(np.abs(energies)))
fermi = float(result["fermi_energy"])
vbm_rel = float(sample["refined_vbm"]) - fermi
cbm_rel = float(sample["refined_cbm"]) - fermi
margin = 3.0 * float(result["eta"])
lower = vbm_rel + margin
upper = cbm_rel - margin
gap_mask = (energies >= lower) & (energies <= upper) if lower < upper else np.zeros_like(energies, dtype=bool)
max_residual = float(np.max(np.asarray(result["residuals"], dtype=float)))
max_iterations = int(np.max(np.asarray(result["iterations"], dtype=int)))
return {
 "surface_eta": float(result["eta"]),
 "energy_window": float(result["energy_window"]),
 "left_weight_at_fermi": float(np.max(left[fermi_index, :])),
 "right_weight_at_fermi": float(np.max(right[fermi_index, :])),
 "left_max_weight_in_bulk_gap": float(np.max(left[gap_mask, :])) if np.any(gap_mask) else float("nan"),
 "right_max_weight_in_bulk_gap": float(np.max(right[gap_mask, :])) if np.any(gap_mask) else float("nan"),
 "max_green_iterations": max_iterations,
 "max_green_residual": max_residual,
}

Finite-ribbon cross-check, corresponding to WannierTools ek_ribbon.f90

def ribbon_hamiltonian(kx: float, params: pd.Series | dict, ny: int) -> np.ndarray:
 onsite, hopping = ribbon_blocks(kx, params)
 dimension = 4 * ny
 hamiltonian = np.zeros((dimension, dimension), dtype=np.complex128)
 for cell in range(ny):
 block = slice(4 * cell, 4 * (cell + 1))
 hamiltonian[block, block] = onsite
 if cell < ny - 1:
 next_block = slice(4 * (cell + 1), 4 * (cell + 2))
 hamiltonian[block, next_block] = hopping
 hamiltonian[next_block, block] = hopping.conj().T
 return hamiltonian

def ribbon_spin_block(
 kx: float, params: pd.Series | dict, ny: int, spin: str
) -> np.ndarray:
 full = ribbon_hamiltonian(kx, params, ny)
 within_cell = SPIN_INDICES[spin]
 indices = [4 * cell + orbital for cell in range(ny) for orbital in within_cell]
 return full[np.ix_(indices, indices)]

def calculate_spin_resolved_ribbon(
 sample: pd.Series,
 ny: int,
 nk: int,
 edge_cells: int,
) -> pd.DataFrame:
 sample_id = str(sample.get("sample_id", sample.name))
 k_values = np.linspace(-np.pi, np.pi, nk)
 rows = []
 block_dimension = 2 * ny
 left_indices = np.arange(0, 2 * edge_cells)
 right_indices = np.arange(2 * (ny - edge_cells), 2 * ny)
 for k_index, kx in enumerate(k_values):
 states = []
 for spin_name, spin_value in (("up", 1), ("down", -1)):
 energies, eigenvectors = eigh(

```

---

```

 ribbon_spin_block(float(kx), sample, ny, spin_name)
)
 probabilities = np.abs(eigenvectors) ** 2
 left_weight = probabilities[left_indices, :].sum(axis=0)
 right_weight = probabilities[right_indices, :].sum(axis=0)
 for local_band in range(block_dimension):
 states.append(
 {
 "energy": float(energies[local_band]),
 "spin": spin_name,
 "spin_value": spin_value,
 "left_weight": float(left_weight[local_band]),
 "right_weight": float(right_weight[local_band]),
 "total_edge_weight": float(
 left_weight[local_band] + right_weight[local_band]
),
 }
)
 states.sort(key=lambda row: row["energy"])
 for band_index, state in enumerate(states):
 total_edge = state["total_edge_weight"]
 edge_polarization = (
 (state["left_weight"] - state["right_weight"]) / total_edge
 if total_edge > EPS
 else 0.0
)
 rows.append(
 {
 "sample_id": sample_id,
 "phase_label": sample["phase_label"],
 "ny": ny,
 "k_index": k_index,
 "kx": float(kx),
 "kx_over_pi": float(kx / np.pi),
 "band_index": band_index,
 **state,
 "edge_polarization": float(edge_polarization),
 }
)
)
 spectrum = pd.DataFrame(rows)
 fermi = 0.5 * (float(sample["refined_vbm"]) + float(sample["refined_cbm"]))
 spectrum["fermi_energy"] = fermi
 spectrum["energy_relative_to_fermi"] = spectrum["energy"] - fermi
 return spectrum

def plot_finite_ribbon_wanniertools_style(
 spectrum: pd.DataFrame,
 sample: pd.Series,
 ny: int,
 figures_dir: Path,
) -> None:
 """Exact ribbon eigenvalues colored by edge weight, like ribbonek.gnu."""
 figures_dir.mkdir(parents=True, exist_ok=True)
 sample_id = str(sample.get("sample_id", sample.name))
 gap = float(sample["final_indirect_gap"])
 energy_window = max(0.22, min(0.60, 2.2 * gap + 0.08))
 edge_cmap = LinearSegmentedColormap.from_list(
 "wanniertools_ribbon",
 [(0.0, 0.45, 0.0), (1.0, 0.85, 0.0), (0.85, 0.0, 0.0)],
 N=256,
)

 fig, ax = plt.subplots(figsize=(7.0, 5.4))
 image = ax.scatter(
 spectrum["kx_over_pi"],
 spectrum["energy_relative_to_fermi"],
 c=np.clip(spectrum["total_edge_weight"], 0.0, 1.0),
 cmap=edge_cmap,

```

---

```

 vmin=0.0,
 vmax=1.0,
 s=2.2,
 linewidths=0.0,
 rasterized=True,
)
 ax.axhline(0.0, color="0.35", linewidth=0.8, linestyle="--")
 ax.set_xlim(-1.0, 1.0)
 ax.set_ylim(-energy_window, energy_window)
 ax.set_xticks([-1.0, 0.0, 1.0], [r"$-X$", r"Γ", r"X"])
 ax.set_xlabel(r"edge momentum k_x")
 ax.set_ylabel(r"$E-E_F$")
 ax.set_title(f"Finite ribbon | {sample_id} | Ny={ny}")
 fig.colorbar(image, ax=ax, label="left + right edge weight")
 fig.tight_layout()
 save_figure(fig, figures_dir / f"{sample_id}_Ny{ny}_wanniertools_ribbonek")
 plt.close(fig)

def ribbon_width_metrics(
 spectrum: pd.DataFrame,
 edge_threshold: float,
) -> dict[str, float]:
 edge = spectrum[spectrum["total_edge_weight"] >= edge_threshold]
 if edge.empty:
 return {
 "n_edge_states": 0,
 "min_abs_edge_energy": float("nan"),
 "apparent_edge_gap": float("nan"),
 "max_edge_weight_near_fermi": 0.0,
 "n_edge_states_within_0p01": 0,
 }
 min_abs = float(edge["energy_relative_to_fermi"].abs().min())
 near = spectrum[spectrum["energy_relative_to_fermi"].abs() <= 0.01]
 return {
 "n_edge_states": int(len(edge)),
 "min_abs_edge_energy": min_abs,
 "apparent_edge_gap": 2.0 * min_abs,
 "max_edge_weight_near_fermi": (
 float(near["total_edge_weight"].max()) if len(near) else 0.0
),
 "n_edge_states_within_0p01": int(
 (edge["energy_relative_to_fermi"].abs() <= 0.01).sum()
),
 }

def run_edge_analysis(
 master: pd.DataFrame,
 selection_file: Path,
 settings: Settings,
 output_dir: Path,
 figures_dir: Path,
) -> tuple[pd.DataFrame, pd.DataFrame]:
 selection = pd.read_csv(selection_file)
 ensure_columns(selection, ["role", "sample_id"], "Step 07 ribbon selection")
 master_index = master.set_index("sample_id")

 validation_sample = master_index.loc[selection.iloc[0]["sample_id"]]
 error = ribbon_reconstruction_error(validation_sample, settings.random_seed)
 write_json(
 output_dir / "step08_05_principal_layer_bloch_reconstruction.json",
 {"maximum_reconstruction_error": error, "passed": bool(error < 1.0e-10)},
)
 if error >= 1.0e-10:
 raise RuntimeError(f"Principal-layer Bloch reconstruction failed: {error}")

 surface_metric_rows: list[dict] = []
 ribbon_metric_rows: list[dict] = []

```

---

```

if settings.edge_method in {"green", "both"}:
 surface_dir = output_dir / "surface_green_data"
 surface_figures = figures_dir / "wanniertools_surface_green"
 surface_dir.mkdir(parents=True, exist_ok=True)
 surface_figures.mkdir(parents=True, exist_ok=True)
 for row in selection.itertuples(index=False):
 sample = master_index.loc[row.sample_id]
 result = calculate_surface_spectral_function(
 sample,
 nk=settings.surface_nk,
 energy_points=settings.surface_energy_points,
 eta=settings.surface_eta,
 tolerance=settings.surface_tol,
 max_iter=settings.surface_max_iter,
 energy_window=settings.surface_energy_window,
)
 prefix = surface_dir / f"step08_05_{row.sample_id}_surface_green"
 write_wanniertools_style_surface_data(
 result, prefix, write_text=settings.write_surface_text
)
 plot_wanniertools_surface_spectrum(
 result,
 sample,
 surface_figures,
 contrast=settings.surface_log_contrast,
)
 surface_metric_rows.append(
 {
 "role": row.role,
 "sample_id": row.sample_id,
 "phase_label": sample["phase_label"],
 "nk": settings.surface_nk,
 "energy_points": settings.surface_energy_points,
 **surface_spectrum_metrics(result, sample),
 }
)

if settings.edge_method in {"ribbon", "both"}:
 ribbon_dir = output_dir / "ribbon_data"
 ribbon_figures = figures_dir / "wanniertools_finite_ribbon"
 ribbon_dir.mkdir(parents=True, exist_ok=True)
 ribbon_figures.mkdir(parents=True, exist_ok=True)
 largest_width = max(settings.ribbon_widths)
 for row in selection.itertuples(index=False):
 sample = master_index.loc[row.sample_id]
 for ny in settings.ribbon_widths:
 edge_cells = min(settings.edge_cells, max(1, ny // 6))
 spectrum = calculate_spin_resolved_ribbon(
 sample,
 ny=ny,
 nk=settings.ribbon_nk,
 edge_cells=edge_cells,
)
 spectrum.to_csv(
 ribbon_dir
 / f"step08_05_{row.sample_id}_Ny{ny}_spin_left_right_spectrum.csv",
 index=False,
)
 ribbon_metric_rows.append(
 {
 "role": row.role,
 "sample_id": row.sample_id,
 "phase_label": sample["phase_label"],
 "ny": ny,
 "nk": settings.ribbon_nk,
 "edge_cells": edge_cells,
 **ribbon_width_metrics(spectrum, settings.edge_threshold),
 }
)

```



```

)
 if ny in {40, largest_width}:
 plot_finite_ribbon_wanniertools_style(
 spectrum, sample, ny, ribbon_figures
)

surface_metrics = pd.DataFrame(surface_metric_rows)
ribbon_metrics = pd.DataFrame(ribbon_metric_rows)
surface_metrics.to_csv(
 output_dir / "step08_05_surface_green_metrics.csv", index=False
)
ribbon_metrics.to_csv(
 output_dir / "step08_05_ribbon_width_convergence.csv", index=False
)

if len(ribbon_metrics):
 fig, ax = plt.subplots(figsize=(7.2, 5.2))
 for role, part in ribbon_metrics.groupby("role"):
 ax.plot(part["ny"], part["apparent_edge_gap"], marker="o", label=role)
 ax.set_xlabel(r"ribbon width N_y")
 ax.set_ylabel(r"$2\min|E_{\rm edge}-E_F|$")
 ax.set_title("Finite-width convergence of the edge crossing")
 ax.legend(frameon=False)
 fig.tight_layout()
 save_figure(fig, figures_dir / "step08_05_ribbon_width_convergence")
 plt.close(fig)

return surface_metrics, ribbon_metrics

def write_current_phase_taxonomy(master: pd.DataFrame, output_dir: Path) -> dict[str, object]:
 """Record exactly what Step 01--08 has and has not classified.

 The present labels are first-order spin-Chern labels. The two signs of the
 spin Chern number are retained as distinct sectors instead of being hidden
 inside one binary type-II-QSH target. Real-Chern / Stiefel-Whitney and
 higher-order labels are deliberately marked as not yet computed.
 """
 labels = []
 for row in master.itertuples(index=False):
 cup = int(row.chern_up_int)
 cdn = int(row.chern_down_int)
 ctot = int(row.chern_total_int)
 if (cup, cdn, ctot) == (0, 0, 0):
 labels.append("trivial_spin_chern")
 elif (cup, cdn, ctot) == (1, -1, 0):
 labels.append("typeII_QSH_Cup_plus")
 elif (cup, cdn, ctot) == (-1, 1, 0):
 labels.append("typeII_QSH_Cup_minus")
 else:
 labels.append(f"other_Cup_{cup}_Cdown_{cdn}_Ctotal_{ctot}")
 table = master[["sample_id", "phase_label", "chern_up_int", "chern_down_int", "chern_total_int"]].copy()
 table["first_order_sector"] = labels
 table.to_csv(output_dir / "step08_07_current_first_order_phase_taxonomy.csv", index=False)
 counts =
 ↪ table["first_order_sector"].value_counts().rename_axis("first_order_sector").reset_index(name="count")
 counts.to_csv(output_dir / "step08_07_current_first_order_phase_counts.csv", index=False)

 scope = {
 "current_model_space": "four-band spin-conserving MSG 123.342 Lieb Hamiltonian at half filling",
 "current_labels_are": [
 "full-BZ metal/boundary/insulator audit",
 "spin-resolved Chern numbers Cup and Cdown",
 "total Chern number",
 "trivial versus type-II QSH",
 "two opposite spin-Chern chirality sectors",
],
 "not_yet_computed": [
 "C2Zt real-Chern / second Stiefel-Whitney invariant",

```

---

```

 "mirror-resolved real topology",
 "symmetry eigenvalue or magnetic-TQC indicator",
 "nested Wilson loop or quantized multipole invariant",
 "finite-square corner spectrum and corner localization",
 "boundary-obstructed versus intrinsic higher-order diagnosis",
],
 "important_rule": (
 "Do not relabel the current binary spin-Chern data as a generic classifier "
 "for all topological insulators or higher-order topological insulators."
),
 "first_order_sector_counts": {
 str(row.first_order_sector): int(row.count) for row in counts.itertuples(index=False)
 },
}
write_json(output_dir / "step08_07_topology_scope_manifest.json", scope)
return scope

=====
Main
=====

def main() -> int:
 args = parse_args()
 context = resolve_step07_context(args.step07_dir)

 inherited_seed = infer_seed(context)
 inherited_pairs = infer_interpolation_pairs(context)
 inherited_widths = infer_ribbon_widths(context)
 random_seed = args.random_seed if args.random_seed is not None else inherited_seed
 interpolation_pairs = (
 args.interpolation_pairs
 if args.interpolation_pairs is not None
 else inherited_pairs
)
 ribbon_widths = (
 tuple(sorted(set(args.ribbon_widths)))
 if args.ribbon_widths is not None
 else inherited_widths
)
 if any(width < 4 for width in ribbon_widths):
 raise ValueError(f"All ribbon widths must be >= 4: {ribbon_widths}")

 if args.quick:
 settings = Settings(
 random_seed=random_seed,
 n_splits=5,
 test_size=DEFAULT_TEST_SIZE,
 interpolation_pairs=min(2, interpolation_pairs),
 root_grid=501,
 bz_nk=80,
 chern_grids=(15, 21),
 chern_shifts=((0.0, 0.0), (0.5, 0.5)),
 edge_method=("green" if args.skip_ribbon and args.edge_method == "both" else args.edge_method),
 surface_nk=(args.surface_nk or 61),
 surface_energy_points=(args.surface_energy_points or 201),
 surface_eta=(args.surface_eta or 0.006),
 surface_max_iter=DEFAULT_SURFACE_MAX_ITER,
 surface_tol=1.0e-12,
 surface_log_contrast=DEFAULT_SURFACE_LOG_CONTRAST,
 surface_energy_window=args.surface_energy_window,
 write_surface_text=args.write_surface_text,
 ribbon_widths=tuple(ribbon_widths[:2]),
 ribbon_nk=61,
 edge_cells=2,
 edge_threshold=DEFAULT_EDGE_THRESHOLD,
 quick=True,
)

```

---

```

else:
 settings = Settings(
 random_seed=random_seed,
 n_splits=DEFAULT_N_SPLITS,
 test_size=DEFAULT_TEST_SIZE,
 interpolation_pairs=interpolation_pairs,
 root_grid=DEFAULT_ROOT_GRID,
 bz_nk=DEFAULT_BZ_NK,
 chern_grids=DEFAULT_CHERN_GRIDS,
 chern_shifts=DEFAULT_CHERN_SHIFTS,
 edge_method=("green" if args.skip_ribbon and args.edge_method == "both" else args.edge_method),
 surface_nk=(args.surface_nk or DEFAULT_SURFACE_NK),
 surface_energy_points=(args.surface_energy_points or DEFAULT_SURFACE_ENERGY_POINTS),
 surface_eta=(args.surface_eta or DEFAULT_SURFACE_ETA),
 surface_max_iter=DEFAULT_SURFACE_MAX_ITER,
 surface_tol=DEFAULT_SURFACE_TOL,
 surface_log_contrast=DEFAULT_SURFACE_LOG_CONTRAST,
 surface_energy_window=args.surface_energy_window,
 write_surface_text=args.write_surface_text,
 ribbon_widths=ribbon_widths,
 ribbon_nk=DEFAULT_RIBBON_NK,
 edge_cells=DEFAULT_EDGE_CELLS,
 edge_threshold=DEFAULT_EDGE_THRESHOLD,
 quick=False,
)

output_name = (
 "outputs_step08_analytic_boundary_wanniertools_edge_"
 f"{context.run_tag}{'_quick' if settings.quick else ''}"
)
output_dir = (
 args.output_dir.expanduser().resolve()
 if args.output_dir is not None
 else (context.source_parent / output_name).resolve()
)
figures_dir = output_dir / "figures"
output_dir.mkdir(parents=True, exist_ok=True)
figures_dir.mkdir(parents=True, exist_ok=True)

master_file = context.require(
 "analysis_master",
 "step07_01_consensus_and_engineered_feature_master__*.csv",
)
pair_file = context.require(
 "nearest_pairs",
 "step07_04_nearest_topology_trivial_parameter_pairs__*.csv",
)
required_files = {
 "analysis_master": master_file,
 "nearest_pairs": pair_file,
}

master = pd.read_csv(master_file)
pair_table = pd.read_csv(pair_file)
ensure_columns(
 master,
 [
 "sample_id",
 "phase_label",
 "target_typeII",
 "parameter_cluster",
 "final_direct_gap",
 "final_indirect_gap",
 "refined_vbm",
 "refined_cbm",
 *PHYS7,
 *RAW8,
 "spin_splitting_1",
 "spin_splitting_2",
]

```

---

```

],
 "Step 07 master",
)
if len(master) == 0:
 raise ValueError("Step 07 analysis master is empty.")
if master["sample_id"].duplicated().any():
 duplicated = master.loc[master["sample_id"].duplicated(), "sample_id"].tolist()
 raise ValueError(f"Duplicate sample_id values in Step 07 master: {duplicated[:10]}")
ensure_columns(
 pair_table,
 ["topology_sample_id", "trivial_sample_id", "standardized_distance"],
 "Step 07 nearest-pair table",
)
if args.skip_edge:
 ribbon_selection_file = context.files.get("ribbon_selection")
 edge_selection_source = "not_required_skip_edge"
else:
 ribbon_selection_file, edge_selection_source = resolve_edge_selection_file(
 context, master, pair_table, output_dir
)
 required_files["edge_pair_selection"] = ribbon_selection_file
write_input_compatibility_report(context, output_dir, required_files)

analytic = add_analytic_features(master)
analytic.to_csv(output_dir / "step08_01_analytic_feature_master.csv", index=False)
exact_rule_metrics, exact_rule_table = audit_exact_mass_product_rule(analytic, output_dir)
topology_scope = write_current_phase_taxonomy(analytic, output_dir)
plot_mechanism_overview(analytic, figures_dir)
stats = mass_boundary_statistics(analytic, output_dir)
split_df, model_summary, coefficient_summary = grouped_logistic_comparison(
 analytic, settings, output_dir
)

roots, paths = refine_transition_paths(
 analytic,
 pair_table,
 settings,
 output_dir,
 figures_dir,
 skip_chern=args.skip_chern,
)

if args.skip_edge:
 surface_metrics = pd.DataFrame()
 ribbon_metrics = pd.DataFrame()
else:
 surface_metrics, ribbon_metrics = run_edge_analysis(
 analytic,
 ribbon_selection_file,
 settings,
 output_dir,
 figures_dir,
)

targeted_table, targeted_summary = targeted_boundary_validation(
 args.targeted_samples,
 args.targeted_width,
 settings,
 output_dir,
)

summary = {
 "step07_dir": str(context.directory),
 "step07_input_mode": "existing_directory_only",
 "step07_run_tag": context.run_tag,
 "zip_reading_enabled": False,
 "edge_selection_source": edge_selection_source,
 "output_dir": str(output_dir),
 "settings": settings.__dict__,

```

```

 "n_high_confidence_insulators": int(len(analytic)),
 "n_typeII": int(analytic["target_typeII"].sum()),
 "n_trivial": int((1 - analytic["target_typeII"]).sum()),
 "analytic_mass_boundaries": [
 "M_plus = m_e + d1 + d2 = 0",
 "M_minus = -m_e + d1 + d2 = 0",
],
 "n_refined_root_candidates": int(len(roots)),
 "exact_mass_product_rule": exact_rule_metrics,
 "topology_scope": topology_scope,
 "targeted_validation": targeted_summary,
 "model_summary": model_summary.to_dict(orient="records"),
 "surface_green_metrics": surface_metrics.to_dict(orient="records"),
 "ribbon_width_metrics": ribbon_metrics.to_dict(orient="records"),
 "important_interpretation": [
 "Delta_scale and Delta_log_anisotropy replace the redundant Delta_min/product/imbalance trio.",
 "M_plus and M_minus are two distinct X/Y mass branches.",
 "A zero of t1 or t2 is only a candidate boundary; the axis d_z condition must also be satisfied.",
 "The primary edge calculation is the semi-infinite iterative surface Green function used by

 ↪ WannierTools.",
 "Finite-ribbon diagonalization is retained as an independent width-convergence check.",
 "Surface-only maps use surface spectral weight minus the bulk principal-layer contribution, following

 ↪ surfstat.f90.",
 "The full-BZ grid is always even and contains Gamma, X, Y, and M exactly.",
 "Chern numbers beside a closing are evaluated at the largest safe insulating offset, not arbitrarily

 ↪ close to the singular point.",
 "Current labels diagnose first-order spin-Chern topology only; higher-order and real topology require

 ↪ new invariants and finite-square calculations.",
],
]
}
write_json(output_dir / "step08_00_run_summary.json", summary)
write_step08_registry(output_dir, context.run_tag)

print("\nStep 08 v5 completed successfully.")
print(f"Input : {context.directory}")
print(f"Run tag: {context.run_tag}")
print(f"Output: {output_dir}")
print("\nExact audited-insulator rule:")
print(json.dumps(exact_rule_metrics, indent=2, ensure_ascii=False))
print("\nModel comparison:")
print(model_summary.to_string(index=False))
print("\nRefined root candidates:", len(roots))
if len(surface_metrics):
 print("\nSemi-infinite surface Green-function metrics:")
 print(surface_metrics.to_string(index=False))
if len(ribbon_metrics):
 print("\nFinite-ribbon width convergence:")
 print(ribbon_metrics.to_string(index=False))
return 0

if __name__ == "__main__":
 try:
 raise SystemExit(main())
 except Exception as exc:
 print(f"ERROR: {exc}", file=sys.stderr)
 raise

```

## ► 代码清单 A.48.2 (python, 886 行)

```

#!/usr/bin/env python3
-*- coding: utf-8 -*-
"""

```

---

Step 09: Analytic Chern formula and three-sector phase map for the spin-conserving MSG 123.342 Lieb Hamiltonian.

This script reads an EXISTING Step 08 output directory directly. It never reads or extracts ZIP archives.

Main result audited by this script

-----

For nonzero  $t_1$ ,  $t_2$  and nonzero Dirac masses

$$\begin{aligned}M_{\text{plus}} &= m_e + d_1 + d_2, \\M_{\text{minus}} &= -m_e + d_1 + d_2,\end{aligned}$$

we test the model-specific analytic formula

$$\begin{aligned}C_{\text{up}} &= -\text{sgn}(t_1 t_2)/2 * [\text{sgn}(M_{\text{plus}}) + \text{sgn}(M_{\text{minus}})], \\C_{\text{down}} &= -C_{\text{up}}, \\C_{\text{total}} &= 0.\end{aligned}$$

The formula follows from the two spin-block d-vectors and the massive-Dirac contributions at  $X=(\pi,0)$  and  $Y=(0,\pi)$ . The script validates the formula against all reliable numerical Chern labels available in Step 08, separately for the original Sobol set and the independent targeted-boundary set.

Outputs include:

- exact spin-block d-vector formulas (JSON and LaTeX),
  - Dirac point masses, Jacobian chiralities and Chern contributions,
  - analytic-vs-numerical Chern master table,
  - formula metrics and mismatch samples,
  - three-sector phase taxonomy,
  - figures suitable for mechanism analysis,
  - a concise research-significance manifest.
- """

```
from __future__ import annotations
```

```
import argparse
import json
import math
import sys
from dataclasses import dataclass
from pathlib import Path
from typing import Iterable, Sequence
```

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

```
EPS = 1.0e-12
BOUNDARY_TOL = 1.0e-10
STEP08_PREFIX = "outputs_step08_analytic_boundary_wanniertools_edge_"
STEP09_PREFIX = "outputs_step09_analytic_chern_formula_"
```

```
REQUIRED_BASE_COLUMNS = [
 "sample_id", "m_e", "t1", "t2", "s1", "d1", "s2", "d2",
 "M_plus", "M_minus", "chern_up_int", "chern_down_int",
 "chern_total_int", "phase_label", "final_direct_gap",
 "final_indirect_gap", "chern_reliable",
]
```

```
REQUIRED_TARGET_COLUMNS = [
 "targeted_sample_id", "sampling_family", "m_e", "t1", "t2",
 "s1", "d1", "s2", "d2", "M_plus", "M_minus",
 "chern_up_int", "chern_down_int", "chern_total_int", "phase_label",
 "fine_direct_gap", "fine_indirect_gap", "gap_high_confidence",
 "chern_reliable",
]
```

---

```

@dataclass(frozen=True)
class Paths:
 step08_dir: Path
 output_dir: Path
 figures_dir: Path

def parse_args() -> argparse.Namespace:
 parser = argparse.ArgumentParser(
 description=(
 "Read an existing Step 08 directory and derive/audit the exact "
 "spin-resolved Chern formula for the MSG 123.342 Lieb model."
)
)
 parser.add_argument(
 "--step08-dir",
 type=Path,
 default=None,
 help=(
 "Existing Step 08 output directory. If omitted, the newest directory "
 f"matching '{STEP08_PREFIX}*' is auto-detected beside this script or "
 "in the current working directory. ZIP files are rejected."
),
)
 parser.add_argument(
 "--output-dir",
 type=Path,
 default=None,
 help="Optional Step 09 output directory.",
)
 parser.add_argument(
 "--boundary-tol",
 type=float,
 default=BOUNDARY_TOL,
 help="Absolute tolerance used to mark formula-boundary samples.",
)
 parser.add_argument(
 "--validation-kpoints",
 type=int,
 default=64,
 help="Random k points per selected sample for d-vector reconstruction audit.",
)
 parser.add_argument(
 "--validation-samples",
 type=int,
 default=40,
 help="Number of samples used in the Hamiltonian reconstruction audit.",
)
 parser.add_argument("--seed", type=int, default=20260711)
 return parser.parse_args()

def reject_zip(path: Path) -> None:
 if path.suffix.lower() == ".zip" or path.is_file():
 raise ValueError(
 f"Step 09 requires an existing Step 08 DIRECTORY, not a file/ZIP: {path}"
)

def candidate_roots() -> list[Path]:
 roots = [Path.cwd(), Path(__file__).resolve().parent]
 unique: list[Path] = []
 for root in roots:
 root = root.resolve()
 if root not in unique:
 unique.append(root)
 return unique

```

---

```

def discover_step08_dir(explicit: Path | None) -> Path:
 if explicit is not None:
 path = explicit.expanduser().resolve()
 reject_zip(path)
 if not path.is_dir():
 raise FileNotFoundError(f"Step 08 directory does not exist: {path}")
 return path

 candidates: list[Path] = []
 for root in candidate_roots():
 candidates.extend(p for p in root.glob(f"{STEP08_PREFIX}*") if p.is_dir())
 if not candidates:
 searched = ", ".join(str(p) for p in candidate_roots())
 raise FileNotFoundError(
 f"No Step 08 directory matching '{STEP08_PREFIX}*' was found in: {searched}"
)
 return max(candidates, key=lambda p: p.stat().st_mtime).resolve()

def infer_run_tag(step08_dir: Path) -> str:
 name = step08_dir.name
 if name.startswith(STEP08_PREFIX):
 return name[len(STEP08_PREFIX):]
 return name

def make_paths(args: argparse.Namespace) -> Paths:
 step08_dir = discover_step08_dir(args.step08_dir)
 run_tag = infer_run_tag(step08_dir)
 output_dir = (
 args.output_dir.expanduser().resolve()
 if args.output_dir is not None
 else step08_dir.parent / f"{STEP09_PREFIX}{run_tag}"
)
 output_dir.mkdir(parents=True, exist_ok=True)
 figures_dir = output_dir / "figures"
 figures_dir.mkdir(parents=True, exist_ok=True)
 return Paths(step08_dir, output_dir, figures_dir)

def write_json(path: Path, payload: dict) -> None:
 path.write_text(
 json.dumps(payload, indent=2, ensure_ascii=False, default=float),
 encoding="utf-8",
)

def save_figure(fig: plt.Figure, stem: Path) -> None:
 stem.parent.mkdir(parents=True, exist_ok=True)
 for suffix, dpi in (("png", 320), ("pdf", None), ("svg", None)):
 kwargs = {"bbox_inches": "tight"}
 if dpi is not None:
 kwargs["dpi"] = dpi
 fig.savefig(stem.with_suffix(suffix), **kwargs)
 plt.close(fig)

def find_required_file(directory: Path, exact_name: str) -> Path:
 path = directory / exact_name
 if path.exists():
 return path
 matches = list(directory.glob(exact_name.replace(".csv", "*.csv")))
 if len(matches) == 1:
 return matches[0]
 raise FileNotFoundError(f"Required Step 08 file not found: {path}")

def require_columns(df: pd.DataFrame, columns: Sequence[str], label: str) -> None:

```



---

```

missing = [column for column in columns if column not in df.columns]
if missing:
 raise KeyError(f"{label} is missing required columns: {missing}")

def load_step08_data(paths: Paths) -> tuple[pd.DataFrame, pd.DataFrame, dict]:
 base_path = find_required_file(paths.step08_dir, "step08_01_analytic_feature_master.csv")
 target_path = find_required_file(
 paths.step08_dir, "step08_06_targeted_boundary_validation_samples.csv"
)
 summary_path = paths.step08_dir / "step08_00_run_summary.json"

 base = pd.read_csv(base_path)
 target = pd.read_csv(target_path)
 require_columns(base, REQUIRED_BASE_COLUMNS, "Step 08 analytic master")
 require_columns(target, REQUIRED_TARGET_COLUMNS, "Step 08 targeted samples")

 base = base[
 (base["chern_reliable"] == 1)
 & base["phase_label"].isin(["typeII_QSH", "trivial_insulator"])
].copy()
 base["source_dataset"] = "sobol_global"
 base["source_sample_id"] = base["sample_id"].astype(str)
 base["direct_gap"] = pd.to_numeric(base["final_direct_gap"], errors="coerce")
 base["indirect_gap"] = pd.to_numeric(base["final_indirect_gap"], errors="coerce")

 target = target[
 (target["gap_high_confidence"] == 1)
 & (target["chern_reliable"] == 1)
 & target["phase_label"].isin(["typeII_QSH", "trivial_insulator"])
].copy()
 target["source_dataset"] = "targeted_boundary"
 target["source_sample_id"] = target["targeted_sample_id"].astype(str)
 target["direct_gap"] = pd.to_numeric(target["fine_direct_gap"], errors="coerce")
 target["indirect_gap"] = pd.to_numeric(target["fine_indirect_gap"], errors="coerce")

 common = [
 "source_dataset", "source_sample_id", "m_e", "t1", "t2", "s1", "d1",
 "s2", "d2", "M_plus", "M_minus", "chern_up_int", "chern_down_int",
 "chern_total_int", "phase_label", "direct_gap", "indirect_gap",
]
 if "sampling_family" not in base.columns:
 base["sampling_family"] = "global_sobol"
 common.append("sampling_family")

 combined = pd.concat([base[common], target[common]], ignore_index=True)
 summary = json.loads(summary_path.read_text(encoding="utf-8")) if summary_path.exists() else {}
 return base, target, {"combined": combined, "step08_summary": summary}

Hamiltonian and d-vector formulas in the atomic gauge

def raw_from_phys(row: pd.Series | dict) -> dict[str, float]:
 me = float(row["m_e"])
 s1, d1, s2, d2 = [float(row[k]) for k in ("s1", "d1", "s2", "d2")]
 return {
 "e1": me,
 "e2": -me,
 "t1": float(row["t1"]),
 "t2": float(row["t2"]),
 "r1": 0.5 * (s1 + d1),
 "r3": 0.5 * (s1 - d1),
 "r2": 0.5 * (s2 + d2),
 "r4": 0.5 * (s2 - d2),
 }

```

---

```

def h_atomic(kx: float, ky: float, row: pd.Series | dict) -> np.ndarray:
 p = raw_from_phys(row)
 e1, e2 = p["e1"], p["e2"]
 t1, t2 = p["t1"], p["t2"]
 r1, r2, r3, r4 = p["r1"], p["r2"], p["r3"], p["r4"]
 a = e1 + 2 * r1 * np.cos(ky) + 2 * r3 * np.cos(kx)
 b = e2 + 2 * r2 * np.cos(ky) + 2 * r4 * np.cos(kx)
 c = e2 + 2 * r2 * np.cos(kx) + 2 * r4 * np.cos(ky)
 d = e1 + 2 * r1 * np.cos(kx) + 2 * r3 * np.cos(ky)
 cp = np.cos((kx + ky) / 2)
 cm = np.cos((kx - ky) / 2)
 u = 2 * (-1j * t1 + t2) * cp + 2 * (1j * t1 + t2) * cm
 v = 2 * (-1j * t1 + t2) * cm + 2 * (1j * t1 + t2) * cp
 return np.array(
 [[a, 0, u, 0], [0, b, 0, v], [np.conj(u), 0, c, 0], [0, np.conj(v), 0, d]],
 dtype=complex,
)

def dvector(kx: float, ky: float, row: pd.Series | dict, spin: str) -> tuple[float, float, float, float]:
 me, t1, t2, s1, d1, s2, d2 = [
 float(row[k]) for k in ("m_e", "t1", "t2", "s1", "d1", "s2", "d2")
]
 sx, sy = np.sin(kx / 2), np.sin(ky / 2)
 cx, cy = np.cos(kx / 2), np.cos(ky / 2)
 dx = 4 * t2 * cx * cy
 S = s1 - s2
 D = d1 + d2

 if spin == "up":
 d0 = 0.5 * (
 (s1 - d1 + s2 + d2) * np.cos(kx)
 + (s1 + d1 + s2 - d2) * np.cos(ky)
)
 dy = -4 * t1 * sx * sy
 dz = me + 0.5 * (S - D) * np.cos(kx) + 0.5 * (S + D) * np.cos(ky)
 elif spin == "down":
 d0 = 0.5 * (
 (s2 - d2 + s1 + d1) * np.cos(kx)
 + (s2 + d2 + s1 - d1) * np.cos(ky)
)
 dy = 4 * t1 * sx * sy
 dz = -me - 0.5 * (S + D) * np.cos(kx) + 0.5 * (-S + D) * np.cos(ky)
 else:
 raise ValueError("spin must be 'up' or 'down'")
 return float(d0), float(dx), float(dy), float(dz)

def block_from_dvector(kx: float, ky: float, row: pd.Series | dict, spin: str) -> np.ndarray:
 d0, dx, dy, dz = dvector(kx, ky, row, spin)
 return np.array(
 [[d0 + dz, dx - 1j * dy], [dx + 1j * dy, d0 - dz]], dtype=complex
)

def reconstruction_audit(
 df: pd.DataFrame, n_samples: int, n_kpoints: int, seed: int
) -> dict[str, float | int]:
 rng = np.random.default_rng(seed)
 if len(df) == 0:
 raise ValueError("No samples available for reconstruction audit")
 chosen = df.sample(min(n_samples, len(df)), random_state=seed)
 errors: list[float] = []
 for _, row in chosen.iterrows():
 for _ in range(n_kpoints):
 kx, ky = rng.uniform(-np.pi, np.pi, size=2)
 full = h_atomic(float(kx), float(ky), row)
 for spin, indices in (("up", [0, 2]), ("down", [1, 3])):

```

---

```

 exact = full[np.ix_(indices, indices)]
 reconstructed = block_from_dvector(float(kx), float(ky), row, spin)
 errors.append(float(np.max(np.abs(exact - reconstructed))))
 return {
 "n_samples": int(len(chosen)),
 "n_kpoints_per_sample": int(n_kpoints),
 "n_spin_block_comparisons": int(len(errors)),
 "max_abs_error": float(np.max(errors)),
 "mean_abs_error": float(np.mean(errors)),
 }

Analytic Chern formula

def sign_with_boundary(values: Iterable[float], tol: float) -> np.ndarray:
 arr = np.asarray(list(values), dtype=float)
 out = np.zeros(arr.shape, dtype=int)
 out[arr > tol] = 1
 out[arr < -tol] = -1
 return out

def sector_from_chern(cup: int, cdn: int, ctot: int) -> str:
 if (cup, cdn, ctot) == (1, -1, 0):
 return "typeII_QSH_Cup_plus"
 if (cup, cdn, ctot) == (-1, 1, 0):
 return "typeII_QSH_Cup_minus"
 if (cup, cdn, ctot) == (0, 0, 0):
 return "trivial_spin_chern"
 return "other_or_unresolved"

def add_analytic_chern(df: pd.DataFrame, tol: float) -> pd.DataFrame:
 out = df.copy()
 # Recompute masses independently, rather than trusting stored derived columns.
 out["M_plus_recomputed"] = out["m_e"] + out["d1"] + out["d2"]
 out["M_minus_recomputed"] = -out["m_e"] + out["d1"] + out["d2"]
 out["mass_recompute_max_error"] = np.maximum(
 np.abs(out["M_plus"] - out["M_plus_recomputed"]),
 np.abs(out["M_minus"] - out["M_minus_recomputed"]),
)

 sign_mp = sign_with_boundary(out["M_plus_recomputed"], tol)
 sign_mm = sign_with_boundary(out["M_minus_recomputed"], tol)
 sign_t1 = sign_with_boundary(out["t1"], tol)
 sign_t2 = sign_with_boundary(out["t2"], tol)
 sign_tp = sign_t1 * sign_t2

 out["sign_M_plus"] = sign_mp
 out["sign_M_minus"] = sign_mm
 out["sign_t1"] = sign_t1
 out["sign_t2"] = sign_t2
 out["sign_t1t2"] = sign_tp
 out["formula_boundary"] = (
 (sign_mp == 0) | (sign_mm == 0) | (sign_t1 == 0) | (sign_t2 == 0)
).astype(int)

 cup = -0.5 * sign_tp * (sign_mp + sign_mm)
 cup = np rint(cup).astype(int)
 cdn = -cup
 ctot = cup + cdn
 cup[out["formula_boundary"].to_numpy(dtype=bool)] = 99
 cdn[out["formula_boundary"].to_numpy(dtype=bool)] = 99
 ctot[out["formula_boundary"].to_numpy(dtype=bool)] = 99

 out["chern_up_analytic"] = cup

```

---

```

out["chern_down_analytic"] = cdn
out["chern_total_analytic"] = ctot
out["numeric_sector"] = [
 sector_from_chern(int(u), int(d), int(t))
 for u, d, t in zip(out["chern_up_int"], out["chern_down_int"], out["chern_total_int"])
]
out["analytic_sector"] = [
 sector_from_chern(int(u), int(d), int(t)) if int(u) != 99 else "formula_boundary"
 for u, d, t in zip(cup, cdn, ctot)
]
valid = out["formula_boundary"] == 0
out["chern_formula_match"] = 0
out.loc[valid, "chern_formula_match"] = (
 (out.loc[valid, "chern_up_int"].astype(int) == out.loc[valid, "chern_up_analytic"])
 & (out.loc[valid, "chern_down_int"].astype(int) == out.loc[valid, "chern_down_analytic"])
 & (out.loc[valid, "chern_total_int"].astype(int) == out.loc[valid, "chern_total_analytic"])
).astype(int)

out["topology_formula_prediction"] = (
 out["M_plus_recomputed"] * out["M_minus_recomputed"] > 0
).astype(int)
out["numeric_typeII"] = (out["phase_label"] == "typeII_QSH").astype(int)
out["topology_formula_match"] = (
 out["topology_formula_prediction"] == out["numeric_typeII"]
).astype(int)
return out

def classification_metrics(df: pd.DataFrame) -> dict:
 valid = df[df["formula_boundary"] == 0].copy()
 matches = valid["chern_formula_match"].astype(int)
 datasets = {}
 for name, part in valid.groupby("source_dataset"):
 datasets[str(name)] = {
 "n": int(len(part)),
 "exact_chern_matches": int(part["chern_formula_match"].sum()),
 "exact_chern_accuracy": float(part["chern_formula_match"].mean()),
 "topology_accuracy": float(part["topology_formula_match"].mean()),
 "sector_counts_numeric": {
 str(k): int(v) for k, v in part["numeric_sector"].value_counts().to_dict().items()
 },
 }
 return {
 "formula": (
 "C_up = -sgn(t1*t2)/2 * [sgn(M_plus)+sgn(M_minus)]; "
 "C_down=-C_up; C_total=0"
),
 "domain": "gapped spin-conserving model with t1*t2*M_plus*M_minus != 0",
 "n_total_labeled": int(len(df)),
 "n_formula_boundary": int(df["formula_boundary"].sum()),
 "n_formula_valid": int(len(valid)),
 "n_exact_matches": int(matches.sum()),
 "exact_chern_accuracy": float(matches.mean()) if len(matches) else float("nan"),
 "n_mismatches": int((1 - matches).sum()),
 "topology_accuracy": float(valid["topology_formula_match"].mean()),
 "max_mass_recompute_error": float(df["mass_recompute_max_error"].max()),
 "by_dataset": datasets,
 }

def dirac_catalog() -> pd.DataFrame:
 rows = [
 {
 "spin": "up", "point": "X", "kx": "pi", "ky": "0",
 "mass": "M_plus", "jacobian_chirality": "+sgn(t1*t2)",
 "lower_band_contribution": "-1/2 sgn(t1*t2) sgn(M_plus)",
 },
 {
 "spin": "up", "point": "Y", "kx": "0", "ky": "pi",

```

```

 "mass": "-M_minus", "jacobian_chirality": "-sgn(t1*t2)",
 "lower_band_contribution": "-1/2 sgn(t1*t2) sgn(M_minus)",
 },
 {
 "spin": "down", "point": "X", "kx": "pi", "ky": "0",
 "mass": "M_minus", "jacobian_chirality": "-sgn(t1*t2)",
 "lower_band_contribution": "+1/2 sgn(t1*t2) sgn(M_minus)",
 },
 {
 "spin": "down", "point": "Y", "kx": "0", "ky": "pi",
 "mass": "-M_plus", "jacobian_chirality": "+sgn(t1*t2)",
 "lower_band_contribution": "+1/2 sgn(t1*t2) sgn(M_plus)",
 },
]
return pd.DataFrame(rows)

```

```

def formula_payload(reconstruction: dict) -> dict:
 return {
 "basis": ["|1,up>", "|1,down>", "|2,up>", "|2,down>"],
 "spin_block_indices": {"up": [0, 2], "down": [1, 3]},
 "definitions": {
 "S": "s1-s2",
 "D": "d1+d2",
 "M_plus": "m_e+d1+d2",
 "M_minus": "-m_e+d1+d2",
 },
 "up_block": {
 "d_x": "4*t2*cos(kx/2)*cos(ky/2)",
 "d_y": "-4*t1*sin(kx/2)*sin(ky/2)",
 "d_z": "m_e + (S-D)/2*cos(kx) + (S+D)/2*cos(ky)",
 "mass_X": "M_plus",
 "mass_Y": "-M_minus",
 },
 "down_block": {
 "d_x": "4*t2*cos(kx/2)*cos(ky/2)",
 "d_y": "-4*t1*sin(kx/2)*sin(ky/2)",
 "d_z": "-m_e - (S+D)/2*cos(kx) + (-S+D)/2*cos(ky)",
 "mass_X": "M_minus",
 "mass_Y": "-M_plus",
 },
 "simultaneous_zero_statement": (
 "For $t1*t2 \neq 0$, $d_x=d_y=0$ only at X and Y (up to periodic equivalents).",
),
 "analytic_chern_formula": {
 "C_up": "-sgn(t1*t2)/2 * [sgn(M_plus)+sgn(M_minus)]",
 "C_down": "+sgn(t1*t2)/2 * [sgn(M_plus)+sgn(M_minus)]",
 "C_total": "0",
 "typeII_condition": "M_plus*M_minus > 0",
 "trivial_condition": "M_plus*M_minus < 0",
 "chirality_control": "sgn(C_up)=-sgn(t1*t2)*sgn(M_plus) in the type-II sectors",
 },
 "restricted_model_reduction": {
 "restriction": "d1=d2=v-u and m_e=m",
 "mass_product": "M_plus*M_minus=4*(v-u)^2-m^2",
 "condition": "2*abs(u-v)>abs(m)",
 },
 "hamiltonian_reconstruction_audit": reconstruction,
 }

```

```

def latex_derivation() -> str:
 return r"""% Auto-generated by Step 09
\section*{Analytic spin-Chern formula for the MSG 123.342 Lieb model}
Define
\begin{equation}
S=s_1-s_2,\quad D=d_1+d_2,
\end{equation}
and

```

---

```

\begin{equation}
M_+=m_e+D,\quad M_-=-m_e+D.
\end{equation}
In the basis $\{|1,\uparrow\rangle,|2,\uparrow\rangle\}$, the spin-up block is
\begin{equation}
h_{\uparrow}(\mathbf{k})=d_{\uparrow}(\mathbf{k})\sigma_0+
\mathbf{d}_{\uparrow}(\mathbf{k})\cdot\boldsymbol{\sigma},
\end{equation}
with
\begin{align}
d_{x,\uparrow}&=4t_2\cos\frac{k_x}{2}\cos\frac{k_y}{2},\\
d_{y,\uparrow}&=-4t_1\sin\frac{k_x}{2}\sin\frac{k_y}{2},\\
d_{z,\uparrow}&=m_e+\frac{S-D}{2}\cos k_x+\frac{S+D}{2}\cos k_y.
\end{align}
For $t_{1,2}\neq 0$, the simultaneous zeros of d_x and d_y are only
 $X=(\pi,0)$ and $Y=(0,\pi)$, modulo reciprocal-lattice equivalence. Their
masses are
\begin{equation}
m_{\uparrow,X}=M_+,\quad m_{\uparrow,Y}=-M_-.
\end{equation}
The corresponding Jacobian chiralities are
\begin{equation}
\chi_{\uparrow,X}=\text{sgn}(t_{1,2}),\quad
\chi_{\uparrow,Y}=-\text{sgn}(t_{1,2}).
\end{equation}
Using the lower-band massive-Dirac contribution
 $C_i=-\chi_i\text{sgn}(m_i)/2$, one obtains
\begin{equation}
\boxed{C_{\uparrow}=-\frac{\text{sgn}(t_{1,2})}{2}
\left[\text{sgn}(M_+)+\text{sgn}(M_-)\right]}.}
\end{equation}
The spin-down block gives
\begin{equation}
\boxed{C_{\downarrow}=-C_{\uparrow},\quad C_{\text{total}}=0.}
\end{equation}
Therefore,
\begin{equation}
M_+M_->0\Longleftrightarrow |C_{\uparrow}|=|C_{\downarrow}|=1,
\end{equation}
whereas
\begin{equation}
M_+M_-<0\Longleftrightarrow C_{\uparrow}=C_{\downarrow}=0.
\end{equation}
Within a type-II QSH sector,
\begin{equation}
\text{sgn}(C_{\uparrow})=-\text{sgn}(t_{1,2})
\text{sgn}(M_+).
\end{equation}
For the restricted parametrization $d_1=d_2=v-u$ and $m_e=m$,
\begin{equation}
M_+M_- = 4(v-u)^2-m^2,
\end{equation}
so the nontrivial condition reduces to
\begin{equation}
2|u-v|>|m|.
\end{equation}
"""

```

```

def make_figures(df: pd.DataFrame, figures_dir: Path) -> None:
 # Three-sector M_- / M_+ map.
 fig, ax = plt.subplots(figsize=(7.2, 6.2))
 sectors = ["trivial_spin_chern", "typeII_QSH_Cup_plus", "typeII_QSH_Cup_minus"]
 markers = ["o", "^", "s"]
 for sector, marker in zip(sectors, markers):
 part = df[df["numeric_sector"] == sector]
 ax.scatter(
 part["M_minus_recomputed"], part["M_plus_recomputed"],
 s=18, alpha=0.65, marker=marker, label=f"{sector} (n={len(part)})"

```

```

)
 ax.axhline(0, linewidth=1)
 ax.axvline(0, linewidth=1)
 ax.set_xlabel(r"M_-")
 ax.set_ylabel(r"M_+")
 ax.set_title("Three spin-Chern sectors in the analytic mass plane")
 ax.legend(fontsize=8)
 save_figure(fig, figures_dir / "step09_01_three_sector_Mplus_Mminus_map")

Numeric vs analytic Cup.
valid = df[df["formula_boundary"] == 0]
matrix = pd.crosstab(valid["chern_up_int"].astype(int), valid["chern_up_analytic"].astype(int))
labels = [-1, 0, 1]
values = matrix.reindex(index=labels, columns=labels, fill_value=0).to_numpy()
fig, ax = plt.subplots(figsize=(5.6, 4.8))
image = ax.imshow(values, aspect="auto")
for i in range(3):
 for j in range(3):
 ax.text(j, i, str(values[i, j]), ha="center", va="center")
ax.set_xticks(range(3), labels)
ax.set_yticks(range(3), labels)
ax.set_xlabel(r"Analytic $C_{\uparrow}$ ")
ax.set_ylabel(r"Numerical $C_{\uparrow}$ ")
ax.set_title("Analytic versus numerical spin Chern number")
fig.colorbar(image, ax=ax, label="Count")
save_figure(fig, figures_dir / "step09_02_analytic_numeric_Cup_confusion")

Chirality mechanism table as scatter.
topo = valid[valid["chern_up_int"].astype(int) != 0].copy()
topo["mass_common_sign"] = np.sign(topo["M_plus_recomputed"]).astype(int)
fig, ax = plt.subplots(figsize=(7.0, 5.0))
jitter = np.linspace(-0.08, 0.08, max(len(topo), 2))[: len(topo)]
x = topo["sign_t1t2"].to_numpy(dtype=float) + jitter
y = topo["mass_common_sign"].to_numpy(dtype=float)
scatter = ax.scatter(x, y, c=topo["chern_up_int"], s=18, alpha=0.7)
ax.set_xticks([-1, 1])
ax.set_yticks([-1, 1])
ax.set_xlabel(r"$\operatorname{sgn}(t_1 t_2)$")
ax.set_ylabel(r"Common sign of M_+ and M_- ")
ax.set_title(r"Spin-Chern chirality: $\operatorname{sgn}(C_{\uparrow}) = -\operatorname{sgn}(t_1 t_2) \operatorname{sgn}(M_+)$ ")
fig.colorbar(scatter, ax=ax, label=r"Numerical $C_{\uparrow}$ ")
save_figure(fig, figures_dir / "step09_03_chirality_control")

Gap versus distance to analytic phase boundaries.
distance = np.minimum(np.abs(valid["M_plus_recomputed"]), np.abs(valid["M_minus_recomputed"]))
fig, ax = plt.subplots(figsize=(7.0, 5.0))
scatter = ax.scatter(distance, valid["direct_gap"], c=valid["chern_up_int"], s=18, alpha=0.65)
ax.set_xlabel(r"$\min(|M_+|, |M_-|)$")
ax.set_ylabel("Audited direct gap")
ax.set_title("Gap robustness relative to the analytic mass boundaries")
fig.colorbar(scatter, ax=ax, label=r"$C_{\uparrow}$")
save_figure(fig, figures_dir / "step09_04_gap_vs_mass_boundary_distance")

def research_significance_manifest(metrics: dict) -> dict:
 return {
 "validated_increment": [
 "Generalizes the known restricted type-II-QSH inversion condition to the full
 ↪ seven-independent-parameter form of the onsite+NN+NNN MSG 123.342 Lieb Hamiltonian.",
 "Separates topology existence (the signs of M_+ and M_-) from spin-Chern chirality (the sign of
 ↪ $t_1 t_2$).",
 "Provides a closed three-sector classification: trivial, $C_{\uparrow}=+1$, and $C_{\uparrow}=-1$.",
 "Closes the ML-to-physics loop: strict full-BZ/Chern labels -> interpretable feature discovery ->
 ↪ analytic Dirac proof -> edge-state verification.",
],
 "practical_meaning": [
 "Within the model domain, the topological sector can be screened algebraically without a new BZ
 ↪ Berry-curvature integration for every parameter point.",
]
 }

```

```

 "The phase-boundary equations identify which microscopic hopping or onsite combinations must be tuned
 ↪ to switch topology or reverse edge-state chirality.",
 "The formula gives a compact bridge from fitted MagneticTB/Wannier parameters to a topological design
 ↪ rule.",
],
 "scope_limits": [
 "The result is for the spin-conserving four-band model at half filling with onsite, nearest-neighbor,
 ↪ and next-nearest-neighbor terms.",
 "It is not a classifier for all topological insulators, non-spin-conserving SOC, or higher-order
 ↪ topology.",
 "Additional symmetry-allowed spin-mixing or longer-range terms require a new invariant analysis.",
],
 "current_validation": metrics,
}

```

```

def write_readme(output_dir: Path, metrics: dict) -> None:
 text = f"""# Step 09 — Analytic Chern Formula and Three-Sector Map

```

```

Input mode

```

```

This program reads an existing Step 08 output directory directly. It does not
read or extract ZIP archives.

```

```

Main formula

```

```

For a gapped parameter point with

```

```

```text
t1*t2*M_plus*M_minus != 0
```

```

```

and

```

```

```text
M_plus = m_e + d1 + d2
M_minus = -m_e + d1 + d2
```

```

```

the audited analytic formula is

```

```

```text
C_up = -sign(t1*t2)/2 * [sign(M_plus) + sign(M_minus)]
C_down = -C_up
C_total = 0
```

```

```

Hence:

```

```

```text
M_plus*M_minus > 0 -> type-II QSH
M_plus*M_minus < 0 -> trivial spin-Chern insulator
```

```

```

Inside the type-II phase:

```

```

```text
sign(C_up) = -sign(t1*t2)*sign(M_plus)
```

```

```

Validation result

```

```

- Total reliable Step 08 labels: {metrics['n_total_labeled']}
- Formula-valid nonboundary samples: {metrics['n_formula_valid']}
- Exact spin-resolved Chern matches: {metrics['n_exact_matches']}
- Mismatches: {metrics['n_mismatches']}
- Exact accuracy: {metrics['exact_chern_accuracy']:.6f}

```



---

## Run

Place this script beside the existing Step 08 output directory and run:

```
```bash
python 09_Lieb8_Analytic_Chern_Formula_and_Three_Sector_Map.py
```
```

Or specify the directory explicitly:

```
```bash
python 09_Lieb8_Analytic_Chern_Formula_and_Three_Sector_Map.py \
--step08-dir outputs_step08_analytic_boundary_wanniertools_edge_<run_tag>
```
```

## Important outputs

```
- `step09_01_spin_block_dvector_formulas.json`
- `step09_01_spin_block_dvector_formulas.tex`
- `step09_02_dirac_point_mass_chirality_catalog.csv`
- `step09_03_analytic_chern_prediction_master.csv`
- `step09_03_analytic_formula_metrics.json`
- `step09_04_formula_mismatch_samples.csv`
- `step09_05_three_sector_counts.csv`
- `step09_06_research_significance_manifest.json`
- `figures/`
"""
```

```
(output_dir / "README_Step09.md").write_text(text, encoding="utf-8")
```

```
def main() -> int:
 args = parse_args()
 paths = make_paths(args)
 print(f"[Step09] Step 08 directory: {paths.step08_dir}")
 print(f"[Step09] Output directory : {paths.output_dir}")

 base, target, loaded = load_step08_data(paths)
 combined = loaded["combined"]
 print(f"[Step09] Global reliable samples : {len(base)}")
 print(f"[Step09] Targeted reliable samples : {len(target)}")
 print(f"[Step09] Combined labels : {len(combined)}")

 reconstruction = reconstruction_audit(
 combined,
 n_samples=args.validation_samples,
 n_kpoints=args.validation_kpoints,
 seed=args.seed,
)
 write_json(paths.output_dir / "step09_01_hamiltonian_reconstruction_audit.json", reconstruction)

 payload = formula_payload(reconstruction)
 write_json(paths.output_dir / "step09_01_spin_block_dvector_formulas.json", payload)
 (paths.output_dir / "step09_01_spin_block_dvector_formulas.tex").write_text(
 latex_derivation(), encoding="utf-8"
)

 catalog = dirac_catalog()
 catalog.to_csv(paths.output_dir / "step09_02_dirac_point_mass_chirality_catalog.csv", index=False)

 master = add_analytic_chern(combined, tol=float(args.boundary_tol))
 master.to_csv(paths.output_dir / "step09_03_analytic_chern_prediction_master.csv", index=False)
 metrics = classification_metrics(master)
 write_json(paths.output_dir / "step09_03_analytic_formula_metrics.json", metrics)

 mismatches = master[
 (master["formula_boundary"] == 0) & (master["chern_formula_match"] == 0)
].copy()
 mismatches.to_csv(paths.output_dir / "step09_04_formula_mismatch_samples.csv", index=False)
```

---

```

counts = (
 master.groupby(["source_dataset", "numeric_sector"], dropna=False)
 .size().rename("count").reset_index()
)
counts.to_csv(paths.output_dir / "step09_05_three_sector_counts.csv", index=False)

sign_table = (
 master[master["formula_boundary"] == 0]
 .groupby(["sign_M_plus", "sign_M_minus", "sign_tlt2", "chern_up_int"])
 .size().rename("count").reset_index()
)
sign_table.to_csv(paths.output_dir / "step09_05_sign_sector_truth_table.csv", index=False)

make_figures(master, paths.figures_dir)

significance = research_significance_manifest(metrics)
write_json(paths.output_dir / "step09_06_research_significance_manifest.json", significance)

run_summary = {
 "step08_dir": str(paths.step08_dir),
 "input_mode": "existing_step08_directory_only",
 "zip_reading_enabled": False,
 "output_dir": str(paths.output_dir),
 "n_global": int(len(base)),
 "n_targeted": int(len(target)),
 "n_combined": int(len(master)),
 "formula_metrics": metrics,
 "hamiltonian_reconstruction": reconstruction,
 "next_recommended_step": (
 "Compare the full seven-parameter phase formula against the restricted "
 "literature subspace and then test controlled symmetry-allowed spin-mixing "
 "terms as a separate Step 10."
),
}
write_json(paths.output_dir / "step09_00_run_summary.json", run_summary)
write_readme(paths.output_dir, metrics)

registry_rows = []
for path in sorted(paths.output_dir.rglob("*")):
 if path.is_file():
 registry_rows.append(
 {
 "relative_path": str(path.relative_to(paths.output_dir)),
 "size_bytes": int(path.stat().st_size),
 }
)
pd.DataFrame(registry_rows).to_csv(
 paths.output_dir / "step09_00_output_file_registry.csv", index=False
)

print("[Step09] Formula:", metrics["formula"])
print(
 f"[Step09] Exact matches: {metrics['n_exact_matches']}/"
 f"{metrics['n_formula_valid']} mismatches={metrics['n_mismatches']}"
)
print(f"[Step09] Reconstruction max error: {reconstruction['max_abs_error']:.3e}")
return 0

if __name__ == "__main__":
 try:
 raise SystemExit(main())
 except Exception as exc:
 print(f"[Step09][ERROR] {exc}", file=sys.stderr)
 raise

```

## ► 代码清单 A.48.3 (python, 518 行)

```
from __future__ import annotations

"""
Paper-ready M+-M- phase diagram for the Lieb model.

Main purpose

Generate the signed-mass phase diagram in a publication-friendly style,
using the user's requested color scheme and Times New Roman font.

Topological masses

 M_plus = m_e + d1 + d2
 M_minus = -m_e + d1 + d2

For a fixed chirality chi = sgn(t1*t2),
 C_up = -chi/2 * [sgn(M_plus) + sgn(M_minus)]

This script is designed for the main-text figure, especially chi = +1.
It can still generate both chirality sectors if needed.

Input

The script reads an existing Lieb Step09 result directory or ZIP:
 outputs_step09_analytic_chern_formula_<run_tag>/
 step09_03_analytic_chern_prediction_master.csv
or the corresponding ZIP file.

If Step09 is unavailable, Step08 can be used as a fallback:
 outputs_step08_analytic_boundary_wanniertools_edge_<run_tag>/
 step08_01_analytic_feature_master.csv

Output

PNG (high dpi), PDF and SVG.
"""

import argparse
import zipfile
from pathlib import Path
from typing import Iterable

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from matplotlib import font_manager, rcParams
from matplotlib.patches import Patch
from matplotlib.text import Text

=====
1. Constants and requested colors
=====
STEP09_PREFIX = "outputs_step09_analytic_chern_formula_"
STEP08_PREFIX = "outputs_step08_analytic_boundary_wanniertools_edge_"
STEP09_CSV = "step09_03_analytic_chern_prediction_master.csv"
STEP08_CSV = "step08_01_analytic_feature_master.csv"
REQUIRED_COLUMNS = ("m_e", "d1", "d2", "t1", "t2", "chern_up_int")

User-requested region colors
COLOR_RED = (169 / 255.0, 36 / 255.0, 37 / 255.0) # C_up = +1
COLOR_BLUE = (63 / 255.0, 99 / 255.0, 173 / 255.0) # C_up = -1
COLOR_GREY = (211 / 255.0, 211 / 255.0, 211 / 255.0) # C_up = 0
```

---

```

def lighten(color: tuple[float, float, float], factor: float = 0.78) -> tuple[float, float, float]:
 """Blend color toward white for region backgrounds."""
 base = np.array(color, dtype=float)
 white = np.ones(3, dtype=float)
 mixed = base * (1 - factor) + white * factor
 return tuple(np.clip(mixed, 0.0, 1.0))

REGION_FILL = {
 +1: lighten(COLOR_RED, 0.70),
 0: lighten(COLOR_GREY, 0.15),
 -1: lighten(COLOR_BLUE, 0.68),
}

Points should be visually distinct from the regions.
Use white-filled markers with colored edges.
POINT_EDGE = {
 +1: COLOR_RED,
 0: (0.25, 0.25, 0.25),
 -1: COLOR_BLUE,
}

=====
2. Plot style
=====
def configure_plot_style() -> str:
 available_fonts = {item.name for item in font_manager.fontManager.ttflist}
 font_name = "Times New Roman" if "Times New Roman" in available_fonts else "DejaVu Serif"

 rcParams["font.family"] = font_name
 rcParams["font.weight"] = "bold"
 rcParams["font.size"] = 9
 rcParams["axes.labelsize"] = 9
 rcParams["axes.labelweight"] = "bold"
 rcParams["axes.titlesize"] = 9
 rcParams["axes.titleweight"] = "bold"
 rcParams["figure.titleweight"] = "bold"
 rcParams["xtick.labelsize"] = 8
 rcParams["ytick.labelsize"] = 8
 rcParams["legend.fontsize"] = 8
 rcParams["mathtext.fontset"] = "custom"
 rcParams["mathtext.rm"] = f"{font_name}:bold"
 rcParams["mathtext.it"] = f"{font_name}:italic:bold"
 rcParams["mathtext.bf"] = f"{font_name}:bold"
 rcParams["mathtext.sf"] = f"{font_name}:bold"
 rcParams["mathtext.tt"] = f"{font_name}:bold"
 rcParams["mathtext.fallback"] = None
 rcParams["axes.linewidth"] = 0.8
 rcParams["xtick.direction"] = "in"
 rcParams["ytick.direction"] = "in"
 rcParams["xtick.top"] = True
 rcParams["ytick.right"] = True
 rcParams["svg.fonttype"] = "none"
 rcParams["pdf.fonttype"] = 42
 rcParams["ps.fonttype"] = 42
 return font_name

def force_bold_times_text(fig: plt.Figure, font_name: str) -> None:
 """Apply the requested font family and weight to every text artist."""
 for artist in fig.findobj(match=Text):
 artist.set_fontfamily(font_name)
 artist.set_fontweight("bold")

=====
3. Data discovery and reading
=====

```

---

```

def zip_contains(archive_path: Path | str, basename: str) -> bool:
 archive_path = Path(archive_path)
 try:
 with zipfile.ZipFile(archive_path) as archive:
 return any(Path(name).name == basename for name in archive.namelist())
 except (OSError, zipfile.BadZipFile):
 return False

def candidate_roots(project_root: Path) -> list[Path]:
 roots = [
 project_root,
 project_root / "lieb",
 Path.cwd(),
 Path.cwd() / "lieb",
 Path(__file__).resolve().parent,
 Path(__file__).resolve().parent / "lieb",
]
 for path in list(roots):
 roots.append(path.parent)

 unique = []
 seen = set()
 for path in roots:
 resolved = path.expanduser().resolve()
 if resolved not in seen:
 seen.add(resolved)
 unique.append(resolved)
 return unique

def discover_lieb_input(project_root: Path | str = ".", allow_step08_fallback: bool = True) -> Path:
 project_root = Path(project_root).expanduser().resolve()
 roots = candidate_roots(project_root)

 for root in roots:
 if not root.is_dir():
 continue
 for directory in sorted(root.glob(f"{STEP09_PREFIX}*")):
 if directory.is_dir() and (directory / STEP09_CSV).is_file():
 return directory
 for archive in sorted(root.glob(f"{STEP09_PREFIX}*.zip")):
 if archive.is_file() and zip_contains(archive, STEP09_CSV):
 return archive

 if project_root.is_dir():
 step09_csv_matches = sorted(project_root.rglob(STEP09_CSV))
 if step09_csv_matches:
 return step09_csv_matches[0].parent
 step09_zip_matches = sorted(project_root.rglob(f"{STEP09_PREFIX}*.zip"))
 for archive in step09_zip_matches:
 if zip_contains(archive, STEP09_CSV):
 return archive

 if allow_step08_fallback:
 for root in roots:
 if not root.is_dir():
 continue
 for directory in sorted(root.glob(f"{STEP08_PREFIX}*")):
 if directory.is_dir() and (directory / STEP08_CSV).is_file():
 return directory
 for archive in sorted(root.glob(f"{STEP08_PREFIX}*.zip")):
 if archive.is_file() and zip_contains(archive, STEP08_CSV):
 return archive

 if project_root.is_dir():
 step08_csv_matches = sorted(project_root.rglob(STEP08_CSV))
 if step08_csv_matches:
 return step08_csv_matches[0].parent

```

---

```

 step08_zip_matches = sorted(project_root.rglob(f"{STEP08_PREFIX}*.zip"))
 for archive in step08_zip_matches:
 if zip_contains(archive, STEP08_CSV):
 return archive

 raise FileNotFoundError(
 "No valid Lieb Step09/Step08 output was found.\n"
 "Expected Step09 file: step09_03_analytic_bern_prediction_master.csv"
)

def _read_csv_from_zip(archive_path: Path, preferred_names: Iterable[str]) -> tuple[pd.DataFrame, str]:
 with zipfile.ZipFile(archive_path) as archive:
 names = archive.namelist()
 for basename in preferred_names:
 matches = [name for name in names if Path(name).name == basename]
 if matches:
 selected = sorted(matches, key=lambda name: (len(Path(name).parts), len(name)))[0]
 with archive.open(selected) as stream:
 return pd.read_csv(stream, low_memory=False), basename
 raise FileNotFoundError(f"Could not find any of {list(preferred_names)} in {archive_path}")

def read_lieb_output(input_source: Path | str) -> tuple[pd.DataFrame, dict]:
 source = Path(input_source).expanduser().resolve()
 preferred = (STEP09_CSV, STEP08_CSV)

 if source.is_file() and source.suffix.lower() == ".zip":
 frame, csv_name = _read_csv_from_zip(source, preferred)
 return frame, {"input_source": str(source), "input_kind": "zip", "csv_name": csv_name}

 if source.is_dir():
 for csv_name in preferred:
 direct = source / csv_name
 if direct.is_file():
 return pd.read_csv(direct, low_memory=False), {
 "input_source": str(source),
 "input_kind": "directory",
 "csv_name": csv_name,
 "csv_path": str(direct),
 }
 for csv_name in preferred:
 matches = sorted(source.rglob(csv_name))
 if matches:
 return pd.read_csv(matches[0], low_memory=False), {
 "input_source": str(source),
 "input_kind": "directory_recursive",
 "csv_name": csv_name,
 "csv_path": str(matches[0]),
 }

 raise FileNotFoundError(f"Cannot read a valid Lieb output from: {source}")

=====
4. Data preparation
=====
def prepare_lieb_data(df: pd.DataFrame) -> pd.DataFrame:
 missing = [column for column in REQUIRED_COLUMNS if column not in df.columns]
 if missing:
 raise KeyError(f"Missing required columns: {missing}\nAvailable columns: {list(df.columns)}")

 out = df.copy()
 for column in REQUIRED_COLUMNS:
 out[column] = pd.to_numeric(out[column], errors="coerce")
 out = out.dropna(subset=list(REQUIRED_COLUMNS)).copy()

 out["M_plus_plot"] = out["m_e"] + out["d1"] + out["d2"]
 out["M_minus_plot"] = -out["m_e"] + out["d1"] + out["d2"]

```

---

```

out["chi_plot"] = np.sign(out["t1"] * out["t2"]).astype(int)
out["chern_up_analytic_plot"] = (
 -0.5 * out["chi_plot"] * (np.sign(out["M_plus_plot"]) + np.sign(out["M_minus_plot"]))
)
out["chern_up_int"] = out["chern_up_int"].astype(int)
return out

def symmetric_limit(*series: Iterable[float], padding: float = 1.08) -> float:
 arrays = []
 for values in series:
 numeric = pd.to_numeric(pd.Series(values), errors="coerce")
 numeric = numeric[np.isfinite(numeric)]
 if len(numeric):
 arrays.append(np.abs(numeric.to_numpy(float)))
 if not arrays:
 return 1.0
 return padding * float(np.max(np.concatenate(arrays)))

=====
5. Plotting helpers
=====
def add_region_labels(ax: plt.Axes, limit: float, chirality: int) -> None:
 """Add compact publication-style annotations."""
 # For chi = +1:
 # top-right -> C=-1 (blue)
 # bottom-left -> C=+1 (red)
 # For chi = -1 signs are reversed.
 c_tr = -chirality
 c_bl = chirality

 text_box = dict(boxstyle="round,pad=0.20", facecolor="white", edgecolor="none", alpha=0.72)

 ax.text(0.58 * limit, 0.58 * limit, rf"$C_{\uparrow}={c_tr:+d}$", ha="center", va="center", bbox=text_box)
 ax.text(-0.58 * limit, -0.58 * limit, rf"$C_{\uparrow}={c_bl:+d}$", ha="center", va="center", bbox=text_box)
 ax.text(-0.58 * limit, 0.58 * limit, rf"$C_{\uparrow}=0$", ha="center", va="center", bbox=text_box)
 ax.text(0.58 * limit, -0.58 * limit, rf"$C_{\uparrow}=0$", ha="center", va="center", bbox=text_box)

=====
6. Main plotting function
=====
def plot_mass_plane(
 data: pd.DataFrame,
 chirality: int,
 output_dir: Path | str,
 png_dpi: int = 2400,
 show_title: bool = False,
 font_name: str = "Times New Roman",
) -> dict[str, Path]:
 if chirality not in (-1, 1):
 raise ValueError("chirality must be +1 or -1")

 subset = data[data["chi_plot"] == chirality].copy()
 if subset.empty:
 raise ValueError(f"No samples for chi={chirality:+d}")

 limit = symmetric_limit(subset["M_plus_plot"], subset["M_minus_plot"])
 fig, ax = plt.subplots(figsize=(8.0 / 2.54, 7.0 / 2.54))

 # Background regions using the requested paper colors.
 # Determine which Chern sector occupies each quadrant for this chirality.
 # Use filled rectangles via ax.axvspan/ax.axhspan combos by contour-like pcolormesh.
 axis = np.linspace(-limit, limit, 401)
 m_plus_grid, m_minus_grid = np.meshgrid(axis, axis)
 chern_grid = -0.5 * chirality * (np.sign(m_plus_grid) + np.sign(m_minus_grid))

 # Manual facecolor map ensures exact requested hues.

```

---

```

region_rgb = np.empty(chern_grid.shape + (3,), dtype=float)
for value, color in REGION_FILL.items():
 mask = np.isclose(chern_grid, value)
 region_rgb[mask] = color

ax.imshow(
 region_rgb,
 extent=[-limit, limit, -limit, limit],
 origin="lower",
 aspect="equal",
 interpolation="nearest",
 zorder=0,
)

Boundary lines
ax.axvline(0.0, linestyle="--", linewidth=1.0, color="black", zorder=2)
ax.axhline(0.0, linestyle="--", linewidth=1.0, color="black", zorder=2)

Points: use white fill + colored edges for strong contrast with the colored regions.
for c_val in (-1, 0, 1):
 group = subset[subset["chern_up_int"] == c_val]
 if len(group) == 0:
 continue
 ax.scatter(
 group["M_plus_plot"],
 group["M_minus_plot"],
 s=18,
 facecolors="white",
 edgecolors=[POINT_EDGE[c_val]],
 linewidths=0.8,
 alpha=1.0,
 zorder=3,
)

add_region_labels(ax, limit, chirality)

ax.set_xlim(-limit, limit)
ax.set_ylim(-limit, limit)
ax.set_aspect("equal", adjustable="box")
ax.set_xlabel(r"$M_{+}=m_e+d_1+d_2$")
ax.set_ylabel(r"$M_{-}=-m_e+d_1+d_2$")

Main-text friendly expression: no long title; only a compact chi tag.
if show_title:
 ax.set_title(rf"Signed-mass phase diagram ($\chi=\{chirality:d\}$)")
else:
 ax.text(
 0.03,
 0.97,
 rf"$\chi=\{chirality:d\}$",
 transform=ax.transAxes,
 ha="left",
 va="top",
 bbox=dict(boxstyle="round,pad=0.20", facecolor="white", edgecolor="none", alpha=0.82),
)

Compact legend with clear separation between region meaning and numerical points.
patches = [
 Patch(facecolor=REGION_FILL[+1], edgecolor="none", label=r"Region: $C_{\uparrow}=+1$ "),
 Patch(facecolor=REGION_FILL[0], edgecolor="none", label=r"Region: $C_{\uparrow}=0$ "),
 Patch(facecolor=REGION_FILL[-1], edgecolor="none", label=r"Region: $C_{\uparrow}=-1$ "),
]
leg1 = ax.legend(handles=patches, loc="lower left", frameon=True, framealpha=0.88, borderpad=0.35)
ax.add_artist(leg1)

Dummy handles for point style legend.
point_handles = []
point_labels = []

```



---

```

for c_val, label in [(+1, r"Numerical: $C_\uparrow=+1$"), (0, r"Numerical: $C_\uparrow=0$"), (-1, r"Numerical:
↪ $C_\uparrow=-1$")]:
 point_handles.append(
 plt.Line2D(
 [0], [0],
 marker='o',
 linestyle='None',
 markerfacecolor='white',
 markeredgecolor=POINT_EDGE[c_val],
 markeredgewidth=0.9,
 markersize=5,
)
)
 point_labels.append(label)
ax.legend(point_handles, point_labels, loc="upper right", frameon=True, framealpha=0.88, borderpad=0.35)

force_bold_times_text(fig, font_name)
fig.tight_layout()

output_dir = Path(output_dir).expanduser().resolve()
output_dir.mkdir(parents=True, exist_ok=True)
stem = output_dir / f"lieb_mass_plane_chi_{chirality:+d}"

paths = {
 "png": stem.with_suffix(".png"),
 "pdf": stem.with_suffix(".pdf"),
 "svg": stem.with_suffix(".svg"),
}
fig.savefig(paths["png"], dpi=png_dpi, bbox_inches="tight")
fig.savefig(paths["pdf"], bbox_inches="tight")
fig.savefig(paths["svg"], bbox_inches="tight")
plt.close(fig)
return paths

=====
7. Workflow
=====
def run_paper_mass_plane(input_source: Path | str | None = None, project_root: Path | str = ".", output_dir: Path
↪ | str = "lieb_mass_plane_paper", png_dpi: int = 2400, generate_chi_minus: bool = True) -> dict:
 font_name = configure_plot_style()

 if input_source is None:
 input_source = discover_lieb_input(project_root)
 input_source = Path(input_source).expanduser().resolve()

 raw, metadata = read_lieb_output(input_source)
 data = prepare_lieb_data(raw)

 output_dir = Path(output_dir).expanduser().resolve()
 output_dir.mkdir(parents=True, exist_ok=True)

 data.to_csv(output_dir / "lieb_signed_mass_plane_points.csv", index=False)

 # Main-text figure: chi = +1
 figures = {
 +1: plot_mass_plane(
 data=data,
 chirality=+1,
 output_dir=output_dir,
 png_dpi=png_dpi,
 show_title=False,
 font_name=font_name,
)
 }

 if generate_chi_minus and (data["chi_plot"] == -1).any():
 figures[-1] = plot_mass_plane(

```

```

 data=data,
 chirality=-1,
 output_dir=output_dir,
 png_dpi=png_dpi,
 show_title=False,
 font_name=font_name,
)

 return {
 "metadata": metadata,
 "input_source": input_source,
 "data": data,
 "figures": figures,
 "output_dir": output_dir,
 }

=====
8. CLI
=====
def parse_args() -> argparse.Namespace:
 parser = argparse.ArgumentParser(description="Generate a paper-ready Lieb M+-M- phase diagram.")
 parser.add_argument("--input-source", type=Path, default=None, help="Step09/Step08 output directory or ZIP. If
 ↪ omitted, search automatically.")
 parser.add_argument("--project-root", type=Path, default=Path('.'), help="Root for automatic search when input
 ↪ is omitted.")
 parser.add_argument("--output-dir", type=Path, default=Path("lieb_mass_plane_paper"))
 parser.add_argument("--png-dpi", type=int, default=2400)
 parser.add_argument("--no-chi-minus", action="store_true", help="Generate only chi=+1, recommended for the
 ↪ main-text figure.")
 return parser.parse_args()

if __name__ == "__main__":
 args = parse_args()
 result = run_paper_mass_plane(
 input_source=args.input_source,
 project_root=args.project_root,
 output_dir=args.output_dir,
 png_dpi=args.png_dpi,
 generate_chi_minus=not args.no_chi_minus,
)
 print("Input source:", result["input_source"])
 print("Detected CSV:", result["metadata"].get("csv_name"))
 print("Processed samples:", len(result["data"]))
 print("Output directory:", result["output_dir"])

```

## TTS 体系全局拓扑标注与分层机器学习代码

### ► 代码清单 A.49.1 (python, 1066 行)

```

#!/usr/bin/env python3
-*- coding: utf-8 -*-
"""
TTS Step 01 v2 —八带交错磁 Hamiltonian 与严格拓扑标签器调试

```

本脚本参考用户提供的 01\_Lieb8\_AMTI\_pipeline\_debug.ipynb 的组织方式，  
 将同一套“模型定义 → 单元测试 → 能带 → 全 BZ 带隙 → 周期规范 →  
 spin-Chern → 小批量标签”流程改写为 tts 网络版本。

## 核心模型

```

- tts tessellation: 3.3.4.3.4
- magnetic space group: P4'/mbm' (BNS 127.391)
- magnetic Wyckoff position: 4h
- representative coordinate: (x, x+1/2, 1/2), x = 0.183
- raw parameters: (e1,e2,t1,t2,r1,r2,r3,r4)
- reduced parameters after removing e0: (m_e,t1,t2,r1,r2,r3,r4)
- eight bands, half filling: four occupied bands
```

## 重要实现说明

```

1. 全部计算均为串行, 不使用 ProcessPoolExecutor, 因此可直接在 Windows/Jupyter 运行。
2. Chern 数使用周期规范 Hamiltonian; 不能直接在含分数坐标相位的 atomic gauge 上跨 BZ 边界。
3. 只有在完整 BZ 中最低四带始终为 $2\uparrow+2\downarrow$, 且直接带隙为正时, 才计算物理 spin-Chern 标签。
4. "spin_chern_TI_candidate" 仅表示模型级候选; 边缘态、DOS 与 SHC 留给后续步骤验证。
"""
```

```
from __future__ import annotations

from dataclasses import asdict, dataclass
from datetime import datetime, timezone
from pathlib import Path
from typing import Callable, Dict, Iterable, Sequence
import argparse
import json
import math
import os
import time

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

np.set_printoptions(precision=10, suppress=True)

=====
Step 0. Constants and configuration
=====

CODE_VERSION = "TTS_STEP01_V2_20260713"
RAW8 = ["e1", "e2", "t1", "t2", "r1", "r2", "r3", "r4"]
REDUCED7 = ["m_e", "t1", "t2", "r1", "r2", "r3", "r4"]

N_OCC_TOTAL = 4
N_OCC_SPIN = 2

X_TTS = 0.183
ALPHA = 2.0 * X_TTS # 0.366 = 183/500
BETA = 0.5 - 2.0 * X_TTS # 0.134 = 67/500

SPIN_UP_INDICES = [0, 2, 4, 6]
SPIN_DOWN_INDICES = [1, 3, 5, 7]

MagneticTB basis positions in the cached tts.nb Hamiltonian.
ORBITAL_POSITIONS = np.array(
 [
 [X_TTS, X_TTS + 0.5], [X_TTS, X_TTS + 0.5],
 [X_TTS + 0.5, -X_TTS], [X_TTS + 0.5, -X_TTS],
 [0.5 - X_TTS, X_TTS], [0.5 - X_TTS, X_TTS],
 [-X_TTS, 0.5 - X_TTS], [-X_TTS, 0.5 - X_TTS],
],
 dtype=float,
)

PAPER_PARAMS: Dict[str, float] = {
 "e1": 0.5,
 "e2": -0.4,
 "t1": 0.2,
```

```

 "t2": 0.1,
 "r1": 0.3,
 "r2": -0.1,
 "r3": 0.1,
 "r4": -0.4,
}

Step 01 小批量中发现的正全局带隙 spin-Chern 候选。
TOPO_PARAMS: Dict[str, float] = {
 "e1": 0.003886630742,
 "e2": -0.003886630742,
 "t1": 0.330302133158,
 "t2": 0.691254202294,
 "r1": 0.793309071205,
 "r2": -0.491195093495,
 "r3": -0.542657657699,
 "r4": -0.578737306339,
}

DEBUG_BOUNDS = {
 "m_e": (-1.0, 1.0),
 "t1": (-0.8, 0.8),
 "t2": (-0.8, 0.8),
 "r1": (-0.8, 0.8),
 "r2": (-0.8, 0.8),
 "r3": (-0.8, 0.8),
 "r4": (-0.8, 0.8),
}

@dataclass
class Step01Config:
 output_dir: Path = Path("outputs_tts_step01_v2")
 run_small_batch: bool = True
 n_batch: int = 24
 batch_seed: int = 20260713
 debug_gap_nk: int = 41
 debug_chern_nk: int = 21
 paper_gap_nk: int = 81
 paper_chern_grids: tuple[int, ...] = (21, 31, 41)
 strict_gap_grids: tuple[int, ...] = (51, 71, 101)
 strict_chern_grids: tuple[int, ...] = (21, 31, 41)
 gap_shifts: tuple[tuple[float, float], ...] = (
 (0.0, 0.0), (0.5, 0.0), (0.0, 0.5), (0.5, 0.5)
)
 chern_shifts: tuple[tuple[float, float], ...] = ((0.0, 0.0), (0.5, 0.5))
 gap_tol: float = 1.0e-3
 chern_tol: float = 0.08
 min_det_tol: float = 1.0e-7

 def normalized(self) -> "Step01Config":
 self.output_dir = Path(self.output_dir)
 self.output_dir.mkdir(parents=True, exist_ok=True)
 if self.n_batch < 0:
 raise ValueError("n_batch must be >= 0")
 for name in ("debug_gap_nk", "debug_chern_nk", "paper_gap_nk"):
 if int(getattr(self, name)) < 5:
 raise ValueError(f"{name} must be >= 5")
 return self

=====
Step 1. Parameters and tts Hamiltonian
=====

def validate_params(params: Dict[str, float]) -> Dict[str, float]:
 missing = [name for name in RAW8 if name not in params]
 if missing:
 raise ValueError(f"Missing parameters: {missing}")

```

---

```

clean = {name: float(params[name]) for name in RAW8}
if not all(np.isfinite(v) for v in clean.values()):
 raise ValueError("Parameter vector contains NaN or infinity")
return clean

def raw8_from_reduced7(reduced: Dict[str, float], e0: float = 0.0) -> Dict[str, float]:
 missing = [name for name in REDUCED7 if name not in reduced]
 if missing:
 raise ValueError(f"Missing reduced parameters: {missing}")
 m_e = float(reduced["m_e"])
 return {
 "e1": float(e0) + m_e,
 "e2": float(e0) - m_e,
 "t1": float(reduced["t1"]),
 "t2": float(reduced["t2"]),
 "r1": float(reduced["r1"]),
 "r2": float(reduced["r2"]),
 "r3": float(reduced["r3"]),
 "r4": float(reduced["r4"]),
 }

def reduced7_from_raw8(params: Dict[str, float]) -> Dict[str, float]:
 p = validate_params(params)
 return {
 "m_e": 0.5 * (p["e1"] - p["e2"]),
 "t1": p["t1"], "t2": p["t2"],
 "r1": p["r1"], "r2": p["r2"],
 "r3": p["r3"], "r4": p["r4"],
 }

def derived_features(params: Dict[str, float]) -> Dict[str, float]:
 p = validate_params(params)
 root2 = math.sqrt(2.0)
 return {
 "e0": 0.5 * (p["e1"] + p["e2"]),
 "m_e": 0.5 * (p["e1"] - p["e2"]),
 "t_sum": p["t1"] + p["t2"],
 "t_diff": p["t1"] - p["t2"],
 "t_product": p["t1"] * p["t2"],
 "t_s": (p["t1"] + p["t2"]) / root2,
 "t_d": (p["t1"] - p["t2"]) / root2,
 "r13_sum": p["r1"] + p["r3"],
 "r13_diff": p["r1"] - p["r3"],
 "r24_sum": p["r2"] + p["r4"],
 "r24_diff": p["r2"] - p["r4"],
 "r_all_sum": p["r1"] + p["r2"] + p["r3"] + p["r4"],
 }

def h_tts_atomic(kx: float, ky: float, params: Dict[str, float]) -> np.ndarray:
 """Supplemental Material Eqs. S24-S25 in the atomic Bloch gauge."""
 p = validate_params(params)
 e1, e2, t1, t2, r1, r2, r3, r4 = [p[name] for name in RAW8]

 r = 1j * r1 + r3
 rp = 1j * r2 + r4

 xi = np.exp(1j * (0.5 * kx + BETA * ky))
 xip = np.exp(1j * (-0.5 * kx + BETA * ky))
 eta = np.exp(1j * (BETA * kx + 0.5 * ky))
 etap = np.exp(1j * (-BETA * kx + 0.5 * ky))
 phi = np.exp(1j * (ALPHA * kx + ALPHA * ky))
 phip = np.exp(1j * (-ALPHA * kx + ALPHA * ky))

 h = np.zeros((8, 8), dtype=np.complex128)
 np.fill_diagonal(h, [e1, e2, e2, e1, e2, e1, e1, e2])

```

---

```

h[0, 2] = xi * r + xip * np.conj(rp)
h[0, 4] = eta * np.conj(r) + np.conj(eta) * rp
h[0, 6] = np.conj(phi) * t1

h[1, 3] = xip * np.conj(r) + xi * rp
h[1, 5] = np.conj(eta) * r + eta * np.conj(rp)
h[1, 7] = np.conj(phi) * t2

h[2, 4] = phip * t2
h[2, 6] = eta * r + np.conj(eta) * np.conj(rp)

h[3, 5] = phip * t1
h[3, 7] = np.conj(eta) * np.conj(r) + eta * rp

h[4, 6] = xi * np.conj(r) + xip * rp
h[5, 7] = xip * r + xi * np.conj(rp)

h = h + np.triu(h, 1).conj().T
return h

def periodic_gauge_matrix(kx: float, ky: float) -> np.ndarray:
 phases = np.exp(
 1j * (
 ORBITAL_POSITIONS[:, 0] * float(kx)
 + ORBITAL_POSITIONS[:, 1] * float(ky)
)
)
 return np.diag(phases).astype(np.complex128)

def h_tts_periodic(kx: float, ky: float, params: Dict[str, float]) -> np.ndarray:
 u = periodic_gauge_matrix(kx, ky)
 h = h_tts_atomic(kx, ky, params)
 return u @ h @ u.conj().T

def spin_indices(spin: str) -> list[int]:
 key = spin.lower()
 if key == "up":
 return SPIN_UP_INDICES.copy()
 if key == "down":
 return SPIN_DOWN_INDICES.copy()
 raise ValueError("spin must be 'up' or 'down'")

def h_spin_block_atomic(kx: float, ky: float, params: Dict[str, float], spin: str) -> np.ndarray:
 idx = spin_indices(spin)
 h = h_tts_atomic(kx, ky, params)
 return h[np.ix_(idx, idx)]

def h_spin_block_periodic(kx: float, ky: float, params: Dict[str, float], spin: str) -> np.ndarray:
 idx = spin_indices(spin)
 h = h_tts_periodic(kx, ky, params)
 return h[np.ix_(idx, idx)]

def eigvals_full(kx: float, ky: float, params: Dict[str, float]) -> np.ndarray:
 return np.linalg.eigvalsh(h_tts_atomic(kx, ky, params))

=====
Step 2. Hard audits
=====

def spin_spectrum_difference(kx: float, ky: float, params: Dict[str, float]) -> float:
 eu = np.linalg.eigvalsh(h_spin_block_atomic(kx, ky, params, "up"))

```

---

```

ed = np.linalg.eigvalsh(h_spin_block_atomic(kx, ky, params, "down"))
return float(np.max(np.abs(np.sort(eu) - np.sort(ed))))

```

```

def model_tests(params: Dict[str, float], n_random: int = 40, tol: float = 1.0e-10) -> Dict[str, float]:
 rng = np.random.default_rng(127391)
 herm = spin_mixing = per_x = per_y = 0.0

```

```

 for _ in range(n_random):
 kx, ky = rng.uniform(-np.pi, np.pi, size=2)
 h = h_tts_atomic(float(kx), float(ky), params)
 herm = max(herm, float(np.max(np.abs(h - h.conj().T))))
 spin_mixing = max(
 spin_mixing,
 float(np.max(np.abs(h[np.ix_(SPIN_UP_INDICES, SPIN_DOWN_INDICES)]))),
 float(np.max(np.abs(h[np.ix_(SPIN_DOWN_INDICES, SPIN_UP_INDICES)]))),
)
 hp = h_tts_periodic(float(kx), float(ky), params)
 per_x = max(
 per_x,
 float(np.max(np.abs(h_tts_periodic(kx + 2*np.pi, ky, params) - hp))),
)
 per_y = max(
 per_y,
 float(np.max(np.abs(h_tts_periodic(kx, ky + 2*np.pi, params) - hp))),
)

```

```

def pair_error(kx: float, ky: float) -> float:
 e = np.sort(eigvals_full(kx, ky, params))
 return float(max(abs(e[2*i + 1] - e[2*i]) for i in range(4)))

```

```

qs = np.linspace(-np.pi, np.pi, 101)
results = {
 "hermiticity_error": herm,
 "spin_mixing_error": spin_mixing,
 "periodicity_error_x": per_x,
 "periodicity_error_y": per_y,
 "gamma_pair_degeneracy_error": pair_error(0.0, 0.0),
 "M_pair_degeneracy_error": pair_error(np.pi, np.pi),
 "Mprime_pair_degeneracy_error": pair_error(-np.pi, np.pi),
 "delta_ky0_spin_degeneracy_error": max(
 spin_spectrum_difference(float(q), 0.0, params) for q in qs
),
 "delta_prime_kx0_spin_degeneracy_error": max(
 spin_spectrum_difference(0.0, float(q), params) for q in qs
),
 "Z_kxpi_spin_degeneracy_error": max(
 spin_spectrum_difference(np.pi, float(q), params) for q in qs
),
 "Z_prime_kypi_spin_degeneracy_error": max(
 spin_spectrum_difference(float(q), np.pi, params) for q in qs
),
 "sigma_kx_eq_ky_max_spin_splitting": max(
 spin_spectrum_difference(float(q), float(q), params) for q in qs
),
 "sigma_prime_minus_kx_eq_ky_max_spin_splitting": max(
 spin_spectrum_difference(float(-q), float(q), params) for q in qs
),
}

```

```

hard_keys = [
 "hermiticity_error", "spin_mixing_error",
 "periodicity_error_x", "periodicity_error_y",
 "gamma_pair_degeneracy_error", "M_pair_degeneracy_error",
 "Mprime_pair_degeneracy_error",
 "delta_ky0_spin_degeneracy_error",
 "delta_prime_kx0_spin_degeneracy_error",
 "Z_kxpi_spin_degeneracy_error",
 "Z_prime_kypi_spin_degeneracy_error",

```

```

]
failed = {key: results[key] for key in hard_keys if results[key] > tol}
if failed:
 raise AssertionError(f"tts Hamiltonian hard audit failed: {failed}")
if results["sigma_kx_eq_ky_max_spin_splitting"] < 1.0e-6:
 raise AssertionError("Expected Sigma spin splitting was not found")
if results["sigma_prime_minus_kx_eq_ky_max_spin_splitting"] < 1.0e-6:
 raise AssertionError("Expected Sigma' spin splitting was not found")
return results

=====
Step 3. High-symmetry path and bands
=====

HIGH_SYM_POINTS = [
 ("Γ", np.array([0.0, 0.0])),
 ("X", np.array([np.pi, 0.0])),
 ("M", np.array([np.pi, np.pi])),
 ("Γ", np.array([0.0, 0.0])),
 ("M'", np.array([-np.pi, np.pi])),
 ("X'", np.array([0.0, np.pi])),
 ("Γ", np.array([0.0, 0.0])),
]

def make_k_path(
 points: Sequence[tuple[str, np.ndarray]] = HIGH_SYM_POINTS,
 n_per_segment: int = 100,
):
 k_list: list[np.ndarray] = []
 x_list: list[float] = []
 tick_positions = [0.0]
 tick_labels = [points[0][0]]
 distance = 0.0

 for seg in range(len(points) - 1):
 _, ka = points[seg]
 label_b, kb = points[seg + 1]
 ts = np.linspace(0.0, 1.0, n_per_segment, endpoint=False)
 for t in ts:
 k = (1.0 - t) * ka + t * kb
 if k_list:
 distance += float(np.linalg.norm(k - k_list[-1]))
 k_list.append(k)
 x_list.append(distance)
 distance += float(np.linalg.norm(kb - k_list[-1]))
 k_list.append(kb.copy())
 x_list.append(distance)
 tick_positions.append(distance)
 tick_labels.append(label_b)

 return np.asarray(k_list), np.asarray(x_list), tick_positions, tick_labels

def path_band_data(params: Dict[str, float], n_per_segment: int = 100):
 k_list, x_axis, ticks, labels = make_k_path(n_per_segment=n_per_segment)
 up = np.array([
 np.linalg.eigvalsh(h_spin_block_atomic(float(k[0]), float(k[1]), params, "up"))
 for k in k_list
])
 down = np.array([
 np.linalg.eigvalsh(h_spin_block_atomic(float(k[0]), float(k[1]), params, "down"))
 for k in k_list
])
 return k_list, x_axis, ticks, labels, up, down

def plot_path_bands(

```



---

```

params: Dict[str, float],
save_path: Path,
title: str,
n_per_segment: int = 100,
) -> pd.DataFrame:
 k_list, x_axis, ticks, labels, up, down = path_band_data(params, n_per_segment)

 fig, ax = plt.subplots(figsize=(9.0, 5.8))
 for ib in range(up.shape[1]):
 ax.plot(x_axis, up[:, ib], linewidth=1.25, label="spin up" if ib == 0 else None)
 ax.plot(
 x_axis, down[:, ib], linewidth=1.10, linestyle="--",
 label="spin down" if ib == 0 else None,
)
 for tick in ticks:
 ax.axvline(tick, linewidth=0.6, alpha=0.45)
 ax.axhline(0.0, linewidth=0.7, alpha=0.5)
 ax.set_xlim(x_axis[0], x_axis[-1])
 ax.set_xticks(ticks)
 ax.set_xticklabels(labels)
 ax.set_ylabel("Energy")
 ax.set_title(title)
 ax.legend(frameon=False)
 fig.tight_layout()
 fig.savefig(save_path, dpi=240, bbox_inches="tight")
 plt.close(fig)

 data = {
 "path_x": x_axis,
 "kx": k_list[:, 0],
 "ky": k_list[:, 1],
 }
 for ib in range(4):
 data[f"up_{ib+1}"] = up[:, ib]
 data[f"down_{ib+1}"] = down[:, ib]
 return pd.DataFrame(data)

=====
Step 4. Full-BZ gap and spin-filling audit
=====

def bz_grid(nk: int, shift: tuple[float, float] = (0.0, 0.0)) -> tuple[np.ndarray, np.ndarray]:
 # Periodic grid in [-pi, pi], shifted by fractions of one mesh spacing.
 kx = -np.pi + 2.0 * np.pi * (np.arange(nk) + float(shift[0])) / nk
 ky = -np.pi + 2.0 * np.pi * (np.arange(nk) + float(shift[1])) / nk
 return kx, ky

def scan_band_gaps(
 params: Dict[str, float],
 nk: int = 81,
 shift: tuple[float, float] = (0.0, 0.0),
 gap_tol: float = 1.0e-3,
) -> Dict[str, float | int]:
 ks_x, ks_y = bz_grid(int(nk), shift)

 min_direct = np.inf
 max_valence = -np.inf
 min_conduction = np.inf
 min_balanced = np.inf
 min_spin_up = np.inf
 min_spin_down = np.inf
 mismatch_count = 0
 min_n_up = N_OCC_TOTAL
 max_n_up = 0

 direct_k = (np.nan, np.nan)
 vbm_k = (np.nan, np.nan)

```

---

```

cbm_k = (np.nan, np.nan)
balanced_k = (np.nan, np.nan)

for kx in ks_x:
 for ky in ks_y:
 kx_f, ky_f = float(kx), float(ky)
 e = eigvals_full(kx_f, ky_f, params)
 direct = float(e[N_OCC_TOTAL] - e[N_OCC_TOTAL - 1])
 if direct < min_direct:
 min_direct = direct
 direct_k = (kx_f, ky_f)
 if float(e[N_OCC_TOTAL - 1]) > max_valence:
 max_valence = float(e[N_OCC_TOTAL - 1])
 vbm_k = (kx_f, ky_f)
 if float(e[N_OCC_TOTAL]) < min_conduction:
 min_conduction = float(e[N_OCC_TOTAL])
 cbm_k = (kx_f, ky_f)

 eu = np.linalg.eigvalsh(h_spin_block_atomic(kx_f, ky_f, params, "up"))
 ed = np.linalg.eigvalsh(h_spin_block_atomic(kx_f, ky_f, params, "down"))
 min_spin_up = min(min_spin_up, float(eu[2] - eu[1]))
 min_spin_down = min(min_spin_down, float(ed[2] - ed[1]))

 balanced = float(min(eu[2], ed[2]) - max(eu[1], ed[1]))
 if balanced < min_balanced:
 min_balanced = balanced
 balanced_k = (kx_f, ky_f)

 tagged = [(float(x), 1) for x in eu] + [(float(x), 0) for x in ed]
 tagged.sort(key=lambda item: item[0])
 n_up = int(sum(tag for _, tag in tagged[:N_OCC_TOTAL]))
 min_n_up = min(min_n_up, n_up)
 max_n_up = max(max_n_up, n_up)
 mismatch_count += int(n_up != N_OCC_SPIN)

indirect = float(min_conduction - max_valence)
return {
 "grid_nk": int(nk),
 "shift_x": float(shift[0]),
 "shift_y": float(shift[1]),
 "min_direct_gap": float(min_direct),
 "indirect_gap": indirect,
 "direct_gap_kx": direct_k[0], "direct_gap_ky": direct_k[1],
 "vbm": float(max_valence), "vbm_kx": vbm_k[0], "vbm_ky": vbm_k[1],
 "cbm": float(min_conduction), "cbm_kx": cbm_k[0], "cbm_ky": cbm_k[1],
 "min_spin_gap_up": float(min_spin_up),
 "min_spin_gap_down": float(min_spin_down),
 "min_balanced_sector_gap": float(min_balanced),
 "balanced_gap_kx": balanced_k[0], "balanced_gap_ky": balanced_k[1],
 "spin_occupancy_mismatch_count": int(mismatch_count),
 "min_n_up_in_lowest4": int(min_n_up),
 "max_n_up_in_lowest4": int(max_n_up),
 "is_direct_gapped": int(min_direct > gap_tol),
 "is_physical_insulator": int(indirect > gap_tol),
 "is_balanced_spin_sector": int(min_balanced > gap_tol and mismatch_count == 0),
}

```

```

=====
Step 5. Non-Abelian Fukui spin-Chern
=====

```

```

def _normalize_link(value: complex, eps: float = 1.0e-14) -> complex:
 amp = abs(value)
 if amp < eps:
 raise FloatingPointError(f"Link determinant too small: {amp:.3e}")
 return value / amp

```

---

```

def fukui_chern_subspace(
 h_func: Callable[[float, float], np.ndarray],
 n_occ: int,
 nk: int,
 shift: tuple[float, float] = (0.0, 0.0),
) -> Dict[str, float | int]:
 dim = h_func(0.0, 0.0).shape[0]
 ks_x = 2.0 * np.pi * (np.arange(nk) + float(shift[0])) / nk
 ks_y = 2.0 * np.pi * (np.arange(nk) + float(shift[1])) / nk
 occ = np.empty((nk, nk, dim, n_occ), dtype=np.complex128)

 for ix, kx in enumerate(ks_x):
 for iy, ky in enumerate(ks_y):
 _, vec = np.linalg.eigh(h_func(float(kx), float(ky)))
 occ[ix, iy] = vec[:, :n_occ]

 total_phase = 0.0
 min_det = 1.0
 for ix in range(nk):
 for iy in range(nk):
 v = occ[ix, iy]
 vx = occ[(ix + 1) % nk, iy]
 vy = occ[ix, (iy + 1) % nk]
 vxy = occ[(ix + 1) % nk, (iy + 1) % nk]

 lx = np.linalg.det(v.conj().T @ vx)
 ly = np.linalg.det(v.conj().T @ vy)
 lx_y = np.linalg.det(vy.conj().T @ vxy)
 ly_x = np.linalg.det(vx.conj().T @ vxy)
 min_det = min(min_det, abs(lx), abs(ly), abs(lx_y), abs(ly_x))

 plaquette = (
 _normalize_link(lx) * _normalize_link(ly_x)
 / (_normalize_link(lx_y) * _normalize_link(ly))
)
 total_phase += float(np.angle(plaquette))

 return {
 "chern": total_phase / (2.0 * np.pi),
 "min_det_amp": float(min_det),
 "nk": int(nk),
 }

def calculate_spin_cherns(
 params: Dict[str, float],
 nk: int = 31,
 shift: tuple[float, float] = (0.0, 0.0),
) -> Dict[str, float | int]:
 up = fukui_chern_subspace(
 lambda kx, ky: h_spin_block_periodic(kx, ky, params, "up"),
 N_OCC_SPIN, int(nk), shift,
)
 down = fukui_chern_subspace(
 lambda kx, ky: h_spin_block_periodic(kx, ky, params, "down"),
 N_OCC_SPIN, int(nk), shift,
)
 total = fukui_chern_subspace(
 lambda kx, ky: h_tts_periodic(kx, ky, params),
 N_OCC_TOTAL, int(nk), shift,
)

 c_up = float(up["chern"])
 c_down = float(down["chern"])
 c_total = float(total["chern"])
 return {
 "chern_nk": int(nk),
 "shift_x": float(shift[0]),
 "shift_y": float(shift[1]),
 }

```

---

```

 "chern_up": c_up,
 "chern_down": c_down,
 "chern_total": c_total,
 "spin_chern": 0.5 * (c_up - c_down),
 "sum_rule_error": abs(c_total - c_up - c_down),
 "min_det_up": float(up["min_det_amp"]),
 "min_det_down": float(down["min_det_amp"]),
 "min_det_total": float(total["min_det_amp"]),
 }

def rounded_integer_if_close(value: float, tolerance: float = 0.08) -> int | None:
 nearest = int(np rint(value))
 return nearest if abs(value - nearest) <= tolerance else None

def chern_convergence_table(
 params: Dict[str, float],
 grids: Iterable[int] = (21, 31, 41),
 shifts: Iterable[tuple[float, float]] = ((0.0, 0.0), (0.5, 0.5)),
) -> pd.DataFrame:
 rows: list[dict] = []
 for nk in grids:
 for shift in shifts:
 try:
 rows.append(calculate_spin_cherns(params, int(nk), shift))
 except Exception as exc:
 rows.append({
 "chern_nk": int(nk),
 "shift_x": float(shift[0]),
 "shift_y": float(shift[1]),
 "error": repr(exc),
 })
 return pd.DataFrame(rows)

=====
Step 6. Unified phase label
=====

def calculate_sample(
 sample_id: str,
 params: Dict[str, float],
 gap_nk: int = 41,
 chern_nk: int = 21,
 gap_tol: float = 1.0e-3,
 chern_tol: float = 0.08,
 min_det_tol: float = 1.0e-7,
) -> Dict[str, object]:
 p = validate_params(params)
 row: Dict[str, object] = {
 "sample_id": sample_id,
 **p,
 **derived_features(p),
 "phase_label": "numeric_error",
 "error": "",
 "chern_reliable": 0,
 "is_spin_chern_topological": 0,
 "is_spin_chern_TI_candidate": 0,
 "is_typeII_QSH_confirmed": 0,
 }

 try:
 row.update(scan_band_gaps(p, nk=gap_nk, gap_tol=gap_tol))
 except Exception as exc:
 row["error"] = f"gap_scan_failed: {exc!r}"
 return row

 if float(row["min_direct_gap"]) <= gap_tol:

```

---

```

 row["phase_label"] = "noninsulating_or_gap_closing"
 return row
 if int(row["is_balanced_spin_sector"]) != 1:
 row["phase_label"] = "spin_sector_filling_mismatch"
 return row

 try:
 row.update(calculate_spin_cherns(p, nk=chern_nk))
 except Exception as exc:
 row["phase_label"] = "chern_unreliable"
 row["error"] = f"chern_failed: {exc!r}"
 return row

 cu = rounded_integer_if_close(float(row["chern_up"]), chern_tol)
 cd = rounded_integer_if_close(float(row["chern_down"]), chern_tol)
 ct = rounded_integer_if_close(float(row["chern_total"]), chern_tol)
 row.update({"chern_up_int": cu, "chern_down_int": cd, "chern_total_int": ct})

 reliable = (
 cu is not None and cd is not None and ct is not None
 and float(row["min_det_up"]) > min_det_tol
 and float(row["min_det_down"]) > min_det_tol
 and float(row["min_det_total"]) > min_det_tol
 and float(row["sum_rule_error"]) <= 0.15
)
 row["chern_reliable"] = int(reliable)
 if not reliable:
 row["phase_label"] = "chern_unreliable"
 return row

 topological = bool(cu == -cd and abs(cu) >= 1 and ct == 0)
 insulator = bool(float(row["indirect_gap"]) > gap_tol)
 row["is_spin_chern_topological"] = int(topological)

 if topological and insulator:
 row["phase_label"] = "spin_chern_TI_candidate"
 row["is_spin_chern_TI_candidate"] = 1
 elif topological:
 row["phase_label"] = "spin_chern_band_metal"
 elif insulator and cu == 0 and cd == 0:
 row["phase_label"] = "trivial_insulator"
 elif cu == 0 and cd == 0:
 row["phase_label"] = "indirect_overlap"
 else:
 row["phase_label"] = "other_spin_chern_sector"
 return row

=====
Step 7. Strict multi-grid verification
=====

def strict_verify_sample(
 sample_id: str,
 params: Dict[str, float],
 config: Step01Config,
) -> Dict[str, object]:
 config = config.normalized()
 gap_rows: list[dict] = []
 chern_rows: list[dict] = []

 for nk in config.strict_gap_grids:
 for shift in config.gap_shifts:
 row = scan_band_gaps(
 params,
 nk=int(nk),
 shift=shift,
 gap_tol=config.gap_tol,
)

```

---

```

 row["sample_id"] = sample_id
 gap_rows.append(row)

for nk in config.strict_chern_grids:
 for shift in config.chern_shifts:
 row = calculate_spin_cherns(params, int(nk), shift)
 row["sample_id"] = sample_id
 chern_rows.append(row)

gap_df = pd.DataFrame(gap_rows)
chern_df = pd.DataFrame(chern_rows)
gap_df.to_csv(config.output_dir / f"{sample_id}_strict_gap_checks.csv", index=False)
chern_df.to_csv(config.output_dir / f"{sample_id}_strict_chern_checks.csv", index=False)

cu = [rounded_integer_if_close(v, config.chern_tol) for v in chern_df["chern_up"]]
cd = [rounded_integer_if_close(v, config.chern_tol) for v in chern_df["chern_down"]]
ct = [rounded_integer_if_close(v, config.chern_tol) for v in chern_df["chern_total"]]
tuples = list(zip(cu, cd, ct))

gaps_ok = bool(
 (gap_df["min_direct_gap"] > config.gap_tol).all()
 and (gap_df["indirect_gap"] > config.gap_tol).all()
 and (gap_df["spin_occupancy_mismatch_count"] == 0).all()
 and (gap_df["is_balanced_spin_sector"] == 1).all()
)
chern_ok = bool(
 all(x is not None for triple in tuples for x in triple)
 and len(set(tuples)) == 1
 and tuples[0][0] == -tuples[0][1]
 and abs(int(tuples[0][0])) >= 1
 and tuples[0][2] == 0
 and float(chern_df[["min_det_up", "min_det_down", "min_det_total"]].min().min())
 > config.min_det_tol
)

summary = {
 "sample_id": sample_id,
 **validate_params(params),
 **derived_features(params),
 "strict_min_direct_gap": float(gap_df["min_direct_gap"].min()),
 "strict_min_indirect_gap": float(gap_df["indirect_gap"].min()),
 "strict_max_spin_occupancy_mismatch_count": int(
 gap_df["spin_occupancy_mismatch_count"].max()
),
 "strict_min_link_determinant": float(
 chern_df[["min_det_up", "min_det_down", "min_det_total"]].min().min()
),
 "strict_chern_tuple": str(tuples[0]) if tuples else "None",
 "strict_gap_consensus": int(gaps_ok),
 "strict_chern_consensus": int(chern_ok),
 "strict_phase_label": (
 "spin_chern_TI_candidate" if gaps_ok and chern_ok else "boundary_or_unreliable"
),
}
pd.DataFrame([summary]).to_csv(
 config.output_dir / f"{sample_id}_strict_summary.csv", index=False
)
return summary

```

```

=====
Step 8. Small serial debug batch
=====

```

```

def sample_debug_parameters(n_samples: int, seed: int) -> pd.DataFrame:
 rng = np.random.default_rng(seed)
 rows = []
 for i in range(int(n_samples)):
 reduced = {

```

```

 name: float(rng.uniform(*DEBUG_BOUNDS[name]))
 for name in REDUCED7
}
 rows.append({"sample_id": f"debug_{i:04d}", **reduced})
return pd.DataFrame(rows)

```

```

def run_small_batch(config: Step01Config) -> pd.DataFrame:
 config = config.normalized()
 parameter_df = sample_debug_parameters(config.n_batch, config.batch_seed)
 parameter_df.to_csv(config.output_dir / "tts_debug_parameters.csv", index=False)

 rows: list[dict] = []
 total = len(parameter_df)
 for i, item in parameter_df.iterrows():
 reduced = {name: float(item[name]) for name in REDUCED7}
 params = raw8_from_reduced7(reduced)
 rows.append(
 calculate_sample(
 str(item["sample_id"]),
 params,
 gap_nk=config.debug_gap_nk,
 chern_nk=config.debug_chern_nk,
 gap_tol=config.gap_tol,
 chern_tol=config.chern_tol,
 min_det_tol=config.min_det_tol,
)
)
 print(f" serial debug sample {i + 1:>3d}/{total}")

 df = pd.DataFrame(rows)
 df.to_csv(config.output_dir / "tts_debug_batch_results.csv", index=False)
 counts = (
 df["phase_label"].value_counts(dropna=False)
 .rename_axis("phase_label").reset_index(name="count")
)
 counts.to_csv(config.output_dir / "tts_debug_phase_counts.csv", index=False)

 fig, ax = plt.subplots(figsize=(8.2, 4.8))
 ax.bar(counts["phase_label"].astype(str), counts["count"])
 ax.set_ylabel("Count")
 ax.set_title("TTS Step 01 serial debug phase counts")
 ax.tick_params(axis="x", rotation=30)
 fig.tight_layout()
 fig.savefig(config.output_dir / "tts_debug_phase_counts.png", dpi=220)
 plt.close(fig)
 return df

```

```

=====
Step 9. Complete Step 01 workflow
=====

```

```

def run_step01(config: Step01Config | None = None) -> Dict[str, object]:
 config = (config or Step01Config()).normalized()
 started = time.time()

 metadata = {
 "code_version": CODE_VERSION,
 "created_utc": datetime.now(timezone.utc).isoformat(),
 "model": "tts eight-band MagneticTB minimal Hamiltonian",
 "msg_bns": "127.391",
 "wyckoff": "4h",
 "x_tts": X_TTS,
 "raw_parameters": RAW8,
 "reduced_parameters": REDUCED7,
 "serial_execution_only": True,
 "config": {
 **asdict(config),

```

---

```

 "output_dir": str(config.output_dir),
 },
}
(config.output_dir / "tts_step01_metadata.json").write_text(
 json.dumps(metadata, indent=2, ensure_ascii=False), encoding="utf-8"
)

print("[1/6] Hamiltonian hard audit")
tests = model_tests(PAPER_PARAMS)
pd.DataFrame([tests]).to_csv(config.output_dir / "tts_model_tests.csv", index=False)
print(pd.DataFrame([tests]).to_string(index=False))

print("\n[2/6] Literature parameter bands and label")
paper_bands = plot_path_bands(
 PAPER_PARAMS,
 config.output_dir / "tts_paper_parameter_bands.png",
 "tts eight-band model: literature parameter set",
)
paper_bands.to_csv(config.output_dir / "tts_paper_path_bands.csv", index=False)
paper_gap = scan_band_gaps(
 PAPER_PARAMS, nk=config.paper_gap_nk, gap_tol=config.gap_tol
)
pd.DataFrame([paper_gap]).to_csv(config.output_dir / "tts_paper_gap_audit.csv", index=False)
paper_chern = chern_convergence_table(
 PAPER_PARAMS, config.paper_chern_grids, config.chern_shifts
)
paper_chern.to_csv(config.output_dir / "tts_paper_chern_convergence.csv", index=False)
paper_label = calculate_sample(
 "paper_tts",
 PAPER_PARAMS,
 gap_nk=config.paper_gap_nk,
 chern_nk=max(config.paper_chern_grids),
 gap_tol=config.gap_tol,
 chern_tol=config.chern_tol,
 min_det_tol=config.min_det_tol,
)
pd.DataFrame([paper_label]).to_csv(
 config.output_dir / "tts_paper_sample_label.csv", index=False
)
print(
 " paper:", paper_label["phase_label"],
 "Edir=", f"{float(paper_label['min_direct_gap']):.8f}",
 "Eind=", f"{float(paper_label['indirect_gap']):.8f}",
)

print("\n[3/6] Known topological anchor bands and basic label")
topo_bands = plot_path_bands(
 TOPO_PARAMS,
 config.output_dir / "tts_topological_anchor_bands.png",
 "tts eight-band model: Step 01 topological anchor",
)
topo_bands.to_csv(config.output_dir / "tts_topological_anchor_path_bands.csv", index=False)
topo_label = calculate_sample(
 "tts_topological_anchor",
 TOPO_PARAMS,
 gap_nk=config.paper_gap_nk,
 chern_nk=max(config.paper_chern_grids),
 gap_tol=config.gap_tol,
 chern_tol=config.chern_tol,
 min_det_tol=config.min_det_tol,
)
pd.DataFrame([topo_label]).to_csv(
 config.output_dir / "tts_topological_anchor_label.csv", index=False
)
print(
 " anchor:", topo_label["phase_label"],
 "Edir=", f"{float(topo_label['min_direct_gap']):.8f}",
 "Eind=", f"{float(topo_label['indirect_gap']):.8f}",
 "C=", (topo_label.get("chern_up_int"), topo_label.get("chern_down_int")),
)

```



```

)

print("\n[4/6] Strict multi-grid/multi-shift anchor verification")
strict_topo = strict_verify_sample("tts_topological_anchor", TOPO_PARAMS, config)
print(pd.DataFrame([strict_topo]).to_string(index=False))

print("\n[5/6] Small serial debug batch")
if config.run_small_batch and config.n_batch > 0:
 batch = run_small_batch(config)
 print(batch["phase_label"].value_counts(dropna=False).to_string())
else:
 batch = pd.DataFrame()
 print(" skipped")

print("\n[6/6] Summary")
summary = {
 "code_version": CODE_VERSION,
 "paper_phase_label": paper_label["phase_label"],
 "paper_min_direct_gap": float(paper_label["min_direct_gap"]),
 "paper_indirect_gap": float(paper_label["indirect_gap"]),
 "paper_chern_up": float(paper_label.get("chern_up", np.nan)),
 "paper_chern_down": float(paper_label.get("chern_down", np.nan)),
 "topological_anchor_phase_label": topo_label["phase_label"],
 "topological_anchor_strict_phase_label": strict_topo["strict_phase_label"],
 "topological_anchor_strict_min_direct_gap": strict_topo["strict_min_direct_gap"],
 "topological_anchor_strict_min_indirect_gap": strict_topo["strict_min_indirect_gap"],
 "topological_anchor_strict_chern_tuple": strict_topo["strict_chern_tuple"],
 "small_batch_size": int(len(batch)),
 "elapsed_seconds": float(time.time() - started),
 "next_step": "TTS Step 02: seven-dimensional Sobol global scan",
}
(config.output_dir / "tts_step01_summary.json").write_text(
 json.dumps(summary, indent=2, ensure_ascii=False), encoding="utf-8"
)
print(json.dumps(summary, indent=2, ensure_ascii=False))
print("\nStep 01 finished:", config.output_dir.resolve())
return {
 "summary": summary,
 "model_tests": pd.DataFrame([tests]),
 "paper_label": pd.DataFrame([paper_label]),
 "topological_anchor_label": pd.DataFrame([topo_label]),
 "strict_topological_anchor": pd.DataFrame([strict_topo]),
 "debug_batch": batch,
}

def build_arg_parser() -> argparse.ArgumentParser:
 parser = argparse.ArgumentParser(description=__doc__)
 parser.add_argument("--output-dir", default="outputs_tts_step01_v2")
 parser.add_argument("--n-batch", type=int, default=24)
 parser.add_argument("--seed", type=int, default=20260713)
 parser.add_argument("--skip-batch", action="store_true")
 parser.add_argument(
 "--quick", action="store_true",
 help="Use reduced meshes and four debug samples for an environment test.",
)
 return parser

def config_from_args(args: argparse.Namespace) -> Step01Config:
 if args.quick:
 return Step01Config(
 output_dir=Path(args.output_dir),
 run_small_batch=not args.skip_batch,
 n_batch=min(max(args.n_batch, 0), 4),
 batch_seed=args.seed,
 debug_gap_nk=21,
 debug_chern_nk=15,
 paper_gap_nk=31,

```

```

 paper_chern_grids=(15, 21),
 strict_gap_grids=(31, 41),
 strict_chern_grids=(15, 21),
)
 return Step01Config(
 output_dir=Path(args.output_dir),
 run_small_batch=not args.skip_batch,
 n_batch=args.n_batch,
 batch_seed=args.seed,
)

def main() -> int:
 parser = build_arg_parser()
 args = parser.parse_args()
 config = config_from_args(args)
 run_step01(config)
 return 0

if __name__ == "__main__":
 raise SystemExit(main())

```

## ► 代码清单 A.49.2 (python, 1489 行)

```

#!/usr/bin/env python3
-*- coding: utf-8 -*-
"""
TTS Step 02 v3 — 稳健物理标签、Hamiltonian 指纹与可解释机器学习

本脚本对应用户提供的 Lieb 工作流中的:
 02v3_Lieb8_Robust_Hamiltonian_fingerprint_and_ML.ipynb

但模型、标签与特征已经改写为 tts 八带交错磁体系:
- tts tessellation: 3.3.4.3.4
- MSG: P4'/mbm' (BNS 127.391)
- 8 bands, half filling = 4 occupied bands
- each conserved-spin block is 4x4 with 2 occupied bands
- frozen Step 01 core: TTS_step01_model_and_label_audit_v2.py

本阶段的研究主线:
 四来源参数采样
 -> 完整 BZ 稳健物理标签
 -> 去冗余 Hamiltonian fingerprint
 -> 采样来源捷径诊断
 -> 重复交叉验证与 global_random 外部测试
 -> 分组 permutation importance

与此前报错版本不同:
1. 全部物理批量计算严格串行;
2. sklearn/joblib 默认 n_jobs=1;
3. 不使用 ProcessPoolExecutor, 因此可直接在 Windows Jupyter 中运行;
4. 先保存 gap, 再决定是否计算 Chern, 近闭隙不会被误记为 numeric_error;
5. fingerprint 使用 4x4 自旋块的谱不变量和占据子空间投影量,
 不把 tts 错误压缩成 Lieb 的 2x2 Pauli 模型。

注意:
- spin_chern_TI_candidate 仍是模型级候选; 边缘态、DOS、SHC 在后续步骤验证。
- 本阶段用于发现候选控制变量, 不用于直接宣称最终解析机制。
"""

from __future__ import annotations

```

---

```

from dataclasses import asdict, dataclass
from pathlib import Path
from typing import Dict, Iterable, Sequence
import argparse
import hashlib
import json
import math
import time
import warnings

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

try:
 from joblib import dump
 from sklearn.ensemble import RandomForestClassifier
 from sklearn.inspection import permutation_importance
 from sklearn.linear_model import LogisticRegression
 from sklearn.metrics import (
 accuracy_score,
 balanced_accuracy_score,
 classification_report,
 confusion_matrix,
 ConfusionMatrixDisplay,
 f1_score,
 precision_score,
 recall_score,
)
 from sklearn.model_selection import (
 RepeatedStratifiedKFold,
 cross_validate,
 train_test_split,
)
 from sklearn.pipeline import Pipeline
 from sklearn.preprocessing import StandardScaler
except ImportError as exc: # pragma: no cover
 raise ImportError(
 "Step 02 需要 scikit-learn 与 joblib. 请运行: \n"
 " conda install scikit-learn joblib\n"
 " 或: \n"
 " pip install scikit-learn joblib"
) from exc

try:
 import TTS_step01_model_and_label_audit_v2 as core
except ImportError as exc: # pragma: no cover
 raise ImportError(
 " 未找到 TTS_step01_model_and_label_audit_v2.py. \n"
 " 请把 Step 01 v2 的 py 文件与本脚本放在同一目录。"
) from exc

EXPECTED_CORE_VERSION = "TTS_STEP01_V2_20260713"
if getattr(core, "CODE_VERSION", None) != EXPECTED_CORE_VERSION:
 raise RuntimeError(
 "Step 01 核心版本不一致: \n"
 f" expected = {EXPECTED_CORE_VERSION}\n"
 f" loaded = {getattr(core, 'CODE_VERSION', None)}\n"
 " 请使用配套代码包中的 Step 01 v2 文件。"
)

np.set_printoptions(precision=10, suppress=True)

CODE_VERSION = "TTS_STEP02V3_20260713"
WORKFLOW_STEP = "step02v3"
SYSTEM_TAG = "tts8_bns127391"
TASK_TAG = "robust_hamiltonian_fingerprint_ml"
REDUCED7 = core.REDUCED7.copy()
RAW8 = core.RAW8.copy()

```

---

```

=====
Configuration
=====

@dataclass
class Step02Config:
 output_dir: Path = Path(
 "outputs_tts_step02v3_robust_hamiltonian_fingerprint_ml"
)

 # Four source regions, following the uploaded Lieb Step 02 v3 workflow.
 n_topo_local: int = 40
 n_trivial_local: int = 40
 n_bridge: int = 40
 n_global: int = 40
 include_exact_anchors: bool = True
 random_seed: int = 20260713

 # Physics labels.
 gap_nk: int = 60
 chern_grids: tuple[int, ...] = (21, 31)
 chern_shifts: tuple[tuple[float, float], ...] = (
 (0.0, 0.0),
 (0.5, 0.5),
)
 direct_gap_skip_chern: float = 5.0e-3
 gap_tol: float = 1.0e-3
 chern_integer_tol: float = 0.08
 min_link_tol: float = 1.0e-7
 sum_rule_tol: float = 0.15
 min_chern_consensus: int = 2

 # Fingerprint.
 path_n: int = 31

 # ML evaluation.
 cv_splits: int = 5
 cv_repeats: int = 5
 validation_fraction: float = 0.25
 random_forest_estimators: int = 400
 permutation_repeats: int = 15
 ml_n_jobs: int = 1

 # Workflow controls.
 checkpoint_every: int = 10
 force_recalculate_physics: bool = False
 force_recalculate_fingerprint: bool = False

 def normalized(self) -> "Step02Config":
 self.output_dir = Path(self.output_dir)
 self.output_dir.mkdir(parents=True, exist_ok=True)
 (self.output_dir / "figures").mkdir(parents=True, exist_ok=True)
 (self.output_dir / "models").mkdir(parents=True, exist_ok=True)

 for name in ("n_topo_local", "n_trivial_local", "n_bridge", "n_global"):
 if int(getattr(self, name)) < 0:
 raise ValueError(f"{name} must be >= 0")
 if self.gap_nk < 8:
 raise ValueError("gap_nk must be >= 8")
 if self.path_n < 5:
 raise ValueError("path_n must be >= 5")
 if self.checkpoint_every < 1:
 raise ValueError("checkpoint_every must be >= 1")
 if self.ml_n_jobs == 0:
 raise ValueError("ml_n_jobs cannot be 0")
 return self

```

---

```

@property
def n_requested(self) -> int:
 return (
 int(self.n_topo_local)
 + int(self.n_trivial_local)
 + int(self.n_bridge)
 + int(self.n_global)
 + (2 if self.include_exact_anchors else 0)
)

Reduced seven-dimensional centers. The global energy shift e0 is fixed to zero.
TOPO_CENTER = core.reduced7_from_raw8(core.TOP0_PARAMS)
TRIVIAL_CENTER = core.reduced7_from_raw8(core.PAPER_PARAMS)

Local windows are deliberately moderate. Step 02 is a robustness/fingerprint stage,
not the final global existence scan.
TOP0_HALF_WIDTH = {
 "m_e": 0.08,
 "t1": 0.08,
 "t2": 0.12,
 "r1": 0.10,
 "r2": 0.10,
 "r3": 0.12,
 "r4": 0.12,
}
TRIVIAL_HALF_WIDTH = {
 "m_e": 0.20,
 "t1": 0.18,
 "t2": 0.18,
 "r1": 0.20,
 "r2": 0.20,
 "r3": 0.20,
 "r4": 0.20,
}
GLOBAL_RANGES = {name: (-1.0, 1.0) for name in REDUCED7}

=====
Output registry and safe file helpers
=====

def _config_tag(config: Step02Config) -> str:
 return (
 f"n{config.n_requested}_gap{config.gap_nk}"
 f"_chern{'-'.join(map(str, config.chern_grids))}"
 f"_seed{config.random_seed}"
)

def build_output_registry(config: Step02Config) -> Dict[str, Path]:
 config = config.normalized()
 run_tag = f"{SYSTEM_TAG}_{_config_tag(config)}"
 figures = config.output_dir / "figures"
 models = config.output_dir / "models"

 def out(substep: int, content: str, ext: str, *, figure: bool = False) -> Path:
 base = figures if figure else config.output_dir
 return base / (
 f"{WORKFLOW_STEP}_{substep:02d}_{content}__{run_tag}.{ext.lstrip('.')}"
)

 return {
 "run_config": out(0, "run_configuration", "json"),
 "registry": out(0, "output_file_registry", "csv"),
 "candidates": out(3, "candidate_parameters_four_source_regions", "csv"),
 "physics_checkpoint": out(4, "physics_labels_checkpoint", "csv"),
 "physics_final": out(4, "physics_labels_final_robust", "csv"),
 "chern_attempts": out(4, "chern_adaptive_attempt_records", "csv"),
 }

```

```

"phase_counts": out(4, "phase_counts", "csv"),
"phase_counts_fig": out(4, "phase_counts", "png", figure=True),
"fingerprint": out(5, "hamiltonian_fingerprint_independent_channels", "csv"),
"feature_metadata": out(5, "fingerprint_feature_metadata", "csv"),
"binary_dataset": out(6, "strict_binary_dataset_ti_vs_trivial", "csv"),
"source_label_diagnostic": out(6, "source_label_shortcut_diagnostic", "csv"),
"cv_metrics": out(8, "repeated_cross_validation_metrics", "csv"),
"holdout_metrics": out(8, "holdout_metrics", "csv"),
"external_global_metrics": out(8, "external_global_test_metrics", "csv"),
"classification_report": out(9, "best_model_classification_report", "txt"),
"validation_predictions": out(9, "best_model_validation_predictions", "csv"),
"confusion_matrix": out(9, "best_model_confusion_matrix", "png", figure=True),
"individual_importance": out(10, "individual_permutation_importance", "csv"),
"group_importance_family": out(10, "group_importance_by_family", "csv"),
"group_importance_kregion": out(10, "group_importance_by_kregion", "csv"),
"group_importance_spin": out(10, "group_importance_by_spin", "csv"),
"group_importance_family_fig": out(10, "group_importance_by_family", "png", figure=True),
"class_feature_means": out(11, "top_feature_class_means", "csv"),
"validation_subset": out(12, "magnetictb_validation_subset", "csv"),
"summary": out(13, "run_summary", "json"),
"model": models / f"{WORKFLOW_STEP}_13_best_random_forest_{run_tag}.joblib",
}

```

```

def atomic_write_csv(df: pd.DataFrame, path: Path) -> None:
 path = Path(path)
 tmp = path.with_suffix(path.suffix + ".tmp")
 df.to_csv(tmp, index=False, encoding="utf-8-sig")
 tmp.replace(path)

```

```

def file_sha256(path: Path) -> str:
 h = hashlib.sha256()
 with Path(path).open("rb") as handle:
 for chunk in iter(lambda: handle.read(1024 * 1024), b""):
 h.update(chunk)
 return h.hexdigest()

```

```

def write_run_metadata(config: Step02Config, outputs: Dict[str, Path]) -> None:
 core_path = Path(core.__file__).resolve()
 payload = {
 "code_version": CODE_VERSION,
 "workflow_step": WORKFLOW_STEP,
 "system_tag": SYSTEM_TAG,
 "task_tag": TASK_TAG,
 "step01_core_version": core.CODE_VERSION,
 "step01_core_file": str(core_path),
 "step01_core_sha256": file_sha256(core_path),
 "config": {
 **asdict(config),
 "output_dir": str(config.output_dir),
 "chern_shifts": [list(x) for x in config.chern_shifts],
 },
 "topological_center": TOPO_CENTER,
 "trivial_center": TRIVIAL_CENTER,
 "interpretation_warning": (
 "spin_chern_TI_candidate is a model-level candidate; "
 "edge states, DOS and SHC are not yet verified."
),
 }
 outputs["run_config"].write_text(
 json.dumps(payload, indent=2, ensure_ascii=False),
 encoding="utf-8",
)

```

```

registry = pd.DataFrame(
 [
 {

```

---

```

 "output_key": key,
 "filename": path.name,
 "relative_path": str(path),
 }
 for key, path in outputs.items()
]
)
atomic_write_csv(registry, outputs["registry"])

=====
Step 3. Four-source parameter sampling
=====

def _sample_local(
 center: Dict[str, float],
 widths: Dict[str, float],
 n: int,
 source: str,
 prefix: str,
 rng: np.random.Generator,
) -> list[dict]:
 rows: list[dict] = []
 for i in range(int(n)):
 row = {"sample_id": f"{prefix}_{i:06d}", "source_region": source}
 for key in REDUCED7:
 row[key] = float(
 rng.uniform(center[key] - widths[key], center[key] + widths[key])
)
 rows.append(row)
 return rows

def _sample_bridge(n: int, rng: np.random.Generator) -> list[dict]:
 rows: list[dict] = []
 for i in range(int(n)):
 alpha = float(rng.uniform(0.0, 1.0))
 row = {
 "sample_id": f"bridge_{i:06d}",
 "source_region": "bridge",
 "bridge_alpha": alpha,
 }
 for key in REDUCED7:
 center = (1.0 - alpha) * TOPO_CENTER[key] + alpha * TRIVIAL_CENTER[key]
 if key == "m_e":
 sigma = 0.06
 elif key.startswith("t"):
 sigma = 0.05
 else:
 sigma = 0.08
 row[key] = float(center + rng.normal(0.0, sigma))
 rows.append(row)
 return rows

def _sample_global(n: int, rng: np.random.Generator) -> list[dict]:
 rows: list[dict] = []
 for i in range(int(n)):
 row = {
 "sample_id": f"global_{i:06d}",
 "source_region": "global_random",
 }
 for key in REDUCED7:
 lo, hi = GLOBAL_RANGES[key]
 row[key] = float(rng.uniform(lo, hi))
 rows.append(row)
 return rows

```

---

```

def generate_candidates(config: Step02Config, outputs: Dict[str, Path]) -> pd.DataFrame:
 config = config.normalized()
 rng = np.random.default_rng(config.random_seed)
 rows: list[dict] = []

 if config.include_exact_anchors:
 rows.append({
 "sample_id": "anchor_topological_exact",
 "source_region": "exact_anchor",
 **TOPO_CENTER,
 })
 rows.append({
 "sample_id": "anchor_trivial_paper_exact",
 "source_region": "exact_anchor",
 **TRIVIAL_CENTER,
 })

 rows += _sample_local(
 TOPO_CENTER,
 TOPO_HALF_WIDTH,
 config.n_topo_local,
 "topo_local",
 "topolocal",
 rng,
)
 rows += _sample_local(
 TRIVIAL_CENTER,
 TRIVIAL_HALF_WIDTH,
 config.n_trivial_local,
 "trivial_local",
 "trivlocal",
 rng,
)
 rows += _sample_bridge(config.n_bridge, rng)
 rows += _sample_global(config.n_global, rng)

 candidates = pd.DataFrame(rows)
 for key in REDUCED7:
 candidates[key] = pd.to_numeric(candidates[key], errors="raise")
 candidates = candidates.drop_duplicates("sample_id", keep="last").reset_index(drop=True)
 atomic_write_csv(candidates, outputs["candidates"])
 return candidates

=====
Step 4. Robust physical labels
=====

def _integer_if_close(value: float, tol: float) -> int | None:
 if not np.isfinite(value):
 return None
 nearest = int(np rint(value))
 return nearest if abs(float(value) - nearest) <= tol else None

def one_chern_attempt(
 params: Dict[str, float],
 nk: int,
 shift: tuple[float, float],
 config: Step02Config,
) -> dict:
 row: dict = {
 "chern_nk": int(nk),
 "shift_x": float(shift[0]),
 "shift_y": float(shift[1]),
 "attempt_reliable": 0,
 "attempt_error": "",
 }
 try:

```



---

```

 row.update(core.calculate_spin_cherns(params, int(nk), shift))
 except Exception as exc:
 row["attempt_error"] = repr(exc)
 return row

cu = _integer_if_close(float(row["chern_up"]), config.chern_integer_tol)
cd = _integer_if_close(float(row["chern_down"]), config.chern_integer_tol)
ct = _integer_if_close(float(row["chern_total"]), config.chern_integer_tol)
row.update({"chern_up_int": cu, "chern_down_int": cd, "chern_total_int": ct})

reliable = bool(
 cu is not None
 and cd is not None
 and ct is not None
 and float(row["min_det_up"]) > config.min_link_tol
 and float(row["min_det_down"]) > config.min_link_tol
 and float(row["min_det_total"]) > config.min_link_tol
 and float(row["sum_rule_error"]) <= config.sum_rule_tol
)
row["attempt_reliable"] = int(reliable)
return row

def adaptive_spin_chern(
 params: Dict[str, float],
 config: Step02Config,
) -> tuple[dict, list[dict]]:
 attempts: list[dict] = []
 for nk in config.chern_grids:
 for shift in config.chern_shifts:
 attempts.append(one_chern_attempt(params, int(nk), shift, config))

 reliable = [row for row in attempts if int(row.get("attempt_reliable", 0)) == 1]
 if not reliable:
 return {
 "chern_reliable": 0,
 "chern_consensus_count": 0,
 "chern_status": "unresolved",
 "chern_up": np.nan,
 "chern_down": np.nan,
 "chern_total": np.nan,
 "spin_chern": np.nan,
 "chern_up_int": np.nan,
 "chern_down_int": np.nan,
 "chern_total_int": np.nan,
 "min_link_consensus": np.nan,
 }, attempts

 tuples = [
 (
 int(row["chern_up_int"]),
 int(row["chern_down_int"]),
 int(row["chern_total_int"]),
)
 for row in reliable
]
 counts = pd.Series(tuples, dtype="object").value_counts()
 consensus_tuple = counts.index[0]
 consensus_count = int(counts.iloc[0])
 consensus_ok = consensus_count >= int(config.min_chern_consensus)

 matching = [
 row for row in reliable
 if (
 int(row["chern_up_int"]),
 int(row["chern_down_int"]),
 int(row["chern_total_int"]),
) == consensus_tuple
]

```

---

```

chosen = matching[-1]
min_link = min(
 min(float(row["min_det_up"]), float(row["min_det_down"]), float(row["min_det_total"]))
 for row in matching
)
cu, cd, ct = consensus_tuple

return {
 "chern_reliable": int(consensus_ok),
 "chern_consensus_count": consensus_count,
 "chern_status": "consensus" if consensus_ok else "single_valid_attempt",
 "chern_up": float(chosen["chern_up"]),
 "chern_down": float(chosen["chern_down"]),
 "chern_total": float(chosen["chern_total"]),
 "spin_chern": 0.5 * (float(chosen["chern_up"]) - float(chosen["chern_down"])),
 "chern_up_int": int(cu),
 "chern_down_int": int(cd),
 "chern_total_int": int(ct),
 "min_link_consensus": float(min_link),
}, attempts

def evaluate_sample_physics(row: pd.Series, config: Step02Config) -> tuple[dict, list[dict]]:
 reduced = {name: float(row[name]) for name in REDUCED7}
 params = core.raw8_from_reduced7(reduced, e0=0.0)
 sample_id = str(row["sample_id"])

 out: dict = {
 "sample_id": sample_id,
 "source_region": str(row["source_region"]),
 **reduced,
 **params,
 **core.derived_features(params),
 "phase_label": "true_numeric_error",
 "error": "",
 "chern_reliable": 0,
 "is_spin_chern_topological": 0,
 "is_spin_chern_TI_candidate": 0,
 "is_typeII_QSH_confirmed": 0,
 }

 if "bridge_alpha" in row and pd.notna(row.get("bridge_alpha", np.nan)):
 out["bridge_alpha"] = float(row["bridge_alpha"])

 attempts: list[dict] = []
 try:
 # Save all gap/filling information before any Chern calculation.
 gap = core.scan_band_gaps(
 params,
 nk=int(config.gap_nk),
 shift=(0.0, 0.0),
 gap_tol=config.gap_tol,
)
 out.update(gap)

 if float(gap["min_direct_gap"]) <= config.gap_tol:
 out["phase_label"] = "noninsulating_or_gap_closing"
 out["chern_status"] = "not_computed_for_gap_closing"
 return out, attempts

 if int(gap["is_balanced_spin_sector"]) != 1:
 out["phase_label"] = "spin_sector_filling_mismatch"
 out["chern_status"] = "not_computed_for_filling_mismatch"
 return out, attempts

 if float(gap["min_direct_gap"]) <= config.direct_gap_skip_chern:
 out["phase_label"] = "boundary_or_small_gap"
 out["chern_status"] = "skipped_small_direct_gap"
 return out, attempts

```

---

```

 chern, raw_attempts = adaptive_spin_chern(params, config)
 for attempt in raw_attempts:
 item = dict(attempt)
 item["sample_id"] = sample_id
 attempts.append(item)
 out.update(chern)

 if int(chern["chern_reliable"]) != 1:
 out["phase_label"] = "chern_unresolved"
 return out, attempts

 cu = int(chern["chern_up_int"])
 cd = int(chern["chern_down_int"])
 ct = int(chern["chern_total_int"])
 topological = bool(cu == -cd and abs(cu) >= 1 and ct == 0)
 indirect_positive = bool(float(gap["indirect_gap"]) > config.gap_tol)

 out["is_spin_chern_topological"] = int(topological)
 if topological and indirect_positive:
 out["phase_label"] = "spin_chern_TI_candidate"
 out["is_spin_chern_TI_candidate"] = 1
 elif topological:
 out["phase_label"] = "spin_chern_band_metal"
 elif cu == 0 and cd == 0 and ct == 0 and indirect_positive:
 out["phase_label"] = "trivial_insulator"
 elif cu == 0 and cd == 0 and ct == 0:
 out["phase_label"] = "indirect_overlap"
 elif ct != 0:
 out["phase_label"] = "Chern_insulator"
 else:
 out["phase_label"] = "other_gapped_phase"

except Exception as exc:
 out["phase_label"] = "true_numeric_error"
 out["chern_status"] = "exception"
 out["error"] = repr(exc)

return out, attempts

def run_physics_labels(
 config: Step02Config,
 outputs: Dict[str, Path],
 candidates: pd.DataFrame,
) -> tuple[pd.DataFrame, pd.DataFrame]:
 checkpoint = outputs["physics_checkpoint"]
 attempts_path = outputs["chern_attempts"]

 if checkpoint.exists() and not config.force_recalculate_physics:
 physics_df = pd.read_csv(checkpoint, low_memory=False)
 done_ids = set(physics_df["sample_id"].astype(str))
 print(f"Loaded physics checkpoint: {len(done_ids)} samples")
 else:
 physics_df = pd.DataFrame()
 done_ids: set[str] = set()

 if attempts_path.exists() and not config.force_recalculate_physics:
 attempts_df = pd.read_csv(attempts_path, low_memory=False)
 else:
 attempts_df = pd.DataFrame()

 pending: list[dict] = []
 pending_attempts: list[dict] = []
 total = len(candidates)
 started = time.time()

 for _, row in candidates.iterrows():
 sample_id = str(row["sample_id"])
 if sample_id in done_ids:

```

---

```

 continue

 result, attempts = evaluate_sample_physics(row, config)
 pending.append(result)
 pending_attempts.extend(attempts)

 if len(pending) >= config.checkpoint_every:
 physics_df = pd.concat([physics_df, pd.DataFrame(pending)], ignore_index=True)
 physics_df = physics_df.drop_duplicates("sample_id", keep="last")
 atomic_write_csv(physics_df, checkpoint)
 done_ids.update(str(x["sample_id"]) for x in pending)
 pending = []

 if pending_attempts:
 attempts_df = pd.concat(
 [attempts_df, pd.DataFrame(pending_attempts)],
 ignore_index=True,
)
 attempts_df = attempts_df.drop_duplicates(
 ["sample_id", "chern_nk", "shift_x", "shift_y"],
 keep="last",
)
 atomic_write_csv(attempts_df, attempts_path)
 pending_attempts = []
 print(f"Physics labels: {len(done_ids)}/{total}")

 if pending:
 physics_df = pd.concat([physics_df, pd.DataFrame(pending)], ignore_index=True)
 physics_df = physics_df.drop_duplicates("sample_id", keep="last")
 atomic_write_csv(physics_df, checkpoint)
 atomic_write_csv(physics_df, outputs["physics_final"])

 if pending_attempts:
 attempts_df = pd.concat([attempts_df, pd.DataFrame(pending_attempts)], ignore_index=True)
 if not attempts_df.empty:
 attempts_df = attempts_df.drop_duplicates(
 ["sample_id", "chern_nk", "shift_x", "shift_y"],
 keep="last",
)
 atomic_write_csv(attempts_df, attempts_path)

 counts = (
 physics_df["phase_label"]
 .value_counts(dropna=False)
 .rename_axis("phase_label")
 .reset_index(name="count")
)
 counts["fraction"] = counts["count"] / max(len(physics_df), 1)
 atomic_write_csv(counts, outputs["phase_counts"])

 fig, ax = plt.subplots(figsize=(9, 5.4))
 ax.bar(counts["phase_label"].astype(str), counts["count"])
 ax.set_ylabel("Count")
 ax.set_title("TTS Step 02 v3 robust phase labels")
 ax.tick_params(axis="x", rotation=35)
 fig.tight_layout()
 fig.savefig(outputs["phase_counts_fig"], dpi=220)
 plt.close(fig)

 print(f"Physics stage elapsed: {time.time() - started:.1f} s")
 return physics_df, attempts_df

```

```

=====
Step 5. Gauge-consistent 4x4-spin-block Hamiltonian fingerprint
=====

```

```

K_POINTS = {
 "G": (0.0, 0.0),

```

---

```

 "X": (np.pi, 0.0),
 "Y": (0.0, np.pi),
 "M": (np.pi, np.pi),
 "Mp": (-np.pi, np.pi),
 "S50": (0.50 * np.pi, 0.50 * np.pi),
 "Sp50": (-0.50 * np.pi, 0.50 * np.pi),
 "Z50": (np.pi, 0.50 * np.pi),
 "Zp50": (0.50 * np.pi, np.pi),
}

PATHS = {
 "GM_Sigma": (np.array([0.0, 0.0]), np.array([np.pi, np.pi])),
 "GMp_SigmaPrime": (np.array([0.0, 0.0]), np.array([-np.pi, np.pi])),
 "GX_Delta": (np.array([0.0, 0.0]), np.array([np.pi, 0.0])),
 "GY_DeltaPrime": (np.array([0.0, 0.0]), np.array([0.0, np.pi])),
 "XM_Z": (np.array([np.pi, 0.0]), np.array([np.pi, np.pi])),
 "YMp_ZPrime": (np.array([0.0, np.pi]), np.array([-np.pi, np.pi])),
}

def _spectral_moments(evals: np.ndarray) -> dict:
 e = np.asarray(evals, dtype=float)
 center = float(np.mean(e))
 centered = e - center
 return {
 "center": center,
 "bandwidth": float(e[-1] - e[0]),
 "moment2": float(np.mean(centered ** 2)),
 "moment3": float(np.mean(centered ** 3)),
 }

def _occupied_projector(h: np.ndarray, n_occ: int) -> np.ndarray:
 _, vec = np.linalg.eigh(h)
 occ = vec[:, : int(n_occ)]
 return occ @ occ.conj().T

def _projector_distance(p: np.ndarray, q: np.ndarray) -> float:
 return float(np.linalg.norm(p - q, ord="fro") / math.sqrt(2.0))

def fixed_k_features(params: Dict[str, float], klabel: str, kx: float, ky: float) -> tuple[dict, list[dict]]:
 feat: dict = {}
 meta: list[dict] = []

 spin_evals: dict[str, np.ndarray] = {}
 for spin in ("up", "down"):
 h4 = core.h_spin_block_periodic(kx, ky, params, spin)
 evals = np.linalg.eigvalsh(h4)
 spin_evals[spin] = evals
 values = {
 "occ_center": float(np.mean(evals[:2])),
 "unocc_center": float(np.mean(evals[2:])),
 "block_gap": float(evals[2] - evals[1]),
 "bandwidth": float(evals[-1] - evals[0]),
 }
 for channel, value in values.items():
 name = f"{spin}_{channel}_{klabel}"
 feat[name] = value
 meta.append({
 "feature": name,
 "spin": spin,
 "channel": channel,
 "k_region": klabel,
 "feature_family": "fixed_k_spin_spectrum",
 })

 full = np.linalg.eigvalsh(core.h_tts_periodic(kx, ky, params))

```

---

```

full_values = {
 "full_direct_gap": float(full[4] - full[3]),
 "full_bandwidth": float(full[-1] - full[0]),
 "spin_spectrum_l2": float(np.linalg.norm(spin_evals["up"] - spin_evals["down"])),
 "spin_spectrum_max": float(np.max(np.abs(spin_evals["up"] - spin_evals["down"]))),
}
for channel, value in full_values.items():
 name = f"{channel}_{klabel}"
 feat[name] = value
 meta.append({
 "feature": name,
 "spin": "both",
 "channel": channel,
 "k_region": klabel,
 "feature_family": "fixed_k_full_spectrum",
 })
return feat, meta

def path_summary(
 params: Dict[str, float],
 start: np.ndarray,
 end: np.ndarray,
 n: int,
) -> dict:
 t_values = np.linspace(0.0, 1.0, int(n))
 full_gaps: list[float] = []
 up_gaps: list[float] = []
 down_gaps: list[float] = []
 split_l2: list[float] = []
 split_max: list[float] = []

 p_up: list[np.ndarray] = []
 p_down: list[np.ndarray] = []
 p_full: list[np.ndarray] = []

 for t in t_values:
 k = (1.0 - t) * start + t * end
 kx, ky = float(k[0]), float(k[1])
 h_up = core.h_spin_block_periodic(kx, ky, params, "up")
 h_down = core.h_spin_block_periodic(kx, ky, params, "down")
 h_full = core.h_tts_periodic(kx, ky, params)

 eu = np.linalg.eigvalsh(h_up)
 ed = np.linalg.eigvalsh(h_down)
 ef = np.linalg.eigvalsh(h_full)
 up_gaps.append(float(eu[2] - eu[1]))
 down_gaps.append(float(ed[2] - ed[1]))
 full_gaps.append(float(ef[4] - ef[3]))
 split_l2.append(float(np.linalg.norm(eu - ed)))
 split_max.append(float(np.max(np.abs(eu - ed))))

 p_up.append(_occupied_projector(h_up, 2))
 p_down.append(_occupied_projector(h_down, 2))
 p_full.append(_occupied_projector(h_full, 4))

 full_arr = np.asarray(full_gaps)
 up_arr = np.asarray(up_gaps)
 down_arr = np.asarray(down_gaps)
 l2_arr = np.asarray(split_l2)
 max_arr = np.asarray(split_max)

 def path_length(projectors: list[np.ndarray]) -> float:
 return float(sum(
 _projector_distance(projectors[i], projectors[i + 1])
 for i in range(len(projectors) - 1)
))

 return {

```

---

```

 "full_gap_min": float(np.min(full_arr)),
 "full_gap_mean": float(np.mean(full_arr)),
 "full_gap_argmin_t": float(t_values[int(np.argmin(full_arr))]),
 "up_gap_min": float(np.min(up_arr)),
 "down_gap_min": float(np.min(down_arr)),
 "spin_split_l2_max": float(np.max(l2_arr)),
 "spin_split_l2_mean": float(np.mean(l2_arr)),
 "spin_split_max_max": float(np.max(max_arr)),
 "up_projector_endpoint_distance": _projector_distance(p_up[0], p_up[-1]),
 "down_projector_endpoint_distance": _projector_distance(p_down[0], p_down[-1]),
 "full_projector_endpoint_distance": _projector_distance(p_full[0], p_full[-1]),
 "full_projector_path_length": path_length(p_full),
 }

def extract_fingerprint(sample_id: str, params: Dict[str, float], path_n: int) -> tuple[dict, list[dict]]:
 feat: dict = {"sample_id": sample_id}
 meta: list[dict] = []

 for klabel, (kx, ky) in K_POINTS.items():
 fixed, fixed_meta = fixed_k_features(params, klabel, float(kx), float(ky))
 feat.update(fixed)
 meta.extend(fixed_meta)

 for path_name, (start, end) in PATHS.items():
 summary = path_summary(params, start, end, path_n)
 for channel, value in summary.items():
 name = f"{channel}_{path_name}"
 feat[name] = value
 spin = (
 "up" if channel.startswith("up_")
 else "down" if channel.startswith("down_")
 else "both"
)
 feat_meta = {
 "feature": name,
 "spin": spin,
 "channel": channel,
 "k_region": path_name,
 "feature_family": (
 "path_projector" if "projector" in channel else "path_spectrum"
),
 }
 meta.append(feat_meta)
 return feat, meta

def run_fingerprint(
 config: Step02Config,
 outputs: Dict[str, Path],
 candidates: pd.DataFrame,
) -> tuple[pd.DataFrame, pd.DataFrame]:
 fp_path = outputs["fingerprint"]
 meta_path = outputs["feature_metadata"]

 if (
 fp_path.exists()
 and meta_path.exists()
 and not config.force_recalculate_fingerprint
):
 return (
 pd.read_csv(fp_path, low_memory=False),
 pd.read_csv(meta_path, low_memory=False),
)

 fp_rows: list[dict] = []
 meta_rows: list[dict] = []
 started = time.time()
 total = len(candidates)

```

---

```

for count, (_, row) in enumerate(candidates.iterrows(), start=1):
 reduced = {name: float(row[name]) for name in REDUCED7}
 params = core.raw8_from_reduced7(reduced, e0=0.0)
 feat, meta = extract_fingerprint(str(row["sample_id"]), params, config.path_n)
 fp_rows.append(feat)
 meta_rows.extend(meta)
 if count % max(config.checkpoint_every, 1) == 0 or count == total:
 print(f"Fingerprint: {count}/{total}")

fingerprint_df = pd.DataFrame(fp_rows)
metadata_df = pd.DataFrame(meta_rows).drop_duplicates("feature", keep="first")
atomic_write_csv(fingerprint_df, fp_path)
atomic_write_csv(metadata_df, meta_path)
print(f"Fingerprint stage elapsed: {time.time() - started:.1f} s")
return fingerprint_df, metadata_df

=====
Steps 6-13. Binary ML, shortcut diagnostics and explainability
=====

def build_binary_dataset(
 outputs: Dict[str, Path],
 physics_df: pd.DataFrame,
 fingerprint_df: pd.DataFrame,
) -> tuple[pd.DataFrame, pd.DataFrame]:
 full_df = physics_df.merge(fingerprint_df, on="sample_id", how="inner")
 binary = full_df[
 full_df["phase_label"].isin(
 ["spin_chern_TI_candidate", "trivial_insulator"]
)
].copy()
 binary["target_typeII"] = (
 binary["phase_label"] == "spin_chern_TI_candidate"
).astype(int)
 atomic_write_csv(binary, outputs["binary_dataset"])

 if binary.empty:
 diagnostic = pd.DataFrame(columns=[
 "source_region", "max_label_fraction", "dominant_label", "shortcut_warning"
])
 else:
 fractions = pd.crosstab(
 binary["source_region"], binary["phase_label"], normalize="index"
)
 rows = []
 for source, values in fractions.iterrows():
 rows.append({
 "source_region": source,
 "max_label_fraction": float(values.max()),
 "dominant_label": str(values.idxmax()),
 "shortcut_warning": int(float(values.max()) >= 0.90),
 })
 diagnostic = pd.DataFrame(rows)
 atomic_write_csv(diagnostic, outputs["source_label_diagnostic"])

 if not diagnostic.empty and diagnostic["shortcut_warning"].any():
 warnings.warn(
 " 至少一个采样来源中单一标签占比 >= 90%。随机拆分可能学习采样来源捷径； "
 " 必须结合 global_random 外部测试解释。"
)

 return binary, diagnostic

def make_model(name: str, config: Step02Config, random_state: int = 42):
 if name == "logistic":
 return Pipeline([
 ("scale", StandardScaler()),

```



---

```

 ("model", LogisticRegression(
 max_iter=5000,
 class_weight="balanced",
 random_state=random_state,
)),
])
if name == "random_forest":
 return RandomForestClassifier(
 n_estimators=int(config.random_forest_estimators),
 min_samples_leaf=2,
 class_weight="balanced_subsample",
 random_state=random_state,
 n_jobs=int(config.ml_n_jobs),
)
raise ValueError(name)

def metric_dict(y_true: Iterable[int], y_pred: Iterable[int]) -> dict:
 y_true_arr = np.asarray(list(y_true), dtype=int)
 y_pred_arr = np.asarray(list(y_pred), dtype=int)
 return {
 "accuracy": float(accuracy_score(y_true_arr, y_pred_arr)),
 "balanced_accuracy": float(balanced_accuracy_score(y_true_arr, y_pred_arr)),
 "precision": float(precision_score(y_true_arr, y_pred_arr, zero_division=0)),
 "recall": float(recall_score(y_true_arr, y_pred_arr, zero_division=0)),
 "f1": float(f1_score(y_true_arr, y_pred_arr, zero_division=0)),
 }

def _adaptive_cv(binary_df: pd.DataFrame, config: Step02Config):
 counts = binary_df["target_typeII"].value_counts()
 if len(counts) < 2:
 return None
 n_splits = min(int(config.cv_splits), int(counts.min()))
 if n_splits < 2:
 return None
 return RepeatedStratifiedKFold(
 n_splits=n_splits,
 n_repeats=int(config.cv_repeats),
 random_state=42,
)

def grouped_permutation_importance(
 model,
 X: pd.DataFrame,
 y: pd.Series,
 groups: Dict[str, list[str]],
 repeats: int,
 seed: int = 42,
) -> pd.DataFrame:
 rng = np.random.default_rng(seed)
 baseline = balanced_accuracy_score(y, model.predict(X))
 rows: list[dict] = []

 for group_name, features in groups.items():
 features = [f for f in features if f in X.columns]
 if not features:
 continue
 drops: list[float] = []
 for _ in range(int(repeats)):
 permuted = X.copy()
 order = rng.permutation(len(permuted))
 # Permute the group jointly to preserve within-group correlations.
 permuted.loc[:, features] = permuted.iloc[order][features].to_numpy()
 score = balanced_accuracy_score(y, model.predict(permuted))
 drops.append(float(baseline - score))
 rows.append({
 "group": group_name,

```

---

```

 "n_features": len(features),
 "importance_mean": float(np.mean(drops)),
 "importance_std": float(np.std(drops)),
 "baseline_balanced_accuracy": float(baseline),
 })
 return pd.DataFrame(rows).sort_values("importance_mean", ascending=False)

def run_machine_learning(
 config: Step02Config,
 outputs: Dict[str, Path],
 binary_df: pd.DataFrame,
 metadata_df: pd.DataFrame,
) -> dict:
 summary: dict = {
 "ml_status": "not_run",
 "n_binary": int(len(binary_df)),
 "n_topological": int(binary_df.get("target_typeII", pd.Series(dtype=int)).sum()),
 "n_trivial": int((1 - binary_df.get("target_typeII", pd.Series(dtype=int))).sum()) if len(binary_df) else
 ↪ 0,
 }

 empty_csvs = [
 "cv_metrics", "holdout_metrics", "external_global_metrics",
 "validation_predictions", "individual_importance",
 "group_importance_family", "group_importance_kregion",
 "group_importance_spin", "class_feature_means", "validation_subset",
]

 if binary_df.empty or binary_df["target_typeII"].nunique() < 2:
 summary["ml_status"] = "insufficient_two_class_data"
 for key in empty_csvs:
 atomic_write_csv(pd.DataFrame(), outputs[key])
 outputs["classification_report"].write_text(
 "ML not run: strict binary dataset does not contain both classes.\n",
 encoding="utf-8",
)
 return summary

 fp_features = metadata_df["feature"].astype(str).tolist()
 fp_features = [f for f in fp_features if f in binary_df.columns]
 feature_sets = {
 "Reduced7": REDUCED7.copy(),
 "HamiltonianFingerprint": fp_features,
 "Combined": REDUCED7.copy() + fp_features,
 }

 cv = _adaptive_cv(binary_df, config)
 cv_rows: list[dict] = []
 scoring = {
 "accuracy": "accuracy",
 "balanced_accuracy": "balanced_accuracy",
 "precision": "precision",
 "recall": "recall",
 "f1": "f1",
 }

 if cv is not None:
 for feature_name, features in feature_sets.items():
 X = binary_df[features].astype(float)
 y = binary_df["target_typeII"].astype(int)
 for model_name in ("logistic", "random_forest"):
 result = cross_validate(
 make_model(model_name, config),
 X,
 y,
 cv=cv,
 scoring=scoring,
 n_jobs=int(config.ml_n_jobs),
 return_train_score=False,

```

---

```

 error_score=np.nan,
)
 row = {"feature_set": feature_name, "model": model_name}
 for metric in scoring:
 values = np.asarray(result[f"test_{metric}"], dtype=float)
 row[f"{metric}_mean"] = float(np.nanmean(values))
 row[f"{metric}_std"] = float(np.nanstd(values))
 cv_rows.append(row)
cv_metrics = pd.DataFrame(cv_rows)
if not cv_metrics.empty:
 cv_metrics = cv_metrics.sort_values("balanced_accuracy_mean", ascending=False)
atomic_write_csv(cv_metrics, outputs["cv_metrics"])

Holdout uses Combined features; fallback to all data if stratified split is impossible.
best_features = feature_sets["Combined"]
X_all = binary_df[best_features].astype(float)
y_all = binary_df["target_typeII"].astype(int)
class_counts = y_all.value_counts()
can_holdout = bool(class_counts.min() >= 2 and len(binary_df) >= 8)

if can_holdout:
 X_train, X_val, y_train, y_val = train_test_split(
 X_all,
 y_all,
 test_size=float(config.validation_fraction),
 random_state=42,
 stratify=y_all,
)
else:
 X_train, X_val, y_train, y_val = X_all, X_all, y_all, y_all
 warnings.warn(" 样本过少, holdout 使用训练集回代; 该指标不能作为泛化证据。")

best_model = make_model("random_forest", config)
best_model.fit(X_train, y_train)
y_pred = best_model.predict(X_val)
y_prob = best_model.predict_proba(X_val)[: , 1]

holdout = pd.DataFrame([
 **metric_dict(y_val, y_pred),
 "n_train": int(len(X_train)),
 "n_validation": int(len(X_val)),
 "holdout_is_independent": int(can_holdout),
])
atomic_write_csv(holdout, outputs["holdout_metrics"])

val_rows = binary_df.loc[X_val.index, [
 "sample_id", "source_region", "phase_label", *REDUCED7
]].copy()
val_rows["target_typeII"] = y_val
val_rows["predicted_typeII"] = y_pred
val_rows["probability_typeII"] = y_prob
atomic_write_csv(val_rows, outputs["validation_predictions"])

report = classification_report(
 y_val,
 y_pred,
 target_names=["trivial", "spin_chern_TI_candidate"],
 zero_division=0,
)
outputs["classification_report"].write_text(report, encoding="utf-8")

fig, ax = plt.subplots(figsize=(5.5, 5.0))
ConfusionMatrixDisplay(
 confusion_matrix=confusion_matrix(y_val, y_pred, labels=[0, 1]),
 display_labels=["trivial", "TI candidate"],
).plot(ax=ax, values_format="d")
ax.set_title("TTS Step 02 v3 validation")
fig.tight_layout()
fig.savefig(outputs["confusion_matrix"], dpi=220)

```

---

```

plt.close(fig)

External global_random test, trained only on non-global samples.
train_external = binary_df[binary_df["source_region"] != "global_random"].copy()
test_external = binary_df[binary_df["source_region"] == "global_random"].copy()
external_rows: list[dict] = []
if train_external["target_typeII"].nunique() == 2 and len(test_external) > 0:
 external_model = make_model("random_forest", config)
 external_model.fit(
 train_external[best_features].astype(float),
 train_external["target_typeII"].astype(int),
)
 ext_pred = external_model.predict(test_external[best_features].astype(float))
 row = metric_dict(test_external["target_typeII"].astype(int), ext_pred)
 row.update({
 "n_train": int(len(train_external)),
 "n_test_global": int(len(test_external)),
 "n_test_topological": int(test_external["target_typeII"].sum()),
 "n_test_trivial": int((1 - test_external["target_typeII"].sum()),
 "test_contains_both_classes": int(test_external["target_typeII"].nunique() == 2),
 })
 external_rows.append(row)
else:
 external_rows.append({
 "n_train": int(len(train_external)),
 "n_test_global": int(len(test_external)),
 "warning": "external test unavailable or one-class training set",
 })
atomic_write_csv(pd.DataFrame(external_rows), outputs["external_global_metrics"])

Individual importance.
individual = permutation_importance(
 best_model,
 X_val,
 y_val,
 n_repeats=int(config.permutation_repeats),
 random_state=42,
 scoring="balanced_accuracy",
 n_jobs=int(config.ml_n_jobs),
)
individual_df = pd.DataFrame({
 "feature": best_features,
 "importance_mean": individual.importances_mean,
 "importance_std": individual.importances_std,
}).sort_values("importance_mean", ascending=False)
atomic_write_csv(individual_df, outputs["individual_importance"])

meta = metadata_df.set_index("feature")
family_groups: Dict[str, list[str]] = {"reduced7_parameters": REDUCED7.copy()}
kregion_groups: Dict[str, list[str]] = {}
spin_groups: Dict[str, list[str]] = {"reduced7_parameters": REDUCED7.copy()}
for feature in fp_features:
 if feature not in meta.index:
 continue
 row = meta.loc[feature]
 family_groups.setdefault(str(row["feature_family"]), []).append(feature)
 kregion_groups.setdefault(str(row["k_region"]), []).append(feature)
 spin_groups.setdefault(str(row["spin"]), []).append(feature)

family_imp = grouped_permutation_importance(
 best_model, X_val, y_val, family_groups, config.permutation_repeats
)
kregion_imp = grouped_permutation_importance(
 best_model, X_val, y_val, kregion_groups, config.permutation_repeats
)
spin_imp = grouped_permutation_importance(
 best_model, X_val, y_val, spin_groups, config.permutation_repeats
)
atomic_write_csv(family_imp, outputs["group_importance_family"])

```

---

```

atomic_write_csv(kregion_imp, outputs["group_importance_kregion"])
atomic_write_csv(spin_imp, outputs["group_importance_spin"])

if not family_imp.empty:
 fig, ax = plt.subplots(figsize=(8, 5.2))
 plot_df = family_imp.sort_values("importance_mean", ascending=True)
 ax.barh(plot_df["group"], plot_df["importance_mean"], xerr=plot_df["importance_std"])
 ax.set_xlabel("Balanced-accuracy decrease")
 ax.set_title("Grouped permutation importance by feature family")
 fig.tight_layout()
 fig.savefig(outputs["group_importance_family_fig"], dpi=220)
 plt.close(fig)

Class means for top individual features.
top_features = individual_df.head(20)["feature"].tolist()
class_means = binary_df.groupby("phase_label")[top_features].mean().T
if {
 "spin_chern_TI_candidate", "trivial_insulator"
}.issubset(class_means.columns):
 class_means["TI_minus_trivial"] = (
 class_means["spin_chern_TI_candidate"]
 - class_means["trivial_insulator"]
)
class_means = class_means.reset_index(names="feature")
atomic_write_csv(class_means, outputs["class_feature_means"])

Representative Hamiltonians for independent MagneticTB or later strict checks.
representatives: list[pd.DataFrame] = []
for label in ("spin_chern_TI_candidate", "trivial_insulator"):
 part = binary_df[binary_df["phase_label"] == label].copy()
 if part.empty:
 continue
 part = part.sort_values(
 ["indirect_gap", "min_direct_gap"], ascending=False
).head(5)
 representatives.append(part[[
 "sample_id", "source_region", "phase_label",
 *REDUCED7, *RAW8,
 "min_direct_gap", "indirect_gap",
 "chern_up_int", "chern_down_int", "chern_total_int",
]])
validation_subset = (
 pd.concat(representatives, ignore_index=True)
 if representatives else pd.DataFrame()
)
atomic_write_csv(validation_subset, outputs["validation_subset"])

dump(best_model, outputs["model"])
summary.update({
 "ml_status": "completed",
 "n_features_reduced7": len(REDUCED7),
 "n_features_fingerprint": len(fp_features),
 "n_features_combined": len(best_features),
 "holdout_balanced_accuracy": float(holdout.iloc[0]["balanced_accuracy"]),
 "holdout_f1": float(holdout.iloc[0]["f1"]),
 "holdout_is_independent": int(can_holdout),
 "best_model_path": str(outputs["model"]),
})
return summary

```

```

=====
Full workflow
=====

```

```

def run_step02(config: Step02Config | None = None) -> Dict[str, object]:
 config = (config or Step02Config()).normalized()
 outputs = build_output_registry(config)
 write_run_metadata(config, outputs)

```

---

```

print("=" * 78)
print("TTS Step 02 v3 — robust Hamiltonian fingerprint and ML")
print("Step 01 core:", core.CODE_VERSION)
print("Output directory:", config.output_dir.resolve())
print("All physics calculations: SERIAL")
print("ML n_jobs:", config.ml_n_jobs)
print("=" * 78)

print("[1/7] Four-source parameter sampling")
candidates = generate_candidates(config, outputs)
print(candidates.groupby("source_region").size())

print("[2/7] Robust full-BZ physics labels")
physics_df, attempts_df = run_physics_labels(config, outputs, candidates)
print(physics_df["phase_label"].value_counts(dropna=False))
print(pd.crosstab(physics_df["source_region"], physics_df["phase_label"]))

print("[3/7] Gauge-consistent 4x4-spin-block fingerprint")
fingerprint_df, metadata_df = run_fingerprint(config, outputs, candidates)
print("Fingerprint shape:", fingerprint_df.shape)

print("[4/7] Strict binary dataset and source shortcut diagnosis")
binary_df, source_diag = build_binary_dataset(outputs, physics_df, fingerprint_df)
print(binary_df["phase_label"].value_counts(dropna=False))
if not source_diag.empty:
 print(source_diag.to_string(index=False))

print("[5/7] Repeated CV, holdout and external global test")
ml_summary = run_machine_learning(config, outputs, binary_df, metadata_df)
print(json.dumps(ml_summary, indent=2, ensure_ascii=False))

print("[6/7] Save run summary")
phase_counts = physics_df["phase_label"].value_counts(dropna=False).to_dict()
summary = {
 "code_version": CODE_VERSION,
 "step01_core_version": core.CODE_VERSION,
 "n_candidates": int(len(candidates)),
 "n_physics_rows": int(len(physics_df)),
 "n_bernoulli_attempt_rows": int(len(attempts_df)),
 "n_fingerprint_features": int(fingerprint_df.shape[1] - 1),
 "phase_counts": {str(k): int(v) for k, v in phase_counts.items()},
 "n_strict_binary": int(len(binary_df)),
 **ml_summary,
 "next_decision": (
 "If both classes are sufficiently populated, proceed to Step 03 global Sobol. "
 "Do not interpret local random-split accuracy as a final physical law."
),
}
outputs["summary"].write_text(
 json.dumps(summary, indent=2, ensure_ascii=False),
 encoding="utf-8",
)

print("[7/7] Done")
print("Summary:", outputs["summary"].resolve())
return {
 "config": config,
 "outputs": outputs,
 "candidates": candidates,
 "physics": physics_df,
 "chern_attempts": attempts_df,
 "fingerprint": fingerprint_df,
 "feature_metadata": metadata_df,
 "binary": binary_df,
 "source_diagnostic": source_diag,
 "summary": summary,
}

```

---

```

=====
CLI
=====

def build_arg_parser() -> argparse.ArgumentParser:
 parser = argparse.ArgumentParser(description=__doc__)
 parser.add_argument(
 "--output-dir",
 type=Path,
 default=Step02Config.output_dir,
)
 parser.add_argument("--n-topo-local", type=int, default=40)
 parser.add_argument("--n-trivial-local", type=int, default=40)
 parser.add_argument("--n-bridge", type=int, default=40)
 parser.add_argument("--n-global", type=int, default=40)
 parser.add_argument("--seed", type=int, default=20260713)
 parser.add_argument("--gap-nk", type=int, default=60)
 parser.add_argument("--path-n", type=int, default=31)
 parser.add_argument("--ml-n-jobs", type=int, default=1)
 parser.add_argument("--force", action="store_true")
 parser.add_argument("--quick", action="store_true")
 return parser

def config_from_args(args: argparse.Namespace) -> Step02Config:
 if args.quick:
 return Step02Config(
 output_dir=args.output_dir,
 n_topo_local=min(args.n_topo_local, 4),
 n_trivial_local=min(args.n_trivial_local, 4),
 n_bridge=min(args.n_bridge, 4),
 n_global=min(args.n_global, 4),
 random_seed=args.seed,
 gap_nk=min(args.gap_nk, 24),
 chern_grids=(11, 15),
 chern_shifts=((0.0, 0.0), (0.5, 0.5)),
 path_n=min(args.path_n, 11),
 cv_splits=3,
 cv_repeats=2,
 random_forest_estimators=80,
 permutation_repeats=2,
 ml_n_jobs=args.ml_n_jobs,
 checkpoint_every=4,
 force_recalculate_physics=args.force,
 force_recalculate_fingerprint=args.force,
)
 return Step02Config(
 output_dir=args.output_dir,
 n_topo_local=args.n_topo_local,
 n_trivial_local=args.n_trivial_local,
 n_bridge=args.n_bridge,
 n_global=args.n_global,
 random_seed=args.seed,
 gap_nk=args.gap_nk,
 path_n=args.path_n,
 ml_n_jobs=args.ml_n_jobs,
 force_recalculate_physics=args.force,
 force_recalculate_fingerprint=args.force,
)

def main() -> int:
 parser = build_arg_parser()
 args = parser.parse_args()
 config = config_from_args(args)
 run_step02(config)
 return 0

```

```
if __name__ == "__main__":
 raise SystemExit(main())
```

## ► 代码清单 A.49.3 (python, 1790 行)

```
#!/usr/bin/env python3
-*- coding: utf-8 -*-
"""
TTS Step 03 ——七维全局 Sobol、严格 spin-Chern 分扇区与分层机器学习

研究目标

1. 在去除整体能量平移后的七维参数空间
 (m_e, t1, t2, r1, r2, r3, r4)
 中进行全局 Sobol 低差异采样;
2. 使用冻结的 TTS Step 01 Hamiltonian、周期规范、完整 BZ 带隙与
 非阿贝尔 spin-Chern 标签器;
3. 对所有非零 Chern 或网格不一致样本执行更严格的多网格、多 shift 复核;
4. 保留 C_up = 0, ±1, ±2, ... 的完整分扇区标签, 而不是只做“拓扑/平庸”合并;
5. 建立分层模型:
 Level A: 全局绝缘体门控;
 Level B1: 严格绝缘体中的 spin-Chern 拓扑/平庸二分类;
 Level B2: C_up 精确扇区多分类 (样本数足够时自动运行);
6. 使用参数空间整簇留出与独立 Sobol 序列评估空间外推能力。

关键实现原则

- 物理计算完全串行; 不使用 ProcessPoolExecutor, 兼容 Windows/Jupyter。
- sklearn/joblib 默认 n_jobs=1, 避免 BrokenProcessPool。
- 全局 Sobol 主样本与独立 Sobol 外部样本严格分开。
- 已知 C_up=-1、C_up=±2 和文献平庸点仅作为回归测试 control,
 不参与全局相比比例统计, 也不参与机器学习训练或测试。
- overall energy shift e0 固定为 0。默认将 Sobol 七维向量归一化到单位范数,
 消除不改变本征态和拓扑的整体能量缩放自由度。
- spin_chern_TI_candidate 仍是模型级候选; 边缘态、DOS、SHC 在后续步骤验证。
"""

from __future__ import annotations

from dataclasses import asdict, dataclass
from pathlib import Path
from typing import Dict, Iterable, Sequence, Any
import argparse
import hashlib
import json
import math
import time
import warnings

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.stats import qmc

try:
 from joblib import dump
 from sklearn.cluster import KMeans
 from sklearn.ensemble import RandomForestClassifier
 from sklearn.inspection import permutation_importance
 from sklearn.linear_model import LogisticRegression
 from sklearn.metrics import (
 accuracy_score,
```



---

```

 balanced_accuracy_score,
 classification_report,
 ConfusionMatrixDisplay,
 f1_score,
 precision_score,
 recall_score,
)
 from sklearn.model_selection import (
 GroupShuffleSplit,
 RepeatedStratifiedKFold,
 cross_validate,
 train_test_split,
)
 from sklearn.pipeline import Pipeline
 from sklearn.preprocessing import StandardScaler
except ImportError as exc: # pragma: no cover
 raise ImportError(
 "Step 03 需要 scipy、scikit-learn 与 joblib。请运行: \n"
 " conda install scipy scikit-learn joblib\n"
 " 或: \n"
 " pip install scipy scikit-learn joblib"
) from exc

try:
 import TTS_step01_model_and_label_audit_v2 as core
except ImportError as exc: # pragma: no cover
 raise ImportError(
 " 未找到 TTS_step01_model_and_label_audit_v2.py。 \n"
 " 请把 Step 01 v2 的 py 文件与本脚本放在同一目录。"
) from exc

try:
 import TTS_step02v3_robust_hamiltonian_fingerprint_and_ML as fp_core
except ImportError as exc: # pragma: no cover
 raise ImportError(
 " 未找到 TTS_step02v3_robust_hamiltonian_fingerprint_and_ML.py。 \n"
 "Step 03 复用 Step 02 v3 已审计的 4x4 自旋块 Hamiltonian fingerprint, \n"
 " 请把配套文件放在同一目录。"
) from exc

EXPECTED_STEP01_VERSION = "TTS_STEP01_V2_20260713"
EXPECTED_STEP02_VERSION = "TTS_STEP02V3_20260713"
if getattr(core, "CODE_VERSION", None) != EXPECTED_STEP01_VERSION:
 raise RuntimeError(
 "Step 01 核心版本不一致: \n"
 f" expected = {EXPECTED_STEP01_VERSION}\n"
 f" loaded = {getattr(core, 'CODE_VERSION', None)}"
)
if getattr(fp_core, "CODE_VERSION", None) != EXPECTED_STEP02_VERSION:
 raise RuntimeError(
 "Step 02 fingerprint 版本不一致: \n"
 f" expected = {EXPECTED_STEP02_VERSION}\n"
 f" loaded = {getattr(fp_core, 'CODE_VERSION', None)}"
)

np.set_printoptions(precision=10, suppress=True)

CODE_VERSION = "TTS_STEP03_V1_20260713"
WORKFLOW_STEP = "step03"
SYSTEM_TAG = "tts8_bns127391"
TASK_TAG = "global_sobol_hierarchical_topology_ml"
REDUCED7 = core.REDUCED7.copy()
RAW8 = core.RAW8.copy()

=====
Configuration
=====

```

---

```

@dataclass
class Step03Config:
 output_dir: Path = Path("outputs_tts_step03_global_sobol_hierarchical_topology_ml")

 # Independent Sobol sequences.
 train_sobol_power: int = 9 # 2^9 = 512
 external_sobol_power: int = 7 # 2^7 = 128
 train_seed: int = 20260713
 external_seed: int = 20260731
 normalize_sobol_vectors: bool = True
 include_control_points: bool = True

 # Physics label: all samples first receive this full-BZ audit.
 gap_nk: int = 60
 initial_chern_grids: tuple[int, ...] = (21, 31)
 initial_chern_shifts: tuple[tuple[float, float], ...] = (
 (0.0, 0.0),
 (0.5, 0.5),
)

 # Any nonzero Chern or inconsistent initial result is sent here.
 strict_gap_grids: tuple[int, ...] = (51, 71)
 strict_gap_shifts: tuple[tuple[float, float], ...] = (
 (0.0, 0.0),
 (0.5, 0.0),
 (0.0, 0.5),
 (0.5, 0.5),
)

 strict_chern_grids: tuple[int, ...] = (31, 41, 51)
 strict_chern_shifts: tuple[tuple[float, float], ...] = (
 (0.0, 0.0),
 (0.5, 0.5),
)

 direct_gap_skip_chern: float = 5.0e-3
 gap_tol: float = 1.0e-3
 chern_integer_tol: float = 0.08
 min_link_tol: float = 1.0e-7
 sum_rule_tol: float = 0.15

 # Hamiltonian fingerprint reused from Step 02 v3.
 fingerprint_path_n: int = 21

 # Machine learning.
 n_parameter_clusters: int = 8
 group_holdout_splits: int = 20
 group_holdout_fraction: float = 0.25
 random_validation_fraction: float = 0.25
 cv_splits: int = 5
 cv_repeats: int = 3
 random_forest_estimators: int = 500
 permutation_repeats: int = 20
 min_class_count_for_training: int = 5
 ml_n_jobs: int = 1

 # Workflow controls.
 checkpoint_every: int = 10
 force_recalculate_physics: bool = False
 force_recalculate_fingerprint: bool = False
 force_recalculate_ml: bool = False

 def normalized(self) -> "Step03Config":
 self.output_dir = Path(self.output_dir)
 self.output_dir.mkdir(parents=True, exist_ok=True)
 (self.output_dir / "figures").mkdir(parents=True, exist_ok=True)
 (self.output_dir / "models").mkdir(parents=True, exist_ok=True)
 if self.train_sobol_power < 2:
 raise ValueError("train_sobol_power must be >= 2")
 if self.external_sobol_power < 0:

```

---

```

 raise ValueError("external_sobol_power must be >= 0")
 if self.gap_nk < 8:
 raise ValueError("gap_nk must be >= 8")
 if self.checkpoint_every < 1:
 raise ValueError("checkpoint_every must be >= 1")
 if self.ml_n_jobs == 0:
 raise ValueError("ml_n_jobs cannot be 0")
 if self.n_parameter_clusters < 2:
 raise ValueError("n_parameter_clusters must be >= 2")
 return self

@property
def n_train(self) -> int:
 return 2 ** int(self.train_sobol_power)

@property
def n_external(self) -> int:
 return 0 if self.external_sobol_power == 0 else 2 ** int(self.external_sobol_power)

Known points are controls only. They are excluded from all ML datasets and global fractions.
CONTROL_REDUCED7: dict[str, dict[str, float]] = {
 "control_topological_Cm1": core.reduced7_from_raw8(core.TOPO_PARAMS),
 "control_trivial_paper": core.reduced7_from_raw8(core.PAPER_PARAMS),
 "control_high_chern_Cp2": {
 "m_e": 0.227401,
 "t1": -0.325125,
 "t2": 0.494735,
 "r1": -0.970710,
 "r2": -0.332244,
 "r3": 0.448718,
 "r4": -0.575438,
 },
 "control_high_chern_Cm2": {
 "m_e": 0.251241,
 "t1": -0.907558,
 "t2": 0.610720,
 "r1": -0.609335,
 "r2": 0.657497,
 "r3": 0.597549,
 "r4": -0.532312,
 },
}
CONTROL_EXPECTED_CUP = {
 "control_topological_Cm1": -1,
 "control_trivial_paper": 0,
 "control_high_chern_Cp2": 2,
 "control_high_chern_Cm2": -2,
}

=====
Output registry and safe file helpers
=====

def _run_tag(config: Step03Config) -> str:
 return (
 f"{SYSTEM_TAG}_train{config.n_train}_external{config.n_external}"
 f"_gap{config.gap_nk}"
 f"_chern{'-'.join(map(str, config.initial_chern_grids))}"
 f"_strict{'-'.join(map(str, config.strict_chern_grids))}"
 f"_seed{config.train_seed}"
)

def build_output_registry(config: Step03Config) -> Dict[str, Path]:
 config = config.normalized()
 run_tag = _run_tag(config)

```

---

```

figures = config.output_dir / "figures"
models = config.output_dir / "models"

def out(substep: int, content: str, ext: str, *, figure: bool = False, model: bool = False) -> Path:
 base = models if model else figures if figure else config.output_dir
 return base / f"{WORKFLOW_STEP}_{substep:02d}_{content}_{run_tag}.{ext.lstrip('.')}"

return {
 "run_config": out(0, "run_configuration", "json"),
 "registry": out(0, "output_file_registry", "csv"),
 "parameters": out(1, "train_external_sobol_and_controls", "csv"),
 "physics_checkpoint": out(2, "physics_labels_checkpoint", "csv"),
 "physics_final": out(2, "physics_labels_final", "csv"),
 "chern_attempts": out(2, "chern_attempt_records", "csv"),
 "strict_gap_checks": out(2, "strict_gap_check_records", "csv"),
 "phase_counts": out(3, "global_phase_counts", "csv"),
 "phase_counts_source": out(3, "phase_counts_by_sobol_source", "csv"),
 "chern_sector_counts": out(3, "strict_insulator_chern_sector_counts", "csv"),
 "control_audit": out(3, "control_point_regression_audit", "csv"),
 "topological_candidates": out(3, "strict_spin_chern_TI_candidates", "csv"),
 "best_by_sector": out(3, "best_gap_candidate_by_chern_sector", "csv"),
 "phase_counts_fig": out(3, "global_phase_counts", "png", figure=True),
 "chern_sector_fig": out(3, "strict_insulator_chern_sector_counts", "png", figure=True),
 "fingerprint": out(4, "hamiltonian_fingerprint", "csv"),
 "fingerprint_metadata": out(4, "hamiltonian_fingerprint_metadata", "csv"),
 "analytic_features": out(4, "symmetry_motivated_parameter_features", "csv"),
 "combined_features": out(4, "combined_ml_feature_table", "csv"),
 "cluster_assignments": out(5, "train_parameter_space_cluster_assignments", "csv"),
 "levelA_dataset": out(5, "levelA_global_insulator_gate_dataset", "csv"),
 "levelB_binary_dataset": out(5, "levelB_binary_topological_vs_trivial_dataset", "csv"),
 "levelB_multiclass_dataset": out(5, "levelB_multiclass_chern_sector_dataset", "csv"),
 "random_cv_metrics": out(6, "random_repeated_cv_metrics", "csv"),
 "cluster_holdout_metrics": out(6, "parameter_cluster_holdout_metrics", "csv"),
 "external_metrics": out(6, "independent_sobol_external_metrics", "csv"),
 "best_model_registry": out(6, "best_model_registry", "csv"),
 "external_predictions": out(7, "independent_sobol_external_predictions", "csv"),
 "cluster_predictions": out(7, "best_cluster_holdout_predictions", "csv"),
 "classification_reports": out(7, "classification_reports", "txt"),
 "levelA_confusion": out(7, "levelA_external_confusion", "png", figure=True),
 "levelB_confusion": out(7, "levelB_binary_external_confusion", "png", figure=True),
 "levelB_multi_confusion": out(7, "levelB_multiclass_external_confusion", "png", figure=True),
 "group_importance": out(8, "levelB_binary_grouped_permutation_importance", "csv"),
 "individual_importance": out(8, "levelB_binary_individual_permutation_importance", "csv"),
 "importance_fig": out(8, "levelB_binary_grouped_permutation_importance", "png", figure=True),
 "misclassified": out(9, "independent_sobol_misclassified_hamiltonians", "csv"),
 "summary": out(10, "run_summary", "json"),
 "levelA_model": out(10, "best_levelA_model", "joblib", model=True),
 "levelB_model": out(10, "best_levelB_binary_model", "joblib", model=True),
 "levelB_multi_model": out(10, "best_levelB_multiclass_model", "joblib", model=True),
}

def atomic_write_csv(df: pd.DataFrame, path: Path) -> None:
 path.parent.mkdir(parents=True, exist_ok=True)
 temp = path.with_suffix(path.suffix + ".tmp")
 df.to_csv(temp, index=False, encoding="utf-8-sig")
 temp.replace(path)

def atomic_write_text(text: str, path: Path) -> None:
 path.parent.mkdir(parents=True, exist_ok=True)
 temp = path.with_suffix(path.suffix + ".tmp")
 temp.write_text(text, encoding="utf-8")
 temp.replace(path)

def file_sha256(path: Path) -> str:
 digest = hashlib.sha256()
 with path.open("rb") as handle:

```

---

```

 for chunk in iter(lambda: handle.read(1024 * 1024), b''):
 digest.update(chunk)
 return digest.hexdigest()

def write_run_metadata(config: Step03Config, outputs: Dict[str, Path]) -> None:
 payload = asdict(config)
 payload["output_dir"] = str(config.output_dir)
 payload.update({
 "code_version": CODE_VERSION,
 "step01_core_version": getattr(core, "CODE_VERSION", None),
 "step02_fingerprint_version": getattr(fp_core, "CODE_VERSION", None),
 "n_train": config.n_train,
 "n_external": config.n_external,
 "run_tag": _run_tag(config),
 })
 atomic_write_text(json.dumps(payload, indent=2, ensure_ascii=False), outputs["run_config"])

 rows = []
 for key, path in outputs.items():
 rows.append({"key": key, "path": str(path), "exists": int(path.exists())})
 atomic_write_csv(pd.DataFrame(rows), outputs["registry"])

=====
Step 1. Global Sobol generation
=====

def _sobol_rows(power: int, seed: int, source: str, normalize: bool) -> list[dict]:
 if power <= 0:
 return []
 sampler = qmc.Sobol(d=len(REDUCED7), scramble=True, seed=int(seed))
 unit = sampler.random_base2(m=int(power))
 raw_vectors = 2.0 * unit - 1.0
 norms = np.linalg.norm(raw_vectors, axis=1)
 if np.any(norms < 1.0e-12):
 raise RuntimeError("Sobol generated a near-zero parameter vector; change the seed.")
 vectors = raw_vectors / norms[:, None] if normalize else raw_vectors

 rows: list[dict] = []
 prefix = "train" if source == "train_sobol" else "external"
 for index, (raw_vec, vec, norm) in enumerate(zip(raw_vectors, vectors, norms)):
 reduced = {name: float(value) for name, value in zip(REDUCED7, vec)}
 raw8 = core.raw8_from_reduced7(reduced, e0=0.0)
 row = {
 "sample_id": f"sobol_{prefix}_{index:06d}",
 "sample_source": source,
 "sobol_index": int(index),
 "is_control": 0,
 "is_ml_eligible": 1,
 "pre_normalization_norm": float(norm),
 "post_normalization_norm": float(np.linalg.norm(vec)),
 **reduced,
 **raw8,
 }
 for name, value in zip(REDUCED7, raw_vec):
 row[f"pre_norm_{name}"] = float(value)
 rows.append(row)
 return rows

def generate_parameters(config: Step03Config, outputs: Dict[str, Path]) -> pd.DataFrame:
 path = outputs["parameters"]
 if path.exists():
 existing = pd.read_csv(path, low_memory=False)
 expected = config.n_train + config.n_external + (len(CONTROL_REDUCED7) if config.include_control_points
 ↪ else 0)
 if len(existing) == expected:

```

---

```

 return existing

 rows = _sobol_rows(
 config.train_sobol_power,
 config.train_seed,
 "train_sobol",
 config.normalize_sobol_vectors,
)
 rows.extend(_sobol_rows(
 config.external_sobol_power,
 config.external_seed,
 "external_sobol",
 config.normalize_sobol_vectors,
))

 if config.include_control_points:
 for sample_id, reduced in CONTROL_REDUCED7.items():
 raw8 = core.raw8_from_reduced7(reduced, e0=0.0)
 rows.append({
 "sample_id": sample_id,
 "sample_source": "control",
 "sobol_index": -1,
 "is_control": 1,
 "is_ml_eligible": 0,
 "pre_normalization_norm": float(np.linalg.norm([reduced[k] for k in REDUCED7])),
 "post_normalization_norm": float(np.linalg.norm([reduced[k] for k in REDUCED7])),
 **reduced,
 **raw8,
 })

 df = pd.DataFrame(rows)
 if df["sample_id"].duplicated().any():
 raise RuntimeError("Duplicate sample_id generated.")
 atomic_write_csv(df, path)
 return df

=====
Step 2. Physics labels with strict nonzero-Chern verification
=====

def _integer_if_close(value: float, tol: float) -> int | None:
 if not np.isfinite(value):
 return None
 nearest = int(np rint(value))
 return nearest if abs(float(value) - nearest) <= tol else None

def one_chern_attempt(
 sample_id: str,
 params: Dict[str, float],
 nk: int,
 shift: tuple[float, float],
 stage: str,
 config: Step03Config,
) -> dict:
 row: dict[str, Any] = {
 "sample_id": sample_id,
 "stage": stage,
 "chern_nk": int(nk),
 "shift_x": float(shift[0]),
 "shift_y": float(shift[1]),
 "attempt_ok": 0,
 "attempt_reliable": 0,
 "error": "",
 }
 try:
 calc = core.calculate_spin_cherns(params, nk=int(nk), shift=shift)

```

---

```

 row.update(calc)
 cu = _integer_if_close(float(calc["chern_up"]), config.chern_integer_tol)
 cd = _integer_if_close(float(calc["chern_down"]), config.chern_integer_tol)
 ct = _integer_if_close(float(calc["chern_total"]), config.chern_integer_tol)
 reliable = bool(
 cu is not None and cd is not None and ct is not None
 and float(calc["min_det_up"]) > config.min_link_tol
 and float(calc["min_det_down"]) > config.min_link_tol
 and float(calc["min_det_total"]) > config.min_link_tol
 and float(calc["sum_rule_error"]) <= config.sum_rule_tol
)
 row.update({
 "chern_up_int": np.nan if cu is None else int(cu),
 "chern_down_int": np.nan if cd is None else int(cd),
 "chern_total_int": np.nan if ct is None else int(ct),
 "attempt_ok": 1,
 "attempt_reliable": int(reliable),
 })
 except Exception as exc: # numerical failures are retained as data
 row["error"] = repr(exc)
 return row

def _exact_consensus(attempts: Sequence[dict]) -> dict:
 total = len(attempts)
 reliable = [a for a in attempts if int(a.get("attempt_reliable", 0)) == 1]
 keys: list[tuple[int, int, int]] = []
 for a in reliable:
 keys.append((
 int(a["chern_up_int"]),
 int(a["chern_down_int"]),
 int(a["chern_total_int"]),
))
 unique = sorted(set(keys))
 exact = bool(total > 0 and len(reliable) == total and len(unique) == 1)
 if exact:
 cu, cd, ct = unique[0]
 else:
 cu = cd = ct = None
 return {
 "attempt_count": int(total),
 "reliable_count": int(len(reliable)),
 "n_unique_reliable_sectors": int(len(unique)),
 "exact_consensus": int(exact),
 "consensus_chern_up": cu,
 "consensus_chern_down": cd,
 "consensus_chern_total": ct,
 }

def _classify_from_gap_and_chern(
 gap: dict,
 consensus: dict,
 config: Step03Config,
 verification_level: str,
) -> dict:
 row: dict[str, Any] = {}
 row.update({
 "verification_level": verification_level,
 "chern_exact_consensus": int(consensus["exact_consensus"]),
 "chern_attempt_count": int(consensus["attempt_count"]),
 "chern_reliable_count": int(consensus["reliable_count"]),
 "chern_unique_sector_count": int(consensus["n_unique_reliable_sectors"]),
 })
 if not consensus["exact_consensus"]:
 row.update({
 "phase_label": "boundary_or_chern_unreliable",
 "chern_up_int": np.nan,
 "chern_down_int": np.nan,

```

---

```

 "chern_total_int": np.nan,
 "is_strict_insulator": 0,
 "is_spin_chern_topological": 0,
 "is_spin_chern_TI_candidate": 0,
 })
 return row

cu = int(consensus["consensus_chern_up"])
cd = int(consensus["consensus_chern_down"])
ct = int(consensus["consensus_chern_total"])
topological = bool(cu == -cd and abs(cu) >= 1 and ct == 0)
direct_gapped = bool(float(gap["min_direct_gap"]) > config.gap_tol)
indirect_gapped = bool(float(gap["indirect_gap"]) > config.gap_tol)
balanced = bool(
 int(gap["spin_occupancy_mismatch_count"]) == 0
 and float(gap["min_balanced_sector_gap"]) > config.gap_tol
)

if not direct_gapped:
 label = "noninsulating_or_gap_closing"
elif not balanced:
 label = "spin_sector_filling_mismatch"
elif topological and indirect_gapped:
 label = "spin_chern_TI_candidate"
elif topological:
 label = "spin_chern_band_metal"
elif cu == 0 and cd == 0 and ct == 0 and indirect_gapped:
 label = "trivial_insulator"
elif cu == 0 and cd == 0 and ct == 0:
 label = "indirect_overlap"
else:
 label = "other_gapped_chern_sector" if indirect_gapped else "other_overlapping_chern_sector"

row.update({
 "phase_label": label,
 "chern_up_int": cu,
 "chern_down_int": cd,
 "chern_total_int": ct,
 "spin_chern_int": int((cu - cd) // 2) if (cu - cd) % 2 == 0 else 0.5 * (cu - cd),
 "is_strict_insulator": int(label in {"spin_chern_TI_candidate", "trivial_insulator"}),
 "is_spin_chern_topological": int(topological),
 "is_spin_chern_TI_candidate": int(label == "spin_chern_TI_candidate"),
})
return row

def evaluate_sample_physics(
 row: pd.Series,
 config: Step03Config,
) -> tuple[dict, list[dict], list[dict]]:
 sample_id = str(row["sample_id"])
 reduced = {name: float(row[name]) for name in REDUCED7}
 params = core.raw8_from_reduced7(reduced, e0=0.0)

 result: dict[str, Any] = {
 "sample_id": sample_id,
 "sample_source": str(row["sample_source"]),
 "is_control": int(row["is_control"]),
 "is_ml_eligible": int(row["is_ml_eligible"]),
 **reduced,
 **params,
 **core.derived_features(params),
 "physics_error": "",
 }
 chern_rows: list[dict] = []
 strict_gap_rows: list[dict] = []

 try:
 coarse_gap = core.scan_band_gaps(

```



---

```

 params,
 nk=int(config.gap_nk),
 shift=(0.0, 0.0),
 gap_tol=config.gap_tol,
)
result.update(coarse_gap)
result["coarse_min_direct_gap"] = float(coarse_gap["min_direct_gap"])
result["coarse_indirect_gap"] = float(coarse_gap["indirect_gap"])

if float(coarse_gap["min_direct_gap"]) <= config.direct_gap_skip_chern:
 result.update({
 "phase_label": "noninsulating_or_gap_closing",
 "verification_level": "gap_only",
 "chern_exact_consensus": 0,
 "chern_attempt_count": 0,
 "chern_reliable_count": 0,
 "chern_unique_sector_count": 0,
 "chern_up_int": np.nan,
 "chern_down_int": np.nan,
 "chern_total_int": np.nan,
 "is_strict_insulator": 0,
 "is_spin_chern_topological": 0,
 "is_spin_chern_TI_candidate": 0,
 })
 return result, chern_rows, strict_gap_rows

if (
 int(coarse_gap["spin_occupancy_mismatch_count"]) > 0
 or float(coarse_gap["min_balanced_sector_gap"]) <= config.gap_tol
):
 result.update({
 "phase_label": "spin_sector_filling_mismatch",
 "verification_level": "gap_and_filling",
 "chern_exact_consensus": 0,
 "chern_attempt_count": 0,
 "chern_reliable_count": 0,
 "chern_unique_sector_count": 0,
 "chern_up_int": np.nan,
 "chern_down_int": np.nan,
 "chern_total_int": np.nan,
 "is_strict_insulator": 0,
 "is_spin_chern_topological": 0,
 "is_spin_chern_TI_candidate": 0,
 })
 return result, chern_rows, strict_gap_rows

initial_attempts: list[dict] = []
for nk in config.initial_chern_grids:
 for shift in config.initial_chern_shifts:
 attempt = one_chern_attempt(sample_id, params, int(nk), shift, "initial", config)
 initial_attempts.append(attempt)
 chern_rows.append(attempt)
initial_consensus = _exact_consensus(initial_attempts)

initial_nonzero = bool(
 initial_consensus["exact_consensus"]
 and int(initial_consensus["consensus_chern_up"]) != 0
)
needs_strict = bool(
 int(row["is_control"]) == 1
 or not initial_consensus["exact_consensus"]
 or initial_nonzero
)

if not needs_strict:
 result.update(_classify_from_gap_and_chern(
 coarse_gap,
 initial_consensus,
 config,

```

---

```

 verification_level="initial_all_grid_consensus",
))
 return result, chern_rows, strict_gap_rows

Strict gap audit for all nonzero/ambiguous/control samples.
strict_gap_results: list[dict] = []
for nk in config.strict_gap_grids:
 for shift in config.strict_gap_shifts:
 g = core.scan_band_gaps(params, nk=int(nk), shift=shift, gap_tol=config.gap_tol)
 g = {"sample_id": sample_id, "stage": "strict", **g}
 strict_gap_results.append(g)
 strict_gap_rows.append(g)

strict_gap = dict(coarse_gap)
strict_gap["min_direct_gap"] = float(min(g["min_direct_gap"] for g in strict_gap_results))
strict_gap["indirect_gap"] = float(min(g["indirect_gap"] for g in strict_gap_results))
strict_gap["min_balanced_sector_gap"] = float(min(g["min_balanced_sector_gap"] for g in
↪ strict_gap_results))
strict_gap["spin_occupancy_mismatch_count"] = int(max(g["spin_occupancy_mismatch_count"] for g in
↪ strict_gap_results))
strict_gap["min_spin_gap_up"] = float(min(g["min_spin_gap_up"] for g in strict_gap_results))
strict_gap["min_spin_gap_down"] = float(min(g["min_spin_gap_down"] for g in strict_gap_results))
result["min_direct_gap"] = strict_gap["min_direct_gap"]
result["indirect_gap"] = strict_gap["indirect_gap"]
result["min_balanced_sector_gap"] = strict_gap["min_balanced_sector_gap"]
result["spin_occupancy_mismatch_count"] = strict_gap["spin_occupancy_mismatch_count"]
result["strict_gap_verified"] = 1
result["strict_gap_check_count"] = len(strict_gap_results)

strict_attempts: list[dict] = []
for nk in config.strict_chern_grids:
 for shift in config.strict_chern_shifts:
 attempt = one_chern_attempt(sample_id, params, int(nk), shift, "strict", config)
 strict_attempts.append(attempt)
 chern_rows.append(attempt)
strict_consensus = _exact_consensus(strict_attempts)
result.update(_classify_from_gap_and_chern(
 strict_gap,
 strict_consensus,
 config,
 verification_level="strict_all_grid_consensus",
))
result["strict_chern_verified"] = int(strict_consensus["exact_consensus"])
return result, chern_rows, strict_gap_rows

except Exception as exc:
 result.update({
 "phase_label": "true_numeric_error",
 "physics_error": repr(exc),
 "verification_level": "error",
 "is_strict_insulator": 0,
 "is_spin_chern_topological": 0,
 "is_spin_chern_TI_candidate": 0,
 })
 return result, chern_rows, strict_gap_rows

def run_physics_labels(
 config: Step03Config,
 outputs: Dict[str, Path],
 parameters: pd.DataFrame,
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
 final_path = outputs["physics_final"]
 checkpoint_path = outputs["physics_checkpoint"]
 chern_path = outputs["chern_attempts"]
 strict_gap_path = outputs["strict_gap_checks"]

 if final_path.exists() and not config.force_recalculate_physics:

```

---

```

 final = pd.read_csv(final_path, low_memory=False)
 if set(final["sample_id"]) == set(parameters["sample_id"]):
 attempts = pd.read_csv(chern_path, low_memory=False) if chern_path.exists() else pd.DataFrame()
 strict_gaps = pd.read_csv(strict_gap_path, low_memory=False) if strict_gap_path.exists() else
 ↪ pd.DataFrame()
 return final, attempts, strict_gaps

if checkpoint_path.exists() and not config.force_recalculate_physics:
 done_df = pd.read_csv(checkpoint_path, low_memory=False)
 done_ids = set(done_df["sample_id"].astype(str))
 result_rows = done_df.to_dict("records")
else:
 done_ids = set()
 result_rows = []

if chern_path.exists() and not config.force_recalculate_physics:
 chern_rows = pd.read_csv(chern_path, low_memory=False).to_dict("records")
else:
 chern_rows = []
if strict_gap_path.exists() and not config.force_recalculate_physics:
 strict_gap_rows = pd.read_csv(strict_gap_path, low_memory=False).to_dict("records")
else:
 strict_gap_rows = []

total = len(parameters)
started = time.time()
processed_now = 0
for _, row in parameters.iterrows():
 sample_id = str(row["sample_id"])
 if sample_id in done_ids:
 continue
 result, attempts, gap_checks = evaluate_sample_physics(row, config)
 result_rows.append(result)
 chern_rows.extend(attempts)
 strict_gap_rows.extend(gap_checks)
 processed_now += 1

 if processed_now % config.checkpoint_every == 0:
 atomic_write_csv(pd.DataFrame(result_rows), checkpoint_path)
 atomic_write_csv(pd.DataFrame(chern_rows), chern_path)
 atomic_write_csv(pd.DataFrame(strict_gap_rows), strict_gap_path)
 print(
 f"Physics: {len(result_rows)}/{total} | "
 f"elapsed {time.time() - started:.1f} s"
)

physics_df = pd.DataFrame(result_rows)
Restore deterministic parameter order.
order = {sid: i for i, sid in enumerate(parameters["sample_id"].astype(str))}
physics_df["__order"] = physics_df["sample_id"].map(order)
physics_df = physics_df.sort_values("__order").drop(columns="__order").reset_index(drop=True)
atomic_write_csv(physics_df, final_path)
atomic_write_csv(physics_df, checkpoint_path)
atomic_write_csv(pd.DataFrame(chern_rows), chern_path)
atomic_write_csv(pd.DataFrame(strict_gap_rows), strict_gap_path)
return physics_df, pd.DataFrame(chern_rows), pd.DataFrame(strict_gap_rows)

```

```

=====
Step 3. Phase and Chern-sector summaries
=====

```

```

def summarize_phases(
 config: Step03Config,
 outputs: Dict[str, Path],
 physics: pd.DataFrame,
) -> dict[str, pd.DataFrame]:
 global_df = physics[physics["is_control"] == 0].copy()

```

---

```

phase_counts = (
 global_df["phase_label"].value_counts(dropna=False)
 .rename_axis("phase_label").reset_index(name="count")
)
phase_counts["fraction"] = phase_counts["count"] / max(len(global_df), 1)
atomic_write_csv(phase_counts, outputs["phase_counts"])

by_source = (
 global_df.groupby(["sample_source", "phase_label"], dropna=False)
 .size().reset_index(name="count")
)
by_source["source_total"] = by_source.groupby("sample_source")["count"].transform("sum")
by_source["fraction_within_source"] = by_source["count"] / by_source["source_total"]
atomic_write_csv(by_source, outputs["phase_counts_source"])

strict_ins = global_df[global_df["is_strict_insulator"] == 1].copy()
sector_counts = (
 strict_ins["chern_up_int"].value_counts(dropna=False).sort_index()
 .rename_axis("chern_up_int").reset_index(name="count")
)
if len(sector_counts):
 sector_counts["fraction_of_strict_insulators"] = sector_counts["count"] / len(strict_ins)
atomic_write_csv(sector_counts, outputs["chern_sector_counts"])

topo = global_df[global_df["phase_label"] == "spin_chern_TI_candidate"].copy()
topo = topo.sort_values(["chern_up_int", "indirect_gap"], ascending=[True, False])
atomic_write_csv(topo, outputs["topological_candidates"])

if len(topo):
 best = (
 topo.sort_values("indirect_gap", ascending=False)
 .groupby("chern_up_int", as_index=False).head(1)
 .sort_values("chern_up_int")
)
else:
 best = pd.DataFrame(columns=topo.columns)
atomic_write_csv(best, outputs["best_by_sector"])

controls = physics[physics["is_control"] == 1].copy()
if len(controls):
 controls["expected_chern_up"] = controls["sample_id"].map(CONTROL_EXPECTED_CUP)
 controls["control_chern_match"] = (
 controls["chern_up_int"].fillna(9999).astype(float)
 == controls["expected_chern_up"].astype(float)
).astype(int)
atomic_write_csv(controls, outputs["control_audit"])

Default-color plots only.
fig, ax = plt.subplots(figsize=(9, 5))
if len(phase_counts):
 ax.bar(phase_counts["phase_label"].astype(str), phase_counts["count"])
 ax.tick_params(axis="x", rotation=35)
ax.set_ylabel("Count")
ax.set_title("TTS Step 03 global Sobol phase counts")
fig.tight_layout()
fig.savefig(outputs["phase_counts_fig"], dpi=220, bbox_inches="tight")
plt.close(fig)

fig, ax = plt.subplots(figsize=(7, 4.5))
if len(sector_counts):
 ax.bar(sector_counts["chern_up_int"].astype(str), sector_counts["count"])
ax.set_xlabel("C_up")
ax.set_ylabel("Strict-insulator count")
ax.set_title("Spin-Chern sectors among strict insulators")
fig.tight_layout()
fig.savefig(outputs["chern_sector_fig"], dpi=220, bbox_inches="tight")
plt.close(fig)

return {

```

```

 "phase_counts": phase_counts,
 "phase_counts_by_source": by_source,
 "chern_sector_counts": sector_counts,
 "topological_candidates": topo,
 "best_by_sector": best,
 "control_audit": controls,
 }

=====
Step 4. Features: symmetry combinations + Step 02 v3 fingerprint
=====

def symmetry_parameter_features(parameters: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
 rows: list[dict] = []
 meta: list[dict] = []
 root2 = math.sqrt(2.0)
 for _, row in parameters.iterrows():
 p = {name: float(row[name]) for name in REDUCED7}
 me, t1, t2, r1, r2, r3, r4 = [p[k] for k in REDUCED7]
 values = {
 "sample_id": str(row["sample_id"]),
 **p,
 "abs_m_e": abs(me),
 "abs_t1": abs(t1),
 "abs_t2": abs(t2),
 "t_sum": t1 + t2,
 "t_diff": t1 - t2,
 "t_product": t1 * t2,
 "t_norm": math.sqrt(t1*t1 + t2*t2),
 "t_s": (t1 + t2) / root2,
 "t_d": (t1 - t2) / root2,
 "r13_sum": r1 + r3,
 "r13_diff": r1 - r3,
 "r24_sum": r2 + r4,
 "r24_diff": r2 - r4,
 "r12_sum": r1 + r2,
 "r34_sum": r3 + r4,
 "r_all_sum": r1 + r2 + r3 + r4,
 "r_all_norm": math.sqrt(r1*r1 + r2*r2 + r3*r3 + r4*r4),
 "r13_product": r1 * r3,
 "r24_product": r2 * r4,
 "r_cross_product": (r1 + r3) * (r2 + r4),
 "r_anisotropy_norm": math.sqrt((r1-r3)**2 + (r2-r4)**2),
 "r_pair_sum_mismatch": (r1 + r3) - (r2 + r4),
 "r_pair_diff_mismatch": (r1 - r3) - (r2 - r4),
 "m_e_t_sum": me * (t1 + t2),
 "m_e_t_diff": me * (t1 - t2),
 "m_e_r13_sum": me * (r1 + r3),
 "m_e_r24_sum": me * (r2 + r4),
 "t_product_r_cross": (t1*t2) * ((r1+r3)*(r2+r4)),
 }
 rows.append(values)

 df = pd.DataFrame(rows)
 for feature in df.columns:
 if feature == "sample_id":
 continue
 family = "raw_parameters" if feature in REDUCED7 else "symmetry_parameter_combinations"
 meta.append({
 "feature": feature,
 "feature_family": family,
 "k_region": "parameter_space",
 "spin": "none",
 "channel": feature,
 })
 return df, pd.DataFrame(meta)

```

---

```

def run_fingerprint(
 config: Step03Config,
 outputs: Dict[str, Path],
 parameters: pd.DataFrame,
) -> tuple[pd.DataFrame, pd.DataFrame]:
 fp_path = outputs["fingerprint"]
 meta_path = outputs["fingerprint_metadata"]
 if fp_path.exists() and meta_path.exists() and not config.force_recalculate_fingerprint:
 fp_df = pd.read_csv(fp_path, low_memory=False)
 if set(fp_df["sample_id"]) == set(parameters["sample_id"]):
 return fp_df, pd.read_csv(meta_path, low_memory=False)

 if fp_path.exists() and not config.force_recalculate_fingerprint:
 done = pd.read_csv(fp_path, low_memory=False)
 done_ids = set(done["sample_id"].astype(str))
 fp_rows = done.to_dict("records")
 else:
 done_ids = set()
 fp_rows = []

 meta_rows: list[dict] = []
 total = len(parameters)
 started = time.time()
 processed = 0
 for _, row in parameters.iterrows():
 sample_id = str(row["sample_id"])
 if sample_id in done_ids:
 continue
 reduced = {name: float(row[name]) for name in REDUCED7}
 params = core.raw8_from_reduced7(reduced, e0=0.0)
 feat, meta = fp_core.extract_fingerprint(sample_id, params, config.fingerprint_path_n)
 fp_rows.append(feat)
 meta_rows.extend(meta)
 processed += 1
 if processed % config.checkpoint_every == 0:
 atomic_write_csv(pd.DataFrame(fp_rows), fp_path)
 print(f"Fingerprint: {len(fp_rows)}/{total} | elapsed {time.time()-started:.1f} s")

 fp_df = pd.DataFrame(fp_rows)
 order = {sid: i for i, sid in enumerate(parameters["sample_id"].astype(str))}
 fp_df["__order"] = fp_df["sample_id"].map(order)
 fp_df = fp_df.sort_values("__order").drop(columns="__order").reset_index(drop=True)

 # If resuming, regenerate metadata from one sample to guarantee a complete table.
 if not meta_rows and len(parameters):
 first = parameters.iloc[0]
 reduced = {name: float(first[name]) for name in REDUCED7}
 _, meta_rows = fp_core.extract_fingerprint(
 str(first["sample_id"]),
 core.raw8_from_reduced7(reduced, e0=0.0),
 config.fingerprint_path_n,
)
 meta_df = pd.DataFrame(meta_rows).drop_duplicates("feature", keep="first")
 atomic_write_csv(fp_df, fp_path)
 atomic_write_csv(meta_df, meta_path)
 return fp_df, meta_df

def build_feature_table(
 outputs: Dict[str, Path],
 parameters: pd.DataFrame,
 physics: pd.DataFrame,
 fingerprint: pd.DataFrame,
 fingerprint_meta: pd.DataFrame,
) -> tuple[pd.DataFrame, pd.DataFrame]:
 analytic, analytic_meta = symmetry_parameter_features(parameters)
 atomic_write_csv(analytic, outputs["analytic_features"])
 feature_meta = pd.concat([analytic_meta, fingerprint_meta], ignore_index=True)

```

---

```

feature_meta = feature_meta.drop_duplicates("feature", keep="first")

labels = physics[[
 "sample_id", "sample_source", "is_control", "is_ml_eligible",
 "phase_label", "is_strict_insulator", "is_spin_chern_topological",
 "is_spin_chern_TI_candidate", "chern_up_int", "chern_down_int",
 "chern_total_int", "min_direct_gap", "indirect_gap",
 "min_balanced_sector_gap", "spin_occupancy_mismatch_count",
 "verification_level",
]].copy()
combined = labels.merge(analytic, on="sample_id", how="inner").merge(
 fingerprint, on="sample_id", how="inner"
)
atomic_write_csv(combined, outputs["combined_features"])
return combined, feature_meta

=====
Step 5-8. Hierarchical ML with cluster holdout and independent Sobol test
=====

def _metric_binary(y_true: Sequence[int], y_pred: Sequence[int]) -> dict:
 y_true = np.asarray(y_true, dtype=int)
 y_pred = np.asarray(y_pred, dtype=int)
 both = len(np.unique(y_true)) >= 2
 return {
 "n_test": int(len(y_true)),
 "accuracy": float(accuracy_score(y_true, y_pred)) if len(y_true) else np.nan,
 "balanced_accuracy": float(balanced_accuracy_score(y_true, y_pred)) if both else np.nan,
 "precision": float(precision_score(y_true, y_pred, zero_division=0)) if len(y_true) else np.nan,
 "recall": float(recall_score(y_true, y_pred, zero_division=0)) if len(y_true) else np.nan,
 "f1": float(f1_score(y_true, y_pred, zero_division=0)) if len(y_true) else np.nan,
 "n_positive_true": int(np.sum(y_true == 1)),
 "n_positive_pred": int(np.sum(y_pred == 1)),
 }

def _metric_multiclass(y_true: Sequence[str], y_pred: Sequence[str]) -> dict:
 y_true = np.asarray(y_true)
 y_pred = np.asarray(y_pred)
 return {
 "n_test": int(len(y_true)),
 "accuracy": float(accuracy_score(y_true, y_pred)) if len(y_true) else np.nan,
 "balanced_accuracy": float(balanced_accuracy_score(y_true, y_pred)) if len(np.unique(y_true)) >= 2 else
 ↪ np.nan,
 "f1_macro": float(f1_score(y_true, y_pred, average="macro", zero_division=0)) if len(y_true) else np.nan,
 "n_classes_true": int(len(np.unique(y_true))),
 "n_classes_pred": int(len(np.unique(y_pred))),
 }

def _make_model(model_name: str, config: Step03Config, multiclass: bool = False):
 if model_name == "logistic":
 return Pipeline([
 ("scale", StandardScaler()),
 ("model", LogisticRegression(
 max_iter=5000,
 class_weight="balanced",
 solver="lbfgs",
 random_state=42,
)),
])
 if model_name == "random_forest":
 return RandomForestClassifier(
 n_estimators=int(config.random_forest_estimators),
 class_weight="balanced_subsample",
 random_state=42,
 n_jobs=int(config.ml_n_jobs),

```

---

```

 min_samples_leaf=2,
)
 raise ValueError(f"Unknown model: {model_name}")

def _feature_sets(combined: pd.DataFrame, metadata: pd.DataFrame) -> dict[str, list[str]]:
 numeric = set(combined.select_dtypes(include=[np.number]).columns)
 raw7 = [f for f in REDUCED7 if f in numeric]
 symmetry = metadata.loc[
 metadata["feature_family"].isin(["raw_parameters", "symmetry_parameter_combinations"]),
 "feature",
].tolist()
 projector = metadata.loc[metadata["feature_family"] == "path_projector", "feature"].tolist()
 spectrum = metadata.loc[metadata["feature_family"].isin([
 "fixed_k_spin_spectrum", "fixed_k_full_spectrum", "path_spectrum"
]), "feature"].tolist()
 fingerprint_all = metadata.loc[
 metadata["feature_family"].isin(["raw_parameters", "symmetry_parameter_combinations"]),
 "feature",
].tolist()

 sets = {
 "raw7": raw7,
 "symmetry_parameters": [f for f in symmetry if f in numeric],
 "spectrum_fingerprint": [f for f in spectrum if f in numeric],
 "projector_fingerprint": [f for f in projector if f in numeric],
 "full_fingerprint": [f for f in fingerprint_all if f in numeric],
 "combined_all": [f for f in dict.fromkeys(symmetry + fingerprint_all) if f in numeric],
 }
 return {name: cols for name, cols in sets.items() if len(cols) > 0}

def _build_datasets(
 config: Step03Config,
 outputs: Dict[str, Path],
 combined: pd.DataFrame,
) -> dict[str, pd.DataFrame]:
 train = combined[(combined["sample_source"] == "train_sobol") & (combined["is_ml_eligible"] == 1)].copy()
 external = combined[(combined["sample_source"] == "external_sobol") & (combined["is_ml_eligible"] ==
↵ 1)].copy()

 train["levelA_target"] = train["is_strict_insulator"].astype(int)
 external["levelA_target"] = external["is_strict_insulator"].astype(int)

 train_B = train[train["phase_label"].isin(["trivial_insulator", "spin_chern_TI_candidate"])]
 external_B = external[external["phase_label"].isin(["trivial_insulator", "spin_chern_TI_candidate"])]
 train_B["levelB_binary_target"] = (train_B["phase_label"] == "spin_chern_TI_candidate").astype(int)
 external_B["levelB_binary_target"] = (external_B["phase_label"] == "spin_chern_TI_candidate").astype(int)
 train_B["levelB_multiclass_target"] = train_B["chern_up_int"].round().astype(int).map(lambda x:
↵ f"C_up_{x:d}")
 external_B["levelB_multiclass_target"] = external_B["chern_up_int"].round().astype(int).map(lambda x:
↵ f"C_up_{x:d}")

 atomic_write_csv(train, outputs["levelA_dataset"])
 atomic_write_csv(train_B, outputs["levelB_binary_dataset"])
 atomic_write_csv(train_B, outputs["levelB_multiclass_dataset"])
 return {
 "train_A": train,
 "external_A": external,
 "train_B": train_B,
 "external_B": external_B,
 }

def _assign_parameter_clusters(
 config: Step03Config,
 outputs: Dict[str, Path],
 datasets: dict[str, pd.DataFrame],

```



---

```

) -> dict[str, pd.DataFrame]:
 train_A = datasets["train_A"].copy()
 n_clusters = min(config.n_parameter_clusters, max(2, len(train_A) // 20))
 n_clusters = min(n_clusters, max(2, len(train_A) - 1))
 km = KMeans(n_clusters=n_clusters, random_state=42, n_init=20)
 labels = km.fit_predict(train_A[REDUCED7].astype(float))
 train_A["parameter_cluster"] = labels

 # Carry the same train-sample cluster labels to Level B.
 mapping = train_A.set_index("sample_id")["parameter_cluster"]
 train_B = datasets["train_B"].copy()
 train_B["parameter_cluster"] = train_B["sample_id"].map(mapping).astype(int)

 assignments = train_A[["sample_id", *REDUCED7, "parameter_cluster"]].copy()
 atomic_write_csv(assignments, outputs["cluster_assignments"])
 datasets = dict(datasets)
 datasets["train_A"] = train_A
 datasets["train_B"] = train_B
 return datasets

def _valid_binary_dataset(df: pd.DataFrame, target: str, min_count: int) -> bool:
 counts = df[target].value_counts()
 return len(counts) == 2 and int(counts.min()) >= min_count

def _valid_multiclass_dataset(df: pd.DataFrame, target: str, min_count: int) -> bool:
 counts = df[target].value_counts()
 return len(counts) >= 2 and int(counts.min()) >= min_count

def _random_cv(
 df: pd.DataFrame,
 target: str,
 task: str,
 feature_sets: dict[str, list[str]],
 config: Step03Config,
) -> pd.DataFrame:
 rows: list[dict] = []
 multiclass = task.endswith("multiclass")
 valid = _valid_multiclass_dataset(df, target, config.min_class_count_for_training) if multiclass else
 ↪ _valid_binary_dataset(df, target, config.min_class_count_for_training)
 if not valid:
 return pd.DataFrame([
 {
 "task": task,
 "status": "skipped_insufficient_class_counts",
 "class_counts": json.dumps(df[target].value_counts().to_dict(), ensure_ascii=False),
 }
])

 min_count = int(df[target].value_counts().min())
 n_splits = min(config.cv_splits, min_count)
 if n_splits < 2:
 return pd.DataFrame([{"task": task, "status": "skipped_cv_splits_lt_2"}])
 cv = RepeatedStratifiedKFold(
 n_splits=n_splits,
 n_repeats=config.cv_repeats,
 random_state=42,
)
 scoring = {
 "accuracy": "accuracy",
 "balanced_accuracy": "balanced_accuracy",
 "f1": "f1_macro" if multiclass else "f1",
 }
 for feature_name, features in feature_sets.items():
 X = df[features].astype(float)
 y = df[target]
 for model_name in ("logistic", "random_forest"):
 model = _make_model(model_name, config, multiclass=multiclass)
 scores = cross_validate(

```

---

```

 model,
 X,
 y,
 cv=cv,
 scoring=scoring,
 n_jobs=int(config.ml_n_jobs),
 error_score=np.nan,
)
 rows.append({
 "task": task,
 "status": "ok",
 "feature_set": feature_name,
 "model": model_name,
 "n_samples": len(df),
 "n_features": len(features),
 "accuracy_mean": float(np.nanmean(scores["test_accuracy"])),
 "accuracy_std": float(np.nanstd(scores["test_accuracy"])),
 "balanced_accuracy_mean": float(np.nanmean(scores["test_balanced_accuracy"])),
 "balanced_accuracy_std": float(np.nanstd(scores["test_balanced_accuracy"])),
 "f1_mean": float(np.nanmean(scores["test_f1"])),
 "f1_std": float(np.nanstd(scores["test_f1"])),
 })
return pd.DataFrame(rows)

def _cluster_holdout(
 df: pd.DataFrame,
 target: str,
 task: str,
 feature_sets: dict[str, list[str]],
 config: Step03Config,
) -> tuple[pd.DataFrame, dict[tuple, dict]]:
 rows: list[dict] = []
 fitted: dict[tuple, dict] = {}
 multiclass = task.endswith("multiclass")
 valid = _valid_multiclass_dataset(df, target, config.min_class_count_for_training) if multiclass else
 ↪ _valid_binary_dataset(df, target, config.min_class_count_for_training)
 if not valid or "parameter_cluster" not in df.columns:
 return pd.DataFrame([
 {
 "task": task,
 "status": "skipped_insufficient_class_counts_or_clusters",
 "class_counts": json.dumps(df[target].value_counts().to_dict(), ensure_ascii=False),
 }
]), fitted

 splitter = GroupShuffleSplit(
 n_splits=config.group_holdout_splits,
 test_size=config.group_holdout_fraction,
 random_state=42,
)
 groups = df["parameter_cluster"].to_numpy()
 y_all = df[target]

 for split_index, (train_idx, test_idx) in enumerate(splitter.split(df, y_all, groups)):
 y_train = y_all.iloc[train_idx]
 y_test = y_all.iloc[test_idx]
 if multiclass:
 # Every class in test must be known to the training set; otherwise the split is not scoreable.
 if not set(y_test.unique()).issubset(set(y_train.unique())) or len(y_test.unique()) < 2:
 continue
 else:
 if len(np.unique(y_train)) < 2 or len(np.unique(y_test)) < 2:
 continue

 for feature_name, features in feature_sets.items():
 X_train = df.iloc[train_idx][features].astype(float)
 X_test = df.iloc[test_idx][features].astype(float)
 for model_name in ("logistic", "random_forest"):
 model = _make_model(model_name, config, multiclass=multiclass)
 model.fit(X_train, y_train)

```

---

```

 pred = model.predict(X_test)
 metrics = _metric_multiclass(y_test, pred) if multiclass else _metric_binary(y_test, pred)
 record = {
 "task": task,
 "status": "ok",
 "split_index": int(split_index),
 "feature_set": feature_name,
 "model": model_name,
 "n_train": len(train_idx),
 "n_test": len(test_idx),
 "train_clusters": json.dumps(sorted(set(groups[train_idx].tolist()))),
 "test_clusters": json.dumps(sorted(set(groups[test_idx].tolist()))),
 **metrics,
 }
 rows.append(record)
 key = (task, feature_name, model_name, split_index)
 fitted[key] = {
 "model": model,
 "features": features,
 "train_idx": train_idx,
 "test_idx": test_idx,
 "y_test": y_test.copy(),
 "pred": pred,
 }
 if not rows:
 return pd.DataFrame([{"task": task, "status": "no_valid_group_split"}]), fitted
 return pd.DataFrame(rows), fitted

def _select_best_from_cluster(cluster_metrics: pd.DataFrame, task: str) -> dict | None:
 part = cluster_metrics[(cluster_metrics.get("task") == task) & (cluster_metrics.get("status") == "ok")].copy()
 if not len(part):
 return None
 score_col = "balanced_accuracy"
 fl_col = "f1_macro" if "f1_macro" in part.columns and part["f1_macro"].notna().any() else "f1"
 grouped = (
 part.groupby(["feature_set", "model"], as_index=False)
 .agg(
 balanced_accuracy_mean=(score_col, "mean"),
 balanced_accuracy_std=(score_col, "std"),
 f1_mean=(fl_col, "mean"),
 valid_splits=("split_index", "nunique"),
)
 .sort_values(["balanced_accuracy_mean", "f1_mean", "valid_splits"], ascending=False)
)
 if not len(grouped):
 return None
 return grouped.iloc[0].to_dict()

def _external_evaluate(
 train_df: pd.DataFrame,
 external_df: pd.DataFrame,
 target: str,
 task: str,
 best_spec: dict | None,
 feature_sets: dict[str, list[str]],
 config: Step03Config,
) -> tuple[dict, Any | None, pd.DataFrame]:
 if best_spec is None:
 return ({
 "task": task,
 "status": "skipped_no_best_cluster_model",
 }, None, pd.DataFrame())
 feature_name = str(best_spec["feature_set"])
 model_name = str(best_spec["model"])
 features = feature_sets[feature_name]
 multiclass = task.endswith("multiclass")

 if len(external_df) == 0:
 return ({
 "task": task,
 "status": "skipped_no_external_samples",
 }, None, pd.DataFrame())
 if multiclass:

```

---

```

 if not set(external_df[target].unique()).issubset(set(train_df[target].unique())):
 unknown = sorted(set(external_df[target].unique()) - set(train_df[target].unique()))
 return ({
 "task": task,
 "status": "skipped_external_contains_unseen_classes",
 "unseen_classes": json.dumps(unknown),
 }, None, pd.DataFrame())
 else:
 if len(train_df[target].unique()) < 2:
 return ({"task": task, "status": "skipped_train_has_one_class"}, None, pd.DataFrame())

 model = _make_model(model_name, config, multiclass=multiclass)
 model.fit(train_df[features].astype(float), train_df[target])
 pred = model.predict(external_df[features].astype(float))
 metrics = _metric_multiclass(external_df[target], pred) if multiclass else _metric_binary(external_df[target],
↪ pred)
 record = {
 "task": task,
 "status": "ok",
 "feature_set": feature_name,
 "model": model_name,
 "n_train": len(train_df),
 **metrics,
 }
 pred_df = external_df[[
 "sample_id", "sample_source", "phase_label", "chern_up_int",
 "min_direct_gap", "indirect_gap", *REDUCED7,
]].copy()
 pred_df["task"] = task
 pred_df["target_true"] = external_df[target].astype(str).to_numpy()
 pred_df["target_pred"] = np.asarray(pred).astype(str)
 if hasattr(model, "predict_proba"):
 probabilities = model.predict_proba(external_df[features].astype(float))
 classes = [str(c) for c in model.classes_]
 for i, cls in enumerate(classes):
 pred_df[f"prob_{cls}"] = probabilities[:, i]
 return record, model, pred_df

def _plot_confusion_from_predictions(pred_df: pd.DataFrame, task: str, path: Path) -> None:
 subset = pred_df[pred_df["task"] == task]
 if not len(subset):
 return
 fig, ax = plt.subplots(figsize=(5.5, 5.0))
 ConfusionMatrixDisplay.from_predictions(
 subset["target_true"], subset["target_pred"], ax=ax, xticks_rotation=35
)
 ax.set_title(task)
 fig.tight_layout()
 fig.savefig(path, dpi=220, bbox_inches="tight")
 plt.close(fig)

def grouped_permutation_importance(
 model,
 X: pd.DataFrame,
 y: pd.Series,
 groups: dict[str, list[str]],
 config: Step03Config,
) -> pd.DataFrame:
 if len(X) == 0 or len(np.unique(y)) < 2:
 return pd.DataFrame()
 rng = np.random.default_rng(42)
 baseline = balanced_accuracy_score(y, model.predict(X))
 rows = []
 for group_name, group_features in groups.items():
 active = [f for f in group_features if f in X.columns]
 if not active:
 continue

```

---

```

 drops = []
 for _ in range(config.permutation_repeats):
 perm = rng.permutation(len(X))
 xp = X.copy()
 xp.loc[:, active] = X.iloc[perm][active].to_numpy()
 drops.append(baseline - balanced_accuracy_score(y, model.predict(xp)))
 rows.append({
 "group": group_name,
 "n_features": len(active),
 "importance_mean": float(np.mean(drops)),
 "importance_std": float(np.std(drops)),
 "baseline_balanced_accuracy": float(baseline),
 })
 return pd.DataFrame(rows).sort_values("importance_mean", ascending=False)

def run_machine_learning(
 config: Step03Config,
 outputs: Dict[str, Path],
 combined: pd.DataFrame,
 metadata: pd.DataFrame,
) -> Dict[str, Any]:
 datasets = _build_datasets(config, outputs, combined)
 datasets = _assign_parameter_clusters(config, outputs, datasets)
 feature_sets = _feature_sets(combined, metadata)

 tasks = [
 ("LevelA_insulator_gate", datasets["train_A"], datasets["external_A"], "levelA_target"),
 ("LevelB_binary_topology", datasets["train_B"], datasets["external_B"], "levelB_binary_target"),
 ("LevelB_multiclass_chern", datasets["train_B"], datasets["external_B"], "levelB_multiclass_target"),
]

 random_rows = []
 cluster_rows = []
 all_fitted: Dict[tuple, Dict] = {}
 best_registry_rows = []
 external_rows = []
 external_prediction_frames = []
 fitted_external_models: Dict[str, Any] = {}

 for task, train_df, external_df, target in tasks:
 random_df = _random_cv(train_df, target, task, feature_sets, config)
 random_rows.append(random_df)
 cluster_df, fitted = _cluster_holdout(train_df, target, task, feature_sets, config)
 cluster_rows.append(cluster_df)
 all_fitted.update(fitted)
 best = _select_best_from_cluster(cluster_df, task)
 if best is not None:
 best_registry_rows.append({"task": task, **best})
 ext_record, ext_model, ext_predictions = _external_evaluate(
 train_df, external_df, target, task, best, feature_sets, config
)
 external_rows.append(ext_record)
 if ext_model is not None:
 fitted_external_models[task] = ext_model
 if len(ext_predictions):
 external_prediction_frames.append(ext_predictions)

 random_metrics = pd.concat(random_rows, ignore_index=True, sort=False)
 cluster_metrics = pd.concat(cluster_rows, ignore_index=True, sort=False)
 external_metrics = pd.DataFrame(external_rows)
 best_registry = pd.DataFrame(best_registry_rows)
 external_predictions = (
 pd.concat(external_prediction_frames, ignore_index=True, sort=False)
 if external_prediction_frames else pd.DataFrame()
)

 atomic_write_csv(random_metrics, outputs["random_cv_metrics"])
 atomic_write_csv(cluster_metrics, outputs["cluster_holdout_metrics"])

```

---

```

atomic_write_csv(external_metrics, outputs["external_metrics"])
atomic_write_csv(best_registry, outputs["best_model_registry"])
atomic_write_csv(external_predictions, outputs["external_predictions"])

Save one representative best cluster split per task.
cluster_pred_frames = []
report_parts = []
for _, best in best_registry.iterrows():
 task = str(best["task"])
 matches = cluster_metrics[
 (cluster_metrics["task"] == task)
 & (cluster_metrics["feature_set"] == best["feature_set"])
 & (cluster_metrics["model"] == best["model"])
 & (cluster_metrics["status"] == "ok")
].sort_values("balanced_accuracy", ascending=False)
 if not len(matches):
 continue
 split_index = int(matches.iloc[0]["split_index"])
 key = (task, str(best["feature_set"]), str(best["model"]), split_index)
 fit = all_fitted.get(key)
 if fit is None:
 continue
 source_df = datasets["train_A"] if task == "LevelA_insulator_gate" else datasets["train_B"]
 test_rows = source_df.iloc[fit["test_idx"]].copy()
 out = test_rows[["sample_id", "phase_label", "chern_up_int", "parameter_cluster", *REDUCED7]].copy()
 out["task"] = task
 out["target_true"] = fit["y_test"].astype(str).to_numpy()
 out["target_pred"] = np.asarray(fit["pred"]).astype(str)
 cluster_pred_frames.append(out)
 report_parts.append(
 f"==== {task} / best cluster split =====\n"
 + classification_report(out["target_true"], out["target_pred"], zero_division=0)
)

cluster_predictions = (
 pd.concat(cluster_pred_frames, ignore_index=True, sort=False)
 if cluster_pred_frames else pd.DataFrame()
)
atomic_write_csv(cluster_predictions, outputs["cluster_predictions"])

for task in ("LevelA_insulator_gate", "LevelB_binary_topology", "LevelB_multiclass_chern"):
 sub = external_predictions[external_predictions["task"] == task] if len(external_predictions) else
 ↪ pd.DataFrame()
 if len(sub):
 report_parts.append(
 f"==== {task} / independent Sobol external =====\n"
 + classification_report(sub["target_true"], sub["target_pred"], zero_division=0)
)
atomic_write_text("\n\n".join(report_parts), outputs["classification_reports"])

if len(external_predictions):
 _plot_confusion_from_predictions(external_predictions, "LevelA_insulator_gate",
 ↪ outputs["levelA_confusion"])
 _plot_confusion_from_predictions(external_predictions, "LevelB_binary_topology",
 ↪ outputs["levelB_confusion"])
 _plot_confusion_from_predictions(external_predictions, "LevelB_multiclass_chern",
 ↪ outputs["levelB_multi_confusion"])

Save external-fitted models.
if "LevelA_insulator_gate" in fitted_external_models:
 dump(fitted_external_models["LevelA_insulator_gate"], outputs["levelA_model"])
if "LevelB_binary_topology" in fitted_external_models:
 dump(fitted_external_models["LevelB_binary_topology"], outputs["levelB_model"])
if "LevelB_multiclass_chern" in fitted_external_models:
 dump(fitted_external_models["LevelB_multiclass_chern"], outputs["levelB_multi_model"])

Level B binary importance, evaluated on the independent external Sobol subset.
importance_group = pd.DataFrame()

```

---

```

importance_individual = pd.DataFrame()
best_binary = best_registry[best_registry["task"] == "LevelB_binary_topology"] if len(best_registry) else
↳ pd.DataFrame()
model_binary = fitted_external_models.get("LevelB_binary_topology")
ext_B = datasets["external_B"]
if len(best_binary) and model_binary is not None and len(ext_B) and
↳ len(ext_B["levelB_binary_target"].unique()) >= 2:
 feature_name = str(best_binary.iloc[0]["feature_set"])
 features = feature_sets[feature_name]
 X_ext = ext_B[features].astype(float)
 y_ext = ext_B["levelB_binary_target"].astype(int)
 family_groups = {
 str(family): metadata.loc[metadata["feature_family"] == family, "feature"].tolist()
 for family in metadata["feature_family"].dropna().unique()
 }
 importance_group = grouped_permutation_importance(
 model_binary, X_ext, y_ext, family_groups, config
)
 atomic_write_csv(importance_group, outputs["group_importance"])

 perm = permutation_importance(
 model_binary,
 X_ext,
 y_ext,
 scoring="balanced_accuracy",
 n_repeats=config.permutation_repeats,
 random_state=42,
 n_jobs=config.ml_n_jobs,
)
 importance_individual = pd.DataFrame({
 "feature": features,
 "importance_mean": perm.importances_mean,
 "importance_std": perm.importances_std,
 }).sort_values("importance_mean", ascending=False)
 atomic_write_csv(importance_individual, outputs["individual_importance"])

 fig, ax = plt.subplots(figsize=(8, 5))
 plot_df = importance_group.sort_values("importance_mean")
 ax.barh(plot_df["group"], plot_df["importance_mean"], xerr=plot_df["importance_std"])
 ax.set_xlabel("Grouped permutation importance")
 ax.set_title("Level B binary importance on independent Sobol set")
 fig.tight_layout()
 fig.savefig(outputs["importance_fig"], dpi=220, bbox_inches="tight")
 plt.close(fig)
else:
 atomic_write_csv(importance_group, outputs["group_importance"])
 atomic_write_csv(importance_individual, outputs["individual_importance"])

Misclassified external Hamiltonians.
if len(external_predictions):
 misclassified = external_predictions[
 external_predictions["target_true"] != external_predictions["target_pred"]
].copy()
else:
 misclassified = pd.DataFrame()
atomic_write_csv(misclassified, outputs["misclassified"])

return {
 "datasets": datasets,
 "feature_sets": feature_sets,
 "random_metrics": random_metrics,
 "cluster_metrics": cluster_metrics,
 "external_metrics": external_metrics,
 "best_registry": best_registry,
 "external_predictions": external_predictions,
 "cluster_predictions": cluster_predictions,
 "group_importance": importance_group,
 "individual_importance": importance_individual,

```

---

```

 "misclassified": misclassified,
 }

=====
Step 10. Orchestration
=====

def run_step03(config: Step03Config | None = None) -> Dict[str, Any]:
 config = (config or Step03Config()).normalized()
 outputs = build_output_registry(config)
 write_run_metadata(config, outputs)

 print("[1/7] Generate global and independent Sobol parameters")
 parameters = generate_parameters(config, outputs)
 print(f" total rows = {len(parameters)}")

 print("[2/7] Full-BZ physics labels and strict Chern verification")
 physics, chern_attempts, strict_gap_checks = run_physics_labels(config, outputs, parameters)
 print(physics["phase_label"].value_counts(dropna=False).to_string())

 print("[3/7] Phase and exact Chern-sector summaries")
 phase_summary = summarize_phases(config, outputs, physics)

 print("[4/7] Hamiltonian fingerprint")
 fingerprint, fp_metadata = run_fingerprint(config, outputs, parameters)

 print("[5/7] Symmetry parameter combinations and combined feature table")
 combined, metadata = build_feature_table(
 outputs, parameters, physics, fingerprint, fp_metadata
)

 print("[6/7] Hierarchical ML: random CV, cluster holdout, independent Sobol")
 ml = run_machine_learning(config, outputs, combined, metadata)

 print("[7/7] Save run summary")
 global_physics = physics[physics["is_control"] == 0]
 controls = phase_summary["control_audit"]
 summary = {
 "code_version": CODE_VERSION,
 "run_tag": _run_tag(config),
 "n_train_sobol": config.n_train,
 "n_external_sobol": config.n_external,
 "n_control": int((physics["is_control"] == 1).sum()),
 "global_phase_counts": {
 str(k): int(v) for k, v in global_physics["phase_label"].value_counts().items()
 },
 "strict_insulator_chern_sector_counts": {
 str(k): int(v)
 for k, v in global_physics.loc[
 global_physics["is_strict_insulator"] == 1, "chern_up_int"
].value_counts().sort_index().items()
 },
 "n_strict_TI_candidates": int((global_physics["phase_label"] == "spin_chern_TI_candidate").sum()),
 "n_high_spin_chern_TI_candidates": int((
 (global_physics["phase_label"] == "spin_chern_TI_candidate")
 & (global_physics["chern_up_int"].abs() >= 2)
).sum()),
 "control_all_chern_match": int(
 len(controls) > 0 and controls.get("control_chern_match", pd.Series(dtype=int)).eq(1).all()
),
 "best_model_registry": ml["best_registry"].to_dict("records"),
 "external_metrics": ml["external_metrics"].to_dict("records"),
 "output_dir": str(config.output_dir),
 }
 atomic_write_text(json.dumps(summary, indent=2, ensure_ascii=False), outputs["summary"])
 write_run_metadata(config, outputs)
 print(json.dumps(summary, indent=2, ensure_ascii=False))

```



---

```

return {
 "config": config,
 "outputs": outputs,
 "parameters": parameters,
 "physics": physics,
 "chern_attempts": chern_attempts,
 "strict_gap_checks": strict_gap_checks,
 "phase_summary": phase_summary,
 "fingerprint": fingerprint,
 "combined_features": combined,
 "metadata": metadata,
 "ml": ml,
 "summary": summary,
}

=====
CLI
=====

def build_arg_parser() -> argparse.ArgumentParser:
 parser = argparse.ArgumentParser(description=__doc__)
 parser.add_argument("--output-dir", type=Path, default=Step03Config.output_dir)
 parser.add_argument("--train-power", type=int, default=Step03Config.train_sobol_power)
 parser.add_argument("--external-power", type=int, default=Step03Config.external_sobol_power)
 parser.add_argument("--train-seed", type=int, default=Step03Config.train_seed)
 parser.add_argument("--external-seed", type=int, default=Step03Config.external_seed)
 parser.add_argument("--gap-nk", type=int, default=Step03Config.gap_nk)
 parser.add_argument("--path-n", type=int, default=Step03Config.fingerprint_path_n)
 parser.add_argument("--ml-n-jobs", type=int, default=1)
 parser.add_argument("--checkpoint-every", type=int, default=Step03Config.checkpoint_every)
 parser.add_argument("--no-normalize", action="store_true")
 parser.add_argument("--no-controls", action="store_true")
 parser.add_argument("--force-physics", action="store_true")
 parser.add_argument("--force-fingerprint", action="store_true")
 parser.add_argument("--smoke-test", action="store_true")
 return parser

def config_from_args(args: argparse.Namespace) -> Step03Config:
 if args.smoke_test:
 return Step03Config(
 output_dir=args.output_dir,
 train_sobol_power=3, # 8
 external_sobol_power=2, # 4
 train_seed=args.train_seed,
 external_seed=args.external_seed,
 normalize_sobol_vectors=not args.no_normalize,
 include_control_points=not args.no_controls,
 gap_nk=17,
 initial_chern_grids=(11, 13),
 initial_chern_shifts=((0.0, 0.0), (0.5, 0.5)),
 strict_gap_grids=(17, 21),
 strict_gap_shifts=((0.0, 0.0), (0.5, 0.5)),
 strict_chern_grids=(13, 15),
 strict_chern_shifts=((0.0, 0.0), (0.5, 0.5)),
 fingerprint_path_n=7,
 n_parameter_clusters=3,
 group_holdout_splits=4,
 cv_splits=2,
 cv_repeats=1,
 random_forest_estimators=40,
 permutation_repeats=3,
 min_class_count_for_training=2,
 checkpoint_every=2,
 ml_n_jobs=1,
 force_recalculate_physics=args.force_physics,

```

```

 force_recalculate_fingerprint=args.force_fingerprint,
)
 return Step03Config(
 output_dir=args.output_dir,
 train_sobol_power=args.train_power,
 external_sobol_power=args.external_power,
 train_seed=args.train_seed,
 external_seed=args.external_seed,
 normalize_sobol_vectors=not args.no_normalize,
 include_control_points=not args.no_controls,
 gap_nk=args.gap_nk,
 fingerprint_path_n=args.path_n,
 ml_n_jobs=args.ml_n_jobs,
 checkpoint_every=args.checkpoint_every,
 force_recalculate_physics=args.force_physics,
 force_recalculate_fingerprint=args.force_fingerprint,
)

def main() -> int:
 parser = build_arg_parser()
 args = parser.parse_args()
 config = config_from_args(args)
 run_step03(config)
 return 0

if __name__ == "__main__":
 raise SystemExit(main())

```

## ► 代码清单 A.49.4 (python, 1294 行)

```

#!/usr/bin/env python3
-*- coding: utf-8 -*-
"""
TTS Step 04 — Chern 扇区定向扩充、拓扑—平庸边界扫描与闭隙谷追踪

研究目标

1. 从 TTS Step 03 的严格结果中分别选择 $C_{up} = -2, -1, +1, +2$
 (以及数据中实际存在的其他非零整数扇区) 的代表性拓扑锚点;
2. 围绕每个 Chern 扇区进行局部 Sobol 定向扩充, 确认拓扑区域不是孤立点,
 并为后续多分类和解析机制积累分扇区样本;
3. 为每个主锚点寻找参数空间中最近的严格平庸绝缘体;
4. 沿单位参数球面上的最短路径 (slerp) 扫描拓扑点—平庸点之间的相变;
5. 稀疏但严格地复核路径上的 Chern 数, 并自动识别 Chern 改变区间;
6. 对直接带隙最小处联合优化 (λ, k_x, k_y), 定位真正的闭隙谷;
7. 判断 $|C|=1$ 是否主要由 Γ/M 高对称谷驱动, $|C|=2$ 是否主要由
 $C4/D4$ 对称相关的普通动量谷共同驱动。

重要原则

- 复用冻结的 Step 01 Hamiltonian、周期规范和非阿贝尔 spin-Chern 标签器;
- 复用 Step 03 的严格多网格标签逻辑;
- 全部物理计算串行, 不使用 ProcessPoolExecutor, 兼容 Windows/Jupyter;
- 路径密集扫描只计算能隙, Chern 仅在自适应选中的点严格复核;
- 本步骤输出的是机制证据和相边界数据, 不把机器学习预测当成拓扑结论。
"""

from __future__ import annotations

from dataclasses import asdict, dataclass
from pathlib import Path

```

---

```

from typing import Any, Dict, Iterable, Sequence
import argparse
import io
import json
import math
import time
import warnings
import zipfile

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.optimize import minimize
from scipy.stats import qmc

try:
 import TTS_step01_model_and_label_audit_v2 as core
except ImportError as exc: # pragma: no cover
 raise ImportError(
 " 未找到 TTS_step01_model_and_label_audit_v2.py. \n"
 " 请把 Step 01 v2 的 py 文件与本脚本放在同一目录。"
) from exc

try:
 import TTS_step03_global_sobol_hierarchical_topology_ML as step3
except ImportError:
 try:
 import TTS_step03_global_sobol_hierarchical_topology_ML_final as step3
 except ImportError:
 try:
 import TTS_step03_global_sobol_hierarchical_topology_ML_fixed as step3
 except ImportError as exc: # pragma: no cover
 raise ImportError(
 " 未找到 Step 03 配套 py 文件。支持以下文件名: \n"
 " TTS_step03_global_sobol_hierarchical_topology_ML.py\n"
 " TTS_step03_global_sobol_hierarchical_topology_ML_final.py\n"
 " TTS_step03_global_sobol_hierarchical_topology_ML_fixed.py"
) from exc

EXPECTED_STEP01_VERSION = "TTS_STEP01_V2_20260713"
EXPECTED_STEP03_VERSION = "TTS_STEP03_V1_20260713"
if getattr(core, "CODE_VERSION", None) != EXPECTED_STEP01_VERSION:
 raise RuntimeError(
 f"Step 01 核心版本不一致: expected={EXPECTED_STEP01_VERSION}, "
 f"loaded={getattr(core, 'CODE_VERSION', None)}"
)
if getattr(step3, "CODE_VERSION", None) != EXPECTED_STEP03_VERSION:
 raise RuntimeError(
 f"Step 03 核心版本不一致: expected={EXPECTED_STEP03_VERSION}, "
 f"loaded={getattr(step3, 'CODE_VERSION', None)}"
)

np.set_printoptions(precision=10, suppress=True)

CODE_VERSION = "TTS_STEP04_V1_20260713"
WORKFLOW_STEP = "step04"
SYSTEM_TAG = "tts8_bns127391"
TASK_TAG = "chern_sector_boundary_valley_tracking"
REDUCED7 = core.REDUCED7.copy()

=====
Configuration
=====

@dataclass
class Step04Config:
 output_dir: Path = Path("outputs_tts_step04_chern_sector_boundary_valley_tracking")
 step3_input: Path | None = None

```

---

```

Representative anchors selected independently in each nonzero Chern sector.
target_sectors: tuple[int, ...] | None = (-2, -1, 1, 2)
n_anchors_per_sector: int = 2
anchor_candidate_pool: int = 24
path_anchors_per_sector: int = 1
trivial_partner_modes: tuple[str, ...] = ("nearest", "same_t_sign")

Local sector-directed Sobol augmentation on the normalized parameter sphere.
local_angular_radii: tuple[float, ...] = (0.05, 0.10)
local_sobol_power_per_radius: int = 4 # 16 samples/radius/anchor
local_seed: int = 20260714

Physics verification for local samples and sparse path Chern points.
local_gap_nk: int = 41
initial_chern_grids: tuple[int, ...] = (21, 31)
initial_chern_shifts: tuple[tuple[float, float], ...] = ((0.0, 0.0), (0.5, 0.5))
strict_gap_grids: tuple[int, ...] = (51, 71)
strict_gap_shifts: tuple[tuple[float, float], ...] = (
 (0.0, 0.0), (0.5, 0.0), (0.0, 0.5), (0.5, 0.5)
)
strict_chern_grids: tuple[int, ...] = (31, 41, 51)
strict_chern_shifts: tuple[tuple[float, float], ...] = ((0.0, 0.0), (0.5, 0.5))
gap_tol: float = 1.0e-3
direct_gap_skip_chern: float = 3.0e-3
chern_integer_tol: float = 0.08
min_link_tol: float = 1.0e-7
sum_rule_tol: float = 0.15

Dense path gap scan and adaptive sparse Chern audit.
path_n_lambda: int = 121
path_gap_nk: int = 35
path_chern_stride: int = 10
path_extra_gap_minima: int = 6
max_chern_points_per_path: int = 25
max_transitions_per_path: int = 4

Continuous critical valley refinement.
critical_k_symmetry_tol: float = 0.10
critical_optimizer_starts: int = 6
critical_optimizer_maxiter: int = 350

Workflow controls.
checkpoint_every: int = 8
force_recalculate_local: bool = False
force_recalculate_paths: bool = False
force_recalculate_chern: bool = False

```

```

def normalized(self) -> "Step04Config":
 self.output_dir = Path(self.output_dir)
 self.output_dir.mkdir(parents=True, exist_ok=True)
 (self.output_dir / "figures").mkdir(parents=True, exist_ok=True)
 if self.step3_input is not None:
 self.step3_input = Path(self.step3_input)
 if self.n_anchors_per_sector < 1:
 raise ValueError("n_anchors_per_sector must be >= 1")
 if self.path_anchors_per_sector < 1:
 raise ValueError("path_anchors_per_sector must be >= 1")
 if self.local_sobol_power_per_radius < 0:
 raise ValueError("local_sobol_power_per_radius must be >= 0")
 if self.path_n_lambda < 11:
 raise ValueError("path_n_lambda must be >= 11")
 if self.path_gap_nk < 9:
 raise ValueError("path_gap_nk must be >= 9")
 if self.checkpoint_every < 1:
 raise ValueError("checkpoint_every must be >= 1")
 return self

```

@property

---

```

def n_local_per_radius(self) -> int:
 return 2 ** int(self.local_sobol_power_per_radius)

=====
Safe I/O and Step 03 input discovery
=====

def atomic_write_csv(df: pd.DataFrame, path: Path) -> None:
 path.parent.mkdir(parents=True, exist_ok=True)
 tmp = path.with_suffix(path.suffix + ".tmp")
 df.to_csv(tmp, index=False, encoding="utf-8-sig")
 tmp.replace(path)

def atomic_write_json(obj: dict, path: Path) -> None:
 path.parent.mkdir(parents=True, exist_ok=True)
 tmp = path.with_suffix(path.suffix + ".tmp")
 tmp.write_text(json.dumps(obj, indent=2, ensure_ascii=False, default=str), encoding="utf-8")
 tmp.replace(path)

def _find_default_step3_input() -> Path:
 candidates = [
 Path.cwd() / "outputs_tts_step03_global_sobol_hierarchical_topology_ml.zip",
 Path.cwd().parent / "outputs_tts_step03_global_sobol_hierarchical_topology_ml.zip",
 Path("/mnt/data") / "outputs_tts_step03_global_sobol_hierarchical_topology_ml.zip",
 Path.cwd() / "outputs_tts_step03_global_sobol_hierarchical_topology_ml",
 Path.cwd().parent / "outputs_tts_step03_global_sobol_hierarchical_topology_ml",
 Path("/mnt/data") / "outputs_tts_step03_global_sobol_hierarchical_topology_ml",
]
 for candidate in candidates:
 if candidate.exists():
 return candidate
 raise FileNotFoundError(
 " 未找到 Step 03 结果。请把结果 ZIP 放在当前目录, 或在配置中设置 step3_input。"
)

def _read_unique_csv_from_zip(zip_path: Path, pattern: str) -> tuple[pd.DataFrame, str]:
 with zipfile.ZipFile(zip_path) as zf:
 matches = [
 name for name in zf.namelist()
 if pattern in Path(name).name and name.lower().endswith(".csv")
]
 if len(matches) != 1:
 raise FileNotFoundError(
 f"ZIP 中匹配 {pattern!r} 的 CSV 数量为 {len(matches)}: {matches}"
)
 member = matches[0]
 with zf.open(member) as fh:
 return pd.read_csv(fh, low_memory=False), f"{zip_path}/{member}"

def _read_unique_csv_from_directory(root: Path, pattern: str) -> tuple[pd.DataFrame, str]:
 matches = [p for p in root.rglob("*.csv") if pattern in p.name]
 if len(matches) != 1:
 raise FileNotFoundError(
 f"目录中匹配 {pattern!r} 的 CSV 数量为 {len(matches)}: {matches}"
)
 return pd.read_csv(matches[0], low_memory=False), str(matches[0])

def load_step3_physics(config: Step04Config) -> tuple[pd.DataFrame, str]:
 source = config.step3_input or _find_default_step3_input()
 source = Path(source)
 pattern = "step03_02_physics_labels_final_"
 if source.is_file() and source.suffix.lower() == ".zip":
 return _read_unique_csv_from_zip(source, pattern)

```

---

```

 if source.is_dir():
 return _read_unique_csv_from_directory(source, pattern)
 raise FileNotFoundError(f" 无效的 Step 03 输入: {source}")

=====
Parameter geometry on the normalized seven-dimensional sphere
=====

def row_vector(row: pd.Series | dict) -> np.ndarray:
 return np.array([float(row[name]) for name in REDUCED7], dtype=float)

def normalize_vector(vector: Sequence[float]) -> np.ndarray:
 v = np.asarray(vector, dtype=float)
 norm = float(np.linalg.norm(v))
 if not np.isfinite(norm) or norm < 1.0e-14:
 raise ValueError("Cannot normalize zero/nonfinite parameter vector")
 return v / norm

def vector_to_reduced(vector: Sequence[float]) -> dict[str, float]:
 v = normalize_vector(vector)
 return {name: float(value) for name, value in zip(REDUCED7, v)}

def angular_distance(a: Sequence[float], b: Sequence[float]) -> float:
 aa = normalize_vector(a)
 bb = normalize_vector(b)
 return float(np.arccos(np.clip(float(np.dot(aa, bb)), -1.0, 1.0)))

def slerp(a: Sequence[float], b: Sequence[float], lam: float) -> np.ndarray:
 aa = normalize_vector(a)
 bb = normalize_vector(b)
 dot = float(np.clip(np.dot(aa, bb), -1.0, 1.0))
 omega = float(np.arccos(dot))
 lam = float(lam)
 if omega < 1.0e-10:
 return aa.copy()
 if abs(math.pi - omega) < 1.0e-7:
 # Nearly antipodal: normalized linear interpolation is the least arbitrary fallback.
 mixed = (1.0 - lam) * aa + lam * bb
 if np.linalg.norm(mixed) < 1.0e-12:
 mixed = aa.copy()
 mixed[0] += 1.0e-6
 return normalize_vector(mixed)
 return normalize_vector(
 math.sin((1.0 - lam) * omega) / math.sin(omega) * aa
 + math.sin(lam * omega) / math.sin(omega) * bb
)

=====
Anchor and nearest-trivial selection
=====

def _strict_topological(physics: pd.DataFrame) -> pd.DataFrame:
 mask = physics["phase_label"].eq("spin_chern_TI_candidate")
 if "strict_chern_verified" in physics:
 mask &= physics["strict_chern_verified"].fillna(0).astype(int).eq(1)
 if "strict_gap_verified" in physics:
 mask &= physics["strict_gap_verified"].fillna(0).astype(int).eq(1)
 return physics.loc[mask].copy()

def _strict_trivial(physics: pd.DataFrame) -> pd.DataFrame:
 mask = physics["phase_label"].eq("trivial_insulator")
 if "is_strict_insulator" in physics:

```

---

```

 mask &= physics["is_strict_insulator"].fillna(0).astype(int).eq(1)
 return physics.loc[mask].copy()

def select_sector_anchors(physics: pd.DataFrame, config: Step04Config) -> pd.DataFrame:
 topo = _strict_topological(physics)
 if topo.empty:
 raise RuntimeError("Step 03 中没有严格 spin-Chern TI 候选。")
 topo["chern_up_int"] = topo["chern_up_int"].astype(int)

 available = sorted(int(x) for x in topo["chern_up_int"].unique() if int(x) != 0)
 requested = available if config.target_sectors is None else [
 int(x) for x in config.target_sectors if int(x) in available
]
 missing = [] if config.target_sectors is None else [
 int(x) for x in config.target_sectors if int(x) not in available
]
 if missing:
 warnings.warn(f"Step 03 中未找到这些 Chern 扇区, 将跳过: {missing}")

 selected_rows: list[dict] = []
 for sector in requested:
 group = topo[topo["chern_up_int"].astype(int) == sector].copy()
 group = group.sort_values("indirect_gap", ascending=False).head(config.anchor_candidate_pool)
 if group.empty:
 continue

 chosen_indices: list[int] = [int(group.index[0])]
 while len(chosen_indices) < min(config.n_anchors_per_sector, len(group)):
 best_idx = None
 best_score = -np.inf
 for idx, row in group.iterrows():
 idx_i = int(idx)
 if idx_i in chosen_indices:
 continue
 candidate = row_vector(row)
 min_dist = min(
 angular_distance(candidate, row_vector(physics.loc[j]))
 for j in chosen_indices
)
 # Gap is a weak tie-breaker; diversity is primary.
 score = min_dist + 1.0e-3 * float(row["indirect_gap"])
 if score > best_score:
 best_score = score
 best_idx = idx_i
 if best_idx is None:
 break
 chosen_indices.append(best_idx)

 for rank, idx in enumerate(chosen_indices, start=1):
 row = physics.loc[idx].to_dict()
 row.update({
 "anchor_id": f"Cup{sector:+d}_anchor{rank:02d}",
 "anchor_sector": int(sector),
 "anchor_rank": int(rank),
 "is_primary_path_anchor": int(rank <= config.path_anchors_per_sector),
 })
 selected_rows.append(row)

 anchors = pd.DataFrame(selected_rows)
 if anchors.empty:
 raise RuntimeError("没有可用的分扇区拓扑锚点。")
 return anchors.reset_index(drop=True)

def select_trivial_partners(
 physics: pd.DataFrame,
 anchors: pd.DataFrame,
 config: Step04Config,

```

```

) -> pd.DataFrame:
 trivial = _strict_trivial(physics).copy()
 if trivial.empty:
 raise RuntimeError("Step 03 中没有严格平庸绝缘体。")

 rows: list[dict] = []
 for _, anchor in anchors[anchors["is_primary_path_anchor"].astype(int) == 1].iterrows():
 av = row_vector(anchor)
 anchor_t_sign = int(np.sign(float(anchor["t1"]) * float(anchor["t2"])))
 used_sample_ids: set[str] = set()

 for mode in config.trivial_partner_modes:
 candidates = trivial
 if mode == "same_t_sign":
 t_sign = np.sign(candidates["t1"].astype(float) * candidates["t2"].astype(float)).astype(int)
 candidates = candidates[t_sign == anchor_t_sign]
 elif mode != "nearest":
 raise ValueError(f"Unknown trivial partner mode: {mode}")

 if candidates.empty:
 continue
 scored: list[tuple[float, int]] = []
 for idx, candidate in candidates.iterrows():
 sid = str(candidate["sample_id"])
 if sid in used_sample_ids:
 continue
 scored.append((angular_distance(av, row_vector(candidate)), int(idx)))
 if not scored:
 continue
 dist, idx = min(scored, key=lambda item: item[0])
 partner = physics.loc[idx]
 used_sample_ids.add(str(partner["sample_id"]))
 rows.append({
 "path_id": f"{anchor['anchor_id']}_{mode}",
 "anchor_id": str(anchor["anchor_id"]),
 "anchor_sample_id": str(anchor["sample_id"]),
 "anchor_sector": int(anchor["anchor_sector"]),
 "partner_mode": mode,
 "trivial_sample_id": str(partner["sample_id"]),
 "angular_distance": float(dist),
 "euclidean_distance": float(np.linalg.norm(normalize_vector(av) -
 ↪ normalize_vector(row_vector(partner)))),
 "anchor_t_product": float(anchor["t1"] * anchor["t2"]),
 "trivial_t_product": float(partner["t1"] * partner["t2"]),
 **{f"anchor_{name}": float(anchor[name]) for name in REDUCED7},
 **{f"trivial_{name}": float(partner[name]) for name in REDUCED7},
 "anchor_indirect_gap": float(anchor["indirect_gap"]),
 "trivial_indirect_gap": float(partner["indirect_gap"]),
 })
 partners = pd.DataFrame(rows)
 if partners.empty:
 raise RuntimeError(" 无法构造拓扑—平庸路径配对。")
 return partners

```

```

=====
Local Sobol augmentation and strict physics labels
=====

```

```

def generate_local_sobol(anchors: pd.DataFrame, config: Step04Config) -> pd.DataFrame:
 rows: list[dict] = []
 sample_counter = 0
 for anchor_pos, (_, anchor) in enumerate(anchors.iterrows()):
 a = normalize_vector(row_vector(anchor))
 for radius_pos, radius in enumerate(config.local_angular_radii):
 engine = qmc.Sobol(
 d=len(REDUCED7),
 scramble=True,
 seed=int(config.local_seed + 1009 * anchor_pos + 97 * radius_pos),
)

```



```

)
cube = 2.0 * engine.random_base2(config.local_sobol_power_per_radius) - 1.0
for local_idx, raw in enumerate(cube):
 tangent = raw - float(np.dot(raw, a)) * a
 tangent_norm = float(np.linalg.norm(tangent))
 if tangent_norm < 1.0e-12:
 tangent = np.roll(a, 1) - float(np.dot(np.roll(a, 1), a)) * a
 tangent_norm = float(np.linalg.norm(tangent))
 tangent /= tangent_norm
 radial_fraction = max(0.10, min(1.0, float(np.linalg.norm(raw) / math.sqrt(len(REDUCED7)))))
 angle = float(radius) * radial_fraction
 candidate = normalize_vector(math.cos(angle) * a + math.sin(angle) * tangent)
 reduced = vector_to_reduced(candidate)
 rows.append({
 "sample_id": f"local_{sample_counter:06d}",
 "sample_source": "sector_local_sobol",
 "is_control": 0,
 "is_ml_eligible": 1,
 "parent_anchor_id": str(anchor["anchor_id"]),
 "parent_sample_id": str(anchor["sample_id"]),
 "parent_chern_up": int(anchor["anchor_sector"]),
 "local_radius_max": float(radius),
 "local_angle_actual": float(angle),
 "local_index": int(local_idx),
 **reduced,
 })
 sample_counter += 1
return pd.DataFrame(rows)

```

```

def make_physics_config(config: Step04Config) -> step3.Step03Config:

```

```

 return step3.Step03Config(
 output_dir=config.output_dir / "_step03_core_unused",
 train_sobol_power=2,
 external_sobol_power=0,
 gap_nk=config.local_gap_nk,
 initial_chern_grids=config.initial_chern_grids,
 initial_chern_shifts=config.initial_chern_shifts,
 strict_gap_grids=config.strict_gap_grids,
 strict_gap_shifts=config.strict_gap_shifts,
 strict_chern_grids=config.strict_chern_grids,
 strict_chern_shifts=config.strict_chern_shifts,
 direct_gap_skip_chern=config.direct_gap_skip_chern,
 gap_tol=config.gap_tol,
 chern_integer_tol=config.chern_integer_tol,
 min_link_tol=config.min_link_tol,
 sum_rule_tol=config.sum_rule_tol,
 checkpoint_every=config.checkpoint_every,
 ml_n_jobs=1,
)

```

```

def run_local_physics(

```

```

 local_params: pd.DataFrame,
 config: Step04Config,

```

```

) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:

```

```

 out = config.output_dir
 final_path = out / "step04_02_local_physics_labels_final.csv"
 checkpoint_path = out / "step04_02_local_physics_labels_checkpoint.csv"
 attempts_path = out / "step04_02_local_chern_attempts.csv"
 strict_gap_path = out / "step04_02_local_strict_gap_checks.csv"

```

```

 if final_path.exists() and not config.force_recalculate_local:

```

```

 final = pd.read_csv(final_path, low_memory=False)

```

```

 if set(final["sample_id"].astype(str)) == set(local_params["sample_id"].astype(str)):

```

```

 attempts = pd.read_csv(attempts_path, low_memory=False) if attempts_path.exists() else pd.DataFrame()

```

```

 gaps = pd.read_csv(strict_gap_path, low_memory=False) if strict_gap_path.exists() else pd.DataFrame()

```

```

 return final, attempts, gaps

```

---

```

if checkpoint_path.exists() and not config.force_recalculate_local:
 final_rows = pd.read_csv(checkpoint_path, low_memory=False).to_dict("records")
 done_ids = {str(row["sample_id"]) for row in final_rows}
else:
 final_rows, done_ids = [], set()
attempts_rows = (
 pd.read_csv(attempts_path, low_memory=False).to_dict("records")
 if attempts_path.exists() and not config.force_recalculate_local else []
)
strict_gap_rows = (
 pd.read_csv(strict_gap_path, low_memory=False).to_dict("records")
 if strict_gap_path.exists() and not config.force_recalculate_local else []
)

pconfig = make_physics_config(config)
started = time.time()
processed = 0
metadata_cols = [
 "parent_anchor_id", "parent_sample_id", "parent_bernoulli",
 "local_radius_max", "local_angle_actual", "local_index",
]
for _, row in local_params.iterrows():
 sid = str(row["sample_id"])
 if sid in done_ids:
 continue
 result, attempts, gaps = step3.evaluate_sample_physics(row, pconfig)
 for col in metadata_cols:
 result[col] = row[col]
 final_rows.append(result)
 for attempt in attempts:
 attempt.update({col: row[col] for col in metadata_cols})
 for gap in gaps:
 gap.update({col: row[col] for col in metadata_cols})
 attempts_rows.extend(attempts)
 strict_gap_rows.extend(gaps)
 processed += 1

 if processed % config.checkpoint_every == 0:
 atomic_write_csv(pd.DataFrame(final_rows), checkpoint_path)
 atomic_write_csv(pd.DataFrame(attempts_rows), attempts_path)
 atomic_write_csv(pd.DataFrame(strict_gap_rows), strict_gap_path)
 print(f"Local physics: {len(final_rows)}/{len(local_params)} | elapsed {time.time()-started:.1f} s")

final = pd.DataFrame(final_rows)
attempts = pd.DataFrame(attempts_rows)
gaps = pd.DataFrame(strict_gap_rows)
atomic_write_csv(final, final_path)
atomic_write_csv(attempts, attempts_path)
atomic_write_csv(gaps, strict_gap_path)
return final, attempts, gaps

```

```

def summarize_local(local: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
 phase_counts = (
 local.groupby(["parent_bernoulli", "parent_anchor_id", "local_radius_max", "phase_label"], dropna=False)
 .size().reset_index(name="count")
)
 summary_rows: list[dict] = []
 for keys, group in local.groupby(["parent_bernoulli", "parent_anchor_id", "local_radius_max"]):
 sector, anchor_id, radius = keys
 same_sector = (
 group["phase_label"].eq("spin_bernoulli_T1_candidate")
 & group["bernoulli_int"].fillna(999).astype(int).eq(int(sector))
)
 summary_rows.append({
 "parent_bernoulli": int(sector),
 "parent_anchor_id": str(anchor_id),
 "local_radius_max": float(radius),
 "n_samples": int(len(group)),

```

---

```

 "n_same_sector_TI": int(same_sector.sum()),
 "same_sector_retention_fraction": float(same_sector.mean()),
 "n_other_TI_sector": int((group["phase_label"].eq("spin_chern_TI_candidate") & ~same_sector).sum()),
 "n_trivial_insulator": int(group["phase_label"].eq("trivial_insulator").sum()),
 "n_band_metal": int(group["phase_label"].eq("spin_chern_band_metal").sum()),
 "n_boundary_or_closing": int(group["phase_label"].isin([
 "noninsulating_or_gap_closing", "boundary_or_chern_unreliable"
]).sum()),
 "best_indirect_gap_same_sector": float(group.loc[same_sector, "indirect_gap"].max()) if
 ↪ same_sector.any() else np.nan,
 })
return phase_counts, pd.DataFrame(summary_rows)

=====
Dense path gap scan
=====

def fast_gap_scan(params: Dict[str, float], nk: int) -> dict[str, float]:
 kxs, kys = core.bz_grid(int(nk), (0.0, 0.0))
 min_direct = np.inf
 max_valence = -np.inf
 min_conduction = np.inf
 direct_k = (np.nan, np.nan)
 vbm_k = (np.nan, np.nan)
 cbm_k = (np.nan, np.nan)

 for kx in kxs:
 for ky in kys:
 eig = core.eigvals_full(float(kx), float(ky), params)
 direct = float(eig[4] - eig[3])
 if direct < min_direct:
 min_direct = direct
 direct_k = (float(kx), float(ky))
 if float(eig[3]) > max_valence:
 max_valence = float(eig[3])
 vbm_k = (float(kx), float(ky))
 if float(eig[4]) < min_conduction:
 min_conduction = float(eig[4])
 cbm_k = (float(kx), float(ky))
 return {
 "min_direct_gap": float(min_direct),
 "indirect_gap": float(min_conduction - max_valence),
 "direct_gap_kx": direct_k[0],
 "direct_gap_ky": direct_k[1],
 "vbm": float(max_valence),
 "vbm_kx": vbm_k[0],
 "vbm_ky": vbm_k[1],
 "cbm": float(min_conduction),
 "cbm_kx": cbm_k[0],
 "cbm_ky": cbm_k[1],
 }

def generate_path_parameters(partners: pd.DataFrame, config: Step04Config) -> pd.DataFrame:
 rows: list[dict] = []
 lambdas = np.linspace(0.0, 1.0, config.path_n_lambda)
 for _, pair in partners.iterrows():
 a = np.array([float(pair[f"anchor_{name}"]) for name in REDUCED7])
 b = np.array([float(pair[f"trivial_{name}"]) for name in REDUCED7])
 for index, lam in enumerate(lambdas):
 reduced = vector_to_reduced(slerp(a, b, float(lam)))
 rows.append({
 "path_point_id": f"{pair['path_id']}__p{index:04d}",
 "path_id": str(pair["path_id"]),
 "path_index": int(index),
 "lambda": float(lam),
 "anchor_sector": int(pair["anchor_sector"]),
 "partner_mode": str(pair["partner_mode"]),
 })

```

---

```

 "anchor_sample_id": str(pair["anchor_sample_id"]),
 "trivial_sample_id": str(pair["trivial_sample_id"]),
 **reduced,
 })
 return pd.DataFrame(rows)

def run_path_gap_scans(path_params: pd.DataFrame, config: Step04Config) -> pd.DataFrame:
 output = config.output_dir / "step04_04_path_dense_gap_scan.csv"
 checkpoint = config.output_dir / "step04_04_path_dense_gap_scan_checkpoint.csv"
 if output.exists() and not config.force_recalculate_paths:
 df = pd.read_csv(output, low_memory=False)
 if set(df["path_point_id"].astype(str)) == set(path_params["path_point_id"].astype(str)):
 return df

 if checkpoint.exists() and not config.force_recalculate_paths:
 rows = pd.read_csv(checkpoint, low_memory=False).to_dict("records")
 done = {str(row["path_point_id"]) for row in rows}
 else:
 rows, done = [], set()

 started = time.time()
 processed = 0
 for _, row in path_params.iterrows():
 point_id = str(row["path_point_id"])
 if point_id in done:
 continue
 reduced = {name: float(row[name]) for name in REDUCED7}
 params = core.raw8_from_reduced7(reduced, e0=0.0)
 scan = fast_gap_scan(params, config.path_gap_nk)
 rows.append(**row.to_dict(), **scan, "path_gap_nk": int(config.path_gap_nk))
 processed += 1
 if processed % max(config.checkpoint_every, 10) == 0:
 atomic_write_csv(pd.DataFrame(rows), checkpoint)
 print(f"Path gap: {len(rows)}/{len(path_params)} | elapsed {time.time()-started:.1f} s")

 result = pd.DataFrame(rows).sort_values(["path_id", "path_index"]).reset_index(drop=True)
 atomic_write_csv(result, output)
 return result

def select_adaptive_chern_points(path_gap: pd.DataFrame, config: Step04Config) -> pd.DataFrame:
 selected_rows: list[pd.Series] = []
 for path_id, group in path_gap.groupby("path_id", sort=False):
 group = group.sort_values("path_index").reset_index(drop=True)
 n = len(group)
 indices: set[int] = set(range(0, n, max(1, config.path_chern_stride)))
 indices.update({0, n - 1})

 # Global and secondary smallest gap points.
 smallest = group.nsmallest(min(config.path_extra_gap_minima, n), "min_direct_gap").index
 for idx in smallest:
 for offset in (-1, 0, 1):
 if 0 <= int(idx) + offset < n:
 indices.add(int(idx) + offset)

 # All local minima of the discrete path gap.
 gaps = group["min_direct_gap"].to_numpy(float)
 for idx in range(1, n - 1):
 if gaps[idx] <= gaps[idx - 1] and gaps[idx] <= gaps[idx + 1]:
 indices.update({idx - 1, idx, idx + 1})

 if len(indices) > config.max_chern_points_per_path:
 mandatory = {0, n - 1, int(np.argmin(gaps))}
 remaining = sorted(indices - mandatory, key=lambda i: gaps[i])
 keep = mandatory | set(remaining[: max(0, config.max_chern_points_per_path - len(mandatory))])
 indices = keep

 for idx in sorted(indices):

```

---

```

 row = group.iloc[idx].copy()
 row["sample_id"] = str(row["path_point_id"])
 row["sample_source"] = "adaptive_path_chern"
 row["is_control"] = 1 # force strict Chern verification for every selected point
 row["is_ml_eligible"] = 0
 selected_rows.append(row)
 return pd.DataFrame(selected_rows)

```

```

def run_path_chern_audit(
 selected: pd.DataFrame,
 config: Step04Config,
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
 final_path = config.output_dir / "step04_04_path_adaptive_chern_labels.csv"
 attempts_path = config.output_dir / "step04_04_path_chern_attempts.csv"
 gaps_path = config.output_dir / "step04_04_path_strict_gap_checks.csv"
 checkpoint = config.output_dir / "step04_04_path_adaptive_chern_checkpoint.csv"

 if final_path.exists() and not config.force_recalculate_chern:
 final = pd.read_csv(final_path, low_memory=False)
 if set(final["sample_id"].astype(str)) == set(selected["sample_id"].astype(str)):
 attempts = pd.read_csv(attempts_path, low_memory=False) if attempts_path.exists() else pd.DataFrame()
 gaps = pd.read_csv(gaps_path, low_memory=False) if gaps_path.exists() else pd.DataFrame()
 return final, attempts, gaps

 if checkpoint.exists() and not config.force_recalculate_chern:
 rows = pd.read_csv(checkpoint, low_memory=False).to_dict("records")
 done = {str(row["sample_id"]) for row in rows}
 else:
 rows, done = [], set()
 attempts_rows = (
 pd.read_csv(attempts_path, low_memory=False).to_dict("records")
 if attempts_path.exists() and not config.force_recalculate_chern else []
)
 gap_rows = (
 pd.read_csv(gaps_path, low_memory=False).to_dict("records")
 if gaps_path.exists() and not config.force_recalculate_chern else []
)

 pconfig = make_physics_config(config)
 started = time.time()
 processed = 0
 metadata = [
 "path_id", "path_point_id", "path_index", "lambda", "anchor_sector",
 "partner_mode", "anchor_sample_id", "trivial_sample_id",
]

 for _, row in selected.iterrows():
 sid = str(row["sample_id"])
 if sid in done:
 continue
 result, attempts, gaps = step3.evaluate_sample_physics(row, pconfig)
 for col in metadata:
 result[col] = row[col]
 for attempt in attempts:
 attempt.update({col: row[col] for col in metadata})
 for gap in gaps:
 gap.update({col: row[col] for col in metadata})
 rows.append(result)
 attempts_rows.extend(attempts)
 gap_rows.extend(gaps)
 processed += 1
 if processed % config.checkpoint_every == 0:
 atomic_write_csv(pd.DataFrame(rows), checkpoint)
 atomic_write_csv(pd.DataFrame(attempts_rows), attempts_path)
 atomic_write_csv(pd.DataFrame(gap_rows), gaps_path)
 print(f"Path Chern: {len(rows)}/{len(selected)} | elapsed {time.time()-started:.1f} s")

 final = pd.DataFrame(rows).sort_values(["path_id", "lambda"]).reset_index(drop=True)
 attempts = pd.DataFrame(attempts_rows)

```

---

```

gaps = pd.DataFrame(gap_rows)
atomic_write_csv(final, final_path)
atomic_write_csv(attempts, attempts_path)
atomic_write_csv(gaps, gaps_path)
return final, attempts, gaps

=====
Transition brackets and continuous critical-valley refinement
=====

def wrap_k(value: float) -> float:
 return float((value + math.pi) % (2.0 * math.pi) - math.pi)

def torus_delta(a: float, b: float) -> float:
 return wrap_k(float(a) - float(b))

def torus_distance(k: tuple[float, float], q: tuple[float, float]) -> float:
 return float(math.hypot(torus_delta(k[0], q[0]), torus_delta(k[1], q[1])))

def classify_k_region(kx: float, ky: float, tol: float) -> dict[str, Any]:
 k = (wrap_k(kx), wrap_k(ky))
 high_symmetry = {
 "Gamma": [(0.0, 0.0)],
 "X": [(math.pi, 0.0), (-math.pi, 0.0)],
 "Y": [(0.0, math.pi), (0.0, -math.pi)],
 "M": [
 (math.pi, math.pi), (math.pi, -math.pi),
 (-math.pi, math.pi), (-math.pi, -math.pi),
],
 }
 distances = {
 name: min(torus_distance(k, point) for point in points)
 for name, points in high_symmetry.items()
 }
 nearest_name = min(distances, key=distances.get)
 if distances[nearest_name] <= tol:
 region = nearest_name
 is_high = 1
 else:
 diag_plus = abs(torus_delta(k[0], k[1]))
 diag_minus = abs(torus_delta(k[0], -k[1]))
 axis_x = abs(wrap_k(k[1]))
 axis_y = abs(wrap_k(k[0]))
 if diag_plus <= tol:
 region = "generic_Sigma_kx_eq_ky"
 elif diag_minus <= tol:
 region = "generic_SigmaPrime_kx_eq_minus_ky"
 elif axis_x <= tol or abs(abs(k[1]) - math.pi) <= tol:
 region = "generic_horizontal"
 elif axis_y <= tol or abs(abs(k[0]) - math.pi) <= tol:
 region = "generic_vertical"
 else:
 region = "generic"
 is_high = 0
 return {
 "critical_k_region": region,
 "critical_is_high_symmetry": int(is_high),
 "nearest_high_symmetry": nearest_name,
 "distance_to_nearest_high_symmetry": float(distances[nearest_name]),
 "distance_to_Sigma": float(abs(torus_delta(k[0], k[1]))),
 "distance_to_SigmaPrime": float(abs(torus_delta(k[0], -k[1]))),
 }

def _unique_k_points(points: Iterable[tuple[float, float]], tol: float = 1.0e-7) -> list[tuple[float, float]]:

```

---

```

unique: list[tuple[float, float]] = []
for point in points:
 p = (wrap_k(point[0]), wrap_k(point[1]))
 if not any(torus_distance(p, q) <= tol for q in unique):
 unique.append(p)
return unique

def c4_orbit(kx: float, ky: float) -> list[tuple[float, float]]:
 return _unique_k_points([
 (kx, ky), (-ky, kx), (-kx, -ky), (ky, -kx),
])

def d4_orbit(kx: float, ky: float) -> list[tuple[float, float]]:
 base = c4_orbit(kx, ky)
 mirrors = [(x, -y) for x, y in base]
 return _unique_k_points(base + mirrors)

def detect_transition_brackets(path_chern: pd.DataFrame, path_gap: pd.DataFrame, config: Step04Config) ->
 pd.DataFrame:
 rows: list[dict] = []
 for path_id, gap_group in path_gap.groupby("path_id", sort=False):
 ch = path_chern[path_chern["path_id"].astype(str) == str(path_id)].copy()
 ch = ch.dropna(subset=["chern_up_int"]).sort_values("lambda")
 valid = ch[ch["chern_exact_consensus"].fillna(0).astype(int) == 1]
 transition_count = 0
 if len(valid) >= 2:
 valid_rows = list(valid.to_dict("records"))
 for left, right in zip(valid_rows[:-1], valid_rows[1:]):
 c_left = int(left["chern_up_int"])
 c_right = int(right["chern_up_int"])
 if c_left == c_right:
 continue
 rows.append({
 "path_id": str(path_id),
 "transition_id": f"{path_id}__transition{transition_count:02d}",
 "lambda_left": float(left["lambda"]),
 "lambda_right": float(right["lambda"]),
 "chern_left": c_left,
 "chern_right": c_right,
 "delta_chern_up": int(c_right - c_left),
 "bracket_source": "strict_chern_change",
 })
 transition_count += 1
 if transition_count >= config.max_transitions_per_path:
 break

 if transition_count == 0:
 group = gap_group.sort_values("lambda").reset_index(drop=True)
 idx = int(group["min_direct_gap"].astype(float).idxmin())
 # idx above is original index; convert to positional index.
 pos = int(np.argmax(group["min_direct_gap"].to_numpy(float)))
 left_pos = max(0, pos - 1)
 right_pos = min(len(group) - 1, pos + 1)
 rows.append({
 "path_id": str(path_id),
 "transition_id": f"{path_id}__gap_minimum00",
 "lambda_left": float(group.iloc[left_pos]["lambda"]),
 "lambda_right": float(group.iloc[right_pos]["lambda"]),
 "chern_left": np.nan,
 "chern_right": np.nan,
 "delta_chern_up": np.nan,
 "bracket_source": "dense_gap_minimum_fallback",
 })
 return pd.DataFrame(rows)

```

---

```

def direct_gap_at(kx: float, ky: float, reduced: dict[str, float]) -> float:
 params = core.raw8_from_reduced7(reduced, e0=0.0)
 eig = core.eigvals_full(wrap_k(kx), wrap_k(ky), params)
 return float(max(0.0, eig[4] - eig[3]))

def refine_one_transition(
 bracket: pd.Series,
 pair: pd.Series,
 path_gap: pd.DataFrame,
 config: Step04Config,
) -> dict[str, Any]:
 path_id = str(bracket["path_id"])
 a = np.array([float(pair[f"anchor_{name}"]) for name in REDUCED7])
 b = np.array([float(pair[f"trivial_{name}"]) for name in REDUCED7])
 lam_lo = float(min(bracket["lambda_left"], bracket["lambda_right"]))
 lam_hi = float(max(bracket["lambda_left"], bracket["lambda_right"]))
 if lam_hi - lam_lo < 1.0e-7:
 lam_lo = max(0.0, lam_lo - 1.0 / max(10, config.path_n_lambda - 1))
 lam_hi = min(1.0, lam_hi + 1.0 / max(10, config.path_n_lambda - 1))

 local = path_gap[
 (path_gap["path_id"].astype(str) == path_id)
 & (path_gap["lambda"].astype(float) >= lam_lo - 1.0e-12)
 & (path_gap["lambda"].astype(float) <= lam_hi + 1.0e-12)
].copy()
 if local.empty:
 local = path_gap[path_gap["path_id"].astype(str) == path_id].nsmallest(
 config.critical_optimizer_starts, "min_direct_gap"
)
 else:
 local = local.nsmallest(config.critical_optimizer_starts, "min_direct_gap")

 starts: list[np.ndarray] = []
 for _, row in local.iterrows():
 starts.append(np.array([
 float(np.clip(row["lambda"], lam_lo, lam_hi)),
 float(row["direct_gap_kx"]), float(row["direct_gap_ky"]),
]))
 if not starts:
 starts = [np.array([(lam_lo + lam_hi) / 2.0, 0.0, 0.0])]

 def objective(x: np.ndarray) -> float:
 lam, kx, ky = float(x[0]), float(x[1]), float(x[2])
 reduced = vector_to_reduced(slerp(a, b, lam))
 return direct_gap_at(kx, ky, reduced)

 best = None
 for start in starts:
 result = minimize(
 objective,
 x0=start,
 method="L-BFGS-B",
 bounds=[(lam_lo, lam_hi), (-math.pi, math.pi), (-math.pi, math.pi)],
 options={"maxiter": int(config.critical_optimizer_maxiter), "ftol": 1.0e-14},
)
 if best is None or float(result.fun) < float(best.fun):
 best = result
 assert best is not None

 lam = float(best.x[0])
 kx, ky = wrap_k(float(best.x[1])), wrap_k(float(best.x[2]))
 reduced = vector_to_reduced(slerp(a, b, lam))
 c4 = c4_orbit(kx, ky)
 d4 = d4_orbit(kx, ky)
 c4_gaps = [direct_gap_at(x, y, reduced) for x, y in c4]
 d4_gaps = [direct_gap_at(x, y, reduced) for x, y in d4]

 return {

```



```

 **bracket.to_dict(),
 "anchor_sector": int(pair["anchor_sector"]),
 "partner_mode": str(pair["partner_mode"]),
 "critical_lambda": lam,
 "critical_direct_gap": float(best.fun),
 "critical_kx": kx,
 "critical_ky": ky,
 "optimizer_success": int(bool(best.success)),
 "optimizer_status": int(best.status),
 "optimizer_message": str(best.message),
 "optimizer_nfev": int(best.nfev),
 "c4_orbit_size": int(len(c4)),
 "d4_orbit_size": int(len(d4)),
 "c4_gap_min": float(min(c4_gaps)),
 "c4_gap_max": float(max(c4_gaps)),
 "c4_gap_spread": float(max(c4_gaps) - min(c4_gaps)),
 "d4_gap_min": float(min(d4_gaps)),
 "d4_gap_max": float(max(d4_gaps)),
 "d4_gap_spread": float(max(d4_gaps) - min(d4_gaps)),
 **classify_k_region(kx, ky, config.critical_k_symmetry_tol),
 **{name: reduced[name] for name in REDUCED7},
 }

def refine_critical_valleys(
 brackets: pd.DataFrame,
 partners: pd.DataFrame,
 path_gap: pd.DataFrame,
 config: Step04Config,
) -> pd.DataFrame:
 rows: list[dict] = []
 partner_map = {str(row["path_id"]): row for _, row in partners.iterrows()}
 started = time.time()
 for count, (_, bracket) in enumerate(brackets.iterrows(), start=1):
 pair = partner_map[str(bracket["path_id"])]
 rows.append(refine_one_transition(bracket, pair, path_gap, config))
 print(f"Critical valley: {count}/{len(brackets)} | elapsed {time.time()-started:.1f} s")
 return pd.DataFrame(rows)

=====
Mechanism summaries and figures
=====

def mechanism_summary(critical: pd.DataFrame) -> pd.DataFrame:
 if critical.empty:
 return pd.DataFrame()
 work = critical.copy()
 work["abs_anchor_sector"] = work["anchor_sector"].abs().astype(int)
 return (
 work.groupby(["abs_anchor_sector", "critical_k_region", "critical_is_high_symmetry"], dropna=False)
 .agg(
 n_transitions=("transition_id", "size"),
 median_critical_gap=("critical_direct_gap", "median"),
 min_critical_gap=("critical_direct_gap", "min"),
 median_c4_orbit_size=("c4_orbit_size", "median"),
 median_d4_orbit_size=("d4_orbit_size", "median"),
 median_c4_gap_spread=("c4_gap_spread", "median"),
)
 .reset_index()
)

def plot_local_retention(summary: pd.DataFrame, path: Path) -> None:
 if summary.empty:
 return
 pivot = summary.pivot_table(
 index="parent_anchor_id",
 columns="local_radius_max",

```

---

```

 values="same_sector_retention_fraction",
 aggfunc="mean",
)
 ax = pivot.plot(kind="bar", figsize=(11, 5))
 ax.set_ylabel("Same-sector TI retention fraction")
 ax.set_xlabel("Parent anchor")
 ax.set_ylim(0.0, 1.05)
 ax.set_title("TTS Step 04: local Chern-sector stability")
 plt.tight_layout()
 plt.savefig(path, dpi=180)
 plt.close()

def plot_paths(path_gap: pd.DataFrame, critical: pd.DataFrame, figure_dir: Path) -> None:
 for path_id, group in path_gap.groupby("path_id", sort=False):
 group = group.sort_values("lambda")
 fig, ax = plt.subplots(figsize=(7.5, 4.8))
 ax.plot(group["lambda"], group["min_direct_gap"], label="direct gap")
 ax.plot(group["lambda"], group["indirect_gap"], label="indirect gap")
 ax.axhline(0.0, linewidth=1.0)
 subset = critical[critical["path_id"].astype(str) == str(path_id)]
 for _, row in subset.iterrows():
 ax.axvline(float(row["critical_lambda"]), linestyle="--", alpha=0.7)
 ax.set_xlabel("Spherical path coordinate λ ")
 ax.set_ylabel("Gap")
 ax.set_title(str(path_id))
 ax.legend()
 plt.tight_layout()
 safe = "".join(ch if ch.isalnum() or ch in "_-" else "_" for ch in str(path_id))
 plt.savefig(figure_dir / f"step04_path_{safe}.png", dpi=180)
 plt.close(fig)

def plot_critical_k(critical: pd.DataFrame, path: Path) -> None:
 if critical.empty:
 return
 fig, ax = plt.subplots(figsize=(6.5, 6.0))
 for sector, group in critical.groupby("anchor_sector"):
 ax.scatter(group["critical_kx"], group["critical_ky"], label=f"Cup={int(sector):+d}")
 ax.set_xlim(-math.pi, math.pi)
 ax.set_ylim(-math.pi, math.pi)
 ax.set_xticks([-math.pi, 0.0, math.pi], ["- π ", "0", " π "])
 ax.set_yticks([-math.pi, 0.0, math.pi], ["- π ", "0", " π "])
 ax.set_xlabel("kx")
 ax.set_ylabel("ky")
 ax.set_title("Refined gap-closing valleys")
 ax.legend()
 ax.set_aspect("equal", adjustable="box")
 plt.tight_layout()
 plt.savefig(path, dpi=180)
 plt.close(fig)

=====
Main workflow
=====

def run_step04(config: Step04Config | None = None) -> Dict[str, Any]:
 config = (config or Step04Config()).normalized()
 started = time.time()

 print("[1/8] Load Step 03 strict physics labels")
 physics, physics_source = load_step3_physics(config)
 required = {"sample_id", "phase_label", "chern_up_int", "indirect_gap", *REDUCED7}
 missing = sorted(required - set(physics.columns))
 if missing:
 raise KeyError(f"Step 03 physics CSV 缺少列: {missing}")

 print("[2/8] Select sector anchors and nearest trivial partners")

```

---

```

anchors = select_sector_anchors(physics, config)
partners = select_trivial_partners(physics, anchors, config)
atomic_write_csv(anchors, config.output_dir / "step04_01_selected_chern_sector_anchors.csv")
atomic_write_csv(partners, config.output_dir / "step04_01_nearest_trivial_path_partners.csv")

print("[3/8] Generate sector-directed local Sobol samples")
local_params = generate_local_sobol(anchors, config)
atomic_write_csv(local_params, config.output_dir / "step04_02_local_sobol_parameters.csv")

print("[4/8] Strict local physics labels")
local_physics, local_attempts, local_strict_gaps = run_local_physics(local_params, config)
local_phase_counts, local_summary = summarize_local(local_physics)
atomic_write_csv(local_phase_counts, config.output_dir / "step04_03_local_phase_counts.csv")
atomic_write_csv(local_summary, config.output_dir / "step04_03_local_sector_stability_summary.csv")

print("[5/8] Dense topology-to-trivial path gap scans")
path_params = generate_path_parameters(partners, config)
atomic_write_csv(path_params, config.output_dir / "step04_04_path_parameters.csv")
path_gap = run_path_gap_scans(path_params, config)

print("[6/8] Adaptive strict Chern audit on path")
selected_chern = select_adaptive_chern_points(path_gap, config)
atomic_write_csv(selected_chern, config.output_dir / "step04_04_path_selected_chern_points.csv")
path_chern, path_attempts, path_strict_gaps = run_path_chern_audit(selected_chern, config)

print("[7/8] Detect transition brackets and refine critical valleys")
brackets = detect_transition_brackets(path_chern, path_gap, config)
atomic_write_csv(brackets, config.output_dir / "step04_05_transition_brackets.csv")
critical = refine_critical_valleys(brackets, partners, path_gap, config)
atomic_write_csv(critical, config.output_dir / "step04_05_refined_critical_valleys.csv")
mech = mechanism_summary(critical)
atomic_write_csv(mech, config.output_dir / "step04_05_mechanism_summary.csv")

print("[8/8] Figures and run summary")
plot_local_retention(local_summary, config.output_dir / "figures" / "step04_local_sector_retention.png")
plot_paths(path_gap, critical, config.output_dir / "figures")
plot_critical_k(critical, config.output_dir / "figures" / "step04_refined_critical_valleys.png")

sector_counts = {
 str(int(k)): int(v)
 for k, v in anchors["anchor_sector"].value_counts().sort_index().items()
}
local_same = int((
 local_physics["phase_label"].eq("spin_chern_TI_candidate")
 & local_physics["chern_up_int"].fillna(999).astype(int).eq(
 local_physics["parent_chern_up"].astype(int)
)
).sum())
summary = {
 "code_version": CODE_VERSION,
 "step01_version": core.CODE_VERSION,
 "step03_version": step3.CODE_VERSION,
 "step3_physics_source": physics_source,
 "n_step3_rows": int(len(physics)),
 "anchor_counts_by_sector": sector_counts,
 "n_anchors": int(len(anchors)),
 "n_path_pairs": int(len(partners)),
 "n_local_samples": int(len(local_physics)),
 "n_local_same_sector_TI": local_same,
 "local_same_sector_TI_fraction": float(local_same / len(local_physics)) if len(local_physics) else np.nan,
 "n_dense_path_points": int(len(path_gap)),
 "n_adaptive_chern_points": int(len(path_chern)),
 "n_transition_brackets": int(len(brackets)),
 "n_refined_critical_valleys": int(len(critical)),
 "critical_region_counts": {
 str(k): int(v) for k, v in critical["critical_k_region"].value_counts().items()
 } if not critical.empty else {},
 "elapsed_seconds": float(time.time() - started),
 "configuration": asdict(config),

```

---

```

 }
 atomic_write_json(summary, config.output_dir / "step04_06_run_summary.json")
 print(json.dumps(summary, indent=2, ensure_ascii=False, default=str))

 return {
 "summary": summary,
 "physics_step3": physics,
 "anchors": anchors,
 "partners": partners,
 "local_parameters": local_params,
 "local_physics": local_physics,
 "local_chern_attempts": local_attempts,
 "local_strict_gap_checks": local_strict_gaps,
 "local_phase_counts": local_phase_counts,
 "local_stability_summary": local_summary,
 "path_parameters": path_params,
 "path_gap": path_gap,
 "path_chern": path_chern,
 "path_chern_attempts": path_attempts,
 "path_strict_gap_checks": path_strict_gaps,
 "transition_brackets": brackets,
 "critical_valleys": critical,
 "mechanism_summary": mech,
 }

=====
CLI
=====

def build_arg_parser() -> argparse.ArgumentParser:
 parser = argparse.ArgumentParser(description=__doc__)
 parser.add_argument("--output-dir", type=Path, default=Step04Config.output_dir)
 parser.add_argument("--step3-input", type=Path, default=None)
 parser.add_argument("--anchors-per-sector", type=int, default=2)
 parser.add_argument("--local-power", type=int, default=4)
 parser.add_argument("--path-n-lambda", type=int, default=121)
 parser.add_argument("--path-gap-nk", type=int, default=35)
 parser.add_argument("--smoke-test", action="store_true")
 parser.add_argument("--force", action="store_true")
 return parser

def config_from_args(args: argparse.Namespace) -> Step04Config:
 if args.smoke_test:
 return Step04Config(
 output_dir=Path(args.output_dir),
 step3_input=args.step3_input,
 target_sectors=(-2, -1, 1, 2),
 n_anchors_per_sector=1,
 path_anchors_per_sector=1,
 trivial_partner_modes=("nearest",),
 local_angular_radii=(0.06,),
 local_sobol_power_per_radius=1,
 local_gap_nk=17,
 initial_chern_grids=(15, 17),
 strict_gap_grids=(17, 21),
 strict_gap_shifts=((0.0, 0.0), (0.5, 0.5)),
 strict_chern_grids=(17, 21),
 path_n_lambda=21,
 path_gap_nk=15,
 path_chern_stride=7,
 path_extra_gap_minima=2,
 max_chern_points_per_path=7,
 critical_optimizer_starts=2,
 critical_optimizer_maxiter=80,
 checkpoint_every=2,
 force_recalculate_local=args.force,
 force_recalculate_paths=args.force,

```

```

 force_recalculate_chern=args.force,
)
 return Step04Config(
 output_dir=Path(args.output_dir),
 step3_input=args.step3_input,
 n_anchors_per_sector=int(args.anchors_per_sector),
 local_sobol_power_per_radius=int(args.local_power),
 path_n_lambda=int(args.path_n_lambda),
 path_gap_nk=int(args.path_gap_nk),
 force_recalculate_local=args.force,
 force_recalculate_paths=args.force,
 force_recalculate_chern=args.force,
)

def main() -> int:
 args = build_arg_parser().parse_args()
 config = config_from_args(args)
 run_step04(config)
 return 0

if __name__ == "__main__":
 raise SystemExit(main())

```

## ► 代码清单 A.49.5 (python, 1492 行)

```

#!/usr/bin/env python3
-*- coding: utf-8 -*-
"""

```

TTS Step 05 —自旋分辨临界谷、低能 k·p 与 valley 拓扑电荷

本步骤承接 TTS Step 04, 但不再使用总八带半填充直接带隙来判定 spin-Chern 跳变的位置。真正的  $C_{up}$  跳变要求 spin-up  $4 \times 4$  块的第 2/3 条能带闭隙;  $C_{down}$  同理。因此本脚本执行:

1. 从 Step 04 的严格 Chern 跳变区间重新搜索 spin-up 内部闭隙;
2. 在  $\Gamma/M/X/Y$  和  $\Sigma/\Sigma'$  线上做确定性多起点搜索, 必要时全 BZ 回退;
3. 枚举 D4 对称轨道并判断每个 valley 属于 up 还是 down 自旋块;
4. 将临界两带投影为  $H_{eff} = d_0 I + d \cdot \sigma$ , 计算三参数 Jacobian;
5. 在  $(k_x, k_y, \lambda)$  小球上用非阿贝尔 determinant-link Berry flux 直接计算每个 valley 的整数拓扑电荷;
6. 验证 valley 电荷之和是否严格等于 Step 04 观测的  $\Delta C_{up}$ ;
7. 输出高对称质量梯度与局域半解析 k·p 系数, 为后续解析公式准备数据。

模型: tts 3.3.4.3.4, P4'/mbm', BNS 127.391, 自旋守恒八带模型。  
全部计算串行, 适合 Windows/Jupyter, 不使用 ProcessPoolExecutor。  
"""

```

from __future__ import annotations

from dataclasses import asdict, dataclass
from pathlib import Path
from typing import Any, Dict, Iterable, Sequence
import argparse
import json
import math
import time
import zipfile

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

```

---

```

from scipy.optimize import OptimizeResult, differential_evolution, minimize, minimize_scalar
from scipy.spatial import ConvexHull

import TTS_step01_model_and_label_audit_v2 as core
import TTS_step04_chern_sector_boundary_valley_tracking as step4

CODE_VERSION = "TTS_STEP05_V2_20260714"
REDUCED7 = list(core.REDUCED7)
SPINS = ("up", "down")

@dataclass
class Step05Config:
 output_dir: Path = Path("outputs_tts_step05_kp_valley_topological_charge")
 step4_input: Path = Path("outputs_tts_step04_chern_sector_boundary_valley_tracking.zip")

 # Spin-resolved critical-point search.
 coarse_lambda_points: int = 9
 coarse_diagonal_k_points: int = 51
 local_starts_per_manifold: int = 6
 powell_maxiter: int = 500
 full_bz_fallback: bool = True
 full_bz_de_popsizer: int = 10
 full_bz_de_maxiter: int = 160
 spin_gap_accept_tol: float = 2.0e-7
 candidate_lambda_dedup_tol: float = 2.0e-4
 candidate_k_dedup_tol: float = 2.0e-3
 active_spin_gap_tol: float = 2.0e-6

 # Finite-difference low-energy k.p.
 fd_k: float = 1.0e-5
 fd_lambda: float = 1.0e-5
 fd_parameter: float = 1.0e-6
 jacobian_singular_tol: float = 1.0e-8

 # Berry sphere: several radii must give the same integer charge.
 sphere_subdivision: int = 2
 sphere_k_radii: tuple[float, ...] = (0.02, 0.04, 0.08)
 sphere_lambda_radius_min: float = 8.0e-4
 sphere_lambda_radius_max: float = 1.5e-2
 sphere_integer_tol: float = 0.08
 sphere_min_link_tol: float = 1.0e-6

 # Optional Chern verification immediately on both sides of each transition.
 side_chern_grids: tuple[int, ...] = (31,)
 side_chern_shifts: tuple[tuple[float, float], ...] = ((0.0, 0.0),)
 side_lambda_fraction: float = 0.18
 chern_integer_tol: float = 0.08

 force_recalculate: bool = False
 random_seed: int = 20260714

 def normalized(self) -> "Step05Config":
 self.output_dir = Path(self.output_dir)
 self.step4_input = Path(self.step4_input)
 self.output_dir.mkdir(parents=True, exist_ok=True)
 (self.output_dir / "figures").mkdir(parents=True, exist_ok=True)
 if self.coarse_lambda_points < 5:
 raise ValueError("coarse_lambda_points must be >= 5")
 if self.coarse_diagonal_k_points < 21:
 raise ValueError("coarse_diagonal_k_points must be >= 21")
 return self

=====
I/O
=====

```

---

```

def atomic_write_csv(df: pd.DataFrame, path: Path) -> None:
 tmp = path.with_suffix(path.suffix + ".tmp")
 df.to_csv(tmp, index=False)
 tmp.replace(path)

def atomic_write_json(obj: dict, path: Path) -> None:
 tmp = path.with_suffix(path.suffix + ".tmp")
 tmp.write_text(json.dumps(obj, ensure_ascii=False, indent=2), encoding="utf-8")
 tmp.replace(path)

def _find_default_step4_input() -> Path:
 candidates = [
 Path("outputs_tts_step04_chern_sector_boundary_valley_tracking.zip"),
 Path("outputs_tts_step04_chern_sector_boundary_valley_tracking"),
]
 for p in candidates:
 if p.exists():
 return p
 zips = sorted(Path.cwd().glob("*step04*boundary*valley*.zip"))
 if zips:
 return zips[0]
 raise FileNotFoundError(
 "Cannot find Step 04 result ZIP/directory. Place it beside this script "
 "or pass --step4-input."
)

def _read_csv_from_zip(zip_path: Path, basename: str) -> pd.DataFrame:
 with zipfile.ZipFile(zip_path, "r") as zf:
 names = [n for n in zf.namelist() if Path(n).name == basename]
 if len(names) != 1:
 raise FileNotFoundError(
 f"Expected exactly one {basename!r} in {zip_path}; found {names}"
)
 with zf.open(names[0], "r") as fh:
 return pd.read_csv(fh)

def _read_csv_from_directory(root: Path, basename: str) -> pd.DataFrame:
 names = list(root.rglob(basename))
 if len(names) != 1:
 raise FileNotFoundError(
 f"Expected exactly one {basename!r} under {root}; found {names}"
)
 return pd.read_csv(names[0])

def load_step4_tables(config: Step05Config) -> dict[str, pd.DataFrame]:
 source = config.step4_input
 if not source.exists():
 source = _find_default_step4_input()
 config.step4_input = source
 files = {
 "partners": "step04_01_nearest_trivial_path_partners.csv",
 "path_chern": "step04_04_path_adaptive_chern_labels.csv",
 "brackets": "step04_05_transition_brackets.csv",
 "step4_critical": "step04_05_refined_critical_valleys.csv",
 }
 out: dict[str, pd.DataFrame] = {}
 for key, basename in files.items():
 if source.is_file() and source.suffix.lower() == ".zip":
 out[key] = _read_csv_from_zip(source, basename)
 else:
 out[key] = _read_csv_from_directory(source, basename)
 return out

```

---

```

=====
Path and Hamiltonian helpers
=====

def pair_vectors(pair: pd.Series) -> tuple[np.ndarray, np.ndarray]:
 a = np.array([float(pair[f"anchor_{name}"]) for name in REDUCED7], dtype=float)
 b = np.array([float(pair[f"trivial_{name}"]) for name in REDUCED7], dtype=float)
 return a, b

def reduced_on_path(pair: pd.Series, lam: float) -> dict[str, float]:
 a, b = pair_vectors(pair)
 return step4.vector_to_reduced(step4.slerp(a, b, float(lam)))

def raw_on_path(pair: pd.Series, lam: float) -> dict[str, float]:
 return core.raw8_from_reduced7(reduced_on_path(pair, lam), e0=0.0)

def spin_block(kx: float, ky: float, pair: pd.Series, lam: float, spin: str) -> np.ndarray:
 return core.h_spin_block_periodic(float(kx), float(ky), raw_on_path(pair, lam), spin)

def spin_middle_gap(kx: float, ky: float, pair: pd.Series, lam: float, spin: str) -> float:
 eig = np.linalg.eigvalsh(spin_block(kx, ky, pair, lam, spin))
 return float(max(0.0, eig[2] - eig[1]))

def rounded_integer(value: float, tol: float) -> int | None:
 if not np.isfinite(value):
 return None
 nearest = int(np rint(value))
 return nearest if abs(float(value) - nearest) <= tol else None

=====
Spin-resolved gap-closing search
=====

HIGH_SYMMETRY_POINTS: dict[str, tuple[float, float]] = {
 "Gamma": (0.0, 0.0),
 "X": (math.pi, 0.0),
 "Y": (0.0, math.pi),
 "M": (math.pi, math.pi),
}

def _bounded_powell_2d(
 objective,
 start: np.ndarray,
 bounds: Sequence[tuple[float, float]],
 maxiter: int,
) -> OptimizeResult:
 """Robust bounded local minimization for the two diagonal coordinates.

 Older/newer SciPy combinations can make bounded Powell construct a zero
 search direction and raise ``ValueError: zero-size array to reduction``.
 The physics does not require the Powell algorithm itself, so this wrapper
 uses L-BFGS-B first and bounded Nelder-Mead as a derivative-free fallback.
 Any local failure is converted into a normal unsuccessful OptimizeResult;
 the deterministic differential-evolution search below still runs.
 """
 x0 = np.asarray(start, dtype=float).reshape(-1)
 bnd = [(float(lo), float(hi)) for lo, hi in bounds]
 if x0.size != len(bnd):
 raise ValueError(f"start has {x0.size} variables but {len(bnd)} bounds were supplied")

```



---

```

lower = np.asarray([lo for lo, _ in bnd], dtype=float)
upper = np.asarray([hi for _, hi in bnd], dtype=float)
if (not np.all(np.isfinite(x0)) or not np.all(np.isfinite(lower))
 or not np.all(np.isfinite(upper))):
 return OptimizeResult(
 x=np.clip(np.nan_to_num(x0, nan=0.0), lower, upper),
 fun=np.inf, success=False, nfev=0,
 message="invalid non-finite local-search start or bounds",
 method="none",
)
if np.any(upper < lower):
 return OptimizeResult(
 x=np.clip(x0, np.minimum(lower, upper), np.maximum(lower, upper)),
 fun=np.inf, success=False, nfev=0,
 message="invalid reversed local-search bounds",
 method="none",
)

Keep the starting point strictly inside nonzero-width intervals.
widths = upper - lower
margin = np.maximum(1.0e-12, 1.0e-10 * np.maximum(widths, 1.0))
interior_lo = np.where(widths > 2.0 * margin, lower + margin, lower)
interior_hi = np.where(widths > 2.0 * margin, upper - margin, upper)
x0 = np.minimum(np.maximum(x0, interior_lo), interior_hi)

attempts: list[OptimizeResult] = []
errors: list[str] = []

methods = [
 (
 "L-BFGS-B",
 {
 "maxiter": int(maxiter),
 "ftol": 1.0e-15,
 "gtol": 1.0e-10,
 "maxls": 80,
 },
),
 (
 "Nelder-Mead",
 {
 "maxiter": int(maxiter),
 "xatol": 1.0e-11,
 "fatol": 1.0e-22,
 "adaptive": True,
 },
),
]

for method, options in methods:
 try:
 result = minimize(
 objective,
 x0=x0,
 method=method,
 bounds=bnd,
 options=options,
)
 result.method = method
 if np.all(np.isfinite(np.asarray(result.x, dtype=float))) and np.isfinite(float(result.fun)):
 attempts.append(result)
 except Exception as exc: # Local search must never abort the full workflow.
 errors.append(f"{method}: {type(exc).__name__}: {exc}")

if attempts:
 best = min(attempts, key=lambda r: float(r.fun))
 if errors:
 best.message = f"{best.message}; recovered after {' | '.join(errors)}"
 return best

```

---

```

try:
 fallback_fun = float(objective(x0))
except Exception as exc:
 fallback_fun = np.inf
 errors.append(f"start evaluation: {type(exc).__name__}: {exc}")
return OptimizeResult(
 x=x0, fun=fallback_fun, success=False, nfev=1,
 message="all bounded local optimizers failed: " + " | ".join(errors),
 method="start_point_fallback",
)

def search_high_symmetry_candidates(
 pair: pd.Series,
 bracket: pd.Series,
 config: Step05Config,
) -> list[dict[str, Any]]:
 lo, hi = sorted([float(bracket["lambda_left"]), float(bracket["lambda_right"])])
 rows: list[dict[str, Any]] = []
 for region, (kx, ky) in HIGH_SYMMETRY_POINTS.items():
 result = minimize_scalar(
 lambda lam: spin_middle_gap(kx, ky, pair, float(lam), "up") ** 2,
 bounds=(lo, hi),
 method="bounded",
 options={"xatol": 1.0e-13, "maxiter": 1000},
)
 gap = spin_middle_gap(kx, ky, pair, float(result.x), "up")
 rows.append({
 "search_manifold": region,
 "critical_lambda": float(result.x),
 "critical_kx": float(step4.wrap_k(kx)),
 "critical_ky": float(step4.wrap_k(ky)),
 "spin_up_gap": gap,
 "optimizer_success": int(bool(result.success)),
 "optimizer_message": str(result.message),
 "optimizer_nfev": int(result.nfev),
 })
 return rows

def diagonal_coordinates(kind: str, kappa: float) -> tuple[float, float]:
 if kind == "Sigma":
 return float(kappa), float(kappa)
 if kind == "SigmaPrime":
 return float(kappa), float(-kappa)
 raise ValueError(kind)

def search_one_diagonal(
 pair: pd.Series,
 bracket: pd.Series,
 kind: str,
 config: Step05Config,
 seed_offset: int,
) -> list[dict[str, Any]]:
 lo, hi = sorted([float(bracket["lambda_left"]), float(bracket["lambda_right"])])
 lambdas = np.linspace(lo, hi, int(config.coarse_lambda_points))
 kappas = np.linspace(-math.pi, math.pi, int(config.coarse_diagonal_k_points), endpoint=False)

 coarse: list[tuple[float, float, float]] = []
 for lam in lambdas:
 for kappa in kappas:
 kx, ky = diagonal_coordinates(kind, float(kappa))
 coarse.append((spin_middle_gap(kx, ky, pair, float(lam), "up"), float(lam), float(kappa)))
 coarse.sort(key=lambda x: x[0])

 starts: list[np.ndarray] = []
 for _, lam, kap in coarse:

```

---

```

candidate = np.array([lam, kap], dtype=float)
if all(
 abs(candidate[0] - old[0]) > 0.01 * max(hi - lo, 1.0e-5)
 or abs(step4.torus_delta(candidate[1], old[1])) > 0.08
 for old in starts
):
 starts.append(candidate)
if len(starts) >= int(config.local_starts_per_manifold):
 break

def objective(x: np.ndarray) -> float:
 lam, kappa = float(x[0]), float(x[1])
 kx, ky = diagonal_coordinates(kind, kappa)
 gap = spin_middle_gap(kx, ky, pair, lam, "up")
 return gap * gap

results: list[dict[str, Any]] = []
for start in starts:
 opt = _bounded_powell_2d(
 objective,
 start,
 [(lo, hi), (-math.pi, math.pi)],
 config.powell_maxiter,
)
 lam, kap = float(opt.x[0]), float(opt.x[1])
 kx, ky = diagonal_coordinates(kind, kap)
 gap = spin_middle_gap(kx, ky, pair, lam, "up")
 results.append({
 "search_manifold": kind,
 "critical_lambda": lam,
 "critical_kx": float(step4.wrap_k(kx)),
 "critical_ky": float(step4.wrap_k(ky)),
 "spin_up_gap": gap,
 "optimizer_success": int(bool(opt.success)),
 "optimizer_message": str(opt.message),
 "optimizer_nfev": int(getattr(opt, "nfev", 0)),
 "optimizer_method": str(getattr(opt, "method", "local_hybrid")),
 })

Differential evolution is a deterministic fallback against narrow valleys.
de = differential_evolution(
 objective,
 bounds=[(lo, hi), (-math.pi, math.pi)],
 seed=int(config.random_seed + seed_offset),
 popsize=max(8, int(config.full_bz_de_popsize // 2)),
 maxiter=max(120, int(config.full_bz_de_maxiter // 2)),
 tol=1.0e-11,
 polish=True,
 workers=1,
 updating="immediate",
)
lam, kap = float(de.x[0]), float(de.x[1])
kx, ky = diagonal_coordinates(kind, kap)
results.append({
 "search_manifold": kind + "_DE",
 "critical_lambda": lam,
 "critical_kx": float(step4.wrap_k(kx)),
 "critical_ky": float(step4.wrap_k(ky)),
 "spin_up_gap": spin_middle_gap(kx, ky, pair, lam, "up"),
 "optimizer_success": int(bool(de.success)),
 "optimizer_message": str(de.message),
 "optimizer_nfev": int(de.nfev),
})
return results

def search_full_bz_fallback(
 pair: pd.Series,
 bracket: pd.Series,

```

---

```

 config: Step05Config,
 seed_offset: int,
) -> dict[str, Any]:
 lo, hi = sorted([float(bracket["lambda_left"]), float(bracket["lambda_right"])]])

 def objective(x: np.ndarray) -> float:
 gap = spin_middle_gap(float(x[1]), float(x[2]), pair, float(x[0]), "up")
 return gap * gap

 result = differential_evolution(
 objective,
 bounds=[(lo, hi), (-math.pi, math.pi), (-math.pi, math.pi)],
 seed=int(config.random_seed + 1000 + seed_offset),
 popsize=int(config.full_bz_de_popsize),
 maxiter=int(config.full_bz_de_maxiter),
 tol=1.0e-11,
 polish=True,
 workers=1,
 updating="immediate",
)
 lam, kx, ky = [float(v) for v in result.x]
 return {
 "search_manifold": "full_BZ_DE",
 "critical_lambda": lam,
 "critical_kx": float(step4.wrap_k(kx)),
 "critical_ky": float(step4.wrap_k(ky)),
 "spin_up_gap": spin_middle_gap(kx, ky, pair, lam, "up"),
 "optimizer_success": int(bool(result.success)),
 "optimizer_message": str(result.message),
 "optimizer_nfev": int(result.nfev),
 }

def _snap_boundary_k(value: float, tol: float = 5.0e-4) -> float:
 wrapped = step4.wrap_k(float(value))
 # +pi and -pi are the same torus point. Numerical optimizers often return
 # values a few 1e-7 away from opposite sides of the seam.
 if abs(abs(wrapped) - math.pi) <= tol:
 return -math.pi
 if abs(wrapped) <= tol:
 return 0.0
 return wrapped

def orbit_signature(kx: float, ky: float, digits: int = 4) -> tuple[tuple[float, float], ...]:
 orbit = step4.d4_orbit(_snap_boundary_k(kx), _snap_boundary_k(ky))
 points = []
 for x, y in orbit:
 points.append((round(_snap_boundary_k(x), digits), round(_snap_boundary_k(y), digits)))
 return tuple(sorted(set(points)))

def deduplicate_closure_candidates(
 candidates: list[dict[str, Any]],
 config: Step05Config,
) -> list[dict[str, Any]]:
 accepted = [r for r in candidates if float(r["spin_up_gap"]) <= config.spin_gap_accept_tol]
 accepted.sort(key=lambda r: float(r["spin_up_gap"]))
 unique: list[dict[str, Any]] = []
 for row in accepted:
 sig = orbit_signature(float(row["critical_kx"]), float(row["critical_ky"]))
 duplicate = False
 for old in unique:
 old_sig = orbit_signature(float(old["critical_kx"]), float(old["critical_ky"]))
 if (
 sig == old_sig
 and abs(float(row["critical_lambda"]) - float(old["critical_lambda"]))
 <= config.candidate_lambda_dedup_tol
):

```

---

```

 duplicate = True
 break
 if not duplicate:
 unique.append(row)
 return unique

def search_transition_closures(
 bracket: pd.Series,
 pair: pd.Series,
 config: Step05Config,
 transition_index: int,
) -> tuple[list[dict[str, Any]], list[dict[str, Any]]:
 candidates = search_high_symmetry_candidates(pair, bracket, config)
 candidates.extend(search_one_diagonal(pair, bracket, "Sigma", config, 10 * transition_index + 1))
 candidates.extend(search_one_diagonal(pair, bracket, "SigmaPrime", config, 10 * transition_index + 2))

 unique = deduplicate_closure_candidates(candidates, config)
 if not unique and config.full_bz_fallback:
 candidates.append(search_full_bz_fallback(pair, bracket, config, transition_index))
 unique = deduplicate_closure_candidates(candidates, config)

 # If a full-BZ fallback finds a lower zero not represented by the symmetry searches,
 # retain it as an additional orbit.
 if config.full_bz_fallback:
 best_gap = min((float(r["spin_up_gap"]) for r in candidates), default=np.inf)
 if best_gap > config.spin_gap_accept_tol:
 candidates.append(search_full_bz_fallback(pair, bracket, config, transition_index))
 unique = deduplicate_closure_candidates(candidates, config)
 return candidates, unique

=====
D4 orbit and spin assignment
=====

def enumerate_spin_valleys(
 closure: dict[str, Any],
 pair: pd.Series,
 config: Step05Config,
) -> pd.DataFrame:
 lam = float(closure["critical_lambda"])
 orbit = step4.d4_orbit(float(closure["critical_kx"]), float(closure["critical_ky"]))
 rows: list[dict[str, Any]] = []
 all_gaps: list[float] = []
 temporary: list[dict[str, Any]] = []
 for orbit_index, (kx, ky) in enumerate(orbit):
 for spin in SPINS:
 gap = spin_middle_gap(kx, ky, pair, lam, spin)
 all_gaps.append(gap)
 temporary.append({
 "orbit_index": int(orbit_index),
 "spin": spin,
 "kx": float(kx),
 "ky": float(ky),
 "spin_gap": float(gap),
 })
 min_gap = min(all_gaps)
 threshold = max(config.active_spin_gap_tol, 100.0 * min_gap)
 for row in temporary:
 row["is_active_valley"] = int(float(row["spin_gap"]) <= threshold)
 row.update(step4.classify_k_region(float(row["kx"]), float(row["ky"]), 0.08))
 rows.append(row)
 return pd.DataFrame(rows)

=====
Two-band k-p projection

```

---

```

=====

PAULI_LABELS = ("x", "y", "z")

def pauli_decompose(matrix: np.ndarray) -> tuple[float, np.ndarray]:
 m = np.asarray(matrix, dtype=np.complex128)
 d0 = 0.5 * float((m[0, 0] + m[1, 1]).real)
 d = np.array([
 float(m[0, 1].real),
 float(-m[0, 1].imag),
 0.5 * float((m[0, 0] - m[1, 1]).real),
])
 return d0, d

def central_derivative(func, x0: float, step: float) -> np.ndarray:
 return (func(x0 + step) - func(x0 - step)) / (2.0 * step)

def crossing_basis(
 pair: pd.Series,
 lam: float,
 kx: float,
 ky: float,
 spin: str,
 config: Step05Config,
) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
 h0 = spin_block(kx, ky, pair, lam, spin)
 eig, vec = np.linalg.eigh(h0)
 u = vec[:, 1:3]

 dhl = central_derivative(
 lambda ll: spin_block(kx, ky, pair, ll, spin),
 lam,
 config.fd_lambda,
)
 projected = u.conj().T @ dhl @ u
 _, mass_vec = pauli_decompose(projected)
 # Diagonalize the λ derivative to define a reproducible local mass axis.
 _, rotation = np.linalg.eigh(projected)
 u = u @ rotation
 return eig, u, mass_vec

def projected_derivative(
 u: np.ndarray,
 derivative: np.ndarray,
) -> tuple[float, np.ndarray, np.ndarray]:
 p = u.conj().T @ derivative @ u
 d0, d = pauli_decompose(p)
 return d0, d, p

def parameter_derivative_matrix(
 pair: pd.Series,
 lam: float,
 kx: float,
 ky: float,
 spin: str,
 parameter: str,
 step: float,
) -> np.ndarray:
 reduced = reduced_on_path(pair, lam)
 plus = dict(reduced)
 minus = dict(reduced)
 plus[parameter] += step
 minus[parameter] -= step

```

---

```

hp = core.h_spin_block_periodic(
 kx, ky, core.raw8_from_reduced7(plus, e0=0.0), spin
)
hm = core.h_spin_block_periodic(
 kx, ky, core.raw8_from_reduced7(minus, e0=0.0), spin
)
return (hp - hm) / (2.0 * step)

def kp_analysis_one_valley(
 transition_id: str,
 closure_id: str,
 valley_id: str,
 pair: pd.Series,
 lam: float,
 kx: float,
 ky: float,
 spin: str,
 config: Step05Config,
) -> tuple[dict[str, Any], list[dict[str, Any]], list[dict[str, Any]]:
 eig, u, _ = crossing_basis(pair, lam, kx, ky, spin, config)

 hk = config.fd_k
 hl = config.fd_lambda
 dh_kx = central_derivative(lambda xx: spin_block(xx, ky, pair, lam, spin), kx, hk)
 dh_ky = central_derivative(lambda yy: spin_block(kx, yy, pair, lam, spin), ky, hk)
 dh_lam = central_derivative(lambda ll: spin_block(kx, ky, pair, ll, spin), lam, hl)

 derivative_rows: list[dict[str, Any]] = []
 vectors: dict[str, np.ndarray] = {}
 d0s: dict[str, float] = {}
 for coordinate, derivative in (("kx", dh_kx), ("ky", dh_ky), ("lambda", dh_lam)):
 d0, d, projected = projected_derivative(u, derivative)
 vectors[coordinate] = d
 d0s[coordinate] = d0
 derivative_rows.append({
 "transition_id": transition_id,
 "closure_id": closure_id,
 "valley_id": valley_id,
 "spin": spin,
 "coordinate": coordinate,
 "d0": d0,
 "dx": float(d[0]),
 "dy": float(d[1]),
 "dz": float(d[2]),
 "projected_00_real": float(projected[0, 0].real),
 "projected_11_real": float(projected[1, 1].real),
 "projected_01_real": float(projected[0, 1].real),
 "projected_01_imag": float(projected[0, 1].imag),
 })

 jacobian = np.column_stack([vectors["kx"], vectors["ky"], vectors["lambda"]])
 singular = np.linalg.svd(jacobian, compute_uv=False)
 det = float(np.linalg.det(jacobian))
 linear_charge = int(-np.sign(det)) if min(singular) > config.jacobian_singular_tol else 0

 # Diagonal coordinates are useful for Σ/Σ' valleys.
 d_parallel_sigma = (vectors["kx"] + vectors["ky"]) / math.sqrt(2.0)
 d_perp_sigma = (vectors["kx"] - vectors["ky"]) / math.sqrt(2.0)
 d_parallel_sigmap = (vectors["kx"] - vectors["ky"]) / math.sqrt(2.0)
 d_perp_sigmap = (vectors["kx"] + vectors["ky"]) / math.sqrt(2.0)

 gradient_rows: list[dict[str, Any]] = []
 mass_gradient = []
 for parameter in REDUCED7:
 derivative = parameter_derivative_matrix(
 pair, lam, kx, ky, spin, parameter, config.fd_parameter
)
 d0, d, _ = projected_derivative(u, derivative)

```

---

```

The basis diagonalizes dH/dλ, so dz is the local mass-direction component.
mass_gradient.append(float(d[2]))
gradient_rows.append({
 "transition_id": transition_id,
 "closure_id": closure_id,
 "valley_id": valley_id,
 "spin": spin,
 "parameter": parameter,
 "mass_gradient": float(d[2]),
 "d0_gradient": float(d[0]),
 "dx_gradient": float(d[0]),
 "dy_gradient": float(d[1]),
 "dz_gradient": float(d[2]),
})
grad_norm = float(np.linalg.norm(mass_gradient))
if grad_norm > 0:
 for row in gradient_rows:
 row["normalized_mass_gradient"] = float(row["mass_gradient"] / grad_norm)
else:
 for row in gradient_rows:
 row["normalized_mass_gradient"] = np.nan

summary = {
 "transition_id": transition_id,
 "closure_id": closure_id,
 "valley_id": valley_id,
 "spin": spin,
 "critical_lambda": float(lam),
 "critical_kx": float(kx),
 "critical_ky": float(ky),
 "crossing_energy_lower": float(eig[1]),
 "crossing_energy_upper": float(eig[2]),
 "residual_spin_gap": float(eig[2] - eig[1]),
 "jacobian_det_kx_ky_lambda": det,
 "jacobian_singular_1": float(singular[0]),
 "jacobian_singular_2": float(singular[1]),
 "jacobian_singular_3": float(singular[2]),
 "jacobian_rank": int(np.sum(singular > config.jacobian_singular_tol)),
 "linear_kp_predicted_charge": linear_charge,
 "norm_d_kx": float(np.linalg.norm(vectors["kx"])),
 "norm_d_ky": float(np.linalg.norm(vectors["ky"])),
 "norm_d_lambda": float(np.linalg.norm(vectors["lambda"])),
 "norm_d_parallel_Sigma": float(np.linalg.norm(d_parallel_sigma)),
 "norm_d_perp_Sigma": float(np.linalg.norm(d_perp_sigma)),
 "norm_d_parallel_SigmaPrime": float(np.linalg.norm(d_parallel_sigmap)),
 "norm_d_perp_SigmaPrime": float(np.linalg.norm(d_perp_sigmap)),
 **step4.classify_k_region(kx, ky, 0.08),
}
return summary, derivative_rows, gradient_rows

```

```

=====
Non-Abelian Berry flux on an icosphere
=====

```

```

def make_icosphere(subdivision: int) -> tuple[np.ndarray, np.ndarray]:
 phi = (1.0 + math.sqrt(5.0)) / 2.0
 vertices = np.array([
 (-1, phi, 0), (1, phi, 0), (-1, -phi, 0), (1, -phi, 0),
 (0, -1, phi), (0, 1, phi), (0, -1, -phi), (0, 1, -phi),
 (phi, 0, -1), (phi, 0, 1), (-phi, 0, -1), (-phi, 0, 1),
], dtype=float)
 vertices /= np.linalg.norm(vertices, axis=1)[:, None]
 faces = ConvexHull(vertices).simplices.tolist()

 def orient(face: Sequence[int], verts: np.ndarray) -> list[int]:
 i, j, k = [int(x) for x in face]
 a, b, c = verts[[i, j, k]]

```



---

```

 if float(np.dot(np.cross(b - a, c - a), (a + b + c) / 3.0)) < 0:
 return [i, k, j]
 return [i, j, k]

faces = [orient(face, vertices) for face in faces]
verts_list = [v.copy() for v in vertices]
for _ in range(int(subdivision)):
 cache: dict[tuple[int, int], int] = {}
 new_faces: list[list[int]] = []

 def midpoint(i: int, j: int) -> int:
 key = tuple(sorted((int(i), int(j))))
 if key in cache:
 return cache[key]
 p = verts_list[i] + verts_list[j]
 p = p / np.linalg.norm(p)
 idx = len(verts_list)
 verts_list.append(p)
 cache[key] = idx
 return idx

 for i, j, k in faces:
 a = midpoint(i, j)
 b = midpoint(j, k)
 c = midpoint(k, i)
 new_faces.extend([[i, a, c], [a, j, b], [c, b, k], [a, b, c]])
 vertices_now = np.array(verts_list, dtype=float)
 faces = [orient(face, vertices_now) for face in new_faces]
return np.array(verts_list, dtype=float), np.array(faces, dtype=int)

def occupied_spin_subspace(
 pair: pd.Series,
 lam: float,
 kx: float,
 ky: float,
 spin: str,
) -> np.ndarray:
 h = spin_block(kx, ky, pair, lam, spin)
 _, vec = np.linalg.eigh(h)
 return vec[:, :2]

def determinant_link(u: np.ndarray, v: np.ndarray) -> tuple[complex, float]:
 value = np.linalg.det(u.conj().T @ v)
 amp = float(abs(value))
 if amp <= 1.0e-15:
 raise FloatingPointError("Near-singular determinant link on Berry sphere")
 return value / amp, amp

def berry_sphere_charge(
 pair: pd.Series,
 lam: float,
 kx: float,
 ky: float,
 spin: str,
 k_radius: float,
 lambda_radius: float,
 subdivision: int,
) -> dict[str, float]:
 vertices, faces = make_icosphere(subdivision)
 subspaces: list[np.ndarray] = []
 for nx, ny, nz in vertices:
 ll = float(lam + lambda_radius * nz)
 # Do not wrap k around $\pm\pi$. The periodic-gauge Hamiltonian is 2π periodic,
 # and an unwrapped local chart avoids an artificial seam at M.
 subspaces.append(occupied_spin_subspace(
 pair,

```

---

```

 ll,
 float(kx + k_radius * nx),
 float(ky + k_radius * ny),
 spin,
))

total_phase = 0.0
min_link = np.inf
max_triangle_phase = 0.0
for i, j, k in faces:
 lij, aij = determinant_link(subspaces[i], subspaces[j])
 ljk, ajk = determinant_link(subspaces[j], subspaces[k])
 lki, aki = determinant_link(subspaces[k], subspaces[i])
 phase = float(np.angle(lij * ljk * lki))
 total_phase += phase
 min_link = min(min_link, aij, ajk, aki)
 max_triangle_phase = max(max_triangle_phase, abs(phase))
return {
 "berry_sphere_charge": float(total_phase / (2.0 * math.pi)),
 "sphere_min_link": float(min_link),
 "sphere_max_triangle_phase": float(max_triangle_phase),
 "sphere_vertex_count": int(len(vertices)),
 "sphere_face_count": int(len(faces)),
}

def automatic_lambda_radius(kp_row: dict[str, Any], k_radius: float, lam: float, config: Step05Config) -> float:
 dk = max(float(kp_row["norm_d_kx"]), float(kp_row["norm_d_ky"]), 1.0e-12)
 dl = max(float(kp_row["norm_d_lambda"]), 1.0e-12)
 radius = float(k_radius * dk / dl)
 boundary_cap = max(1.0e-5, 0.35 * min(lam, 1.0 - lam))
 return float(np.clip(
 radius,
 config.sphere_lambda_radius_min,
 min(config.sphere_lambda_radius_max, boundary_cap),
))

def berry_charge_attempts_one_valley(
 kp_row: dict[str, Any],
 pair: pd.Series,
 config: Step05Config,
) -> tuple[list[dict[str, Any]], dict[str, Any]]:
 attempts: list[dict[str, Any]] = []
 lam = float(kp_row["critical_lambda"])
 for k_radius in config.sphere_k_radii:
 lambda_radius = automatic_lambda_radius(kp_row, float(k_radius), lam, config)
 result = berry_sphere_charge(
 pair,
 lam,
 float(kp_row["critical_kx"]),
 float(kp_row["critical_ky"]),
 str(kp_row["spin"]),
 float(k_radius),
 lambda_radius,
 int(config.sphere_subdivision),
)
 integer = rounded_integer(result["berry_sphere_charge"], config.sphere_integer_tol)
 attempts.append({
 "transition_id": kp_row["transition_id"],
 "closure_id": kp_row["closure_id"],
 "valley_id": kp_row["valley_id"],
 "spin": kp_row["spin"],
 "k_radius": float(k_radius),
 "lambda_radius": lambda_radius,
 **result,
 "berry_charge_int": np.nan if integer is None else int(integer),
 "attempt_reliable": int(
 integer is not None and result["sphere_min_link"] > config.sphere_min_link_tol
)
 })

```

---

```

),
 })

 reliable = [r for r in attempts if int(r["attempt_reliable"]) == 1]
 integers = [int(r["berry_charge_int"]) for r in reliable]
 consensus = bool(len(reliable) == len(attempts) and len(set(integers)) == 1)
 summary = {
 "transition_id": kp_row["transition_id"],
 "closure_id": kp_row["closure_id"],
 "valley_id": kp_row["valley_id"],
 "spin": kp_row["spin"],
 "berry_charge_consensus": int(consensus),
 "berry_charge_int": int(integers[0]) if consensus else np.nan,
 "berry_charge_attempt_count": int(len(attempts)),
 "berry_charge_reliable_count": int(len(reliable)),
 "berry_charge_unique_integer_count": int(len(set(integers))),
 "berry_charge_max_abs_residual": float(max(
 abs(float(r["berry_sphere_charge"]) - float(r["berry_charge_int"]))
 for r in reliable
)) if reliable else np.nan,
 "berry_sphere_min_link_over_attempts": float(min(
 r["sphere_min_link"] for r in attempts
)),
 }
 return attempts, summary

=====
Chern checks on both sides of the corrected transition
=====

def side_chern_checks(
 transition_id: str,
 pair: pd.Series,
 bracket: pd.Series,
 critical_lambda: float,
 config: Step05Config,
) -> list[dict[str, Any]]:
 lo, hi = sorted([float(bracket["lambda_left"]), float(bracket["lambda_right"])])
 # The Step 04 bracket endpoints are themselves strict, reliable Chern points.
 # Using them avoids a coarse Chern mesh being asked to resolve an extremely
 # small gap immediately next to the critical lambda.
 lambdas = {"left": lo, "right": hi}
 rows: list[dict[str, Any]] = []
 for side, lam in lambdas.items():
 raw = raw_on_path(pair, lam)
 for nk in config.side_chern_grids:
 for shift in config.side_chern_shifts:
 base = {
 "transition_id": transition_id,
 "side": side,
 "lambda": float(lam),
 "chern_nk": int(nk),
 "shift_x": float(shift[0]),
 "shift_y": float(shift[1]),
 "attempt_ok": 0,
 "error": "",
 }
 try:
 up = core.fukui_chern_subspace(
 lambda kx, ky: core.h_spin_block_periodic(kx, ky, raw, "up"),
 core.N_OCC_SPIN, int(nk), shift,
)
 down = core.fukui_chern_subspace(
 lambda kx, ky: core.h_spin_block_periodic(kx, ky, raw, "down"),
 core.N_OCC_SPIN, int(nk), shift,
)
 cu = float(up["chern"])

```

---

```

 cd = float(down["chern"])
 base.update({
 "attempt_ok": 1,
 "chern_up": cu,
 "chern_down": cd,
 "chern_up_int": rounded_integer(cu, config.chern_integer_tol),
 "chern_down_int": rounded_integer(cd, config.chern_integer_tol),
 "min_det_up": float(up["min_det_amp"]),
 "min_det_down": float(down["min_det_amp"]),
 })
 except Exception as exc:
 base["error"] = repr(exc)
 rows.append(base)

return rows

=====
Summaries and plots
=====

def normalized_mass_formula(group: pd.DataFrame) -> str:
 g = group.sort_values("parameter")
 terms = []
 for _, row in g.iterrows():
 coefficient = float(row["normalized_mass_gradient"])
 if abs(coefficient) >= 0.08:
 terms.append(f"{coefficient:+.3f}*d{row['parameter']}")
 return " ".join(terms) if terms else "all coefficients < 0.08"

def mass_gradient_similarity(gradients: pd.DataFrame) -> pd.DataFrame:
 if gradients.empty:
 return pd.DataFrame()
 pivot = gradients.pivot_table(
 index=["transition_id", "closure_id", "valley_id", "spin"],
 columns="parameter",
 values="normalized_mass_gradient",
 aggfunc="first",
).reset_index()
 rows: list[dict[str, Any]] = []
 for i in range(len(pivot)):
 for j in range(i + 1, len(pivot)):
 a = pivot.iloc[i]
 b = pivot.iloc[j]
 va = np.array([a.get(p, np.nan) for p in REDUCED7], dtype=float)
 vb = np.array([b.get(p, np.nan) for p in REDUCED7], dtype=float)
 if not np.isfinite(va).all() or not np.isfinite(vb).all():
 continue
 rows.append({
 "transition_id_a": a["transition_id"],
 "valley_id_a": a["valley_id"],
 "spin_a": a["spin"],
 "transition_id_b": b["transition_id"],
 "valley_id_b": b["valley_id"],
 "spin_b": b["spin"],
 "cosine_similarity": float(np.dot(va, vb)),
 "absolute_cosine_similarity": float(abs(np.dot(va, vb))),
 })
 return pd.DataFrame(rows)

def plot_corrected_valleys(valleys: pd.DataFrame, path: Path) -> None:
 if valleys.empty:
 return
 active = valleys[valleys["is_active_valley"].astype(int) == 1]
 fig, ax = plt.subplots(figsize=(6.5, 6.0))
 for spin, group in active.groupby("spin"):
 ax.scatter(group["kx"], group["ky"], s=55, label=spin, alpha=0.8)

```

---

```

ax.axline((-math.pi, -math.pi), (math.pi, math.pi), linewidth=0.8, linestyle="--")
ax.axline((-math.pi, math.pi), (math.pi, -math.pi), linewidth=0.8, linestyle="--")
ax.set_xlim(-math.pi, math.pi)
ax.set_ylim(-math.pi, math.pi)
ax.set_aspect("equal")
ax.set_xlabel(r"k_x")
ax.set_ylabel(r"k_y")
ax.set_title("Spin-resolved critical valleys")
ax.legend()
fig.tight_layout()
fig.savefig(path, dpi=220)
plt.close(fig)

def plot_charge_certificate(certificate: pd.DataFrame, path: Path) -> None:
 if certificate.empty:
 return
 fig, ax = plt.subplots(figsize=(6.8, 5.2))
 x = certificate["observed_delta_chern_up"].to_numpy(float)
 y = certificate["berry_charge_sum_up"].to_numpy(float)
 ax.scatter(x, y, s=65)
 low = min(-2.5, float(np.nanmin([x, y])))
 high = max(2.5, float(np.nanmax([x, y])))
 ax.plot([low, high], [low, high], linestyle="--", linewidth=1.0)
 for _, row in certificate.iterrows():
 ax.annotate(str(row["transition_id"]).split("__transition")[0],
 (row["observed_delta_chern_up"], row["berry_charge_sum_up"]),
 fontsize=7, xytext=(3, 3), textcoords="offset points")
 ax.set_xlabel(r"Observed $\Delta C_{\uparrow}$")
 ax.set_ylabel("Sum of spin-up valley charges")
 ax.set_title("Valley-charge certificate")
 fig.tight_layout()
 fig.savefig(path, dpi=220)
 plt.close(fig)

def plot_mass_gradients(gradients: pd.DataFrame, path: Path) -> None:
 if gradients.empty:
 return
 up = gradients[gradients["spin"] == "up"].copy()
 labels = up[["transition_id", "valley_id"]].drop_duplicates()
 keys = [(r.transition_id, r.valley_id) for r in labels.itertuples(index=False)]
 matrix = []
 for transition_id, valley_id in keys:
 group = up[(up["transition_id"] == transition_id) & (up["valley_id"] == valley_id)]
 mapping = dict(zip(group["parameter"], group["normalized_mass_gradient"]))
 matrix.append([mapping.get(p, np.nan) for p in REDUCED7])
 arr = np.asarray(matrix, dtype=float)
 fig, ax = plt.subplots(figsize=(8.2, max(4.0, 0.35 * len(keys) + 1.8)))
 image = ax.imshow(arr, aspect="auto", vmin=-1, vmax=1)
 ax.set_xticks(range(len(REDUCED7)), REDUCED7, rotation=45, ha="right")
 ax.set_yticks(range(len(keys)), [f"{a.split('__transition')[0]} | {b}" for a, b in keys], fontsize=7)
 ax.set_title("Normalized local mass gradients")
 fig.colorbar(image, ax=ax, label="normalized coefficient")
 fig.tight_layout()
 fig.savefig(path, dpi=220)
 plt.close(fig)

=====
Main workflow
=====

def run_step05(config: Step05Config | None = None) -> Dict[str, Any]:
 if config is None:
 config = Step05Config()
 config = config.normalized()
 started = time.time()

```

---

```

tables = load_step4_tables(config)
partners = tables["partners"]
brackets = tables["brackets"].copy()
step4_critical = tables["step4_critical"].copy()
partner_map = {str(row["path_id"]): row for _, row in partners.iterrows()}

print("[1/8] Reload Step 04 strict transition brackets")
print(f" transitions = {len(brackets)}")

candidate_checkpoint = config.output_dir / "step05_01_spin_gap_search_candidates_checkpoint.csv"
closure_checkpoint = config.output_dir / "step05_01_corrected_spin_critical_points_checkpoint.csv"
progress_checkpoint = config.output_dir / "step05_01_search_progress_checkpoint.json"

candidate_rows: list[dict[str, Any]] = []
closure_rows: list[dict[str, Any]] = []
completed_transitions: set[str] = set()

if not config.force_recalculate and progress_checkpoint.exists():
 try:
 progress = json.loads(progress_checkpoint.read_text(encoding="utf-8"))
 if str(progress.get("code_version")) == CODE_VERSION:
 completed_transitions = {str(x) for x in progress.get("completed_transition_ids", [])}
 if candidate_checkpoint.exists():
 candidate_rows = pd.read_csv(candidate_checkpoint).to_dict(orient="records")
 if closure_checkpoint.exists():
 closure_rows = pd.read_csv(closure_checkpoint).to_dict(orient="records")
 print(
 " resume checkpoint: "
 f"{len(completed_transitions)}/{len(brackets)} transitions already completed"
)
 except Exception as exc:
 print(f" warning: ignored unreadable Step 05 search checkpoint: {exc}")
 candidate_rows = []
 closure_rows = []
 completed_transitions = set()

print("[2/8] Search spin-up internal gap closings")
for index, (_, bracket) in enumerate(brackets.iterrows(), start=1):
 transition_id = str(bracket["transition_id"])
 if transition_id in completed_transitions:
 n_existing = sum(str(r.get("transition_id")) == transition_id for r in closure_rows)
 print(
 f" {index}/{len(brackets)} {transition_id}: "
 f"resume skip, accepted spin closures = {n_existing}"
)
 continue

 path_id = str(bracket["path_id"])
 pair = partner_map[path_id]
 candidates, closures = search_transition_closures(bracket, pair, config, index)
 step4_match = step4_critical[step4_critical["transition_id"].astype(str) == str(bracket["transition_id"])]
 old_region = str(step4_match.iloc[0]["critical_k_region"]) if not step4_match.empty else "missing"
 old_gap = float(step4_match.iloc[0]["critical_direct_gap"]) if not step4_match.empty else np.nan

 for row in candidates:
 candidate_rows.append({
 **bracket.to_dict(),
 "step4_critical_region": old_region,
 "step4_full_direct_gap": old_gap,
 **row,
 })
 for closure_index, row in enumerate(closures):
 closure_id = f"{bracket['transition_id']}__spinclosure{closure_index:02d}"
 classified = step4.classify_k_region(
 float(row["critical_kx"]), float(row["critical_ky"]), 0.08
)
 closure = {
 **bracket.to_dict(),

```

---

```

 "closure_id": closure_id,
 "step4_critical_region": old_region,
 "step4_full_direct_gap": old_gap,
 **row,
 **classified,
 "step4_region_matches_spin_closure": int(old_region == classified["critical_k_region"]),
 **reduced_on_path(pair, float(row["critical_lambda"])),
 }
 closure_rows.append(closure)
 completed_transitions.add(transition_id)
 atomic_write_csv(pd.DataFrame(candidate_rows), candidate_checkpoint)
 atomic_write_csv(pd.DataFrame(closure_rows), closure_checkpoint)
 atomic_write_json(
 {
 "code_version": CODE_VERSION,
 "completed_transition_ids": sorted(completed_transitions),
 "updated_at_unix": float(time.time()),
 },
 progress_checkpoint,
)
 print(
 f" {index}/{len(brackets)} {bracket['transition_id']}: "
 f"accepted spin closures = {len(closures)}"
)

candidate_df = pd.DataFrame(candidate_rows)
closure_df = pd.DataFrame(closure_rows)

Reconstruct all closure objects, including rows restored from a checkpoint.
bracket_lookup = {str(r["transition_id"]): r for _, r in brackets.iterrows()}
closure_objects: list[tuple[dict[str, Any], pd.Series, pd.Series]] = []
for closure in closure_rows:
 transition_id = str(closure["transition_id"])
 bracket = bracket_lookup[transition_id]
 pair = partner_map[str(bracket["path_id"])]
 closure_objects.append((closure, bracket, pair))
atomic_write_csv(candidate_df, config.output_dir / "step05_01_spin_gap_search_candidates.csv")
atomic_write_csv(closure_df, config.output_dir / "step05_01_corrected_spin_critical_points.csv")
if closure_df.empty:
 raise RuntimeError("No spin-resolved critical points were found.")

print("[3/8] Assign D4-related valleys to spin-up and spin-down blocks")
valley_frames: list[pd.DataFrame] = []
closure_lookup = {c[0]["closure_id"]: c for c in closure_objects}
for closure, _, pair in closure_objects:
 df = enumerate_spin_valleys(closure, pair, config)
 df.insert(0, "closure_id", closure["closure_id"])
 df.insert(0, "transition_id", closure["transition_id"])
 df.insert(2, "observed_delta_chern_up", closure["delta_chern_up"])
 valley_frames.append(df)
valleys = pd.concat(valley_frames, ignore_index=True)
valleys["valley_id"] = [
 f"{row.closure_id}__{row.spin}_v{int(row.orbit_index):02d}"
 for row in valleys.itertuples(index=False)
]
atomic_write_csv(valleys, config.output_dir / "step05_02_spin_valley_assignment.csv")

print("[4/8] Build projected two-band k.p Hamiltonians and Jacobians")
kp_rows: list[dict[str, Any]] = []
derivative_rows: list[dict[str, Any]] = []
gradient_rows: list[dict[str, Any]] = []
active = valleys[valleys["is_active_valley"].astype(int) == 1].copy()
for count, (_, valley) in enumerate(active.iterrows(), start=1):
 closure, _, pair = closure_lookup[str(valley["closure_id"])]
 summary, derivatives, gradients = kp_analysis_one_valley(
 str(valley["transition_id"]),
 str(valley["closure_id"]),
 str(valley["valley_id"]),
 pair,
)

```

```

 float(closure["critical_lambda"]),
 float(valley["kx"]),
 float(valley["ky"]),
 str(valley["spin"]),
 config,
)
 summary["observed_delta_chern_up"] = int(closure["delta_chern_up"])
 kp_rows.append(summary)
 derivative_rows.extend(derivatives)
 gradient_rows.extend(gradients)
 print(f" k.p {count}/{len(active)}")
kp_df = pd.DataFrame(kp_rows)
derivatives_df = pd.DataFrame(derivative_rows)
gradients_df = pd.DataFrame(gradient_rows)
atomic_write_csv(kp_df, config.output_dir / "step05_03_kp_summary.csv")
atomic_write_csv(derivatives_df, config.output_dir / "step05_03_kp_derivative_coefficients.csv")
atomic_write_csv(gradients_df, config.output_dir / "step05_03_local_mass_gradients.csv")

print("[5/8] Compute non-Abelian Berry charge on small spheres")
sphere_attempt_rows: list[dict[str, Any]] = []
charge_rows: list[dict[str, Any]] = []
kp_lookup = {str(row["valley_id"]): row for row in kp_rows}
for count, row in enumerate(kp_rows, start=1):
 closure, _, pair = closure_lookup[str(row["closure_id"])]
 attempts, charge = berry_charge_attempts_one_valley(row, pair, config)
 sphere_attempt_rows.extend(attempts)
 charge["linear_kp_predicted_charge"] = row["linear_kp_predicted_charge"]
 charge["kp_and_sphere_charge_match"] = int(
 int(charge["berry_charge_consensus"]) == 1
 and int(charge["berry_charge_int"]) == int(row["linear_kp_predicted_charge"])
)
 charge_rows.append(charge)
 print(f" Berry sphere {count}/{len(kp_rows)}")
sphere_attempts = pd.DataFrame(sphere_attempt_rows)
charges = pd.DataFrame(charge_rows)
atomic_write_csv(sphere_attempts, config.output_dir / "step05_04_berry_sphere_attempts.csv")
atomic_write_csv(charges, config.output_dir / "step05_04_valley_topological_charges.csv")

print("[6/8] Verify side Chern numbers and valley-charge sum rule")
side_rows: list[dict[str, Any]] = []
One corrected closure orbit is expected per Step 04 bracket in the present data.
The code still supports multiple closure orbits and sums all charges.
for closure, bracket, pair in closure_objects:
 side_rows.extend(side_chern_checks(
 str(closure["transition_id"]),
 pair,
 bracket,
 float(closure["critical_lambda"]),
 config,
))
side_df = pd.DataFrame(side_rows)
atomic_write_csv(side_df, config.output_dir / "step05_05_side_chern_checks.csv")

charge_with_spin = charges.merge(
 kp_df[["valley_id", "spin", "critical_k_region"]],
 on=["valley_id", "spin"],
 how="left",
)
certificate_rows: list[dict[str, Any]] = []
for transition_id, group in charge_with_spin.groupby("transition_id", sort=False):
 bracket = brackets[brackets["transition_id"].astype(str) == str(transition_id)].iloc[0]
 up = group[group["spin"] == "up"]
 down = group[group["spin"] == "down"]
 up_valid = up[up["berry_charge_consensus"].astype(int) == 1]
 down_valid = down[down["berry_charge_consensus"].astype(int) == 1]
 sum_up = float(up_valid["berry_charge_int"].sum())
 sum_down = float(down_valid["berry_charge_int"].sum())
 expected = int(bracket["delta_chern_up"])
 certificate_rows.append({

```



---

```

 "transition_id": transition_id,
 "chern_left": int(bracket["chern_left"]),
 "chern_right": int(bracket["chern_right"]),
 "observed_delta_chern_up": expected,
 "n_active_spin_up_valleys": int(len(up)),
 "n_active_spin_down_valleys": int(len(down)),
 "n_reliable_spin_up_charges": int(len(up_valid)),
 "n_reliable_spin_down_charges": int(len(down_valid)),
 "berry_charge_sum_up": sum_up,
 "berry_charge_sum_down": sum_down,
 "up_charge_matches_delta_chern": int(abs(sum_up - expected) < 0.1),
 "down_charge_matches_opposite_delta": int(abs(sum_down + expected) < 0.1),
 "spin_charge_sum_zero": int(abs(sum_up + sum_down) < 0.1),
 "all_valley_charges_integer_consensus": int(
 len(up_valid) == len(up) and len(down_valid) == len(down)
),
 "all_kp_jacobians_match_sphere": int(
 (group["kp_and_sphere_charge_match"].astype(int) == 1).all()
),
 "critical_regions_up": ";".join(sorted(set(up["critical_k_region"].astype(str)))),
 "mechanism_certificate_pass": int(
 abs(sum_up - expected) < 0.1
 and abs(sum_down + expected) < 0.1
 and len(up_valid) == len(up)
 and len(down_valid) == len(down)
),
 })
certificate = pd.DataFrame(certificate_rows)
atomic_write_csv(certificate, config.output_dir / "step05_05_valley_charge_certificate.csv")

print("[7/8] Summarize local mass formulas")
formula_rows: list[dict[str, Any]] = []
for keys, group in gradients_df.groupby(
 ["transition_id", "closure_id", "valley_id", "spin"], sort=False
):
 formula_rows.append({
 "transition_id": keys[0],
 "closure_id": keys[1],
 "valley_id": keys[2],
 "spin": keys[3],
 "normalized_local_mass_formula": normalized_mass_formula(group),
 })
formulas = pd.DataFrame(formula_rows)
similarities = mass_gradient_similarity(gradients_df)
atomic_write_csv(formulas, config.output_dir / "step05_06_local_mass_formulas.csv")
atomic_write_csv(similarities, config.output_dir / "step05_06_mass_gradient_similarity.csv")

plot_corrected_valleys(
 valleys,
 config.output_dir / "figures" / "step05_spin_resolved_critical_valleys.png",
)
plot_charge_certificate(
 certificate,
 config.output_dir / "figures" / "step05_valley_charge_certificate.png",
)
plot_mass_gradients(
 gradients_df,
 config.output_dir / "figures" / "step05_local_mass_gradients.png",
)

print("[8/8] Save run summary")
summary = {
 "code_version": CODE_VERSION,
 "step01_version": core.CODE_VERSION,
 "step04_version": step4.CODE_VERSION,
 "step4_input": str(config.step4_input),
 "n_transition_brackets": int(len(brackets)),
 "n_corrected_spin_closure_orbits": int(len(closure_df)),
 "n_active_spin_up_valleys": int(((valleys["spin"] == "up") & (valleys["is_active_valley"] == 1)).sum()),

```

---

```

 "n_active_spin_down_valleys": int(((valleys["spin"] == "down") & (valleys["is_active_valley"] ==
 ↪ 1)).sum()),
 "n_kp_valleys": int(len(kp_df)),
 "n_integer_berry_charge_valleys": int(charges["berry_charge_consensus"].astype(int).sum()),
 "n_mechanism_certificates_pass": int(certificate["mechanism_certificate_pass"].astype(int).sum()),
 "n_mechanism_certificates_total": int(len(certificate)),
 "step4_region_corrections": int((closure_df["step4_region_matches_spin_closure"].astype(int) == 0).sum()),
 "critical_region_counts_spin_up": (
 kp_df[kp_df["spin"] == "up"]["critical_k_region"].value_counts().to_dict()
),
 "elapsed_seconds": float(time.time() - started),
 "configuration": {
 **asdict(config),
 "output_dir": str(config.output_dir),
 "step4_input": str(config.step4_input),
 },
}
atomic_write_json(summary, config.output_dir / "step05_07_run_summary.json")
return {
 "summary": summary,
 "closures": closure_df,
 "valleys": valleys,
 "kp": kp_df,
 "charges": charges,
 "certificate": certificate,
 "formulas": formulas,
}

=====
CLI
=====

def build_arg_parser() -> argparse.ArgumentParser:
 parser = argparse.ArgumentParser(description=__doc__)
 parser.add_argument("--step4-input", type=Path, default=None)
 parser.add_argument("--output-dir", type=Path, default=None)
 parser.add_argument("--quick", action="store_true", help="Reduce search and sphere settings for a smoke test")
 parser.add_argument("--no-full-bz-fallback", action="store_true")
 parser.add_argument("--force-recalculate", action="store_true")
 return parser

def config_from_args(args: argparse.Namespace) -> Step05Config:
 config = Step05Config()
 if args.step4_input is not None:
 config.step4_input = args.step4_input
 if args.output_dir is not None:
 config.output_dir = args.output_dir
 config.full_bz_fallback = not bool(args.no_full_bz_fallback)
 config.force_recalculate = bool(args.force_recalculate)
 if args.quick:
 config.output_dir = Path(str(config.output_dir) + "_quick")
 config.coarse_lambda_points = 9
 config.coarse_diagonal_k_points = 51
 config.local_starts_per_manifold = 6
 config.powell_maxiter = 500
 config.full_bz_de_popsizer = 10
 config.full_bz_de_maxiter = 160
 config.sphere_subdivision = 1
 config.sphere_k_radii = (0.03, 0.06)
 config.side_chern_grids = (21,)
 config.side_chern_shifts = ((0.0, 0.0),)
 return config.normalized()

def main() -> int:
 args = build_arg_parser().parse_args()

```

```

 config = config_from_args(args)
 results = run_step05(config)
 print(json.dumps(results["summary"], ensure_ascii=False, indent=2))
 return 0

if __name__ == "__main__":
 raise SystemExit(main())

```

## ► 代码清单 A.49.6 (python, 1166 行)

```

#!/usr/bin/env python3
-*- coding: utf-8 -*-
"""
TTS Step 06 —解析高对称质量分支、局域对角谷质量图册与加性 Chern 相图

```

本步骤承接 Step 04/05，目标不是再次计算 Chern，而是把已经严格确认的 9 个 Chern 跳变组织成可解释的“质量分支图册”：

1. 在  $\Gamma/M$  点利用固定反对角宇称算符把 spin-up  $4 \times 4$  块严格分解为两个  $2 \times 2$  块，给出四条闭式本征值分支；
2. 自动识别每个高对称临界点究竟是哪两条解析分支发生交换，并构造精确的有符号质量  $m_{\Gamma}$  或  $m_M$ ；
3. 对  $\Sigma/\Sigma'$  普通动量谷，在临界二带子空间中固定 diabatic 基，由于 Hamiltonian 对七个参数线性，得到局域但精确线性的质量坐标  $m_{\text{valley}}(p) = \sum_j a_j p_j$ ；
4. 将单谷 Berry 电荷和质量翻转组合成路径级加性公式  $C_{\text{up}}(p) = C_{\text{anchor}} + \sum_i \Delta C_i \Theta[m_i(p)]$ ；
5. 用 Step 04 的全部严格路径 Chern 点验证该公式，而不是用  $\lambda$  位置直接硬编码相变；
6. 可选地在 Step 03 全局严格绝缘体上审计 ( $m_{\Gamma}, m_M$ ) 的信息量，明确高对称质量为何不足以单独区分  $|C|=2$  普通谷相。

### 重要限定

```

- Γ/M 分支公式是全参数空间严格解析公式；
- Σ/Σ' 质量是围绕各临界 valley 的局部解析图册，不宣称单一全局线性质量能够覆盖全部普通动量谷；
- 本步骤给出的是“分支解析 + 局部图册 + 加性相变网络”，若数据表明多个质量方向不共线，代码会明确报告不存在单一全局质量坐标。
"""

```

```

from __future__ import annotations

from dataclasses import asdict, dataclass
from pathlib import Path
from typing import Any, Callable, Sequence
import argparse
import json
import math
import time
import zipfile

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

try:
 import TTS_step01_model_and_label_audit_v2 as core
except ImportError as exc: # pragma: no cover
 raise ImportError(
 " 未找到 TTS_step01_model_and_label_audit_v2.py。请把 Step 01 v2 脚本与本脚本放在同一目录。 "
) from exc

```

---

```

EXPECTED_STEP01_VERSION = "TTS_STEP01_V2_20260713"
if getattr(core, "CODE_VERSION", None) != EXPECTED_STEP01_VERSION:
 raise RuntimeError(
 f"Step 01 核心版本不一致: expected={EXPECTED_STEP01_VERSION}, "
 f"loaded={getattr(core, 'CODE_VERSION', None)}"
)

CODE_VERSION = "TTS_STEP06_V1_20260714"
REDUCED7 = core.REDUCED7.copy()
POINTS = {"Gamma": (0.0, 0.0), "M": (math.pi, math.pi)}

@dataclass
class Step06Config:
 output_dir: Path = Path("outputs_tts_step06_analytic_mass_branch_chern_atlas")
 step4_input: Path | None = None
 step5_input: Path | None = None
 step3_input: Path | None = None
 formula_validation_samples: int = 128
 formula_validation_seed: int = 20260714
 finite_lambda_step: float = 1.0e-5
 branch_degeneracy_tol: float = 1.0e-7
 chart_zero_tol: float = 1.0e-8
 chart_sign_tol: float = 1.0e-10
 path_prediction_zero_tol: float = 1.0e-7
 global_audit: bool = True

 def normalized(self) -> "Step06Config":
 self.output_dir = Path(self.output_dir)
 self.output_dir.mkdir(parents=True, exist_ok=True)
 (self.output_dir / "figures").mkdir(parents=True, exist_ok=True)
 if self.step4_input is not None:
 self.step4_input = Path(self.step4_input)
 if self.step5_input is not None:
 self.step5_input = Path(self.step5_input)
 if self.step3_input is not None:
 self.step3_input = Path(self.step3_input)
 if self.formula_validation_samples < 4:
 raise ValueError("formula_validation_samples must be >= 4")
 return self

=====
I/O helpers
=====

def atomic_write_csv(df: pd.DataFrame, path: Path) -> None:
 path.parent.mkdir(parents=True, exist_ok=True)
 tmp = path.with_suffix(path.suffix + ".tmp")
 df.to_csv(tmp, index=False, encoding="utf-8-sig")
 tmp.replace(path)

def atomic_write_json(obj: dict, path: Path) -> None:
 path.parent.mkdir(parents=True, exist_ok=True)
 tmp = path.with_suffix(path.suffix + ".tmp")
 tmp.write_text(json.dumps(obj, indent=2, ensure_ascii=False, default=str), encoding="utf-8")
 tmp.replace(path)

def _find_default(name: str) -> Path:
 candidates = [
 Path.cwd() / name,
 Path.cwd().parent / name,
 Path("/mnt/data") / name,
 Path.cwd() / name.removesuffix(".zip"),
 Path.cwd().parent / name.removesuffix(".zip"),
]

```

---

```

 Path("/mnt/data") / name.removesuffix(".zip"),
]
 for path in candidates:
 if path.exists():
 return path
 raise FileNotFoundError(f" 未找到输入 {name}。请把结果 ZIP 放在当前目录或显式设置路径。")

def _read_csv(source: Path, basename: str) -> pd.DataFrame:
 source = Path(source)
 if source.is_file() and source.suffix.lower() == ".zip":
 with zipfile.ZipFile(source) as zf:
 matches = [n for n in zf.namelist() if Path(n).name == basename]
 if len(matches) != 1:
 raise FileNotFoundError(
 f"{source} 中匹配 {basename!r} 的文件数量为 {len(matches)}: {matches}"
)
 with zf.open(matches[0]) as fh:
 return pd.read_csv(fh, low_memory=False)
 if source.is_dir():
 matches = list(source.rglob(basename))
 if len(matches) != 1:
 raise FileNotFoundError(
 f"{source} 中匹配 {basename!r} 的文件数量为 {len(matches)}: {matches}"
)
 return pd.read_csv(matches[0], low_memory=False)
 raise FileNotFoundError(source)

def _read_csv_pattern(source: Path, pattern: str) -> pd.DataFrame:
 source = Path(source)
 if source.is_file() and source.suffix.lower() == ".zip":
 with zipfile.ZipFile(source) as zf:
 matches = [
 n for n in zf.namelist()
 if pattern in Path(n).name and Path(n).suffix.lower() == ".csv"
]
 if len(matches) != 1:
 raise FileNotFoundError(
 f"{source} 中匹配 {pattern!r} 的 CSV 数量为 {len(matches)}: {matches}"
)
 with zf.open(matches[0]) as fh:
 return pd.read_csv(fh, low_memory=False)
 if source.is_dir():
 matches = [p for p in source.rglob("*.csv") if pattern in p.name]
 if len(matches) != 1:
 raise FileNotFoundError(
 f"{source} 中匹配 {pattern!r} 的 CSV 数量为 {len(matches)}: {matches}"
)
 return pd.read_csv(matches[0], low_memory=False)
 raise FileNotFoundError(source)

def load_inputs(config: Step06Config) -> dict[str, pd.DataFrame]:
 step4 = config.step4_input or _find_default(
 "outputs_tts_step04_chern_sector_boundary_valley_tracking.zip"
)
 step5 = config.step5_input or _find_default(
 "outputs_tts_step05_kp_valley_topological_charge.zip"
)
 config.step4_input = Path(step4)
 config.step5_input = Path(step5)

 tables = {
 "partners": _read_csv(step4, "step04_01_nearest_trivial_path_partners.csv"),
 "dense": _read_csv(step4, "step04_04_path_dense_gap_scan.csv"),
 "adaptive": _read_csv(step4, "step04_04_path_adaptive_chern_labels.csv"),
 "brackets": _read_csv(step4, "step04_05_transition_brackets.csv"),
 "critical": _read_csv(step5, "step05_01_corrected_spin_critical_points.csv"),
 }

```

---

```

 "valleys": _read_csv(step5, "step05_02_spin_valley_assignment.csv"),
 "kp": _read_csv(step5, "step05_03_kp_summary.csv"),
 "gradients": _read_csv(step5, "step05_03_local_mass_gradients.csv"),
 "charges": _read_csv(step5, "step05_04_valley_topological_charges.csv"),
 "certificates": _read_csv(step5, "step05_05_valley_charge_certificate.csv"),
}

if config.global_audit:
 try:
 step3 = config.step3_input or _find_default(
 "outputs_tts_step03_global_sobol_hierarchical_topology_ml.zip"
)
 config.step3_input = Path(step3)
 tables["step3_physics"] = _read_csv_pattern(
 step3, "step03_02_physics_labels_final__"
)
 except FileNotFoundError:
 tables["step3_physics"] = pd.DataFrame()
 else:
 tables["step3_physics"] = pd.DataFrame()
 return tables

=====
Parameter path geometry
=====

def normalize_vector(vector: Sequence[float]) -> np.ndarray:
 v = np.asarray(vector, dtype=float)
 norm = float(np.linalg.norm(v))
 if norm < 1.0e-14 or not np.isfinite(norm):
 raise ValueError("invalid parameter vector")
 return v / norm

def slerp(a: Sequence[float], b: Sequence[float], lam: float) -> np.ndarray:
 aa = normalize_vector(a)
 bb = normalize_vector(b)
 dot = float(np.clip(np.dot(aa, bb), -1.0, 1.0))
 omega = float(np.arccos(dot))
 lam = float(lam)
 if omega < 1.0e-12:
 return aa.copy()
 if abs(math.pi - omega) < 1.0e-7:
 return normalize_vector((1.0 - lam) * aa + lam * bb)
 return normalize_vector(
 math.sin((1.0 - lam) * omega) / math.sin(omega) * aa
 + math.sin(lam * omega) / math.sin(omega) * bb
)

def pair_vectors(pair: pd.Series) -> tuple[np.ndarray, np.ndarray]:
 a = np.array([float(pair[f"anchor_{name}"]) for name in REDUCED7], dtype=float)
 b = np.array([float(pair[f"trivial_{name}"]) for name in REDUCED7], dtype=float)
 return a, b

def reduced_on_path(pair: pd.Series, lam: float) -> dict[str, float]:
 a, b = pair_vectors(pair)
 vector = slerp(a, b, float(lam))
 return {name: float(value) for name, value in zip(REDUCED7, vector)}

def raw_from_reduced(reduced: dict[str, float]) -> dict[str, float]:
 return core.raw8_from_reduced7(reduced, e0=0.0)

def spin_block_from_reduced(

```

---

```

 kx: float, ky: float, reduced: dict[str, float], spin: str = "up"
) -> np.ndarray:
 return core.h_spin_block_periodic(
 float(kx), float(ky), raw_from_reduced(reduced), spin
)

=====
Exact Gamma/M parity decomposition
=====

PARITY_J = np.fliplr(np.eye(4, dtype=np.complex128))
EYE4 = np.eye(4, dtype=np.complex128)
_e = np.eye(4, dtype=np.complex128)
PARITY_BASIS = np.column_stack([
 (_e[:, 0] + _e[:, 3]) / math.sqrt(2.0),
 (_e[:, 1] + _e[:, 2]) / math.sqrt(2.0),
 (_e[:, 0] - _e[:, 3]) / math.sqrt(2.0),
 (_e[:, 1] - _e[:, 2]) / math.sqrt(2.0),
])

def parity_blocks(point: str, reduced: dict[str, float]) -> tuple[np.ndarray, np.ndarray]:
 me, t1, t2, r1, r2, r3, r4 = [float(reduced[x]) for x in REDUCED7]
 if point == "Gamma":
 plus = np.array([
 [me + t1, 2.0 * (r3 + r4)],
 [2.0 * (r3 + r4), -me + t2],
], dtype=np.complex128)
 minus = np.array([
 [me - t1, 2.0j * (r1 - r2)],
 [-2.0j * (r1 - r2), -me - t2],
], dtype=np.complex128)
 elif point == "M":
 plus = np.array([
 [me + t1, 2.0 * (r4 - r3)],
 [2.0 * (r4 - r3), -me + t2],
], dtype=np.complex128)
 minus = np.array([
 [me - t1, -2.0j * (r1 + r2)],
 [2.0j * (r1 + r2), -me - t2],
], dtype=np.complex128)
 else:
 raise ValueError(point)
 return plus, minus

def exact_branches(point: str, reduced: dict[str, float]) -> dict[str, float]:
 me, t1, t2, r1, r2, r3, r4 = [float(reduced[x]) for x in REDUCED7]
 cp = 0.5 * (t1 + t2)
 cm = -cp
 dp = me + 0.5 * (t1 - t2)
 dm = me - 0.5 * (t1 - t2)
 if point == "Gamma":
 op = 2.0 * (r3 + r4)
 om = 2.0 * (r1 - r2)
 elif point == "M":
 op = 2.0 * (r4 - r3)
 om = 2.0 * (r1 + r2)
 else:
 raise ValueError(point)
 sp = math.sqrt(dp * dp + op * op)
 sm = math.sqrt(dm * dm + om * om)
 return {
 f"{point}_P+_lower": cp - sp,
 f"{point}_P+_upper": cp + sp,
 f"{point}_P-_lower": cm - sm,
 f"{point}_P-_upper": cm + sm,
 }

```

---

```

}

def branch_formula_strings(point: str) -> dict[str, str]:
 if point == "Gamma":
 rp = "4*(r3+r4)^2"
 rm = "4*(r1-r2)^2"
 elif point == "M":
 rp = "4*(r4-r3)^2"
 rm = "4*(r1+r2)^2"
 else:
 raise ValueError(point)
 return {
 f"{point}_P+_lower": f"(t1+t2)/2 - sqrt((m_e+(t1-t2)/2)^2 + {rp})",
 f"{point}_P+_upper": f"(t1+t2)/2 + sqrt((m_e+(t1-t2)/2)^2 + {rp})",
 f"{point}_P-_lower": f"(t1+t2)/2 - sqrt((m_e-(t1-t2)/2)^2 + {rm})",
 f"{point}_P-_upper": f"(t1+t2)/2 + sqrt((m_e-(t1-t2)/2)^2 + {rm})",
 }

def build_formula_tables() -> tuple[pd.DataFrame, pd.DataFrame]:
 block_rows = [
 {"point": "Gamma", "parity": "+", "matrix": "[[m_e+t1, 2(r3+r4)], [2(r3+r4), -m_e+t2]]"},
 {"point": "Gamma", "parity": "-", "matrix": "[[m_e-t1, 2i(r1-r2)], [-2i(r1-r2), -m_e-t2]]"},
 {"point": "M", "parity": "+", "matrix": "[[m_e+t1, 2(r4-r3)], [2(r4-r3), -m_e+t2]]"},
 {"point": "M", "parity": "-", "matrix": "[[m_e-t1, -2i(r1+r2)], [2i(r1+r2), -m_e-t2]]"},
]
 branch_rows = []
 for point in ("Gamma", "M"):
 for label, formula in branch_formula_strings(point).items():
 branch_rows.append({"point": point, "branch_label": label, "formula": formula})
 return pd.DataFrame(block_rows), pd.DataFrame(branch_rows)

def validate_exact_formulas(config: Step06Config) -> pd.DataFrame:
 rng = np.random.default_rng(config.formula_validation_seed)
 rows: list[dict[str, Any]] = []
 for index in range(config.formula_validation_samples):
 vector = normalize_vector(rng.normal(size=7))
 reduced = {name: float(value) for name, value in zip(REDUCED7, vector)}
 for point, (kx, ky) in POINTS.items():
 h = spin_block_from_reduced(kx, ky, reduced, "up")
 numerical = np.linalg.eigvalsh(h)
 analytic = np.sort(np.array(list(exact_branches(point, reduced).values())))
 transformed = PARITY_BASIS.conj().T @ h @ PARITY_BASIS
 offblock = transformed[:2, 2:]
 plus, minus = parity_blocks(point, reduced)
 block_error = max(
 float(np.max(np.abs(transformed[:2, :2] - plus))),
 float(np.max(np.abs(transformed[2:, 2:] - minus))),
)
 rows.append({
 "sample_index": index,
 "point": point,
 "eigenvalue_max_abs_error": float(np.max(np.abs(numerical - analytic))),
 "parity_commutator_norm": float(np.linalg.norm(PARITY_J @ h - h @ PARITY_J)),
 "off_block_norm": float(np.linalg.norm(offblock)),
 "explicit_block_max_abs_error": block_error,
 })
 return pd.DataFrame(rows)

=====
Diabatic local mass charts
=====

def pauli_decompose(matrix: np.ndarray) -> tuple[float, np.ndarray]:
 m = np.asarray(matrix, dtype=np.complex128)

```



---

```

d0 = 0.5 * float((m[0, 0] + m[1, 1]).real)
d = np.array([
 float(m[0, 1].real),
 float(-m[0, 1].imag),
 0.5 * float((m[0, 0] - m[1, 1]).real),
])
return d0, d

def crossing_basis(
 pair: pd.Series,
 lam: float,
 kx: float,
 ky: float,
 spin: str,
 config: Step06Config,
) -> tuple[np.ndarray, np.ndarray]:
 reduced = reduced_on_path(pair, lam)
 h0 = spin_block_from_reduced(kx, ky, reduced, spin)
 eig, vec = np.linalg.eigh(h0)
 u = vec[:, 1:3]
 step = config.finite_lambda_step
 hp = spin_block_from_reduced(kx, ky, reduced_on_path(pair, lam + step), spin)
 hm = spin_block_from_reduced(kx, ky, reduced_on_path(pair, lam - step), spin)
 derivative = (hp - hm) / (2.0 * step)
 projected = u.conj().T @ derivative @ u
 _, rotation = np.linalg.eigh(projected)
 u = u @ rotation
 return eig, u

def exact_linear_chart_coefficients(
 u: np.ndarray,
 kx: float,
 ky: float,
 spin: str,
) -> tuple[dict[str, float], dict[str, float], dict[str, float], dict[str, float]]:
 mass: dict[str, float] = {}
 d0c: dict[str, float] = {}
 dxc: dict[str, float] = {}
 dyc: dict[str, float] = {}
 for parameter in REDUCED7:
 unit = {name: 0.0 for name in REDUCED7}
 unit[parameter] = 1.0
 h = spin_block_from_reduced(kx, ky, unit, spin)
 projected = u.conj().T @ h @ u
 d0, d = pauli_decompose(projected)
 d0c[parameter] = float(d0)
 dxc[parameter] = float(d[0])
 dyc[parameter] = float(d[1])
 mass[parameter] = float(d[2])
 return mass, d0c, dxc, dyc

def dot_formula(coefficients: dict[str, float], precision: int = 8) -> str:
 terms = []
 for parameter in REDUCED7:
 coefficient = float(coefficients.get(parameter, 0.0))
 terms.append(f"{coefficient:+.{precision}g}*{parameter}")
 return " ".join(terms)

def evaluate_linear(coefficients: dict[str, float], reduced: dict[str, float]) -> float:
 return float(sum(float(coefficients[p]) * float(reduced[p]) for p in REDUCED7))

def identify_high_symmetry_branch_pair(point: str, reduced: dict[str, float]) -> tuple[str, str, float]:
 branches = exact_branches(point, reduced)
 ordered = sorted(branches.items(), key=lambda item: item[1])

```

---

```

label_a, energy_a = ordered[1]
label_b, energy_b = ordered[2]
return label_a, label_b, float(energy_b - energy_a)

def evaluate_branch_difference(
 point: str,
 label_a: str,
 label_b: str,
 reduced: dict[str, float],
) -> float:
 branches = exact_branches(point, reduced)
 return float(branches[label_b] - branches[label_a])

def choose_orientation(left: float, right: float, tol: float) -> tuple[int, int]:
 if left < -tol and right > tol:
 return 1, 1
 if left > tol and right < -tol:
 return -1, 1
 # Keep a deterministic orientation even when a coarse bracket is too close.
 return (1 if right >= left else -1), 0

=====
Build transition mass atlas
=====

def build_transition_atlas(
 tables: dict[str, pd.DataFrame], config: Step06Config
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
 partners = tables["partners"].set_index("path_id", drop=False)
 critical = tables["critical"].copy()
 kp = tables["kp"].copy()
 gradients = tables["gradients"].copy()
 charges = tables["charges"].copy()
 certificates = tables["certificates"].set_index("transition_id", drop=False)

 atlas_rows: list[dict[str, Any]] = []
 coefficient_rows: list[dict[str, Any]] = []
 branch_rows: list[dict[str, Any]] = []

 for _, closure in critical.sort_values(["path_id", "critical_lambda"]).iterrows():
 transition_id = str(closure["transition_id"])
 path_id = str(closure["path_id"])
 pair = partners.loc[path_id]
 lam_c = float(closure["critical_lambda"])
 lam_l = float(closure["lambda_left"])
 lam_r = float(closure["lambda_right"])
 p_c = {name: float(closure[name]) for name in REDUCED7}
 p_l = reduced_on_path(pair, lam_l)
 p_r = reduced_on_path(pair, lam_r)
 p_anchor = reduced_on_path(pair, 0.0)
 region = str(closure["critical_k_region"])
 delta_c = int(closure["delta_chern_up"])
 cert = certificates.loc[transition_id]

 common = {
 "transition_id": transition_id,
 "path_id": path_id,
 "chern_left": int(closure["chern_left"]),
 "chern_right": int(closure["chern_right"]),
 "delta_chern_up": delta_c,
 "critical_lambda": lam_c,
 "critical_kx": float(closure["critical_kx"]),
 "critical_ky": float(closure["critical_ky"]),
 "critical_k_region": region,
 "n_active_spin_up_valleys": int(cert["n_active_spin_up_valleys"]),

```

---

```

 "berry_charge_sum_up": int(round(float(cert["berry_charge_sum_up"]))),
 "mechanism_certificate_pass": int(cert["mechanism_certificate_pass"]),
 }

 if region in ("Gamma", "M"):
 point = region
 label_a, label_b, residual = identify_high_symmetry_branch_pair(point, p_c)
 raw_l = evaluate_branch_difference(point, label_a, label_b, p_l)
 raw_c = evaluate_branch_difference(point, label_a, label_b, p_c)
 raw_r = evaluate_branch_difference(point, label_a, label_b, p_r)
 raw_anchor = evaluate_branch_difference(point, label_a, label_b, p_anchor)
 orientation, sign_flip = choose_orientation(raw_l, raw_r, config.chart_sign_tol)
 formulas = branch_formula_strings(point)
 exact_formula = f"{{orientation:+d}}*({{formulas[label_b]}} - {{formulas[label_a]}})"
 atlas_rows.append({
 **common,
 "chart_id": f"{{transition_id}}_exact_{{point}}",
 "chart_type": "exact_high_symmetry_branch",
 "mass_point": point,
 "mass_branch_a": label_a,
 "mass_branch_b": label_b,
 "mass_orientation": orientation,
 "mass_formula": exact_formula,
 "raw_mass_left": raw_l,
 "raw_mass_critical": raw_c,
 "raw_mass_right": raw_r,
 "oriented_mass_anchor": orientation * raw_anchor,
 "oriented_mass_left": orientation * raw_l,
 "oriented_mass_critical": orientation * raw_c,
 "oriented_mass_right": orientation * raw_r,
 "mass_sign_flip_verified": sign_flip,
 "critical_mass_abs_residual": abs(raw_c),
 "step05_gradient_cosine": np.nan,
 "offdiagonal_norm_at_critical": 0.0,
 })
 branch_rows.append({
 "transition_id": transition_id,
 "point": point,
 "branch_a": label_a,
 "branch_b": label_b,
 "branch_a_energy": exact_branches(point, p_c)[label_a],
 "branch_b_energy": exact_branches(point, p_c)[label_b],
 "branch_gap_abs": residual,
 "branch_a_formula": formulas[label_a],
 "branch_b_formula": formulas[label_b],
 })
 else:
 # One representative active spin-up valley is enough to define the chart;
 # the symmetry partner has the same mass coefficients.
 candidates = kp[(kp["transition_id"].astype(str) == transition_id) & (kp["spin"] == "up")].copy()
 if candidates.empty:
 raise RuntimeError(f"No spin-up k.p row for {transition_id}")
 rep = candidates.sort_values(["critical_kx", "critical_ky"]).iloc[0]
 kx = float(rep["critical_kx"])
 ky = float(rep["critical_ky"])
 eig, u = crossing_basis(pair, lam_c, kx, ky, "up", config)
 mass, d0c, dxc, dyc = exact_linear_chart_coefficients(u, kx, ky, "up")
 raw_l = evaluate_linear(mass, p_l)
 raw_c = evaluate_linear(mass, p_c)
 raw_r = evaluate_linear(mass, p_r)
 raw_anchor = evaluate_linear(mass, p_anchor)
 orientation, sign_flip = choose_orientation(raw_l, raw_r, config.chart_sign_tol)
 oriented = {name: orientation * value for name, value in mass.items()}

 grad = gradients[
 (gradients["transition_id"].astype(str) == transition_id)
 & (gradients["valley_id"].astype(str) == str(rep["valley_id"]))
 & (gradients["spin"] == "up")
].set_index("parameter")

```

---

```

grad_vector = np.array([float(grad.loc[p, "mass_gradient"]) for p in REDUCED7])
chart_vector = np.array([float(mass[p]) for p in REDUCED7])
cosine = float(np.dot(grad_vector, chart_vector) / (
 np.linalg.norm(grad_vector) * np.linalg.norm(chart_vector)
))

hcrit = spin_block_from_reduced(kx, ky, p_c, "up")
projected = u.conj().T @ hcrit @ u
_, dcrit = pauli_decompose(projected)
offdiag = float(np.linalg.norm(dcrit[:2]))

atlas_rows.append({
 **common,
 "chart_id": f"{transition_id}__local_{region}",
 "chart_type": "local_exact_linear_diabatic_mass",
 "mass_point": region,
 "mass_branch_a": "critical_subspace_state_0",
 "mass_branch_b": "critical_subspace_state_1",
 "mass_orientation": orientation,
 "mass_formula": dot_formula(oriented),
 "raw_mass_left": raw_l,
 "raw_mass_critical": raw_c,
 "raw_mass_right": raw_r,
 "oriented_mass_anchor": orientation * raw_anchor,
 "oriented_mass_left": orientation * raw_l,
 "oriented_mass_critical": orientation * raw_c,
 "oriented_mass_right": orientation * raw_r,
 "mass_sign_flip_verified": sign_flip,
 "critical_mass_abs_residual": abs(raw_c),
 "step05_gradient_cosine": cosine,
 "offdiagonal_norm_at_critical": offdiag,
})
for parameter in REDUCED7:
 coefficient_rows.append({
 "transition_id": transition_id,
 "chart_id": f"{transition_id}__local_{region}",
 "critical_k_region": region,
 "critical_kx": kx,
 "critical_ky": ky,
 "parameter": parameter,
 "raw_mass_coefficient": mass[parameter],
 "oriented_mass_coefficient": oriented[parameter],
 "d0_coefficient": d0c[parameter],
 "dx_coefficient": dxc[parameter],
 "dy_coefficient": dyc[parameter],
 "step05_mass_gradient": float(grad.loc[parameter, "mass_gradient"]),
 })

atlas = pd.DataFrame(atlas_rows).sort_values(["path_id", "critical_lambda"]).reset_index(drop=True)
coefficients = pd.DataFrame(coefficient_rows)
branches = pd.DataFrame(branch_rows)
return atlas, coefficients, branches

=====
Chart evaluators and additive Chern prediction
=====

def coefficient_map(coefficients: pd.DataFrame, transition_id: str) -> dict[str, float]:
 subset = coefficients[coefficients["transition_id"].astype(str) == str(transition_id)]
 return {
 str(row["parameter"]): float(row["oriented_mass_coefficient"])
 for _, row in subset.iterrows()
 }

def evaluate_oriented_chart(
 chart: pd.Series,

```

---

```

 reduced: dict[str, float],
 coefficients: pd.DataFrame,
) -> float:
 if chart["chart_type"] == "exact_high_symmetry_branch":
 raw = evaluate_branch_difference(
 str(chart["mass_point"]),
 str(chart["mass_branch_a"]),
 str(chart["mass_branch_b"]),
 reduced,
)
 return float(chart["mass_orientation"]) * raw
 coeff = coefficient_map(coefficients, str(chart["transition_id"]))
 return evaluate_linear(coeff, reduced)

def predict_path_chern(
 tables: dict[str, pd.DataFrame],
 atlas: pd.DataFrame,
 coefficients: pd.DataFrame,
 config: Step06Config,
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame, pd.DataFrame]:
 partners = tables["partners"].set_index("path_id", drop=False)
 adaptive = tables["adaptive"].copy()
 dense = tables["dense"].copy()

 adaptive_rows: list[dict[str, Any]] = []
 dense_rows: list[dict[str, Any]] = []
 formula_rows: list[dict[str, Any]] = []

 for path_id, charts in atlas.groupby("path_id", sort=False):
 charts = charts.sort_values("critical_lambda").reset_index(drop=True)
 pair = partners.loc[path_id]
 anchor_chern = int(pair["anchor_sector"])
 formula_terms = []
 for _, chart in charts.iterrows():
 formula_terms.append(
 f"({int(chart['delta_chern_up']):+d})*Theta({chart['chart_id']})"
)
 formula_rows.append({
 "path_id": path_id,
 "anchor_chern": anchor_chern,
 "n_transition_charts": len(charts),
 "additive_formula": f"C_up = {anchor_chern:+d} " + " ".join(formula_terms),
 "transition_ids": ";".join(charts["transition_id"].astype(str)),
 })

 def one_prediction(reduced: dict[str, float]) -> tuple[int, list[float], list[int]]:
 predicted = anchor_chern
 masses: list[float] = []
 crossed_flags: list[int] = []
 for _, chart in charts.iterrows():
 mass = evaluate_oriented_chart(chart, reduced, coefficients)
 masses.append(mass)
 crossed = int(mass > config.path_prediction_zero_tol)
 crossed_flags.append(crossed)
 if crossed:
 predicted += int(chart["delta_chern_up"])
 return int(predicted), masses, crossed_flags

 a_subset = adaptive[adaptive["path_id"].astype(str) == str(path_id)]
 for _, row in a_subset.iterrows():
 reduced = {name: float(row[name]) for name in REDUCED7}
 pred, masses, flags = one_prediction(reduced)
 actual = int(round(float(row["chern_up_int"]))) if np.isfinite(row["chern_up_int"]) else None
 record = {
 "path_id": path_id,
 "path_point_id": row["path_point_id"],
 "path_index": int(row["path_index"]),
 "lambda": float(row["lambda"]),

```

---

```

 "actual_chern_up": actual,
 "has_actual_chern": int(actual is not None),
 "predicted_chern_up": pred,
 "prediction_match": int(actual == pred) if actual is not None else np.nan,
 "minimum_abs_chart_mass": float(min(abs(x) for x in masses)),
 }
 for j, (mass, flag) in enumerate(zip(masses, flags)):
 record[f"chart{j:02d}_oriented_mass"] = mass
 record[f"chart{j:02d}_crossed"] = flag
 adaptive_rows.append(record)

d_subset = dense[dense["path_id"].astype(str) == str(path_id)]
for _, row in d_subset.iterrows():
 reduced = {name: float(row[name]) for name in REDUCED7}
 pred, masses, flags = one_prediction(reduced)
 record = {
 "path_id": path_id,
 "path_point_id": row["path_point_id"],
 "path_index": int(row["path_index"]),
 "lambda": float(row["lambda"]),
 "predicted_chern_up": pred,
 "min_direct_gap": float(row["min_direct_gap"]),
 "indirect_gap": float(row["indirect_gap"]),
 "minimum_abs_chart_mass": float(min(abs(x) for x in masses)),
 }
 for j, (mass, flag) in enumerate(zip(masses, flags)):
 record[f"chart{j:02d}_oriented_mass"] = mass
 record[f"chart{j:02d}_crossed"] = flag
 dense_rows.append(record)

adaptive_pred = pd.DataFrame(adaptive_rows)
dense_pred = pd.DataFrame(dense_rows)
formulas = pd.DataFrame(formula_rows)
metrics_rows = []
valid_all = adaptive_pred[adaptive_pred["has_actual_chern"].eq(1)].copy()
for path_id, group_all in adaptive_pred.groupby("path_id", sort=False):
 group = group_all[group_all["has_actual_chern"].eq(1)]
 metrics_rows.append({
 "path_id": path_id,
 "n_strict_chern_points": len(group),
 "n_unresolved_points_excluded": len(group_all) - len(group),
 "n_correct": int(group["prediction_match"].sum()) if len(group) else 0,
 "accuracy": float(group["prediction_match"].mean()) if len(group) else np.nan,
 "minimum_abs_chart_mass_over_strict_points": float(group["minimum_abs_chart_mass"].min()) if
 ↪ len(group) else np.nan,
 })
metrics = pd.DataFrame(metrics_rows)
metrics = pd.concat([
 metrics,
 pd.DataFrame([
 "path_id": "ALL_PATHS",
 "n_strict_chern_points": len(valid_all),
 "n_unresolved_points_excluded": len(adaptive_pred) - len(valid_all),
 "n_correct": int(valid_all["prediction_match"].sum()),
 "accuracy": float(valid_all["prediction_match"].mean()),
 "minimum_abs_chart_mass_over_strict_points": float(valid_all["minimum_abs_chart_mass"].min()),
])
], ignore_index=True)
return adaptive_pred, dense_pred, formulas, metrics

=====
Transition network and global high-symmetry audit
=====

def build_transition_network(atlas: pd.DataFrame) -> pd.DataFrame:
 rows = []
 for keys, group in atlas.groupby(

```

---

```

 ["chern_left", "chern_right", "critical_k_region", "delta_chern_up"],
 dropna=False,
):
 left, right, region, delta = keys
 rows.append({
 "chern_left": int(left),
 "chern_right": int(right),
 "delta_chern_up": int(delta),
 "critical_k_region": str(region),
 "n_observed_transitions": len(group),
 "transition_ids": ";".join(group["transition_id"].astype(str)),
 "mean_active_spin_up_valleys": float(group["n_active_spin_up_valleys"].mean()),
 "all_mechanism_certificates_pass": int(group["mechanism_certificate_pass"].eq(1).all()),
 })
 return pd.DataFrame(rows).sort_values(["chern_left", "chern_right", "critical_k_region"])

def canonical_high_symmetry_masses(reduced: dict[str, float]) -> tuple[float, float]:
 g = exact_branches("Gamma", reduced)
 m = exact_branches("M", reduced)
 m_gamma = g["Gamma_P+_upper"] - g["Gamma_P-_lower"]
 m_m = m["M_P+_upper"] - m["M_P+_lower"]
 return float(m_gamma), float(m_m)

def global_high_symmetry_audit(physics: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
 if physics.empty:
 return pd.DataFrame(), pd.DataFrame()
 strict = physics.copy()
 if "is_strict_insulator" in strict:
 strict = strict[strict["is_strict_insulator"].fillna(0).astype(int).eq(1)]
 strict = strict[np.isfinite(strict["chern_up_int"])]
 rows = []
 for _, row in strict.iterrows():
 reduced = {name: float(row[name]) for name in REDUCED7}
 mg, mm = canonical_high_symmetry_masses(reduced)
 rows.append({
 "sample_id": row.get("sample_id", ""),
 "sample_source": row.get("sample_source", ""),
 "chern_up_int": int(round(float(row["chern_up_int"]))),
 "phase_label": row.get("phase_label", ""),
 "m_Gamma_exact": mg,
 "m_M_exact": mm,
 "sign_m_Gamma": int(np.sign(mg)),
 "sign_m_M": int(np.sign(mm)),
 **reduced,
 })
 audit = pd.DataFrame(rows)
 ambiguity_rows = []
 for (sg, sm), group in audit.groupby(["sign_m_Gamma", "sign_m_M"]):
 sectors = sorted(group["chern_up_int"].unique().tolist())
 ambiguity_rows.append({
 "sign_m_Gamma": int(sg),
 "sign_m_M": int(sm),
 "n_samples": len(group),
 "chern_sectors": ";".join(str(int(x)) for x in sectors),
 "n_distinct_chern_sectors": len(sectors),
 "high_symmetry_signs_are_unique_classifier": int(len(sectors) == 1),
 })
 return audit, pd.DataFrame(ambiguity_rows)

=====
Figures
=====

def plot_formula_validation(validation: pd.DataFrame, path: Path) -> None:
 fig, ax = plt.subplots(figsize=(7.2, 4.6))

```

---

```

data = [
 validation[validation["point"] == point]["eigenvalue_max_abs_error"].to_numpy()
 for point in ("Gamma", "M")
]
ax.boxplot(data, tick_labels=["Gamma", "M"], showfliers=True)
ax.set_yscale("log")
ax.set_ylabel("max |E_exact - E_numeric|")
ax.set_title("Exact parity-branch validation")
ax.grid(True, axis="y", alpha=0.3)
fig.tight_layout()
fig.savefig(path, dpi=220)
plt.close(fig)

def plot_mass_charts(dense_pred: pd.DataFrame, atlas: pd.DataFrame, path: Path) -> None:
 paths = list(dense_pred["path_id"].drop_duplicates())
 ncols = 2
 nrows = int(math.ceil(len(paths) / ncols))
 fig, axes = plt.subplots(nrows, ncols, figsize=(12, 3.4 * nrows), squeeze=False)
 for ax, path_id in zip(axes.flat, paths):
 group = dense_pred[dense_pred["path_id"] == path_id].sort_values("lambda")
 charts = atlas[atlas["path_id"] == path_id].sort_values("critical_lambda")
 mass_cols = sorted([c for c in group.columns if c.endswith("_oriented_mass") and c.startswith("chart")])
 for index, col in enumerate(mass_cols):
 label = str(charts.iloc[index]["critical_k_region"]) if index < len(charts) else col
 ax.plot(group["lambda"], group[col], label=label)
 if index < len(charts):
 ax.axvline(float(charts.iloc[index]["critical_lambda"]), linestyle="--", linewidth=0.9)
 ax.axhline(0.0, linewidth=0.8)
 ax.set_title(path_id, fontsize=9)
 ax.set_xlabel("path lambda")
 ax.set_ylabel("oriented mass")
 ax.grid(True, alpha=0.25)
 ax.legend(fontsize=7)
 for ax in axes.flat[len(paths):]:
 ax.axis("off")
 fig.tight_layout()
 fig.savefig(path, dpi=220)
 plt.close(fig)

def plot_path_chern_prediction(
 dense_pred: pd.DataFrame,
 adaptive_pred: pd.DataFrame,
 atlas: pd.DataFrame,
 path: Path,
) -> None:
 paths = list(dense_pred["path_id"].drop_duplicates())
 ncols = 2
 nrows = int(math.ceil(len(paths) / ncols))
 fig, axes = plt.subplots(nrows, ncols, figsize=(12, 3.2 * nrows), squeeze=False)
 for ax, path_id in zip(axes.flat, paths):
 dense = dense_pred[dense_pred["path_id"] == path_id].sort_values("lambda")
 strict = adaptive_pred[adaptive_pred["path_id"] == path_id].sort_values("lambda")
 charts = atlas[atlas["path_id"] == path_id]
 ax.step(dense["lambda"], dense["predicted_chern_up"], where="mid", label="mass-atlas prediction")
 strict_plot = strict[np.isfinite(strict["actual_chern_up"])]
 ax.scatter(strict_plot["lambda"], strict_plot["actual_chern_up"], marker="o", s=22, label="strict Chern")
 for value in charts["critical_lambda"]:
 ax.axvline(float(value), linestyle="--", linewidth=0.8)
 ax.set_title(path_id, fontsize=9)
 ax.set_xlabel("path lambda")
 ax.set_ylabel("C_up")
 tick_values = dense["predicted_chern_up"].astype(int).tolist()
 tick_values += strict_plot["actual_chern_up"].astype(int).tolist()
 ax.set_yticks(sorted(set(tick_values)))
 ax.grid(True, alpha=0.25)
 ax.legend(fontsize=7)
 for ax in axes.flat[len(paths):]:

```



---

```

 ax.axis("off")
 fig.tight_layout()
 fig.savefig(path, dpi=220)
 plt.close(fig)

def plot_transition_network(network: pd.DataFrame, path: Path) -> None:
 nodes = sorted(set(network["chern_left"].astype(int)) | set(network["chern_right"].astype(int)))
 positions = {node: (i, 0.0) for i, node in enumerate(nodes)}
 fig, ax = plt.subplots(figsize=(10, 3.8))
 for node, (x, y) in positions.items():
 circle = plt.Circle((x, y), 0.18, fill=False, linewidth=1.8)
 ax.add_patch(circle)
 ax.text(x, y, f"C={node:d}", ha="center", va="center")
 offsets: dict[tuple[int, int], int] = {}
 for _, row in network.iterrows():
 left, right = int(row["chern_left"]), int(row["chern_right"])
 key = (left, right)
 count = offsets.get(key, 0)
 offsets[key] = count + 1
 x1, _ = positions[left]
 x2, _ = positions[right]
 rad = 0.25 + 0.16 * count
 ax.annotate(
 "",
 xy=(x2 - 0.18 * np.sign(x2 - x1), 0.0),
 xytext=(x1 + 0.18 * np.sign(x2 - x1), 0.0),
 arrowprops=dict(arrowstyle="->", connectionstyle=f"arc3,rad={rad}"),
)
 ax.text(
 0.5 * (x1 + x2),
 0.32 + 0.18 * count,
 f"{row['critical_k_region']} ΔC={int(row['delta_chern_up']):d}",
 ha="center", fontsize=8,
)
 ax.set_xlim(-0.6, len(nodes) - 0.4)
 ax.set_ylim(-0.6, 1.2)
 ax.set_aspect("equal")
 ax.axis("off")
 ax.set_title("TTS additive Chern transition network")
 fig.tight_layout()
 fig.savefig(path, dpi=220)
 plt.close(fig)

def plot_global_audit(audit: pd.DataFrame, path: Path) -> None:
 if audit.empty:
 return
 fig, ax = plt.subplots(figsize=(7.2, 6.0))
 for sector, group in audit.groupby("chern_up_int"):
 ax.scatter(group["m_Gamma_exact"], group["m_M_exact"], s=20, alpha=0.75, label=f"C={int(sector):d}")
 ax.axhline(0.0, linewidth=0.8)
 ax.axvline(0.0, linewidth=0.8)
 ax.set_xlabel("m_Gamma exact branch difference")
 ax.set_ylabel("m_M exact branch difference")
 ax.set_title("Global strict insulators: high-symmetry mass audit")
 ax.grid(True, alpha=0.25)
 ax.legend(ncol=2, fontsize=8)
 fig.tight_layout()
 fig.savefig(path, dpi=220)
 plt.close(fig)

=====
Main workflow
=====

def run_step06(config: Step06Config) -> dict[str, Any]:

```

---

```

config = config.normalized()
started = time.time()
out = config.output_dir
figures = out / "figures"

print("[1/8] Load Step 04/05 strict transition evidence")
tables = load_inputs(config)
print(f" transitions = {len(tables['critical'])}")

print("[2/8] Derive and validate exact Gamma/M parity branches")
blocks, branch_formulas = build_formula_tables()
validation = validate_exact_formulas(config)
atomic_write_csv(blocks, out / "step06_01_high_symmetry_parity_blocks.csv")
atomic_write_csv(branch_formulas, out / "step06_01_high_symmetry_branch_formulas.csv")
atomic_write_csv(validation, out / "step06_01_exact_formula_validation.csv")

print("[3/8] Build high-symmetry and generic-valley mass charts")
atlas, coefficients, branch_ids = build_transition_atlas(tables, config)
atomic_write_csv(atlas, out / "step06_02_transition_mass_atlas.csv")
atomic_write_csv(coefficients, out / "step06_02_generic_valley_mass_coefficients.csv")
atomic_write_csv(branch_ids, out / "step06_02_high_symmetry_branch_identification.csv")

print("[4/8] Construct additive Chern formulas on all Step 04 paths")
adaptive_pred, dense_pred, path_formulas, metrics = predict_path_chern(
 tables, atlas, coefficients, config
)
atomic_write_csv(path_formulas, out / "step06_03_path_additive_chern_formulas.csv")
atomic_write_csv(adaptive_pred, out / "step06_03_strict_path_chern_prediction.csv")
atomic_write_csv(dense_pred, out / "step06_03_dense_path_mass_phase_map.csv")
atomic_write_csv(metrics, out / "step06_03_path_prediction_metrics.csv")

print("[5/8] Build branch-resolved Chern transition network")
network = build_transition_network(atlas)
atomic_write_csv(network, out / "step06_04_additive_chern_transition_network.csv")

print("[6/8] Audit global information carried by m_Gamma and m_M")
global_audit, ambiguity = global_high_symmetry_audit(tables["step3_physics"])
if not global_audit.empty:
 atomic_write_csv(global_audit, out / "step06_05_global_high_symmetry_mass_audit.csv")
 atomic_write_csv(ambiguity, out / "step06_05_high_symmetry_sign_ambiguity.csv")

print("[7/8] Generate figures")
plot_formula_validation(validation, figures / "step06_exact_formula_validation.png")
plot_mass_charts(dense_pred, atlas, figures / "step06_transition_mass_charts.png")
plot_path_chern_prediction(
 dense_pred, adaptive_pred, atlas, figures / "step06_additive_chern_path_validation.png"
)
plot_transition_network(network, figures / "step06_additive_chern_network.png")
if not global_audit.empty:
 plot_global_audit(global_audit, figures / "step06_global_high_symmetry_mass_audit.png")

print("[8/8] Save summary and certificates")
high_sym = atlas[atlas["chart_type"] == "exact_high_symmetry_branch"]
generic = atlas[atlas["chart_type"] == "local_exact_linear_diabatic_mass"]
all_accuracy = float(metrics.loc[metrics["path_id"] == "ALL_PATHS", "accuracy"].iloc[0])
formula_max_error = float(validation["eigenvalue_max_abs_error"].max())
single_global_mass_supported = False
if not coefficients.empty:
 vectors = []
 for _, group in coefficients.groupby("transition_id"):
 v = group.set_index("parameter").loc[REDUCED7, "oriented_mass_coefficient"].to_numpy(float)
 vectors.append(v / np.linalg.norm(v))
 if len(vectors) >= 2:
 similarities = [abs(float(np.dot(a, b))) for i, a in enumerate(vectors) for b in vectors[i + 1:]]
 single_global_mass_supported = bool(min(similarities) > 0.95)

certificate = pd.DataFrame([
 "exact_high_symmetry_formula_max_error": formula_max_error,
 "all_high_symmetry_branch_crossings_resolved": int(len(high_sym) == 4),

```

---

```

 "all_generic_charts_sign_flip": int(generic["mass_sign_flip_verified"].eq(1).all()),
 "all_critical_mass_residual_below_tol":
 ↪ int(atlas["critical_mass_abs_residual"].lt(config.branch_degeneracy_tol).all()),
 "all_step05_mechanism_certificates_pass": int(atlas["mechanism_certificate_pass"].eq(1).all()),
 "strict_path_additive_chern_accuracy": all_accuracy,
 "all_strict_path_chern_points_matched": int(all_accuracy == 1.0),
 "single_global_generic_mass_coordinate_supported": int(single_global_mass_supported),
 "final_certificate_pass": int(
 formula_max_error < 1.0e-12
 and atlas["mass_sign_flip_verified"].eq(1).all()
 and atlas["mechanism_certificate_pass"].eq(1).all()
 and all_accuracy == 1.0
),
})
atomic_write_csv(certificate, out / "step06_06_analytic_mass_chern_certificate.csv")

summary = {
 "code_version": CODE_VERSION,
 "step01_version": core.CODE_VERSION,
 "step4_input": str(config.step4_input),
 "step5_input": str(config.step5_input),
 "step3_input": str(config.step3_input) if config.step3_input else None,
 "n_transition_charts": len(atlas),
 "n_exact_high_symmetry_charts": len(high_sym),
 "n_generic_valley_charts": len(generic),
 "n_strict_path_chern_points": len(adaptive_pred),
 "strict_path_additive_chern_accuracy": all_accuracy,
 "exact_formula_max_abs_error": formula_max_error,
 "single_global_generic_mass_coordinate_supported": single_global_mass_supported,
 "final_certificate_pass": int(certificate.iloc[0]["final_certificate_pass"]),
 "elapsed_seconds": time.time() - started,
 "configuration": asdict(config),
}
atomic_write_json(summary, out / "step06_07_run_summary.json")
print(f" certificate pass = {summary['final_certificate_pass']}")
print(f" strict path accuracy = {all_accuracy:.6f}")
print(f" elapsed = {summary['elapsed_seconds']:.1f} s")
return {
 "summary": summary,
 "atlas": atlas,
 "certificate": certificate,
 "metrics": metrics,
 "network": network,
}

=====
CLI
=====

def build_config_from_args(args: argparse.Namespace) -> Step06Config:
 return Step06Config(
 output_dir=Path(args.output_dir),
 step4_input=Path(args.step4_input) if args.step4_input else None,
 step5_input=Path(args.step5_input) if args.step5_input else None,
 step3_input=Path(args.step3_input) if args.step3_input else None,
 formula_validation_samples=24 if args.quick else args.formula_validation_samples,
 global_audit=not args.skip_global_audit,
)

def main() -> None:
 parser = argparse.ArgumentParser(description=__doc__)
 parser.add_argument(
 "--output-dir",
 default="outputs_tts_step06_analytic_mass_branch_chern_atlas",
)
 parser.add_argument("--step4-input", default=None)

```

```

parser.add_argument("--step5-input", default=None)
parser.add_argument("--step3-input", default=None)
parser.add_argument("--formula-validation-samples", type=int, default=128)
parser.add_argument("--skip-global-audit", action="store_true")
parser.add_argument("--quick", action="store_true")
args = parser.parse_args()
result = run_step06(build_config_from_args(args))
print(json.dumps(result["summary"], indent=2, ensure_ascii=False, default=str))

if __name__ == "__main__":
 main()

```

## ► 代码清单 A.49.7 (python, 1360 行)

```

#!/usr/bin/env python3
-*- coding: utf-8 -*-
"""
TTS Step 07 — final observable validation
=====

Final validation of the tts eight-band altermagnetic Hamiltonian through:

1. bulk bands on the square-lattice high-symmetry path;
2. finite ribbons for axial and diagonal terminations;
3. semi-infinite surface spectral functions with spin resolution;
4. bulk DOS around the global insulating gap;
5. energy-resolved intrinsic spin Hall conductivity (SHC);
6. an observable certificate for $C_{\text{up}} = 0, \pm 1, \pm 2$ representatives.

Plotting conventions follow the uploaded WannierTools ``src`` style:
- thick borders and large labels;
- ribbon edge weight: green -> yellow -> red;
- spin-resolved surface spectrum: blue -> white -> red;
- DOS and conductivity: clean thick-line panels;
- 1920x1680-like publication aspect ratios at high DPI.

The code is serial and Windows/Jupyter safe. It does not use ProcessPoolExecutor.
"""

from __future__ import annotations

from dataclasses import asdict, dataclass
from datetime import datetime, timezone
from pathlib import Path
from typing import Any, Dict, Iterable, Sequence
import argparse
import io
import json
import math
import os
import time
import zipfile

Dense eigensolvers become extremely slow on some Windows/Jupyter and CI
environments when BLAS starts many threads for small/medium matrices.
Keep these calculations deterministic and avoid thread oversubscription.
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.environ.setdefault("OMP_NUM_THREADS", "1")
os.environ.setdefault("MKL_NUM_THREADS", "1")
os.environ.setdefault("NUMEXPR_NUM_THREADS", "1")

import matplotlib as mpl

```

---

```

import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap, Normalize, TwoSlopeNorm
import numpy as np
import pandas as pd
from scipy.linalg import eigh
from scipy.signal import find_peaks
from threadpoolctl import threadpool_limits

Apply the limit even when NumPy/SciPy was imported earlier by a notebook.
_THREADPOOL_LIMITER = threadpool_limits(limits=1)

import TTS_step01_model_and_label_audit_v2 as step1

CODE_VERSION = "TTS_STEP07_V1_20260714"
PARAM7 = tuple(step1.REDUCED7)
PARAM8 = tuple(step1.RAW8)

Two boundary orientations. Columns are real-space supercell vectors A1, A2.
A1 is periodic along the ribbon; A2 is finite / surface-normal.
EDGE_SPECS: Dict[str, Dict[str, Any]] = {
 "axial_x": {
 "label": r"axial edge $A_1=(1,0)$",
 "A": np.array([[1, 0], [0, 1]], dtype=int),
 },
 "diagonal_11": {
 "label": r"diagonal edge $A_1=(1,1)$",
 "A": np.array([[1, 1], [1, -1]], dtype=int),
 },
}

WannierTools-like palettes inferred from uploaded src.
WT_EDGE_CMAP = LinearSegmentedColormap.from_list(
 "wt_edge_green_yellow_red", ["green", "yellow", "red"]
)
WT_SPIN_CMAP = LinearSegmentedColormap.from_list(
 "wt_spin_blue_white_red", ["#194eff", "white", "red"]
)

@dataclass
class Step07Config:
 step3_input: Path = Path("outputs_tts_step03_global_sobol_hierarchical_topology_ml.zip")
 output_dir: Path = Path("outputs_tts_step07_observable_validation")

 # Representative Chern sectors. The paper parameter set is added as C=0.
 chern_sectors: tuple[int, ...] = (-2, -1, 1, 2)

 # Bulk and response grids.
 gap_nk: int = 101
 band_points_per_segment: int = 70
 response_nk: int = 81
 response_energy_points: int = 321
 dos_sigma_fraction_of_gap: float = 0.025
 minimum_dos_sigma: float = 0.002

 # Ribbon calculations.
 ribbon_k_points: int = 161
 ribbon_width_axial: int = 36
 ribbon_width_diagonal: int = 24
 ribbon_edge_layers: int = 3
 ribbon_plot_margin_factor: float = 0.75
 ribbon_min_plot_margin: float = 0.35
 ribbon_edge_weight_threshold: float = 0.25

 # Surface Green functions.
 surface_k_points: int = 141
 surface_energy_points: int = 181
 surface_eta_fraction_of_gap: float = 0.012

```

---

```

surface_min_eta: float = 0.002
surface_max_iter: int = 80
surface_tol: float = 1.0e-12
surface_peak_prominence_fraction: float = 0.08

Fourier decomposition of periodic Hamiltonian.
fourier_n: int = 8
fourier_tolerance: float = 1.0e-10

Observable certificate tolerances.
shc_plateau_tolerance: float = 0.08
charge_hall_tolerance: float = 0.08
normalized_gap_dos_tolerance: float = 0.06

dpi: int = 240
quick: bool = False
force_recalculate: bool = False

def normalized(self) -> "Step07Config":
 self.step3_input = Path(self.step3_input)
 self.output_dir = Path(self.output_dir)
 self.output_dir.mkdir(parents=True, exist_ok=True)
 (self.output_dir / "figures").mkdir(parents=True, exist_ok=True)
 (self.output_dir / "data").mkdir(parents=True, exist_ok=True)

 if self.quick:
 self.chern_sectors = (-1, 2)
 self.gap_nk = min(self.gap_nk, 41)
 self.band_points_per_segment = min(self.band_points_per_segment, 24)
 self.response_nk = min(self.response_nk, 31)
 self.response_energy_points = min(self.response_energy_points, 101)
 self.ribbon_k_points = min(self.ribbon_k_points, 41)
 self.ribbon_width_axial = min(self.ribbon_width_axial, 14)
 self.ribbon_width_diagonal = min(self.ribbon_width_diagonal, 10)
 self.ribbon_edge_layers = min(self.ribbon_edge_layers, 2)
 self.surface_k_points = min(self.surface_k_points, 41)
 self.surface_energy_points = min(self.surface_energy_points, 61)
 self.surface_max_iter = min(self.surface_max_iter, 50)
 self.dpi = min(self.dpi, 150)

 for name in (
 "gap_nk", "band_points_per_segment", "response_nk",
 "response_energy_points", "ribbon_k_points", "surface_k_points",
 "surface_energy_points", "fourier_n",
):
 if int(getattr(self, name)) < 5:
 raise ValueError(f"{name} must be >= 5")
 return self

General utilities

def utc_now() -> str:
 return datetime.now(timezone.utc).isoformat()

def atomic_write_csv(df: pd.DataFrame, path: Path) -> None:
 path = Path(path)
 tmp = path.with_suffix(path.suffix + ".tmp")
 df.to_csv(tmp, index=False)
 tmp.replace(path)

def atomic_write_json(payload: Dict[str, Any], path: Path) -> None:
 path = Path(path)
 tmp = path.with_suffix(path.suffix + ".tmp")
 tmp.write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")

```

---

```

tmp.replace(path)

def wrap_angle(x: float) -> float:
 return float((float(x) + np.pi) % (2.0 * np.pi) - np.pi)

def configure_plot_style() -> None:
 mpl.rcParams.update(
 {
 "font.family": "DejaVu Sans",
 "font.size": 12,
 "axes.linewidth": 2.0,
 "axes.labelsize": 14,
 "axes.titlesize": 14,
 "xtick.labelsize": 11,
 "ytick.labelsize": 11,
 "xtick.major.width": 1.6,
 "ytick.major.width": 1.6,
 "xtick.major.size": 5,
 "ytick.major.size": 5,
 "legend.fontsize": 10,
 "savefig.bbox": "tight",
 }
)

def resolve_existing_path(path: Path) -> Path:
 path = Path(path)
 if path.exists():
 return path
 candidates = sorted(Path.cwd().glob(f"*{path.name}*"))
 if candidates:
 return candidates[0]
 raise FileNotFoundError(path)

def read_matching_csv_from_zip(zip_path: Path, keyword: str) -> pd.DataFrame:
 with zipfile.ZipFile(zip_path) as zf:
 names = [n for n in zf.namelist() if keyword in n and n.lower().endswith(".csv")]
 if not names:
 raise FileNotFoundError(f"No CSV containing {keyword!r} in {zip_path}")
 if len(names) > 1:
 names.sort(key=len)
 return pd.read_csv(zf.open(names[0]))

def raw_params_from_row(row: pd.Series) -> Dict[str, float]:
 if all(name in row.index for name in PARAM8):
 return {name: float(row[name]) for name in PARAM8}
 reduced = {name: float(row[name]) for name in PARAM7}
 return step1.raw8_from_reduced7(reduced)

def select_representatives(step3_zip: Path, sectors: Sequence[int]) -> pd.DataFrame:
 """Select the largest-gap strict representative in each requested sector."""
 best = read_matching_csv_from_zip(step3_zip, "best_gap_candidate_by_chern_sector")
 rows: list[dict[str, Any]] = []
 for c in sectors:
 subset = best[np.rint(best["chern_up_int"]).astype(int) == int(c)]
 if subset.empty:
 strict = read_matching_csv_from_zip(step3_zip, "strict_spin_chern_TI_candidates")
 subset = strict[np.rint(strict["chern_up_int"]).astype(int) == int(c)]
 if subset.empty:
 raise RuntimeError(f"No strict representative found for C_up={c}")
 row = subset.sort_values("indirect_gap", ascending=False).iloc[0]
 rec = {key: row[key] for key in row.index}
 rec["representative_label"] = f"C_up_{int(c):+d}"
 rec["expected_chern_up"] = int(c)

```

---

```

 rec["expected_chern_down"] = -int(c)
 rec["selection_reason"] = "largest strict indirect gap in Step 03"
 rows.append(rec)

The exact literature parameter set is the clean trivial control.
paper_gap = step1.scan_band_gaps(step1.PAPER_PARAMS, nk=81, shift=(0.0, 0.0))
paper = {
 "sample_id": "paper_trivial_exact",
 "sample_source": "literature_parameter_control",
 "representative_label": "C_up+0",
 "expected_chern_up": 0,
 "expected_chern_down": 0,
 "selection_reason": "exact tts literature parameter set",
 **step1.reduced7_from_raw8(step1.PAPER_PARAMS),
 **step1.PAPER_PARAMS,
 **paper_gap,
}
rows.append(paper)

result = pd.DataFrame(rows)
Put trivial first, followed by increasing Chern.
order = {0: 0, -1: 1, 1: 2, -2: 3, 2: 4}
result["_order"] = result["expected_chern_up"].map(order).fillna(99)
result = result.sort_values("_order").drop(columns="_order").reset_index(drop=True)
return result

Finite Fourier representation $H(k) = \sum_R H_R \exp(i \cdot k \cdot R)$

def extract_hopping_matrices(
 params: Dict[str, float],
 nfft: int = 8,
 tolerance: float = 1.0e-10,
) -> Dict[tuple[int, int], np.ndarray]:
 nfft = int(nfft)
 ks = 2.0 * np.pi * np.arange(nfft) / nfft
 hk = np.empty((nfft, nfft, 8, 8), dtype=np.complex128)
 for ix, kx in enumerate(ks):
 for iy, ky in enumerate(ks):
 hk[ix, iy] = step1.h_tts_periodic(float(kx), float(ky), params)

 coeff = np.fft.fft2(hk, axes=(0, 1)) / float(nfft * nfft)
 hoppings: Dict[tuple[int, int], np.ndarray] = {}
 for ix in range(nfft):
 rx = ix if ix <= nfft // 2 else ix - nfft
 for iy in range(nfft):
 ry = iy if iy <= nfft // 2 else iy - nfft
 mat = coeff[ix, iy]
 if float(np.max(np.abs(mat))) > float(tolerance):
 hoppings[(int(rx), int(ry))] = np.asarray(mat, dtype=np.complex128)

Hard reconstruction audit.
for kx, ky in ((0.173, -1.029), (2.037, 0.691), (-2.4, 2.1)):
 rec = np.zeros((8, 8), dtype=np.complex128)
 for (rx, ry), mat in hoppings.items():
 rec += mat * np.exp(1j * (kx * rx + ky * ry))
 ref = step1.h_tts_periodic(kx, ky, params)
 err = float(np.max(np.abs(rec - ref)))
 if err > 1.0e-9:
 raise RuntimeError(f"Fourier reconstruction failed: {err:.3e}")
return hoppings

def bloch_from_hoppings(
 kx: float,
 ky: float,
 hoppings: Dict[tuple[int, int], np.ndarray],

```



---

```

) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
 h = np.zeros((8, 8), dtype=np.complex128)
 vx = np.zeros_like(h)
 vy = np.zeros_like(h)
 for (rx, ry), mat in hoppings.items():
 phase = np.exp(1j * (float(kx) * rx + float(ky) * ry))
 h += mat * phase
 vx += 1j * rx * mat * phase
 vy += 1j * ry * mat * phase
 h = 0.5 * (h + h.conj().T)
 vx = 0.5 * (vx + vx.conj().T)
 vy = 0.5 * (vy + vy.conj().T)
 return h, vx, vy

Supercell / ribbon geometry

def integer_coset_representatives(A: np.ndarray) -> list[np.ndarray]:
 A = np.asarray(A, dtype=int)
 det = abs(int(round(np.linalg.det(A))))
 if det < 1:
 raise ValueError("Supercell matrix must be nonsingular")
 invA = np.linalg.inv(A.astype(float))

 candidates = [np.array([0, 0], dtype=int)]
 radius = 1
 while len(candidates) < 20 * det:
 for x in range(-radius, radius + 1):
 for y in range(-radius, radius + 1):
 candidates.append(np.array([x, y], dtype=int))
 radius += 1
 if radius > 8:
 break

 reps: list[np.ndarray] = []
 for candidate in candidates:
 equivalent = False
 for rep in reps:
 delta = invA @ (candidate - rep)
 if np.allclose(delta, np rint(delta), atol=1.0e-10):
 equivalent = True
 break
 if not equivalent:
 reps.append(candidate.copy())
 if len(reps) == det:
 break
 if len(reps) != det:
 raise RuntimeError("Failed to construct integer supercell cosets")
 return reps

def locate_coset(q: np.ndarray, A: np.ndarray, reps: Sequence[np.ndarray]) -> tuple[int, np.ndarray]:
 invA = np.linalg.inv(np.asarray(A, dtype=float))
 for i, rep in enumerate(reps):
 delta = invA @ (np.asarray(q, dtype=float) - np.asarray(rep, dtype=float))
 rounded = np rint(delta)
 if np.allclose(delta, rounded, atol=1.0e-9):
 return int(i), rounded.astype(int)
 raise RuntimeError(f"Unable to map primitive cell {q} into supercell")

def transformed_principal_layer_blocks(
 k_parallel: float,
 hoppings: Dict[tuple[int, int], np.ndarray],
 A: np.ndarray,
) -> tuple[np.ndarray, np.ndarray, np.ndarray, list[np.ndarray]]:
 """Return H0, H(+normal), H(-normal) for one principal layer."""

```

---

```

A = np.asarray(A, dtype=int)
reps = integer_coset_representatives(A)
ncoset = len(reps)
n = 8 * ncoset
blocks = {-1: np.zeros((n, n), complex), 0: np.zeros((n, n), complex), 1: np.zeros((n, n), complex)}

for ia, ca in enumerate(reps):
 for (rx, ry), mat in hoppings.items():
 q = ca + np.array([rx, ry], dtype=int)
 ib, delta = locate_coset(q, A, reps)
 dpar, dperp = int(delta[0]), int(delta[1])
 if dperp not in blocks:
 raise RuntimeError(
 f"Principal layer too thin: normal displacement {dperp}; A={A.tolist()}"
)
 rs = slice(8 * ia, 8 * (ia + 1))
 cs = slice(8 * ib, 8 * (ib + 1))
 blocks[dperp][rs, cs] += mat * np.exp(1j * float(k_parallel) * dpar)

h0 = 0.5 * (blocks[0] + blocks[0].conj().T)
hp = 0.5 * (blocks[1] + blocks[-1].conj().T)
hm = hp.conj().T
return h0, hp, hm, reps

def ribbon_hamiltonian(
 k_parallel: float,
 width: int,
 hoppings: Dict[tuple[int, int], np.ndarray],
 A: np.ndarray,
) -> tuple[np.ndarray, int]:
 h0, hp, hm, reps = transformed_principal_layer_blocks(k_parallel, hoppings, A)
 layer_dim = h0.shape[0]
 width = int(width)
 h = np.zeros((layer_dim * width, layer_dim * width), dtype=np.complex128)
 for layer in range(width):
 sl = slice(layer * layer_dim, (layer + 1) * layer_dim)
 h[sl, sl] = h0
 if layer + 1 < width:
 sr = slice((layer + 1) * layer_dim, (layer + 2) * layer_dim)
 h[sl, sr] = hp
 h[sr, sl] = hm
 h = 0.5 * (h + h.conj().T)
 return h, layer_dim

def spin_operator_for_layer(ncoset: int) -> np.ndarray:
 primitive = np.diag([1.0 if i in step1.SPIN_UP_INDICES else -1.0 for i in range(8)])
 return np.kron(np.eye(int(ncoset)), primitive)

def projected_bulk_bands(
 k_parallel_values: np.ndarray,
 hoppings: Dict[tuple[int, int], np.ndarray],
 A: np.ndarray,
 n_perp: int = 61,
) -> tuple[np.ndarray, np.ndarray]:
 invAT = np.linalg.inv(np.asarray(A, dtype=float).T)
 kperp_values = np.linspace(-np.pi, np.pi, int(n_perp), endpoint=False)
 records_k: list[float] = []
 records_e: list[float] = []
 for kp in k_parallel_values:
 for kn in kperp_values:
 kx, ky = invAT @ np.array([kp, kn], dtype=float)
 h, _, _ = bloch_from_hoppings(float(kx), float(ky), hoppings)
 energies = np.linalg.eigvalsh(h)
 records_k.extend([float(kp)] * len(energies))
 records_e.extend([float(x) for x in energies])
 return np.asarray(records_k), np.asarray(records_e)

```

---

```

Bulk bands, ribbon spectra and edge crossing diagnostics

def high_symmetry_path(n_per_segment: int) -> tuple[np.ndarray, np.ndarray, list[float], list[str]]:
 points = [
 ("Γ", (0.0, 0.0)),
 ("X", (np.pi, 0.0)),
 ("M", (np.pi, np.pi)),
 ("Γ", (0.0, 0.0)),
 ("M'", (-np.pi, np.pi)),
 ("X'", (0.0, np.pi)),
 ("Γ", (0.0, 0.0)),
]
 klist: list[tuple[float, float]] = []
 distance: list[float] = []
 tick_positions: list[float] = [0.0]
 labels = [points[0][0]]
 current = 0.0
 previous: np.ndarray | None = None
 for iseg in range(len(points) - 1):
 p0 = np.asarray(points[iseg][1], float)
 p1 = np.asarray(points[iseg + 1][1], float)
 for j in range(int(n_per_segment)):
 if iseg > 0 and j == 0:
 continue
 t = j / float(n_per_segment)
 k = (1.0 - t) * p0 + t * p1
 if previous is not None:
 current += float(np.linalg.norm(k - previous))
 klist.append((float(k[0]), float(k[1])))
 distance.append(current)
 previous = k
 k = p1
 current += float(np.linalg.norm(k - previous)) if previous is not None else 0.0
 klist.append((float(k[0]), float(k[1])))
 distance.append(current)
 previous = k
 tick_positions.append(current)
 labels.append(points[iseg + 1][0])
 return np.asarray(klist), np.asarray(distance), tick_positions, labels

def calculate_bulk_band_data(
 params: Dict[str, float],
 ef: float,
 n_per_segment: int,
) -> pd.DataFrame:
 klist, distance, _, _ = high_symmetry_path(n_per_segment)
 rows: list[dict[str, Any]] = []
 for ik, ((kx, ky), xpos) in enumerate(zip(klist, distance)):
 h = step1.h_tts_atomic(float(kx), float(ky), params)
 evals, evecs = np.linalg.eigh(h)
 for band, energy in enumerate(evals):
 spin = float(np.sum(np.abs(evecs[step1.SPIN_UP_INDICES, band]) ** 2) -
 ↳ np.sum(np.abs(evecs[step1.SPIN_DOWN_INDICES, band]) ** 2))
 rows.append(
 {
 "k_index": ik,
 "path_coordinate": float(xpos),
 "kx": float(kx),
 "ky": float(ky),
 "band": int(band + 1),
 "energy": float(energy - ef),
 "spin_z": spin,
 }
)
)

```

---

```

return pd.DataFrame(rows)

def calculate_ribbon_data(
 params: Dict[str, float],
 hoppings: Dict[tuple[int, int], np.ndarray],
 edge_name: str,
 width: int,
 k_points: int,
 edge_layers: int,
 ef: float,
 vbm_rel: float,
 cbm_rel: float,
 edge_weight_threshold: float,
) -> tuple[pd.DataFrame, Dict[str, Any]]:
 A = np.asarray(EDGE_SPECS[edge_name]["A"], int)
 reps = integer_coset_representatives(A)
 ncset = len(reps)
 sz_layer = spin_operator_for_layer(ncset)
 kvalues = np.linspace(-np.pi, np.pi, int(k_points), endpoint=True)
 rows: list[dict[str, Any]] = []

 for ik, kp in enumerate(kvalues):
 h, layer_dim = ribbon_hamiltonian(float(kp), int(width), hoppings, A)
 evals, evects = eigh(h, overwrite_a=True, check_finite=False)
 edge_layers_eff = min(int(edge_layers), max(1, int(width) // 3))
 left_dim = edge_layers_eff * layer_dim
 right_start = (int(width) - edge_layers_eff) * layer_dim

 probability = np.abs(evects) ** 2
 left_weight = np.sum(probability[:left_dim, :], axis=0)
 right_weight = np.sum(probability[right_start:, :], axis=0)
 total_edge = left_weight + right_weight

 spin_diag = np.tile(np.diag(sz_layer), int(width))
 spin_z = np.real(np.sum(probability * spin_diag[:, None], axis=0))

 for ib, energy in enumerate(evals):
 rows.append(
 {
 "edge_name": edge_name,
 "k_index": int(ik),
 "k_parallel": float(kp),
 "band": int(ib + 1),
 "energy": float(energy - ef),
 "left_edge_weight": float(left_weight[ib]),
 "right_edge_weight": float(right_weight[ib]),
 "total_edge_weight": float(total_edge[ib]),
 "spin_z": float(spin_z[ib]),
 "is_global_gap_state": int(vbm_rel < float(energy - ef) < cbm_rel),
 "is_edge_localized": int(float(total_edge[ib]) >= float(edge_weight_threshold)),
 }
)

 df = pd.DataFrame(rows)
 ingap = df[(df["is_global_gap_state"] == 1) & (df["is_edge_localized"] == 1)]
 summary = {
 "edge_name": edge_name,
 "width": int(width),
 "principal_layer_orbitals": int(8 * ncset),
 "in_gap_edge_state_records": int(len(ingap)),
 "in_gap_spin_up_records": int(np.count_nonzero(ingap["spin_z"].to_numpy(float) > 0.5)),
 "in_gap_spin_down_records": int(np.count_nonzero(ingap["spin_z"].to_numpy(float) < -0.5)),
 "max_in_gap_edge_weight": float(ingap["total_edge_weight"].max()) if not ingap.empty else 0.0,
 "edge_states_detected": int(not ingap.empty),
 }

 return df, summary

```

---

```

Semi-infinite surface Green function

def sancho_surface_green(
 energy: float,
 eta: float,
 h0: np.ndarray,
 hp: np.ndarray,
 max_iter: int,
 tol: float,
) -> np.ndarray:
 z = complex(float(energy), float(eta))
 n = h0.shape[0]
 identity = np.eye(n, dtype=np.complex128)

 eps = h0.copy()
 eps_surface = h0.copy()
 alpha = hp.copy()
 beta = hp.conj().T

 for _ in range(int(max_iter)):
 g = np.linalg.inv(z * identity - eps)
 agb = alpha @ g @ beta
 bga = beta @ g @ alpha
 eps_surface = eps_surface + agb
 eps = eps + agb + bga
 alpha_new = alpha @ g @ alpha
 beta_new = beta @ g @ beta
 alpha, beta = alpha_new, beta_new
 if max(float(np.linalg.norm(alpha, ord="fro")), float(np.linalg.norm(beta, ord="fro"))) < float(tol):
 break
 return np.linalg.inv(z * identity - eps_surface)

def calculate_surface_spectrum(
 hoppings: Dict[tuple[int, int], np.ndarray],
 edge_name: str,
 k_points: int,
 energies: np.ndarray,
 ef: float,
 eta: float,
 max_iter: int,
 tol: float,
) -> tuple[Dict[str, np.ndarray], Dict[str, Any]]:
 A = np.asarray(EDGE_SPECS[edge_name]["A"], int)
 reps = integer_coset_representatives(A)
 ncset = len(reps)
 sz = spin_operator_for_layer(ncset)
 up_mask = np.diag((np.diag(sz) > 0).astype(float))
 down_mask = np.diag((np.diag(sz) < 0).astype(float))

 kvalues = np.linspace(-np.pi, np.pi, int(k_points), endpoint=True)
 energies_abs = np.asarray(energies, float) + float(ef)
 total = np.zeros((len(energies), len(kvalues)), dtype=float)
 spin = np.zeros_like(total)
 up = np.zeros_like(total)
 down = np.zeros_like(total)

 for ik, kp in enumerate(kvalues):
 h0, hp, _, _ = transformed_principal_layer_blocks(float(kp), hoppings, A)
 for ie, energy in enumerate(energies_abs):
 g = sancho_surface_green(float(energy), eta, h0, hp, max_iter, tol)
 total[ie, ik] = max(0.0, float(-np.imag(np.trace(g)) / np.pi))
 up[ie, ik] = max(0.0, float(-np.imag(np.trace(up_mask @ g)) / np.pi))
 down[ie, ik] = max(0.0, float(-np.imag(np.trace(down_mask @ g)) / np.pi))
 spin[ie, ik] = float(up[ie, ik] - down[ie, ik])

 # Midgap peak count by spin. These peaks are a direct surface-state signature.

```

---

```

imid = int(np.argmin(np.abs(energies)))
peak_counts: Dict[str, int] = {}
for name, curve in (("up", up[imid]), ("down", down[imid]), ("total", total[imid])):
 scale = max(float(np.max(curve)), 1.0e-14)
 prominence = max(1.0e-12, 0.08 * scale)
 peaks, _ = find_peaks(curve, prominence=prominence, distance=max(1, len(kvalues) // 50))
 peak_counts[name] = int(len(peaks))

intensity = np.log1p(total)
normalized_intensity = intensity / max(float(np.max(intensity)), 1.0e-14)
polarization = spin / np.maximum(total, 1.0e-14)
spin_field = np.clip(polarization, -1.0, 1.0) * normalized_intensity

data = {
 "k_parallel": kvalues,
 "energy": np.asarray(energies, float),
 "surface_dos": total,
 "surface_dos_up": up,
 "surface_dos_down": down,
 "spin_polarization": polarization,
 "spin_weighted_log_intensity": spin_field,
}
summary = {
 "edge_name": edge_name,
 "surface_eta": float(eta),
 "midgap_peak_count_up": peak_counts["up"],
 "midgap_peak_count_down": peak_counts["down"],
 "midgap_peak_count_total": peak_counts["total"],
 "max_surface_dos": float(np.max(total)),
}
return data, summary

Bulk DOS and energy-resolved Hall responses

def band_berry_curvature(
 h: np.ndarray,
 vx: np.ndarray,
 vy: np.ndarray,
 denominator_floor: float = 1.0e-12,
) -> tuple[np.ndarray, np.ndarray]:
 energies, vectors = np.linalg.eigh(h)
 n = len(energies)
 omega = np.zeros(n, dtype=float)
 # Sign chosen to match the determinant-link Fukui convention used in Steps 01-06.
 for ib in range(n):
 value = 0.0
 for jb in range(n):
 if ib == jb:
 continue
 de = float(energies[ib] - energies[jb])
 denom = max(de * de, float(denominator_floor))
 a = np.vdot(vectors[:, ib], vx @ vectors[:, jb])
 b = np.vdot(vectors[:, jb], vy @ vectors[:, ib])
 value += 2.0 * float(np.imag(a * b)) / denom
 omega[ib] = value
 return energies, omega

def calculate_dos_and_hall(
 hoppings: Dict[tuple[int, int], np.ndarray],
 ef: float,
 energy_grid: np.ndarray,
 nk: int,
 dos_sigma: float,
) -> tuple[pd.DataFrame, Dict[str, Any]]:
 all_energies: list[float] = []

```

---

```

spin_records: Dict[str, list[tuple[float, float]]] = {"up": [], "down": []}
area_weight = (2.0 * np.pi / int(nk)) ** 2 / (2.0 * np.pi)
kvalues = -np.pi + 2.0 * np.pi * (np.arange(int(nk)) + 0.5) / int(nk)

for kx in kvalues:
 for ky in kvalues:
 h, vx, vy = bloch_from_hoppings(float(kx), float(ky), hoppings)
 evals = np.linalg.eigvalsh(h)
 all_energies.extend([float(e - ef) for e in evals])
 for spin_name, indices in (
 ("up", step1.SPIN_UP_INDICES),
 ("down", step1.SPIN_DOWN_INDICES),
):
 idx = np.asarray(indices, int)
 es, omega = band_berry_curvature(
 h[np.ix_(idx, idx)],
 vx[np.ix_(idx, idx)],
 vy[np.ix_(idx, idx)],
)
 for energy, curvature in zip(es, omega):
 spin_records[spin_name].append((float(energy - ef), float(curvature * area_weight)))

energy_grid = np.asarray(energy_grid, float)
all_e = np.asarray(all_energies, float)
sigma = max(float(dos_sigma), 1.0e-8)
dos = np.zeros_like(energy_grid)
prefactor = 1.0 / (math.sqrt(2.0 * math.pi) * sigma * int(nk) * int(nk))
Chunk the Gaussian evaluation to keep memory bounded.
for start in range(0, len(all_e), 1024):
 chunk = all_e[start:start + 1024]
 x = (energy_grid[:, None] - chunk[None, :]) / sigma
 dos += prefactor * np.sum(np.exp(-0.5 * x * x), axis=1)

cumulative: Dict[str, np.ndarray] = {}
plateau_chern: Dict[str, float] = {}
for spin_name, records in spin_records.items():
 rec = np.asarray(records, dtype=float)
 order = np.argsort(rec[:, 0])
 energies_sorted = rec[order, 0]
 weights_sorted = rec[order, 1]
 cumulative_weight = np.cumsum(weights_sorted)
 indices = np.searchsorted(energies_sorted, energy_grid, side="right") - 1
 values = np.zeros_like(energy_grid)
 valid = indices >= 0
 values[valid] = cumulative_weight[indices[valid]]
 cumulative[spin_name] = values
 mid = int(np.argmin(np.abs(energy_grid)))
 plateau_chern[spin_name] = float(values[mid])

charge_hall = cumulative["up"] + cumulative["down"]
Dimensionless spin Hall conductivity in units e/(2*pi).
spin_hall = 0.5 * (cumulative["up"] - cumulative["down"])
df = pd.DataFrame(
 {
 "energy": energy_grid,
 "dos": dos,
 "chern_up_below_energy": cumulative["up"],
 "chern_down_below_energy": cumulative["down"],
 "charge_hall_e2_over_h": charge_hall,
 "spin_hall_e_over_2pi": spin_hall,
 }
)
summary = {
 "response_nk": int(nk),
 "dos_sigma": float(sigma),
 "midgap_chern_up_kubo": plateau_chern["up"],
 "midgap_chern_down_kubo": plateau_chern["down"],
 "midgap_charge_hall_e2_over_h": float(charge_hall[np.argmin(np.abs(energy_grid))]),
 "midgap_spin_hall_e_over_2pi": float(spin_hall[np.argmin(np.abs(energy_grid))]),
}

```

---

```

 }
 return df, summary

Plotting

def add_gap_shading(ax: plt.Axes, vbm: float, cbm: float) -> None:
 ax.axhspan(float(vbm), float(cbm), color="0.92", zorder=0)
 ax.axhline(0.0, color="0.25", lw=1.2, ls="--", zorder=1)

def plot_bulk_bands(
 df: pd.DataFrame,
 ticks: Sequence[float],
 labels: Sequence[str],
 vbm: float,
 cbm: float,
 title: str,
 output: Path,
 dpi: int,
) -> None:
 fig, ax = plt.subplots(figsize=(7.2, 5.5))
 add_gap_shading(ax, vbm, cbm)
 for band in sorted(df["band"].unique()):
 sub = df[df["band"] == band]
 ax.plot(sub["path_coordinate"], sub["energy"], color="black", lw=1.2)
 for x in ticks[1:-1]:
 ax.axvline(float(x), color="0.7", lw=0.8)
 ax.set_xticks(ticks, labels)
 ax.set_xlim(float(min(ticks)), float(max(ticks)))
 margin = max(0.35, 0.7 * float(cbm - vbm))
 ax.set_ylim(float(vbm - margin), float(cbm + margin))
 ax.set_ylabel("Energy relative to midgap")
 ax.set_title(title)
 for spine in ax.spines.values():
 spine.set_linewidth(2.0)
 fig.savefig(output, dpi=dpi)
 plt.close(fig)

def plot_ribbon(
 df: pd.DataFrame,
 bulk_k: np.ndarray,
 bulk_e: np.ndarray,
 vbm: float,
 cbm: float,
 title: str,
 output: Path,
 dpi: int,
) -> None:
 fig, ax = plt.subplots(figsize=(7.5, 6.0))
 add_gap_shading(ax, vbm, cbm)
 margin = max(0.35, 0.75 * float(cbm - vbm))
 ymin, ymax = float(vbm - margin), float(cbm + margin)
 mask_bulk = (bulk_e >= ymin) & (bulk_e <= ymax)
 ax.scatter(bulk_k[mask_bulk], bulk_e[mask_bulk], s=1.0, color="0.83", alpha=0.35, rasterized=True)

 mask = (df["energy"] >= ymin) & (df["energy"] <= ymax)
 sub = df[mask]
 weight = np.clip(sub["total_edge_weight"].to_numpy(float), 0.0, 1.0)
 order = np.argsort(weight)
 scatter = ax.scatter(
 sub["k_parallel"].to_numpy(float)[order],
 sub["energy"].to_numpy(float)[order],
 c=weight[order],
 cmap=WT_EDGE_CMAP,
 norm=Normalize(0.0, 1.0),

```



---

```

 s=3.5 + 14.0 * weight[order],
 linewidths=0,
 rasterized=True,
)
 cbar = fig.colorbar(scatter, ax=ax, pad=0.02)
 cbar.set_label("Edge weight")
 ax.set_xlim(-np.pi, np.pi)
 ax.set_ylim(ymin, ymax)
 ax.set_xticks([-np.pi, 0.0, np.pi], [r"$-\pi$", "0", r"π"])
 ax.set_xlabel(r"$k_{\parallel}$")
 ax.set_ylabel("Energy relative to midgap")
 ax.set_title(title)
 fig.savefig(output, dpi=dpi)
 plt.close(fig)

def plot_surface_spectrum(
 data: Dict[str, np.ndarray],
 vbm: float,
 cbm: float,
 title: str,
 output: Path,
 dpi: int,
) -> None:
 k = data["k_parallel"]
 e = data["energy"]
 field = data["spin_weighted_log_intensity"]
 fig, ax = plt.subplots(figsize=(7.5, 6.0))
 mesh = ax.pcolormesh(k, e, field, shading="auto", cmap=WT_SPIN_CMAP, vmin=-1.0, vmax=1.0, rasterized=True)
 ax.axhline(0.0, color="black", lw=1.1, ls="--")
 ax.axhline(vbm, color="0.25", lw=0.8, ls=":")
 ax.axhline(cbm, color="0.25", lw=0.8, ls=":")
 ax.set_xlim(-np.pi, np.pi)
 ax.set_ylim(float(e.min()), float(e.max()))
 ax.set_xticks([-np.pi, 0.0, np.pi], [r"$-\pi$", "0", r"π"])
 ax.set_xlabel(r"$k_{\parallel}$")
 ax.set_ylabel("Energy relative to midgap")
 ax.set_title(title + "\nred: spin up, blue: spin down")
 cbar = fig.colorbar(mesh, ax=ax, pad=0.02)
 cbar.set_label("Spin-weighted log spectral intensity")
 fig.savefig(output, dpi=dpi)
 plt.close(fig)

def plot_dos_shc(
 df: pd.DataFrame,
 vbm: float,
 cbm: float,
 expected_chern: int,
 title: str,
 output: Path,
 dpi: int,
) -> None:
 fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.8), sharex=True)
 energy = df["energy"].to_numpy(float)

 axes[0].axvspan(vbm, cbm, color="0.92")
 axes[0].plot(energy, df["dos"], color="black", lw=2.0)
 axes[0].axvline(0.0, color="0.3", lw=1.0, ls="--")
 axes[0].set_xlabel("Energy relative to midgap")
 axes[0].set_ylabel("DOS (states / model energy / cell)")
 axes[0].set_title("Bulk DOS")
 axes[0].set_ylim(bottom=0.0)

 axes[1].axvspan(vbm, cbm, color="0.92")
 axes[1].plot(energy, df["spin_hall_e_over_2pi"], color="red", lw=2.2, label=r"σ^s_{xy}")
 axes[1].plot(energy, df["charge_hall_e2_over_h"], color="#194eff", lw=1.6, label=r"σ_{xy}")
 axes[1].axhline(float(expected_chern), color="0.25", lw=1.0, ls=":", label=r"expected $C_{\uparrow}$")
 axes[1].axhline(0.0, color="0.3", lw=0.9)

```

---

```

axes[1].axvline(0.0, color="0.3", lw=1.0, ls="--")
axes[1].set_xlabel("Energy relative to midgap")
axes[1].set_ylabel(r"Hall response: σ^s in $e/2\pi$, σ in e^2/h ")
axes[1].set_title("Energy-resolved Hall response")
axes[1].legend(frameon=False, loc="best")

fig.suptitle(title, y=1.02, fontsize=15)
for ax in axes:
 for spine in ax.spines.values():
 spine.set_linewidth(2.0)
fig.tight_layout()
fig.savefig(output, dpi=dpi)
plt.close(fig)

def plot_observable_certificate(summary: pd.DataFrame, output: Path, dpi: int) -> None:
 ordered = summary.sort_values("expected_chern_up")
 labels = [f"C $\uparrow$ ={int(x):+d}" for x in ordered["expected_chern_up"]]
 expected = ordered["expected_chern_up"].to_numpy(float)
 measured = ordered["shc_plateau_median"].to_numpy(float)
 dos_ratio = ordered["normalized_gap_dos_max"].to_numpy(float)

 fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.6))
 x = np.arange(len(labels))
 axes[0].plot(x, expected, "o-", color="black", lw=1.8, label="Expected C $\uparrow$ ")
 axes[0].plot(x, measured, "s--", color="red", lw=1.8, label=r"SHC plateau ($e/2\pi$)")
 axes[0].axhline(0, color="0.5", lw=0.8)
 axes[0].set_xticks(x, labels)
 axes[0].set_ylabel("Topological response")
 axes[0].set_title("Quantized spin Hall plateau")
 axes[0].legend(frameon=False)

 axes[1].bar(x, dos_ratio, color="0.45")
 axes[1].set_xticks(x, labels)
 axes[1].set_ylabel("max gap DOS / max DOS")
 axes[1].set_title("Clean bulk insulating window")
 axes[1].axhline(0.06, color="red", lw=1.2, ls="--")
 fig.tight_layout()
 fig.savefig(output, dpi=dpi)
 plt.close(fig)

Main pipeline

def energy_window_from_gap(gap: Dict[str, Any], margin_factor: float, min_margin: float) -> tuple[float, float,
↪ float, float, float]:
 vbm = float(gap["vbm"])
 cbm = float(gap["cbm"])
 ef = 0.5 * (vbm + cbm)
 vbm_rel = vbm - ef
 cbm_rel = cbm - ef
 gap_value = cbm - vbm
 margin = max(float(min_margin), float(margin_factor) * gap_value)
 return ef, vbm_rel, cbm_rel, vbm_rel - margin, cbm_rel + margin

def plateau_metrics(
 response: pd.DataFrame,
 vbm: float,
 cbm: float,
 expected_chern: int,
) -> Dict[str, float]:
 lo = float(vbm + 0.2 * (cbm - vbm))
 hi = float(cbm - 0.2 * (cbm - vbm))
 central = response[(response["energy"] >= lo) & (response["energy"] <= hi)]
 if central.empty:
 central = response.iloc[[int(np.argmin(np.abs(response["energy"].to_numpy(float))))]]

```

---

```

shc = central["spin_hall_e_over_2pi"].to_numpy(float)
charge = central["charge_hall_e2_over_h"].to_numpy(float)
dos = central["dos"].to_numpy(float)
max_dos_all = max(float(response["dos"].max()), 1.0e-14)
return {
 "plateau_energy_min": lo,
 "plateau_energy_max": hi,
 "shc_plateau_median": float(np.median(shc)),
 "shc_plateau_max_error": float(np.max(np.abs(shc - float(expected_chern)))),
 "charge_hall_plateau_max_abs": float(np.max(np.abs(charge))),
 "gap_dos_max": float(np.max(dos)),
 "normalized_gap_dos_max": float(np.max(dos) / max_dos_all),
}

def run_step07(config: Step07Config | None = None) -> Dict[str, Any]:
 if config is None:
 config = Step07Config()
 config = config.normalized()
 configure_plot_style()
 started = time.time()

 step3_zip = resolve_existing_path(config.step3_input)
 print("[1/8] Select strict Chern-sector representatives")
 representatives = select_representatives(step3_zip, config.chern_sectors)
 atomic_write_csv(representatives, config.output_dir / "step07_01_selected_observable_representatives.csv")
 print(representatives[["sample_id", "expected_chern_up", "selection_reason"]].to_string(index=False))

 summary_rows: list[dict[str, Any]] = []
 ribbon_summaries: list[dict[str, Any]] = []
 surface_summaries: list[dict[str, Any]] = []
 bulk_gap_rows: list[dict[str, Any]] = []

 _, _, band_ticks, band_labels = high_symmetry_path(config.band_points_per_segment)

 print("[2/8] Bulk gap, band and Fourier audits")
 for irow, row in representatives.iterrows():
 sample_id = str(row["sample_id"])
 expected_c = int(row["expected_chern_up"])
 params = raw_params_from_row(row)
 prefix = f"{irow:02d}_{sample_id}_Cup{expected_c:d}".replace("+", "p").replace("-", "m")
 print(f" {irow+1}/{len(representatives)} {sample_id}: C_up={expected_c:d}")

 gap = step1.scan_band_gaps(params, nk=config.gap_nk, shift=(0.0, 0.0))
 ef, vbm_rel, cbm_rel, plot_emin, plot_emax = energy_window_from_gap(
 gap, config.ribbon_plot_margin_factor, config.ribbon_min_plot_margin
)
 gap_record = {
 "sample_id": sample_id,
 "expected_chern_up": expected_c,
 "fermi_midgap": ef,
 "vbm_relative": vbm_rel,
 "cbm_relative": cbm_rel,
 **gap,
 }
 bulk_gap_rows.append(gap_record)

 hoppings = extract_hopping_matrices(params, config.fourier_n, config.fourier_tolerance)
 hr_rows = []
 for (rx, ry), mat in sorted(hoppings.items()):
 hr_rows.append(
 {
 "sample_id": sample_id,
 "Rx": rx,
 "Ry": ry,
 "max_abs_matrix_element": float(np.max(np.abs(mat))),
 "frobenius_norm": float(np.linalg.norm(mat, ord="fro")),
 }
)

```

---

```

atomic_write_csv(pd.DataFrame(hr_rows), config.output_dir / "data" / f"{prefix}_hopping_summary.csv")

band_df = calculate_bulk_band_data(params, ef, config.band_points_per_segment)
atomic_write_csv(band_df, config.output_dir / "data" / f"{prefix}_bulk_bands.csv")
plot_bulk_bands(
 band_df, band_ticks, band_labels, vbm_rel, cbm_rel,
 f"tts bulk bands — {sample_id}, Cr={expected_c:+d}",
 config.output_dir / "figures" / f"{prefix}_bulk_bands.png",
 config.dpi,
)

print("[3/8] Ribbon spectra for axial and diagonal terminations")
sample_ribbon_records: list[dict[str, Any]] = []
for edge_name, edge_spec in EDGE_SPECS.items():
 width = config.ribbon_width_axial if edge_name == "axial_x" else config.ribbon_width_diagonal
 ribbon_df, ribbon_summary = calculate_ribbon_data(
 params, hoppings, edge_name, width, config.ribbon_k_points,
 config.ribbon_edge_layers, ef, vbm_rel, cbm_rel,
 config.ribbon_edge_weight_threshold,
)
 ribbon_summary.update({"sample_id": sample_id, "expected_chern_up": expected_c})
 ribbon_summaries.append(ribbon_summary)
 sample_ribbon_records.append(ribbon_summary)
 atomic_write_csv(ribbon_df, config.output_dir / "data" / f"{prefix}_ribbon_{edge_name}.csv")

A = np.asarray(edge_spec["A"], int)
bulk_k, bulk_e_abs = projected_bulk_bands(
 np.linspace(-np.pi, np.pi, config.ribbon_k_points), hoppings, A,
 n_perp=31 if config.quick else 61,
)
bulk_e = bulk_e_abs - ef
plot_ribbon(
 ribbon_df, bulk_k, bulk_e, vbm_rel, cbm_rel,
 f"tts ribbon — {edge_spec['label']}, Cr={expected_c:+d}",
 config.output_dir / "figures" / f"{prefix}_ribbon_{edge_name}.png",
 config.dpi,
)

print("[4/8] Semi-infinite spin-resolved surface spectral functions")
surface_margin = max(config.ribbon_min_plot_margin, 0.7 * float(gap["indirect_gap"]))
surface_energies = np.linspace(vbm_rel - surface_margin, cbm_rel + surface_margin,
 ↪ config.surface_energy_points)
eta = max(config.surface_min_eta, config.surface_eta_fraction_of_gap * float(gap["indirect_gap"]))
sample_surface_records: list[dict[str, Any]] = []
for edge_name, edge_spec in EDGE_SPECS.items():
 surface_data, surface_summary = calculate_surface_spectrum(
 hoppings, edge_name, config.surface_k_points, surface_energies,
 ef, eta, config.surface_max_iter, config.surface_tol,
)
 gap_lo = vbm_rel + 0.10 * (cbm_rel - vbm_rel)
 gap_hi = cbm_rel - 0.10 * (cbm_rel - vbm_rel)
 gap_mask = (surface_data["energy"] >= gap_lo) & (surface_data["energy"] <= gap_hi)
 up_global_max = max(float(np.max(surface_data["surface_dos_up"])), 1.0e-14)
 down_global_max = max(float(np.max(surface_data["surface_dos_down"])), 1.0e-14)
 if np.any(gap_mask):
 gap_up_max = float(np.max(surface_data["surface_dos_up"][gap_mask, :]))
 gap_down_max = float(np.max(surface_data["surface_dos_down"][gap_mask, :]))
 else:
 gap_up_max = 0.0
 gap_down_max = 0.0
 surface_summary.update({
 "sample_id": sample_id,
 "expected_chern_up": expected_c,
 "central_gap_surface_up_max": gap_up_max,
 "central_gap_surface_down_max": gap_down_max,
 "central_gap_surface_up_ratio": gap_up_max / up_global_max,
 "central_gap_surface_down_ratio": gap_down_max / down_global_max,
 })
 surface_summaries.append(surface_summary)

```

---

```

sample_surface_records.append(surface_summary)
np.savez_compressed(
 config.output_dir / "data" / f"{prefix}_surface_{edge_name}.npz",
 **surface_data,
)
plot_surface_spectrum(
 surface_data, vbm_rel, cbm_rel,
 f"tts semi-infinite surface — {edge_spec['label']}, C↑={expected_c:+d}",
 config.output_dir / "figures" / f"{prefix}_surface_{edge_name}.png",
 config.dpi,
)

print("[5/8] Bulk DOS and intrinsic spin Hall conductivity")
Cover the full near-gap ribbon plotting window, with modest extra response margin.
response_energies = np.linspace(plot_emin, plot_emax, config.response_energy_points)
dos_sigma = max(
 config.minimum_dos_sigma,
 config.dos_sigma_fraction_of_gap * float(gap["indirect_gap"]),
)
response_df, response_summary = calculate_dos_and_hall(
 hoppings, ef, response_energies, config.response_nk, dos_sigma
)
atomic_write_csv(response_df, config.output_dir / "data" / f"{prefix}_dos_shc.csv")
plot_dos_shc(
 response_df, vbm_rel, cbm_rel, expected_c,
 f"tts observable response — {sample_id}, C↑={expected_c:+d}",
 config.output_dir / "figures" / f"{prefix}_dos_shc.png",
 config.dpi,
)

metrics = plateau_metrics(response_df, vbm_rel, cbm_rel, expected_c)
axial_ribbon = next(x for x in sample_ribbon_records if x["edge_name"] == "axial_x")
diagonal_ribbon = next(x for x in sample_ribbon_records if x["edge_name"] == "diagonal_11")
axial_surface = next(x for x in sample_surface_records if x["edge_name"] == "axial_x")
diagonal_surface = next(x for x in sample_surface_records if x["edge_name"] == "diagonal_11")

is_topological = expected_c != 0
shc_pass = int(metrics["shc_plateau_max_error"] <= config.shc_plateau_tolerance)
charge_pass = int(metrics["charge_hall_plateau_max_abs"] <= config.charge_hall_tolerance)
dos_pass = int(metrics["normalized_gap_dos_max"] <= config.normalized_gap_dos_tolerance)
ribbon_pass = int(
 (not is_topological)
 or (
 axial_ribbon["edge_states_detected"] == 1
 and diagonal_ribbon["edge_states_detected"] == 1
)
)
A topological edge branch must traverse the gap, but it need not cross exactly
at the chosen midgap energy. Use central-gap spectral weight as the hard test;
retain the exact-midgap peak counts as a diagnostic.
surface_ratio_threshold = 0.02
surface_pass = int(
 (not is_topological)
 or (
 axial_surface["central_gap_surface_up_ratio"] >= surface_ratio_threshold
 and axial_surface["central_gap_surface_down_ratio"] >= surface_ratio_threshold
 and diagonal_surface["central_gap_surface_up_ratio"] >= surface_ratio_threshold
 and diagonal_surface["central_gap_surface_down_ratio"] >= surface_ratio_threshold
)
)

summary_rows.append(
 {
 "sample_id": sample_id,
 "representative_label": str(row["representative_label"]),
 "expected_chern_up": expected_c,
 "expected_chern_down": -expected_c,
 "indirect_gap": float(gap["indirect_gap"]),
 "min_direct_gap": float(gap["min_direct_gap"]),
 }
)

```

```

 "fermi_midgap": ef,
 "vbm_relative": vbm_rel,
 "cbm_relative": cbm_rel,
 **response_summary,
 **metrics,
 "axial_in_gap_edge_records": axial_ribbon["in_gap_edge_state_records"],
 "diagonal_in_gap_edge_records": diagonal_ribbon["in_gap_edge_state_records"],
 "axial_surface_midgap_peaks_up": axial_surface["midgap_peak_count_up"],
 "axial_surface_midgap_peaks_down": axial_surface["midgap_peak_count_down"],
 "diagonal_surface_midgap_peaks_up": diagonal_surface["midgap_peak_count_up"],
 "diagonal_surface_midgap_peaks_down": diagonal_surface["midgap_peak_count_down"],
 "axial_surface_gap_ratio_up": axial_surface["central_gap_surface_up_ratio"],
 "axial_surface_gap_ratio_down": axial_surface["central_gap_surface_down_ratio"],
 "diagonal_surface_gap_ratio_up": diagonal_surface["central_gap_surface_up_ratio"],
 "diagonal_surface_gap_ratio_down": diagonal_surface["central_gap_surface_down_ratio"],
 "shc_quantization_pass": shc_pass,
 "charge_hall_zero_pass": charge_pass,
 "bulk_dos_gap_pass": dos_pass,
 "ribbon_edge_state_pass": ribbon_pass,
 "surface_spectral_pass": surface_pass,
 "observable_certificate_pass": int(shc_pass and charge_pass and dos_pass and ribbon_pass and
 ↪ surface_pass),
 }
)

print("[6/8] Build representative observable certificate")
summary_df = pd.DataFrame(summary_rows)
ribbon_summary_df = pd.DataFrame(ribbon_summaries)
surface_summary_df = pd.DataFrame(surface_summaries)
gap_df = pd.DataFrame(bulk_gap_rows)
atomic_write_csv(gap_df, config.output_dir / "step07_02_bulk_gap_audit.csv")
atomic_write_csv(ribbon_summary_df, config.output_dir / "step07_03_ribbon_edge_state_summary.csv")
atomic_write_csv(surface_summary_df, config.output_dir / "step07_04_surface_spectral_summary.csv")
atomic_write_csv(summary_df, config.output_dir / "step07_05_observable_certificate.csv")
plot_observable_certificate(
 summary_df,
 config.output_dir / "figures" / "step07_observable_certificate.png",
 config.dpi,
)

print("[7/8] Final interpretation checks")
topological = summary_df[summary_df["expected_chern_up"] != 0]
trivial = summary_df[summary_df["expected_chern_up"] == 0]
all_topological_pass = int(not topological.empty and bool((topological["observable_certificate_pass"] ==
↪ 1).all()))
trivial_response_pass = int(
 not trivial.empty
 and bool((trivial["shc_quantization_pass"] == 1).all())
 and bool((trivial["charge_hall_zero_pass"] == 1).all())
 and bool((trivial["bulk_dos_gap_pass"] == 1).all())
)
high_chern_edge_enhancement = np.nan
if not topological.empty:
 low = topological[np.abs(topological["expected_chern_up"]) == 1]
 high = topological[np.abs(topological["expected_chern_up"]) == 2]
 if not low.empty and not high.empty:
 high_chern_edge_enhancement = float(
 high[["axial_in_gap_edge_records", "diagonal_in_gap_edge_records"]].to_numpy(float).mean()
 / max(low[["axial_in_gap_edge_records", "diagonal_in_gap_edge_records"]].to_numpy(float).mean(),
 ↪ 1.0)
)

final_certificate = {
 "code_version": CODE_VERSION,
 "created_utc": utc_now(),
 "n_representatives": int(len(summary_df)),
 "chern_sectors": [int(x) for x in summary_df["expected_chern_up"].tolist()],
 "all_topological_observable_certificates_pass": all_topological_pass,
}

```

---

```

 "trivial_bulk_response_control_pass": trivial_response_pass,
 "high_chnern_to_unit_chnern_edge_record_ratio": high_chnern_edge_enhancement,
 "spin_hall_unit": "e/(2*pi); plateau equals C_up for spin-conserved model",
 "charge_hall_unit": "e^2/h",
 "plot_style_reference": {
 "source": "uploaded WannierTools src archive",
 "ribbon_palette": "green-yellow-red edge weight",
 "surface_palette": "blue-white-red spin-resolved spectral intensity",
 "visual_style": "thick borders, large labels, high-resolution raster output",
 },
 "final_observable_validation_pass": int(all_topological_pass and trivial_response_pass),
 "elapsed_seconds": float(time.time() - started),
 "config": asdict(config) | {"step3_input": str(config.step3_input), "output_dir": str(config.output_dir)},
 }
 atomic_write_json(final_certificate, config.output_dir / "step07_06_final_observable_certificate.json")

 print("[8/8] Save summary")
 run_summary = {
 "code_version": CODE_VERSION,
 "created_utc": utc_now(),
 "step3_input": str(step3_zip),
 "output_dir": str(config.output_dir),
 "n_representatives": int(len(summary_df)),
 "observable_certificate_pass_count": int(summary_df["observable_certificate_pass"].sum()),
 "final_observable_validation_pass": final_certificate["final_observable_validation_pass"],
 "elapsed_seconds": float(time.time() - started),
 }
 atomic_write_json(run_summary, config.output_dir / "step07_07_run_summary.json")
 print(json.dumps(run_summary, ensure_ascii=False, indent=2))

 return {
 "summary": run_summary,
 "representatives": representatives,
 "observable_certificate": summary_df,
 "final_certificate": final_certificate,
 }

CLI

def build_arg_parser() -> argparse.ArgumentParser:
 parser = argparse.ArgumentParser(description="TTS Step 07 final observable validation")
 parser.add_argument("--step3-input", type=Path, default=Step07Config.step3_input)
 parser.add_argument("--output-dir", type=Path, default=Step07Config.output_dir)
 parser.add_argument("--quick", action="store_true")
 parser.add_argument("--force-recalculate", action="store_true")
 parser.add_argument("--response-nk", type=int, default=Step07Config.response_nk)
 parser.add_argument("--ribbon-k-points", type=int, default=Step07Config.ribbon_k_points)
 parser.add_argument("--surface-k-points", type=int, default=Step07Config.surface_k_points)
 parser.add_argument("--surface-energy-points", type=int, default=Step07Config.surface_energy_points)
 return parser

def config_from_args(args: argparse.Namespace) -> Step07Config:
 return Step07Config(
 step3_input=args.step3_input,
 output_dir=args.output_dir,
 quick=bool(args.quick),
 force_recalculate=bool(args.force_recalculate),
 response_nk=int(args.response_nk),
 ribbon_k_points=int(args.ribbon_k_points),
 surface_k_points=int(args.surface_k_points),
 surface_energy_points=int(args.surface_energy_points),
)

def main() -> int:

```

```

 parser = build_arg_parser()
 args = parser.parse_args()
 run_step07(config_from_args(args))
 return 0

if __name__ == "__main__":
 raise SystemExit(main())

```

## TTS 体系机制感知机器学习、局部质量相界与验证代码

### ► 代码清单 A.50.1 (python, 1641 行)

```

#!/usr/bin/env python3
-*- coding: utf-8 -*-
"""
TTS Step 08M —机制感知机器学习：从相分类转向 valley 质量规律发现

本步骤承接冻结的 Step 01–07 结果与 formal r3-r4 相图，目标不是重新计算
Hamiltonian 或 Chern 数，而是回答机器学习在 TTS 中能否提炼出可验证的
物理规律：

1. 在 Step 03 独立 Sobol 数据上，用稀疏逻辑回归检验参数组合是否能够
 区分 $|C_{up}|=1$ 与 $|C_{up}|=2$ ，并单独检验 Chern 手性符号是否存在稳定全局规则；
2. 将 Step 04–06 的九个严格相变扩展为 transition-grouped 机制数据集，
 使用 leave-one-transition-out 验证，避免同一路径数据泄漏；
3. 学习 “高对称单谷” 与 “Sigma/Sigma' 对称双谷” 机制，并分别检验
 Gamma/M 与 Sigma/Sigma' 的可分性；
4. 对 Step 05 的局域质量梯度做无监督聚类，寻找质量方向族；
5. 在 formal 上侧 $+2 < -> 0$ 相图中，用分区 group holdout 的线性/二次逻辑回归
 恢复数值边界，并与 Step 06 的 Sigma' 局域质量方向比较；
6. 对 formal 下侧两个 $C_{up}=-2$ 连通分量进行局域质量图册对齐，判断它们是否
 对应不同的局部 valley 质量机制；
7. 输出 “支持什么、不支持什么” 的机制发现证书，不把预测准确率当成拓扑证明。

重要原则

- 所有标签均来自已冻结的严格物理结果；
- 机器学习只提炼候选规律，最终机制仍需 Step 05/06 的 valley 电荷和解析质量验证；
- transition 数据采用整条相变留出，不进行随机点拆分；
- formal 相图采用参数条带 group holdout，避免相邻网格泄漏；
- 本步骤不修改 Step 01–07 的任何输出。
"""

from __future__ import annotations

from dataclasses import asdict, dataclass
from pathlib import Path
from typing import Any, Iterable
import argparse
import io
import json
import math
import re
import time
import warnings
import zipfile

Silence scikit-learn version-deprecation messages without hiding numerical warnings.
warnings.filterwarnings("ignore", category=FutureWarning, module="sklearn")

```



---

```

warnings.filterwarnings(
 "ignore",
 message=r"Inconsistent values: penalty=l1.*",
 category=UserWarning,
)

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

try:
 from joblib import dump
 from sklearn.cluster import KMeans
 from sklearn.decomposition import PCA
 from sklearn.ensemble import RandomForestClassifier
 from sklearn.linear_model import LogisticRegression
 from sklearn.metrics import (
 accuracy_score,
 balanced_accuracy_score,
 classification_report,
 confusion_matrix,
 f1_score,
 silhouette_score,
)
 from sklearn.model_selection import (
 GroupKFold,
 LeaveOneGroupOut,
 RepeatedStratifiedKFold,
 cross_val_predict,
 cross_val_score,
)
 from sklearn.pipeline import Pipeline
 from sklearn.preprocessing import PolynomialFeatures, StandardScaler
except ImportError as exc: # pragma: no cover
 raise ImportError(
 "Step 08M 需要 numpy、pandas、matplotlib、scikit-learn 与 joblib.\n"
 " 请运行: pip install numpy pandas matplotlib scikit-learn joblib"
) from exc

CODE_VERSION = "TTS_STEP08M_V1_20260724"
WORKFLOW_STEP = "step08M"
SYSTEM_TAG = "tts8_bns127391"
TASK_TAG = "mechanism_aware_machine_learning"

REDUCED7 = ["m_e", "t1", "t2", "r1", "r2", "r3", "r4"]
EXPECTED_VERSIONS = {
 "TTS_step01_model_and_label_audit_v2.py": "TTS_STEP01_V2_20260713",
 "TTS_step02v3_robust_hamiltonian_fingerprint_and_ML.py": "TTS_STEP02V3_20260713",
 "TTS_step03_global_sobol_hierarchical_topology_ML.py": "TTS_STEP03_V1_20260713",
 "TTS_step04_chern_sector_boundary_valley_tracking.py": "TTS_STEP04_V1_20260713",
 "TTS_step05_kp_valley_topological_charge.py": "TTS_STEP05_V2_20260714",
 "TTS_step06_analytic_mass_branch_chern_atlas.py": "TTS_STEP06_V1_20260714",
 "TTS_step07_observable_validation.py": "TTS_STEP07_V1_20260714",
}

SYMMETRY_FEATURES = [
 "abs_m_e", "abs_t1", "abs_t2",
 "t_sum", "t_diff", "t_product", "t_norm", "t_s", "t_d",
 "r13_sum", "r13_diff", "r24_sum", "r24_diff",
 "r12_sum", "r34_sum", "r_all_sum", "r_all_norm",
 "r13_product", "r24_product", "r_cross_product",
 "r_anisotropy_norm", "r_pair_sum_mismatch", "r_pair_diff_mismatch",
 "m_e_t_sum", "m_e_t_diff", "m_e_r13_sum", "m_e_r24_sum",
 "t_product_r_cross",
]

PARAMETER_FEATURES = REDUCED7 + SYMMETRY_FEATURES

MECHANISM_ORDER = ["Gamma", "M", "Sigma", "SigmaPrime"]

```

---

```

@dataclass
class Step08MConfig:
 output_dir: Path = Path("outputs_tts_step08M_mechanism_aware_ml")
 tts_archive: Path = Path("tts(1).zip")
 formal_scan_input: Path = Path("TTS_Cup2_Formal_Refined_Scans(1).zip")

 random_seed: int = 20260724
 n_jobs: int = 1
 random_forest_estimators: int = 700
 bootstrap_repeats: int = 300

 transition_lambda_window: float = 0.18
 upper_group_bins: int = 8
 upper_linear_C: float = 100.0
 upper_quadratic_C: float = 1.0

 global_logistic_C_grid: tuple[float, ...] = (0.03, 0.1, 0.3, 1.0, 3.0)
 transition_logistic_C: float = 0.3
 mass_cluster_k_min: int = 2
 mass_cluster_k_max: int = 5

 require_frozen_versions: bool = True

 def normalized(self) -> "Step08MConfig":
 self.output_dir = Path(self.output_dir)
 self.tts_archive = Path(self.tts_archive)
 self.formal_scan_input = Path(self.formal_scan_input)
 self.output_dir.mkdir(parents=True, exist_ok=True)
 (self.output_dir / "figures").mkdir(parents=True, exist_ok=True)
 (self.output_dir / "models").mkdir(parents=True, exist_ok=True)
 if not self.tts_archive.exists():
 raise FileNotFoundError(f" 未找到 TTS 冻结档案: {self.tts_archive}")
 if not self.formal_scan_input.exists():
 raise FileNotFoundError(f" 未找到 formal 相图结果: {self.formal_scan_input}")
 if self.transition_lambda_window <= 0:
 raise ValueError("transition_lambda_window must be > 0")
 if self.upper_group_bins < 4:
 raise ValueError("upper_group_bins must be >= 4")
 return self

=====
Safe I/O
=====

def atomic_write_csv(df: pd.DataFrame, path: Path) -> None:
 path.parent.mkdir(parents=True, exist_ok=True)
 tmp = path.with_suffix(path.suffix + ".tmp")
 df.to_csv(tmp, index=False, encoding="utf-8-sig")
 tmp.replace(path)

def atomic_write_json(obj: dict[str, Any], path: Path) -> None:
 path.parent.mkdir(parents=True, exist_ok=True)
 tmp = path.with_suffix(path.suffix + ".tmp")
 tmp.write_text(json.dumps(obj, ensure_ascii=False, indent=2, default=str), encoding="utf-8")
 tmp.replace(path)

def atomic_write_text(text: str, path: Path) -> None:
 path.parent.mkdir(parents=True, exist_ok=True)
 tmp = path.with_suffix(path.suffix + ".tmp")
 tmp.write_text(text, encoding="utf-8")
 tmp.replace(path)

def configure_plot_style() -> None:

```

---

```

plt.rcParams.update({
 "font.family": "DejaVu Serif",
 "font.size": 9,
 "axes.labelsize": 9,
 "axes.titlesize": 9,
 "xtick.labelsize": 8,
 "ytick.labelsize": 8,
 "legend.fontsize": 8,
 "axes.linewidth": 0.8,
 "xtick.direction": "in",
 "ytick.direction": "in",
 "xtick.top": True,
 "ytick.right": True,
 "pdf.fonttype": 42,
 "ps.fonttype": 42,
 "svg.fonttype": "none",
})

def _list_members(source: Path) -> list[str]:
 if source.is_file() and source.suffix.lower() == ".zip":
 with zipfile.ZipFile(source) as zf:
 return zf.namelist()
 if source.is_dir():
 return [str(p.relative_to(source)).replace("\\", "/") for p in source.rglob("*") if p.is_file()]
 raise FileNotFoundError(source)

def _read_bytes(source: Path, member: str) -> bytes:
 if source.is_file() and source.suffix.lower() == ".zip":
 with zipfile.ZipFile(source) as zf:
 return zf.read(member)
 path = source / member
 return path.read_bytes()

def _find_member(source: Path, *, exact_basename: str | None = None,
 contains: str | None = None, suffix: str | None = None) -> str:
 members = _list_members(source)
 matches = []
 for member in members:
 name = Path(member).name
 if exact_basename is not None and name != exact_basename:
 continue
 if contains is not None and contains not in name:
 continue
 if suffix is not None and not name.lower().endswith(suffix.lower()):
 continue
 matches.append(member)
 if len(matches) != 1:
 raise FileNotFoundError(
 f"{source} 中匹配 exact={exact_basename!r}, contains={contains!r}, "
 f"suffix={suffix!r} 的文件数为 {len(matches)}: {matches[:20]}"
)
 return matches[0]

def read_csv_pattern(source: Path, pattern: str) -> pd.DataFrame:
 member = _find_member(source, contains=pattern, suffix=".csv")
 return pd.read_csv(io.BytesIO(_read_bytes(source, member)), low_memory=False)

def read_json_member(source: Path, member: str) -> dict[str, Any]:
 return json.loads(_read_bytes(source, member).decode("utf-8"))

def read_json_pattern(source: Path, pattern: str) -> dict[str, Any]:
 member = _find_member(source, contains=pattern, suffix=".json")
 return read_json_member(source, member)

```

---

```

=====
Frozen-version audit
=====

def audit_frozen_versions(config: Step08MConfig) -> dict[str, Any]:
 members = _list_members(config.tts_archive)
 rows = []
 all_pass = True
 for basename, expected in EXPECTED_VERSIONS.items():
 matches = [m for m in members if Path(m).name == basename]
 loaded = None
 status = "missing"
 if len(matches) == 1:
 text = _read_bytes(config.tts_archive, matches[0]).decode("utf-8", errors="replace")
 match = re.search(r'CODE_VERSION\s*=\s*"([^\"]+)"', text)
 loaded = match.group(1) if match else None
 status = "pass" if loaded == expected else "mismatch"
 if status != "pass":
 all_pass = False
 rows.append({
 "script": basename,
 "expected_version": expected,
 "loaded_version": loaded,
 "status": status,
 })
 audit = {
 "step08M_code_version": CODE_VERSION,
 "all_frozen_versions_pass": bool(all_pass),
 "version_rows": rows,
 }
 if config.require_frozen_versions and not all_pass:
 raise RuntimeError(
 "Step 01-07 冻结版本审计失败。详见输入审计表；"
 " 如确需分析其他版本，请显式设置 require_frozen_versions=False。"
)
 return audit

=====
Data loading and feature engineering
=====

def load_inputs(config: Step08MConfig) -> dict[str, Any]:
 tts = config.tts_archive
 formal = config.formal_scan_input

 inputs = {
 "step03_combined": read_csv_pattern(tts, "step03_04_combined_ml_feature_table"),
 "step04_path_parameters": read_csv_pattern(tts, "step04_04_path_parameters"),
 "step04_transition_brackets": read_csv_pattern(tts, "step04_05_transition_brackets"),
 "step04_refined_valleys": read_csv_pattern(tts, "step04_05_refined_critical_valleys"),
 "step05_assignment": read_csv_pattern(tts, "step05_02_spin_valley_assignment"),
 "step05_gradients": read_csv_pattern(tts, "step05_03_local_mass_gradients"),
 "step05_charge_certificate": read_csv_pattern(tts, "step05_05_valley_charge_certificate"),
 "step06_atlas": read_csv_pattern(tts, "step06_02_transition_mass_atlas"),
 "step06_dense_mass": read_csv_pattern(tts, "step06_03_dense_path_mass_phase_map"),
 }

 if formal.is_file() and formal.suffix.lower() == ".zip":
 # There are two files with the same basename, so use full folder names explicitly.
 with zipfile.ZipFile(formal) as zf:
 inputs["formal_upper"] = pd.read_csv(
 zf.open("outputs_formal_upper_Cup2_to_0_boundary/tts_scan_r3_r4_grid.csv"),
 low_memory=False,
)
 inputs["formal_lower"] = pd.read_csv(
 zf.open("outputs_formal_lower_Cup2_Cum2_region/tts_scan_r3_r4_grid.csv"),

```

---

```

 low_memory=False,
)
 inputs["formal_upper_meta"] = json.load(
 zf.open("outputs_formal_upper_Cup2_to_0_boundary/scan_metadata.json")
)
 inputs["formal_lower_meta"] = json.load(
 zf.open("outputs_formal_lower_Cup2_Cum2_region/scan_metadata.json")
)
elif formal.is_dir():
 inputs["formal_upper"] = pd.read_csv(
 formal / "outputs_formal_upper_Cup2_to_0_boundary" / "tts_scan_r3_r4_grid.csv"
)
 inputs["formal_lower"] = pd.read_csv(
 formal / "outputs_formal_lower_Cup2_Cum2_region" / "tts_scan_r3_r4_grid.csv"
)
 inputs["formal_upper_meta"] = json.loads(
 (formal / "outputs_formal_upper_Cup2_to_0_boundary" / "scan_metadata.json")
 .read_text(encoding="utf-8")
)
 inputs["formal_lower_meta"] = json.loads(
 (formal / "outputs_formal_lower_Cup2_Cum2_region" / "scan_metadata.json")
 .read_text(encoding="utf-8")
)
else:
 raise FileNotFoundError(formal)

return inputs

def add_symmetry_features(df: pd.DataFrame) -> pd.DataFrame:
 out = df.copy()
 for name in REDUCED7:
 out[name] = pd.to_numeric(out[name], errors="coerce")
 me, t1, t2, r1, r2, r3, r4 = [out[name] for name in REDUCED7]
 root2 = math.sqrt(2.0)
 out["abs_me"] = me.abs()
 out["abs_t1"] = t1.abs()
 out["abs_t2"] = t2.abs()
 out["t_sum"] = t1 + t2
 out["t_diff"] = t1 - t2
 out["t_product"] = t1 * t2
 out["t_norm"] = np.sqrt(t1*t1 + t2*t2)
 out["t_s"] = (t1 + t2) / root2
 out["t_d"] = (t1 - t2) / root2
 out["r13_sum"] = r1 + r3
 out["r13_diff"] = r1 - r3
 out["r24_sum"] = r2 + r4
 out["r24_diff"] = r2 - r4
 out["r12_sum"] = r1 + r2
 out["r34_sum"] = r3 + r4
 out["r_all_sum"] = r1 + r2 + r3 + r4
 out["r_all_norm"] = np.sqrt(r1*r1 + r2*r2 + r3*r3 + r4*r4)
 out["r13_product"] = r1 * r3
 out["r24_product"] = r2 * r4
 out["r_cross_product"] = (r1 + r3) * (r2 + r4)
 out["r_anisotropy_norm"] = np.sqrt((r1-r3)**2 + (r2-r4)**2)
 out["r_pair_sum_mismatch"] = (r1 + r3) - (r2 + r4)
 out["r_pair_diff_mismatch"] = (r1 - r3) - (r2 - r4)
 out["m_e_t_sum"] = me * (t1 + t2)
 out["m_e_t_diff"] = me * (t1 - t2)
 out["m_e_r13_sum"] = me * (r1 + r3)
 out["m_e_r24_sum"] = me * (r2 + r4)
 out["t_product_r_cross"] = (t1*t2) * ((r1+r3)*(r2+r4))
 return out

def normalize_mechanism(region: str) -> str:
 mapping = {
 "Gamma": "Gamma",

```

---

```

 "M": "M",
 "generic_Sigma_kx_eq_ky": "Sigma",
 "generic_SigmaPrime_kx_eq_minus_ky": "SigmaPrime",
 }
 return mapping.get(str(region), str(region))

=====
Global Sobol rule discovery
=====

def _sparse_logistic(C: float, seed: int) -> Pipeline:
 return Pipeline([
 ("scale", StandardScaler()),
 ("model", LogisticRegression(
 penalty="l1",
 solver="liblinear",
 C=float(C),
 class_weight="balanced",
 max_iter=5000,
 random_state=seed,
)),
])

def _rf(config: Step08MConfig) -> RandomForestClassifier:
 return RandomForestClassifier(
 n_estimators=config.random_forest_estimators,
 class_weight="balanced_subsample",
 min_samples_leaf=2,
 max_features="sqrt",
 random_state=config.random_seed,
 n_jobs=config.n_jobs,
)

def select_logistic_C(X: pd.DataFrame, y: pd.Series, config: Step08MConfig) -> tuple[float, pd.DataFrame]:
 min_class = int(y.value_counts().min())
 n_splits = min(5, min_class)
 if n_splits < 2:
 return float(config.global_logistic_C_grid[0]), pd.DataFrame()
 cv = RepeatedStratifiedKFold(
 n_splits=n_splits,
 n_repeats=10,
 random_state=config.random_seed,
)
 rows = []
 for C in config.global_logistic_C_grid:
 model = _sparse_logistic(C, config.random_seed)
 scores = cross_val_score(model, X, y, scoring="balanced_accuracy", cv=cv)
 rows.append({
 "C": float(C),
 "cv_balanced_accuracy_mean": float(scores.mean()),
 "cv_balanced_accuracy_std": float(scores.std(ddof=1)),
 })
 table = pd.DataFrame(rows).sort_values(
 ["cv_balanced_accuracy_mean", "cv_balanced_accuracy_std", "C"],
 ascending=[False, True, True],
)
 return float(table.iloc[0]["C"]), table

def bootstrap_sparse_coefficients(
 X: pd.DataFrame,
 y: pd.Series,
 feature_names: list[str],
 C: float,
 config: Step08MConfig,
) -> pd.DataFrame:

```

---

```

rng = np.random.default_rng(config.random_seed)
coeffs = []
n = len(X)
for _ in range(config.bootstrap_repeats):
 for _attempt in range(100):
 idx = rng.integers(0, n, size=n)
 yb = y.iloc[idx]
 if yb.nunique() == 2:
 break
 else:
 continue
 model = _sparse_logistic(C, config.random_seed)
 with warnings.catch_warnings():
 warnings.simplefilter("ignore")
 model.fit(X.iloc[idx], yb)
 coeffs.append(model.named_steps["model"].coef_[0])
if not coeffs:
 return pd.DataFrame()
arr = np.asarray(coeffs, float)
rows = []
for j, feature in enumerate(feature_names):
 values = arr[:, j]
 nonzero = np.abs(values) > 1.0e-12
 nz_values = values[nonzero]
 rows.append({
 "feature": feature,
 "coefficient_median": float(np.median(values)),
 "coefficient_abs_median": float(np.median(np.abs(values))),
 "nonzero_fraction": float(nonzero.mean()),
 "positive_fraction": float((values > 0).mean()),
 "negative_fraction": float((values < 0).mean()),
 "sign_stability": float(max((values > 0).mean(), (values < 0).mean())),
 "q05": float(np.quantile(values, 0.05)),
 "q95": float(np.quantile(values, 0.95)),
 "n_bootstrap": int(arr.shape[0]),
 })
return pd.DataFrame(rows).sort_values(
 ["coefficient_abs_median", "nonzero_fraction"], ascending=False
)

```

```

def run_global_rules(inputs: dict[str, Any], config: Step08MConfig) -> dict[str, Any]:
 df = inputs["step03_combined"].copy()
 df = add_symmetry_features(df)
 strict_ti = df[
 (df["phase_label"] == "spin_chern_TI_candidate")
 & (pd.to_numeric(df["is_ml_eligible"], errors="coerce").fillna(0).astype(int) == 1)
 & df["chern_up_int"].notna()
].copy()

 tasks = {
 "chern_magnitude_abs2_vs_abs1": strict_ti.copy(),
 "high_chern_sign_positive_vs_negative": strict_ti[
 strict_ti["chern_up_int"].abs() == 2
].copy(),
 }

 metric_rows = []
 coef_tables = []
 dataset_rows = []
 model_paths = {}
 cv_tables = []

 for task_name, task_df in tasks.items():
 if task_name.startswith("chern_magnitude"):
 task_df["target"] = (task_df["chern_up_int"].abs() == 2).astype(int)
 positive_label = "|C_up|=2"
 negative_label = "|C_up|=1"
 else:

```

---

```

 task_df["target"] = (task_df["chern_up_int"] > 0).astype(int)
 positive_label = "C_up=+2"
 negative_label = "C_up=-2"

train = task_df[task_df["sample_source"] == "train_sobol"].copy()
external = task_df[task_df["sample_source"] == "external_sobol"].copy()
if train["target"].nunique() < 2 or external["target"].nunique() < 2:
 continue

X_train = train[PARAMETER_FEATURES].astype(float)
y_train = train["target"].astype(int)
X_external = external[PARAMETER_FEATURES].astype(float)
y_external = external["target"].astype(int)

selected_C, cv_table = select_logistic_C(X_train, y_train, config)
if not cv_table.empty:
 cv_table.insert(0, "task", task_name)
 cv_tables.append(cv_table)

models = {
 "sparse_logistic": _sparse_logistic(selected_C, config.random_seed),
 "random_forest": _rf(config),
}
for model_name, model in models.items():
 with warnings.catch_warnings():
 warnings.simplefilter("ignore")
 model.fit(X_train, y_train)
 pred = model.predict(X_external)
 metric_rows.append({
 "task": task_name,
 "model": model_name,
 "n_train": int(len(train)),
 "n_external": int(len(external)),
 "train_class_0": int((y_train == 0).sum()),
 "train_class_1": int((y_train == 1).sum()),
 "external_class_0": int((y_external == 0).sum()),
 "external_class_1": int((y_external == 1).sum()),
 "balanced_accuracy": float(balanced_accuracy_score(y_external, pred)),
 "accuracy": float(accuracy_score(y_external, pred)),
 "macro_f1": float(f1_score(y_external, pred, average="macro")),
 "tn": int(confusion_matrix(y_external, pred, labels=[0, 1])[0, 0]),
 "fp": int(confusion_matrix(y_external, pred, labels=[0, 1])[0, 1]),
 "fn": int(confusion_matrix(y_external, pred, labels=[0, 1])[1, 0]),
 "tp": int(confusion_matrix(y_external, pred, labels=[0, 1])[1, 1]),
 "positive_label": positive_label,
 "negative_label": negative_label,
 "selected_C": selected_C if model_name == "sparse_logistic" else np.nan,
 })
 model_path = config.output_dir / "models" / f"step08M_global_{task_name}_{model_name}.joblib"
 dump(model, model_path)
 model_paths[f"{task_name}:{model_name}"] = str(model_path)

sparse = models["sparse_logistic"]
scale = sparse.named_steps["scale"]
clf = sparse.named_steps["model"]
standardized = clf.coef_[0]
raw_coef = standardized / scale.scale_
raw_intercept = float(clf.intercept_[0] - np.dot(standardized, scale.mean_ / scale.scale_))
coef = pd.DataFrame({
 "task": task_name,
 "feature": PARAMETER_FEATURES,
 "standardized_coefficient": standardized,
 "raw_space_coefficient": raw_coef,
 "abs_standardized_coefficient": np.abs(standardized),
 "selected_C": selected_C,
 "raw_space_intercept": raw_intercept,
}).sort_values("abs_standardized_coefficient", ascending=False)

boot = bootstrap_sparse_coefficients(

```



---

```

 X_train, y_train, PARAMETER_FEATURES, selected_C, config
)
 if not boot.empty:
 boot.insert(0, "task", task_name)
 coef = coef.merge(boot, on=["task", "feature"], how="left")
 coef_tables.append(coef)

 task_export = task_df[
 ["sample_id", "sample_source", "chern_up_int", "target"] + PARAMETER_FEATURES
].copy()
 task_export.insert(0, "task", task_name)
 dataset_rows.append(task_export)

metrics = pd.DataFrame(metric_rows)
coefficients = pd.concat(coef_tables, ignore_index=True) if coef_tables else pd.DataFrame()
datasets = pd.concat(dataset_rows, ignore_index=True) if dataset_rows else pd.DataFrame()
cv_results = pd.concat(cv_tables, ignore_index=True) if cv_tables else pd.DataFrame()

atomic_write_csv(datasets, config.output_dir / "step08M_01_global_rule_dataset.csv")
atomic_write_csv(cv_results, config.output_dir / "step08M_02_global_logistic_C_selection.csv")
atomic_write_csv(metrics, config.output_dir / "step08M_03_global_external_metrics.csv")
atomic_write_csv(coefficients, config.output_dir / "step08M_04_global_sparse_coefficients.csv")

plot_global_coefficients(coefficients, config.output_dir / "figures" /
 ↪ "step08M_global_sparse_coefficients.png")

return {
 "metrics": metrics,
 "coefficients": coefficients,
 "datasets": datasets,
 "model_paths": model_paths,
}

def plot_global_coefficients(coefficients: pd.DataFrame, output: Path) -> None:
 if coefficients.empty:
 return
 configure_plot_style()
 tasks = coefficients["task"].unique().tolist()
 fig, axes = plt.subplots(1, len(tasks), figsize=(6.4 * len(tasks), 5.2), squeeze=False)
 for ax, task in zip(axes[0], tasks):
 sub = coefficients[coefficients["task"] == task].nlargest(
 12, "abs_standardized_coefficient"
).sort_values("standardized_coefficient")
 ax.barh(sub["feature"], sub["standardized_coefficient"])
 ax.axvline(0, color="black", linewidth=0.8)
 ax.set_title(task.replace("_", " "))
 ax.set_xlabel("standardized sparse-logistic coefficient")
 fig.tight_layout()
 fig.savefig(output, dpi=350, bbox_inches="tight")
 plt.close(fig)

=====
Transition-grouped mechanism learning
=====

def build_transition_dataset(inputs: dict[str, Any], config: Step08MConfig) -> pd.DataFrame:
 params = inputs["step04_path_parameters"].copy()
 mass = inputs["step06_dense_mass"].copy()
 atlas = inputs["step06_atlas"].copy()

 merged = params.merge(
 mass,
 on=["path_id", "path_point_id", "path_index", "lambda"],
 how="inner",
 validate="one_to_one",
)
 rows = []

```

---

```

for path_id, transitions in atlas.groupby("path_id"):
 transitions = transitions.sort_values("critical_lambda").reset_index(drop=True)
 path_rows = merged[merged["path_id"] == path_id].copy()
 for chart_index, transition in transitions.iterrows():
 mass_column = f"chart{chart_index:02d}_oriented_mass"
 if mass_column not in path_rows.columns:
 raise KeyError(
 f"{path_id} 缺少 {mass_column}; Step06 path chart 顺序与 atlas 不一致。"
)
 near = path_rows[
 (path_rows["lambda"] - float(transition["critical_lambda"])).abs()
 <= config.transition_lambda_window
].copy()
 near["transition_id"] = str(transition["transition_id"])
 near["critical_lambda"] = float(transition["critical_lambda"])
 near["critical_k_region"] = str(transition["critical_k_region"])
 near["mechanism_fine"] = normalize_mechanism(transition["critical_k_region"])
 near["mechanism_coarse"] = np.where(
 near["mechanism_fine"].isin(["Gamma", "M"]),
 "single_high_symmetry_valley",
 "paired_generic_valleys",
)
 near["delta_chern_up"] = int(transition["delta_chern_up"])
 near["abs_delta_chern_up"] = abs(int(transition["delta_chern_up"]))
 near["oriented_mass"] = pd.to_numeric(near[mass_column], errors="coerce")
 near["mass_sign"] = np.sign(near["oriented_mass"]).astype(int)
 near["lambda_minus_critical"] = near["lambda"] - near["critical_lambda"]
 near["chart_index"] = int(chart_index)
 rows.append(near)

dataset = pd.concat(rows, ignore_index=True)
dataset = add_symmetry_features(dataset)
keep = [
 "transition_id", "path_id", "path_point_id", "path_index", "lambda",
 "critical_lambda", "lambda_minus_critical", "critical_k_region",
 "mechanism_fine", "mechanism_coarse", "delta_chern_up",
 "abs_delta_chern_up", "oriented_mass", "mass_sign",
] + PARAMETER_FEATURES
return dataset[keep].copy()

def _logo_predictions(
 X: pd.DataFrame,
 y: pd.Series,
 groups: pd.Series,
 model: Any,
) -> np.ndarray:
 logo = LeaveOneGroupOut()
 pred = np.empty(len(y), dtype=object)
 for train_idx, test_idx in logo.split(X, y, groups):
 with warnings.catch_warnings():
 warnings.simplefilter("ignore")
 model.fit(X.iloc[train_idx], y.iloc[train_idx])
 pred[test_idx] = model.predict(X.iloc[test_idx])
 return pred

def run_transition_mechanism_learning(
 inputs: dict[str, Any], config: Step08MConfig
) -> dict[str, Any]:
 dataset = build_transition_dataset(inputs, config)
 atomic_write_csv(dataset, config.output_dir / "step08M_05_transition_mechanism_dataset.csv")

 task_specs = {
 "coarse_single_vs_paired": (
 dataset,
 "mechanism_coarse",
 ["single_high_symmetry_valley", "paired_generic_valleys"],
),
 }

```

---

```

 "fine_four_family": (
 dataset,
 "mechanism_fine",
 MECHANISM_ORDER,
),
 "high_symmetry_Gamma_vs_M": (
 dataset[dataset["mechanism_fine"].isin(["Gamma", "M"]).copy(),
 "mechanism_fine",
 ["Gamma", "M"],
),
 "generic_Sigma_vs_SigmaPrime": (
 dataset[dataset["mechanism_fine"].isin(["Sigma", "SigmaPrime"]).copy(),
 "mechanism_fine",
 ["Sigma", "SigmaPrime"],
),
}

metric_rows = []
prediction_rows = []
coefficient_rows = []
model_paths = {}

for task_name, (task_df, target_col, labels) in task_specs.items():
 if task_df[target_col].nunique() < 2:
 continue
 # Require at least two independent transitions per class for meaningful LOGO.
 group_class = task_df[["transition_id", target_col]].drop_duplicates()
 if group_class.groupby(target_col)["transition_id"].nunique().min() < 2:
 continue

 X = task_df[PARAMETER_FEATURES].astype(float)
 y = task_df[target_col].astype(str)
 groups = task_df["transition_id"].astype(str)

 models = {
 "logistic": Pipeline([
 ("scale", StandardScaler()),
 ("model", LogisticRegression(
 C=config.transition_logistic_C,
 solver="lbfgs",
 class_weight="balanced",
 max_iter=5000,
 random_state=config.random_seed,
)),
]),
 "random_forest": _rf(config),
 }

 for model_name, model in models.items():
 pred = _logo_predictions(X, y, groups, model)
 metric_rows.append({
 "task": task_name,
 "model": model_name,
 "n_rows": int(len(task_df)),
 "n_transitions": int(groups.nunique()),
 "n_classes": int(y.nunique()),
 "balanced_accuracy": float(balanced_accuracy_score(y, pred)),
 "accuracy": float(accuracy_score(y, pred)),
 "macro_f1": float(f1_score(y, pred, average="macro")),
 })
 prediction_rows.append(pd.DataFrame({
 "task": task_name,
 "model": model_name,
 "transition_id": groups.to_numpy(),
 "path_point_id": task_df["path_point_id"].to_numpy(),
 "actual": y.to_numpy(),
 "predicted": pred,
 "correct": (y.to_numpy() == pred).astype(int),
 })))

```

---

```

with warnings.catch_warnings():
 warnings.simplefilter("ignore")
 model.fit(X, y)
model_path = config.output_dir / "models" / f"step08M_transition_{task_name}_{model_name}.joblib"
dump(model, model_path)
model_paths[f"{task_name}:{model_name}"] = str(model_path)

if model_name == "logistic":
 clf = model.named_steps["model"]
 if clf.coef_.shape[0] == 1 and len(clf.classes_) == 2:
 class_vectors = [
 (str(clf.classes_[0]), -clf.coef_[0]),
 (str(clf.classes_[1]), clf.coef_[0]),
]
 else:
 class_vectors = [
 (str(class_name), clf.coef_[class_index])
 for class_index, class_name in enumerate(clf.classes_)
]
 for class_name, vector in class_vectors:
 for feature, value in zip(PARAMETER_FEATURES, vector):
 coefficient_rows.append({
 "task": task_name,
 "class": class_name,
 "feature": feature,
 "standardized_coefficient": float(value),
 "abs_standardized_coefficient": float(abs(value)),
 })

metrics = pd.DataFrame(metric_rows)
predictions = pd.concat(prediction_rows, ignore_index=True) if prediction_rows else pd.DataFrame()
coefficients = pd.DataFrame(coefficient_rows)

atomic_write_csv(metrics, config.output_dir / "step08M_06_transition_grouped_cv_metrics.csv")
atomic_write_csv(predictions, config.output_dir / "step08M_07_transition_grouped_predictions.csv")
atomic_write_csv(coefficients, config.output_dir / "step08M_08_transition_logistic_coefficients.csv")

plot_transition_confusions(
 predictions,
 config.output_dir / "figures" / "step08M_transition_grouped_confusions.png",
)
return {
 "dataset": dataset,
 "metrics": metrics,
 "predictions": predictions,
 "coefficients": coefficients,
 "model_paths": model_paths,
}

```

```

def plot_transition_confusions(predictions: pd.DataFrame, output: Path) -> None:
 if predictions.empty:
 return
 configure_plot_style()
 tasks = [
 task for task in [
 "coarse_single_vs_paired",
 "fine_four_family",
 "high_symmetry_Gamma_vs_M",
 "generic_Sigma_vs_SigmaPrime",
]
 if task in set(predictions["task"])
]
 fig, axes = plt.subplots(2, 2, figsize=(10, 8))
 for ax, task in zip(axes.ravel(), tasks):
 sub = predictions[
 (predictions["task"] == task) & (predictions["model"] == "logistic")
]

```

---

```

 labels = sorted(set(sub["actual"]) | set(sub["predicted"]))
 cm = confusion_matrix(sub["actual"], sub["predicted"], labels=labels)
 im = ax.imshow(cm, cmap="Blues")
 ax.set_xticks(range(len(labels)), labels=labels, rotation=30, ha="right")
 ax.set_yticks(range(len(labels)), labels=labels)
 ax.set_xlabel("predicted")
 ax.set_ylabel("actual")
 ax.set_title(task.replace("_", " "))
 for i in range(cm.shape[0]):
 for j in range(cm.shape[1]):
 ax.text(j, i, str(cm[i, j]), ha="center", va="center")
 for ax in axes.ravel()[len(tasks):]:
 ax.axis("off")
 fig.tight_layout()
 fig.savefig(output, dpi=350, bbox_inches="tight")
 plt.close(fig)

=====
Local-mass-gradient clustering
=====

def build_mass_gradient_table(inputs: dict[str, Any]) -> pd.DataFrame:
 assignment = inputs["step05_assignment"].copy()
 gradients = inputs["step05_gradients"].copy()
 active = assignment[
 (assignment["spin"] == "up")
 & (pd.to_numeric(assignment["is_active_valley"], errors="coerce").fillna(0).astype(int) == 1)
][["transition_id", "closure_id", "valley_id", "critical_k_region"]].copy()

 merged = gradients[
 gradients["spin"] == "up"
].merge(
 active,
 on=["transition_id", "closure_id", "valley_id"],
 how="inner",
)
 pivot = merged.pivot_table(
 index=["transition_id", "critical_k_region"],
 columns="parameter",
 values="normalized_mass_gradient",
 aggfunc="mean",
).reset_index()
 pivot = pivot[["transition_id", "critical_k_region"] + REDUCED7].copy()

 vectors = pivot[REDUCED7].to_numpy(float)
 for i, vector in enumerate(vectors):
 max_index = int(np.argmax(np.abs(vector)))
 if vector[max_index] < 0:
 vectors[i] *= -1.0
 pivot.loc[:, REDUCED7] = vectors
 pivot["mechanism_fine"] = pivot["critical_k_region"].map(normalize_mechanism)
 return pivot

def run_mass_gradient_clustering(
 inputs: dict[str, Any], config: Step08MConfig
) -> dict[str, Any]:
 table = build_mass_gradient_table(inputs)
 X = table[REDUCED7].to_numpy(float)

 selection_rows = []
 fits = {}
 max_k = min(config.mass_cluster_k_max, len(table) - 1)
 for k in range(config.mass_cluster_k_min, max_k + 1):
 model = KMeans(
 n_clusters=k,
 random_state=config.random_seed,
 n_init=100,

```

---

```

).fit(X)
 score = float(silhouette_score(X, model.labels_, metric="cosine"))
 selection_rows.append({"n_clusters": k, "cosine_silhouette": score})
 fits[k] = model
 selection = pd.DataFrame(selection_rows).sort_values(
 ["cosine_silhouette", "n_clusters"], ascending=[False, True]
)
 best_k = int(selection.iloc[0]["n_clusters"])
 model = fits[best_k]
 table["mass_direction_cluster"] = model.labels_.astype(int)

 pca = PCA(n_components=2, random_state=config.random_seed)
 xy = pca.fit_transform(X)
 table["pca_1"] = xy[:, 0]
 table["pca_2"] = xy[:, 1]

 centroid_rows = []
 for cluster_id, centroid in enumerate(model.cluster_centers_):
 row = {"mass_direction_cluster": int(cluster_id)}
 row.update({name: float(value) for name, value in zip(REDUCED7, centroid)})
 centroid_rows.append(row)
 centroids = pd.DataFrame(centroid_rows)

 atomic_write_csv(table, config.output_dir / "step08M_09_mass_gradient_clusters.csv")
 atomic_write_csv(selection, config.output_dir / "step08M_10_mass_cluster_selection.csv")
 atomic_write_csv(centroids, config.output_dir / "step08M_11_mass_cluster_centroids.csv")
 dump(model, config.output_dir / "models" / "step08M_mass_gradient_kmeans.joblib")

 plot_mass_gradient_pca(
 table,
 config.output_dir / "figures" / "step08M_mass_gradient_clusters.png",
)
 return {
 "table": table,
 "selection": selection,
 "centroids": centroids,
 "best_k": best_k,
 "best_silhouette": float(selection.iloc[0]["cosine_silhouette"]),
 }

def plot_mass_gradient_pca(table: pd.DataFrame, output: Path) -> None:
 configure_plot_style()
 fig, ax = plt.subplots(figsize=(6.4, 5.2))
 for mechanism, sub in table.groupby("mechanism_fine"):
 ax.scatter(sub["pca_1"], sub["pca_2"], s=55, label=mechanism)
 for _, row in sub.iterrows():
 short = str(row["transition_id"]).replace("_anchor01", "").replace("__transition00",
 ↪ "0").replace("__transition01", "1")
 ax.annotate(short, (row["pca_1"], row["pca_2"]), fontsize=6, xytext=(3, 3), textcoords="offset
 ↪ points")
 ax.set_xlabel("PCA 1 of canonicalized mass gradients")
 ax.set_ylabel("PCA 2 of canonicalized mass gradients")
 ax.legend(frameon=True)
 fig.tight_layout()
 fig.savefig(output, dpi=350, bbox_inches="tight")
 plt.close(fig)

=====
Formal upper boundary: ML recovery of the valley-mass direction
=====

def parse_linear_mass_formula(formula: str) -> dict[str, float]:
 compact = str(formula).replace(" ", "")
 pairs = re.findall(r"([+-]?[d+](?:\.d+)?)*([A-Za-z0-9_]+)", compact)
 return {parameter: float(coefficient) for coefficient, parameter in pairs}

```

---

```

def upper_mass_chart(meta: dict[str, Any]) -> dict[str, Any]:
 charts = meta.get("generic_mass_charts", [])
 for chart in charts:
 if (
 int(chart.get("chern_left", 999)) == 2
 and int(chart.get("chern_right", 999)) == 0
 and "SigmaPrime" in str(chart.get("critical_k_region", ""))
):
 coeff = parse_linear_mass_formula(chart["mass_formula"])
 return {**chart, "coefficients": coeff}
 raise ValueError("formal upper metadata 中未找到 +2->0 SigmaPrime 质量图。")

def grouped_upper_cv(
 data: pd.DataFrame,
 config: Step08MConfig,
) -> tuple[pd.DataFrame, pd.DataFrame, dict[str, Any]]:
 work = data[data["phase_code"].isin([0, 2])].copy()
 work["target"] = (work["phase_code"] == 2).astype(int)
 work["r3_group"] = pd.qcut(
 work["scan_x"],
 q=config.upper_group_bins,
 labels=False,
 duplicates="drop",
)
 n_groups = int(work["r3_group"].nunique())
 cv = GroupKFold(n_splits=n_groups)

 specs = {
 "linear_r3_r4": Pipeline([
 ("scale", StandardScaler()),
 ("model", LogisticRegression(
 C=config.upper_linear_C,
 solver="lbfgs",
 class_weight="balanced",
 max_iter=5000,
 random_state=config.random_seed,
)),
]),
 "quadratic_r3_r4": Pipeline([
 ("poly", PolynomialFeatures(degree=2, include_bias=False)),
 ("scale", StandardScaler()),
 ("model", LogisticRegression(
 C=config.upper_quadratic_C,
 solver="lbfgs",
 class_weight="balanced",
 max_iter=5000,
 random_state=config.random_seed,
)),
]),
 }
 metric_rows = []
 prediction_rows = []
 fitted = {}
 X = work[["scan_x", "scan_y"]].astype(float)
 y = work["target"].astype(int)
 groups = work["r3_group"]

 for name, model in specs.items():
 pred = cross_val_predict(model, X, y, groups=groups, cv=cv)
 metric_rows.append({
 "model": name,
 "n_points": int(len(work)),
 "n_groups": n_groups,
 "balanced_accuracy": float(balanced_accuracy_score(y, pred)),
 "accuracy": float(accuracy_score(y, pred)),
 "macro_f1": float(f1_score(y, pred, average="macro")),
 })

```

---

```

 prediction_rows.append(pd.DataFrame({
 "model": name,
 "sample_id": work["sample_id"].to_numpy(),
 "scan_x": work["scan_x"].to_numpy(),
 "scan_y": work["scan_y"].to_numpy(),
 "actual": y.to_numpy(),
 "predicted": pred,
 "r3_group": groups.to_numpy(),
 }))
 model.fit(X, y)
 fitted[name] = model

 return pd.DataFrame(metric_rows), pd.concat(prediction_rows, ignore_index=True), fitted

def raw_linear_logistic_rule(model: Pipeline) -> dict[str, float]:
 scale = model.named_steps["scale"]
 clf = model.named_steps["model"]
 standardized = clf.coef_[0]
 beta = standardized / scale.scale_
 intercept = float(clf.intercept_[0] - np.dot(standardized, scale.mean_ / scale.scale_))
 if abs(beta[1]) < 1.0e-14:
 slope = np.nan
 line_intercept = np.nan
 else:
 slope = float(-beta[0] / beta[1])
 line_intercept = float(-intercept / beta[1])
 return {
 "beta_r3": float(beta[0]),
 "beta_r4": float(beta[1]),
 "intercept": intercept,
 "boundary_slope": slope,
 "boundary_intercept": line_intercept,
 }

def run_upper_boundary_rule(
 inputs: dict[str, Any], config: Step08MConfig
) -> dict[str, Any]:
 upper = inputs["formal_upper"].copy()
 meta = inputs["formal_upper_meta"]
 metrics, predictions, fitted = grouped_upper_cv(upper, config)
 linear_rule = raw_linear_logistic_rule(fitted["linear_r3_r4"])

 chart = upper_mass_chart(meta)
 coeff = chart["coefficients"]
 mass_vector = np.array([coeff["r3"], coeff["r4"]], float)
 ml_vector = np.array([linear_rule["beta_r3"], linear_rule["beta_r4"]], float)
 cosine = float(abs(np.dot(mass_vector, ml_vector)) / (
 np.linalg.norm(mass_vector) * np.linalg.norm(ml_vector)
))

 anchor = meta["anchor_params"]
 constant = sum(
 float(value) * float(anchor[parameter])
 for parameter, value in coeff.items()
 if parameter not in {"r3", "r4"}
)
 mass_slope = float(-coeff["r3"] / coeff["r4"])
 mass_intercept = float(-constant / coeff["r4"])

 rule_table = pd.DataFrame([
 *linear_rule,
 "step06_mass_r3_coefficient": float(coeff["r3"]),
 "step06_mass_r4_coefficient": float(coeff["r4"]),
 "step06_mass_boundary_slope": mass_slope,
 "step06_mass_boundary_intercept": mass_intercept,
 "absolute_cosine_ml_vs_step06_mass_direction": cosine,
 "transition_id": chart["transition_id"],
])

```



---

```

 "critical_k_region": chart["critical_k_region"],
 "mass_formula": chart["mass_formula"],
 })

 atomic_write_csv(metrics, config.output_dir / "step08M_12_upper_grouped_cv_metrics.csv")
 atomic_write_csv(predictions, config.output_dir / "step08M_13_upper_grouped_predictions.csv")
 atomic_write_csv(rule_table, config.output_dir / "step08M_14_upper_mass_direction_recovery.csv")
 dump(
 fitted["linear_r3_r4"],
 config.output_dir / "models" / "step08M_upper_linear_boundary_logistic.joblib",
)
 dump(
 fitted["quadratic_r3_r4"],
 config.output_dir / "models" / "step08M_upper_quadratic_boundary_logistic.joblib",
)

 plot_upper_boundary_rule(
 upper,
 linear_rule,
 mass_slope,
 mass_intercept,
 config.output_dir / "figures" / "step08M_upper_boundary_mass_recovery.png",
)
 return {
 "metrics": metrics,
 "predictions": predictions,
 "rule_table": rule_table,
 "linear_rule": linear_rule,
 "mass_slope": mass_slope,
 "mass_intercept": mass_intercept,
 "cosine": cosine,
 }

def plot_upper_boundary_rule(
 upper: pd.DataFrame,
 linear_rule: dict[str, float],
 mass_slope: float,
 mass_intercept: float,
 output: Path,
) -> None:
 configure_plot_style()
 phase_colors = {2: "#F28E8B", 0: "#B2B2B2", -2: "#3F63AD", 99: "white"}
 fig, ax = plt.subplots(figsize=(6.4, 5.2))
 for phase, sub in upper.groupby("phase_code"):
 ax.scatter(
 sub["scan_x"], sub["scan_y"],
 s=10,
 color=phase_colors.get(int(phase), "white"),
 edgecolors="black" if int(phase) == 99 else "none",
 linewidths=0.3,
 label=f"C_up={int(phase)}" if int(phase) != 99 else "unresolved",
)
 xs = np.linspace(float(upper["scan_x"].min()), float(upper["scan_x"].max()), 300)
 ax.plot(
 xs,
 linear_rule["boundary_slope"] * xs + linear_rule["boundary_intercept"],
 color="black",
 linewidth=1.4,
 label="grouped-CV ML boundary",
)
 ax.plot(
 xs,
 mass_slope * xs + mass_intercept,
 color="black",
 linewidth=1.0,
 linestyle="--",
 label="Step06 local mass zero",
)

```

---

```

ax.set_xlabel(r"r_3")
ax.set_ylabel(r"r_4")
ax.legend(frameon=True)
fig.tight_layout()
fig.savefig(output, dpi=350, bbox_inches="tight")
plt.close(fig)

=====
Formal lower scan: disconnected -2 components and mass-chart alignment
=====

def connected_components(
 df: pd.DataFrame,
 phase_code: int,
) -> list[list[Any]]:
 rows = {
 (int(row.scan_ix), int(row.scan_iy)): row
 for row in df.itertuples()
 if int(row.phase_code) == int(phase_code)
 }
 visited: set[tuple[int, int]] = set()
 components: list[list[Any]] = []
 for key in rows:
 if key in visited:
 continue
 stack = [key]
 visited.add(key)
 component = []
 while stack:
 node = stack.pop()
 component.append(rows[node])
 ix, iy = node
 for neighbor in ((ix+1, iy), (ix-1, iy), (ix, iy+1), (ix, iy-1)):
 if neighbor in rows and neighbor not in visited:
 visited.add(neighbor)
 stack.append(neighbor)
 component.append(component)
 components.append(component)
 components.sort(key=len, reverse=True)
 return components

def run_lower_component_alignment(
 inputs: dict[str, Any], config: Step08MConfig
) -> dict[str, Any]:
 lower = inputs["formal_lower"].copy()
 meta = inputs["formal_lower_meta"]
 grid = {
 (int(row.scan_ix), int(row.scan_iy)): row
 for row in lower.itertuples()
 }
 charts = meta["generic_mass_charts"]
 chart_columns = [chart["column"] for chart in charts]
 chart_map = {chart["column"]: chart for chart in charts}

 components = connected_components(lower, -2)
 component_rows = []
 alignment_rows = []

 for component_id, component in enumerate(components):
 if len(component) < 5:
 continue
 keys = {(int(row.scan_ix), int(row.scan_iy)) for row in component}
 boundary = []
 for key in keys:
 ix, iy = key
 if any(
 neighbor in grid and int(grid[neighbor].phase_code) != -2
 for neighbor in ((ix+1, iy), (ix-1, iy), (ix, iy+1), (ix, iy-1))
):
 boundary.append(key)

```

---

```

):
 boundary.append(grid[key])
boundary_df = pd.DataFrame([row._asdict() for row in boundary])

component_rows.append({
 "component_id": component_id,
 "n_points": len(component),
 "n_boundary_points": len(boundary),
 "r3_min": min(float(row.scan_x) for row in component),
 "r3_max": max(float(row.scan_x) for row in component),
 "r4_min": min(float(row.scan_y) for row in component),
 "r4_max": max(float(row.scan_y) for row in component),
 "median_indirect_gap": float(np.median([float(row.indirect_gap) for row in component])),
 "max_indirect_gap": float(np.max([float(row.indirect_gap) for row in component])),
})

nearest = boundary_df[chart_columns].abs().idxmin(axis=1)
for column in chart_columns:
 chart = chart_map[column]
 alignment_rows.append({
 "component_id": component_id,
 "component_size": len(component),
 "n_boundary_points": len(boundary_df),
 "chart_column": column,
 "transition_id": chart["transition_id"],
 "chern_left": chart["chern_left"],
 "chern_right": chart["chern_right"],
 "critical_k_region": chart["critical_k_region"],
 "median_abs_mass_on_component_boundary": float(boundary_df[column].abs().median()),
 "mean_abs_mass_on_component_boundary": float(boundary_df[column].abs().mean()),
 "fraction_nearest_mass_chart": float((nearest == column).mean()),
 "mass_formula": chart["mass_formula"],
 })

components_df = pd.DataFrame(component_rows)
alignment = pd.DataFrame(alignment_rows).sort_values(
 ["component_id", "median_abs_mass_on_component_boundary"]
)

atomic_write_csv(components_df, config.output_dir / "step08M_15_lower_minus2_components.csv")
atomic_write_csv(alignment, config.output_dir / "step08M_16_lower_component_mass_alignment.csv")
plot_lower_alignment(
 alignment,
 config.output_dir / "figures" / "step08M_lower_component_mass_alignment.png",
)
return {
 "components": components_df,
 "alignment": alignment,
}

```

```

def plot_lower_alignment(alignment: pd.DataFrame, output: Path) -> None:
 if alignment.empty:
 return
 configure_plot_style()
 component_ids = sorted(alignment["component_id"].unique())
 fig, axes = plt.subplots(1, len(component_ids), figsize=(6.2 * len(component_ids), 4.6), squeeze=False)
 for ax, component_id in zip(axes[0], component_ids):
 sub = alignment[alignment["component_id"] == component_id].copy()
 sub["short"] = sub["transition_id"].str.replace("_anchor01", "", regex=False)
 sub = sub.sort_values("median_abs_mass_on_component_boundary")
 ax.barh(sub["short"], sub["median_abs_mass_on_component_boundary"])
 ax.set_title(f"Cup=2 component {component_id}")
 ax.set_xlabel("median |local chart mass| on boundary")
 fig.tight_layout()
 fig.savefig(output, dpi=350, bbox_inches="tight")
 plt.close(fig)

```

---

```

=====
Summary and certificate
=====

def _best_metric(metrics: pd.DataFrame, task: str) -> dict[str, Any] | None:
 sub = metrics[metrics["task"] == task]
 if sub.empty:
 return None
 row = sub.sort_values(["balanced_accuracy", "macro_f1"], ascending=False).iloc[0]
 return row.to_dict()

def write_rule_summary(
 global_result: dict[str, Any],
 transition_result: dict[str, Any],
 cluster_result: dict[str, Any],
 upper_result: dict[str, Any],
 lower_result: dict[str, Any],
 config: Step08MConfig,
) -> dict[str, Any]:
 global_metrics = global_result["metrics"]
 transition_metrics = transition_result["metrics"]

 magnitude = _best_metric(global_metrics, "chern_magnitude_abs2_vs_abs1")
 sign = _best_metric(global_metrics, "high_chern_sign_positive_vs_negative")
 coarse = _best_metric(transition_metrics, "coarse_single_vs_paired")
 fine = _best_metric(transition_metrics, "fine_four_family")
 high = _best_metric(transition_metrics, "high_symmetry_Gamma_vs_M")
 generic = _best_metric(transition_metrics, "generic_Sigma_vs_SigmaPrime")

 upper_linear = upper_result["metrics"][
 upper_result["metrics"]["model"] == "linear_r3_r4"
].iloc[0].to_dict()
 upper_quad = upper_result["metrics"][
 upper_result["metrics"]["model"] == "quadratic_r3_r4"
].iloc[0].to_dict()

 alignment = lower_result["alignment"]
 dominant_rows = []
 if not alignment.empty:
 for component_id, sub in alignment.groupby("component_id"):
 dominant_rows.append(
 sub.sort_values(
 ["fraction_nearest_mass_chart", "median_abs_mass_on_component_boundary"],
 ascending=[False, True],
).iloc[0].to_dict()
)

 certificate = {
 "code_version": CODE_VERSION,
 "global_abs_chern_rule_supported": bool(
 magnitude is not None and magnitude["balanced_accuracy"] >= 0.75
),
 "global_high_chern_sign_rule_supported": bool(
 sign is not None and sign["balanced_accuracy"] >= 0.70
),
 "coarse_single_vs_paired_valley_rule_supported": bool(
 coarse is not None and coarse["balanced_accuracy"] >= 0.65
),
 "single_global_four_family_classifier_supported": bool(
 fine is not None and fine["balanced_accuracy"] >= 0.75
),
 "high_symmetry_Gamma_vs_M_rule_supported": bool(
 high is not None and high["balanced_accuracy"] >= 0.80
),
 "global_Sigma_vs_SigmaPrime_rule_supported": bool(
 generic is not None and generic["balanced_accuracy"] >= 0.70
),
 "upper_linear_boundary_rule_supported": bool(

```

```

 upper_linear["balanced_accuracy"] >= 0.95
),
 "upper_quadratic_correction_useful": bool(
 upper_quad["balanced_accuracy"] - upper_linear["balanced_accuracy"] >= 0.005
),
 "upper_step06_mass_direction_recovered": bool(
 upper_result["cosine"] >= 0.98
),
 "multiple_minus2_components_supported": bool(
 len(lower_result["components"]) >= 2
),
 "distinct_minus2_local_mass_charts_supported": bool(
 len({row["transition_id"] for row in dominant_rows}) >= 2
),
 "mass_gradient_best_cluster_count": int(cluster_result["best_k"]),
 "mass_gradient_cosine_silhouette": float(cluster_result["best_silhouette"]),
 "key_metrics": {
 "global_magnitude": magnitude,
 "global_sign": sign,
 "transition_coarse": coarse,
 "transition_fine": fine,
 "high_symmetry_axis": high,
 "generic_diagonal_axis": generic,
 "upper_linear": upper_linear,
 "upper_quadratic": upper_quad,
 "upper_mass_direction_cosine": upper_result["cosine"],
 },
 "dominant_minus2_component_charts": dominant_rows,
 "interpretation": {
 "supported": [
 " 参数规则能够在独立 Sobol 上较好地区分 |Cup|=1 与 |Cup|=2。",
 "transition-grouped 学习能够识别单个高对称谷与成对普通谷两类粗机制。",
 "Gamma 与 M 高对称单谷机制具有稳定可分的参数特征。",
 "formal 上侧相图中的 ML 线性边界方向与 Step06 Sigma' 质量方向一致。",
 " 两个主要 Cup=-2 连通分量分别对齐到不同的局部质量图表。",
],
 "not_supported": [
 " 高 Chern 手性符号不存在可靠的单一全局参数判据。",
 "Sigma 与 Sigma' 不能由一个跨 transition 的全局分类器稳定区分。",
 " 四类 valley 机制不能压缩为单一全局分类规则。",
],
 },
}

lines = [
 "# TTS Step 08M 机制感知机器学习结论",
 "",
 "## 一、机器学习支持的规律",
 "",
]
for item in certificate["interpretation"]["supported"]:
 lines.append(f"- {item}")
lines += ["", "## 二、机器学习不支持的过度简化", ""]
for item in certificate["interpretation"]["not_supported"]:
 lines.append(f"- {item}")
lines += [
 "",
 "## 三、上侧 +2+0 质量方向",
 "",
 (
 f"- grouped-CV 线性逻辑回归 balanced accuracy = "
 f"{upper_linear['balanced_accuracy']:.4f}"
),
 (
 f"- 二次逻辑回归 balanced accuracy = "
 f"{upper_quad['balanced_accuracy']:.4f}"
),
 (
 f"- ML 边界方向与 Step06 Sigma' 局域质量方向的 |cosine| = "

```

```

 f"{upper_result['cosine']:.6f}"
),
 (
 f"- ML 边界: r4 = {upper_result['linear_rule']['boundary_slope']:.6f} r3 "
 f"+ {upper_result['linear_rule']['boundary_intercept']:.6f}"
),
 (
 f"- Step06 质量零线: r4 = {upper_result['mass_slope']:.6f} r3 "
 f"+ {upper_result['mass_intercept']:.6f}"
),
 "",
 "## 四、推荐物理表述",
 "",
 (
 "TTS 的机器学习结果不支持 Lieb 式单一全局质量坐标。更合适的结论是: "
 " 七维参数中存在可学习的层级规律——首先区分高对称单谷与成对普通谷机制, "
 " 随后在各机制内部使用局部质量坐标。ML 可以恢复上侧 Sigma' 质量方向并识别"
 " 两个 -2 区域对应不同局部质量图表, 但不能用一个全局公式同时决定 valley "
 " 方向、Chern 手性和全部相区。"
),
]

atomic_write_json(
 certificate,
 config.output_dir / "step08M_17_mechanism_discovery_certificate.json",
)
atomic_write_text(
 "\n".join(lines),
 config.output_dir / "step08M_18_candidate_mechanism_rules.md",
)
return certificate

```

```

=====
Main workflow
=====

```

```

def run_step08M(config: Step08MConfig) -> dict[str, Any]:
 config = config.normalized()
 start = time.time()

 audit = audit_frozen_versions(config)
 atomic_write_csv(
 pd.DataFrame(audit["version_rows"]),
 config.output_dir / "step08M_00_frozen_version_audit.csv",
)

 inputs = load_inputs(config)
 input_summary = {
 key: (
 {"rows": int(value.shape[0]), "columns": int(value.shape[1])}
 if isinstance(value, pd.DataFrame)
 else {"type": type(value).__name__}
)
 for key, value in inputs.items()
 }
 atomic_write_json(
 {
 "code_version": CODE_VERSION,
 "configuration": asdict(config),
 "frozen_version_audit": audit,
 "input_summary": input_summary,
 },
 config.output_dir / "step08M_00_run_configuration.json",
)

 global_result = run_global_rules(inputs, config)
 transition_result = run_transition_mechanism_learning(inputs, config)
 cluster_result = run_mass_gradient_clustering(inputs, config)

```

---

```

upper_result = run_upper_boundary_rule(inputs, config)
lower_result = run_lower_component_alignment(inputs, config)

certificate = write_rule_summary(
 global_result,
 transition_result,
 cluster_result,
 upper_result,
 lower_result,
 config,
)

summary = {
 "code_version": CODE_VERSION,
 "output_dir": str(config.output_dir.resolve()),
 "elapsed_seconds": float(time.time() - start),
 "all_frozen_versions_pass": audit["all_frozen_versions_pass"],
 "certificate": certificate,
 "output_files": [
 str(path.relative_to(config.output_dir))
 for path in sorted(config.output_dir.rglob("*"))
 if path.is_file()
],
}
atomic_write_json(summary, config.output_dir / "step08M_19_run_summary.json")
return summary

def build_arg_parser() -> argparse.ArgumentParser:
 parser = argparse.ArgumentParser(
 description="TTS Step08M mechanism-aware machine learning"
)
 parser.add_argument(
 "--tts-archive",
 type=Path,
 default=Path("tts(1).zip"),
)
 parser.add_argument(
 "--formal-scan-input",
 type=Path,
 default=Path("TTS_Cup2_Forma1_Refined_Scans(1).zip"),
)
 parser.add_argument(
 "--output-dir",
 type=Path,
 default=Path("outputs_tts_step08M_mechanism_aware_ml"),
)
 parser.add_argument("--n-jobs", type=int, default=1)
 parser.add_argument("--bootstrap-repeats", type=int, default=300)
 parser.add_argument("--transition-lambda-window", type=float, default=0.18)
 parser.add_argument("--allow-version-mismatch", action="store_true")
 return parser

def main() -> None:
 args = build_arg_parser().parse_args()
 config = Step08MConfig(
 output_dir=args.output_dir,
 tts_archive=args.tts_archive,
 formal_scan_input=args.formal_scan_input,
 n_jobs=args.n_jobs,
 bootstrap_repeats=args.bootstrap_repeats,
 transition_lambda_window=args.transition_lambda_window,
 require_frozen_versions=not args.allow_version_mismatch,
)
 summary = run_step08M(config)
 print("TTS Step08M completed.")
 print("Output directory:", summary["output_dir"])
 print(

```

```

 "Upper mass direction recovered:",
 summary["certificate"]["upper_step06_mass_direction_recovered"],
)
 print(
 "Single global four-family rule supported:",
 summary["certificate"]["single_global_four_family_classifier_supported"],
)

if __name__ == "__main__":
 main()

```

## ► 代码清单 A.50.2 (python, 1603 行)

```

from __future__ import annotations

"""
TTS Step11M — Lieb-aligned fixed-slice mechanism closure
=====

This stage deliberately keeps the five background parameters

 (m_e, t1, t2, r1, r2)

fixed, exactly as a representative reduced-parameter phase diagram should do.

The workflow follows the same logic used in the Lieb study:
1. reduce the topology problem to a physically interpretable two-parameter plane;
2. complete and certify the phase boundaries in that plane;
3. define signed local mass coordinates from those certified boundaries;
4. validate the mass-sign rule with spatially blocked holdout tests;
5. resolve the only remaining local junction by strict Chern mapping and
 selected multi-closure edge certificates.

It does NOT claim a universal seven-dimensional mass formula.
"""

import json
import math
import zipfile
from dataclasses import asdict, dataclass
from pathlib import Path
from typing import Any, Iterable

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from matplotlib.colors import BoundaryNorm, ListedColormap
from sklearn.metrics import accuracy_score, balanced_accuracy_score, f1_score

import TTS_step09M_minus2_local_kp_refinement as base
import TTS_step09M_minus2_linear_signed_v2 as linear_v2
import TTS_step09M_D_multiclosure_path_atlas as step09d
import TTS_step10M_generic_four_valley_boundary_atlas as step10

CODE_VERSION = "TTS_STEP11M_LIEB_ALIGNED_FIXED_SLICE_V1_20260724"
REDUCED7 = list(base.REDUCED7)
FIXED5 = ["m_e", "t1", "t2", "r1", "r2"]
PHASE_CODES = {-2, -1, 0, 1, 2}

PHASE_ORDER = [-2, -1, 0, 1, 2, 99]
PHASE_COLORS = {

```



---

```

2: (242/255, 142/255, 139/255),
1: (127/255, 203/255, 161/255),
0: (178/255, 178/255, 178/255),
-1: (138/255, 163/255, 205/255),
-2: (63/255, 99/255, 173/255),
99: (1.0, 1.0, 1.0),
}

@dataclass
class Step11Config:
 output_dir: Path

 # Left-lower branch completion.
 left_bridge_target_count: int = 2
 left_bridge_min_gap_in_r3: float = 0.020
 bridge_normal_half_width: float = 0.030

 # Right-upper junction strict map.
 junction_r3_range: tuple[float, float] = (0.052, 0.068)
 junction_r4_range: tuple[float, float] = (0.116, 0.140)
 junction_grid_n: int = 7
 junction_max_edge_certificates: int = 3

 # Strict physics.
 strict_gap_grids: tuple[int, ...] = (71, 101)
 strict_chern_grids: tuple[int, ...] = (61, 81, 101)
 strict_chern_shifts: tuple[tuple[float, float], ...] = (
 (0.0, 0.0),
 (0.5, 0.5),
)

 # Local mass-coordinate validation.
 local_mass_band_half_width: float = 0.045
 boundary_exclusion_width: float = 0.004
 spatial_folds: int = 4

 quick: bool = False
 random_seed: int = 20260724

 def normalized(self) -> "Step11Config":
 self.output_dir = Path(self.output_dir).expanduser().resolve()
 self.output_dir.mkdir(parents=True, exist_ok=True)
 (self.output_dir / "bridge_points").mkdir(exist_ok=True)
 (self.output_dir / "junction_points").mkdir(exist_ok=True)
 (self.output_dir / "junction_edges").mkdir(exist_ok=True)
 (self.output_dir / "figures").mkdir(exist_ok=True)
 (self.output_dir / "_strict_core").mkdir(exist_ok=True)
 (self.output_dir / "_search_core").mkdir(exist_ok=True)
 if self.junction_grid_n < 3:
 raise ValueError("junction_grid_n must be >= 3")
 if self.spatial_folds < 2:
 raise ValueError("spatial_folds must be >= 2")
 return self

Generic archive I/O

def read_csv_token(source: str | Path, token: str) -> pd.DataFrame:
 source = Path(source).expanduser().resolve()
 if source.is_file() and source.suffix.lower() == ".zip":
 with zipfile.ZipFile(source) as archive:
 matches = [
 name for name in archive.namelist()
 if token in Path(name).name or token in name
]

```

---

```

 matches = [name for name in matches if name.lower().endswith(".csv")]
 if not matches:
 raise FileNotFoundError(f"No CSV containing {token!r} in {source}")
 selected = sorted(matches, key=lambda x: (len(Path(x).parts), len(x)))[0]
 with archive.open(selected) as stream:
 return pd.read_csv(stream, low_memory=False)
 if source.is_dir():
 matches = sorted(source.rglob(f"*{token}*"))
 if not matches:
 raise FileNotFoundError(token)
 return pd.read_csv(matches[0], low_memory=False)
 raise FileNotFoundError(source)

def read_json_token(source: str | Path, token: str) -> dict[str, Any]:
 source = Path(source).expanduser().resolve()
 if source.is_file() and source.suffix.lower() == ".zip":
 with zipfile.ZipFile(source) as archive:
 matches = [
 name for name in archive.namelist()
 if token in Path(name).name or token in name
]
 matches = [name for name in matches if name.lower().endswith(".json")]
 if not matches:
 raise FileNotFoundError(f"No JSON containing {token!r} in {source}")
 selected = sorted(matches, key=lambda x: (len(Path(x).parts), len(x)))[0]
 with archive.open(selected) as stream:
 return json.load(stream)
 if source.is_dir():
 matches = sorted(source.rglob(f"*{token}*"))
 if not matches:
 raise FileNotFoundError(token)
 return json.loads(matches[0].read_text(encoding="utf-8"))
 raise FileNotFoundError(source)

def phase_code(row: pd.Series | dict[str, Any]) -> int:
 if isinstance(row, dict):
 get = row.get
 else:
 get = row.get
 value = get("paper_phase_code", np.nan)
 if pd.notna(value):
 value = int(value)
 return value if value in PHASE_CODES else 99
 label = str(get("phase_label", ""))
 cup = get("chern_up_int", np.nan)
 if label in {"spin_chern_TI_candidate", "trivial_insulator"} and pd.notna(cup):
 cup = int(cup)
 return cup if cup in PHASE_CODES else 99
 return 99

Fixed-background and branch models

def load_step10_tables(step10_source: str | Path) -> dict[str, pd.DataFrame]:
 return {
 "points": read_csv_token(
 step10_source,
 "step10M_03_certified_generic_boundary_points.csv",
),
 "fits": read_csv_token(
 step10_source,
 "step10M_13_generic_boundary_curve_fits.csv",
),
 "mass_atlas": read_csv_token(

```

---

```

 step10_source,
 "step10M_14_generic_branch_mass_atlas.csv",
),
}

def validate_fixed_background(points: pd.DataFrame, tolerance: float = 1.0e-10) -> dict[str, float]:
 background = {}
 for name in FIXED5:
 values = points[name].to_numpy(float)
 spread = float(np.max(values) - np.min(values))
 if spread > tolerance:
 raise RuntimeError(
 f"Step10 points do not belong to one fixed slice: "
 f"{name} spread={spread:.3e}"
)
 background[name] = float(np.mean(values))
 return background

def fit_row_for_branch(fits: pd.DataFrame, branch: str) -> pd.Series:
 selected = fits[
 fits["branch_hint"].astype(str).eq(branch)
 & fits["fit_available"].astype(int).eq(1)
]
 if len(selected) != 1:
 raise RuntimeError(f"Expected one available fit for {branch}, found {len(selected)}")
 return selected.iloc[0]

def polynomial_value(fit: pd.Series, x: float | np.ndarray) -> float | np.ndarray:
 return (
 float(fit["coefficient_2"]) * np.asarray(x) ** 2
 + float(fit["coefficient_1"]) * np.asarray(x)
 + float(fit["coefficient_0"])
)

def polynomial_derivative(fit: pd.Series, x: float) -> float:
 return 2.0 * float(fit["coefficient_2"]) * float(x) + float(fit["coefficient_1"])

def branch_mass_vector(mass_atlas: pd.DataFrame, branch: str) -> np.ndarray:
 group = (
 mass_atlas[mass_atlas["branch_hint"].astype(str).eq(branch)]
 .set_index("parameter")
 .reindex(REDUCED7)
)
 vector = group["mean_normalized_mass_gradient"].to_numpy(float)
 vector /= max(float(np.linalg.norm(vector)), 1.0e-15)
 return vector

def signed_mass_definition(
 fits: pd.DataFrame,
 mass_atlas: pd.DataFrame,
 branch: str,
) -> dict[str, Any]:
 fit = fit_row_for_branch(fits, branch)
 vector = branch_mass_vector(mass_atlas, branch)
 independent = str(fit["independent_parameter"])
 dependent = str(fit["dependent_parameter"])

 x_mid = 0.5 * (float(fit["x_min"]) + float(fit["x_max"]))
 derivative = polynomial_derivative(fit, x_mid)

 if independent == "r3" and dependent == "r4":
 # F = r4 - f(r3)
 raw_gradient = np.array([-derivative, 1.0], dtype=float)

```

---

```

 raw_expression = (
 f"r4 - ({float(fit['coefficient_2']):+.10g}*r3^2 "
 f"{float(fit['coefficient_1']):+.10g}*r3 "
 f"{float(fit['coefficient_0']):+.10g})"
)
 elif independent == "r4" and dependent == "r3":
 # F = r3 - f(r4)
 raw_gradient = np.array([1.0, -derivative], dtype=float)
 raw_expression = (
 f"r3 - ({float(fit['coefficient_2']):+.10g}*r4^2 "
 f"{float(fit['coefficient_1']):+.10g}*r4 "
 f"{float(fit['coefficient_0']):+.10g})"
)
 else:
 raise RuntimeError(
 f"Unsupported branch fit coordinates: {independent}->{dependent}"
)

plane_mass = vector[-2:]
orientation = 1.0 if float(np.dot(raw_gradient, plane_mass)) >= 0 else -1.0
return {
 "branch_hint": branch,
 "independent_parameter": independent,
 "dependent_parameter": dependent,
 "selected_degree": int(fit["selected_degree"]),
 "coefficient_2": float(fit["coefficient_2"]),
 "coefficient_1": float(fit["coefficient_1"]),
 "coefficient_0": float(fit["coefficient_0"]),
 "x_min": float(fit["x_min"]),
 "x_max": float(fit["x_max"]),
 "orientation": float(orientation),
 "formula": f"{orientation:+.0f} * [{raw_expression}]",
 "mass_gradient_plane_r3": float(plane_mass[0]),
 "mass_gradient_plane_r4": float(plane_mass[1]),
}

def evaluate_signed_mass(definition: dict[str, Any], frame: pd.DataFrame) -> np.ndarray:
 a2 = float(definition["coefficient_2"])
 a1 = float(definition["coefficient_1"])
 a0 = float(definition["coefficient_0"])
 orientation = float(definition["orientation"])
 if definition["independent_parameter"] == "r3":
 raw = frame["r4"].to_numpy(float) - (
 a2 * frame["r3"].to_numpy(float) ** 2
 + a1 * frame["r3"].to_numpy(float)
 + a0
)
 else:
 raw = frame["r3"].to_numpy(float) - (
 a2 * frame["r4"].to_numpy(float) ** 2
 + a1 * frame["r4"].to_numpy(float)
 + a0
)
 return orientation * raw

Left-lower branch completion

def prepare_left_bridge_targets(
 points: pd.DataFrame,
 fits: pd.DataFrame,
 *,
 target_count: int,
 minimum_gap: float,
) -> pd.DataFrame:

```

---

```

branch = "generic_left_lower"
group = (
 points[points["branch_hint"].astype(str).eq(branch)]
 .sort_values("r3")
 .reset_index(drop=True)
)
fit = fit_row_for_branch(fits, branch)
gaps = []
for i in range(len(group) - 1):
 left = float(group.iloc[i]["r3"])
 right = float(group.iloc[i + 1]["r3"])
 gap = right - left
 if gap >= float(minimum_gap):
 gaps.append((gap, left, right))
gaps.sort(reverse=True)

target_values: list[float] = []
for gap, left, right in gaps:
 remaining = max(1, int(round(gap / max(minimum_gap, 1.0e-6))) - 1)
 count = min(remaining, target_count - len(target_values))
 if count <= 0:
 break
 target_values.extend(
 np.linspace(left, right, count + 2, dtype=float)[1:-1].tolist()
)
 if len(target_values) >= target_count:
 break

rows = []
for index, r3 in enumerate(sorted(target_values[:target_count])):
 r4 = float(polynomial_value(fit, r3))
 nearest_index = int(
 np.argmin(
 np.hypot(
 group["r3"].to_numpy(float) - r3,
 group["r4"].to_numpy(float) - r4,
)
)
)
 nearest = group.iloc[nearest_index]
 rows.append({
 "target_id": f"left_bridge_target_{index+1:02d}",
 "branch_hint": branch,
 "target_r3": r3,
 "target_r4": r4,
 "nearest_point_id": str(nearest["point_id"]),
 "nearest_kx": float(nearest["critical_kx_representative"]),
 "nearest_ky": float(nearest["critical_ky_representative"]),
 "nearest_normal_r3": float(nearest["normal_r3"]),
 "nearest_normal_r4": float(nearest["normal_r4"]),
 })
return pd.DataFrame(rows)

```

```

def run_left_bridge_completion(
 *,
 tts_archive: str | Path,
 step10_source: str | Path,
 output_dir: str | Path,
 config: Step11Config,
) -> pd.DataFrame:
 config = config.normalized()
 tables = load_step10_tables(step10_source)
 background = validate_fixed_background(tables["points"])
 targets = prepare_left_bridge_targets(
 tables["points"],
 tables["fits"],
 target_count=config.left_bridge_target_count,
 minimum_gap=config.left_bridge_min_gap_in_r3,
)

```

---

```

)
targets.to_csv(
 config.output_dir / "step11M_01_left_bridge_target_manifest.csv",
 index=False,
 encoding="utf-8-sig",
)
if targets.empty:
 return pd.DataFrame()

core, step4, step5, source_dir = base.import_reference_modules(tts_archive)
linear_v2.patch_step5_linear_path(step5, core)
step3 = linear_v2.import_step3(source_dir)

continuation = step10.ContinuationConfig(
 output_dir=config.output_dir / "_bridge_internal",
 normal_half_width=config.bridge_normal_half_width,
 strict_gap_grids=tuple(config.strict_gap_grids),
 strict_chern_grids=tuple(config.strict_chern_grids),
 strict_chern_shifts=tuple(config.strict_chern_shifts),
 quick=config.quick,
 random_seed=config.random_seed,
).normalized()
strict_config = step10.strict_step3_config(
 step3,
 config.output_dir / "_strict_core",
 continuation,
)
kp_config = step10.step5_config(
 step5,
 config.output_dir / "_search_core",
 continuation,
)
step5._step10m_config = kp_config

fit = fit_row_for_branch(tables["fits"], "generic_left_lower")
result_rows = []
for target_index, target in targets.iterrows():
 point_id = str(target["target_id"])
 point_file = config.output_dir / "bridge_points" / f"{point_id}.json"
 if point_file.is_file():
 payload = json.loads(point_file.read_text(encoding="utf-8"))
 summary = payload.get("point_summary", {})
 if summary:
 result_rows.append(summary)
 continue

 r3 = float(target["target_r3"])
 r4 = float(target["target_r4"])
 derivative = polynomial_derivative(fit, r3)
 normal = np.array([-derivative, 1.0], dtype=float)
 nearest_normal = np.array(
 [float(target["nearest_normal_r3"]), float(target["nearest_normal_r4"])],
 dtype=float,
)
 if float(np.dot(normal, nearest_normal)) < 0:
 normal *= -1.0
 normal /= max(float(np.linalg.norm(normal)), 1.0e-15)

 center = {
 **background,
 "r3": r3,
 "r4": r4,
 }
 previous_k = (
 float(target["nearest_kx"]),
 float(target["nearest_ky"]),
)

closure = None

```

---

```

pair = None
candidate_rows = []
for width_index, width in enumerate(
 (
 config.bridge_normal_half_width,
 1.5 * config.bridge_normal_half_width,
 2.0 * config.bridge_normal_half_width,
)
):
 pair = step10.build_pair(center, normal, float(width), point_id)
 closure, candidates = step10.search_generic_closure(
 step5,
 pair,
 previous_k,
 continuation,
 5000 + 100 * target_index + width_index,
)
 candidates["point_id"] = point_id
 candidates["normal_half_width"] = float(width)
 candidate_rows.extend(candidates.to_dict("records"))
 if closure is not None:
 break

if closure is None or pair is None:
 payload = {
 "point_summary": {
 "point_id": point_id,
 "branch_hint": "generic_left_lower",
 "point_certificate_pass": 0,
 "failure_reason": "no_generic_four_valley_closure",
 "predictor_r3": r3,
 "predictor_r4": r4,
 },
 "search_candidates": candidate_rows,
 }
else:
 payload = step10.analyse_boundary_point(
 core=core,
 step5=step5,
 step3=step3,
 strict_config=strict_config,
 pair=pair,
 closure=closure,
 point_id=point_id,
 branch_hint="generic_left_lower",
 seed_id=str(target["nearest_point_id"]),
 trace_direction=0,
 step_index=int(target_index + 1),
 config=continuation,
)
 payload["search_candidates"] = candidate_rows

point_file.write_text(
 json.dumps(payload, ensure_ascii=False, indent=2),
 encoding="utf-8",
)
summary = payload.get("point_summary", {})
if summary:
 result_rows.append(summary)

result = pd.DataFrame(result_rows)
result.to_csv(
 config.output_dir / "step11M_02_left_bridge_completion_results.csv",
 index=False,
 encoding="utf-8-sig",
)
return result

```

---

```

Strict junction grid

def junction_checkpoint_path(config: Step11Config, ix: int, iy: int) -> Path:
 return config.output_dir / "junction_points" / f"junction_{ix:02d}_{iy:02d}.json"

def run_junction_strict_grid(
 *,
 tts_archive: str | Path,
 step10_source: str | Path,
 output_dir: str | Path,
 config: Step11Config,
) -> pd.DataFrame:
 config = config.normalized()
 tables = load_step10_tables(step10_source)
 background = validate_fixed_background(tables["points"])

 core, step4, step5, source_dir = base.import_reference_modules(tts_archive)
 linear_v2.patch_step5_linear_path(step5, core)
 step3 = linear_v2.import_step3(source_dir)

 continuation = step10.ContinuationConfig(
 output_dir=config.output_dir / "_junction_internal",
 strict_gap_grids=tuple(config.strict_gap_grids),
 strict_chern_grids=tuple(config.strict_chern_grids),
 strict_chern_shifts=tuple(config.strict_chern_shifts),
 quick=config.quick,
 random_seed=config.random_seed,
).normalized()
 strict_config = step10.strict_step3_config(
 step3,
 config.output_dir / "_strict_core",
 continuation,
)

 r3_values = np.linspace(
 config.junction_r3_range[0],
 config.junction_r3_range[1],
 int(config.junction_grid_n),
)
 r4_values = np.linspace(
 config.junction_r4_range[0],
 config.junction_r4_range[1],
 int(config.junction_grid_n),
)

 rows = []
 for iy, r4 in enumerate(r4_values):
 for ix, r3 in enumerate(r3_values):
 path = junction_checkpoint_path(config, ix, iy)
 if path.is_file():
 summary = json.loads(path.read_text(encoding="utf-8"))
 else:
 reduced = {**background, "r3": float(r3), "r4": float(r4)}
 result = step10.evaluate_strict(
 step3,
 strict_config,
 reduced,
 f"step11M_junction_{ix:02d}_{iy:02d}",
)
 summary = result["summary"]
 summary.update({"junction_ix": ix, "junction_iy": iy})
 path.write_text(
 json.dumps(summary, ensure_ascii=False, indent=2),
 encoding="utf-8",
)

```



---

```

 summary = dict(summary)
 summary["junction_ix"] = ix
 summary["junction_iy"] = iy
 summary["paper_phase_code"] = phase_code(summary)
 rows.append(summary)

frame = pd.DataFrame(rows)
frame.to_csv(
 config.output_dir / "step11M_03_right_junction_strict_grid.csv",
 index=False,
 encoding="utf-8-sig",
)
return frame

def junction_boundary_edges(frame: pd.DataFrame) -> pd.DataFrame:
 lookup = {
 (int(row.junction_ix), int(row.junction_iy)): row
 for row in frame.itertuples()
 }
 rows = []
 for (ix, iy), row in lookup.items():
 for neighbour in ((ix + 1, iy), (ix, iy + 1)):
 if neighbour not in lookup:
 continue
 other = lookup[neighbour]
 left_valid = (
 int(getattr(row, "strict_gap_verified", 0) or 0) == 1
 and int(getattr(row, "strict_chern_verified", 0) or 0) == 1
 and int(row.paper_phase_code) in PHASE_CODES
)
 right_valid = (
 int(getattr(other, "strict_gap_verified", 0) or 0) == 1
 and int(getattr(other, "strict_chern_verified", 0) or 0) == 1
 and int(other.paper_phase_code) in PHASE_CODES
)
 if not (left_valid and right_valid):
 continue
 c0 = int(row.paper_phase_code)
 c1 = int(other.paper_phase_code)
 if c0 == c1:
 continue
 delta = c1 - c0
 if abs(delta) == 4:
 candidate = "generic_four_valley_candidate"
 elif abs(delta) == 2:
 candidate = "two_valley_or_multiclosure_candidate"
 elif abs(delta) == 1:
 candidate = "single_valley_candidate"
 else:
 candidate = "multiclosure_candidate"
 rows.append({
 "edge_id": f"edge_{ix:02d}_{iy:02d}_{neighbour[0]:02d}_{neighbour[1]:02d}",
 "ix0": ix,
 "iy0": iy,
 "ix1": neighbour[0],
 "iy1": neighbour[1],
 "r3_0": float(row.r3),
 "r4_0": float(row.r4),
 "chern_0": c0,
 "r3_1": float(other.r3),
 "r4_1": float(other.r4),
 "chern_1": c1,
 "delta_chern_up": delta,
 "mid_r3": 0.5 * (float(row.r3) + float(other.r3)),
 "mid_r4": 0.5 * (float(row.r4) + float(other.r4)),
 "candidate_mechanism": candidate,
 })
 return pd.DataFrame(rows)

```

---

```

def select_representative_junction_edges(
 edges: pd.DataFrame,
 max_edges: int,
) -> pd.DataFrame:
 if edges.empty:
 return edges.copy()
 selected_rows = []
 # First retain one edge for every distinct signed Chern transition.
 for transition, group in edges.groupby(["chern_0", "chern_1"]):
 group = group.copy()
 center = group[["mid_r3", "mid_r4"]].mean().to_numpy(float)
 distance = np.linalg.norm(
 group[["mid_r3", "mid_r4"]].to_numpy(float) - center,
 axis=1,
)
 selected_rows.append(group.iloc[int(np.argmin(distance))])
 selected = pd.DataFrame(selected_rows)
 if len(selected) > max_edges:
 selected = selected.head(max_edges)
 elif len(selected) < max_edges:
 remaining = edges[~edges["edge_id"].isin(selected["edge_id"])].copy()
 while len(selected) < max_edges and len(remaining):
 selected_points = selected[["mid_r3", "mid_r4"]].to_numpy(float)
 candidates = remaining[["mid_r3", "mid_r4"]].to_numpy(float)
 min_dist = np.min(
 np.linalg.norm(
 candidates[:, None, :] - selected_points[None, :, :],
 axis=2,
),
 axis=1,
)
 pick = int(np.argmax(min_dist))
 selected = pd.concat(
 [selected, remaining.iloc[[pick]]],
 ignore_index=True,
)
 remaining = remaining.drop(remaining.index[pick])
 return selected.reset_index(drop=True)

Selected junction-edge multi-closure certificates

def build_pair_from_edge(
 edge: pd.Series,
 background: dict[str, float],
) -> pd.Series:
 start = pd.Series(
 {
 **background,
 "r3": float(edge["r3_0"]),
 "r4": float(edge["r4_0"]),
 }
)
 end = pd.Series(
 {
 **background,
 "r3": float(edge["r3_1"]),
 "r4": float(edge["r4_1"]),
 }
)
 return base.build_pair_series(start, end, path_id=str(edge["edge_id"]))

def analyse_junction_edge_multiclosure(

```

```

*,
edge: pd.Series,
background: dict[str, float],
core,
step5,
step3,
strict_config,
search_config,
multi_config: step09d.MultiClosureConfig,
) -> dict[str, Any]:
 edge_id = str(edge["edge_id"])
 pair = build_pair_from_edge(edge, background)

 gap_scan = step09d.dense_gap_scan(step5, pair, multi_config, edge_id)
 coarse, coarse_attempts = step09d.coarse_chern_scan(
 core,
 step5,
 pair,
 multi_config,
 edge_id,
)
 brackets = step09d.detect_candidate_brackets(
 gap_scan,
 coarse,
 multi_config,
)

 all_closures = []
 search_rows = []
 for bracket_index, bracket_row in brackets.iterrows():
 bracket = pd.Series({
 "transition_id": f"{edge_id}_bracket{bracket_index:02d}",
 "path_id": edge_id,
 "lambda_left": float(bracket_row["lambda_left"]),
 "lambda_right": float(bracket_row["lambda_right"]),
 "chern_left": np.nan,
 "chern_right": np.nan,
 })
 candidates, closures = step5.search_transition_closures(
 bracket,
 pair,
 search_config,
 9000 + int(bracket_index),
)
 search_rows.extend(
 [
 {
 "edge_id": edge_id,
 "bracket_index": int(bracket_index),
 **row,
 }
 for row in candidates
]
)
 for closure in closures:
 item = dict(closure)
 item["source_bracket_index"] = int(bracket_index)
 all_closures.append(item)

 unique = step09d.global_deduplicate_closures(
 step5,
 all_closures,
 multi_config.closure_lambda_dedup_tol,
)
 groups = step09d.closure_groups(unique)
 centres = [
 float(np.mean([float(row["critical_lambda"]) for row in group]))
 for group in groups
]

```

---

```

segment_results = []
segment_rows = []
boundaries = [0.0, *centres, 1.0]
for segment_index, (left, right) in enumerate(
 zip(boundaries[:-1], boundaries[1:]))
):
 probe = step09d.find_reliable_segment_probe(
 step3,
 strict_config,
 pair,
 edge_id,
 float(left),
 float(right),
 multi_config.segment_probe_fractions,
 segment_index,
)
 segment_results.append(probe)
 summary = probe["summary"]
 segment_rows.append({
 "edge_id": edge_id,
 "segment_index": segment_index,
 "lambda_left_bound": left,
 "lambda_right_bound": right,
 "lambda": summary.get("lambda"),
 "strict_chern_up": step09d.reliable_chern(summary),
 "strict_phase_label": summary.get("phase_label"),
 "min_direct_gap": summary.get("min_direct_gap"),
 })

closure_rows = []
kp_rows = []
gradient_rows = []
berry_rows = []
group_certificates = []

for group_index, group in enumerate(groups):
 group_id = f"{edge_id}_group{group_index:02d}"
 group_charge = 0
 all_consensus = True
 all_rank3 = True
 valley_count = 0
 for closure_index, closure in enumerate(group):
 closure_id = f"{group_id}_closure{closure_index:02d}"
 reduced = linear_v2.linear_reduced_on_path(
 pair,
 float(closure["critical_lambda"]),
)
 closure_rows.append({
 "edge_id": edge_id,
 "closure_group_id": group_id,
 "closure_id": closure_id,
 **closure,
 **reduced,
 })
 analysis = step09d.analyse_closure(
 step5,
 pair,
 search_config,
 edge_id,
 group_id,
 closure_id,
 closure,
)
 kp_rows.extend(analysis["kp_rows"])
 gradient_rows.extend(analysis["gradient_rows"])
 berry_rows.extend(analysis["berry_summary_rows"])
 group_charge += int(analysis["charge_sum"])
 all_consensus = all_consensus and bool(analysis["all_consensus"])

```

---

```

 all_rank3 = all_rank3 and bool(analysis["all_rank3"])
 valley_count += int(analysis["n_active_valleys"])

 left_chern = (
 step09d.reliable_chern(segment_results[group_index]["summary"])
 if group_index < len(segment_results) else None
)
 right_chern = (
 step09d.reliable_chern(segment_results[group_index + 1]["summary"])
 if group_index + 1 < len(segment_results) else None
)
 delta = (
 None
 if left_chern is None or right_chern is None
 else int(right_chern - left_chern)
)
 signed_match = (
 delta is not None
 and all_consensus
 and int(group_charge) == int(delta)
)
 group_certificates.append({
 "edge_id": edge_id,
 "closure_group_id": group_id,
 "mean_critical_lambda": centres[group_index],
 "n_closure_orbits": len(group),
 "n_active_spin_up_valleys": valley_count,
 "left_chern_up": left_chern,
 "right_chern_up": right_chern,
 "observed_delta_chern_up": delta,
 "berry_charge_sum_up": group_charge,
 "all_berry_charges_consensus": int(all_consensus),
 "all_kp_jacobians_rank3": int(all_rank3),
 "signed_charge_matches": int(signed_match),
 "closure_group_certificate_pass": int(signed_match and all_rank3),
 })

 endpoint_delta = int(edge["chern_1"] - edge["chern_0"])
 total_charge = int(sum(row["berry_charge_sum_up"] for row in group_certificates))
 all_groups_pass = bool(
 len(group_certificates)
 and all(
 int(row["closure_group_certificate_pass"]) == 1
 for row in group_certificates
)
)
 edge_certificate = {
 "edge_id": edge_id,
 "grid_chern_0": int(edge["chern_0"]),
 "grid_chern_1": int(edge["chern_1"]),
 "grid_endpoint_delta_chern_up": endpoint_delta,
 "n_closure_groups": len(groups),
 "n_closure_orbits": len(unique),
 "n_active_spin_up_valleys_total": int(
 sum(row["n_active_spin_up_valleys"] for row in group_certificates)
),
 "berry_charge_sum_over_all_groups": total_charge,
 "signed_total_charge_matches_grid_delta": int(
 total_charge == endpoint_delta
),
 "all_closure_groups_certified": int(all_groups_pass),
 "edge_multiclosure_certificate_pass": int(
 all_groups_pass and total_charge == endpoint_delta
),
 }
}
return {
 "edge": edge.to_dict(),
 "gap_scan": gap_scan.to_dict("records"),
 "coarse_chern": coarse.to_dict("records"),

```

---

```

 "coarse_chn_attempts": coarse_attempts.to_dict("records"),
 "candidate_brackets": brackets.to_dict("records"),
 "search_candidates": search_rows,
 "closures": closure_rows,
 "segments": segment_rows,
 "kp_summaries": kp_rows,
 "mass_gradients": gradient_rows,
 "berry_summaries": berry_rows,
 "group_certificates": group_certificates,
 "edge_certificate": edge_certificate,
 }

def run_junction_edge_certificates(
 *,
 tts_archive: str | Path,
 step10_source: str | Path,
 junction_grid: pd.DataFrame,
 output_dir: str | Path,
 config: Step11Config,
) -> pd.DataFrame:
 config = config.normalized()
 tables = load_step10_tables(step10_source)
 background = validate_fixed_background(tables["points"])
 edges = junction_boundary_edges(junction_grid)
 edges.to_csv(
 config.output_dir / "step11M_04_right_junction_boundary_edges.csv",
 index=False,
 encoding="utf-8-sig",
)
 selected = select_representative_junction_edges(
 edges,
 config.junction_max_edge_certificates,
)
 selected.to_csv(
 config.output_dir / "step11M_05_selected_junction_edges.csv",
 index=False,
 encoding="utf-8-sig",
)
 if selected.empty:
 return pd.DataFrame()

 core, step4, step5, source_dir = base.import_reference_modules(tts_archive)
 linear_v2.patch_step5_linear_path(step5, core)
 step3 = linear_v2.import_step3(source_dir)

 multi_config = step09d.MultiClosureConfig(
 output_dir=config.output_dir / "_junction_multiclosure",
 lambda_points=(151 if config.quick else 301),
 diagonal_k_points=(81 if config.quick else 141),
 candidate_gap_ceiling=2.0e-2,
 candidate_log_prominence=0.25,
 candidate_min_index_distance=4,
 bracket_half_width_indices=6,
 coarse_chn_lambda_points=(11 if config.quick else 17),
 coarse_chn_grids=((21,) if config.quick else (21, 31)),
 coarse_chn_shifts=((0.0, 0.0),),
 strict_chn_grids=tuple(config.strict_chn_grids),
 strict_chn_shifts=tuple(config.strict_chn_shifts),
 strict_gap_grids=tuple(config.strict_gap_grids),
 full_search_quick=config.quick,
 random_seed=config.random_seed,
).normalized()

 strict_config = step09d.configure_strict_step3(
 step3,
 config.output_dir / "_strict_core",
 multi_config,
)

```

---

```

search_config = step09d.configure_step5_search(
 step5,
 config.output_dir / "_search_core",
 multi_config,
)

certificate_rows = []
for _, edge in selected.iterrows():
 edge_id = str(edge["edge_id"])
 path = config.output_dir / "junction_edges" / f"{edge_id}.json"
 if path.is_file():
 payload = json.loads(path.read_text(encoding="utf-8"))
 else:
 payload = analyse_junction_edge_multiclosure(
 edge=edge,
 background=background,
 core=core,
 step5=step5,
 step3=step3,
 strict_config=strict_config,
 search_config=search_config,
 multi_config=multi_config,
)
 path.write_text(
 json.dumps(payload, ensure_ascii=False, indent=2),
 encoding="utf-8",
)
 certificate_rows.append(payload["edge_certificate"])

result = pd.DataFrame(certificate_rows)
result.to_csv(
 config.output_dir / "step11M_06_junction_edge_certificates.csv",
 index=False,
 encoding="utf-8-sig",
)
return result

Lieb-style local signed-mass validation

def load_formal_lower_grid(formal_scan_source: str | Path) -> pd.DataFrame:
 frame = base.load_lower_grid(formal_scan_source)
 frame = frame.copy()
 frame["paper_phase_code"] = frame.apply(phase_code, axis=1)
 if "r3" not in frame:
 frame["r3"] = frame["scan_x"]
 if "r4" not in frame:
 frame["r4"] = frame["scan_y"]
 return frame

def mapping_from_train(sign_values: np.ndarray, labels: np.ndarray) -> dict[int, int]:
 mapping = {}
 for sign in (-1, 1):
 selected = labels[sign_values == sign]
 if len(selected) == 0:
 continue
 values, counts = np.unique(selected, return_counts=True)
 mapping[sign] = int(values[int(np.argmax(counts))])
 return mapping

def validate_one_branch_mass(
 grid: pd.DataFrame,
 definition: dict[str, Any],
 *,

```

---

```

band_half_width: float,
boundary_exclusion: float,
spatial_folds: int,
) -> tuple[pd.DataFrame, dict[str, Any]]:
 frame = grid.copy()
 frame["signed_mass"] = evaluate_signed_mass(definition, frame)
 independent = definition["independent_parameter"]
 x = frame[independent].to_numpy(float)

 x_min = float(definition["x_min"])
 x_max = float(definition["x_max"])
 x_margin = max(0.02, 0.15 * (x_max - x_min))
 selected = frame[
 frame["paper_phase_code"].isin([-2, 2])
 & (frame["signed_mass"].abs() <= float(band_half_width))
 & (frame["signed_mass"].abs() >= float(boundary_exclusion))
 & (frame[independent] >= x_min - x_margin)
 & (frame[independent] <= x_max + x_margin)
].copy()
 if len(selected) < 8:
 return selected, {
 "branch_hint": definition["branch_hint"],
 "n_points": int(len(selected)),
 "n_folds_evaluated": 0,
 "balanced_accuracy": np.nan,
 "accuracy": np.nan,
 "macro_f1": np.nan,
 "status": "insufficient_local_grid_points",
 }

 selected["mass_sign"] = np.where(selected["signed_mass"] >= 0, 1, -1)
 edges = np.linspace(
 float(selected[independent].min()),
 float(selected[independent].max()),
 int(spatial_folds) + 1,
)
 selected["spatial_fold"] = np.clip(
 np.digitize(selected[independent], edges[1:-1], right=False),
 0,
 spatial_folds - 1,
)

 prediction_rows = []
 for fold in sorted(selected["spatial_fold"].unique()):
 train = selected[selected["spatial_fold"] != fold]
 test = selected[selected["spatial_fold"] == fold]
 if len(test) == 0:
 continue
 mapping = mapping_from_train(
 train["mass_sign"].to_numpy(int),
 train["paper_phase_code"].to_numpy(int),
)
 if set(mapping) != {-1, 1}:
 continue
 predicted = np.array(
 [mapping[int(sign)] for sign in test["mass_sign"]],
 dtype=int,
)
 part = test[
 ["r3", "r4", "signed_mass", "mass_sign", "paper_phase_code", "spatial_fold"]
].copy()
 part["prediction"] = predicted
 part["branch_hint"] = definition["branch_hint"]
 prediction_rows.append(part)

 if not prediction_rows:
 return selected, {
 "branch_hint": definition["branch_hint"],
 "n_points": int(len(selected)),

```



---

```

 "n_folds_evaluated": 0,
 "balanced_accuracy": np.nan,
 "accuracy": np.nan,
 "macro_f1": np.nan,
 "status": "no_valid_spatial_folds",
 }

 predictions = pd.concat(prediction_rows, ignore_index=True)
 y = predictions["paper_phase_code"].to_numpy(int)
 p = predictions["prediction"].to_numpy(int)
 metrics = {
 "branch_hint": definition["branch_hint"],
 "n_points": int(len(selected)),
 "n_predictions": int(len(predictions)),
 "n_folds_evaluated": int(predictions["spatial_fold"].nunique()),
 "balanced_accuracy": float(balanced_accuracy_score(y, p)),
 "accuracy": float(accuracy_score(y, p)),
 "macro_f1": float(f1_score(y, p, average="macro")),
 "status": "evaluated",
 }
 return predictions, metrics

Aggregation and figures

def collect_bridge_payloads(config: Step11Config) -> list[dict[str, Any]]:
 rows = []
 for path in sorted((config.output_dir / "bridge_points").glob("*.json")):
 rows.append(json.loads(path.read_text(encoding="utf-8")))
 return rows

def refit_completed_branches(
 step10_points: pd.DataFrame,
 bridge_payloads: list[dict[str, Any]],
 config: Step11Config,
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
 rows = [step10_points]
 new_rows = []
 for payload in bridge_payloads:
 summary = payload.get("point_summary", {})
 if int(summary.get("point_certificate_pass", 0) or 0) == 1:
 new_rows.append(summary)
 if new_rows:
 rows.append(pd.DataFrame(new_rows))
 points = pd.concat(rows, ignore_index=True)
 points = step10.deduplicate_points(points, distance=2.5e-3)

 continuation = step10.ContinuationConfig(
 output_dir=config.output_dir / "_fit_internal",
 min_branch_points_for_fit=4,
)
 fit_rows = [
 step10.fit_branch(group, branch, 4)
 for branch, group in points.groupby("branch_hint")
]
 fits = pd.DataFrame(fit_rows)
 mass_atlas = step10.branch_mass_atlas(
 points,
 bootstrap_repeats=(200 if config.quick else 1000),
 seed=config.random_seed,
)
 return points, fits, mass_atlas

def plot_fixed_slice_phase_map(

```

---

```

formal_grid: pd.DataFrame,
certified_points: pd.DataFrame,
fits: pd.DataFrame,
junction_grid: pd.DataFrame,
output_path: Path,
) -> None:
 cmap = ListedColormap([PHASE_COLORS[value] for value in PHASE_ORDER])
 norm = BoundaryNorm(
 [-2.5, -1.5, -0.5, 0.5, 1.5, 2.5, 99.5],
 cmap.N,
)

 pivot = formal_grid.pivot(
 index="scan_iy",
 columns="scan_ix",
 values="paper_phase_code",
).sort_index()
 x = formal_grid.pivot(
 index="scan_iy",
 columns="scan_ix",
 values="scan_x",
).sort_index().iloc[0].to_numpy(float)
 y = formal_grid.pivot(
 index="scan_iy",
 columns="scan_ix",
 values="scan_y",
).sort_index().iloc[:, 0].to_numpy(float)

 def edges(values: np.ndarray) -> np.ndarray:
 mids = 0.5 * (values[:-1] + values[1:])
 return np.r_[
 values[0] - (mids[0] - values[0]),
 mids,
 values[-1] + (values[-1] - mids[-1]),
]

 fig, ax = plt.subplots(figsize=(16.0 / 2.54, 10.0 / 2.54))
 ax.pcolormesh(
 edges(x),
 edges(y),
 pivot.to_numpy(float),
 cmap=cmap,
 norm=norm,
 shading="flat",
)

 markers = {
 "generic_left_lower": "s",
 "generic_right_upper": "^",
 }
 for branch, group in certified_points.groupby("branch_hint"):
 ax.scatter(
 group["r3"],
 group["r4"],
 s=25,
 marker=markers.get(branch, "D"),
 facecolors="white",
 edgecolors="black",
 linewidths=0.8,
 label=branch.replace("generic_", "") + " certified",
)
 fit = fits[
 fits["branch_hint"].astype(str).eq(branch)
 & fits["fit_available"].astype(int).eq(1)
]
 if len(fit):
 row = fit.iloc[0]
 xx = np.linspace(float(row["x_min"]), float(row["x_max"]), 300)
 yy = polynomial_value(row, xx)

```

---

```

 if row["independent_parameter"] == "r3":
 ax.plot(xx, yy, "k-", linewidth=1.0)
 else:
 ax.plot(yy, xx, "k-", linewidth=1.0)

if len(junction_grid):
 reliable = junction_grid[
 junction_grid["paper_phase_code"].isin([-2, 0, 2])
]
 ax.scatter(
 reliable["r3"],
 reliable["r4"],
 s=10,
 marker=".",
 color="black",
 label="strict junction grid",
)

ax.set_xlabel(r"r_3")
ax.set_ylabel(r"r_4")
ax.set_xlim(-0.82, 0.11)
ax.set_ylim(-0.21, 0.16)
ax.tick_params(direction="in", top=True, right=True)
ax.legend(frameon=False, loc="upper left", bbox_to_anchor=(1.01, 1.0))
fig.tight_layout()
fig.savefig(output_path.with_suffix(".png"), dpi=1200, bbox_inches="tight")
fig.savefig(output_path.with_suffix(".pdf"), bbox_inches="tight")
fig.savefig(output_path.with_suffix(".svg"), bbox_inches="tight")
plt.close(fig)

def plot_mass_coordinate_validation(
 predictions: pd.DataFrame,
 output_path: Path,
) -> None:
 if predictions.empty:
 return
 fig, ax = plt.subplots(figsize=(12.0 / 2.54, 7.0 / 2.54))
 markers = {
 "generic_left_lower": "s",
 "generic_right_upper": "^",
 }
 for branch, group in predictions.groupby("branch_hint"):
 ax.scatter(
 group["signed_mass"],
 group["paper_phase_code"],
 s=20,
 marker=markers.get(branch, "o"),
 facecolors="none",
 edgecolors="black",
 linewidths=0.7,
 label=branch.replace("generic_", ""),
)
 ax.axvline(0.0, linestyle="--", linewidth=0.8, color="black")
 ax.set_xlabel("local signed mass coordinate")
 ax.set_ylabel(r"$C_{\uparrow}$")
 ax.set_yticks([-2, 2])
 ax.tick_params(direction="in", top=True, right=True)
 ax.legend(frameon=False)
 fig.tight_layout()
 fig.savefig(output_path.with_suffix(".png"), dpi=1200, bbox_inches="tight")
 fig.savefig(output_path.with_suffix(".pdf"), bbox_inches="tight")
 fig.savefig(output_path.with_suffix(".svg"), bbox_inches="tight")
 plt.close(fig)

def aggregate_step11(
 *,
 formal_scan_source: str | Path,

```

---

```

step10_source: str | Path,
output_dir: str | Path,
config: Step11Config,
) -> dict[str, Any]:
 config = config.normalized()
 tables = load_step10_tables(step10_source)
 background = validate_fixed_background(tables["points"])
 bridge_payloads = collect_bridge_payloads(config)
 points, fits, mass_atlas = refit_completed_branches(
 tables["points"],
 bridge_payloads,
 config,
)

 points.to_csv(
 config.output_dir / "step11M_07_completed_certified_boundary_points.csv",
 index=False,
 encoding="utf-8-sig",
)
 fits.to_csv(
 config.output_dir / "step11M_08_fixed_slice_boundary_curve_fits.csv",
 index=False,
 encoding="utf-8-sig",
)
 mass_atlas.to_csv(
 config.output_dir / "step11M_09_fixed_slice_local_mass_atlas.csv",
 index=False,
 encoding="utf-8-sig",
)

 definitions = []
 for branch in sorted(points["branch_hint"].dropna().unique()):
 definitions.append(
 signed_mass_definition(fits, mass_atlas, str(branch))
)
 definitions_df = pd.DataFrame(definitions)
 definitions_df.to_csv(
 config.output_dir / "step11M_10_local_signed_mass_definitions.csv",
 index=False,
 encoding="utf-8-sig",
)

 formal_grid = load_formal_lower_grid(formal_scan_source)
 prediction_frames = []
 metric_rows = []
 for definition in definitions:
 predictions, metrics = validate_one_branch_mass(
 formal_grid,
 definition,
 band_half_width=config.local_mass_band_half_width,
 boundary_exclusion=config.boundary_exclusion_width,
 spatial_folds=config.spatial_folds,
)
 if not predictions.empty and "prediction" in predictions:
 prediction_frames.append(predictions)
 metric_rows.append(metrics)

 predictions_df = (
 pd.concat(prediction_frames, ignore_index=True)
 if prediction_frames else pd.DataFrame()
)
 metrics_df = pd.DataFrame(metric_rows)
 predictions_df.to_csv(
 config.output_dir / "step11M_11_spatial_holdout_predictions.csv",
 index=False,
 encoding="utf-8-sig",
)
 metrics_df.to_csv(
 config.output_dir / "step11M_12_spatial_holdout_mass_metrics.csv",

```

---

```

 index=False,
 encoding="utf-8-sig",
)

 junction_path = (
 config.output_dir / "step11M_03_right_junction_strict_grid.csv"
)
 junction_grid = (
 pd.read_csv(junction_path, low_memory=False)
 if junction_path.is_file() else pd.DataFrame()
)
 edge_cert_path = (
 config.output_dir / "step11M_06_junction_edge_certificates.csv"
)
 edge_certificates = (
 pd.read_csv(edge_cert_path, low_memory=False)
 if edge_cert_path.is_file() else pd.DataFrame()
)

 plot_fixed_slice_phase_map(
 formal_grid,
 points,
 fits,
 junction_grid,
 config.output_dir / "figures" / "step11M_fixed_slice_mechanism_map",
)
 plot_mass_coordinate_validation(
 predictions_df,
 config.output_dir / "figures" / "step11M_local_mass_coordinate_validation",
)

 background_json = {
 name: float(value) for name, value in background.items()
 }
 (config.output_dir / "step11M_00_fixed_background.json").write_text(
 json.dumps(background_json, ensure_ascii=False, indent=2),
 encoding="utf-8",
)

 bridge_new = points[
 points["point_id"].astype(str).str.startswith("left_bridge_target_")
]
 certificate = {
 "code_version": CODE_VERSION,
 "research_design": "Lieb_aligned_fixed_reduced_plane",
 "fixed_background_parameters": background_json,
 "free_plane_parameters": ["r3", "r4"],
 "n_step10_input_points": int(len(tables["points"])),
 "n_new_certified_bridge_points": int(len(bridge_new)),
 "n_completed_certified_points": int(len(points)),
 "n_local_signed_mass_definitions": int(len(definitions_df)),
 "all_background_parameters_remain_fixed": True,
 "mass_coordinate_validation": metrics_df.to_dict("records"),
 "junction_grid_completed": bool(len(junction_grid) > 0),
 "n_junction_reliable_points": int(
 junction_grid["paper_phase_code"].isin([-2, 0, 2]).sum()
) if len(junction_grid) else 0,
 "n_junction_edge_certificates": int(len(edge_certificates)),
 "n_junction_edges_certified": int(
 edge_certificates.get(
 "edge_multiclosure_certificate_pass",
 pd.Series(dtype=int),
).fillna(0).astype(int).sum()
) if len(edge_certificates) else 0,
 "interpretation": {
 "methodological_scope": (
 "This is a representative fixed-background r3-r4 mechanism plane, "
 "analogous to the fixed-parameter planes used in the Lieb analysis."
),
 },
 },

```

---

```

 "valid_claim": (
 "The certified local signed masses organize the -2/+2 boundaries "
 "within this reduced plane."
),
 "not_claimed": (
 "No universal seven-dimensional phase boundary is claimed or required."
),
 },
}
(config.output_dir / "step11M_13_fixed_slice_certificate.json").write_text(
 json.dumps(certificate, ensure_ascii=False, indent=2),
 encoding="utf-8",
)

formula_lines = [
 "# TTS Step11M local signed masses",
 "",
 "Fixed background:",
 *[
 f"- {name} = {background[name]:+.10g}"
 for name in FIXED5
],
 "",
]
for definition in definitions:
 formula_lines.extend([
 f"## {definition['branch_hint']}",
 "",
 f"M = {definition['formula']}",
 "",
 (
 f"Validated only for "
 f"{definition['independent_parameter']} in "
 f"[{definition['x_min']:+.6f}, {definition['x_max']:+.6f}]."
),
 "",
])
(config.output_dir / "step11M_14_local_signed_mass_formulas.md").write_text(
 "\n".join(formula_lines),
 encoding="utf-8",
)
return certificate

Complete public workflow

def run_step11m(
 *,
 tts_archive: str | Path,
 formal_scan_source: str | Path,
 step10_source: str | Path,
 output_dir: str | Path,
 config: Step11Config | None = None,
 run_bridge: bool = True,
 run_unction_grid: bool = True,
 run_unction_edges: bool = True,
) -> dict[str, Any]:
 if config is None:
 config = Step11Config(output_dir=Path(output_dir))
 config.output_dir = Path(output_dir)
 config = config.normalized()

 if run_bridge:
 run_left_bridge_completion(
 tts_archive=tts_archive,
 step10_source=step10_source,

```

```

 output_dir=output_dir,
 config=config,
)

 junction_grid = pd.DataFrame()
 if run_junction_grid:
 junction_grid = run_junction_strict_grid(
 tts_archive=tts_archive,
 step10_source=step10_source,
 output_dir=output_dir,
 config=config,
)
 else:
 path = config.output_dir / "step11M_03_right_junction_strict_grid.csv"
 if path.is_file():
 junction_grid = pd.read_csv(path, low_memory=False)

 if run_junction_edges and len(junction_grid):
 run_junction_edge_certificates(
 tts_archive=tts_archive,
 step10_source=step10_source,
 junction_grid=junction_grid,
 output_dir=output_dir,
 config=config,
)

 (config.output_dir / "step11M_run_config.json").write_text(
 json.dumps(asdict(config), ensure_ascii=False, indent=2, default=str),
 encoding="utf-8",
)
 return aggregate_step11(
 formal_scan_source=formal_scan_source,
 step10_source=step10_source,
 output_dir=output_dir,
 config=config,
)

```

## ► 代码清单 A.50.3 (python, 1738 行)

```

from __future__ import annotations

"""
TTS Step12M — right-junction multi-closure repair and publication phase map
=====

Purpose

Resolve the only failed Step11M edge certificate:

 edge_03_03_03_04:
 (r3, r4) = (0.060, 0.128) -> (0.060, 0.132)
 C_up = -2 -> 0

Step11M found a certified Sigma' two-valley closing with Berry charge -2, but
that charge did not equal the endpoint Chern change +2. Step12M therefore
forces a separate full-Brillouin-zone search for a preceding generic four-valley
closing. The target hypothesis is

 C_up: -2 --(four generic valleys, +4)--> +2
 --(two Sigma' valleys, -2)--> 0.

The workflow also recomputes a refined strict 13x13 junction map and produces
publication-oriented figures with phase colors, unreliable-point markers,

```

---

legends, a junction zoom, the target path, and certified closure annotations.

Scope

-----

All background parameters ( $m_e$ ,  $t_1$ ,  $t_2$ ,  $r_1$ ,  $r_2$ ) remain fixed. Only the same Lieb-aligned  $r_3$ - $r_4$  plane is studied. No universal seven-dimensional boundary is claimed.

"""

```
import argparse
import json
import math
import sys
import zipfile
from dataclasses import asdict, dataclass
from pathlib import Path
from typing import Any, Iterable

import matplotlib.pyplot as plt
from matplotlib.colors import BoundaryNorm, ListedColormap
from matplotlib.lines import Line2D
from matplotlib.patches import Patch, Rectangle
import numpy as np
import pandas as pd
from scipy.optimize import differential_evolution, minimize, minimize_scalar

import TTS_step09M_minus2_local_kp_refinement as base
import TTS_step09M_minus2_linear_signed_v2 as linear_v2
import TTS_step09M_D_multiclosure_path_atlas as step09d
import TTS_step10M_generic_four_valley_boundary_atlas as step10
import TTS_step11M_lieb_aligned_fixed_slice as step11
```

```
CODE_VERSION = "TTS_STEP12M_JUNCTION_MULTICLOSURE_REPAIR_V1_2_FORCE_SOURCE_LOAD_20260724"
REDUCED7 = list(base.REDUCED7)
FIXED5 = ["m_e", "t1", "t2", "r1", "r2"]
TARGET_EDGE_ID = "edge_03_03__03_04"
```

```
PHASE_COLORS = {
 -2: "#3B6FB6",
 -1: "#7FB6D6",
 0: "#F2F2F2",
 1: "#F2B36D",
 2: "#C84A3A",
}
PHASE_ORDER = [-2, -1, 0, 1, 2]
```

```
@dataclass
class Step12Config:
 output_dir: Path

 # Target edge is fixed by the Step11M junction grid.
 target_edge_id: str = TARGET_EDGE_ID

 # Targeted generic four-valley search. The expected direct boundary from
 # the certified right-upper branch intersects $r_3=0.060$ near $\lambda=0.624$.
 generic_lambda_bracket: tuple[float, float] = (0.42, 0.80)
 sigma_prime_lambda_bracket: tuple[float, float] = (0.80, 1.00)
 generic_predicted_lambda: float = 0.6244
 generic_k_box_half_width: float = 0.90
 generic_powell_restarts: int = 12
 generic_de_repeats: int = 4
 closure_gap_tol: float = 2.0e-8
 closure_lambda_dedup_tol: float = 8.0e-4

 # Diagnostic gap tracks along the edge.
 gap_track_lambda_points: int = 241
 sigma_prime_kappa_points: int = 241
```



---

```

Refined strict junction map. This range contains r3=0.060 and both
endpoints r4=0.128, 0.132, while extending above the second closing.
refined_r3_range: tuple[float, float] = (0.056, 0.064)
refined_r4_range: tuple[float, float] = (0.128, 0.134)
refined_grid_n: int = 13

strict_gap_grids: tuple[int, ...] = (71, 101)
strict_chern_grids: tuple[int, ...] = (61, 81, 101)
strict_chern_shifts: tuple[tuple[float, float], ...] = (
 (0.0, 0.0),
 (0.5, 0.5),
)

quick: bool = False
force_recalculate: bool = False
random_seed: int = 20260724

def normalized(self) -> "Step12Config":
 self.output_dir = Path(self.output_dir).expanduser().resolve()
 self.output_dir.mkdir(parents=True, exist_ok=True)
 for sub in [
 "figures",
 "refined_junction_points",
 "_search_core",
 "_strict_core",
]:
 (self.output_dir / sub).mkdir(exist_ok=True)
 if self.refined_grid_n < 5:
 raise ValueError("refined_grid_n must be >= 5")
 if self.gap_track_lambda_points < 61:
 raise ValueError("gap_track_lambda_points must be >= 61")
 lo, hi = sorted(self.generic_lambda_bracket)
 if not (0.0 <= lo < hi <= 1.0):
 raise ValueError("generic_lambda_bracket must lie in [0,1]")
 return self

=====
Generic I/O
=====

def _select_member(zf: zipfile.ZipFile, token: str, suffix: str | None = None) -> str:
 matches = []
 for name in zf.namelist():
 if token not in Path(name).name and token not in name:
 continue
 if suffix is not None and not name.lower().endswith(suffix.lower()):
 continue
 matches.append(name)
 if not matches:
 raise FileNotFoundError(f"No member containing {token!r}")
 return sorted(matches, key=lambda x: (len(Path(x).parts), len(x)))[0]

def read_csv_token(source: str | Path, token: str) -> pd.DataFrame:
 source = Path(source).expanduser().resolve()
 if source.is_file() and source.suffix.lower() == ".zip":
 with zipfile.ZipFile(source) as zf:
 member = _select_member(zf, token, ".csv")
 with zf.open(member) as stream:
 return pd.read_csv(stream, low_memory=False)
 if source.is_dir():
 matches = sorted(source.rglob(f"*{token}*"))
 if not matches:
 raise FileNotFoundError(token)
 return pd.read_csv(matches[0], low_memory=False)
 raise FileNotFoundError(source)

```

---

```

def safe_read_csv_path(
 path: str | Path,
 *,
 columns: list[str] | tuple[str, ...] | None = None,
 required: bool = False,
) -> pd.DataFrame:
 """Read a CSV without failing on a valid zero-row/zero-byte checkpoint.

 Some Step12M diagnostics, especially the refined transition-edge table, may
 legitimately contain zero rows. Older code wrote such a DataFrame without
 column names, which creates an empty CSV and makes pandas raise
 ``EmptyDataError`` during the plotting-only reload. Returning an empty frame
 with a stable schema keeps the physical result intact and lets plotting run.
 """
 path = Path(path).expanduser().resolve()
 schema = list(columns) if columns is not None else None
 if not path.is_file():
 if required:
 raise FileNotFoundError(path)
 return pd.DataFrame(columns=schema)
 try:
 frame = pd.read_csv(path, low_memory=False)
 except pd.errors.EmptyDataError:
 return pd.DataFrame(columns=schema)
 if schema is not None:
 for column in schema:
 if column not in frame.columns:
 frame[column] = pd.Series(dtype="object")
 frame = frame.reindex(columns=schema + [c for c in frame.columns if c not in schema])
 return frame

def read_json_token(source: str | Path, token: str) -> dict[str, Any]:
 source = Path(source).expanduser().resolve()
 if source.is_file() and source.suffix.lower() == ".zip":
 with zipfile.ZipFile(source) as zf:
 member = _select_member(zf, token, ".json")
 with zf.open(member) as stream:
 return json.load(stream)
 if source.is_dir():
 matches = sorted(source.rglob(f"*{token}*"))
 if not matches:
 raise FileNotFoundError(token)
 return json.loads(matches[0].read_text(encoding="utf-8"))
 raise FileNotFoundError(source)

def atomic_json(payload: dict[str, Any], path: Path) -> None:
 tmp = path.with_suffix(path.suffix + ".tmp")
 tmp.write_text(
 json.dumps(payload, ensure_ascii=False, indent=2, default=_json_default),
 encoding="utf-8",
)
 tmp.replace(path)

def atomic_csv(frame: pd.DataFrame, path: Path) -> None:
 tmp = path.with_suffix(path.suffix + ".tmp")
 frame.to_csv(tmp, index=False, encoding="utf-8-sig")
 tmp.replace(path)

def _json_default(value: Any) -> Any:
 if isinstance(value, (np.bool_,)):
 return bool(value)
 if isinstance(value, (np.integer,)):
 return int(value)

```

---

```

 if isinstance(value, (np.floating,)):
 return float(value)
 if isinstance(value, np.ndarray):
 return value.tolist()
 if isinstance(value, Path):
 return str(value)
 raise TypeError(type(value).__name__)

def safe_int(value: Any) -> int | None:
 if value is None or pd.isna(value):
 return None
 return int(value)

=====
Input reconstruction
=====

def load_background(step11_source: str | Path) -> dict[str, float]:
 payload = read_json_token(step11_source, "step11M_00_fixed_background.json")
 if "fixed_background_parameters" in payload:
 payload = payload["fixed_background_parameters"]
 background = {name: float(payload[name]) for name in FIXED5}
 return background

def load_target_edge(step11_source: str | Path, edge_id: str) -> pd.Series:
 edges = read_csv_token(step11_source, "step11M_04_right_junction_boundary_edges.csv")
 selected = edges[edges["edge_id"].astype(str).eq(edge_id)]
 if selected.empty:
 # Fallback to selected edge table.
 selected_edges = read_csv_token(step11_source, "step11M_05_selected_junction_edges.csv")
 selected = selected_edges[selected_edges["edge_id"].astype(str).eq(edge_id)]
 if selected.empty:
 raise KeyError(f"Target edge {edge_id!r} not found in Step11M output")
 return selected.iloc[0].copy()

def load_right_upper_seed(step11_source: str | Path, edge: pd.Series) -> pd.Series:
 points = read_csv_token(step11_source, "step11M_07_completed_certified_boundary_points.csv")
 selected = points[
 points["branch_hint"].astype(str).eq("generic_right_upper")
 & points["point_certificate_pass"].astype(int).eq(1)
].copy()
 if selected.empty:
 raise RuntimeError("No certified generic_right_upper seed in Step11M output")
 target = np.array([
 0.5 * (float(edge["r3_0"]) + float(edge["r3_1"])),
 0.5 * (float(edge["r4_0"]) + float(edge["r4_1"])),
])
 distance = np.hypot(
 selected["r3"].to_numpy(float) - target[0],
 selected["r4"].to_numpy(float) - target[1],
)
 return selected.iloc[int(np.argmin(distance))].copy()

def validate_fixed_background(
 background: dict[str, float],
 edge: pd.Series,
 seed: pd.Series,
 tolerance: float = 1.0e-10,
) -> None:
 for name in FIXED5:
 expected = float(background[name])
 if name in seed and abs(float(seed[name]) - expected) > tolerance:
 raise ValueError(f"Seed background mismatch in {name}")

```

---

```

 if int(edge["chern_0"]) != -2 or int(edge["chern_1"]) != 0:
 raise ValueError(
 f'Expected target edge -2 -> 0, got {edge['chern_0']} -> {edge['chern_1']}'
)

=====
Frozen physical core
=====

def configure_physics(
 tts_archive: str | Path,
 config: Step12Config,
):
 core, step4, step5, source_dir = base.import_reference_modules(tts_archive)
 linear_v2.patch_step5_linear_path(step5, core)
 step3 = linear_v2.import_step3(source_dir)

 continuation = step10.ContinuationConfig(
 output_dir=config.output_dir / "_search_core",
 strict_gap_grids=tuple(config.strict_gap_grids),
 strict_chern_grids=tuple(config.strict_chern_grids),
 strict_chern_shifts=tuple(config.strict_chern_shifts),
 closure_gap_tol=float(config.closure_gap_tol),
 quick=bool(config.quick),
 random_seed=int(config.random_seed),
).normalized()
 strict_config = step10.strict_step3_config(
 step3,
 config.output_dir / "_strict_core",
 continuation,
)
 search_config = step10.step5_config(
 step5,
 config.output_dir / "_search_core",
 continuation,
)
 # Explicitly strengthen full-BZ search for the missed generic orbit.
 if config.quick:
 search_config.full_bz_de_popsiz = 10
 search_config.full_bz_de_maxiter = 160
 search_config.sphere_k_radii = (0.03, 0.06)
 else:
 search_config.full_bz_de_popsiz = 20
 search_config.full_bz_de_maxiter = 520
 search_config.local_starts_per_manifold = 14
 search_config.powell_maxiter = 1400
 search_config.sphere_k_radii = (0.02, 0.04, 0.08)
 search_config.spin_gap_accept_tol = float(config.closure_gap_tol)
 search_config.active_spin_gap_tol = max(2.0e-6, 20.0 * config.closure_gap_tol)
 search_config.random_seed = int(config.random_seed)
 step5._step10m_config = search_config
 return core, step4, step5, step3, strict_config, search_config

=====
Targeted full-BZ closure search
=====

def wrap_k(value: float) -> float:
 return float((float(value) + math.pi) % (2.0 * math.pi) - math.pi)

def d4_start_points(step4, kx: float, ky: float) -> list[tuple[float, float]]:
 orbit = step4.d4_orbit(wrap_k(kx), wrap_k(ky))
 starts: list[tuple[float, float]] = []
 for x, y in orbit:

```

---

```

 point = (wrap_k(x), wrap_k(y))
 if all(math.hypot(wrap_k(point[0] - a), wrap_k(point[1] - b)) > 1.0e-6 for a, b in starts):
 starts.append(point)
 return starts

def full_bz_gap_objective(step5, pair: pd.Series, x: np.ndarray) -> float:
 lam = float(np.clip(x[0], 0.0, 1.0))
 kx = wrap_k(float(x[1]))
 ky = wrap_k(float(x[2]))
 gap = step5.spin_middle_gap(kx, ky, pair, lam, "up")
 return float(gap * gap)

def _candidate_row(
 *,
 method: str,
 result,
 pair: pd.Series,
 step5,
 source_bracket: str,
 run_index: int,
) -> dict[str, Any]:
 lam, kx, ky = [float(v) for v in result.x]
 kx = wrap_k(kx)
 ky = wrap_k(ky)
 gap = float(step5.spin_middle_gap(kx, ky, pair, lam, "up"))
 return {
 "search_method": method,
 "source_bracket": source_bracket,
 "run_index": int(run_index),
 "critical_lambda": lam,
 "critical_kx": kx,
 "critical_ky": ky,
 "spin_up_gap": gap,
 "optimizer_success": int(bool(getattr(result, "success", True))),
 "optimizer_message": str(getattr(result, "message", "")),
 "optimizer_nfev": int(getattr(result, "nfev", -1)),
 }

def search_generic_four_valley(
 *,
 step4,
 step5,
 pair: pd.Series,
 seed: pd.Series,
 config: Step12Config,
) -> tuple[list[dict[str, Any]], list[dict[str, Any]]]:
 lo, hi = sorted(config.generic_lambda_bracket)
 seed_kx = float(seed["critical_kx_representative"])
 seed_ky = float(seed["critical_ky_representative"])
 orbit_starts = d4_start_points(step4, seed_kx, seed_ky)

 # The local optimizers are initialized at the Step10M continuation valley.
 # A few nearby lambda starts prevent a flat local basin from hiding the zero.
 lambda_starts = np.linspace(
 max(lo, config.generic_predicted_lambda - 0.10),
 min(hi, config.generic_predicted_lambda + 0.10),
 max(3, min(7, config.generic_powell_restarts // 2)),
)
 starts: list[np.ndarray] = []
 for lam in lambda_starts:
 for kx, ky in orbit_starts[:4]:
 starts.append(np.array([lam, kx, ky], dtype=float))
 starts = starts[: max(4, int(config.generic_powell_restarts))]

 candidates: list[dict[str, Any]] = []
 bounds = [(lo, hi), (-math.pi, math.pi), (-math.pi, math.pi)]

```

---

```

for index, start in enumerate(starts):
 result = minimize(
 lambda x: full_bz_gap_objective(step5, pair, x),
 start,
 method="Powell",
 bounds=bounds,
 options={
 "xtol": 1.0e-12,
 "ftol": 1.0e-22,
 "maxiter": (700 if config.quick else 1800),
 },
)
 candidates.append(
 _candidate_row(
 method="generic_seeded_Powell",
 result=result,
 pair=pair,
 step5=step5,
 source_bracket="generic",
 run_index=index,
)
)

Always perform several independent full-BZ DE searches. Step11M only
invoked full-BZ fallback when no diagonal zero existed, which is why the
earlier generic closing could be missed after Sigma' had already closed.
de_repeats = 2 if config.quick else int(config.generic_de_repeats)
for index in range(de_repeats):
 result = differential_evolution(
 lambda x: full_bz_gap_objective(step5, pair, x),
 bounds=bounds,
 seed=int(config.random_seed + 1009 * (index + 1)),
 popsize=(10 if config.quick else 22),
 maxiter=(180 if config.quick else 600),
 tol=1.0e-12,
 polish=True,
 workers=1,
 updating="immediate",
)
 candidates.append(
 _candidate_row(
 method="generic_full_BZ_DE",
 result=result,
 pair=pair,
 step5=step5,
 source_bracket="generic",
 run_index=index,
)
)

accepted = list(dict[str, Any]) = []
for row in sorted(candidates, key=lambda r: float(r["spin_up_gap"])):
 if float(row["spin_up_gap"]) > float(config.closure_gap_tol):
 continue
 closure = {**row, "search_manifold": row["search_method"]}
 valleys = step5.enumerate_spin_valleys(closure, pair, step5._step10m_config)
 active = base.unique_active_spin_up_valleys(valleys, step5)
 row["n_active_spin_up_valleys"] = int(len(active))
 row["active_k_regions"] = ";".join(sorted(set(active["critical_k_region"].astype(str))))
 if len(active) != 4:
 continue
 if not active["critical_k_region"].astype(str).eq("generic").all():
 continue
 accepted.append(closure)

accepted = step09d.global_deduplicate_closures(
 step5,
 accepted,
 float(config.closure_lambda_dedup_tol),

```

---

```

)
 return candidates, accepted

def search_sigma_prime_closure(
 *,
 step5,
 pair: pd.Series,
 config: Step12Config,
) -> tuple[list[dict[str, Any]], list[dict[str, Any]]:
 lo, hi = sorted(config.sigma_prime_lambda_bracket)
 bracket = pd.Series(
 {
 "transition_id": TARGET_EDGE_ID + "_sigma_prime",
 "path_id": TARGET_EDGE_ID,
 "lambda_left": lo,
 "lambda_right": hi,
 "chern_left": np.nan,
 "chern_right": np.nan,
 }
)
 candidates, closures = step5.search_transition_closures(
 bracket,
 pair,
 step5._step10m_config,
 12000,
)
 selected: list[dict[str, Any]] = []
 for closure in closures:
 valleys = step5.enumerate_spin_valleys(closure, pair, step5._step10m_config)
 active = base.unique_active_spin_up_valleys(valleys, step5)
 if len(active) != 2:
 continue
 regions = set(active["critical_k_region"].astype(str))
 if not any("SigmaPrime" in region for region in regions):
 continue
 selected.append(dict(closure))
 selected = step09d.global_deduplicate_closures(
 step5,
 selected,
 float(config.closure_lambda_dedup_tol),
)
 return candidates, selected

def combine_closures(
 step5,
 generic: list[dict[str, Any]],
 sigma_prime: list[dict[str, Any]],
 config: Step12Config,
) -> list[dict[str, Any]]:
 closures = [dict(row) for row in generic + sigma_prime]
 return step09d.global_deduplicate_closures(
 step5,
 closures,
 float(config.closure_lambda_dedup_tol),
)

=====
Gap tracks for diagnostics
=====

def minimize_k_in_box(
 *,
 step5,
 pair: pd.Series,
 lam: float,

```

---

```

 center: tuple[float, float],
 half_width: float,
) -> dict[str, float]:
 cx, cy = center
 lower_x = max(-math.pi, cx - half_width)
 upper_x = min(math.pi, cx + half_width)
 lower_y = max(-math.pi, cy - half_width)
 upper_y = min(math.pi, cy + half_width)

 def objective(x: np.ndarray) -> float:
 gap = step5.spin_middle_gap(float(x[0]), float(x[1]), pair, float(lam), "up")
 return float(gap * gap)

 result = minimize(
 objective,
 np.array([np.clip(cx, lower_x, upper_x), np.clip(cy, lower_y, upper_y)]),
 method="Powell",
 bounds=[(lower_x, upper_x), (lower_y, upper_y)],
 options={"xtol": 1.0e-10, "ftol": 1.0e-18, "maxiter": 600},
)
 kx, ky = [float(v) for v in result.x]
 return {
 "generic_gap": float(step5.spin_middle_gap(kx, ky, pair, float(lam), "up")),
 "generic_kx": kx,
 "generic_ky": ky,
 }

def sigma_prime_gap_at_lambda(
 *,
 step5,
 pair: pd.Series,
 lam: float,
 kappa_points: int,
) -> dict[str, float]:
 kappas = np.linspace(-math.pi, math.pi, int(kappa_points), endpoint=False)
 gaps = np.array(
 [
 step5.spin_middle_gap(kappa, -kappa, pair, float(lam), "up")
 for kappa in kappas
],
 dtype=float,
)
 index = int(np.argmin(gaps))
 left = float(kappas[max(0, index - 1)])
 right = float(kappas[min(len(kappas) - 1, index + 1)])
 if right > left:
 result = minimize_scalar(
 lambda kap: step5.spin_middle_gap(float(kap), -float(kap), pair, float(lam), "up") ** 2,
 bounds=(left, right),
 method="bounded",
 options={"xatol": 1.0e-12, "maxiter": 600},
)
 kappa = float(result.x)
 else:
 kappa = float(kappas[index])
 return {
 "sigma_prime_gap": float(
 step5.spin_middle_gap(kappa, -kappa, pair, float(lam), "up")
),
 "sigma_prime_kappa": kappa,
 }

def compute_gap_tracks(
 *,
 step5,
 pair: pd.Series,
 seed: pd.Series,

```



---

```

 config: Step12Config,
) -> pd.DataFrame:
 lambdas = np.linspace(0.0, 1.0, int(config.gap_track_lambda_points))
 generic_center = (
 float(seed["critical_kx_representative"]),
 float(seed["critical_ky_representative"]),
)
 rows: list[dict[str, Any]] = []
 for index, lam in enumerate(lambdas):
 row: dict[str, Any] = {"lambda": float(lam), "lambda_index": int(index)}
 if config.generic_lambda_bracket[0] - 0.05 <= lam <= config.generic_lambda_bracket[1] + 0.05:
 result = minimize_k_in_box(
 step5=step5,
 pair=pair,
 lam=float(lam),
 center=generic_center,
 half_width=float(config.generic_k_box_half_width),
)
 row.update(result)
 generic_center = (result["generic_kx"], result["generic_ky"])
 else:
 row.update({"generic_gap": np.nan, "generic_kx": np.nan, "generic_ky": np.nan})
 row.update(
 sigma_prime_gap_at_lambda(
 step5=step5,
 pair=pair,
 lam=float(lam),
 kappa_points=(121 if config.quick else int(config.sigma_prime_kappa_points)),
)
)
 finite = [
 value for value in [row["generic_gap"], row["sigma_prime_gap"]]
 if np.isfinite(value)
]
 row["tracked_min_gap"] = min(finite) if finite else np.nan
 rows.append(row)
 return pd.DataFrame(rows)

```

```

=====
Strict segment labels and closure certificates
=====

```

```

def strict_segment_probes(
 *,
 step3,
 strict_config,
 pair: pd.Series,
 closures: list[dict[str, Any]],
 path_id: str,
) -> tuple[list[dict[str, Any]], list[dict[str, Any]]]:
 groups = step09d.closure_groups(closures)
 centres = [
 float(np.mean([float(row["critical_lambda"]) for row in group]))
 for group in groups
]
 boundaries = [0.0, *centres, 1.0]
 results: list[dict[str, Any]] = []
 rows: list[dict[str, Any]] = []
 for index, (left, right) in enumerate(zip(boundaries[:-1], boundaries[1:])):
 probe = step09d.find_reliable_segment_probe(
 step3,
 strict_config,
 pair,
 path_id,
 float(left),
 float(right),
 (0.5, 0.35, 0.65, 0.2, 0.8),

```

---

```

 index,
)
 results.append(probe)
 summary = probe["summary"]
 rows.append(
 {
 "path_id": path_id,
 "segment_index": index,
 "lambda_left_bound": float(left),
 "lambda_right_bound": float(right),
 "probe_lambda": summary.get("lambda"),
 "strict_chern_up": step09d.reliable_chern(summary),
 "strict_phase_label": summary.get("phase_label"),
 "min_direct_gap": summary.get("min_direct_gap"),
 "indirect_gap": summary.get("indirect_gap"),
 "strict_gap_verified": summary.get("strict_gap_verified"),
 "strict_chern_verified": summary.get("strict_chern_verified"),
 }
)
)
return results, rows

def certify_target_edge(
 *,
 edge: pd.Series,
 pair: pd.Series,
 closures: list[dict[str, Any]],
 step5,
 search_config,
 segment_results: list[dict[str, Any]],
) -> dict[str, Any]:
 groups = step09d.closure_groups(closures)
 closure_rows: list[dict[str, Any]] = []
 valley_rows: list[dict[str, Any]] = []
 kp_rows: list[dict[str, Any]] = []
 derivative_rows: list[dict[str, Any]] = []
 gradient_rows: list[dict[str, Any]] = []
 berry_attempt_rows: list[dict[str, Any]] = []
 berry_rows: list[dict[str, Any]] = []
 group_rows: list[dict[str, Any]] = []

 for group_index, group in enumerate(groups):
 group_id = f"{TARGET_EDGE_ID}_step12_group{group_index:02d}"
 centre = float(np.mean([float(row["critical_lambda"]) for row in group]))
 group_charge = 0
 all_consensus = True
 all_rank3 = True
 valley_count = 0
 region_set: set[str] = set()
 for closure_index, closure in enumerate(group):
 closure_id = f"{group_id}_closure{closure_index:02d}"
 reduced = linear_v2.linear_reduced_on_path(pair, float(closure["critical_lambda"]))
 closure_rows.append(
 {
 "edge_id": TARGET_EDGE_ID,
 "closure_group_id": group_id,
 "closure_id": closure_id,
 **closure,
 **reduced,
 }
)
 analysis = step09d.analyse_closure(
 step5,
 pair,
 search_config,
 TARGET_EDGE_ID,
 group_id,
 closure_id,
 closure,

```

---

```

)
 valleys = analysis["valleys"].copy()
 if len(valleys):
 valley_rows.extend(valleys.to_dict("records"))
 region_set.update(valleys["critical_k_region"].astype(str))
 kp_rows.extend(analysis["kp_rows"])
 derivative_rows.extend(analysis["derivative_rows"])
 gradient_rows.extend(analysis["gradient_rows"])
 berry_attempt_rows.extend(analysis["berry_attempt_rows"])
 berry_rows.extend(analysis["berry_summary_rows"])
 group_charge += int(analysis["charge_sum"])
 all_consensus = all_consensus and bool(analysis["all_consensus"])
 all_rank3 = all_rank3 and bool(analysis["all_rank3"])
 valley_count += int(analysis["n_active_valleys"])

 left_chern = step09d.reliable_chern(segment_results[group_index]["summary"])
 right_chern = step09d.reliable_chern(segment_results[group_index + 1]["summary"])
 delta = None if left_chern is None or right_chern is None else int(right_chern - left_chern)
 signed_match = (
 delta is not None
 and all_consensus
 and int(group_charge) == int(delta)
)
 if valley_count == 4 and all("generic" in region for region in region_set):
 mechanism = "generic_four_valley"
 elif valley_count == 2 and any("SigmaPrime" in region for region in region_set):
 mechanism = "SigmaPrime_two_valley"
 else:
 mechanism = "other_or_mixed"
 group_rows.append(
 {
 "edge_id": TARGET_EDGE_ID,
 "closure_group_id": group_id,
 "mean_critical_lambda": centre,
 "mechanism": mechanism,
 "n_closure_orbits": len(group),
 "n_active_spin_up_valleys": valley_count,
 "left_chern_up": left_chern,
 "right_chern_up": right_chern,
 "observed_delta_chern_up": delta,
 "berry_charge_sum_up": int(group_charge),
 "all_berry_charges_consensus": int(all_consensus),
 "all_kp_jacobians_rank3": int(all_rank3),
 "signed_charge_matches": int(signed_match),
 "closure_group_certificate_pass": int(signed_match and all_rank3),
 }
)
)

endpoint_delta = int(edge["chern_1"] - edge["chern_0"])
total_charge = int(sum(row["berry_charge_sum_up"] for row in group_rows))
all_groups_pass = bool(
 len(group_rows)
 and all(int(row["closure_group_certificate_pass"]) == 1 for row in group_rows)
)
sequence = [step09d.reliable_chern(item["summary"]) for item in segment_results]
expected_sequence = sequence == [-2, 2, 0]
edge_certificate = {
 "code_version": CODE_VERSION,
 "edge_id": TARGET_EDGE_ID,
 "grid_chern_0": int(edge["chern_0"]),
 "grid_chern_1": int(edge["chern_1"]),
 "grid_endpoint_delta_chern_up": endpoint_delta,
 "strict_segment_chern_sequence": sequence,
 "target_sequence_minus2_to_plus2_to_zero": int(expected_sequence),
 "n_closure_groups": len(groups),
 "n_closure_orbits": len(closures),
 "n_active_spin_up_valleys_total": int(
 sum(row["n_active_spin_up_valleys"] for row in group_rows)
),
}

```

---

```

 "berry_charge_sum_over_all_groups": total_charge,
 "signed_total_charge_matches_grid_delta": int(total_charge == endpoint_delta),
 "all_closure_groups_certified": int(all_groups_pass),
 "edge_multiclosure_certificate_pass": int(
 expected_sequence and all_groups_pass and total_charge == endpoint_delta
),
 "interpretation": (
 "Pass requires two separately certified closure groups with strict "
 "segment sequence -2 -> +2 -> 0, signed Berry charges +4 and -2, "
 "and total charge +2 equal to the endpoint Chern change."
),
 }
}
return {
 "closure_rows": closure_rows,
 "valley_rows": valley_rows,
 "kp_rows": kp_rows,
 "derivative_rows": derivative_rows,
 "gradient_rows": gradient_rows,
 "berry_attempt_rows": berry_attempt_rows,
 "berry_rows": berry_rows,
 "group_rows": group_rows,
 "edge_certificate": edge_certificate,
}

=====
Refined junction strict map
=====

def refined_checkpoint(config: Step12Config, ix: int, iy: int) -> Path:
 return config.output_dir / "refined_junction_points" / f"refined_{ix:02d}_{iy:02d}.json"

def phase_code_from_summary(summary: dict[str, Any]) -> int:
 label = str(summary.get("phase_label", ""))
 cup = summary.get("chern_up_int")
 strict_gap = int(summary.get("strict_gap_verified", 0) or 0)
 strict_chern = int(summary.get("strict_chern_verified", 0) or 0)
 if strict_gap == 1 and strict_chern == 1 and cup is not None and not pd.isna(cup):
 value = int(cup)
 if label in {"spin_chern_TI_candidate", "trivial_insulator"} and value in PHASE_ORDER:
 return value
 return 99

def run_refined_junction_grid(
 *,
 background: dict[str, float],
 step3,
 strict_config,
 config: Step12Config,
) -> pd.DataFrame:
 r3_values = np.linspace(*config.refined_r3_range, int(config.refined_grid_n))
 r4_values = np.linspace(*config.refined_r4_range, int(config.refined_grid_n))
 rows: list[dict[str, Any]] = []
 for iy, r4 in enumerate(r4_values):
 for ix, r3 in enumerate(r3_values):
 path = refined_checkpoint(config, ix, iy)
 if path.is_file() and not config.force_recalculate:
 payload = json.loads(path.read_text(encoding="utf-8"))
 else:
 reduced = {**background, "r3": float(r3), "r4": float(r4)}
 result = step10.evaluate_strict(
 step3,
 strict_config,
 reduced,
 f"step12M_refined_{ix:02d}_{iy:02d}",
)
 rows.append(result)
 return pd.DataFrame(rows)

```

---

```

 payload = result
 atomic_json(payload, path)
 summary = dict(payload["summary"])
 summary.update(
 {
 "refined_ix": int(ix),
 "refined_iy": int(iy),
 "paper_phase_code": phase_code_from_summary(summary),
 }
)
 rows.append(summary)
 print(
 f"refined junction {iy * len(r3_values) + ix + 1:3d}/"
 f"{len(r3_values) * len(r4_values)} | r3={r3:.6f} r4={r4:.6f} "
 f"C={summary['paper_phase_code']}"
)
 frame = pd.DataFrame(rows)
 atomic_csv(frame, config.output_dir / "step12M_10_refined_junction_strict_grid.csv")
 return frame

def combine_junction_grids(
 step11_grid: pd.DataFrame,
 refined: pd.DataFrame,
) -> pd.DataFrame:
 old = step11_grid.copy()
 old["grid_source"] = "step11M_7x7"
 old["grid_resolution_priority"] = 0
 new = refined.copy()
 new["grid_source"] = "step12M_13x13"
 new["grid_resolution_priority"] = 1
 combined = pd.concat([old, new], ignore_index=True, sort=False)
 combined["r3_round"] = combined["r3"].astype(float).round(10)
 combined["r4_round"] = combined["r4"].astype(float).round(10)
 combined = (
 combined.sort_values("grid_resolution_priority")
 .drop_duplicates(["r3_round", "r4_round"], keep="last")
 .drop(columns=["r3_round", "r4_round"])
 .sort_values(["r4", "r3"])
 .reset_index(drop=True)
)
 return combined

REFINED_TRANSITION_EDGE_COLUMNS = [
 "ix0", "iy0", "ix1", "iy1",
 "r3_0", "r4_0", "chern_0",
 "r3_1", "r4_1", "chern_1",
 "mid_r3", "mid_r4", "delta_chern_up", "orientation",
]

```

```

def refined_transition_edges(refined: pd.DataFrame) -> pd.DataFrame:
 by_key = {
 (int(row.refined_ix), int(row.refined_iy)): row
 for row in refined.itertuples()
 }
 rows: list[dict[str, Any]] = []
 seen: set[tuple[tuple[int, int], tuple[int, int]]] = set()
 for key, row in by_key.items():
 if int(row.paper_phase_code) not in {-2, 0, 2}:
 continue
 ix, iy = key
 for other_key in [(ix + 1, iy), (ix, iy + 1)]:
 if other_key not in by_key:
 continue
 other = by_key[other_key]
 if int(other.paper_phase_code) not in {-2, 0, 2}:
 continue

```

---

```

 if int(row.paper_phase_code) == int(other.paper_phase_code):
 continue
 edge_key = tuple(sorted((key, other_key)))
 if edge_key in seen:
 continue
 seen.add(edge_key)
 rows.append(
 {
 "ix0": ix,
 "iy0": iy,
 "ix1": other_key[0],
 "iy1": other_key[1],
 "r3_0": float(row.r3),
 "r4_0": float(row.r4),
 "chern_0": int(row.paper_phase_code),
 "r3_1": float(other.r3),
 "r4_1": float(other.r4),
 "chern_1": int(other.paper_phase_code),
 "mid_r3": 0.5 * (float(row.r3) + float(other.r3)),
 "mid_r4": 0.5 * (float(row.r4) + float(other.r4)),
 "delta_chern_up": int(other.paper_phase_code - row.paper_phase_code),
 "orientation": "vertical_boundary" if ix != other_key[0] else "horizontal_boundary",
 }
)
 return pd.DataFrame(rows, columns=REFINED_TRANSITION_EDGE_COLUMNS)

=====
Publication plotting
=====

def publication_rc() -> None:
 plt.rcParams.update(
 {
 "font.family": "DejaVu Sans",
 "font.size": 8.5,
 "axes.linewidth": 0.8,
 "xtick.direction": "in",
 "ytick.direction": "in",
 "xtick.top": True,
 "ytick.right": True,
 "savefig.facecolor": "white",
 "figure.facecolor": "white",
 "axes.facecolor": "white",
 }
)

def phase_cmap() -> tuple[ListedColormap, BoundaryNorm]:
 cmap = ListedColormap([PHASE_COLORS[value] for value in PHASE_ORDER])
 norm = BoundaryNorm([-2.5, -1.5, -0.5, 0.5, 1.5, 2.5], cmap.N)
 return cmap, norm

def formal_phase_grid(formal_scan_source: str | Path) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
 frame = base.load_lower_grid(formal_scan_source)
 frame["paper_phase_code"] = frame.apply(base.phase_code, axis=1)
 x = np.sort(frame["scan_x"].unique().astype(float))
 y = np.sort(frame["scan_y"].unique().astype(float))
 matrix = np.full((len(y), len(x)), np.nan)
 x_index = {round(value, 12): i for i, value in enumerate(x)}
 y_index = {round(value, 12): i for i, value in enumerate(y)}
 for row in frame.itertuples():
 value = int(row.paper_phase_code)
 if value not in PHASE_ORDER:
 continue
 matrix[y_index[round(float(row.scan_y), 12)], x_index[round(float(row.scan_x), 12)]] = value
 return x, y, matrix

```

---

```

def plot_main_phase_map(
 *,
 formal_scan_source: str | Path,
 completed_points: pd.DataFrame,
 fits: pd.DataFrame,
 combined_grid: pd.DataFrame,
 config: Step12Config,
) -> None:
 publication_rc()
 cmap, norm = phase_cmap()
 x, y, matrix = formal_phase_grid(formal_scan_source)
 fig, ax = plt.subplots(figsize=(16.0 / 2.54, 10.5 / 2.54))
 ax.imshow(
 matrix,
 origin="lower",
 extent=[x.min(), x.max(), y.min(), y.max()],
 aspect="auto",
 cmap=cmap,
 norm=norm,
 interpolation="nearest",
 alpha=0.92,
)

 markers = {"generic_left_lower": "s", "generic_right_upper": "^"}
 for branch, group in completed_points.groupby("branch_hint"):
 ax.scatter(
 group["r3"],
 group["r4"],
 s=27,
 marker=markers.get(str(branch), "o"),
 facecolors="white",
 edgecolors="black",
 linewidths=0.8,
 zorder=6,
)

 for row in fits.itertuples():
 branch = str(row.branch_hint)
 degree = int(row.selected_degree)
 coefficients = [float(getattr(row, f"coefficient_{i}")) for i in range(degree + 1)]
 if branch == "generic_left_lower":
 xx = np.linspace(float(row.x_min), float(row.x_max), 300)
 yy = np.polyval(coefficients, xx)
 ax.plot(xx, yy, color="black", linewidth=1.1, zorder=5)
 else:
 yy = np.linspace(float(row.x_min), float(row.x_max), 300)
 xx = np.polyval(coefficients, yy)
 ax.plot(xx, yy, color="black", linewidth=1.1, zorder=5)

 reliable = combined_grid[combined_grid["paper_phase_code"].isin([-2, 0, 2])]
 unreliable = combined_grid[~combined_grid["paper_phase_code"].isin([-2, 0, 2])]
 for phase in [-2, 0, 2]:
 group = reliable[reliable["paper_phase_code"].astype(int).eq(phase)]
 if len(group):
 ax.scatter(
 group["r3"], group["r4"],
 s=18, marker="s",
 c=PHASE_COLORS[phase], edgecolors="black", linewidths=0.25,
 zorder=7,
)
 if len(unreliable):
 ax.scatter(
 unreliable["r3"], unreliable["r4"],
 s=17, marker="x", color="#555555", linewidths=0.65, zorder=8,
)

 plot_xmin = min(float(x.min()), float(combined_grid["r3"].min()))

```

---

```

plot_xmax = max(float(x.max()), float(combined_grid["r3"].max()))
plot_ymin = min(float(y.min()), float(combined_grid["r4"].min()))
plot_ymax = max(float(y.max()), float(combined_grid["r4"].max())) + 0.004

r3_lo, r3_hi = config.refined_r3_range
r4_lo, r4_hi = config.refined_r4_range
ax.add_patch(
 Rectangle(
 (r3_lo, r4_lo), r3_hi - r3_lo, r4_hi - r4_lo,
 fill=False, edgecolor="black", linewidth=0.9, linestyle="--", zorder=9,
)
)
ax.annotate(
 "right junction\n(refined in Step12M)",
 xy=(0.5 * (r3_lo + r3_hi), 0.5 * (r4_lo + r4_hi)),
 xytext=(r3_lo - 0.020, r4_hi + 0.006),
 arrowprops={"arrowstyle": "-", "linewidth": 0.7},
 fontsize=7.3, ha="right", va="top",
)

handles: list[Any] = [
 Patch(facecolor=PHASE_COLORS[value], edgecolor="black", linewidth=0.3, label=rf"$C_{\uparrow}={value}$")
 for value in PHASE_ORDER
]
handles.extend(
 [
 Line2D([0], [0], marker="s", color="none", markerfacecolor="white", markeredgcolor="black",
 ↪ label="certified left branch"),
 Line2D([0], [0], marker="^", color="none", markerfacecolor="white", markeredgcolor="black",
 ↪ label="certified right branch"),
 Line2D([0], [0], marker="x", color="#555555", linestyle="none", label="boundary / unreliable"),
]
)
ax.legend(
 handles=handles, frameon=True, facecolor="white", edgecolor="none",
 framealpha=0.86, ncol=2, fontsize=6.8, loc="upper left",
 borderpad=0.35, handletextpad=0.45, columnspacing=0.8,
)
ax.set_xlabel(r"r_3")
ax.set_ylabel(r"r_4")
ax.set_title("TTS fixed-background topological phase map")
ax.set_xlim(plot_xmin, plot_xmax)
ax.set_ylim(plot_ymin, plot_ymax)
fig.tight_layout()
out = config.output_dir / "figures" / "step12M_final_fixed_slice_phase_map"
fig.savefig(out.with_suffix(".png"), dpi=900, bbox_inches="tight")
fig.savefig(out.with_suffix(".pdf"), bbox_inches="tight")
fig.savefig(out.with_suffix(".svg"), bbox_inches="tight")
plt.close(fig)

def _grid_spacing(frame: pd.DataFrame) -> tuple[float, float]:
 x = np.sort(frame["r3"].unique().astype(float))
 y = np.sort(frame["r4"].unique().astype(float))
 dx = float(np.median(np.diff(x))) if len(x) > 1 else 0.001
 dy = float(np.median(np.diff(y))) if len(y) > 1 else 0.001
 return dx, dy

def plot_junction_zoom(
 *,
 refined: pd.DataFrame,
 transition_edges: pd.DataFrame,
 closure_rows: pd.DataFrame,
 edge: pd.Series,
 right_fit: pd.Series | None,
 config: Step12Config,
) -> None:

```



---

```

publication_rc()
fig, ax = plt.subplots(figsize=(12.5 / 2.54, 10.0 / 2.54))
reliable = refined[refined["paper_phase_code"].isin([-2, 0, 2])]
unreliable = refined[~refined["paper_phase_code"].isin([-2, 0, 2])]
dx, dy = _grid_spacing(refined)
marker_size = 135
for phase in [-2, 0, 2]:
 group = reliable[reliable["paper_phase_code"].astype(int).eq(phase)]
 ax.scatter(
 group["r3"], group["r4"],
 s=marker_size, marker="s", c=PHASE_COLORS[phase],
 edgecolors="white", linewidths=0.45, zorder=2,
 label=r"$C_{\uparrow}={phase}$",
)
if len(unreliable):
 ax.scatter(
 unreliable["r3"], unreliable["r4"],
 s=60, marker="x", color="#4F4F4F", linewidths=0.9,
 zorder=5, label="boundary / unreliable",
)

Pixel-edge boundary segments inferred only from reliable neighbouring points.
for row in transition_edges.itertuples():
 if str(row.orientation) == "vertical_boundary":
 ax.plot(
 [row.mid_r3, row.mid_r3],
 [row.mid_r4 - 0.48 * dy, row.mid_r4 + 0.48 * dy],
 color="black", linewidth=1.0, zorder=4,
)
 else:
 ax.plot(
 [row.mid_r3 - 0.48 * dx, row.mid_r3 + 0.48 * dx],
 [row.mid_r4, row.mid_r4],
 color="black", linewidth=1.0, zorder=4,
)

Target path.
ax.plot(
 [float(edge["r3_0"]), float(edge["r3_1"])],
 [float(edge["r4_0"]), float(edge["r4_1"])],
 color="black", linewidth=1.1, linestyle="--", zorder=6,
 label="target edge path",
)
ax.scatter(
 [float(edge["r3_0"]), float(edge["r3_1"])],
 [float(edge["r4_0"]), float(edge["r4_1"])],
 s=36, facecolors="white", edgecolors="black", zorder=7,
)

if len(closure_rows):
 for index, row in closure_rows.sort_values("critical_lambda").reset_index(drop=True).iterrows():
 lam = float(row["critical_lambda"])
 r3 = (1.0 - lam) * float(edge["r3_0"]) + lam * float(edge["r3_1"])
 r4 = (1.0 - lam) * float(edge["r4_0"]) + lam * float(edge["r4_1"])
 if "full_BZ" in str(row.get("search_manifold", "")) or "generic" in str(row.get("search_manifold",
 ↪ "")):
 marker, text = "*", "four-valley"
 size = 120
 else:
 marker, text = "D", r"two-valley Σ'"
 size = 48
 ax.scatter([r3], [r4], marker=marker, s=size, c="black", zorder=9)
 ax.annotate(
 text,
 xy=(r3, r4), xytext=(5, 6 if index == 0 else -13),
 textcoords="offset points", fontsize=7.2,
 ha="left", va="bottom" if index == 0 else "top",
)

```

---

```

Certified right-upper fit is solid only in its original validated domain;
the extension toward the newly certified generic closure is dashed.
if right_fit is not None:
 degree = int(right_fit["selected_degree"])
 coeff = [float(right_fit[f"coefficient_{i}"]) for i in range(degree + 1)]
 y_valid = np.linspace(float(right_fit["x_min"]), float(right_fit["x_max"]), 120)
 x_valid = np.polyval(coeff, y_valid)
 ax.plot(x_valid, y_valid, color="black", linewidth=1.0, zorder=6)
 y_ext = np.linspace(float(right_fit["x_max"]), config.refined_r4_range[1], 100)
 x_ext = np.polyval(coeff, y_ext)
 ax.plot(x_ext, y_ext, color="black", linewidth=0.8, linestyle=":", zorder=6)

ax.set_xlim(config.refined_r3_range[0] - 0.0004, config.refined_r3_range[1] + 0.0004)
ax.set_ylim(config.refined_r4_range[0] - 0.0003, config.refined_r4_range[1] + 0.0003)
ax.set_xlabel(r"r_3")
ax.set_ylabel(r"r_4")
ax.set_title("Right-upper junction: strict Chern map")
ax.legend(frameon=False, fontsize=7.0, loc="upper left")
fig.tight_layout()
out = config.output_dir / "figures" / "step12M_right_junction_zoom"
fig.savefig(out.with_suffix(".png"), dpi=900, bbox_inches="tight")
fig.savefig(out.with_suffix(".pdf"), bbox_inches="tight")
fig.savefig(out.with_suffix(".svg"), bbox_inches="tight")
plt.close(fig)

def plot_target_path_atlas(
 *,
 gap_tracks: pd.DataFrame,
 closure_rows: pd.DataFrame,
 segment_rows: pd.DataFrame,
 config: Step12Config,
) -> None:
 publication_rc()
 fig, ax = plt.subplots(figsize=(14.0 / 2.54, 8.5 / 2.54))
 if "generic_gap" in gap_tracks:
 ax.semilogy(
 gap_tracks["lambda"], np.maximum(gap_tracks["generic_gap"], 1.0e-16),
 linewidth=1.1, label="generic full-BZ valley track",
)
 ax.semilogy(
 gap_tracks["lambda"], np.maximum(gap_tracks["sigma_prime_gap"], 1.0e-16),
 linewidth=1.0, label=r"Σ valley track",
)
 for index, row in closure_rows.sort_values("critical_lambda").reset_index(drop=True).iterrows():
 lam = float(row["critical_lambda"])
 ax.axvline(lam, color="black", linewidth=0.8, linestyle="--")
 ax.text(
 lam, 2.0e-15 if index == 0 else 7.0e-15,
 "4 valleys" if index == 0 else "2 valleys",
 rotation=90, fontsize=7.0, ha="right", va="bottom",
)
 ax.set_xlabel(r"path coordinate λ")
 ax.set_ylabel(r"spin-up internal gap")
 ax.set_ylim(5.0e-16, max(0.05, float(np.nanmax(gap_tracks[["generic_gap", "sigma_prime_gap"]].to_numpy())) *
 ↪ 1.3))
 ax.legend(frameon=False, fontsize=7.2, loc="upper right")

 ax2 = ax.twinx()
 for row in segment_rows.itertuples():
 if pd.isna(row.strict_chern_up):
 continue
 ax2.hlines(
 int(row.strict_chern_up),
 float(row.lambda_left_bound),
 float(row.lambda_right_bound),
 color="#333333", linewidth=2.0, alpha=0.75,
)
 ax2.scatter([float(row.probe_lambda)], [int(row.strict_chern_up)], marker="s", s=18, color="#333333")

```

---

```

ax2.set_ylabel(r"strict $\mathbb{C}$ \uparrow")
ax2.set_yticks([-2, 0, 2])
ax2.set_ylim(-2.7, 2.7)
fig.tight_layout()
out = config.output_dir / "figures" / "step12M_target_edge_multiclosure_atlas"
fig.savefig(out.with_suffix(".png"), dpi=900, bbox_inches="tight")
fig.savefig(out.with_suffix(".pdf"), bbox_inches="tight")
fig.savefig(out.with_suffix(".svg"), bbox_inches="tight")
plt.close(fig)

def plot_mass_sign_mapping(
 *,
 step11_source: str | Path,
 config: Step12Config,
) -> None:
 predictions = read_csv_token(step11_source, "step11M_11_spatial_holdout_predictions.csv")
 definitions = read_csv_token(step11_source, "step11M_10_local_signed_mass_definitions.csv")
 publication_rc()
 fig, ax = plt.subplots(figsize=(12.5 / 2.54, 7.5 / 2.54))
 markers = {"generic_left_lower": "s", "generic_right_upper": "^"}
 for branch, group in predictions.groupby("branch_hint"):
 for phase in [-2, 2]:
 subset = group[group["paper_phase_code"].astype(int).eq(phase)]
 if len(subset):
 ax.scatter(
 subset["signed_mass"],
 np.full(len(subset), phase),
 s=26,
 marker=markers.get(str(branch), "o"),
 c=PHASE_COLORS[phase],
 edgecolors="black", linewidths=0.35,
 alpha=0.85,
 label=f"{branch.replace('generic_', '')}: C={phase}",
)
 ax.axvline(0.0, color="black", linestyle="--", linewidth=0.8)
 ax.set_xlabel("local signed mass coordinate")
 ax.set_ylabel(r"strict $\mathbb{C}$ \uparrow")
 ax.set_yticks([-2, 2])
 ax.set_ylim(-2.8, 2.8)
 ax.set_title("Local mass sign and topological sector")
 # Explicit sign mapping inferred from the validated Step11M predictions.
 text_lines = []
 for branch, group in predictions.groupby("branch_hint"):
 mapping = (
 group.groupby("mass_sign")["paper_phase_code"]
 .agg(lambda values: int(pd.Series(values).mode().iloc[0]))
 .to_dict()
)
 if -1 in mapping and 1 in mapping:
 short = str(branch).replace("generic_", "").replace("_", "-")
 text_lines.append(
 f"{short}: $M < 0 \rightarrow C = \{mapping[-1]\}$, $M > 0 \rightarrow C = \{mapping[1]\}$ "
)
 if text_lines:
 ax.text(
 0.02, 0.98, "\n".join(text_lines), transform=ax.transAxes,
 ha="left", va="top", fontsize=7.1,
 bbox={"facecolor": "white", "edgecolor": "none", "alpha": 0.85},
)
 legend_handles = [
 Line2D([0], [0], marker="s", color="none", markerfacecolor="white", markeredgcolor="black",
 ↪ label="left-lower branch"),
 Line2D([0], [0], marker="^", color="none", markerfacecolor="white", markeredgcolor="black",
 ↪ label="right-upper branch"),
 Patch(facecolor=PHASE_COLORS[-2], edgecolor="black", linewidth=0.35, label=r"strict $\mathbb{C}$ \uparrow=-2"),
 Patch(facecolor=PHASE_COLORS[2], edgecolor="black", linewidth=0.35, label=r"strict $\mathbb{C}$ \uparrow=+2"),
]

```

---

```

ax.legend(
 handles=legend_handles, frameon=False, fontsize=6.8,
 loc="center left", bbox_to_anchor=(1.01, 0.5), borderaxespad=0.0,
)
fig.tight_layout()
out = config.output_dir / "figures" / "step12M_local_mass_sign_mapping"
fig.savefig(out.with_suffix(".png"), dpi=900, bbox_inches="tight")
fig.savefig(out.with_suffix(".pdf"), bbox_inches="tight")
fig.savefig(out.with_suffix(".svg"), bbox_inches="tight")
plt.close(fig)

=====
Aggregation and orchestration
=====

def save_analysis_tables(
 *,
 config: Step12Config,
 edge: pd.Series,
 generic_candidates: list[dict[str, Any]],
 sigma_candidates: list[dict[str, Any]],
 closures: list[dict[str, Any]],
 gap_tracks: pd.DataFrame,
 segment_rows: list[dict[str, Any]],
 analysis: dict[str, Any],
) -> None:
 atomic_csv(pd.DataFrame([edge.to_dict()]), config.output_dir / "step12M_01_target_edge_definition.csv")
 candidates = pd.DataFrame(
 [
 {"candidate_family": "generic", **row}
 for row in generic_candidates
]
 + [
 {"candidate_family": "SigmaPrime", **row}
 for row in sigma_candidates
]
)
 atomic_csv(candidates, config.output_dir / "step12M_02_full_bz_search_candidates.csv")
 atomic_csv(pd.DataFrame(closures), config.output_dir / "step12M_03_accepted_closure_orbits.csv")
 atomic_csv(gap_tracks, config.output_dir / "step12M_04_target_edge_gap_tracks.csv")
 atomic_csv(pd.DataFrame(segment_rows), config.output_dir / "step12M_05_strict_segment_bern_labels.csv")
 mapping = {
 "closure_rows": "step12M_06_critical_closure_parameters.csv",
 "valley_rows": "step12M_07_critical_valley_orbits.csv",
 "kp_rows": "step12M_08_local_kp_summaries.csv",
 "derivative_rows": "step12M_09_local_kp_derivatives.csv",
 "gradient_rows": "step12M_10_local_mass_gradients.csv",
 "berry_attempt_rows": "step12M_11_berry_sphere_attempts.csv",
 "berry_rows": "step12M_12_berry_charge_consensus.csv",
 "group_rows": "step12M_13_closure_group_certificates.csv",
 }
 for key, filename in mapping.items():
 atomic_csv(pd.DataFrame(analysis[key]), config.output_dir / filename)
 atomic_json(analysis["edge_certificate"], config.output_dir /
 ↪ "step12M_14_target_edge_multiclosure_certificate.json")

def build_final_certificate(
 *,
 config: Step12Config,
 background: dict[str, float],
 edge_certificate: dict[str, Any],
 refined: pd.DataFrame,
 transition_edges: pd.DataFrame,
) -> dict[str, Any]:
 reliable = refined[refined["paper_phase_code"].isin([-2, 0, 2])]
 counts = reliable["paper_phase_code"].astype(int).value_counts().sort_index().to_dict()

```

```

return {
 "code_version": CODE_VERSION,
 "research_design": "targeted_fixed_slice_right_junction_repair",
 "fixed_background_parameters": background,
 "free_plane_parameters": ["r3", "r4"],
 "target_edge_id": config.target_edge_id,
 "target_edge_certificate": edge_certificate,
 "refined_junction_grid": {
 "r3_range": list(config.refined_r3_range),
 "r4_range": list(config.refined_r4_range),
 "grid_n": int(config.refined_grid_n),
 "n_total_points": int(len(refined)),
 "n_reliable_points": int(len(reliable)),
 "phase_counts": {str(k): int(v) for k, v in counts.items()},
 "n_reliable_transition_edges": int(len(transition_edges)),
 },
 "all_background_parameters_remain_fixed": True,
 "step12M_research_complete": bool(
 int(edge_certificate.get("edge_multiclosure_certificate_pass", 0)) == 1
),
 "valid_claim_if_pass": (
 "Along the formerly unresolved r3=0.060 junction edge, the strict "
 "Chern sequence is -2 -> +2 -> 0. A generic four-valley closure "
 "carries +4 and a later Sigma' two-valley closure carries -2, so "
 "the total signed charge +2 equals the endpoint Chern change."
),
 "not_claimed": (
 "The result is local to the fixed-background r3-r4 plane and does "
 "not define a universal seven-dimensional mass formula."
),
}

```

```

def make_plots_from_outputs(
 *,
 formal_scan_source: str | Path,
 step11_source: str | Path,
 config: Step12Config,
) -> None:
 completed = read_csv_token(step11_source, "step11M_07_completed_certified_boundary_points.csv")
 fits = read_csv_token(step11_source, "step11M_08_fixed_slice_boundary_curve_fits.csv")
 step11_grid = read_csv_token(step11_source, "step11M_03_right_junction_strict_grid.csv")
 refined_path = config.output_dir / "step12M_15_refined_junction_strict_grid.csv"
 if not refined_path.is_file():
 refined_path = config.output_dir / "step12M_10_refined_junction_strict_grid.csv"
 refined = safe_read_csv_path(refined_path, required=True)
 combined_path = config.output_dir / "step12M_16_combined_junction_grid.csv"
 combined = safe_read_csv_path(combined_path) if combined_path.is_file() else
 ↪ combine_junction_grids(step11_grid, refined)
 transitions_path = config.output_dir / "step12M_17_refined_transition_edges.csv"
 if transitions_path.is_file():
 transitions = safe_read_csv_path(
 transitions_path, columns=REFINED_TRANSITION_EDGE_COLUMNS
)
 else:
 transitions = refined_transition_edges(refined)
 closure_path = config.output_dir / "step12M_06_critical_closure_parameters.csv"
 closure_rows = safe_read_csv_path(closure_path) if closure_path.is_file() else pd.DataFrame()
 edge = load_target_edge(step11_source, config.target_edge_id)
 right = fits[fits["branch_hint"].astype(str).eq("generic_right_upper")]
 right_fit = right.iloc[0] if len(right) else None

 plot_main_phase_map(
 formal_scan_source=formal_scan_source,
 completed_points=completed,
 fits=fits,
 combined_grid=combined,
 config=config,
)

```

---

```

plot_junction_zoom(
 refined=refined,
 transition_edges=transitions,
 closure_rows=closure_rows,
 edge=edge,
 right_fit=right_fit,
 config=config,
)
gap_path = config.output_dir / "step12M_04_target_edge_gap_tracks.csv"
segment_path = config.output_dir / "step12M_05_strict_segment_chern_labels.csv"
if gap_path.is_file() and segment_path.is_file() and len(closure_rows):
 plot_target_path_atlas(
 gap_tracks=safe_read_csv_path(gap_path, required=True),
 closure_rows=closure_rows,
 segment_rows=safe_read_csv_path(segment_path, required=True),
 config=config,
)
plot_mass_sign_mapping(step11_source=step11_source, config=config)

def run_step12m(
 *,
 tts_archive: str | Path,
 formal_scan_source: str | Path,
 step11_source: str | Path,
 output_dir: str | Path,
 config: Step12Config | None = None,
 run_physics: bool = True,
 run_refined_grid_flag: bool = True,
 run_plots: bool = True,
) -> dict[str, Any]:
 if config is None:
 config = Step12Config(output_dir=Path(output_dir))
 config.output_dir = Path(output_dir)
 config = config.normalized()
 atomic_json(asdict(config), config.output_dir / "step12M_00_run_configuration.json")

 background = load_background(step11_source)
 edge = load_target_edge(step11_source, config.target_edge_id)
 seed = load_right_upper_seed(step11_source, edge)
 validate_fixed_background(background, edge, seed)
 atomic_json(
 {
 "code_version": CODE_VERSION,
 "fixed_background_parameters": background,
 "target_edge": edge.to_dict(),
 "nearest_certified_generic_seed": seed.to_dict(),
 },
 config.output_dir / "step12M_00_input_audit.json",
)

 if not run_physics:
 if run_plots:
 make_plots_from_outputs(
 formal_scan_source=formal_scan_source,
 step11_source=step11_source,
 config=config,
)
 return {"status": "plot_only", "output_dir": str(config.output_dir)}

 core, step4, step5, step3, strict_config, search_config = configure_physics(
 tts_archive,
 config,
)
 pair = step11.build_pair_from_edge(edge, background)

 print("[1/5] Full-BZ generic four-valley search")
 generic_candidates, generic_closures = search_generic_four_valley(
 step4=step4,

```

---

```

 step5=step5,
 pair=pair,
 seed=seed,
 config=config,
)
 print(f" accepted generic closures = {len(generic_closures)}")

 print("[2/5] SigmaPrime two-valley search")
 sigma_candidates, sigma_closures = search_sigma_prime_closure(
 step5=step5,
 pair=pair,
 config=config,
)
 print(f" accepted SigmaPrime closures = {len(sigma_closures)}")

 closures = combine_closures(step5, generic_closures, sigma_closures, config)
 if len(closures) < 2:
 # Preserve all diagnostics before failing clearly.
 atomic_csv(pd.DataFrame(generic_candidates), config.output_dir /
 ↪ "step12M_02a_generic_search_candidates.csv")
 atomic_csv(pd.DataFrame(sigma_candidates), config.output_dir / "step12M_02b_sigma_search_candidates.csv")
 atomic_csv(pd.DataFrame(closures), config.output_dir / "step12M_03_accepted_closure_orbits.csv")
 raise RuntimeError(
 "Step12M did not find two distinct closure groups. Inspect step12M_02a/02b candidates; "
 "do not reinterpret the Step11M charge sign manually."
)

 print("[3/5] Gap tracks, strict segment Chern labels, k.p and Berry charges")
 gap_tracks = compute_gap_tracks(step5=step5, pair=pair, seed=seed, config=config)
 segment_results, segment_rows = strict_segment_probes(
 step3=step3,
 strict_config=strict_config,
 pair=pair,
 closures=closures,
 path_id=config.target_edge_id,
)
 analysis = certify_target_edge(
 edge=edge,
 pair=pair,
 closures=closures,
 step5=step5,
 search_config=search_config,
 segment_results=segment_results,
)
 save_analysis_tables(
 config=config,
 edge=edge,
 generic_candidates=generic_candidates,
 sigma_candidates=sigma_candidates,
 closures=closures,
 gap_tracks=gap_tracks,
 segment_rows=segment_rows,
 analysis=analysis,
)

 print("[4/5] Refined 13x13 strict junction map")
 if run_refined_grid_flag:
 refined = run_refined_junction_grid(
 background=background,
 step3=step3,
 strict_config=strict_config,
 config=config,
)
 else:
 refined = read_csv_token(step11_source, "step11M_03_right_junction_strict_grid.csv")
 # Normalize names so plotting works, but this is not considered a Step12 refined map.
 refined = refined.rename(columns={"junction_ix": "refined_ix", "junction_iy": "refined_iy"})
 atomic_csv(refined, config.output_dir / "step12M_15_refined_junction_strict_grid.csv")
 step11_grid = read_csv_token(step11_source, "step11M_03_right_junction_strict_grid.csv")

```

---

```

combined = combine_junction_grids(step11_grid, refined)
transitions = refined_transition_edges(refined)
atomic_csv(combined, config.output_dir / "step12M_16_combined_junction_grid.csv")
atomic_csv(transitions, config.output_dir / "step12M_17_refined_transition_edges.csv")

certificate = build_final_certificate(
 config=config,
 background=background,
 edge_certificate=analysis["edge_certificate"],
 refined=refined,
 transition_edges=transitions,
)
atomic_json(certificate, config.output_dir / "step12M_18_final_mechanism_certificate.json")

print("[5/5] Publication figures")
if run_plots:
 make_plots_from_outputs(
 formal_scan_source=formal_scan_source,
 step11_source=step11_source,
 config=config,
)

print(json.dumps(certificate, ensure_ascii=False, indent=2, default=_json_default))
return certificate

=====
CLI
=====

def build_parser() -> argparse.ArgumentParser:
 parser = argparse.ArgumentParser(
 description="Repair the unresolved TTS Step11M right-junction edge and redraw the fixed-slice phase map."
)
 parser.add_argument("--tts-archive", required=True, type=Path)
 parser.add_argument("--formal-scan", required=True, type=Path)
 parser.add_argument("--step11-source", required=True, type=Path)
 parser.add_argument("--output-dir", default=Path("outputs_tts_step12M_junction_multiclosure_repair"),
 ↪ type=Path)
 parser.add_argument("--quick", action="store_true", help="Reduced numerical settings for code testing only.")
 parser.add_argument("--force", action="store_true", help="Recalculate existing refined-grid checkpoints.")
 parser.add_argument("--skip-refined-grid", action="store_true", help="Run only the target edge certificate and
 ↪ plots.")
 parser.add_argument("--skip-plots", action="store_true")
 parser.add_argument("--plot-only", action="store_true", help="Regenerate figures from existing Step12M CSV
 ↪ files.")
 return parser

def main() -> int:
 args = build_parser().parse_args()
 config = Step12Config(
 output_dir=args.output_dir,
 quick=bool(args.quick),
 force_recalculate=bool(args.force),
)
 run_step12m(
 tts_archive=args.tts_archive,
 formal_scan_source=args.formal_scan,
 step11_source=args.step11_source,
 output_dir=args.output_dir,
 config=config,
 run_physics=not bool(args.plot_only),
 run_refined_grid_flag=not bool(args.skip_refined_grid),
 run_plots=not bool(args.skip_plots),
)
 return 0

```



---

```
if __name__ == "__main__":
 raise SystemExit(main())
```

## ► 代码清单 A.50.4 (python, 1200 行)

```
from __future__ import annotations

"""
TTS Step13M — adaptive Wilson-loop certification of the narrow Cup=+2 sector
=====

This workflow is deliberately local. It does not rescan the seven-dimensional
parameter space. It takes the two Step12M closures on the unresolved r3=0.060
edge and independently certifies the very narrow intermediate spin-up Chern
sector with non-Abelian Wilson loops of the occupied spin-up subspace.

Target sequence

 Cup = -2 -- generic four-valley charge +4 --> +2
 -- Sigma' two-valley charge -2 --> 0

The Wilson-loop sign convention is calibrated against the independently strict
Step12M pre-closure control Cup=-2. The post-closure Cup=0 control is then an
out-of-sample validation of that calibration.
"""

import argparse
import json
import math
import sys
import zipfile
from dataclasses import asdict, dataclass
from pathlib import Path
from typing import Any, Callable, Iterable

import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
import numpy as np
import pandas as pd
from scipy.optimize import linear_sum_assignment, minimize

import TTS_step12M_junction_multiclosure_repair as step12
import TTS_step11M_lieb_aligned_fixed_slice as step11

CODE_VERSION = "TTS_STEP13M_ADAPTIVE_WILSON_INTERMEDIATE_CHERN_V1_20260724"
TARGET_EDGE_ID = "edge_03_03_03_04"
TWO_PI = 2.0 * math.pi

PHASE_COLORS = {
 -2: "#3B6FB6",
 -1: "#7FB6D6",
 0: "#B8B8B8",
 1: "#F2B36D",
 2: "#C84A3A",
}

}

@dataclass
class Step13Config:
 output_dir: Path
```

---

```

Fractions inside the interval between the two Step12M closures.
middle_fractions: tuple[float, ...] = (0.25, 0.40, 0.50, 0.65, 0.80)
minimum_middle_certified_points: int = 3

Three independent Wilson-loop discretizations. Pairwise entries are used
together: (nkx[i], initial_nky[i], shift[i]).
wilson_nkx: tuple[int, ...] = (401, 601, 801)
wilson_initial_nky: tuple[int, ...] = (81, 101, 121)
wilson_shifts: tuple[tuple[float, float], ...] = (
 (0.00, 0.00),
 (0.50, 0.37),
 (0.25, 0.73),
)
adaptive_phase_step: float = 0.32 * math.pi
adaptive_max_points: int = 801
adaptive_max_rounds: int = 8
integer_residual_tolerance: float = 0.08
minimum_overlap_singular_value: float = 1.0e-4

Independent occupied-subspace gap audit at every probe.
gap_grid_n: int = 101
gap_local_starts: int = 16
minimum_positive_spin_gap: float = 1.0e-8

quick: bool = False
force_recalculate: bool = False
random_seed: int = 20260724

def normalized(self) -> "Step13Config":
 self.output_dir = Path(self.output_dir).expanduser().resolve()
 self.output_dir.mkdir(parents=True, exist_ok=True)
 for sub in ["figures", "wilson_checkpoints", "wilson_traces", "_physics_core"]:
 (self.output_dir / sub).mkdir(exist_ok=True)
 if len(self.wilson_nkx) != len(self.wilson_initial_nky):
 raise ValueError("wilson_nkx and wilson_initial_nky must have equal length")
 if len(self.wilson_shifts) != len(self.wilson_nkx):
 raise ValueError("wilson_shifts must match the number of Wilson attempts")
 if not self.middle_fractions:
 raise ValueError("middle_fractions cannot be empty")
 if any(not (0.0 < x < 1.0) for x in self.middle_fractions):
 raise ValueError("middle_fractions must lie strictly inside (0,1)")
 if self.minimum_middle_certified_points < 1:
 raise ValueError("minimum_middle_certified_points must be positive")
 if self.quick:
 self.wilson_nkx = (201, 301)
 self.wilson_initial_nky = (61, 81)
 self.wilson_shifts = ((0.0, 0.0), (0.5, 0.37))
 self.adaptive_max_points = min(self.adaptive_max_points, 401)
 self.gap_grid_n = min(self.gap_grid_n, 61)
 self.gap_local_starts = min(self.gap_local_starts, 8)
 return self

```

```

=====
Safe I/O
=====

```

```

def _select_member(zf: zipfile.ZipFile, token: str, suffix: str | None = None) -> str:
 matches = []
 for name in zf.namelist():
 if token not in Path(name).name and token not in name:
 continue
 if suffix and not name.lower().endswith(suffix.lower()):
 continue
 matches.append(name)
 if not matches:
 raise FileNotFoundError(f"No archive member containing {token!r}")

```

---

```

return sorted(matches, key=lambda n: (len(Path(n).parts), len(n)))[0]

def read_csv_token(source: str | Path, token: str) -> pd.DataFrame:
 source = Path(source).expanduser().resolve()
 if source.is_file() and source.suffix.lower() == ".zip":
 with zipfile.ZipFile(source) as zf:
 member = _select_member(zf, token, ".csv")
 with zf.open(member) as stream:
 try:
 return pd.read_csv(stream, low_memory=False)
 except pd.errors.EmptyDataError:
 return pd.DataFrame()
 if source.is_dir():
 matches = sorted(source.rglob(f"*{token}*"))
 if not matches:
 raise FileNotFoundError(token)
 try:
 return pd.read_csv(matches[0], low_memory=False)
 except pd.errors.EmptyDataError:
 return pd.DataFrame()
 raise FileNotFoundError(source)

def read_json_token(source: str | Path, token: str) -> dict[str, Any]:
 source = Path(source).expanduser().resolve()
 if source.is_file() and source.suffix.lower() == ".zip":
 with zipfile.ZipFile(source) as zf:
 member = _select_member(zf, token, ".json")
 with zf.open(member) as stream:
 return json.load(stream)
 if source.is_dir():
 matches = sorted(source.rglob(f"*{token}*"))
 if not matches:
 raise FileNotFoundError(token)
 return json.loads(matches[0].read_text(encoding="utf-8"))
 raise FileNotFoundError(source)

def _json_default(value: Any) -> Any:
 if isinstance(value, (np.bool_,)):
 return bool(value)
 if isinstance(value, (np.integer,)):
 return int(value)
 if isinstance(value, (np.floating,)):
 return float(value)
 if isinstance(value, np.ndarray):
 return value.tolist()
 if isinstance(value, Path):
 return str(value)
 raise TypeError(type(value).__name__)

def atomic_json(payload: dict[str, Any], path: Path) -> None:
 tmp = path.with_suffix(path.suffix + ".tmp")
 tmp.write_text(
 json.dumps(payload, ensure_ascii=False, indent=2, default=_json_default),
 encoding="utf-8",
)
 tmp.replace(path)

def atomic_csv(frame: pd.DataFrame, path: Path) -> None:
 tmp = path.with_suffix(path.suffix + ".tmp")
 frame.to_csv(tmp, index=False, encoding="utf-8-sig")
 tmp.replace(path)

def wrap_k(value: float) -> float:

```

---

```

 return float((float(value) + math.pi) % TWO_PI - math.pi)

def principal_angle(value: complex) -> float:
 return float(np.angle(value))

=====
Step12M reconstruction
=====

def load_step12_inputs(step12_source: str | Path) -> dict[str, Any]:
 final_certificate = read_json_token(step12_source, "step12M_18_final_mechanism_certificate.json")
 edge = read_csv_token(step12_source, "step12M_01_target_edge_definition.csv").iloc[0].copy()
 closures = read_csv_token(step12_source, "step12M_06_critical_closure_parameters.csv")
 segment_labels = read_csv_token(step12_source, "step12M_05_strict_segment_chern_labels.csv")
 group_certificates = read_csv_token(step12_source, "step12M_13_closure_group_certificates.csv")
 berry = read_csv_token(step12_source, "step12M_12_berry_charge_consensus.csv")
 grid = read_csv_token(step12_source, "step12M_15_refined_junction_strict_grid.csv")
 gap_track = read_csv_token(step12_source, "step12M_04_target_edge_gap_tracks.csv")

 if str(edge.get("edge_id")) != TARGET_EDGE_ID:
 raise ValueError(f"Unexpected target edge: {edge.get('edge_id')}")
 background = {
 name: float(final_certificate["fixed_background_parameters"][name])
 for name in ["m_e", "t1", "t2", "r1", "r2"]
 }

 if "closure_group_id" in closures.columns:
 grouped = closures.groupby("closure_group_id", sort=False)["critical_lambda"].mean()
 closure_lambdas = sorted(float(x) for x in grouped.to_list())
 else:
 closure_lambdas = sorted(float(x) for x in closures["critical_lambda"].to_list())
 if len(closure_lambdas) != 2:
 raise RuntimeError(f"Step13M requires exactly two Step12M closure groups, found {closure_lambdas}")

 return {
 "final_certificate": final_certificate,
 "edge": edge,
 "closures": closures,
 "segment_labels": segment_labels,
 "group_certificates": group_certificates,
 "berry": berry,
 "grid": grid,
 "gap_track": gap_track,
 "background": background,
 "closure_lambdas": closure_lambdas,
 }

def build_probe_table(inputs: dict[str, Any], config: Step13Config) -> pd.DataFrame:
 lam1, lam2 = inputs["closure_lambdas"]
 segments = inputs["segment_labels"].sort_values("segment_index")
 pre_rows = segments[segments["segment_index"].astype(int).eq(0)]
 post_rows = segments[segments["segment_index"].astype(int).eq(2)]
 pre_lambda = float(pre_rows.iloc[0]["probe_lambda"]) if not pre_rows.empty else 0.5 * lam1
 post_lambda = float(post_rows.iloc[0]["probe_lambda"]) if not post_rows.empty else 0.5 * (lam2 + 1.0)

 rows = [
 {
 "probe_id": "control_pre_minus2",
 "probe_role": "pre_control",
 "lambda": pre_lambda,
 "expected_chern_up": -2,
 "middle_fraction": np.nan,
 }
]
 for index, fraction in enumerate(config.middle_fractions):

```

---

```

 rows.append(
 {
 "probe_id": f"middle_{index+1:02d}",
 "probe_role": "intermediate",
 "lambda": lam1 + float(fraction) * (lam2 - lam1),
 "expected_chern_up": 2,
 "middle_fraction": float(fraction),
 }
)
 rows.append(
 {
 "probe_id": "control_post_zero",
 "probe_role": "post_control",
 "lambda": post_lambda,
 "expected_chern_up": 0,
 "middle_fraction": np.nan,
 }
)
 edge = inputs["edge"]
 for row in rows:
 lam = float(row["lambda"])
 row["r3"] = float(edge["r3_0"] + lam * (edge["r3_1"] - edge["r3_0"]))
 row["r4"] = float(edge["r4_0"] + lam * (edge["r4_1"] - edge["r4_0"]))
 return pd.DataFrame(rows)

```

```

def closure_ky_anchors(inputs: dict[str, Any]) -> list[float]:
 valleys = read_csv_token_from_optional(inputs, "step12M_07_critical_valley_orbits.csv")
 anchors: list[float] = []
 if valleys.empty:
 return anchors
 ky_column = "critical_ky" if "critical_ky" in valleys.columns else ("ky" if "ky" in valleys.columns else None)
 if ky_column is None:
 return anchors
 selected = valleys.copy()
 if "spin" in selected.columns:
 selected = selected[selected["spin"].astype(str).eq("up")]
 if "is_active_valley" in selected.columns:
 selected = selected[selected["is_active_valley"].astype(int).eq(1)]
 for value in selected[ky_column].dropna().to_numpy(float):
 for offset in (0.0, -0.08, -0.04, -0.02, 0.02, 0.04, 0.08):
 anchors.append(wrap_k(float(value) + offset))
 return sorted(set(round(x, 12) for x in anchors))

```

```

def read_csv_token_from_optional(inputs: dict[str, Any], token: str) -> pd.DataFrame:
 source = inputs.get("_step12_source")
 if source is None:
 return pd.DataFrame()
 try:
 return read_csv_token(source, token)
 except FileNotFoundError:
 return pd.DataFrame()

```

```

=====
Physical core and occupied-subspace gap audit
=====

```

```

def configure_physics(tts_archive: str | Path, config: Step13Config):
 step12_config = step12.Step12Config(
 output_dir=config.output_dir / "_physics_core",
 quick=bool(config.quick),
 force_recalculate=False,
 random_seed=int(config.random_seed),
).normalized()
 core, step4, step5, step3, strict_config, search_config = step12.configure_physics(
 tts_archive,

```

---

```

 step12_config,
)
 return core, step4, step5, step3, strict_config, search_config

def spin_hamiltonian(core, raw: dict[str, float], kx: float, ky: float) -> np.ndarray:
 matrix = np.asarray(core.h_spin_block_periodic(float(kx), float(ky), raw, "up"), dtype=complex)
 if matrix.ndim != 2 or matrix.shape[0] != matrix.shape[1]:
 raise ValueError(f"Invalid spin Hamiltonian shape {matrix.shape}")
 return 0.5 * (matrix + matrix.conj().T)

def occupied_frame(core, raw: dict[str, float], kx: float, ky: float) -> tuple[np.ndarray, np.ndarray]:
 values, vectors = np.linalg.eigh(spin_hamiltonian(core, raw, kx, ky))
 n_occ = int(core.N_OCC_SPIN)
 if not (0 < n_occ < len(values)):
 raise ValueError(f"Invalid N_OCC_SPIN={n_occ} for dimension {len(values)}")
 return values, vectors[:, :n_occ]

def spin_internal_gap(core, raw: dict[str, float], kx: float, ky: float) -> float:
 values = np.linalg.eigvalsh(spin_hamiltonian(core, raw, kx, ky))
 n_occ = int(core.N_OCC_SPIN)
 return float(values[n_occ] - values[n_occ - 1])

def audit_global_spin_gap(
 core,
 raw: dict[str, float],
 *,
 grid_n: int,
 local_starts: int,
) -> dict[str, Any]:
 grid_n = int(grid_n)
 ks = np.linspace(-math.pi, math.pi, grid_n, endpoint=False)
 candidates: list[tuple[float, float, float]] = []
 for kx in ks:
 for ky in ks:
 gap = spin_internal_gap(core, raw, float(kx), float(ky))
 candidates.append((gap, float(kx), float(ky)))
 candidates.sort(key=lambda item: item[0])

 best = {"gap": float(candidates[0][0]), "kx": candidates[0][1], "ky": candidates[0][2]}
 unique_starts: list[tuple[float, float]] = []
 for _, kx, ky in candidates:
 if all(math.hypot(wrap_k(kx-a), wrap_k(ky-b)) > 0.05 for a, b in unique_starts):
 unique_starts.append((kx, ky))
 if len(unique_starts) >= int(local_starts):
 break

 def objective(x: np.ndarray) -> float:
 gap = spin_internal_gap(core, raw, wrap_k(float(x[0])), wrap_k(float(x[1])))
 return float(gap * gap)

 for start in unique_starts:
 result = minimize(
 objective,
 np.asarray(start, dtype=float),
 method="Powell",
 bounds=[(-math.pi, math.pi), (-math.pi, math.pi)],
 options={"xtol": 1.0e-11, "ftol": 1.0e-22, "maxiter": 1200},
)
 kx, ky = wrap_k(result.x[0]), wrap_k(result.x[1])
 gap = spin_internal_gap(core, raw, kx, ky)
 if gap < best["gap"]:
 best = {"gap": float(gap), "kx": float(kx), "ky": float(ky)}

 return {
 "spin_internal_gap": float(best["gap"]),

```

---

```

 "gap_kx": float(best["kx"]),
 "gap_ky": float(best["ky"]),
 "gap_grid_n": int(grid_n),
 "gap_local_starts": int(len(unique_starts)),
 }

=====
Non-Abelian Wilson loop
=====

def polar_unitary(matrix: np.ndarray) -> tuple[np.ndarray, float]:
 u, singular, vh = np.linalg.svd(matrix, full_matrices=False)
 return u @ vh, float(np.min(singular))

def wilson_loop_at_ky(
 h_fn: Callable[[float, float], np.ndarray],
 n_occ: int,
 ky: float,
 *,
 nkx: int,
 shift_x: float,
) -> dict[str, Any]:
 kxs = -math.pi + TWO_PI * (np.arange(int(nkx), dtype=float) + float(shift_x)) / int(nkx)
 frames: list[np.ndarray] = []
 direct_gaps: list[float] = []
 for kx in kxs:
 matrix = h_fn(float(kx), float(ky))
 values, vectors = np.linalg.eigh(0.5 * (matrix + matrix.conj().T))
 frames.append(vectors[:, :n_occ])
 direct_gaps.append(float(values[n_occ] - values[n_occ - 1]))

 wilson = np.eye(n_occ, dtype=complex)
 min_singular = 1.0
 for index in range(len(frames)):
 overlap = frames[index].conj().T @ frames[(index + 1) % len(frames)]
 link, singular = polar_unitary(overlap)
 min_singular = min(min_singular, singular)
 wilson = wilson @ link

 eigenvalues = np.linalg.eigvals(wilson)
 eigenphases = np.sort(np.angle(eigenvalues))
 determinant = np.linalg.det(wilson)
 determinant /= abs(determinant) if abs(determinant) else 1.0
 return {
 "ky": float(ky),
 "determinant": complex(determinant),
 "det_phase": float(np.angle(determinant)),
 "eigenphases": np.asarray(eigenphases, dtype=float),
 "min_overlap_singular": float(min_singular),
 "min_kx_direct_gap": float(np.min(direct_gaps)),
 }

def track_phase_branches(phase_rows: list[np.ndarray]) -> np.ndarray:
 if not phase_rows:
 return np.empty((0, 0), dtype=float)
 tracked = [np.sort(np.asarray(phase_rows[0], dtype=float))]
 for raw in phase_rows[1:]:
 raw = np.asarray(raw, dtype=float)
 previous = tracked[-1]
 previous_wrapped = (previous + math.pi) % TWO_PI - math.pi
 cost = np.abs(np.angle(np.exp(1j * (raw[None, :] - previous_wrapped[:, None]))))
 row_ind, col_ind = linear_sum_assignment(cost)
 ordered = np.empty_like(raw)
 ordered[row_ind] = raw[col_ind]
 delta = np.angle(np.exp(1j * (ordered - previous_wrapped)))

```

---

```

 tracked.append(previous + delta)
 return np.asarray(tracked, dtype=float)

def adaptive_wilson_winding(
 h_fn: Callable[[float, float], np.ndarray],
 n_occ: int,
 *,
 nkx: int,
 initial_nky: int,
 shift_x: float,
 shift_y: float,
 phase_step: float,
 max_points: int,
 max_rounds: int,
 ky_anchors: Iterable[float] = (),
) -> tuple[dict[str, Any], pd.DataFrame]:
 start = -math.pi + TWO_PI * float(shift_y) / int(initial_nky)
 coords = list(start + TWO_PI * np.arange(int(initial_nky), dtype=float) / int(initial_nky))
 for anchor in ky_anchors:
 value = float(anchor)
 while value < start:
 value += TWO_PI
 while value >= start + TWO_PI:
 value -= TWO_PI
 coords.append(value)
 coords = sorted(set(round(float(x), 14) for x in coords))

 cache: dict[float, dict[str, Any]] = {}

 def evaluate(coord: float) -> dict[str, Any]:
 key = round(float(coord), 14)
 if key not in cache:
 cache[key] = wilson_loop_at_ky(
 h_fn,
 n_occ,
 wrap_k(float(coord)),
 nkx=int(nkx),
 shift_x=float(shift_x),
)
 cache[key]["ky_coordinate"] = float(coord)
 return cache[key]

 refinement_rounds = 0
 for round_index in range(int(max_rounds) + 1):
 coords = sorted(coords)
 rows = [evaluate(x) for x in coords]
 new_points: list[float] = []
 for index, left in enumerate(coords):
 right = coords[index + 1] if index + 1 < len(coords) else coords[0] + TWO_PI
 left_row = rows[index]
 right_row = rows[index + 1] if index + 1 < len(rows) else rows[0]
 increment = principal_angle(right_row["determinant"] * np.conj(left_row["determinant"]))
 if abs(increment) > float(phase_step):
 new_points.append(0.5 * (left + right))
 if not new_points or len(coords) >= int(max_points) or round_index == int(max_rounds):
 refinement_rounds = round_index
 break
 remaining = int(max_points) - len(coords)
 coords.extend(new_points[:remaining])
 coords = sorted(set(round(float(x), 14) for x in coords))
 refinement_rounds = round_index + 1

 coords = sorted(coords)
 rows = [evaluate(x) for x in coords]
 determinants = np.asarray([row["determinant"] for row in rows], dtype=complex)
 increments = []
 for index in range(len(rows)):
 next_index = (index + 1) % len(rows)

```



```

 increments.append(principal_angle(determinants[next_index] * np.conj(determinants[index])))
winding_float = float(np.sum(increments) / TWO_PI)
winding_int = int(np rint(winding_float))
integer_residual = float(abs(winding_float - winding_int))

det_unwrapped = [float(rows[0]["det_phase"])]
for index in range(1, len(rows)):
 det_unwrapped.append(det_unwrapped[-1] + principal_angle(determinants[index] * np.conj(determinants[index
↵ - 1])))
branch_array = track_phase_branches([row["eigenphases"] for row in rows])

trace_rows: list[dict[str, Any]] = []
for index, row in enumerate(rows):
 item = {
 "ky_coordinate": float(coords[index]),
 "ky_wrapped": float(wrap_k(coords[index])),
 "det_phase": float(row["det_phase"]),
 "det_phase_unwrapped": float(det_unwrapped[index]),
 "min_overlap_singular": float(row["min_overlap_singular"]),
 "min_kx_direct_gap": float(row["min_kx_direct_gap"]),
 }
 for branch in range(branch_array.shape[1]):
 item[f"wilson_phase_branch_{branch}"] = float(branch_array[index, branch])
 trace_rows.append(item)

Add a closing point for plotting the full winding.
closing = dict(trace_rows[0])
closing["ky_coordinate"] = float(coords[0] + TWO_PI)
closing["ky_wrapped"] = float(trace_rows[0]["ky_wrapped"])
closing["det_phase_unwrapped"] = float(det_unwrapped[0] + TWO_PI * winding_float)
trace_rows.append(closing)

summary = {
 "raw_winding_float": winding_float,
 "raw_winding_int": winding_int,
 "integer_residual": integer_residual,
 "n_ky_points_final": int(len(coords)),
 "adaptive_refinement_rounds": int(refinement_rounds),
 "max_abs_phase_increment": float(np.max(np.abs(increments))),
 "min_overlap_singular": float(min(row["min_overlap_singular"] for row in rows)),
 "min_kx_direct_gap": float(min(row["min_kx_direct_gap"] for row in rows)),
}
return summary, pd.DataFrame(trace_rows)

```

```

=====
Probe execution and consensus
=====

```

```

def run_wilson_attempts(
 *,
 core,
 step5,
 pair: pd.Series,
 probes: pd.DataFrame,
 ky_anchors: list[float],
 config: Step13Config,
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
 attempt_rows: list[dict[str, Any]] = []
 trace_frames: list[pd.DataFrame] = []
 gap_rows: list[dict[str, Any]] = []

 for _, probe in probes.iterrows():
 probe_id = str(probe["probe_id"])
 probe_role = str(probe["probe_role"])
 probe_lambda = float(probe["lambda"])
 probe_r3 = float(probe["r3"])
 probe_r4 = float(probe["r4"])

```

---

```

probe_expected = int(probe["expected_chern_up"])
raw = step5.raw_on_path(pair, probe_lambda)
gap_checkpoint = config.output_dir / "wilson_checkpoints" / f"{probe_id}_gap.json"
if gap_checkpoint.is_file() and not config.force_recalculate:
 gap = json.loads(gap_checkpoint.read_text(encoding="utf-8"))
else:
 gap = audit_global_spin_gap(
 core,
 raw,
 grid_n=int(config.gap_grid_n),
 local_starts=int(config.gap_local_starts),
)
 atomic_json(gap, gap_checkpoint)
gap_rows.append({
 "probe_id": probe_id,
 "probe_role": probe_role,
 "lambda": probe_lambda,
 "r3": probe_r3,
 "r4": probe_r4,
 "expected_chern_up": probe_expected,
 **gap,
})

h_fn = lambda kx, ky, raw=raw: spin_hamiltonian(core, raw, kx, ky)
for attempt_index, (nkx, nky, shift) in enumerate(
 zip(config.wilson_nkx, config.wilson_initial_nky, config.wilson_shifts)
):
 checkpoint = config.output_dir / "wilson_checkpoints" / f"{probe_id}_attempt{attempt_index:02d}.json"
 trace_path = config.output_dir / "wilson_traces" / f"{probe_id}_attempt{attempt_index:02d}.csv"
 if checkpoint.is_file() and trace_path.is_file() and not config.force_recalculate:
 summary = json.loads(checkpoint.read_text(encoding="utf-8"))
 trace = pd.read_csv(trace_path, low_memory=False)
 else:
 summary, trace = adaptive_wilson_winding(
 h_fn,
 int(core.N_OCC_SPIN),
 nkx=int(nkx),
 initial_nky=int(nky),
 shift_x=float(shift[0]),
 shift_y=float(shift[1]),
 phase_step=float(config.adaptive_phase_step),
 max_points=int(config.adaptive_max_points),
 max_rounds=int(config.adaptive_max_rounds),
 ky_anchors=ky_anchors,
)
 atomic_json(summary, checkpoint)
 atomic_csv(trace, trace_path)
 row = {
 "probe_id": probe_id,
 "probe_role": probe_role,
 "lambda": probe_lambda,
 "r3": probe_r3,
 "r4": probe_r4,
 "expected_chern_up": probe_expected,
 "attempt_index": int(attempt_index),
 "nkx": int(nkx),
 "initial_nky": int(nky),
 "shift_x": float(shift[0]),
 "shift_y": float(shift[1]),
 **summary,
 }
 attempt_rows.append(row)
 trace = trace.copy()
 trace.insert(0, "attempt_index", int(attempt_index))
 trace.insert(0, "probe_id", probe_id)
 trace_frames.append(trace)

return pd.DataFrame(attempt_rows), pd.concat(trace_frames, ignore_index=True), pd.DataFrame(gap_rows)

```

---

```

def determine_orientation(attempts: pd.DataFrame, config: Step13Config) -> dict[str, Any]:
 control = attempts[attempts["probe_id"].astype(str).eq("control_pre_minus2")].copy()
 valid = control[
 control["integer_residual"].astype(float).le(float(config.integer_residual_tolerance))
 & control["min_overlap_singular"].astype(float).ge(float(config.minimum_overlap_singular_value))
 & control["max_abs_phase_increment"].astype(float).le(1.05 * float(config.adaptive_phase_step))
]
 raw_values = valid["raw_winding_int"].astype(int).to_list()
 if not raw_values or len(set(raw_values)) != 1 or abs(raw_values[0]) != 2:
 return {
 "orientation_calibrated": 0,
 "orientation_factor": np.nan,
 "raw_control_values": raw_values,
 "reason": "pre-control Wilson winding did not have a unique |C|=2 consensus",
 }
 factor = int(-2 // raw_values[0])
 return {
 "orientation_calibrated": 1,
 "orientation_factor": factor,
 "raw_control_values": raw_values,
 "reason": "calibrated against independently strict Step12M C_up=-2 control",
 }

def build_probe_consensus(
 attempts: pd.DataFrame,
 gaps: pd.DataFrame,
 orientation: dict[str, Any],
 config: Step13Config,
) -> tuple[pd.DataFrame, pd.DataFrame]:
 attempts = attempts.copy()
 factor = orientation.get("orientation_factor")
 if orientation.get("orientation_calibrated"):
 attempts["oriented_chern_up"] = attempts["raw_winding_int"].astype(int) * int(factor)
 else:
 attempts["oriented_chern_up"] = np.nan
 attempts["attempt_reliable"] = (
 attempts["integer_residual"].astype(float).le(float(config.integer_residual_tolerance))
 & attempts["min_overlap_singular"].astype(float).ge(float(config.minimum_overlap_singular_value))
 & attempts["max_abs_phase_increment"].astype(float).le(1.05 * float(config.adaptive_phase_step))
).astype(int)

 gap_map = gaps.set_index("probe_id").to_dict("index")
 rows: list[dict[str, Any]] = []
 for probe_id, group in attempts.groupby("probe_id", sort=False):
 reliable = group[group["attempt_reliable"].astype(int).eq(1)]
 values = reliable["oriented_chern_up"].dropna().astype(int).to_list()
 consensus = bool(values) and len(set(values)) == 1
 consensus_value = int(values[0]) if consensus else None
 expected = int(group.iloc[0]["expected_chern_up"])
 gap = gap_map[str(probe_id)]
 positive_gap = float(gap["spin_internal_gap"]) > float(config.minimum_positive_spin_gap)
 rows.append({
 "probe_id": str(probe_id),
 "probe_role": str(group.iloc[0]["probe_role"]),
 "lambda": float(group.iloc[0]["lambda"]),
 "r3": float(group.iloc[0]["r3"]),
 "r4": float(group.iloc[0]["r4"]),
 "expected_chern_up": expected,
 "wilson_attempt_count": int(len(group)),
 "wilson_reliable_attempt_count": int(len(reliable)),
 "wilson_unique_oriented_integer_count": int(len(set(values))),
 "wilson_chern_up_consensus": np.nan if consensus_value is None else consensus_value,
 "wilson_consensus": int(consensus),
 "spin_internal_gap": float(gap["spin_internal_gap"]),
 "positive_spin_gap": int(positive_gap),
 "expected_value_match": int(consensus and consensus_value == expected),
 "probe_certificate_pass": int(consensus and consensus_value == expected and positive_gap),
 })

```

---

```

 "minimum_overlap_singular": float(group["min_overlap_singular"].min()),
 "maximum_integer_residual": float(group["integer_residual"].max()),
 "maximum_phase_increment": float(group["max_abs_phase_increment"].max()),
 })
 return attempts, pd.DataFrame(rows)

=====
Certificate update
=====

def update_closure_certificates(
 original: pd.DataFrame,
 probe_consensus: pd.DataFrame,
) -> pd.DataFrame:
 frame = original.sort_values("mean_critical_lambda").reset_index(drop=True).copy()
 middle = probe_consensus[probe_consensus["probe_role"].astype(str).eq("intermediate")]
 middle_pass = int((middle["probe_certificate_pass"].astype(int) == 1).sum()) >= 1
 pre_pass = bool(
 probe_consensus.loc[
 probe_consensus["probe_id"].astype(str).eq("control_pre_minus2"),
 "probe_certificate_pass",
].astype(int).eq(1).all()
)
 post_pass = bool(
 probe_consensus.loc[
 probe_consensus["probe_id"].astype(str).eq("control_post_zero"),
 "probe_certificate_pass",
].astype(int).eq(1).all()
)
 if len(frame) >= 2 and pre_pass and middle_pass and post_pass:
 assignments = [(-2, 2), (2, 0)]
 for index, (left, right) in enumerate(assignments):
 frame.loc[index, "left_chern_up"] = left
 frame.loc[index, "right_chern_up"] = right
 frame.loc[index, "observed_delta_chern_up"] = right - left
 charge = int(frame.loc[index, "berry_charge_sum_up"])
 match = int(charge == right - left)
 frame.loc[index, "signed_charge_matches"] = match
 frame.loc[index, "closure_group_certificate_pass"] = int(
 match
 and int(frame.loc[index, "all_berry_charges_consensus"]) == 1
 and int(frame.loc[index, "all_kp_jacobians_rank3"]) == 1
)
 return frame

def build_final_certificate(
 *,
 inputs: dict[str, Any],
 probes: pd.DataFrame,
 attempts: pd.DataFrame,
 probe_consensus: pd.DataFrame,
 orientation: dict[str, Any],
 updated_groups: pd.DataFrame,
 config: Step13Config,
) -> dict[str, Any]:
 middle = probe_consensus[probe_consensus["probe_role"].astype(str).eq("intermediate")]
 n_middle_pass = int(middle["probe_certificate_pass"].astype(int).sum())
 no_middle_contradiction = not bool(
 ((middle["wilson_consensus"].astype(int) == 1)
 & (middle["wilson_chern_up_consensus"].fillna(999).astype(int) != 2)).any()
)
 pre_pass = bool(
 probe_consensus.loc[
 probe_consensus["probe_id"].astype(str).eq("control_pre_minus2"),
 "probe_certificate_pass",
].astype(int).eq(1).all()
)

```

```

)
post_pass = bool(
 probe_consensus.loc[
 probe_consensus["probe_id"].astype(str).eq("control_post_zero"),
 "probe_certificate_pass",
].astype(int).eq(1).all()
)
intermediate_pass = (
 n_middle_pass >= int(config.minimum_middle_certified_points)
 and no_middle_contradiction
)
groups_pass = bool(len(updated_groups) == 2 and
 ↪ updated_groups["closure_group_certificate_pass"].astype(int).eq(1).all())

original = inputs["final_certificate"]
target = original["target_edge_certificate"]
charge_total = int(target["berry_charge_sum_over_all_groups"])
charge_match = bool(target["signed_total_charge_matches_grid_delta"])
complete = bool(
 orientation.get("orientation_calibrated")
 and pre_pass
 and post_pass
 and intermediate_pass
 and groups_pass
 and charge_total == 2
 and charge_match
)

return {
 "code_version": CODE_VERSION,
 "research_design": "adaptive_non_abelian_wilson_loop_intermediate_sector_certification",
 "fixed_background_parameters": inputs["background"],
 "target_edge_id": TARGET_EDGE_ID,
 "closure_lambdas": inputs["closure_lambdas"],
 "orientation_calibration": orientation,
 "control_pre_minus2_pass": int(pre_pass),
 "control_post_zero_pass": int(post_pass),
 "n_intermediate_probe_points": int(len(middle)),
 "n_intermediate_points_certified_as_plus2": n_middle_pass,
 "minimum_required_intermediate_points": int(config.minimum_middle_certified_points),
 "no_reliable_middle_contradiction": int(no_middle_contradiction),
 "intermediate_Cup_plus2_certified": int(intermediate_pass),
 "updated_strict_segment_chern_sequence": [-2, 2, 0] if intermediate_pass else [-2, None, 0],
 "generic_four_valley_charge": 4,
 "SigmaPrime_two_valley_charge": -2,
 "berry_charge_sum_over_all_groups": charge_total,
 "signed_total_charge_matches_endpoint_delta": int(charge_match),
 "all_updated_closure_groups_certified": int(groups_pass),
 "target_sequence_minus2_to_plus2_to_zero": int(intermediate_pass and groups_pass),
 "edge_multiclosure_certificate_pass": int(complete),
 "step13M_research_complete": bool(complete),
 "valid_claim_if_pass": (
 "Along the fixed-background $r_3=0.060$ junction path, adaptive non-Abelian Wilson loops "
 "independently certify a narrow $C_{up}=+2$ sector between a generic four-valley $+4$ closure "
 "and a Sigma' two-valley -2 closure, completing the strict sequence $-2 \rightarrow +2 \rightarrow 0$."
),
 "not_claimed": (
 "This certification is local to the fixed-background target path. It does not fill the entire "
 "two-dimensional uncertain band with $C_{up}=+2$ and does not define a universal seven-dimensional rule."
),
 "probe_summary": probe_consensus.to_dict("records"),
}

```

```

=====
Figures
=====

```

---

```

def save_figure(fig: plt.Figure, output_base: Path) -> None:
 fig.savefig(output_base.with_suffix(".png"), dpi=600, bbox_inches="tight")
 fig.savefig(output_base.with_suffix(".pdf"), bbox_inches="tight")
 fig.savefig(output_base.with_suffix(".svg"), bbox_inches="tight")
 plt.close(fig)

def plot_wilson_flows(
 traces: pd.DataFrame,
 consensus: pd.DataFrame,
 orientation: dict[str, Any],
 output_dir: Path,
) -> None:
 selected_ids = ["control_pre_minus2", "middle_03", "control_post_zero"]
 available = [pid for pid in selected_ids if pid in set(traces["probe_id"].astype(str))]
 if not available:
 return
 factor = int(orientation.get("orientation_factor", 1)) if orientation.get("orientation_calibrated") else 1
 fig, axes = plt.subplots(1, len(available), figsize=(4.4 * len(available), 3.8), squeeze=False)
 for ax, probe_id in zip(axes[0], available):
 group = traces[traces["probe_id"].astype(str).eq(probe_id)]
 best_attempt = int(group["attempt_index"].max())
 curve = group[group["attempt_index"].astype(int).eq(best_attempt)].sort_values("ky_coordinate")
 x = (curve["ky_coordinate"].to_numpy(float) - curve["ky_coordinate"].min()) / TWO_PI
 y = factor * (curve["det_phase_unwrapped"].to_numpy(float) - curve["det_phase_unwrapped"].iloc[0]) /
 TWO_PI
 ax.plot(x, y, linewidth=1.6)
 row = consensus[consensus["probe_id"].astype(str).eq(probe_id)].iloc[0]
 value = row["wilson_chern_up_consensus"]
 title_value = "unresolved" if pd.isna(value) else f"Cr = {int(value):+d}"
 ax.set_title(f"{probe_id}\n{title_value}")
 ax.set_xlabel(r"k_y cycle")
 ax.grid(alpha=0.25)
 axes[0][0].set_ylabel(r"oriented $\Delta \arg \det W_x / 2 \pi$")
 fig.suptitle("Adaptive occupied-subspace Wilson-loop winding")
 fig.tight_layout()
 save_figure(fig, output_dir / "figures" / "step13M_wilson_determinant_winding")

def plot_probe_consensus(
 attempts: pd.DataFrame,
 consensus: pd.DataFrame,
 closure_lambdas: list[float],
 output_dir: Path,
) -> None:
 fig, ax = plt.subplots(figsize=(7.6, 4.4))
 reliable = attempts[attempts["attempt_reliable"].astype(int).eq(1)]
 for attempt_index, group in reliable.groupby("attempt_index"):
 ax.scatter(
 group["lambda"], group["oriented_chern_up"],
 s=42, alpha=0.65, label=f"Wilson discretization {int(attempt_index)+1}",
)
 passed = consensus[consensus["probe_certificate_pass"].astype(int).eq(1)]
 ax.scatter(
 passed["lambda"], passed["wilson_chern_up_consensus"],
 marker="o", facecolors="none", edgecolors="black", s=110, linewidths=1.4,
 label="certified probe",
)
 lam1, lam2 = closure_lambdas
 ax.axvline(lam1, linestyle="--", linewidth=1.2)
 ax.axvline(lam2, linestyle="--", linewidth=1.2)
 ax.plot([0, lam1, lam1, lam2, lam2, 1], [-2, -2, 2, 2, 0, 0], linewidth=1.2, alpha=0.7, label="target
 ↪ sequence")
 ax.set_xlim(0, 1)
 ax.set_ylim(-2.6, 2.6)
 ax.set_yticks([-2, -1, 0, 1, 2])
 ax.set_xlabel(r"path coordinate λ")
 ax.set_ylabel(r"spin-up Chern number $C_{\uparrow}$")

```

---

```

ax.set_title("Direct Wilson-loop certification of the narrow intermediate sector")
ax.grid(alpha=0.25)
ax.legend(frameon=False, fontsize=8, ncol=2)
fig.tight_layout()
save_figure(fig, output_dir / "figures" / "step13M_intermediate_wilson_consensus")

def plot_target_path_final(
 gap_track: pd.DataFrame,
 consensus: pd.DataFrame,
 closure_lambdas: list[float],
 output_dir: Path,
) -> None:
 lam1, lam2 = closure_lambdas
 fig, axes = plt.subplots(2, 1, figsize=(7.5, 6.4), sharex=True, gridspec_kw={"height_ratios": [1.2, 1.0]})
 ax = axes[0]
 ax.semilogy(gap_track["lambda"], np.maximum(gap_track["tracked_min_gap"], 1e-16), linewidth=1.4)
 ax.axvline(lam1, linestyle="--", linewidth=1.2)
 ax.axvline(lam2, linestyle="--", linewidth=1.2)
 ax.set_ylabel("tracked spin-up internal gap")
 ax.set_title("Resolved multi-closure path and directly certified Chern sequence")
 ax.grid(alpha=0.25)

 ax = axes[1]
 ax.plot([0, lam1, lam1, lam2, lam2, 1], [-2, -2, 2, 2, 0, 0], linewidth=2.0)
 for _, row in consensus.iterrows():
 if int(row["probe_certificate_pass"]) == 1:
 c = int(row["wilson_chern_up_consensus"])
 ax.scatter(float(row["lambda"]), c, s=65, color=PHASE_COLORS[c], edgecolor="black", linewidth=0.6,
 ↪ zorder=5)
 else:
 ax.scatter(float(row["lambda"]), 0, marker="x", s=55, color="black", zorder=5)
 ax.axvline(lam1, linestyle="--", linewidth=1.2)
 ax.axvline(lam2, linestyle="--", linewidth=1.2)
 ax.text(lam1, 2.35, "generic four-valley\ncharge +4", ha="center", va="bottom", fontsize=8)
 ax.text(lam2, 2.35, r"Σ two-valley" + "\ncharge -2", ha="center", va="bottom", fontsize=8)
 ax.set_xlabel(r"path coordinate λ")
 ax.set_ylabel(r"$C_{\uparrow}$")
 ax.set_ylim(-2.7, 2.8)
 ax.set_yticks([-2, 0, 2])
 ax.grid(alpha=0.25)
 fig.tight_layout()
 save_figure(fig, output_dir / "figures" / "step13M_final_target_path_atlas")

def plot_junction_zoom(
 grid: pd.DataFrame,
 consensus: pd.DataFrame,
 edge: pd.Series,
 closure_lambdas: list[float],
 output_dir: Path,
) -> None:
 fig, ax = plt.subplots(figsize=(6.6, 5.7))
 reliable = grid[grid["paper_phase_code"].isin([-2, -1, 0, 1, 2])]
 unreliable = grid[~grid["paper_phase_code"].isin([-2, -1, 0, 1, 2])]
 for phase, group in reliable.groupby("paper_phase_code"):
 phase = int(phase)
 ax.scatter(group["r3"], group["r4"], s=31, color=PHASE_COLORS[phase], label=rf"strict
 ↪ $C_{\uparrow}={phase:d}$")
 if not unreliable.empty:
 ax.scatter(unreliable["r3"], unreliable["r4"], marker="x", s=26, color="black", alpha=0.55,
 ↪ label="uniform-grid boundary/unreliable")

 r3 = float(edge["r3_0"])
 r4_start, r4_end = float(edge["r4_0"]), float(edge["r4_1"])
 lam1, lam2 = closure_lambdas
 r4_1 = r4_start + lam1 * (r4_end - r4_start)
 r4_2 = r4_start + lam2 * (r4_end - r4_start)

```

---

```

ax.plot([r3, r3], [r4_start, r4_1], color=PHASE_COLORS[-2], linewidth=3.0)
ax.plot([r3, r3], [r4_1, r4_2], color=PHASE_COLORS[2], linewidth=3.0)
ax.plot([r3, r3], [r4_2, r4_end], color=PHASE_COLORS[0], linewidth=3.0)
ax.scatter([r3, r3], [r4_1, r4_2], marker="*", s=150, color="black", zorder=7, label="certified closure")

passed = consensus[consensus["probe_certificate_pass"].astype(int).eq(1)]
for _, row in passed.iterrows():
 c = int(row["wilson_chern_up_consensus"])
 ax.scatter(float(row["r3"]), float(row["r4"]), s=80, color=PHASE_COLORS[c], edgecolor="black",
 ↪ linewidth=0.8, zorder=8)

ax.set_xlabel(r"r_3")
ax.set_ylabel(r"r_4")
ax.set_title("Right-junction zoom with path-resolved Wilson certification")
ax.grid(alpha=0.20)
handles, labels = ax.get_legend_handles_labels()
dedup = dict(zip(labels, handles))
ax.legend(dedup.values(), dedup.keys(), frameon=False, fontsize=8, loc="best")
fig.tight_layout()
save_figure(fig, output_dir / "figures" / "step13M_right_junction_wilson_certified")

=====
Main workflow
=====

def run_step13m(
 *,
 tts_archive: str | Path,
 step12_source: str | Path,
 output_dir: str | Path | None = None,
 config: Step13Config | None = None,
 run_physics: bool = True,
 run_plots: bool = True,
) -> dict[str, Any]:
 if config is None:
 if output_dir is None:
 output_dir = Path.cwd() / "outputs_tts_step13M_adaptive_wilson_intermediate_chern"
 config = Step13Config(output_dir=Path(output_dir))
 elif output_dir is not None:
 config.output_dir = Path(output_dir)
 config = config.normalized()

 print("[1/5] Load Step12M evidence and define Wilson probes")
 inputs = load_step12_inputs(step12_source)
 inputs["_step12_source"] = str(Path(step12_source).expanduser().resolve())
 probes = build_probe_table(inputs, config)
 atomic_csv(probes, config.output_dir / "step13M_01_probe_definitions.csv")
 audit = {
 "code_version": CODE_VERSION,
 "tts_archive": str(Path(tts_archive).expanduser().resolve()),
 "step12_source": str(Path(step12_source).expanduser().resolve()),
 "fixed_background_parameters": inputs["background"],
 "target_edge": inputs["edge"].to_dict(),
 "closure_lambdas": inputs["closure_lambdas"],
 "config": asdict(config),
 }
 atomic_json(audit, config.output_dir / "step13M_00_input_audit.json")

 if not run_physics:
 return {"code_version": CODE_VERSION, "physics_run": False, "probe_count": len(probes)}

 print("[2/5] Configure frozen TTS Hamiltonian and audit occupied-subspace gaps")
 core, step4, step5, step3, strict_config, search_config = configure_physics(tts_archive, config)
 pair = step11.build_pair_from_edge(inputs["edge"], inputs["background"])
 ky_anchors = closure_ky_anchors(inputs)

 print("[3/5] Adaptive non-Abelian Wilson loops")

```



---

```

attempts, traces, gaps = run_wilson_attempts(
 core=core,
 step5=step5,
 pair=pair,
 probes=probes,
 ky_anchors=ky_anchors,
 config=config,
)
atomic_csv(gaps, config.output_dir / "step13M_02_spin_gap_audit.csv")
atomic_csv(attempts, config.output_dir / "step13M_03_wilson_attempts_raw.csv")
atomic_csv(traces, config.output_dir / "step13M_05_wilson_phase_traces.csv")

orientation = determine_orientation(attempts, config)
attempts_oriented, consensus = build_probe_consensus(attempts, gaps, orientation, config)
atomic_csv(attempts_oriented, config.output_dir / "step13M_03_wilson_attempts.csv")
atomic_csv(consensus, config.output_dir / "step13M_04_wilson_probe_consensus.csv")
atomic_json(orientation, config.output_dir / "step13M_04b_orientation_calibration.json")

print("[4/5] Update closure-group and final mechanism certificates")
updated_groups = update_closure_certificates(inputs["group_certificates"], consensus)
atomic_csv(updated_groups, config.output_dir / "step13M_07_updated_closure_group_certificates.csv")
certificate = build_final_certificate(
 inputs=inputs,
 probes=probes,
 attempts=attempts_oriented,
 probe_consensus=consensus,
 orientation=orientation,
 updated_groups=updated_groups,
 config=config,
)
atomic_json(certificate, config.output_dir / "step13M_08_final_intermediate_chern_certificate.json")

summary = pd.DataFrame([
 {
 "updated_segment_0_chern": -2,
 "updated_segment_1_chern": 2 if certificate["intermediate_Cup_plus2_certified"] else np.nan,
 "updated_segment_2_chern": 0,
 "generic_four_valley_charge": 4,
 "SigmaPrime_two_valley_charge": -2,
 "total_charge": 2,
 "edge_multiclosure_certificate_pass": certificate["edge_multiclosure_certificate_pass"],
 }
])
atomic_csv(summary, config.output_dir / "step13M_06_updated_strict_segment_sequence.csv")

print("[5/5] Publication figures")
if run_plots:
 plot_wilson_flows(traces, consensus, orientation, config.output_dir)
 plot_probe_consensus(attempts_oriented, consensus, inputs["closure_lambdas"], config.output_dir)
 plot_target_path_final(inputs["gap_track"], consensus, inputs["closure_lambdas"], config.output_dir)
 plot_junction_zoom(inputs["grid"], consensus, inputs["edge"], inputs["closure_lambdas"],
 ↪ config.output_dir)

print(json.dumps(certificate, ensure_ascii=False, indent=2, default=_json_default))
return certificate

=====
Synthetic validation of the Wilson engine
=====

def _qwz_hamiltonian(mass: float) -> Callable[[float, float], np.ndarray]:
 sx = np.array([[0, 1], [1, 0]], dtype=complex)
 sy = np.array([[0, -1j], [1j, 0]], dtype=complex)
 sz = np.array([[1, 0], [0, -1]], dtype=complex)
 def h(kx: float, ky: float) -> np.ndarray:
 return math.sin(kx)*sx + math.sin(ky)*sy + (mass + math.cos(kx) + math.cos(ky))*sz
 return h

```

---

```

def synthetic_wilson_validation() -> dict[str, Any]:
 topological, _ = adaptive_wilson_winding(
 _qwz_hamiltonian(-1.0), 1,
 nkx=121, initial_nky=61, shift_x=0.0, shift_y=0.0,
 phase_step=0.35*math.pi, max_points=301, max_rounds=6,
)
 trivial, _ = adaptive_wilson_winding(
 _qwz_hamiltonian(3.0), 1,
 nkx=121, initial_nky=61, shift_x=0.0, shift_y=0.0,
 phase_step=0.35*math.pi, max_points=301, max_rounds=6,
)
 return {
 "topological_abs_winding": abs(int(topological["raw_winding_int"])),
 "trivial_winding": int(trivial["raw_winding_int"]),
 "synthetic_validation_pass": bool(abs(int(topological["raw_winding_int"])) == 1 and
 ↪ int(trivial["raw_winding_int"]) == 0),
 "topological_summary": topological,
 "trivial_summary": trivial,
 }

def build_parser() -> argparse.ArgumentParser:
 parser = argparse.ArgumentParser(description=__doc__)
 parser.add_argument("--tts-archive", required=True, type=Path)
 parser.add_argument("--step12-source", required=True, type=Path)
 parser.add_argument("--output-dir", type=Path,
 ↪ default=Path("outputs_tts_step13M_adaptive_wilson_intermediate_chern"))
 parser.add_argument("--quick", action="store_true")
 parser.add_argument("--force-recalculate", action="store_true")
 parser.add_argument("--no-plots", action="store_true")
 return parser

def main(argv: list[str] | None = None) -> int:
 args = build_parser().parse_args(argv)
 config = Step13Config(
 output_dir=args.output_dir,
 quick=bool(args.quick),
 force_recalculate=bool(args.force_recalculate),
)
 run_step13m(
 tts_archive=args.tts_archive,
 step12_source=args.step12_source,
 config=config,
 run_plots=not args.no_plots,
)
 return 0

if __name__ == "__main__":
 raise SystemExit(main())

```

## ► 代码清单 A.50.5 (python, 1211 行)

```

"""
TTS Step14M: physics-certified, mechanism-aware ML rule extraction.

This program does NOT search for new topology. It learns interpretable rules from
strictly certified Step11-Step13 outputs and the formal fixed-slice scans.

Main tasks

```

- 
- ```

-----
1. Build a physics-certified r3-r4 phase dataset with source-priority deduplication.
2. Compare polynomial logistic regression and random forest using spatial group holdout.
3. Recover sparse local mass formulas near the left-lower and right-upper boundaries.
4. Build a certified transition-event table and perform an explicitly exploratory
   generic-four-valley vs SigmaPrime-two-valley analysis with a sample-size gate.
5. Export publication-ready figures, tables, Markdown conclusions, and a JSON certificate.

```

Required inputs

- ```

- TTS_Cup2_Formal_Refined_Scans(1).zip (or extracted directory)
- outputs_tts_step11M_lieb_aligned_fixed_slice.zip (or extracted directory)
- outputs_tts_step12M_junction_multiclosure_repair.zip (or extracted directory)
- outputs_tts_step13M_adaptive_wilson_intermediate_chern.zip (or extracted directory)

```

Dependencies: numpy, pandas, matplotlib, scikit-learn

Import hotfix v1.1:

- ```

- Compatible with importlib.util.spec_from_file_location even when callers forget
  to register the module in sys.modules before exec_module.
"""

```

```

import argparse
import json
import math
import shutil
import tempfile
import warnings
import zipfile
from dataclasses import asdict, dataclass, field
from pathlib import Path
from typing import Any, Iterable, Sequence

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from matplotlib.colors import BoundaryNorm, ListedColormap
from sklearn.base import clone
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    balanced_accuracy_score,
    confusion_matrix,
    f1_score,
)
from sklearn.model_selection import GroupKFold
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler

try:
    from sklearn.model_selection import StratifiedGroupKFold
except ImportError:  # pragma: no cover
    StratifiedGroupKFold = None

CODE_VERSION = "TTS_STEP14M_PHYSICS_CERTIFIED_RULE_EXTRACTION_V1_1_IMPORT_HOTFIX_20260724"

@dataclass
class Step14Config:
    output_dir: Path = Path("outputs_tts_step14m_physics_certified_rule_extraction")
    random_state: int = 20260724
    n_spatial_splits: int = 5
    spatial_bins_r3: int = 7
    spatial_bins_r4: int = 7
    rf_n_estimators: int = 400
    rf_min_samples_leaf: int = 4

```

```

rf_max_depth: int | None = None
logistic_c: float = 1.0
mass_l1_c_grid: tuple[float, ...] = (0.01, 0.03, 0.1, 0.3, 1.0, 3.0, 10.0, 30.0)
mass_min_abs_distance: float = 0.0
mechanism_min_events_total: int = 20
mechanism_min_events_per_class: int = 5
dpi: int = 500
save_svg: bool = True
include_wilson_points_in_phase_training: bool = True
phase_allowed_labels: tuple[int, ...] = (-2, -1, 0, 1, 2)
strict_only: bool = True

def normalize(self) -> "Step14Config":
    self.output_dir = Path(self.output_dir)
    return self

# -----
# General I/O helpers
# -----

def _json_default(obj: Any) -> Any:
    if isinstance(obj, Path):
        return str(obj)
    if isinstance(obj, (np.integer,)):
        return int(obj)
    if isinstance(obj, (np.floating,)):
        return float(obj)
    if isinstance(obj, np.ndarray):
        return obj.tolist()
    raise TypeError(f"Object of type {type(obj).__name__} is not JSON serializable")

def write_json(path: Path, payload: Any) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(payload, ensure_ascii=False, indent=2, default=_json_default), encoding="utf-8")

def safe_read_csv(path: Path) -> pd.DataFrame:
    try:
        if not path.is_file() or path.stat().st_size == 0:
            return pd.DataFrame()
        return pd.read_csv(path, low_memory=False)
    except pd.errors.EmptyDataError:
        return pd.DataFrame()

def _extract_zip(zip_path: Path, target: Path) -> Path:
    target.mkdir(parents=True, exist_ok=True)
    with zipfile.ZipFile(zip_path, "r") as zf:
        zf.extractall(target)
    return target

def materialize_source(source: str | Path, work_dir: Path, tag: str) -> Path:
    """Return an extracted/searchable directory for a zip or directory source."""
    source = Path(source).expanduser().resolve()
    if not source.exists():
        raise FileNotFoundError(f"Input source does not exist: {source}")
    if source.is_dir():
        return source
    if source.suffix.lower() != ".zip":
        raise ValueError(f"Expected directory or .zip source, got: {source}")
    out = work_dir / tag
    if out.exists():
        shutil.rmtree(out)
    _extract_zip(source, out)
    return out

```

```

def find_first(root: Path, names: Sequence[str]) -> Path | None:
    name_set = set(names)
    for p in root.rglob("*"):
        if p.is_file() and p.name in name_set:
            return p
    return None

def find_all(root: Path, filename: str) -> list[Path]:
    return sorted(p for p in root.rglob(filename) if p.is_file())

def _series_numeric(df: pd.DataFrame, name: str, default: float = np.nan) -> pd.Series:
    if name not in df.columns:
        return pd.Series(default, index=df.index, dtype=float)
    return pd.to_numeric(df[name], errors="coerce")

def _series_bool01(df: pd.DataFrame, name: str, default: int = 0) -> pd.Series:
    if name not in df.columns:
        return pd.Series(default, index=df.index, dtype=int)
    s = pd.to_numeric(df[name], errors="coerce").fillna(default)
    return (s == 1).astype(int)

# -----
# Certified phase dataset
# -----

def load_formal_phase_points(formal_root: Path, allowed_labels: Sequence[int]) -> pd.DataFrame:
    rows: list[pd.DataFrame] = []
    for path in find_all(formal_root, "tts_scan_r3_r4_grid.csv"):
        df = safe_read_csv(path)
        if df.empty or not {"r3", "r4"}.issubset(df.columns):
            continue
        phase = _series_numeric(df, "phase_code")
        chern = _series_numeric(df, "chern_up_int")
        phase = phase.where(phase.notna(), chern)
        exact = _series_bool01(df, "chern_exact_consensus", default=0)
        strict_ins = _series_bool01(df, "is_strict_insulator", default=0)
        strict_gap = _series_bool01(df, "strict_gap_verified", default=0)
        physical = _series_bool01(df, "is_physical_insulator", default=0)
        strict_chern = _series_bool01(df, "strict_chern_verified", default=0)
        reliable = (
            phase.isin(list(allowed_labels))
            & exact.eq(1)
            & ((strict_ins.eq(1)) | (strict_gap.eq(1) & physical.eq(1)))
            & ((strict_chern.eq(1)) | phase.eq(0) | chern.notna())
        )
        out = pd.DataFrame({
            "sample_id": df.get("sample_id", pd.Series([f"formal_{i}" for i in range(len(df))])),
            "r3": _series_numeric(df, "r3"),
            "r4": _series_numeric(df, "r4"),
            "chern_up": phase,
            "min_direct_gap": _series_numeric(df, "min_direct_gap"),
            "indirect_gap": _series_numeric(df, "indirect_gap"),
            "spin_internal_gap": _series_numeric(df, "min_spin_gap_up"),
            "strict_label": reliable.astype(int),
            "source_type": "formal_scan",
            "source_region": path.parent.name,
            "source_priority": 10,
            "uncertain_or_boundary": (~reliable).astype(int),
        })
        rows.append(out)
    if not rows:
        raise FileNotFoundError("No tts_scan_r3_r4_grid.csv files found in formal scan source")
    return pd.concat(rows, ignore_index=True)

```

```

def load_step12_phase_points(step12_root: Path, allowed_labels: Sequence[int]) -> pd.DataFrame:
    path = find_first(step12_root, ["step12M_15_refined_junction_strict_grid.csv"])
    if path is None:
        return pd.DataFrame()
    df = safe_read_csv(path)
    if df.empty:
        return pd.DataFrame()
    phase = _series_numeric(df, "paper_phase_code")
    chern = _series_numeric(df, "chern_up_int")
    phase = phase.where(phase.notna(), chern)
    reliable = (
        phase.isin(list(allowed_labels))
        & _series_bool01(df, "strict_gap_verified").eq(1)
        & _series_bool01(df, "strict_chern_verified").eq(1)
        & _series_bool01(df, "chern_exact_consensus").eq(1)
    )
    return pd.DataFrame({
        "sample_id": df.get("sample_id", pd.Series([f"step12_{i}" for i in range(len(df))])),
        "r3": _series_numeric(df, "r3"),
        "r4": _series_numeric(df, "r4"),
        "chern_up": phase,
        "min_direct_gap": _series_numeric(df, "min_direct_gap"),
        "indirect_gap": _series_numeric(df, "indirect_gap"),
        "spin_internal_gap": np.nan,
        "strict_label": reliable.astype(int),
        "source_type": "step12_refined_grid",
        "source_region": "right_junction_13x13",
        "source_priority": 30,
        "uncertain_or_boundary": (~reliable).astype(int),
    })

def load_step13_wilson_points(step13_root: Path, allowed_labels: Sequence[int]) -> pd.DataFrame:
    path = find_first(step13_root, ["step13M_04_wilson_probe_consensus.csv"])
    if path is None:
        return pd.DataFrame()
    df = safe_read_csv(path)
    if df.empty:
        return pd.DataFrame()
    phase = _series_numeric(df, "wilson_chern_up_consensus")
    reliable = (
        _series_bool01(df, "probe_certificate_pass").eq(1)
        & _series_bool01(df, "wilson_consensus").eq(1)
        & phase.isin(list(allowed_labels))
    )
    return pd.DataFrame({
        "sample_id": df.get("probe_id", pd.Series([f"step13_{i}" for i in range(len(df))])),
        "r3": _series_numeric(df, "r3"),
        "r4": _series_numeric(df, "r4"),
        "chern_up": phase,
        "min_direct_gap": np.nan,
        "indirect_gap": np.nan,
        "spin_internal_gap": _series_numeric(df, "spin_internal_gap"),
        "strict_label": reliable.astype(int),
        "source_type": "step13_wilson",
        "source_region": df.get("probe_role", pd.Series("wilson_path", index=df.index)).astype(str),
        "source_priority": 40,
        "uncertain_or_boundary": (~reliable).astype(int),
    })

def deduplicate_phase_points(all_points: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
    df = all_points.copy()
    df = df[np.isfinite(df["r3"]) & np.isfinite(df["r4"])].copy()
    df["key_r3"] = df["r3"].round(10)
    df["key_r4"] = df["r4"].round(10)
    conflict_rows: list[dict[str, Any]] = []
    for (r3, r4), g in df[df["strict_label"].eq(1)].groupby(["key_r3", "key_r4"]):

```

```

        labels = sorted(set(int(x) for x in g["chern_up"].dropna()))
        if len(labels) > 1:
            conflict_rows.append({
                "r3": r3,
                "r4": r4,
                "strict_labels": ", ".join(map(str, labels)),
                "sources": ";".join(g["source_type"].astype(str)),
            })
    conflicts = pd.DataFrame(conflict_rows)
    # High-priority source wins. Within equal priority, strict label wins.
    df = df.sort_values(
        ["key_r3", "key_r4", "source_priority", "strict_label"],
        ascending=[True, True, False, False],
    )
    dedup = df.drop_duplicates(["key_r3", "key_r4"], keep="first").copy()
    dedup.drop(columns=["key_r3", "key_r4"], inplace=True)
    dedup.reset_index(drop=True, inplace=True)
    return dedup, conflicts

def make_spatial_groups(df: pd.DataFrame, bins_r3: int, bins_r4: int) -> pd.Series:
    r3 = df["r3"].to_numpy(float)
    r4 = df["r4"].to_numpy(float)
    eps = 1e-12
    edges3 = np.linspace(r3.min() - eps, r3.max() + eps, bins_r3 + 1)
    edges4 = np.linspace(r4.min() - eps, r4.max() + eps, bins_r4 + 1)
    i3 = np.clip(np.digitize(r3, edges3[1:-1], right=False), 0, bins_r3 - 1)
    i4 = np.clip(np.digitize(r4, edges4[1:-1], right=False), 0, bins_r4 - 1)
    return pd.Series([f"a:{02d}_b:{02d}" for a, b in zip(i3, i4)], index=df.index)

def build_phase_dataset(
    formal_root: Path,
    step12_root: Path,
    step13_root: Path,
    config: Step14Config,
) -> tuple[pd.DataFrame, pd.DataFrame, dict[str, Any]]:
    frames = [load_formal_phase_points(formal_root, config.phase_allowed_labels)]
    s12 = load_step12_phase_points(step12_root, config.phase_allowed_labels)
    if not s12.empty:
        frames.append(s12)
    if config.include_wilson_points_in_phase_training:
        s13 = load_step13_wilson_points(step13_root, config.phase_allowed_labels)
        if not s13.empty:
            frames.append(s13)
    raw = pd.concat(frames, ignore_index=True)
    dedup, conflicts = deduplicate_phase_points(raw)
    strict = dedup[dedup["strict_label"].eq(1) & dedup["chern_up"].isin(config.phase_allowed_labels)].copy()
    strict["chern_up"] = strict["chern_up"].astype(int)
    strict["spatial_group"] = make_spatial_groups(strict, config.spatial_bins_r3, config.spatial_bins_r4)
    audit = {
        "n_raw_rows": len(raw),
        "n_deduplicated_rows": len(dedup),
        "n_strict_training_rows": len(strict),
        "strict_label_counts": strict["chern_up"].value_counts().sort_index().to_dict(),
        "n_uncertain_or_boundary_rows": int((dedup["strict_label"] == 0).sum()),
        "n_conflicting_strict_coordinates": len(conflicts),
        "source_counts_deduplicated": dedup["source_type"].value_counts().to_dict(),
        "n_spatial_groups": int(strict["spatial_group"].nunique()),
    }
    return strict, dedup, {"audit": audit, "conflicts": conflicts}

# -----
# Spatial cross-validation phase models
# -----

def _phase_models(config: Step14Config) -> dict[str, Any]:
    poly_logistic = Pipeline([

```

```

        ("poly", PolynomialFeatures(degree=2, include_bias=False)),
        ("scale", StandardScaler()),
        ("model", LogisticRegression(
            C=config.logistic_c,
            max_iter=10000,
            class_weight="balanced",
            solver="lbfgs",
            random_state=config.random_state,
        )),
    ])
    rf = RandomForestClassifier(
        n_estimators=config.rf_n_estimators,
        min_samples_leaf=config.rf_min_samples_leaf,
        max_depth=config.rf_max_depth,
        class_weight="balanced_subsample",
        random_state=config.random_state,
        n_jobs=-1,
        max_features="sqrt",
    )
    return {"poly2_logistic": poly_logistic, "random_forest": rf}

def _make_cv_splits(y: np.ndarray, groups: np.ndarray, n_splits: int, random_state: int):
    n_groups = len(np.unique(groups))
    n_splits = max(2, min(n_splits, n_groups))
    if StratifiedGroupKFold is not None:
        cv = StratifiedGroupKFold(n_splits=n_splits, shuffle=True, random_state=random_state)
        return list(cv.split(np.zeros(len(y)), y, groups))
    cv = GroupKFold(n_splits=n_splits)
    return list(cv.split(np.zeros(len(y)), y, groups))

def spatial_cv_phase_models(phase_df: pd.DataFrame, config: Step14Config):
    features = ["r3", "r4"]
    X = phase_df[features].to_numpy(float)
    y = phase_df["chern_up"].to_numpy(int)
    groups = phase_df["spatial_group"].to_numpy(str)
    classes = np.array(sorted(np.unique(y)), dtype=int)
    splits = _make_cv_splits(y, groups, config.n_spatial_splits, config.random_state)
    models = _phase_models(config)
    metric_rows: list[dict[str, Any]] = []
    prediction_frames: list[pd.DataFrame] = []

    for model_name, estimator in models.items():
        y_pred = np.full_like(y, fill_value=999)
        fold_id = np.full(len(y), -1, dtype=int)
        fold_metrics: list[dict[str, Any]] = []
        for fold, (tr, te) in enumerate(splits):
            est = clone(estimator)
            est.fit(X[tr], y[tr])
            pred = est.predict(X[te]).astype(int)
            y_pred[te] = pred
            fold_id[te] = fold
            fold_metrics.append({
                "model": model_name,
                "fold": fold,
                "n_train": len(tr),
                "n_test": len(te),
                "balanced_accuracy": balanced_accuracy_score(y[te], pred),
                "accuracy": accuracy_score(y[te], pred),
                "macro_f1": f1_score(y[te], pred, average="macro", zero_division=0),
                "test_classes": ", ".join(map(str, sorted(np.unique(y[te])))),
            })
        valid = y_pred != 999
        metric_rows.append({
            "model": model_name,
            "n_samples": int(valid.sum()),
            "n_folds": len(splits),
            "balanced_accuracy": balanced_accuracy_score(y[valid], y_pred[valid]),

```

```

        "accuracy": accuracy_score(y[valid], y_pred[valid]),
        "macro_f1": f1_score(y[valid], y_pred[valid], average="macro", zero_division=0),
    })
    pred_df = phase_df[["sample_id", "r3", "r4", "chern_up", "spatial_group", "source_type"]].copy()
    pred_df["model"] = model_name
    pred_df["fold"] = fold_id
    pred_df["prediction"] = y_pred
    prediction_frames.append(pred_df)
    prediction_frames.append(pd.DataFrame(fold_metrics).assign(record_type="fold_metric"))

metrics = pd.DataFrame(metric_rows).sort_values(
    ["balanced_accuracy", "macro_f1", "accuracy"], ascending=False
).reset_index(drop=True)
predictions = pd.concat(prediction_frames, ignore_index=True, sort=False)
best_name = str(metrics.iloc[0]["model"])
best_model = clone(models[best_name]).fit(X, y)
cm = confusion_matrix(
    y,
    predictions[(predictions.get("record_type").isna()) & (predictions["model"] ==
↵ best_name)]["prediction"].astype(int),
    labels=classes,
)
return metrics, predictions, best_name, best_model, classes, cm

def random_forest_importance(
    phase_df: pd.DataFrame,
    fitted_rf: RandomForestClassifier,
    config: Step14Config,
) -> pd.DataFrame:
    X = phase_df[["r3", "r4"]].to_numpy(float)
    y = phase_df["chern_up"].to_numpy(int)
    result = permutation_importance(
        fitted_rf,
        X,
        y,
        scoring="balanced_accuracy",
        n_repeats=30,
        random_state=config.random_state,
        n_jobs=-1,
    )
    return pd.DataFrame({
        "feature": ["r3", "r4"],
        "permutation_importance_mean": result.importances_mean,
        "permutation_importance_std": result.importances_std,
        "gini_importance": fitted_rf.feature_importances_,
    }).sort_values("permutation_importance_mean", ascending=False)

# -----
# Sparse local mass recovery
# -----
MASS_TERMS = ["constant", "r3", "r4", "r3^2", "r3*r4", "r4^2"]

def raw_poly2_matrix(df: pd.DataFrame) -> np.ndarray:
    r3 = df["r3"].to_numpy(float)
    r4 = df["r4"].to_numpy(float)
    return np.column_stack([r3, r4, r3 * r3, r3 * r4, r4 * r4])

def make_branch_spatial_splits(df: pd.DataFrame, n_splits: int = 4):
    if "spatial_fold" in df.columns and df["spatial_fold"].nunique() >= 2:
        splits = []
        for fold in sorted(df["spatial_fold"].dropna().unique()):
            te = np.where(df["spatial_fold"].to_numpy() == fold)[0]
            tr = np.where(df["spatial_fold"].to_numpy() != fold)[0]
            if len(te) and len(tr):
                splits.append((tr, te))

```

```

        return splits
    group = make_spatial_groups(df, 4, 4).to_numpy()
    return list(GroupKFold(n_splits=min(n_splits, len(np.unique(group)))).split(df, df["paper_phase_code"],
↪ group))

def _fit_l1_poly_raw_coefficients(Xraw: np.ndarray, y: np.ndarray, C: float, seed: int):
    scaler = StandardScaler()
    Xz = scaler.fit_transform(Xraw)
    model = LogisticRegression(
        penalty="l1",
        solver="liblinear",
        C=C,
        class_weight="balanced",
        max_iter=10000,
        random_state=seed,
    )
    model.fit(Xz, y)
    beta_raw = model.coef_[0] / scaler.scale_
    intercept_raw = float(model.intercept_[0] - np.sum(model.coef_[0] * scaler.mean_ / scaler.scale_))
    return model, scaler, np.concatenate([[intercept_raw], beta_raw])

def _physical_mass_vector(defrow: pd.Series) -> np.ndarray:
    # vector order: constant, r3, r4, r3^2, r3*r4, r4^2
    c2 = float(defrow.get("coefficient_2", 0.0))
    c1 = float(defrow.get("coefficient_1", 0.0))
    c0 = float(defrow.get("coefficient_0", 0.0))
    orientation = float(defrow.get("orientation", 1.0))
    independent = str(defrow["independent_parameter"])
    dependent = str(defrow["dependent_parameter"])
    v = np.zeros(6, dtype=float)
    # orientation * [dependent - (c2 independent^2 + c1 independent + c0)]
    v[0] = -orientation * c0
    if dependent == "r3":
        v[1] += orientation
    elif dependent == "r4":
        v[2] += orientation
    if independent == "r3":
        v[1] += -orientation * c1
        v[3] += -orientation * c2
    elif independent == "r4":
        v[2] += -orientation * c1
        v[5] += -orientation * c2
    return v

def _orient_vector_to_physical(ml_vec: np.ndarray, physical_vec: np.ndarray) -> np.ndarray:
    if np.dot(ml_vec[1:], physical_vec[1:]) < 0:
        return -ml_vec
    return ml_vec

def _normalize_mass_vector(v: np.ndarray, branch: str) -> np.ndarray:
    index = 2 if "left" in branch else 1 # left normalize by r4, right by r3
    denom = v[index]
    if abs(denom) < 1e-12:
        denom = np.linalg.norm(v[1:]) or 1.0
    return v / denom

def _formula_from_vector(v: np.ndarray, precision: int = 7) -> str:
    chunks = []
    for coef, term in zip(v, MASS_TERMS):
        if abs(coef) < 1e-8:
            continue
        label = "1" if term == "constant" else term
        chunks.append(f"{coef:+.{precision}g}{label}")
    return " ".join(chunks).rstrip("+") if chunks else "0"

```

```

def _polyfit_loocv(x: np.ndarray, y: np.ndarray, degree: int) -> tuple[np.ndarray, float, float]:
    coeff = np.polyfit(x, y, degree)
    pred = np.polyval(coeff, x)
    rmse = float(np.sqrt(np.mean((pred - y) ** 2)))
    errs = []
    for i in range(len(x)):
        mask = np.arange(len(x)) != i
        c = np.polyfit(x[mask], y[mask], degree)
        errs.append(float(np.polyval(c, x[i]) - y[i]))
    loocv = float(np.sqrt(np.mean(np.square(errs))))
    return coeff, rmse, loocv

def _boundary_regression_vector(
    independent: str,
    dependent: str,
    coeff_descending: np.ndarray,
    orientation: float = 1.0,
) -> np.ndarray:
    """Return orientation*[dependent - polynomial(independent)] in MASS_TERMS order."""
    v = np.zeros(6, dtype=float)
    degree = len(coeff_descending) - 1
    coeff_ascending = coeff_descending[::-1]
    v[0] = -orientation * coeff_ascending[0]
    if dependent == "r3":
        v[1] += orientation
    else:
        v[2] += orientation
    if degree >= 1:
        if independent == "r3":
            v[1] += -orientation * coeff_ascending[1]
        else:
            v[2] += -orientation * coeff_ascending[1]
    if degree >= 2:
        if independent == "r3":
            v[3] += -orientation * coeff_ascending[2]
        else:
            v[5] += -orientation * coeff_ascending[2]
    return v

def recover_sparse_mass_rules(step11_root: Path, config: Step14Config):
    """
    Recover the local mass boundary from certified boundary points, then test the
    discovered mass sign on independent phase points around each branch.

    This avoids an identifiability failure of unconstrained L1 classification in a
    narrow correlated parameter band, where several algebraically different
    polynomials can classify the same finite grid but do not represent the physical
    boundary normal.
    """
    pred_path = find_first(step11_root, ["step11M_11_spatial_holdout_predictions.csv"])
    def_path = find_first(step11_root, ["step11M_10_local_signed_mass_definitions.csv"])
    boundary_path = find_first(step11_root, ["step11M_07_completed_certified_boundary_points.csv"])
    if pred_path is None or def_path is None or boundary_path is None:
        return pd.DataFrame(), pd.DataFrame(), pd.DataFrame()
    phase_data = safe_read_csv(pred_path)
    defs = safe_read_csv(def_path)
    boundary = safe_read_csv(boundary_path)
    if phase_data.empty or defs.empty or boundary.empty:
        return pd.DataFrame(), pd.DataFrame(), pd.DataFrame()
    phase_data = phase_data[phase_data["paper_phase_code"].isin([-2, 2]).copy()]
    boundary = boundary[boundary.get("point_certificate_pass", 0) == 1].copy()

    result_rows: list[dict[str, Any]] = []
    coefficient_rows: list[dict[str, Any]] = []
    prediction_rows: list[pd.DataFrame] = []

```

```

for _, defrow in defs.iterrows():
    branch = str(defrow["branch_hint"])
    b = boundary[boundary["branch_hint"] == branch].copy()
    d = phase_data[phase_data["branch_hint"] == branch].copy()
    if len(b) < 3 or d.empty:
        continue
    independent = str(defrow["independent_parameter"])
    dependent = str(defrow["dependent_parameter"])
    x = b[independent].to_numpy(float)
    y = b[dependent].to_numpy(float)

    candidate_rows = []
    max_degree = min(2, len(b) - 2)
    for degree in range(1, max_degree + 1):
        coeff, fit_rmse, loocv = _polyfit_loocv(x, y, degree)
        candidate_rows.append({
            "branch_hint": branch,
            "degree": degree,
            "fit_rmse": fit_rmse,
            "loocv_rmse": loocv,
            "coefficients_descending": json.dumps(coeff.tolist()),
        })
    cand_df = pd.DataFrame(candidate_rows).sort_values(["loocv_rmse", "degree"])
    selected = cand_df.iloc[0]
    degree = int(selected["degree"])
    coeff = np.array(json.loads(selected["coefficients_descending"]), dtype=float)
    ml_vec = _boundary_regression_vector(independent, dependent, coeff, orientation=1.0)
    physical_vec = _physical_mass_vector(defrow)
    ml_vec = _orient_vector_to_physical(ml_vec, physical_vec)

    # Orient the learned mass so positive mass maps to Cup=+2 on the independent phase points.
    score = _eval_mass(ml_vec, d["r3"].to_numpy(float), d["r4"].to_numpy(float))
    pred = np.where(score >= 0, 2, -2)
    if balanced_accuracy_score(d["paper_phase_code"], pred) < balanced_accuracy_score(d["paper_phase_code"],
↪ -pred):
        ml_vec = -ml_vec
        score = -score
        pred = -pred
    # Align physical vector to the same sign convention for coefficient comparison.
    if np.dot(ml_vec[1:], physical_vec[1:]) < 0:
        physical_vec = -physical_vec

    ml_norm = _normalize_mass_vector(ml_vec, branch)
    phys_norm = _normalize_mass_vector(physical_vec, branch)
    cos = float(np.dot(ml_vec[1:], physical_vec[1:]) / (
        np.linalg.norm(ml_vec[1:]) * np.linalg.norm(physical_vec[1:]) + 1e-15
    ))
    result_rows.append({
        "branch_hint": branch,
        "n_certified_boundary_points": len(b),
        "n_independent_phase_points": len(d),
        "selected_polynomial_degree": degree,
        "boundary_fit_rmse": float(selected["fit_rmse"]),
        "boundary_loocv_rmse": float(selected["loocv_rmse"]),
        "phase_sign_balanced_accuracy": balanced_accuracy_score(d["paper_phase_code"], pred),
        "phase_sign_accuracy": accuracy_score(d["paper_phase_code"], pred),
        "coefficient_cosine_similarity_to_physical_mass": cos,
        "ml_boundary_formula_normalized": _formula_from_vector(ml_norm),
        "physical_mass_formula_normalized": _formula_from_vector(phys_norm),
    })
    for term, a, bcoef in zip(MASS_TERMS, ml_norm, phys_norm):
        coefficient_rows.append({
            "branch_hint": branch,
            "term": term,
            "ml_normalized_coefficient": a,
            "physical_normalized_coefficient": bcoef,
            "absolute_difference": abs(a - bcoef),
        })

```

```

    pr = d[["r3", "r4", "paper_phase_code", "spatial_fold", "branch_hint"].copy()
    pr["ml_prediction"] = pred
    pr["ml_decision_score"] = score
    prediction_rows.append(pr)
    cand_df.to_csv(config.output_dir / f"step14M_mass_degree_search_{branch}.csv", index=False)

    return pd.DataFrame(result_rows), pd.DataFrame(coefficient_rows), pd.concat(prediction_rows,
    ↪ ignore_index=True)

# -----
# Certified transition-event atlas and exploratory mechanism model
# -----

def build_transition_event_dataset(step11_root: Path, step12_root: Path, step13_root: Path) -> pd.DataFrame:
    rows: list[dict[str, Any]] = []
    # Generic four-valley certified boundary points
    p11 = find_first(step11_root, ["step11M_07_completed_certified_boundary_points.csv"])
    if p11 is not None:
        df = safe_read_csv(p11)
        for _, r in df[df.get("point_certificate_pass", 0) == 1].iterrows():
            nval = int(r.get("n_active_spin_up_valleys", 0))
            mech = "generic_four_valley" if nval == 4 else f"certified_{nval}_valley"
            rows.append({
                "event_id": str(r.get("point_id")),
                "event_source": "step11_boundary",
                "r3": float(r.get("r3")),
                "r4": float(r.get("r4")),
                "kx": float(r.get("critical_kx_representative", np.nan)),
                "ky": float(r.get("critical_ky_representative", np.nan)),
                "n_valleys": nval,
                "delta_chern_up": float(r.get("strict_side_delta_chern_up", np.nan)),
                "berry_charge_sum_up": float(r.get("berry_charge_sum_up", np.nan)),
                "mechanism": mech,
                "certificate_pass": 1,
            })
    # Step11 passed two-valley junction events: use selected edge midpoint
    edges_path = find_first(step11_root, ["step11M_05_selected_junction_edges.csv"])
    cert_path = find_first(step11_root, ["step11M_06_junction_edge_certificates.csv"])
    if edges_path is not None and cert_path is not None:
        edges = safe_read_csv(edges_path)
        cert = safe_read_csv(cert_path)
        if not edges.empty and not cert.empty:
            merged = edges.merge(cert, on="edge_id", how="inner")
            for _, r in merged[merged["edge_multiclosure_certificate_pass"] == 1].iterrows():
                rows.append({
                    "event_id": str(r["edge_id"]),
                    "event_source": "step11_junction",
                    "r3": float(r["mid_r3"]),
                    "r4": float(r["mid_r4"]),
                    "kx": np.nan,
                    "ky": np.nan,
                    "n_valleys": int(r["n_active_spin_up_valleys_total"]),
                    "delta_chern_up": float(r["grid_endpoint_delta_chern_up"]),
                    "berry_charge_sum_up": float(r["berry_charge_sum_over_all_groups"]),
                    "mechanism": "SigmaPrime_two_valley" if int(r["n_active_spin_up_valleys_total"]) == 2 else
                    ↪ "multiclosure",
                    "certificate_pass": 1,
                })
    # Step13 updated closure groups joined to Step12 critical coordinates
    c13_path = find_first(step13_root, ["step13M_07_updated_closure_group_certificates.csv"])
    c12_path = find_first(step12_root, ["step12M_06_critical_closure_parameters.csv"])
    if c13_path is not None and c12_path is not None:
        c13 = safe_read_csv(c13_path)
        c12 = safe_read_csv(c12_path)
        if not c13.empty and not c12.empty:
            c12g = c12.groupby("closure_group_id", as_index=False).agg({
                "r3": "mean", "r4": "mean", "critical_kx": "mean", "critical_ky": "mean"
            })

```

```

    })
    merged = c13.merge(c12g, on="closure_group_id", how="left")
    for _, r in merged[merged["closure_group_certificate_pass"] == 1].iterrows():
        rows.append({
            "event_id": str(r["closure_group_id"]),
            "event_source": "step13_updated_closure",
            "r3": float(r["r3"]),
            "r4": float(r["r4"]),
            "kx": float(r["critical_kx"]),
            "ky": float(r["critical_ky"]),
            "n_valleys": int(r["n_active_spin_up_valleys"]),
            "delta_chern_up": float(r["observed_delta_chern_up"]),
            "berry_charge_sum_up": float(r["berry_charge_sum_up"]),
            "mechanism": str(r["mechanism"]),
            "certificate_pass": 1,
        })
    events = pd.DataFrame(rows)
    if events.empty:
        return events
    # Step13 events can duplicate one Step11 generic point only approximately; keep separately because it is a
    ↪ distinct junction mechanism event.
    events["abs_delta_chern_up"] = events["delta_chern_up"].abs()
    events["abs_berry_charge_sum"] = events["berry_charge_sum_up"].abs()
    events["charge_conservation_pass"] = np.isclose(
        events["delta_chern_up"], events["berry_charge_sum_up"], atol=1e-8
    ).astype(int)
    return events

```

```

def exploratory_mechanism_analysis(events: pd.DataFrame, config: Step14Config):
    if events.empty:
        return pd.DataFrame(), pd.DataFrame(), {
            "mechanism_model_status": "no_events",
            "mechanism_ml_conclusion_allowed": False,
        }
    selected = events[events["mechanism"].isin(["generic_four_valley", "SigmaPrime_two_valley"])].copy()
    counts = selected["mechanism"].value_counts()
    gate = (
        len(selected) >= config.mechanism_min_events_total
        and not counts.empty
        and counts.min() >= config.mechanism_min_events_per_class
        and len(counts) == 2
    )
    summary = {
        "n_certified_events": len(events),
        "mechanism_counts": events["mechanism"].value_counts().to_dict(),
        "all_events_charge_conserving": bool(events["charge_conservation_pass"].eq(1).all()),
        "mechanism_sample_size_gate_pass": bool(gate),
        "mechanism_ml_conclusion_allowed": bool(gate),
        "interpretation": (
            "Parameter-only mechanism classification is allowed."
            if gate else
            "Certified event count is too small or too imbalanced for a generalizable parameter-only mechanism"
            ↪ classifier; report the event atlas descriptively and treat any classifier as exploratory only."
        ),
    }
    if len(selected) < 4 or selected["mechanism"].nunique() < 2:
        return pd.DataFrame(), selected, summary

    X = selected[["r3", "r4"]].to_numpy(float)
    y = (selected["mechanism"] == "SigmaPrime_two_valley").astype(int).to_numpy()
    # Leave-one-event-out exploratory predictions, no claim of generalization unless gate passes.
    pred = np.zeros(len(selected), dtype=int)
    score = np.zeros(len(selected), dtype=float)
    for i in range(len(selected)):
        tr = np.arange(len(selected)) != i
        if len(np.unique(y[tr])) < 2:
            pred[i] = int(np.round(y[tr].mean()))

```

```

        score[i] = float(y[tr].mean())
        continue
    pipe = Pipeline([
        ("poly", PolynomialFeatures(2, include_bias=False)),
        ("scale", StandardScaler()),
        ("model", LogisticRegression(C=1.0, class_weight="balanced", max_iter=10000,
        ↪ random_state=config.random_state)),
    ])
    pipe.fit(X[tr], y[tr])
    pred[i] = int(pipe.predict(X[[i]])[0])
    score[i] = float(pipe.predict_proba(X[[i]])[0, 1])
    metrics = pd.DataFrame([
        "model": "exploratory_poly2_logistic_L0E0",
        "n_events": len(selected),
        "balanced_accuracy": balanced_accuracy_score(y, pred),
        "accuracy": accuracy_score(y, pred),
        "macro_f1": f1_score(y, pred, average="macro", zero_division=0),
        "sample_size_gate_pass": int(gate),
        "conclusion_allowed": int(gate),
    ])
    out = selected.copy()
    out["exploratory_prediction"] = np.where(pred == 1, "SigmaPrime_two_valley", "generic_four_valley")
    out["SigmaPrime_probability"] = score
    return metrics, out, summary

# -----
# Figures and reports
# -----
PHASE_COLORS = {
    -2: "#3B6FB6",
    -1: "#8AB6D6",
    0: "#F2C14E",
    1: "#F28E8E",
    2: "#C73E3A",
}

def save_figure(fig: plt.Figure, base: Path, config: Step14Config) -> None:
    base.parent.mkdir(parents=True, exist_ok=True)
    fig.savefig(base.with_suffix(".png"), dpi=config.dpi, bbox_inches="tight", facecolor="white")
    fig.savefig(base.with_suffix(".pdf"), bbox_inches="tight", facecolor="white")
    if config.save_svg:
        fig.savefig(base.with_suffix(".svg"), bbox_inches="tight", facecolor="white")
    plt.close(fig)

def plot_phase_model_metrics(metrics: pd.DataFrame, config: Step14Config) -> None:
    fig, ax = plt.subplots(figsize=(7.2, 4.6))
    x = np.arange(len(metrics))
    width = 0.24
    for j, col in enumerate(["balanced_accuracy", "macro_f1", "accuracy"]):
        ax.bar(x + (j - 1) * width, metrics[col], width=width, label=col.replace("_", " "))
    ax.set_xticks(x, metrics["model"], rotation=0)
    ax.set_ylim(0, 1.05)
    ax.set_ylabel("Spatial holdout score")
    ax.set_title("Physics-certified fixed-slice phase classification")
    ax.legend(frameon=False)
    ax.grid(axis="y", alpha=0.25)
    fig.tight_layout()
    save_figure(fig, config.output_dir / "figures/step14M_phase_model_comparison", config)

def plot_confusion(cm: np.ndarray, classes: np.ndarray, best_name: str, config: Step14Config) -> None:
    fig, ax = plt.subplots(figsize=(5.4, 4.8))
    im = ax.imshow(cm, cmap="Blues")
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            ax.text(j, i, str(cm[i, j]), ha="center", va="center")

```

```

ax.set_xticks(range(len(classes)), [str(c) for c in classes])
ax.set_yticks(range(len(classes)), [str(c) for c in classes])
ax.set_xlabel("Predicted $C_{\uparrow}$")
ax.set_ylabel("Certified $C_{\uparrow}$")
ax.set_title(f"Spatial holdout confusion matrix: {best_name}")
fig.colorbar(im, ax=ax, shrink=0.8, label="Count")
fig.tight_layout()
save_figure(fig, config.output_dir / "figures/step14M_best_phase_confusion_matrix", config)

def plot_phase_decision_map(
    certified: pd.DataFrame,
    all_points: pd.DataFrame,
    model: Any,
    classes: np.ndarray,
    best_name: str,
    config: Step14Config,
) -> None:
    r3_min, r3_max = certified.r3.min(), certified.r3.max()
    r4_min, r4_max = certified.r4.min(), certified.r4.max()
    gx = np.linspace(r3_min, r3_max, 360)
    gy = np.linspace(r4_min, r4_max, 360)
    xx, yy = np.meshgrid(gx, gy)
    pred = model.predict(np.column_stack([xx.ravel(), yy.ravel()])).reshape(xx.shape)
    present = sorted(set(int(x) for x in classes))
    colors = [PHASE_COLORS.get(c, "#999999") for c in present]
    cmap = ListedColormap(colors)
    bounds = [present[0] - 0.5] + [(a + b) / 2 for a, b in zip(present[:-1], present[1:])] + [present[-1] + 0.5]
    norm = BoundaryNorm(bounds, cmap.N)

    fig, ax = plt.subplots(figsize=(7.7, 6.1))
    ax.pcolormesh(xx, yy, pred, cmap=cmap, norm=norm, shading="auto", alpha=0.55)
    uncertain = all_points[all_points["strict_label"].eq(0)]
    if not uncertain.empty:
        ax.scatter(uncertain.r3, uncertain.r4, marker="x", s=12, linewidths=0.45, c="0.45", alpha=0.35,
            label="boundary/unreliable")
    for c in present:
        g = certified[certified["chern_up"] == c]
        ax.scatter(g.r3, g.r4, s=8, c=PHASE_COLORS.get(c), edgecolors="none", label=f"certified
            $C_{\uparrow}$")
    ax.set_xlabel("$r_3$")
    ax.set_ylabel("$r_4$")
    ax.set_title(f"Interpretable phase model ({best_name})\nbackground fixed to Step11--13 values")
    ax.legend(frameon=False, ncol=2, fontsize=8, loc="best")
    ax.set_xlim(r3_min, r3_max)
    ax.set_ylim(r4_min, r4_max)
    fig.tight_layout()
    save_figure(fig, config.output_dir / "figures/step14M_phase_decision_map", config)

def plot_feature_importance(importance: pd.DataFrame, config: Step14Config) -> None:
    fig, ax = plt.subplots(figsize=(5.5, 4.0))
    x = np.arange(len(importance))
    ax.bar(x, importance["permutation_importance_mean"], yerr=importance["permutation_importance_std"], capsize=4)
    ax.set_xticks(x, importance["feature"])
    ax.set_ylabel("Permutation importance\n(balanced accuracy decrease)")
    ax.set_title("Random-forest parameter importance")
    ax.grid(axis="y", alpha=0.25)
    fig.tight_layout()
    save_figure(fig, config.output_dir / "figures/step14M_parameter_importance", config)

def _eval_mass(v: np.ndarray, r3: np.ndarray, r4: np.ndarray) -> np.ndarray:
    return v[0] + v[1]*r3 + v[2]*r4 + v[3]*r3*r3 + v[4]*r3*r4 + v[5]*r4*r4

def plot_mass_rule_recovery(
    coefficient_df: pd.DataFrame,

```



```

branch_predictions: pd.DataFrame,
config: Step14Config,
) -> None:
    if coefficient_df.empty or branch_predictions.empty:
        return
    for branch in coefficient_df["branch_hint"].unique():
        c = coefficient_df[coefficient_df["branch_hint"] == branch].set_index("term")
        ml = np.array([c.loc[t, "ml_normalized_coefficient"] for t in MASS_TERMS])
        ph = np.array([c.loc[t, "physical_normalized_coefficient"] for t in MASS_TERMS])
        d = branch_predictions[branch_predictions["branch_hint"] == branch]
        x = np.linspace(d.r3.min(), d.r3.max(), 500)
        y = np.linspace(d.r4.min(), d.r4.max(), 500)
        xx, yy = np.meshgrid(x, y)
        fig, ax = plt.subplots(figsize=(6.2, 5.2))
        for label in [-2, 2]:
            g = d[d.paper_phase_code == label]
            ax.scatter(g.r3, g.r4, s=22, c=PHASE_COLORS[label], label=f"certified  $C_{\uparrow}$ ={label}$",
                ↪ alpha=0.75)
            ax.contour(xx, yy, _eval_mass(ph, xx, yy), levels=[0], colors="black", linewidths=2.0)
            ax.contour(xx, yy, _eval_mass(ml, xx, yy), levels=[0], colors="darkorange", linewidths=2.0,
                ↪ linestyle="--")
            ax.plot([], [], color="black", lw=2, label="physics-certified mass  $M=0$ $")
            ax.plot([], [], color="darkorange", lw=2, ls="--", label="sparse ML boundary")
            ax.set_xlabel("$r_3$")
            ax.set_ylabel("$r_4$")
            ax.set_title(branch.replace("_", " "))
            ax.legend(frameon=False, fontsize=8)
            fig.tight_layout()
            save_figure(fig, config.output_dir / f"figures/step14M_mass_rule_recovery_{branch}", config)

def plot_transition_events(events: pd.DataFrame, config: Step14Config) -> None:
    if events.empty:
        return
    fig, ax = plt.subplots(figsize=(6.8, 5.4))
    markers = {"generic_four_valley": "o", "SigmaPrime_two_valley": "^"}
    for mech, g in events.groupby("mechanism"):
        ax.scatter(
            g.r3, g.r4,
            s=55 + 12 * g.n_valleys,
            marker=markers.get(mech, "s"),
            label=f"{mech} (n={len(g)}",
            alpha=0.8,
        )
    for _, r in events.iterrows():
        if r["event_source"] == "step13_updated_closure":
            ax.annotate(f"$\\Delta C={int(r['delta_bern_up'])}$", (r.r3, r.r4), xytext=(4, 5), textcoords="offset
                ↪ points", fontsize=8)
    ax.set_xlabel("$r_3$")
    ax.set_ylabel("$r_4$")
    ax.set_title("Physics-certified transition-event atlas")
    ax.legend(frameon=False, fontsize=8)
    ax.grid(alpha=0.2)
    fig.tight_layout()
    save_figure(fig, config.output_dir / "figures/step14M_transition_mechanism_atlas", config)

def write_rule_summary(
    path: Path,
    phase_audit: dict[str, Any],
    metrics: pd.DataFrame,
    mass_results: pd.DataFrame,
    mechanism_summary: dict[str, Any],
    classes: Sequence[int],
) -> None:
    best = metrics.iloc[0].to_dict() if not metrics.empty else {}
    lines = [
        "# TTS Step14M: 物理认证驱动的机器学习规律总结",

```

```

    "",
    f" 代码版本: `{CODE_VERSION}`",
    "",
    "## 1. 数据边界",
    "",
    f"- 严格训练点: {phase_audit.get('n_strict_training_rows', 0)}",
    f"- 纳入的固定切片标签: {list(classes)}",
    f"- 边界/不可靠点: {phase_audit.get('n_uncertain_or_boundary_rows', 0)} (只用于显示, 不用于硬标签训练)",
    "- 本阶段只总结固定背景下的二维 `r3-r4` 规律, 不构成七维全局拓扑判据。",
    "",
    "## 2. 相区分类",
    "",
]
if best:
    lines += [
        f" 空间分块留出中最佳模型为 **{best['model']}**: ",
        f"- balanced accuracy = {best['balanced_accuracy']:.4f}",
        f"- macro F1 = {best['macro_f1']:.4f}",
        f"- accuracy = {best['accuracy']:.4f}",
        "",
        " 该性能用于衡量模型跨空间块重建固定切片相区的能力, 而不是随机网格记忆能力。",
    ]
lines += ["", "## 3. 局域质量公式恢复", ""]
if mass_results.empty:
    lines.append(" 未找到 Step11M 局域质量训练表。")
else:
    for _, r in mass_results.iterrows():
        lines += [
            f"### {r['branch_hint']}",
            f"- 相界两侧独立相点的质量符号 balanced accuracy: {r['phase_sign_balanced_accuracy']:.4f}",
            f"- 与物理质量方向余弦相似度: {r['coefficient_cosine_similarity_to_physical_mass']:.6f}",
            f"- 稀疏 ML 边界: `{r['ml_boundary_formula_normalized']} = 0`",
            f"- 物理认证边界: `{r['physical_mass_formula_normalized']} = 0`",
            "",
        ]
lines += [
    "## 4. 相变机制",
    "",
    f"- 认证事件数: {mechanism_summary.get('n_certified_events', 0)}",
    f"- 事件类别: {mechanism_summary.get('mechanism_counts', {})}",
    f"- 所有事件满足 Berry 荷守恒: {mechanism_summary.get('all_events_charge_conserving', False)}",
    f"- 参数到机制的机器学习结论是否允许: {mechanism_summary.get('mechanism_ml_conclusion_allowed', False)}",
    "",
    mechanism_summary.get("interpretation", ""),
    "",
    "## 5. 可发表的核心规律",
    "",
    "1. 固定背景下, 拓扑相主要由 `r3-r4` 的非线性组合组织; 随机划分不是可信验证, 必须采用空间分块留出。",
    "2. 左下 `C_up=-2/+2` 相界需要二次质量坐标, 右上相界在已认证局部范围内近似线性。",
    "3. 右上 junction 的 `-2 -> +2 -> 0` 不是单一质量翻转, 而是普通四谷 `+4` 与 Sigma` 双谷 `-2` 依次发生。",
    "4. 当前机制事件数量不足以支持七维或跨背景的通用机制分类器; 机制规律应以认证事件图谱和局域公式表述。",
]
path.write_text("\n".join(lines), encoding="utf-8")

```

```

# -----
# Main workflow
# -----

def run_step14m(
    formal_scan_source: str | Path,
    step11_source: str | Path,
    step12_source: str | Path,
    step13_source: str | Path,
    output_dir: str | Path = "outputs_tts_step14M_physics_certified_rule_extraction",
    config: Step14Config | None = None,
) -> dict[str, Any]:
    config = (config or Step14Config()).normalize()
    config.output_dir = Path(output_dir)

```

```

config.output_dir.mkdir(parents=True, exist_ok=True)
(config.output_dir / "figures").mkdir(exist_ok=True)
work_dir = config.output_dir / "_input_extract"
work_dir.mkdir(exist_ok=True)

print(f"Code version: {CODE_VERSION}")
print("[1/6] Materialize and audit inputs")
formal_root = materialize_source(formal_scan_source, work_dir, "formal")
step11_root = materialize_source(step11_source, work_dir, "step11")
step12_root = materialize_source(step12_source, work_dir, "step12")
step13_root = materialize_source(step13_source, work_dir, "step13")

print("[2/6] Build physics-certified phase dataset")
phase_df, all_phase_points, phase_info = build_phase_dataset(formal_root, step12_root, step13_root, config)
phase_df.to_csv(config.output_dir / "step14M_01_certified_phase_dataset.csv", index=False)
all_phase_points.to_csv(config.output_dir / "step14M_01b_all_phase_points_with_uncertain.csv", index=False)
phase_info["conflicts"].to_csv(config.output_dir / "step14M_01c_coordinate_label_conflicts.csv", index=False)
write_json(config.output_dir / "step14M_00_input_and_dataset_audit.json", {
    "code_version": CODE_VERSION,
    "config": asdict(config),
    **phase_info["audit"],
})

print("[3/6] Spatial group-holdout phase models")
metrics, predictions, best_name, best_model, classes, cm = spatial_cv_phase_models(phase_df, config)
metrics.to_csv(config.output_dir / "step14M_02_phase_model_metrics.csv", index=False)
predictions.to_csv(config.output_dir / "step14M_03_phase_spatial_holdout_predictions.csv", index=False)
pd.DataFrame(cm, index=classes, columns=classes).to_csv(config.output_dir /
↳ "step14M_04_best_phase_confusion_matrix.csv")

fitted_models = _phase_models(config)
rf = fitted_models["random_forest"].fit(phase_df[["r3", "r4"]].to_numpy(float),
↳ phase_df["chern_up"].to_numpy(int))
importance = random_forest_importance(phase_df, rf, config)
importance.to_csv(config.output_dir / "step14M_05_parameter_importance.csv", index=False)

print("[4/6] Sparse local mass-rule recovery")
mass_results, mass_coefficients, mass_predictions = recover_sparse_mass_rules(step11_root, config)
mass_results.to_csv(config.output_dir / "step14M_06_sparse_mass_rule_metrics.csv", index=False)
mass_coefficients.to_csv(config.output_dir / "step14M_07_sparse_mass_coefficients.csv", index=False)
mass_predictions.to_csv(config.output_dir / "step14M_08_sparse_mass_predictions.csv", index=False)

print("[5/6] Certified transition-event atlas and mechanism gate")
events = build_transition_event_dataset(step11_root, step12_root, step13_root)
events.to_csv(config.output_dir / "step14M_09_certified_transition_events.csv", index=False)
mechanism_metrics, mechanism_predictions, mechanism_summary = exploratory_mechanism_analysis(events, config)
mechanism_metrics.to_csv(config.output_dir / "step14M_10_exploratory_mechanism_metrics.csv", index=False)
mechanism_predictions.to_csv(config.output_dir / "step14M_11_exploratory_mechanism_predictions.csv",
↳ index=False)
write_json(config.output_dir / "step14M_12_mechanism_sample_size_gate.json", mechanism_summary)

print("[6/6] Figures and final certificate")
plot_phase_model_metrics(metrics, config)
plot_confusion(cm, classes, best_name, config)
plot_phase_decision_map(phase_df, all_phase_points, best_model, classes, best_name, config)
plot_feature_importance(importance, config)
plot_mass_rule_recovery(mass_coefficients, mass_predictions, config)
plot_transition_events(events, config)

write_rule_summary(
    config.output_dir / "step14M_13_interpretable_rule_summary.md",
    phase_info["audit"], metrics, mass_results, mechanism_summary, classes,
)

best = metrics.iloc[0].to_dict()
mass_pass = bool(
    not mass_results.empty
    and mass_results["phase_sign_balanced_accuracy"].min() >= 0.90

```

```

    and mass_results["coefficient_cosine_similarity_to_physical_mass"].min() >= 0.99
)
certificate = {
    "code_version": CODE_VERSION,
    "research_scope": "fixed-background r3-r4 slice only",
    "n_certified_phase_points": len(phase_df),
    "phase_classes_present": [int(x) for x in classes],
    "best_phase_model": best_name,
    "best_phase_balanced_accuracy_spatial_holdout": float(best["balanced_accuracy"]),
    "best_phase_macro_f1_spatial_holdout": float(best["macro_f1"]),
    "phase_rule_extraction_pass": bool(best["balanced_accuracy"] >= 0.80),
    "local_mass_rule_recovery_pass": mass_pass,
    "n_certified_transition_events": len(events),
    "all_transition_events_charge_conserving": bool(mechanism_summary.get("all_events_charge_conserving",
↵ False)),
    "mechanism_sample_size_gate_pass": bool(mechanism_summary.get("mechanism_sample_size_gate_pass", False)),
    "mechanism_ml_conclusion_allowed": bool(mechanism_summary.get("mechanism_ml_conclusion_allowed", False)),
    "step14M_fixed_slice_interpretable_summary_complete": bool(
        best["balanced_accuracy"] >= 0.80 and mass_pass and
        ↵ mechanism_summary.get("all_events_charge_conserving", False)
    ),
    "claims_allowed": [
        "Spatially held-out ML can summarize certified phase regions in the current fixed r3-r4 slice.",
        "Sparse polynomial ML can be compared quantitatively with the two certified local mass coordinates.",
        "Certified four-valley and SigmaPrime two-valley events can be summarized in an event atlas.",
    ],
    "claims_not_allowed": [
        "A global seven-parameter topological rule.",
        "A cross-background universal mass formula.",
        "A generalizable parameter-only mechanism classifier unless the mechanism sample-size gate passes.",
    ],
}
write_json(config.output_dir / "step14M_14_final_rule_extraction_certificate.json", certificate)
print(json.dumps(certificate, ensure_ascii=False, indent=2))
return certificate

```

```

def make_argument_parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument("--formal", default="TTS_Cup2_Formal_Refined_Scans(1).zip")
    p.add_argument("--step11", default="outputs_tts_step11M_lieb_aligned_fixed_slice.zip")
    p.add_argument("--step12", default="outputs_tts_step12M_junction_multiclosure_repair.zip")
    p.add_argument("--step13", default="outputs_tts_step13M_adaptive_wilson_intermediate_chern.zip")
    p.add_argument("--output", default="outputs_tts_step14M_physics_certified_rule_extraction")
    return p

```

```

def main() -> None:
    args = make_argument_parser().parse_args()
    run_step14m(args.formal, args.step11, args.step12, args.step13, args.output)

```

```

if __name__ == "__main__":
    main()

```

► 代码清单 A.50.6 (python, 901 行)

```

from __future__ import annotations

"""
TTS Step15M — exhaustive reclassification of every previously unresolved point
=====

```

Purpose

Re-evaluate every point marked ``boundary / unreliable`` in the fixed-background r3-r4 map and replace the ambiguous plotting label with a physically definite status whenever possible:

1. strict gapped phase with a certified spin Chern number;
2. indirect-overlap metal;
3. true full-gap closing / phase-boundary point;
4. spin-sector internal closing or spin-filling transition;
5. numeric_unresolved (retained only if all escalated methods fail).

A true gap-closing point is *not* assigned a Chern number, because the occupied subspace is not globally isolated there. Gapped points are first tested with multi-grid Fukui calculations and, if necessary, escalated to adaptive non-Abelian Wilson loops.

"""

```
import argparse
import importlib.util
import io
import json
import math
import sys
import time
import zipfile
from dataclasses import asdict, dataclass
from pathlib import Path
from typing import Any, Callable, Iterable

import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
from matplotlib.lines import Line2D
from matplotlib.patches import Patch
import numpy as np
import pandas as pd
from scipy.optimize import linear_sum_assignment, minimize
from scipy.interpolate import NearestNDInterpolator

CODE_VERSION = "TTS_STEP15M_RECLASSIFY_ALL_BOUNDARY_POINTS_V1_20260724"
TWO_PI = 2.0 * math.pi
REDUCED7 = ("m_e", "t1", "t2", "r1", "r2", "r3", "r4")

PHASE_COLORS = {
    -2: "#4F81BD",
    -1: "#8FBAD9",
    0: "#E6E6E6",
    1: "#E8B36A",
    2: "#CF5A47",
}

STATUS_COLORS = {
    "phase_boundary_gap_closing": "#111111",
    "indirect_overlap_metal": "#8C6D31",
    "spin_sector_gap_closing": "#7A3E9D",
    "spin_filling_transition": "#17A589",
    "indirect_overlap_boundary": "#B9770E",
    "numeric_unresolved": "#666666",
}

@dataclass
class Step15Config:
    output_dir: Path = Path("outputs_tts_step15m_reclassify_all_boundary_points")

    # Global gap audit.
    gap_grid_n: int = 101
    gap_local_starts: int = 14
    gap_powell_maxiter: int = 1200
    physical_zero_tolerance: float = 1.0e-9
```

```
positive_gap_tolerance: float = 5.0e-9
indirect_metal_tolerance: float = 5.0e-9
```

```
# Fukui hierarchy.
fukui_grids: tuple[int, ...] = (61, 81, 101, 121)
fukui_shifts: tuple[tuple[float, float], ...] = (
    (0.00, 0.00),
    (0.50, 0.50),
    (0.25, 0.73),
)
fukui_integer_tolerance: float = 0.055
fukui_min_link: float = 1.0e-9
fukui_sum_rule_tolerance: float = 0.06
fukui_min_consensus: int = 3
fukui_min_dominant_fraction: float = 0.75
```

```
# Adaptive Wilson fallback.
wilson_nkx: tuple[int, ...] = (401, 601, 801)
wilson_initial_nky: tuple[int, ...] = (81, 101, 121)
wilson_shifts: tuple[tuple[float, float], ...] = (
    (0.00, 0.00),
    (0.50, 0.37),
    (0.25, 0.73),
)
wilson_phase_step: float = 0.30 * math.pi
wilson_max_points: int = 901
wilson_max_rounds: int = 9
wilson_integer_tolerance: float = 0.06
wilson_min_overlap: float = 5.0e-5
```

```
checkpoint_every: int = 1
force_recalculate: bool = False
quick: bool = False
random_seed: int = 20260724
```

```
def normalized(self) -> "Step15Config":
    self.output_dir = Path(self.output_dir).expanduser().resolve()
    self.output_dir.mkdir(parents=True, exist_ok=True)
    for name in ["checkpoints", "wilson_traces", "figures", "_physics_source"]:
        (self.output_dir / name).mkdir(exist_ok=True)
    if self.quick:
        self.gap_grid_n = min(self.gap_grid_n, 51)
        self.gap_local_starts = min(self.gap_local_starts, 6)
        self.fukui_grids = (41, 61)
        self.fukui_shifts = ((0.0, 0.0), (0.5, 0.5))
        self.fukui_min_consensus = 2
        self.wilson_nkx = (201, 301)
        self.wilson_initial_nky = (61, 81)
        self.wilson_shifts = ((0.0, 0.0), (0.5, 0.37))
        self.wilson_max_points = min(self.wilson_max_points, 401)
    if len(self.wilson_nkx) != len(self.wilson_initial_nky):
        raise ValueError("wilson_nkx and wilson_initial_nky must have equal length")
    if len(self.wilson_nkx) != len(self.wilson_shifts):
        raise ValueError("wilson_shifts must match Wilson attempts")
    return self
```

```
# =====
# Safe archive I/O
# =====
```

```
def _select_member(zf: zipfile.ZipFile, token: str, suffix: str | None = None) -> str:
    matches = []
    for name in zf.namelist():
        if token not in Path(name).name and token not in name:
            continue
        if suffix and not name.lower().endswith(suffix.lower()):
            continue
```

```

        matches.append(name)
    if not matches:
        raise FileNotFoundError(f"No archive member containing {token!r}")
    return sorted(matches, key=lambda x: (len(Path(x).parts), len(x)))[0]

def read_csv_token(source: str | Path, token: str) -> pd.DataFrame:
    source = Path(source).expanduser().resolve()
    if source.is_file() and source.suffix.lower() == ".zip":
        with zipfile.ZipFile(source) as zf:
            member = _select_member(zf, token, ".csv")
            with zf.open(member) as stream:
                try:
                    return pd.read_csv(stream, low_memory=False)
                except pd.errors.EmptyDataError:
                    return pd.DataFrame()
    if source.is_dir():
        matches = sorted(source.rglob(f"*{token}*"))
        if not matches:
            raise FileNotFoundError(token)
        try:
            return pd.read_csv(matches[0], low_memory=False)
        except pd.errors.EmptyDataError:
            return pd.DataFrame()
    raise FileNotFoundError(source)

def read_json_token(source: str | Path, token: str) -> dict[str, Any]:
    source = Path(source).expanduser().resolve()
    if source.is_file() and source.suffix.lower() == ".zip":
        with zipfile.ZipFile(source) as zf:
            member = _select_member(zf, token, ".json")
            with zf.open(member) as stream:
                return json.load(stream)
    if source.is_dir():
        matches = sorted(source.rglob(f"*{token}*"))
        if not matches:
            raise FileNotFoundError(token)
        return json.loads(matches[0].read_text(encoding="utf-8"))
    raise FileNotFoundError(source)

def _json_default(value: Any) -> Any:
    if isinstance(value, Path):
        return str(value)
    if isinstance(value, np.ndarray):
        return value.tolist()
    if isinstance(value, (np.bool_,)):
        return bool(value)
    if isinstance(value, (np.integer,)):
        return int(value)
    if isinstance(value, (np.floating,)):
        return float(value)
    raise TypeError(type(value).__name__)

def atomic_json(payload: dict[str, Any], path: Path) -> None:
    tmp = path.with_suffix(path.suffix + ".tmp")
    tmp.write_text(json.dumps(payload, ensure_ascii=False, indent=2, default=_json_default), encoding="utf-8")
    tmp.replace(path)

def atomic_csv(frame: pd.DataFrame, path: Path) -> None:
    tmp = path.with_suffix(path.suffix + ".tmp")
    frame.to_csv(tmp, index=False, encoding="utf-8-sig")
    tmp.replace(path)

# =====

```

```

# Frozen Hamiltonian source
# =====

def extract_tts_core(tts_archive: str | Path, destination: Path) -> Path:
    archive = Path(tts_archive).expanduser().resolve()
    if not archive.is_file():
        raise FileNotFoundError(archive)
    marker = destination / "TTS_step01_model_and_label_audit_v2.py"
    if marker.is_file():
        return destination
    with zipfile.ZipFile(archive) as zf:
        candidates = [n for n in zf.namelist() if Path(n).name == "TTS_step01_model_and_label_audit_v2.py"]
        destination.mkdir(parents=True, exist_ok=True)
        if candidates:
            member = sorted(candidates, key=lambda x: (len(Path(x).parts), len(x)))[0]
            with zf.open(member) as src, marker.open("wb") as dst:
                dst.write(src.read())
            return destination

        nested = [n for n in zf.namelist() if Path(n).name.lower() in {"tts(1).zip", "tts.zip"}]
        if not nested:
            raise FileNotFoundError("Neither Step01 source nor a nested tts(1).zip was found")
        nested_member = sorted(nested, key=lambda x: (len(Path(x).parts), len(x)))[0]
        nested_bytes = zf.read(nested_member)
        with zipfile.ZipFile(io.BytesIO(nested_bytes)) as inner:
            inner_candidates = [n for n in inner.namelist() if Path(n).name ==
                                ↪ "TTS_step01_model_and_label_audit_v2.py"]
            if not inner_candidates:
                raise FileNotFoundError("TTS_step01_model_and_label_audit_v2.py not found in nested tts archive")
            member = sorted(inner_candidates, key=lambda x: (len(Path(x).parts), len(x)))[0]
            with inner.open(member) as src, marker.open("wb") as dst:
                dst.write(src.read())
    return destination

def import_module_from_path(name: str, path: Path):
    sys.modules.pop(name, None)
    spec = importlib.util.spec_from_file_location(name, path)
    if spec is None or spec.loader is None:
        raise ImportError(path)
    module = importlib.util.module_from_spec(spec)
    sys.modules[name] = module
    try:
        spec.loader.exec_module(module)
    except Exception:
        sys.modules.pop(name, None)
        raise
    return module

def load_core(tts_archive: str | Path, config: Step15Config):
    root = extract_tts_core(tts_archive, config.output_dir / "_physics_source")
    return import_module_from_path("tts_step15_core", root / "TTS_step01_model_and_label_audit_v2.py")

# =====
# Gap audit
# =====

def wrap_k(value: float) -> float:
    return float((float(value) + math.pi) % TWO_PI - math.pi)

def raw_parameters(background: dict[str, float], row: pd.Series, core) -> dict[str, float]:
    reduced = {**background, "r3": float(row["r3"]), "r4": float(row["r4"])}
    return core.raw8_from_reduced7(reduced, e0=0.0)

```

```

def spectral_values(core, raw: dict[str, float], kx: float, ky: float) -> dict[str, Any]:
    full = np.linalg.eigvalsh(np.asarray(core.h_tts_periodic(kx, ky, raw), dtype=complex))
    up = np.linalg.eigvalsh(np.asarray(core.h_spin_block_periodic(kx, ky, raw, "up"), dtype=complex))
    down = np.linalg.eigvalsh(np.asarray(core.h_spin_block_periodic(kx, ky, raw, "down"), dtype=complex))
    n_total = int(core.N_OCC_TOTAL)
    n_spin = int(core.N_OCC_SPIN)
    return {
        "direct": float(full[n_total] - full[n_total - 1]),
        "valence": float(full[n_total - 1]),
        "conduction": float(full[n_total]),
        "spin_up": float(up[n_spin] - up[n_spin - 1]),
        "spin_down": float(down[n_spin] - down[n_spin - 1]),
        "balanced": float(min(up[n_spin], down[n_spin]) - max(up[n_spin - 1], down[n_spin - 1])),
    }

def unique_starts(candidates: list[tuple[float, float, float]], n: int, distance: float = 0.08) ->
↳ list[tuple[float, float]]:
    starts: list[tuple[float, float]] = []
    for _, kx, ky in sorted(candidates, key=lambda x: x[0]):
        if all(math.hypot(wrap_k(kx-a), wrap_k(ky-b)) > distance for a, b in starts):
            starts.append((kx, ky))
        if len(starts) >= n:
            break
    return starts

def optimize_periodic(
    objective: Callable[[float, float], float],
    starts: list[tuple[float, float]],
    *,
    maxiter: int,
) -> tuple[float, float, float]:
    best = (float("inf"), float(starts[0][0]), float(starts[0][1]))
    for start in starts:
        result = minimize(
            lambda x: float(objective(wrap_k(x[0]), wrap_k(x[1]))),
            np.asarray(start, dtype=float),
            method="Powell",
            bounds=[(-math.pi, math.pi), (-math.pi, math.pi)],
            options={"xtol": 1.0e-11, "ftol": 1.0e-20, "maxiter": int(maxiter)},
        )
        kx, ky = wrap_k(result.x[0]), wrap_k(result.x[1])
        value = float(objective(kx, ky))
        if value < best[0]:
            best = (value, kx, ky)
    return best

def adaptive_gap_audit(core, raw: dict[str, float], config: Step15Config) -> dict[str, Any]:
    ks = np.linspace(-math.pi, math.pi, int(config.gap_grid_n), endpoint=False)
    candidates = {key: [] for key in ["direct", "spin_up", "spin_down", "balanced", "neg_valence", "conduction"]}
    for kx in ks:
        for ky in ks:
            s = spectral_values(core, raw, float(kx), float(ky))
            candidates["direct"].append((s["direct"], float(kx), float(ky)))
            candidates["spin_up"].append((s["spin_up"], float(kx), float(ky)))
            candidates["spin_down"].append((s["spin_down"], float(kx), float(ky)))
            candidates["balanced"].append((s["balanced"], float(kx), float(ky)))
            candidates["neg_valence"].append((-s["valence"], float(kx), float(ky)))
            candidates["conduction"].append((s["conduction"], float(kx), float(ky)))

    starts = {k: unique_starts(v, int(config.gap_local_starts)) for k, v in candidates.items()}

    direct = optimize_periodic(lambda x, y: spectral_values(core, raw, x, y)["direct"], starts["direct"],
↳ maxiter=config.gap_powell_maxiter)
    spin_up = optimize_periodic(lambda x, y: spectral_values(core, raw, x, y)["spin_up"], starts["spin_up"],
↳ maxiter=config.gap_powell_maxiter)

```

```

spin_down = optimize_periodic(lambda x, y: spectral_values(core, raw, x, y)["spin_down"], starts["spin_down"],
↪ maxiter=config.gap_powell_maxiter)
balanced = optimize_periodic(lambda x, y: spectral_values(core, raw, x, y)["balanced"], starts["balanced"],
↪ maxiter=config.gap_powell_maxiter)
neg_v = optimize_periodic(lambda x, y: -spectral_values(core, raw, x, y)["valence"], starts["neg_valence"],
↪ maxiter=config.gap_powell_maxiter)
cond = optimize_periodic(lambda x, y: spectral_values(core, raw, x, y)["conduction"], starts["conduction"],
↪ maxiter=config.gap_powell_maxiter)

max_valence = -neg_v[0]
min_conduction = cond[0]
indirect = min_conduction - max_valence
return {
    "min_direct_gap": direct[0], "direct_gap_kx": direct[1], "direct_gap_ky": direct[2],
    "min_spin_gap_up": spin_up[0], "spin_up_gap_kx": spin_up[1], "spin_up_gap_ky": spin_up[2],
    "min_spin_gap_down": spin_down[0], "spin_down_gap_kx": spin_down[1], "spin_down_gap_ky": spin_down[2],
    "min_balanced_sector_gap": balanced[0], "balanced_gap_kx": balanced[1], "balanced_gap_ky": balanced[2],
    "max_valence": max_valence, "vbm_kx": neg_v[1], "vbm_ky": neg_v[2],
    "min_conduction": min_conduction, "cbm_kx": cond[1], "cbm_ky": cond[2],
    "indirect_gap": indirect,
    "gap_grid_n": int(config.gap_grid_n),
    "gap_local_starts": int(config.gap_local_starts),
}

def preliminary_status(gap: dict[str, Any], config: Step15Config) -> str:
    z = float(config.physical_zero_tolerance)
    if float(gap["min_direct_gap"]) <= z:
        return "phase_boundary_gap_closing"
    if float(gap["indirect_gap"]) < -float(config.indirect_metal_tolerance):
        return "indirect_overlap_metal"
    if float(gap["indirect_gap"]) <= z:
        return "indirect_overlap_boundary"
    if float(gap["min_balanced_sector_gap"]) <= z:
        return "spin_filling_transition"
    if min(float(gap["min_spin_gap_up"]), float(gap["min_spin_gap_down"])) <= z:
        return "spin_sector_gap_closing"
    # Any strictly positive isolated occupied subspace is sent to topology
    # certification, even when the gap is extremely small.
    return "strict_gapped_candidate"

# =====
# Fukui classification
# =====

def integer_if_close(value: float, tolerance: float) -> int | None:
    if not np.isfinite(value):
        return None
    nearest = int(np rint(value))
    return nearest if abs(float(value) - nearest) <= float(tolerance) else None

def fukui_attempt(core, raw: dict[str, float], nk: int, shift: tuple[float, float], config: Step15Config) ->
↪ dict[str, Any]:
    row: dict[str, Any] = {
        "nk": int(nk), "shift_x": float(shift[0]), "shift_y": float(shift[1]),
        "attempt_reliable": 0, "attempt_error": "",
    }
    try:
        result = core.calculate_spin_cherns(raw, int(nk), shift)
        row.update(result)
        cu = integer_if_close(float(result["chern_up"]), config.fukui_integer_tolerance)
        cd = integer_if_close(float(result["chern_down"]), config.fukui_integer_tolerance)
        ct = integer_if_close(float(result["chern_total"]), config.fukui_integer_tolerance)
        row.update({"chern_up_int": cu, "chern_down_int": cd, "chern_total_int": ct})
        reliable = (

```

```

        cu is not None and cd is not None and ct is not None
        and min(float(result["min_det_up"]), float(result["min_det_down"]), float(result["min_det_total"])) >
        ↪ float(config.fukui_min_link)
        and float(result["sum_rule_error"]) <= float(config.fukui_sum_rule_tolerance)
    )
    row["attempt_reliable"] = int(reliable)
except Exception as exc:
    row["attempt_error"] = repr(exc)
return row

def fukui_consensus(attempts: list[dict[str, Any]], config: Step15Config) -> dict[str, Any]:
    reliable = [r for r in attempts if int(r.get("attempt_reliable", 0)) == 1]
    if not reliable:
        return {"fukui_consensus": 0, "fukui_chern_up": None, "fukui_chern_down": None, "fukui_chern_total": None,
        ↪ "fukui_consensus_count": 0}
    values = [(int(r["chern_up_int"]), int(r["chern_down_int"]), int(r["chern_total_int"])) for r in reliable]
    counts = pd.Series(values, dtype="object").value_counts()
    best = counts.index[0]
    count = int(counts.iloc[0])
    dominant_fraction = count / max(len(reliable), 1)
    ok = (
        count >= int(config.fukui_min_consensus)
        and dominant_fraction >= float(config.fukui_min_dominant_fraction)
    )
    return {
        "fukui_consensus": int(ok),
        "fukui_chern_up": int(best[0]) if ok else None,
        "fukui_chern_down": int(best[1]) if ok else None,
        "fukui_chern_total": int(best[2]) if ok else None,
        "fukui_consensus_count": count,
        "fukui_reliable_attempt_count": int(len(reliable)),
        "fukui_dominant_fraction": float(dominant_fraction),
        "fukui_unique_reliable_tuples": int(len(counts)),
    }

# =====
# Adaptive Wilson loop
# =====

def polar_unitary(matrix: np.ndarray) -> tuple[np.ndarray, float]:
    u, singular, vh = np.linalg.svd(matrix, full_matrices=False)
    return u @ vh, float(np.min(singular))

def spin_hamiltonian(core, raw: dict[str, float], spin: str, kx: float, ky: float) -> np.ndarray:
    h = np.asarray(core.h_spin_block_periodic(float(kx), float(ky), raw, spin), dtype=complex)
    return 0.5 * (h + h.conj().T)

def wilson_loop_at_ky(
    h_fn: Callable[[float, float], np.ndarray], n_occ: int, ky: float, *, nkx: int, shift_x: float,
) -> dict[str, Any]:
    kxs = -math.pi + TWO_PI * (np.arange(int(nkx), dtype=float) + float(shift_x)) / int(nkx)
    frames: list[np.ndarray] = []
    direct_gaps: list[float] = []
    for kx in kxs:
        values, vectors = np.linalg.eigh(h_fn(float(kx), float(ky)))
        frames.append(vectors[:, :n_occ])
        direct_gaps.append(float(values[n_occ] - values[n_occ - 1]))
    wilson = np.eye(n_occ, dtype=complex)
    min_singular = 1.0
    for i in range(len(frames)):
        overlap = frames[i].conj().T @ frames[(i + 1) % len(frames)]
        link, singular = polar_unitary(overlap)
        min_singular = min(min_singular, singular)

```

```

        wilson = wilson @ link
    determinant = np.linalg.det(wilson)
    determinant /= abs(determinant) if abs(determinant) else 1.0
    return {
        "det_phase": float(np.angle(determinant)),
        "min_overlap_singular": float(min_singular),
        "min_kx_direct_gap": float(np.min(direct_gaps)),
    }

def adaptive_wilson_winding(
    h_fn: Callable[[float, float], np.ndarray], n_occ: int, *, nkx: int, initial_nky: int,
    shift_x: float, shift_y: float, phase_step: float, max_points: int, max_rounds: int,
    ky_anchors: Iterable[float] = (),
) -> tuple[dict[str, Any], pd.DataFrame]:
    start = -math.pi + TWO_PI * float(shift_y) / int(initial_nky)
    coords = list(start + TWO_PI * np.arange(int(initial_nky), dtype=float) / int(initial_nky))
    for anchor in ky_anchors:
        value = float(anchor)
        while value < start:
            value += TWO_PI
        while value >= start + TWO_PI:
            value -= TWO_PI
        coords.append(value)
    coords = sorted(set(round(float(x), 14) for x in coords))
    cache: dict[float, dict[str, Any]] = {}

    def evaluate(coord: float) -> dict[str, Any]:
        key = round(float(coord), 14)
        if key not in cache:
            row = wilson_loop_at_ky(h_fn, n_occ, wrap_k(coord), nkx=int(nkx), shift_x=float(shift_x))
            row["ky_coordinate"] = float(coord)
            cache[key] = row
        return cache[key]

    rounds = 0
    for round_index in range(int(max_rounds) + 1):
        coords = sorted(coords)
        rows = [evaluate(x) for x in coords]
        new_points: list[float] = []
        for i, left in enumerate(coords):
            right = coords[i + 1] if i + 1 < len(coords) else coords[0] + TWO_PI
            phase_left = rows[i]["det_phase"]
            phase_right = rows[(i + 1) % len(rows)]["det_phase"]
            delta = float(np.angle(np.exp(1j * (phase_right - phase_left))))
            if abs(delta) > float(phase_step) and len(coords) + len(new_points) < int(max_points):
                new_points.append(0.5 * (left + right))
        if not new_points:
            rounds = round_index
            break
        coords.extend(new_points)
        rounds = round_index + 1

    coords = sorted(coords)
    rows = [evaluate(x) for x in coords]
    phases = np.asarray([r["det_phase"] for r in rows], dtype=float)
    increments = np.angle(np.exp(1j * (np.roll(phases, -1) - phases)))
    raw = float(np.sum(increments) / TWO_PI)
    raw_int = int(np rint(raw))
    trace = pd.DataFrame({
        "ky_coordinate": coords,
        "ky_wrapped": [wrap_k(x) for x in coords],
        "det_phase": phases,
        "min_overlap_singular": [r["min_overlap_singular"] for r in rows],
        "min_kx_direct_gap": [r["min_kx_direct_gap"] for r in rows],
    })
    summary = {
        "raw_winding": raw,
        "raw_winding_int": raw_int,
    }

```

```

        "integer_residual": abs(raw - raw_int),
        "min_overlap_singular": float(trace["min_overlap_singular"].min()),
        "max_abs_phase_increment": float(np.max(np.abs(increments))),
        "final_nky": int(len(coords)),
        "refinement_rounds": int(rounds),
    }
    return summary, trace

def wilson_attempts_for_spin(
    *, core: dict[str, float], spin: str, orientation_factor: int, gap: dict[str, Any],
    config: Step15Config, sample_id: str,
) -> tuple[list[dict[str, Any]], list[pd.DataFrame]]:
    ky0 = float(gap[f"spin_{spin}_gap_ky"])
    anchors = [wrap_k(ky0 + d) for d in (0.0, -0.08, -0.04, -0.02, 0.02, 0.04, 0.08)]
    attempts: list[dict[str, Any]] = []
    traces: list[pd.DataFrame] = []
    h_fn = lambda kx, ky: spin_hamiltonian(core, raw, spin, kx, ky)
    for index, (nkx, nky, shift) in enumerate(zip(config.wilson_nkx, config.wilson_initial_nky,
    ↪ config.wilson_shifts)):
        summary, trace = adaptive_wilson_winding(
            h_fn, int(core.N_OCC_SPIN), nkx=int(nkx), initial_nky=int(nky),
            shift_x=float(shift[0]), shift_y=float(shift[1]), phase_step=float(config.wilson_phase_step),
            max_points=int(config.wilson_max_points), max_rounds=int(config.wilson_max_rounds),
            ↪ ky_anchors=anchors,
        )
        oriented = int(summary["raw_winding_int"]) * int(orientation_factor)
        reliable = (
            float(summary["integer_residual"]) <= float(config.wilson_integer_tolerance)
            and float(summary["min_overlap_singular"]) >= float(config.wilson_min_overlap)
            and float(summary["max_abs_phase_increment"]) <= 1.10 * float(config.wilson_phase_step)
        )
        attempts.append({
            "sample_id": sample_id, "spin": spin, "attempt_index": index,
            "nkx": int(nkx), "initial_nky": int(nky), "shift_x": shift[0], "shift_y": shift[1],
            **summary, "oriented_chern": oriented, "attempt_reliable": int(reliable),
        })
        trace = trace.copy()
        trace.insert(0, "attempt_index", index)
        trace.insert(0, "spin", spin)
        trace.insert(0, "sample_id", sample_id)
        traces.append(trace)
    return attempts, traces

def wilson_consensus(attempts: list[dict[str, Any]]) -> tuple[bool, int | None, int]:
    values = [int(x["oriented_chern"]) for x in attempts if int(x["attempt_reliable"]) == 1]
    if not values:
        return False, None, 0
    counts = pd.Series(values).value_counts()
    value = int(counts.index[0])
    count = int(counts.iloc[0])
    return bool(count >= 2 and len(counts) == 1), value, count

# =====
# Final phase label
# =====

def phase_label_from_cherns(cu: int, cd: int, ct: int, indirect_gap: float) -> tuple[str, int | None]:
    if indirect_gap <= 0:
        return "indirect_overlap_metal", None
    if cu == -cd and ct == 0 and abs(cu) >= 1:
        return "spin_chern_II", cu
    if cu == 0 and cd == 0 and ct == 0:
        return "trivial_insulator", 0
    if ct != 0:

```

```

    return "chern_insulator", cu
return "other_gapped_phase", cu

```

```

def classify_one(
    *, core, background: dict[str, float], row: pd.Series, orientation_factor: int,
    config: Step15Config,
) -> tuple[dict[str, Any], list[dict[str, Any]], list[dict[str, Any]], list[pd.DataFrame]]:
    sample_id = str(row["sample_id"])
    raw = raw_parameters(background, row, core)
    gap = adaptive_gap_audit(core, raw, config)
    status = preliminary_status(gap, config)
    result: dict[str, Any] = {
        "sample_id": sample_id, "r3": float(row["r3"]), "r4": float(row["r4"]),
        "original_source_type": str(row.get("source_type", "")),
        "original_source_region": str(row.get("source_region", "")),
        **gap, "preliminary_status": status,
        "final_status": status, "final_chern_up": np.nan, "final_chern_down": np.nan,
        "final_chern_total": np.nan, "topology_method": "gap_audit_only",
        "strict_phase_code": 99, "definite_physical_conclusion": 1,
    }
    fukui_rows: list[dict[str, Any]] = []
    wilson_rows: list[dict[str, Any]] = []
    traces: list[pd.DataFrame] = []

    if status not in {"strict_gapped_candidate"}:
        return result, fukui_rows, wilson_rows, traces

    attempts: list[dict[str, Any]] = []
    for nk in config.fukui_grids:
        for shift in config.fukui_shifts:
            item = fukui_attempt(core, raw, int(nk), shift, config)
            item["sample_id"] = sample_id
            attempts.append(item)
    fukui_rows.extend(attempts)
    fcons = fukui_consensus(attempts, config)
    result.update(fcons)
    if int(fcons["fukui_consensus"]) == 1:
        cu, cd, ct = int(fcons["fukui_chern_up"]), int(fcons["fukui_chern_down"]), int(fcons["fukui_chern_total"])
        label, code = phase_label_from_cherns(cu, cd, ct, float(gap["indirect_gap"]))
        result.update({
            "final_status": label, "final_chern_up": cu, "final_chern_down": cd,
            "final_chern_total": ct, "topology_method": "multigrid_fukui_consensus",
            "strict_phase_code": 99 if code is None else int(code),
        })
        return result, fukui_rows, wilson_rows, traces

    # Wilson fallback for both spin sectors.
    up_attempts, up_traces = wilson_attempts_for_spin(
        core=core, raw=raw, spin="up", orientation_factor=orientation_factor,
        gap=gap, config=config, sample_id=sample_id,
    )
    down_attempts, down_traces = wilson_attempts_for_spin(
        core=core, raw=raw, spin="down", orientation_factor=orientation_factor,
        gap=gap, config=config, sample_id=sample_id,
    )
    wilson_rows.extend(up_attempts + down_attempts)
    traces.extend(up_traces + down_traces)
    up_ok, cu, up_count = wilson_consensus(up_attempts)
    down_ok, cd, down_count = wilson_consensus(down_attempts)
    result.update({
        "wilson_up_consensus": int(up_ok), "wilson_down_consensus": int(down_ok),
        "wilson_up_consensus_count": up_count, "wilson_down_consensus_count": down_count,
    })
    if up_ok and down_ok and cu is not None and cd is not None:
        ct = int(cu + cd)
        label, code = phase_label_from_cherns(cu, cd, ct, float(gap["indirect_gap"]))
        result.update({
            "final_status": label, "final_chern_up": cu, "final_chern_down": cd,

```

```

        "final_chern_total": ct, "topology_method": "adaptive_wilson_consensus",
        "strict_phase_code": 99 if code is None else int(code),
    })
else:
    result.update({
        "final_status": "numeric_unresolved", "topology_method": "fukui_and_wilson_failed",
        "definite_physical_conclusion": 0,
    })
return result, fukui_rows, wilson_rows, traces

# =====
# Plotting
# =====

def configure_plot_style() -> None:
    plt.rcParams.update({
        "font.family": "Times New Roman",
        "font.weight": "bold",
        "axes.labelweight": "bold",
        "axes.titleweight": "bold",
        "mathtext.fontset": "stix",
        "axes.unicode_minus": False,
    })

def plot_reclassified_region(
    revised: pd.DataFrame, boundary_results: pd.DataFrame, *, xlim: tuple[float, float], ylim: tuple[float,
    ↪ float],
    title: str, output_stem: Path,
) -> None:
    configure_plot_style()
    strict = revised[revised["strict_phase_code"].isin([-2, -1, 0, 1, 2])].copy()
    if strict.empty:
        return
    interp = NearestNDInterpolator(strict[["r3", "r4"]].to_numpy(float),
    ↪ strict["strict_phase_code"].to_numpy(int))
    x = np.linspace(xlim[0], xlim[1], 700)
    y = np.linspace(ylim[0], ylim[1], 700)
    X, Y = np.meshgrid(x, y)
    Z = interp(X, Y)
    rgba = np.zeros((len(y), len(x), 4), dtype=float)
    for code, color in PHASE_COLORS.items():
        rgba[Z == code] = plt.matplotlib.colors.to_rgba(color)

    fig, ax = plt.subplots(figsize=(8.2, 8.2))
    ax.imshow(rgba, origin="lower", extent=[*xlim, *ylim], interpolation="nearest", aspect="auto")
    local = boundary_results[
        boundary_results["r3"].between(*xlim) & boundary_results["r4"].between(*ylim)
    ]
    markers = {
        "phase_boundary_gap_closing": ("x", STATUS_COLORS["phase_boundary_gap_closing"]),
        "indirect_overlap_metal": ("P", STATUS_COLORS["indirect_overlap_metal"]),
        "spin_sector_gap_closing": ("D", STATUS_COLORS["spin_sector_gap_closing"]),
        "spin_filling_transition": ("v", STATUS_COLORS["spin_filling_transition"]),
        "indirect_overlap_boundary": ("h", STATUS_COLORS["indirect_overlap_boundary"]),
        "numeric_unresolved": ("X", STATUS_COLORS["numeric_unresolved"]),
    }
    for status, (marker, color) in markers.items():
        rows = local[local["final_status"].eq(status)]
        if not rows.empty:
            ax.scatter(rows["r3"], rows["r4"], marker=marker, s=55, color=color, linewidths=1.2, zorder=6)

    ax.set_xlim(*xlim); ax.set_ylim(*ylim); ax.set_box_aspect(1)
    ax.set_xlabel(r"$\boldsymbol{\mathrm{\mathit{r}}}_{\mathrm{3}}$", fontsize=24)
    ax.set_ylabel(r"$\boldsymbol{\mathrm{\mathit{r}}}_{\mathrm{4}}$", fontsize=24)
    ax.set_title(title, fontsize=23, pad=14)

```

```

ax.tick_params(direction="in", top=True, right=True, length=7, width=1.7, labelsz=15)
for spine in ax.spines.values(): spine.set_linewidth(1.7)
handles = [Patch(facecolor=PHASE_COLORS[c], edgecolor="black",
    ↳ label=r"$\boldsymbol{\{\mathit{\{C\}}\}_{\uparrow}}=\{c\}$") for c in
    ↳ sorted(set(strict["strict_phase_code"]).intersection(PHASE_COLORS))]
handles += [Line2D([0],[0], marker=m, linestyle="None", color=col, markersize=9, label=s.replace("_", " "))
    ↳ for s,(m,col) in markers.items() if (local["final_status"] == s).any()]
leg = ax.legend(handles=handles, loc="best", fontsize=11, frameon=True)
for text in leg.get_texts(): text.set_fontweight("bold")
fig.tight_layout()
for suffix in ["png", "pdf", "svg"]:
    fig.savefig(output_stem.with_suffix("." + suffix), dpi=600 if suffix == "png" else None,
        ↳ bbox_inches="tight")
plt.close(fig)

# =====
# Main workflow
# =====

def run_step15m(
    *, tts_archive: str | Path, step11_source: str | Path, step13_source: str | Path,
    step14_source: str | Path, output_dir: str | Path | None = None,
    config: Step15Config | None = None,
) -> dict[str, Any]:
    if config is None:
        config = Step15Config(output_dir=Path(output_dir or "outputs_tts_step15M_reclassify_all_boundary_points"))
    elif output_dir is not None:
        config.output_dir = Path(output_dir)
    config = config.normalized()

    all_points = read_csv_token(step14_source, "step14M_01b_all_phase_points_with_uncertain.csv")
    unresolved = all_points[all_points["uncertain_or_boundary"].astype(int).eq(1)].copy().reset_index(drop=True)
    background = read_json_token(step11_source,
        ↳ "step11M_13_fixed_slice_certificate.json")["fixed_background_parameters"]
    orientation = read_json_token(step13_source, "step13M_04b_orientation_calibration.json")
    if int(orientation.get("orientation_calibrated", 0)) != 1:
        raise RuntimeError("Step13 Wilson orientation is not calibrated")
    orientation_factor = int(orientation["orientation_factor"])
    core = load_core(tts_archive, config)

    atomic_csv(unresolved, config.output_dir / "step15M_01_original_unresolved_points.csv")
    atomic_json({
        "code_version": CODE_VERSION,
        "n_all_points": int(len(all_points)),
        "n_original_unresolved": int(len(unresolved)),
        "fixed_background_parameters": background,
        "wilson_orientation_factor": orientation_factor,
        "config": asdict(config),
    }, config.output_dir / "step15M_00_run_audit.json")

    results: list[dict[str, Any]] = []
    fukui_all: list[dict[str, Any]] = []
    wilson_all: list[dict[str, Any]] = []
    start = time.time()
    for index, row in unresolved.iterrows():
        sample_id = str(row["sample_id"])
        checkpoint = config.output_dir / "checkpoints" / f"{sample_id}.json"
        if checkpoint.is_file() and not config.force_recalculate:
            payload = json.loads(checkpoint.read_text(encoding="utf-8"))
            results.append(payload["result"])
            fukui_all.extend(payload.get("fukui_attempts", []))
            wilson_all.extend(payload.get("wilson_attempts", []))
            continue
        print(f"[{index+1}/{len(unresolved)}] {sample_id} r3={row['r3']:.8f} r4={row['r4']:.8f}")
        result, fukui_rows, wilson_rows, traces = classify_one(
            core=core, background=background, row=row, orientation_factor=orientation_factor, config=config,

```



```

)
payload = {"result": result, "fukui_attempts": fukui_rows, "wilson_attempts": wilson_rows}
atomic_json(payload, checkpoint)
for trace in traces:
    sid = str(trace.iloc[0]["sample_id"]); spin = str(trace.iloc[0]["spin"]); attempt =
    ↪ int(trace.iloc[0]["attempt_index"])
    atomic_csv(trace, config.output_dir / "wilson_traces" / f"{sid}_{spin}_{attempt:02d}.csv")
    results.append(result); fukui_all.extend(fukui_rows); wilson_all.extend(wilson_rows)
print("    ->", result["final_status"], result.get("final_chern_up"))

result_df = pd.DataFrame(results)
atomic_csv(result_df, config.output_dir / "step15M_05_final_boundary_reclassification.csv")
atomic_csv(pd.DataFrame(fukui_all), config.output_dir / "step15M_03_fukui_attempts.csv")
atomic_csv(pd.DataFrame(wilson_all), config.output_dir / "step15M_04_wilson_attempts.csv")

revised = all_points.copy()
revised["strict_phase_code"] = np.where(revised["uncertain_or_boundary"].astype(int).eq(0),
↪ revised["chern_up"], 99).astype(int)
revised["final_status"] = np.where(revised["uncertain_or_boundary"].astype(int).eq(0),
↪ "previously_certified_phase", "original_unresolved")
by_id = result_df.set_index("sample_id")
for idx, row in revised[revised["uncertain_or_boundary"].astype(int).eq(1)].iterrows():
    res = by_id.loc[str(row["sample_id"])]
    revised.loc[idx, "final_status"] = res["final_status"]
    revised.loc[idx, "strict_phase_code"] = int(res["strict_phase_code"])
    revised.loc[idx, "chern_up"] = res["final_chern_up"]
    revised.loc[idx, "uncertain_or_boundary"] = int(res["definite_physical_conclusion"] == 0)
atomic_csv(revised, config.output_dir / "step15M_06_revised_all_phase_points.csv")

counts =
↪ result_df["final_status"].value_counts(dropna=False).rename_axis("final_status").reset_index(name="count")
counts["fraction"] = counts["count"] / max(len(result_df), 1)
atomic_csv(counts, config.output_dir / "step15M_07_final_status_counts.csv")

plot_reclassified_region(
    revised, result_df, xlim=(-0.67, -0.52), ylim=(-0.180, -0.135),
    title="TTS left-lower phase boundary after exhaustive reclassification",
    output_stem=config.output_dir / "figures" / "step15M_left_lower_reclassified",
)
plot_reclassified_region(
    revised, result_df, xlim=(0.054, 0.066), ylim=(0.128, 0.134),
    title="TTS right junction after exhaustive reclassification",
    output_stem=config.output_dir / "figures" / "step15M_right_junction_reclassified",
)

n_numeric = int(result_df["final_status"].eq("numeric_unresolved").sum())
n_definite = int(result_df["definite_physical_conclusion"].astype(int).sum())
certificate = {
    "code_version": CODE_VERSION,
    "research_design": "exhaustive_adaptive_reclassification_of_all_previous_boundary_points",
    "n_original_unresolved_points": int(len(unresolved)),
    "n_points_with_definite_physical_conclusion": n_definite,
    "n_strict_chern_classified": int(result_df["strict_phase_code"].isin([-2,-1,0,1,2]).sum()),
    "n_true_gap_closing_or_near_boundary":
    ↪ int(result_df["final_status"].eq("phase_boundary_gap_closing").sum()),
    "n_indirect_overlap_metals": int(result_df["final_status"].eq("indirect_overlap_metal").sum()),
    "n_spin_sector_gap_closings": int(result_df["final_status"].eq("spin_sector_gap_closing").sum()),
    "n_spin_filling_transitions": int(result_df["final_status"].eq("spin_filling_transition").sum()),
    "n_indirect_overlap_boundaries": int(result_df["final_status"].eq("indirect_overlap_boundary").sum()),
    "n_numeric_unresolved": n_numeric,
    "all_previous_boundary_points_physically_resolved": bool(n_numeric == 0 and n_definite ==
    ↪ len(unresolved)),
    "elapsed_seconds": float(time.time() - start),
    "interpretation": {
        "strict_chern_phase": "A positive global insulating gap and isolated spin subspaces permit a
        ↪ spin-Chern label.",
        "true_boundary": "A gap-closing point is a definite phase-boundary conclusion but has no well-defined
        ↪ Chern number at the critical parameter.",
    }
}

```

```

        "metal": "An indirect-overlap point is not a topological insulator even if direct gaps at fixed k
        ↪ remain positive.",
    },
}
atomic_json(certificate, config.output_dir / "step15M_08_final_reclassification_certificate.json")
print(json.dumps(certificate, ensure_ascii=False, indent=2, default=_json_default))
return certificate

def build_parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument("--tts-archive", required=True)
    p.add_argument("--step11-source", required=True)
    p.add_argument("--step13-source", required=True)
    p.add_argument("--step14-source", required=True)
    p.add_argument("--output-dir", default="outputs_tts_step15M_reclassify_all_boundary_points")
    p.add_argument("--quick", action="store_true")
    p.add_argument("--force-recalculate", action="store_true")
    return p

def main() -> None:
    args = build_parser().parse_args()
    cfg = Step15Config(output_dir=Path(args.output_dir), quick=bool(args.quick),
    ↪ force_recalculate=bool(args.force_recalculate))
    run_step15m(
        tts_archive=args.tts_archive, step11_source=args.step11_source,
        step13_source=args.step13_source, step14_source=args.step14_source,
        output_dir=args.output_dir, config=cfg,
    )

if __name__ == "__main__":
    main()

```

► 代码清单 A.50.7 (python, 674 行)

```

from __future__ import annotations

import io
import json
import math
import zipfile
from dataclasses import dataclass, asdict
from pathlib import Path
from typing import Optional

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from matplotlib.colors import ListedColormap
from matplotlib.lines import Line2D
from matplotlib.patches import Patch

CODE_VERSION = "TTS_STEP16M_MASS_COORDINATE_PHASE_MAPS_V1_20260726"

# -----
# Plot style
# -----
plt.rcParams["font.family"] = "Times New Roman"
plt.rcParams["font.weight"] = "bold"
plt.rcParams["axes.labelweight"] = "bold"
plt.rcParams["axes.titleweight"] = "bold"

```

```

plt.rcParams["figure.titleweight"] = "bold"
plt.rcParams["axes.unicode_minus"] = False
plt.rcParams["mathtext.fontset"] = "custom"
plt.rcParams["mathtext.rm"] = "Times New Roman:bold"
plt.rcParams["mathtext.it"] = "Times New Roman:italic:bold"
plt.rcParams["mathtext.bf"] = "Times New Roman:bold"
plt.rcParams["mathtext.sf"] = "Times New Roman:bold"
plt.rcParams["mathtext.tt"] = "Times New Roman:bold"
plt.rcParams["mathtext.fallback"] = None
plt.rcParams["svg.fonttype"] = "none"
plt.rcParams["pdf.fonttype"] = 42
plt.rcParams["ps.fonttype"] = 42

PHASE_COLORS = {
    -2: "#4F81BD",
    0: "#D9D9D9",
    2: "#CF5A47",
}

@dataclass
class Step16Config:
    output_dir: Path = Path("outputs_tts_step16M_mass_coordinate_phase_maps")

    # Local windows are used only to select the already-certified data.
    left_r3_min: float = -0.665
    left_r3_max: float = -0.525
    left_r4_min: float = -0.180
    left_r4_max: float = -0.135

    right_r3_min: float = 0.054
    right_r3_max: float = 0.066
    right_r4_min: float = 0.128
    right_r4_max: float = 0.134

    # Certified left local-mass formula:
    #  $M_L = r_4 + a_L r_3^2 + b_L r_3 + c_L$ 
    left_a: float = 0.4489248315
    left_b: float = 0.7973688821
    left_c: float = 0.4712000815

    # Certified right four-valley mass:
    #  $M_{4v} = r_3 - \alpha_{4v} r_4$ 
    alpha_4v: float = 0.4597785904

    # Certified closure anchors. If Step12M is provided, these are replaced by
    # the values read from step12M_06_critical_closure_parameters.csv.
    closure_A_r3: float = 0.0600000000
    closure_A_r4: float = 0.1304975940
    closure_B_r3: float = 0.0600000000
    closure_B_r4: float = 0.1316899010

    # Plot limits in normalized mass coordinates. None means determine from data.
    left_mass_padding: float = 1.15
    right_mass_padding: float = 1.15
    figure_size_single: tuple[float, float] = (7.6, 7.6)
    figure_size_combined: tuple[float, float] = (14.6, 6.8)
    point_size: float = 58.0

    # -----
    # ZIP-stream readers: no extraction, avoids Windows WinError 206.
    # -----
    def _find_member_by_basename(zf: zipfile.ZipFile, filename: str) -> str:
        matches = [name for name in zf.namelist() if Path(name).name == filename]
        if not matches:
            raise FileNotFoundError(f"{filename} was not found inside the ZIP archive")
        matches.sort(key=lambda x: (len(Path(x).parts), len(x), x))
        return matches[0]

```

```

def read_csv_any(source: str | Path, filename: str) -> pd.DataFrame:
    source = Path(source)
    if source.is_dir():
        matches = list(source.rglob(filename))
        if not matches:
            raise FileNotFoundError(f"{filename} was not found under {source}")
        matches.sort(key=lambda p: (len(p.parts), str(p)))
        return pd.read_csv(matches[0], low_memory=False)
    if source.is_file() and source.suffix.lower() == ".csv":
        return pd.read_csv(source, low_memory=False)
    if source.is_file() and source.suffix.lower() == ".zip":
        with zipfile.ZipFile(source) as zf:
            member = _find_member_by_basename(zf, filename)
            return pd.read_csv(io.BytesIO(zf.read(member)), low_memory=False)
    raise FileNotFoundError(f"Unsupported or missing source: {source}")

def optional_csv(source: str | Path | None, filename: str) -> Optional[pd.DataFrame]:
    if source is None:
        return None
    path = Path(source)
    if not path.exists():
        return None
    try:
        return read_csv_any(path, filename)
    except FileNotFoundError:
        return None

# -----
# Basic helpers
# -----
def in_window(df: pd.DataFrame, x0: float, x1: float, y0: float, y1: float) -> pd.Series:
    return (
        df["r3"].between(x0, x1, inclusive="both")
        & df["r4"].between(y0, y1, inclusive="both")
    )

def robust_scale(values: np.ndarray, quantile: float = 0.95) -> float:
    values = np.asarray(values, dtype=float)
    scale = float(np.quantile(np.abs(values[np.isfinite(values)]), quantile))
    if not np.isfinite(scale) or scale <= 1e-14:
        scale = float(np.std(values))
    if not np.isfinite(scale) or scale <= 1e-14:
        scale = 1.0
    return scale

def style_axis(ax, title: str):
    ax.set_box_aspect(1)
    ax.tick_params(direction="in", length=7, width=1.7, top=True, right=True, labelsz=15)
    for spine in ax.spines.values():
        spine.set_linewidth(1.7)
    ax.set_title(title, fontsize=21, fontweight="bold", pad=12)

def phase_handles(include_zero: bool = True):
    handles = [
        Patch(facecolor=PHASE_COLORS[-2], edgecolor="black", linewidth=0.8,
              label=r"$\mathbf{\hat{C}}_{\uparrow}=-2$"),
        Patch(facecolor=PHASE_COLORS[2], edgecolor="black", linewidth=0.8,
              label=r"$\mathbf{\hat{C}}_{\uparrow}=2$"),
    ]
    if include_zero:
        handles.insert(1, Patch(facecolor=PHASE_COLORS[0], edgecolor="black", linewidth=0.8,
                                label=r"$\mathbf{\hat{C}}_{\uparrow}=0$"))

```

```

    return handles

# -----
# Left mass coordinates
# -----
def left_boundary_r4(r3: np.ndarray, cfg: Step16Config) -> np.ndarray:
    return -(cfg.left_a * r3**2 + cfg.left_b * r3 + cfg.left_c)

def left_boundary_slope(r3: np.ndarray, cfg: Step16Config) -> np.ndarray:
    return -(2.0 * cfg.left_a * r3 + cfg.left_b)

def left_arc_coordinate(r3_values: np.ndarray, cfg: Step16Config) -> np.ndarray:
    """Signed arc length along the fitted left boundary, referenced to its center."""
    r3_values = np.asarray(r3_values, dtype=float)
    lo = min(float(np.min(r3_values)), cfg.left_r3_min)
    hi = max(float(np.max(r3_values)), cfg.left_r3_max)
    grid = np.linspace(lo, hi, 10001)
    slope = left_boundary_slope(grid, cfg)
    integrand = np.sqrt(1.0 + slope**2)
    dg = np.diff(grid)
    cumulative = np.concatenate([[0.0], np.cumsum(0.5 * (integrand[1:] + integrand[:-1]) * dg)])
    ref_r3 = 0.5 * (cfg.left_r3_min + cfg.left_r3_max)
    ref_s = float(np.interp(ref_r3, grid, cumulative))
    return np.interp(r3_values, grid, cumulative) - ref_s

def transform_left(df: pd.DataFrame, cfg: Step16Config) -> tuple[pd.DataFrame, dict]:
    left = df[
        in_window(df, cfg.left_r3_min, cfg.left_r3_max, cfg.left_r4_min, cfg.left_r4_max)
        & df["strict_phase_code"].isin([-2, 0, 2])
    ].copy()
    if left.empty:
        raise RuntimeError("No certified points were found in the left local window")

    slope = left_boundary_slope(left["r3"].to_numpy(), cfg)
    raw_mass = (
        left["r4"].to_numpy()
        + cfg.left_a * left["r3"].to_numpy() ** 2
        + cfg.left_b * left["r3"].to_numpy()
        + cfg.left_c
    )
    # Geometric signed normal distance to M_L=0.
    normal_mass = raw_mass / np.sqrt(1.0 + slope**2)
    tangent = left_arc_coordinate(left["r3"].to_numpy(), cfg)

    mass_scale = robust_scale(normal_mass)
    tangent_scale = robust_scale(tangent)
    left["M_L_raw"] = raw_mass
    left["M_L_normal"] = normal_mass
    left["S_L_arc"] = tangent
    left["M_L_tilde"] = normal_mass / mass_scale
    left["S_L_tilde"] = tangent / tangent_scale

    nonzero = left[left["strict_phase_code"].isin([-2, 2])]
    sign_pred = np.where(nonzero["M_L_normal"].to_numpy() > 0, 2, -2)
    sign_accuracy = float(np.mean(sign_pred == nonzero["strict_phase_code"].to_numpy()))

    meta = {
        "formula": (
            f"M_L = r4 + ({cfg.left_a:.10g}) r3^2 + "
            f"({cfg.left_b:.10g}) r3 + ({cfg.left_c:.10g})"
        ),
        "normal_mass_definition": "M_L_normal = M_L / sqrt(1 + (dr4_boundary/dr3)^2)",
        "tangent_coordinate_definition": "S_L is signed arc length along M_L=0",
        "mass_scale": mass_scale,
        "tangent_scale": tangent_scale,
    }

```

```

        "nonzero_phase_sign_accuracy": sign_accuracy,
        "n_points": int(len(left)),
        "n_minus2": int((left["strict_phase_code"] == -2).sum()),
        "n_zero": int((left["strict_phase_code"] == 0).sum()),
        "n_plus2": int((left["strict_phase_code"] == 2).sum()),
    }
    return left, meta

# -----
# Right double-mass coordinates
# -----
def detect_plus2_zero_transition_midpoints(right: pd.DataFrame) -> pd.DataFrame:
    rows = []
    for r3, group in right.groupby("r3"):
        group = group.sort_values("r4")
        arr = group[["r4", "strict_phase_code"]].to_numpy()
        candidates = []
        for (y1, c1), (y2, c2) in zip(arr[:-1], arr[1:]):
            if {int(c1), int(c2)} == {0, 2}:
                candidates.append(0.5 * (float(y1) + float(y2)))
        if candidates:
            # The right-junction data contain one relevant +2/0 transition per r3.
            rows.append({"r3": float(r3), "r4_mid": float(np.median(candidates))})
    return pd.DataFrame(rows)

def fit_anchored_sigma_boundary(
    transition_points: pd.DataFrame,
    anchor_r3: float,
    anchor_r4: float,
) -> dict:
    """Fit an anchored local boundary  $r_4 = \text{anchor\_r4} + \beta (r_3 - \text{anchor\_r3})$ .

    A linear anchored fit is deliberately used because it is the simplest local
    mass formula and, for the current certified junction data, classifies all
    strict -2/0/+2 points consistently when paired with M_4v.
    """
    if len(transition_points) < 2:
        raise RuntimeError("Too few +2/0 transition midpoints to fit M_Sigma")
    x = transition_points["r3"].to_numpy(dtype=float) - anchor_r3
    y = transition_points["r4_mid"].to_numpy(dtype=float) - anchor_r4
    denom = float(np.dot(x, x))
    if denom <= 1e-20:
        raise RuntimeError("Degenerate transition-point geometry")
    beta = float(np.dot(x, y) / denom)
    predicted = anchor_r4 + beta * (transition_points["r3"].to_numpy() - anchor_r3)
    residual = transition_points["r4_mid"].to_numpy() - predicted
    return {
        "beta": beta,
        "rmse": float(np.sqrt(np.mean(residual**2))),
        "max_abs_residual": float(np.max(np.abs(residual))),
        "n_boundary_midpoints": int(len(transition_points)),
    }

def transform_right(df: pd.DataFrame, cfg: Step16Config) -> tuple[pd.DataFrame, pd.DataFrame, dict]:
    right = df[
        in_window(df, cfg.right_r3_min, cfg.right_r3_max, cfg.right_r4_min, cfg.right_r4_max)
        & df["strict_phase_code"].isin([-2, 0, 2])
    ].copy()
    if right.empty:
        raise RuntimeError("No certified points were found in the right junction window")

    transitions = detect_plus2_zero_transition_midpoints(right)
    sigma_fit = fit_anchored_sigma_boundary(
        transitions,
        cfg.closure_B_r3,
        cfg.closure_B_r4,
    )

```

```

)
beta = sigma_fit["beta"]

r3 = right["r3"].to_numpy(dtype=float)
r4 = right["r4"].to_numpy(dtype=float)

# Signed normal distance to the certified four-valley boundary.
m4_raw = r3 - cfg.alpha_4v * r4
m4_normal = m4_raw / math.sqrt(1.0 + cfg.alpha_4v**2)

# Signed normal distance to the locally fitted Sigma' two-valley boundary.
sigma_boundary = cfg.closure_B_r4 + beta * (r3 - cfg.closure_B_r3)
ms_raw = r4 - sigma_boundary
ms_normal = ms_raw / math.sqrt(1.0 + beta**2)

m4_scale = robust_scale(m4_normal)
ms_scale = robust_scale(ms_normal)

right["M_4v_raw"] = m4_raw
right["M_4v_normal"] = m4_normal
right["M_Sigma_raw"] = ms_raw
right["M_Sigma_normal"] = ms_normal
right["M_4v_tilde"] = m4_normal / m4_scale
right["M_Sigma_tilde"] = ms_normal / ms_scale

# Mechanism-sector rule inferred from the ordered -2 -> +2 -> 0 path:
#   M_Sigma > 0           -> C_up = 0
#   M_Sigma < 0 and M_4v > 0 -> C_up = -2
#   M_Sigma < 0 and M_4v < 0 -> C_up = +2
predicted = np.where(
    right["M_Sigma_normal"].to_numpy() > 0,
    0,
    np.where(right["M_4v_normal"].to_numpy() > 0, -2, 2),
)
right["mass_sector_prediction"] = predicted
right["mass_sector_match"] = predicted == right["strict_phase_code"].to_numpy()

labels = [-2, 0, 2]
recall = {}
for label in labels:
    mask = right["strict_phase_code"].to_numpy() == label
    recall[str(label)] = float(np.mean(predicted[mask] == label)) if np.any(mask) else None
balanced_accuracy = float(np.mean([v for v in recall.values() if v is not None]))
accuracy = float(np.mean(right["mass_sector_match"]))

meta = {
    "M_4v_formula": (
        f"M_4v = r3 - ({cfg.alpha_4v:.10g}) r4"
    ),
    "M_Sigma_formula": (
        f"M_Sigma = r4 - [{cfg.closure_B_r4:.10g} + "
        f"({beta:.10g})(r3 - {cfg.closure_B_r3:.10g})]"
    ),
    "M_4v_normal_definition": "M_4v / sqrt(1 + alpha_4v^2)",
    "M_Sigma_normal_definition": "M_Sigma / sqrt(1 + beta^2)",
    "sigma_boundary_fit": sigma_fit,
    "M_4v_scale": m4_scale,
    "M_Sigma_scale": ms_scale,
    "sector_accuracy": accuracy,
    "sector_balanced_accuracy": balanced_accuracy,
    "per_class_recall": recall,
    "n_points": int(len(right)),
    "n_minus2": int((right["strict_phase_code"] == -2).sum()),
    "n_zero": int((right["strict_phase_code"] == 0).sum()),
    "n_plus2": int((right["strict_phase_code"] == 2).sum()),
    "interpretation": {
        "M_Sigma_positive": "C_up = 0",
        "M_Sigma_negative_and_M_4v_positive": "C_up = -2",
        "M_Sigma_negative_and_M_4v_negative": "C_up = +2",
    }
}

```

```

    },
}
return right, transitions, meta

# -----
# Plotting
# -----
def scatter_phases(ax, data: pd.DataFrame, xcol: str, ycol: str, size: float):
    for phase in [-2, 0, 2]:
        sub = data[data["strict_phase_code"] == phase]
        if sub.empty:
            continue
        ax.scatter(
            sub[xcol], sub[ycol],
            s=size,
            facecolors="white",
            edgecolors=PHASE_COLORS[phase],
            linewidths=1.5,
            zorder=5,
        )

def plot_left(left: pd.DataFrame, meta: dict, cfg: Step16Config, output_dir: Path):
    x = left["M_L_tilde"].to_numpy()
    y = left["S_L_tilde"].to_numpy()
    xlim = (-cfg.left_mass_padding * max(abs(x.min()), abs(x.max())),
            cfg.left_mass_padding * max(abs(x.min()), abs(x.max())))
    ylim = (-cfg.left_mass_padding * max(abs(y.min()), abs(y.max())),
            cfg.left_mass_padding * max(abs(y.min()), abs(y.max())))

    fig, ax = plt.subplots(figsize=cfg.figure_size_single)
    ax.axvspan(xlim[0], 0.0, facecolor=PHASE_COLORS[-2], alpha=0.25, zorder=0)
    ax.axvspan(0.0, xlim[1], facecolor=PHASE_COLORS[2], alpha=0.25, zorder=0)
    ax.axvline(0.0, color="black", linewidth=1.9, zorder=3)
    scatter_phases(ax, left, "M_L_tilde", "S_L_tilde", cfg.point_size)

    ax.set_xlim(*xlim)
    ax.set_ylim(*ylim)
    ax.set_xlabel(r"$\widetilde{\mathbf{\mathit{M}}}_{\mathrm{L}}$", fontsize=24, fontweight="bold")
    ax.set_ylabel(r"$\widetilde{\mathbf{\mathit{S}}}_{\mathrm{L}}$", fontsize=24, fontweight="bold")
    style_axis(ax, "Left four-valley boundary in local mass coordinates")

    handles = phase_handles(include_zero=True) + [
        Line2D([0], [0], color="black", linewidth=1.9,
              label=r"$\mathbf{\mathit{M}}_{\mathrm{L}}=0$"),
    ]
    leg = ax.legend(handles=handles, loc="upper left", fontsize=13, frameon=True, ncol=2)
    leg.get_frame().set_linewidth(1.1)
    leg.get_frame().set_edgecolor("black")

    ax.text(
        0.03, 0.025,
        f"sign-rule accuracy = {meta['nonzero_phase_sign_accuracy']:.3f}",
        transform=ax.transAxes,
        fontsize=12,
        fontweight="bold",
        bbox=dict(facecolor="white", edgecolor="black", alpha=0.85, boxstyle="round,pad=0.3"),
    )
    plt.tight_layout()
    for ext in ["png", "pdf", "svg"]:
        fig.savefig(output_dir / f"step16M_left_mass_coordinate_phase_map.{ext}",
                    dpi=600 if ext == "png" else None, bbox_inches="tight")
    plt.close(fig)

def plot_right(right: pd.DataFrame, meta: dict, cfg: Step16Config, output_dir: Path):
    x = right["M_4v_tilde"].to_numpy()
    y = right["M_Sigma_tilde"].to_numpy()

```



```

xmax = cfg.right_mass_padding * max(abs(x.min()), abs(x.max()))
ymax = cfg.right_mass_padding * max(abs(y.min()), abs(y.max()))
xlim = (-xmax, xmax)
ylim = (-ymax, ymax)

fig, ax = plt.subplots(figsize=cfg.figure_size_single)
# Clean mechanism sectors in the double-mass plane.
ax.axhspan(0.0, ylim[1], facecolor=PHASE_COLORS[0], alpha=0.45, zorder=0)
ax.fill_between([xlim[0], 0.0], ylim[0], 0.0, color=PHASE_COLORS[2], alpha=0.35, zorder=0)
ax.fill_between([0.0, xlim[1]], ylim[0], 0.0, color=PHASE_COLORS[-2], alpha=0.35, zorder=0)
ax.axvline(0.0, color="black", linewidth=1.8, zorder=3)
ax.axhline(0.0, color="black", linewidth=1.8, zorder=3)

scatter_phases(ax, right, "M_4v_tilde", "M_Sigma_tilde", cfg.point_size)
mismatch = right[~right["mass_sector_match"]]
if not mismatch.empty:
    ax.scatter(mismatch["M_4v_tilde"], mismatch["M_Sigma_tilde"], marker="x",
               s=90, color="black", linewidths=1.8, zorder=8)

ax.set_xlim(*xlim)
ax.set_ylim(*ylim)
ax.set_xlabel(r"$\widetilde{\mathbf{\hat{M}}}_{4\mathrm{v}}$", fontsize=24, fontweight="bold")
ax.set_ylabel(r"$\widetilde{\mathbf{\hat{M}}}_{\mathrm{Sigma}}$", fontsize=24, fontweight="bold")
style_axis(ax, "Right junction in double-mass coordinates")

handles = phase_handles(include_zero=True) + [
    Line2D([0], [0], color="black", linewidth=1.8,
           label=r"$\mathbf{\hat{M}}_{4\mathrm{v}}=0$ or $\mathbf{\hat{M}}_{\mathrm{Sigma}}=0$"),
]
if not mismatch.empty:
    handles.append(Line2D([0], [0], marker="x", linestyle="None", color="black",
                          markersize=9, label="mass-rule mismatch"))
leg = ax.legend(handles=handles, loc="upper right", fontsize=12.5, frameon=True, ncol=2)
leg.get_frame().set_linewidth(1.1)
leg.get_frame().set_edgecolor("black")

ax.text(
    0.03, 0.025,
    f"sector accuracy = {meta['sector_accuracy']:.3f}\n"
    f"balanced accuracy = {meta['sector_balanced_accuracy']:.3f}",
    transform=ax.transAxes,
    fontsize=12,
    fontweight="bold",
    bbox=dict(facecolor="white", edgecolor="black", alpha=0.85, boxstyle="round,pad=0.3"),
)
plt.tight_layout()
for ext in ["png", "pdf", "svg"]:
    fig.savefig(output_dir / f"step16M_right_double_mass_phase_map.{ext}",
               dpi=600 if ext == "png" else None, bbox_inches="tight")
plt.close(fig)

```

```

def plot_combined(left: pd.DataFrame, left_meta: dict, right: pd.DataFrame, right_meta: dict,
                  cfg: Step16Config, output_dir: Path):
    fig, axes = plt.subplots(1, 2, figsize=cfg.figure_size_combined)

    # Left panel
    ax = axes[0]
    x = left["M_L_tilde"].to_numpy(); y = left["S_L_tilde"].to_numpy()
    xmax = cfg.left_mass_padding * max(abs(x.min()), abs(x.max()))
    ymax = cfg.left_mass_padding * max(abs(y.min()), abs(y.max()))
    ax.axvspan(-xmax, 0.0, facecolor=PHASE_COLORS[-2], alpha=0.25)
    ax.axvspan(0.0, xmax, facecolor=PHASE_COLORS[2], alpha=0.25)
    ax.axvline(0.0, color="black", linewidth=1.8)
    scatter_phases(ax, left, "M_L_tilde", "S_L_tilde", cfg.point_size * 0.85)
    ax.set_xlim(-xmax, xmax); ax.set_ylim(-ymax, ymax)
    ax.set_xlabel(r"$\widetilde{\mathbf{\hat{M}}}_{\mathrm{L}}$", fontsize=22, fontweight="bold")
    ax.set_ylabel(r"$\widetilde{\mathbf{\hat{S}}}_{\mathrm{L}}$", fontsize=22, fontweight="bold")
    style_axis(ax, "(a) Left local mass plane")

```

```

# Right panel
ax = axes[1]
x = right["M_4v_tilde"].to_numpy(); y = right["M_Sigma_tilde"].to_numpy()
xmax = cfg.right_mass_padding * max(abs(x.min()), abs(x.max()))
ymax = cfg.right_mass_padding * max(abs(y.min()), abs(y.max()))
ax.axhspan(0.0, ymax, facecolor=PHASE_COLORS[0], alpha=0.45)
ax.fill_between([-xmax, 0.0], -ymax, 0.0, color=PHASE_COLORS[2], alpha=0.35)
ax.fill_between([0.0, xmax], -ymax, 0.0, color=PHASE_COLORS[-2], alpha=0.35)
ax.axvline(0.0, color="black", linewidth=1.8)
ax.axhline(0.0, color="black", linewidth=1.8)
scatter_phases(ax, right, "M_4v_tilde", "M_Sigma_tilde", cfg.point_size * 0.85)
mismatch = right[~right["mass_sector_match"]]
if not mismatch.empty:
    ax.scatter(mismatch["M_4v_tilde"], mismatch["M_Sigma_tilde"], marker="x",
               s=80, color="black", linewidths=1.7, zorder=8)
ax.set_xlim(-xmax, xmax); ax.set_ylim(-ymax, ymax)
ax.set_xlabel(r"$\widetilde{\mathbf{M}}_{4v}$", fontsize=22, fontweight="bold")
ax.set_ylabel(r"$\widetilde{\mathbf{M}}_{\Sigma}$", fontsize=22, fontweight="bold")
style_axis(ax, "(b) Right double-mass plane")

handles = phase_handles(include_zero=True)
fig.legend(handles=handles, loc="upper center", bbox_to_anchor=(0.5, 1.03),
           ncol=3, fontsize=14, frameon=True)
plt.tight_layout(rect=[0, 0, 1, 0.96])
for ext in ["png", "pdf", "svg"]:
    fig.savefig(output_dir / f"step16M_combined_mass_coordinate_phase_maps.{ext}",
               dpi=600 if ext == "png" else None, bbox_inches="tight")
plt.close(fig)

# -----
# Main
# -----
def run_step16M(
    phase_source: str | Path = "outputs_tts_step15C_repair_and_redraw_phase_maps.zip",
    step11_source: str | Path | None = "outputs_tts_step11M_lieb_aligned_fixed_slice.zip",
    step12_source: str | Path | None = "outputs_tts_step12M_junction_multiclosure_repair.zip",
    output_dir: str | Path = "outputs_tts_step16M_mass_coordinate_phase_maps",
    config: Step16Config | None = None,
):
    if config is None:
        config = Step16Config(output_dir=Path(output_dir))
    config.output_dir = Path(output_dir)
    config.output_dir.mkdir(parents=True, exist_ok=True)

    phase = read_csv_any(phase_source, "step15C_00_final_phase_points_repaired.csv")

    # Use exact closure coordinates from Step12M when available.
    closure = optional_csv(step12_source, "step12M_06_critical_closure_parameters.csv")
    if closure is not None and len(closure) >= 2:
        closure = closure.sort_values("critical_lambda").reset_index(drop=True)
        config.closure_A_r3 = float(closure.loc[0, "r3"])
        config.closure_A_r4 = float(closure.loc[0, "r4"])
        config.closure_B_r3 = float(closure.loc[1, "r3"])
        config.closure_B_r4 = float(closure.loc[1, "r4"])

    left, left_meta = transform_left(phase, config)
    right, transition_points, right_meta = transform_right(phase, config)

    # Optional certified points from Step11M are transformed and stored for audit.
    step11 = optional_csv(step11_source, "step11M_07_completed_certified_boundary_points.csv")
    if step11 is not None:
        left_cert = step11[step11["branch_hint"].eq("generic_left_lower")].copy()
        if not left_cert.empty:
            slope = left_boundary_slope(left_cert["r3"].to_numpy(), config)
            mass = (
                left_cert["r4"].to_numpy()
                + config.left_a * left_cert["r3"].to_numpy() ** 2

```

```

        + config.left_b * left_cert["r3"].to_numpy()
        + config.left_c
    ) / np.sqrt(1.0 + slope**2)
    tangent = left_arc_coordinate(left_cert["r3"].to_numpy(), config)
    left_cert["M_L_normal"] = mass
    left_cert["S_L_arc"] = tangent
    left_cert.to_csv(config.output_dir / "step16M_03_left_certified_boundary_points_mass_coordinates.csv",
        ↪ index=False)

transformed = pd.concat([
    left.assign(mass_map_region="left_local"),
    right.assign(mass_map_region="right_junction"),
], ignore_index=True, sort=False)
transformed.to_csv(config.output_dir / "step16M_00_transformed_mass_coordinate_points.csv", index=False)
transition_points.to_csv(config.output_dir / "step16M_01_sigma_transition_midpoints.csv", index=False)

plot_left(left, left_meta, config, config.output_dir)
plot_right(right, right_meta, config, config.output_dir)
plot_combined(left, left_meta, right, right_meta, config, config.output_dir)

report = {
    "code_version": CODE_VERSION,
    "configuration": asdict(config),
    "left_mass_coordinates": left_meta,
    "right_double_mass_coordinates": right_meta,
    "evidence_scope": {
        "left": "Local formula valid only near the certified left four-valley branch.",
        "right": (
            "M_4v is the certified local four-valley mass. M_Sigma is an exploratory "
            "local mass coordinate fitted from the certified +2/0 boundary and anchored "
            "at the Sigma' two-valley closure. It is not yet a globally derived k.p mass."
        ),
        "global_warning": (
            "These local mass coordinates must not be used as a global reparameterization "
            "of the entire fixed-background r3-r4 phase map."
        ),
    },
    "generated_figures": [
        "step16M_left_mass_coordinate_phase_map.pdf",
        "step16M_right_double_mass_phase_map.pdf",
        "step16M_combined_mass_coordinate_phase_maps.pdf",
    ],
}
with open(config.output_dir / "step16M_02_mass_coordinate_report.json", "w", encoding="utf-8") as f:
    json.dump(report, f, ensure_ascii=False, indent=2, default=str)

md = rf"""# TTS Step16M mass-coordinate phase maps

## Left local coordinates

\[
M_{\{\mathrm{L}\}}=r_4+\{config.left_a:.10g\}r_3^2+\{config.left_b:.10g\}r_3+\{config.left_c:.10g\}.
\]

The plotted horizontal coordinate is the normalized signed normal distance
 $\widetilde{M}_{\{\mathrm{L}\}}$ ; the vertical coordinate
 $\widetilde{S}_{\{\mathrm{L}\}}$  is normalized arc length along  $(M_{\{\mathrm{L}\}}=0)$ .
The sign rule reproduces the certified  $(C_{\uparrow}=\pm 2)$  labels with accuracy
 $\{left\_meta['nonzero\_phase\_sign\_accuracy']:.6f\}$ %.

## Right double-mass coordinates

\[
M_{\{4v\}}=r_3-\{config.alpha_{4v}:.10g\}r_4,
\]

\[
M_{\{\Sigma'\}}=r_4-\left[\{config.closure\_B\_r4:.10g\}
+\{right\_meta['sigma\_boundary\_fit']['beta']:.10g\}(r_3-\{config.closure\_B\_r3:.10g\})\right].

```

\]

The mechanism-sector rule gives accuracy `**{right_meta['sector_accuracy']:.6f}**` and balanced accuracy `**{right_meta['sector_balanced_accuracy']:.6f}**`.

Scope

The left and right coordinates are `**local mechanism coordinates**`, not a global coordinate transformation. In particular, $\backslash(M_{\{\backslash\Sigma'\}}\backslash)$ is fitted from the certified local $\backslash(+2/0\backslash)$ boundary and anchored to the certified two-valley closure. It should be described as an exploratory effective mass coordinate until an independent low-energy $\backslash(k\backslash\cdot p\backslash)$ derivation is completed.

```
"""
    (config.output_dir / "step16M_02_mass_coordinate_report.md").write_text(md, encoding="utf-8")

print(json.dumps({
    "code_version": CODE_VERSION,
    "output_dir": str(config.output_dir),
    "left_sign_accuracy": left_meta["nonzero_phase_sign_accuracy"],
    "right_sector_accuracy": right_meta["sector_accuracy"],
    "right_sector_balanced_accuracy": right_meta["sector_balanced_accuracy"],
    "M_Sigma_beta": right_meta["sigma_boundary_fit"]["beta"],
}, ensure_ascii=False, indent=2))
return report

if __name__ == "__main__":
    run_step16M()
```

► 代码清单 A.50.8 (python, 819 行)

```
from __future__ import annotations

import importlib.util
import io
import json
import math
import sys
import time
import zipfile
from dataclasses import asdict, dataclass
from pathlib import Path
from typing import Any

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from matplotlib.lines import Line2D
from matplotlib.patches import Patch

CODE_VERSION = "TTS_STEP17M_RANDOM_MASS_COORDINATE_VALIDATION_V1_20260727"

PHASE_COLORS = {
    -2: "#4F81BD",
    0: "#D9D9D9",
    2: "#CF5A47",
    99: "#666666",
}

@dataclass
class Step17Config:
    output_dir: Path = Path("outputs_tts_step17M_random_mass_coordinate_validation")
```

```

random_seed: int = 20260727

# Random interior validation counts.
n_left_minus2: int = 15
n_left_plus2: int = 15
n_right_minus2: int = 15
n_right_plus2: int = 15
n_right_zero: int = 15

# Boundary stress-test points. These are not included in interior accuracy.
n_left_boundary: int = 6
n_right_four_valley_boundary: int = 6
n_right_sigma_boundary: int = 6

# Safety margins in normalized mass coordinates. Interior points are sampled
# away from the zero-mass boundaries; boundary points are sampled inside a
# narrow strip around zero.
interior_mass_margin: float = 0.18
boundary_half_width: float = 0.055

# Sampling domains in normalized mass coordinates.
left_abs_mass_max: float = 1.05
left_s_min: float = -0.85
left_s_max: float = 0.90

right_abs_m4_max: float = 0.95
right_msigma_min: float = -1.05
right_msigma_max: float = 0.90

# Valid raw-parameter windows. The left r3 interval is restricted to the
# certified local-mass range rather than the entire plotting window.
left_r3_min: float = -0.643364
left_r3_max: float = -0.542527
left_r4_min: float = -0.180
left_r4_max: float = -0.135

right_r3_min: float = 0.056
right_r3_max: float = 0.064
right_r4_min: float = 0.128
right_r4_max: float = 0.134

# Reject samples too close to an existing regular-grid coordinate. This
# ensures that the validation points are genuinely new parameter points.
min_distance_from_existing_raw: float = 2.0e-5

# Rejection sampler controls.
maximum_draws_per_region: int = 200000

# Physics settings. quick=True is only for pipeline testing and must not be
# used for paper conclusions.
quick: bool = False
force_recalculate: bool = False

# Plot settings.
point_size: float = 62.0
figure_size_combined: tuple[float, float] = (14.8, 6.8)

def normalized(self) -> "Step17Config":
    self.output_dir = Path(self.output_dir).expanduser().resolve()
    self.output_dir.mkdir(parents=True, exist_ok=True)
    for name in ["checkpoints", "wilson_traces", "figures", "_physics_source"]:
        (self.output_dir / name).mkdir(exist_ok=True)
    return self

# =====
# ZIP-stream readers: no full extraction of result archives.
# =====

```

```

def _find_member_by_basename(zf: zipfile.ZipFile, filename: str) -> str:
    matches = [name for name in zf.namelist() if Path(name).name == filename]
    if not matches:
        raise FileNotFoundError(f"{filename} was not found inside {zf.filename}")
    matches.sort(key=lambda name: (len(Path(name).parts), len(name), name))
    return matches[0]

def read_csv_any(source: str | Path, filename: str) -> pd.DataFrame:
    source = Path(source)
    if source.is_dir():
        matches = sorted(source.rglob(filename), key=lambda p: (len(p.parts), str(p)))
        if not matches:
            raise FileNotFoundError(f"{filename} was not found under {source}")
        return pd.read_csv(matches[0], low_memory=False)
    if source.is_file() and source.suffix.lower() == ".csv":
        return pd.read_csv(source, low_memory=False)
    if source.is_file() and source.suffix.lower() == ".zip":
        with zipfile.ZipFile(source) as zf:
            member = _find_member_by_basename(zf, filename)
            return pd.read_csv(io.BytesIO(zf.read(member)), low_memory=False)
    raise FileNotFoundError(f"Unsupported or missing source: {source}")

def read_json_any(source: str | Path, filename: str) -> dict[str, Any]:
    source = Path(source)
    if source.is_dir():
        matches = sorted(source.rglob(filename), key=lambda p: (len(p.parts), str(p)))
        if not matches:
            raise FileNotFoundError(f"{filename} was not found under {source}")
        return json.loads(matches[0].read_text(encoding="utf-8"))
    if source.is_file() and source.suffix.lower() == ".json":
        return json.loads(source.read_text(encoding="utf-8"))
    if source.is_file() and source.suffix.lower() == ".zip":
        with zipfile.ZipFile(source) as zf:
            member = _find_member_by_basename(zf, filename)
            return json.loads(zf.read(member).decode("utf-8"))
    raise FileNotFoundError(f"Unsupported or missing source: {source}")

# =====
# Dynamic physics-module import and TTS source resolution.
# =====

def import_module_from_path(module_name: str, path: str | Path):
    path = Path(path).expanduser().resolve()
    if not path.is_file():
        raise FileNotFoundError(path)
    spec = importlib.util.spec_from_file_location(module_name, path)
    if spec is None or spec.loader is None:
        raise ImportError(f"Could not create import spec for {path}")
    module = importlib.util.module_from_spec(spec)
    # Required for dataclasses under importlib.exec_module.
    sys.modules[module_name] = module
    spec.loader.exec_module(module)
    return module

def resolve_step15_module(source: str | Path) -> Path:
    source = Path(source).expanduser()
    if source.is_file() and source.suffix.lower() == ".py":
        return source.resolve()
    if source.is_dir():
        matches = sorted(source.rglob("TTS_step15M_reclassify_all_boundary_points.py"))
        if matches:
            return matches[0].resolve()
    if source.is_file() and source.suffix.lower() == ".zip":

```

```

        # Extract only the single Python module, not the whole ZIP.
        with zipfile.ZipFile(source) as zf:
            member = _find_member_by_basename(zf, "TTS_step15M_reclassify_all_boundary_points.py")
            target = Path.cwd() / "TTS_step15M_reclassify_all_boundary_points.py"
            if not target.is_file():
                target.write_bytes(zf.read(member))
            return target.resolve()
        raise FileNotFoundError(f"Could not resolve Step15M support module from {source}")

def _valid_tts_source(path: Path) -> bool:
    if not path.is_file() or path.suffix.lower() != ".zip":
        return False
    try:
        with zipfile.ZipFile(path) as zf:
            names = zf.namelist()
    except zipfile.BadZipFile:
        return False
    return any(Path(n).name == "TTS_step01_model_and_label_audit_v2.py" for n in names) or any(
        Path(n).name.lower() in {"tts(1).zip", "tts.zip"} for n in names
    )

def resolve_tts_archive(requested: str | Path) -> Path:
    requested = Path(requested).expanduser()
    candidates: list[Path] = []
    if requested.is_file():
        candidates.append(requested.resolve())
    if not requested.is_absolute() and (Path.cwd() / requested).is_file():
        candidates.append((Path.cwd() / requested).resolve())
    for name in ["tts(1).zip", "tts.zip", "TTS_Step08M_Mechanism_Aware_ML.zip"]:
        candidate = Path.cwd() / name
        if candidate.is_file() and candidate.resolve() not in candidates:
            candidates.append(candidate.resolve())
    for candidate in candidates:
        if _valid_tts_source(candidate):
            print(f"[input] frozen TTS model source: {candidate}")
            return candidate
    raise FileNotFoundError(
        "No valid TTS model source was found. Put tts(1).zip or "
        "TTS_Step08M_Mechanism_Aware_ML.zip beside this script."
    )

# =====
# Mass-coordinate transformations and inverse maps.
# =====

def left_boundary_slope(r3: np.ndarray, a: float, b: float) -> np.ndarray:
    return -(2.0 * a * r3 + b)

def build_left_arc_map(r3_lo: float, r3_hi: float, ref_lo: float, ref_hi: float,
                      a: float, b: float, n: int = 30001) -> tuple[np.ndarray, np.ndarray]:
    grid = np.linspace(min(r3_lo, ref_lo), max(r3_hi, ref_hi), n)
    slope = left_boundary_slope(grid, a, b)
    integrand = np.sqrt(1.0 + slope**2)
    dg = np.diff(grid)
    cumulative = np.concatenate([[0.0], np.cumsum(0.5 * (integrand[1:] + integrand[:-1]) * dg)])
    ref_r3 = 0.5 * (ref_lo + ref_hi)
    ref_s = float(np.interp(ref_r3, grid, cumulative))
    return grid, cumulative - ref_s

def inverse_left_mass_coordinates(m_tilde: float, s_tilde: float, *, mass_scale: float,
                                tangent_scale: float, a: float, b: float, c: float,
                                arc_r3: np.ndarray, arc_s: np.ndarray) -> tuple[float, float]:
    target_s = float(s_tilde * mass_scale)
    target_r3 = float(arc_r3[np.interp(target_s, arc_s, arc_r3)])
    return target_r3, target_s

```

```

    r3 = float(np.interp(target_s, arc_s, arc_r3))
    slope = float(left_boundary_slope(np.array([r3]), a, b)[0])
    normal_mass = float(m_tilde * mass_scale)
    raw_mass = normal_mass * math.sqrt(1.0 + slope**2)
    r4 = raw_mass - a * r3**2 - b * r3 - c
    return r3, float(r4)

def forward_left_mass_coordinates(r3: float, r4: float, *, mass_scale: float,
                                tangent_scale: float, a: float, b: float, c: float,
                                arc_r3: np.ndarray, arc_s: np.ndarray) -> tuple[float, float]:
    slope = -(2.0 * a * r3 + b)
    raw_mass = r4 + a * r3**2 + b * r3 + c
    normal_mass = raw_mass / math.sqrt(1.0 + slope**2)
    s_arc = float(np.interp(r3, arc_r3, arc_s))
    return float(normal_mass / mass_scale), float(s_arc / tangent_scale)

def inverse_right_mass_coordinates(m4_tilde: float, ms_tilde: float, *,
                                  m4_scale: float, ms_scale: float,
                                  alpha: float, beta: float,
                                  anchor_r3: float, anchor_r4: float) -> tuple[float, float]:
    m4_raw = float(m4_tilde * m4_scale * math.sqrt(1.0 + alpha**2))
    ms_raw = float(ms_tilde * ms_scale * math.sqrt(1.0 + beta**2))
    constant = anchor_r4 - beta * anchor_r3
    denominator = 1.0 - alpha * beta
    if abs(denominator) < 1.0e-8:
        raise RuntimeError("The two right-junction mass coordinates are nearly linearly dependent")
    r3 = (m4_raw + alpha * (ms_raw + constant)) / denominator
    r4 = ms_raw + beta * r3 + constant
    return float(r3), float(r4)

def forward_right_mass_coordinates(r3: float, r4: float, *,
                                  m4_scale: float, ms_scale: float,
                                  alpha: float, beta: float,
                                  anchor_r3: float, anchor_r4: float) -> tuple[float, float]:
    m4_raw = r3 - alpha * r4
    ms_raw = r4 - (anchor_r4 + beta * (r3 - anchor_r3))
    m4_normal = m4_raw / math.sqrt(1.0 + alpha**2)
    ms_normal = ms_raw / math.sqrt(1.0 + beta**2)
    return float(m4_normal / m4_scale), float(ms_normal / ms_scale)

# =====
# Random design generation.
# =====

def _far_from_existing(r3: float, r4: float, existing: np.ndarray, minimum_distance: float) -> bool:
    if existing.size == 0:
        return True
    distance = np.sqrt((existing[:, 0] - r3) ** 2 + (existing[:, 1] - r4) ** 2)
    return bool(np.min(distance) >= minimum_distance)

def _accept_unique(r3: float, r4: float, accepted: list[tuple[float, float]], threshold: float = 1.0e-7) -> bool:
    return all(math.hypot(r3 - x, r4 - y) >= threshold for x, y in accepted)

def generate_random_design(cfg: Step17Config, step16_report: dict[str, Any],
                           existing_points: pd.DataFrame) -> pd.DataFrame:
    rng = np.random.default_rng(cfg.random_seed)
    conf = step16_report["configuration"]
    left_meta = step16_report["left_mass_coordinates"]
    right_meta = step16_report["right_double_mass_coordinates"]

    a = float(conf["left_a"]); b = float(conf["left_b"]); c = float(conf["left_c"])
    alpha = float(conf["alpha_4v"])

```

```

beta = float(right_meta["sigma_boundary_fit"]["beta"])
anchor_r3 = float(conf["closure_B_r3"]); anchor_r4 = float(conf["closure_B_r4"])
left_mass_scale = float(left_meta["mass_scale"])
left_tangent_scale = float(left_meta["tangent_scale"])
m4_scale = float(right_meta["M_4v_scale"])
ms_scale = float(right_meta["M_Sigma_scale"])

arc_r3, arc_s = build_left_arc_map(
    cfg.left_r3_min, cfg.left_r3_max,
    float(conf["left_r3_min"]), float(conf["left_r3_max"]), a, b,
)
# Limit the sampled S range to the certified r3 interval after conversion.
s_cert_lo = float(np.interp(cfg.left_r3_min, arc_r3, arc_s) / left_tangent_scale)
s_cert_hi = float(np.interp(cfg.left_r3_max, arc_r3, arc_s) / left_tangent_scale)
left_s_lo = max(cfg.left_s_min, min(s_cert_lo, s_cert_hi))
left_s_hi = min(cfg.left_s_max, max(s_cert_lo, s_cert_hi))
if left_s_lo >= left_s_hi:
    raise RuntimeError("No overlap between the requested left S range and the certified r3 range")

existing = existing_points[["r3", "r4"]].dropna().to_numpy(dtype=float)
accepted_raw: list[tuple[float, float]] = []
rows: list[dict[str, Any]] = []

def add_left(region: str, count: int, m_lo: float, m_hi: float, expected: int | None):
    found = 0
    for _ in range(cfg.maximum_draws_per_region):
        if found >= count:
            break
        mt = float(rng.uniform(m_lo, m_hi))
        st = float(rng.uniform(left_s_lo, left_s_hi))
        r3, r4 = inverse_left_mass_coordinates(
            mt, st, mass_scale=left_mass_scale, tangent_scale=left_tangent_scale,
            a=a, b=b, c=c, arc_r3=arc_r3, arc_s=arc_s,
        )
        if not (cfg.left_r3_min <= r3 <= cfg.left_r3_max and cfg.left_r4_min <= r4 <= cfg.left_r4_max):
            continue
        if not _far_from_existing(r3, r4, existing, cfg.min_distance_from_existing_raw):
            continue
        if not _accept_unique(r3, r4, accepted_raw):
            continue
        mt_check, st_check = forward_left_mass_coordinates(
            r3, r4, mass_scale=left_mass_scale, tangent_scale=left_tangent_scale,
            a=a, b=b, c=c, arc_r3=arc_r3, arc_s=arc_s,
        )
        sample_id = f"L_{region}_{found+1:03d}"
        rows.append({
            "sample_id": sample_id, "mass_map_region": "left_local",
            "validation_region": region, "expected_phase_code": expected,
            "M_L_tilde_target": mt, "S_L_tilde_target": st,
            "M_L_tilde": mt_check, "S_L_tilde": st_check,
            "M_4v_tilde": np.nan, "M_Sigma_tilde": np.nan,
            "r3": r3, "r4": r4,
            "source_type": "step17_random_inverse_mass_sample",
            "source_region": "left_local",
            "is_boundary_stress": int(expected is None),
        })
        accepted_raw.append((r3, r4)); found += 1
    if found != count:
        raise RuntimeError(f"Generated only {found}/{count} samples for {region}")

def add_right(region: str, count: int,
              m4_lo: float, m4_hi: float, ms_lo: float, ms_hi: float,
              expected: int | None):
    found = 0
    for _ in range(cfg.maximum_draws_per_region):
        if found >= count:
            break
        m4t = float(rng.uniform(m4_lo, m4_hi))
        mst = float(rng.uniform(ms_lo, ms_hi))

```

```

r3, r4 = inverse_right_mass_coordinates(
    m4t, mst, m4_scale=m4_scale, ms_scale=ms_scale,
    alpha=alpha, beta=beta, anchor_r3=anchor_r3, anchor_r4=anchor_r4,
)
if not (cfg.right_r3_min <= r3 <= cfg.right_r3_max and cfg.right_r4_min <= r4 <= cfg.right_r4_max):
    continue
if not _far_from_existing(r3, r4, existing, cfg.min_distance_from_existing_raw):
    continue
if not _accept_unique(r3, r4, accepted_raw):
    continue
m4_check, ms_check = forward_right_mass_coordinates(
    r3, r4, m4_scale=m4_scale, ms_scale=ms_scale,
    alpha=alpha, beta=beta, anchor_r3=anchor_r3, anchor_r4=anchor_r4,
)
sample_id = f"R_{region}_{found+1:03d}"
rows.append({
    "sample_id": sample_id, "mass_map_region": "right_junction",
    "validation_region": region, "expected_phase_code": expected,
    "M_L_tilde_target": np.nan, "S_L_tilde_target": np.nan,
    "M_L_tilde": np.nan, "S_L_tilde": np.nan,
    "M_4v_tilde": m4_check, "M_Sigma_tilde": ms_check,
    "r3": r3, "r4": r4,
    "source_type": "step17_random_inverse_mass_sample",
    "source_region": "right_junction",
    "is_boundary_stress": int(expected is None),
})
accepted_raw.append((r3, r4)); found += 1
if found != count:
    raise RuntimeError(f"Generated only {found}/{count} samples for {region}")

margin = cfg.interior_mass_margin
bw = cfg.boundary_half_width

add_left("minus2_interior", cfg.n_left_minus2, -cfg.left_abs_mass_max, -margin, -2)
add_left("plus2_interior", cfg.n_left_plus2, margin, cfg.left_abs_mass_max, 2)
add_left("mass_boundary_stress", cfg.n_left_boundary, -bw, bw, None)

add_right("minus2_interior", cfg.n_right_minus2,
    margin, cfg.right_abs_m4_max, cfg.right_msigma_min, -margin, -2)
add_right("plus2_interior", cfg.n_right_plus2,
    -cfg.right_abs_m4_max, -margin, cfg.right_msigma_min, -margin, 2)
add_right("zero_interior", cfg.n_right_zero,
    -cfg.right_abs_m4_max, cfg.right_abs_m4_max, margin, cfg.right_msigma_max, 0)
add_right("four_valley_boundary_stress", cfg.n_right_four_valley_boundary,
    -bw, bw, cfg.right_msigma_min, -margin, None)
add_right("sigma_boundary_stress", cfg.n_right_sigma_boundary,
    -cfg.right_abs_m4_max, cfg.right_abs_m4_max, -bw, bw, None)

design = pd.DataFrame(rows)
design["draw_order"] = np.arange(1, len(design) + 1)
# Randomize execution order so that checkpoints accumulate across regions.
design = design.sample(frac=1.0, random_state=cfg.random_seed).reset_index(drop=True)
return design

```

```

# =====
# Physics evaluation, metrics, and plotting.
# =====

```

```

def configure_step15_physics(step15, cfg: Step17Config):
    p = step15.Step15Config(output_dir=cfg.output_dir / "strict_support")
    p.quick = cfg.quick
    p.force_recalculate = cfg.force_recalculate
    # Paper validation uses a slightly stricter Fukui hierarchy than the original
    # broad Step15M scan while retaining Wilson fallback for difficult points.
    if not cfg.quick:
        p.gap_grid_n = 121
        p.gap_local_starts = 18

```

```

p.fukui_grids = (61, 81, 101, 121)
p.fukui_shifts = ((0.00, 0.00), (0.50, 0.50), (0.25, 0.73))
p.wilson_nkx = (401, 601, 801)
p.wilson_initial_nky = (81, 101, 121)
p.wilson_shifts = ((0.00, 0.00), (0.50, 0.37), (0.25, 0.73))
return p.normalized()

```

```

def evaluate_design(design: pd.DataFrame, *, step15, core, background: dict[str, float],
                   orientation_factor: int, physics_cfg, cfg: Step17Config):
    results: list[dict[str, Any]] = []
    fukui_all: list[dict[str, Any]] = []
    wilson_all: list[dict[str, Any]] = []
    start = time.time()

    for index, row in design.iterrows():
        sample_id = str(row["sample_id"])
        checkpoint = cfg.output_dir / "checkpoints" / f"{sample_id}.json"
        if checkpoint.is_file() and not cfg.force_recalculate:
            payload = json.loads(checkpoint.read_text(encoding="utf-8"))
            results.append(payload["result"])
            fukui_all.extend(payload.get("fukui_attempts", []))
            wilson_all.extend(payload.get("wilson_attempts", []))
            print(f"[{index+1}/{len(design)}] {sample_id}: checkpoint")
            continue

        print(
            f"[{index+1}/{len(design)}] {sample_id} "
            f"region={row['validation_region']} r3={row['r3']:.9f} r4={row['r4']:.9f}"
        )
        result, fukui_rows, wilson_rows, traces = step15.classify_one(
            core=core, background=background, row=row,
            orientation_factor=orientation_factor, config=physics_cfg,
        )
        # Preserve validation design metadata in the final result.
        for key in [
            "mass_map_region", "validation_region", "expected_phase_code",
            "M_L_tilde", "S_L_tilde", "M_4v_tilde", "M_Sigma_tilde",
            "is_boundary_stress",
        ]:
            result[key] = row.get(key, np.nan)
        payload = {
            "result": result,
            "fukui_attempts": fukui_rows,
            "wilson_attempts": wilson_rows,
        }
        step15.atomic_json(payload, checkpoint)
        for trace in traces:
            sid = str(trace.iloc[0]["sample_id"])
            spin = str(trace.iloc[0]["spin"])
            attempt = int(trace.iloc[0]["attempt_index"])
            step15.atomic_csv(
                trace,
                cfg.output_dir / "wilson_traces" / f"{sid}__{spin}__attempt{attempt:02d}.csv",
            )
        results.append(result)
        fukui_all.extend(fukui_rows)
        wilson_all.extend(wilson_rows)
        print(
            f"    -> {result['final_status']} C_up={result.get('final_chern_up')} "
            f"method={result.get('topology_method')}"
        )

    return pd.DataFrame(results), pd.DataFrame(fukui_all), pd.DataFrame(wilson_all), time.time() - start

```

```

def validation_metrics(results: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame, dict[str, Any]]:
    results = results.copy()
    interior = results[results["is_boundary_stress"].astype(int).eq(0)].copy()

```

```

interior["actual_phase_code"] = pd.to_numeric(interior["strict_phase_code"], errors="coerce")
interior["prediction_match"] = (
    interior["actual_phase_code"].eq(pd.to_numeric(interior["expected_phase_code"], errors="coerce"))
).astype(int)

labels = [-2, 0, 2]
matrix = pd.crosstab(
    pd.Categorical(interior["expected_phase_code"], categories=labels),
    pd.Categorical(interior["actual_phase_code"], categories=labels),
    dropna=False,
)
matrix.index.name = "expected_phase_code"
matrix.columns.name = "actual_phase_code"
matrix_long = matrix.stack(future_stack=True).rename("count").reset_index()

region_rows = []
for region, group in interior.groupby("validation_region", sort=False):
    n = len(group)
    region_rows.append({
        "validation_region": region,
        "expected_phase_code": group["expected_phase_code"].iloc[0],
        "n_points": n,
        "n_match": int(group["prediction_match"].sum()),
        "accuracy": float(group["prediction_match"].mean()) if n else np.nan,
        "n_numeric_unresolved": int(group["final_status"].eq("numeric_unresolved").sum()),
        "n_noninsulating_or_boundary": int((~group["strict_phase_code"].isin([-2, 0, 2])).sum()),
    })
region_summary = pd.DataFrame(region_rows)

boundary = results[results["is_boundary_stress"].astype(int).eq(1)].copy()
if boundary.empty:
    boundary_summary = pd.DataFrame()
else:
    boundary_summary = (
        boundary.groupby(["validation_region", "final_status"], dropna=False)
        .size().rename("count").reset_index()
    )

recalls = []
for label in labels:
    group = interior[pd.to_numeric(interior["expected_phase_code"], errors="coerce").eq(label)]
    if len(group):
        recalls.append(float(group["prediction_match"].mean()))
certificate = {
    "n_random_points_total": int(len(results)),
    "n_interior_validation_points": int(len(interior)),
    "n_boundary_stress_points": int(len(boundary)),
    "n_interior_matches": int(interior["prediction_match"].sum()),
    "interior_accuracy": float(interior["prediction_match"].mean()) if len(interior) else None,
    "interior_balanced_accuracy": float(np.mean(recalls)) if recalls else None,
    "n_interior_numeric_unresolved": int(interior["final_status"].eq("numeric_unresolved").sum()),
    "n_interior_noninsulating_or_boundary": int((~interior["strict_phase_code"].isin([-2, 0, 2])).sum()),
    "n_boundary_gap_closing_or_noninsulating": int((~boundary["strict_phase_code"].isin([-2, 0, 2])).sum()),
    "all_interior_random_points_match_mass_sector": bool(
        len(interior) > 0 and interior["prediction_match"].eq(1).all()
    ),
}
return interior, matrix_long, region_summary, boundary_summary, certificate

def _plot_points(ax, points: pd.DataFrame, x: str, y: str):
    for code in [-2, 0, 2]:
        subset = points[pd.to_numeric(points["strict_phase_code"], errors="coerce").eq(code)]
        if subset.empty:
            continue
        ax.scatter(
            subset[x], subset[y], s=64,
            facecolors="white", edgecolors=PHASE_COLORS[code], linewidths=1.7,
            zorder=5,

```

```

    )
    unresolved = points[~pd.to_numeric(points["strict_phase_code"], errors="coerce").isin([-2, 0, 2])]
    if not unresolved.empty:
        ax.scatter(
            unresolved[x], unresolved[y], s=70, marker="x",
            color=PHASE_COLORS[99], linewidths=1.8, zorder=6,
        )
    mismatch = points[
        points["is_boundary_stress"].astype(int).eq(0)
        & pd.to_numeric(points["expected_phase_code"], errors="coerce").ne(
            pd.to_numeric(points["strict_phase_code"], errors="coerce")
        )
    ]
    if not mismatch.empty:
        ax.scatter(
            mismatch[x], mismatch[y], s=115, facecolors="none",
            edgecolors="black", linewidths=2.0, zorder=7,
        )

def configure_publication_fonts():
    plt.rcParams.update({
        "font.family": "Times New Roman",
        "font.weight": "bold",
        "axes.labelweight": "bold",
        "axes.titleweight": "bold",
        "figure.titleweight": "bold",
        "axes.unicode_minus": False,
        "mathtext.fontset": "custom",
        # Keep accents such as the tilde over mass coordinates at regular
        # weight; all other upright math glyphs are explicitly bolded below.
        "mathtext.rm": "Times New Roman:style=normal:weight=normal",
        "mathtext.it": "Times New Roman:italic:bold",
        "mathtext.bf": "Times New Roman:bold",
        "mathtext.sf": "Times New Roman:bold",
        "mathtext.tt": "Times New Roman:bold",
        "mathtext.fallback": None,
        "svg.fonttype": "none",
        "pdf.fonttype": 42,
        "ps.fonttype": 42,
    })

def plot_validation_maps(results: pd.DataFrame, cfg: Step17Config):
    configure_publication_fonts()
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=cfg.figure_size_combined)

    # Left ideal sector background.
    ax1.axvspan(-1.4, 0, color=PHASE_COLORS[-2], alpha=0.28, zorder=0)
    ax1.axvspan(0, 1.4, color=PHASE_COLORS[2], alpha=0.28, zorder=0)
    ax1.axvline(0, color="black", linewidth=1.8)
    left = results[results["mass_map_region"].eq("left_local")]
    _plot_points(ax1, left, "M_L_tilde", "S_L_tilde")
    ax1.set_xlim(-1.35, 1.35); ax1.set_ylim(-1.15, 1.15)
    ax1.set_xlabel(r"$\overset{\text{\tiny{M}}}{\mathbf{L}}$", fontsize=25, fontweight="normal")
    ax1.set_ylabel(r"$\overset{\text{\tiny{S}}}{\mathbf{L}}$", fontsize=25, fontweight="normal")
    ax1.set_title("(a) Random validation in the left local-mass plane", fontsize=20, pad=12)

    # Right ideal sector background.
    ax2.axhspan(0, 1.4, color=PHASE_COLORS[0], alpha=0.45, zorder=0)
    ax2.fill_between([-1.4, 0], -1.4, 0, color=PHASE_COLORS[2], alpha=0.35, zorder=0)
    ax2.fill_between([0, 1.4], -1.4, 0, color=PHASE_COLORS[-2], alpha=0.35, zorder=0)
    ax2.axvline(0, color="black", linewidth=1.8)
    ax2.axhline(0, color="black", linewidth=1.8)
    right = results[results["mass_map_region"].eq("right_junction")]
    _plot_points(ax2, right, "M_4v_tilde", "M_Sigma_tilde")
    ax2.set_xlim(-1.35, 1.35); ax2.set_ylim(-1.35, 1.35)
    ax2.set_xlabel(r"$\overset{\text{\tiny{M}}}{\mathbf{4v}}$", fontsize=25, fontweight="normal")
    ax2.set_ylabel(r"$\overset{\text{\tiny{M}}}{\mathbf{\Sigma^{\prime}}}$", fontsize=25, fontweight="normal")

```

```

ax2.set_title("(b) Random validation in the right double-mass plane", fontsize=20, pad=12)

for ax in [ax1, ax2]:
    ax.set_box_aspect(1)
    ax.tick_params(direction="in", length=7, width=1.7, top=True, right=True, labels=15)
    for spine in ax.spines.values():
        spine.set_linewidth(1.7)

handles = [
    Patch(facecolor=PHASE_COLORS[-2], edgecolor="black", label=r"$C_{\mathbf{\uparrow}\mathbf{=-2}}$"),
    Patch(facecolor=PHASE_COLORS[0], edgecolor="black", label=r"$C_{\mathbf{\uparrow}\mathbf{=0}}$"),
    Patch(facecolor=PHASE_COLORS[2], edgecolor="black", label=r"$C_{\mathbf{\uparrow}\mathbf{=2}}$"),
    Line2D([0], [0], marker="x", linestyle="None", color=PHASE_COLORS[99],
           markersize=9, markeredgewidth=1.8, label="gapless / boundary / unresolved"),
    Line2D([0], [0], marker="o", linestyle="None", color="black",
           markerfacecolor="none", markersize=10, markeredgewidth=1.8,
           label="interior mismatch"),
]
fig.legend(handles=handles, loc="upper center", ncol=5, frameon=True,
           bbox_to_anchor=(0.5, 1.02), fontsize=13)
fig.tight_layout(rect=(0, 0, 1, 0.94))

stem = cfg.output_dir / "figures" / "step17M_random_mass_coordinate_validation"
for ext in ["png", "pdf", "svg"]:
    fig.savefig(stem.with_suffix(f".{ext}"), dpi=600 if ext == "png" else None, bbox_inches="tight")
plt.close(fig)

def plot_confusion_matrix(matrix_long: pd.DataFrame, cfg: Step17Config):
    configure_publication_fonts()
    labels = [-2, 0, 2]
    matrix = matrix_long.pivot(index="expected_phase_code", columns="actual_phase_code", values="count")
    matrix = matrix.reindex(index=labels, columns=labels, fill_value=0).fillna(0).to_numpy(dtype=int)
    fig, ax = plt.subplots(figsize=(6.4, 5.8))
    image = ax.imshow(matrix, cmap="Blues")
    for i in range(3):
        for j in range(3):
            ax.text(j, i, str(matrix[i, j]), ha="center", va="center", fontsize=15, fontweight="bold")
    ax.set_xticks(range(3), labels); ax.set_yticks(range(3), labels)
    ax.set_xlabel(r"Calculated $C_{\mathbf{\uparrow}}$", fontsize=17, fontweight="bold")
    ax.set_ylabel(r"Mass-sector prediction", fontsize=17, fontweight="bold")
    ax.set_title("Random out-of-grid validation", fontsize=18, fontweight="bold")
    ax.tick_params(direction="in", top=True, right=True, labels=14)
    fig.colorbar(image, ax=ax, label="Count")
    fig.tight_layout()
    stem = cfg.output_dir / "figures" / "step17M_random_validation_confusion_matrix"
    for ext in ["png", "pdf", "svg"]:
        fig.savefig(stem.with_suffix(f".{ext}"), dpi=600 if ext == "png" else None, bbox_inches="tight")
    plt.close(fig)

# =====
# Main workflow.
# =====

def run_step17M(
    *,
    tts_archive: str | Path = "tts(1).zip",
    step11_source: str | Path = "outputs_tts_step11M_lieb_aligned_fixed_slice.zip",
    step13_source: str | Path = "outputs_tts_step13M_adaptive_wilson_intermediate_chern.zip",
    step16_source: str | Path = "outputs_tts_step16M_mass_coordinate_phase_maps",
    step15_module_source: str | Path = "TTS_step15M_reclassify_all_boundary_points.py",
    output_dir: str | Path = "outputs_tts_step17M_random_mass_coordinate_validation",
    config: Step17Config | None = None,
    run_physics: bool = True,
) -> dict[str, Any]:
    if config is None:
        config = Step17Config(output_dir=Path(output_dir))

```

```

else:
    config.output_dir = Path(output_dir)
    config = config.normalized()

print("Code version:", CODE_VERSION)
step16_report = read_json_any(step16_source, "step16M_02_mass_coordinate_report.json")
existing_points = read_csv_any(step16_source, "step16M_00_transformed_mass_coordinate_points.csv")

design = generate_random_design(config, step16_report, existing_points)
design_path = config.output_dir / "step17M_01_random_inverse_mass_samples.csv"
design.to_csv(design_path, index=False)
audit = {
    "code_version": CODE_VERSION,
    "sampling_design": "uniform_random_sampling_in_local_mass sectors followed by exact inverse mapping to
↔ r3-r4",
    "config": asdict(config),
    "n_samples": int(len(design)),
    "counts_by_region": design["validation_region"].value_counts().to_dict(),
    "step16_mass_report_code_version": step16_report.get("code_version"),
    "warnings": [
        "The mass coordinates are local and must not be extrapolated beyond the certified windows.",
        "Boundary-stress points are excluded from interior classification accuracy.",
        "quick=True is for pipeline testing only and is not paper evidence.",
    ],
}
(config.output_dir / "step17M_00_sampling_audit.json").write_text(
    json.dumps(audit, ensure_ascii=False, indent=2, default=str), encoding="utf-8"
)

if not run_physics:
    summary = {
        "code_version": CODE_VERSION,
        "sampling_only": True,
        "n_samples_generated": int(len(design)),
        "design_csv": str(design_path),
    }
    print(json.dumps(summary, ensure_ascii=False, indent=2))
    return summary

module_path = resolve_step15_module(step15_module_source)
step15 = import_module_from_path("tts_step15_support_for_step17", module_path)
physics_cfg = configure_step15_physics(step15, config)
model_archive = resolve_tts_archive(tts_archive)
core = step15.load_core(model_archive, physics_cfg)
background = step15.read_json_token(step11_source, "step11M_13_fixed_slice_certificate.json")[
    "fixed_background_parameters"
]
orientation = step15.read_json_token(step13_source, "step13M_04b_orientation_calibration.json")
if int(orientation.get("orientation_calibrated", 0)) != 1:
    raise RuntimeError("Step13 Wilson orientation was not calibrated")
orientation_factor = int(orientation["orientation_factor"])

results, fukui, wilson, elapsed = evaluate_design(
    design, step15=step15, core=core, background=background,
    orientation_factor=orientation_factor, physics_cfg=physics_cfg, cfg=config,
)
results.to_csv(config.output_dir / "step17M_02_random_validation_results.csv", index=False)
fukui.to_csv(config.output_dir / "step17M_03_fukui_attempts.csv", index=False)
wilson.to_csv(config.output_dir / "step17M_04_wilson_attempts.csv", index=False)

interior, matrix_long, region_summary, boundary_summary, certificate = validation_metrics(results)
interior.to_csv(config.output_dir / "step17M_05_interior_validation_points.csv", index=False)
matrix_long.to_csv(config.output_dir / "step17M_06_confusion_matrix.csv", index=False)
region_summary.to_csv(config.output_dir / "step17M_07_region_accuracy.csv", index=False)
boundary_summary.to_csv(config.output_dir / "step17M_08_boundary_stress_summary.csv", index=False)

mismatches = interior[interior["prediction_match"].eq(0)].copy()
mismatches.to_csv(config.output_dir / "step17M_09_interior_mismatches.csv", index=False)

```

```

certificate.update({
    "code_version": CODE_VERSION,
    "elapsed_seconds": float(elapsed),
    "fixed_background_parameters": background,
    "wilson_orientation_factor": orientation_factor,
    "physics_quick_mode": bool(config.quick),
    "validation_claim": (
        "Random out-of-grid parameter points were sampled in local mass-coordinate sectors, "
        "inverse-mapped to the original Hamiltonian parameters, and independently recalculated."
    ),
    "evidence_scope": {
        "left": "Only the certified left local four-valley window is tested.",
        "right": "Only the refined right-junction window is tested.",
        "global": "This is not a validation of a universal seven-dimensional mass rule.",
    },
})
(config.output_dir / "step17M_10_final_random_validation_certificate.json").write_text(
    json.dumps(certificate, ensure_ascii=False, indent=2, default=str), encoding="utf-8"
)

plot_validation_maps(results, config)
plot_confusion_matrix(matrix_long, config)

print(json.dumps(certificate, ensure_ascii=False, indent=2, default=str))
return certificate

if __name__ == "__main__":
    run_step17M()

```

三体系能带、边缘态、Berry 曲率与霍尔响应代码

► 代码清单 A.51.1 (python, 1892 行)

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Step 18: three-system WannierTools-style edge spectra and Hall examples.

The workflow selects exactly three states for each of the Lieb, TTS and FES
models:

1. a comparatively large-gap topological representative;
2. the refined critical Hamiltonian on the same transition path;
3. a gapped Hamiltonian on the Chern-changed side of that boundary.

It then calculates:

- square-lattice bulk bands;
- a finite ribbon spectrum with left/right edge weights and spin character;
- a semi-infinite surface spectral function using the iterative Sancho method;
- energy-resolved spin-up, spin-down, charge and spin Hall responses.

The numerical definitions follow the existing WannierTools-style project
scripts. Plot colors are taken exactly from the corresponding phase maps.
No existing result directory is modified.
"""

from __future__ import annotations

```

```

from dataclasses import dataclass
from datetime import datetime, timezone
from pathlib import Path
from typing import Any, Callable, Iterable
import argparse
import importlib.util
import json
import math
import os
import sys
import time

os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.environ.setdefault("OMP_NUM_THREADS", "1")
os.environ.setdefault("MKL_NUM_THREADS", "1")
os.environ.setdefault("NUMEXPR_NUM_THREADS", "1")

import matplotlib as mpl
import matplotlib.pyplot as plt
from matplotlib.colors import to_rgb
import numpy as np
import pandas as pd
from scipy.linalg import eigh
from threadpoolctl import threadpool_limits

_THREADPOOL_LIMITER = threadpool_limits(limits=1)

CODE_VERSION = "TTS_STEP18_THREE_SYSTEM_WT_STYLE_V1_20260730"
SCRIPT_DIR = Path(__file__).resolve().parent
PROJECT_ROOT = SCRIPT_DIR.parent
LIEB_ROOT = PROJECT_ROOT / "lieb"
TTS_ROOT = PROJECT_ROOT / "tts"
FES_ROOT = PROJECT_ROOT / "fes"

DEFAULT_OUTPUT = (
    TTS_ROOT / "outputs_three_system_wanniertools_style_edge_ahc_examples"
)

STATE_ORDER = ("topological", "critical", "chern_changed")
STATE_LABELS = {
    "topological": "Topological",
    "critical": "Critical",
    "chern_changed": "Chern-changed",
}
SYSTEM_LABELS = {"lieb": "Lieb", "tts": "TTS", "fes": "FES"}

# Exact strong colors used by the phase maps. Region backgrounds are derived
# by blending these colors toward white.
LIEB_FES_COLORS = {
    +1: "#A92425",
    0: "#D3D3D3",
    -1: "#3F63AD",
}
TTS_COLORS = {
    +2: "#F28E8B",
    +1: "#7FCBA1",
    0: "#B2B2B2",
    -1: "#8AA3CD",
    -2: "#3F63AD",
}
CRITICAL_COLOR = "#4D4D4D"
BODY_COLOR = "#D7D7D7"
BODY_DARK = "#AFAFAF"
BLACK = "#111111"

@dataclass
class ModelSpec:

```

```

key: str
label: str
h_periodic: Callable[[float, float, dict[str, float]], np.ndarray]
h_atomic: Callable[[float, float, dict[str, float]], np.ndarray]
spin_up_indices: tuple[int, ...]
spin_down_indices: tuple[int, ...]
n_occ_total: int
n_occ_spin: int
palette: dict[int, str]
parameter_names: tuple[str, ...]
ribbon_width: int
energy_reference: str = "global_midgap"

```

```

@dataclass
class SelectedState:
    system: str
    state: str
    source_id: str
    path_id: str
    params: dict[str, float]
    expected_chern_up: int | None
    expected_chern_down: int | None
    critical_kx: float
    critical_ky: float
    selection_note: str

```

```

@dataclass
class RunSettings:
    gap_nk: int
    chern_nk: int
    band_points_per_segment: int
    ribbon_k_points: int
    surface_k_points: int
    surface_energy_points: int
    response_nk: int
    response_energy_points: int
    fourier_n: int
    dpi: int
    quick: bool

```

```

def utc_now() -> str:
    return datetime.now(timezone.utc).isoformat()

```

```

def load_module(name: str, path: Path):
    spec = importlib.util.spec_from_file_location(name, path)
    if spec is None or spec.loader is None:
        raise ImportError(f"Unable to load module from {path}")
    module = importlib.util.module_from_spec(spec)
    sys.modules[name] = module
    spec.loader.exec_module(module)
    return module

```

```

def configure_plot_style() -> None:
    mpl.rcParams.update(
        {
            "font.family": "Times New Roman",
            "font.weight": "bold",
            "axes.labelweight": "bold",
            "axes.titleweight": "bold",
            "axes.linewidth": 1.8,
            "font.size": 10.5,
            "axes.labelsize": 12,
            "axes.titlesize": 11.5,
            "xtick.labelsize": 10,

```

```

        "ytick.labelsize": 10,
        "legend.fontsize": 8.5,
        "mathtext.fontset": "stix",
        "mathtext.default": "it",
        "savefig.facecolor": "white",
        "figure.facecolor": "white",
        "svg.fonttype": "none",
        "pdf.fonttype": 42,
        "ps.fonttype": 42,
    }
)

def style_axis(ax: plt.Axes) -> None:
    ax.tick_params(
        which="both",
        direction="in",
        top=True,
        right=True,
        width=1.35,
        length=4.5,
    )
    for spine in ax.spines.values():
        spine.set_linewidth(1.8)
        spine.set_color(BLACK)
    for label in ax.get_xticklabels() + ax.get_yticklabels():
        label.set_fontweight("bold")

def save_figure(fig: plt.Figure, stem: Path, dpi: int) -> list[str]:
    paths: list[str] = []
    for suffix in (".png", ".pdf", ".svg"):
        path = stem.with_suffix(suffix)
        fig.savefig(path, dpi=dpi, bbox_inches="tight", pad_inches=0.04)
        paths.append(str(path))
    plt.close(fig)
    return paths

def lighten(color: str, factor: float = 0.78) -> tuple[float, float, float]:
    rgb = np.asarray(to_rgb(color), dtype=float)
    return tuple(np.clip(rgb * (1.0 - factor) + factor, 0.0, 1.0))

def phase_color(spec: ModelSpec, chern: int | None) -> str:
    if chern is None:
        return CRITICAL_COLOR
    return spec.palette[int(chern)]

def build_model_specs() -> tuple[dict[str, ModelSpec], dict[str, Any]]:
    lieb = load_module(
        "step18_lieb_source",
        LIEB_ROOT / "08_Lieb8_Analytic_Mechanism_and_WannierTools_Edge_v5.py",
    )
    fes = load_module("step18_fes_source", FES_ROOT / "FES_step01_v2.py")
    tts = load_module(
        "step18_tts_source",
        TTS_ROOT / "TTS_step01_model_and_label_audit_v2.py",
    )

    specs = {
        "lieb": ModelSpec(
            key="lieb",
            label="Lieb",
            h_periodic=lieb.h_periodic,
            h_atomic=lieb.h_lieb_atomic,
            spin_up_indices=tuple(int(x) for x in lieb.SPIN_INDICES["up"]),
            spin_down_indices=tuple(int(x) for x in lieb.SPIN_INDICES["down"]),
        ),
    }

```

```

        n_occ_total=2,
        n_occ_spin=1,
        palette=LIEB_FES_COLORS,
        parameter_names=tuple(lieb.RAW8),
        ribbon_width=60,
    ),
    "tts": ModelSpec(
        key="tts",
        label="TTS",
        h_periodic=tts.h_tts_periodic,
        h_atomic=tts.h_tts_atomic,
        spin_up_indices=tuple(int(x) for x in tts.SPIN_UP_INDICES),
        spin_down_indices=tuple(int(x) for x in tts.SPIN_DOWN_INDICES),
        n_occ_total=int(tts.N_OCC_TOTAL),
        n_occ_spin=int(tts.N_OCC_SPIN),
        palette=TTS_COLORS,
        parameter_names=tuple(tts.RAW8),
        ribbon_width=36,
    ),
    "fes": ModelSpec(
        key="fes",
        label="FES",
        h_periodic=fes.h_fes_periodic,
        h_atomic=fes.h_fes_atomic,
        spin_up_indices=tuple(int(x) for x in fes.SPIN_UP_INDICES),
        spin_down_indices=tuple(int(x) for x in fes.SPIN_DOWN_INDICES),
        n_occ_total=int(fes.N_OCC_TOTAL),
        n_occ_spin=int(fes.N_OCC_SPIN),
        palette=LIEB_FES_COLORS,
        parameter_names=tuple(fes.RAW6),
        ribbon_width=44,
        energy_reference="critical_valley_midgap",
    ),
}
return specs, {"lieb": lieb, "tts": tts, "fes": fes}

def _clean_params(
    params: dict[str, float],
    names: Iterable[str],
) -> dict[str, float]:
    return {name: float(params[name]) for name in names}

def select_lieb_states(lieb: Any) -> list[SelectedState]:
    output = (
        LIEB_ROOT
        / "outputs_step08_analytic_boundary_wanniertools_edge_msg123342_lieb8_from_step06_n1024_pairs4_seed202607"
        ↪ 11"
    )
    roots = pd.read_csv(output / "step08_04_refined_analytic_transition_roots.csv")
    profiles = pd.read_csv(output / "step08_04_dense_transition_path_profiles.csv")

    direct_flip = roots[
        (roots["boundary"].astype(str) == "t2")
        & (pd.to_numeric(roots["before_bern_up_int"], errors="coerce")
         * pd.to_numeric(roots["after_bern_up_int"], errors="coerce") < 0)
    ].copy()
    if direct_flip.empty:
        raise RuntimeError("No direct Lieb Chern-sign flip was found")
    root = direct_flip.sort_values("verified_direct_gap").iloc[0]
    pair_id = int(root["pair_id"])
    tc = float(root["t_critical"])

    pair_roots = roots[roots["pair_id"].astype(int) == pair_id].sort_values("t_critical")
    later_roots = pair_roots[pair_roots["t_critical"] > tc + 1.0e-8]
    upper = float(later_roots["t_critical"].min()) if not later_roots.empty else 1.0

```

```

profile = profiles[profiles["pair_id"].astype(int) == pair_id].copy()
pre = profile[profile["t"] <= tc - 0.04].sort_values(
    ["indirect_gap", "direct_gap"], ascending=False
).iloc[0]
post = profile[
    (profile["t"] >= tc + 0.04) & (profile["t"] <= upper - 0.04)
].sort_values(["indirect_gap", "direct_gap"], ascending=False).iloc[0]

def raw_from_row(row: pd.Series) -> dict[str, float]:
    vector = [float(row[name]) for name in lieb.PHYS7]
    return _clean_params(lieb.phys7_to_raw8(vector), lieb.RAW8)

critical_params = raw_from_row(root)
pre_c = int(root["before_chn_up_int"])
post_c = int(root["after_chn_up_int"])
critical_kx = float(root["global_direct_kx"])
critical_ky = float(root["global_direct_ky"])
path_id = f"pair_{pair_id}_direct_t2_flip"
return [
    SelectedState(
        system="lieb",
        state="topological",
        source_id=f"pair_{pair_id}_t{float(pre['t']):.6f}",
        path_id=path_id,
        params=raw_from_row(pre),
        expected_chn_up=pre_c,
        expected_chn_down=-pre_c,
        critical_kx=critical_kx,
        critical_ky=critical_ky,
        selection_note=(
            "Largest indirect gap on the pre-critical Chern-sign segment "
            f"(t < {tc:.8f})."
        ),
    ),
    SelectedState(
        system="lieb",
        state="critical",
        source_id=f"pair_{pair_id}_tcritical_{tc:.10f}",
        path_id=path_id,
        params=critical_params,
        expected_chn_up=None,
        expected_chn_down=None,
        critical_kx=critical_kx,
        critical_ky=critical_ky,
        selection_note="Refined exact t2=0 Chern-sign-flip root.",
    ),
    SelectedState(
        system="lieb",
        state="chern_changed",
        source_id=f"pair_{pair_id}_t{float(post['t']):.6f}",
        path_id=path_id,
        params=raw_from_row(post),
        expected_chn_up=post_c,
        expected_chn_down=-post_c,
        critical_kx=critical_kx,
        critical_ky=critical_ky,
        selection_note=(
            "Largest indirect gap after the direct sign flip and before "
            "the next analytic boundary."
        ),
    ),
]

```

```

def select_tts_states(tts: Any) -> list[SelectedState]:
    output = TTS_ROOT / "outputs_tts_step04_chern_sector_boundary_valley_tracking"
    refined = pd.read_csv(output / "step04_05_refined_critical_valleys.csv")
    gaps = pd.read_csv(output / "step04_04_path_dense_gap_scan.csv")

```

```

path_id = "Cup+1_anchor01_nearest"
candidates = refined[
    (refined["path_id"].astype(str) == path_id)
    & (pd.to_numeric(refined["chern_left"], errors="coerce") == 1)
    & (pd.to_numeric(refined["chern_right"], errors="coerce") == 0)
]
if candidates.empty:
    raise RuntimeError("The requested TTS C_up=+1 to 0 transition is missing")
root = candidates.iloc[0]
lc = float(root["critical_lambda"])
path = gaps[gaps["path_id"].astype(str) == path_id].copy()
pre = path[path["lambda"] <= lc - 0.05].sort_values(
    ["indirect_gap", "min_direct_gap"], ascending=False
).iloc[0]
post = path[path["lambda"] >= lc + 0.05].sort_values(
    ["indirect_gap", "min_direct_gap"], ascending=False
).iloc[0]

def raw_from_row(row: pd.Series) -> dict[str, float]:
    reduced = {name: float(row[name]) for name in tts.REDUCED7}
    return _clean_params(tts.raw8_from_reduced7(reduced), tts.RAW8)

kx = float(root["critical_kx"])
ky = float(root["critical_ky"])
return [
    SelectedState(
        system="tts",
        state="topological",
        source_id=str(pre["path_point_id"]),
        path_id=path_id,
        params=raw_from_row(pre),
        expected_chern_up=1,
        expected_chern_down=-1,
        critical_kx=kx,
        critical_ky=ky,
        selection_note="Largest indirect gap on the C_up=+1 side of the path.",
    ),
    SelectedState(
        system="tts",
        state="critical",
        source_id=str(root["transition_id"]),
        path_id=path_id,
        params=raw_from_row(root),
        expected_chern_up=None,
        expected_chern_down=None,
        critical_kx=kx,
        critical_ky=ky,
        selection_note="Refined M-valley critical Hamiltonian.",
    ),
    SelectedState(
        system="tts",
        state="chern_changed",
        source_id=str(post["path_point_id"]),
        path_id=path_id,
        params=raw_from_row(post),
        expected_chern_up=0,
        expected_chern_down=0,
        critical_kx=kx,
        critical_ky=ky,
        selection_note="Largest indirect gap on the C_up=0 side of the same path.",
    ),
]

```

```

def select_fes_states(fes: Any) -> list[SelectedState]:
    boundary = FES_ROOT / "outputs_fes6_step04_boundary"
    kp = FES_ROOT / "outputs_fes6_step06_kp"
    strict = pd.read_csv(boundary / "fes_step04_strict_verified_results.csv")
    roots = pd.read_csv(kp / "fes_step06_critical_mass_roots.csv")

```

```

reliable = strict[
    (pd.to_numeric(strict["verified_chern_reliable"], errors="coerce") == 1)
    & (pd.to_numeric(strict["verified_chern_up_int"], errors="coerce").abs() == 1)
].copy()
topo = reliable.sort_values(
    ["verified_min_direct_gap", "verified_indirect_gap"],
    ascending=False,
).iloc[0]
point_id = str(topo["point_id"])
root_candidates = roots[
    (roots["point_id"].astype(str) == point_id)
    & (roots["valley"].astype(str) == "Gamma")
    & (pd.to_numeric(roots["root_found"], errors="coerce") == 1)
]
if root_candidates.empty:
    raise RuntimeError(f"No FES Gamma critical root for {point_id}")
root = root_candidates.iloc[0]

reduced_topo = {
    name: float(topo[name]) for name in ("m_e", "t1", "t2", "r1", "r2")
}
reduced_critical = dict(reduced_topo)
reduced_critical["m_e"] = float(root["m_critical"])

direction = float(np.sign(float(root["m_critical"]) - float(topo["m_e"])))
if direction == 0.0:
    direction = 1.0
reduced_changed = dict(reduced_critical)
reduced_changed["m_e"] += direction * 0.03

raw_topo = _clean_params(fes.raw6_from_reduced5(reduced_topo), fes.RAW6)
raw_critical = _clean_params(
    fes.raw6_from_reduced5(reduced_critical), fes.RAW6
)
raw_changed = _clean_params(
    fes.raw6_from_reduced5(reduced_changed), fes.RAW6
)
expected_topo = int(topo["verified_chern_up_int"])
path_id = f"{point_id}_Gamma_mass_crossing"
return [
    SelectedState(
        system="fes",
        state="topological",
        source_id=point_id,
        path_id=path_id,
        params=raw_topo,
        expected_chern_up=expected_topo,
        expected_chern_down=-expected_topo,
        critical_kx=0.0,
        critical_ky=0.0,
        selection_note=(
            "Largest verified direct gap among reliable FES |Cup|=1 "
            "samples; the indirect gap remains negative by the FES no-go result."
        ),
    ),
    SelectedState(
        system="fes",
        state="critical",
        source_id=f"{point_id}_Gamma_mcrit",
        path_id=path_id,
        params=raw_critical,
        expected_chern_up=None,
        expected_chern_down=None,
        critical_kx=0.0,
        critical_ky=0.0,
        selection_note="Analytic Gamma-valley mass root from FES Step06.",
    ),
    SelectedState(

```

```

        system="fes",
        state="chern_changed",
        source_id=f"{point_id}_Gamma_post_0p03",
        path_id=path_id,
        params=raw_changed,
        expected_chern_up=0,
        expected_chern_down=0,
        critical_kx=0.0,
        critical_ky=0.0,
        selection_note=(
            "C_up=0 point 0.03 in m_e beyond the same Gamma boundary."
        ),
    ),
]

def select_all_states(modules: dict[str, Any]) -> list[SelectedState]:
    return (
        select_lieb_states(modules["lieb"])
        + select_tts_states(modules["tts"])
        + select_fes_states(modules["fes"])
    )

def extract_hoppings(
    spec: ModelSpec,
    params: dict[str, float],
    nfft: int,
    tolerance: float = 1.0e-10,
) -> dict[tuple[int, int], np.ndarray]:
    nfft = int(nfft)
    ks = 2.0 * np.pi * np.arange(nfft) / nfft
    dim = int(spec.h_periodic(0.0, 0.0, params).shape[0])
    hk = np.empty((nfft, nfft, dim, dim), dtype=np.complex128)
    for ix, kx in enumerate(ks):
        for iy, ky in enumerate(ks):
            hk[ix, iy] = spec.h_periodic(float(kx), float(ky), params)
    coeff = np.fft.fft2(hk, axes=(0, 1)) / float(nfft * nfft)
    hoppings: dict[tuple[int, int], np.ndarray] = {}
    for ix in range(nfft):
        rx = ix if ix <= nfft // 2 else ix - nfft
        for iy in range(nfft):
            ry = iy if iy <= nfft // 2 else iy - nfft
            mat = coeff[ix, iy]
            if float(np.max(np.abs(mat))) > tolerance:
                hoppings[(int(rx), int(ry))] = np.asarray(
                    mat, dtype=np.complex128
                )
    for kx, ky in ((0.173, -1.029), (2.037, 0.691), (-2.4, 2.1)):
        rec = np.zeros((dim, dim), dtype=np.complex128)
        for (rx, ry), mat in hoppings.items():
            rec += mat * np.exp(1j * (kx * rx + ky * ry))
        ref = spec.h_periodic(kx, ky, params)
        error = float(np.max(np.abs(rec - ref)))
        if error > 2.0e-9:
            raise RuntimeError(
                f"{spec.key} Fourier reconstruction failed: {error:.3e}"
            )
    if any(abs(ry) > 1 for _, ry in hoppings):
        raise RuntimeError(
            f"{spec.key} axial principal layer is too thin for hoppings "
            f"{sorted(hoppings)}"
        )
    return hoppings

def bloch_from_hoppings(
    kx: float,
    ky: float,

```

```

    hoppings: dict[tuple[int, int], np.ndarray],
) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    dim = next(iter(hoppings.values())).shape[0]
    h = np.zeros((dim, dim), dtype=np.complex128)
    vx = np.zeros_like(h)
    vy = np.zeros_like(h)
    for (rx, ry), mat in hoppings.items():
        phase = np.exp(1j * (float(kx) * rx + float(ky) * ry))
        h += mat * phase
        vx += 1j * rx * mat * phase
        vy += 1j * ry * mat * phase
    h = 0.5 * (h + h.conj().T)
    vx = 0.5 * (vx + vx.conj().T)
    vy = 0.5 * (vy + vy.conj().T)
    return h, vx, vy

def axial_blocks(
    k_parallel: float,
    hoppings: dict[tuple[int, int], np.ndarray],
) -> tuple[np.ndarray, np.ndarray]:
    dim = next(iter(hoppings.values())).shape[0]
    blocks = {
        -1: np.zeros((dim, dim), dtype=np.complex128),
        0: np.zeros((dim, dim), dtype=np.complex128),
        1: np.zeros((dim, dim), dtype=np.complex128),
    }
    for (rx, ry), mat in hoppings.items():
        blocks[int(ry)] += mat * np.exp(1j * float(k_parallel) * rx)
    h0 = 0.5 * (blocks[0] + blocks[0].conj().T)
    hp = 0.5 * (blocks[1] + blocks[-1].conj().T)
    return h0, hp

def gap_audit(
    spec: ModelSpec,
    selected: SelectedState,
    nk: int,
) -> dict[str, float | int]:
    nk = int(nk)
    if nk % 2:
        nk += 1
    ks = np.linspace(-np.pi, np.pi, nk, endpoint=False)
    min_direct = np.inf
    direct_kx = np.nan
    direct_ky = np.nan
    vbm = -np.inf
    cbm = np.inf
    vbm_kx = vbm_ky = cbm_kx = cbm_ky = np.nan
    for kx in ks:
        for ky in ks:
            energies = np.linalg.eigvalsh(
                spec.h_periodic(float(kx), float(ky), selected.params)
            )
            dg = float(
                energies[spec.n_occ_total] - energies[spec.n_occ_total - 1]
            )
            if dg < min_direct:
                min_direct = dg
                direct_kx, direct_ky = float(kx), float(ky)
            valence = float(energies[spec.n_occ_total - 1])
            conduction = float(energies[spec.n_occ_total])
            if valence > vbm:
                vbm, vbm_kx, vbm_ky = valence, float(kx), float(ky)
            if conduction < cbm:
                cbm, cbm_kx, cbm_ky = conduction, float(kx), float(ky)

    kc_e = np.linalg.eigvalsh(
        spec.h_periodic(

```

```

        float(selected.critical_kx),
        float(selected.critical_ky),
        selected.params,
    )
)
critical_valence = float(kc_e[spec.n_occ_total - 1])
critical_conduction = float(kc_e[spec.n_occ_total])
critical_gap = critical_conduction - critical_valence
# The Lieb sign-flip closes at an off-grid axis momentum. Include the
# explicitly refined transition momentum in the global extrema audit so
# the reported indirect gap cannot spuriously exceed the exact direct gap.
if critical_valence > vbm:
    vbm = critical_valence
    vbm_kx = float(selected.critical_kx)
    vbm_ky = float(selected.critical_ky)
if critical_conduction < cbm:
    cbm = critical_conduction
    cbm_kx = float(selected.critical_kx)
    cbm_ky = float(selected.critical_ky)
if critical_gap < min_direct:
    min_direct = critical_gap
    direct_kx = float(selected.critical_kx)
    direct_ky = float(selected.critical_ky)

global_midgap = 0.5 * (vbm + cbm)
local_midgap = 0.5 * (critical_valence + critical_conduction)
if selected.state == "critical":
    ef = local_midgap
    reference_kind = "critical_closing_energy"
elif spec.energy_reference == "critical_valley_midgap":
    ef = local_midgap
    reference_kind = "critical_valley_local_midgap"
else:
    ef = global_midgap
    reference_kind = "global_midgap"

return {
    "gap_nk": int(nk),
    "min_direct_gap": float(min_direct),
    "indirect_gap": float(cbm - vbm),
    "direct_gap_kx": float(direct_kx),
    "direct_gap_ky": float(direct_ky),
    "vbm": float(vbm),
    "vbm_kx": float(vbm_kx),
    "vbm_ky": float(vbm_ky),
    "cbm": float(cbm),
    "cbm_kx": float(cbm_kx),
    "cbm_ky": float(cbm_ky),
    "critical_valley_gap": float(critical_gap),
    "critical_valley_midgap": float(local_midgap),
    "energy_reference": float(ef),
    "energy_reference_kind": reference_kind,
    "is_direct_gapped": int(min_direct > 1.0e-4),
    "is_global_insulator": int((cbm - vbm) > 1.0e-4),
}

def _normalized_link(value: complex, floor: float = 1.0e-14) -> complex:
    amplitude = abs(value)
    if amplitude < floor:
        raise FloatingPointError(f"Link determinant too small: {amplitude:.3e}")
    return value / amplitude

def fukui_chern(
    h_func: Callable[[float, float], np.ndarray],
    n_occ: int,
    nk: int,
) -> tuple[float, float]:

```

```

sample = h_func(0.0, 0.0)
dim = sample.shape[0]
ks = 2.0 * np.pi * np.arange(int(nk)) / int(nk)
occupied = np.empty((nk, nk, dim, n_occ), dtype=np.complex128)
for ix, kx in enumerate(ks):
    for iy, ky in enumerate(ks):
        _, vectors = np.linalg.eigh(h_func(float(kx), float(ky)))
        occupied[ix, iy] = vectors[:, :n_occ]
total = 0.0
min_det = 1.0
for ix in range(nk):
    for iy in range(nk):
        v = occupied[ix, iy]
        vx = occupied[(ix + 1) % nk, iy]
        vy = occupied[ix, (iy + 1) % nk]
        vxy = occupied[(ix + 1) % nk, (iy + 1) % nk]
        lx = np.linalg.det(v.conj().T @ vx)
        ly = np.linalg.det(v.conj().T @ vy)
        lx_y = np.linalg.det(vy.conj().T @ vxy)
        ly_x = np.linalg.det(vx.conj().T @ vxy)
        min_det = min(min_det, abs(lx), abs(ly), abs(lx_y), abs(ly_x))
        plaquette = (
            _normalized_link(lx)
            * _normalized_link(ly_x)
            / (_normalized_link(lx_y) * _normalized_link(ly))
        )
        total += float(np.angle(plaquette))
return float(total / (2.0 * np.pi)), float(min_det)

def chern_audit(
    spec: ModelSpec,
    selected: SelectedState,
    nk: int,
) -> dict[str, float | int]:
    if selected.state == "critical":
        return {
            "chern_nk": int(nk),
            "chern_up": np.nan,
            "chern_down": np.nan,
            "chern_up_int": np.nan,
            "chern_down_int": np.nan,
            "min_link_up": np.nan,
            "min_link_down": np.nan,
            "chern_defined": 0,
        }
    up_idx = np.asarray(spec.spin_up_indices, dtype=int)
    down_idx = np.asarray(spec.spin_down_indices, dtype=int)

    def block(indices: np.ndarray, kx: float, ky: float) -> np.ndarray:
        h = spec.h_periodic(kx, ky, selected.params)
        return h[np.ix_(indices, indices)]

    cup, det_up = fukui_chern(
        lambda kx, ky: block(up_idx, kx, ky),
        spec.n_occ_spin,
        int(nk),
    )
    cdown, det_down = fukui_chern(
        lambda kx, ky: block(down_idx, kx, ky),
        spec.n_occ_spin,
        int(nk),
    )
    return {
        "chern_nk": int(nk),
        "chern_up": float(cup),
        "chern_down": float(cdown),
        "chern_up_int": int(np rint(cup)),
        "chern_down_int": int(np rint(cdown)),
    }

```

```

        "min_link_up": float(det_up),
        "min_link_down": float(det_down),
        "chern_defined": 1,
    }

def high_symmetry_path(
    points_per_segment: int,
) -> tuple[np.ndarray, np.ndarray, list[float], list[str]]:
    named = [
        (r"$\Gamma$", (0.0, 0.0)),
        ("X", (np.pi, 0.0)),
        ("M", (np.pi, np.pi)),
        (r"$\Gamma$", (0.0, 0.0)),
    ]
    klist: list[np.ndarray] = []
    distance: list[float] = []
    ticks = [0.0]
    labels = [named[0][0]]
    current = 0.0
    previous: np.ndarray | None = None
    for iseg in range(len(named) - 1):
        p0 = np.asarray(named[iseg][1], dtype=float)
        p1 = np.asarray(named[iseg + 1][1], dtype=float)
        for j in range(points_per_segment + 1):
            if iseg > 0 and j == 0:
                continue
            t = j / float(points_per_segment)
            k = (1.0 - t) * p0 + t * p1
            if previous is not None:
                current += float(np.linalg.norm(k - previous))
            klist.append(k)
            distance.append(current)
            previous = k
            ticks.append(current)
            labels.append(named[iseg + 1][0])
    return np.asarray(klist), np.asarray(distance), ticks, labels

def calculate_bulk_bands(
    spec: ModelSpec,
    selected: SelectedState,
    ef: float,
    points_per_segment: int,
) -> tuple[pd.DataFrame, list[float], list[str]]:
    klist, distance, ticks, labels = high_symmetry_path(points_per_segment)
    spin_diag = np.zeros(
        spec.h_atomic(0.0, 0.0, selected.params).shape[0], dtype=float
    )
    spin_diag[list(spec.spin_up_indices)] = 1.0
    spin_diag[list(spec.spin_down_indices)] = -1.0
    rows: list[dict[str, float | int]] = []
    for ik, ((kx, ky), xpos) in enumerate(zip(klist, distance)):
        energies, vectors = np.linalg.eigh(
            spec.h_atomic(float(kx), float(ky), selected.params)
        )
        probability = np.abs(vectors) ** 2
        spin = np.sum(probability * spin_diag[:, None], axis=0)
        for ib, energy in enumerate(energies):
            rows.append(
                {
                    "k_index": int(ik),
                    "path_coordinate": float(xpos),
                    "kx": float(kx),
                    "ky": float(ky),
                    "band": int(ib + 1),
                    "energy": float(energy - ef),
                    "spin_z": float(spin[ib]),
                }
            )

```

```

    )
    return pd.DataFrame(rows), ticks, labels

def ribbon_hamiltonian(
    k_parallel: float,
    width: int,
    hoppings: dict[tuple[int, int], np.ndarray],
) -> tuple[np.ndarray, int]:
    h0, hp = axial_blocks(k_parallel, hoppings)
    dim = h0.shape[0]
    h = np.zeros((width * dim, width * dim), dtype=np.complex128)
    for layer in range(width):
        sl = slice(layer * dim, (layer + 1) * dim)
        h[sl, sl] = h0
        if layer + 1 < width:
            sr = slice((layer + 1) * dim, (layer + 2) * dim)
            h[sl, sr] = hp
            h[sr, sl] = hp.conj().T
    return 0.5 * (h + h.conj().T), dim

def calculate_ribbon(
    spec: ModelSpec,
    selected: SelectedState,
    hoppings: dict[tuple[int, int], np.ndarray],
    ef: float,
    k_points: int,
    width: int,
    edge_layers: int = 3,
) -> pd.DataFrame:
    kvalues = np.linspace(-np.pi, np.pi, int(k_points), endpoint=True)
    spin_primitive = np.zeros(
        next(iter(hoppings.values())).shape[0], dtype=float
    )
    spin_primitive[list(spec.spin_up_indices)] = 1.0
    spin_primitive[list(spec.spin_down_indices)] = -1.0
    rows: list[dict[str, float | int]] = []
    for ik, kp in enumerate(kvalues):
        h, dim = ribbon_hamiltonian(float(kp), int(width), hoppings)
        energies, vectors = eigh(h, overwrite_a=True, check_finite=False)
        probability = np.abs(vectors) ** 2
        nedge = min(edge_layers, max(1, width // 3))
        left_weight = np.sum(probability[: nedge * dim], axis=0)
        right_weight = np.sum(probability[(width - nedge) * dim :], axis=0)
        total_edge = left_weight + right_weight
        spin_diag = np.tile(spin_primitive, width)
        spin_z = np.real(np.sum(probability * spin_diag[:, None], axis=0))
        for ib, energy in enumerate(energies):
            rows.append(
                {
                    "k_index": int(ik),
                    "k_parallel": float(kp),
                    "band": int(ib + 1),
                    "energy": float(energy - ef),
                    "left_edge_weight": float(left_weight[ib]),
                    "right_edge_weight": float(right_weight[ib]),
                    "total_edge_weight": float(total_edge[ib]),
                    "spin_z": float(spin_z[ib]),
                }
            )
    return pd.DataFrame(rows)

def sancho_surface_green(
    energy: float,
    eta: float,
    h0: np.ndarray,
    hp: np.ndarray,

```

```

    max_iter: int = 100,
    tolerance: float = 1.0e-12,
) -> np.ndarray:
    z = complex(float(energy), float(eta))
    identity = np.eye(h0.shape[0], dtype=np.complex128)
    eps = h0.copy()
    eps_surface = h0.copy()
    alpha = hp.copy()
    beta = hp.conj().T
    for _ in range(max_iter):
        g = np.linalg.inv(z * identity - eps)
        agb = alpha @ g @ beta
        bga = beta @ g @ alpha
        eps_surface += agb
        eps += agb + bga
        alpha_new = alpha @ g @ alpha
        beta_new = beta @ g @ beta
        alpha, beta = alpha_new, beta_new
        if max(
            float(np.linalg.norm(alpha, ord="fro")),
            float(np.linalg.norm(beta, ord="fro")),
        ) < tolerance:
            break
    return np.linalg.inv(z * identity - eps_surface)

def calculate_surface(
    spec: ModelSpec,
    hoppings: dict[tuple[int, int], np.ndarray],
    ef: float,
    energy_window: float,
    k_points: int,
    energy_points: int,
    eta: float,
) -> dict[str, np.ndarray | float]:
    kvalues = np.linspace(-np.pi, np.pi, int(k_points), endpoint=True)
    energies = np.linspace(-energy_window, energy_window, int(energy_points))
    dim = next(iter(hoppings.values())).shape[0]
    up_mask = np.zeros(dim, dtype=float)
    down_mask = np.zeros(dim, dtype=float)
    up_mask[list(spec.spin_up_indices)] = 1.0
    down_mask[list(spec.spin_down_indices)] = 1.0
    total = np.zeros((len(energies), len(kvalues)), dtype=float)
    up = np.zeros_like(total)
    down = np.zeros_like(total)
    for ik, kp in enumerate(kvalues):
        h0, hp = axial_blocks(float(kp), hoppings)
        for ie, energy_rel in enumerate(energies):
            g = sancho_surface_green(float(energy_rel + ef), eta, h0, hp)
            diagonal = np.maximum(
                0.0, -np.imag(np.diag(g)) / np.pi
            )
            total[ie, ik] = float(np.sum(diagonal))
            up[ie, ik] = float(np.sum(diagonal * up_mask))
            down[ie, ik] = float(np.sum(diagonal * down_mask))
    return {
        "k_parallel": kvalues,
        "energy": energies,
        "surface_dos": total,
        "surface_dos_up": up,
        "surface_dos_down": down,
        "eta": float(eta),
    }

def band_berry_curvature(
    h: np.ndarray,
    vx: np.ndarray,
    vy: np.ndarray,

```

```

    denominator_floor: float = 1.0e-12,
) -> tuple[np.ndarray, np.ndarray]:
    energies, vectors = np.linalg.eigh(h)
    omega = np.zeros(len(energies), dtype=float)
    for ib in range(len(energies)):
        value = 0.0
        for jb in range(len(energies)):
            if ib == jb:
                continue
            de = float(energies[ib] - energies[jb])
            denom = max(de * de, denominator_floor)
            a = np.vdot(vectors[:, ib], vx @ vectors[:, jb])
            b = np.vdot(vectors[:, jb], vy @ vectors[:, ib])
            value += 2.0 * float(np.imag(a * b)) / denom
        omega[ib] = value
    return energies, omega

def calculate_hall(
    spec: ModelSpec,
    hoppings: dict[tuple[int, int], np.ndarray],
    ef: float,
    energy_window: float,
    nk: int,
    energy_points: int,
) -> tuple[pd.DataFrame, dict[str, float]]:
    records: dict[str, list[tuple[float, float]]] = {"up": [], "down": []}
    area_weight = (2.0 * np.pi / nk) ** 2 / (2.0 * np.pi)
    kvalues = -np.pi + 2.0 * np.pi * (np.arange(nk) + 0.5) / nk
    spin_blocks = (
        ("up", np.asarray(spec.spin_up_indices, dtype=int)),
        ("down", np.asarray(spec.spin_down_indices, dtype=int)),
    )
    for kx in kvalues:
        for ky in kvalues:
            h, vx, vy = bloch_from_hoppings(float(kx), float(ky), hoppings)
            for spin_name, indices in spin_blocks:
                es, omega = band_berry_curvature(
                    h[np.ix_(indices, indices)],
                    vx[np.ix_(indices, indices)],
                    vy[np.ix_(indices, indices)],
                )
                for energy, curvature in zip(es, omega):
                    records[spin_name].append(
                        (float(energy - ef), float(curvature * area_weight))
                    )

    energy = np.linspace(-energy_window, energy_window, int(energy_points))
    cumulative: dict[str, np.ndarray] = {}
    for spin_name, values in records.items():
        rec = np.asarray(values, dtype=float)
        order = np.argsort(rec[:, 0])
        e_sorted = rec[order, 0]
        w_sorted = rec[order, 1]
        cumulative_weight = np.cumsum(w_sorted)
        indices = np.searchsorted(e_sorted, energy, side="right") - 1
        curve = np.zeros_like(energy)
        valid = indices >= 0
        curve[valid] = cumulative_weight[indices[valid]]
        cumulative[spin_name] = curve
    charge = cumulative["up"] + cumulative["down"]
    spin = 0.5 * (cumulative["up"] - cumulative["down"])
    df = pd.DataFrame(
        {
            "energy": energy,
            "sigma_xy_up_e2_over_h": cumulative["up"],
            "sigma_xy_down_e2_over_h": cumulative["down"],
            "sigma_xy_charge_e2_over_h": charge,
            "sigma_xy_spin_e_over_2pi": spin,
        }
    )

```

```

    }
)
mid = int(np.argmin(np.abs(energy)))
summary = {
    "response_nk": int(nk),
    "mid_reference_sigma_up": float(cumulative["up"][mid]),
    "mid_reference_sigma_down": float(cumulative["down"][mid]),
    "mid_reference_sigma_charge": float(charge[mid]),
    "mid_reference_sigma_spin": float(spin[mid]),
}
return df, summary

def choose_system_window(
    audits: list[dict[str, Any]],
    system: str,
) -> float:
    direct = [
        abs(float(row["min_direct_gap"]))
        for row in audits
        if row["system"] == system and row["state"] != "critical"
    ]
    base = max(direct, default=0.05)
    minimum = {"lieb": 0.24, "tts": 0.36, "fes": 0.10}[system]
    maximum = {"lieb": 0.55, "tts": 0.60, "fes": 0.24}[system]
    return float(np.clip(2.2 * base + 0.08, minimum, maximum))

def state_title(selected: SelectedState) -> str:
    if selected.expected_chern_up is None:
        return "Critical"
    return rf"{STATE_LABELS[selected.state]} $\rightarrow$ {selected.expected_chern_up:d}$"

def spin_colors(spec: ModelSpec, selected: SelectedState) -> tuple[str, str]:
    if selected.expected_chern_up is None:
        return CRITICAL_COLOR, BODY_DARK
    return (
        phase_color(spec, selected.expected_chern_up),
        phase_color(spec, selected.expected_chern_down),
    )

def surface_rgb(
    spec: ModelSpec,
    selected: SelectedState,
    surface: dict[str, np.ndarray | float],
) -> np.ndarray:
    up = np.log1p(np.asarray(surface["surface_dos_up"], dtype=float))
    down = np.log1p(np.asarray(surface["surface_dos_down"], dtype=float))
    scale = max(float(np.max(up)), float(np.max(down)), 1.0e-14)
    up /= scale
    down /= scale
    intensity = np.maximum(up, down)
    total = np.maximum(up + down, 1.0e-14)
    color_up, color_down = spin_colors(spec, selected)
    cup = np.asarray(to_rgb(color_up), dtype=float)
    cdown = np.asarray(to_rgb(color_down), dtype=float)
    mixed = (
        up[..., None] * cup[None, None, :]
        + down[..., None] * cdown[None, None, :]
    ) / total[..., None]
    white = np.ones_like(mixed)
    rgb = white * (1.0 - intensity[..., None]) + mixed * intensity[..., None]
    return np.clip(rgb, 0.0, 1.0)

def plot_bulk_grid(
    results: list[dict[str, Any]],

```

```

specs: dict[str, ModelSpec],
systems: list[str],
output_stem: Path,
dpi: int,
) -> list[str]:
    fig, axes = plt.subplots(
        len(systems),
        3,
        figsize=(10.7, 3.0 * len(systems)),
        squeeze=False,
        sharey="row",
    )
    for irow, system in enumerate(systems):
        spec = specs[system]
        for icol, state in enumerate(STATE_ORDER):
            result = next(
                x for x in results
                if x["selected"].system == system and x["selected"].state == state
            )
            selected = result["selected"]
            df = result["bulk"]
            ax = axes[irow, icol]
            color_up, color_down = spin_colors(spec, selected)
            for _, band in df.groupby("band"):
                mean_spin = float(band["spin_z"].mean())
                color = color_up if mean_spin >= 0.0 else color_down
                ax.plot(
                    band["path_coordinate"],
                    band["energy"],
                    color=color,
                    lw=1.35,
                    alpha=0.96,
                )
            for tick in result["bulk_ticks"]:
                ax.axvline(tick, color="#B8B8B8", lw=0.65, zorder=0)
            ax.axhline(0.0, color=BLACK, lw=1.0, ls="--")
            ax.set_xticks(result["bulk_ticks"])
            ax.set_xticklabels(result["bulk_labels"])
            ax.set_ylim(-result["window"], result["window"])
            ax.set_title(state_title(selected))
            if icol == 0:
                ax.set_ylabel(
                    f"{spec.label}\n" + r"$E-E_{\mathrm{ref}}$"
                )
            style_axis(ax)
        fig.supxlabel("Bulk momentum path", fontweight="bold", y=0.01)
    fig.tight_layout(rect=(0.0, 0.025, 1.0, 1.0))
    return save_figure(fig, output_stem, dpi)

def plot_ribbon_grid(
    results: list[dict[str, Any]],
    specs: dict[str, ModelSpec],
    systems: list[str],
    output_stem: Path,
    dpi: int,
) -> list[str]:
    fig, axes = plt.subplots(
        len(systems),
        3,
        figsize=(10.7, 3.0 * len(systems)),
        squeeze=False,
        sharex=True,
        sharey="row",
    )
    for irow, system in enumerate(systems):
        spec = specs[system]
        for icol, state in enumerate(STATE_ORDER):
            result = next(

```

```

        x for x in results
        if x["selected"].system == system and x["selected"].state == state
    )
    selected = result["selected"]
    df = result["ribbon"]
    ax = axes[irow, icol]
    visible = df[np.abs(df["energy"]) <= result["window"]]
    ax.scatter(
        visible["k_parallel"],
        visible["energy"],
        s=2.0,
        color=BODY_COLOR,
        alpha=0.40,
        linewidths=0,
        rasterized=True,
    )
    edge = visible[visible["total_edge_weight"] >= 0.28].copy()
    up = edge[edge["spin_z"] >= 0.0]
    down = edge[edge["spin_z"] < 0.0]
    color_up, color_down = spin_colors(spec, selected)
    for frame, color, marker in (
        (up, color_up, "o"),
        (down, color_down, "o"),
    ):
        ax.scatter(
            frame["k_parallel"],
            frame["energy"],
            s=4.0 + 9.0 * frame["total_edge_weight"],
            color=color,
            alpha=0.92,
            marker=marker,
            linewidths=0,
            rasterized=True,
        )
    ax.axhline(0.0, color=BLACK, lw=1.0, ls="--")
    ax.set_xlim(-np.pi, np.pi)
    ax.set_ylim(-result["window"], result["window"])
    ax.set_xticks([-np.pi, 0.0, np.pi])
    ax.set_xticklabels([r"$-\pi$", "0", r"$\pi$"])
    ax.set_title(state_title(selected))
    if icol == 0:
        ax.set_ylabel(
            f"{spec.label}\n" + r"$E-E_{\mathrm{ref}}$"
        )
    style_axis(ax)
    fig.supxlabel(r"$k_{\parallel}$", fontweight="bold", y=0.01)
    fig.tight_layout(rect=(0.0, 0.025, 1.0, 1.0))
    return save_figure(fig, output_stem, dpi)

def plot_surface_grid(
    results: list[dict[str, Any]],
    specs: dict[str, ModelSpec],
    systems: list[str],
    output_stem: Path,
    dpi: int,
) -> list[str]:
    fig, axes = plt.subplots(
        len(systems),
        3,
        figsize=(10.7, 3.0 * len(systems)),
        squeeze=False,
        sharex=True,
        sharey="row",
    )
    for irow, system in enumerate(systems):
        spec = specs[system]
        for icol, state in enumerate(STATE_ORDER):
            result = next(

```

```

        x for x in results
        if x["selected"].system == system and x["selected"].state == state
    )
    selected = result["selected"]
    surface = result["surface"]
    rgb = surface_rgb(spec, selected, surface)
    ax = axes[irow, icol]
    ax.imshow(
        rgb,
        origin="lower",
        extent=[
            -np.pi,
            np.pi,
            -result["window"],
            result["window"],
        ],
        aspect="auto",
        interpolation="nearest",
        rasterized=True,
    )
    ax.axhline(0.0, color=BLACK, lw=1.0, ls="--")
    ax.set_xticks([-np.pi, 0.0, np.pi])
    ax.set_xticklabels([r"$-\pi$", "0", r"$\pi$"])
    ax.set_title(state_title(selected))
    if icol == 0:
        ax.set_ylabel(
            f"{spec.label}\n" + r"$E_{\mathrm{ref}}$"
        )
    style_axis(ax)
    fig.supxlabel(r"$k_{\parallel}$", fontweight="bold", y=0.01)
    fig.tight_layout(rect=(0.0, 0.025, 1.0, 1.0))
    return save_figure(fig, output_stem, dpi)

def plot_hall_grid(
    results: list[dict[str, Any]],
    specs: dict[str, ModelSpec],
    systems: list[str],
    output_stem: Path,
    dpi: int,
) -> list[str]:
    fig, axes = plt.subplots(
        len(systems),
        3,
        figsize=(10.7, 3.0 * len(systems)),
        squeeze=False,
        sharex="row",
        sharey=True,
    )
    for irow, system in enumerate(systems):
        spec = specs[system]
        for icol, state in enumerate(STATE_ORDER):
            result = next(
                x for x in results
                if x["selected"].system == system and x["selected"].state == state
            )
            selected = result["selected"]
            df = result["hall"]
            ax = axes[irow, icol]
            color_up, color_down = spin_colors(spec, selected)
            ax.axvspan(
                -0.04 * result["window"],
                0.04 * result["window"],
                color=lighten(
                    phase_color(spec, selected.expected_chern_up), 0.76
                ),
                alpha=0.65,
                zorder=0,
            )

```

```

ax.plot(
    df["energy"],
    df["sigma_xy_up_e2_over_h"],
    color=color_up,
    lw=2.0,
    label=r"$\sigma_{xy}^{\uparrow}$",
)
ax.plot(
    df["energy"],
    df["sigma_xy_down_e2_over_h"],
    color=color_down,
    lw=2.0,
    label=r"$\sigma_{xy}^{\downarrow}$",
)
ax.plot(
    df["energy"],
    df["sigma_xy_charge_e2_over_h"],
    color=BLACK,
    lw=1.3,
    ls="--",
    label=r"$\sigma_{xy}^{\mathrm{tot}}$",
)
ax.axhline(0.0, color="#777777", lw=0.8)
ax.axvline(0.0, color=BLACK, lw=0.9, ls=":")
ax.set_xlim(-result["window"], result["window"])
ax.set_ylim(-2.35, 2.35)
ax.set_title(state_title(selected))
if icol == 0:
    ax.set_ylabel(
        f"{spec.label}\n" + r"$\sigma_{xy}$ (e2/h)$"
    )
if irow == 0 and icol == 0:
    ax.legend(
        loc="upper left",
        frameon=False,
        handlelength=2.0,
    )
style_axis(ax)
fig.supxlabel(r"$E_{\mathrm{ref}}$", fontweight="bold", y=0.01)
fig.tight_layout(rect=(0.0, 0.025, 1.0, 1.0))
return save_figure(fig, output_stem, dpi)

```

```

def build_settings(quick: bool) -> RunSettings:
    if quick:
        return RunSettings(
            gap_nk=48,
            chern_nk=21,
            band_points_per_segment=28,
            ribbon_k_points=51,
            surface_k_points=51,
            surface_energy_points=71,
            response_nk=25,
            response_energy_points=101,
            fourier_n=8,
            dpi=150,
            quick=True,
        )
    return RunSettings(
        gap_nk=120,
        chern_nk=41,
        band_points_per_segment=70,
        ribbon_k_points=161,
        surface_k_points=141,
        surface_energy_points=181,
        response_nk=81,
        response_energy_points=241,
        fourier_n=8,
        dpi=320,
    )

```

```

        quick=False,
    )

def hall_mesh_for_state(
    selected: SelectedState,
    settings: RunSettings,
) -> int:
    """Use denser Hall meshes only where the Berry curvature is concentrated."""
    if settings.quick:
        return int(settings.response_nk)
    if selected.system == "lieb":
        # The post-t2 sign-flip state has a small generic-valley gap and needs
        # a much denser Kubo mesh to recover the integer plateau.
        return 241 if selected.state == "chern_changed" else 121
    if selected.system == "fes":
        # The topological and critical FES examples have millielectronvolt-
        # scale direct gaps, although the topological point is an indirect
        # overlap metal and therefore has no quantized Fermi-level plateau.
        # At the exact Gamma critical point an odd midpoint mesh contains
        #  $k=(0,0)$ , where the band-resolved Kubo curvature is singular. The
        # neighboring even mesh is the symmetric principal-value convention.
        if selected.state == "critical":
            return 160
        return 161 if selected.state == "topological" else 81
    return 81

def refresh_hall_outputs(
    output_dir: Path,
    quick: bool = False,
) -> dict[str, Any]:
    """Refresh Hall data whose stored mesh differs from the current policy."""
    configure_plot_style()
    settings = build_settings(quick)
    output_dir = Path(output_dir)
    figures_dir = output_dir / "figures"
    data_dir = output_dir / "data"
    audit_path = output_dir / "step18_01_selected_state_audit.csv"
    summary_path = output_dir / "step18_02_hall_summary.csv"
    if not audit_path.is_file():
        raise FileNotFoundError(audit_path)
    specs, modules = build_model_specs()
    selected_states = select_all_states(modules)
    audit_df = pd.read_csv(audit_path)
    old_summary = (
        pd.read_csv(summary_path)
        if summary_path.is_file()
        else pd.DataFrame()
    )
    old_lookup = {
        (str(row.system), str(row.state)): row
        for row in old_summary.itertuples(index=False)
    }
    audit_records = audit_df.to_dict(orient="records")
    windows = {
        system: choose_system_window(audit_records, system)
        for system in ("lieb", "tts", "fes")
    }
    results: list[dict[str, Any]] = []
    summaries: list[dict[str, Any]] = []
    refreshed: list[str] = []
    for selected in selected_states:
        spec = specs[selected.system]
        desired_nk = hall_mesh_for_state(selected, settings)
        sample_dir = data_dir / f"{selected.system}_{selected.state}"
        hall_path = sample_dir / "wanniertools_style_hall.csv"
        old = old_lookup.get((selected.system, selected.state))
        stored_nk = int(old.response_nk) if old is not None else -1

```

```

audit = audit_df[
    (audit_df["system"].astype(str) == selected.system)
    & (audit_df["state"].astype(str) == selected.state)
].iloc[0]
ef = float(audit["energy_reference"])
window = float(windows[selected.system])
if hall_path.is_file() and stored_nk == desired_nk:
    hall = pd.read_csv(hall_path)
else:
    hoppings = extract_hoppings(
        spec,
        selected.params,
        nfft=settings.fourier_n,
    )
    hall, _ = calculate_hall(
        spec,
        hoppings,
        ef,
        window,
        desired_nk,
        settings.response_energy_points,
    )
    hall.to_csv(hall_path, index=False)
    refreshed.append(f"{selected.system}:{selected.state}")
mid = int(np.argmax(np.abs(hall["energy"]).to_numpy(float)))
summaries.append(
    {
        "system": selected.system,
        "state": selected.state,
        "source_id": selected.source_id,
        "expected_chern_up": selected.expected_chern_up,
        "response_nk": int(desired_nk),
        "mid_reference_sigma_up": float(
            hall["sigma_xy_up_e2_over_h"].iloc[mid]
        ),
        "mid_reference_sigma_down": float(
            hall["sigma_xy_down_e2_over_h"].iloc[mid]
        ),
        "mid_reference_sigma_charge": float(
            hall["sigma_xy_charge_e2_over_h"].iloc[mid]
        ),
        "mid_reference_sigma_spin": float(
            hall["sigma_xy_spin_e_over_2pi"].iloc[mid]
        ),
    }
)
results.append(
    {
        "selected": selected,
        "window": window,
        "hall": hall,
    }
)
pd.DataFrame(summaries).to_csv(summary_path, index=False)
plot_hall_grid(
    results,
    specs,
    ["lieb", "tts", "fes"],
    figures_dir / "step18_three_system_hall_trptychs",
    settings.dpi,
)
for system in ("lieb", "tts", "fes"):
    plot_hall_grid(
        results,
        specs,
        [system],
        figures_dir / f"step18_{system}_hall_trptych",
        settings.dpi,
    )

```

```

certificate_path = output_dir / "step18_04_final_certificate.json"
if certificate_path.is_file():
    certificate = json.loads(certificate_path.read_text(encoding="utf-8"))
    certificate["hall_refresh_utc"] = utc_now()
    certificate["hall_refreshed_states"] = refreshed
    certificate["critical_hall_policy"] = (
        "The exact FES Gamma critical state uses an even 160x160 midpoint "
        "mesh, which excludes the singular closing momentum and implements "
        "the symmetric principal-value Kubo limit. Critical Hall values are "
        "not topological plateaus."
    )
    certificate_path.write_text(
        json.dumps(certificate, indent=2, ensure_ascii=False),
        encoding="utf-8",
    )
return {"refreshed": refreshed, "summary_rows": len(summaries)}

def run(
    output_dir: Path,
    quick: bool = False,
) -> dict[str, Any]:
    started = time.time()
    configure_plot_style()
    settings = build_settings(quick)
    output_dir = Path(output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)
    figures_dir = output_dir / "figures"
    data_dir = output_dir / "data"
    figures_dir.mkdir(parents=True, exist_ok=True)
    data_dir.mkdir(parents=True, exist_ok=True)

    specs, modules = build_model_specs()
    selected_states = select_all_states(modules)
    if len(selected_states) != 9:
        raise RuntimeError(f"Expected exactly 9 selected states, got {len(selected_states)}")

    print("[1/6] Gap and Chern audits for nine selected states", flush=True)
    audits: list[dict[str, Any]] = []
    for selected in selected_states:
        spec = specs[selected.system]
        gap = gap_audit(spec, selected, settings.gap_nk)
        chern = chern_audit(spec, selected, settings.chern_nk)
        audit = {
            "system": selected.system,
            "state": selected.state,
            "source_id": selected.source_id,
            "path_id": selected.path_id,
            "expected_chern_up": selected.expected_chern_up,
            "expected_chern_down": selected.expected_chern_down,
            "critical_kx": selected.critical_kx,
            "critical_ky": selected.critical_ky,
            "selection_note": selected.selection_note,
            **selected.params,
            **gap,
            **chern,
        }
        if selected.expected_chern_up is None:
            audit["chern_matches_selection"] = np.nan
            audit["critical_gap_pass"] = int(
                float(gap["critical_valley_gap"]) < 2.0e-5
            )
        else:
            audit["chern_matches_selection"] = int(
                int(chern["chern_up_int"]) == selected.expected_chern_up
                and int(chern["chern_down_int"]) == selected.expected_chern_down
            )
            audit["critical_gap_pass"] = np.nan
        audits.append(audit)

```

```

        print(
            f" {selected.system:4s} {selected.state:13s} "
            f"gap={float(gap['min_direct_gap']):.4e} "
            f"indirect={float(gap['indirect_gap']):.4e} "
            f"Cup={chern['chern_up_int']}",
            flush=True,
        )
    )
    audit_df = pd.DataFrame(audits)
    audit_df.to_csv(output_dir / "step18_01_selected_state_audit.csv", index=False)

    windows = {
        system: choose_system_window(audits, system)
        for system in ("lieb", "tts", "fes")
    }

    print("[2/6] Wannier hopping reconstruction and bulk bands", flush=True)
    results: list[dict[str, Any]] = []
    hall_summaries: list[dict[str, Any]] = []
    for index, selected in enumerate(selected_states, start=1):
        spec = specs[selected.system]
        audit = next(
            row for row in audits
            if row["system"] == selected.system and row["state"] == selected.state
        )
        ef = float(audit["energy_reference"])
        window = float(windows[selected.system])
        sample_dir = data_dir / f"{selected.system}_{selected.state}"
        sample_dir.mkdir(parents=True, exist_ok=True)

        print(
            f" [{index}/9] {selected.system} {selected.state}: Fourier/bulk",
            flush=True,
        )
        hoppings = extract_hoppings(
            spec, selected.params, nfft=settings.fourier_n
        )
        hopping_rows = [
            {
                "rx": rx,
                "ry": ry,
                "max_abs_hopping": float(np.max(np.abs(mat))),
            }
            for (rx, ry), mat in sorted(hoppings.items())
        ]
        pd.DataFrame(hopping_rows).to_csv(
            sample_dir / "wannier_hopping_summary.csv", index=False
        )
        bulk, ticks, labels = calculate_bulk_bands(
            spec,
            selected,
            ef,
            settings.band_points_per_segment,
        )
        bulk.to_csv(sample_dir / "bulk_bands.csv", index=False)

        width = spec.ribbon_width
        if quick:
            width = min(width, 18 if selected.system == "lieb" else 16)
        print(
            f" [{index}/9] {selected.system} {selected.state}: ribbon width={width}",
            flush=True,
        )
        ribbon = calculate_ribbon(
            spec,
            selected,
            hoppings,
            ef,
            settings.ribbon_k_points,
            width,

```

```

    )
    ribbon.to_csv(
        sample_dir / "wanniertools_style_ribbon.csv.gz",
        index=False,
        compression="gzip",
    )

    eta = max(
        5.0e-4 if selected.system == "fes" else 1.5e-3,
        0.006 * window,
    )
    print(
        f" [{index}/9] {selected.system} {selected.state}: "
        f"semi-infinite surface eta={eta:.3e}",
        flush=True,
    )
    surface = calculate_surface(
        spec,
        hoppings,
        ef,
        window,
        settings.surface_k_points,
        settings.surface_energy_points,
        eta,
    )
    np.savez_compressed(
        sample_dir / "wanniertools_style_surface_spectrum.npz",
        **surface,
    )

    print(
        f" [{index}/9] {selected.system} {selected.state}: Hall response",
        flush=True,
    )
    hall_nk = hall_mesh_for_state(selected, settings)
    hall, hall_summary = calculate_hall(
        spec,
        hoppings,
        ef,
        window,
        hall_nk,
        settings.response_energy_points,
    )
    hall.to_csv(sample_dir / "wanniertools_style_hall.csv", index=False)
    hall_summaries.append(
        {
            "system": selected.system,
            "state": selected.state,
            "source_id": selected.source_id,
            "expected_chern_up": selected.expected_chern_up,
            **hall_summary,
        }
    )
    results.append(
        {
            "selected": selected,
            "audit": audit,
            "window": window,
            "bulk": bulk,
            "bulk_ticks": ticks,
            "bulk_labels": labels,
            "ribbon": ribbon,
            "surface": surface,
            "hall": hall,
            "width": int(width),
        }
    )

pd.DataFrame(hall_summaries).to_csv(

```

```

    output_dir / "step18_02_hall_summary.csv", index=False
)

print("[3/6] Three-system publication figures", flush=True)
figure_manifest: list[dict[str, str]] = []
plotters = (
    ("bulk", plot_bulk_grid),
    ("ribbon", plot_ribbon_grid),
    ("surface", plot_surface_grid),
    ("hall", plot_hall_grid),
)
for kind, plotter in plotters:
    paths = plotter(
        results,
        specs,
        ["lieb", "tts", "fes"],
        figures_dir / f"step18_three_system_{kind}_trptychs",
        settings.dpi,
    )
    for path in paths:
        figure_manifest.append(
            {"scope": "three_system", "kind": kind, "path": path}
        )

print("[4/6] Individual-system figures", flush=True)
for system in ("lieb", "tts", "fes"):
    for kind, plotter in plotters:
        paths = plotter(
            results,
            specs,
            [system],
            figures_dir / f"step18_{system}_{kind}_trptych",
            settings.dpi,
        )
        for path in paths:
            figure_manifest.append(
                {"scope": system, "kind": kind, "path": path}
            )
pd.DataFrame(figure_manifest).to_csv(
    output_dir / "step18_03_figure_manifest.csv", index=False
)

print("[5/6] Physics and color certificates", flush=True)
noncritical = audit_df[audit_df["state"] != "critical"]
critical = audit_df[audit_df["state"] == "critical"]
certificate = {
    "code_version": CODE_VERSION,
    "created_utc": utc_now(),
    "quick": bool(quick),
    "selected_state_count": int(len(audit_df)),
    "exactly_three_states_per_system": bool(
        audit_df.groupby("system")["state"].nunique().eq(3).all()
    ),
    "noncritical_chern_match": bool(
        noncritical["chern_matches_selection"].fillna(0).astype(int).eq(1).all()
    ),
    "critical_gap_closure_pass": bool(
        critical["critical_gap_pass"].fillna(0).astype(int).eq(1).all()
    ),
    "fes_topological_global_insulator": bool(
        audit_df[
            (audit_df["system"] == "fes")
            & (audit_df["state"] == "topological")
        ][["is_global_insulator"]].iloc[0]
    ),
    "fes_topological_classification": (
        "global_insulator"
        if bool(
            audit_df[

```

```

        (audit_df["system"] == "fes")
        & (audit_df["state"] == "topological")
    ][["is_global_insulator"].iloc[0]
    )
    else "spin_chern_band_metal"
),
"palettes": {
    "lieb": LIEB_FES_COLORS,
    "fes": LIEB_FES_COLORS,
    "tts": TTS_COLORS,
    "critical": CRITICAL_COLOR,
    "bulk": BODY_COLOR,
},
"color_policy": (
    "Exact phase-map strong colors for spin-resolved edge and Hall data; "
    "critical spectra use neutral dark gray; bulk states use light gray."
),
"energy_reference_policy": {
    "lieb": "global midpoint between VBM and CBM",
    "tts": "global midpoint between VBM and CBM",
    "fes": (
        "local midpoint at the selected Gamma transition valley because "
        "the topological representative is an indirect-overlap band metal"
    ),
    "critical": "critical closing energy",
},
"hall_units": {
    "spin_resolved_and_charge": "e2/h",
    "spin_hall": "e/(2*pi)",
},
"run_settings": settings.__dict__,
"elapsed_seconds": float(time.time() - started),
}
with (output_dir / "step18_04_final_certificate.json").open(
    "w", encoding="utf-8"
) as handle:
    json.dump(certificate, handle, indent=2, ensure_ascii=False)

print("[6/6] Complete", flush=True)
print(json.dumps(certificate, indent=2, ensure_ascii=False), flush=True)
return certificate

```

```

def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument(
        "--output-dir",
        type=Path,
        default=DEFAULT_OUTPUT,
    )
    parser.add_argument("--quick", action="store_true")
    parser.add_argument(
        "--refresh-hall-only",
        action="store_true",
        help="Reuse existing nine-state outputs and refresh Hall data/figures only.",
    )
    return parser

```

```

def main() -> int:
    args = build_parser().parse_args()
    if args.refresh_hall_only:
        result = refresh_hall_outputs(
            Path(args.output_dir),
            quick=bool(args.quick),
        )
        print(json.dumps(result, indent=2, ensure_ascii=False))
    else:
        run(Path(args.output_dir), quick=bool(args.quick))

```

```

    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

► 代码清单 A.51.2 (python, 1984 行)

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Step 19: main-text package with four non-trivial edge/Hall examples.

The main-text selection is intentionally compact:

    (a) Lieb, C_s = +1
    (b) FES, C_s = +1
    (c) TTS, C_s = +1
    (d) TTS, C_s = +2

Critical and trivial examples from Step 18 remain available as supplementary
material and are not repeated here. Each main-text case contains a
WannierTools-style semi-infinite edge spectral function, spin-resolved Hall
conductivity, bulk-band data, the exact parameter vector and validation
metadata. Plot colors are inherited from the phase-map palettes in Step 18.
"""

from __future__ import annotations

from pathlib import Path
from typing import Any
import argparse
import base64
import json
import re
import struct
import time

import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap, to_rgb
from matplotlib.lines import Line2D
from matplotlib.patches import Rectangle
from mpl_toolkits.axes_grid1.inset_locator import inset_axes
import numpy as np
import pandas as pd

import TTS_step18_three_system_wanniertools_style_edge_ahc_examples as core

CODE_VERSION = "TTS_STEP19_MAINTTEXT_FOUR_TOPOLOGICAL_V1_20260730"
DEFAULT_OUTPUT = (
    core.DEFAULT_OUTPUT / "main_text_four_topological"
)
CASE_ORDER = (
    "a_lieb_Cs_plus1",
    "b_fes_Cs_plus1",
    "c_tts_Cs_plus1",
    "d_tts_Cs_plus2",
)
PANEL_LABEL = {
    "a_lieb_Cs_plus1": "(a)",
    "b_fes_Cs_plus1": "(b)",
    "c_tts_Cs_plus1": "(c)",

```

```

    "d_tts-Cs_plus2": "(d)",
}
DISPLAY_TITLE = {
    "a_lieb-Cs_plus1": r"Lieb: $C_s=+1$",
    "b_fes-Cs_plus1": r"FES: $C_s=+1$",
    "c_tts-Cs_plus1": r"TTS: $C_s=+1$",
    "d_tts-Cs_plus2": r"TTS: $C_s=+2$",
}

def configure_step19_plot_style() -> None:
    """Times New Roman throughout, with bold italic/roman math preserved."""
    core.configure_plot_style()
    plt.rcParams.update(
        {
            "font.family": "Times New Roman",
            "font.weight": "bold",
            "axes.labelweight": "bold",
            "axes.titleweight": "bold",
            "mathtext.fontset": "custom",
            "mathtext.rm": "Times New Roman:bold",
            "mathtext.it": "Times New Roman:italic:bold",
            "mathtext.bf": "Times New Roman:bold",
            "mathtext.sf": "Times New Roman:bold",
            "mathtext.tt": "Times New Roman:bold",
            "mathtext.fallback": "stix",
            "svg.fonttype": "none",
            "axes.unicode_minus": True,
        }
    )

```

```

def save_figure_inkscape_safe(
    fig: plt.Figure,
    stem: Path,
    dpi: int,
    primary_svg_editable: bool = False,
) -> list[str]:
    """Save a complete SVG plus a separately editable Matplotlib SVG.

```

Some Inkscape builds intermittently omit one or more of Matplotlib's embedded raster layers when a figure contains several surface-spectrum images and inset colour bars. The primary SVG therefore contains one flattened, self-contained PNG layer and also carries a relative-file fallback for older Inkscape versions. The companion ``\*\_editable.svg`` retains vector text, axes, and bulk-band curves.

```

    """
    stem.parent.mkdir(parents=True, exist_ok=True)
    png_path = stem.with_suffix(".png")
    pdf_path = stem.with_suffix(".pdf")
    svg_path = stem.with_suffix(".svg")
    editable_svg_path = stem.with_name(stem.name + "_editable").with_suffix(
        ".svg"
    )
    save_kwargs = {
        "dpi": dpi,
        "bbox_inches": "tight",
        "pad_inches": 0.04,
        "facecolor": "white",
    }
    fig.savefig(png_path, **save_kwargs)
    fig.savefig(pdf_path, **save_kwargs)
    fig.savefig(editable_svg_path, **save_kwargs)

    if primary_svg_editable:
        fig.savefig(svg_path, **save_kwargs)
        make_svg_raster_layers_inkscape_compatible(svg_path)
    else:
        png_bytes = png_path.read_bytes()

```

```

if png_bytes[:8] != b"\x89PNG\r\n\x1a\n":
    raise RuntimeError(f"Unexpected PNG header: {png_path}")
width, height = struct.unpack(">II", png_bytes[16:24])
encoded = base64.b64encode(png_bytes).decode("ascii")
absolute_png = str(png_path.resolve()).replace("\\", "/")
svg_text = (
    '<?xml version="1.0" encoding="UTF-8" standalone="no"?>\n'
    '<svg xmlns="http://www.w3.org/2000/svg"\n'
    '    xmlns:xlink="http://www.w3.org/1999/xlink"\n'
    '    xmlns:sodipodi="http://sodipodi.sourceforge.net/DTD/'
    'sodipodi-0.dtd"\n'
    f'        width="{width}" height="{height}" '
    f'viewBox="0 0 {width} {height}">\n'
    '    <title>Inkscape-compatible complete figure</title>\n'
    f'    <image x="0" y="0" width="{width}" height="{height}"\n'
    '        preserveAspectRatio="xMidYMid meet"\n'
    f'        href="data:image/png;base64,{encoded}"\n'
    f'        xlink:href="{png_path.name}"\n'
    f'        sodipodi:absref="{absolute_png}">/>\n'
    '</svg>\n'
)
svg_path.write_text(svg_text, encoding="utf-8")
plt.close(fig)
return [
    str(png_path),
    str(pdf_path),
    str(svg_path),
    str(editable_svg_path),
]

def make_svg_raster_layers_inkscape_compatible(svg_path: Path) -> list[Path]:
    """Give each scientific raster layer an external fallback for Inkscape."""
    svg_text = svg_path.read_text(encoding="utf-8")
    pattern = re.compile(
        r'xlink:href="data:image/png;base64,\s*([^\s]+)"',
        flags=re.DOTALL,
    )
    layer_paths: list[Path] = []
    layer_index = 0

    def replace_layer(match: re.Match[str]) -> str:
        nonlocal layer_index
        layer_index += 1
        encoded = re.sub(r"\s+", "", match.group(1))
        layer_path = svg_path.with_name(
            f"{svg_path.stem}_raster_layer_{layer_index:02d}.png"
        )
        layer_path.write_bytes(base64.b64decode(encoded))
        layer_paths.append(layer_path)
        return (
            f'href="data:image/png;base64,{encoded}" '
            f'xlink:href="{layer_path.name}" '
        )

    updated = pattern.sub(replace_layer, svg_text)
    if layer_index == 0:
        raise RuntimeError(f"No raster layer found in editable SVG: {svg_path}")
    # Keep STIXGeneral on MathText accent/symbol glyphs. Matplotlib encodes the
    # overbar and parallel sign as STIX private-use glyphs; changing their font
    # family makes Inkscape display a circled stroke or a short minus instead.
    svg_path.write_text(updated, encoding="utf-8")
    return layer_paths

def save_editable_svg_only(
    fig: plt.Figure,
    svg_path: Path,
    dpi: int,

```

```

) -> list[str]:
    """Update only one editable SVG, leaving PNG/PDF/other figures untouched."""
    svg_path.parent.mkdir(parents=True, exist_ok=True)
    fig.savefig(
        svg_path,
        dpi=dpi,
        bbox_inches="tight",
        pad_inches=0.04,
        facecolor="white",
    )
    make_svg_raster_layers_inkscape_compatible(svg_path)
    plt.close(fig)
    return [str(svg_path)]

def clean_params(values: dict[str, float], names: tuple[str, ...]) -> dict[str, float]:
    return {name: float(values[name]) for name in names}

def select_lieb_main(lieb: Any) -> core.SelectedState:
    master_path = (
        core.LIEB_ROOT
        / "outputs_step08_analytic_boundary_wanniertools_edge_msg123342_lieb8_from_step06_n1024_pairs4_seed202607"
        ↪ 11"
        / "step08_01_analytic_feature_master.csv"
    )
    master = pd.read_csv(master_path)
    candidates = master[
        (pd.to_numeric(master["chern_reliable"], errors="coerce") == 1)
        & (pd.to_numeric(master["gap_convergence_ok"], errors="coerce") == 1)
        & (pd.to_numeric(master["chern_up_int"], errors="coerce") == 1)
        & (pd.to_numeric(master["final_indirect_gap"], errors="coerce") > 0)
    ].copy()
    if candidates.empty:
        raise RuntimeError("No reliable insulating Lieb Cs=+1 candidate")
    # The largest-gap candidate (about 1.88 eV) makes the bulk/edge panel look
    # artificially empty. Select a clean, converged intermediate-gap example
    # close to the roughly 0.5 eV gap scale in the user's reference figure.
    target_gap = 0.50
    candidates["display_gap_score"] = (
        (
            pd.to_numeric(candidates["final_indirect_gap"], errors="coerce")
            - target_gap
        ).abs()
        + 0.25
        * (
            pd.to_numeric(candidates["final_direct_gap"], errors="coerce")
            - pd.to_numeric(
                candidates["final_indirect_gap"], errors="coerce"
            )
        ).abs()
    )
    row = candidates.sort_values(
        ["display_gap_score", "final_direct_gap"],
        ascending=[True, False],
    ).iloc[0]
    physical = [float(row[name]) for name in lieb.PHYS7]
    params = clean_params(lieb.phys7_to_raw8(physical), tuple(lieb.RAW8))
    return core.SelectedState(
        system="lieb",
        state="topological",
        source_id=str(row["sample_id"]),
        path_id="lieb_step08_intermediate_gap-Cs_plus1_near_0p5eV",
        params=params,
        expected_chern_up=1,
        expected_chern_down=-1,
        critical_kx=float(row["refined_direct_kx"]),
        critical_ky=float(row["refined_direct_ky"]),
    )

```

```

        selection_note=(
            "Reliable converged Lieb C_s=+1 insulator selected near a 0.5 eV "
            "indirect gap for visual comparability with the reference panel; "
            "the previous maximum-gap example was intentionally not used."
        ),
    )
)

def select_fes_main(fes: Any) -> core.SelectedState:
    strict_path = (
        core.FES_ROOT
        / "outputs_fes6_step04_boundary"
        / "fes_step04_strict_verified_results.csv"
    )
    strict = pd.read_csv(strict_path)
    candidates = strict[
        (pd.to_numeric(strict["verified_chern_reliable"], errors="coerce") == 1)
        & (pd.to_numeric(strict["verified_chern_up_int"], errors="coerce") == 1)
    ].copy()
    if candidates.empty:
        raise RuntimeError("No reliable FES C_s=+1 candidate")
    row = candidates.sort_values(
        ["verified_min_direct_gap", "verified_indirect_gap"],
        ascending=False,
    ).iloc[0]
    reduced = {
        name: float(row[name])
        for name in ("m_e", "t1", "t2", "r1", "r2")
    }
    params = clean_params(fes.raw6_from_reduced5(reduced), tuple(fes.RAW6))
    return core.SelectedState(
        system="fes",
        state="topological",
        source_id=str(row["point_id"]),
        path_id="fes_step04_largest_verified_direct_gap-Cs_plus1",
        params=params,
        expected_chern_up=1,
        expected_chern_down=-1,
        critical_kx=float(row["verified_direct_gap_kx"]),
        critical_ky=float(row["verified_direct_gap_ky"]),
        selection_note=(
            "Largest verified direct gap among reliable FES C_s=+1 samples. "
            "Its negative indirect gap makes it a spin-Chern band metal, not "
            "a globally insulating quantized Hall plateau."
        ),
    )
)

def select_tts_main(tts: Any, chern_up: int) -> core.SelectedState:
    representatives_path = (
        core.TTS_ROOT
        / "outputs_tts_step07_observable_validation"
        / "step07_01_selected_observable_representatives.csv"
    )
    representatives = pd.read_csv(representatives_path)
    candidates = representatives[
        pd.to_numeric(
            representatives["expected_chern_up"], errors="coerce"
        ) == int(chern_up)
    ]
    if candidates.empty:
        raise RuntimeError(f"No TTS C_s={chern_up:+d} representative")
    row = candidates.sort_values(
        ["indirect_gap", "min_direct_gap"],
        ascending=False,
    ).iloc[0]
    reduced = {
        name: float(row[name])
        for name in tts.REDUCED7
    }

```

```

    }
    params = clean_params(tts.raw8_from_reduced7(reduced), tuple(tts.RAW8))
    return core.SelectedState(
        system="tts",
        state="topological",
        source_id=str(row["sample_id"]),
        path_id=f"tts_step07_largest_strict_indirect_gap-Cs_{chern_up:+d}",
        params=params,
        expected_chern_up=int(chern_up),
        expected_chern_down=-int(chern_up),
        critical_kx=float(row["direct_gap_kx"]),
        critical_ky=float(row["direct_gap_ky"]),
        selection_note=str(row["selection_reason"]),
    )

def select_main_cases(modules: dict[str, Any]) -> list[dict[str, Any]]:
    cases = [
        {
            "case_id": "a_lieb-Cs_plus1",
            "selected": select_lieb_main(modules["lieb"]),
        },
        {
            "case_id": "b_fes-Cs_plus1",
            "selected": select_fes_main(modules["fes"]),
        },
        {
            "case_id": "c_tts-Cs_plus1",
            "selected": select_tts_main(modules["tts"], +1),
        },
        {
            "case_id": "d_tts-Cs_plus2",
            "selected": select_tts_main(modules["tts"], +2),
        },
    ]
    if tuple(case["case_id"] for case in cases) != CASE_ORDER:
        raise RuntimeError("Unexpected main-text case ordering")
    return cases

def choose_case_window(system: str, direct_gap: float) -> float:
    if system == "lieb":
        return float(np.clip(0.62 * direct_gap + 0.08, 0.55, 1.20))
    if system == "fes":
        return 0.10
    return 0.60

def classification(audit: dict[str, Any]) -> str:
    if int(audit["is_global_insulator"]) == 1:
        return "spin_chern_insulator"
    if int(audit["is_direct_gapped"]) == 1:
        return "spin_chern_band_metal"
    return "gapless"

def title_for(case_id: str, panel: bool = True) -> str:
    prefix = PANEL_LABEL[case_id] + " " if panel else ""
    return prefix + DISPLAY_TITLE[case_id]

def draw_surface(
    ax: plt.Axes,
    result: dict[str, Any],
    specs: dict[str, core.ModelSpec],
    title: str,
) -> None:
    selected = result["selected"]
    spec = specs[selected.system]

```

```

rgb = core.surface_rgb(spec, selected, result["surface"])
ax.imshow(
    rgb,
    origin="lower",
    extent=[
        -np.pi,
        np.pi,
        -result["window"],
        result["window"],
    ],
    aspect="auto",
    interpolation="nearest",
    rasterized=True,
)
ax.axhline(0.0, color=core.BLACK, lw=1.0, ls="--")
ax.set_xlim(-np.pi, np.pi)
ax.set_ylim(-result["window"], result["window"])
ax.set_xticks([-np.pi, 0.0, np.pi])
ax.set_xticklabels([r"$-\pi$", "0", r"$\pi$"])
ax.set_xlabel(r"$k_{\parallel}$", fontweight="bold")
ax.set_ylabel(
    r"${E-E_{\mathrm{ref}}}\backslash \mathrm{(eV)}$",
    fontweight="bold",
)
ax.set_title(title, fontweight="bold")
core.style_axis(ax)
# A taller-than-wide box shortens the physical horizontal axis without
# changing the displayed momentum interval [-pi, pi].
ax.set_box_aspect(1.08)

def draw_hall(
    ax: plt.Axes,
    result: dict[str, Any],
    specs: dict[str, core.ModelSpec],
    title: str,
    show_legend: bool,
) -> None:
    selected = result["selected"]
    spec = specs[selected.system]
    hall = result["hall"]
    color_up, color_down = core.spin_colors(spec, selected)
    phase = core.phase_color(spec, selected.expected_chn_up)
    audit = result["audit"]
    ef = float(audit["energy_reference"])
    if int(audit["is_global_insulator"]) == 1:
        interval_low = float(audit["vbm"]) - ef
        interval_high = float(audit["cbm"]) - ef
    else:
        # FES is an indirect-overlap metal. Highlight only its local direct
        # gap around the reference valley; do not imply a global bulk gap.
        half_local_gap = 0.5 * float(audit["critical_valley_gap"])
        interval_low = -half_local_gap
        interval_high = +half_local_gap
    ax.axvspan(
        interval_low,
        interval_high,
        color=core.lighten(phase, 0.76),
        alpha=0.65,
        zorder=0,
    )
    ax.plot(
        hall["energy"],
        hall["sigma_xy_up_e2_over_h"],
        color=color_up,
        lw=2.0,
        label=r"$\sigma_{xy}^{\uparrow}$",
    )
    ax.plot(

```

```

        hall["energy"],
        hall["sigma_xy_down_e2_over_h"],
        color=color_down,
        lw=2.0,
        label=r"$\sigma_{xy}^{\downarrow}$",
    )
    ax.axhline(0.0, color="#777777", lw=0.8)
    ax.axvline(0.0, color=core.BLACK, lw=0.9, ls=":")
    ax.set_xlim(-result["window"], result["window"])
    ax.set_ylim(-2.35, 2.35)
    ax.set_xlabel(r"$E-E_{\mathrm{ref}}$", fontweight="bold")
    ax.set_ylabel(r"$\sigma_{xy} \backslash (e^2/h)$", fontweight="bold")
    ax.set_title(title, fontweight="bold")
    if show_legend:
        ax.legend(loc="best", frameon=False, handlelength=2.0)
    core.style_axis(ax)
    ax.set_box_aspect(1.08)

def plot_individual_pair(
    result: dict[str, Any],
    specs: dict[str, core.ModelSpec],
    stem: Path,
    dpi: int,
) -> list[str]:
    fig, axes = plt.subplots(1, 2, figsize=(5.85, 3.35))
    case_id = result["case_id"]
    draw_surface(axes[0], result, specs, title_for(case_id, panel=False))
    draw_hall(axes[1], result, specs, "Spin-resolved Hall response", True)
    fig.tight_layout(w_pad=1.25)
    return save_figure_inkscape_safe(fig, stem, dpi)

def plot_surface_four(
    results: list[dict[str, Any]],
    specs: dict[str, core.ModelSpec],
    stem: Path,
    dpi: int,
) -> list[str]:
    fig, axes = plt.subplots(2, 2, figsize=(5.85, 6.65))
    for ax, result in zip(axes.flat, results):
        draw_surface(
            ax,
            result,
            specs,
            title_for(result["case_id"]),
        )
    fig.tight_layout(h_pad=1.0, w_pad=1.05)
    return save_figure_inkscape_safe(fig, stem, dpi)

def plot_hall_four(
    results: list[dict[str, Any]],
    specs: dict[str, core.ModelSpec],
    stem: Path,
    dpi: int,
) -> list[str]:
    fig, axes = plt.subplots(2, 2, figsize=(5.85, 6.65), sharey=True)
    for index, (ax, result) in enumerate(zip(axes.flat, results)):
        draw_hall(
            ax,
            result,
            specs,
            title_for(result["case_id"]),
            show_legend=(index == 0),
        )
    fig.tight_layout(h_pad=1.0, w_pad=1.05)
    return save_figure_inkscape_safe(fig, stem, dpi)

```

```

def principal_edge_window(result: dict[str, Any]) -> float:
    """Return an energy window focused on the edge states near the main gap."""
    audit = result["audit"]
    selected = result["selected"]
    surface_energy = np.asarray(result["surface"]["energy"], dtype=float)
    full_window = float(np.max(np.abs(surface_energy)))
    if selected.system in ("lieb", "tts"):
        return min(1.5, full_window)
    if int(audit["is_global_insulator"]) == 1:
        ef = float(audit["energy_reference"])
        half_gap_extent = max(
            abs(float(audit["vbm"]) - ef),
            abs(float(audit["cbm"]) - ef),
        )
        if result["selected"].system == "lieb":
            focused = 1.25 * half_gap_extent
        else:
            # Include appreciable projected bulk-band background outside the
            # topological gap while keeping the edge branches visually central.
            focused = max(0.42, 2.05 * half_gap_extent)
    else:
        # The FES example is an indirect-overlap metal. Focus on the local
        # direct-gap neighborhood plus enough surrounding projected bulk bands.
        focused = max(0.075, 12.0 * float(audit["critical_valley_gap"]))
    return float(min(full_window, focused))

def edge_spin_colors(
    selected: core.SelectedState | None = None,
) -> tuple[str, str]:
    """Use a fixed red/blue convention for spin-up/spin-down edge spectra."""
    del selected
    return core.LIEB_FES_COLORS[+1], core.LIEB_FES_COLORS[-1]

def edge_spin_colormap() -> LinearSegmentedColormap:
    """WannierTools-style spin map: blue -1, white 0, red +1."""
    color_up, color_down = edge_spin_colors()
    return LinearSegmentedColormap.from_list(
        "edge_spin_down_white_up",
        [color_down, "white", color_up],
        N=256,
    )

def edge_surface_rgb(
    selected: core.SelectedState,
    surface: dict[str, np.ndarray | float],
) -> np.ndarray:
    """Use the project's WannierTools-style spin-resolved spectral mixing."""
    visible_up = np.loglp(
        np.asarray(surface["surface_dos_up"], dtype=float)
    )
    visible_down = np.loglp(
        np.asarray(surface["surface_dos_down"], dtype=float)
    )
    scale = max(
        float(np.max(visible_up)),
        float(np.max(visible_down)),
        1.0e-14,
    )
    visible_up /= scale
    visible_down /= scale
    intensity = np.maximum(visible_up, visible_down)
    total = np.maximum(visible_up + visible_down, 1.0e-14)
    color_up, color_down = edge_spin_colors(selected)
    rgb_up = np.asarray(to_rgb(color_up), dtype=float)
    rgb_down = np.asarray(to_rgb(color_down), dtype=float)

```

```

mixed = (
    visible_up[..., None] * rgb_up[None, None, :]
    + visible_down[..., None] * rgb_down[None, None, :]
) / total[..., None]
white = np.ones_like(mixed)
rgb = white * (1.0 - intensity[..., None]) + mixed * intensity[..., None]
return np.clip(rgb, 0.0, 1.0)

def add_spin_colorbar(ax: plt.Axes) -> None:
    """Add an editable vector blue-white-red spin-polarization colour bar."""
    color_up, color_down = edge_spin_colors()
    cmap = edge_spin_colormap()
    cax = inset_axes(
        ax,
        width="3.2%",
        height="94%",
        loc="lower left",
        bbox_to_anchor=(1.025, 0.03, 1.0, 1.0),
        bbox_transform=ax.transAxes,
        borderpad=0.0,
    )
    strip_count = 96
    for index in range(strip_count):
        y0 = index / float(strip_count)
        y1 = (index + 1) / float(strip_count)
        color = cmap((index + 0.5) / float(strip_count))
        cax.add_patch(
            Rectangle(
                (0.0, y0),
                1.0,
                y1 - y0 + 1.0e-5,
                transform=cax.transAxes,
                facecolor=color,
                edgecolor="none",
                linewidth=0.0,
            )
        )
    cax.set_xlim(0.0, 1.0)
    cax.set_ylim(0.0, 1.0)
    cax.set_xticks([])
    cax.set_yticks([])
    for spine in cax.spines.values():
        spine.set_visible(True)
        spine.set_color(core.BLACK)
        spine.set_linewidth(1.25)
    cax.set_ylabel(
        r"$P_z$",
        rotation=90,
        labelpad=7,
        fontweight="bold",
    )
    cax.yaxis.set_label_position("right")
    cax.text(
        1.55,
        1.0,
        r"$\uparrow$",
        color=color_up,
        ha="left",
        va="center",
        transform=cax.transAxes,
        fontsize=14,
        fontweight="bold",
        clip_on=False,
    )
    cax.text(
        1.55,
        0.0,
        r"$\downarrow$",

```

```

        color=color_down,
        ha="left",
        va="center",
        transform=cax.transAxes,
        fontsize=14,
        fontweight="bold",
        clip_on=False,
    )

def draw_edge_only(
    ax: plt.Axes,
    result: dict[str, Any],
    specs: dict[str, core.ModelSpec],
    title: str,
    show_legend: bool = True,
) -> None:
    selected = result["selected"]
    spec = specs[selected.system]
    surface = result["surface"]
    rgb = edge_surface_rgb(selected, surface)
    # Retain the established WannierTools-like continuum, but reduce its
    # contrast slightly relative to the earlier 1.28 enhancement.
    rgb = np.clip(1.0 - 1.15 * (1.0 - rgb), 0.0, 1.0)
    full_window = float(
        np.max(np.abs(np.asarray(surface["energy"], dtype=float)))
    )
    focused_window = principal_edge_window(result)
    ax.imshow(
        rgb,
        origin="lower",
        extent=[-np.pi, np.pi, -full_window, full_window],
        aspect="auto",
        interpolation="nearest",
    )
    ax.axhline(0.0, color=core.BLACK, lw=1.0, ls="--")
    ax.set_xlim(-np.pi, np.pi)
    ax.set_ylim(-focused_window, focused_window)
    ax.set_xticks([-np.pi, 0.0, np.pi])
    # The calculation uses A1=(1,0) as the periodic direction and A2=(0,1)
    # as the open/surface-normal direction. Thus k_parallel=kx and the
    # projected 1D edge BZ follows X-Gamma-X. A Y-Gamma-Y path
    # would instead require an x-normal edge and a new surface calculation.
    edge_tick_labels = ax.set_xticklabels(
        ["X", "\u0393", "X"]
    )
    for tick_label in edge_tick_labels:
        tick_label.set_fontfamily("Times New Roman")
        tick_label.set_fontweight("bold")
        tick_label.set_fontstyle("normal")
    ax.set_xlabel(r"$k_{\parallel}$", fontweight="bold")
    ax.set_ylabel(
        r"${E-E_{\mathrm{ref}}}\backslash \mathrm{(eV)}$",
        fontweight="bold",
    )
    ax.set_title(title + r" $(010)$ edge", fontweight="bold")
    if show_legend:
        color_up, color_down = edge_spin_colors(selected)
        cup = int(selected.expected_chern_up)
        cdown = int(selected.expected_chern_down)
        handles = [
            Line2D(
                [0],
                [0],
                color=color_up,
                lw=3.0,
                label=r"$C_{\uparrow}={cup:+d}$",
            ),
            Line2D(

```

```

        [0],
        [0],
        color=color_down,
        lw=3.0,
        label=r"$C_{\{\downarrow\}}=\{c_{down}:+d\}$",
    ),
]
ax.legend(
    handles=handles,
    loc="upper right",
    frameon=True,
    facecolor="white",
    edgecolor="none",
    framealpha=0.72,
    borderaxespad=0.35,
    handlelength=1.7,
)
add_spin_colorbar(ax)
core.style_axis(ax)
# Preserve k_parallel in [-pi, pi], but make the printed axis physically
# shorter and emphasize the gap-region dispersion.
ax.set_box_aspect(1.12)

def plot_edge_only_individual(
    result: dict[str, Any],
    specs: dict[str, core.ModelSpec],
    stem: Path,
    dpi: int,
) -> list[str]:
    fig, ax = plt.subplots(figsize=(3.15, 3.45))
    draw_edge_only(
        ax,
        result,
        specs,
        specs[result["selected"].system].label,
        show_legend=True,
    )
    fig.tight_layout()
    return save_figure_inkscape_safe(fig, stem, dpi)

def plot_edge_only_three_systems(
    results: list[dict[str, Any]],
    specs: dict[str, core.ModelSpec],
    stem: Path,
    dpi: int,
) -> list[str]:
    cs1_results = [
        result
        for result in results
        if int(result["selected"].expected_chern_up) == 1
    ]
    if [result["selected"].system for result in cs1_results] != [
        "lieb",
        "fes",
        "tts",
    ]:
        raise RuntimeError("Expected Lieb/FES/TTS C_s=+1 edge cases")
    fig, axes = plt.subplots(1, 3, figsize=(8.55, 3.65))
    for index, (ax, result) in enumerate(zip(axes, cs1_results)):
        draw_edge_only(
            ax,
            result,
            specs,
            f"({chr(ord('a') + index)}) "
            + specs[result["selected"].system].label,
            show_legend=True,
        )

```

```

fig.tight_layout(w_pad=2.1)
return save_figure_inkscape_safe(fig, stem, dpi)

def square_bulk_path(
    points_per_segment: int,
) -> tuple[np.ndarray, np.ndarray, list[float], list[str]]:
    """Full square-BZ path Gamma-X-M-Y-Gamma."""
    named = [
        (r"$\Gamma$", (0.0, 0.0)),
        ("X", (np.pi, 0.0)),
        ("M", (np.pi, np.pi)),
        ("Y", (0.0, np.pi)),
        (r"$\Gamma$", (0.0, 0.0)),
    ]
    klist: list[np.ndarray] = []
    distance: list[float] = []
    ticks = [0.0]
    labels = [named[0][0]]
    current = 0.0
    previous: np.ndarray | None = None
    for segment in range(len(named) - 1):
        start = np.asarray(named[segment][1], dtype=float)
        stop = np.asarray(named[segment + 1][1], dtype=float)
        for index in range(int(points_per_segment) + 1):
            if segment > 0 and index == 0:
                continue
            fraction = index / float(points_per_segment)
            kpoint = (1.0 - fraction) * start + fraction * stop
            if previous is not None:
                current += float(np.linalg.norm(kpoint - previous))
            klist.append(kpoint)
            distance.append(current)
            previous = kpoint
            ticks.append(current)
            labels.append(named[segment + 1][0])
    return np.asarray(klist), np.asarray(distance), ticks, labels

def tts_altermagnetic_bulk_path(
    points_per_segment: int,
) -> tuple[np.ndarray, np.ndarray, list[float], list[str]]:
    """TTS path containing Sigma and Sigma-prime spin-split diagonals."""
    named = [
        (r"$\Gamma$", (0.0, 0.0)),
        ("X", (np.pi, 0.0)),
        ("M", (np.pi, np.pi)),
        (r"$\Gamma$", (0.0, 0.0)),
        (r"$M^\text{prime}$", (-np.pi, np.pi)),
        ("Y", (0.0, np.pi)),
        (r"$\Gamma$", (0.0, 0.0)),
    ]
    klist: list[np.ndarray] = []
    distance: list[float] = []
    ticks = [0.0]
    labels = [named[0][0]]
    current = 0.0
    previous: np.ndarray | None = None
    for segment in range(len(named) - 1):
        start = np.asarray(named[segment][1], dtype=float)
        stop = np.asarray(named[segment + 1][1], dtype=float)
        for index in range(int(points_per_segment) + 1):
            if segment > 0 and index == 0:
                continue
            fraction = index / float(points_per_segment)
            kpoint = (1.0 - fraction) * start + fraction * stop
            if previous is not None:
                current += float(np.linalg.norm(kpoint - previous))
            klist.append(kpoint)

```

```

        distance.append(current)
        previous = kpoint
        ticks.append(current)
        labels.append(named[segment + 1][0])
    return np.asarray(klist), np.asarray(distance), ticks, labels

def display_bulk_path(
    system: str,
    points_per_segment: int,
) -> tuple[np.ndarray, np.ndarray, list[float], list[str]]:
    if system == "tts":
        return tts_altermagnetic_bulk_path(points_per_segment)
    return square_bulk_path(points_per_segment)

def calculate_square_bulk_bands(
    spec: core.ModelSpec,
    selected: core.SelectedState,
    energy_reference: float,
    points_per_segment: int,
) -> tuple[pd.DataFrame, list[float], list[str]]:
    """Spin-resolved bands on the system-appropriate display path."""
    klist, distance, ticks, labels = display_bulk_path(
        spec.key,
        points_per_segment,
    )
    spin_blocks = (
        ("up", np.asarray(spec.spin_up_indices, dtype=int), 1.0),
        ("down", np.asarray(spec.spin_down_indices, dtype=int), -1.0),
    )
    rows: list[dict[str, float | int | str]] = []
    for k_index, ((kx, ky), coordinate) in enumerate(
        zip(klist, distance)
    ):
        hamiltonian = spec.h_atomic(
            float(kx), float(ky), selected.params
        )
        for spin_sector, indices, spin_z in spin_blocks:
            spin_hamiltonian = hamiltonian[np.ix_(indices, indices)]
            energies = np.linalg.eigvalsh(spin_hamiltonian)
            for band_index, energy in enumerate(energies):
                rows.append(
                    {
                        "k_index": int(k_index),
                        "path_coordinate": float(coordinate),
                        "kx": float(kx),
                        "ky": float(ky),
                        "spin_sector": spin_sector,
                        "band": int(band_index + 1),
                        "energy": float(energy - energy_reference),
                        "spin_z": float(spin_z),
                    }
                )
    return pd.DataFrame(rows), ticks, labels

def draw_square_bulk(
    ax: plt.Axes,
    result: dict[str, Any],
    specs: dict[str, core.ModelSpec],
    title: str,
) -> None:
    selected = result["selected"]
    spec = specs[selected.system]
    bulk = result["bulk_square"]
    color_up, color_down = edge_spin_colors(selected)
    up_bands = {
        int(index): band

```

```

        for index, band in bulk[bulk["spin_sector"] == "up"].groupby("band")
    }
    down_bands = {
        int(index): band
        for index, band in bulk[bulk["spin_sector"] == "down"].groupby("band")
    }
    for band_index in sorted(set(up_bands) | set(down_bands)):
        up_band = up_bands.get(band_index)
        down_band = down_bands.get(band_index)
        degenerate = (
            up_band is not None
            and down_band is not None
            and len(up_band) == len(down_band)
            and np.allclose(
                up_band["energy"].to_numpy(),
                down_band["energy"].to_numpy(),
                rtol=0.0,
                atol=1.0e-10,
            )
        )
        if up_band is not None:
            ax.plot(
                up_band["path_coordinate"],
                up_band["energy"],
                color=color_up,
                ls="--",
                lw=2.25 if degenerate else 1.60,
                alpha=0.98,
                zorder=2,
            )
        if down_band is not None:
            ax.plot(
                down_band["path_coordinate"],
                down_band["energy"],
                color=color_down,
                ls="--",
                lw=1.15 if degenerate else 1.60,
                alpha=0.98,
                zorder=3,
            )
    for tick in result["bulk_square_ticks"]:
        ax.axvline(tick, color="#AFAFAF", lw=0.75, ls="--", zorder=0)
    ax.axhline(0.0, color=core.BLACK, lw=1.0, ls="--")
    ax.set_xlim(
        float(result["bulk_square_ticks"][0]),
        float(result["bulk_square_ticks"][-1]),
    )
    ax.set_ylim(
        -principal_edge_window(result),
        principal_edge_window(result),
    )
    ax.set_xticks(result["bulk_square_ticks"])
    ax.set_xticklabels(result["bulk_square_labels"])
    ax.set_xlabel("Bulk momentum path", fontweight="bold")
    ax.set_ylabel(
        r"${E-E_{\mathrm{ref}}}\backslash \mathrm{eV}$",
        fontweight="bold",
    )
    ax.set_title(title, fontweight="bold")
    core.style_axis(ax)
    ax.set_box_aspect(1.12)

def plot_bulk_edge_individual(
    result: dict[str, Any],
    specs: dict[str, core.ModelSpec],
    stem: Path,
    dpi: int,
) -> list[str]:

```

```

label = specs[result["selected"].system].label
fig, axes = plt.subplots(1, 2, figsize=(6.65, 3.55))
draw_square_bulk(
    axes[0],
    result,
    specs,
    label + ": bulk",
)
draw_edge_only(
    axes[1],
    result,
    specs,
    label,
    show_legend=True,
)
fig.subplots_adjust(
    left=0.10,
    right=0.93,
    bottom=0.18,
    top=0.88,
    wspace=0.52,
)
return save_figure_inkscape_safe(fig, stem, dpi)

def plot_bulk_edge_rows(
    results: list[dict[str, Any]],
    specs: dict[str, core.ModelSpec],
    stem: Path,
    dpi: int,
    primary_svg_editable: bool = False,
    svg_only: bool = False,
) -> list[str]:
    """Reference-style rows: bulk on the left, matching edge on the right."""
    row_count = len(results)
    fig, axes = plt.subplots(
        row_count,
        2,
        figsize=(6.85, 3.35 * row_count),
        squeeze=False,
    )
    for row_index, result in enumerate(results):
        system_label = specs[result["selected"].system].label
        is_band_metal = (
            result["selected"].system == "fes"
            and int(result["audit"]["is_global_insulator"]) == 0
        )
        bulk_letter = chr(ord("a") + 2 * row_index)
        edge_letter = chr(ord("a") + 2 * row_index + 1)
        cs_value = int(result["selected"].expected_chn_up)
        bulk_title = (
            rf"({bulk_letter}) {system_label}: bulk, "
            rf"$C_s={cs_value:+d}$"
        )
        edge_title = rf"({edge_letter}) {system_label}"
        if is_band_metal:
            bulk_title += " (band metal)"
            edge_title += " (band metal)"
        draw_square_bulk(
            axes[row_index, 0],
            result,
            specs,
            bulk_title,
        )
        draw_edge_only(
            axes[row_index, 1],
            result,
            specs,
            edge_title,

```

```

        show_legend=True,
    )
fig.subplots_adjust(
    left=0.11,
    right=0.92,
    bottom=0.07,
    top=0.97,
    hspace=0.48,
    wspace=0.54,
)
if svg_only:
    return save_editable_svg_only(fig, stem.with_suffix(".svg"), dpi)
return save_figure_inkscape_safe(
    fig,
    stem,
    dpi,
    primary_svg_editable=primary_svg_editable,
)

def parameter_record(case_id: str, selected: core.SelectedState) -> dict[str, Any]:
    record: dict[str, Any] = {
        "case_id": case_id,
        "system": selected.system,
        "source_id": selected.source_id,
        "C_s": selected.expected_chern_up,
        "C_up": selected.expected_chern_up,
        "C_down": selected.expected_chern_down,
    }
    record.update(selected.params)
    return record

def run(output_dir: Path, quick: bool = False) -> dict[str, Any]:
    started = time.time()
    configure_step19_plot_style()
    settings = core.build_settings(quick)
    output_dir = Path(output_dir)
    figures_dir = output_dir / "figures"
    data_dir = output_dir / "data"
    figures_dir.mkdir(parents=True, exist_ok=True)
    data_dir.mkdir(parents=True, exist_ok=True)

    specs, modules = core.build_model_specs()
    cases = select_main_cases(modules)
    results: list[dict[str, Any]] = []
    audit_records: list[dict[str, Any]] = []
    hall_records: list[dict[str, Any]] = []
    parameter_records: list[dict[str, Any]] = []

    print("[1/4] Four main-text representatives", flush=True)
    for index, case in enumerate(cases, start=1):
        case_id = str(case["case_id"])
        selected: core.SelectedState = case["selected"]
        spec = specs[selected.system]
        sample_dir = data_dir / case_id
        sample_dir.mkdir(parents=True, exist_ok=True)
        print(
            f" [{index}/4] {case_id}: gap/Chern/Fourier",
            flush=True,
        )

        gap = core.gap_audit(spec, selected, settings.gap_nk)
        chern = core.chern_audit(spec, selected, settings.chern_nk)
        audit = {
            "case_id": case_id,
            "panel": PANEL_LABEL[case_id],
            "system": selected.system,
            "source_id": selected.source_id,

```

```

        "path_id": selected.path_id,
        "expected_C_s": selected.expected_chern_up,
        "expected_chern_up": selected.expected_chern_up,
        "expected_chern_down": selected.expected_chern_down,
        "selection_note": selected.selection_note,
        **gap,
        **chern,
    }
    audit["C_s_numeric"] = (
        float(chern["chern_up"]) - float(chern["chern_down"])
    ) / 2.0
    audit["chern_matches_selection"] = int(
        int(chern["chern_up_int"]) == selected.expected_chern_up
        and int(chern["chern_down_int"]) == selected.expected_chern_down
    )
    audit["classification"] = classification(audit)
    if selected.system == "tts":
        magnetic_audit = modules["tts"].model_tests(
            selected.params,
            n_random=12,
        )
        audit["sigma_max_spin_splitting"] = float(
            magnetic_audit[
                "sigma_kx_eq_ky_max_spin_splitting"
            ]
        )
        audit["sigma_prime_max_spin_splitting"] = float(
            magnetic_audit[
                "sigma_prime_minus_kx_eq_ky_max_spin_splitting"
            ]
        )
        audit["symmetry_axis_max_spin_splitting"] = float(
            max(
                magnetic_audit[
                    "delta_ky0_spin_degeneracy_error"
                ],
                magnetic_audit[
                    "delta_prime_kx0_spin_degeneracy_error"
                ],
                magnetic_audit[
                    "Z_kxpi_spin_degeneracy_error"
                ],
                magnetic_audit[
                    "Z_prime_kypi_spin_degeneracy_error"
                ],
            )
        )
    audit_records.append(audit)
    parameter_records.append(parameter_record(case_id, selected))

    window = choose_case_window(
        selected.system,
        float(gap["min_direct_gap"]),
    )
    ef = float(gap["energy_reference"])
    hoppings = core.extract_hoppings(
        spec,
        selected.params,
        nfft=settings.fourier_n,
    )
    hopping_rows = [
        {
            "rx": rx,
            "ry": ry,
            "max_abs_hopping": float(np.max(np.abs(matrix))),
        }
        for (rx, ry), matrix in sorted(hoppings.items())
    ]
    pd.DataFrame(hopping_rows).to_csv(

```

```

        sample_dir / "wannier_hopping_summary.csv",
        index=False,
    )

    bulk, bulk_ticks, bulk_labels = core.calculate_bulk_bands(
        spec,
        selected,
        ef,
        settings.band_points_per_segment,
    )
    bulk.to_csv(sample_dir / "bulk_bands.csv", index=False)
    bulk_square, bulk_square_ticks, bulk_square_labels = (
        calculate_square_bulk_bands(
            spec,
            selected,
            ef,
            settings.band_points_per_segment,
        )
    )
    bulk_square.to_csv(
        sample_dir / "bulk_bands_display_path.csv",
        index=False,
    )

    eta = max(
        5.0e-4 if selected.system == "fes" else 1.5e-3,
        0.006 * window,
    )
    print(
        f" [{index}/4] {case_id}: semi-infinite edge "
        f"(window={window:.4f}, eta={eta:.3e})",
        flush=True,
    )
    surface = core.calculate_surface(
        spec,
        hoppings,
        ef,
        window,
        settings.surface_k_points,
        settings.surface_energy_points,
        eta,
    )
    np.savez_compressed(
        sample_dir / "wanniertools_style_surface_spectrum.npz",
        **surface,
    )

    hall_nk = core.hall_mesh_for_state(selected, settings)
    print(
        f" [{index}/4] {case_id}: Hall response nk={hall_nk}",
        flush=True,
    )
    hall, hall_summary = core.calculate_hall(
        spec,
        hoppings,
        ef,
        window,
        hall_nk,
        settings.response_energy_points,
    )
    hall.to_csv(
        sample_dir / "wanniertools_style_hall.csv",
        index=False,
    )
    hall_record = {
        "case_id": case_id,
        "panel": PANEL_LABEL[case_id],
        "system": selected.system,
        "source_id": selected.source_id,
    }

```

```

        "expected_C_s": selected.expected_chern_up,
        "classification": audit["classification"],
        **hall_summary,
    }
    hall_records.append(hall_record)
    (sample_dir / "parameters_and_reference.json").write_text(
        json.dumps(
            {
                "case_id": case_id,
                "code_version": CODE_VERSION,
                "selected_state": {
                    "system": selected.system,
                    "source_id": selected.source_id,
                    "path_id": selected.path_id,
                    "C_s": selected.expected_chern_up,
                    "C_up": selected.expected_chern_up,
                    "C_down": selected.expected_chern_down,
                    "selection_note": selected.selection_note,
                },
                "parameters": selected.params,
                "energy_reference": ef,
                "energy_reference_kind": gap["energy_reference_kind"],
                "plot_window": window,
                "surface_eta": eta,
                "classification": audit["classification"],
            },
            indent=2,
            ensure_ascii=False,
        ),
        encoding="utf-8",
    )
    results.append(
        {
            "case_id": case_id,
            "selected": selected,
            "audit": audit,
            "window": window,
            "bulk": bulk,
            "bulk_ticks": bulk_ticks,
            "bulk_labels": bulk_labels,
            "bulk_square": bulk_square,
            "bulk_square_ticks": bulk_square_ticks,
            "bulk_square_labels": bulk_square_labels,
            "surface": surface,
            "hall": hall,
        }
    )
    print(
        f"        direct={float(gap['min_direct_gap']):.6g}, "
        f"indirect={float(gap['indirect_gap']):.6g}, "
        f"C_s={float(audit['C_s_numeric']):+.6f}",
        flush=True,
    )

print("[2/4] Summary tables", flush=True)
audit_df = pd.DataFrame(audit_records)
hall_df = pd.DataFrame(hall_records)
params_df = pd.DataFrame(parameter_records)
audit_df.to_csv(
    output_dir / "step19_01_maintext_case_summary.csv",
    index=False,
)
hall_df.to_csv(
    output_dir / "step19_02_hall_summary.csv",
    index=False,
)
params_df.to_csv(
    output_dir / "step19_03_parameter_table.csv",
    index=False,

```

```

)

print("[3/4] Four individual figures and two optional composites", flush=True)
manifest: list[dict[str, str]] = []
for result in results:
    case_id = result["case_id"]
    files = plot_individual_pair(
        result,
        specs,
        figures_dir / f"Fig_maintext_{case_id}_edge_and_hall",
        settings.dpi,
    )
    for path in files:
        manifest.append(
            {
                "figure_role": "individual_maintext_edge_and_hall",
                "case_id": case_id,
                "path": path,
            }
        )
    edge_files = plot_edge_only_individual(
        result,
        specs,
        figures_dir / f"Fig_edge_only_{case_id}_Cup_Cdown",
        settings.dpi,
    )
    for path in edge_files:
        manifest.append(
            {
                "figure_role": "individual_edge_only_Cup_Cdown",
                "case_id": case_id,
                "path": path,
            }
        )
    bulk_edge_files = plot_bulk_edge_individual(
        result,
        specs,
        figures_dir / f"Fig_bulk_edge_pair_{case_id}",
        settings.dpi,
    )
    for path in bulk_edge_files:
        manifest.append(
            {
                "figure_role": "individual_bulk_edge_pair",
                "case_id": case_id,
                "path": path,
            }
        )
    for path in plot_edge_only_three_systems(
        results,
        specs,
        figures_dir / "Fig_edge_only_three_systems_Cup_Cdown",
        settings.dpi,
    ):
        manifest.append(
            {
                "figure_role": "three_system_edge_only_Cup_Cdown",
                "case_id": "lieb_fes_tts-Cs_plus1",
                "path": path,
            }
        )
    csl_results = [
        result
        for result in results
        if int(result["selected"].expected_chern_up) == 1
    ]
    for role, row_results, stem_name in (
        (
            "three_system_bulk_edge_rows",

```

```

        csl_results,
        "Fig_bulk_edge_three_systems-Cs_plus1",
    ),
    (
        "four_case_bulk_edge_rows",
        results,
        "Fig_bulk_edge_four_topological_cases",
    ),
):
    for path in plot_bulk_edge_rows(
        row_results,
        specs,
        figures_dir / stem_name,
        settings.dpi,
        primary_svg_editable=(
            stem_name == "Fig_bulk_edge_four_topological_cases"
        ),
    ):
        manifest.append(
            {
                "figure_role": role,
                "case_id": "multiple",
                "path": path,
            }
        )
for role, files in (
    (
        "optional_four_panel_edge_composite",
        plot_surface_four(
            results,
            specs,
            figures_dir / "Fig_maintext_four_edge_spectra",
            settings.dpi,
        ),
    ),
    (
        "optional_four_panel_hall_composite",
        plot_hall_four(
            results,
            specs,
            figures_dir / "Fig_maintext_four_hall_conductivity",
            settings.dpi,
        ),
    ),
):
    for path in files:
        manifest.append(
            {"figure_role": role, "case_id": "all_four", "path": path}
        )
pd.DataFrame(manifest).to_csv(
    output_dir / "step19_04_figure_manifest.csv",
    index=False,
)

print("[4/4] Validation certificate", flush=True)
hall_by_case = {
    str(row["case_id"]): row
    for row in hall_records
}
insulating_case_ids = [
    row["case_id"]
    for row in audit_records
    if row["classification"] == "spin_chern_insulator"
]
plateau_errors = {
    case_id: abs(
        float(hall_by_case[case_id]["mid_reference_sigma_up"])
        - float(
            next(

```

```

        row["expected_C_s"]
        for row in audit_records
            if row["case_id"] == case_id
        )
    )
)
for case_id in insulating_case_ids
}
certificate = {
    "code_version": CODE_VERSION,
    "generated_utc": core.utcnow(),
    "quick_mode": bool(quick),
    "case_count": len(results),
    "case_order": list(CASE_ORDER),
    "all_chern_match": bool(
        all(int(row["chern_matches_selection"]) == 1 for row in audit_records)
    ),
    "all_direct_gapped": bool(
        all(int(row["is_direct_gapped"]) == 1 for row in audit_records)
    ),
    "global_insulator_count": int(
        sum(int(row["is_global_insulator"]) for row in audit_records)
    ),
    "band_metal_cases": [
        row["case_id"]
        for row in audit_records
        if row["classification"] == "spin_chern_band_metal"
    ],
    "insulating_midgap_hall_plateau_abs_errors": plateau_errors,
    "insulating_hall_plateaus_pass": bool(
        all(error < (0.12 if quick else 0.035) for error in plateau_errors.values())
    ),
    "mid_reference_charge_cancellation_pass": bool(
        all(
            abs(float(row["mid_reference_sigma_charge"])) < 5.0e-6
            for row in hall_records
        )
    ),
    "fes_interpretation": (
        "The selected FES C_s=+1 state is direct-gapped but has a negative "
        "indirect gap. It is therefore reported as a spin-Chern band metal; "
        "its Fermi-level Hall value is not claimed as a quantized plateau."
    ),
    "supplementary_relation": (
        "The Step18 nine-state topological/critical/Chern-changed triptychs "
        "remain unchanged and are intended for supplementary material."
    ),
    "runtime_seconds": float(time.time() - started),
}
(output_dir / "step19_05_validation_certificate.json").write_text(
    json.dumps(certificate, indent=2, ensure_ascii=False),
    encoding="utf-8",
)
readme = (
    "# Main-text four-topology package\n\n"
    "The four individual `Fig_maintext*_edge_and_hall` files are the "
    "recommended main-text choices. Each combines the semi-infinite edge "
    "spectral function and the matching spin-resolved Hall response. "
    "`Fig_maintext_four_edge_spectra` and "
    "`Fig_maintext_four_hall_conductivity` are optional 2x2 composites.\n\n"
    "Cases: (a) Lieb C_s=+1; (b) FES C_s=+1; "
    "(c) TTS C_s=+1; (d) TTS C_s=+2.\n\n"
    "Important: the FES case is a direct-gapped spin-Chern band metal "
    "because its indirect gap is negative. Its Hall response is therefore "
    "not described as a quantized insulating plateau.\n"
)
(output_dir / "README.md").write_text(readme, encoding="utf-8")
if not certificate["all_chern_match"]:
    raise RuntimeError("At least one main-text Chern number failed")

```

```

if not certificate["all_direct_gapped"]:
    raise RuntimeError("At least one main-text case is not direct-gapped")
if not certificate["insulating_hall_plateaus_pass"]:
    raise RuntimeError("An insulating Hall plateau did not converge")
if not certificate["mid_reference_charge_cancellation_pass"]:
    raise RuntimeError("Time-reversal charge Hall cancellation failed")
print(
    f"Done: {output_dir} ({certificate['runtime_seconds']:.1f} s)",
    flush=True,
)
return certificate

def refresh_wide_edge_surfaces(output_dir: Path) -> dict[str, Any]:
    """Calculate only the wide-energy surface spectra needed by edge figures."""
    configure_step19_plot_style()
    output_dir = Path(output_dir)
    data_dir = output_dir / "data"
    specs, modules = core.build_model_specs()
    cases = select_main_cases(modules)
    settings = core.build_settings(False)
    refreshed: list[str] = []
    for case in cases:
        selected: core.SelectedState = case["selected"]
        if selected.system not in ("lieb", "tts"):
            continue
        case_id = str(case["case_id"])
        sample_dir = data_dir / case_id
        metadata = json.loads(
            (sample_dir / "parameters_and_reference.json").read_text(
                encoding="utf-8"
            )
        )
        ef = float(metadata["energy_reference"])
        hoppings = core.extract_hoppings(
            specs[selected.system],
            selected.params,
            nfft=settings.fourier_n,
        )
        eta = 0.0072 if selected.system == "lieb" else 0.0045
        print(
            f"Wide edge surface {case_id}: E=[-1.5,1.5], "
            f"nk={settings.surface_k_points}, nE=361, eta={eta:.4g}",
            flush=True,
        )
        surface = core.calculate_surface(
            specs[selected.system],
            hoppings,
            ef,
            1.5,
            settings.surface_k_points,
            361,
            eta,
        )
        np.savez_compressed(
            sample_dir
            / "wanniertools_style_surface_spectrum_wide_Eminus1p5_to_1p5.npz",
            **surface,
        )
        refreshed.append(case_id)
    certificate = replot_existing(output_dir)
    certificate_path = output_dir / "step19_05_validation_certificate.json"
    certificate["wide_edge_surface_refresh_utc"] = core.utcnow()
    certificate["wide_edge_surface_cases"] = refreshed
    certificate["wide_edge_surface_energy_range"] = [-1.5, 1.5]
    certificate["edge_spin_color_policy"] = (
        "C_up is red and C_down is blue for all edge-only figures."
    )
    certificate_path.write_text(

```

```

        json.dumps(certificate, indent=2, ensure_ascii=False),
        encoding="utf-8",
    )
    return certificate

def refresh_four_case_editable_svg(output_dir: Path) -> Path:
    """Regenerate only the four-case SVG as an Inkscape-editable hybrid."""
    configure_step19_plot_style()
    output_dir = Path(output_dir)
    figures_dir = output_dir / "figures"
    data_dir = output_dir / "data"
    audit_path = output_dir / "step19_01_maintext_case_summary.csv"
    if not audit_path.is_file():
        raise FileNotFoundError(audit_path)
    specs, modules = core.build_model_specs()
    cases = select_main_cases(modules)
    settings = core.build_settings(False)
    audit_df = pd.read_csv(audit_path)
    results: list[dict[str, Any]] = []
    for case in cases:
        case_id = str(case["case_id"])
        selected: core.SelectedState = case["selected"]
        sample_dir = data_dir / case_id
        metadata = json.loads(
            (sample_dir / "parameters_and_reference.json").read_text(
                encoding="utf-8"
            )
        )
        wide_surface_path = (
            sample_dir
            / "wanniertools_style_surface_spectrum_wide_Eminus1p5_to_1p5.npz"
        )
        surface_path = (
            wide_surface_path
            if wide_surface_path.is_file()
            else sample_dir / "wanniertools_style_surface_spectrum.npz"
        )
        with np.load(surface_path, allow_pickle=False) as stored:
            surface = {name: stored[name] for name in stored.files}
        bulk_square, bulk_square_ticks, bulk_square_labels = (
            calculate_square_bulk_bands(
                specs[selected.system],
                selected,
                float(metadata["energy_reference"]),
                settings.band_points_per_segment,
            )
        )
        audit = audit_df[
            audit_df["case_id"].astype(str) == case_id
        ].iloc[0].to_dict()
        results.append(
            {
                "case_id": case_id,
                "selected": selected,
                "audit": audit,
                "window": float(metadata["plot_window"]),
                "surface": surface,
                "bulk_square": bulk_square,
                "bulk_square_ticks": bulk_square_ticks,
                "bulk_square_labels": bulk_square_labels,
            }
        )
    svg_path = (
        figures_dir / "Fig_bulk_edge_four_topological_cases.svg"
    )
    plot_bulk_edge_rows(
        results,
        specs,

```

```

        svg_path.with_suffix(""),
        settings.dpi,
        primary_svg_editable=True,
        svg_only=True,
    )
    print(f"Editable four-case SVG refreshed: {svg_path}", flush=True)
    return svg_path

def replot_existing(output_dir: Path) -> dict[str, Any]:
    """Rebuild figures from saved numerical data without recalculation."""
    configure_step19_plot_style()
    output_dir = Path(output_dir)
    figures_dir = output_dir / "figures"
    data_dir = output_dir / "data"
    audit_path = output_dir / "step19_01_maintext_case_summary.csv"
    if not audit_path.is_file():
        raise FileNotFoundError(audit_path)
    specs, modules = core.build_model_specs()
    cases = select_main_cases(modules)
    settings = core.build_settings(False)
    audit_df = pd.read_csv(audit_path)
    results: list[dict[str, Any]] = []
    for case in cases:
        case_id = str(case["case_id"])
        selected: core.SelectedState = case["selected"]
        sample_dir = data_dir / case_id
        metadata = json.loads(
            (sample_dir / "parameters_and_reference.json").read_text(
                encoding="utf-8"
            )
        )
        wide_surface_path = (
            sample_dir
            / "wanniertools_style_surface_spectrum_wide_Eminus1p5_to_1p5.npz"
        )
        surface_path = (
            wide_surface_path
            if wide_surface_path.is_file()
            else sample_dir / "wanniertools_style_surface_spectrum.npz"
        )
        with np.load(surface_path, allow_pickle=False) as stored:
            surface = {name: stored[name] for name in stored.files}
        hall = pd.read_csv(sample_dir / "wanniertools_style_hall.csv")
        bulk_square_path = (
            sample_dir / "bulk_bands_display_path.csv"
        )
        bulk_square, bulk_square_ticks, bulk_square_labels = (
            calculate_square_bulk_bands(
                specs[selected.system],
                selected,
                float(metadata["energy_reference"]),
                settings.band_points_per_segment,
            )
        )
        bulk_square.to_csv(bulk_square_path, index=False)
        audit = audit_df[
            audit_df["case_id"].astype(str) == case_id
        ].iloc[0].to_dict()
        results.append(
            {
                "case_id": case_id,
                "selected": selected,
                "audit": audit,
                "window": float(metadata["plot_window"]),
                "surface": surface,
                "hall": hall,
                "bulk_square": bulk_square,
                "bulk_square_ticks": bulk_square_ticks,
            }
        )

```

```

        "bulk_square_labels": bulk_square_labels,
    }
)

dpi = settings.dpi
manifest: list[dict[str, str]] = []
for result in results:
    case_id = result["case_id"]
    files = plot_individual_pair(
        result,
        specs,
        figures_dir / f"Fig_maintext_{case_id}_edge_and_hall",
        dpi,
    )
    for path in files:
        manifest.append(
            {
                "figure_role": "individual_spin_resolved_edge_and_hall",
                "case_id": case_id,
                "path": path,
            }
        )
    edge_files = plot_edge_only_individual(
        result,
        specs,
        figures_dir / f"Fig_edge_only_{case_id}_Cup_Cdown",
        dpi,
    )
    for path in edge_files:
        manifest.append(
            {
                "figure_role": "individual_edge_only_Cup_Cdown",
                "case_id": case_id,
                "path": path,
            }
        )
    bulk_edge_files = plot_bulk_edge_individual(
        result,
        specs,
        figures_dir / f"Fig_bulk_edge_pair_{case_id}",
        dpi,
    )
    for path in bulk_edge_files:
        manifest.append(
            {
                "figure_role": "individual_bulk_edge_pair",
                "case_id": case_id,
                "path": path,
            }
        )
    for path in plot_edge_only_three_systems(
        results,
        specs,
        figures_dir / "Fig_edge_only_three_systems_Cup_Cdown",
        dpi,
    ):
        manifest.append(
            {
                "figure_role": "three_system_edge_only_Cup_Cdown",
                "case_id": "lieb_fes_tts-Cs_plus1",
                "path": path,
            }
        )
csl_results = [
    result
    for result in results
    if int(result["selected"].expected_chern_up) == 1
]
for role, row_results, stem_name in (

```

```

(
    "three_system_bulk_edge_rows",
    cs1_results,
    "Fig_bulk_edge_three_systems-Cs_plus1",
),
(
    "four_case_bulk_edge_rows",
    results,
    "Fig_bulk_edge_four_topological_cases",
),
):
    for path in plot_bulk_edge_rows(
        row_results,
        specs,
        figures_dir / stem_name,
        dpi,
        primary_svg_editable=(
            stem_name == "Fig_bulk_edge_four_topological_cases"
        ),
    ):
        manifest.append(
            {
                "figure_role": role,
                "case_id": "multiple",
                "path": path,
            }
        )
for role, files in (
    (
        "four_panel_spin_resolved_edge_composite",
        plot_surface_four(
            results,
            specs,
            figures_dir / "Fig_maintext_four_edge_spectra",
            dpi,
        ),
    ),
    (
        "four_panel_spin_resolved_hall_composite",
        plot_hall_four(
            results,
            specs,
            figures_dir / "Fig_maintext_four_hall_conductivity",
            dpi,
        ),
    ),
):
    for path in files:
        manifest.append(
            {"figure_role": role, "case_id": "all_four", "path": path}
        )
pd.DataFrame(manifest).to_csv(
    output_dir / "step19_04_figure_manifest.csv",
    index=False,
)
certificate_path = output_dir / "step19_05_validation_certificate.json"
certificate = json.loads(certificate_path.read_text(encoding="utf-8"))
certificate["figure_revision_utc"] = core.utcnow()
certificate["figure_revision"] = (
    "Bulk and edge spectra are paired case by case. All spin-up bulk "
    "bands are solid red and all spin-down bulk bands are solid blue; "
    "exactly degenerate red/blue curves use nested solid strokes so both "
    "remain visible. Primary SVG files are complete Inkscape-compatible "
    "single-layer figures, with companion *_editable.svg files retaining "
    "vector text, axes, and bulk curves."
)
certificate_path.write_text(
    json.dumps(certificate, indent=2, ensure_ascii=False),
    encoding="utf-8",
)

```

```

    )
    print(
        f"Replotted without numerical recalculation: {figures_dir}",
        flush=True,
    )
    return certificate

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument(
        "--output-dir",
        type=Path,
        default=DEFAULT_OUTPUT,
    )
    parser.add_argument(
        "--quick",
        action="store_true",
        help="Small numerical grids for pipeline testing only.",
    )
    parser.add_argument(
        "--replot-only",
        action="store_true",
        help="Regenerate figures from saved Step19 data without recalculation.",
    )
    parser.add_argument(
        "--refresh-wide-edge-only",
        action="store_true",
        help=(
            "Calculate Lieb/TTS surface spectra on E in [-1.5,1.5] and "
            "regenerate edge figures; Hall and bulk data are not recalculated."
        ),
    )
    parser.add_argument(
        "--editable-four-case-svg-only",
        action="store_true",
        help=(
            "Regenerate only Fig_bulk_edge_four_topological_cases.svg with "
            "editable vector axes, curves, text, legends, and colour bars."
        ),
    )
    return parser.parse_args()

def main() -> None:
    args = parse_args()
    if args.editable_four_case_svg_only:
        svg_path = refresh_four_case_editable_svg(args.output_dir)
        print(
            json.dumps(
                {"editable_four_case_svg": str(svg_path)},
                indent=2,
                ensure_ascii=False,
            ),
            flush=True,
        )
        return
    if args.refresh_wide_edge_only:
        certificate = refresh_wide_edge_surfaces(args.output_dir)
    elif args.replot_only:
        certificate = replot_existing(args.output_dir)
    else:
        certificate = run(args.output_dir, quick=bool(args.quick))
    print(json.dumps(certificate, indent=2, ensure_ascii=False), flush=True)

if __name__ == "__main__":
    main()

```


► 代码清单 A.51.3 (python, 914 行)

```
from __future__ import annotations

import argparse
import importlib.util
import json
import sys
from pathlib import Path
from typing import Any

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy import ndimage
from matplotlib.colors import LinearSegmentedColormap, Normalize
from matplotlib.cm import ScalarMappable

ROOT = Path(r"D:\1_ML\tts")
CORE_PATH = ROOT / "TTS_step18_three_system_wanniertools_style_edge_ahc_examples.py"
DEFAULT_OUTPUT = ROOT / "outputs_tts_step20_edge_representative_rescreen"

LIEB_MASTER = Path(
    r"D:\1_ML\lieb"
    r"\outputs_step08_analytic_boundary_wanniertools_edge_msg123342_lieb8_from_step06_n1024_pairs4_seed20260711"
    r"\step08_01_analytic_feature_master.csv"
)
FES_STRICT = Path(
    r"D:\1_ML\fes\outputs_fes6_step04_boundary"
    r"\fes_step04_strict_verified_results.csv"
)
TTS_STRICT = (
    ROOT
    / "outputs_tts_stepA_pm1_screening_v2"
    / "tts_stepA_01_all_normalized_strict_points.csv"
)

CASE_ORDER = (
    "a_lieb-Cs_plus1",
    "b_fes-Cs_plus1",
    "c_tts-Cs_plus1",
    "d_tts-Cs_plus2",
)
CASE_LABELS = {
    "a_lieb-Cs_plus1": r"Lieb  $C_{\uparrow}=+1$",
    "b_fes-Cs_plus1": r"FES  $C_{\uparrow}=+1$",
    "c_tts-Cs_plus1": r"TTS  $C_{\uparrow}=+1$",
    "d_tts-Cs_plus2": r"TTS  $C_{\uparrow}=+2$",
}
WINDOWS = {
    "a_lieb-Cs_plus1": 1.5,
    "b_fes-Cs_plus1": 0.075,
    "c_tts-Cs_plus1": 1.5,
    "d_tts-Cs_plus2": 1.5,
}
ETAS = {
    "a_lieb-Cs_plus1": 0.0072,
    "b_fes-Cs_plus1": 0.00045,
    "c_tts-Cs_plus1": 0.0045,
    "d_tts-Cs_plus2": 0.0045,
}
TARGET_GAPS = {
    "a_lieb-Cs_plus1": 0.50,
    "b_fes-Cs_plus1": 0.0025,
    "c_tts-Cs_plus1": 0.32,
    "d_tts-Cs_plus2": 0.25,
```

```

}

RED = np.asarray((231, 36, 28), dtype=float) / 255.0
BLUE = np.asarray((44, 88, 167), dtype=float) / 255.0

def load_module(name: str, path: Path) -> Any:
    spec = importlib.util.spec_from_file_location(name, path)
    if spec is None or spec.loader is None:
        raise ImportError(f"Cannot import {path}")
    module = importlib.util.module_from_spec(spec)
    sys.modules[name] = module
    spec.loader.exec_module(module)
    return module

def selected_from_row(
    core: Any,
    system: str,
    source_id: str,
    params: dict[str, float],
    chern_up: int,
    kx: float,
    ky: float,
    note: str,
) -> Any:
    return core.SelectedState(
        system=system,
        state="topological",
        source_id=str(source_id),
        path_id="step20_visual_rescreen",
        params={key: float(value) for key, value in params.items()},
        expected_chern_up=int(chern_up),
        expected_chern_down=-int(chern_up),
        critical_kx=float(kx),
        critical_ky=float(ky),
        selection_note=note,
    )

def build_candidates(core: Any, modules: dict[str, Any]) -> dict[str, list[Any]]:
    lieb = modules["lieb"]
    fes = modules["fes"]
    tts = modules["tts"]

    lieb_master = pd.read_csv(LIEB_MASTER)
    lieb_rows = lieb_master[
        (pd.to_numeric(lieb_master["chern_reliable"], errors="coerce") == 1)
        & (pd.to_numeric(lieb_master["gap_convergence_ok"], errors="coerce") == 1)
        & (pd.to_numeric(lieb_master["chern_up_int"], errors="coerce") == 1)
        & (pd.to_numeric(lieb_master["final_indirect_gap"], errors="coerce") > 0)
        & (
            pd.to_numeric(
                lieb_master["distance_to_any_analytic_boundary"],
                errors="coerce",
            )
            > 0.06
        )
    ].copy()
    lieb_rows["prefilter"] = (
        (
            pd.to_numeric(lieb_rows["final_indirect_gap"], errors="coerce")
            - TARGET_GAPS["a_lieb-Cs_plus1"]
        ).abs()
        + 0.20
    ) * (
        pd.to_numeric(lieb_rows["final_direct_gap"], errors="coerce")
        - pd.to_numeric(lieb_rows["final_indirect_gap"], errors="coerce")
    ).abs()

```

```

- 0.12
* pd.to_numeric(
    lieb_rows["distance_to_any_analytic_boundary"], errors="coerce"
)
)
lieb_rows = lieb_rows.sort_values("prefilter").head(36)
lieb_candidates: list[Any] = []
for _, row in lieb_rows.iterrows():
    physical = [float(row[name]) for name in lieb.PHYS7]
    params = lieb.phys7_to_raw8(physical)
    lieb_candidates.append(
        selected_from_row(
            core,
            "lieb",
            str(row["sample_id"]),
            params,
            +1,
            float(row["refined_direct_kx"]),
            float(row["refined_direct_ky"]),
            "Reliable converged phase-interior Lieb C_s=+1 point.",
        )
    )

fes_rows = pd.read_csv(FES_STRICT)
fes_rows = fes_rows[
    (pd.to_numeric(fes_rows["verified_chern_reliable"], errors="coerce") == 1)
    & (pd.to_numeric(fes_rows["verified_chern_up_int"], errors="coerce") == 1)
    & (
        pd.to_numeric(
            fes_rows["verified_min_direct_gap"], errors="coerce"
        )
        > 8.0e-4
    )
].copy()
fes_candidates: list[Any] = []
for _, row in fes_rows.iterrows():
    reduced = {
        name: float(row[name])
        for name in ("m_e", "t1", "t2", "r1", "r2")
    }
    params = fes.raw6_from_reduced5(reduced)
    fes_candidates.append(
        selected_from_row(
            core,
            "fes",
            str(row["point_id"]),
            params,
            +1,
            float(row["verified_direct_gap_kx"]),
            float(row["verified_direct_gap_ky"]),
            "Strictly verified FES C_s=+1 phase-interior point.",
        )
    )

tts_rows = pd.read_csv(TTS_STRICT)
tts_candidates: dict[int, list[Any]] = {+1: [], +2: []}
for chern_up in (+1, +2):
    subset = tts_rows[
        (pd.to_numeric(tts_rows["chern_up_int"], errors="coerce") == chern_up)
        & (pd.to_numeric(tts_rows["strict_gap_verified"], errors="coerce") == 1)
        & (
            pd.to_numeric(
                tts_rows["strict_chern_verified"], errors="coerce"
            )
            == 1
        )
        & (pd.to_numeric(tts_rows["indirect_gap"], errors="coerce") > 0.01)
    ]
    for _, row in subset.iterrows():

```

```

        reduced = {name: float(row[name]) for name in tts.REDUCED7}
        params = tts.raw8_from_reduced7(reduced)
        tts_candidates[chern_up].append(
            selected_from_row(
                core,
                "tts",
                str(row["point_id"]),
                params,
                chern_up,
                float(row["direct_gap_kx"]),
                float(row["direct_gap_ky"]),
                f"Strictly verified TTS C_s={chern_up:+d} phase-interior point.",
            )
        )
    )

return {
    "a_lieb-Cs_plus1": lieb_candidates,
    "b_fes-Cs_plus1": fes_candidates,
    "c_tts-Cs_plus1": tts_candidates[+1],
    "d_tts-Cs_plus2": tts_candidates[+2],
}

def projected_bulk_metrics(
    spec: Any,
    selected: Any,
    ef: float,
    window: float,
    nk: int,
) -> tuple[dict[str, float], dict[str, np.ndarray]]:
    k_parallel = np.linspace(-np.pi, np.pi, int(nk), endpoint=True)
    k_normal = np.linspace(-np.pi, np.pi, int(nk), endpoint=False)
    valence = np.empty_like(k_parallel)
    conduction = np.empty_like(k_parallel)
    all_relative: list[np.ndarray] = []
    for ix, kx in enumerate(k_parallel):
        eigenvalues = np.asarray(
            [
                np.linalg.eigvalsh(
                    spec.h_periodic(float(kx), float(ky), selected.params)
                )
                for ky in k_normal
            ],
            dtype=float,
        )
        eigenvalues -= float(ef)
        valence[ix] = float(
            np.max(eigenvalues[:, spec.n_occ_total - 1])
        )
        conduction[ix] = float(
            np.min(eigenvalues[:, spec.n_occ_total])
        )
        all_relative.append(eigenvalues.ravel())

    all_energy = np.concatenate(all_relative)
    projected_gap = conduction - valence
    gap_center = 0.5 * (conduction + valence)
    energy_scale = max(float(window), 1.0e-8)
    envelope_scale = max(
        float(np.ptp(valence) + np.ptp(conduction)),
        1.0e-8,
    )
    symmetry_error = float(
        np.mean(np.abs(valence - valence[::-1]))
        + np.mean(np.abs(conduction - conduction[::-1]))
    ) / envelope_scale
    roughness = float(
        np.mean(np.abs(np.diff(valence, n=2)))
        + np.mean(np.abs(np.diff(conduction, n=2)))
    )

```

```

    ) / envelope_scale
    positive_gap = np.maximum(projected_gap, 0.0)
    overlap = np.maximum(-projected_gap, 0.0)
    metrics = {
        "projected_gap_fraction": float(np.mean(projected_gap > 0.0)),
        "projected_gap_median": float(np.median(positive_gap)),
        "projected_overlap_mean": float(np.mean(overlap)),
        "projected_gap_center_abs": float(np.median(np.abs(gap_center))),
        "projected_symmetry_error": symmetry_error,
        "projected_envelope_roughness": roughness,
        "bulk_near_reference_fraction": float(
            np.mean(np.abs(all_energy) < 0.20 * energy_scale)
        ),
    }
}
arrays = {
    "k_parallel": k_parallel,
    "projected_valence": valence,
    "projected_conduction": conduction,
}
return metrics, arrays

def percentile_rank(values: pd.Series, high_is_good: bool) -> pd.Series:
    return values.rank(pct=True, ascending=high_is_good, method="average")

def score_coarse(group: pd.DataFrame) -> pd.DataFrame:
    group = group.copy()
    target = float(TARGET_GAPS[str(group["case_id"].iloc[0])])
    gap_match = np.exp(
        -np.abs(group["indirect_gap"] - target) / max(target, 1.0e-6)
    )
    group["coarse_score"] = (
        0.25 * percentile_rank(group["projected_gap_fraction"], True)
        + 0.18 * percentile_rank(group["projected_gap_median"], True)
        + 0.12 * percentile_rank(group["projected_overlap_mean"], False)
        + 0.14 * percentile_rank(group["projected_symmetry_error"], False)
        + 0.14 * percentile_rank(group["projected_envelope_roughness"], False)
        + 0.07
        * percentile_rank(group["bulk_near_reference_fraction"], False)
        + 0.10 * gap_match
    )
    return group.sort_values("coarse_score", ascending=False)

def run_coarse(
    core: Any,
    specs: dict[str, Any],
    candidates: dict[str, list[Any]],
    output_dir: Path,
    nk_gap: int,
    nk_projected: int,
) -> pd.DataFrame:
    records: list[dict[str, Any]] = []
    envelope_dir = output_dir / "projected_bulk_envelopes"
    envelope_dir.mkdir(parents=True, exist_ok=True)
    total = sum(len(items) for items in candidates.values())
    done = 0
    for case_id in CASE_ORDER:
        for selected in candidates[case_id]:
            done += 1
            print(
                f"[coarse {done:02d}/{total:02d}] "
                f"{case_id} {selected.source_id}",
                flush=True,
            )
            spec = specs[selected.system]
            gap = core.gap_audit(spec, selected, nk=nk_gap)
            metrics, arrays = projected_bulk_metrics(

```

```

        spec,
        selected,
        ef=float(gap["energy_reference"]),
        window=WINDOWS[case_id],
        nk=nk_projected,
    )
    record = {
        "case_id": case_id,
        "system": selected.system,
        "source_id": selected.source_id,
        "expected_chern_up": selected.expected_chern_up,
        **{key: float(value) for key, value in gap.items()
           if isinstance(value, (int, float, np.integer, np.floating))},
        **metrics,
        "params_json": json.dumps(selected.params, sort_keys=True),
        "critical_kx": float(selected.critical_kx),
        "critical_ky": float(selected.critical_ky),
    }
    records.append(record)
    np.savez_compressed(
        envelope_dir / f"{case_id}__{selected.source_id}.npz",
        **arrays,
    )
coarse = pd.DataFrame.from_records(records)
scored = pd.concat(
    [score_coarse(group) for _, group in coarse.groupby("case_id")],
    ignore_index=True,
)
scored = scored.sort_values(
    ["case_id", "coarse_score"], ascending=[True, False]
)
scored.to_csv(output_dir / "step20_01_coarse_candidate_audit.csv", index=False)
return scored

```

```

def reconstruct_selected(core: Any, row: pd.Series) -> Any:
    return selected_from_row(
        core,
        str(row["system"]),
        str(row["source_id"]),
        json.loads(str(row["params_json"])),
        int(row["expected_chern_up"]),
        float(row["critical_kx"]),
        float(row["critical_ky"]),
        "Step20 visual rescreen shortlist.",
    )

```

```

def surface_metrics(surface: dict[str, Any]) -> dict[str, float]:
    up = np.loglp(np.asarray(surface["surface_dos_up"], dtype=float))
    down = np.loglp(np.asarray(surface["surface_dos_down"], dtype=float))
    scale = max(float(np.quantile(np.maximum(up, down), 0.997)), 1.0e-14)
    up = np.clip(up / scale, 0.0, 1.5)
    down = np.clip(down / scale, 0.0, 1.5)
    intensity = np.maximum(up, down)
    polarization = (up - down) / np.maximum(up + down, 1.0e-14)
    purity = np.abs(polarization)
    effective = np.clip(
        np.power(intensity, 0.85) * np.power(purity, 0.60),
        0.0,
        1.0,
    )
    weight = intensity / max(float(np.sum(intensity)), 1.0e-14)
    dirty_overlap = float(np.sum(weight * (1.0 - purity)))
    ridge_strength = np.max(effective, axis=0)
    ridge_index = np.argmax(effective, axis=0)
    energy = np.asarray(surface["energy"], dtype=float)
    ridge_energy = energy[ridge_index]
    strong = ridge_strength > 0.15

```

```

    if np.count_nonzero(strong) >= 5:
        smoothness = float(
            np.median(np.abs(np.diff(ridge_energy[strong], n=2)))
            ) / max(float(np.ptp(energy)), 1.0e-12)
    else:
        smoothness = 1.0
    return {
        "surface_dirty_overlap": dirty_overlap,
        "surface_mean_polarization": float(np.sum(weight * purity)),
        "surface_ridge_coverage": float(np.mean(strong)),
        "surface_ridge_strength": float(np.mean(ridge_strength)),
        "surface_ridge_smoothness": smoothness,
        "surface_colored_fraction": float(np.mean(effective > 0.14)),
    }

def clean_spin_rgb(
    dos_up: np.ndarray,
    dos_down: np.ndarray,
    height: int = 900,
    width: int = 720,
) -> np.ndarray:
    up = np.loglp(np.asarray(dos_up, dtype=float))
    down = np.loglp(np.asarray(dos_down, dtype=float))
    scale = max(float(np.quantile(np.maximum(up, down), 0.997)), 1.0e-14)
    zoom = (height / up.shape[0], width / up.shape[1])
    up = ndimage.zoom(up / scale, zoom=zoom, order=3, mode="nearest")
    down = ndimage.zoom(down / scale, zoom=zoom, order=3, mode="nearest")
    up = ndimage.gaussian_filter(np.clip(up, 0.0, 1.3), sigma=(0.8, 1.6))
    down = ndimage.gaussian_filter(
        np.clip(down, 0.0, 1.3), sigma=(0.8, 1.6)
    )
    intensity = np.maximum(up, down)
    polarization = (up - down) / np.maximum(up + down, 1.0e-14)
    effective = np.clip(
        np.power(intensity, 0.85)
        * np.power(np.abs(polarization), 0.60),
        0.0,
        0.96,
    )
    endpoint = np.where(
        (polarization >= 0.0)[..., None],
        RED[None, None, :],
        BLUE[None, None, :],
    )
    return np.clip(
        1.0 - effective[..., None] * (1.0 - endpoint),
        0.0,
        1.0,
    )

def score_surfaces(group: pd.DataFrame) -> pd.DataFrame:
    group = group.copy()
    group["surface_score"] = (
        0.25 * percentile_rank(group["surface_dirty_overlap"], False)
        + 0.20 * percentile_rank(group["surface_mean_polarization"], True)
        + 0.18 * percentile_rank(group["surface_ridge_coverage"], True)
        + 0.15 * percentile_rank(group["surface_ridge_strength"], True)
        + 0.12 * percentile_rank(group["surface_ridge_smoothness"], False)
        + 0.10 * percentile_rank(group["surface_colored_fraction"], False)
    )
    group["combined_score"] = (
        0.35 * group["coarse_score"] + 0.65 * group["surface_score"]
    )
    return group.sort_values("combined_score", ascending=False)

def draw_contact_sheet(audit: pd.DataFrame, output_dir: Path) -> None:

```

```

fig, axes = plt.subplots(
    len(CASE_ORDER),
    3,
    figsize=(7.8, 10.0),
    constrained_layout=True,
)
for row_index, case_id in enumerate(CASE_ORDER):
    subset = audit[audit["case_id"] == case_id].sort_values(
        "combined_score", ascending=False
    )
    for col_index, (_, row) in enumerate(subset.head(3).iterrows()):
        ax = axes[row_index, col_index]
        with np.load(str(row["surface_npz"])) as data:
            energy = np.asarray(data["energy"], dtype=float)
            rgb = clean_spin_rgb(
                data["surface_dos_up"],
                data["surface_dos_down"],
            )
        ax.imshow(
            rgb,
            origin="lower",
            extent=(-np.pi, np.pi, energy[0], energy[-1]),
            aspect="auto",
            interpolation="bicubic",
        )
        ax.set_xticks((-np.pi, 0.0, np.pi), ("X", r"$\Gamma$", "X"))
        if col_index == 0:
            ax.set_ylabel(CASE_LABELS[case_id] + "\nEnergy (eV)")
        ax.set_title(
            f"{row['source_id']}\nscore={row['combined_score']:.3f}",
            fontsize=8,
        )
        ax.tick_params(direction="in", top=True, right=True, labelsiz=8)
fig.savefig(output_dir / "step20_03_surface_shortlist_contact_sheet.png", dpi=300)
fig.savefig(output_dir / "step20_03_surface_shortlist_contact_sheet.svg")
plt.close(fig)

```

```

def run_surface_shortlist(
    core: Any,
    specs: dict[str, Any],
    coarse: pd.DataFrame,
    output_dir: Path,
    shortlist: int,
    k_points: int,
    energy_points: int,
    fourier_n: int,
) -> pd.DataFrame:
    records: list[dict[str, Any]] = []
    data_dir = output_dir / "surface_shortlist"
    data_dir.mkdir(parents=True, exist_ok=True)
    chosen_rows: list[pd.DataFrame] = []
    for case_id in CASE_ORDER:
        group = coarse[coarse["case_id"] == case_id].sort_values(
            "coarse_score", ascending=False
        )
        target = TARGET_GAPS[case_id]
        priority_indices: list[int] = list(group.head(2).index)
        priority_indices.append(int(group["indirect_gap"].idxmax()))
        priority_indices.append(
            int((group["indirect_gap"] - target).abs().idxmin())
        )
        priority_indices.append(int(group["min_direct_gap"].idxmax()))
        unique_indices: list[int] = []
        for row_index in priority_indices + list(group.index):
            if row_index not in unique_indices:
                unique_indices.append(row_index)
            if len(unique_indices) >= int(shortlist):
                break

```

```

        chosen_rows.append(group.loc[unique_indices])
    short = pd.concat(chosen_rows, ignore_index=False)
    total = len(short)
    for index, (_, row) in enumerate(short.iterrows(), start=1):
        case_id = str(row["case_id"])
        selected = reconstruct_selected(core, row)
        spec = specs[selected.system]
        print(
            f"[surface {index:02d}/{total:02d}] "
            f"{case_id} {selected.source_id}",
            flush=True,
        )
        hoppings = core.extract_hoppings(
            spec, selected.params, nfft=int(fourier_n)
        )
        surface = core.calculate_surface(
            spec,
            hoppings,
            ef=float(row["energy_reference"]),
            energy_window=WINDOWS[case_id],
            k_points=int(k_points),
            energy_points=int(energy_points),
            eta=ETAS[case_id],
        )
        case_dir = data_dir / f"{case_id}_{selected.source_id}"
        case_dir.mkdir(parents=True, exist_ok=True)
        npz_path = case_dir / "surface_preview.npz"
        np.savez_compressed(npz_path, **surface)
        records.append(
            {
                **row.to_dict(),
                **surface_metrics(surface),
                "surface_npz": str(npz_path),
            }
        )
    audit = pd.DataFrame.from_records(records)
    audit = pd.concat(
        [score_surfaces(group) for _, group in audit.groupby("case_id")],
        ignore_index=True,
    ).sort_values(["case_id", "combined_score"], ascending=[True, False])
    audit.to_csv(output_dir / "step20_02_surface_shortlist_audit.csv", index=False)
    draw_contact_sheet(audit, output_dir)
    return audit

```

```

def write_selection_json(audit: pd.DataFrame, output_dir: Path) -> None:
    selections: dict[str, Any] = {}
    for case_id in CASE_ORDER:
        row = (
            audit[audit["case_id"] == case_id]
            .sort_values("combined_score", ascending=False)
            .iloc[0]
        )
        selections[case_id] = {
            "source_id": str(row["source_id"]),
            "system": str(row["system"]),
            "expected_chern_up": int(row["expected_chern_up"]),
            "min_direct_gap": float(row["min_direct_gap"]),
            "indirect_gap": float(row["indirect_gap"]),
            "coarse_score": float(row["coarse_score"]),
            "surface_score": float(row["surface_score"]),
            "combined_score": float(row["combined_score"]),
            "surface_npz": str(row["surface_npz"]),
            "params": json.loads(str(row["params_json"])),
            "critical_kx": float(row["critical_kx"]),
            "critical_ky": float(row["critical_ky"]),
        }
    (output_dir / "step20_04_provisional_best_points.json").write_text(
        json.dumps(selections, ensure_ascii=False, indent=2),
    )

```

```

        encoding="utf-8",
    )

def run_final(
    core: Any,
    specs: dict[str, Any],
    output_dir: Path,
    k_points: int,
    energy_points: int,
    fourier_n: int,
) -> list[dict[str, Any]]:
    selection_path = output_dir / "step20_04_provisional_best_points.json"
    selections = json.loads(selection_path.read_text(encoding="utf-8"))
    step19 = load_module(
        "step20_plot_support",
        ROOT / "TTS_step19_maintext_four_topological_edge_hall.py",
    )
    final_dir = output_dir / "final_selected_data"
    final_dir.mkdir(parents=True, exist_ok=True)
    results: list[dict[str, Any]] = []
    audit_records: list[dict[str, Any]] = []
    for index, case_id in enumerate(CASE_ORDER, start=1):
        item = selections[case_id]
        selected = selected_from_row(
            core,
            str(item["system"]),
            str(item["source_id"]),
            item["params"],
            int(item["expected_chern_up"]),
            float(item["critical_kx"]),
            float(item["critical_ky"]),
            "Final Step20 visually screened representative.",
        )
        spec = specs[selected.system]
        print(
            f"[final {index}/4] {case_id} {selected.source_id}",
            flush=True,
        )
        gap = core.gap_audit(spec, selected, nk=72)
        chern = core.chern_audit(spec, selected, nk=48)
        if (
            int(chern["chern_up_int"]) != int(selected.expected_chern_up)
            or int(chern["chern_down_int"])
            != int(selected.expected_chern_down)
        ):
            raise RuntimeError(
                f"Chern verification failed for {case_id}: {chern}"
            )
        bulk, ticks, labels = step19.calculate_square_bulk_bands(
            spec,
            selected,
            energy_reference=float(gap["energy_reference"]),
            points_per_segment=120,
        )
        hoppings = core.extract_hoppings(
            spec, selected.params, nfft=int(fourier_n)
        )
        surface = core.calculate_surface(
            spec,
            hoppings,
            ef=float(gap["energy_reference"]),
            energy_window=WINDOWS[case_id],
            k_points=int(k_points),
            energy_points=int(energy_points),
            eta=ETAS[case_id],
        )
        case_dir = final_dir / case_id
        case_dir.mkdir(parents=True, exist_ok=True)

```

```

surface_path = case_dir / "surface_spectrum_final.npz"
bulk_path = case_dir / "bulk_bands_final.csv"
np.savez_compressed(surface_path, **surface)
bulk.to_csv(bulk_path, index=False)
result = {
    "case_id": case_id,
    "selected": selected,
    "gap": gap,
    "chern": chern,
    "bulk": bulk,
    "bulk_ticks": ticks,
    "bulk_labels": labels,
    "surface": surface,
    "surface_path": str(surface_path),
    "bulk_path": str(bulk_path),
}
results.append(result)
audit_records.append(
    {
        "case_id": case_id,
        "system": selected.system,
        "source_id": selected.source_id,
        "expected_chern_up": selected.expected_chern_up,
        **gap,
        **chern,
        **selected.params,
    }
)
pd.DataFrame(audit_records).to_csv(
    output_dir / "step20_05_final_physics_audit.csv", index=False
)
draw_final_preview(results, output_dir)
return results

def draw_final_preview(
    results: list[dict[str, Any]],
    output_dir: Path,
) -> None:
    plt.rcParams.update(
        {
            "font.family": "Times New Roman",
            "font.weight": "bold",
            "axes.labelweight": "bold",
            "axes.titleweight": "bold",
            "svg.fonttype": "none",
            "font.size": 9.5,
            "axes.linewidth": 1.3,
        }
    )
    fig, axes = plt.subplots(
        4,
        2,
        figsize=(6.8, 12.4),
        constrained_layout=False,
    )
    cmap = LinearSegmentedColormap.from_list(
        "spin_down_white_up", [BLUE, np.ones(3), RED], N=512
    )
    for row_index, result in enumerate(results):
        case_id = result["case_id"]
        selected = result["selected"]
        ax_bulk, ax_edge = axes[row_index]
        bulk = result["bulk"]
        for spin_sector, color in (("up", RED), ("down", BLUE)):
            subset = bulk[bulk["spin_sector"] == spin_sector]
            for _, band in subset.groupby("band"):
                ax_bulk.plot(
                    band["path_coordinate"],

```

```

        band["energy"],
        color=color,
        lw=1.25,
        ls="-",
        alpha=0.98,
    )
    ax_bulk.set_xlim(
        float(result["bulk_ticks"][0]),
        float(result["bulk_ticks"][-1]),
    )
    ax_bulk.set_ylim(-WINDOWS[case_id], WINDOWS[case_id])
    ax_bulk.set_xticks(result["bulk_ticks"])
    ax_bulk.set_xticklabels(result["bulk_labels"])
    ax_bulk.set_ylabel(r"$E-E_{\mathrm{ref}}$ (eV)")
    ax_bulk.set_title(
        f"({chr(ord('a')) + 2 * row_index}) "
        + CASE_LABELS[case_id]
        + " bulk",
        fontsize=10,
    )

    surface = result["surface"]
    energy = np.asarray(surface["energy"], dtype=float)
    rgb = clean_spin_rgb(
        surface["surface_dos_up"],
        surface["surface_dos_down"],
        height=1200,
        width=900,
    )
    ax_edge.imshow(
        rgb,
        origin="lower",
        extent=(-np.pi, np.pi, energy[0], energy[-1]),
        aspect="auto",
        interpolation="bicubic",
    )
    ax_edge.set_xlim(-np.pi, np.pi)
    ax_edge.set_ylim(-WINDOWS[case_id], WINDOWS[case_id])
    ax_edge.set_xticks((-np.pi, 0.0, np.pi), ("X", r"$\Gamma$", "X"))
    ax_edge.set_ylabel(r"$E-E_{\mathrm{ref}}$ (eV)")
    ax_edge.set_title(
        f"({chr(ord('b')) + 2 * row_index}) "
        + CASE_LABELS[case_id]
        + " edge",
        fontsize=10,
    )
    colorbar = fig.colorbar(
        ScalarMappable(norm=Normalize(-1.0, 1.0), cmap=cmap),
        ax=ax_edge,
        fraction=0.045,
        pad=0.025,
    )
    colorbar.set_ticks((-1.0, 0.0, 1.0))
    colorbar.set_ticklabels(
        (r"$\downarrow$", r"$P_z=0$", r"$\uparrow$")
    )
    for axis in (ax_bulk, ax_edge):
        axis.tick_params(
            direction="in", top=True, right=True, width=1.1, length=4
        )
        for spine in axis.spines.values():
            spine.set_linewidth(1.3)
    fig.subplots_adjust(
        left=0.10,
        right=0.92,
        bottom=0.045,
        top=0.97,
        wspace=0.40,
        hspace=0.47,
    )

```

```

    )
    fig.savefig(
        output_dir / "step20_06_rescreened_bulk_edge_preview.png",
        dpi=450,
        facecolor="white",
    )
    fig.savefig(
        output_dir / "step20_06_rescreened_bulk_edge_preview.svg",
        dpi=450,
        facecolor="white",
    )
    plt.close(fig)

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser()
    parser.add_argument("--output-dir", type=Path, default=DEFAULT_OUTPUT)
    parser.add_argument(
        "--stage",
        choices=("coarse", "surface", "final", "all"),
        default="all",
    )
    parser.add_argument("--gap-nk", type=int, default=40)
    parser.add_argument("--projected-nk", type=int, default=49)
    parser.add_argument("--shortlist", type=int, default=3)
    parser.add_argument("--surface-k", type=int, default=81)
    parser.add_argument("--surface-energy", type=int, default=241)
    parser.add_argument("--fourier-n", type=int, default=8)
    return parser.parse_args()

def main() -> None:
    args = parse_args()
    args.output_dir.mkdir(parents=True, exist_ok=True)
    core = load_module("step20_core", CORE_PATH)
    specs, modules = core.build_model_specs()
    if args.stage in ("surface", "final"):
        coarse = pd.read_csv(
            args.output_dir / "step20_01_coarse_candidate_audit.csv"
        )
    else:
        candidates = build_candidates(core, modules)
        inventory = {
            case_id: len(candidates[case_id]) for case_id in CASE_ORDER
        }
        print("Candidate inventory:", inventory, flush=True)
        coarse = run_coarse(
            core,
            specs,
            candidates,
            args.output_dir,
            nk_gap=args.gap_nk,
            nk_projected=args.projected_nk,
        )
    if args.stage == "coarse":
        return
    if args.stage == "final":
        run_final(
            core,
            specs,
            args.output_dir,
            k_points=141,
            energy_points=361,
            fourier_n=args.fourier_n,
        )
    return
    audit = run_surface_shortlist(
        core,
        specs,

```

```

        coarse,
        args.output_dir,
        shortlist=args.shortlist,
        k_points=args.surface_k,
        energy_points=args.surface_energy,
        fourier_n=args.fourier_n,
    )
    write_selection_json(audit, args.output_dir)
    if args.stage == "all":
        run_final(
            core,
            specs,
            args.output_dir,
            k_points=141,
            energy_points=361,
            fourier_n=args.fourier_n,
        )

if __name__ == "__main__":
    main()

```

► 代码清单 A.51.4 (python, 420 行)

```

from __future__ import annotations

import importlib.util
import json
import sys
from pathlib import Path
from typing import Any

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

ROOT = Path(r"D:\1_ML\tts")
OUTPUT = ROOT / "outputs_tts_step21_small_gap_edge_rescreen"
STEP20_OUTPUT = ROOT / "outputs_tts_step20_edge_representative_rescreen"
LIEB_MASTER = Path(
    r"D:\1_ML\lieb"
    r"\outputs_step08_analytic_boundary_wanniertools_edge_msg123342_lieb8_from_step06_n1024_pairs4_seed20260711"
    r"\step08_01_analytic_feature_master.csv"
)
TTS_STRICT = (
    ROOT
    / "outputs_tts_stepA_pm1_screening_v2"
    / "tts_stepA_01_all_normalized_strict_points.csv"
)

WINDOWS = {
    "a_lieb-Cs_plus1": 1.5,
    "c_tts-Cs_plus1": 1.5,
    "d_tts-Cs_plus2": 1.5,
}
BASE_ETA = {
    "a_lieb-Cs_plus1": 0.0072,
    "c_tts-Cs_plus1": 0.0045,
    "d_tts-Cs_plus2": 0.0045,
}
BENCHMARK_IDS = {
    "a_lieb-Cs_plus1": "sobol_000555",

```

```

    "c_tts-Cs_plus1": "sobol_train_000456",
    "d_tts-Cs_plus2": "sobol_external_000117",
}

def load_module(name: str, path: Path) -> Any:
    spec = importlib.util.spec_from_file_location(name, path)
    if spec is None or spec.loader is None:
        raise ImportError(path)
    module = importlib.util.module_from_spec(spec)
    sys.modules[name] = module
    spec.loader.exec_module(module)
    return module

def selected(
    core: Any,
    system: str,
    source_id: str,
    params: dict[str, float],
    chern_up: int,
    kx: float,
    ky: float,
) -> Any:
    return core.SelectedState(
        system=system,
        state="topological",
        source_id=source_id,
        path_id="step21_small_gap_visual_rescreen",
        params={key: float(value) for key, value in params.items()},
        expected_chern_up=chern_up,
        expected_chern_down=-chern_up,
        critical_kx=float(kx),
        critical_ky=float(ky),
        selection_note="Small-gap visual rescreen candidate.",
    )

def build_lieb_candidates(
    core: Any,
    lieb: Any,
) -> list[Any]:
    master = pd.read_csv(LIEB_MASTER)
    candidates = master[
        (pd.to_numeric(master["chern_reliable"], errors="coerce") == 1)
        & (pd.to_numeric(master["gap_convergence_ok"], errors="coerce") == 1)
        & (pd.to_numeric(master["chern_up_int"], errors="coerce") == 1)
        & (
            pd.to_numeric(master["final_indirect_gap"], errors="coerce")
            .between(0.07, 0.25)
        )
        & (
            pd.to_numeric(
                master["distance_to_any_analytic_boundary"], errors="coerce"
            )
            > 0.05
        )
    ].copy()
    candidates["prefilter"] = (
        (
            pd.to_numeric(candidates["final_indirect_gap"], errors="coerce")
            - 0.14
        ).abs()
        + 0.45
        * (
            pd.to_numeric(candidates["final_direct_gap"], errors="coerce")
            - pd.to_numeric(
                candidates["final_indirect_gap"], errors="coerce"
            )
        )
    )

```

```

        ).abs()
        - 0.18
        * pd.to_numeric(
            candidates["distance_to_any_analytic_boundary"], errors="coerce"
        )
    )
    rows = candidates.sort_values("prefilter").head(8)
    output: list[Any] = []
    for _, row in rows.iterrows():
        params = lieb.phys7_to_raw8(
            [float(row[name]) for name in lieb.PHYS7]
        )
        output.append(
            selected(
                core,
                "lieb",
                str(row["sample_id"]),
                params,
                +1,
                float(row["refined_direct_kx"]),
                float(row["refined_direct_ky"]),
            )
        )
    benchmark = master[master["sample_id"] == BENCHMARK_IDS["a_lieb-Cs_plus1"]]
    row = benchmark.iloc[0]
    output.append(
        selected(
            core,
            "lieb",
            str(row["sample_id"]),
            lieb.phys7_to_raw8([float(row[name]) for name in lieb.PHYS7]),
            +1,
            float(row["refined_direct_kx"]),
            float(row["refined_direct_ky"]),
        )
    )
    return output

def build_tts_candidates(
    core: Any,
    tts: Any,
    chern_up: int,
) -> list[Any]:
    table = pd.read_csv(TTS_STRICT)
    if chern_up == 1:
        low, high = 0.018, 0.12
        case_id = "c_tts-Cs_plus1"
    else:
        low, high = 0.03, 0.20
        case_id = "d_tts-Cs_plus2"
    rows = table[
        (pd.to_numeric(table["chern_up_int"], errors="coerce") == chern_up)
        & (pd.to_numeric(table["strict_gap_verified"], errors="coerce") == 1)
        & (
            pd.to_numeric(table["strict_chern_verified"], errors="coerce")
            == 1
        )
        & (
            pd.to_numeric(table["indirect_gap"], errors="coerce").between(
                low, high
            )
        )
    ].sort_values("indirect_gap")
    output: list[Any] = []
    for _, row in rows.iterrows():
        params = tts.raw8_from_reduced7(
            {name: float(row[name]) for name in tts.REDUCED7}
        )

```

```

        output.append(
            selected(
                core,
                "tts",
                str(row["point_id"]),
                params,
                chern_up,
                float(row["direct_gap_kx"]),
                float(row["direct_gap_ky"]),
            )
        )
    benchmark = table[table["point_id"] == BENCHMARK_IDS[case_id]].iloc[0]
    output.append(
        selected(
            core,
            "tts",
            str(benchmark["point_id"]),
            tts.raw8_from_reduced7(
                {name: float(benchmark[name]) for name in tts.REDUCED7}
            ),
            chern_up,
            float(benchmark["direct_gap_kx"]),
            float(benchmark["direct_gap_ky"]),
        )
    )
    return output

def effective_eta(case_id: str, direct_gap: float) -> float:
    floor = 8.0e-4
    return float(
        min(
            BASE_ETA[case_id],
            max(floor, float(direct_gap) / 18.0),
        )
    )

def visual_score(group: pd.DataFrame) -> pd.DataFrame:
    group = group.copy()
    ratio = np.clip(group["min_direct_gap"] / group["eta"], 0.0, 20.0) / 20.0
    rank = lambda column, ascending: group[column].rank(
        pct=True, ascending=ascending, method="average"
    )
    group["small_gap_visual_score"] = (
        0.27 * rank("surface_colored_fraction", ascending=False)
        + 0.18 * rank("surface_dirty_overlap", ascending=False)
        + 0.15 * rank("surface_mean_polarization", ascending=True)
        + 0.17 * rank("surface_ridge_strength", ascending=True)
        + 0.10 * rank("surface_ridge_smoothness", ascending=False)
        + 0.13 * ratio
    )
    return group.sort_values("small_gap_visual_score", ascending=False)

def calculate_candidates(
    core: Any,
    step20: Any,
    specs: dict[str, Any],
    candidates: dict[str, list[Any]],
) -> pd.DataFrame:
    OUTPUT.mkdir(parents=True, exist_ok=True)
    data_dir = OUTPUT / "surface_candidates"
    data_dir.mkdir(parents=True, exist_ok=True)
    records: list[dict[str, Any]] = []
    total = sum(len(items) for items in candidates.values())
    count = 0
    for case_id, items in candidates.items():
        for item in items:

```

```

count += 1
spec = specs[item.system]
gap = core.gap_audit(spec, item, nk=48)
eta = effective_eta(case_id, float(gap["min_direct_gap"]))
print(
    f"[{count:02d}/{total:02d}] {case_id} {item.source_id} "
    f"gap={gap['indirect_gap']:.6f} eta={eta:.6f}",
    flush=True,
)
hoppings = core.extract_hoppings(spec, item.params, nfft=8)
surface = core.calculate_surface(
    spec,
    hoppings,
    ef=float(gap["energy_reference"]),
    energy_window=WINDOWS[case_id],
    k_points=81,
    energy_points=241,
    eta=eta,
)
case_dir = data_dir / f"{case_id}__{item.source_id}"
case_dir.mkdir(parents=True, exist_ok=True)
npz_path = case_dir / "surface_preview.npz"
np.savez_compressed(npz_path, **surface)
records.append(
    {
        "case_id": case_id,
        "system": item.system,
        "source_id": item.source_id,
        "is_step20_benchmark": int(
            item.source_id == BENCHMARK_IDS[case_id]
        ),
        "expected_chern_up": item.expected_chern_up,
        "min_direct_gap": float(gap["min_direct_gap"]),
        "indirect_gap": float(gap["indirect_gap"]),
        "energy_reference": float(gap["energy_reference"]),
        "eta": eta,
        "gap_to_eta": float(gap["min_direct_gap"]) / eta,
        **step20.surface_metrics(surface),
        "params_json": json.dumps(item.params, sort_keys=True),
        "critical_kx": item.critical_kx,
        "critical_ky": item.critical_ky,
        "surface_npz": str(npz_path),
    }
)
audit = pd.DataFrame(records)
audit = pd.concat(
    [visual_score(group) for _, group in audit.groupby("case_id")],
    ignore_index=True,
).sort_values(
    ["case_id", "small_gap_visual_score"],
    ascending=[True, False],
)
audit.to_csv(OUTPUT / "step21_01_small_gap_surface_audit.csv", index=False)
return audit

```

```

def draw_case_sheets(step20: Any, audit: pd.DataFrame) -> None:
    for case_id, group in audit.groupby("case_id", sort=False):
        group = group.sort_values(
            "small_gap_visual_score", ascending=False
        ).reset_index(drop=True)
        columns = 3
        rows = int(np.ceil(len(group) / columns))
        fig, axes = plt.subplots(
            rows,
            columns,
            figsize=(8.1, 2.65 * rows),
            squeeze=False,
            constrained_layout=True,

```

```

)
for index, (_, record) in enumerate(group.iterrows()):
    ax = axes.flat[index]
    with np.load(record["surface_npz"]) as data:
        energy = np.asarray(data["energy"], dtype=float)
        rgb = step20.clean_spin_rgb(
            data["surface_dos_up"],
            data["surface_dos_down"],
            height=900,
            width=720,
        )
    ax.imshow(
        rgb,
        origin="lower",
        extent=(-np.pi, np.pi, energy[0], energy[-1]),
        aspect="auto",
        interpolation="bicubic",
    )
    ax.set_xticks((-np.pi, 0.0, np.pi), ("X", r"$\Gamma$", "X"))
    benchmark = " [current]" if record["is_step20_benchmark"] else ""
    ax.set_title(
        f"{record['source_id']}{benchmark}\n"
        f"gap={record['indirect_gap']:.4f}, "
        f"score={record['small_gap_visual_score']:.3f}",
        fontsize=8,
    )
    ax.tick_params(direction="in", top=True, right=True, labelsize=8)
for ax in axes.flat[len(group):]:
    ax.axis("off")
fig.savefig(
    OUTPUT / f"step21_02_{case_id}_comparison.png",
    dpi=320,
    facecolor="white",
)
plt.close(fig)

```

```

def write_provisional_selection(audit: pd.DataFrame) -> None:
    selection = {}
    for case_id, group in audit.groupby("case_id", sort=False):
        row = group.sort_values(
            "small_gap_visual_score", ascending=False
        ).iloc[0]
        selection[case_id] = {
            key: (
                float(row[key])
                if isinstance(row[key], (float, np.floating))
                else int(row[key])
                if isinstance(row[key], (int, np.integer))
                else row[key]
            )
            for key in (
                "source_id",
                "system",
                "expected_chern_up",
                "min_direct_gap",
                "indirect_gap",
                "eta",
                "gap_to_eta",
                "small_gap_visual_score",
                "surface_npz",
                "params_json",
                "critical_kx",
                "critical_ky",
            )
        }
    (OUTPUT / "step21_03_provisional_selection.json").write_text(
        json.dumps(selection, ensure_ascii=False, indent=2),
        encoding="utf-8",
    )

```

```

)

def main() -> None:
    core = load_module(
        "step21_core",
        ROOT / "TTS_step18_three_system_wanniertools_style_edge_ahc_examples.py",
    )
    step20 = load_module(
        "step21_render",
        ROOT / "TTS_step20_edge_representative_rescreen.py",
    )
    specs, modules = core.build_model_specs()
    candidates = {
        "a_lieb-Cs_plus1": build_lieb_candidates(
            core, modules["lieb"]
        ),
        "c_tts-Cs_plus1": build_tts_candidates(
            core, modules["tts"], +1
        ),
        "d_tts-Cs_plus2": build_tts_candidates(
            core, modules["tts"], +2
        ),
    }
    print(
        "Candidate inventory:",
        {key: len(value) for key, value in candidates.items()},
        flush=True,
    )
    audit = calculate_candidates(core, step20, specs, candidates)
    draw_case_sheets(step20, audit)
    write_provisional_selection(audit)

if __name__ == "__main__":
    main()

```

► 代码清单 A.51.5 (python, 179 行)

```

from __future__ import annotations

import importlib.util
import json
import sys
from pathlib import Path
from typing import Any

import numpy as np
import pandas as pd

ROOT = Path(r"D:\1_ML\tts")
OUTPUT = ROOT / "outputs_tts_step21_small_gap_edge_rescreen"
FINAL_DIR = OUTPUT / "final_selected_data"
LIEB_MASTER = Path(
    r"D:\1_ML\lieb"
    r"\outputs_step08_analytic_boundary_wanniertools_edge_msg123342_lieb8_from_step06_n1024_pairs4_seed20260711"
    r"\step08_01_analytic_feature_master.csv"
)
TTS_STRICT = (
    ROOT
    / "outputs_tts_stepA_pml_screening_v2"
    / "tts_stepA_01_all_normalized_strict_points.csv"

```

```

)
SELECTIONS = {
    "a_lieb-Cs_plus1": {
        "system": "lieb",
        "source_id": "sobol_000367",
        "chern_up": +1,
        "eta": 0.0072,
    },
    "c_tts-Cs_plus1": {
        "system": "tts",
        "source_id": "sobol_train_000343",
        "chern_up": +1,
        "eta": 0.002121,
    },
    "d_tts-Cs_plus2": {
        "system": "tts",
        "source_id": "sobol_train_000376",
        "chern_up": +2,
        "eta": 0.002863,
    },
}

def load_module(name: str, path: Path) -> Any:
    spec = importlib.util.spec_from_file_location(name, path)
    if spec is None or spec.loader is None:
        raise ImportError(path)
    module = importlib.util.module_from_spec(spec)
    sys.modules[name] = module
    spec.loader.exec_module(module)
    return module

def make_selected(
    core: Any,
    modules: dict[str, Any],
    case_id: str,
    item: dict[str, Any],
) -> Any:
    if item["system"] == "lieb":
        table = pd.read_csv(LIEB_MASTER)
        row = table[table["sample_id"] == item["source_id"]].iloc[0]
        module = modules["lieb"]
        params = module.phys7_to_raw8(
            [float(row[name]) for name in module.PHYS7]
        )
        kx = float(row["refined_direct_kx"])
        ky = float(row["refined_direct_ky"])
    else:
        table = pd.read_csv(TTS_STRICT)
        row = table[table["point_id"] == item["source_id"]].iloc[0]
        module = modules["tts"]
        params = module.raw8_from_reduced7(
            {name: float(row[name]) for name in module.REDUCED7}
        )
        kx = float(row["direct_gap_kx"])
        ky = float(row["direct_gap_ky"])
    return core.SelectedState(
        system=str(item["system"]),
        state="topological",
        source_id=str(item["source_id"]),
        path_id="step21_final_small_gap_selection",
        params={key: float(value) for key, value in params.items()},
        expected_chern_up=int(item["chern_up"]),
        expected_chern_down=-int(item["chern_up"]),
        critical_kx=kx,
        critical_ky=ky,
        selection_note=(
            "Small-gap phase-interior point selected to reduce projected-bulk "

```

```

        "color occupancy while retaining resolvable topological branches."
    ),
)

def main() -> None:
    FINAL_DIR.mkdir(parents=True, exist_ok=True)
    core = load_module(
        "step21_final_core",
        ROOT / "TTS_step18_three_system_wanniertools_style_edge_ahc_examples.py",
    )
    step19 = load_module(
        "step21_final_plot",
        ROOT / "TTS_step19_maintext_four_topological_edge_hall.py",
    )
    specs, modules = core.build_model_specs()
    audit_records = []
    manifest = {}
    for index, (case_id, item) in enumerate(SELECTIONS.items(), start=1):
        selected = make_selected(core, modules, case_id, item)
        spec = specs[selected.system]
        print(
            f"[{index}/3] {case_id} {selected.source_id}",
            flush=True,
        )
        gap = core.gap_audit(spec, selected, nk=72)
        chern = core.chern_audit(spec, selected, nk=48)
        if (
            int(chern["chern_up_int"]) != selected.expected_chern_up
            or int(chern["chern_down_int"]) != selected.expected_chern_down
        ):
            raise RuntimeError(f"Chern mismatch for {case_id}: {chern}")
        bulk, ticks, labels = step19.calculate_square_bulk_bands(
            spec,
            selected,
            energy_reference=float(gap["energy_reference"]),
            points_per_segment=120,
        )
        hoppings = core.extract_hoppings(spec, selected.params, nfft=8)
        surface = core.calculate_surface(
            spec,
            hoppings,
            ef=float(gap["energy_reference"]),
            energy_window=1.5,
            k_points=141,
            energy_points=361,
            eta=float(item["eta"]),
        )
        case_dir = FINAL_DIR / case_id
        case_dir.mkdir(parents=True, exist_ok=True)
        surface_path = case_dir / "surface_spectrum_final.npz"
        bulk_path = case_dir / "bulk_bands_final.csv"
        np.savez_compressed(surface_path, **surface)
        bulk.to_csv(bulk_path, index=False)
        audit_records.append(
            {
                "case_id": case_id,
                "system": selected.system,
                "source_id": selected.source_id,
                "expected_chern_up": selected.expected_chern_up,
                "eta": float(item["eta"]),
                "gap_to_eta": float(gap["min_direct_gap"]) / float(item["eta"]),
                **gap,
                **chern,
                **selected.params,
            }
        )
        manifest[case_id] = {
            "source_id": selected.source_id,

```

```

        "surface_npz": str(surface_path),
        "bulk_csv": str(bulk_path),
        "bulk_ticks": [float(value) for value in ticks],
        "bulk_labels": labels,
        "params": selected.params,
    }
    pd.DataFrame(audit_records).to_csv(
        OUTPUT / "step21_04_final_physics_audit.csv", index=False
    )
    (OUTPUT / "step21_05_final_manifest.json").write_text(
        json.dumps(manifest, ensure_ascii=False, indent=2),
        encoding="utf-8",
    )

if __name__ == "__main__":
    main()

```

► 代码清单 A.51.6 (python, 301 行)

```

from __future__ import annotations

import base64
import importlib.util
import io
import json
import shutil
import sys
from pathlib import Path
from typing import Any

import numpy as np
import pandas as pd
from lxml import etree
from PIL import Image
from scipy import ndimage

ROOT = Path(r"D:\1_ML\tts")
OUTPUT = ROOT / "outputs_tts_step22_user_selected_dual_color"
FINAL_DATA = OUTPUT / "final_selected_data"
STEP20_DATA = (
    ROOT
    / "outputs_tts_step20_edge_representative_rescreen"
    / "final_selected_data"
)
LIEB_MASTER = Path(
    r"D:\1_ML\lieb"
    r"\outputs_step08_analytic_boundary_wanniertools_edge_msg123342_lieb8_from_step06_n1024_pairs4_seed20260711"
    r"\step08_01_analytic_feature_master.csv"
)
FES_STRICT = Path(
    r"D:\1_ML\fes\outputs_fes6_step04_boundary"
    r"\fes_step04_strict_verified_results.csv"
)
TTS_STRICT = (
    ROOT
    / "outputs_tts_stepA_pm1_screening_v2"
    / "tts_stepA_01_all_normalized_strict_points.csv"
)

SELECTIONS = {
    "a_lieb-Cs_plus1": {

```

```

        "system": "lieb",
        "source_id": "sobol_000544",
        "chern_up": +1,
        "eta": 0.005686,
        "surface_source": "calculate",
    },
    "b_fes-Cs_plus1": {
        "system": "fes",
        "source_id": "rs-rd_i0031_j0044",
        "chern_up": +1,
        "eta": 0.00045,
        "surface_source": "step20",
    },
    "c_tts-Cs_plus1": {
        "system": "tts",
        "source_id": "sobol_train_000456",
        "chern_up": +1,
        "eta": 0.0045,
        "surface_source": "step20",
    },
    "d_tts-Cs_plus2": {
        "system": "tts",
        "source_id": "sobol_external_000117",
        "chern_up": +2,
        "eta": 0.0045,
        "surface_source": "step20",
    },
}
WINDOWS = {
    "a_lieb-Cs_plus1": 1.5,
    "b_fes-Cs_plus1": 0.075,
    "c_tts-Cs_plus1": 1.5,
    "d_tts-Cs_plus2": 1.5,
}
RED = np.asarray((231, 36, 28), dtype=float) / 255.0
BLUE = np.asarray((44, 88, 167), dtype=float) / 255.0

def load_module(name: str, path: Path) -> Any:
    spec = importlib.util.spec_from_file_location(name, path)
    if spec is None or spec.loader is None:
        raise ImportError(path)
    module = importlib.util.module_from_spec(spec)
    sys.modules[name] = module
    spec.loader.exec_module(module)
    return module

def make_selected(
    core: Any,
    modules: dict[str, Any],
    item: dict[str, Any],
) -> Any:
    system = str(item["system"])
    source_id = str(item["source_id"])
    if system == "lieb":
        table = pd.read_csv(LIEB_MASTER)
        row = table[table["sample_id"] == source_id].iloc[0]
        module = modules["lieb"]
        params = module.phys7_to_raw8(
            [float(row[name]) for name in module.PHYS7]
        )
        kx = float(row["refined_direct_kx"])
        ky = float(row["refined_direct_ky"])
    elif system == "fes":
        table = pd.read_csv(FES_STRICT)
        row = table[table["point_id"] == source_id].iloc[0]
        module = modules["fes"]
        params = module.raw6_from_reduced5(

```



```

        {
            name: float(row[name])
            for name in ("m_e", "t1", "t2", "r1", "r2")
        }
    )
    kx = float(row["verified_direct_gap_kx"])
    ky = float(row["verified_direct_gap_ky"])
else:
    table = pd.read_csv(TTS_STRICT)
    row = table[table["point_id"] == source_id].iloc[0]
    module = modules["tts"]
    params = module.raw8_from_reduced7(
        {name: float(row[name]) for name in module.REDUCED7}
    )
    kx = float(row["direct_gap_kx"])
    ky = float(row["direct_gap_ky"])
chern_up = int(item["chern_up"])
return core.SelectedState(
    system=system,
    state="topological",
    source_id=source_id,
    path_id="step22_user_selected",
    params={key: float(value) for key, value in params.items()},
    expected_chern_up=chern_up,
    expected_chern_down=-chern_up,
    critical_kx=kx,
    critical_ky=ky,
    selection_note="Representative explicitly selected by the user.",
)

```

```

def prepare_data() -> pd.DataFrame:
    OUTPUT.mkdir(parents=True, exist_ok=True)
    FINAL_DATA.mkdir(parents=True, exist_ok=True)
    core = load_module(
        "step22_core",
        ROOT / "TTS_step18_three_system_wanniertools_style_edge_ahc_examples.py",
    )
    step19 = load_module(
        "step22_plot",
        ROOT / "TTS_step19_maintext_four_topological_edge_hall.py",
    )
    specs, modules = core.build_model_specs()
    records = []
    manifest = {}
    for index, (case_id, item) in enumerate(SELECTIONS.items(), start=1):
        selected = make_selected(core, modules, item)
        spec = specs[selected.system]
        print(f"[{index}/4] {case_id} {selected.source_id}", flush=True)
        gap = core.gap_audit(spec, selected, nk=72)
        chern = core.chern_audit(spec, selected, nk=48)
        if (
            int(chern["chern_up_int"]) != selected.expected_chern_up
            or int(chern["chern_down_int"]) != selected.expected_chern_down
        ):
            raise RuntimeError(f"Chern mismatch for {case_id}: {chern}")
        bulk, ticks, labels = step19.calculate_square_bulk_bands(
            spec,
            selected,
            energy_reference=float(gap["energy_reference"]),
            points_per_segment=120,
        )
        case_dir = FINAL_DATA / case_id
        case_dir.mkdir(parents=True, exist_ok=True)
        bulk_path = case_dir / "bulk_bands_final.csv"
        surface_path = case_dir / "surface_spectrum_final.npz"
        bulk.to_csv(bulk_path, index=False)
        if item["surface_source"] == "step20":
            source_surface = (

```

```

        STEP20_DATA / case_id / "surface_spectrum_final.npz"
    )
    shutil.copy2(source_surface, surface_path)
else:
    hoppings = core.extract_hoppings(spec, selected.params, nfft=8)
    surface = core.calculate_surface(
        spec,
        hoppings,
        ef=float(gap["energy_reference"]),
        energy_window=WINDOWS[case_id],
        k_points=141,
        energy_points=361,
        eta=float(item["eta"]),
    )
    np.savez_compressed(surface_path, **surface)
records.append(
    {
        "case_id": case_id,
        "system": selected.system,
        "source_id": selected.source_id,
        "eta": float(item["eta"]),
        "gap_to_eta": float(gap["min_direct_gap"]) / float(item["eta"]),
        **gap,
        **chern,
        **selected.params,
    }
)
manifest[case_id] = {
    "source_id": selected.source_id,
    "surface_npz": str(surface_path),
    "bulk_csv": str(bulk_path),
    "bulk_ticks": [float(value) for value in ticks],
    "bulk_labels": labels,
    "params": selected.params,
}
audit = pd.DataFrame(records)
audit.to_csv(OUTPUT / "step22_01_final_physics_audit.csv", index=False)
(OUTPUT / "step22_02_manifest.json").write_text(
    json.dumps(manifest, ensure_ascii=False, indent=2),
    encoding="utf-8",
)
return audit

```

```

def upsample_fields(
    dos_up: np.ndarray,
    dos_down: np.ndarray,
    height: int = 1420,
    width: int = 1268,
) -> tuple[np.ndarray, np.ndarray]:
    up = np.log1p(np.asarray(dos_up, dtype=float))
    down = np.log1p(np.asarray(dos_down, dtype=float))
    scale = max(float(np.quantile(np.maximum(up, down), 0.997)), 1.0e-14)
    zoom = (height / up.shape[0], width / up.shape[1])
    up = ndimage.zoom(up / scale, zoom=zoom, order=3, mode="nearest")
    down = ndimage.zoom(down / scale, zoom=zoom, order=3, mode="nearest")
    up = ndimage.gaussian_filter(np.clip(up, 0.0, 1.3), sigma=(1.0, 3.0))
    down = ndimage.gaussian_filter(
        np.clip(down, 0.0, 1.3), sigma=(1.0, 3.0)
    )
    return up, down

```

```

def white_overlap_rgb(
    dos_up: np.ndarray,
    dos_down: np.ndarray,
) -> np.ndarray:
    up, down = upsample_fields(dos_up, dos_down)
    intensity = np.maximum(up, down)

```

```

polarization = (up - down) / np.maximum(up + down, 1.0e-14)
effective = np.clip(
    np.power(intensity, 0.85)
    * np.power(np.abs(polarization), 0.60),
    0.0,
    0.96,
)
endpoint = np.where(
    (polarization >= 0.0)[..., None],
    RED[None, None, :],
    BLUE[None, None, :],
)
return np.clip(
    1.0 - effective[..., None] * (1.0 - endpoint),
    0.0,
    1.0,
)

def direct_mix_rgb(
    dos_up: np.ndarray,
    dos_down: np.ndarray,
) -> np.ndarray:
    up, down = upsample_fields(dos_up, dos_down)
    total = np.maximum(up + down, 1.0e-14)
    mixed = (
        up[..., None] * RED[None, None, :]
        + down[..., None] * BLUE[None, None, :]
    ) / total[..., None]
    intensity = np.clip(np.power(np.maximum(up, down), 0.85), 0.0, 0.96)
    return np.clip(
        1.0 - intensity[..., None] * (1.0 - mixed),
        0.0,
        1.0,
    )

def encode_png(rgb: np.ndarray) -> tuple[str, bytes]:
    array = np.asarray(np rint(np.clip(rgb, 0.0, 1.0) * 255.0), dtype=np.uint8)
    image = Image.fromarray(array, mode="RGB")
    buffer = io.BytesIO()
    image.save(buffer, format="PNG", compress_level=6)
    payload = buffer.getvalue()
    return (
        "data:image/png;base64," + base64.b64encode(payload).decode("ascii"),
        payload,
    )

if __name__ == "__main__":
    prepare_data()

```

► 代码清单 A.51.7 (python, 579 行)

```

from __future__ import annotations

import hashlib
import importlib.util
import json
import math
import re
import sys
from pathlib import Path

```

```

from typing import Any

import numpy as np
import pandas as pd
from lxml import etree

ROOT = Path(r"D:\1_ML\tts")
SOURCE_SVG = Path(r"D:\1_ML\picture\Fig4.svg")
OUTPUT = ROOT / "outputs_tts_step31_fig4_path_berry_curvature"
OUTPUT_SVG = OUTPUT / "Fig4_with_path_Berry_curvature.svg"
DATA_DIR = OUTPUT / "path_berry_curvature_data"

CORE_PATH = ROOT / "TTS_step18_three_system_wanniertools_style_edge_ahc_examples.py"
STEP19_PATH = ROOT / "TTS_step19_maintext_four_topological_edge_hall.py"
STEP22_PATH = ROOT / "TTS_step22_user_selection_dual_color_render.py"

SVG_NS = "http://www.w3.org/2000/svg"
INKSCAPE_NS = "http://www.inkscape.org/namespaces/inkscape"
NS = {"svg": SVG_NS}
GREEN = "#009E55"

# The four bulk-band axes in the supplied Fig4.svg. The identifiers are
# Matplotlib clip paths already present in that exact source file.
CASE_CLIPS = {
    "a_lieb-Cs_plus1": "pfa3bb03856",
    "b_fes-Cs_plus1": "pd58dd54c92",
    "c_tts-Cs_plus1": "pa7505dfcc6",
    "d_tts-Cs_plus2": "p91e0cc4bd4",
}
PATH_POINTS_PER_SEGMENT = {
    "lieb": 120,
    "fes": 2400,
    "tts": 120,
}

def load_module(name: str, path: Path) -> Any:
    spec = importlib.util.spec_from_file_location(name, path)
    if spec is None or spec.loader is None:
        raise ImportError(path)
    module = importlib.util.module_from_spec(spec)
    sys.modules[name] = module
    spec.loader.exec_module(module)
    return module

def sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for block in iter(lambda: handle.read(1024 * 1024), b''):
            digest.update(block)
    return digest.hexdigest()

def parse_transform(value: str | None) -> np.ndarray:
    """Parse the simple affine transforms used by the supplied SVG."""
    result = np.eye(3, dtype=float)
    if not value:
        return result
    pattern = re.compile(r"([A-Za-z]+\s*\([^\)]*\))")
    for name, payload in pattern.findall(value):
        numbers = [
            float(token)
            for token in re.split(r"[\s,]+", payload.strip())
            if token
        ]
        transform = np.eye(3, dtype=float)
        if name == "matrix" and len(numbers) == 6:

```

```

        a, b, c, d, e, f = numbers
        transform = np.asarray(
            [[a, c, e], [b, d, f], [0.0, 0.0, 1.0]],
            dtype=float,
        )
    elif name == "translate" and len(numbers) in (1, 2):
        transform[0, 2] = numbers[0]
        transform[1, 2] = numbers[1] if len(numbers) == 2 else 0.0
    elif name == "scale" and len(numbers) in (1, 2):
        transform[0, 0] = numbers[0]
        transform[1, 1] = numbers[1] if len(numbers) == 2 else numbers[0]
    else:
        raise ValueError(f"Unsupported SVG transform: {name}({payload})")
    result = result @ transform
return result

def transform_to_root(node: etree._Element) -> np.ndarray:
    chain: list[etree._Element] = []
    current: etree._Element | None = node
    while current is not None:
        chain.append(current)
        current = current.getparent()
    result = np.eye(3, dtype=float)
    for element in reversed(chain):
        result = result @ parse_transform(element.get("transform"))
    return result

def find_panel_bounds(
    root: etree._Element,
    clip_id: str,
) -> tuple[float, float, float, float]:
    clips = root.xpath(
        ".*//svg:clipPath[@id=$clip_id]",
        namespaces=NS,
        clip_id=clip_id,
    )
    if len(clips) != 1:
        raise RuntimeError(f"Expected one clipPath {clip_id}, found {len(clips)}")
    rectangles = clips[0].xpath(".*//svg:rect", namespaces=NS)
    if len(rectangles) != 1:
        raise RuntimeError(f"Clip {clip_id} is not a single rectangle")
    rect = rectangles[0]

    candidates = root.xpath(
        ".*//svg:path[contains(@clip-path, $clip_id)]",
        namespaces=NS,
        clip_id=clip_id,
    )
    candidates = [
        node
        for node in candidates
        if (
            ("a92425" in node.get("style", "").lower())
            or ("3f63ad" in node.get("style", "").lower())
        )
        and len(node.get("d", "")) > 100
    ]
    if not candidates:
        raise RuntimeError(f"No transformed band path found for {clip_id}")
    # Some panels contain an old, displaced duplicate path that shares the
    # same clipPath. The visible publication band is the densely encoded
    # path (largest d attribute); using the first XML match displaced the FES
    # overlay by one imported-group translation.
    visible_band = max(candidates, key=lambda node: len(node.get("d", "")))
    matrix = transform_to_root(visible_band)

    x = float(rect.get("x"))

```

```

y = float(rect.get("y"))
width = float(rect.get("width"))
height = float(rect.get("height"))
corners = np.asarray(
    [
        [x, y, 1.0],
        [x + width, y, 1.0],
        [x, y + height, 1.0],
        [x + width, y + height, 1.0],
    ],
    dtype=float,
)
mapped = (matrix @ corners.T).T
x0 = float(np.min(mapped[:, 0]))
x1 = float(np.max(mapped[:, 0]))
y0 = float(np.min(mapped[:, 1]))
y1 = float(np.max(mapped[:, 1]))
return x0, y0, x1, y1

def nice_limit(maximum: float) -> float:
    """Symmetric panel limit with a small amount of headroom."""
    target = max(float(maximum) * 1.06, 1.0e-12)
    exponent = math.floor(math.log10(target))
    scale = 10.0**exponent
    fraction = target / scale
    for candidate in (1.0, 1.2, 1.5, 2.0, 2.5, 3.0, 3.2, 4.0, 5.0, 6.0, 8.0, 10.0):
        if fraction <= candidate + 1.0e-12:
            return float(candidate * scale)
    return float(10.0 * scale)

def format_tick(value: float) -> str:
    absolute = abs(float(value))
    if absolute >= 100.0:
        return f"{absolute:.0f}"
    if absolute >= 10.0:
        return f"{absolute:.0f}"
    if absolute >= 1.0:
        return f"{absolute:.1f}".rstrip("0").rstrip(".")
    if absolute >= 0.1:
        return f"{absolute:.2f}".rstrip("0").rstrip(".")
    return f"{absolute:.2g}"

def calculate_path_curvature(
    core: Any,
    step19: Any,
    step22: Any,
) -> tuple[dict[str, pd.DataFrame], list[dict[str, Any]]]:
    specs, modules = core.build_model_specs()
    curves: dict[str, pd.DataFrame] = {}
    audit: list[dict[str, Any]] = []
    DATA_DIR.mkdir(parents=True, exist_ok=True)

    for case_id, item in step22.SELECTIONS.items():
        selected = step22.make_selected(core, modules, item)
        spec = specs[selected.system]
        hoppings = core.extract_hoppings(spec, selected.params, nfft=8)
        klist, distance, ticks, labels = step19.display_bulk_path(
            spec.key,
            points_per_segment=PATH_POINTS_PER_SEGMENT[spec.key],
        )
        up_indices = np.asarray(spec.spin_up_indices, dtype=int)
        down_indices = np.asarray(spec.spin_down_indices, dtype=int)
        rows: list[dict[str, float | int | str]] = []

        for index, ((kx, ky), coordinate) in enumerate(zip(klist, distance)):
            hamiltonian, velocity_x, velocity_y = core.bloch_from_hoppings(

```

```

        float(kx),
        float(ky),
        hoppings,
    )
    _, omega_up_bands = core.band_berry_curvature(
        hamiltonian[np.ix_(up_indices, up_indices)],
        velocity_x[np.ix_(up_indices, up_indices)],
        velocity_y[np.ix_(up_indices, up_indices)],
    )
    _, omega_down_bands = core.band_berry_curvature(
        hamiltonian[np.ix_(down_indices, down_indices)],
        velocity_x[np.ix_(down_indices, down_indices)],
        velocity_y[np.ix_(down_indices, down_indices)],
    )
    omega_up = float(np.sum(omega_up_bands[: spec.n_occ_spin]))
    omega_down = float(np.sum(omega_down_bands[: spec.n_occ_spin]))
    rows.append(
        {
            "k_index": int(index),
            "path_coordinate": float(coordinate),
            "kx": float(kx),
            "ky": float(ky),
            "omega_z_up": omega_up,
            "omega_z_down": omega_down,
            "omega_z_total": omega_up + omega_down,
        }
    )

    frame = pd.DataFrame(rows)
    curves[case_id] = frame
    frame.to_csv(
        DATA_DIR / f"{case_id}_path_berry_curvature.csv",
        index=False,
    )

    maximum = float(np.max(np.abs(frame["omega_z_total"].to_numpy())))
    limit = nice_limit(maximum)
    audit.append(
        {
            "case_id": case_id,
            "system": selected.system,
            "source_id": selected.source_id,
            "expected_chern_up": selected.expected_chern_up,
            "expected_chern_down": selected.expected_chern_down,
            "path_labels": " | ".join(
                label.replace("$", "")
                .replace("\\Gamma", "Γ")
                .replace("M^\\prime", "M'")
                for label in labels
            ),
            "path_ticks": json.dumps([float(value) for value in ticks]),
            "omega_z_total_min": float(frame["omega_z_total"].min()),
            "omega_z_total_max": float(frame["omega_z_total"].max()),
            "right_axis_limit": limit,
            "path_points_per_segment": PATH_POINTS_PER_SEGMENT[spec.key],
        }
    )
)
return curves, audit

```

```

def svg_path(
    frame: pd.DataFrame,
    bounds: tuple[float, float, float, float],
    limit: float,
) -> str:
    x0, y0, x1, y1 = bounds
    coordinate = frame["path_coordinate"].to_numpy(dtype=float)
    omega = frame["omega_z_total"].to_numpy(dtype=float)
    x = x0 + (coordinate - coordinate[0]) / (
        coordinate[-1] - coordinate[0]
    )

```

```

) * (x1 - x0)
y = y0 + (limit - omega) / (2.0 * limit) * (y1 - y0)
commands = [f"M {x[0]:.6f},{y[0]:.6f}"]
commands.extend(
    f"L {x_value:.6f},{y_value:.6f}"
    for x_value, y_value in zip(x[1:], y[1:])
)
return " ".join(commands)

def add_text(
    parent: etree._Element,
    *,
    x: float,
    y: float,
    text: str,
    anchor: str = "start",
    size: float = 3.0,
    element_id: str,
) -> etree._Element:
    node = etree.SubElement(
        parent,
        f"{{{SVG_NS}}}text",
        {
            "id": element_id,
            "x": f"{x:.6f}",
            "y": f"{y:.6f}",
            "text-anchor": anchor,
            "style": (
                f"font-family:'Times New Roman';font-size:{size:.3f}px;"
                f"font-weight:bold;font-style:normal;fill:{GREEN};"
                "stroke:none"
            ),
        },
    )
    node.text = text
    return node

def add_omega_label(
    parent: etree._Element,
    *,
    x: float,
    y: float,
    element_id: str,
) -> None:
    text = etree.SubElement(
        parent,
        f"{{{SVG_NS}}}text",
        {
            "id": element_id,
            "x": f"{x:.6f}",
            "y": f"{y:.6f}",
            "style": (
                "font-family:'Times New Roman';font-size:3.45px;"
                f"font-weight:bold;fill:{GREEN};stroke:none"
            ),
        },
    )
    omega = etree.SubElement(
        text,
        f"{{{SVG_NS}}}tspan",
        {"style": "font-style:italic"},
    )
    omega.text = "Ω"
    subscript = etree.SubElement(
        text,
        f"{{{SVG_NS}}}tspan",
        {

```



```

        "style": "font-style:italic;font-size:70%",
        "baseline-shift": "sub",
    },
)
subscript.text = "z"

def add_svg_overlays(
    curves: dict[str, pd.DataFrame],
    audit: list[dict[str, Any]],
) -> list[dict[str, Any]]:
    parser = etree.XMLParser(huge_tree=True, remove_blank_text=False)
    tree = etree.parse(str(SOURCE_SVG), parser)
    root = tree.getroot()
    if root.xpath("./*[@id='berry-curvature-overlays']"):
        raise RuntimeError("Source SVG already contains a Berry-curvature overlay")

    defs_nodes = root.xpath("./svg:defs", namespaces=NS)
    if defs_nodes:
        defs = defs_nodes[0]
    else:
        defs = etree.Element(f"{{{SVG_NS}}}defs")
        root.insert(0, defs)

    group = etree.SubElement(
        root,
        f"{{{SVG_NS}}}g",
        {
            "id": "berry-curvature-overlays",
            f"{{{INKSCAPE_NS}}}groupmode": "layer",
            f"{{{INKSCAPE_NS}}}label": "Path Berry curvature Omega_z",
        },
    )
    audit_by_case = {str(record["case_id"]): record for record in audit}
    overlay_records: list[dict[str, Any]] = []

    for case_id, clip_id in CASE_CLIPS.items():
        bounds = find_panel_bounds(root, clip_id)
        x0, y0, x1, y1 = bounds
        limit = float(audit_by_case[case_id]["right_axis_limit"])
        overlay_clip_id = f"berry-clip-{case_id}"
        clip = etree.SubElement(
            defs,
            f"{{{SVG_NS}}}clipPath",
            {
                "id": overlay_clip_id,
                "clipPathUnits": "userSpaceOnUse",
            },
        )
        etree.SubElement(
            clip,
            f"{{{SVG_NS}}}rect",
            {
                "x": f"{x0:.6f}",
                "y": f"{y0:.6f}",
                "width": f"{x1 - x0:.6f}",
                "height": f"{y1 - y0:.6f}",
            },
        )

    panel_group = etree.SubElement(
        group,
        f"{{{SVG_NS}}}g",
        {
            "id": f"berry-overlay-{case_id}",
            f"{{{INKSCAPE_NS}}}label": f"{case_id}: total Omega_z",
        },
    )
    curve_width = 0.50 if case_id == "b_fes-Cs_plus1" else 0.58

```

```

etree.SubElement(
    panel_group,
    f"{{{SVG_NS}}}{path}",
    {
        "id": f"berry-curve-{case_id}",
        "d": svg_path(curves[case_id], bounds, limit),
        "clip-path": f"url("#{overlay_clip_id})",
        "style": (
            f"fill:none;stroke:{GREEN};stroke-width:{curve_width:.2f};"
            "stroke-linecap:round;stroke-linejoin:round;"
            "stroke-opacity:1"
        ),
    },
)

# The right axis is deliberately minimal, matching the cited paper:
# a green spine, endpoint ticks/numbers, and Omega_z at the top.
etree.SubElement(
    panel_group,
    f"{{{SVG_NS}}}{path}",
    {
        "id": f"berry-right-spine-{case_id}",
        "d": f"M {x1:.6f},{y0:.6f} L {x1:.6f},{y1:.6f}",
        "style": f"fill:none;stroke:{GREEN};stroke-width:0.55",
    },
)

tick_length = 1.25
if case_id == "b_fes-Cs_plus1":
    # Keep the numbers clear of the final  $\Gamma$  label. The curve still
    # uses the full  $\pm 25$  range; only the labelled ticks are at  $\pm 20$ .
    tick_value = 0.8 * limit
    tick_top_y = y0 + (limit - tick_value) / (2.0 * limit) * (y1 - y0)
    tick_bottom_y = (
        y0 + (limit + tick_value) / (2.0 * limit) * (y1 - y0)
    )
else:
    tick_value = limit
    tick_top_y = y0
    tick_bottom_y = y1
for suffix, y in (("top", tick_top_y), ("bottom", tick_bottom_y)):
    etree.SubElement(
        panel_group,
        f"{{{SVG_NS}}}{path}",
        {
            "id": f"berry-right-tick-{suffix}-{case_id}",
            "d": f"M {x1:.6f},{y:.6f} L {x1 + tick_length:.6f},{y:.6f}",
            "style": f"fill:none;stroke:{GREEN};stroke-width:0.48",
        },
    )

tick_text = format_tick(tick_value)
if case_id == "b_fes-Cs_plus1":
    label_top_y = tick_top_y + 0.9
    label_bottom_y = tick_bottom_y + 0.9
    label_size = 2.55
else:
    label_top_y = y0 + 3.0
    label_bottom_y = y1 - 0.8
    label_size = 2.75
add_text(
    panel_group,
    x=x1 + 1.55,
    y=label_top_y,
    text=tick_text,
    size=label_size,
    element_id=f"berry-right-label-top-{case_id}",
)
add_text(
    panel_group,

```

```

        x=x1 + 1.55,
        y=label_bottom_y,
        text=f"-{tick_text}",
        size=label_size,
        element_id=f"berry-right-label-bottom-{case_id}",
    )
    add_omega_label(
        panel_group,
        x=x1 + 1.35,
        y=y0 - 1.4,
        element_id=f"berry-right-title-{case_id}",
    )
    overlay_records.append(
        {
            "case_id": case_id,
            "source_clip_id": clip_id,
            "overlay_clip_id": overlay_clip_id,
            "panel_x0": x0,
            "panel_y0": y0,
            "panel_x1": x1,
            "panel_y1": y1,
            "right_axis_limit": limit,
            "right_axis_labelled_tick": tick_value,
            "curve_id": f"berry-curve-{case_id}",
        }
    )
)

tree.write(
    str(OUTPUT_SVG),
    encoding="utf-8",
    xml_declaration=True,
    pretty_print=False,
)
return overlay_records

def main() -> None:
    OUTPUT.mkdir(parents=True, exist_ok=True)
    source_hash_before = sha256(SOURCE_SVG)
    core = load_module("step31_core", CORE_PATH)
    step19 = load_module("step31_plot", STEP19_PATH)
    step22 = load_module("step31_selection", STEP22_PATH)

    curves, audit = calculate_path_curvature(core, step19, step22)
    overlay_records = add_svg_overlays(curves, audit)
    source_hash_after = sha256(SOURCE_SVG)
    if source_hash_before != source_hash_after:
        raise RuntimeError("The source Fig4.svg was unexpectedly modified")

    pd.DataFrame(audit).to_csv(
        OUTPUT / "step31_01_path_berry_curvature_audit.csv",
        index=False,
    )
    pd.DataFrame(overlay_records).to_csv(
        OUTPUT / "step31_02_svg_overlay_manifest.csv",
        index=False,
    )
    manifest = {
        "source_svg": str(SOURCE_SVG),
        "source_sha256_before": source_hash_before,
        "source_sha256_after": source_hash_after,
        "source_preserved": source_hash_before == source_hash_after,
        "output_svg": str(OUTPUT_SVG),
        "quantity_plotted": "Omega_z_total(k) = Omega_z_up(k) + Omega_z_down(k)",
        "right_axis": "Berry curvature along the same high-symmetry path",
        "not_plotted": (
            "Anomalous Hall conductivity sigma_xy is a Brillouin-zone "
            "integral and is not a function of this one-dimensional k path."
        ),
    },

```

```

        "curve_color": GREEN,
        "svg_editability": (
            "Each curve, right spine, tick, and label is an independent "
            "vector object in the 'Path Berry curvature Omega_z' layer."
        ),
        "cases": overlay_records,
    }
    (OUTPUT / "step31_03_manifest.json").write_text(
        json.dumps(manifest, ensure_ascii=False, indent=2),
        encoding="utf-8",
    )
    print(f"Source preserved: {source_hash_before == source_hash_after}")
    print(f"Output SVG: {OUTPUT_SVG}")
    print(f>Data: {DATA_DIR}")

if __name__ == "__main__":
    main()

```

► 代码清单 A.51.8 (python, 230 行)

```

from __future__ import annotations

import json
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

import TTS_step18_three_system_wanniertools_style_edge_ahc_examples as core
import TTS_step19_maintext_four_topological_edge_hall as step19
import TTS_step22_user_selection_dual_color_render as step22

ROOT = Path(r"D:\1_ML\tts")
OUTPUT = ROOT / "outputs_submission_hall_current_fig4"
FIGURES = OUTPUT / "figures"
DATA = OUTPUT / "data"
STEP22_AUDIT = (
    ROOT
    / "outputs_tts_step22_user_selected_dual_color"
    / "step22_01_final_physics_audit.csv"
)

CASE_ORDER = tuple(step22.SELECTIONS)
PANEL_TITLE = {
    "a_lieb-Cs_plus1": r"(a) Lieb, $C_s=+1$",
    "b_fes-Cs_plus1": r"(b) FES, fixed-index $C_s=+1$ metal",
    "c_tts-Cs_plus1": r"(c) TTS, $C_s=+1$",
    "d_tts-Cs_plus2": r"(d) TTS, $C_s=+2$",
}

ENERGY_WINDOW = {
    "a_lieb-Cs_plus1": 0.55,
    "b_fes-Cs_plus1": 0.10,
    "c_tts-Cs_plus1": 0.60,
    "d_tts-Cs_plus2": 0.60,
}

MAIN_MESH = {
    "a_lieb-Cs_plus1": 121,
    "b_fes-Cs_plus1": 161,
    "c_tts-Cs_plus1": 81,
    "d_tts-Cs_plus2": 81,
}

```

```

}
CHECK_MESH = {
    "a_lieb-Cs_plus1": 101,
    "b_fes-Cs_plus1": 141,
    "c_tts-Cs_plus1": 61,
    "d_tts-Cs_plus2": 61,
}
ENERGY_POINTS = 241
FOURIER_N = 8

def value_at_reference(curve: pd.DataFrame, column: str) -> float:
    index = int(np.argmin(np.abs(curve["energy"].to_numpy(float))))
    return float(curve.iloc[index][column])

def main() -> None:
    FIGURES.mkdir(parents=True, exist_ok=True)
    DATA.mkdir(parents=True, exist_ok=True)
    step19.configure_step19_plot_style()
    specs, modules = core.build_model_specs()
    audit_table = pd.read_csv(STEP22_AUDIT).set_index("case_id")

    results: list[dict[str, object]] = []
    summaries: list[dict[str, object]] = []
    for case_id in CASE_ORDER:
        item = step22.SELECTIONS[case_id]
        selected = step22.make_selected(core, modules, item)
        spec = specs[selected.system]
        audit = audit_table.loc[case_id]
        energy_reference = float(audit["energy_reference"])
        window = float(ENERGY_WINDOW[case_id])
        hoppings = core.extract_hoppings(
            spec, selected.params, nfft=FOURIER_N
        )

        curves: dict[int, pd.DataFrame] = {}
        for nk in (CHECK_MESH[case_id], MAIN_MESH[case_id]):
            print(f"{case_id}: Hall mesh {nk} x {nk}", flush=True)
            curve, _ = core.calculate_hall(
                spec,
                hoppings,
                energy_reference,
                window,
                nk,
                ENERGY_POINTS,
            )
            curves[nk] = curve

        final_curve = curves[MAIN_MESH[case_id]]
        check_curve = curves[CHECK_MESH[case_id]]
        case_dir = DATA / case_id
        case_dir.mkdir(parents=True, exist_ok=True)
        final_curve.to_csv(case_dir / "spin_resolved_hall.csv", index=False)

        up = value_at_reference(final_curve, "sigma_xy_up_e2_over_h")
        down = value_at_reference(final_curve, "sigma_xy_down_e2_over_h")
        charge = value_at_reference(final_curve, "sigma_xy_charge_e2_over_h")
        check_up = value_at_reference(check_curve, "sigma_xy_up_e2_over_h")
        check_down = value_at_reference(check_curve, "sigma_xy_down_e2_over_h")
        expected = int(selected.expected_chnern_up)
        is_insulator = int(audit["is_global_insulator"]) == 1
        summaries.append(
            {
                "case_id": case_id,
                "system": selected.system,
                "source_id": selected.source_id,
                "C_up": expected,
                "C_down": -expected,
            }
        )

```

```

        "is_global_insulator": int(is_insulator),
        "energy_reference_eV": energy_reference,
        "energy_min_eV": -window,
        "energy_max_eV": window,
        "energy_points": ENERGY_POINTS,
        "main_mesh": MAIN_MESH[case_id],
        "check_mesh": CHECK_MESH[case_id],
        "sigma_up_at_reference_e2_over_h": up,
        "sigma_down_at_reference_e2_over_h": down,
        "sigma_charge_at_reference_e2_over_h": charge,
        "mesh_change_up": abs(up - check_up),
        "mesh_change_down": abs(down - check_down),
        "quantized_plateau_error_up": (
            abs(up - expected) if is_insulator else np.nan
        ),
        "quantized_plateau_error_down": (
            abs(down + expected) if is_insulator else np.nan
        ),
    }
)
results.append(
    {
        "case_id": case_id,
        "selected": selected,
        "audit": audit,
        "curve": final_curve,
        "window": window,
    }
)

summary = pd.DataFrame(summaries)
summary.to_csv(OUTPUT / "step35_01_hall_convergence_summary.csv", index=False)

fig, axes = plt.subplots(2, 2, figsize=(6.10, 6.25))
for index, (ax, result) in enumerate(zip(axes.flat, results)):
    case_id = str(result["case_id"])
    selected = result["selected"]
    audit = result["audit"]
    curve = result["curve"]
    window = float(result["window"])
    spec = specs[selected.system]
    color_up, color_down = core.spin_colors(spec, selected)
    phase_color = core.phase_color(spec, selected.expected_chern_up)

    if int(audit["is_global_insulator"]) == 1:
        low = float(audit["vbm"]) - float(audit["energy_reference"])
        high = float(audit["cbm"]) - float(audit["energy_reference"])
        hatch = None
    else:
        half_gap = 0.5 * float(audit["critical_valley_gap"])
        low, high = -half_gap, half_gap
        hatch = "////"
    ax.axvspan(
        low,
        high,
        color=core.lighten(phase_color, 0.78),
        alpha=0.65,
        hatch=hatch,
        edgecolor=phase_color if hatch else "none",
        linewidth=0.0,
        zorder=0,
    )
    ax.plot(
        curve["energy"],
        curve["sigma_xy_up_e2_over_h"],
        color=color_up,
        lw=1.9,
        label=r"$\sigma_{xy}^{\uparrow}$",
    )

```

```

ax.plot(
    curve["energy"],
    curve["sigma_xy_down_e2_over_h"],
    color=color_down,
    lw=1.9,
    label=r"$\sigma_{xy}^{\downarrow}$",
)
ax.axhline(0.0, color="#777777", lw=0.7)
ax.axvline(0.0, color="#111111", lw=0.8, ls=":")
ax.set_xlim(-window, window)
ax.set_ylim(-2.35, 2.35)
ax.set_xlabel(r"$\mu$-E_{\mathrm{ref}} \ \mathrm{(eV)}$")
ax.set_ylabel(r"$\sigma_{xy}^{\downarrow}$ \ (e2/h)$")
ax.set_title(PANEL_TITLE[case_id])
if index == 0:
    ax.legend(loc="best", frameon=False, fontsize=8)
core.style_axis(ax)
ax.set_box_aspect(0.92)

fig.tight_layout(h_pad=1.15, w_pad=1.05)
stem = FIGURES / "FigS6_spin_resolved_hall_current_fig4"
fig.savefig(stem.with_suffix(".pdf"), bbox_inches="tight")
fig.savefig(stem.with_suffix(".png"), dpi=600, bbox_inches="tight")
plt.close(fig)

metadata = {
    "calculation": "zero-temperature occupation-weighted Kubo-Berry integral",
    "k_mesh": "uniform midpoint mesh in [-pi, pi]^2",
    "energy_points": ENERGY_POINTS,
    "fourier_reconstruction_n": FOURIER_N,
    "figure": str(stem.with_suffix(".pdf")),
    "cases": summaries,
}
(OUTPUT / "step35_02_metadata.json").write_text(
    json.dumps(metadata, indent=2, ensure_ascii=False), encoding="utf-8"
)

insulating = summary[summary["is_global_insulator"] == 1]
if float(insulating["quantized_plateau_error_up"].max()) >= 0.035:
    raise RuntimeError("An insulating spin-up Hall plateau failed tolerance")
if float(insulating["quantized_plateau_error_down"].max()) >= 0.035:
    raise RuntimeError("An insulating spin-down Hall plateau failed tolerance")
if float(summary["sigma_charge_at_reference_e2_over_h"].abs().max()) >= 5e-6:
    raise RuntimeError("Charge Hall cancellation failed")
print(summary.to_string(index=False), flush=True)
print(f"Saved {stem.with_suffix('.pdf')}", flush=True)

if __name__ == "__main__":
    main()

```

kp 构造对称哈密顿量的测试

► 代码清单 A.53.1 (plaintext, 10 行)

```

# 生成对称哈密顿量并遍历输出
for m in kp.symmetric_hamiltonian(
    c2y, # 传入 C2y 对称性
    parity, # 传入宇称对称性
    time_reversal, # 传入时间反演对称性

```

```

    expr_basis=kp.monomial_basis(order), # 使用单项式基矢, 阶数为 order
    repr_basis=basis # 使用之前定义的泡利矩阵张量积基矢
):
    print(m) # 打印每个对称的哈密顿量矩阵
print() # 空行分隔

```

(原创脚本) 基于对称性构造 TB 模型 (n 带紧束缚模型)——边缘态计算与拟合

► 代码清单 A.54.1 (python, 23 行)

```

from pythtb import W90
import matplotlib.pyplot as plt
import numpy as np

nso = W90(".", "wannier90")

# hard coded fermi level in eV
fermi_ev = -1.065140

# all pair distances between the orbitals
print("Shells:\n", nso.shells())

# plot hopping terms as a function of distance on a log scale
(dist, ham) = nso.dist_hop()
fig, ax = plt.subplots()
ax.scatter(dist, np.abs(ham))
ax.hlines(
    1e-3, xmin=0, xmax=max(dist), colors="r", linestyle="dashed", label="Cutoff = 1meV"
)
ax.legend()
ax.set_xlabel("Distance (Å)")
ax.set_ylabel(r"$H$ (eV)")
ax.set_yscale("log")

```

► 代码清单 A.54.2 (python, 12 行)

```

# get tb model in which some small terms are ignored
my_model = nso.model(
    zero_energy=fermi_ev,
    min_hopping_norm=1e-3,
)

(w90_kpt, w90_evals, w90_k_dist, w90_k_nodes, w90_k_labels) = nso.bands_w90(
    return_k_dist=True, return_k_nodes=True
)

print("k-point labels:", w90_k_labels)
print("k-point nodes (fractional):\n", w90_k_nodes)

```


► 代码清单 A.54.3 (python, 19 行)

```
k_vec, k_dist, k_node_dist = my_model.k_path(w90_k_nodes, nk=500, report=False)

int_evals = my_model.solve_ham(w90_kpt)

fig, ax = plt.subplots(figsize=(10, 6))

ax.plot(w90_k_dist, w90_evals[:, 0] - fermi_ev, "k-", zorder=0, label="Wannier90")
ax.plot(w90_k_dist, w90_evals[:, 1:] - fermi_ev, "k-", zorder=0)
ax.plot(w90_k_dist, int_evals[:, 0], "r--", zorder=1, label="TBModel")
ax.plot(w90_k_dist, int_evals[:, 1:], "r--", zorder=1)

# set x-ticks at k-point nodes
ax.set_xticks(k_node_dist)
for n in range(len(w90_k_nodes)):
    ax.axvline(x=k_node_dist[n], linewidth=0.5, color="k", zorder=1)
ax.set_xticklabels(w90_k_labels, size=12)
ax.set_xlim(k_node_dist[0], k_node_dist[-1])
ax.set_ylabel("Band energy (eV)")
plt.show()
```

► 代码清单 A.54.4 (python, 27 行)

```
# 新模型, 边缘态能谱的计算

fin_model = my_model.make_finite(periodic_dirs=[0], num_cells=[26])

#k_nodes = [[0, 0], [0.5, 0], [0.5, 0.5], [0, 0.5], [0, 0.0]] # G X M Y G
k_nodes = [[0, 0], [0.5, 0], [0, 0]]
k_labels = [
    r"$\bar{\Gamma}$",
    r"$\bar{X}$",
    r"$\bar{\Gamma}$",
]

fig, ax = plt.subplots(figsize=(8, 6))
fin_model.plot_bands(
    k_nodes=k_nodes, k_node_labels=k_labels, lw=1, nk=500, fig=fig, ax=ax
)

#ax.axhline(y=0, color='red', linestyle='--', linewidth=1.5, alpha=0.7,
#           label=r'$E_F$ (Fermi level)')
ax.set_ylim(-0.5, 0.5)

plt.show()
fig.savefig(
    "edge_band.png",
    dpi=300,
    bbox_inches="tight"
)
```

► 代码清单 A.54.5 (python, 102 行)

```
# 创建有限模型: x 方向周期性, y 方向开边界 (26 个原胞)
fin_model = my_model.make_finite(periodic_dirs=[0], num_cells=[26])
```

```

# 定义 x 方向的 k 路径:  $\Gamma \rightarrow X \rightarrow \Gamma$ 
# 由于 y 方向是开边界, 所以 ky 分量设为 0
k_points = [
    [0, 0],      #  $\Gamma$  点
    [0.5, 0],   # X 点
    [0, 0]       # 返回  $\Gamma$  点
]

# 在路径上采样
nk_segment = 150 # 每段路径的采样点数
nk_total = 2 * nk_segment # 总点数 (两段)

# 生成路径上的 k 点
k_path = []
k_labels = [] # 用于标记高对称点
segment_lengths = [] # 每段路径的长度

# 第一段:  $\Gamma \rightarrow X$ 
for i in range(nk_segment):
    # 线性插值
    t = i / (nk_segment - 1)
    kx = (1 - t) * k_points[0][0] + t * k_points[1][0]
    ky = (1 - t) * k_points[0][1] + t * k_points[1][1]
    k_path.append([kx, ky])

# 第二段:  $X \rightarrow \Gamma$ 
for i in range(nk_segment):
    t = i / (nk_segment - 1)
    kx = (1 - t) * k_points[1][0] + t * k_points[2][0]
    ky = (1 - t) * k_points[1][1] + t * k_points[2][1]
    k_path.append([kx, ky])

# 计算 k 点路径的距离坐标 (用于绘图 x 轴)
k_path_array = np.array(k_path)
k_dist = [0] # 距离坐标从 0 开始

# 计算累积距离
for i in range(1, len(k_path)):
    # 两点之间的欧几里得距离
    dk = k_path_array[i] - k_path_array[i-1]
    dist = np.sqrt(np.sum(dk**2))
    k_dist.append(k_dist[-1] + dist)

k_dist = np.array(k_dist)

# 计算能带
bands = []
for k in k_path:
    # 注意: y 方向是开边界, 所以 ky 分量应该设为 0
    # 但为了通用性, 我们保留 ky 分量
    evals = fin_model.solve_ham(k)
    bands.append(evals)

bands = np.array(bands) # shape: (nk_total, n_bands)

# 高对称点在距离坐标中的位置
k_node_positions = []
for k in k_points:
    # 找到路径上最近的 k 点位置
    k_array = np.array(k)
    distances = np.sqrt(np.sum((k_path_array - k_array)**2, axis=1))
    idx = np.argmin(distances)
    k_node_positions.append(k_dist[idx])

# 高对称点标签
k_node_labels = [
    r"$\bar{\Gamma}$",
    r"$\bar{X}$",

```

```

    r"$\bar{\Gamma}$"
]

# 绘制能带
fig, ax = plt.subplots(figsize=(8, 6))

# 绘制所有能带
for n in range(bands.shape[1]):
    ax.plot(k_dist, bands[:, n], color="k", lw=0.5, alpha=0.7)

# 标记高对称点
for pos, label in zip(k_node_positions, k_node_labels):
    ax.axvline(x=pos, color="red", linestyle="--", alpha=0.5, lw=1)

# 设置 x 轴
ax.set_xticks(k_node_positions)
ax.set_xticklabels([r"$\bar{\Gamma}$", r"$\bar{X}$", r"$\bar{\Gamma}$"])

# 设置 y 轴
ax.set_ylim(-0.5, 0.5)
ax.set_xlim(k_dist[0], k_dist[-1])
ax.set_xlabel(r"$k_{\parallel}$ (沿 x 方向)")
ax.set_ylabel("Energy (eV)")

plt.tight_layout()
plt.show()

# 可选: 保存数据
# np.save("edge_bands_gamma_X_gamma.npy", bands)
# np.save("k_dist_gamma_X_gamma.npy", k_dist)
# np.save("k_path_gamma_X_gamma.npy", k_path_array)

```

► 代码清单 A.54.6 (python, 9 行)

```

# 有限体系下的边缘态波函数图
#fin_model = my_model.make_finite(
#    periodic_dirs=[0, 1], num_cells=[10, 10], glue_edges=[False, False]
#)
fin_model = my_model.make_finite(
    periodic_dirs=[0, 1], num_cells=[3, 3], glue_edges=[True, False]
)
(evals, evecs) = fin_model.solve_ham(return_eigvecs=True)
print("Number of states:", len(evals))

```

► 代码清单 A.54.7 (python, 9 行)

```

# 可视化
(fig, ax) = fin_model_half.visualize(
    proj_plane=[0, 1], eig_dr=evecs_half[ed, :], draw_hoppings=False
)
ax.set_title("Edge state for finite model periodic in one direction")
ax.set_xlabel("x coordinate")
ax.set_ylabel("y coordinate")
fig.tight_layout()
plt.show()

```

► 代码清单 A.54.8 (python, 663 行)

```
# -*- coding: utf-8 -*-
"""
Fit symmetry-based TB parameters (e/t/r) against Wannier band reference (wannier90_band.dat)

Stages:
    Stage-1: fit e only
    Stage-2: fit t only (e fixed)
    Stage-3: fit r only (e,t fixed)
    Stage-4: fit e,t,r together (warm start from stage1-3)

Band-matching protection (recommended):
    Stage-4 uses eigenvector-overlap band tracking + Hungarian anchoring at kref_match,
    to mitigate band crossings / index swapping issues.

Windows-ready. Target band set is fixed by reference band indices selected at kref.
"""

import os
import numpy as np
from scipy.optimize import least_squares, linear_sum_assignment

# Wolfram WXF deserializer (needs: pip install wolframclient)
from wolframclient.deserializers import binary_deserialize
from wolframclient.language.expression import WLFunction, WLSymbol

# =====
# 0) Config (EDIT PATHS HERE)
# =====
TB_FILE = r"E:\马睿骁\组会汇报\Nb20Sse\pythTB\tb_basis.wxf"
WANNIER_BAND_FILE = r"E:\马睿骁\组会汇报\Nb20Sse\pythTB\wannier90_band.dat"

# Optional: for cross validation (DFT BAND.dat from VASPKIT)
DFT_BAND_FILE = None # r"E:\...\BAND.dat"

# DFT Fermi energy baseline: you said "use DFT EF=0"
EF_DFT = 0.0
EF_SHIFT_WANNIER = 0.0

# Select 12 bands: fixed band indices (global columns)
N_TARGET_BANDS = 12
KREF_FOR_PICK = 0 # select target bands at this ref index in reference bands
E_TARGET = 0.0

# Regularization (keeps parameters from exploding)
LAMBDA_E = 1e-8
LAMBDA_T = 1e-8
LAMBDA_R = 1e-8

# Bounds (typical low-energy window)
E_BOUND = 5.0
T_BOUND = 2.5
R_BOUND = 2.5

# Optimizer controls
MAX_NFEV_1 = 2000
MAX_NFEV_2 = 3000
MAX_NFEV_3 = 3000
MAX_NFEV_4 = 6000

DIFF_STEP = 1e-3

# Stage-4 band-matching protection (HIGHLY recommended)
USE_BANDMATCH_STAGE4 = True
KREF_MATCH = KREF_FOR_PICK # anchor for Hungarian matching + tracking
```

```

# =====
# 1) Wolfram expression -> Python number
# =====
def _head_name(h):
    return h.name if isinstance(h, WLSymbol) else str(h)

def wl_to_number(x):
    if isinstance(x, np.generic):
        x = x.item()
    if isinstance(x, (int, float, complex, np.number)):
        return x
    if isinstance(x, (list, tuple)):
        # Some WXF encodes Complex as [re, im]
        if len(x) == 2:
            return complex(float(wl_to_number(x[0])), float(wl_to_number(x[1])))
        if len(x) == 1:
            return wl_to_number(x[0])
    if isinstance(x, WLFunction):
        name = _head_name(x.head)
        args = list(x.args)
        if name == "Complex":
            return complex(wl_to_number(args[0]), wl_to_number(args[1]))
        if name == "Rational":
            return float(wl_to_number(args[0])) / float(wl_to_number(args[1]))
        if name == "Plus":
            return sum(wl_to_number(a) for a in args)
        if name == "Times":
            out = 1
            for a in args:
                out *= wl_to_number(a)
            return out
        if name == "Power":
            return wl_to_number(args[0]) ** wl_to_number(args[1])
        raise TypeError(f"Unsupported WLFunction: {name}")
    # fallback
    return complex(x)

def mat_to_complex_array(mat, nb):
    out = np.empty((nb, nb), dtype=np.complex128)
    for i in range(nb):
        for j in range(nb):
            out[i, j] = wl_to_number(mat[i][j])
    return out

# =====
# 2) Load TB basis from WXF
# =====
def load_tb_basis_wxf(filename, debug=False):
    if not os.path.exists(filename):
        raise FileNotFoundError(f"TB WXF not found: {filename}")

    with open(filename, "rb") as f:
        expr = binary_deserialize(f)

    pars = np.array([str(p) for p in expr["pars"]])
    kpts = np.array(expr["kpts"], dtype=float) # (Nk,2) or (Nk,3) depending on export
    H0k_raw = expr["H0k"]
    Hbk_raw = expr["Hbk"]

    Nk = len(H0k_raw)
    nb = len(H0k_raw[0])
    Np = len(pars)

    H0k = np.empty((Nk, nb, nb), dtype=np.complex128)
    Hbk = np.empty((Nk, Np, nb, nb), dtype=np.complex128)

```

```

for ik in range(Nk):
    H0k[ik] = mat_to_complex_array(H0k_raw[ik], nb)
    for jp in range(Np):
        Hbk[ik, jp] = mat_to_complex_array(Hbk_raw[ik][jp], nb)

# Hermitian safety
H0k = (H0k + H0k.transpose(0, 2, 1).conj()) / 2.0
Hbk = (Hbk + Hbk.transpose(0, 1, 3, 2).conj()) / 2.0

if debug:
    print(f"[TB] Nk={Nk}, nb={nb}, Nparams={Np}")
    print("kpts[0:3]:", kpts[:3])

return pars, kpts, H0k, Hbk

# =====
# 3) Parse wannier90_band.dat
# Format: two columns (kdist, energy), bands separated by blank lines
# =====
def load_wannier_bandedat(filename, ef_shift=0.0, debug=False):
    if not os.path.exists(filename):
        raise FileNotFoundError(f"Wannier band file not found: {filename}")

    blocks, buf = [], []
    with open(filename, "r") as f:
        for line in f:
            s = line.strip()
            if (not s) or s.startswith("#"):
                if buf:
                    blocks.append(np.array(buf, dtype=float))
                    buf = []
                continue
            parts = s.split()
            if len(parts) < 2:
                continue
            kd = float(parts[0])
            en = float(parts[1]) - ef_shift
            buf.append([kd, en])
    if buf:
        blocks.append(np.array(buf, dtype=float))

    if len(blocks) == 0:
        raise ValueError("No band blocks parsed from wannier90_band.dat")

    Nk = len(blocks[0])
    kdist = blocks[0][:, 0]
    bands = []
    for b in blocks:
        if len(b) != Nk:
            continue
        bands.append(b[:, 1])
    Eall = np.stack(bands, axis=1) # (Nk, Nb_total)

    if debug:
        print(f"[WANNIER] Nk={Nk}, Nb_total={Eall.shape[1]}")
        print("kdist[0:3]:", kdist[:3])
        print("E range (min,max):", float(Eall.min()), float(Eall.max()))

    return kdist, Eall

# =====
# 4) Optional: Parse VASPKIT BAND.dat (band blocks)
# =====
def load_vaspkit_bandedat_auto(filename, ef):
    blocks, buf = [], []
    with open(filename, "r") as f:
        for line in f:

```

```

        s = line.strip()
        if (not s) or s.startswith("#"):
            if buf:
                blocks.append(np.array(buf, dtype=float))
                buf = []
            continue
        a, b = s.split()[:2]
        buf.append([float(a), float(b) - ef])
    if buf:
        blocks.append(np.array(buf, dtype=float))

    Nk = len(blocks[0])
    kdist = blocks[0][:, 0]
    bands = [b[:, 1] for b in blocks if len(b) == Nk]
    Eall = np.stack(bands, axis=1) # (Nk, Nb_total)
    return kdist, Eall

# =====
# 5) Fixed band index selection (global 12 columns)
# =====
def pick_fixed_band_indices(Eall, n=12, kref=0, target=0.0):
    if kref < 0 or kref >= Eall.shape[0]:
        raise ValueError("kref out of range")
    e0 = Eall[kref, :]
    order = np.argsort(np.abs(e0 - target))
    chosen = np.sort(order[:n]) # keep stable band order by index
    return chosen

def select_bands_fixed_order(Eall, indices):
    return Eall[:, indices]

# =====
# 6) kdist utilities
# =====
def kdist_from_kpts(kpts):
    kpts = np.asarray(kpts, dtype=float)
    dk = np.linalg.norm(np.diff(kpts, axis=0), axis=1)
    kd = np.concatenate([[0.0], np.cumsum(dk)])
    return kd

def rescale_kdist(kd_tb, kd_ref):
    if kd_tb[-1] == 0:
        return kd_tb.copy()
    return kd_tb * (kd_ref[-1] / kd_tb[-1])

def make_unique_monotonic(x, Y):
    """
    For duplicate x (common at high symmetry joints):
    - average Y for identical x
    - return strictly monotonic x
    """
    x = np.asarray(x)
    Y = np.asarray(Y)
    ux, inv = np.unique(x, return_inverse=True)
    Y2 = np.zeros((len(ux),) + Y.shape[1:], dtype=float)
    cnt = np.zeros(len(ux), dtype=int)
    for i, g in enumerate(inv):
        Y2[g] += Y[i]
        cnt[g] += 1
    if Y2.ndim == 1:
        Y2 = Y2 / cnt
    else:
        Y2 = Y2 / cnt.reshape(-1, *[1] * (Y2.ndim - 1))
    return ux, Y2

# =====

```

```

# 7) TB eigenvalues (simple: energy-sorted at each k)
# =====
def tb_bands_allk(H0k, Hbk, p_full):
    Nk = H0k.shape[0]
    nb = H0k.shape[1]
    E = np.empty((Nk, nb), dtype=float)
    for ik in range(Nk):
        H = H0k[ik] + np.tensordot(p_full, Hbk[ik], axes=(0, 0))
        H = (H + H.conj().T) / 2.0
        E[ik] = np.linalg.eigvalsh(H).real # ascending
    return E

# =====
# 8) TB eigenvalues with band-matching protection (tracking)
# =====
def tb_bands_allk_tracked(H0k, Hbk, p_full, Eref_k0=None, kref_match=0):
    """
    Compute TB eigenvalues on all k points, with band-order tracking protection.

    Steps:
    A) At kref_match: align TB band order to reference Eref_k0 by energy matching (Hungarian).
    B) Track forward/backward by maximizing eigenvector overlaps.

    Returns:
    E_tracked: (Nk, nb) float, energies in a consistent tracked order.
    """
    Nk = H0k.shape[0]
    nb = H0k.shape[1]

    # solve eigensystem at all k
    evals = np.empty((Nk, nb), dtype=float)
    evecs = np.empty((Nk, nb, nb), dtype=np.complex128) # columns are eigenvectors
    for ik in range(Nk):
        H = H0k[ik] + np.tensordot(p_full, Hbk[ik], axes=(0, 0))
        H = (H + H.conj().T) / 2.0
        w, v = np.linalg.eigh(H) # ascending w; v columns are eigenvectors
        evals[ik] = w.real
        evecs[ik] = v

    perm = np.zeros((Nk, nb), dtype=int)

    # anchor matching at kref_match
    if Eref_k0 is None:
        perm[kref_match] = np.arange(nb, dtype=int)
    else:
        tb0 = evals[kref_match]
        ref0 = np.asarray(Eref_k0, dtype=float)
        if ref0.shape[0] != nb:
            raise ValueError(f"Eref_k0 length {ref0.shape[0]} != nb {nb}")

        C = (tb0[:, None] - ref0[None, :]) ** 2
        row_ind, col_ind = linear_sum_assignment(C) # minimize cost
        # want mapping tracked(ref index) -> tb eigen-index
        ref_to_tb = np.zeros(nb, dtype=int)
        ref_to_tb[col_ind] = row_ind
        perm[kref_match] = ref_to_tb

    def match_by_overlap(v_prev, v_cur, prev_map):
        # v_prev, v_cur: (nb, nb), columns are eigenvectors
        Vp = v_prev[:, prev_map] # columns in tracked order
        Vc = v_cur
        O = np.abs(Vp.conj().T @ Vc) ** 2 # (nb, nb)
        cost = 1.0 - O # maximize overlap
        row_ind, col_ind = linear_sum_assignment(cost)
        cur_map = np.zeros(nb, dtype=int)
        cur_map[row_ind] = col_ind
        return cur_map

```

```

# forward
for ik in range(kref_match + 1, Nk):
    perm[ik] = match_by_overlap(evecs[ik - 1], evecs[ik], perm[ik - 1])

# backward
for ik in range(kref_match - 1, -1, -1):
    perm[ik] = match_by_overlap(evecs[ik + 1], evecs[ik], perm[ik + 1])

E_tracked = np.empty_like(evals)
for ik in range(Nk):
    E_tracked[ik] = evals[ik, perm[ik]]

return E_tracked

# =====
# 9) Residual builders (kdist-aligned)
# =====
def residual_kdist_align_fixed_order(p_var, pars, kpts, H0k, Hbk,
                                     kd_ref, Eref,
                                     idx_e, idx_t, idx_r,
                                     e_fixed=None, t_fixed=None, r_fixed=None,
                                     lam=0.0,
                                     fit_mask=("e", "t", "r")):
    """
    Fixed-order residual: assumes energy-sorted eigenvalues at each k already correspond to Eref order.
    """
    Np = len(pars)
    p_full = np.zeros(Np, dtype=float)

    if e_fixed is not None:
        p_full[idx_e] = e_fixed
    if t_fixed is not None:
        p_full[idx_t] = t_fixed
    if r_fixed is not None:
        p_full[idx_r] = r_fixed

    cursor = 0
    if "e" in fit_mask:
        p_full[idx_e] = p_var[cursor: cursor + len(idx_e)]
        cursor += len(idx_e)
    if "t" in fit_mask:
        p_full[idx_t] = p_var[cursor: cursor + len(idx_t)]
        cursor += len(idx_t)
    if "r" in fit_mask:
        p_full[idx_r] = p_var[cursor: cursor + len(idx_r)]
        cursor += len(idx_r)

    E_tb = tb_bands_allk(H0k, Hbk, p_full)
    kd_tb = rescale_kdist(kdist_from_kpts(kpts), kd_ref)

    kd_tb_u, E_tb_u = make_unique_monotonic(kd_tb, E_tb)
    kd_ref_u, Eref_u = make_unique_monotonic(kd_ref, Eref)

    E_tb_on_ref = np.empty_like(Eref_u)
    for n in range(Eref_u.shape[1]):
        E_tb_on_ref[:, n] = np.interp(kd_ref_u, kd_tb_u, E_tb_u[:, n])

    res = (E_tb_on_ref - Eref_u).ravel()

    if lam and lam > 0:
        res = np.concatenate([res, np.sqrt(lam) * p_var])

    return res

def residual_kdist_align_bandmatch(p_var, pars, kpts, H0k, Hbk,
                                   kd_ref, Eref,
                                   idx_e, idx_t, idx_r,

```

```

        e_fixed=None, t_fixed=None, r_fixed=None,
        lam=0.0,
        fit_mask=("e", "t", "r"),
        kref_match=0):
    """
    Band-matching protected residual:
    - Hungarian energy matching at kref_match to align TB band order to Eref
    - eigenvector-overlap tracking along k to avoid band-index swapping
    """
    Np = len(pars)
    p_full = np.zeros(Np, dtype=float)

    if e_fixed is not None:
        p_full[idx_e] = e_fixed
    if t_fixed is not None:
        p_full[idx_t] = t_fixed
    if r_fixed is not None:
        p_full[idx_r] = r_fixed

    cursor = 0
    if "e" in fit_mask:
        p_full[idx_e] = p_var[cursor: cursor + len(idx_e)]
        cursor += len(idx_e)
    if "t" in fit_mask:
        p_full[idx_t] = p_var[cursor: cursor + len(idx_t)]
        cursor += len(idx_t)
    if "r" in fit_mask:
        p_full[idx_r] = p_var[cursor: cursor + len(idx_r)]
        cursor += len(idx_r)

    # Anchor energies at kref_match from reference bands (assumes consistent k-path ordering)
    Eref_anchor = np.asarray(Eref[kref_match], dtype=float).copy()

    E_tb = tb_bands_allk_tracked(H0k, Hbk, p_full, Eref_k0=Eref_anchor, kref_match=kref_match)

    kd_tb = rescale_kdist(kdist_from_kpts(kpts), kd_ref)

    kd_tb_u, E_tb_u = make_unique_monotonic(kd_tb, E_tb)
    kd_ref_u, Eref_u = make_unique_monotonic(kd_ref, Eref)

    E_tb_on_ref = np.empty_like(Eref_u)
    for n in range(Eref_u.shape[1]):
        E_tb_on_ref[:, n] = np.interp(kd_ref_u, kd_tb_u, E_tb_u[:, n])

    res = (E_tb_on_ref - Eref_u).ravel()

    if lam and lam > 0:
        res = np.concatenate([res, np.sqrt(lam) * p_var])

    return res

# =====
# 10) IO helpers
# =====
def save_params_txt(filename, pars, idx_list, values):
    with open(filename, "w", encoding="utf-8") as f:
        for idx, val in zip(idx_list, values):
            f.write(f"{pars[idx]} {val:.10f}\n")

# =====
# 11) Main Stage1-4
# =====
def main():
    # ----- Load TB basis -----
    pars, kpts, H0k, Hbk = load_tb_basis_wxf(TB_FILE, debug=True)
    nb = H0k.shape[1]
    print(f"[TB] nb = {nb} (your TB model size)")

```

```

# param indices
idx_e = np.where(np.char.startswith(pars, "e"))[0]
idx_t = np.where(np.char.startswith(pars, "t"))[0]
idx_r = np.where(np.char.startswith(pars, "r"))[0]
print(f"# e params = {len(idx_e)}")
print(f"# t params = {len(idx_t)}")
print(f"# r params = {len(idx_r)}")
if nb != N_TARGET_BANDS:
    print(f"WARNING: TB nb={nb} but N_TARGET_BANDS={N_TARGET_BANDS}. "
          f"Your TB model should be 12-band if you fit 12 bands.")

# ----- Load Wannier reference -----
kd_w, Eall_w = load_wannier_bandedat(WANNIER_BAND_FILE, ef_shift=EF_SHIFT_WANNIER, debug=True)
print("Eall_w shape:", Eall_w.shape)

# ----- Pick fixed 12 band indices -----
fixed_indices = pick_fixed_band_indices(Eall_w, n=N_TARGET_BANDS, kref=KREF_FOR_PICK, target=E_TARGET)
print("Fixed band indices (0-based):", fixed_indices.tolist())
print("Fixed band indices (1-based):", (fixed_indices + 1).tolist())
Eref = select_bands_fixed_order(Eall_w, fixed_indices) # (Nk_ref, 12)

# ----- Bounds -----
lb_e = -E_BOUND * np.ones(len(idx_e))
ub_e = +E_BOUND * np.ones(len(idx_e))
lb_t = -T_BOUND * np.ones(len(idx_t))
ub_t = +T_BOUND * np.ones(len(idx_t))
lb_r = -R_BOUND * np.ones(len(idx_r))
ub_r = +R_BOUND * np.ones(len(idx_r))

# ----- Stage-1: fit e only -----
print("\n===== Stage-1: fitting e only (Wannier ref) =====")
p0_e = np.zeros(len(idx_e), dtype=float)
res1 = least_squares(
    residual_kdist_align_fixed_order,
    p0_e,
    bounds=(lb_e, ub_e),
    args=(pars, kpts, H0k, Hbk, kd_w, Eref, idx_e, idx_t, idx_r,
          None, np.zeros(len(idx_t)), np.zeros(len(idx_r)),
          LAMBDA_E, ("e",)),
    method="trf",
    loss="linear",
    x_scale="jac",
    diff_step=DIFF_STEP,
    verbose=2,
    max_nfev=MAX_NFEV_1
)
e_fit = res1.x.copy()
save_params_txt("fitted_stage1_e_wannier.txt", pars, idx_e, e_fit)
print("Stage-1 cost =", res1.cost)
print("Saved: fitted_stage1_e_wannier.txt")

# ----- Stage-2: fit t only (e fixed) -----
print("\n===== Stage-2: fitting t only (e fixed, Wannier ref) =====")
p0_t = np.zeros(len(idx_t), dtype=float)
res2 = least_squares(
    residual_kdist_align_fixed_order,
    p0_t,
    bounds=(lb_t, ub_t),
    args=(pars, kpts, H0k, Hbk, kd_w, Eref, idx_e, idx_t, idx_r,
          e_fit, None, np.zeros(len(idx_r)),
          LAMBDA_T, ("t",)),
    method="trf",
    loss="linear",
    x_scale="jac",
    diff_step=DIFF_STEP,
    verbose=2,
    max_nfev=MAX_NFEV_2
)

```

```

t_fit = res2.x.copy()
save_params_txt("fitted_stage2_t_wannier.txt", pars, idx_t, t_fit)
print("Stage-2 cost =", res2.cost)
print("Saved: fitted_stage2_t_wannier.txt")

# ----- Stage-3: fit r only (e,t fixed) -----
print("\n===== Stage-3: fitting r only (e,t fixed, Wannier ref) =====")
p0_r = np.zeros(len(idx_r), dtype=float)
res3 = least_squares(
    residual_kdist_align_fixed_order,
    p0_r,
    bounds=(lb_r, ub_r),
    args=(pars, kpts, H0k, Hbk, kd_w, Eref, idx_e, idx_t, idx_r,
          e_fit, t_fit, None,
          LAMBDA_R, ("r",)),
    method="trf",
    loss="linear",
    x_scale="jac",
    diff_step=DIFF_STEP,
    verbose=2,
    max_nfev=MAX_NFEV_3
)
r_fit = res3.x.copy()
save_params_txt("fitted_stage3_r_wannier.txt", pars, idx_r, r_fit)
print("Stage-3 cost =", res3.cost)
print("Saved: fitted_stage3_r_wannier.txt")

# ----- Stage-4: fit e,t,r together (warm start) -----
print("\n===== Stage-4: fitting e,t,r together (warm start, Wannier ref) =====")
p0_etr = np.concatenate([e_fit, t_fit, r_fit], axis=0)
lb_etr = np.concatenate([lb_e, lb_t, lb_r], axis=0)
ub_etr = np.concatenate([ub_e, ub_t, ub_r], axis=0)

lam_etr = 1e-8

if USE_BANDMATCH_STAGE4:
    print(f"[Stage-4] Using band-matching protection (kref_match={KREF_MATCH})")
    res4 = least_squares(
        residual_kdist_align_bandmatch,
        p0_etr,
        bounds=(lb_etr, ub_etr),
        args=(pars, kpts, H0k, Hbk, kd_w, Eref, idx_e, idx_t, idx_r,
              None, None, None,
              lam_etr, ("e", "t", "r"),
              KREF_MATCH),
        method="trf",
        loss="linear",
        x_scale="jac",
        diff_step=DIFF_STEP,
        verbose=2,
        max_nfev=MAX_NFEV_4
    )
else:
    print("[Stage-4] Using fixed-order residual (no band-matching)")
    res4 = least_squares(
        residual_kdist_align_fixed_order,
        p0_etr,
        bounds=(lb_etr, ub_etr),
        args=(pars, kpts, H0k, Hbk, kd_w, Eref, idx_e, idx_t, idx_r,
              None, None, None,
              lam_etr, ("e", "t", "r")),
        method="trf",
        loss="linear",
        x_scale="jac",
        diff_step=DIFF_STEP,
        verbose=2,
        max_nfev=MAX_NFEV_4
    )

```

```

etr_fit = res4.x.copy()
e4 = etr_fit[:len(idx_e)]
t4 = etr_fit[len(idx_e):len(idx_e) + len(idx_t)]
r4 = etr_fit[len(idx_e) + len(idx_t):]

save_params_txt("fitted_stage4_etr_wannier_e.txt", pars, idx_e, e4)
save_params_txt("fitted_stage4_etr_wannier_t.txt", pars, idx_t, t4)
save_params_txt("fitted_stage4_etr_wannier_r.txt", pars, idx_r, r4)
print("Stage-4 cost =", res4.cost)
print("Saved: fitted_stage4_etr_wannier_[e/t/r].txt")

# ----- Optional: Cross validation on DFT BAND.dat -----
if DFT_BAND_FILE is not None and os.path.exists(DFT_BAND_FILE):
    print("\n==== Cross validation on DFT BAND.dat (no refit) =====")
    kd_d, Eall_d = load_vaspkit_bandedat_auto(DFT_BAND_FILE, EF_DFT)
    dft_indices = pick_fixed_band_indices(Eall_d, n=N_TARGET_BANDS, kref=KREF_FOR_PICK, target=0.0)
    Eref_d = select_bands_fixed_order(Eall_d, dft_indices)

    p_full = np.zeros(len(pars))
    p_full[idx_e] = e4
    p_full[idx_t] = t4
    p_full[idx_r] = r4

    if USE_BANDMATCH_STAGE4:
        # band tracking for evaluation as well (anchor to DFT at kref)
        E_tb = tb_bands_allk_tracked(H0k, Hbk, p_full, Eref_k0=Eref_d[KREF_MATCH], kref_match=KREF_MATCH)
    else:
        E_tb = tb_bands_allk(H0k, Hbk, p_full)

    kd_tb = rescale_kdist(kdist_from_kpts(kpts), kd_d)
    kd_tb_u, E_tb_u = make_unique_monotonic(kd_tb, E_tb)
    kd_d_u, Eref_d_u = make_unique_monotonic(kd_d, Eref_d)

    E_tb_on_d = np.empty_like(Eref_d_u)
    for n in range(Eref_d_u.shape[1]):
        E_tb_on_d[:, n] = np.interp(kd_d_u, kd_tb_u, E_tb_u[:, n])

    rms = np.sqrt(np.mean((E_tb_on_d - Eref_d_u) ** 2))
    print(f"DFT RMS error (eV) = {rms:.6f}")
    print("NOTE: DFT band indices used (1-based):", (dft_indices + 1).tolist())

print("\nDONE.")

if __name__ == "__main__":
    main()

```

(原创自研) 利用 wannier 验证超超交换路径

► 代码清单 A.57.1 (python, 3 行)

```

wannier90_centres.xyz
wannier90_hr.dat
wannier90.wout

```

► 代码清单 A.57.2 (python, 3 行)

```
wannier90_hr.dat
wannier90_centres.xyz
wannier90.win
```

► 代码清单 A.57.3 (python, 2 行)

```
1step.csv
2step.csv
```

► 代码清单 A.57.4 (python, 357 行)

```
import re
import csv
import math
import argparse
from collections import defaultdict, namedtuple

import numpy as np

Edge = namedtuple("Edge", "pair shell dist absH re im Rx Ry Rz m n atom_m atom_n elem_m elem_n")

# =====
# Parse wannier90.win
# =====
def _extract_block(txt, block_name):
    m = re.search(rf"begin\s+{block_name}(.*?)end\s+{block_name}", txt, re.S | re.I)
    return None if not m else m.group(1).strip()

def parse_win_lattice_and_atoms(win_path):
    """
    Read unit_cell_cart and atoms_cart from wannier90.win.
    Assumes Angstrom if unit not specified (your file matches this).
    Returns:
        A (3x3) with columns = a1,a2,a3 in Angstrom, so cart = A @ frac
        atoms: list of dict {id, elem, r_cart(np.array shape(3))}
    """
    with open(win_path, "r", encoding="utf-8", errors="ignore") as f:
        txt = f.read()

    cell_block = _extract_block(txt, "unit_cell_cart")
    if cell_block is None:
        raise RuntimeError("Cannot find unit_cell_cart block in wannier90.win")

    lines = [ln.strip() for ln in cell_block.splitlines() if ln.strip()]
    # Some files might include "Ang" or "Bohr" as first line; yours doesn't.
    unit = None
    if re.match(r"^(ang|angstrom|bohr)\b", lines[0], re.I):
        unit = lines[0].lower()
        vec_lines = lines[1:4]
    else:
        unit = "ang"
        vec_lines = lines[0:3]
```

```

if len(vec_lines) < 3:
    raise RuntimeError("unit_cell_cart has < 3 lattice vectors")

a1 = np.array([float(x) for x in vec_lines[0].split()[1:3]], dtype=float)
a2 = np.array([float(x) for x in vec_lines[1].split()[1:3]], dtype=float)
a3 = np.array([float(x) for x in vec_lines[2].split()[1:3]], dtype=float)

if "bohr" in unit:
    bohr_to_ang = 0.52917721092
    a1 *= bohr_to_ang
    a2 *= bohr_to_ang
    a3 *= bohr_to_ang

# A columns are lattice vectors
A = np.stack([a1, a2, a3], axis=1) # shape (3,3)

atoms_block = _extract_block(txt, "atoms_cart")
if atoms_block is None:
    raise RuntimeError("Cannot find atoms_cart block in wannier90.win")

atoms = []
for idx, ln in enumerate([x for x in atoms_block.splitlines() if x.strip()], start=1):
    parts = ln.split()
    if len(parts) < 4:
        continue
    elem = parts[0]
    r = np.array([float(parts[1]), float(parts[2]), float(parts[3])], dtype=float)
    atoms.append({"id": idx, "elem": elem, "r": r})

if not atoms:
    raise RuntimeError("atoms_cart parsed but got 0 atoms")

return A, atoms

# =====
# Parse wannier90_centres.xyz
# =====
def parse_centres_xyz(xyz_path):
    """
    Parse wannier90_centres.xyz:
    line1 = N
    line2 = comment
    then N lines: <label> x y z
    Return centers list indexed 1..N: centers[wf] = np.array([x,y,z])
    """
    with open(xyz_path, "r", encoding="utf-8", errors="ignore") as f:
        lines = [ln.strip() for ln in f if ln.strip()]

    n = int(lines[0])
    if len(lines) < 2 + n:
        raise RuntimeError(f"centres.xyz incomplete: need {2+n} lines, got {len(lines)}")

    centers = [None] * (n + 1)
    for i in range(n):
        parts = lines[2 + i].split()
        if len(parts) < 4:
            raise RuntimeError(f"Bad centres.xyz line: {lines[2+i]}")
        centers[i + 1] = np.array([float(parts[1]), float(parts[2]), float(parts[3])], dtype=float)
    return centers

# =====
# Parse wannier90_hr.dat
# =====
def parse_hr_dat(hr_path):
    """
    Yields (Rx, Ry, Rz, m, n, reH, imH)
    """

```

```

with open(hr_path, "r", encoding="utf-8", errors="ignore") as f:
    _ = f.readline() # comment
    num_wann = int(f.readline().strip())
    nrpts = int(f.readline().strip())

    # read degeneracy list
    degen = []
    while len(degen) < nrpts:
        ln = f.readline()
        if not ln:
            raise RuntimeError("Unexpected EOF while reading degeneracy list")
        ln = ln.strip()
        if not ln:
            continue
        degen += [int(x) for x in ln.split()]

    # data
    for ln in f:
        ln = ln.strip()
        if not ln:
            continue
        parts = ln.split()
        if len(parts) < 7:
            continue
        Rx, Ry, Rz = int(parts[0]), int(parts[1]), int(parts[2])
        m, n = int(parts[3]), int(parts[4])
        reH, imH = float(parts[5]), float(parts[6])
        yield Rx, Ry, Rz, m, n, reH, imH

# =====
# WF grouping (edit ranges here)
# =====
def wf_group(wf):
    # Your current mapping:
    if 1 <= wf <= 10:
        return "Tc_d"
    if 11 <= wf <= 20:
        return "Ir_d"
    if 21 <= wf <= 56:
        return "Se_p"
    if 57 <= wf <= 68:
        return "Ge_p"
    return "OTHER"

def pair_name(g1, g2):
    # Use a fixed ordering to avoid the earlier "Se_p<->Tc_d" name surprise.
    order = {"Tc_d": 1, "Ir_d": 2, "Se_p": 3, "Ge_p": 4, "OTHER": 99}
    if order[g1] <= order[g2]:
        return f"{g1}<->{g2}"
    return f"{g2}<->{g1}"

# =====
# Minimum-image mapping: WF -> nearest atom
# =====
def cart_to_frac(A, r_cart):
    # cart = A @ frac
    return np.linalg.solve(A, r_cart)

def frac_to_cart(A, f):
    return A @ f

def wrap_delta_frac(df):
    # to [-0.5, 0.5)
    return df - np.round(df)

def map_wf_to_atoms(A, centers, atoms):

```

```

"""
Returns:
    wf_atom_id[wf] = atom index in atoms list (0-based)
    wf_atom_elem[wf] = element string
Uses minimum-image distance in fractional space.
"""

inv_needed = False # we use solve each time; stable enough for 68 WF

atom_frac = []
for a in atoms:
    atom_frac.append(cart_to_frac(A, a["r"]))
atom_frac = np.array(atom_frac) # shape (Nat,3)

wf_atom_id = [None] * len(centers)
wf_atom_elem = [None] * len(centers)

for wf in range(1, len(centers)):
    r = centers[wf]
    f_w = cart_to_frac(A, r)

    # compute minimum image distance to each atom
    best_i = None
    best_d = 1e30
    for i, f_a in enumerate(atom_frac):
        df = wrap_delta_frac(f_w - f_a)
        dr = frac_to_cart(A, df)
        d = float(np.linalg.norm(dr))
        if d < best_d:
            best_d = d
            best_i = i

    wf_atom_id[wf] = best_i
    wf_atom_elem[wf] = atoms[best_i]["elem"]

return wf_atom_id, wf_atom_elem

# =====
# Main
# =====
def main():
    ap = argparse.ArgumentParser(description="Step2 (interatomic): summarize hoppings by subspace pair + distance
    ↪ shell")
    ap.add_argument("--win", required=True, help="wannier90.win")
    ap.add_argument("--centres", required=True, help="wannier90_centres.xyz")
    ap.add_argument("--hr", required=True, help="wannier90_hr.dat")
    ap.add_argument("--tol", type=float, default=0.10, help="distance bin size (Ang), e.g. 0.05~0.15")
    ap.add_argument("--min_absH", type=float, default=1e-4, help="min |H| kept (eV)")
    ap.add_argument("--topk", type=int, default=30, help="topK edges per (pair,shell)")
    ap.add_argument("--skip_same_atom_R0", action="store_true",
        help="skip terms where (atom_m==atom_n) AND (R==0). Recommended to remove onsite/local
        ↪ terms.")
    ap.add_argument("--skip_diag_R0", action="store_true",
        help="skip diagonal terms (m==n) at R==0 (onsite). Usually redundant if skip_same_atom_R0 is
        ↪ on.")
    ap.add_argument("--pairs", default="ALL", help="comma-separated allowed pairs like Tc_d<->Se_p, or ALL")
    ap.add_argument("--out_summary", default="hopping_summary_by_shell.csv")
    ap.add_argument("--out_edges", default="", help="optional: write filtered edges csv (empty=off)")
    args = ap.parse_args()

    A, atoms = parse_win_lattice_and_atoms(args.win)
    centers = parse_centres_xyz(args.centres)

    wf_atom_id, wf_atom_elem = map_wf_to_atoms(A, centers, atoms)

    # Allowed pairs
    allowed_pairs = None
    if args.pairs.strip().upper() != "ALL":

```

```

    allowed_pairs = set([p.strip() for p in args.pairs.split(",") if p.strip()])

# stats
cnt = defaultdict(int)
sumsq = defaultdict(float)
maxv = defaultdict(float)
top_edges = defaultdict(list)
filtered_edges = []

# Precompute lattice vectors for translation
a1 = A[:, 0]
a2 = A[:, 1]
a3 = A[:, 2]

def R_to_T(Rx, Ry, Rz):
    return Rx * a1 + Ry * a2 + Rz * a3

for Rx, Ry, Rz, m, n, reH, imH in parse_hr_dat(args.hr):
    absH = math.hypot(reH, imH)
    if absH < args.min_absH:
        continue

    gm = wf_group(m)
    gn = wf_group(n)
    if gm == "OTHER" or gn == "OTHER":
        continue

    pair = pair_name(gm, gn)
    if allowed_pairs is not None and pair not in allowed_pairs:
        continue

    atom_m = wf_atom_id[m]
    atom_n = wf_atom_id[n]
    elem_m = wf_atom_elem[m]
    elem_n = wf_atom_elem[n]

    # Skip onsite/local terms if requested
    if (Rx, Ry, Rz) == (0, 0, 0):
        if args.skip_diag_R0 and (m == n):
            continue
        if args.skip_same_atom_R0 and (atom_m == atom_n):
            continue

    # distance between WF centers considering translation R
    T = R_to_T(Rx, Ry, Rz)
    dr = (centers[n] + T) - centers[m]
    dist = float(np.linalg.norm(dr))

    # bin to shell
    shell = round(dist / args.tol) * args.tol
    key = (pair, shell)

    cnt[key] += 1
    sumsq[key] += absH * absH
    if absH > maxv[key]:
        maxv[key] = absH

    e = Edge(pair, shell, dist, absH, reH, imH, Rx, Ry, Rz, m, n, atom_m, atom_n, elem_m, elem_n)

    lst = top_edges[key]
    lst.append(e)
    lst.sort(key=lambda x: x.absH, reverse=True)
    if len(lst) > args.topk:
        lst[:] = lst[:args.topk]

    if args.out_edges:
        filtered_edges.append(e)

# write summary

```

```

with open(args.out_summary, "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f)
    w.writerow([
        "pair", "shell_A", "count", "max_absH_eV", "rms_absH_eV",
        f"top{args.topk}_edges(m,n,atom_m,atom_n,elem_m,elem_n,Rx,Ry,Rz,absH,dist)"
    ])
    for (pair, shell) in sorted(cnt.keys(), key=lambda x: (x[0], x[1])):
        c = cnt[(pair, shell)]
        rms = math.sqrt(sumsq[(pair, shell)] / c) if c else 0.0
        tops = top_edges[(pair, shell)]
        tops_str = "; ".join([
            f"{e.m}-{e.n}|a{e.atom_m+1}-a{e.atom_n+1}|{e.elem_m}-{e.elem_n}"
            f"@({e.Rx},{e.Ry},{e.Rz})|{e.absH:.6g}|d={e.dist:.3f}"
            for e in tops
        ])
        w.writerow([pair, f"{shell:.3f}", c, f"{maxv[(pair, shell)]:.6g}", f"{rms:.6g}", tops_str])

# optional edges
if args.out_edges:
    with open(args.out_edges, "w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow([
            "pair", "shell_A", "dist_A", "absH_eV", "re", "im",
            "Rx", "Ry", "Rz", "m", "n",
            "atom_m", "atom_n", "elem_m", "elem_n"
        ])
        for e in filtered_edges:
            w.writerow([
                e.pair, f"{e.shell:.3f}", f"{e.dist:.6f}", f"{e.absH:.8g}",
                f"{e.re:.8g}", f"{e.im:.8g}",
                e.Rx, e.Ry, e.Rz, e.m, e.n,
                e.atom_m+1, e.atom_n+1, e.elem_m, e.elem_n
            ])

print("Done.")
print("Summary:", args.out_summary)
if args.out_edges:
    print("Edges:", args.out_edges)

if __name__ == "__main__":
    main()

```

► 代码清单 A.57.6 (python, 271 行)

```

import csv
import math
import argparse
from dataclasses import dataclass
from collections import defaultdict
from typing import Tuple, List, Dict, Iterable

@dataclass(frozen=True)
class DirEdge:
    # directed edge: u(cell_shift) -> v(cell_shift + R)
    u_atom: int          # 1-based
    v_atom: int          # 1-based
    u_elem: str
    v_elem: str
    u_wf: int
    v_wf: int
    Rx: int

```

```

Ry: int
Rz: int
absH: float
dist: float

def parse_edges_csv(path: str) -> List[DirEdge]:
    """
    Read out_edges csv produced by step2_interatomic.py (your 2step.csv).
    Expected columns include:
        pair, shell_A, dist_A, absH_eV, re, im, Rx, Ry, Rz, m, n, atom_m, atom_n, elem_m, elem_n
    atom_* are 1-based in the file.
    """
    edges = []
    with open(path, "r", encoding="utf-8", errors="ignore", newline="") as f:
        r = csv.DictReader(f)
        required = ["dist_A", "absH_eV", "Rx", "Ry", "Rz", "m", "n", "atom_m", "atom_n", "elem_m", "elem_n"]
        for k in required:
            if k not in r.fieldnames:
                raise RuntimeError(f"Missing column '{k}' in {path}. Found: {r.fieldnames}")

        for row in r:
            try:
                dist = float(row["dist_A"])
                absH = float(row["absH_eV"])
                Rx, Ry, Rz = int(row["Rx"]), int(row["Ry"]), int(row["Rz"])
                m, n = int(row["m"]), int(row["n"])
                atom_m, atom_n = int(row["atom_m"]), int(row["atom_n"])
                elem_m, elem_n = row["elem_m"].strip(), row["elem_n"].strip()
            except Exception:
                continue

            # Create BOTH directions:
            # m(home) -> n(shift R)
            edges.append(DirEdge(atom_m, atom_n, elem_m, elem_n, m, n, Rx, Ry, Rz, absH, dist))
            # n(home) -> m(shift -R)
            edges.append(DirEdge(atom_n, atom_m, elem_n, elem_m, n, m, -Rx, -Ry, -Rz, absH, dist))
    return edges

def is_pair(e: DirEdge, a: str, b: str) -> bool:
    return (e.u_elem == a and e.v_elem == b)

def add_shift(s: Tuple[int, int, int], R: Tuple[int, int, int]) -> Tuple[int, int, int]:
    return (s[0] + R[0], s[1] + R[1], s[2] + R[2])

def abs_R(R: Tuple[int, int, int]) -> int:
    return abs(R[0]) + abs(R[1]) + abs(R[2])

def main():
    ap = argparse.ArgumentParser(description="Step4: build Tc-Se-Ir-Se-Tc paths from out_edges csv")
    ap.add_argument("--edges", required=True, help="Input edges csv, e.g. 2step.csv")
    ap.add_argument("--out", default="top_paths.csv", help="Output paths csv")

    ap.add_argument("--d_tcse", type=float, default=3.0, help="Max distance for Tc-Se edges (Ang)")
    ap.add_argument("--d_seir", type=float, default=3.0, help="Max distance for Se-Ir edges (Ang)")

    ap.add_argument("--min_absH", type=float, default=1e-3,
                    help="Min |t| for edges used in path building (eV). "
                    "Use 1e-4 if you want everything, 1e-3 speeds up a lot.")
    ap.add_argument("--top_per_se", type=int, default=60,
                    help="Keep top N Tc-Se edges per Se atom (after filtering)")
    ap.add_argument("--top_per_ir_se", type=int, default=80,
                    help="Keep top N Se-Ir edges per Se atom for each direction (Se->Ir and Ir->Se)")
    ap.add_argument("--top_paths", type=int, default=500,
                    help="Number of top paths to write")

```

```

ap.add_argument("--require_same_tc_atom", action="store_true",
                help="Require end Tc atom id == start Tc atom id (useful when there is only one Tc in
                ↪ primitive cell)")
ap.add_argument("--max_netR_L1", type=int, default=6,
                help="Filter paths by L1 norm of net cell shift |dRx|+|dRy|+|dRz| <= this. "
                "Set large (e.g. 999) to disable.")
args = ap.parse_args()

# ----- load directed edges -----
all_dir = parse_edges_csv(args.edges)

# Identify Tc/Ir/Se atom ids from elements (in directed edges)
tc_atoms = sorted({e.u_atom for e in all_dir if e.u_elem == "Tc"} | {e.v_atom for e in all_dir if e.v_elem ==
↪ "Tc"})
ir_atoms = sorted({e.u_atom for e in all_dir if e.u_elem == "Ir"} | {e.v_atom for e in all_dir if e.v_elem ==
↪ "Ir"})
se_atoms = sorted({e.u_atom for e in all_dir if e.u_elem == "Se"} | {e.v_atom for e in all_dir if e.v_elem ==
↪ "Se"})

if not tc_atoms:
    raise RuntimeError("No Tc atoms found in edges file (elem == 'Tc').")
if not ir_atoms:
    raise RuntimeError("No Ir atoms found in edges file (elem == 'Ir').")
if not se_atoms:
    raise RuntimeError("No Se atoms found in edges file (elem == 'Se').")

# For this material there is typically 1 Ir in cell; if multiple, we allow any.
# We'll build paths via any Ir atom (still correct).
print(f"[INFO] Tc atoms: {tc_atoms}")
print(f"[INFO] Ir atoms: {ir_atoms}")
print(f"[INFO] Se atoms: {se_atoms}")

# ----- filter edges for path building -----
# We need:
# Tc -> Se (directed) AND Se -> Tc (directed) for last hop
# Se -> Ir (directed) AND Ir -> Se (directed) for middle hops
tc_to_se = defaultdict(list) # key: (Tc_atom) -> list[DirEdge] where Tc->Se
se_to_tc = defaultdict(list) # key: (Se_atom) -> list[DirEdge] where Se->Tc
se_to_ir = defaultdict(list) # key: (Se_atom) -> list[DirEdge] where Se->Ir
ir_to_se = defaultdict(list) # key: (Ir_atom) -> list[DirEdge] where Ir->Se

for e in all_dir:
    if e.absH < args.min_absH:
        continue

    if is_pair(e, "Tc", "Se") and e.dist <= args.d_tcse:
        tc_to_se[e.u_atom].append(e)
    elif is_pair(e, "Se", "Tc") and e.dist <= args.d_tcse:
        se_to_tc[e.u_atom].append(e)
    elif is_pair(e, "Se", "Ir") and e.dist <= args.d_seir:
        se_to_ir[e.u_atom].append(e)
    elif is_pair(e, "Ir", "Se") and e.dist <= args.d_seir:
        ir_to_se[e.u_atom].append(e)

# ----- per-atom pruning (keep strongest edges) -----
def keep_top(d: Dict[int, List[DirEdge]], topn: int):
    for k in list(d.keys()):
        lst = d[k]
        lst.sort(key=lambda x: x.absH, reverse=True)
        if len(lst) > topn:
            d[k] = lst[:topn]

# for Tc->Se: prune per Tc atom? We want per Se too, but easiest is per Tc then per Se later in path
keep_top(tc_to_se, topn=max(args.top_per_se, 50))
keep_top(se_to_tc, topn=max(args.top_per_se, 50))
keep_top(se_to_ir, topn=max(args.top_per_ir_se, 80))
keep_top(ir_to_se, topn=max(args.top_per_ir_se, 80))

```

```

# Build helper indices by Se atom for faster chaining
# Tc->Se edges grouped by Se target
tc_to_se_by_se = defaultdict(list) # Se_atom -> edges
for tc, lst in tc_to_se.items():
    for e in lst:
        tc_to_se_by_se[e.v_atom].append(e)
for se, lst in tc_to_se_by_se.items():
    lst.sort(key=lambda x: x.absH, reverse=True)
    if len(lst) > args.top_per_se:
        tc_to_se_by_se[se] = lst[:args.top_per_se]

# Se->Tc edges grouped by Se source already: se_to_tc[Se]
for se, lst in se_to_tc.items():
    lst.sort(key=lambda x: x.absH, reverse=True)
    if len(lst) > args.top_per_se:
        se_to_tc[se] = lst[:args.top_per_se]

# Se->Ir per Se
for se, lst in se_to_ir.items():
    lst.sort(key=lambda x: x.absH, reverse=True)
    if len(lst) > args.top_per_ir_se:
        se_to_ir[se] = lst[:args.top_per_ir_se]

# Ir->Se per Ir
for ir, lst in ir_to_se.items():
    lst.sort(key=lambda x: x.absH, reverse=True)
    if len(lst) > args.top_per_ir_se:
        ir_to_se[ir] = lst[:args.top_per_ir_se]

# ----- enumerate paths -----
# State uses (atom_id, cell_shift). Start Tc in home cell (0,0,0).
# Directed edge always moves to (v_atom, shift + (Rx,Ry,Rz)).
paths = []

def push_path(W, e1, e2, e3, e4, netR, tc0, tc1, se1, ir, se2):
    paths.append((
        W,
        tc0, se1, ir, se2, tc1,
        netR,
        e1, e2, e3, e4
    ))

# For each start Tc atom
for tc0 in tc_atoms:
    shift0 = (0, 0, 0)

    # Step 1: Tc -> Se1
    for e1 in tc_to_se.get(tc0, []):
        shift_se1 = add_shift(shift0, (e1.Rx, e1.Ry, e1.Rz))
        se1 = e1.v_atom

    # Step 2: Se1 -> Ir
    for e2 in se_to_ir.get(se1, []):
        shift_ir = add_shift(shift_se1, (e2.Rx, e2.Ry, e2.Rz))
        ir = e2.v_atom

    # Step 3: Ir -> Se2
    for e3 in ir_to_se.get(ir, []):
        shift_se2 = add_shift(shift_ir, (e3.Rx, e3.Ry, e3.Rz))
        se2 = e3.v_atom

    # Step 4: Se2 -> Tc1
    for e4 in se_to_tc.get(se2, []):
        shift_tc1 = add_shift(shift_se2, (e4.Rx, e4.Ry, e4.Rz))
        tc1 = e4.v_atom

    if args.require_same_tc_atom and (tc1 != tc0):
        continue

```

```

        netR = shift_tc1 # Tc1 cell relative to Tc0(home)
        if args.max_netR_L1 < 999 and abs_R(netR) > args.max_netR_L1:
            continue

        # Path weight (simple, reproducible)
        W = e1.absH * e2.absH * e3.absH * e4.absH
        push_path(W, e1, e2, e3, e4, netR, tc0, tc1, se1, ir, se2)

    if not paths:
        raise RuntimeError("No paths found. Try lowering --min_absH, increasing distance windows, or relaxing
        ↪ --max_netR_L1.")

    # Sort and keep top
    paths.sort(key=lambda x: x[0], reverse=True)
    paths = paths[:args.top_paths]

    # ----- write output -----
    with open(args.out, "w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow([
            "W_path",
            "Tc0_atom", "Se1_atom", "Ir_atom", "Se2_atom", "Tc1_atom",
            "net_dRx", "net_dRy", "net_dRz",
            # step1
            "t1_absH", "d1_A", "wf1_u", "wf1_v", "R1x", "R1y", "R1z",
            # step2
            "t2_absH", "d2_A", "wf2_u", "wf2_v", "R2x", "R2y", "R2z",
            # step3
            "t3_absH", "d3_A", "wf3_u", "wf3_v", "R3x", "R3y", "R3z",
            # step4
            "t4_absH", "d4_A", "wf4_u", "wf4_v", "R4x", "R4y", "R4z",
        ])

    for W, tc0, se1, ir, se2, tc1, netR, e1, e2, e3, e4 in paths:
        w.writerow([
            f"{W:.8g}",
            tc0, se1, ir, se2, tc1,
            netR[0], netR[1], netR[2],

            f"{e1.absH:.8g}", f"{e1.dist:.6f}", e1.u_wf, e1.v_wf, e1.Rx, e1.Ry, e1.Rz,
            f"{e2.absH:.8g}", f"{e2.dist:.6f}", e2.u_wf, e2.v_wf, e2.Rx, e2.Ry, e2.Rz,
            f"{e3.absH:.8g}", f"{e3.dist:.6f}", e3.u_wf, e3.v_wf, e3.Rx, e3.Ry, e3.Rz,
            f"{e4.absH:.8g}", f"{e4.dist:.6f}", e4.u_wf, e4.v_wf, e4.Rx, e4.Ry, e4.Rz,
        ])

    print(f"[DONE] wrote {len(paths)} paths to: {args.out}")
    print("[TIP] If you get too many paths or it runs slow: increase --min_absH (e.g. 2e-3 or 5e-3), or lower
    ↪ --top_per_se/--top_per_ir_se.")

if __name__ == "__main__":
    main()

```

► 代码清单 A.57.7 (shell, 7 行)

```

python step4_build_paths.py --edges 2step.csv --out top_paths.csv ^
--d_tcse 3.0 --d_seir 3.0 ^
--min_absH 1e-3 ^
--top_per_se 60 --top_per_ir_se 80 ^
--top_paths 500 ^
--require_same_tc_atom ^
--max_netR_L1 6

```

► 代码清单 A.57.8 (shell, 7 行)

```
python step4_build_paths.py --edges 2step.csv --out top_paths.csv ^
--d_tcse 3.0 --d_seir 3.0 ^
--min_absH 1e-4 ^
--top_per_se 80 --top_per_ir_se 120 ^
--top_paths 1000 ^
--require_same_tc_atom ^
--max_netR_L1 8
```

► 代码清单 A.57.9 (python, 265 行)

```
import csv
import argparse
from dataclasses import dataclass
from collections import defaultdict
from typing import Tuple, List, Dict

@dataclass(frozen=True)
class DirEdge:
    # directed edge: u(cell_shift) -> v(cell_shift + R)
    u_atom: int          # 1-based
    v_atom: int          # 1-based
    u_elem: str
    v_elem: str
    u_wf: int
    v_wf: int
    Rx: int
    Ry: int
    Rz: int
    absH: float
    dist: float

def parse_edges_csv(path: str) -> List[DirEdge]:
    """
    Read out_edges csv produced by your step2 script (2step.csv).
    Columns expected:
        dist_A, absH_eV, Rx,Ry,Rz, m,n, atom_m,atom_n, elem_m,elem_n
    """
    edges = []
    with open(path, "r", encoding="utf-8", errors="ignore", newline="") as f:
        r = csv.DictReader(f)
        required = ["dist_A", "absH_eV", "Rx", "Ry", "Rz",
                    "m", "n", "atom_m", "atom_n", "elem_m", "elem_n"]
        for k in required:
            if k not in r.fieldnames:
                raise RuntimeError(f"Missing column '{k}' in {path}. Found: {r.fieldnames}")

        for row in r:
            try:
                dist = float(row["dist_A"])
                absH = float(row["absH_eV"])
                Rx, Ry, Rz = int(row["Rx"]), int(row["Ry"]), int(row["Rz"])
                m, n = int(row["m"]), int(row["n"])
                atom_m, atom_n = int(row["atom_m"]), int(row["atom_n"])
                elem_m, elem_n = row["elem_m"].strip(), row["elem_n"].strip()
            except Exception:
                continue

            # add BOTH directions (m@0->n@R) and (n@0->m@-R)
```

```

        edges.append(DirEdge(atom_m, atom_n, elem_m, elem_n, m, n, Rx, Ry, Rz, absH, dist))
        edges.append(DirEdge(atom_n, atom_m, elem_n, elem_m, n, m, -Rx, -Ry, -Rz, absH, dist))
    return edges

def add_shift(s: Tuple[int, int, int], R: Tuple[int, int, int]) -> Tuple[int, int, int]:
    return (s[0] + R[0], s[1] + R[1], s[2] + R[2])

def abs_R_L1(R: Tuple[int, int, int]) -> int:
    return abs(R[0]) + abs(R[1]) + abs(R[2])

def keep_top_per_key(d: Dict[int, List[DirEdge]], topn: int):
    for k in list(d.keys()):
        lst = d[k]
        lst.sort(key=lambda x: x.absH, reverse=True)
        if len(lst) > topn:
            d[k] = lst[:topn]

def main():
    ap = argparse.ArgumentParser(
        description="Build Tc-Se-X-Se-Tc (X in {Ir,Ge}) paths from out_edges csv and rank by hopping-product
        ↪ weight."
    )
    ap.add_argument("--edges", required=True, help="Input edges csv (e.g., 2step.csv)")
    ap.add_argument("--out", default="top_paths.csv", help="Output paths csv")

    # distance windows (Ang)
    ap.add_argument("--d_tcse", type=float, default=3.0, help="Max distance for Tc-Se edges (Ang)")
    ap.add_argument("--d_sex", type=float, default=3.0, help="Max distance for Se-X edges (Ang), applies to all
    ↪ mediators")

    # which mediators to consider
    ap.add_argument("--mediators", default="Ir,Ge",
        help="Comma-separated mediator elements, e.g. Ir,Ge or Ir only or Ge only")

    # pruning
    ap.add_argument("--min_absH", type=float, default=1e-3,
        help="Min |t| for edges used in path building (eV). 1e-3 is fast; 1e-4 is exhaustive but
        ↪ slower.")
    ap.add_argument("--top_per_se_tc", type=int, default=60,
        help="Keep top N Tc-Se edges per Se atom")
    ap.add_argument("--top_per_se_x", type=int, default=120,
        help="Keep top N Se-X edges per Se atom (for each mediator)")
    ap.add_argument("--top_per_x", type=int, default=120,
        help="Keep top N X-Se edges per X atom (for each mediator)")

    ap.add_argument("--top_paths", type=int, default=800,
        help="Number of top paths (overall) to write")

    ap.add_argument("--require_same_tc_atom", action="store_true",
        help="Require end Tc atom id == start Tc atom id (useful when there is only one Tc in
        ↪ primitive cell)")
    ap.add_argument("--max_netR_L1", type=int, default=6,
        help="Filter by |dRx|+|dRy|+|dRz| <= this (set 999 to disable)")

    args = ap.parse_args()

    mediators = [x.strip() for x in args.mediators.split(",") if x.strip()]
    if not mediators:
        raise RuntimeError("No mediators specified. Use --mediators Ir,Ge or similar.")

    all_dir = parse_edges_csv(args.edges)

    # detect atom sets
    tc_atoms = sorted({e.u_atom for e in all_dir if e.u_elem == "Tc"} | {e.v_atom for e in all_dir if e.v_elem ==
    ↪ "Tc"})

```

```

se_atoms = sorted({e.u_atom for e in all_dir if e.u_elem == "Se"} | {e.v_atom for e in all_dir if e.v_elem ==
↪ "Se"})
if not tc_atoms:
    raise RuntimeError("No Tc atoms found (elem == 'Tc').")
if not se_atoms:
    raise RuntimeError("No Se atoms found (elem == 'Se').")

med_atoms = {M: sorted({e.u_atom for e in all_dir if e.u_elem == M} | {e.v_atom for e in all_dir if e.v_elem
↪ == M})
               for M in mediators}

print(f"[INFO] Tc atoms: {tc_atoms}")
print(f"[INFO] Se atoms: {se_atoms}")
for M in mediators:
    print(f"[INFO] {M} atoms: {med_atoms.get(M, [])}")

# ----- build filtered edge pools -----
# Tc->Se and Se->Tc
tc_to_se = defaultdict(list) # key Tc atom -> edges Tc->Se
se_to_tc = defaultdict(list) # key Se atom -> edges Se->Tc

# Se->X and X->Se for each mediator X
se_to_x = {M: defaultdict(list) for M in mediators} # key Se atom -> edges Se->M
x_to_se = {M: defaultdict(list) for M in mediators} # key M atom -> edges M->Se

for e in all_dir:
    if e.absH < args.min_absH:
        continue

    # Tc<->Se
    if e.u_elem == "Tc" and e.v_elem == "Se" and e.dist <= args.d_tcse:
        tc_to_se[e.u_atom].append(e)
        continue
    if e.u_elem == "Se" and e.v_elem == "Tc" and e.dist <= args.d_tcse:
        se_to_tc[e.u_atom].append(e)
        continue

    # Se<->X
    if e.dist <= args.d_sex:
        for M in mediators:
            if e.u_elem == "Se" and e.v_elem == M:
                se_to_x[M][e.u_atom].append(e)
            elif e.u_elem == M and e.v_elem == "Se":
                x_to_se[M][e.u_atom].append(e)

# ----- prune edges -----
keep_top_per_key(tc_to_se, topn=max(args.top_per_se_tc, 30))
keep_top_per_key(se_to_tc, topn=max(args.top_per_se_tc, 30))

for M in mediators:
    keep_top_per_key(se_to_x[M], topn=max(args.top_per_se_x, 50))
    keep_top_per_key(x_to_se[M], topn=max(args.top_per_x, 50))

# Build by-Se index for Tc->Se (so we can start from Se if needed)
tc_to_se_by_se = defaultdict(list) # key Se atom -> edges Tc->Se (from any Tc)
for tc, lst in tc_to_se.items():
    for e in lst:
        tc_to_se_by_se[e.v_atom].append(e)
for se, lst in tc_to_se_by_se.items():
    lst.sort(key=lambda x: x.absH, reverse=True)
    if len(lst) > args.top_per_se_tc:
        tc_to_se_by_se[se] = lst[:args.top_per_se_tc]

# ----- enumerate paths -----
# Path: Tc0@0 -> Se1 -> X -> Se2 -> Tc1
paths = []

def push_path(M, W, tc0, se1, x, se2, tc1, netR, e1, e2, e3, e4):

```

```

    paths.append((
        M, W,
        tc0, se1, x, se2, tc1,
        netR,
        e1, e2, e3, e4
    ))

for tc0 in tc_atoms:
    shift0 = (0, 0, 0)

    for e1 in tc_to_se.get(tc0, []): # Tc->Se1
        se1 = e1.v_atom
        shift_se1 = add_shift(shift0, (e1.Rx, e1.Ry, e1.Rz))

        for M in mediators:
            # Se1 -> X
            for e2 in se_to_x[M].get(se1, []):
                x = e2.v_atom
                shift_x = add_shift(shift_se1, (e2.Rx, e2.Ry, e2.Rz))

            # X -> Se2
            for e3 in x_to_se[M].get(x, []):
                se2 = e3.v_atom
                shift_se2 = add_shift(shift_x, (e3.Rx, e3.Ry, e3.Rz))

            # Se2 -> Tc1
            for e4 in se_to_tc.get(se2, []):
                tc1 = e4.v_atom
                shift_tc1 = add_shift(shift_se2, (e4.Rx, e4.Ry, e4.Rz))

            if args.require_same_tc_atom and (tc1 != tc0):
                continue

            netR = shift_tc1
            if args.max_netR_L1 < 999 and abs_R_L1(netR) > args.max_netR_L1:
                continue

            W = e1.absH * e2.absH * e3.absH * e4.absH
            push_path(M, W, tc0, se1, x, se2, tc1, netR, e1, e2, e3, e4)

if not paths:
    raise RuntimeError(
        "No paths found. Try: lower --min_absH, increase --d_tcse/--d_sex, "
        "or relax --max_netR_L1 / remove --require_same_tc_atom."
    )

# sort by weight
paths.sort(key=lambda x: x[1], reverse=True)
paths = paths[:args.top_paths]

# ----- write output -----
with open(args.out, "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f)
    w.writerow([
        "mediator", "W_path",
        "Tc0_atom", "Se1_atom", "X_atom", "Se2_atom", "Tc1_atom",
        "net_dRx", "net_dRy", "net_dRz",
        # step1 Tc->Se1
        "t1_absH", "d1_A", "wf1_u", "wf1_v", "R1x", "R1y", "R1z",
        # step2 Se1->X
        "t2_absH", "d2_A", "wf2_u", "wf2_v", "R2x", "R2y", "R2z",
        # step3 X->Se2
        "t3_absH", "d3_A", "wf3_u", "wf3_v", "R3x", "R3y", "R3z",
        # step4 Se2->Tc1
        "t4_absH", "d4_A", "wf4_u", "wf4_v", "R4x", "R4y", "R4z",
    ])

for M, W, tc0, se1, x, se2, tc1, netR, e1, e2, e3, e4 in paths:
    w.writerow([

```

```

        M, f"{W:.8g}",
        tc0, se1, x, se2, tc1,
        netR[0], netR[1], netR[2],

        f"{e1.absH:.8g}", f"{e1.dist:.6f}", e1.u_wf, e1.v_wf, e1.Rx, e1.Ry, e1.Rz,
        f"{e2.absH:.8g}", f"{e2.dist:.6f}", e2.u_wf, e2.v_wf, e2.Rx, e2.Ry, e2.Rz,
        f"{e3.absH:.8g}", f"{e3.dist:.6f}", e3.u_wf, e3.v_wf, e3.Rx, e3.Ry, e3.Rz,
        f"{e4.absH:.8g}", f"{e4.dist:.6f}", e4.u_wf, e4.v_wf, e4.Rx, e4.Ry, e4.Rz,
    ])

    print(f"[DONE] wrote {len(paths)} paths to: {args.out}")
    print("[TIP] If too slow: raise --min_absH (e.g. 2e-3~5e-3) or lower --top_per_*.")

if __name__ == "__main__":
    main()

```

► 代码清单 A.57.12 (python, 224 行)

```

import csv
import argparse
from dataclasses import dataclass
from collections import defaultdict
from typing import Tuple, List, Dict

@dataclass(frozen=True)
class DirEdge:
    u_atom: int          # 1-based atom number
    v_atom: int
    u_elem: str
    v_elem: str
    u_wf: int
    v_wf: int
    Rx: int
    Ry: int
    Rz: int
    absH: float          # hopping magnitude
    dist: float          # distance

def parse_edges_csv(path: str) -> List[DirEdge]:
    edges = []
    with open(path, "r", encoding="utf-8", errors="ignore", newline="") as f:
        r = csv.DictReader(f)
        required = ["dist_A", "absH_eV", "Rx", "Ry", "Rz",
                    "m", "n", "atom_m", "atom_n", "elem_m", "elem_n"]
        for k in required:
            if k not in r.fieldnames:
                raise RuntimeError(f"Missing column '{k}' in {path}. Found: {r.fieldnames}")

        for row in r:
            try:
                dist = float(row["dist_A"])
                absH = float(row["absH_eV"])
                Rx, Ry, Rz = int(row["Rx"]), int(row["Ry"]), int(row["Rz"])
                m, n = int(row["m"]), int(row["n"])
                atom_m, atom_n = int(row["atom_m"]), int(row["atom_n"])
                elem_m, elem_n = row["elem_m"].strip(), row["elem_n"].strip()
            except Exception:
                continue

            # Add both directions of the edge

```

```

        edges.append(DirEdge(atom_m, atom_n, elem_m, elem_n, m, n, Rx, Ry, Rz, absH, dist))
        edges.append(DirEdge(atom_n, atom_m, elem_n, elem_m, n, m, -Rx, -Ry, -Rz, absH, dist))
    return edges

def add_shift(s: Tuple[int, int, int], R: Tuple[int, int, int]) -> Tuple[int, int, int]:
    return (s[0] + R[0], s[1] + R[1], s[2] + R[2])

def abs_R_L1(R: Tuple[int, int, int]) -> int:
    return abs(R[0]) + abs(R[1]) + abs(R[2])

def keep_top_per_key(d: Dict[int, List[DirEdge]], topn: int):
    for k in list(d.keys()):
        lst = d[k]
        lst.sort(key=lambda x: x.absH, reverse=True)
        if len(lst) > topn:
            d[k] = lst[:topn]

def main():
    ap = argparse.ArgumentParser(description="Build Tc-Se-X-Se-Tc paths from out_edges csv and rank by
    ↪ hopping-product weight.")
    ap.add_argument("--edges", required=True, help="Input edges csv (e.g., 2step.csv)")
    ap.add_argument("--out", default="top_paths.csv", help="Output paths csv")
    ap.add_argument("--d_tcse", type=float, default=3.0, help="Max distance for Tc-Se edges (Ang)")
    ap.add_argument("--d_sex", type=float, default=3.0, help="Max distance for Se-X edges (Ang), applies to all
    ↪ mediators")
    ap.add_argument("--mediators", default="Ir,Ge", help="Comma-separated mediator elements, e.g. Ir,Ge or Ir only
    ↪ or Ge only")
    ap.add_argument("--min_absH", type=float, default=1e-3, help="Min |t| for edges used in path building (eV).
    ↪ 1e-3 is fast; 1e-4 is exhaustive but slower.")
    ap.add_argument("--top_per_se_tc", type=int, default=60, help="Keep top N Tc-Se edges per Se atom")
    ap.add_argument("--top_per_se_x", type=int, default=120, help="Keep top N Se-X edges per Se atom (for each
    ↪ mediator)")
    ap.add_argument("--top_per_x", type=int, default=120, help="Keep top N X-Se edges per X atom (for each
    ↪ mediator)")
    ap.add_argument("--top_paths", type=int, default=800, help="Number of top paths (overall) to write")
    ap.add_argument("--require_same_tc_atom", action="store_true", help="Require end Tc atom id == start Tc atom
    ↪ id (useful when there is only one Tc in primitive cell)")
    ap.add_argument("--max_netR_L1", type=int, default=6, help="Filter by |dRx|+|dRy|+|dRz| <= this (set 999 to
    ↪ disable)")

    args = ap.parse_args()

    mediators = [x.strip() for x in args.mediators.split(",") if x.strip()]
    if not mediators:
        raise RuntimeError("No mediators specified. Use --mediators Ir,Ge or similar.")

    all_dir = parse_edges_csv(args.edges)

    # detect atom sets
    tc_atoms = sorted({e.u_atom for e in all_dir if e.u_elem == "Tc"} | {e.v_atom for e in all_dir if e.v_elem ==
    ↪ "Tc"})
    se_atoms = sorted({e.u_atom for e in all_dir if e.u_elem == "Se"} | {e.v_atom for e in all_dir if e.v_elem ==
    ↪ "Se"})
    if not tc_atoms:
        raise RuntimeError("No Tc atoms found (elem == 'Tc').")
    if not se_atoms:
        raise RuntimeError("No Se atoms found (elem == 'Se').")
    med_atoms = {M: sorted({e.u_atom for e in all_dir if e.u_elem == M} | {e.v_atom for e in all_dir if e.v_elem
    ↪ == M}) for M in mediators}

    print(f"[INFO] Tc atoms: {tc_atoms}")
    print(f"[INFO] Se atoms: {se_atoms}")
    for M in mediators:
        print(f"[INFO] {M} atoms: {med_atoms.get(M, [])}")

```

```

# ----- build filtered edge pools -----
tc_to_se = defaultdict(list) # Tc->Se edges
se_to_tc = defaultdict(list) # Se->Tc edges
se_to_x = {M: defaultdict(list) for M in mediators} # Se->X edges for each mediator
x_to_se = {M: defaultdict(list) for M in mediators} # X->Se edges for each mediator

for e in all_dir:
    if e.absH < args.min_absH:
        continue

    # Tc<->Se
    if e.u_elem == "Tc" and e.v_elem == "Se" and e.dist <= args.d_tcse:
        tc_to_se[e.u_atom].append(e)
        continue
    if e.u_elem == "Se" and e.v_elem == "Tc" and e.dist <= args.d_tcse:
        se_to_tc[e.u_atom].append(e)
        continue

    # Se<->X
    if e.dist <= args.d_sex:
        for M in mediators:
            if e.u_elem == "Se" and e.v_elem == M:
                se_to_x[M][e.u_atom].append(e)
            elif e.u_elem == M and e.v_elem == "Se":
                x_to_se[M][e.u_atom].append(e)

# ----- prune edges -----
keep_top_per_key(tc_to_se, topn=max(args.top_per_se_tc, 30))
keep_top_per_key(se_to_tc, topn=max(args.top_per_se_tc, 30))
for M in mediators:
    keep_top_per_key(se_to_x[M], topn=max(args.top_per_se_x, 50))
    keep_top_per_key(x_to_se[M], topn=max(args.top_per_x, 50))

# ----- build paths -----
paths = []

def push_path(M, W, tc0, se1, x, se2, tc1, netR, e1, e2, e3, e4):
    paths.append((
        M, W,
        tc0, se1, x, se2, tc1,
        netR,
        e1, e2, e3, e4
    ))

for tc0 in tc_atoms:
    shift0 = (0, 0, 0)

    for e1 in tc_to_se.get(tc0, []): # Tc->Se1
        se1 = e1.v_atom
        shift_se1 = add_shift(shift0, (e1.Rx, e1.Ry, e1.Rz))

        for M in mediators:
            # Se1 -> X
            for e2 in se_to_x[M].get(se1, []):
                x = e2.v_atom
                shift_x = add_shift(shift_se1, (e2.Rx, e2.Ry, e2.Rz))

            # X -> Se2
            for e3 in x_to_se[M].get(x, []):
                se2 = e3.v_atom
                shift_se2 = add_shift(shift_x, (e3.Rx, e3.Ry, e3.Rz))

            # Se2 -> Tc1
            for e4 in se_to_tc.get(se2, []):
                tc1 = e4.v_atom
                shift_tc1 = add_shift(shift_se2, (e4.Rx, e4.Ry, e4.Rz))

            if args.require_same_tc_atom and (tc1 != tc0):

```

```

        continue

    netR = shift_tc1
    if args.max_netR_L1 < 999 and abs_R_L1(netR) > args.max_netR_L1:
        continue

    W = e1.absH * e2.absH * e3.absH * e4.absH
    push_path(M, W, tc0, se1, x, se2, tc1, netR, e1, e2, e3, e4)

if not paths:
    raise RuntimeError(
        "No paths found. Try: lower --min_absH, increase --d_tcse/--d_sex, "
        "or relax --max_netR_L1 / remove --require_same_tc_atom."
    )

# sort by weight
paths.sort(key=lambda x: x[1], reverse=True)
paths = paths[:args.top_paths]

# ----- write output -----
with open(args.out, "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f)
    w.writerow([
        "mediator", "W_path",
        "Tc0_atom", "Se1_atom", "X_atom", "Se2_atom", "Tc1_atom",
        "net_dRx", "net_dRy", "net_dRz",
        # step1 Tc->Se1
        "t1_absH", "d1_A", "wf1_u", "wf1_v", "R1x", "R1y", "R1z",
        # step2 Se1->X
        "t2_absH", "d2_A", "wf2_u", "wf2_v", "R2x", "R2y", "R2z",
        # step3 X->Se2
        "t3_absH", "d3_A", "wf3_u", "wf3_v", "R3x", "R3y", "R3z",
        # step4 Se2->Tc1
        "t4_absH", "d4_A", "wf4_u", "wf4_v", "R4x", "R4y", "R4z",
    ])

    for M, W, tc0, se1, x, se2, tc1, netR, e1, e2, e3, e4 in paths:
        w.writerow([
            M, f"{W:.8g}",
            tc0, se1, x, se2, tc1,
            netR[0], netR[1], netR[2],

            f"{e1.absH:.8g}", f"{e1.dist:.6f}", e1.u_wf, e1.v_wf, e1.Rx, e1.Ry, e1.Rz,
            f"{e2.absH:.8g}", f"{e2.dist:.6f}", e2.u_wf, e2.v_wf, e2.Rx, e2.Ry, e2.Rz,
            f"{e3.absH:.8g}", f"{e3.dist:.6f}", e3.u_wf, e3.v_wf, e3.Rx, e3.Ry, e3.Rz,
            f"{e4.absH:.8g}", f"{e4.dist:.6f}", e4.u_wf, e4.v_wf, e4.Rx, e4.Ry, e4.Rz,
        ])

    print(f"[DONE] wrote {len(paths)} paths to: {args.out}")
    print("[TIP] If too slow: raise --min_absH (e.g. 2e-3~5e-3) or lower --top_per*.")

if __name__ == "__main__":
    main()

```

▶ 代码清单 A.57.13 (python, 46 行)

```

import pandas as pd
import argparse

J1_NETR = {
    ( 1, 0, 0),
    (-1, 0, 0),

```

```

( 0, 1, 0),
( 0,-1, 0),
( 1, 1, 0),
(-1,-1, 0),
}

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--csv", required=True, help="path file (Ir_xxx.csv or Ge_xxx.csv)")
    ap.add_argument("--topk", type=int, default=100, help="sum topK scores after filtering to J1 shell")
    args = ap.parse_args()

    df = pd.read_csv(args.csv)

    # 兼容列名: 文件中通常为 net_dRx/net_dRy/net_dRz 与 score_num_over_denom
    for c in ["net_dRx", "net_dRy", "net_dRz", "score_num_over_denom", "mediator"]:
        if c not in df.columns:
            raise RuntimeError(f"missing column: {c}, have: {list(df.columns)}")

    df["netR"] = list(zip(df["net_dRx"], df["net_dRy"], df["net_dRz"]))
    sub = df[df["netR"].isin(J1_NETR)].copy()

    if sub.empty:
        raise RuntimeError("No rows in J1 shell after filtering. Check your net_dR convention.")

    sub = sub.sort_values("score_num_over_denom", ascending=False)

    topk = min(args.topk, len(sub))
    ssum = sub["score_num_over_denom"].head(topk).sum()
    smax = sub["score_num_over_denom"].iloc[0]
    med = sub["mediator"].iloc[0]

    print(f"[INFO] file: {args.csv}")
    print(f"[INFO] mediator (top row): {med}")
    print(f"[INFO] J1-shell rows: {len(sub)}")
    print(f"[INFO] top1 score: {smax:.6g}")
    print(f"[INFO] sum top{topk} score: {ssum:.6g}")

if __name__ == "__main__":
    main()

```

► 代码清单 A.57.15 (shell, 3 行)

```

python 1step.py --win wannier90.1.win --centres wannier90.1_centres.xyz --hr wannier90.1_hr.dat --tol 0.10
↪ --min_absH 1e-3 --topk 30 --out_summary 1step_up.csv --pairs ALL --skip_same_atom_R0 --out_edges
↪ 2step_up.csv

python 1step.py --win wannier90.2.win --centres wannier90.2_centres.xyz --hr wannier90.2_hr.dat --tol 0.10
↪ --min_absH 1e-3 --topk 30 --out_summary 1step_dw.csv --pairs ALL --skip_same_atom_R0 --out_edges
↪ 2step_dw.csv

```

► 代码清单 A.57.16 (shell, 7 行)

```

python 2step.py --edges 2step_dw.csv --out dw.csv --mediators Ir --d_tcse 3.0 --d_sex 3.0 --min_absH 1e-3
↪ --top_paths 2000 --require_same_tc_atom --max_netR_L1 6 --exclude_netR0 --delta_mode sequential
↪ --delta_floor 1e-3 --hr wannier90.2_hr.dat

```



```
python 2step.py --edges 2step_up.csv --out up.csv --mediators Ir --d_tcse 3.0 --d_sex 3.0 --min_absH 1e-3
↪ --top_paths 2000 --require_same_tc_atom --max_netR_L1 6 --exclude_netR0 --delta_mode sequential
↪ --delta_floor 1e-3 --hr wannier90.1_hr.dat

python 2step.py --edges 2step_dw.csv --out dw.csv --mediators Ge --d_tcse 3.0 --d_sex 3.0 --min_absH 1e-3
↪ --top_paths 2000 --require_same_tc_atom --max_netR_L1 6 --exclude_netR0 --delta_mode sequential
↪ --delta_floor 1e-3 --hr wannier90.2_hr.dat

python 2step.py --edges 2step_up.csv --out up.csv --mediators Ge --d_tcse 3.0 --d_sex 3.0 --min_absH 1e-3
↪ --top_paths 2000 --require_same_tc_atom --max_netR_L1 6 --exclude_netR0 --delta_mode sequential
↪ --delta_floor 1e-3 --hr wannier90.1_hr.dat
```

► 代码清单 A.57.18 (shell, 5 行)

```
[INFO] file: up.csv
[INFO] mediator (top row): Ge
[INFO] J1-shell rows: 2000
[INFO] top1 score: 70.4132
[INFO] sum top100 score: 2565.74
```

(原创自研) 利用 wannier 验证超超交换路径 (2)

► 代码清单 A.58.1 (python, 6 行)

```
num_wann = 88
```

```
Tc: d
Se : p
Ge : d;p
Ir: d
```

► 代码清单 A.58.2 (python, 536 行)

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
1step.py (Python 3.8/3.9 compatible)

NEW: export edges csv (2step.csv) from wannier90_hr.dat, like your old 1step.py did,
but now robust for num_wann=88 with projections:
    Tc: d
    Se: p
    Ge: d;p
    Ir: d

Core features
-----
(1) Parse wannier90.win: unit_cell_cart, atoms_cart, projections
(2) Parse wannier90.wout: Final State WF centres & spreads (authoritative)
```

- (3) Parse wannier90_hr.dat: all hopping matrix elements $H_{\{mn\}}(R)$
- (4) Assign each WF to an element using quota-balanced assignment so element counts match expected from projections & atom counts (handles bond-centred WFs).
- (5) Split Ge WFs into Ge_d / Ge_p using spread / onsite / hybrid.
- (6) Export directed edges to CSV with columns compatible with your downstream 2step.py: shell, dist, absH, re, im, Rx,Ry,Rz, u_wf,v_wf, u_atom,v_atom, u_elem,v_elem, pair

Optional:

- Export a summary CSV per shell.

Usage (export edges)

```
python 1step.py --win wannier90.win --wout wannier90.wout --hr wannier90_hr.dat \
  --out_edges 2step.csv --tol 0.10 --min_absH 1e-6
```

If you want fewer edges:

```
--topk_per_shell 400
```

Ge d/p splitting mode:

```
--ge_split spread    (default)
--ge_split onsite
--ge_split hybrid
"""
```

```
import argparse
import csv
import json
import re
from collections import defaultdict, Counter
from pathlib import Path

import numpy as np

# =====
# WIN parsing
# =====
def _extract_block(txt, name):
    m = re.search(r"begin\s+%(.*?)end\s+%" % (re.escape(name), re.escape(name)), txt, re.S | re.I)
    return None if not m else m.group(1).strip()

def parse_win(win_path):
    txt = Path(win_path).read_text(encoding="utf-8", errors="ignore")

    blk = _extract_block(txt, "unit_cell_cart")
    if blk is None:
        raise RuntimeError("Cannot find begin/end unit_cell_cart in win.")
    lines = [ln.strip() for ln in blk.splitlines() if ln.strip()]
    if lines and lines[0].lower().startswith("ang"):
        lines = lines[1:]
    if len(lines) < 3:
        raise RuntimeError("unit_cell_cart must contain 3 lattice vectors.")
    A = np.array([[float(x) for x in lines[i].split()[1:4]] for i in range(3)], dtype=float)

    blk = _extract_block(txt, "atoms_cart")
    if blk is None:
        raise RuntimeError("Cannot find begin/end atoms_cart in win.")
    atoms = []
    for ln in blk.splitlines():
        ln = ln.strip()
        if not ln:
            continue
        parts = ln.split()
        if len(parts) < 4:
            continue
        elem = parts[0].strip()
        x, y, z = map(float, parts[1:4])
        atoms.append((elem, np.array([x, y, z], float)))
```

```

if not atoms:
    raise RuntimeError("No atoms parsed from atoms_cart.")

blk = _extract_block(txt, "projections")
if blk is None:
    raise RuntimeError("Cannot find begin/end projections in win.")
proj = defaultdict(set)
for ln in blk.splitlines():
    ln = ln.strip()
    if not ln or ln.startswith("#") or ln.startswith("!"):
        continue
    if ":" not in ln:
        continue
    lhs, rhs = ln.split(":", 1)
    elem = lhs.strip()
    rhs = rhs.strip().replace(" ", "")
    for orb in rhs.split(";"):
        if orb:
            proj[elem].add(orb.lower())
if not proj:
    raise RuntimeError("Parsed empty projections. Check win projections block.")

return A, atoms, proj

# =====
# WOUT parsing (Final State)
# =====
def parse_wout_final_centres_spreads(wout_path):
    txt = Path(wout_path).read_text(encoding="utf-8", errors="ignore")
    idx = txt.lower().rfind("final state")
    if idx < 0:
        raise RuntimeError("Cannot find 'Final State' in wannier90.wout")

    centres = {}
    spreads = {}

    pat = re.compile(
        r"^\s*WF centre and
        ↪ spread\s+(\d+)\s+(\s*([-\\d\\.]+)\s*,\s*([-\\d\\.]+)\s*,\s*([-\\d\\.]+)\s*)\s+([-\\d\\.]+)\s*$"
    )

    for ln in txt[idx:].splitlines():
        m = pat.match(ln)
        if not m:
            continue
        i = int(m.group(1))
        x, y, z = float(m.group(2)), float(m.group(3)), float(m.group(4))
        s = float(m.group(5))
        centres[i] = np.array([x, y, z], float)
        spreads[i] = s

    if not centres:
        raise RuntimeError("No WF centres parsed from Final State section.")
    num_wann = max(centres.keys())
    return num_wann, centres, spreads

# =====
# HR parsing
# =====
def read_hr_all(hr_path):
    """
    Return:
        num_wann, onsite(dict), hoppings(list of tuples)
    where hoppings are (Rx,Ry,Rz,m,n,re,im)
    """
    raw = Path(hr_path).read_text(encoding="utf-8", errors="ignore").splitlines()
    if len(raw) < 4:

```

```

        raise RuntimeError("hr.dat too short")
    num_wann = int(row[1].split()[0])
    nrpts = int(row[2].split()[0])

    deg = []
    i = 3
    while len(deg) < nrpts:
        deg.extend([int(x) for x in row[i].split()])
        i += 1
    if i >= len(row):
        raise RuntimeError("Unexpected EOF while reading degeneracies.")

    onsite = {}
    hops = []
    for ln in row[i:]:
        parts = ln.split()
        if len(parts) < 7:
            continue
        Rx, Ry, Rz = map(int, parts[0:3])
        m = int(parts[3])
        n = int(parts[4])
        re_ = float(parts[5])
        im_ = float(parts[6])
        hops.append((Rx, Ry, Rz, m, n, re_, im_))
        if Rx == 0 and Ry == 0 and Rz == 0 and m == n:
            onsite[m] = re_
    return num_wann, onsite, hops

# =====
# PBC helpers
# =====
def pbc_dist(A, r1, r2):
    invA = np.linalg.inv(A)
    f1 = r1 @ invA
    f2 = r2 @ invA
    df = f2 - f1
    df -= np.round(df)
    dr = df @ A
    return float(np.linalg.norm(dr))

def shift_by_R(A, vec, Rx, Ry, Rz):
    return vec + Rx * A[0] + Ry * A[1] + Rz * A[2]

# =====
# Expected WF counts from projections + atoms_cart
# =====
def expected_counts(atoms, proj, num_wann):
    atom_counts = Counter([e for e, _ in atoms])
    orb_dim = {"s": 1, "p": 3, "d": 5, "f": 7}

    base_total = 0
    elem_orb = defaultdict(dict)
    elem_total = {}

    for elem, orbs in proj.items():
        n = atom_counts.get(elem, 0)
        tot = 0
        for o in orbs:
            if o not in orb_dim:
                raise RuntimeError("Unknown orbital '%s' in projections for %s" % (o, elem))
            tot += orb_dim[o] * n
        elem_total[elem] = tot
        base_total += tot
        for o in orbs:
            elem_orb[elem][o] = orb_dim[o] * n

```

```

factor = 2 if num_wann == 2 * base_total else 1

elem_total = {k: v * factor for k, v in elem_total.items()}
for elem in list(elem_orb.keys()):
    for o in list(elem_orb[elem].keys()):
        elem_orb[elem][o] *= factor

s = sum(elem_total.values())
if s != num_wann:
    raise RuntimeError(
        "Expected counts sum %d != num_wann %d. Check atoms_cart/projections or SOC/spinor." % (s, num_wann)
    )
return factor, elem_total, elem_orb

# =====
# Quota-balanced WF->element assignment
# =====
def assign_wfs_to_elements_quota(A, atoms, centres, elem_quota):
    elem_atoms = defaultdict(list)
    for ai, (elem, pos) in enumerate(atoms, start=1):
        elem_atoms[elem].append((ai, pos))

    wf_choices = {}
    for wf, c in centres.items():
        opts = []
        for elem, lst in elem_atoms.items():
            bestd = 1e18
            best_ai = None
            for ai, pos in lst:
                d = pbc_dist(A, c, pos)
                if d < bestd:
                    bestd = d
                    best_ai = ai
            opts.append((bestd, elem, best_ai))
        opts.sort(key=lambda x: x[0])
        wf_choices[wf] = opts

    order = sorted(centres.keys(), key=lambda wf: wf_choices[wf][0][0])

    quota = dict(elem_quota)
    wf_elem = {}
    wf_atom = {}

    for wf in order:
        assigned = False
        for d, elem, ai in wf_choices[wf]:
            if quota.get(elem, 0) > 0:
                wf_elem[wf] = elem
                wf_atom[wf] = ai
                quota[elem] -= 1
                assigned = True
                break
        if not assigned:
            wf_elem[wf] = wf_choices[wf][0][1]
            wf_atom[wf] = wf_choices[wf][0][2]

    remain = {k: v for k, v in quota.items() if v != 0}
    if remain:
        raise RuntimeError("Quota assignment failed, remaining quota: %s" % remain)
    return wf_elem, wf_atom

# =====
# Split Ge into d/p
# =====
def split_ge_dp(wfs_ge, spreads, onsite, n_ge_d, n_ge_p, mode):
    wfs_ge = list(wfs_ge)
    if len(wfs_ge) != n_ge_d + n_ge_p:

```

```

        raise RuntimeError("Ge WF count %d != expected %d" % (len(wfs_ge), n_ge_d + n_ge_p))

    if mode == "spread":
        order = sorted(wfs_ge, key=lambda i: spreads[i])
    elif mode == "onsite":
        order = sorted(wfs_ge, key=lambda i: onsite.get(i, 0.0))
    elif mode == "hybrid":
        sp = np.array([spreads[i] for i in wfs_ge], float)
        ep = np.array([onsite.get(i, 0.0) for i in wfs_ge], float)
        spz = (sp - sp.mean()) / (sp.std() + 1e-12)
        epz = (ep - ep.mean()) / (ep.std() + 1e-12)
        score = spz + epz
        order = [w for _, w in sorted(zip(score, wfs_ge), key=lambda x: x[0])]
    else:
        raise ValueError("Unknown ge_split mode: %s" % mode)

    ge_d = set(order[:n_ge_d])
    ge_p = set(order[n_ge_d:n_ge_d + n_ge_p])
    return ge_d, ge_p

# =====
# Shell clustering by distance
# =====
def assign_shells_by_tol(dist_list, tol):
    """
    Given distances, cluster into shells by tolerance tol.
    Return:
        shell_id per dist index, and shell_centers list
    """
    if not dist_list:
        return [], []
    order = sorted(range(len(dist_list)), key=lambda i: dist_list[i])
    centers = []
    shell_id = [None] * len(dist_list)

    for idx in order:
        d = dist_list[idx]
        placed = False
        for s, c in enumerate(centers):
            if abs(d - c) <= tol:
                shell_id[idx] = s + 1
                # update center (simple running average)
                centers[s] = (c + d) * 0.5
                placed = True
                break
        if not placed:
            centers.append(d)
            shell_id[idx] = len(centers)
    return shell_id, centers

# =====
# Main: export edges
# =====
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--win", required=True)
    ap.add_argument("--wout", required=True)
    ap.add_argument("--hr", required=True)

    ap.add_argument("--out_edges", default="", help="Output edges CSV (e.g. 2step.csv). If empty, no edges
    ↪ exported.")
    ap.add_argument("--out_summary", default="", help="Optional per-shell summary CSV.")

    ap.add_argument("--min_absH", type=float, default=1e-6)
    ap.add_argument("--tol", type=float, default=0.10, help="Distance tolerance for shell clustering (Ang).")
    ap.add_argument("--topk_per_shell", type=int, default=0,
                    help="If >0, keep only topK edges (by absH) per shell.")

```

```

ap.add_argument("--ge_split", default="spread", choices=["spread", "onsite", "hybrid"])
ap.add_argument("--dump_groups", default="", help="Optional JSON to dump WF index sets.")

args = ap.parse_args()

A, atoms, proj = parse_win(args.win)
num_wann_wout, centres, spreads = parse_wout_final_centres_spreads(args.wout)
num_wann_hr, onsite, hops = read_hr_all(args.hr)

if num_wann_wout != num_wann_hr:
    raise RuntimeError("num_wann mismatch: wout=%d, hr=%d. Use matching files." % (num_wann_wout,
↪ num_wann_hr))
num_wann = num_wann_wout

factor, elem_total, elem_orb = expected_counts(atoms, proj, num_wann)
print("[INFO] num_wann=%d factor=%d" % (num_wann, factor))
print("[INFO] expected per-element totals: %s" % dict(elem_total))
print("[INFO] expected per-element-orbital: %s" % {k: dict(v) for k, v in elem_orb.items()})

wf_elem, wf_atom = assign_wfs_to_elements_quota(A, atoms, centres, elem_total)

elem_wfs = defaultdict(list)
for wf in range(1, num_wann + 1):
    elem_wfs[wf_elem[wf]].append(wf)

Tc_set = set(elem_wfs.get("Tc", []))
Se_set = set(elem_wfs.get("Se", []))
Ir_set = set(elem_wfs.get("Ir", []))
Ge_set = set(elem_wfs.get("Ge", []))

# split Ge into d/p (only if Ge has both in projections)
n_ge_d = elem_orb.get("Ge", {}).get("d", 0)
n_ge_p = elem_orb.get("Ge", {}).get("p", 0)
if n_ge_d > 0 and n_ge_p > 0:
    Ge_d, Ge_p = split_ge_dp(list(Ge_set), spreads, onsite, n_ge_d, n_ge_p, mode=args.ge_split)
else:
    Ge_d, Ge_p = set(Ge_set), set()

print("[INFO] derived sizes: Tc=%d Se=%d Ir=%d Ge=%d (Ge_d=%d Ge_p=%d)" %
      (len(Tc_set), len(Se_set), len(Ir_set), len(Ge_set), len(Ge_d), len(Ge_p)))

if args.dump_groups:
    out = {
        "Tc": sorted(Tc_set),
        "Se": sorted(Se_set),
        "Ir": sorted(Ir_set),
        "Ge": sorted(Ge_set),
        "Ge_d": sorted(Ge_d),
        "Ge_p": sorted(Ge_p),
    }
    Path(args.dump_groups).write_text(json.dumps(out, indent=2), encoding="utf-8")
    print("[INFO] dumped group WF indices -> %s" % args.dump_groups)

# Build edges from hr.dat
rows = []
dist_list = []

for (Rx, Ry, Rz, m, n, re_, im_) in hops:
    if m == n and Rx == 0 and Ry == 0 and Rz == 0:
        continue
    if m < 1 or m > num_wann or n < 1 or n > num_wann:
        continue
    absH = float(np.hypot(re_, im_))
    if absH < args.min_absH:
        continue

    # distance between WF centers (n shifted by lattice R)
    cm = centres[m]

```

```

cn = shift_by_R(A, centres[n], Rx, Ry, Rz)
dist = float(np.linalg.norm(cn - cm))

u_elem = wf_elem[m]
v_elem = wf_elem[n]
u_atom = wf_atom[m]
v_atom = wf_atom[n]

# pair label (for downstream filtering/analysis)
# Additionally label Ge as Ge_d / Ge_p when possible.
def label_elem_wf(elem, wf):
    if elem == "Ge":
        if wf in Ge_d:
            return "Ge_d"
        if wf in Ge_p:
            return "Ge_p"
        return "Ge"
    if elem == "Tc":
        return "Tc_d" # your projections are Tc:d
    if elem == "Se":
        return "Se_p" # Se:p
    if elem == "Ir":
        return "Ir_d" # Ir:d
    return elem

u_tag = label_elem_wf(u_elem, m)
v_tag = label_elem_wf(v_elem, n)
pair = "%s-%s" % (u_tag, v_tag)

rows.append({
    "shell": 0, # fill later
    "dist": dist,
    "absH": absH,
    "re": re_,
    "im": im_,
    "Rx": Rx, "Ry": Ry, "Rz": Rz,
    "u_wf": m, "v_wf": n,
    "u_atom": u_atom, "v_atom": v_atom,
    "u_elem": u_elem, "v_elem": v_elem,
    "pair": pair
})
dist_list.append(dist)

if not rows:
    raise RuntimeError("No edges passed filters. Lower --min_absH or check hr.dat.")

# Assign shells
shell_id, centers = assign_shells_by_tol(dist_list, tol=args.tol)
for i, sid in enumerate(shell_id):
    rows[i]["shell"] = sid

# Optionally keep topK per shell
if args.topk_per_shell and args.topk_per_shell > 0:
    by_shell = defaultdict(list)
    for r in rows:
        by_shell[r["shell"]].append(r)
    new_rows = []
    for sh, lst in by_shell.items():
        lst.sort(key=lambda x: x["absH"], reverse=True)
        new_rows.extend(lst[:args.topk_per_shell])
    rows = new_rows
    print("[INFO] after topk_per_shell=%d: edges=%d" % (args.topk_per_shell, len(rows)))

# Write edges
if args.out_edges:
    outp = Path(args.out_edges)
    outp.parent.mkdir(parents=True, exist_ok=True)
    fieldnames = ["pair", "shell", "dist", "absH", "re", "im",
                  "Rx", "Ry", "Rz", "u_wf", "v_wf", "u_atom", "v_atom", "u_elem", "v_elem"]

```

```

with outp.open("w", newline="", encoding="utf-8") as f:
    w = csv.DictWriter(f, fieldnames=fieldnames)
    w.writeheader()
    for r in rows:
        w.writerow({k: r[k] for k in fieldnames})
print("[INFO] wrote edges -> %s (rows=%d)" % (str(outp), len(rows)))

# Optional summary per shell
if args.out_summary:
    by_shell = defaultdict(list)
    for r in rows:
        by_shell[r["shell"]].append(r)

    summ = []
    for sh in sorted(by_shell.keys()):
        lst = by_shell[sh]
        dmean = float(np.mean([x["dist"] for x in lst]))
        amax = float(np.max([x["absH"] for x in lst]))
        asum = float(np.sum([x["absH"] for x in lst]))
        n = len(lst)
        summ.append({
            "shell": sh,
            "N_edges": n,
            "dist_mean": dmean,
            "absH_max": amax,
            "absH_sum": asum
        })

    outp = Path(args.out_summary)
    outp.parent.mkdir(parents=True, exist_ok=True)
    with outp.open("w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=list(summ[0].keys()))
        w.writeheader()
        for r in summ:
            w.writerow(r)
    print("[INFO] wrote summary -> %s" % str(outp))

if __name__ == "__main__":
    main()

```

► 代码清单 A.58.4 (shell, 2 行)

Summary: 1step_up.csv
Edges: 2step_up.csv

► 代码清单 A.58.5 (python, 430 行)

```

import csv
import argparse
from dataclasses import dataclass
from collections import defaultdict
from typing import Tuple, List, Dict, Optional

@dataclass(frozen=True)
class DirEdge:
    # directed edge: u(cell_shift) -> v(cell_shift + R)

```

```

u_atom: int
v_atom: int
u_elem: str
v_elem: str
u_wf: int # 1-based wf index
v_wf: int # 1-based wf index
Rx: int
Ry: int
Rz: int
absH: float
dist: float
pair: str = "" # optional: "Se_p-Ge_d", "Se_p-Ir_d", ...
shell: int = 0 # optional

def _pick(row: dict, *keys: str, default=None):
    for k in keys:
        if k in row and row[k] not in (None, ""):
            return row[k]
    return default

def parse_edges_csv(path: str, add_reverse: bool = False) -> List[DirEdge]:
    """
    Read edges csv produced by:
    - old lstep: dist_A, absH_eV, Rx,Ry,Rz, m,n, atom_m,atom_n, elem_m,elem_n
    - new lstep: dist, absH, Rx,Ry,Rz, u_wf,v_wf, u_atom,v_atom, u_elem,v_elem, pair(optional), shell(optional)

    For new lstep outputs, edges are already directed and include both directions in hr.dat,
    so add_reverse should usually be False to avoid doubling.
    """
    edges: List[DirEdge] = []
    with open(path, "r", encoding="utf-8", errors="ignore", newline="") as f:
        r = csv.DictReader(f)
        fns = r.fieldnames or []
        fset = set(fns)

        is_new = ("u_wf" in fset and "v_wf" in fset and "u_atom" in fset and "v_atom" in fset)
        is_old = ("m" in fset and "n" in fset and "atom_m" in fset and "atom_n" in fset)

        if not (is_new or is_old):
            raise RuntimeError(f"Unrecognized edges schema in {path}. Found columns: {fns}")

        for row in r:
            try:
                dist = float(_pick(row, "dist", "dist_A"))
                absH = float(_pick(row, "absH", "absH_eV"))
                Rx, Ry, Rz = int(_pick(row, "Rx")), int(_pick(row, "Ry")), int(_pick(row, "Rz"))

                u_wf = int(_pick(row, "u_wf", "m"))
                v_wf = int(_pick(row, "v_wf", "n"))

                u_atom = int(_pick(row, "u_atom", "atom_m"))
                v_atom = int(_pick(row, "v_atom", "atom_n"))

                u_elem = str(_pick(row, "u_elem", "elem_m")).strip()
                v_elem = str(_pick(row, "v_elem", "elem_n")).strip()

                pair = str(_pick(row, "pair", default="") or "")
                shell = int(_pick(row, "shell", default="0") or 0)
            except Exception:
                continue

            e = DirEdge(u_atom, v_atom, u_elem, v_elem, u_wf, v_wf, Rx, Ry, Rz, absH, dist, pair=pair,
                        ↪ shell=shell)
            edges.append(e)

        if add_reverse:
            edges.append(DirEdge(

```

```

        v_atom, u_atom, v_elem, u_elem,
        v_wf, u_wf, -Rx, -Ry, -Rz,
        absH, dist, pair=pair, shell=shell
    ))

    return edges

def load_onsite_from_hr(hr_path: str, num_wann: Optional[int] = None) -> Dict[int, float]:
    """Extract onsite energies eps[m] = Re(H_mm(R=0)) from wannier90_hr.dat."""
    with open(hr_path, "r", encoding="utf-8", errors="ignore") as f:
        lines = f.readlines()
    if len(lines) < 4:
        raise RuntimeError(f"hr.dat too short: {hr_path}")

    try:
        nw = int(lines[1].split()[0])
    except Exception:
        raise RuntimeError("Failed to parse num_wann from hr.dat line2.")
    if num_wann is not None and nw != num_wann:
        print(f"[WARN] num_wann mismatch: hr={nw} vs --num_wann={num_wann}. Using hr value.")
    num_wann = nw

    try:
        nrpts = int(lines[2].split()[0])
    except Exception:
        raise RuntimeError("Failed to parse nrpts from hr.dat line3.")

    deg = []
    idx = 3
    while idx < len(lines) and len(deg) < nrpts:
        for p in lines[idx].split():
            try:
                deg.append(int(p))
            except Exception:
                pass
        idx += 1
    if len(deg) < nrpts:
        raise RuntimeError("Failed to read full degeneracy list from hr.dat.")

    eps: Dict[int, float] = {}
    for j in range(idx, len(lines)):
        parts = lines[j].split()
        if len(parts) < 7:
            continue
        try:
            Rx, Ry, Rz = int(parts[0]), int(parts[1]), int(parts[2])
            m, n = int(parts[3]), int(parts[4])
            re = float(parts[5])
        except Exception:
            continue
        if Rx == 0 and Ry == 0 and Rz == 0 and m == n:
            eps[m] = re

    if len(eps) < num_wann:
        print(f"[WARN] onsite found {len(eps)}/{num_wann}. Some WFs missing onsite.")
    else:
        print(f"[INFO] onsite found {len(eps)}/{num_wann}.")
    return eps

def add_shift(s: Tuple[int, int, int], R: Tuple[int, int, int]) -> Tuple[int, int, int]:
    return (s[0] + R[0], s[1] + R[1], s[2] + R[2])

def abs_R_L1(R: Tuple[int, int, int]) -> int:
    return abs(R[0]) + abs(R[1]) + abs(R[2])

```

```

def keep_top_per_key(d: Dict[int, List[DirEdge]], topn: int):
    for k in list(d.keys()):
        lst = d[k]
        lst.sort(key=lambda x: x.absH, reverse=True)
        if len(lst) > topn:
            d[k] = lst[:topn]

def delta_pair(eps: Dict[int, float], wf_a: int, wf_b: int, floor: float) -> float:
    ea = eps.get(wf_a, None)
    eb = eps.get(wf_b, None)
    if ea is None or eb is None:
        return float("nan")
    d = abs(ea - eb)
    return max(d, floor)

def print_panel_summary(tag: str, uniq_sorted: List[tuple], panel_topk: int):
    """
    uniq_sorted elements: (score, N, D, d1, d2, d3, sig, ...)
    """
    if not uniq_sorted:
        print(f"[PANEL] {tag}: no paths")
        return

    k = min(panel_topk, len(uniq_sorted))
    top1 = uniq_sorted[0][0]
    sumk = sum(x[0] for x in uniq_sorted[:k])

    # also show N and D behavior (helps diagnose delta blow-up)
    N1, D1 = uniq_sorted[0][1], uniq_sorted[0][2]
    Nk = [x[1] for x in uniq_sorted[:k]]
    Dk = [x[2] for x in uniq_sorted[:k]]

    def _safe_mean(arr):
        return sum(arr) / max(1, len(arr))

    print(f"[PANEL] {tag}")
    print(f"  unique_paths : {len(uniq_sorted)}")
    print(f"  top1 score   : {top1:.6g}    (N={N1:.6g}, D={D1:.6g})")
    print(f"  sum top{k}   : {sumk:.6g}")
    print(f"  top{k} <N>   : {_safe_mean(Nk):.6g}")
    print(f"  top{k} <D>   : {_safe_mean(Dk):.6g}")

def main():
    ap = argparse.ArgumentParser(
        description="Rank Tc-Se-X-Se-Tc paths by (product |t|) / (product Δ) using onsite energies from hr.dat."
    )
    ap.add_argument("--edges", required=True, help="Input edges csv (from lstep.py)")
    ap.add_argument("--hr", required=True, help="wannier90_hr.dat path for onsite extraction")
    ap.add_argument("--out", default="top_paths_ratio.csv", help="Output ranked paths csv")

    ap.add_argument("--mediators", default="Ir,Ge", help="Comma-separated mediators, e.g. Ir,Ge or Ir only")
    ap.add_argument("--d_tcse", type=float, default=3.0, help="Max distance for Tc-Se edges (Ang)")
    ap.add_argument("--d_sex", type=float, default=3.0, help="Max distance for Se-X edges (Ang)")
    ap.add_argument("--min_absH", type=float, default=1e-6, help="Min |t| (eV) used for edges in path building")

    # channel filter using 'pair' column
    ap.add_argument("--X_orb", choices=["auto", "Ir_d", "Ge_d", "Ge_p", "Ge_all"],
                    default="auto",
                    help=("Filter mediator orbital channel using edge 'pair' labels from lstep.py. "
                          "auto: Ir->Ir_d, Ge->Ge_all."))

    ap.add_argument("--top_per_se_tc", type=int, default=60)
    ap.add_argument("--top_per_se_x", type=int, default=120)
    ap.add_argument("--top_per_x", type=int, default=120)
    ap.add_argument("--top_paths", type=int, default=2000, help="How many top paths to output after ranking")

```

```

ap.add_argument("--require_same_tc_atom", action="store_true")
ap.add_argument("--max_netR_L1", type=int, default=6, help="Filter by |dRx|+|dRy|+|dRz| <= this (999
↳ disables)")
ap.add_argument("--exclude_netR0", action="store_true", help="Drop net_dR=(0,0,0) loops")

ap.add_argument("--delta_mode", choices=["sequential", "pairwise"], default="sequential")
ap.add_argument("--delta_floor", type=float, default=1e-3)

ap.add_argument("--add_reverse_edges", action="store_true",
                help="Also add reversed edges. Usually OFF for new 1step edges (already directed).")

# NEW: panel printing
ap.add_argument("--panel_topk", type=int, default=100,
                help="In-shell panel: report sum of topK scores (default 100).")

args = ap.parse_args()

mediators = [x.strip() for x in args.mediators.split(",") if x.strip()]
if not mediators:
    raise RuntimeError("No mediators specified.")

eps = load_onsite_from_hr(args.hr)
all_dir = parse_edges_csv(args.edges, add_reverse=args.add_reverse_edges)

tc_atoms = sorted({e.u_atom for e in all_dir if e.u_elem == "Tc"} | {e.v_atom for e in all_dir if e.v_elem ==
↳ "Tc"})
se_atoms = sorted({e.u_atom for e in all_dir if e.u_elem == "Se"} | {e.v_atom for e in all_dir if e.v_elem ==
↳ "Se"})
if not tc_atoms:
    raise RuntimeError("No Tc atoms found.")
if not se_atoms:
    raise RuntimeError("No Se atoms found.")

print(f"[INFO] Tc atoms: {tc_atoms}")
print(f"[INFO] Se atoms: {se_atoms}")
for M in mediators:
    med_atoms = sorted({e.u_atom for e in all_dir if e.u_elem == M} | {e.v_atom for e in all_dir if e.v_elem
↳ == M})
    print(f"[INFO] {M} atoms: {med_atoms}")

def edge_pass_channel(e: DirEdge, M: str) -> bool:
    if not e.pair:
        return True
    p = e.pair
    if M == "Ir":
        # Ir is d in your setting
        return ("Ir_d" in p)
    if M == "Ge":
        if args.X_orb == "Ge_d":
            return ("Ge_d" in p)
        if args.X_orb == "Ge_p":
            return ("Ge_p" in p)
        if args.X_orb in ("Ge_all", "auto"):
            return ("Ge_d" in p) or ("Ge_p" in p) or ("Ge" in p)
    return True

tc_to_se = defaultdict(list)
se_to_tc = defaultdict(list)
se_to_x = {M: defaultdict(list) for M in mediators}
x_to_se = {M: defaultdict(list) for M in mediators}

for e in all_dir:
    if e.absH < args.min_absH:
        continue

    if e.u_elem == "Tc" and e.v_elem == "Se" and e.dist <= args.d_tcse:
        tc_to_se[e.u_atom].append(e)

```

```

        continue
    if e.u_elem == "Se" and e.v_elem == "Tc" and e.dist <= args.d_tcse:
        se_to_tc[e.u_atom].append(e)
        continue

    if e.dist <= args.d_sex:
        for M in mediators:
            if e.u_elem == "Se" and e.v_elem == M and edge_pass_channel(e, M):
                se_to_x[M][e.u_atom].append(e)
            elif e.u_elem == M and e.v_elem == "Se" and edge_pass_channel(e, M):
                x_to_se[M][e.u_atom].append(e)

keep_top_per_key(tc_to_se, topn=max(args.top_per_se_tc, 30))
keep_top_per_key(se_to_tc, topn=max(args.top_per_se_tc, 30))
for M in mediators:
    keep_top_per_key(se_to_x[M], topn=max(args.top_per_se_x, 50))
    keep_top_per_key(x_to_se[M], topn=max(args.top_per_x, 50))

def compute_denoms(tc_wf, se1_wf, x_wf, se2_wf):
    floor = args.delta_floor
    if args.delta_mode == "sequential":
        d1 = delta_pair(eps, se1_wf, tc_wf, floor)
        d2 = delta_pair(eps, x_wf, se1_wf, floor)
        d3 = delta_pair(eps, se2_wf, x_wf, floor)
    else:
        d1 = delta_pair(eps, se1_wf, tc_wf, floor)
        d2 = delta_pair(eps, x_wf, tc_wf, floor)
        d3 = delta_pair(eps, se2_wf, tc_wf, floor)

    if any([d != d for d in (d1, d2, d3)]):
        return float("nan"), d1, d2, d3
    return d1 * d2 * d3, d1, d2, d3

# ----- enumerate -----
paths_by_M = {M: [] for M in mediators}

for tc0 in tc_atoms:
    shift0 = (0, 0, 0)

    for e1 in tc_to_se.get(tc0, []):
        se1 = e1.v_atom
        shift_sel = add_shift(shift0, (e1.Rx, e1.Ry, e1.Rz))

        for M in mediators:
            for e2 in se_to_x[M].get(se1, []):
                x = e2.v_atom
                shift_x = add_shift(shift_sel, (e2.Rx, e2.Ry, e2.Rz))

                for e3 in x_to_se[M].get(x, []):
                    se2 = e3.v_atom
                    shift_se2 = add_shift(shift_x, (e3.Rx, e3.Ry, e3.Rz))

                    for e4 in se_to_tc.get(se2, []):
                        tc1 = e4.v_atom
                        shift_tc1 = add_shift(shift_se2, (e4.Rx, e4.Ry, e4.Rz))

                        if args.require_same_tc_atom and (tc1 != tc0):
                            continue

                        netR = shift_tc1
                        if args.exclude_netR0 and netR == (0, 0, 0):
                            continue
                        if args.max_netR_L1 < 999 and abs_R_L1(netR) > args.max_netR_L1:
                            continue

                        N = e1.absH * e2.absH * e3.absH * e4.absH
                        D, d1, d2, d3 = compute_denoms(e1.u_wf, e1.v_wf, e2.v_wf, e3.v_wf)
                        if D != D:
                            continue

```

```

        score = N / D

        sig = (
            M,
            tc0, se1, x, se2, tc1,
            netR[0], netR[1], netR[2],
            (e1.u_wf, e1.v_wf, e1.Rx, e1.Ry, e1.Rz),
            (e2.u_wf, e2.v_wf, e2.Rx, e2.Ry, e2.Rz),
            (e3.u_wf, e3.v_wf, e3.Rx, e3.Ry, e3.Rz),
            (e4.u_wf, e4.v_wf, e4.Rx, e4.Ry, e4.Rz),
        )

        paths_by_M[M].append((score, N, D, d1, d2, d3, sig,
                               M, tc0, se1, x, se2, tc1, netR, e1, e2, e3, e4))

# ----- de-dup within each mediator -----
uniq_by_M = {}
for M, paths in paths_by_M.items():
    if not paths:
        uniq_by_M[M] = []
        continue
    best = {}
    for item in paths:
        sc, sig = item[0], item[6]
        if sig not in best or sc > best[sig][0]:
            best[sig] = item
    uniq = list(best.values())
    uniq.sort(key=lambda x: x[0], reverse=True)
    uniq_by_M[M] = uniq

# ----- panel print -----
print("\n===== SHELL PANEL SUMMARY =====")
for M in mediators:
    tag = f"{M} (X_orb={args.X_orb})"
    print_panel_summary(tag, uniq_by_M[M], panel_topk=args.panel_topk)

# ----- merge for output CSV -----
merged = []
for M in mediators:
    merged.extend(uniq_by_M[M])

if not merged:
    raise RuntimeError("No paths found after applying constraints. Try loosening windows or lowering
↳ --min_absH.")

merged.sort(key=lambda x: x[0], reverse=True)
merged = merged[:args.top_paths]

# ----- write output -----
with open(args.out, "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f)
    w.writerow([
        "mediator", "X_orb",
        "score_num_over_denom", "N_prod_absH", "D_prod_delta",
        "delta1", "delta2", "delta3",
        "Tc0_atom", "Se1_atom", "X_atom", "Se2_atom", "Tc1_atom",
        "net_dRx", "net_dRy", "net_dRz",
        "t1_absH", "d1_A", "wf1_u", "wf1_v", "R1x", "R1y", "R1z",
        "t2_absH", "d2_A", "wf2_u", "wf2_v", "R2x", "R2y", "R2z",
        "t3_absH", "d3_A", "wf3_u", "wf3_v", "R3x", "R3y", "R3z",
        "t4_absH", "d4_A", "wf4_u", "wf4_v", "R4x", "R4y", "R4z",
    ])

for score, N, D, d1, d2, d3, sig, M, tc0, se1, x, se2, tc1, netR, e1, e2, e3, e4 in merged:
    w.writerow([
        M, args.X_orb,
        f"{score:.10g}", f"{N:.10g}", f"{D:.10g}",
        f"{d1:.10g}", f"{d2:.10g}", f"{d3:.10g}",
        tc0, se1, x, se2, tc1,

```

```

        netR[0], netR[1], netR[2],
        f"{e1.absH:.10g}", f"{e1.dist:.6f}", e1.u_wf, e1.v_wf, e1.Rx, e1.Ry, e1.Rz,
        f"{e2.absH:.10g}", f"{e2.dist:.6f}", e2.u_wf, e2.v_wf, e2.Rx, e2.Ry, e2.Rz,
        f"{e3.absH:.10g}", f"{e3.dist:.6f}", e3.u_wf, e3.v_wf, e3.Rx, e3.Ry, e3.Rz,
        f"{e4.absH:.10g}", f"{e4.dist:.6f}", e4.u_wf, e4.v_wf, e4.Rx, e4.Ry, e4.Rz,
    ])

    print(f"\n[DONE] wrote ranked paths to: {args.out}")
    print(f"[INFO] delta_mode={args.delta_mode}, delta_floor={args.delta_floor} eV")

if __name__ == "__main__":
    main()

```

(原创自研) 利用 wannier 验证超超交换路径 (3)

► 代码清单 A.59.2 (python, 402 行)

```

import re
import csv
import math
import argparse
from collections import defaultdict, namedtuple

import numpy as np

Edge = namedtuple(
    "Edge",
    "pair shell dist absH re im Rx Ry Rz m n atom_m atom_n elem_m elem_n group_m group_n"
)

# =====
# Parse wannier90.win
# =====
def _extract_block(txt, block_name):
    m = re.search(rf"begin\s+{block_name}(.*?)end\s+{block_name}", txt, re.S | re.I)
    return None if not m else m.group(1).strip()

def parse_win_lattice_and_atoms(win_path):
    """
    Read unit_cell_cart and atoms_cart from wannier90.win.
    Assumes Angstrom if unit not specified.
    Returns:
        A (3x3) with columns = a1,a2,a3 in Angstrom, so cart = A @ frac
        atoms: list of dict {id, elem, r_cart(np.array shape(3))}
    """
    with open(win_path, "r", encoding="utf-8", errors="ignore") as f:
        txt = f.read()

    cell_block = _extract_block(txt, "unit_cell_cart")
    if cell_block is None:
        raise RuntimeError("Cannot find unit_cell_cart block in wannier90.win")

    lines = [ln.strip() for ln in cell_block.splitlines() if ln.strip()]
    if re.match(r"^(ang|angstrom|bohr)\b", lines[0], re.I):
        unit = lines[0].lower()
        vec_lines = lines[1:4]
    else:
        unit = "ang"
        vec_lines = lines[0:3]

```

```

if len(vec_lines) < 3:
    raise RuntimeError("unit_cell_cart has < 3 lattice vectors")

a1 = np.array([float(x) for x in vec_lines[0].split()[3:]], dtype=float)
a2 = np.array([float(x) for x in vec_lines[1].split()[3:]], dtype=float)
a3 = np.array([float(x) for x in vec_lines[2].split()[3:]], dtype=float)

if "bohr" in unit:
    bohr_to_ang = 0.52917721092
    a1 *= bohr_to_ang
    a2 *= bohr_to_ang
    a3 *= bohr_to_ang

A = np.stack([a1, a2, a3], axis=1) # (3,3), columns are lattice vectors

atoms_block = _extract_block(txt, "atoms_cart")
if atoms_block is None:
    raise RuntimeError("Cannot find atoms_cart block in wannier90.win")

atoms = []
for idx, ln in enumerate([x for x in atoms_block.splitlines() if x.strip()], start=1):
    parts = ln.split()
    if len(parts) < 4:
        continue
    elem = parts[0]
    r = np.array([float(parts[1]), float(parts[2]), float(parts[3])], dtype=float)
    atoms.append({"id": idx, "elem": elem, "r": r})

if not atoms:
    raise RuntimeError("atoms_cart parsed but got 0 atoms")

return A, atoms

# =====
# Parse wannier90_centres.xyz
# =====
def parse_centres_xyz(xyz_path):
    """
    Parse wannier90_centres.xyz:
    line1 = N
    line2 = comment
    then N lines: <label> x y z
    Return centers list indexed 1..N: centers[wf] = np.array([x,y,z])
    """
    with open(xyz_path, "r", encoding="utf-8", errors="ignore") as f:
        lines = [ln.strip() for ln in f if ln.strip()]

    n = int(lines[0])
    if len(lines) < 2 + n:
        raise RuntimeError(f"centres.xyz incomplete: need {2+n} lines, got {len(lines)}")

    centers = [None] * (n + 1)
    for i in range(n):
        parts = lines[2 + i].split()
        if len(parts) < 4:
            raise RuntimeError(f"Bad centres.xyz line: {lines[2+i]}")
        centers[i + 1] = np.array([float(parts[1]), float(parts[2]), float(parts[3])], dtype=float)
    return centers

# =====
# Parse wannier90_hr.dat
# =====
def parse_hr_dat(hr_path):
    """
    Yields (Rx, Ry, Rz, m, n, reH, imH)
    """

```

```

with open(hr_path, "r", encoding="utf-8", errors="ignore") as f:
    _ = f.readline() # comment
    _num_wann = int(f.readline().strip())
    nrpts = int(f.readline().strip())

    # read degeneracy list
    degen = []
    while len(degen) < nrpts:
        ln = f.readline()
        if not ln:
            raise RuntimeError("Unexpected EOF while reading degeneracy list")
        ln = ln.strip()
        if not ln:
            continue
        degen += [int(x) for x in ln.split()]

    # data
    for ln in f:
        ln = ln.strip()
        if not ln:
            continue
        parts = ln.split()
        if len(parts) < 7:
            continue
        Rx, Ry, Rz = int(parts[0]), int(parts[1]), int(parts[2])
        m, n = int(parts[3]), int(parts[4])
        reH, imH = float(parts[5]), float(parts[6])
        yield Rx, Ry, Rz, m, n, reH, imH

# =====
# Group mapping: element -> group label
# =====
def parse_group_map(s):
    """
    "Tc:Tc_d,Ir:Ir_d,Se:Se_p,Ge:Ge_p" -> dict
    """
    mp = {}
    s = (s or "").strip()
    if not s:
        return mp
    for item in s.split(","):
        item = item.strip()
        if not item:
            continue
        if ":" not in item:
            raise ValueError(f"Bad group_map item '{item}', expected Elem:Group")
        k, v = item.split(":", 1)
        mp[k.strip()] = v.strip()
    return mp

def elem_to_group(elem, group_map):
    """
    Default rules (can be overridden by --group_map):
    Tc -> Tc_d, Ir -> Ir_d, Se -> Se_p, Ge -> Ge_p
    """
    if elem in group_map:
        return group_map[elem]
    if elem == "Tc":
        return "Tc_d"
    if elem == "Ir":
        return "Ir_d"
    if elem == "Se":
        return "Se_p"
    if elem == "Ge":
        return "Ge_p"
    return "OTHER"

def pair_name(g1, g2, order_map):

```

```

"""
Deterministic ordering, avoid Se_p<->Tc_d flip.
"""
o1 = order_map.get(g1, 999)
o2 = order_map.get(g2, 999)
if o1 < o2:
    return f"{g1}<->{g2}"
if o2 < o1:
    return f"{g2}<->{g1}"
return f"{min(g1,g2)}<->{max(g1,g2)}"

# =====
# Minimum-image mapping: WF -> nearest atom
# =====
def cart_to_frac(A, r_cart):
    return np.linalg.solve(A, r_cart)

def frac_to_cart(A, f):
    return A @ f

def wrap_delta_frac(df):
    return df - np.round(df)

def map_wf_to_atoms(A, centers, atoms):
    """
    Returns:
        wf_atom_id[wf] = atom index in atoms list (0-based)
        wf_atom_elem[wf] = element string
    Uses minimum-image distance in fractional space.
    """
    atom_frac = []
    for a in atoms:
        atom_frac.append(cart_to_frac(A, a["r"]))
    atom_frac = np.array(atom_frac) # (Nat,3)

    wf_atom_id = [None] * len(centers)
    wf_atom_elem = [None] * len(centers)

    for wf in range(1, len(centers)):
        f_w = cart_to_frac(A, centers[wf])

        best_i = None
        best_d = 1e30
        for i, f_a in enumerate(atom_frac):
            df = wrap_delta_frac(f_w - f_a)
            dr = frac_to_cart(A, df)
            d = float(np.linalg.norm(dr))
            if d < best_d:
                best_d = d
                best_i = i

        wf_atom_id[wf] = best_i
        wf_atom_elem[wf] = atoms[best_i]["elem"]

    return wf_atom_id, wf_atom_elem

# =====
# Main
# =====
def main():
    ap = argparse.ArgumentParser(
        description="Summarize hoppings by (group-pair, distance shell) using WF centres + lattice translations."
    )
    ap.add_argument("--win", required=True, help="wannier90.win")
    ap.add_argument("--centres", required=True, help="wannier90_centres.xyz")
    ap.add_argument("--hr", required=True, help="wannier90_hr.dat")
    ap.add_argument("--tol", type=float, default=0.10, help="distance bin size (Ang), e.g. 0.05~0.15")

```

```

ap.add_argument("--min_absH", type=float, default=1e-4, help="min |H| kept (eV)")
ap.add_argument("--topk", type=int, default=30, help="topK edges per (pair,shell)")
ap.add_argument("--skip_same_atom_R0", action="store_true",
                help="skip terms where (atom_m==atom_n) AND (R==0). Recommended to remove onsite/local
                ↪ terms.")
ap.add_argument("--skip_diag_R0", action="store_true",
                help="skip diagonal terms (m==n) at R==0 (onsite).")
ap.add_argument("--pairs", default="ALL", help="comma-separated allowed pairs like Tc_d<->Se_p, or ALL")
ap.add_argument("--out_summary", default="hopping_summary_by_shell.csv")
ap.add_argument("--out_edges", default="", help="optional: write filtered edges csv (empty=off)")

# NEW: element -> group mapping (to replace hard-coded wf index ranges)
ap.add_argument("--group_map", default="Tc:Tc_d,Ir:Ir_d,Se:Se_p,Ge:Ge_p",
                help="Override element->group mapping, e.g. 'Tc:Tc_d,Ir:Ir_d,Se:Se_p'.")
ap.add_argument("--group_order", default="Tc_d,Ir_d,Se_p,Ge_p,OTHER",
                help="Group ordering used to format A<->B deterministically.")

args = ap.parse_args()

group_map = parse_group_map(args.group_map)
order_list = [x.strip() for x in args.group_order.split(",") if x.strip()]
order_map = {g: i for i, g in enumerate(order_list, start=1)}

A, atoms = parse_win_lattice_and_atoms(args.win)
centers = parse_centres_xyz(args.centres)

wf_atom_id, wf_atom_elem = map_wf_to_atoms(A, centers, atoms)

# quick sanity print: how many WFs mapped to each element
elem_counts = defaultdict(int)
for wf in range(1, len(centers)):
    elem_counts[wf_atom_elem[wf]] += 1
print("[INFO] WF->nearest-atom element counts:", dict(sorted(elem_counts.items(), key=lambda x: x[0])))

# Allowed pairs
allowed_pairs = None
if args.pairs.strip().upper() != "ALL":
    allowed_pairs = set([p.strip() for p in args.pairs.split(",") if p.strip()])

# stats
cnt = defaultdict(int)
sumsq = defaultdict(float)
maxv = defaultdict(float)
top_edges = defaultdict(list)
filtered_edges = []

# Precompute lattice vectors for translation
a1 = A[:, 0]
a2 = A[:, 1]
a3 = A[:, 2]

def R_to_T(Rx, Ry, Rz):
    return Rx * a1 + Ry * a2 + Rz * a3

for Rx, Ry, Rz, m, n, reH, imH in parse_hr_dat(args.hr):
    absH = math.hypot(reH, imH)
    if absH < args.min_absH:
        continue

    atom_m = wf_atom_id[m]
    atom_n = wf_atom_id[n]
    elem_m = wf_atom_elem[m]
    elem_n = wf_atom_elem[n]

    gm = elem_to_group(elem_m, group_map)
    gn = elem_to_group(elem_n, group_map)

    # drop OTHER by default (consistent with your original behavior)
    if gm == "OTHER" or gn == "OTHER":

```

```

        continue

    pair = pair_name(gm, gn, order_map=order_map)
    if allowed_pairs is not None and pair not in allowed_pairs:
        continue

    # Skip onsite/local terms if requested
    if (Rx, Ry, Rz) == (0, 0, 0):
        if args.skip_diag_R0 and (m == n):
            continue
        if args.skip_same_atom_R0 and (atom_m == atom_n):
            continue

    # distance between WF centers considering translation R
    T = R_to_T(Rx, Ry, Rz)
    dr = (centers[n] + T) - centers[m]
    dist = float(np.linalg.norm(dr))

    # bin to shell
    shell = round(dist / args.tol) * args.tol
    key = (pair, shell)

    cnt[key] += 1
    sumsq[key] += absH * absH
    if absH > maxv[key]:
        maxv[key] = absH

    e = Edge(pair, shell, dist, absH, reH, imH, Rx, Ry, Rz, m, n,
            atom_m, atom_n, elem_m, elem_n, gm, gn)

    lst = top_edges[key]
    lst.append(e)
    lst.sort(key=lambda x: x.absH, reverse=True)
    if len(lst) > args.topk:
        lst[:] = lst[:args.topk]

    if args.out_edges:
        filtered_edges.append(e)

# write summary
with open(args.out_summary, "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f)
    w.writerow([
        "pair", "shell_A", "count", "max_absH_eV", "rms_absH_eV",
        f"top{args.topk}_edges(m,n,atom_m,atom_n,elem_m,elem_n,group_m,group_n,Rx,Ry,Rz,absH,dist)"
    ])
    for (pair, shell) in sorted(cnt.keys(), key=lambda x: (x[0], x[1])):
        c = cnt[(pair, shell)]
        rms = math.sqrt(sumsq[(pair, shell)] / c) if c else 0.0
        tops = top_edges[(pair, shell)]
        tops_str = "; ".join([
            f"{e.m}-{e.n}|a{e.atom_m+1}-a{e.atom_n+1}|{e.elem_m}-{e.elem_n}"
            f"|{e.group_m}-{e.group_n}"
            f"@({e.Rx},{e.Ry},{e.Rz})|{e.absH:.6g}|d={e.dist:.3f}"
            for e in tops
        ])
        w.writerow([pair, f"{shell:.3f}", c, f"{maxv[(pair, shell)]:.6g}", f"{rms:.6g}", tops_str])

# optional edges
if args.out_edges:
    with open(args.out_edges, "w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow([
            "pair", "shell_A", "dist_A", "absH_eV", "re", "im",
            "Rx", "Ry", "Rz", "m", "n",
            "atom_m", "atom_n", "elem_m", "elem_n",
            "group_m", "group_n"
        ])
        for e in filtered_edges:

```

```

        w.writerow([
            e.pair, f"{e.shell:.3f}", f"{e.dist:.6f}", f"{e.absH:.8g}",
            f"{e.re:.8g}", f"{e.im:.8g}",
            e.Rx, e.Ry, e.Rz, e.m, e.n,
            e.atom_m + 1, e.atom_n + 1, e.elem_m, e.elem_n,
            e.group_m, e.group_n
        ])

    print("Done.")
    print("Summary:", args.out_summary)
    if args.out_edges:
        print("Edges:", args.out_edges)

if __name__ == "__main__":
    main()

```

► 代码清单 A.59.4 (python, 386 行)

```

import csv
import argparse
from dataclasses import dataclass
from collections import defaultdict
from typing import Tuple, List, Dict, Optional

@dataclass(frozen=True)
class DirEdge:
    # directed edge: u(cell_shift) -> v(cell_shift + R)
    u_atom: int
    v_atom: int
    u_elem: str
    v_elem: str
    u_group: str
    v_group: str
    u_wf: int # 1-based wf index
    v_wf: int # 1-based wf index
    Rx: int
    Ry: int
    Rz: int
    absH: float
    dist: float

def parse_edges_csv(path: str, prefer_group: bool = True) -> List[DirEdge]:
    """
    Read out_edges csv produced by NEW lstep (recommended) or old format.
    Required columns (minimum):
        dist_A, absH_eV, Rx,Ry,Rz, m,n, atom_m,atom_n, elem_m,elem_n
    Optional (NEW, preferred):
        group_m, group_n

    If group_m/group_n exists and prefer_group=True, use them; otherwise fallback to elem.
    """
    edges: List[DirEdge] = []
    with open(path, "r", encoding="utf-8", errors="ignore", newline="") as f:
        r = csv.DictReader(f)
        required = ["dist_A", "absH_eV", "Rx", "Ry", "Rz",
                    "m", "n", "atom_m", "atom_n", "elem_m", "elem_n"]
        for k in required:
            if k not in r.fieldnames:
                raise RuntimeError(f"Missing column '{k}' in {path}. Found: {r.fieldnames}")

```

```

has_group = ("group_m" in r.fieldnames) and ("group_n" in r.fieldnames)
if prefer_group and (not has_group):
    print("[WARN] group_m/group_n not found. Falling back to elem_m/elem_n filtering.")

for row in r:
    try:
        dist = float(row["dist_A"])
        absH = float(row["absH_eV"])
        Rx, Ry, Rz = int(row["Rx"]), int(row["Ry"]), int(row["Rz"])
        m, n = int(row["m"]), int(row["n"])
        atom_m, atom_n = int(row["atom_m"]), int(row["atom_n"])
        elem_m, elem_n = row["elem_m"].strip(), row["elem_n"].strip()
    except Exception:
        continue

    if has_group:
        gm = row["group_m"].strip()
        gn = row["group_n"].strip()
    else:
        gm = elem_m
        gn = elem_n

    # Add both directions to make path enumeration easier.
    edges.append(DirEdge(atom_m, atom_n, elem_m, elem_n, gm, gn, m, n, Rx, Ry, Rz, absH, dist))
    edges.append(DirEdge(atom_n, atom_m, elem_n, elem_m, gn, gm, n, m, -Rx, -Ry, -Rz, absH, dist))

return edges

def load_onsite_from_hr(hr_path: str, num_wann: Optional[int] = None) -> Dict[int, float]:
    """
    Parse wannier90_hr.dat and extract onsite energies eps[m] = Re(H_mm(R=0)).
    Returns dict: wf_index(1-based) -> eps_eV
    """
    with open(hr_path, "r", encoding="utf-8", errors="ignore") as f:
        lines = f.readlines()

    if len(lines) < 4:
        raise RuntimeError(f"hr.dat too short: {hr_path}")

    try:
        nw = int(lines[1].split()[0])
    except Exception:
        raise RuntimeError("Failed to parse num_wann from hr.dat line2.")
    if num_wann is not None and nw != num_wann:
        print(f"[WARN] num_wann mismatch: hr={nw} vs --num_wann={num_wann}. Using hr value.")
    num_wann = nw

    try:
        nrpts = int(lines[2].split()[0])
    except Exception:
        raise RuntimeError("Failed to parse nrpts from hr.dat line3.")

    deg = []
    idx = 3
    while idx < len(lines) and len(deg) < nrpts:
        parts = lines[idx].split()
        for p in parts:
            try:
                deg.append(int(p))
            except Exception:
                pass
        idx += 1
    if len(deg) < nrpts:
        raise RuntimeError("Failed to read full degeneracy list from hr.dat.")

    eps: Dict[int, float] = {}
    for j in range(idx, len(lines)):
        parts = lines[j].split()

```

```

    if len(parts) < 7:
        continue
    try:
        Rx, Ry, Rz = int(parts[0]), int(parts[1]), int(parts[2])
        m, n = int(parts[3]), int(parts[4])
        re = float(parts[5])
    except Exception:
        continue

    if Rx == 0 and Ry == 0 and Rz == 0 and m == n:
        eps[m] = re

if len(eps) < num_wann:
    print(f"[WARN] onsite found {len(eps)}/{num_wann}. Some WFs missing onsite (unusual).")
else:
    print(f"[INFO] onsite found {len(eps)}/{num_wann}.")
return eps

def add_shift(s: Tuple[int, int, int], R: Tuple[int, int, int]) -> Tuple[int, int, int]:
    return (s[0] + R[0], s[1] + R[1], s[2] + R[2])

def abs_R_L1(R: Tuple[int, int, int]) -> int:
    return abs(R[0]) + abs(R[1]) + abs(R[2])

def keep_top_per_key(d: Dict[int, List[DirEdge]], topn: int):
    for k in list(d.keys()):
        lst = d[k]
        lst.sort(key=lambda x: x.absH, reverse=True)
        if len(lst) > topn:
            d[k] = lst[:topn]

def delta_pair(eps: Dict[int, float], wf_a: int, wf_b: int, floor: float) -> float:
    ea = eps.get(wf_a, None)
    eb = eps.get(wf_b, None)
    if ea is None or eb is None:
        return float("nan")
    d = abs(ea - eb)
    return max(d, floor)

def main():
    ap = argparse.ArgumentParser(
        description="Rank Tc-Se-X-Se-Tc paths by (product |t|) / (product Δ) using onsite energies from hr.dat."
    )
    ap.add_argument("--edges", required=True, help="Input edges csv (NEW lstep out_edges is recommended)")
    ap.add_argument("--hr", required=True, help="wannier90_hr.dat path for onsite extraction")
    ap.add_argument("--out", default="top_paths_ratio.csv", help="Output ranked paths csv")

    # NEW: use group labels (Tc_d / Se_p / Ir_d ...) if present
    ap.add_argument("--prefer_group", action="store_true", help="Prefer group_m/group_n if present
    ↪ (recommended).")
    ap.add_argument("--tc_group", default="Tc_d", help="Group label for Tc-centered d-like WFs (default: Tc_d)")
    ap.add_argument("--se_group", default="Se_p", help="Group label for Se-centered p-like WFs (default: Se_p)")
    ap.add_argument("--mediators", default="Ir_d",
        help="Comma-separated mediator group labels (e.g. Ir_d,Ge_p). If no group cols, use element
        ↪ symbols (Ir,Ge).")

    ap.add_argument("--d_tcse", type=float, default=3.0, help="Max distance for Tc-Se edges (Ang)")
    ap.add_argument("--d_sex", type=float, default=3.0, help="Max distance for Se-X edges (Ang)")
    ap.add_argument("--min_absH", type=float, default=1e-3, help="Min |t| (eV) used for edges in path building")

    ap.add_argument("--top_per_se_tc", type=int, default=60)
    ap.add_argument("--top_per_se_x", type=int, default=120)
    ap.add_argument("--top_per_x", type=int, default=120)

```

```

ap.add_argument("--top_paths", type=int, default=2000, help="How many top paths to output after ranking")

ap.add_argument("--require_same_tc_atom", action="store_true")
ap.add_argument("--max_netR_L1", type=int, default=6, help="Filter by |dRx|+|dRy|+|dRz| <= this (999
↳ disables)")
ap.add_argument("--exclude_netR0", action="store_true", help="Drop net_dR=(0,0,0) loops (recommended for Tc-Tc
↳ exchange)")

ap.add_argument("--delta_mode", choices=["sequential", "pairwise"], default="sequential",
                help=("How to define  $\Delta$  product:\n"
                    " sequential:  $\Delta_1=|\epsilon(\text{Se1})-\epsilon(\text{Tc})|$ ,  $\Delta_2=|\epsilon(\text{X})-\epsilon(\text{Se1})|$ ,  $\Delta_3=|\epsilon(\text{Se2})-\epsilon(\text{X})|$ \n"
                    " pairwise:  $\Delta_1=|\epsilon(\text{Se1})-\epsilon(\text{Tc})|$ ,  $\Delta_2=|\epsilon(\text{X})-\epsilon(\text{Tc})|$ ,  $\Delta_3=|\epsilon(\text{Se2})-\epsilon(\text{Tc})|$ \n"
                    "Both are proxies; sequential is closer to step-by-step virtual hopping."))
ap.add_argument("--delta_floor", type=float, default=1e-3,
                help="Lower bound for each  $\Delta$  (eV) to avoid division blow-up when  $\epsilon$  nearly equal")

args = ap.parse_args()

mediators = [x.strip() for x in args.mediators.split(",") if x.strip()]
if not mediators:
    raise RuntimeError("No mediators specified.")

eps = load_onsite_from_hr(args.hr)

# IMPORTANT: prefer_group=True only makes sense if group_m/group_n exist.
all_dir = parse_edges_csv(args.edges, prefer_group=args.prefer_group)

# Decide whether we are filtering by group or elem:
# If group columns existed, DirEdge.u_group != DirEdge.u_elem in general; else group==elem.
use_group = True if args.prefer_group else False

# Helper accessors
def Utag(e: DirEdge) -> str:
    return e.u_group if use_group else e.u_elem

def Vtag(e: DirEdge) -> str:
    return e.v_group if use_group else e.v_elem

tc_tag = args.tc_group if use_group else "Tc"
se_tag = args.se_group if use_group else "Se"

# Build node lists
tc_atoms = sorted({e.u_atom for e in all_dir if Utag(e) == tc_tag} |
                  {e.v_atom for e in all_dir if Vtag(e) == tc_tag})
se_atoms = sorted({e.u_atom for e in all_dir if Utag(e) == se_tag} |
                  {e.v_atom for e in all_dir if Vtag(e) == se_tag})

if not tc_atoms:
    raise RuntimeError(f"No Tc nodes found under tag '{tc_tag}'. Check --prefer_group/--tc_group.")
if not se_atoms:
    raise RuntimeError(f"No Se nodes found under tag '{se_tag}'. Check --prefer_group/--se_group.")

print(f"[INFO] use_group={use_group}")
print(f"[INFO] Tc tag: {tc_tag}, Tc atoms: {tc_atoms}")
print(f"[INFO] Se tag: {se_tag}, Se atoms: {se_atoms}")
for M in mediators:
    med_atoms = sorted({e.u_atom for e in all_dir if Utag(e) == M} |
                      {e.v_atom for e in all_dir if Vtag(e) == M})
    print(f"[INFO] mediator '{M}' atoms: {med_atoms}")

# Build filtered edge pools
tc_to_se = defaultdict(list) # Tc atom -> edges Tc->Se
se_to_tc = defaultdict(list) # Se atom -> edges Se->Tc
se_to_x = {M: defaultdict(list) for M in mediators} # Se atom -> edges Se->M
x_to_se = {M: defaultdict(list) for M in mediators} # M atom -> edges M->Se

for e in all_dir:
    if e.absH < args.min_absH:

```

```

        continue

    u = Utag(e)
    v = Vtag(e)

    if u == tc_tag and v == se_tag and e.dist <= args.d_tcse:
        tc_to_se[e.u_atom].append(e)
        continue
    if u == se_tag and v == tc_tag and e.dist <= args.d_tcse:
        se_to_tc[e.u_atom].append(e)
        continue

    if e.dist <= args.d_sex:
        for M in mediators:
            if u == se_tag and v == M:
                se_to_x[M][e.u_atom].append(e)
            elif u == M and v == se_tag:
                x_to_se[M][e.u_atom].append(e)

    keep_top_per_key(tc_to_se, topn=max(args.top_per_se_tc, 30))
    keep_top_per_key(se_to_tc, topn=max(args.top_per_se_tc, 30))
    for M in mediators:
        keep_top_per_key(se_to_x[M], topn=max(args.top_per_se_x, 50))
        keep_top_per_key(x_to_se[M], topn=max(args.top_per_x, 50))

    paths = []

def compute_denoms(tc_wf, sel_wf, x_wf, se2_wf) -> Tuple[float, float, float, float]:
    floor = args.delta_floor
    if args.delta_mode == "sequential":
        d1 = delta_pair(eps, sel_wf, tc_wf, floor)
        d2 = delta_pair(eps, x_wf, sel_wf, floor)
        d3 = delta_pair(eps, se2_wf, x_wf, floor)
    else:
        d1 = delta_pair(eps, sel_wf, tc_wf, floor)
        d2 = delta_pair(eps, x_wf, tc_wf, floor)
        d3 = delta_pair(eps, se2_wf, tc_wf, floor)

    if any([d != d for d in (d1, d2, d3)]): # NaN
        return float("nan"), d1, d2, d3
    return d1 * d2 * d3, d1, d2, d3

for tc0 in tc_atoms:
    shift0 = (0, 0, 0)

    for e1 in tc_to_se.get(tc0, []): # Tc->Sel
        sel = e1.v_atom
        shift_sel = add_shift(shift0, (e1.Rx, e1.Ry, e1.Rz))

        for M in mediators:
            for e2 in se_to_x[M].get(sel, []): # Sel->X
                x = e2.v_atom
                shift_x = add_shift(shift_sel, (e2.Rx, e2.Ry, e2.Rz))

                for e3 in x_to_se[M].get(x, []): # X->Se2
                    se2 = e3.v_atom
                    shift_se2 = add_shift(shift_x, (e3.Rx, e3.Ry, e3.Rz))

                    for e4 in se_to_tc.get(se2, []): # Se2->Tc1
                        tc1 = e4.v_atom
                        shift_tc1 = add_shift(shift_se2, (e4.Rx, e4.Ry, e4.Rz))

                        if args.require_same_tc_atom and (tc1 != tc0):
                            continue

                        netR = shift_tc1
                        if args.exclude_netR0 and netR == (0, 0, 0):
                            continue
                        if args.max_netR_L1 < 999 and abs_R_L1(netR) > args.max_netR_L1:

```

```

        continue

    N = e1.absH * e2.absH * e3.absH * e4.absH
    D, d1, d2, d3 = compute_denoms(e1.u_wf, e1.v_wf, e2.v_wf, e3.v_wf)
    if D != D:
        continue

    score = N / D

    sig = (
        M,
        tc0, se1, x, se2, tc1,
        netR[0], netR[1], netR[2],
        (e1.u_wf, e1.v_wf, e1.Rx, e1.Ry, e1.Rz),
        (e2.u_wf, e2.v_wf, e2.Rx, e2.Ry, e2.Rz),
        (e3.u_wf, e3.v_wf, e3.Rx, e3.Ry, e3.Rz),
        (e4.u_wf, e4.v_wf, e4.Rx, e4.Ry, e4.Rz),
    )

    paths.append((score, N, D, d1, d2, d3, sig, M,
                  tc0, se1, x, se2, tc1, netR, e1, e2, e3, e4))

if not paths:
    raise RuntimeError("No paths found. Try lowering --min_absH or loosening distance windows "
                      "or check tc/se/mediator tags.")

best = {}
for item in paths:
    score = item[0]
    sig = item[6]
    if sig not in best or score > best[sig][0]:
        best[sig] = item
uniq = list(best.values())
print(f"[INFO] paths raw={len(paths)}, unique={len(uniq)} (by signature)")

uniq.sort(key=lambda x: x[0], reverse=True)
uniq = uniq[:args.top_paths]

with open(args.out, "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f)
    w.writerow([
        "mediator",
        "score_num_over_denom", "N_prod_absH", "D_prod_delta",
        "delta1", "delta2", "delta3",
        "Tc0_atom", "Se1_atom", "X_atom", "Se2_atom", "Tc1_atom",
        "net_dRx", "net_dRy", "net_dRz",
        "t1_absH", "d1_A", "wf1_u", "wf1_v", "R1x", "R1y", "R1z",
        "t2_absH", "d2_A", "wf2_u", "wf2_v", "R2x", "R2y", "R2z",
        "t3_absH", "d3_A", "wf3_u", "wf3_v", "R3x", "R3y", "R3z",
        "t4_absH", "d4_A", "wf4_u", "wf4_v", "R4x", "R4y", "R4z",
    ])

for score, N, D, d1, d2, d3, sig, M, tc0, se1, x, se2, tc1, netR, e1, e2, e3, e4 in uniq:
    w.writerow([
        M,
        f"{score:.10g}", f"{N:.10g}", f"{D:.10g}",
        f"{d1:.10g}", f"{d2:.10g}", f"{d3:.10g}",
        tc0, se1, x, se2, tc1,
        netR[0], netR[1], netR[2],
        f"{e1.absH:.10g}", f"{e1.dist:.6f}", e1.u_wf, e1.v_wf, e1.Rx, e1.Ry, e1.Rz,
        f"{e2.absH:.10g}", f"{e2.dist:.6f}", e2.u_wf, e2.v_wf, e2.Rx, e2.Ry, e2.Rz,
        f"{e3.absH:.10g}", f"{e3.dist:.6f}", e3.u_wf, e3.v_wf, e3.Rx, e3.Ry, e3.Rz,
        f"{e4.absH:.10g}", f"{e4.dist:.6f}", e4.u_wf, e4.v_wf, e4.Rx, e4.Ry, e4.Rz,
    ])

print(f"[DONE] wrote ranked unique paths to: {args.out}")
print(f"[INFO] delta_mode={args.delta_mode}, delta_floor={args.delta_floor} eV")
print(f"[INFO] prefer_group={args.prefer_group}, tc_group={args.tc_group}, se_group={args.se_group},
↪ mediators={mediators}")

```

```
if __name__ == "__main__":
    main()
```

► 代码清单 A.59.6 (python, 472 行)

```
import csv
import argparse
from dataclasses import dataclass
from collections import defaultdict
from typing import Tuple, List, Dict, Optional

@dataclass(frozen=True)
class DirEdge:
    # directed edge: u(cell_shift) -> v(cell_shift + R)
    u_atom: int
    v_atom: int
    u_elem: str
    v_elem: str
    u_group: str
    v_group: str
    u_wf: int # 1-based wf index
    v_wf: int # 1-based wf index
    Rx: int
    Ry: int
    Rz: int
    absH: float
    dist: float

def parse_edges_csv(path: str, prefer_group: bool = True) -> Tuple[List[DirEdge], Dict[int, str]]:
    """
    Read edges.csv produced by 1step.py.

    Required columns:
        dist_A, absH_eV, Rx,Ry,Rz, m,n, atom_m,atom_n, elem_m,elem_n
    Optional (recommended):
        group_m, group_n

    Returns:
        dir_edges: directed edges (both directions expanded)
        wf_tag: mapping wf_index -> tag (group if present else element)
    """
    edges: List[DirEdge] = []
    wf_tag: Dict[int, str] = {}

    with open(path, "r", encoding="utf-8", errors="ignore", newline="") as f:
        r = csv.DictReader(f)
        required = ["dist_A", "absH_eV", "Rx", "Ry", "Rz",
                    "m", "n", "atom_m", "atom_n", "elem_m", "elem_n"]
        for k in required:
            if k not in (r.fieldnames or []):
                raise RuntimeError(f"Missing column '{k}' in {path}. Found: {r.fieldnames}")

        has_group = ("group_m" in (r.fieldnames or [])) and ("group_n" in (r.fieldnames or []))
        if prefer_group and (not has_group):
            print("[WARN] group_m/group_n not found. Falling back to elem_m/elem_n filtering.")

        for row in r:
            try:
                dist = float(row["dist_A"])
```

```

        absH = float(row["absH_eV"])
        Rx, Ry, Rz = int(row["Rx"]), int(row["Ry"]), int(row["Rz"])
        m, n = int(row["m"]), int(row["n"])
        atom_m, atom_n = int(row["atom_m"]), int(row["atom_n"])
        elem_m, elem_n = row["elem_m"].strip(), row["elem_n"].strip()
    except Exception:
        continue

    if has_group:
        gm = row["group_m"].strip()
        gn = row["group_n"].strip()
    else:
        gm = elem_m
        gn = elem_n

    # wf -> tag (group or element)
    if m not in wf_tag:
        wf_tag[m] = gm
    if n not in wf_tag:
        wf_tag[n] = gn

    # Add both directions
    edges.append(DirEdge(atom_m, atom_n, elem_m, elem_n, gm, gn, m, n, Rx, Ry, Rz, absH, dist))
    edges.append(DirEdge(atom_n, atom_m, elem_n, elem_m, gn, gm, n, m, -Rx, -Ry, -Rz, absH, dist))

return edges, wf_tag

def load_onsite_from_hr(hr_path: str) -> Dict[int, float]:
    """
    Extract onsite energies eps[m] = Re(H_mm(R=0)) from wannier90_hr.dat.
    """
    with open(hr_path, "r", encoding="utf-8", errors="ignore") as f:
        lines = f.readlines()

    if len(lines) < 4:
        raise RuntimeError(f"hr.dat too short: {hr_path}")

    num_wann = int(lines[1].split()[0])
    nrpts = int(lines[2].split()[0])

    # degeneracy list
    deg = []
    idx = 3
    while idx < len(lines) and len(deg) < nrpts:
        parts = lines[idx].split()
        for p in parts:
            try:
                deg.append(int(p))
            except Exception:
                pass
        idx += 1
    if len(deg) < nrpts:
        raise RuntimeError("Failed to read full degeneracy list from hr.dat.")

    eps: Dict[int, float] = {}
    for j in range(idx, len(lines)):
        parts = lines[j].split()
        if len(parts) < 7:
            continue
        try:
            Rx, Ry, Rz = int(parts[0]), int(parts[1]), int(parts[2])
            m, n = int(parts[3]), int(parts[4])
            re = float(parts[5])
        except Exception:
            continue
        if Rx == 0 and Ry == 0 and Rz == 0 and m == n:
            eps[m] = re

```

```

    if len(eps) < num_wann:
        print(f"[WARN] onsite found {len(eps)}/{num_wann}. Some WFs missing onsite.")
    else:
        print(f"[INFO] onsite found {len(eps)}/{num_wann}.")
    return eps

def add_shift(s: Tuple[int, int, int], R: Tuple[int, int, int]) -> Tuple[int, int, int]:
    return (s[0] + R[0], s[1] + R[1], s[2] + R[2])

def abs_R_L1(R: Tuple[int, int, int]) -> int:
    return abs(R[0]) + abs(R[1]) + abs(R[2])

def keep_top_per_key(d: Dict[int, List[DirEdge]], topn: int):
    for k in list(d.keys()):
        lst = d[k]
        lst.sort(key=lambda x: x.absH, reverse=True)
        if len(lst) > topn:
            d[k] = lst[:topn]

def delta_pair_with_UJ(
    eps: Dict[int, float],
    wf_tag: Dict[int, str],
    wf_a: int,
    wf_b: int,
    floor: float,
    enable_UJ: bool,
    U_Tc: float,
    U_Ir: float,
    JH_Tc: float,
    JH_Ir: float,
    hund_coef: float,
    tc_tag_for_UJ: str,
    ir_tag_for_UJ: str,
) -> float:
    """
     $\Delta(a,b) = \max(|\epsilon_a - \epsilon_b| + UJ(a,b), \text{floor})$ 

    UJ(a,b) rule (minimal & robust):
        if either endpoint tag matches tc_tag_for_UJ -> add (U_Tc + hund_coef * JH_Tc)
        if either endpoint tag matches ir_tag_for_UJ -> add (U_Ir + hund_coef * JH_Ir)

    NOTE:
        - This is a controlled phenomenological correction.
        - hund_coef default is set to 2.0 (more suitable when d-manifold is not 5-fold degenerate).
    """
    ea = eps.get(wf_a, None)
    eb = eps.get(wf_b, None)
    if ea is None or eb is None:
        return float("nan")

    base = abs(ea - eb)
    if not enable_UJ:
        return max(base, floor)

    ta = wf_tag.get(wf_a, "")
    tb = wf_tag.get(wf_b, "")

    uj = 0.0
    if (ta == tc_tag_for_UJ) or (tb == tc_tag_for_UJ):
        uj += (U_Tc + hund_coef * JH_Tc)
    if (ta == ir_tag_for_UJ) or (tb == ir_tag_for_UJ):
        uj += (U_Ir + hund_coef * JH_Ir)

    return max(base + uj, floor)

```

```

def main():
    ap = argparse.ArgumentParser(
        description="Rank Tc-Se-X-Se-Tc paths by (product |t|) / (product Δ). "
        "Δ can include U + hund_coef*JH when Tc/Ir d tags are involved."
    )
    ap.add_argument("--edges", required=True, help="edges.csv from 1step.py")
    ap.add_argument("--hr", required=True, help="wannier90_hr.dat for onsite extraction")
    ap.add_argument("--out", default="top_paths_ratio.csv", help="Output ranked paths csv")

    # Filtering mode
    ap.add_argument("--prefer_group", action="store_true",
        help="Prefer group_m/group_n if present. If not, fall back to elem.")
    ap.add_argument("--tc_group", default="Tc_d", help="Tc tag when prefer_group is ON (default Tc_d)")
    ap.add_argument("--se_group", default="Se_p", help="Se tag when prefer_group is ON (default Se_p)")
    ap.add_argument("--mediators", default="Ir_d",
        help="Comma-separated mediator tags. If prefer_group OFF, use elements Ir/Ge...")

    # Geometry and pruning
    ap.add_argument("--d_tcse", type=float, default=3.0, help="Max distance for Tc-Se edges (Ang)")
    ap.add_argument("--d_sex", type=float, default=3.0, help="Max distance for Se-X edges (Ang)")
    ap.add_argument("--min_absH", type=float, default=1e-4, help="Min |t| (eV) used for edges in path building")
    ap.add_argument("--top_per_se_tc", type=int, default=60)
    ap.add_argument("--top_per_se_x", type=int, default=120)
    ap.add_argument("--top_per_x", type=int, default=120)
    ap.add_argument("--top_paths", type=int, default=2000)

    ap.add_argument("--require_same_tc_atom", action="store_true")
    ap.add_argument("--max_netR_L1", type=int, default=6)
    ap.add_argument("--exclude_netR0", action="store_true")

    # Denominator form
    ap.add_argument("--delta_mode", choices=["sequential", "pairwise"], default="sequential")
    ap.add_argument("--delta_floor", type=float, default=0.1, help="Lower bound for each Δ (eV)")

    # U/JH controls
    ap.add_argument("--enable_UJ", action="store_true",
        help="Enable adding U + hund_coef*JH to Δ when Tc/Ir tags are involved.")
    ap.add_argument("--U_Tc", type=float, default=3.0, help="Tc Hubbard U (eV). You set: 3.0")
    ap.add_argument("--U_Ir", type=float, default=1.0, help="Ir Hubbard U (eV). You set: 1.0")
    ap.add_argument("--JH_Tc", type=float, default=0.5, help="Tc Hund J_H (eV). Default 0.5")
    ap.add_argument("--JH_Ir", type=float, default=0.4, help="Ir Hund J_H (eV). Default 0.4")

    # IMPORTANT: default 2.0 (recommended for your split d-manifold)
    ap.add_argument("--hund_coef", type=float, default=2.0,
        help="Coefficient in U + hund_coef*JH. Default 2.0 (recommended). "
        "Try 1/2/4 for sensitivity test.")

    args = ap.parse_args()

    mediators = [x.strip() for x in args.mediators.split(",") if x.strip()]
    if not mediators:
        raise RuntimeError("No mediators specified.")

    eps = load_onsite_from_hr(args.hr)
    all_dir, wf_tag = parse_edges_csv(args.edges, prefer_group=args.prefer_group)

    # Decide tag space
    use_group = True if args.prefer_group else False

    def Utag(e: DirEdge) -> str:
        return e.u_group if use_group else e.u_elem

    def Vtag(e: DirEdge) -> str:
        return e.v_group if use_group else e.v_elem

    tc_tag = args.tc_group if use_group else "Tc"
    se_tag = args.se_group if use_group else "Se"

```

```

# These are the tags that trigger UJ penalty:
tc_tag_for_UJ = tc_tag
ir_tag_for_UJ = ("Ir_d" if use_group else "Ir")

tc_atoms = sorted({e.u_atom for e in all_dir if Utag(e) == tc_tag} |
                  {e.v_atom for e in all_dir if Vtag(e) == tc_tag})
se_atoms = sorted({e.u_atom for e in all_dir if Utag(e) == se_tag} |
                  {e.v_atom for e in all_dir if Vtag(e) == se_tag})

if not tc_atoms:
    raise RuntimeError(f"No Tc nodes found under tag '{tc_tag}'. Check --prefer_group/--tc_group.")
if not se_atoms:
    raise RuntimeError(f"No Se nodes found under tag '{se_tag}'. Check --prefer_group/--se_group.")

print(f"[INFO] use_group={use_group}, enable_UJ={args.enable_UJ}, hund_coef={args.hund_coef}")
print(f"[INFO] Tc tag: {tc_tag}, Tc atoms: {tc_atoms}")
print(f"[INFO] Se tag: {se_tag}, Se atoms: {se_atoms}")
print(f"[INFO] mediators: {mediators}")
if args.enable_UJ:
    print(f"[INFO] U_Tc={args.U_Tc} eV, U_Ir={args.U_Ir} eV, JH_Tc={args.JH_Tc} eV, JH_Ir={args.JH_Ir} eV")

# Build filtered edge pools
tc_to_se = defaultdict(list) # Tc atom -> edges Tc->Se
se_to_tc = defaultdict(list) # Se atom -> edges Se->Tc
se_to_x = {M: defaultdict(list) for M in mediators} # Se atom -> edges Se->M
x_to_se = {M: defaultdict(list) for M in mediators} # M atom -> edges M->Se

for e in all_dir:
    if e.absH < args.min_absH:
        continue

    u = Utag(e)
    v = Vtag(e)

    if u == tc_tag and v == se_tag and e.dist <= args.d_tcse:
        tc_to_se[e.u_atom].append(e)
        continue
    if u == se_tag and v == tc_tag and e.dist <= args.d_tcse:
        se_to_tc[e.u_atom].append(e)
        continue

    if e.dist <= args.d_sex:
        for M in mediators:
            if u == se_tag and v == M:
                se_to_x[M][e.u_atom].append(e)
            elif u == M and v == se_tag:
                x_to_se[M][e.u_atom].append(e)

keep_top_per_key(tc_to_se, topn=max(args.top_per_se_tc, 30))
keep_top_per_key(se_to_tc, topn=max(args.top_per_se_tc, 30))
for M in mediators:
    keep_top_per_key(se_to_x[M], topn=max(args.top_per_se_x, 50))
    keep_top_per_key(x_to_se[M], topn=max(args.top_per_x, 50))

def compute_denoms(tc_wf, sel_wf, x_wf, se2_wf) -> Tuple[float, float, float, float]:
    floor = args.delta_floor
    if args.delta_mode == "sequential":
        d1 = delta_pair_with_UJ(
            eps, wf_tag, sel_wf, tc_wf, floor,
            enable_UJ=args.enable_UJ,
            U_Tc=args.U_Tc, U_Ir=args.U_Ir, JH_Tc=args.JH_Tc, JH_Ir=args.JH_Ir,
            hund_coef=args.hund_coef,
            tc_tag_for_UJ=tc_tag_for_UJ, ir_tag_for_UJ=ir_tag_for_UJ
        )
    d2 = delta_pair_with_UJ(
        eps, wf_tag, x_wf, sel_wf, floor,
        enable_UJ=args.enable_UJ,
        U_Tc=args.U_Tc, U_Ir=args.U_Ir, JH_Tc=args.JH_Tc, JH_Ir=args.JH_Ir,
        hund_coef=args.hund_coef,

```

```

        tc_tag_for_UJ=tc_tag_for_UJ, ir_tag_for_UJ=ir_tag_for_UJ
    )
    d3 = delta_pair_with_UJ(
        eps, wf_tag, se2_wf, x_wf, floor,
        enable_UJ=args.enable_UJ,
        U_Tc=args.U_Tc, U_Ir=args.U_Ir, JH_Tc=args.JH_Tc, JH_Ir=args.JH_Ir,
        hund_coef=args.hund_coef,
        tc_tag_for_UJ=tc_tag_for_UJ, ir_tag_for_UJ=ir_tag_for_UJ
    )
else:
    d1 = delta_pair_with_UJ(
        eps, wf_tag, se1_wf, tc_wf, floor,
        enable_UJ=args.enable_UJ,
        U_Tc=args.U_Tc, U_Ir=args.U_Ir, JH_Tc=args.JH_Tc, JH_Ir=args.JH_Ir,
        hund_coef=args.hund_coef,
        tc_tag_for_UJ=tc_tag_for_UJ, ir_tag_for_UJ=ir_tag_for_UJ
    )
    d2 = delta_pair_with_UJ(
        eps, wf_tag, x_wf, tc_wf, floor,
        enable_UJ=args.enable_UJ,
        U_Tc=args.U_Tc, U_Ir=args.U_Ir, JH_Tc=args.JH_Tc, JH_Ir=args.JH_Ir,
        hund_coef=args.hund_coef,
        tc_tag_for_UJ=tc_tag_for_UJ, ir_tag_for_UJ=ir_tag_for_UJ
    )
    d3 = delta_pair_with_UJ(
        eps, wf_tag, se2_wf, tc_wf, floor,
        enable_UJ=args.enable_UJ,
        U_Tc=args.U_Tc, U_Ir=args.U_Ir, JH_Tc=args.JH_Tc, JH_Ir=args.JH_Ir,
        hund_coef=args.hund_coef,
        tc_tag_for_UJ=tc_tag_for_UJ, ir_tag_for_UJ=ir_tag_for_UJ
    )

if any([d != d for d in (d1, d2, d3)]): # NaN
    return float("nan"), d1, d2, d3
return d1 * d2 * d3, d1, d2, d3

paths = []

for tc0 in tc_atoms:
    shift0 = (0, 0, 0)

    for e1 in tc_to_se.get(tc0, []): # Tc->Se1
        se1 = e1.v_atom
        shift_se1 = add_shift(shift0, (e1.Rx, e1.Ry, e1.Rz))

        for M in mediators:
            for e2 in se_to_x[M].get(se1, []): # Se1->X
                x = e2.v_atom
                shift_x = add_shift(shift_se1, (e2.Rx, e2.Ry, e2.Rz))

                for e3 in x_to_se[M].get(x, []): # X->Se2
                    se2 = e3.v_atom
                    shift_se2 = add_shift(shift_x, (e3.Rx, e3.Ry, e3.Rz))

                    for e4 in se_to_tc.get(se2, []): # Se2->Tc1
                        tc1 = e4.v_atom
                        shift_tc1 = add_shift(shift_se2, (e4.Rx, e4.Ry, e4.Rz))

                        if args.require_same_tc_atom and (tc1 != tc0):
                            continue

                        netR = shift_tc1
                        if args.exclude_netR0 and netR == (0, 0, 0):
                            continue
                        if args.max_netR_L1 < 999 and abs_R_L1(netR) > args.max_netR_L1:
                            continue

                        N = e1.absH * e2.absH * e3.absH * e4.absH
                        D, d1, d2, d3 = compute_denoms(e1.u_wf, e1.v_wf, e2.v_wf, e3.v_wf)

```

```

        if D != D:
            continue

        score = N / D
        paths.append((score, N, D, d1, d2, d3,
                      M, tc0, se1, x, se2, tc1, netR,
                      e1, e2, e3, e4))

if not paths:
    raise RuntimeError("No paths found. Try loosening distances, lowering --min_absH, "
                      "or check tags (prefer_group/tc_group/se_group/mediators).")

# de-dup by signature
best = {}
for item in paths:
    score = item[0]
    key = (item[6], item[7], item[8], item[9], item[10], item[11], item[12],
          (item[13].u_wf, item[13].v_wf, item[13].Rx, item[13].Ry, item[13].Rz),
          (item[14].u_wf, item[14].v_wf, item[14].Rx, item[14].Ry, item[14].Rz),
          (item[15].u_wf, item[15].v_wf, item[15].Rx, item[15].Ry, item[15].Rz),
          (item[16].u_wf, item[16].v_wf, item[16].Rx, item[16].Ry, item[16].Rz))
    if key not in best or score > best[key][0]:
        best[key] = item

uniq = list(best.values())
uniq.sort(key=lambda x: x[0], reverse=True)
uniq = uniq[:args.top_paths]

with open(args.out, "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f)
    w.writerow([
        # run meta
        "use_group", "enable_UJ", "U_Tc", "U_Ir", "JH_Tc", "JH_Ir", "hund_coef", "delta_mode", "delta_floor",
        # path score
        "mediator",
        "score_num_over_denom", "N_prod_absH", "D_prod_delta",
        "delta1", "delta2", "delta3",
        "Tc0_atom", "Se1_atom", "X_atom", "Se2_atom", "Tc1_atom",
        "net_dRx", "net_dRy", "net_dRz",
        "t1_absH", "d1_A", "wf1_u", "wf1_v", "R1x", "R1y", "R1z",
        "t2_absH", "d2_A", "wf2_u", "wf2_v", "R2x", "R2y", "R2z",
        "t3_absH", "d3_A", "wf3_u", "wf3_v", "R3x", "R3y", "R3z",
        "t4_absH", "d4_A", "wf4_u", "wf4_v", "R4x", "R4y", "R4z",
    ])

for (score, N, D, d1, d2, d3,
     M, tc0, se1, x, se2, tc1, netR,
     e1, e2, e3, e4) in uniq:
    w.writerow([
        str(use_group), str(args.enable_UJ),
        f"{args.U_Tc:.6g}", f"{args.U_Ir:.6g}", f"{args.JH_Tc:.6g}", f"{args.JH_Ir:.6g}",
        f"{args.hund_coef:.6g}", args.delta_mode, f"{args.delta_floor:.6g}",
        M,
        f"{score:.10g}", f"{N:.10g}", f"{D:.10g}",
        f"{d1:.10g}", f"{d2:.10g}", f"{d3:.10g}",
        tc0, se1, x, se2, tc1,
        netR[0], netR[1], netR[2],
        f"{e1.absH:.10g}", f"{e1.dist:.6f}", e1.u_wf, e1.v_wf, e1.Rx, e1.Ry, e1.Rz,
        f"{e2.absH:.10g}", f"{e2.dist:.6f}", e2.u_wf, e2.v_wf, e2.Rx, e2.Ry, e2.Rz,
        f"{e3.absH:.10g}", f"{e3.dist:.6f}", e3.u_wf, e3.v_wf, e3.Rx, e3.Ry, e3.Rz,
        f"{e4.absH:.10g}", f"{e4.dist:.6f}", e4.u_wf, e4.v_wf, e4.Rx, e4.Ry, e4.Rz,
    ])

print(f"[DONE] wrote ranked unique paths to: {args.out}")
print(f"[INFO] delta_mode={args.delta_mode}, delta_floor={args.delta_floor} eV")
print(f"[INFO] enable_UJ={args.enable_UJ}, U_Tc={args.U_Tc}, U_Ir={args.U_Ir}, "
      f"JH_Tc={args.JH_Tc}, JH_Ir={args.JH_Ir}, hund_coef={args.hund_coef}")

```

```
if __name__ == "__main__":
    main()
```

► 代码清单 A.59.8 (python, 140 行)

```
#!/usr/bin/env python3
import argparse
import pandas as pd

def find_col(df, candidates, required=True, name=""):
    for c in candidates:
        if c in df.columns:
            return c
    if required:
        raise RuntimeError(f"Cannot find column for {name}. Tried {candidates}. "
                           f"Available columns: {list(df.columns)}")
    return None

def pretty_mediator(tag: str) -> str:
    # Ir_d -> Ir, Ge_p -> Ge
    if isinstance(tag, str) and "_" in tag:
        return tag.split("_")[0]
    return str(tag)

def load_2step(csv_path: str, source_label: str, topk: int):
    df = pd.read_csv(csv_path)

    score_col = find_col(df, ["score_num_over_denom", "score", "S"], True, "score")
    med_col = find_col(df, ["mediator", "mediator_tag", "M"], True, "mediator")

    Tc0 = find_col(df, ["Tc0", "Tc0_atom"], True, "Tc0")
    Se1 = find_col(df, ["Se1", "Se1_atom"], True, "Se1")
    X = find_col(df, ["X", "X_atom"], True, "X")
    Se2 = find_col(df, ["Se2", "Se2_atom"], True, "Se2")
    Tc1 = find_col(df, ["Tc1", "Tc1_atom"], True, "Tc1")

    netRx = find_col(df, ["net_dRx", "netRx", "net_Rx"], required=False, name="netRx")
    netRy = find_col(df, ["net_dRy", "netRy", "net_Ry"], required=False, name="netRy")
    netRz = find_col(df, ["net_dRz", "netRz", "net_Rz"], required=False, name="netRz")
    has_netR = (netRx is not None) and (netRy is not None) and (netRz is not None)

    df = df.copy()
    df[score_col] = pd.to_numeric(df[score_col], errors="coerce")
    df = df.dropna(subset=[score_col])
    df = df.sort_values(score_col, ascending=False)

    if topk and topk > 0:
        df = df.head(topk)

    df["source"] = source_label
    df["mediator"] = df[med_col].astype(str)

    df["Tc0"] = df[Tc0].astype(int)
    df["Se1"] = df[Se1].astype(int)
    df["X"] = df[X].astype(int)
    df["Se2"] = df[Se2].astype(int)
    df["Tc1"] = df[Tc1].astype(int)

    if has_netR:
        df["netRx"] = df[netRx].astype(int)
```

```

        df["netRy"] = df[netRy].astype(int)
        df["netRz"] = df[netRz].astype(int)
    else:
        df["netRx"] = 0
        df["netRy"] = 0
        df["netRz"] = 0

    # path string: Tc2-Se9-Ir1-Se8-Tc2
    def mk(row):
        M = pretty_mediator(row["mediator"])
        return f"Tc{row['Tc0']}-Se{row['Se1']}-{M}{row['X']}-Se{row['Se2']}-Tc{row['Tc1']}"
    df["path_str"] = df.apply(mk, axis=1)

    return df

def main():
    ap = argparse.ArgumentParser(
        description="Merge Ir/Ge 2step outputs, group by atom-chain, and keep top-3 rows per chain by score."
    )
    ap.add_argument("--ir_csv", default="Ir_d_UJ.csv")
    ap.add_argument("--ge_csv", default="Ge_1e-3_0.1.csv")
    ap.add_argument("--top_each", type=int, default=2000,
        help="Read top K rows from each file before grouping (0 = read all). Default 2000.")
    ap.add_argument("--top_per_chain", type=int, default=3,
        help="Keep top N rows per atom-chain (default 3).")
    ap.add_argument("--key_with_netR", action="store_true",
        help="Include netR (net_dR*) in chain key (recommended if netR exists).")
    ap.add_argument("--out", default="chains_top3.csv")
    args = ap.parse_args()

    topk = None if args.top_each == 0 else args.top_each

    df_ir = load_2step(args.ir_csv, "Ir", 0 if topk is None else topk)
    df_ge = load_2step(args.ge_csv, "Ge", 0 if topk is None else topk)

    df = pd.concat([df_ir, df_ge], ignore_index=True)

    # group key: atom chain (+ optional netR)
    key_cols = ["Tc0", "Se1", "X", "Se2", "Tc1"]
    if args.key_with_netR:
        key_cols += ["netRx", "netRy", "netRz"]

    # rank chains by their best score (for stable chain_id)
    best = df.groupby(key_cols, dropna=False)["score"].max().reset_index()
    best = best.sort_values("score", ascending=False).reset_index(drop=True)
    best["chain_id"] = range(1, len(best) + 1)

    df = df.merge(best[key_cols + ["chain_id"]], on=key_cols, how="left")

    # within each chain keep top N
    df = df.sort_values("score", ascending=False).copy()
    df["rank_in_chain"] = df.groupby("chain_id")["score"].rank(method="first", ascending=False).astype(int)
    out = df[df["rank_in_chain"] <= args.top_per_chain].copy()
    out = out.sort_values(["chain_id", "rank_in_chain"], ascending=[True, True])

    # choose output columns (keep more if present)
    cols = ["chain_id", "rank_in_chain", "source", "path_str", "score",
        "Tc0", "Se1", "X", "Se2", "Tc1"]
    if args.key_with_netR:
        cols += ["netRx", "netRy", "netRz"]

    # optional physics columns if present
    for c in ["t1_absH", "t2_absH", "t3_absH", "t4_absH", "delta1", "delta2", "delta3"]:
        if c in out.columns:
            cols.append(c)

    # also include raw mediator tag (Ir_d / Ge_p) if you want
    if "mediator" in out.columns:

```

```

        cols.insert(cols.index("path_str"), "mediator")

out[cols].to_csv(args.out, index=False)

# brief preview
print("\n=== Per-chain top results (top per chain) ===")
print(f"[INFO] merged_rows={len(df)} chains={best.shape[0]} output_rows={len(out)}")
print(out[cols].head(30).to_string(index=False))
print(f"\n[DONE] wrote: {args.out}\n")

if __name__ == "__main__":
    main()

```

► 代码清单 A.59.11 (python, 86 行)

```

#!/usr/bin/env python3
import argparse
import pandas as pd

def main():
    ap = argparse.ArgumentParser(
        description="Extract Tc-Se, Se-X, X-Se, Se-Tc hoppings from chains_top3.csv and print numeric values."
    )
    ap.add_argument("--in_csv", default="chains_top3.csv", help="Input chains table from 2step grouping")
    ap.add_argument("--out_csv", default="hopping_edges_from_chains.csv", help="Output expanded edge list")
    ap.add_argument("--top_edge_each", type=int, default=10, help="Top-N edges per edge-type by |t| (default 10)")
    args = ap.parse_args()

    df = pd.read_csv(args.in_csv)

    # required columns
    req = ["path_str", "t1_absH", "t2_absH", "t3_absH", "t4_absH"]
    miss = [c for c in req if c not in df.columns]
    if miss:
        raise RuntimeError(f"Missing columns in {args.in_csv}: {miss}. Available: {list(df.columns)}")

    # atom indices if present
    atom_cols = ["Tc0", "Se1", "X", "Se2", "Tc1"]
    has_atoms = all(c in df.columns for c in atom_cols)

    # mediator tag if present
    med_col = "mediator" if "mediator" in df.columns else None

    rows = []
    for _, r in df.iterrows():
        path = r["path_str"]
        source = r["source"] if "source" in df.columns else ""
        chain_id = r["chain_id"] if "chain_id" in df.columns else None
        rank_in_chain = r["rank_in_chain"] if "rank_in_chain" in df.columns else None

        if has_atoms:
            Tc0, Se1, X, Se2, Tc1 = int(r["Tc0"]), int(r["Se1"]), int(r["X"]), int(r["Se2"]), int(r["Tc1"])
        else:
            Tc0 = Se1 = X = Se2 = Tc1 = None

        med = str(r[med_col]) if med_col else ""
        # Expand 4 steps
        rows.append({
            "chain_id": chain_id, "rank_in_chain": rank_in_chain, "source": source, "mediator": med,
            "path_str": path, "edge_type": "Tc-Se", "edge_str": f"Tc{Tc0}-Se{Se1}" if has_atoms else "",
            "absH_eV": float(r["t1_absH"])
        })

```

```

    })
    rows.append({
        "chain_id": chain_id, "rank_in_chain": rank_in_chain, "source": source, "mediator": med,
        "path_str": path, "edge_type": "Se-X", "edge_str": f"Se{Se1}-X{X}" if has_atoms else "",
        "absH_eV": float(r["t2_absH"])
    })
    rows.append({
        "chain_id": chain_id, "rank_in_chain": rank_in_chain, "source": source, "mediator": med,
        "path_str": path, "edge_type": "X-Se", "edge_str": f"X{X}-Se{Se2}" if has_atoms else "",
        "absH_eV": float(r["t3_absH"])
    })
    rows.append({
        "chain_id": chain_id, "rank_in_chain": rank_in_chain, "source": source, "mediator": med,
        "path_str": path, "edge_type": "Se-Tc", "edge_str": f"Se{Se2}-Tc{Tc1}" if has_atoms else "",
        "absH_eV": float(r["t4_absH"])
    })
    })

out = pd.DataFrame(rows)
out.to_csv(args.out_csv, index=False)

# Print per-path detailed values
print("\n=== Per-path hoppings (|t| in eV) ===")
show_cols = ["chain_id", "rank_in_chain", "source", "path_str", "edge_type", "edge_str", "absH_eV"]
if "chain_id" not in out.columns:
    show_cols = ["path_str", "edge_type", "edge_str", "absH_eV"]
print(out[show_cols].to_string(index=False))

# Top edges per type
print(f"\n=== Top {args.top_edge_each} edges per edge_type (by |t|) ===")
for et in ["Tc-Se", "Se-X", "X-Se", "Se-Tc"]:
    sub = out[out["edge_type"] == et].sort_values("absH_eV", ascending=False).head(args.top_edge_each)
    print(f"\n[{et}]")
    print(sub[["path_str", "edge_str", "absH_eV"]].to_string(index=False))

print(f"\n[DONE] wrote: {args.out_csv}\n")

if __name__ == "__main__":
    main()

```

► 代码清单 A.59.13 (python, 121 行)

```

#!/usr/bin/env python3
import argparse
import pandas as pd

def main():
    ap = argparse.ArgumentParser(
        description="Extract Tc-Tc direct-exchange hoppings from edges.csv (Tc_d <-> Tc_d), ranked by |H|."
    )
    ap.add_argument("--edges", default="edges.csv", help="Input edges.csv")
    ap.add_argument("--out", default="TcTc_direct_hoppings_top.csv", help="Output csv")
    ap.add_argument("--topn", type=int, default=50, help="Top-N hoppings to output (default 50)")

    ap.add_argument("--prefer_group", action="store_true",
        help="Use group_m/group_n tags if present (recommended).")
    ap.add_argument("--tc_tag", default="Tc_d",
        help="Tc tag when using group columns (default Tc_d). "
        "If not using group, this becomes element symbol (default Tc).")
    ap.add_argument("--dist_max", type=float, default=0.0,
        help="Optional distance cutoff in Å (0 = no cutoff).")
    ap.add_argument("--min_absH", type=float, default=0.0,

```

```

        help="Optional minimum |H| cutoff in eV (default 0).")

ap.add_argument("--undirected_dedup", action="store_true",
                help="Deduplicate A->B and B->A (keep the larger |H|). Recommended for clean tables.")
args = ap.parse_args()

df = pd.read_csv(args.edges)

# Basic required columns
required = ["absH_eV", "dist_A", "atom_m", "atom_n", "Rx", "Ry", "Rz", "m", "n", "elem_m", "elem_n"]
missing = [c for c in required if c not in df.columns]
if missing:
    raise RuntimeError(f"Missing columns in {args.edges}: {missing}\nAvailable: {list(df.columns)}")

has_group = ("group_m" in df.columns) and ("group_n" in df.columns)
use_group = args.prefer_group and has_group

# Normalize tag in element-mode
if use_group:
    tc_tag = args.tc_tag
    tag_m = df["group_m"].astype(str).str.strip()
    tag_n = df["group_n"].astype(str).str.strip()
else:
    tc_tag = args.tc_tag.split("_")[0] # Tc_d -> Tc
    tag_m = df["elem_m"].astype(str).str.strip()
    tag_n = df["elem_n"].astype(str).str.strip()

# Filter Tc-Tc (direct exchange channel)
sub = df[(tag_m == tc_tag) & (tag_n == tc_tag)].copy()

# Cutoffs
sub["absH_eV"] = pd.to_numeric(sub["absH_eV"], errors="coerce")
sub["dist_A"] = pd.to_numeric(sub["dist_A"], errors="coerce")
sub = sub.dropna(subset=["absH_eV", "dist_A"])

if args.min_absH > 0:
    sub = sub[sub["absH_eV"] >= args.min_absH]
if args.dist_max and args.dist_max > 0:
    sub = sub[sub["dist_A"] <= args.dist_max]

if sub.empty:
    raise RuntimeError(
        f"No Tc-Tc edges found. Check --prefer_group and --tc_tag.\n"
        f"use_group={use_group}, tc_tag={tc_tag}, "
        f"rows={len(df)}, columns={list(df.columns)}"
    )

# Optional: keep only one of A->B and B->A
if args.undirected_dedup:
    # Create undirected key: (min(atom), max(atom), Rx,Ry,Rz, min(wf), max(wf)) is too strict.
    # For a clean "direct exchange" table, usually dedup by (min(atom), max(atom), Rx,Ry,Rz) is enough.
    a = sub["atom_m"].astype(int)
    b = sub["atom_n"].astype(int)
    sub["_a"] = a.where(a <= b, b)
    sub["_b"] = b.where(a <= b, a)
    sub["_key"] = (
        sub["_a"].astype(str) + "-" + sub["_b"].astype(str) + "|R=" +
        sub["Rx"].astype(int).astype(str) + "," +
        sub["Ry"].astype(int).astype(str) + "," +
        sub["Rz"].astype(int).astype(str)
    )
    # Keep max |H| per key
    sub = sub.sort_values("absH_eV", ascending=False).groupby("_key", as_index=False).head(1)

# Sort by |H| desc and take topn
sub = sub.sort_values("absH_eV", ascending=False).head(args.topn)

# Build a nice edge label
sub["edge_str"] = "Tc" + sub["atom_m"].astype(int).astype(str) + "-Tc" + sub["atom_n"].astype(int).astype(str)

```

```

# Include re/im if present
cols = [
    "edge_str",
    "absH_eV",
    "dist_A",
    "atom_m", "atom_n",
    "m", "n",
    "Rx", "Ry", "Rz",
]
if "reH_eV" in sub.columns:
    cols.insert(2, "reH_eV")
if "imH_eV" in sub.columns:
    cols.insert(3 if "reH_eV" in sub.columns else 2, "imH_eV")
# Also keep shell/group/pair if present
for c in ["shell", "pair", "group_m", "group_n", "elem_m", "elem_n"]:
    if c in sub.columns and c not in cols:
        cols.append(c)

out = sub[cols].copy()
out.to_csv(args.out, index=False)

# Print preview
print(f"\n[INFO] use_group={use_group}, tc_tag={tc_tag}")
print(f"[INFO] Tc-Tc edges found: {len(df[(tag_m==tc_tag) & (tag_n==tc_tag)])}")
print(f"[INFO] output topn={len(out)} -> {args.out}\n")
print(out.head(min(20, len(out))).to_string(index=False))
print()

if __name__ == "__main__":
    main()

```

(原创自研) 利用 PDOS 验证超超交换路径

► 代码清单 A.60.1 (python, 297 行)

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
1step.py (VB + CB grouping + level diagram)

```

功能:

- 读取 UP/DW 两个 PDOS 文件 (能量网格必须一致)
- 对 d 轨道 (dxy, dyz, dz2, dxz, dx2y2) 在两个能窗内分别:
 - 1) 计算余弦相似度矩阵 sim_matrix
 - 2) 连通分量分组 auto_groups (阈值 thr)
 - 3) 计算每轨道 band center E_center (谱权重质心) 与 N_window
 - 4) 画能级示意图 (按 E_center 排序, 标注简并组)

输出 (prefix = --out_prefix):

- <prefix>_VB_sim_matrix.csv
- <prefix>_VB_orb_centers.csv
- <prefix>_VB_auto_groups.csv
- <prefix>_VB_levels.png
- <prefix>_CB_sim_matrix.csv
- <prefix>_CB_orb_centers.csv
- <prefix>_CB_auto_groups.csv
- <prefix>_CB_levels.png

说明:

- Ef 位置由 --Ef 指定 (默认 0.0), 脚本不从 OUTCAR/DOSCAR 读取。
 - 这里的“能级/分组”是基于指定能窗内 PDOS 形状相似性 + band center (质心), 用于机制图示意, 不等同严格晶场本征值。
 """

```
import argparse
from pathlib import Path
import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# ===== PDOS 文件列布局 (与现有脚本一致) =====
COLS = ["E", "s", "py", "pz", "px", "dxy", "dyz", "dz2", "dxz", "dx2y2", "tot"]
D_ORBS = ["dxy", "dyz", "dz2", "dxz", "dx2y2"]

def load_pdos(path: Path) -> pd.DataFrame:
    return pd.read_csv(path, sep=r"\s+", comment="#", names=COLS, skiprows=1)

def trapz(E: np.ndarray, Y: np.ndarray) -> float:
    return float(np.trapz(Y, E))

def band_center(E: np.ndarray, D: np.ndarray, Emin: float, Emax: float):
    m = (E >= Emin) & (E <= Emax)
    Ew = E[m]
    Dw = D[m]
    denom = trapz(Ew, Dw)
    if abs(denom) < 1e-14:
        return np.nan, 0.0
    num = trapz(Ew, Ew * Dw)
    return num / denom, denom

def cosine_sim(E: np.ndarray, A: np.ndarray, B: np.ndarray, Emin: float, Emax: float):
    m = (E >= Emin) & (E <= Emax)
    Em = E[m]
    a = A[m]
    b = B[m]
    na = np.sqrt(trapz(Em, a * a))
    nb = np.sqrt(trapz(Em, b * b))
    if na < 1e-14 or nb < 1e-14:
        return np.nan
    return trapz(Em, a * b) / (na * nb)

def auto_groups(sim_mat: np.ndarray, labels, thr: float):
    """
    Build graph with edge(u,v) if sim>=thr, return connected components.
    """
    n = len(labels)
    visited = [False] * n
    groups = []
    for i in range(n):
        if visited[i]:
            continue
        stack = [i]
        comp = []
        visited[i] = True
        while stack:
            u = stack.pop()
            comp.append(u)
            for v in range(n):
                if (not visited[v]) and (sim_mat[u, v] >= thr):
                    visited[v] = True
                    stack.append(v)
        groups.append([labels[k] for k in sorted(comp)])
```

```

return groups

def compute_window(E: np.ndarray, D: np.ndarray, Emin: float, Emax: float, thr: float, tag: str):
    """
    返回:
        df_sim, df_orb, df_grp, groups(list[list[str]])
    """
    n = len(D_ORBS)
    sim = np.zeros((n, n), float)
    for i in range(n):
        for j in range(n):
            if i == j:
                sim[i, j] = 1.0
            elif j < i:
                sim[i, j] = sim[j, i]
            else:
                sim[i, j] = cosine_sim(E, D[:, i], D[:, j], Emin, Emax)

    df_sim = pd.DataFrame(sim, index=D_ORBS, columns=D_ORBS)
    groups = auto_groups(sim, D_ORBS, thr=thr)

    # per-orb centers
    orb_rows = []
    for i, orb in enumerate(D_ORBS):
        Ec, Nw = band_center(E, D[:, i], Emin, Emax)
        orb_rows.append({
            "orb": orb,
            "E_center": Ec,
            "N_window": Nw,
            "window": f"[{Emin},{Emax}]",
            "tag": tag
        })
    df_orb = pd.DataFrame(orb_rows).sort_values("E_center")

    # per-group centers (sum DOS)
    grp_rows = []
    for g in groups:
        idx = [D_ORBS.index(o) for o in g]
        Dg = D[:, idx].sum(axis=1)
        Ec, Nw = band_center(E, Dg, Emin, Emax)
        grp_rows.append({
            "group": "(" + ",".join(g) + ")",
            "members": ",".join(g),
            "size": len(g),
            "E_center": Ec,
            "N_window": Nw,
            "window": f"[{Emin},{Emax}]",
            "tag": tag
        })
    df_grp = pd.DataFrame(grp_rows).sort_values("E_center")

    return df_sim, df_orb, df_grp, groups

def parse_orb_list(s):
    tokens = re.findall(r"(dxy|dxz|dyz|dz2|dx2y2)", str(s), flags=re.IGNORECASE)
    return [t.lower() for t in tokens]

def draw_level_diagram(df_grp: pd.DataFrame, out_png: str, title: str, eref: float = 0.0,
                       fontsize: int = 12, linewidth: float = 3.0):
    """
    用 df_grp (每组 E_center) 画能级示意图。
    """
    # 过滤无效
    df = df_grp.copy()
    df = df[np.isfinite(df["E_center"].values)]
    if len(df) == 0:

```

```

        raise RuntimeError("No valid E_center to plot.")

# 按能量排序
df = df.sort_values("E_center").reset_index(drop=True)

ys = (df["E_center"].to_numpy(float) - eref)
labels = []
sizes = []
for _, r in df.iterrows():
    orbs = parse_orb_list(r["members"])
    label = " + ".join(orbs) if orbs else str(r["group"])
    labels.append(label)
    sizes.append(int(r["size"]))

n = len(df)
fig_h = max(4.0, 1.2 + 0.8 * n)
fig, ax = plt.subplots(figsize=(6.4, fig_h))

x0 = 0.0
xspan = 0.55
x_offsets = np.linspace(-0.15, 0.15, n) if n > 1 else np.array([0.0])

for i in range(n):
    y = ys[i]
    x0 = x_offsets[i]
    ax.hlines(y, x0 - xspan + x0, x0 + xspan + x0, linewidth=linewidth)
    ax.text(x0 + xspan + 0.10, y, f"{labels[i]} ({sizes[i]})", va="center", fontsize=fontsize)
    ax.text(x0 - xspan - 0.10, y, f"{y:.3f} eV", va="center", ha="right", fontsize=fontsize-1)

ax.set_xlim(-1.2, 1.9)
ax.set_xticks([])
ax.set_ylabel(f"Energy (eV) [E_center - {eref:g}]", fontsize=fontsize)
ax.grid(True, axis="y", linestyle="--", linewidth=0.6, alpha=0.5)
ax.set_title(title, fontsize=fontsize + 2)

ypad = 0.15 * (float(np.nanmax(ys)) - float(np.nanmin(ys)) + 1e-9)
ax.set_ylim(float(np.nanmin(ys) - ypad), float(np.nanmax(ys) + ypad))

fig.tight_layout()
fig.savefig(out_png, dpi=300)
plt.close(fig)

def main():
    ap = argparse.ArgumentParser()

    ap.add_argument("--up", required=True, help="spin-up PDOS file")
    ap.add_argument("--dw", required=True, help="spin-down PDOS file")

    ap.add_argument("--Ef", type=float, default=0.0,
                    help="Fermi level on PDOS energy axis (used for defining VB/CB windows around it). default=0")

    # windows defined relative to Ef
    ap.add_argument("--vb_width", type=float, default=1.0,
                    help="VB window width below Ef: [Ef-vb_width, Ef]")
    ap.add_argument("--cb_width", type=float, default=1.0,
                    help="CB window width above Ef: [Ef, Ef+cb_width]")

    ap.add_argument("--thr", type=float, default=0.99, help="cosine similarity threshold for grouping")
    ap.add_argument("--out_prefix", default="Auto", help="output prefix, e.g., TcAuto")

    ap.add_argument("--no_plot", action="store_true", help="do not output png level diagrams")

    args = ap.parse_args()

    up = load_pdos(Path(args.up))
    dw = load_pdos(Path(args.dw))

    if not np.allclose(up["E"].values, dw["E"].values, atol=1e-10):

```

```

        raise RuntimeError("Energy grids differ between UP/DW.")

E = up["E"].to_numpy(float)

Dup = up[D_ORBS].to_numpy(float)
# 常见: DW 可能为负, 取 abs 保持与原有逻辑一致
Ddw = np.abs(dw[D_ORBS].to_numpy(float))
D = Dup + Ddw

Ef = args.Ef
vb_Emin, vb_Emax = Ef - args.vb_width, Ef
cb_Emin, cb_Emax = Ef, Ef + args.cb_width

# ===== VB =====
vb_sim, vb_orb, vb_grp, vb_groups = compute_window(
    E, D, vb_Emin, vb_Emax, thr=args.thr, tag="VB"
)
vb_sim.to_csv(f"{args.out_prefix}_VB_sim_matrix.csv")
vb_orb.to_csv(f"{args.out_prefix}_VB_orb_centers.csv", index=False)
vb_grp.to_csv(f"{args.out_prefix}_VB_auto_groups.csv", index=False)

# ===== CB =====
cb_sim, cb_orb, cb_grp, cb_groups = compute_window(
    E, D, cb_Emin, cb_Emax, thr=args.thr, tag="CB"
)
cb_sim.to_csv(f"{args.out_prefix}_CB_sim_matrix.csv")
cb_orb.to_csv(f"{args.out_prefix}_CB_orb_centers.csv", index=False)
cb_grp.to_csv(f"{args.out_prefix}_CB_auto_groups.csv", index=False)

# ===== plots =====
if not args.no_plot:
    draw_level_diagram(
        vb_grp,
        out_png=f"{args.out_prefix}_VB_levels.png",
        title=f"{args.out_prefix}: d-level groups in VB window [{vb_Emin:.2f},{vb_Emax:.2f}] (Ef={Ef:g})",
        eref=0.0
    )
    draw_level_diagram(
        cb_grp,
        out_png=f"{args.out_prefix}_CB_levels.png",
        title=f"{args.out_prefix}: d-level groups in CB window [{cb_Emin:.2f},{cb_Emax:.2f}] (Ef={Ef:g})",
        eref=0.0
    )

print(f"[Ef] using Ef = {Ef} eV on the PDOS energy axis.")
print("\n[VB window]", (vb_Emin, vb_Emax), "thr=", args.thr, "groups=", vb_groups)
print(vb_grp.to_string(index=False))

print("\n[CB window]", (cb_Emin, cb_Emax), "thr=", args.thr, "groups=", cb_groups)
print(cb_grp.to_string(index=False))

print("\n[OK] Outputs:")
print(" ", f"{args.out_prefix}_VB_sim_matrix.csv")
print(" ", f"{args.out_prefix}_VB_orb_centers.csv")
print(" ", f"{args.out_prefix}_VB_auto_groups.csv")
if not args.no_plot:
    print(" ", f"{args.out_prefix}_VB_levels.png")
print(" ", f"{args.out_prefix}_CB_sim_matrix.csv")
print(" ", f"{args.out_prefix}_CB_orb_centers.csv")
print(" ", f"{args.out_prefix}_CB_auto_groups.csv")
if not args.no_plot:
    print(" ", f"{args.out_prefix}_CB_levels.png")

if __name__ == "__main__":
    main()

```

(原创自研) 利用 wannier 验证超超交换路径 (4)

► 代码清单 A.61.2 (python, 350 行)

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import argparse
import csv
from pathlib import Path
import numpy as np
import pandas as pd

# =====
# helpers: parse hr.dat
# =====
def parse_hr_dat(hr_path: Path):
    lines = hr_path.read_text(encoding="utf-8", errors="ignore").splitlines()
    if len(lines) < 4:
        raise RuntimeError("hr.dat too short")
    num_wann = int(lines[1].strip())
    nrpts = int(lines[2].strip())

    degen = []
    idx = 3
    while len(degen) < nrpts:
        degen.extend([int(t) for t in lines[idx].split()])
        idx += 1
    rec_lines = lines[idx:]
    return num_wann, rec_lines

def build_H0(num_wann, rec_lines):
    H0 = np.zeros((num_wann, num_wann), dtype=np.complex128)
    for ln in rec_lines:
        if not ln.strip():
            continue
        toks = ln.split()
        if len(toks) < 7:
            continue
        Rx, Ry, Rz = int(toks[0]), int(toks[1]), int(toks[2])
        if (Rx, Ry, Rz) != (0, 0, 0):
            continue
        m = int(toks[3]) - 1
        n = int(toks[4]) - 1
        re0 = float(toks[5])
        im0 = float(toks[6])
        H0[m, n] = re0 + 1j * im0
    return 0.5 * (H0 + H0.conj().T)

# =====
# helpers: parse edges.csv -> WF -> (atom_id, elem)
# =====
def read_edges_wf_map(edges_path: Path):
    df = pd.read_csv(edges_path)
    needed = ["m", "n", "atom_m", "atom_n", "elem_m", "elem_n"]
    for c in needed:
        if c not in df.columns:
            raise RuntimeError(f"edges.csv missing required column: {c}")

    wf_to_pairs = {}
    for _, r in df.iterrows():
        m = int(r["m"]); n = int(r["n"])
```

```

        am = int(r["atom_m"]); an = int(r["atom_n"])
        em = str(r["elem_m"]); en = str(r["elem_n"])
        wf_to_pairs.setdefault(m, set()).add((am, em))
        wf_to_pairs.setdefault(n, set()).add((an, en))

    bad = []
    wf_map = {}
    for wf, s in wf_to_pairs.items():
        if len(s) != 1:
            bad.append((wf, sorted(list(s))))
        else:
            wf_map[wf] = next(iter(s))

    if bad:
        msg = ["[ERROR] Some WFs map to multiple (atom_id, elem) labels. Fix edges.csv labeling first."]
        for wf, pairs in bad[:50]:
            msg.append(f"  wf={wf}: {pairs}")
        raise RuntimeError("\n".join(msg))

    return wf_map

def atoms_by_element(wf_map, num_wann):
    elem_to_atoms = {}
    for wf in range(1, num_wann + 1):
        aid, elem = wf_map[wf]
        elem_to_atoms.setdefault(elem, set()).add(aid)
    return {e: sorted(list(s)) for e, s in elem_to_atoms.items()}

# =====
# helpers: parse win projections (explicit order)
# =====
def norm_orb(tok: str) -> str:
    t = tok.strip().lower()
    t = t.replace("dx2y2", "dx2-y2").replace("dx2_y2", "dx2-y2")
    t = t.replace("d(z2)", "dz2").replace("d(z^2)", "dz2").replace("d(z**2)", "dz2")
    t = t.replace("d(x2-y2)", "dx2-y2")
    return t

def expand_token(tok: str):
    t = norm_orb(tok)
    if t == "d":
        return ["dxy", "dyz", "dxz", "dz2", "dx2-y2"]
    if t == "p":
        return ["px", "py", "pz"]
    if t == "s":
        return ["s"]
    return [t]

def parse_win_projections(win_path: Path):
    lines = win_path.read_text(encoding="utf-8", errors="ignore").splitlines()
    in_proj = False
    proj_lines = []
    for ln in lines:
        s = ln.strip()
        low = s.lower()
        if low.startswith("begin projections"):
            in_proj = True
            continue
        if low.startswith("end projections"):
            in_proj = False
            continue
        if not in_proj:
            continue
        if (not s) or s.startswith("#") or s.startswith("!"):
            continue

```

```

        proj_lines.append(s)

out = []
for raw in proj_lines:
    if ":" not in raw:
        continue
    elem, rhs = raw.split(":", 1)
    elem = elem.strip()
    rhs = rhs.replace(",", " ").replace(";", " ")
    toks = [t for t in rhs.split() if t.strip()]
    orbs = []
    for t in toks:
        orbs += expand_token(t)
    out.append((elem, orbs))

if not out:
    raise RuntimeError("[ERROR] No projections found in win. Check begin/end projections block.")
return out

def build_wf_labels_from_win(proj_spec, elem_to_atoms, num_wann):
    """
    Construct expected WF label sequence in the SAME ordering as Wannier90 projections expansion.
    We assume SOC spinor duplication if num_wann == 2 * spatial_count.
    Return: list labels (1..num_wann): (atom_id, elem, orb)
    """
    spatial = []
    for elem, orbs in proj_spec:
        if elem not in elem_to_atoms:
            raise RuntimeError(f"[ERROR] win projections element '{elem}' not in edges elements
                               ↳ {list(elem_to_atoms.keys())}")
        for aid in elem_to_atoms[elem]:
            for orb in orbs:
                spatial.append((aid, elem, orb))

    if len(spatial) == num_wann:
        return spatial, False
    if 2 * len(spatial) == num_wann:
        spinor = []
        for item in spatial:
            spinor.append(item)
            spinor.append(item)
        return spinor, True

    raise RuntimeError(
        "[ERROR] projection expansion count mismatch.\n"
        f"  spatial_count={len(spatial)}, num_wann={num_wann}\n"
        "Expected num_wann == spatial_count (no spinor) OR num_wann == 2*spatial_count (SOC spinor).\n"
    )

# =====
# local diag + orbital labeling from basis
# =====
def diag_local(H0, wf_list_lbased):
    idx = [w - 1 for w in wf_list_lbased]
    sub = H0[np.ix_(idx, idx)]
    evals, evecs = np.linalg.eigh(sub)
    order = np.argsort(np.real(evals))
    return np.real(evals[order]), evecs[:, order]

def try_pairs(evals_rel):
    if len(evals_rel) % 2 != 0:
        return []
    out = []
    for i in range(0, len(evals_rel), 2):
        e1 = float(evals_rel[i]); e2 = float(evals_rel[i + 1])
        out.append((0.5 * (e1 + e2), abs(e2 - e1), i, i + 1))

```

```

return out

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--hr", default="wannier90_hr.dat")
    ap.add_argument("--edges", default="edges.csv")
    ap.add_argument("--win", default="wannier90.win")
    ap.add_argument("--Ef", type=float, default=0.0, help="output energies as E_rel = E - Ef (eV)")
    ap.add_argument("--out_prefix", default="cfFINAL")
    ap.add_argument("--pair_tol", type=float, default=0.02, help="warn if pair split > tol (eV)")
    args = ap.parse_args()

    hr_path = Path(args.hr)
    edges_path = Path(args.edges)
    win_path = Path(args.win)
    for p in [hr_path, edges_path, win_path]:
        if not p.exists():
            raise FileNotFoundError(p.resolve())

    num_wann, rec_lines = parse_hr_dat(hr_path)
    H0 = build_H0(num_wann, rec_lines)

    wf_map = read_edges_wf_map(edges_path)
    elem_to_atoms = atoms_by_element(wf_map, num_wann)

    proj_spec = parse_win_projections(win_path)
    expected_labels, spinor_dup = build_wf_labels_from_win(proj_spec, elem_to_atoms, num_wann)

    # Build WF groups per atom from edges (ground truth for which WF belongs to which atom)
    atom_to_wfs = {}
    atom_to_elem = {}
    for wf in range(1, num_wann + 1):
        aid, elem = wf_map[wf]
        atom_to_wfs.setdefault(aid, []).append(wf)
        atom_to_elem[aid] = elem
    for aid in atom_to_wfs:
        atom_to_wfs[aid] = sorted(atom_to_wfs[aid])

    # Now: assign each WF an orbital label by matching its position in the expected sequence.
    # This requires that the WF ordering produced by Wannier90 follows the projection expansion ordering.
    # We'll build wf_orb[wf] = orb_label.
    wf_orb = {}
    wf_expected_atom = {}
    for wf in range(1, num_wann + 1):
        aid_e, elem_e, orb_e = expected_labels[wf - 1]
        wf_expected_atom[wf] = (aid_e, elem_e)
        wf_orb[wf] = orb_e

    # Consistency check: WF->atom from edges should match expected atom from win ordering
    mism = []
    for wf in range(1, num_wann + 1):
        aid_true, elem_true = wf_map[wf]
        aid_e, elem_e = wf_expected_atom[wf]
        if (aid_true != aid_e) or (elem_true != elem_e):
            mism.append((wf, (aid_true, elem_true), (aid_e, elem_e)))
    if mism:
        # We do not abort; but we warn loudly because orbital labeling would be unreliable.
        print("[WARN] WF ordering does NOT match win projection expansion ordering for these WFs (first 20  

        ↪ shown):")
        for it in mism[:20]:
            print(f"  wf={it[0]} edges={it[1]} expected_from_win={it[2]}")
        print("[WARN] In this case, you must reorder WFs by parsing wout spread table or use a stricter mapping  

        ↪ method.")
        # If this happens, stop now to avoid producing wrong orbital order.
        raise RuntimeError("WF ordering mismatch: cannot safely label orbitals from win ordering.")

    # Outputs

```

```

out_levels = f"{args.out_prefix}_levels.csv"
out_order = f"{args.out_prefix}_orbital_order.csv"
out_pairs = f"{args.out_prefix}_pair_centers.csv"

rows_levels = []
rows_pairs = []
lowest = {} # (aid, elem, orb) -> lowest E_rel

for aid in sorted(atom_to_wfs.keys()):
    elem = atom_to_elem[aid]
    wfs = atom_to_wfs[aid]
    evals, evecs = diag_local(H0, wfs)
    evals_rel = evals - args.Ef

    # basis orbital label list for this atom (same length as wfs)
    basis_orbs = [wf_orb[wf] for wf in wfs]

    for j in range(len(evals_rel)):
        c2 = np.abs(evecs[:, j]) ** 2
        # orbital weight = sum |c_i|^2 over basis functions with same orb label
        orb_w = {}
        for i, orb in enumerate(basis_orbs):
            orb_w[orb] = orb_w.get(orb, 0.0) + float(c2[i])
        dom_orb = max(orb_w.keys(), key=lambda k: orb_w[k])
        dom_w = orb_w[dom_orb]

        rows_levels.append({
            "atom_id": aid,
            "elem": elem,
            "n_wf": len(wfs),
            "level_index": j + 1,
            "E_rel": float(evals_rel[j]),
            "dominant_orb": dom_orb,
            "dominant_w": float(dom_w)
        })

    key = (aid, elem, dom_orb)
    lowest[key] = min(lowest.get(key, 1e30), float(evals_rel[j]))

# pair diagnostics
pairs = try_pairs(evals_rel)
for pi, (center, split, i1, i2) in enumerate(pairs, start=1):
    warn = int(split > args.pair_tol)
    # label by combined weights
    c2 = (np.abs(evecs[:, i1]) ** 2 + np.abs(evecs[:, i2]) ** 2)
    orb_w = {}
    for i, orb in enumerate(basis_orbs):
        orb_w[orb] = orb_w.get(orb, 0.0) + float(c2[i])
    dom_orb = max(orb_w.keys(), key=lambda k: orb_w[k])
    dom_w = orb_w[dom_orb]

    rows_pairs.append({
        "atom_id": aid,
        "elem": elem,
        "n_wf": len(wfs),
        "pair_index": pi,
        "center_rel": float(center),
        "split": float(split),
        "dominant_orb_pair": dom_orb,
        "dominant_w_pair": float(dom_w),
        "warn_split_gt_tol": warn
    })

pd.DataFrame(rows_levels).to_csv(out_levels, index=False)
pd.DataFrame(rows_pairs).to_csv(out_pairs, index=False)

out_rows = []
for (aid, elem, orb), E0 in sorted(lowest.items(), key=lambda x: (x[0][1], x[0][0], x[1], x[0][2]]):
    out_rows.append({

```

```

        "atom_id": aid, "elem": elem, "orbital": orb,
        "lowest_E_rel": float(E0)
    })
pd.DataFrame(out_rows).to_csv(out_order, index=False)

print("[OK] Done.")
print(f"  spinor_dup = {spinor_dup}")
print(f"  wrote: {out_levels}")
print(f"  wrote: {out_pairs}")
print(f"  wrote: {out_order}")
print("Notes:")
print("  - This route labels orbitals from win projection expansion order; it is stable and does NOT depend on
↪ chk/amn.")
print("  - If you are FM+SOC (TR broken), pair_centers are diagnostic only; do not treat them as Kramers by
↪ default.")

if __name__ == "__main__":
    main()

```

► 代码清单 A.61.4 (python, 95 行)

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import argparse
from pathlib import Path
import pandas as pd
import numpy as np

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--levels", default="cfA2_levels.csv", help="*_levels.csv from the CF script")
    ap.add_argument("--out_prefix", default="cf_elem", help="output prefix")
    ap.add_argument("--min_dom_w", type=float, default=0.20,
                    help="discard levels whose dominant_w < this threshold (label too mixed)")
    args = ap.parse_args()

    levels_path = Path(args.levels)
    if not levels_path.exists():
        raise FileNotFoundError(levels_path.resolve())

    df = pd.read_csv(levels_path)

    need = {"atom_id", "elem", "n_wf", "level_index", "E_rel", "dominant_orb", "dominant_w"}
    miss = need - set(df.columns)
    if miss:
        raise RuntimeError(f"levels file missing columns: {sorted(list(miss))}")

    # clean
    df["dominant_orb"] = df["dominant_orb"].astype(str).str.strip()
    df = df[df["dominant_orb"] != ""].copy()

    # filter mixed labels
    df_f = df[df["dominant_w"] >= args.min_dom_w].copy()

    # long table output
    long_out = f"{args.out_prefix}_elem_orb_levels_long.csv"
    df_f.to_csv(long_out, index=False)

    # base stats
    stats = (

```

```

df_f.groupby(["elem", "dominant_orb"])["E_rel"]
    .agg(count="count", mean="mean", std="std", min="min", max="max")
    .reset_index()
)

# quantiles (robust way)
qs = (
    df_f.groupby(["elem", "dominant_orb"])["E_rel"]
        .quantile([0.25, 0.50, 0.75])
        .unstack(level=-1)
        .reset_index()
        .rename(columns={0.25: "q25", 0.50: "q50", 0.75: "q75"})
)

stats = stats.merge(qs, on=["elem", "dominant_orb"], how="left")

# per-element splitting summary (range of orbital medians)
med = stats.pivot_table(index="elem", columns="dominant_orb", values="q50", aggfunc="first")

elem_split = []
for elem in med.index:
    vals = med.loc[elem].dropna().to_numpy()
    if len(vals) == 0:
        continue
    elem_split.append({
        "elem": elem,
        "orbital_median_range": float(vals.max() - vals.min()),
        "orbital_median_min": float(vals.min()),
        "orbital_median_max": float(vals.max()),
        "n_orbitals": int(len(vals))
    })
elem_split_df = pd.DataFrame(elem_split).sort_values(["elem"])

# write outputs
stats_out = f"{args.out_prefix}_elem_orb_stats.csv"
split_out = f"{args.out_prefix}_elem_splitting_summary.csv"

stats.sort_values(["elem", "dominant_orb"]).to_csv(stats_out, index=False)
elem_split_df.to_csv(split_out, index=False)

print("[OK] wrote:")
print(" ", long_out)
print(" ", stats_out)
print(" ", split_out)
print(f"[INFO] kept {len(df_f)} / {len(df)} levels (min_dom_w={args.min_dom_w})")

if len(elem_split_df) > 0:
    print("\n[Summary] per-element orbital median range (q50 max-min):")
    for _, r in elem_split_df.iterrows():
        print(f"  {r['elem']}: range={r['orbital_median_range']:.4f} eV over {r['n_orbitals']} orbitals")

if __name__ == "__main__":
    main()

```

► 代码清单 A.61.6 (python, 269 行)

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import argparse
from pathlib import Path
import numpy as np

```

```

import pandas as pd

def parse_hr_dat(hr_path: Path):
    lines = hr_path.read_text(encoding="utf-8", errors="ignore").splitlines()
    num_wann = int(lines[1].strip())
    nrpts = int(lines[2].strip())
    degen = []
    idx = 3
    while len(degen) < nrpts:
        degen.extend([int(t) for t in lines[idx].split()])
        idx += 1
    rec_lines = lines[idx:]
    return num_wann, rec_lines

def build_H0(num_wann, rec_lines):
    H0 = np.zeros((num_wann, num_wann), dtype=np.complex128)
    for ln in rec_lines:
        if not ln.strip():
            continue
        toks = ln.split()
        if len(toks) < 7:
            continue
        Rx, Ry, Rz = int(toks[0]), int(toks[1]), int(toks[2])
        if (Rx, Ry, Rz) != (0, 0, 0):
            continue
        m = int(toks[3]) - 1
        n = int(toks[4]) - 1
        H0[m, n] = float(toks[5]) + 1j * float(toks[6])
    return 0.5 * (H0 + H0.conj().T)

def read_edges_wf_map(edges_path: Path):
    df = pd.read_csv(edges_path)
    needed = ["m", "n", "atom_m", "atom_n", "elem_m", "elem_n"]
    for c in needed:
        if c not in df.columns:
            raise RuntimeError(f"edges.csv missing required column: {c}")

    wf_to_pairs = {}
    for _, r in df.iterrows():
        m = int(r["m"]); n = int(r["n"])
        am = int(r["atom_m"]); an = int(r["atom_n"])
        em = str(r["elem_m"]); en = str(r["elem_n"])
        wf_to_pairs.setdefault(m, set()).add((am, em))
        wf_to_pairs.setdefault(n, set()).add((an, en))

    bad = []
    wf_map = {}
    for wf, s in wf_to_pairs.items():
        if len(s) != 1:
            bad.append((wf, sorted(list(s))))
        else:
            wf_map[wf] = next(iter(s))

    if bad:
        msg = ["[ERROR] WF->(atom,elem) inconsistent in edges.csv"]
        for wf, pairs in bad[:30]:
            msg.append(f"  wf={wf}: {pairs}")
        raise RuntimeError("\n".join(msg))
    return wf_map

def atoms_by_element(wf_map, num_wann):
    elem_to_atoms = {}
    for wf in range(1, num_wann + 1):
        aid, elem = wf_map[wf]
        elem_to_atoms.setdefault(elem, set()).add(aid)
    return {e: sorted(list(s)) for e, s in elem_to_atoms.items()}

```

```

def norm_orb(tok: str) -> str:
    t = tok.strip().lower()
    t = t.replace("dx2y2", "dx2-y2").replace("dx2_y2", "dx2-y2")
    t = t.replace("d(z2)", "dz2").replace("d(z^2)", "dz2").replace("d(z**2)", "dz2")
    t = t.replace("d(x2-y2)", "dx2-y2")
    return t

def expand_token(tok: str):
    t = norm_orb(tok)
    if t == "d":
        return ["dxy", "dyz", "dxz", "dz2", "dx2-y2"]
    if t == "p":
        return ["px", "py", "pz"]
    if t == "s":
        return ["s"]
    return [t]

def parse_win_projections(win_path: Path):
    lines = win_path.read_text(encoding="utf-8", errors="ignore").splitlines()
    in_proj = False
    proj_lines = []
    for ln in lines:
        s = ln.strip()
        low = s.lower()
        if low.startswith("begin projections"):
            in_proj = True
            continue
        if low.startswith("end projections"):
            in_proj = False
            continue
        if not in_proj:
            continue
        if (not s) or s.startswith("#") or s.startswith("!"):
            continue
        proj_lines.append(s)

    out = []
    for raw in proj_lines:
        if ":" not in raw:
            continue
        elem, rhs = raw.split(":", 1)
        elem = elem.strip()
        rhs = rhs.replace(",", " ").replace(";", " ")
        toks = [t for t in rhs.split() if t.strip()]
        orbs = []
        for t in toks:
            orbs += expand_token(t)
        out.append((elem, orbs))
    if not out:
        raise RuntimeError("[ERROR] No projections in win.")
    return out

def build_expected_labels(proj_spec, elem_to_atoms, num_wann):
    spatial = []
    for elem, orbs in proj_spec:
        if elem not in elem_to_atoms:
            raise RuntimeError(f"[ERROR] elem {elem} in win not found in edges-derived elements
                               ↳ {list(elem_to_atoms.keys())}")
        for aid in elem_to_atoms[elem]:
            for orb in orbs:
                spatial.append((aid, elem, orb))
    if len(spatial) == num_wann:
        return spatial, False
    if 2 * len(spatial) == num_wann:

```

```

        spinor = []
        for x in spatial:
            spinor.append(x); spinor.append(x)
        return spinor, True
    raise RuntimeError(f"[ERROR] projection count mismatch: spatial={len(spatial)} num_wann={num_wann}")

def diag_local(H0, wf_list_lbased):
    idx = [w - 1 for w in wf_list_lbased]
    sub = H0[np.ix_(idx, idx)]
    evals, evecs = np.linalg.eigh(sub)
    order = np.argsort(np.real(evals))
    return np.real(evals[order]), evecs[:, order]

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--hr", default="wannier90_hr.dat")
    ap.add_argument("--edges", default="edges.csv")
    ap.add_argument("--win", default="wannier90.win")
    ap.add_argument("--Ef", type=float, default=0.0)
    ap.add_argument("--out_prefix", default="cfFW")
    args = ap.parse_args()

    hr_path = Path(args.hr)
    edges_path = Path(args.edges)
    win_path = Path(args.win)
    for p in [hr_path, edges_path, win_path]:
        if not p.exists():
            raise FileNotFoundError(p.resolve())

    num_wann, rec_lines = parse_hr_dat(hr_path)
    H0 = build_H0(num_wann, rec_lines)

    wf_map = read_edges_wf_map(edges_path)
    elem_to_atoms = atoms_by_element(wf_map, num_wann)

    proj_spec = parse_win_projections(win_path)
    expected_labels, spinor_dup = build_expected_labels(proj_spec, elem_to_atoms, num_wann)

    # verify WF ordering matches expected atom labels
    mism = []
    for wf in range(1, num_wann + 1):
        aid_true, elem_true = wf_map[wf]
        aid_e, elem_e, _ = expected_labels[wf - 1]
        if (aid_true != aid_e) or (elem_true != elem_e):
            mism.append((wf, (aid_true, elem_true), (aid_e, elem_e)))
    if mism:
        print("[WARN] WF ordering mismatch with win expansion ordering; cannot label safely.")
        for it in mism[:20]:
            print("  wf", it[0], "edges", it[1], "expected", it[2])
        raise RuntimeError("WF ordering mismatch; stop.")

    # WF -> orbital label (basis orbital)
    wf_orb = {wf: expected_labels[wf - 1][2] for wf in range(1, num_wann + 1)}

    # group WFs by atom
    atom_to_wfs = {}
    atom_to_elem = {}
    for wf in range(1, num_wann + 1):
        aid, elem = wf_map[wf]
        atom_to_wfs.setdefault(aid, []).append(wf)
        atom_to_elem[aid] = elem
    for aid in atom_to_wfs:
        atom_to_wfs[aid] = sorted(atom_to_wfs[aid])

    # outputs
    out_levels = f"{args.out_prefix}_levels.csv"
    out_full = f"{args.out_prefix}_levels_fullweights.csv"

```

```

out_order = f"{args.out_prefix}_orbital_order.csv"

rows_levels = []
rows_full = []
lowest = {}

for aid in sorted(atom_to_wfs.keys()):
    elem = atom_to_elem[aid]
    wfs = atom_to_wfs[aid]
    evals, evecs = diag_local(H0, wfs)
    evals_rel = evals - args.Ef

    basis_orbs = [wf_orb[wf] for wf in wfs]
    unique_orbs = sorted(list(set(basis_orbs)))

    for j in range(len(evals_rel)):
        c2 = np.abs(evecs[:, j])**2
        orb_w = {o: 0.0 for o in unique_orbs}
        for i, o in enumerate(basis_orbs):
            orb_w[o] += float(c2[i])

        dom_orb = max(orb_w.keys(), key=lambda k: orb_w[k])
        dom_w = orb_w[dom_orb]

        rows_levels.append({
            "atom_id": aid, "elem": elem, "n_wf": len(wfs),
            "level_index": j+1, "E_rel": float(evals_rel[j]),
            "dominant_orb": dom_orb, "dominant_w": float(dom_w)
        })

        r = {"atom_id": aid, "elem": elem, "n_wf": len(wfs),
            "level_index": j+1, "E_rel": float(evals_rel[j])}
        # add all orbital weights as columns
        for o in unique_orbs:
            r[f"w_{o}"] = float(orb_w[o])
        rows_full.append(r)

        key = (aid, elem, dom_orb)
        lowest[key] = min(lowest.get(key, 1e30), float(evals_rel[j]))

pd.DataFrame(rows_levels).to_csv(out_levels, index=False)
pd.DataFrame(rows_full).to_csv(out_full, index=False)

out_rows = []
for (aid, elem, orb), E0 in sorted(lowest.items(), key=lambda x: (x[0][1], x[0][0], x[0][2])):
    out_rows.append({"atom_id": aid, "elem": elem, "orbital": orb, "lowest_E_rel": float(E0)})
pd.DataFrame(out_rows).to_csv(out_order, index=False)

print("[OK] wrote:")
print(" ", out_levels)
print(" ", out_full)
print(" ", out_order)
print("[INFO] spinor_dup =", spinor_dup)

if __name__ == "__main__":
    main()

```

► 代码清单 A.61.8 (python, 119 行)

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-

```

```

import argparse
from pathlib import Path
import numpy as np
import pandas as pd

def weighted_quantile(values, weights, qs):
    """values, weights: 1D arrays; qs in [0,1]"""
    values = np.asarray(values, dtype=float)
    weights = np.asarray(weights, dtype=float)
    m = (weights > 0) & np.isfinite(values)
    values = values[m]; weights = weights[m]
    if len(values) == 0:
        return [np.nan for _ in qs]
    s = np.argsort(values)
    v = values[s]; w = weights[s]
    cw = np.cumsum(w)
    cw /= cw[-1]
    return [float(np.interp(q, cw, v)) for q in qs]

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--full", default="cfFW_levels_fullweights.csv",
                    help="*_levels_fullweights.csv")
    ap.add_argument("--out_prefix", default="elemDist")
    ap.add_argument("--bins", type=int, default=60)
    ap.add_argument("--emin", type=float, default=None)
    ap.add_argument("--emax", type=float, default=None)
    ap.add_argument("--min_weight", type=float, default=1e-6,
                    help="discard contributions with orbital weight < this")
    args = ap.parse_args()

    p = Path(args.full)
    if not p.exists():
        raise FileNotFoundError(p.resolve())

    df = pd.read_csv(p)

    # find weight columns
    wcols = [c for c in df.columns if c.startswith("w_")]
    if not wcols:
        raise RuntimeError("No w_* columns found. Use cfFINAL_export_fullweights.py first.")

    # energy range
    E = df["E_rel"].to_numpy(dtype=float)
    emin = np.nanmin(E) if args.emin is None else args.emin
    emax = np.nanmax(E) if args.emax is None else args.emax
    edges = np.linspace(emin, emax, args.bins + 1)

    stats_rows = []
    hist_rows = []

    # for each element + orbital: treat each level contributes weight w_orb at energy E
    for elem in sorted(df["elem"].unique()):
        sub = df[df["elem"] == elem].copy()
        Es = sub["E_rel"].to_numpy(dtype=float)

        for wc in wcols:
            orb = wc[2:] # remove "w_"
            ws = sub[wc].to_numpy(dtype=float)

            m = ws >= args.min_weight
            if m.sum() == 0:
                continue

            v = Es[m]
            w = ws[m]

```



```

wsum = float(w.sum())
mean = float(np.sum(w * v) / wsum)
var = float(np.sum(w * (v - mean)**2) / wsum)
std = float(np.sqrt(var))

q25, q50, q75 = weighted_quantile(v, w, [0.25, 0.50, 0.75])
vmin = float(np.min(v))
vmax = float(np.max(v))

stats_rows.append({
    "elem": elem, "orbital": orb,
    "weight_sum": wsum,
    "wmean_E": mean,
    "wstd_E": std,
    "wmin_E": vmin,
    "wmax_E": vmax,
    "wq25_E": q25,
    "wq50_E": q50,
    "wq75_E": q75
})

# weighted histogram
hist, _ = np.histogram(v, bins=edges, weights=w)
centers = 0.5 * (edges[:-1] + edges[1:])
for x, h in zip(centers, hist):
    hist_rows.append({
        "elem": elem, "orbital": orb,
        "E_center": float(x),
        "weighted_count": float(h)
    })

stats_df = pd.DataFrame(stats_rows).sort_values(["elem", "orbital"])
hist_df = pd.DataFrame(hist_rows).sort_values(["elem", "orbital", "E_center"])

out_stats = f"{args.out_prefix}_elem_orb_weighted_stats.csv"
out_hist = f"{args.out_prefix}_elem_orb_hist.csv"
stats_df.to_csv(out_stats, index=False)
hist_df.to_csv(out_hist, index=False)

print("[OK] wrote:")
print(" ", out_stats)
print(" ", out_hist)
print(f"[INFO] energy range: [{emin:.6f}, {emax:.6f}] eV, bins={args.bins}")

if __name__ == "__main__":
    main()

```

► 代码清单 A.61.10 (python, 88 行)

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import argparse
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def orbital_sort_key(orb):
    # order p then d if mixed; within sets use common order
    p_order = ["px", "py", "pz"]

```

```

d_order = ["dxy", "dyz", "dxz", "dz2", "dx2-y2"]
if orb in p_order:
    return (0, p_order.index(orb))
if orb in d_order:
    return (1, d_order.index(orb))
return (2, orb)

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--stats", default="elemCFdist_elem_orb_weighted_stats.csv")
    ap.add_argument("--elem", required=True, help="Element symbol, e.g. Se / Tc / Ir / Ge")
    ap.add_argument("--out", default=None, help="Output image path (png/pdf). If omitted, auto name.")
    ap.add_argument("--title", default=None)
    ap.add_argument("--band", action="store_true",
                    help="Draw q25-q75 interval as a vertical band around the median energy.")
    ap.add_argument("--ymin", type=float, default=None)
    ap.add_argument("--ymax", type=float, default=None)
    ap.add_argument("--dpi", type=int, default=300)
    args = ap.parse_args()

    p = Path(args.stats)
    if not p.exists():
        raise FileNotFoundError(p.resolve())

    df = pd.read_csv(p)
    need = {"elem", "orbital", "wq25_E", "wq50_E", "wq75_E", "weight_sum"}
    miss = need - set(df.columns)
    if miss:
        raise RuntimeError(f"Missing columns in stats csv: {sorted(list(miss))}")

    sub = df[df["elem"].astype(str).str.strip() == args.elem].copy()
    if len(sub) == 0:
        raise RuntimeError(f"No rows for elem={args.elem} in {p.name}")

    sub["orbital"] = sub["orbital"].astype(str).str.strip()
    sub = sub.sort_values("orbital", key=lambda s: s.map(orbital_sort_key))

    orbs = sub["orbital"].tolist()
    y50 = sub["wq50_E"].to_numpy(float)
    y25 = sub["wq25_E"].to_numpy(float)
    y75 = sub["wq75_E"].to_numpy(float)

    # x positions: 1..N
    xs = np.arange(1, len(orbs) + 1)

    plt.figure(figsize=(4.8, 5.5))

    # draw median level as horizontal line segments
    for x, e in zip(xs, y50):
        plt.hlines(e, x - 0.35, x + 0.35, linewidth=2)

    # optional: draw q25-q75 as vertical band
    if args.band:
        for x, lo, hi in zip(xs, y25, y75):
            plt.vlines(x, lo, hi, linewidth=2, alpha=0.7)

    plt.xticks(xs, orbs, rotation=0)
    plt.ylabel("Energy (E_rel) [eV]")

    ttl = args.title if args.title else f"{args.elem}: orbital level positions (weighted q50)"
    plt.title(ttl)

    if args.ymin is not None or args.ymax is not None:
        plt.ylim(args.ymin, args.ymax)

    plt.tight_layout()

    out = args.out if args.out else f"{args.elem}_level_diagram.png"

```

```

plt.savefig(out, dpi=args.dpi)
print("[OK] saved:", out)

if __name__ == "__main__":
    main()

```

► 代码清单 A.61.11 (python, 84 行)

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import argparse
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def orbital_sort_key(orb):
    p_order = ["px", "py", "pz"]
    d_order = ["dxy", "dyz", "dxz", "dz2", "dx2-y2"]
    if orb in p_order:
        return (0, p_order.index(orb))
    if orb in d_order:
        return (1, d_order.index(orb))
    return (2, orb)

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--hist", default="elemCFdist_elem_orb_hist.csv")
    ap.add_argument("--elem", required=True)
    ap.add_argument("--out", default=None)
    ap.add_argument("--title", default=None)
    ap.add_argument("--normalize", action="store_true",
                    help="Normalize each orbital curve by its total area (sum of weighted_count).")
    ap.add_argument("--xmin", type=float, default=None)
    ap.add_argument("--xmax", type=float, default=None)
    ap.add_argument("--dpi", type=int, default=300)
    args = ap.parse_args()

    p = Path(args.hist)
    if not p.exists():
        raise FileNotFoundError(p.resolve())

    df = pd.read_csv(p)
    need = {"elem", "orbital", "E_center", "weighted_count"}
    miss = need - set(df.columns)
    if miss:
        raise RuntimeError(f"Missing columns in hist csv: {sorted(list(miss))}")

    sub = df[df["elem"].astype(str).str.strip() == args.elem].copy()
    if len(sub) == 0:
        raise RuntimeError(f"No rows for elem={args.elem} in {p.name}")

    sub["orbital"] = sub["orbital"].astype(str).str.strip()

    orbs = sorted(sub["orbital"].unique(), key=orbital_sort_key)

    plt.figure(figsize=(5.6, 4.8))

    for orb in orbs:

```

```

s = sub[sub["orbital"] == orb].sort_values("E_center")
x = s["E_center"].to_numpy(float)
y = s["weighted_count"].to_numpy(float)

if args.normalize:
    area = float(np.sum(y))
    if area > 0:
        y = y / area

plt.plot(x, y, linewidth=1.8, label=orb)

plt.xlabel("Energy (E_rel) [eV]")
plt.ylabel("Weighted intensity" + (" (normalized)" if args.normalize else ""))

ttl = args.title if args.title else f"{args.elem}: orbital-weighted energy distributions"
plt.title(ttl)

if args.xmin is not None or args.xmax is not None:
    plt.xlim(args.xmin, args.xmax)

plt.legend()
plt.tight_layout()

out = args.out if args.out else f"{args.elem}_orbital_distributions.png"
plt.savefig(out, dpi=args.dpi)
print("[OK] saved:", out)

if __name__ == "__main__":
    main()

```

► 代码清单 A.61.12 (shell, 10 行)

```

python plot_elem_level_diagram.py --stats elemCFdist_elem_orb_weighted_stats.csv --elem Se --out Se_levels.png
python plot_elem_level_diagram.py --stats elemCFdist_elem_orb_weighted_stats.csv --elem Tc --out Tc_levels.png
python plot_elem_level_diagram.py --stats elemCFdist_elem_orb_weighted_stats.csv --elem Ir --out Ir_levels.png
python plot_elem_level_diagram.py --stats elemCFdist_elem_orb_weighted_stats.csv --elem Ge --out Ge_levels.png

python plot_elem_orb_distributions.py --hist elemCFdist_elem_orb_hist.csv --elem Se --out Se_dist.png
python plot_elem_orb_distributions.py --hist elemCFdist_elem_orb_hist.csv --elem Tc --out Tc_dist.png
python plot_elem_orb_distributions.py --hist elemCFdist_elem_orb_hist.csv --elem Ir --out Ir_dist.png
python plot_elem_orb_distributions.py --hist elemCFdist_elem_orb_hist.csv --elem Ge --out Ge_dist.png

```

► 代码清单 A.61.13 (python, 583 行)

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import argparse
from pathlib import Path
import numpy as np
import pandas as pd

# =====
# 1) POSCAR parser (VASP5/6)
# =====

```

```

def read_poscar(poscar_path: Path):
    lines = poscar_path.read_text(encoding="utf-8", errors="ignore").splitlines()
    if len(lines) < 8:
        raise RuntimeError("POSCAR too short")

    comment = lines[0].strip()
    scale = float(lines[1].split()[0])
    lattice = np.array([[float(x) for x in lines[i].split()[3:]] for i in range(2, 5)], dtype=float) * scale

    # VASP5+: line 5 symbols, line 6 counts
    # VASP4: line 5 counts directly
    tokens5 = lines[5].split()
    tokens6 = lines[6].split()

    def is_all_int(toks):
        try:
            for t in toks:
                int(t)
            return True
        except Exception:
            return False

    if is_all_int(tokens5):
        # VASP4 style: no symbols line
        species = [f"X{i+1}" for i in range(len(tokens5))]
        counts = [int(x) for x in tokens5]
        coord_line = 6
    else:
        species = tokens5
        counts = [int(x) for x in tokens6]
        coord_line = 7

    coord_type = lines[coord_line].strip().lower()
    if coord_type.startswith("s"):
        # selective dynamics present
        coord_line += 1
        coord_type = lines[coord_line].strip().lower()

    direct = coord_type.startswith("d")

    n_atoms = sum(counts)
    start = coord_line + 1
    coord_lines = lines[start:start + n_atoms]
    if len(coord_lines) < n_atoms:
        raise RuntimeError("POSCAR missing atomic coordinates")

    frac = []
    for ln in coord_lines:
        toks = ln.split()
        frac.append([float(toks[0]), float(toks[1]), float(toks[2])])
    frac = np.array(frac, dtype=float)

    if not direct:
        # Cartesian -> convert to fractional
        # cart = frac * scale? already included scale in lattice, so treat as Å
        cart = frac.copy()
        frac = cart @ np.linalg.inv(lattice)

    # element per atom list (1-based atom_id)
    elems = []
    for sp, ct in zip(species, counts):
        elems += [sp] * ct

    return {
        "comment": comment,
        "lattice": lattice,      # (3,3) row vectors in Å
        "frac": frac,           # (N,3)
        "elems": elems,         # length N, 1-based id -> elems[id-1]
        "species": species,
    }

```

```

        "counts": counts
    }

def frac_to_cart(frac, lattice):
    # lattice row vectors: a,b,c
    return frac @ lattice

def min_image_frac(dfrac):
    # wrap to [-0.5, 0.5)
    return dfrac - np.round(dfrac)

# =====
# 2) hr.dat onsite H(R=0)
# =====
def parse_hr_dat(hr_path: Path):
    lines = hr_path.read_text(encoding="utf-8", errors="ignore").splitlines()
    if len(lines) < 4:
        raise RuntimeError("hr.dat too short")
    num_wann = int(lines[1].strip())
    nrpts = int(lines[2].strip())

    degen = []
    idx = 3
    while len(degen) < nrpts:
        degen.extend([int(t) for t in lines[idx].split()])
        idx += 1
    rec_lines = lines[idx:]
    return num_wann, rec_lines

def build_H0(num_wann, rec_lines):
    H0 = np.zeros((num_wann, num_wann), dtype=np.complex128)
    for ln in rec_lines:
        if not ln.strip():
            continue
        toks = ln.split()
        if len(toks) < 7:
            continue
        Rx, Ry, Rz = int(toks[0]), int(toks[1]), int(toks[2])
        if (Rx, Ry, Rz) != (0, 0, 0):
            continue
        m = int(toks[3]) - 1
        n = int(toks[4]) - 1
        H0[m, n] = float(toks[5]) + 1j * float(toks[6])
    return 0.5 * (H0 + H0.conj().T)

# =====
# 3) edges.csv WF -> (atom_id, elem)
# =====
def read_edges_wf_map(edges_path: Path):
    df = pd.read_csv(edges_path)
    needed = ["m", "n", "atom_m", "atom_n", "elem_m", "elem_n"]
    for c in needed:
        if c not in df.columns:
            raise RuntimeError(f"edges.csv missing column: {c}")

    wf_to_pairs = {}
    for _, r in df.iterrows():
        m = int(r["m"]); n = int(r["n"])
        am = int(r["atom_m"]); an = int(r["atom_n"])
        em = str(r["elem_m"]); en = str(r["elem_n"])
        wf_to_pairs.setdefault(m, set()).add((am, em))
        wf_to_pairs.setdefault(n, set()).add((an, en))

    wf_map = {}

```

```

bad = []
for wf, s in wf_to_pairs.items():
    if len(s) != 1:
        bad.append((wf, sorted(list(s))))
    else:
        wf_map[wf] = next(iter(s))
if bad:
    msg = ["[ERROR] WF labeling inconsistent in edges.csv (one WF -> multiple atom/elem)."]
    for wf, pairs in bad[:30]:
        msg.append(f" wf={wf}: {pairs}")
    raise RuntimeError("\n".join(msg))
return wf_map

def atoms_by_element_from_edges(wf_map, num_wann):
    elem_to_atoms = {}
    for wf in range(1, num_wann + 1):
        aid, elem = wf_map[wf]
        elem_to_atoms.setdefault(elem, set()).add(aid)
    return {e: sorted(list(s)) for e, s in elem_to_atoms.items()}

# =====
# 4) parse win projections => expected WF orbital labels
# =====
def norm_orb(tok: str) -> str:
    t = tok.strip().lower()
    t = t.replace("dx2y2", "dx2-y2").replace("dx2_y2", "dx2-y2")
    t = t.replace("d(z2)", "dz2").replace("d(z^2)", "dz2").replace("d(z**2)", "dz2")
    t = t.replace("d(x2-y2)", "dx2-y2")
    return t

def expand_token(tok: str):
    t = norm_orb(tok)
    if t == "d":
        return ["dxy", "dyz", "dxz", "dz2", "dx2-y2"]
    if t == "p":
        return ["px", "py", "pz"]
    if t == "s":
        return ["s"]
    return [t]

def parse_win_projections(win_path: Path):
    lines = win_path.read_text(encoding="utf-8", errors="ignore").splitlines()
    in_proj = False
    proj_lines = []
    for ln in lines:
        s = ln.strip()
        low = s.lower()
        if low.startswith("begin projections"):
            in_proj = True
            continue
        if low.startswith("end projections"):
            in_proj = False
            continue
        if not in_proj:
            continue
        if (not s) or s.startswith("#") or s.startswith("!"):
            continue
        proj_lines.append(s)

    out = []
    for raw in proj_lines:
        if ":" not in raw:
            continue
        elem, rhs = raw.split(":", 1)
        elem = elem.strip()

```

```

        rhs = rhs.replace(",", " ").replace(";", " ")
        toks = [t for t in rhs.split() if t.strip()]
        orbs = []
        for t in toks:
            orbs += expand_token(t)
        out.append((elem, orbs))
    if not out:
        raise RuntimeError("[ERROR] No projections found in wannier90.win.")
    return out

def build_expected_labels(proj_spec, elem_to_atoms, num_wann):
    spatial = []
    for elem, orbs in proj_spec:
        if elem not in elem_to_atoms:
            raise RuntimeError(f"[ERROR] elem '{elem}' in win not found in edges-derived elements:
                               ↪ {list(elem_to_atoms.keys())}")
        for aid in elem_to_atoms[elem]:
            for orb in orbs:
                spatial.append((aid, elem, orb))

    if len(spatial) == num_wann:
        return spatial, False
    if 2 * len(spatial) == num_wann:
        spinor = []
        for x in spatial:
            spinor.append(x); spinor.append(x)
        return spinor, True

    raise RuntimeError(f"[ERROR] Projection count mismatch: spatial={len(spatial)}, num_wann={num_wann}")

# =====
# 5) Build Tc 5x5 onsite from spinor-duplicated 10x10
#    by averaging over the two duplicates per orbital label.
# =====
D_ORBS = ["dxy", "dyz", "dxz", "dz2", "dx2-y2"]

def build_orbital_indices_for_atom(atom_id, expected_labels, wf_map, target_orbs):
    """
    Returns: dict orb -> list of wf indices (1-based) belonging to atom_id with that orbital label
    Requires WF ordering to match expected_labels (checked).
    """
    orb2wfs = {o: [] for o in target_orbs}
    for wf in range(1, len(expected_labels) + 1):
        aid_true, elem_true = wf_map[wf]
        aid_e, elem_e, orb_e = expected_labels[wf - 1]
        if (aid_true != aid_e) or (elem_true != elem_e):
            # ordering mismatch
            return None
        if aid_true == atom_id and orb_e in orb2wfs:
            orb2wfs[orb_e].append(wf)
    return orb2wfs

def average_subblock(H0, wfs_a, wfs_b):
    # average over all pairs (a,b)
    A = np.array([w - 1 for w in wfs_a], dtype=int)
    B = np.array([w - 1 for w in wfs_b], dtype=int)
    sub = H0[np.ix_(A, B)]
    return np.mean(sub)

def build_H5_for_atom(H0, orb2wfs):
    # 5x5 in orbital-label space
    H5 = np.zeros((5, 5), dtype=np.complex128)
    for i, oi in enumerate(D_ORBS):
        for j, oj in enumerate(D_ORBS):
            wi = orb2wfs.get(oi, [])

```

```

        wj = orb2wfs.get(oj, [])
        if len(wi) == 0 or len(wj) == 0:
            raise RuntimeError(f"Missing WF indices for orb {oi} or {oj}. Check win projections.")
        H5[i, j] = average_subblock(H0, wi, wj)
    return 0.5 * (H5 + H5.conj().T)

# =====
# 6) Local axis from octahedron (PCA on neighbor vectors)
# =====
def pca_local_axes(vectors, weight_mode="inv_r2"):
    """
    vectors: (6,3) Cartesian vectors from center to neighbors (Å)
    weight_mode: 'none' | 'inv_r2'
    Return R (3,3) with columns = local x',y',z' in global coords (right-handed).
    """
    V = np.array(vectors, dtype=float)
    r = np.linalg.norm(V, axis=1)
    if weight_mode == "inv_r2":
        w = 1.0 / np.maximum(r*r, 1e-12)
    else:
        w = np.ones_like(r)

    M = np.zeros((3, 3), dtype=float)
    for vi, wi in zip(V, w):
        M += wi * np.outer(vi, vi)

    evals, evects = np.linalg.eigh(M) # ascending
    # choose axis order: largest eigenvalue => "principal" axis (often distortion axis)
    order = np.argsort(evals)[::-1]
    evects = evects[:, order]

    # enforce right-handed
    if np.linalg.det(evects) < 0:
        evects[:, 2] *= -1.0

    return evects, evals[order]

# =====
# 7) Rotate real d-orbital basis using quadratic-form trick
#    f(x)=x^T Q x. Under x = R x', Q' = R^T Q R.
# =====
def d_quadratic_matrices():
    """
    Real cubic harmonics basis as traceless symmetric matrices Q:
    dxy ~ xy
    dyz ~ yz
    dxz ~ xz
    dx2-y2 ~ x^2 - y^2
    dz2 ~ 2 z^2 - x^2 - y^2 (proportional to 3z^2-r^2)
    """
    Q = {}
    Q["dxy"] = np.array([[0, 0.5, 0],
                        [0.5, 0, 0],
                        [0, 0, 0]], float)
    Q["dyz"] = np.array([[0, 0, 0],
                        [0, 0, 0.5],
                        [0, 0.5, 0]], float)
    Q["dxz"] = np.array([[0, 0, 0.5],
                        [0, 0, 0],
                        [0.5, 0, 0]], float)
    Q["dx2-y2"] = np.array([[1, 0, 0],
                        [0, -1, 0],
                        [0, 0, 0]], float)
    Q["dz2"] = np.array([[-1, 0, 0],
                        [0, -1, 0],
                        [0, 0, 2]], float)

    return Q

```

```

def vec_sym_traceless(Qm):
    """
    Map symmetric matrix to 5D vector in our basis space using inner products.
    We'll solve coefficients by least squares against basis matrices.
    """
    # flatten symmetric components (xx,yy,zz,xy,xz,yz) for robustness
    return np.array([Qm[0, 0], Qm[1, 1], Qm[2, 2], Qm[0, 1], Qm[0, 2], Qm[1, 2]], float)

def build_d_rotation_matrix(R):
    """
    Given 3x3 rotation R (columns are local axes in global coords),
    construct 5x5 matrix C such that coefficients transform as:
        v_local = C v_global (in the D_ORBS order)
    Using Q'_a = R^T Q_a R and project onto basis.
    """
    Q = d_quadratic_matrices()
    B = np.stack([vec_sym_traceless(Q[o]) for o in D_ORBS], axis=1) # (6,5)
    # solve for each rotated basis matrix
    C = np.zeros((5, 5), float)
    for a, orb in enumerate(D_ORBS):
        Qp = R.T @ Q[orb] @ R # expressed in local coords
        vp = vec_sym_traceless(Qp) # (6,)
        # least squares: B c = vp
        c, *_ = np.linalg.lstsq(B, vp, rcond=None)
        C[:, a] = c
    # Orthonormalize C (numerical)
    # Use QR on C^T then transpose back
    Qt, Rt = np.linalg.qr(C.T)
    C_ortho = Qt.T
    return C_ortho

# =====
# 8) Nearest-neighbor search Tc->6 Se (PBC)
# =====
def get_cart_vectors_to_neighbors(frac, lattice, center_idx0, neighbor_indices0):
    """
    center_idx0: 0-based index
    neighbor_indices0: list of 0-based indices
    return vectors (Nn,3) in Cartesian using minimum-image in fractional space
    """
    fc = frac[center_idx0]
    out = []
    for j0 in neighbor_indices0:
        df = frac[j0] - fc
        df = min_image_frac(df)
        out.append(df @ lattice)
    return np.array(out, float)

def find_k_nearest_neighbors(frac, lattice, center_idx0, candidate_idx0, k=6):
    fc = frac[center_idx0]
    dists = []
    for j0 in candidate_idx0:
        df = frac[j0] - fc
        df = min_image_frac(df)
        v = df @ lattice
        d = float(np.linalg.norm(v))
        dists.append((d, j0))
    dists.sort(key=lambda x: x[0])
    return dists[:k]

# =====
# 9) Main
# =====

```

```

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--poscar", default="POSCAR")
    ap.add_argument("--hr", default="wannier90_hr.dat")
    ap.add_argument("--edges", default="edges.csv")
    ap.add_argument("--win", default="wannier90.win")
    ap.add_argument("--center", default="Tc")
    ap.add_argument("--neighbor", default="Se")
    ap.add_argument("--center_atom_id", type=int, default=None, help="If set, analyze only this POSCAR atom id
↪ (1-based)")
    ap.add_argument("--Ef", type=float, default=0.0, help="Shift energies as E_rel = E - Ef")
    ap.add_argument("--weight_mode", default="inv_r2", choices=["inv_r2", "none"])
    ap.add_argument("--out_prefix", default="TcCF")
    args = ap.parse_args()

    poscar_path = Path(args.poscar)
    hr_path = Path(args.hr)
    edges_path = Path(args.edges)
    win_path = Path(args.win)
    for p in [poscar_path, hr_path, edges_path, win_path]:
        if not p.exists():
            raise FileNotFoundError(p.resolve())

    # POSCAR
    S = read_poscar(poscar_path)
    frac = S["frac"]
    lattice = S["lattice"]
    elems = S["elems"]

    # center/neighbor indices in POSCAR (0-based)
    center_idx0 = [i for i, e in enumerate(elems) if e == args.center]
    neigh_idx0 = [i for i, e in enumerate(elems) if e == args.neighbor]
    if not center_idx0:
        raise RuntimeError(f"No center element '{args.center}' found in POSCAR element list {set(elems)}")
    if not neigh_idx0:
        raise RuntimeError(f"No neighbor element '{args.neighbor}' found in POSCAR element list {set(elems)}")

    if args.center_atom_id is not None:
        i0 = args.center_atom_id - 1
        if i0 < 0 or i0 >= len(elems):
            raise RuntimeError("center_atom_id out of range")
        if elems[i0] != args.center:
            raise RuntimeError(f"center_atom_id={args.center_atom_id} is element {elems[i0]}, not {args.center}")
        center_idx0 = [i0]

    # hr onsite
    num_wann, rec_lines = parse_hr_dat(hr_path)
    H0 = build_H0(num_wann, rec_lines)

    # WF mapping
    wf_map = read_edges_wf_map(edges_path)
    elem_to_atoms_edges = atoms_by_element_from_edges(wf_map, num_wann)
    proj_spec = parse_win_projections(win_path)
    expected_labels, spinor_dup = build_expected_labels(proj_spec, elem_to_atoms_edges, num_wann)

    # Output rows
    rows_axes = []
    rows_levels = []
    rows_neighbors = []

    for c0 in center_idx0:
        atom_id = c0 + 1 # POSCAR 1-based

        # find 6 nearest Se around this Tc
        nn = find_k_nearest_neighbors(frac, lattice, c0, neigh_idx0, k=6)
        nn_ids0 = [j0 for _, j0 in nn]
        nn_d = [d for d, _ in nn]

        # vectors center->Se

```

```

vecs = get_cart_vectors_to_neighbors(frac, lattice, c0, nn_ids0)

# PCA local axes
Rloc, evals = pca_local_axes(vecs, weight_mode=args.weight_mode) # columns: x',y',z' in global coords

# Rotation in d space (global->local coefficients)
C = build_d_rotation_matrix(Rloc) # 5x5 real
# NOTE: coefficients v_local = C v_global

# Build Tc H5 (global d basis) from wf_map/expected_labels using atom_id in edges atom indexing.
# Important: edges atom_id indexing must match POSCAR atom_id (you already established this mapping
↪ earlier).
orb2wfs = build_orbital_indices_for_atom(atom_id, expected_labels, wf_map, D_ORBS)
if orb2wfs is None:
    raise RuntimeError("WF ordering mismatch vs win expansion; cannot build orbital indices safely.")

H5g = build_H5_for_atom(H0, orb2wfs) # 5x5 complex hermitian
# Rotate to local basis (treat C as real orthonormal): H_local = C H_global C^T
H5l = C @ H5g @ C.T

# Diagonalize local 5x5 (energies; eigenvectors in local orbital basis)
evals_d, evecs_d = np.linalg.eigh(H5l)
order = np.argsort(np.real(evals_d))
evals_d = np.real(evals_d[order]) - args.Ef
evecs_d = evecs_d[:, order]

# record neighbor table
for rank, (d, j0) in enumerate(nn, start=1):
    rows_neighbors.append({
        "center_atom_id": atom_id,
        "neighbor_rank": rank,
        "neighbor_atom_id": j0 + 1,
        "neighbor_elem": elems[j0],
        "distance_A": float(d)
    })

# record axes
rows_axes.append({
    "center_atom_id": atom_id,
    "center_elem": args.center,
    "pca_eval1": float(evals[0]),
    "pca_eval2": float(evals[1]),
    "pca_eval3": float(evals[2]),
    "xprime_global": ", ".join([f"{x:.6f}" for x in Rloc[:, 0]]),
    "yprime_global": ", ".join([f"{x:.6f}" for x in Rloc[:, 1]]),
    "zprime_global": ", ".join([f"{x:.6f}" for x in Rloc[:, 2]]),
    "nn6_distances_A": ", ".join([f"{x:.4f}" for x in nn_d]),
    "nn6_mean_A": float(np.mean(nn_d)),
    "nn6_std_A": float(np.std(nn_d)),
    "nn6_min_A": float(np.min(nn_d)),
    "nn6_max_A": float(np.max(nn_d)),
    "spinor_dup": bool(spinor_dup)
})

# record local CF levels and orbital characters in local basis
for i in range(5):
    v = evecs_d[:, i]
    w = np.abs(v)**2
    dom = int(np.argmax(w))
    rows_levels.append({
        "center_atom_id": atom_id,
        "level_index": i + 1,
        "E_rel_eV": float(evals_d[i]),
        "dom_orb_local": D_ORBS[dom] + "",
        "w_dxy_p": float(w[0]),
        "w_dyz_p": float(w[1]),
        "w_dxz_p": float(w[2]),
        "w_dz2_p": float(w[3]),
        "w_dx2y2_p": float(w[4]),
    })

```

```

    })

# write outputs
out_axes = f"{args.out_prefix}_Tc_local_axes.csv"
out_nn = f"{args.out_prefix}_Tc_Se6_neighbors.csv"
out_lv = f"{args.out_prefix}_Tc_local_d_levels.csv"

pd.DataFrame(rows_axes).to_csv(out_axes, index=False)
pd.DataFrame(rows_neighbors).to_csv(out_nn, index=False)
pd.DataFrame(rows_levels).to_csv(out_lv, index=False)

print("[OK] wrote:")
print(" ", out_axes)
print(" ", out_nn)
print(" ", out_lv)
print("[NOTE]")
print(" - Energies are from Tc onsite 5x5 effective d Hamiltonian (R=0), rotated into local octahedron axes by
↪ PCA.")
print(" - dom_orb_local is defined in the local axes (x',y',z'), hence meaningful for distorted octahedron CF
↪ discussion.")

if __name__ == "__main__":
    main()

```

► 代码清单 A.61.15 (python, 2 行)

PDOS_Tc_DW.dat
PDOS_Tc_UP.dat

► 代码清单 A.61.16 (python, 121 行)

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import argparse
from pathlib import Path
import numpy as np
import pandas as pd

COLS = ["E", "s", "py", "pz", "px", "dxy", "dyz", "dz2", "dxz", "dx2y2", "tot"]
D_ORBS = ["dxy", "dyz", "dz2", "dxz", "dx2y2"]

def load_pdos(path: Path):
    return pd.read_csv(path, sep=r"\s+", comment="#", names=COLS, skiprows=1)

def trapz(E, Y):
    return np.trapz(Y, E)

def band_center(E, D, Emin, Emax):
    m = (E >= Emin) & (E <= Emax)
    Ew = E[m]; Dw = D[m]
    denom = trapz(Ew, Dw)
    if abs(denom) < 1e-14:
        return np.nan, 0.0
    num = trapz(Ew, Ew * Dw)
    return num/denom, denom

```

```

def cosine_sim(E, A, B, Emin, Emax):
    m = (E >= Emin) & (E <= Emax)
    a = A[m]; b = B[m]
    # L2-normalize by integral of square (continuous analog)
    na = np.sqrt(trapz(E[m], a*a))
    nb = np.sqrt(trapz(E[m], b*b))
    if na < 1e-14 or nb < 1e-14:
        return np.nan
    return trapz(E[m], a*b) / (na*nb)

def auto_groups(sim_mat, labels, thr=0.95):
    # 简单阈值连通分量: sim>=thr 即连边
    n = len(labels)
    visited = [False]*n
    groups = []
    for i in range(n):
        if visited[i]:
            continue
        stack = [i]
        comp = []
        visited[i] = True
        while stack:
            u = stack.pop()
            comp.append(u)
            for v in range(n):
                if not visited[v] and sim_mat[u,v] >= thr:
                    visited[v] = True
                    stack.append(v)
        groups.append([labels[k] for k in sorted(comp)])
    return groups

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--up", required=True)
    ap.add_argument("--dw", required=True)
    ap.add_argument("--Emin", type=float, default=-1.0)
    ap.add_argument("--Emax", type=float, default=0.0)
    ap.add_argument("--thr", type=float, default=0.95, help="cosine similarity threshold for grouping")
    ap.add_argument("--out_prefix", default="Auto")
    args = ap.parse_args()

    up = load_pdos(Path(args.up))
    dw = load_pdos(Path(args.dw))

    if not np.allclose(up["E"].values, dw["E"].values, atol=1e-10):
        raise RuntimeError("Energy grids differ between UP/DW.")

    E = up["E"].to_numpy(float)
    Dup = up[D_ORBS].to_numpy(float)
    Ddw = np.abs(dw[D_ORBS].to_numpy(float))
    D = Dup + Ddw # spin-summed DOS

    # --- similarity matrix ---
    n = len(D_ORBS)
    sim = np.zeros((n,n), float)
    for i in range(n):
        for j in range(n):
            if i == j:
                sim[i,j] = 1.0
            elif j < i:
                sim[i,j] = sim[j,i]
            else:
                sim[i,j] = cosine_sim(E, D[:,i], D[:,j], args.Emin, args.Emax)

    df_sim = pd.DataFrame(sim, index=D_ORBS, columns=D_ORBS)
    df_sim.to_csv(f"{args.out_prefix}_sim_matrix.csv")

    groups = auto_groups(sim, D_ORBS, thr=args.thr)

```

```

# --- per-orbital band centers ---
orb_rows = []
for i, orb in enumerate(D_ORBS):
    Ec, Nw = band_center(E, D[:,i], args.Emin, args.Emax)
    orb_rows.append({"orb": orb, "E_center": Ec, "N_window": Nw})
df_orb = pd.DataFrame(orb_rows).sort_values("E_center")
df_orb.to_csv(f"{args.out_prefix}_orb_centers.csv", index=False)

# --- per-group band centers (sum DOS in group) ---
grp_rows = []
for g in groups:
    idx = [D_ORBS.index(o) for o in g]
    Dg = D[:,idx].sum(axis=1)
    Ec, Nw = band_center(E, Dg, args.Emin, args.Emax)
    grp_rows.append({"group": "(" + ",".join(g) + ")", "members": ",".join(g), "E_center": Ec, "N_window":
        ↪ Nw})
df_grp = pd.DataFrame(grp_rows).sort_values("E_center")
df_grp.to_csv(f"{args.out_prefix}_auto_groups.csv", index=False)

print("[OK] similarity matrix:", f"{args.out_prefix}_sim_matrix.csv")
print("[OK] orbital centers:", f"{args.out_prefix}_orb_centers.csv")
print("[OK] auto groups:", f"{args.out_prefix}_auto_groups.csv")
print("\n[Auto groups @ thr =", args.thr, "]")
print(df_grp.to_string(index=False))

if __name__ == "__main__":
    main()

```

► 代码清单 A.61.18 (python, 208 行)

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
plot_level_diagram.py

```

通用能级排布图绘制脚本:

- 从 orb_centers.csv 读每个轨道能量中心 E_center
- 从 auto_groups.csv 读简并分组 (哪些轨道一组)

两种使用方式:

A) 显式指定文件 (推荐、最稳)

```
python plot_level_diagram.py --orb XXX_orb_centers.csv --groups XXX_auto_groups.csv --out level.png
```

B) 只给 prefix 自动匹配 (省事)

```
python plot_level_diagram.py --prefix TcAuto --dir . --out Tc_level.png
```

会自动找:

```

<dir>/<prefix>_orb_centers.csv
<dir>/<prefix>_auto_groups.csv

```

可选:

```

--eref -1.46      # 平移能量参考 (如把费米能级平移到 0)
--title "... "    # 标题
"""

```

```

import argparse
import re
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

```

```

def guess_col(df, candidates):
    for c in candidates:
        if c in df.columns:
            return c
    return None

def parse_orb_list(x):
    if pd.isna(x):
        return []
    s = str(x).strip()

    # 提取常见 d 轨道 token (可按需扩展, 比如 p/s/f)
    tokens = re.findall(r"(dxy|dxz|dyz|dz2|dx2y2)", s, flags=re.IGNORECASE)
    if tokens:
        return [t.lower() for t in tokens]

    # fallback: 逗号/空格分割
    s = s.strip("[](){}")
    parts = re.split(r"[,\s]+", s)
    parts = [p.strip().strip("\'").lower() for p in parts if p.strip()]
    return parts

def resolve_inputs(args):
    """
    解析输入:
    - 若用户显式给 --orb 和 --groups: 直接用
    - 否则要求给 --prefix, 并在 --dir 中匹配 <prefix>_orb_centers.csv / <prefix>_auto_groups.csv
    """
    if args.orb and args.groups:
        orb_path = Path(args.orb)
        grp_path = Path(args.groups)
    else:
        if not args.prefix:
            raise ValueError(" 请使用: 要么同时提供 --orb 与 --groups; 要么提供 --prefix (可配合 --dir)。")
        base = Path(args.dir).expanduser().resolve()
        orb_path = base / f"{args.prefix}_orb_centers.csv"
        grp_path = base / f"{args.prefix}_auto_groups.csv"

    if not orb_path.exists():
        raise FileNotFoundError(f" 找不到 orb_centers.csv: {orb_path}")
    if not grp_path.exists():
        raise FileNotFoundError(f" 找不到 auto_groups.csv: {grp_path}")

    return orb_path, grp_path

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--orb", default=None, help=" 显式指定 orb_centers.csv 路径")
    ap.add_argument("--groups", default=None, help=" 显式指定 auto_groups.csv 路径")
    ap.add_argument("--prefix", default=None, help=" 自动匹配前缀: <prefix>_orb_centers.csv / <prefix>_auto_groups.csv")
    ap.add_argument("--dir", default=".", help=" 自动匹配时搜索目录 (默认当前目录) ")

    ap.add_argument("--out", default="level_diagram.png", help=" 输出图片名")
    ap.add_argument("--eref", type=float, default=0.0, help=" 能量参考平移: E_plot = E_center - eref")
    ap.add_argument("--title", default=None, help=" 图标题")
    ap.add_argument("--ymin", type=float, default=None)
    ap.add_argument("--ymax", type=float, default=None)
    ap.add_argument("--fontsize", type=int, default=12)
    ap.add_argument("--linewidth", type=float, default=3.0)
    ap.add_argument("--xspan", type=float, default=0.55, help=" 每条能级线横向半宽度")
    args = ap.parse_args()

    orb_path, grp_path = resolve_inputs(args)

    df_orb = pd.read_csv(orb_path)

```

```

df_grp = pd.read_csv(grp_path)

# ---- 1) orb_centers: 识别轨道列 + 能量列
col_orb = guess_col(df_orb, ["orb", "orbital", "orb_name", "name", "orbital_name"])
col_E = guess_col(df_orb, ["E_center", "Ecentre", "Ecent", "E", "center_eV", "energy_center"])
if col_orb is None or col_E is None:
    raise ValueError(
        f"[orb_centers] 列名识别失败。现有列: {list(df_orb.columns)}\n"
        f" 需要类似: orb/orbital + E_center"
    )

orb2E = {}
for _, r in df_orb.iterrows():
    o = str(r[col_orb]).strip().lower()
    orb2E[o] = float(r[col_E]) - args.eref

# ---- 2) auto_groups: 识别轨道列表列 + 组能量列 (可选) + 简并度列 (可选)
col_list = guess_col(df_grp, ["orbs", "orbitals", "orb_list", "members", "group_orbs"])
col_grpE = guess_col(df_grp, ["E_center", "E_group", "group_E", "E", "center_eV"])
col_size = guess_col(df_grp, ["size", "deg", "degeneracy", "n_orb", "N"])

groups = []
for _, r in df_grp.iterrows():
    orbs = parse_orb_list(r[col_list]) if col_list else []
    orbs = [o.lower() for o in orbs if o]
    if not orbs:
        # 兜底: 从整行提取
        orbs = parse_orb_list(" ".join(map(str, r.values)))

    # 组能量: 优先 groups 表里的 E_center, 否则用成员平均
    if col_grpE and (not pd.isna(r[col_grpE])):
        Eg = float(r[col_grpE]) - args.eref
    else:
        Es = [orb2E[o] for o in orbs if o in orb2E]
        Eg = float(np.mean(Es)) if Es else np.nan

    # 简并度: 优先 size 列, 否则用 orbs 数量
    if col_size and (not pd.isna(r[col_size])):
        gsize = int(r[col_size])
    else:
        gsize = len(orbs)

    # label 排序显示更直观 (可按需扩展)
    pretty_order = ["dxy", "dx2y2", "dz2", "dxz", "dyz"]
    orbs_sorted = sorted(orbs, key=lambda x: pretty_order.index(x) if x in pretty_order else 999)
    label = " + ".join(orbs_sorted) if orbs_sorted else "Group"

    groups.append({"Eg": Eg, "orbs": orbs_sorted, "size": gsize, "label": label})

groups = [g for g in groups if np.isfinite(g["Eg"])]
if not groups:
    raise RuntimeError(" 没有解析到任何有效的 group 能量 Eg, 请检查 groups.csv 格式。")

# 按能量排序 (低-> 高)
groups.sort(key=lambda g: g["Eg"])

# ---- 3) 绘图
n = len(groups)
fig_h = max(4.0, 1.2 + 0.8 * n)
fig, ax = plt.subplots(figsize=(6.4, fig_h))

x0 = 0.0
xspan = args.xspan
x_offsets = np.linspace(-0.15, 0.15, n) if n > 1 else np.array([0.0])

for i, g in enumerate(groups):
    y = g["Eg"]
    xo = x_offsets[i]

```

```

    ax.hlines(y, x0 - xspan + xo, x0 + xspan + xo, linewidth=args.linewidth)

    deg_txt = f"({g['size']})" if g["size"] else ""
    txt = f"{g['label']} {deg_txt}".strip()

    ax.text(x0 + xspan + 0.10, y, txt, va="center", fontsize=args.fontsize)
    ax.text(x0 - xspan - 0.10, y, f"{y:.3f} eV", va="center", ha="right", fontsize=args.fontsize - 1)

ax.set_xlim(-1.2, 1.9)
ax.set_xticks([])
ax.set_ylabel(f"Energy (eV) [E_center - {args.eref:g}]", fontsize=args.fontsize)
ax.grid(True, axis="y", linestyle="--", linewidth=0.6, alpha=0.5)

if args.title:
    ax.set_title(args.title, fontsize=args.fontsize + 2)
else:
    # 默认标题: 用文件名提示来源
    ax.set_title(f"Level diagram from {orb_path.name}", fontsize=args.fontsize + 1)

ys = [g["Eg"] for g in groups]
ypad = 0.15 * (max(ys) - min(ys) + 1e-9)
ymin = (min(ys) - ypad) if args.ymin is None else args.ymin
ymax = (max(ys) + ypad) if args.ymax is None else args.ymax
ax.set_ylim(ymin, ymax)

fig.tight_layout()
fig.savefig(args.out, dpi=300)

print(f"[OK] orb_centers: {orb_path}")
print(f"[OK] auto_groups: {grp_path}")
print(f"[OK] Saved: {args.out}")
print("[INFO] Levels (low -> high):")
for g in groups:
    print(f"   Eg={g['Eg']:.6f} eV   size={g['size']}   {g['label']}")

if __name__ == "__main__":
    main()

```

(原创自研) 利用 **wannier** 计算超超交换过程中的轨道杂化

► 代码清单 A.62.1 (python, 375 行)

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-

"""
1step.py (Index-based robust version) [SOC-ready + output re/im]

输出 edges CSV 现在包含:
    absH_eV, reH_eV, imH_eV, phase_rad, phase_deg
便于后续做:
    - 轨道杂化 (absH / absH^2)
    - 2x2 SOC spinor block Frobenius norm / singular values (更接近文献 "hopping 幅度" 口径)
"""

import re
import csv
import math
import argparse

```

```

from collections import defaultdict, namedtuple
import numpy as np

# -----
# Edge record
# -----
Edge = namedtuple(
    "Edge",
    "pair shell dist absH reH imH phase_rad Rx Ry Rz m n atom_m atom_n elem_m elem_n group_m group_n orb_m orb_n"
)

# =====
# Parse wannier90.win
# =====
def _extract_block(txt, block_name):
    m = re.search(rf"begin\s+{block_name}(.*?)end\s+{block_name}", txt, re.S | re.I)
    return None if not m else m.group(1).strip()

def parse_win_lattice_and_atoms(win_path):
    with open(win_path, "r", encoding="utf-8", errors="ignore") as f:
        txt = f.read()

    cell_block = _extract_block(txt, "unit_cell_cart")
    if cell_block is None:
        raise RuntimeError("Cannot find unit_cell_cart block in wannier90.win")

    lines = [ln.strip() for ln in cell_block.splitlines() if ln.strip()]
    if re.match(r"^(ang|angstrom|bohr)\b", lines[0], re.I):
        unit = lines[0].lower()
        vec_lines = lines[1:4]
    else:
        unit = "ang"
        vec_lines = lines[0:3]

    a1 = np.array([float(x) for x in vec_lines[0].split()[1:4]], dtype=float)
    a2 = np.array([float(x) for x in vec_lines[1].split()[1:4]], dtype=float)
    a3 = np.array([float(x) for x in vec_lines[2].split()[1:4]], dtype=float)

    if "bohr" in unit:
        bohr_to_ang = 0.52917721092
        a1 *= bohr_to_ang
        a2 *= bohr_to_ang
        a3 *= bohr_to_ang

    A = np.stack([a1, a2, a3], axis=1) # columns are lattice vectors

    atoms_block = _extract_block(txt, "atoms_cart")
    if atoms_block is None:
        raise RuntimeError("Cannot find atoms_cart block in wannier90.win")

    atoms = []
    for idx, ln in enumerate([x for x in atoms_block.splitlines() if x.strip()], start=1):
        parts = ln.split()
        if len(parts) < 4:
            continue
        elem = parts[0]
        r = np.array([float(parts[1]), float(parts[2]), float(parts[3])], dtype=float)
        atoms.append({"id": idx, "elem": elem, "r": r})

    if not atoms:
        raise RuntimeError("atoms_cart parsed but got 0 atoms")

    return A, atoms

# =====
# Parse centres.xyz (WF-only)
# =====
def parse_wf_centres_from_xyz(xyz_path, natoms, nwann):
    with open(xyz_path, "r", encoding="utf-8", errors="ignore") as f:

```

```

        lines = [ln.strip() for ln in f if ln.strip()]

N = int(lines[0])
if N < natoms + nwann:
    raise RuntimeError(f"centres.xyz N={N} < natoms+nwann={natoms+nwann}")

start = 2 + natoms
end = start + nwann
if len(lines) < end:
    raise RuntimeError(f"centres.xyz insufficient lines: need >= {end}, got {len(lines)}")

wf_centers = [None] * (nwann + 1)
for i in range(nwann):
    parts = lines[start + i].split()
    if len(parts) < 4:
        raise RuntimeError(f"Bad WF centre line: {lines[start+i]}")
    wf_centers[i + 1] = np.array([float(parts[1]), float(parts[2]), float(parts[3])], dtype=float)

return wf_centers

# =====
# Parse hr.dat
# =====
def read_hr_header_num_wann(hr_path):
    with open(hr_path, "r", encoding="utf-8", errors="ignore") as f:
        f.readline()
        return int(f.readline().strip())

def parse_hr_dat(hr_path):
    """
    Yield (Rx,Ry,Rz,m,n,reH,imH) from wannier90_hr.dat. m,n are 1-based.
    """
    with open(hr_path, "r", encoding="utf-8", errors="ignore") as f:
        _ = f.readline()
        _num_wann = int(f.readline().strip())
        nrpts = int(f.readline().strip())

        deg = []
        while len(deg) < nrpts:
            ln = f.readline()
            if not ln:
                raise RuntimeError("Unexpected EOF while reading degeneracy list")
            ln = ln.strip()
            if not ln:
                continue
            deg += [int(x) for x in ln.split()]

        for ln in f:
            ln = ln.strip()
            if not ln:
                continue
            parts = ln.split()
            if len(parts) < 7:
                continue
            Rx, Ry, Rz = int(parts[0]), int(parts[1]), int(parts[2])
            m, n = int(parts[3]), int(parts[4])
            reH, imH = float(parts[5]), float(parts[6])
            yield Rx, Ry, Rz, m, n, reH, imH

# =====
# Group/order helpers
# =====
def pair_name(g1, g2, order_map):
    o1 = order_map.get(g1, 999)
    o2 = order_map.get(g2, 999)
    if o1 < o2:
        return f"{g1}<->{g2}"
    if o2 < o1:
        return f"{g2}<->{g1}"

```

```

    return f"{min(g1, g2)}<->{max(g1, g2)}"

# =====
# Index-based WF mapping
# =====
D_ORBS = ["dxy", "dyz", "dxz", "dz2", "dx2-y2"]
P_ORBS = ["px", "py", "pz"]

def build_index_mapping(atoms, num_wann):
    if num_wann != 68:
        raise RuntimeError(
            f"Expected num_wann=68 for your system, but got {num_wann}. "
            f"If your projections changed, update index blocks accordingly."
        )

    idx_Tc = [i for i,a in enumerate(atoms) if a["elem"] == "Tc"]
    idx_Ir = [i for i,a in enumerate(atoms) if a["elem"] == "Ir"]
    idx_Se = [i for i,a in enumerate(atoms) if a["elem"] == "Se"]
    idx_Ge = [i for i,a in enumerate(atoms) if a["elem"] == "Ge"]

    if len(idx_Tc) != 1 or len(idx_Ir) != 1 or len(idx_Se) != 6 or len(idx_Ge) != 2:
        raise RuntimeError(
            f"Atom counts mismatch: Tc={len(idx_Tc)}, Ir={len(idx_Ir)}, "
            f"Se={len(idx_Se)}, Ge={len(idx_Ge)}. Check atoms_cart."
        )

    wf_atom = [None] * (num_wann + 1)
    wf_elem = [None] * (num_wann + 1)
    wf_group = [None] * (num_wann + 1)
    wf_orb = [None] * (num_wann + 1)

    for wf in range(1, 11):
        wf_atom[wf] = idx_Tc[0]
        wf_elem[wf] = "Tc"
        wf_group[wf] = "Tc_d"
        i = (wf - 1) // 2
        wf_orb[wf] = D_ORBS[i]

    for wf in range(11, 21):
        wf_atom[wf] = idx_Ir[0]
        wf_elem[wf] = "Ir"
        wf_group[wf] = "Ir_d"
        i = (wf - 11) // 2
        wf_orb[wf] = D_ORBS[i]

    start = 21
    for a_i in range(6):
        atom_idx = idx_Se[a_i]
        for local in range(6):
            wf = start + a_i * 6 + local
            wf_atom[wf] = atom_idx
            wf_elem[wf] = "Se"
            wf_group[wf] = "Se_p"
            j = local // 2
            wf_orb[wf] = P_ORBS[j]

    start = 57
    for a_i in range(2):
        atom_idx = idx_Ge[a_i]
        for local in range(6):
            wf = start + a_i * 6 + local
            wf_atom[wf] = atom_idx
            wf_elem[wf] = "Ge"
            wf_group[wf] = "Ge_p"
            j = local // 2
            wf_orb[wf] = P_ORBS[j]

    return wf_atom, wf_elem, wf_group, wf_orb

```

```

# =====
# Main
# =====
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--win", required=True, help="wannier90.win")
    ap.add_argument("--centres", required=True, help="wannier90_centres.xyz")
    ap.add_argument("--hr", required=True, help="wannier90_hr.dat")
    ap.add_argument("--tol", type=float, default=0.10, help="distance bin size (Ang)")
    ap.add_argument("--min_absH", type=float, default=1e-4, help="min |H| kept (eV)")
    ap.add_argument("--topk", type=int, default=30, help="topK edges per (pair,shell)")
    ap.add_argument("--skip_same_atom_R0", action="store_true")
    ap.add_argument("--skip_diag_R0", action="store_true")
    ap.add_argument("--pairs", default="ALL", help="comma-separated allowed pairs like Tc_d<->Se_p, or ALL")
    ap.add_argument("--out_summary", default="hopping_summary_by_shell.csv")
    ap.add_argument("--out_edges", default="", help="optional edges csv output")
    ap.add_argument("--group_order", default="Tc_d,Ir_d,Se_p,Ge_p,OTHER")

    args = ap.parse_args()

    order_list = [x.strip() for x in args.group_order.split(",") if x.strip()]
    order_map = {g: i for i, g in enumerate(order_list, start=1)}

    A, atoms = parse_win_lattice_and_atoms(args.win)
    natoms = len(atoms)
    num_wann = read_hr_header_num_wann(args.hr)
    wf_centers = parse_wf_centres_from_xyz(args.centres, natoms=natoms, nwann=num_wann)

    wf_atom_id, wf_elem, wf_group, wf_orb = build_index_mapping(atoms, num_wann)

    counts = defaultdict(int)
    for wf in range(1, num_wann + 1):
        counts[wf_elem[wf]] += 1
    print("[INFO] num_wann:", num_wann, " natoms:", natoms)
    print("[INFO] WF counts by elem (index-based):", dict(sorted(counts.items())))

    allowed_pairs = None
    if args.pairs.strip().upper() != "ALL":
        allowed_pairs = set([p.strip() for p in args.pairs.split(",") if p.strip()])

    cnt = defaultdict(int)
    sumsq = defaultdict(float)
    maxv = defaultdict(float)
    top_edges = defaultdict(list)
    filtered_edges = []

    a1 = A[:, 0]; a2 = A[:, 1]; a3 = A[:, 2]
    def R_to_T(Rx, Ry, Rz):
        return Rx * a1 + Ry * a2 + Rz * a3

    for Rx, Ry, Rz, m, n, reH, imH in parse_hr_dat(args.hr):
        absH = math.hypot(reH, imH)
        if absH < args.min_absH:
            continue

        atom_m = wf_atom_id[m]
        atom_n = wf_atom_id[n]
        elem_m = wf_elem[m]
        elem_n = wf_elem[n]
        gm = wf_group[m]
        gn = wf_group[n]
        if gm == "OTHER" or gn == "OTHER":
            continue

        pair = pair_name(gm, gn, order_map=order_map)
        if allowed_pairs is not None and pair not in allowed_pairs:
            continue

        if (Rx, Ry, Rz) == (0, 0, 0):

```

```

        if args.skip_diag_R0 and (m == n):
            continue
        if args.skip_same_atom_R0 and (atom_m == atom_n):
            continue

    T = R_to_T(Rx, Ry, Rz)
    dr = (wf_centers[n] + T) - wf_centers[m]
    dist = float(np.linalg.norm(dr))
    shell = round(dist / args.tol) * args.tol

    # complex phase
    phase_rad = math.atan2(imH, reH) if absH > 0 else 0.0

    key = (pair, shell)
    cnt[key] += 1
    sumsq[key] += absH * absH
    if absH > maxv[key]:
        maxv[key] = absH

    e = Edge(
        pair, shell, dist, absH, reH, imH, phase_rad,
        Rx, Ry, Rz, m, n,
        atom_m, atom_n, elem_m, elem_n,
        gm, gn, wf_orb[m], wf_orb[n]
    )

    lst = top_edges[key]
    lst.append(e)
    lst.sort(key=lambda x: x.absH, reverse=True)
    if len(lst) > args.topk:
        lst[:] = lst[:args.topk]

    if args.out_edges:
        filtered_edges.append(e)

# ---- summary ----
with open(args.out_summary, "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f)
    w.writerow([
        "pair", "shell_A", "count", "max_absH_eV", "rms_absH_eV",
        f"top{args.topk}_edges(m,n,atom_m,atom_n,group_m,group_n,orb_m,orb_n,R,absH,dist,phase)"
    ])
    for (pair, shell) in sorted(cnt.keys(), key=lambda x: (x[0], x[1])):
        c = cnt[(pair, shell)]
        rms = math.sqrt(sumsq[(pair, shell)] / c) if c else 0.0
        tops = top_edges[(pair, shell)]
        tops_str = "; ".join([
            f"{e.m}-{e.n}|a{e.atom_m+1}-a{e.atom_n+1}"
            f"|{e.group_m}-{e.group_n}|{e.orb_m}-{e.orb_n}"
            f"@({e.Rx},{e.Ry},{e.Rz})|{e.absH:.6g}|d={e.dist:.3f}|ph={e.phase_rad:.3f}"
            for e in tops
        ])
        w.writerow([pair, f"{shell:.3f}", c, f"{maxv[(pair, shell)]:.6g}", f"{rms:.6g}", tops_str])

# ---- edges ----
if args.out_edges:
    with open(args.out_edges, "w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow([
            "pair", "shell_A", "dist_A",
            "absH_eV", "reH_eV", "imH_eV", "phase_rad", "phase_deg",
            "Rx", "Ry", "Rz", "m", "n",
            "atom_m", "atom_n", "elem_m", "elem_n",
            "group_m", "group_n", "orb_m", "orb_n"
        ])
        for e in filtered_edges:
            w.writerow([
                e.pair, f"{e.shell:.3f}", f"{e.dist:.6f}",
                f"{e.absH:.10g}", f"{e.reH:.10g}", f"{e.imH:.10g}",

```

```

        f"{e.phase_rad:.10g}", f"{(e.phase_rad * 180.0 / math.pi):.10g}",
        e.Rx, e.Ry, e.Rz, e.m, e.n,
        e.atom_m + 1, e.atom_n + 1, e.elem_m, e.elem_n,
        e.group_m, e.group_n, e.orb_m, e.orb_n
    ])

    print("Done.")
    print("Summary:", args.out_summary)
    if args.out_edges:
        print("Edges :", args.out_edges)

if __name__ == "__main__":
    main()

```

► 代码清单 A.62.2 (shell, 8 行)

```

python lstep.py ^
--win wannier90.win ^
--centres wannier90_centres.xyz ^
--hr wannier90_hr.dat ^
--tol 0.10 --min_absH 1e-4 --topk 2000 ^
--skip_same_atom_R0 ^
--out_summary hopping_summary_by_shell.csv ^
--out_edges edges_with_orb_reim.csv

```

► 代码清单 A.62.3 (python, 361 行)

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import argparse
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

D_ORBS = ["dxy", "dyz", "dxz", "dz2", "dx2-y2"]
P_ORBS = ["px", "py", "pz"]

# ----- POSCAR -----
def read_poscar(path: str):
    lines = Path(path).read_text(encoding="utf-8", errors="ignore").splitlines()
    lines = [ln.strip() for ln in lines if ln.strip()]

    scale = float(lines[1].split()[0])
    a1 = np.array(list(map(float, lines[2].split()[3:]))) * scale
    a2 = np.array(list(map(float, lines[3].split()[3:]))) * scale
    a3 = np.array(list(map(float, lines[4].split()[3:]))) * scale
    A = np.stack([a1, a2, a3], axis=1) # columns

    species = lines[5].split()
    counts = list(map(int, lines[6].split()))
    if len(species) != len(counts):
        raise RuntimeError("POSCAR species/counts mismatch")

    idx = 7
    if lines[idx].lower().startswith("selective"):
        idx += 1

```

```

mode_line = lines[idx].lower()
if mode_line.startswith("d"):
    mode = "Direct"
elif mode_line.startswith("c") or mode_line.startswith("k"):
    mode = "Cartesian"
else:
    raise RuntimeError(f"Unknown coordinate mode: {lines[idx]}")
idx += 1

nat = sum(counts)
coords = np.array([list(map(float, lines[idx+i].split()[1:3])) for i in range(nat)], dtype=float)
if mode == "Cartesian":
    frac = np.linalg.solve(A, coords.T).T
else:
    frac = coords.copy()

elems_by_atom = []
for sp, c in zip(species, counts):
    elems_by_atom += [sp] * c

return A, np.asarray(frac), elems_by_atom

def bond_distance_with_R(A, frac, i0, j0, R):
    df = (frac[j0] + np.array(R, dtype=float)) - frac[i0]
    dr = A @ df
    return float(np.linalg.norm(dr))

# ----- plotting -----
def safe_mkdir(p: Path):
    p.mkdir(parents=True, exist_ok=True)

def heatmap_with_values(mat, xlabels, ylabels, title, out_png: Path, cbar_label="t_eff"):
    fig = plt.figure(figsize=(6.6, 5.0), dpi=180)
    ax = fig.add_subplot(111)
    im = ax.imshow(mat, aspect="auto", interpolation="nearest")

    ax.set_xticks(np.arange(len(xlabels)))
    ax.set_yticks(np.arange(len(ylabels)))
    ax.set_xticklabels(xlabels, rotation=45, ha="right")
    ax.set_yticklabels(ylabels)
    ax.set_title(title)

    cbar = fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
    cbar.set_label(cbar_label)

    finite = np.isfinite(mat)
    vmax = float(np.max(mat[finite])) if np.any(finite) else 1.0
    thresh = 0.55 * vmax

    for i in range(mat.shape[0]):
        for j in range(mat.shape[1]):
            v = mat[i, j]
            s = "0" if abs(v) < 1e-15 else f"{v:.3g}"
            color = "white" if v >= thresh else "black"
            ax.text(j, i, s, ha="center", va="center", color=color, fontsize=9)

    ax.set_xticks(np.arange(-.5, len(xlabels), 1), minor=True)
    ax.set_yticks(np.arange(-.5, len(ylabels), 1), minor=True)
    ax.grid(which="minor", linestyle="-", linewidth=0.6)
    ax.tick_params(which="minor", bottom=False, left=False)

    fig.tight_layout()
    fig.savefig(out_png, bbox_inches="tight")
    plt.close(fig)

# ----- orientation / filter -----
def orient_to_AB(df, A, B):
    """keep (A,B) or (B,A), swap BA->AB so elem_m=A, elem_n=B"""

```

```

df = df.copy()
mAB = (df["elem_m"] == A) & (df["elem_n"] == B)
mBA = (df["elem_m"] == B) & (df["elem_n"] == A)
df_AB = df.loc[mAB].copy()
df_BA = df.loc[mBA].copy()

if len(df_BA) > 0:
    for c1, c2 in [("m", "n"), ("atom_m", "atom_n"), ("elem_m", "elem_n"), ("orb_m", "orb_n")]:
        tmp = df_BA[c1].copy()
        df_BA[c1] = df_BA[c2].values
        df_BA[c2] = tmp.values

return pd.concat([df_AB, df_BA], ignore_index=True)

def filter_kind(df, kind):
    D = set(D_ORBS); P = set(P_ORBS)
    if kind == "dd":
        return df[df["orb_m"].isin(D) & df["orb_n"].isin(D)].copy(), D_ORBS, D_ORBS
    if kind == "pp":
        return df[df["orb_m"].isin(P) & df["orb_n"].isin(P)].copy(), P_ORBS, P_ORBS
    # dp after AB: m is d, n is p
    return df[df["orb_m"].isin(D) & df["orb_n"].isin(P)].copy(), D_ORBS, P_ORBS

# ----- SOC pairing & t_eff -----
def build_spin_of_wf(dfAB, strict=False):
    """
    Determine SOC Kramers pair (2 WFs) for each (atom_id, spatial_orb).
    Assign spin index 0/1 by sorting WF ids.
    """
    groups = {}
    for side in ["m", "n"]:
        wf = dfAB[side].astype(int).to_numpy()
        atom = dfAB[f"atom_{side}"].astype(int).to_numpy()
        orb = dfAB[f"orb_{side}"].astype(str).to_numpy()
        for w,a,o in zip(wf, atom, orb):
            key = (int(a), str(o))
            groups.setdefault(key, set()).add(int(w))

    spin_of_wf = {}
    bad = []
    for (a,o), s in groups.items():
        wfs = sorted(list(s))
        if len(wfs) != 2:
            bad.append((a,o), wfs)
            continue
        spin_of_wf[wfs[0]] = 0
        spin_of_wf[wfs[1]] = 1

    if bad:
        msg = ["[WARN] Some (atom,orb) do not have exactly 2 WFs (SOC)."]
        for (a,o), wfs in bad[:30]:
            msg.append(f"  atom={a}, orb={o}, wfs={wfs}")
        msg.append("  (showing up to 30)")
        print("\n".join(msg))
        if strict:
            raise RuntimeError("strict_spinor enabled: found (atom,orb) with !=2 WFs.")

    return spin_of_wf

def teff_block(df_block, spin_of_wf):
    """
    df_block: fixed bond image + fixed (orb_m, orb_n).
    Construct 2x2 complex H and compute Frobenius norm.
    """
    H = np.zeros((2,2), dtype=np.complex128)
    for _, r in df_block.iterrows():
        m = int(r["m"]); n = int(r["n"])
        sm = spin_of_wf.get(m, None)
        sn = spin_of_wf.get(n, None)

```

```

        if sm is None or sn is None:
            continue
        H[sm, sn] = complex(float(r["reH_eV"]), float(r["imH_eV"]))
    return float(np.sqrt(np.sum(np.abs(H)**2)))

def build_teff_matrix_for_bond(df_bond, A_orbs, B_orbs, spin_of_wf):
    mat = np.zeros((len(A_orbs), len(B_orbs)), dtype=float)
    ia = {o:i for i,o in enumerate(A_orbs)}
    ib = {o:i for i,o in enumerate(B_orbs)}
    for (om, on), g in df_bond.groupby(["orb_m", "orb_n"]):
        if om not in ia or on not in ib:
            continue
        mat[ia[om], ib[on]] = teff_block(g, spin_of_wf)
    return mat

def long_table_orb_pairs(df_bond, spin_of_wf):
    """
    Produce a long table like paper: each row is (orb_m, orb_n, t_eff, and the 2x2 complex matrix entries).
    """
    rows = []
    for (om, on), g in df_bond.groupby(["orb_m", "orb_n"]):
        H = np.zeros((2,2), dtype=np.complex128)
        for _, r in g.iterrows():
            m = int(r["m"]); n = int(r["n"])
            sm = spin_of_wf.get(m, None)
            sn = spin_of_wf.get(n, None)
            if sm is None or sn is None:
                continue
            H[sm, sn] = complex(float(r["reH_eV"]), float(r["imH_eV"]))
        teff = float(np.sqrt(np.sum(np.abs(H)**2)))
        rows.append({
            "orb_m": om, "orb_n": on, "t_eff": teff,
            "H00_re": H[0,0].real, "H00_im": H[0,0].imag,
            "H01_re": H[0,1].real, "H01_im": H[0,1].imag,
            "H10_re": H[1,0].real, "H10_im": H[1,0].imag,
            "H11_re": H[1,1].real, "H11_im": H[1,1].imag,
        })
    dfL = pd.DataFrame(rows)
    if not dfL.empty:
        dfL.sort_values("t_eff", ascending=False, inplace=True)
    return dfL

# ----- main -----
def main():
    ap = argparse.ArgumentParser(
        description="One-shot, maximum literature-like reproduction of Fig.5(b)/(d): bond-resolved NN selection +
        ↪ SOC 2x2 spinor Frobenius t_eff + outputs for plotting."
    )
    ap.add_argument("--poscar", required=True)
    ap.add_argument("--edges", required=True, help="edges_with_orb_reim.csv (must include reH_eV/imH_eV)")
    ap.add_argument("--outdir", required=True)

    ap.add_argument("--A", required=True, help="e.g. Tc")
    ap.add_argument("--B", required=True, help="e.g. Se")
    ap.add_argument("--kind", default="dp", choices=["dd", "dp", "pp"])

    ap.add_argument("--dmin", type=float, required=True, help="NN distance window min (Å)")
    ap.add_argument("--dmax", type=float, required=True, help="NN distance window max (Å)")

    ap.add_argument("--min_absH", type=float, default=0.0, help="pre-filter edges by absH_eV")
    ap.add_argument("--top_bonds", type=int, default=10, help="export top N NN bond-images")
    ap.add_argument("--strict_spinor", action="store_true", help="require exactly 2 WFs per (atom,orb)")

    args = ap.parse_args()

    outdir = Path(args.outdir)
    safe_mkdir(outdir)

    A_lat, frac, elems = read_poscar(args.poscar)

```

```

nat = len(elems)

df = pd.read_csv(args.edges)
need = ["elem_m", "elem_n", "atom_m", "atom_n", "orb_m", "orb_n", "absH_eV",
        "reH_eV", "imH_eV", "Rx", "Ry", "Rz", "m", "n"]
miss = [c for c in need if c not in df.columns]
if miss:
    raise ValueError(f"edges missing columns: {miss}")

if args.min_absH > 0:
    df = df[df["absH_eV"] >= args.min_absH].copy()

# orient + kind filter
dfAB = orient_to_AB(df, args.A, args.B)
dfAB, A_orbs, B_orbs = filter_kind(dfAB, args.kind)
if dfAB.empty:
    raise RuntimeError("No edges after A/B/kind filtering.")

# SOC pairing
spin_of_wf = build_spin_of_wf(dfAB, strict=args.strict_spinor)

# compute bond distance per edge (atom_m -> atom_n + R)
am = dfAB["atom_m"].astype(int).to_numpy()
an = dfAB["atom_n"].astype(int).to_numpy()
Rx = dfAB["Rx"].astype(int).to_numpy()
Ry = dfAB["Ry"].astype(int).to_numpy()
Rz = dfAB["Rz"].astype(int).to_numpy()

if am.min() < 1 or an.min() < 1 or am.max() > nat or an.max() > nat:
    raise RuntimeError("atom indices out of POSCAR range")

d_atom = np.empty(len(dfAB), dtype=float)
for i in range(len(dfAB)):
    d_atom[i] = bond_distance_with_R(A_lat, frac, am[i]-1, an[i]-1, (Rx[i], Ry[i], Rz[i]))

dfAB = dfAB.copy()
dfAB["d_atom_R_A"] = d_atom

# NN window selection (paper-like)
dfNN = dfAB[(dfAB["d_atom_R_A"] >= args.dmin) & (dfAB["d_atom_R_A"] <= args.dmax)].copy()
if dfNN.empty:
    raise RuntimeError("No edges in NN window. Widen dmin/dmax or check POSCAR/edges.")

# bond-image identity
bond_cols = ["atom_m", "atom_n", "Rx", "Ry", "Rz"]
bond_rows = []
mats_cache = {}

for key, g in dfNN.groupby(bond_cols):
    mat = build_teff_matrix_for_bond(g, A_orbs, B_orbs, spin_of_wf)
    score = float(mat.sum())
    dval = float(g["d_atom_R_A"].mean())
    bond_rows.append((*key, dval, len(g), score))
    mats_cache[key] = (mat, g)

bond_sum = pd.DataFrame(
    bond_rows,
    columns=["atom_m", "atom_n", "Rx", "Ry", "Rz", "d_atom_A", "n_edges", "sum_teff"]
).sort_values("sum_teff", ascending=False)

# export global ranking (this is your "choose representative bond like paper")
bond_rank = outdir / f"BONDS_{args.A}-{args.B}_{args.kind}_TEFF_d{args.dmin}-{args.dmax}.csv"
bond_sum.to_csv(bond_rank, index=False)

# export top bonds with full fig5-like package
topN = min(args.top_bonds, len(bond_sum))
summary = []

for bi in range(topN):

```

```

r = bond_sum.iloc[bi]
key = (int(r["atom_m"]), int(r["atom_n"]), int(r["Rx"]), int(r["Ry"]), int(r["Rz"]))
mat, g = mats_cache[key]
a_m, a_n, Rx0, Ry0, Rz0 = key
dval = float(r["d_atom_A"])

tag = f"{args.A}{a_m}-{args.B}{a_n}_R{Rx0}{Ry0}{Rz0}_{args.kind}_TEFF"
bdir = outdir / tag
safe_mkdir(bdir)

# (1) matrix + heatmap (paper-like panel)
pd.DataFrame(mat, index=A_orbs, columns=B_orbs).to_csv(bdir / f"{tag}_matrix.csv")
title = f"{tag} | d={dval:.3f}Å |  $\Sigma t_{\text{eff}}=\{\text{mat.sum}():.4g\}$ "
heatmap_with_values(mat, B_orbs, A_orbs, title, bdir / f"{tag}.png",
                    cbar_label=r"$t^{\mathrm{eff}}=\lvert H^2\times 2\rvert\_F$ (eV)")

# (2) long-table orbital pairs (paper-like list)
dfl = long_table_orb_pairs(g, spin_of_wf)
dfl.to_csv(bdir / f"{tag}_orbital_pairs_long.csv", index=False)

# (3) top pairs list
flat = []
for i, ao in enumerate(A_orbs):
    for j, bo in enumerate(B_orbs):
        flat.append((ao, bo, mat[i, j]))
flat.sort(key=lambda x: x[2], reverse=True)
pd.DataFrame(flat, columns=["A_orb", "B_orb", "t_eff"]).to_csv(
    bdir / f"{tag}_orbital_pairs_matrix_entries.csv", index=False
)

# (4) raw edges used (traceability)
g.to_csv(bdir / f"{tag}_edges.csv", index=False)

summary.append({
    "rank": bi+1,
    "bond_tag": tag,
    "atom_m": a_m, "atom_n": a_n,
    "Rx": Rx0, "Ry": Ry0, "Rz": Rz0,
    "d_atom_A": dval,
    "sum_teff": float(mat.sum()),
    "max_teff": float(mat.max()),
    "argmax_pair": f"{flat[0][0]}-{flat[0][1]}" if flat else "NA",
})

pd.DataFrame(summary).to_csv(outdir / f"SUMMARY_top{topN}_bonds_TEFF.csv", index=False)

print(f"[OK] NN edges: {len(dfNN)} in window [{args.dmin},{args.dmax}] Å")
print(f"[OK] unique NN bond-images: {len(bond_sum)}")
print(f"[OK] bond ranking: {bond_rank}")
print(f"[OK] exported top bond packages: {topN}")
print(f"[OK] summary: {outdir / f'SUMMARY_top{topN}_bonds_TEFF.csv'}")

if __name__ == "__main__":
    main()

```

► 代码清单 A.62.4 (shell, 8 行)

```

python paper_fig5_like_soc.py ^
--poscar POSCAR ^
--edges edges_with_orb_reim.csv ^
--outdir Fig5_like_TcSe ^
--A Tc --B Se --kind dp ^

```

```
--dmin 2.43 --dmax 2.73 ^
--top_bonds 6 ^
--strict_spinor
```

► 代码清单 A.62.5 (python, 384 行)

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

"""
paper_fig5_like_soc.py (final, with --unique_bonds)

One-shot, maximum literature-like reproduction of Fig.5(b)/(d):
- bond-resolved NN selection by atom distance window (POSCAR + Rx,Ry,Rz)
- SOC spinor 2x2 block -> Frobenius norm t_eff
- export: matrix heatmap + long-table + raw edges for top bond-images
- NEW: --unique_bonds to avoid double counting when A==B (e.g., Tc-Tc)
"""

import argparse
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

D_ORBS = ["dxy", "dyz", "dxz", "dz2", "dx2-y2"]
P_ORBS = ["px", "py", "pz"]

# ----- POSCAR -----
def read_poscar(path: str):
    lines = Path(path).read_text(encoding="utf-8", errors="ignore").splitlines()
    lines = [ln.strip() for ln in lines if ln.strip()]

    scale = float(lines[1].split()[0])
    a1 = np.array(list(map(float, lines[2].split()[1:3]))) * scale
    a2 = np.array(list(map(float, lines[3].split()[1:3]))) * scale
    a3 = np.array(list(map(float, lines[4].split()[1:3]))) * scale
    A = np.stack([a1, a2, a3], axis=1) # columns

    species = lines[5].split()
    counts = list(map(int, lines[6].split()))
    if len(species) != len(counts):
        raise RuntimeError("POSCAR species/counts mismatch")

    idx = 7
    if lines[idx].lower().startswith("selective"):
        idx += 1

    mode_line = lines[idx].lower()
    if mode_line.startswith("d"):
        mode = "Direct"
    elif mode_line.startswith("c") or mode_line.startswith("k"):
        mode = "Cartesian"
    else:
        raise RuntimeError(f"Unknown coordinate mode: {lines[idx]}")
    idx += 1

    nat = sum(counts)
    coords = np.array([list(map(float, lines[idx+i].split()[1:3])) for i in range(nat)], dtype=float)
    if mode == "Cartesian":
        frac = np.linalg.solve(A, coords.T).T
    else:
        frac = coords.copy()
```

```

elems_by_atom = []
for sp, c in zip(species, counts):
    elems_by_atom += [sp] * c

return A, np.asarray(frac), elems_by_atom

def bond_distance_with_R(A, frac, i0, j0, R):
    df = (frac[j0] + np.array(R, dtype=float)) - frac[i0]
    dr = A @ df
    return float(np.linalg.norm(dr))

# ----- plotting -----
def safe_mkdir(p: Path):
    p.mkdir(parents=True, exist_ok=True)

def heatmap_with_values(mat, xlabels, ylabels, title, out_png: Path, cbar_label="t_eff"):
    fig = plt.figure(figsize=(6.6, 5.0), dpi=180)
    ax = fig.add_subplot(111)
    im = ax.imshow(mat, aspect="auto", interpolation="nearest")

    ax.set_xticks(np.arange(len(xlabels)))
    ax.set_yticks(np.arange(len(ylabels)))
    ax.set_xticklabels(xlabels, rotation=45, ha="right")
    ax.set_yticklabels(ylabels)
    ax.set_title(title)

    cbar = fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
    cbar.set_label(cbar_label)

    finite = np.isfinite(mat)
    vmax = float(np.max(mat[finite])) if np.any(finite) else 1.0
    thresh = 0.55 * vmax

    for i in range(mat.shape[0]):
        for j in range(mat.shape[1]):
            v = mat[i, j]
            s = "0" if abs(v) < 1e-15 else f"{v:.3g}"
            color = "white" if v >= thresh else "black"
            ax.text(j, i, s, ha="center", va="center", color=color, fontsize=9)

    ax.set_xticks(np.arange(-.5, len(xlabels), 1), minor=True)
    ax.set_yticks(np.arange(-.5, len(ylabels), 1), minor=True)
    ax.grid(which="minor", linestyle="-", linewidth=0.6)
    ax.tick_params(which="minor", bottom=False, left=False)

    fig.tight_layout()
    fig.savefig(out_png, bbox_inches="tight")
    plt.close(fig)

# ----- orientation / filter -----
def orient_to_AB(df, A, B):
    """
    Keep (A,B) or (B,A). Swap BA->AB so elem_m=A, elem_n=B.
    Note: If A=B, this will keep both directions as-is; use --unique_bonds to avoid double counting.
    """
    df = df.copy()
    mAB = (df["elem_m"] == A) & (df["elem_n"] == B)
    mBA = (df["elem_m"] == B) & (df["elem_n"] == A)
    df_AB = df.loc[mAB].copy()
    df_BA = df.loc[mBA].copy()

    if len(df_BA) > 0 and A != B:
        for c1, c2 in [("m", "n"), ("atom_m", "atom_n"), ("elem_m", "elem_n"), ("orb_m", "orb_n")]:
            tmp = df_BA[c1].copy()
            df_BA[c1] = df_BA[c2].values
            df_BA[c2] = tmp.values

    return pd.concat([df_AB, df_BA], ignore_index=True)

```

```

def filter_kind(df, kind):
    D = set(D_ORBS); P = set(P_ORBS)
    if kind == "dd":
        return df[df["orb_m"].isin(D) & df["orb_n"].isin(D)].copy(), D_ORBS, D_ORBS
    if kind == "pp":
        return df[df["orb_m"].isin(P) & df["orb_n"].isin(P)].copy(), P_ORBS, P_ORBS
    # dp after AB: m is d, n is p
    return df[df["orb_m"].isin(D) & df["orb_n"].isin(P)].copy(), D_ORBS, P_ORBS

# ----- SOC pairing & t_eff -----
def build_spin_of_wf(dfAB, strict=False):
    """
    Determine SOC Kramers pair (2 WFs) for each (atom_id, spatial_orb).
    Assign spin index 0/1 by sorting WF ids.
    """
    groups = {}
    for side in ["m", "n"]:
        wf = dfAB[side].astype(int).to_numpy()
        atom = dfAB[f"atom_{side}"].astype(int).to_numpy()
        orb = dfAB[f"orb_{side}"].astype(str).to_numpy()
        for w, a, o in zip(wf, atom, orb):
            key = (int(a), str(o))
            groups.setdefault(key, set()).add(int(w))

    spin_of_wf = {}
    bad = []
    for (a, o), s in groups.items():
        wfs = sorted(list(s))
        if len(wfs) != 2:
            bad.append((a, o), wfs)
            continue
        spin_of_wf[wfs[0]] = 0
        spin_of_wf[wfs[1]] = 1

    if bad:
        msg = ["[WARN] Some (atom,orb) do not have exactly 2 WFs (SOC)."]
        for (a, o), wfs in bad[:30]:
            msg.append(f"  atom={a}, orb={o}, wfs={wfs}")
        msg.append("  (showing up to 30)")
        print("\n".join(msg))
        if strict:
            raise RuntimeError("strict_spinor enabled: found (atom,orb) with !=2 WFs.")

    return spin_of_wf

def teff_block(df_block, spin_of_wf):
    """
    df_block: fixed bond image + fixed (orb_m, orb_n).
    Construct 2x2 complex H and compute Frobenius norm.
    """
    H = np.zeros((2, 2), dtype=np.complex128)
    for _, r in df_block.iterrows():
        m = int(r["m"]); n = int(r["n"])
        sm = spin_of_wf.get(m, None)
        sn = spin_of_wf.get(n, None)
        if sm is None or sn is None:
            continue
        H[sm, sn] = complex(float(r["reH_eV"]), float(r["imH_eV"]))
    return float(np.sqrt(np.sum(np.abs(H) ** 2)))

def build_teff_matrix_for_bond(df_bond, A_orbs, B_orbs, spin_of_wf):
    mat = np.zeros((len(A_orbs), len(B_orbs)), dtype=float)
    ia = {o: i for i, o in enumerate(A_orbs)}
    ib = {o: i for i, o in enumerate(B_orbs)}
    for (om, on), g in df_bond.groupby(["orb_m", "orb_n"]):
        if om not in ia or on not in ib:
            continue
        mat[ia[om], ib[on]] = teff_block(g, spin_of_wf)

```

```

    return mat

def long_table_orb_pairs(df_bond, spin_of_wf):
    """
    Long table: each row is (orb_m, orb_n, t_eff, and the 2x2 complex matrix entries).
    """
    rows = []
    for (om, on), g in df_bond.groupby(["orb_m", "orb_n"]):
        H = np.zeros((2, 2), dtype=np.complex128)
        for _, r in g.iterrows():
            m = int(r["m"]); n = int(r["n"])
            sm = spin_of_wf.get(m, None)
            sn = spin_of_wf.get(n, None)
            if sm is None or sn is None:
                continue
            H[sm, sn] = complex(float(r["reH_eV"]), float(r["imH_eV"]))
            teff = float(np.sqrt(np.sum(np.abs(H) ** 2)))
            rows.append({
                "orb_m": om, "orb_n": on, "t_eff": teff,
                "H00_re": H[0, 0].real, "H00_im": H[0, 0].imag,
                "H01_re": H[0, 1].real, "H01_im": H[0, 1].imag,
                "H10_re": H[1, 0].real, "H10_im": H[1, 0].imag,
                "H11_re": H[1, 1].real, "H11_im": H[1, 1].imag,
            })
    dfl = pd.DataFrame(rows)
    if not dfl.empty:
        dfl.sort_values("t_eff", ascending=False, inplace=True)
    return dfl

# ----- main -----
def main():
    ap = argparse.ArgumentParser(
        description="Fig.5-like (bond-resolved) orbital hybridization with SOC t_eff (Frobenius norm of 2x2 spinor
        ↪ block).")
    )
    ap.add_argument("--poscar", required=True)
    ap.add_argument("--edges", required=True, help="edges_with_orb_reim.csv (must include reH_eV/imH_eV)")
    ap.add_argument("--outdir", required=True)

    ap.add_argument("--A", required=True, help="e.g. Tc")
    ap.add_argument("--B", required=True, help="e.g. Se")
    ap.add_argument("--kind", default="dp", choices=["dd", "dp", "pp"])

    ap.add_argument("--dmin", type=float, required=True, help="NN distance window min (Å)")
    ap.add_argument("--dmax", type=float, required=True, help="NN distance window max (Å)")

    ap.add_argument("--min_absH", type=float, default=0.0, help="pre-filter edges by absH_eV")
    ap.add_argument("--top_bonds", type=int, default=10, help="export top N NN bond-images")
    ap.add_argument("--strict_spinor", action="store_true", help="require exactly 2 WFs per (atom,orb)")
    # ---- NEW ----
    ap.add_argument("--unique_bonds", action="store_true",
        help="When A==B, keep only atom_m < atom_n to avoid double counting.")

    args = ap.parse_args()

    outdir = Path(args.outdir)
    safe_mkdir(outdir)

    A_lat, frac, elems = read_poscar(args.poscar)
    nat = len(elems)

    df = pd.read_csv(args.edges)
    need = ["elem_m", "elem_n", "atom_m", "atom_n", "orb_m", "orb_n", "absH_eV",
            "reH_eV", "imH_eV", "Rx", "Ry", "Rz", "m", "n"]
    miss = [c for c in need if c not in df.columns]
    if miss:
        raise ValueError(f"edges missing columns: {miss}")

    if args.min_absH > 0:

```

```

    df = df[df["absH_eV"] >= args.min_absH].copy()

# orient + kind filter
dfAB = orient_to_AB(df, args.A, args.B)
dfAB, A_orbs, B_orbs = filter_kind(dfAB, args.kind)
if dfAB.empty:
    raise RuntimeError("No edges after A/B/kind filtering.")

# SOC pairing
spin_of_wf = build_spin_of_wf(dfAB, strict=args.strict_spinor)

# compute atom distance per edge (atom_m -> atom_n + R)
am = dfAB["atom_m"].astype(int).to_numpy()
an = dfAB["atom_n"].astype(int).to_numpy()
Rx = dfAB["Rx"].astype(int).to_numpy()
Ry = dfAB["Ry"].astype(int).to_numpy()
Rz = dfAB["Rz"].astype(int).to_numpy()

if am.min() < 1 or an.min() < 1 or am.max() > nat or an.max() > nat:
    raise RuntimeError("atom indices out of POSCAR range")

d_atom = np.empty(len(dfAB), dtype=float)
for i in range(len(dfAB)):
    d_atom[i] = bond_distance_with_R(A_lat, frac, am[i]-1, an[i]-1, (Rx[i], Ry[i], Rz[i]))

dfAB = dfAB.copy()
dfAB["d_atom_R_A"] = d_atom

# NN window selection (paper-like)
dfNN = dfAB[(dfAB["d_atom_R_A"] >= args.dmin) & (dfAB["d_atom_R_A"] <= args.dmax)].copy()
if dfNN.empty:
    raise RuntimeError("No edges in NN window. Widen dmin/dmax or check POSCAR/edges.")

# ---- NEW: avoid double counting for A==B ----
if args.unique_bonds and args.A == args.B:
    dfNN = dfNN[dfNN["atom_m"].astype(int) < dfNN["atom_n"].astype(int)].copy()
    if dfNN.empty:
        raise RuntimeError("After --unique_bonds filtering, no edges left. Check distance window.")

# bond-image identity
bond_cols = ["atom_m", "atom_n", "Rx", "Ry", "Rz"]
bond_rows = []
mats_cache = {}

for key, g in dfNN.groupby(bond_cols):
    mat = build_teff_matrix_for_bond(g, A_orbs, B_orbs, spin_of_wf)
    score = float(mat.sum())
    dval = float(g["d_atom_R_A"].mean())
    bond_rows.append((*key, dval, len(g), score))
    mats_cache[key] = (mat, g)

bond_sum = pd.DataFrame(
    bond_rows,
    columns=["atom_m", "atom_n", "Rx", "Ry", "Rz", "d_atom_A", "n_edges", "sum_teff"]
).sort_values("sum_teff", ascending=False)

# export global ranking
bond_rank = outdir / f"BONDS_{args.A}-{args.B}_{args.kind}_TEFF_d{args.dmin}-{args.dmax}.csv"
bond_sum.to_csv(bond_rank, index=False)

# export top bonds
topN = min(args.top_bonds, len(bond_sum))
summary = []

for bi in range(topN):
    r = bond_sum.iloc[bi]
    key = (int(r["atom_m"]), int(r["atom_n"]), int(r["Rx"]), int(r["Ry"]), int(r["Rz"]))
    mat, g = mats_cache[key]
    a_m, a_n, Rx0, Ry0, Rz0 = key

```

```

dval = float(r["d_atom_A"])

tag = f"{args.A}-{args.B}-{args.n}_R{Rx0}{Ry0}{Rz0}_{args.kind}_TEFF"
bdir = outdir / tag
safe_mkdir(bdir)

# (1) matrix + heatmap
pd.DataFrame(mat, index=A_orbs, columns=B_orbs).to_csv(bdir / f"{tag}_matrix.csv")
title = f"{tag} | d={dval:.3f}Å |  $\Sigma t_{\text{eff}}={\text{mat.sum():.4g}}$ "
heatmap_with_values(
    mat, B_orbs, A_orbs, title, bdir / f"{tag}.png",
    cbar_label=r"$t^{\mathrm{eff}}=\lvert H^{2\times 2}\rvert_F$ (eV)"
)

# (2) long-table orbital pairs
dfl = long_table_orb_pairs(g, spin_of_wf)
dfl.to_csv(bdir / f"{tag}_orbital_pairs_long.csv", index=False)

# (3) flat list from matrix (all entries)
flat = []
for i, ao in enumerate(A_orbs):
    for j, bo in enumerate(B_orbs):
        flat.append((ao, bo, mat[i, j]))
flat.sort(key=lambda x: x[2], reverse=True)
pd.DataFrame(flat, columns=["A_orb", "B_orb", "t_eff"]).to_csv(
    bdir / f"{tag}_orbital_pairs_matrix_entries.csv", index=False
)

# (4) raw edges used
g.to_csv(bdir / f"{tag}_edges.csv", index=False)

summary.append({
    "rank": bi + 1,
    "bond_tag": tag,
    "atom_m": a_m, "atom_n": a_n,
    "Rx": Rx0, "Ry": Ry0, "Rz": Rz0,
    "d_atom_A": dval,
    "sum_teff": float(mat.sum()),
    "max_teff": float(mat.max()),
    "argmax_pair": f"{flat[0][0]}-{flat[0][1]}" if flat else "NA",
})

pd.DataFrame(summary).to_csv(outdir / f"SUMMARY_top{topN}_bonds_TEFF.csv", index=False)

print(f"[OK] NN edges: {len(dfNN)} in window [{args.dmin},{args.dmax}] Å")
print(f"[OK] unique NN bond-images: {len(bond_sum)}")
print(f"[OK] bond ranking: {bond_rank}")
print(f"[OK] exported top bond packages: {topN}")
print(f"[OK] summary: {outdir / f'SUMMARY_top{topN}_bonds_TEFF.csv'}")

if __name__ == "__main__":
    main()

```

(原创脚本) 任意体系的磁参数 J 的磁构型设计与计算

► 代码清单 A.68.1 (python, 676 行)

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""

```

Generate Heisenberg energy-mapping coefficients for single-layer or bilayer magnetic systems.

Hamiltonian convention used here:

$$E = E_0 - \sum_{\langle i,j \rangle, \text{shell/type}} J_{\text{type_shell}} * S_i * S_j$$

where $S_i = +1$ or -1 for collinear magnetic configurations.

For every generated magnetic configuration, the coefficient of a J parameter is:

$$c = - \sum_{\{\text{bonds in that pair type and neighbour shell}\}} S_i * S_j$$

The script:

1. Reads a VASP POSCAR/CONTCAR.
2. Selects magnetic atoms by element or explicit POSCAR indices.
3. Finds unique magnetic bonds under periodic boundary conditions.
4. Splits bonds into neighbour shells separately for each pair type.
5. Generates linearly independent collinear spin configurations.
6. Writes the coefficient matrix and equations for fitting J.

Typical use for a Cr/Ru bilayer heterostructure:

```
python3 generate_J_mapping_coefficients.py POSCAR \  
--mag-elements Cr Ru \  
--group-by element \  
--max-shells 3 \  
--pbc 1 1 0 \  
--n-configs 12 \  
--outdir J_fit_output
```

Typical use for a single magnetic layer with one magnetic element M:

```
python3 generate_J_mapping_coefficients.py POSCAR \  
--mag-elements Cr \  
--group-by element \  
--max-shells 3 \  
--pbc 1 1 0 \  
--n-configs 8 \  
--outdir J_fit_output
```

Author: generated for atomistic spin-model J fitting workflows. (Ruixiao Ma)
"""

```
import argparse  
import csv  
import itertools  
import json  
import math  
import os  
import sys  
from dataclasses import dataclass  
from typing import Dict, List, Tuple, Optional  
  
import numpy as np  
  
@dataclass  
class Atom:  
    index0: int          # 0-based POSCAR atom index  
    element: str  
    frac: np.ndarray     # fractional coordinates, wrapped to [0,1)  
    cart: np.ndarray     # Cartesian coordinates in Angstrom  
    layer: str = "L1"  
  
@dataclass  
class Bond:  
    i: int               # index in mag_atoms list  
    j: int               # index in mag_atoms list  
    shift: Tuple[int, int, int]  
    distance: float  
    pair_label: str  
    shell: int = 0
```

```

def parse_poscar(path: str):
    with open(path, "r", encoding="utf-8", errors="ignore") as f:
        lines = [line.rstrip() for line in f if line.strip()]
    if len(lines) < 8:
        raise ValueError("POSCAR seems too short.")

    scale = float(lines[1].split()[0])
    lattice = np.array([[float(x) for x in lines[i].split()[1:3]] for i in range(2, 5)], dtype=float)
    lattice *= scale

    # VASP5: line 5 elements, line 6 counts. VASP4 has no element names.
    line5 = lines[5].split()
    try:
        counts = [int(x) for x in line5]
        elements = [f"X{i+1}" for i in range(len(counts))]
        coord_line_idx = 6
    except ValueError:
        elements = line5
        counts = [int(x) for x in lines[6].split()]
        coord_line_idx = 7

    if lines[coord_line_idx].lower().startswith("s"):
        coord_line_idx += 1

    coord_type = lines[coord_line_idx].strip().lower()[0]
    coord_start = coord_line_idx + 1
    natoms = sum(counts)
    coord_lines = lines[coord_start:coord_start + natoms]
    if len(coord_lines) < natoms:
        raise ValueError("Not enough coordinate lines in POSCAR.")

    inv_lattice = np.linalg.inv(lattice)
    atoms: List[Atom] = []
    idx = 0
    for el, num in zip(elements, counts):
        for _ in range(num):
            vals = [float(x) for x in coord_lines[idx].split()[1:3]]
            arr = np.array(vals, dtype=float)
            if coord_type in ["d", "f"]: # Direct/Fractional
                frac = arr.copy()
                cart = frac @ lattice
            elif coord_type in ["c", "k"]: # Cartesian/Kpoints style label in VASP
                cart = arr * scale if abs(scale - 1.0) > 1e-12 else arr
                frac = cart @ inv_lattice
            else:
                raise ValueError(f"Unknown coordinate type line: {lines[coord_line_idx]}")
            frac_wrapped = frac - np.floor(frac)
            cart_wrapped = frac_wrapped @ lattice
            atoms.append(Atom(index=idx, element=el, frac=frac_wrapped, cart=cart_wrapped))
            idx += 1

    return lattice, elements, counts, atoms

def parse_index_list(s: str) -> List[int]:
    """Parse comma-separated 1-based or 0-based indices. Ranges allowed, e.g. 1-4,7,8."""
    out = []
    for part in s.replace(" ", "").split(","):
        if not part:
            continue
        if "-" in part:
            a, b = part.split("-", 1)
            a, b = int(a), int(b)
            out.extend(range(a, b + 1))
        else:
            out.append(int(part))
    return out

```

```

def assign_layers_by_z(mag_atoms: List[Atom], n_layers: int):
    if n_layers <= 1:
        for a in mag_atoms:
            a.layer = "L1"
        return
    zvals = np.array([a.frac[2] for a in mag_atoms])
    order = np.argsort(zvals)
    sorted_z = zvals[order]

    # Split sorted atoms into n_layers clusters using the largest internal gaps.
    gaps = sorted_z[1:] - sorted_z[:-1]
    if len(gaps) < n_layers - 1:
        raise ValueError("Not enough magnetic atoms to assign layers.")
    cut_positions = np.argsort(gaps)[-(n_layers - 1):] + 1
    cut_positions = sorted(cut_positions.tolist())
    groups = np.split(order, cut_positions)
    # Label bottom to top as L1, L2, ... according to average z.
    groups_sorted = sorted(groups, key=lambda g: float(np.mean(zvals[g])))
    for lid, group in enumerate(groups_sorted, start=1):
        for idx in group:
            mag_atoms[int(idx)].layer = f"L{lid}"

def get_group_label(atom: Atom, group_by: str) -> str:
    if group_by == "element":
        return atom.element
    if group_by == "layer":
        return atom.layer
    if group_by == "element_layer":
        return f"{atom.element}_{atom.layer}"
    raise ValueError(f"Unknown group_by: {group_by}")

def pair_label(a: Atom, b: Atom, group_by: str) -> str:
    ga = get_group_label(a, group_by)
    gb = get_group_label(b, group_by)
    return "-".join(sorted([ga, gb]))

def canonical_bond_key(i: int, j: int, shift: Tuple[int, int, int]) -> Tuple:
    tx, ty, tz = shift
    rep1 = ((i, 0, 0, 0), (j, tx, ty, tz))
    rep2 = ((j, 0, 0, 0), (i, -tx, -ty, -tz))
    return min(rep1, rep2)

def find_unique_bonds(
    lattice: np.ndarray,
    mag_atoms: List[Atom],
    pbc: Tuple[int, int, int],
    max_image: int,
    group_by: str,
) -> List[Bond]:
    shifts = []
    ranges = []
    for flag in pbc:
        ranges.append(range(-max_image, max_image + 1) if flag else range(0, 1))
    for tx, ty, tz in itertools.product(*ranges):
        shifts.append((tx, ty, tz))

    bonds = []
    seen = set()
    frac_coords = np.array([a.frac for a in mag_atoms])

    for i, ai in enumerate(mag_atoms):
        for j, aj in enumerate(mag_atoms):
            for sh in shifts:
                if i == j and sh == (0, 0, 0):

```

```

        continue
    key = canonical_bond_key(i, j, sh)
    if key in seen:
        continue
    seen.add(key)
    dfrac = frac_coords[j] + np.array(sh, dtype=float) - frac_coords[i]
    dcart = dfrac @ lattice
    dist = float(np.linalg.norm(dcart))
    if dist < 1e-8:
        continue
    bonds.append(Bond(i=i, j=j, shift=sh, distance=dist, pair_label=pair_label(ai, aj, group_by)))
return bonds

def assign_shells(
    bonds: List[Bond],
    max_shells: int,
    shell_tol: float,
    max_distance: Optional[float] = None,
) -> Tuple[List[Bond], Dict[str, List[float]]]:
    kept: List[Bond] = []
    shell_centers: Dict[str, List[float]] = {}
    by_pair: Dict[str, List[Bond]] = {}
    for b in bonds:
        if max_distance is not None and b.distance > max_distance:
            continue
        by_pair.setdefault(b.pair_label, []).append(b)

    for plabel, blist in by_pair.items():
        blist_sorted = sorted(blist, key=lambda x: x.distance)
        centers: List[float] = []
        shell_lists: List[List[Bond]] = []
        for b in blist_sorted:
            assigned = False
            for sidx, cen in enumerate(centers):
                if abs(b.distance - cen) <= shell_tol:
                    shell_lists[sidx].append(b)
                    # update center as mean for stability
                    centers[sidx] = float(np.mean([x.distance for x in shell_lists[sidx]]))
                    assigned = True
                    break
            if not assigned:
                centers.append(b.distance)
                shell_lists.append([b])
        # Stop only after collecting all bonds belonging to max_shells shells.
        order = np.argsort(centers).tolist()
        centers_sorted = [centers[k] for k in order]
        shell_lists_sorted = [shell_lists[k] for k in order]
        shell_centers[plabel] = centers_sorted[:max_shells]
        for shell_no, slist in enumerate(shell_lists_sorted[:max_shells], start=1):
            for b in slist:
                b.shell = shell_no
                kept.append(b)
    kept = sorted(kept, key=lambda b: (b.pair_label, b.shell, b.distance, b.i, b.j, b.shift))
    return kept, shell_centers

def parameter_names(bonds: List[Bond]) -> List[str]:
    params = sorted(set(f"J{b.shell}_{b.pair_label.replace('-', '')}" for b in bonds))
    # Natural sort by pair label then shell if possible
    def keyfn(name):
        # J1_CrRu -> (CrRu, 1)
        left, pair = name.split("_", 1)
        shell = int(left[1:])
        return (pair, shell)
    return sorted(params, key=keyfn)

def coefficient_row(spins: np.ndarray, bonds: List[Bond], params: List[str]) -> List[int]:

```

```

acc = {p: 0 for p in params}
for b in bonds:
    pname = f"J{b.shell}_{b.pair_label.replace('-', '')}"
    acc[pname] += int(spins[b.i] * spins[b.j])
# Hamiltonian E = E0 - sum J SiSj, so coefficient is -sum SiSj.
return [-acc[p] for p in params]

def config_name(k: int) -> str:
    return f"E{k}"

def generate_candidate_spins(mag_atoms: List[Atom], group_by_for_afm: str, seed: int, n_random: int) ->
↳ List[Tuple[str, np.ndarray]]:
    """Generate candidate collinear spin configurations.

    Priority in this version:
    1. FM is always E1 and has the highest priority.
    2. The next candidates are true AFM / compensated AFM-like configurations whenever possible:
        - inter-group/layer AFM, e.g. Cr up + Ru down;
        - exact 50/50 AFM patterns inside each group/layer when the group has an even number of atoms;
        - exact 50/50 AFM patterns in the whole magnetic supercell when the total number is even.
    3. Only if these AFM-priority candidates cannot provide full column rank, random non-AFM
        fallback configurations are used.

    Note:
    For an even number of magnetic atoms in a group, AFM-like means exactly N/2 up and N/2 down.
    The previous version used some "near-balanced" patterns, which could produce 4 up / 2 down
    states for a 6-atom layer. This version avoids that in the AFM-priority pool.
    """
    n = len(mag_atoms)
    rng = np.random.default_rng(seed)
    candidates: List[Tuple[str, np.ndarray]] = []

    def add_candidate(name: str, spins: np.ndarray, seen: set):
        spins = np.array(spins, dtype=int)
        # Keep FM as all-up. For other patterns, remove global spin-reversal duplicates.
        if name != "FM" and spins[0] == -1:
            spins = -spins
        key = tuple(spins.tolist())
        if key not in seen:
            seen.add(key)
            candidates.append((name, spins.copy()))

    def exact_balanced_patterns_for_indices(indices: List[int], max_patterns: int = 200) -> List[np.ndarray]:
        """Return exact 50/50 up/down patterns for selected indices if possible.

        The returned arrays have length len(indices). If len(indices) is odd, this returns
        the closest compensated patterns with floor(N/2) down spins, but these are used
        only when exact compensation is impossible.
        """
        m = len(indices)
        if m <= 1:
            return [np.ones(m, dtype=int)]

        n_down = m // 2
        patterns: List[np.ndarray] = []

        # Deterministic literature-like patterns first.
        deterministic_down_sets = []
        deterministic_down_sets.append(tuple(range(1, m, 2)))           # up/down alternating
        deterministic_down_sets.append(tuple(range(n_down, m)))        # first half up, second half down
        deterministic_down_sets.append(tuple(range(0, m, 2)))          # opposite alternating
        deterministic_down_sets.append(tuple([k for k in range(m) if (k // 2) % 2 == 1]))

        seen_local = set()
        for ds in deterministic_down_sets:
            ds = tuple(sorted(set(ds)))
            if len(ds) != n_down:

```

```

        continue
    if ds in seen_local:
        continue
    seen_local.add(ds)
    vals = np.ones(m, dtype=int)
    vals[list(ds)] = -1
    patterns.append(vals)

# Exhaustive exact 50/50 combinations, capped to avoid huge candidate lists.
for ds in itertools.combinations(range(m), n_down):
    if len(patterns) >= max_patterns:
        break
    if ds in seen_local:
        continue
    seen_local.add(ds)
    vals = np.ones(m, dtype=int)
    vals[list(ds)] = -1
    patterns.append(vals)

return patterns

seen = set()

group_labels = [get_group_label(a, group_by_for_afm) for a in mag_atoms]
unique_groups = sorted(set(group_labels))
group_indices: Dict[str, List[int]] = {g: [i for i, lab in enumerate(group_labels) if lab == g] for g in
↪ unique_groups}

# 1) Highest-priority reference: FM.
add_candidate("FM", np.ones(n, dtype=int), seen)

# 2) Inter-group/layer AFM references, e.g. Cr up/Ru down or top up/bottom down.
# These are not necessarily zero total moment if the two groups have different sizes,
# but they are the standard layer-AFM reference states.
if len(unique_groups) >= 2:
    for g in unique_groups[1:]:
        spins = np.ones(n, dtype=int)
        for idx, lab in enumerate(group_labels):
            if lab == g:
                spins[idx] = -1
        add_candidate(f"inter_AFM_{g}_down", spins, seen)

    spins = np.ones(n, dtype=int)
    base = unique_groups[0]
    for idx, lab in enumerate(group_labels):
        if lab != base:
            spins[idx] = -1
    add_candidate("inter_group_AFM", spins, seen)

# 3) Exact AFM-like patterns within each group/layer, all other groups FM up.
for g in unique_groups:
    patterns = exact_balanced_patterns_for_indices(group_indices[g], max_patterns=200)
    for pidx, vals in enumerate(patterns, start=1):
        spins = np.ones(n, dtype=int)
        for local_i, atom_i in enumerate(group_indices[g]):
            spins[atom_i] = vals[local_i]
        add_candidate(f"AFM_{g}_balanced{pidx}", spins, seen)

# 4) Exact AFM-like patterns simultaneously applied to every group/layer.
if len(unique_groups) >= 2:
    per_group_patterns = []
    for g in unique_groups:
        per_group_patterns.append(exact_balanced_patterns_for_indices(group_indices[g], max_patterns=20))
    for combo_idx, combo in enumerate(itertools.product(*per_group_patterns), start=1):
        if combo_idx > 400:
            break
        spins = np.ones(n, dtype=int)
        for g, vals in zip(unique_groups, combo):
            for local_i, atom_i in enumerate(group_indices[g]):

```

```

        spins[atom_i] = vals[local_i]
        add_candidate(f"AFM_all_groups_balanced{combo_idx}", spins, seen)

        # Also add a version with every second group flipped. This can change interlayer
        # coefficients while preserving each group's exact zero net moment.
        spins2 = spins.copy()
        for gidx, g in enumerate(unique_groups):
            if gidx % 2 == 1:
                for atom_i in group_indices[g]:
                    spins2[atom_i] *= -1
        add_candidate(f"AFM_all_groups_balanced{combo_idx}_interflip", spins2, seen)

# 5) Whole-cell exact balanced AFM-like patterns.
if n > 1:
    whole_patterns = exact_balanced_patterns_for_indices(list(range(n)), max_patterns=500)
    for pidx, vals in enumerate(whole_patterns, start=1):
        add_candidate(f"AFM_whole_balanced{pidx}", vals, seen)

# 6) Balanced random AFM-like patterns.
# For even n, these are exactly half up and half down. For odd n, closest possible.
for _ in range(n_random):
    spins = np.ones(n, dtype=int)
    if n > 1:
        down = rng.choice(n, size=n // 2, replace=False)
        spins[down] = -1
    add_candidate("rand_balanced_AFM", spins, seen)

# 7) Fully random non-AFM fallback patterns. These are intentionally last.
for _ in range(n_random):
    spins = rng.choice([-1, 1], size=n).astype(int)
    add_candidate("rand_fallback_nonAFM", spins, seen)

return candidates

def select_full_rank_configs(
    bonds: List[Bond],
    params: List[str],
    mag_atoms: List[Atom],
    n_configs: int,
    seed: int,
    group_by_for_afm: str,
) -> List[Tuple[str, np.ndarray, List[int]]]:
    unknowns = 1 + len(params)
    candidates = generate_candidate_spins(mag_atoms, group_by_for_afm, seed=seed, n_random=max(5000, 200 *
↪ n_configs))

    selected: List[Tuple[str, np.ndarray, List[int]]] = []

    def rank_of(sel):
        if not sel:
            return 0
        A = np.array([[1] + row[2] for row in sel], dtype=float)
        return int(np.linalg.matrix_rank(A, tol=1e-10))

    # Always include FM first if available. This is the highest-priority reference state.
    fm_added = False
    for name, spins in candidates:
        if name == "FM":
            selected.append((name, spins, coefficient_row(spins, bonds, params)))
            fm_added = True
            break
    if not fm_added:
        spins = np.ones(len(mag_atoms), dtype=int)
        selected.append(("FM", spins, coefficient_row(spins, bonds, params)))

    # Greedily add configurations in candidate priority order. Since the candidate list is
    # ordered as FM -> AFM-like -> balanced random AFM -> random fallback, the algorithm
    # uses AFM-like states whenever they increase rank, and only resorts to non-AFM states
    # if the AFM-like pool is insufficient.

```

```

seen_spins = {tuple(selected[0][1].tolist())}
for name, spins in candidates:
    if len(selected) >= n_configs and rank_of(selected) >= min(unknowns, n_configs):
        break
    key = tuple(spins.tolist())
    if key in seen_spins:
        continue
    coeff = coefficient_row(spins, bonds, params)
    r0 = rank_of(selected)
    test = selected + [(name, spins, coeff)]
    r1 = rank_of(test)
    if r1 > r0:
        selected.append((name, spins, coeff))
        seen_spins.add(key)
    if len(selected) >= n_configs and rank_of(selected) >= unknowns:
        break

# If full rank has been reached but fewer than n_configs were requested, append extra
# AFM-priority candidates first; random fallback candidates appear only after AFM-like
# candidates in the candidate list.
for name, spins in candidates:
    if len(selected) >= n_configs:
        break
    key = tuple(spins.tolist())
    if key in seen_spins:
        continue
    selected.append((name, spins, coefficient_row(spins, bonds, params)))
    seen_spins.add(key)

if rank_of(selected) < unknowns:
    raise RuntimeError(
        f"Failed to generate full-column-rank configurations. Rank={rank_of(selected)}, unknowns={unknowns}. "
        f"Try increasing --n-configs, using a larger magnetic supercell, or changing --seed."
    )

# Rename sequentially E1, E2, ... but keep original note in name suffix.
renamed = []
for k, (name, spins, coeff) in enumerate(selected[:n_configs], start=1):
    renamed.append((f"E{k}_{name}", spins, coeff))
return renamed

def write_csv(path: str, header: List[str], rows: List[List]):
    with open(path, "w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow(header)
        w.writerows(rows)

def equation_string(name: str, coeff: List[int], params: List[str]) -> str:
    terms = ["E0"]
    for c, p in zip(coeff, params):
        if c == 0:
            continue
        if c > 0:
            terms.append(f"+ {c}{p}")
        else:
            terms.append(f"- {abs(c)}{p}")
    return f"{name} = " + " ".join(terms)

def main():
    ap = argparse.ArgumentParser(description="Generate J-fitting magnetic configurations and coefficients.")
    ap.add_argument("poscar", help="Path to POSCAR/CONTCAR")
    ap.add_argument("--mag-elements", nargs="*", default=None, help="Magnetic elements, e.g. Cr Ru")
    ap.add_argument("--mag-indices", default=None, help="Explicit POSCAR atom indices. Default assumes 1-based,
    ↪ e.g. 1-6,13-18")
    ap.add_argument("--indices-zero-based", action="store_true", help="Interpret --mag-indices as 0-based")

```

```

ap.add_argument("--group-by", choices=["element", "layer", "element_layer"], default="element", help="How to
↪ classify pair types")
ap.add_argument("--n-layers", type=int, default=1, help="Number of magnetic layers for layer assignment")
ap.add_argument("--pbc", nargs=3, type=int, default=[1, 1, 0], help="Periodic boundary flags, e.g. 1 1 0")
ap.add_argument("--max-image", type=int, default=2, help="Maximum image shift searched in periodic
↪ directions")
ap.add_argument("--max-shells", type=int, default=3, help="Number of neighbour shells per pair type")
ap.add_argument("--shell-tol", type=float, default=0.08, help="Distance tolerance for grouping shells in
↪ Angstrom")
ap.add_argument("--max-distance", type=float, default=None, help="Optional maximum bond distance in Angstrom")
ap.add_argument("--n-configs", type=int, default=None, help="Number of magnetic configurations to output")
ap.add_argument("--seed", type=int, default=20260610, help="Random seed for generated configurations")
ap.add_argument("--outdir", default="J_fit_output", help="Output directory")
args = ap.parse_args()

lattice, elements, counts, atoms = parse_poscar(args.poscar)

if args.mag_indices:
    raw = parse_index_list(args.mag_indices)
    if args.indices_zero_based:
        idx_set = set(raw)
    else:
        idx_set = set([x - 1 for x in raw])
    mag_atoms = [a for a in atoms if a.index0 in idx_set]
elif args.mag_elements:
    mag_set = set(args.mag_elements)
    mag_atoms = [a for a in atoms if a.element in mag_set]
else:
    raise ValueError("Please provide either --mag-elements or --mag-indices.")

if not mag_atoms:
    raise ValueError("No magnetic atoms selected.")

assign_layers_by_z(mag_atoms, args.n_layers)

all_bonds = find_unique_bonds(lattice, mag_atoms, tuple(args.pbc), args.max_image, args.group_by)
bonds, centers = assign_shells(all_bonds, args.max_shells, args.shell_tol, args.max_distance)
params = parameter_names(bonds)
unknowns = 1 + len(params)
n_configs = args.n_configs if args.n_configs is not None else unknowns + 2

# For group AFM candidates, use layer if n_layers>1, otherwise element.
group_by_for_afm = "layer" if args.n_layers > 1 else "element"
configs = select_full_rank_configs(bonds, params, mag_atoms, n_configs, args.seed, group_by_for_afm)

os.makedirs(args.outdir, exist_ok=True)

# Magnetic atom table
atom_header = ["mag_index", "poscar_index_lbased", "element", "layer", "frac_x", "frac_y", "frac_z"]
atom_rows = []
atom_labels = []
for k, a in enumerate(mag_atoms):
    label = f"atom{a.index0+1}_{a.element}_{a.layer}"
    atom_labels.append(label)
    atom_rows.append([k, a.index0 + 1, a.element, a.layer, *[f"{x:.10f}" for x in a.frac]])
write_csv(os.path.join(args.outdir, "magnetic_atoms.csv"), atom_header, atom_rows)

# Bond table
bond_header = ["pair_label", "shell", "i_mag", "j_mag", "i_poscar", "j_poscar", "shift_a", "shift_b",
↪ "shift_c", "distance_A"]
bond_rows = []
for b in bonds:
    bond_rows.append([
        b.pair_label, b.shell, b.i, b.j, mag_atoms[b.i].index0 + 1, mag_atoms[b.j].index0 + 1,
        b.shift[0], b.shift[1], b.shift[2], f"{b.distance:.8f}"
    ])
write_csv(os.path.join(args.outdir, "bonds_used_for_J.csv"), bond_header, bond_rows)

```

```

# Shell summary
summary = []
for p in sorted(set(b.pair_label for b in bonds)):
    for s in range(1, args.max_shells + 1):
        bl = [b for b in bonds if b.pair_label == p and b.shell == s]
        if not bl:
            continue
        ds = [b.distance for b in bl]
        summary.append([p, s, len(bl), f"{min(ds):.8f}", f"{max(ds):.8f}", f"{np.mean(ds):.8f}"])
write_csv(os.path.join(args.outdir, "neighbor_shell_summary.csv"),
          ["pair_label", "shell", "n_unique_bonds", "dist_min_A", "dist_max_A", "dist_mean_A"], summary)

# Config coefficients
header = ["config"] + atom_labels + params
rows = []
for name, spins, coeff in configs:
    rows.append([name] + ["up" if s == 1 else "down" for s in spins] + coeff)
coeff_path = os.path.join(args.outdir, "J_fit_configurations_coefficients.csv")
write_csv(coeff_path, header, rows)

# Equations
A = np.array([[1] + coeff for _, _, coeff in configs], dtype=float)
rank = int(np.linalg.matrix_rank(A, tol=1e-10))
with open(os.path.join(args.outdir, "J_fit_equations.txt"), "w", encoding="utf-8") as f:
    f.write("Hamiltonian convention:\n")
    f.write("E = E0 - sum J_ij S_i S_j, with S_i = +1/-1.\n\n")
    f.write("Unknown vector:\n")
    f.write("[E0, " + ", ".join(params) + "]\n\n")
    f.write(f"Matrix shape: {A.shape[0]} x {A.shape[1]}\n")
    f.write(f"Rank: {rank}\n")
    f.write(f"Full column rank: {rank == A.shape[1]}\n\n")
    for name, _, coeff in configs:
        f.write(equation_string(name, coeff, params) + "\n")

# Metadata JSON
meta = {
    "hamiltonian": "E = E0 - sum J_ij S_i S_j",
    "poscar": os.path.abspath(args.poscar),
    "pbc": args.pbc,
    "max_image": args.max_image,
    "max_shells": args.max_shells,
    "shell_tol_A": args.shell_tol,
    "group_by": args.group_by,
    "n_layers": args.n_layers,
    "parameters": params,
    "matrix_shape": list(A.shape),
    "rank": rank,
    "full_column_rank": rank == A.shape[1],
}
with open(os.path.join(args.outdir, "metadata.json"), "w", encoding="utf-8") as f:
    json.dump(meta, f, indent=2, ensure_ascii=False)

print("Done.")
print(f"Magnetic atoms: {len(mag_atoms)}")
print(f"Parameters: {params}")
print(f"Coefficient matrix shape: {A.shape}, rank={rank}, full_column_rank={rank == A.shape[1]}")
print(f"Output directory: {args.outdir}")
print(f"Main coefficient file: {coeff_path}")

if __name__ == "__main__":
    main()

```

► 代码清单 A.68.2 (shell, 8 行)

1. E1 必定优先生成 FM, 全 ↑;
2. 后续优先生成严格 AFM 构型;
对偶数个磁性原子, 要求严格 N/2 ↑、N/2 ↓;
例如 6 个 Tc 必须是 3↑3↓;
3. 对双层/多组体系, 优先生成 layer-AFM, 例如 Cr 全 ↑、Ru 全 ↓;
4. 再生成每层内部严格 AFM, 例如 Cr 层 3↑3↓、Ru 层全 ↑;
5. 再生成所有层同时严格 AFM, 例如 Cr 层 3↑3↓、Ru 层 3↑3↓;
6. 如果这些 AFM 构型仍然无法让矩阵满秩, 最后才使用非 AFM 随机构型补足。

高通量结构优化脚本

► 代码清单 A.69.1 (shell, 30 行)

```
#!/bin/bash

echo " 开始处理材料..."

# 正确定义材料列表数组
materials=(
VScGe2S6 VGaGe2S6 VCdGe2Te6 CrScGe2Se6 CrGaGe2S6
MnVGe2S6 MnNiGe2Te6 MnAgGe2Se6 MnAgGe2Te6 MnCuGe2Se6
MnGaGe2Se6 MnHgGe2Te6 MnIrGe2S6 MnIrGe2Se6 MnScGe2Se6
MnCoGe2Te6 FeZnGe2S6 FeZnGe2Se6 CoVGe2S6 CoVGe2Se6
CoVGe2Te6 CoMoGe2Se6 CoMoGe2Te6 CoRuGe2Te6 NbCdGe2Te6
MoAgGe2S6 MoAgGe2Te6 MoCdGe2Se6 MoHgGe2S6 MoPbGe2Se6
MoSnGe2Te6 MoBiGe2Se6 TcAgGe2S6 TcCdGe2S6 ReScGe2Se6
)

# 循环遍历数组中的每个材料
for i in "${materials[@]}"
do
    echo " 正在处理材料: $i"
    target_dir="$i"
    mkdir -p "$target_dir"
    cd "$target_dir" || exit 1

    # 复制 POSCAR 文件
    cp ../POSCAR .

    cd ..
done

echo " 所有材料处理完毕! "
```

► 代码清单 A.69.2 (python, 75 行)

```
#!/usr/bin/env python
import os
import re
from ase.io import read, write
```

```

# 获取当前目录下的所有文件夹
folders = [f for f in os.listdir('.') if os.path.isdir(f)]

# 元素符号正则表达式模式
element_pattern = re.compile(r'[A-Z][a-z]*')

# 处理每个文件夹
for folder in folders:
    # 从文件夹名称中提取元素符号
    folder_elements = element_pattern.findall(folder)

    if not folder_elements:
        print(f" 跳过文件夹 '{folder}': 未提取到元素符号")
        continue

    print(f" 处理文件夹 '{folder}', 提取的元素: {folder_elements}")

    # 构建 POSCAR 文件路径
    poscar_path = os.path.join(folder, 'POSCAR')

    if not os.path.exists(poscar_path):
        print(f" 警告: 文件夹中未找到 POSCAR 文件")
        continue

    # 读取 POSCAR 文件
    atoms = read(poscar_path, format='vasp')

    # 获取当前 POSCAR 中的元素符号
    current_symbols = atoms.get_chemical_symbols()

    # 获取当前 POSCAR 中所有唯一的元素 (按出现顺序)
    current_unique_elements = []
    for symbol in current_symbols:
        if symbol not in current_unique_elements:
            current_unique_elements.append(symbol)

    # 创建一个映射: 将当前元素映射到文件夹名称中的元素 (从左到右)
    element_mapping = {}
    for i, elem in enumerate(folder_elements):
        if i < len(current_unique_elements):
            element_mapping[current_unique_elements[i]] = elem

    # 应用映射到所有原子
    final_symbols = [
        element_mapping.get(symbol, symbol)
        for symbol in current_symbols
    ]

    # 设置新的化学符号
    atoms.set_chemical_symbols(final_symbols)

    # 备份原文件
    backup_path = os.path.join(folder, 'POSCAR.bak')
    os.rename(poscar_path, backup_path)
    print(f" 已备份原文件为: {backup_path}")

    # 保存新文件
    write(poscar_path, atoms, format='vasp', direct=True, vasp5=True)
    print(f" 已更新文件: {poscar_path}")

    # 获取最终的元素顺序
    final_unique_elements = []
    for symbol in final_symbols:
        if symbol not in final_unique_elements:
            final_unique_elements.append(symbol)

    print(f" 最终元素顺序: {final_unique_elements}")
    print()

```

```
print(" 所有文件夹处理完成! ")
```

► 代码清单 A.69.3 (shell, 59 行)

```
#!/bin/bash
#PBS -l nodes=1:ppn=64
#PBS -l walltime=640:01:00
#PBS -j oe
#PBS -q sugon
#PBS -N 35materials_geo

cd $PBS_O_WORKDIR

echo job id is $PBS_JOBID | tee pbslog
echo run nodes is following: | tee -a pbslog
cat $PBS_NODEFILE | tee -a pbslog

echo begin time is `date` | tee -a pbslog
id=`echo $PBS_JOBID|awk -F. '{print $1}'`
NP=`cat $PBS_NODEFILE|wc -l`

source /public/software/profile.d/compiler_intel-compiler-2021.3.0.sh
source /public/software/profile.d/mpi_intelmpi-2021.3.0.sh
RUN_1=/public/home/cssong/app/vasp/intel_2021.3.0/5.4.4/vasp_std
RUN_2=/public/home/cssong/app/vasp/intel_2021.3.0/5.4.4/vasp_ncl
cat $PBS_NODEFILE | uniq -c | awk '{ printf("%s:%s\n", $2, $1); }' >> $PBS_JOBID-$PBS_JOBCOOKIE.hosts

# 正确定义材料列表数组
materials=(
  VScGe2S6 VGaGe2S6 VCdGe2Te6 CrScGe2Se6 CrGaGe2S6
  MnVGe2S6 MnNiGe2Te6 MnAgGe2Se6 MnAgGe2Te6 MnCuGe2Se6
  MnGaGe2Se6 MnHgGe2Te6 MnIrGe2S6 MnIrGe2Se6 MnScGe2Se6
  MnCoGe2Te6 FeZnGe2S6 FeZnGe2Se6 CoVGe2S6 CoVGe2Se6
  CoVGe2Te6 CoMoGe2Se6 CoMoGe2Te6 CoRuGe2Te6 NbCdGe2Te6
  MoAgGe2S6 MoAgGe2Te6 MoCdGe2Se6 MoHgGe2S6 MoPbGe2Se6
  MoSnGe2Te6 MoBiGe2Se6 TcAgGe2S6 TcCdGe2S6 ReScGe2Se6
)

# 循环遍历数组中的每个材料
for i in "${materials[@]}"
do
  echo " 正在处理材料: $i"
  target_dir="$i"
  mkdir -p "$target_dir"
  cd "$target_dir" || exit 1 # 进入目录, 如果失败则退出
  echo -e "102\n1\n0.03" | vaspkit > vaspkit_dos.log 2>&1
  vaspkit_exit_code=$?
  if [ $vaspkit_exit_code -ne 0 ]; then
    echo " 错误: 在 $target_dir 中执行 vaspkit 失败 (错误码 $vaspkit_exit_code) "
    cd ..
    continue
  fi
  cp ../INCAR .
  mpirun -np $NP ${RUN_1} > log 2>&1
  # 计算完成后, 清理大文件
  rm -f CHG CHGCAR WAVECAR

  cd .. # 返回上级目录, 准备处理下一个材料
done

echo " 所有材料处理完毕! "
echo end time is `date` | tee -a pbslog
```


► 代码清单 A.69.4 (shell, 76 行)

```
#!/bin/bash
#PBS -l nodes=node19:ppn=64
#PBS -l walltime=640:01:00
#PBS -j oe
#PBS -q sugon
#PBS -N 35materials_geo2

cd $PBS_O_WORKDIR

echo job id is $PBS_JOBID | tee pbslog
echo run nodes is following: | tee -a pbslog
cat $PBS_NODEFILE | tee -a pbslog

echo begin time is `date` | tee -a pbslog
id=`echo $PBS_JOBID|awk -F. '{print $1}'`
NP=`cat $PBS_NODEFILE|wc -l`

source /public/software/profile.d/compiler_intel-compiler-2021.3.0.sh
source /public/software/profile.d/mpi_intelmpi-2021.3.0.sh
RUN_1=/public/home/cssong/app/vasp/intel_2021.3.0/5.4.4/vasp_std
RUN_2=/public/home/cssong/app/vasp/intel_2021.3.0/5.4.4/vasp_ncl
cat $PBS_NODEFILE | uniq -c | awk '{ printf("%s:%s\n", $2, $1); }' >> $PBS_JOBID-$PBS_JOBCOOKIE.hosts

# 正确定义材料列表数组
materials=(
  VScGe2S6 VGaGe2S6 VCdGe2Te6 CrScGe2Se6 CrGaGe2S6
  MnVGe2S6 MnNiGe2Te6 MnAgGe2Se6 MnAgGe2Te6 MnCuGe2Se6
  MnGaGe2Se6 MnHgGe2Te6 MnIrGe2S6 MnIrGe2Se6 MnScGe2Se6
  MnCoGe2Te6 FeZnGe2S6 FeZnGe2Se6 CoVGe2S6 CoVGe2Se6
  CoVGe2Te6 CoMoGe2Se6 CoMoGe2Te6 CoRuGe2Te6 NbCdGe2Te6
  MoAgGe2S6 MoAgGe2Te6 MoCdGe2Se6 MoHgGe2S6 MoPbGe2Se6
  MoSnGe2Te6 MoBiGe2Se6 TcAgGe2S6 TcCdGe2S6 ReScGe2Se6
)

# 循环遍历数组中的每个材料
for i in "${materials[@]}"
do
  echo "正在处理材料: $i"
  target_dir="$i"

  # 1. 进入材料目录
  cd "$target_dir" || { echo "无法进入目录 $target_dir"; exit 1; }

  # 2. 在材料目录内创建 geo_2 子目录
  mkdir -p "geo_2"

  # 3. 进入 geo_2 目录
  cd "geo_2" || { echo "无法进入 geo_2 目录"; exit 1; }

  # 4. 从上层目录复制 CONTCAR 作为 POSCAR
  cp ../CONTCAR POSCAR || { echo "复制 CONTCAR 失败"; exit 1; }

  # 5. 运行 vaspkit 生成输入文件
  echo -e "102\n1\n0.03" | vaspkit > vaspkit_dos.log 2>&1
  vaspkit_exit_code=$?
  if [ $vaspkit_exit_code -ne 0 ]; then
    echo "错误: 在 $target_dir/geo_2 中执行 vaspkit 失败 (错误码 $vaspkit_exit_code)"
    cd ../../ # 返回原始工作目录
    continue
  fi

  # 6. 从上层目录复制 INCAR
  cp ../../INCAR . || { echo "复制 INCAR 失败"; exit 1; }

  # 7. 运行 VASP 计算
```

```

mpirun -np $NP ${RUN_1} > log 2>&1

# 8. 计算完成后, 清理大文件
rm -f CHG CHGCAR WAVECAR

# 9. 返回原始工作目录 (两次 cd..)
cd ../../
done

echo " 所有材料处理完毕! "
echo end time is `date` | tee -a pbslog

```

► 代码清单 A.69.5 (shell, 172 行)

```

#!/bin/bash
#PBS -l nodes=1:ppn=64
#PBS -l walltime=640:01:00
#PBS -j oe
#PBS -q sugon
#PBS -N 35materials_geo

cd $PBS_O_WORKDIR

echo job id is $PBS_JOBID | tee pbslog
echo run nodes is following: | tee -a pbslog
cat $PBS_NODEFILE | tee -a pbslog

echo begin time is `date` | tee -a pbslog
id=`echo $PBS_JOBID|awk -F. '{print $1}'`
NP=`cat $PBS_NODEFILE|wc -l`

source /public/software/profile.d/compiler_intel-compiler-2021.3.0.sh
source /public/software/profile.d/mpi_intelmpi-2021.3.0.sh
RUN_1=/public/home/cssong/app/vasp/intel_2021.3.0/5.4.4/vasp_std
RUN_2=/public/home/cssong/app/vasp/intel_2021.3.0/5.4.4/vasp_ncl
cat $PBS_NODEFILE | uniq -c | awk '{ printf("%s:%s\n", $2, $1); }' >> $PBS_JOBID-$PBS_JOBCOOKIE.hosts

# 正确定义材料列表数组
#materials=(
#VScGe2S6 VGaGe2S6 VCdGe2Te6 CrScGe2Se 极 CrGaGe2S6
#MnVGe2S6 MnNiGe2Te6 MnAgGe2Se6 MnAgGe2Te6
#MnGaGe2Se6 MnHgGe2Te6 MnIrGe2S6 MnIrGe2Se6 MnScGe2Se6
#MnCoGe2Te6 FeZnGe2S6 FeZnGe2Se6 CoV 极 2S6 CoVGe2Se6
#CoVGe2Te6 CoMoGe2Se6 CoMoGe2Te6 CoRuGe2Te6 NbCdGe2Te6
#MoAgGe2S6 MoAgGe2Te6 MoCdGe2Se6 MoHgGe2S6 MoPbGe2Se6
#MoSnGe2Te6 MoBiGe2Se6 TcAgGe2S6 TcCdGe2S6 ReScGe2Se6
#)

materials=(
MnGaGe2Se6 MnHgGe2Te6 MnIrGe2S6 MnIrGe2Se6 MnScGe2Se6
MnCoGe2Te6 FeZnGe2S6 FeZnGe2Se6 CoV 极 2S6 CoVGe2Se6
CoVGe2Te6 CoMoGe2Se6 CoMoGe2Te6 CoRuGe2Te6 NbCdGe2Te6
MoAgGe2S6 MoAgGe2Te6 MoCdGe2Se6 MoHgGe2S6 MoPbGe2Se6
MoSnGe2Te6 MoBiGe2Se6 TcAgGe2S6 TcCdGe2S6 ReScGe2Se6
)

# 1. 创建 3_geo 文件夹
echo " 创建 3_geo 文件夹..."
mkdir -p 3_geo
if [ $? -eq 0 ]; then
    echo " 成功创建 3_geo 文件夹"
else
    echo " 错误: 无法创建 3_geo 文件夹"

```

```

        exit 1
    fi

# 2. 循环遍历数组中的每个材料
echo " 开始复制 CONTCAR 文件..."
for i in "${materials[@]}"
do
    echo " 正在处理材料: $i"

    # 2.1 检查 geo_2 文件夹是否存在
    if [ ! -d "$i/geo_2" ]; then
        echo " 警告: 目录 $i/geo_2 不存在, 跳过此材料"
        continue
    fi

    # 2.2 进入材料目录的 geo_2 文件夹
    echo " 进入目录 $i/geo_2"
    cd "$i/geo_2" || { echo " 错误: 无法进入目录 $i/geo_2"; continue; }

    # 2.3 检查 CONTCAR 文件是否存在
    if [ ! -f "CONTCAR" ]; then
        echo " 警告: CONTCAR 文件不存在于 $i/geo_2, 跳过此材料"
        cd ../../
        continue
    fi

    # 2.4 复制 CONTCAR
    echo " 复制 CONTCAR 到 ../../3_geo/${i}_CONTCAR"
    cp CONTCAR ../../3_geo/"${i}_CONTCAR"
    if [ $? -eq 0 ]; then
        echo " 成功复制 CONTCAR"
    else
        echo " 错误: 复制 CONTCAR 失败"
        cd ../../
        continue
    fi

    # 2.5 返回两次到脚本所在目录
    cd ../../
    echo " 返回脚本所在目录"
done

echo " 已完成 CONTCAR 复制, 共处理了 $(ls 3_geo/*_CONTCAR 2>/dev/null | wc -l) 个文件"

# 3. 在 3_geo 文件夹中为每种材料创建子文件夹
echo " 开始在 3_geo 中创建材料子文件夹..."
for i in "${materials[@]}"
do
    # 3.1 检查 CONTCAR 文件是否存在
    if [ ! -f "3_geo/${i}_CONTCAR" ]; then
        echo " 警告: CONTCAR 文件 3_geo/${i}_CONTCAR 不存在, 跳过此材料"
        continue
    fi

    # 3.2 在 3_geo 中创建材料文件夹
    echo " 创建目录 3_geo/$i"
    mkdir -p "3_geo/$i"
    if [ $? -eq 0 ]; then
        echo " 成功创建目录 3_geo/$i"
    else
        echo " 错误: 无法创建目录 3_geo/$i"
        continue
    fi

    # 3.3 将复制的 CONTCAR 移动到新文件夹并重命名为 POSCAR
    echo " 移动文件 3_geo/${i}_CONTCAR 到 3_geo/$i/POSCAR"
    mv "3_geo/${i}_CONTCAR" "3_geo/$i/POSCAR"
    if [ $? -eq 0 ]; then
        echo " 成功移动文件"
    fi
done

```

```

else
    echo " 错误: 移动文件失败"
    continue
fi

# 3.4 进入新建的材料文件夹
echo " 进入目录 3_geo/$i"
cd "3_geo/$i" || { echo " 错误: 无法进入目录 3_geo/$i"; continue; }

# 3.5 运行 vaspkit 生成输入文件
echo " 运行 vaspkit 生成输入文件..."
echo -e "102\n1\n0.03" | vaspkit > vaspkit_dos.log 2>&1
vaspkit_exit_code=$?
if [ $vaspkit_exit_code -eq 0 ]; then
    echo "vaspkit 运行成功"
else
    echo " 错误: 在 3_geo/$i 中执行 vaspkit 失败 (错误码 $vaspkit_exit_code) "
    cd ../../ # 返回原始工作目录
    continue
fi

# 3.6 从脚本所在目录复制 INCAR 到当前目录
echo " 复制 INCAR 文件..."
cp $PBS_0_WORKDIR/INCAR .
if [ $? -eq 0 ]; then
    echo " 成功复制 INCAR"
else
    echo " 错误: 复制 INCAR 失败"
    cd ../../
    continue
fi

# 3.7 运行 VASP 计算
echo " 开始 VASP 计算..."
mpirun -np $NP ${RUN_1} > log 2>&1
vasp_exit_code=$?
if [ $vasp_exit_code -eq 0 ]; then
    echo "VASP 计算完成"
else
    echo " 警告: VASP 计算可能出错 (退出码 $vasp_exit_code) "
fi

# 3.8 计算完成后, 清理大文件
echo " 清理大文件..."
rm -f CHG CHGCAR WAVECAR
echo " 清理完成"

# 3.9 返回原始工作目录
cd ../../
echo " 返回脚本所在目录"
done

echo " 所有材料处理完毕! "
echo end time is `date` | tee -a pbslog

```

通过能带判断缺陷、团簇的 python 脚本

► 代码清单 A.70.1 (python, 11 行)

```

import os
import pyprocar

```

```
# 设置 VASP 计算数据目录
data_dir = "/public/home/cssong/song/lmr/9_single_layer/19_ReIrGe2S6/TcIrGeSe/3_static_band/2_band"

# 运行自动能带分析
pyprocar.autobandsplot(
    code="vasp",      # 指定使用 VASP 数据格式
    dirname=data_dir  # 指向计算数据目录
)
```

高通量 DMI 计算脚本

► 代码清单 A.71.1 (shell, 303 行)

```
#!/bin/bash
#PBS -l nodes=1:ppn=64
#PBS -l walltime=640:01:00
#PBS -j oe
#PBS -q sugon
#PBS -N 35materials_dmi

cd $PBS_0_WORKDIR

# 创建详细的日志文件
LOG_FILE="dmi_calculation_$(date +%Y%m%d_%H%M%S).log"
exec > >(tee -a "$LOG_FILE") 2>&1

echo "=== DMI 计算脚本开始执行 ==="
echo " 作业 ID: $PBS_JOBID"
echo " 运行节点:"
cat $PBS_NODEFILE
echo " 开始极间: $(date)"
id=$(echo $PBS_JOBID | awk -F. '{print $1}')
NP=$(cat $PBS_NODEFILE | wc -l)

# 加载环境变量
echo " 加载环境变量..."
source /public/software/profile.d/compiler_intel-compiler-2021.3.0.sh
if [ $? -ne 0 ]; then
    echo " 错误: 无法加载 Intel 编译器环境"
    exit 1
fi

source /public/software/profile.d/mpi_intelmpi-2021.3.0.sh
if [ $? -ne 0 ]; then
    echo " 错误: 无法加载 Intel MPI 环境"
    exit 1
fi

RUN_1="/public/home/cssong/app/vasp/intel_2021.3.0/5.4.4/vasp_std"
RUN_2="/public/home/cssong/app/vasp/intel_2021.3.0/5.4.4/vasp_ncl"

# 检查 VASP 可执行文件是否存在
if [ ! -f "$RUN_1" ]; then
    echo " 错误: VASP 标准版可执行文件不存在: $RUN_1"
    exit 1
fi

if [ ! -f "$RUN_2" ]; then
    echo " 错误: VASP NCL 版可执行文件不存在: $RUN_2"
```

```

        exit 1
    fi

cat $PBS_NODEFILE | uniq -c | awk '{ printf("%s:%s\n", $2, $1); }' >> "$PBS_JOBID-$PBS_JOBCOOKIE.hosts"

# 材料列表数组
materials=(
VScGe2S6 VGaGe2S6 VCdGe2Te6 CrScGe2Se6 CrGaGe2S6
MnVGe2S6 MnNiGe2Te6 MnAgGe2Se6 MnAgGe2Te6
MnGaGe2Se6 MnHgGe2Te6 MnIrGe2S6 MnIrGe2Se6 MnScGe2Se6
MnCoGe2Te6 FeZnGe2S6 FeZnGe2Se6 CoVGe2S6 CoV 极 2Se6
CoVGe2Te6 CoMoGe2Se6 CoMoGe2Te6 CoRuGe2Te6 NbCdGe2Te6
MoAgGe2S6 MoAgGe2Te6 MoCdGe2Se6 MoHgGe2S6 MoPbGe2Se6
MoSnGe2Te6 MoBiGe2Se6 TcAgGe2S6 TcCdGe2S6 ReScGe2Se6
)

echo " 材料列表: ${materials[*]}"

# 1. 创建 4_DMI 文件夹
echo " 步骤 1: 创建 4_DMI 文件夹..."
mkdir -p 4_DMI
if [ $? -ne 0 ]; then
    echo " 错误: 无法创建 4_DMI 文件夹"
    exit 1
fi
echo " 成功创建 4_DMI 文件夹"

# 2. 从 3_geo 文件夹复制 CONTCAR 文件
echo " 步骤 2: 从 3_geo 文件夹复制 CONTCAR 文件..."
COPIED_COUNT=0
for i in "${materials[@]}"
do
    echo " 处理材料: $i"

    # 检查 3_geo 中的材料文件夹是否存在
    if [ ! -d "3_geo/$i" ]; then
        echo " 警告: 目录 3_geo/$i 不存在, 跳过此材料"
        continue
    fi

    # 检查 CONTCAR 文件是否存在
    if [ ! -f "3_geo/$i/CONTCAR" ]; then
        echo " 警告: CONTCAR 文件 3_geo/$i/CONTCAR 不存在, 跳过此材料"
        continue
    fi

    # 在 4_DMI 中创建材料文件夹
    mkdir -p "4_DMI/$i"
    if [ $? -ne 0 ]; then
        echo " 错误: 无法创建目录 4_DMI/$i"
        continue
    fi

    # 复制 CONTCAR 到 4_DMI/材料名/POSCAR
    cp "3_geo/$i/CONTCAR" "4_DMI/$i/POSCAR"
    if [ $? -ne 0 ]; then
        echo " 错误: 复制 CONTCAR 失败, 从 3_geo/$i/CONTCAR 到 4_DMI/$i/POSCAR"
        continue
    fi

    echo " 成功复制 CONTCAR 到 4_DMI/$i/POSCAR"
    ((COPIED_COUNT++))
done

echo " 已完成 CONTCAR 复制, 共处理了 $COPIED_COUNT 个文件"

# 3. 在 4_DMI 文件夹中处理每个材料
echo " 步骤 3: 在 4_DMI 文件夹中处理每个材料..."
PROCESSED_COUNT=0

```

```

for i in "${materials[@]}"
do
    echo " 开始处理材料: $i"

    # 检查 POSCAR 文件是否存在
    if [ ! -f "4_DMI/$i/POSCAR" ]; then
        echo " 警告: POSCAR 文件 4_DMI/$i/POSCAR 不存在, 跳过此材料"
        continue
    fi

    # 进入材料文件夹
    cd "4_DMI/$i"
    if [ $? -ne 0 ]; then
        echo " 错误: 无法进入目录 4_DMI/$i"
        continue
    fi

    # 设置 Fortran 环境变量
    export FORT_BUFFERED=TRUE
    export FORT_CONVERTED_BIG_ENDIAN=TRUE

    # 运行 vaspkit 生成 SC411.vasp
    echo " 运行 vaspkit 生成 SC411.vasp..."

    # 创建输入文件
    echo -e "401\n1\n4 1 1" | vaspkit > vaspkit_dmi.log 2>&1
    VASPKIT_EXIT_CODE=$?

    if [ $VASPKIT_EXIT_CODE -ne 0 ]; then
        echo " 错误: vaspkit 401 命令执行失败 (退出码 $VASPKIT_EXIT_CODE) "
        echo " 查看 vaspkit_dmi.log 获取详细信息"
        cat vaspkit_dmi.log
        cd ../../
        continue
    fi

    # 检查 SC411.vasp 是否生成
    if [ ! -f "SC411.vasp" ]; then
        echo " 错误: SC411.vasp 文件未生成"
        echo "vaspkit 输出:"
        cat vaspkit_dmi.log
        cd ../../
        continue
    fi

    # 删除原有 POSCAR, 将 SC411.vasp 重命名为 POSCAR
    rm -f POSCAR
    mv SC411.vasp POSCAR
    echo " 成功将 SC411.vasp 重命名为 POSCAR"

    # 使用 vaspkit 生成输入文件
    echo " 使用 vaspkit 生成输入文件..."
    echo -e "102\n1\n0.03" | vaspkit > vaspkit_input.log 2>&1
    VASPKIT_EXIT_CODE=$?

    if [ $VASPKIT_EXIT_CODE -ne 0 ]; then
        echo " 错误: vaspkit 102 命令执行失败 (退出码 $VASPKIT_EXIT_CODE) "
        echo " 查看 vaspkit_input.log 获取详细信息"
        cat vaspkit_input.log
        cd ../../
        continue
    fi

    # 从脚本所在目录复制 incar.cw 和 incar.acw
    echo " 复制 incar.cw 和 incar.acw 文件..."
    cp "$PBS_0_WORKDIR/incar.cw" .
    if [ $? -ne 0 ]; then
        echo " 错误: 复制 incar.cw 失败"
        cd ../../
    fi

```

```

    continue
fi

cp "$PBS_0_WORKDIR/incar.acw" .
if [ $? -ne 0 ]; then
    echo " 错误: 复制 incar.acw 失败"
    cd ../../
    continue
fi

# 创建 cw 文件夹
echo " 创建 cw 文件夹..."
mkdir -p cw
if [ $? -ne 0 ]; then
    echo " 错误: 创建 cw 文件夹失败"
    cd ../../
    continue
fi

# 处理 cw 文件夹
echo " 处理 cw 文件夹..."
cd cw
if [ $? -ne 0 ]; then
    echo " 错误: 无法进入 cw 目录"
    cd ..
    cd ../../
    continue
fi

# 复制 incar.cw 并重命名为 INCAR
cp ../incar.cw INCAR
if [ $? -ne 0 ]; then
    echo " 错误: 复制 incar.cw 到 INCAR 失败"
    cd ..
    cd ../../
    continue
fi

# 复制其他必要文件
cp ../KPOINTS .
cp ../POTCAR .
cp ../POSCAR .

# 运行 VASP 计算
echo " 开始 cw 方向的 VASP 计算..."
mpirun -np $NP ${RUN_2} > log 2>&1
VASP_EXIT_CODE=$?
if [ $VASP_EXIT_CODE -ne 0 ]; then
    echo " 警告: VASP 计算可能出错 (退出码 $VASP_EXIT_CODE) "
else
    echo "cw 方向 VASP 计算完成"
fi

# 计算完成后, 清理大文件
echo " 清理 cw 方向的大文件..."
rm -f CHG CHGCAR WAVECAR

# 返回材料目录
cd ..

# 创建 acw 文件夹
echo " 创建 acw 文件夹..."
mkdir -p acw
if [ $? -ne 0 ]; then
    echo " 错误: 创建 acw 文件夹失败"
    cd ../../
    continue
fi

```



```

# 处理 acw 文件夹
echo " 处理 acw 文件夹..."
cd acw
if [ $? -ne 0 ]; then
    echo " 错误: 无法进入 acw 目录"
    cd ../..
    continue
fi

# 复制 incar.acw 并重命名为 INCAR
cp ../incar.acw INCAR
if [ $? -ne 0 ]; then
    echo " 错误: 复制 incar.acw 到 INCAR 失败"
    cd ../..
    continue
fi

# 复制其他必要文件
cp ../KPOINTS .
cp ../POTCAR .
cp ../POSCAR .

# 运行 VASP 计算
echo " 开始 acw 方向的 VASP 计算..."
mpirun -np $NP ${RUN_2} > log 2>&1
VASP_EXIT_CODE=$?
if [ $VASP_EXIT_CODE -ne 0 ]; then
    echo " 警告: VASP 计算可能出错 (退出码 $VASP_EXIT_CODE) "
else
    echo "acw 方向 VASP 计算完成"
fi

# 计算完成后, 清理大文件
echo " 清理 acw 方向的大文件..."
rm -f CHG CHGCAR WAVECAR

# 返回原始工作目录
cd ../../..

echo " 完成材料 $i 的处理"
((PROCESSED_COUNT++))
done

echo " 所有材料 DMI 计算处理完毕! 共成功处理 $PROCESSED_COUNT 个材料"
echo " 结束时间: $(date)"
echo "=== DMI 计算脚本执行完成 ==="

# 同时将重要信息也输出到 pbslog
echo " 所有材料 DMI 计算处理完毕! " | tee -a pbslog
echo " 结束时间: $(date)" | tee -a pbslog

```

► 代码清单 A.71.2 (python, 112 行)

```

#!/usr/bin/env python
import os
import subprocess
import re

# 材料列表
materials = [
    "VScGe2S6", "VGaGe2S6", "VCdGe2Te6", "CrScGe2Se6", "CrGaGe2S6",
    "MnVGe2S6", "MnNiGe2Te6", "MnAgGe2Se6", "MnAgGe2Te6", "MnCuGe2Se6",
    "MnGaGe2Se6", "MnHgGe2Te6", "MnIrGe2S6", "MnIrGe2Se6", "MnScGe2Se6",

```

```

    "MnCoGe2Te6", "FeZnGe2S6", "FeZnGe2Se6", "CoVGe2S6", "CoVGe2Se6",
    "CoVGe2Te6", "CoMoGe2Se6", "CoMoGe2Te6", "CoRuGe2Te6", "NbCdGe2Te6",
    "MoAgGe2S6", "MoAgGe2Te6", "MoCdGe2Se6", "MoHgGe2S6", "MoPbGe2Se6",
    "MoSnGe2Te6", "MoBiGe2Se6", "TcAgGe2S6", "TcCdGe2S6", "ReScGe2Se6"
]

# 存储结果
results = {}

# 检查 4_DMI 文件夹是否存在
if not os.path.exists("4_DMI"):
    print(" 错误: 4_DMI 文件夹不存在")
    exit(1)

# 切换到 4_DMI 文件夹
os.chdir("4_DMI")
print(f" 当前工作目录: {os.getcwd()}")

for material in materials:
    print(f"\n 正在处理材料: {material}")

    # 检查材料文件夹是否存在
    if not os.path.exists(material):
        print(f" 警告: 文件夹 {material} 不存在, 跳过")
        results[material] = " 文件夹不存在"
        continue

    # 进入材料文件夹
    os.chdir(material)
    print(f" 进入文件夹: {os.getcwd()}")

    # 在材料文件夹中运行 qvasp -e 命令
    try:
        # 运行 qvasp -e 命令获取所有能量信息
        result = subprocess.run(["qvasp", "-e"], capture_output=True, text=True, check=True)
        output = result.stdout.strip()
        print(f" qvasp -e 输出:\n{output}")

        # 从输出中提取 cw 和 acw 的能量值
        cw_energy = None
        acw_energy = None

        # 使用正则表达式匹配总结部分
        lines = output.split('\n')
        for line in lines:
            # 匹配 cw 行的能量值
            if 'folder: cw' in line:
                match = re.search(r'energy:\s*([-]?\d+\.\d+)', line)
                if match:
                    cw_energy = float(match.group(1))
                    print(f" 提取到 cw 能量: {cw_energy} eV")

            # 匹配 acw 行的能量值
            elif 'folder: acw' in line:
                match = re.search(r'energy:\s*([-]?\d+\.\d+)', line)
                if match:
                    acw_energy = float(match.group(1))
                    print(f" 提取到 acw 能量: {acw_energy} eV")

        # 计算最终结果 - 先乘 1000 再除以 12
        if cw_energy is not None and acw_energy is not None:
            calculation = (cw_energy - acw_energy) * 1000 / 12
            results[material] = calculation
            print(f" 计算结果: {calculation:.6f} meV")
        else:
            error_msg = ""
            if cw_energy is None:
                error_msg += "cw 能量获取失败 "
            if acw_energy is None:

```

```

        error_msg += "acw 能量获取失败"
        print(f" 警告: {error_msg}")
        results[material] = error_msg

except (subprocess.CalledProcessError, FileNotFoundError) as e:
    print(f" 运行 qvasp -e 命令时出错: {e}")
    results[material] = f" 命令执行错误: {e}"
except ValueError as e:
    print(f" 转换能量值时出错: {e}")
    results[material] = f" 数据转换错误: {e}"

# 返回 4_DMI 文件夹
os.chdir("../")
print(f" 返回目录: {os.getcwd()}")

# 输出所有结果
print("\n\n 最终结果:")
for material, value in results.items():
    if isinstance(value, float):
        print(f"{material}: {value:.6f} meV")
    else:
        print(f"{material}: {value}")

# 将结果保存到文件
with open("DMI_results.txt", "w") as f:
    f.write(" 材料\tDMI 能量 (meV)\n")
    for material, value in results.items():
        if isinstance(value, float):
            f.write(f"{material}\t{value:.6f}\n")
        else:
            f.write(f"{material}\t{value}\n")

print("\n 所有材料处理完毕! 结果已保存到 DMI_results.txt")

```

(原创脚本) 高通量磁耦合参数计算脚本

► 代码清单 A.72.1 (shell, 121 行)

```

#!/bin/bash
#PBS -l nodes=1:ppn=64
#PBS -l walltime=640:01:00
#PBS -j oe
#PBS -q sugon
#PBS -N test

cd $PBS_O_WORKDIR

echo job id is $PBS_JOBID | tee pbslog
echo run nodes is following: | tee -a pbslog
cat $PBS_NODEFILE | tee -a pbslog

echo begin time is `date` | tee -a pbslog
id=`echo $PBS_JOBID|awk -F. '{print $1}'`
NP=`cat $PBS_NODEFILE|wc -l`

source /public/software/profile.d/compiler_intel-compiler-2021.3.0.sh
source /public/software/profile.d/mpi_intelmpi-2021.3.0.sh
RUN_1=/public/home/cssong/app/vasp/intel_2021.3.0/5.4.4/vasp_std
RUN_2=/public/home/cssong/app/vasp/intel_2021.3.0/5.4.4/vasp_ncl

```

```

cat $PBS_NODEFILE | uniq -c | awk '{ printf("%s:%s\n", $2, $1); }' >> $PBS_JOBID-$PBS_JOBCOOKIE.hosts

# 材料列表
materials=(
VScGe2S6 VGaGe2S6 VCdGe2Te6 CrScGe2Se6 CrGaGe2S6
MnVGe2S6 MnNiGe2Te6 MnAgGe2Se6 MnAgGe2Te6 MnCuGe2Se6
MnGaGe2Se6 MnHgGe2Te6 MnIrGe2S6 MnIrGe2Se6 MnScGe2Se6
MnCoGe2Te6 FeZnGe2S6 FeZnGe2Se6 CoVGe2S6 CoVGe2Se6
CoVGe2Te6 CoMoGe2Se6 CoMoGe2Te6 CoRuGe2Te6 NbCdGe2Te6
MoAgGe2S6 MoAgGe2Te6 MoCdGe2Se6 MoHgGe2S6 MoPbGe2Se6
MoSnGe2Te6 MoBiGe2Se6 TcAgGe2S6 TcCdGe2S6 ReScGe2Se6
)

# 创建 5_J 文件夹
mkdir -p 5_J
cd 5_J

# 循环处理每个材料
for material in "${materials[@]}"
do
    echo "正在处理材料: $material"

    # 创建材料文件夹
    mkdir -p "$material"
    cd "$material"

    # 从 3_geo 文件夹复制 CONTCAR 并重命名为 POSCAR
    if [ -f "$PBS_0_WORKDIR/3_geo/$material/CONTCAR" ]; then
        cp "$PBS_0_WORKDIR/3_geo/$material/CONTCAR" POSCAR
        echo "已从 3_geo/$material/CONTCAR 复制 POSCAR"
    else
        echo "错误: 找不到 $PBS_0_WORKDIR/3_geo/$material/CONTCAR"
        cd ..
        continue
    fi

    # 运行 vaspkit 生成超胞 (修正输入格式)
    echo "运行 vaspkit 生成超胞..."
    echo -e "401\n1\n3 2 1" | vaspkit

    # 重命名文件
    if [ -f "POSCAR" ]; then
        mv POSCAR POSCAR-uc
    fi

    # 查找并重命名 SC321.vasp 文件
    if [ -f "SC321.vasp" ]; then
        mv "SC321.vasp" POSCAR
        echo "已将 SC321.vasp 重命名为 POSCAR"
    else
        echo "警告: 未找到 SC321.vasp 文件"
        # 尝试查找其他可能的超胞文件
        vasp_file=$(find . -name "*.vasp" | head -1)
        if [ -n "$vasp_file" ]; then
            mv "$vasp_file" POSCAR
            echo "已将 $vasp_file 重命名为 POSCAR"
        else
            echo "错误: 未找到任何.vasp 文件"
            cd ..
            continue
        fi
    fi

    # 运行 vaspkit 生成 POTCAR 和 KPOINTS
    echo "运行 vaspkit 生成 POTCAR 和 KPOINTS..."
    echo -e "102\n1\n0.03" | vaspkit

    # 复制 INCAR 文件

```

```

cp "$PBS_0_WORKDIR/INCAR1" .
cp "$PBS_0_WORKDIR/INCAR2" .
cp "$PBS_0_WORKDIR/INCAR3" .
cp "$PBS_0_WORKDIR/INCAR4" .

# 运行 VASP 计算
for k in 1 2 3 4
do
    echo "  运行计算 $k"
    mkdir $k
    cd $k
    cp ../POSCAR .
    cp ../POTCAR .
    cp ../KPOINTS .
    cp ../INCAR$k ./INCAR

    # 运行 VASP
    mpirun -np $NP $RUN_1 > log 2>&1

    cd ..
done

cd ..
done

# 清理
rm -rf $PBS_JOBID-$PBS_JOBID-00KIE.hosts

echo "  所有材料计算完成! "
echo end time is `date` | tee -a pbslog

```

二维双层转角材料的代码构建

► 代码清单 A.73.1 (python, 21 行)

```

from twist2d import *

twist_demo = Twist2D()

m = 6
n = 7

super_a1_mult = [m, n]
super_a2_mult = [-n, m+n]
twist_demo.add_layer(super_a1_mult, super_a2_mult, layer_dis=3, prim_poscar="POSCAR")

super_a1_mult = [n, m]
super_a2_mult = [-m, n+m]
twist_demo.add_layer(super_a1_mult, super_a2_mult, prim_poscar="POSCAR")

twist_demo.twist_layers(start_z=0.1)

twist_demo.write_res_to_poscar()

twisted_angles = twist_demo.calc_layers_twist_angles()
print(twisted_angles)

```

► 代码清单 A.73.2 (python, 556 行)

```
"""Module for twisting 2D materials"""
# Author: Yang Li
# Email: yangli18@mails.tsinghua.edu.cn
# Date: 2021.04.27
# Description:
# Module for twisting the 2D materials,
# Special thanks for
# guoxm teach me the wonderful integer supercell generation method

import sys
import os
import numpy as np

# Necessary Constants
FLOAT_PREC = 1e-6
DEFAULT_IN_POSCAR = 'POSCAR'
DEFAULT_OUT_POSCAR = 'POSCAR.T2D.vasp'

#####
### Class for 2d twisted ###
#####
class Twist2D():
    # +-----+
    # | Sys Setting |
    # +-----+
    def __init__(self):
        self.primcell_info_list = []
        self.supercell_info_list = []

    def _exit(self, context='[error] Unknown: Something goes wrong...'):
        """exit the program with some error msg."""
        print(context)
        sys.exit(1)

    # +-----+
    # | Misc Tools |
    # +-----+
    def coord_cart2frac(self, cell_vecs, cart_vec):
        """Transfrom the cart coords to frac coords"""
        cell_vecs_inv = np.linalg.inv(cell_vecs)
        frac_vec = np.dot(cart_vec, cell_vecs_inv)
        return frac_vec

    def float_eq(self, f1, f2, prec=FLOAT_PREC):
        """float equal"""
        return abs(f1 - f2) < prec

    def cross_a2d(self, v1, v2):
        """Calculate the value of cross mult between two 2d array"""
        return v1[0]*v2[1] - v1[1]*v2[0]

    def abs_cross_a2d(self, v1, v2):
        """Calculate the abs value of cross mult between two 2d array"""
        return abs(self.cross_a2d(v1, v2))

    def calc_vectors_angle(self, v1, v2):
```

```

    """calculate the angle between two vectors"""
    # Get the dot product
    v1dotv2 = np.dot(v1, v2)
    # Get the length of v1, v2
    length_v1 = np.sqrt(np.dot(v1, v1))
    length_v2 = np.sqrt(np.dot(v2, v2))
    # Get cos_theta
    cos_theta = v1dotv2 / (length_v1 * length_v2)
    # In the case cos(theta) little larger than 1.0
    if self.float_eq(cos_theta, 1.0):
        cos_theta = 1.0
    # Get theta
    theta = np.arccos(cos_theta)
    return theta

def calc_vectors_angle_wsign(self, v1, v2):
    """calculate the angle between two vectors with the right hand rule"""
    # Get the angle
    theta = self.calc_vectors_angle(v1, v2)
    # Get the sign
    sign = np.sign(self.cross_a2d(v1, v2))
    return sign * theta

def angle_pi2degree(self, angle):
    """angle pi to degree"""
    return angle * 180 / np.pi

# +-----+
# | Read in POSCAR |
# +-----+
def _read_file_lines(self, filename=DEFAULT_IN_POSCAR):
    """Read in the file content to a list"""
    if not os.path.isfile(filename):
        self._exit("[error] Primitive POSCAR: file '%s' not found." % filename)
    with open(filename) as frp:
        lines = frp.readlines()
    return lines

def _poscar_in_direct_mode(self, lines):
    """Check if the POSCAR file is in the 'Direct' coordinates mode."""
    direct_str = lines[7]
    return ('direct' in direct_str.lower())

def _get_poscar_prim_vecs(self, lines):
    """read in the poscar primitive cell vectors"""
    length_unit = float(lines[1])
    a1_x = float(lines[2].split()[0]) * length_unit
    a1_y = float(lines[2].split()[1]) * length_unit
    a1_z = float(lines[2].split()[2]) * length_unit
    a2_x = float(lines[3].split()[0]) * length_unit
    a2_y = float(lines[3].split()[1]) * length_unit
    a2_z = float(lines[3].split()[2]) * length_unit
    a3_x = float(lines[4].split()[0]) * length_unit
    a3_y = float(lines[4].split()[1]) * length_unit
    a3_z = float(lines[4].split()[2]) * length_unit
    prim_vecs = np.array([a1_x, a1_y], [a2_x, a2_y])
    if not (self.float_eq(a1_z, 0.0) and
            self.float_eq(a2_z, 0.0) and
            self.float_eq(a3_x, 0.0) and
            self.float_eq(a3_y, 0.0)):
        self._exit("[error] Primitive POSCAR: The c axis must in z direction!")
    return prim_vecs, a3_z

```

```

def _get_poscar_atoms_coord(self, lines, total_number):
    """Get the POSCAR atoms' fractional coordinates"""
    atom_coord_list = []
    for line_i in range(8, 8+total_number):
        # Get the coord of current line's atom
        coord_a = float(lines[line_i].split()[0])
        coord_b = float(lines[line_i].split()[1])
        coord_c = float(lines[line_i].split()[2])
        curr_coord = np.array([coord_a, coord_b, coord_c])
        atom_coord_list.append(curr_coord)
    return atom_coord_list

def _get_poscar_atoms_info(self, lines):
    """Get the atoms's info in the poscar"""
    # Get the atoms' symbols
    elements_list = lines[5].split()
    # Get the atoms' number
    quantities_list = list(map(int, lines[6].split()))
    # Read in the atoms coords
    atom_coord_list = self._get_poscar_atoms_coord(lines, sum(quantities_list))
    return elements_list, quantities_list, atom_coord_list

def _record_primcell_info(self, primcell_info):
    """Record the poscar info to the poscar data list"""
    self.primcell_info_list.append(primcell_info)

def read_primcell_of_layers(self, filename):
    """Read the info in the init poscar file"""
    # Read in the content of POSCAR
    lines = self._read_file_lines(filename)
    # Check if the POSCAR is in direct coordinate mode
    if not self._poscar_in_direct_mode(lines):
        self._exit("[error] Primitive POSCAR: the atoms' coordinates MUST be in 'Direct' mode!")
    # Get the 2D vector of a1 a2
    prim_vecs, a3_z = self._get_poscar_prim_vecs(lines)
    # Get the atoms' info
    elements_list, quantities_list, atom_coord_list = \
        self._get_poscar_atoms_info(lines)
    # Create and record the result
    primcell_info = {"prim_vecs" : prim_vecs,
                    "a3_z" : a3_z,
                    "elements" : elements_list,
                    "quantities" : quantities_list,
                    "atom_coords" : atom_coord_list}
    self._record_primcell_info(primcell_info)

# +-----+
# | Generate Supercell |
# +-----+
def _get_atoms_num_in_supercell(self, super_mult_vec1, super_mult_vec2):
    """Get the atoms number in the supercell"""
    return self.abs_cross_a2d(super_mult_vec1, super_mult_vec2)

def _get_index_boder_of_atoms(self, a1p, a2p):
    """Get the border of the atoms indices searching"""
    # A border extended by 4 points: (0,0), a1p, a2p, a12p
    a12p = a1p + a2p
    a1_upper = max(0, a1p[0], a2p[0], a12p[0])
    a1_lower = min(0, a1p[0], a2p[0], a12p[0])
    a2_upper = max(0, a1p[1], a2p[1], a12p[1])
    a2_lower = min(0, a1p[1], a2p[1], a12p[1])
    a1_boder = [a1_lower, a1_upper+1]
    a2_boder = [a2_lower, a2_upper+1]
    return a1_boder, a2_boder

```

```

def _get_supercell_vecs(self, supercell_matrix, primitive_vecs, super_a3_z):
    """Get the supercell vectors"""
    # Supercell vectors in 2D
    svcs_2d = np.dot(supercell_matrix, primitive_vecs)
    supercell_vecs = np.array([[svcs_2d[0,0], svcs_2d[0,1], 0],
                               [svcs_2d[1,0], svcs_2d[1,1], 0],
                               [0, 0, super_a3_z]])

    return supercell_vecs

def _get_atoms_cell_shifts(self, a1_boder, a2_boder,
                           supercell_a1p, supercell_a2p):
    '''Get the atoms cell shifts in the supercell for each sub-primitive-cell. new_position_in_supercell =
    ↪ cell_shifts + atom_pos_in_primcell'''
    atoms_cell_shifts = []
    total_area = self.cross_a2d(supercell_a1p, supercell_a2p)
    for a1_i in range(a1_boder[0], a1_boder[1]):
        for a2_i in range(a2_boder[0], a2_boder[1]):
            shift_a1a2 = np.array([a1_i, a2_i])
            # !!! KEY code !!!
            supercell_a1_int = \
                self.cross_a2d(shift_a1a2, supercell_a2p) // total_area
            supercell_a2_int = \
                self.cross_a2d(supercell_a1p, shift_a1a2) // total_area
            if (supercell_a1_int == 0) and (supercell_a2_int == 0):
                atoms_cell_shifts.append(shift_a1a2)
    return atoms_cell_shifts

def _get_coords_z_range(self, coord_list):
    """Get the range of atoms' fractional coordinate z"""
    min_z = min(np.array(coord_list)[:,:2])
    max_z = max(np.array(coord_list)[:,:2])
    return min_z, max_z

def _get_supercell_atoms_coord(self, supercell_matrix_inv, super_a3_z, a3_z,
                               atoms_cell_shifts, atom_coord_list,
                               scell_shift_x, scell_shift_y,
                               supercell_vecs, supercell_shift_z):
    """Get the atomic fractional coordinates in the supercell"""
    # Find the minimal frac_z of the atoms
    min_frac_z, _ = self._get_coords_z_range(atom_coord_list)
    # Transform from the scell shift from the Cart to Frac
    cart_vec = [scell_shift_x, scell_shift_y, 0]
    scell_shift_x, scell_shift_y, _ = \
        self.coord_cart2frac(supercell_vecs, cart_vec)
    # generate the supercell atoms
    supercell_atom_coord_list = []
    for coord in atom_coord_list:
        for shift in atoms_cell_shifts:
            # Get the primitive cell coords
            coord_a = coord[0] + shift[0]
            coord_b = coord[1] + shift[1]
            coord_c = (coord[2] - min_frac_z) * a3_z / super_a3_z + supercell_shift_z
            primit_coord = np.array([coord_a, coord_b])
            # Get the supercell coords
            supercell_coord = np.dot(primit_coord, supercell_matrix_inv)
            supercell_coord[0] += scell_shift_x
            supercell_coord[1] += scell_shift_y
            supercell_coord = np.append(supercell_coord, coord_c)
            # Record
            supercell_atom_coord_list.append(supercell_coord)
    # Record the range of frac_z about the atoms
    frac_z_range = self._get_coords_z_range(supercell_atom_coord_list)
    return supercell_atom_coord_list, frac_z_range

```

```

def _check_supercell_cellnum(self, atoms_cell_shifts, supercell_num):
    """Check the supercell atoms' number"""
    check_supercell_num = len(atoms_cell_shifts)
    if check_supercell_num != supercell_num:
        self._exit("[error] Supercell generation: expect %d primitive cell in supercell, but find %d..."
                    %(supercell_num, check_supercell_num))

def gen_supercell(self, super_mult_vec1, super_mult_vec2,
                  primitive_vecs, a3_z, quantities_list,
                  atom_coord_list, super_a3_z=20.0, scell_shift_x=0.0,
                  scell_shift_y=0.0, supercell_shift_z=0.0):
    """Generate the supercell"""
    # Supercell number
    supercell_num = self._get_atoms_num_in_supercell(super_mult_vec1,
                                                    super_mult_vec2)

    # Supercell number list
    supercell_quantities_list = [val*supercell_num for val in quantities_list]
    # Supercell transform matrix
    supercell_matrix = np.array([super_mult_vec1, super_mult_vec2])
    supercell_matrix_inv = np.linalg.inv(supercell_matrix)
    # Supercell vectors
    supercell_vecs = \
        self._get_supercell_vecs(supercell_matrix, primitive_vecs, super_a3_z)
    # Calculate supercell shifts according to supercell vector a1',a2'
    #-> Get the border of the atomic index searching
    a1_boder, a2_boder = \
        self._get_index_boder_of_atoms(super_mult_vec1, super_mult_vec2)
    #-> Find all atoms' shift vector in the supercell
    atoms_cell_shifts = self._get_atoms_cell_shifts(a1_boder, a2_boder,
                                                    super_mult_vec1,
                                                    super_mult_vec2)

    #-> Check the atom number in supercell
    self._check_supercell_cellnum(atoms_cell_shifts, supercell_num)
    # Get fractional coordinates in supercell (min-z = 0.0)
    supercell_atom_coord_list, frac_z_range = \
        self._get_supercell_atoms_coord(supercell_matrix_inv, super_a3_z, a3_z,
                                         atoms_cell_shifts, atom_coord_list,
                                         scell_shift_x, scell_shift_y,
                                         supercell_vecs, supercell_shift_z)

    return supercell_vecs, supercell_quantities_list, \
        supercell_atom_coord_list, frac_z_range

# +-----+
# | Twist Layers |
# +-----+
def add_layer(self, super_a1_mult, super_a2_mult,
              layer_dis=2, scs_x=0.0, scs_y=0,
              prim_poscar=DEFAULT_IN_POSCAR):
    """Read in the the layers' parameters in supercell"""
    # Read in the primitive cell info
    self.read_primcell_of_layers(prim_poscar)
    # Read in the supercell info
    curr_supercell_info = {"super_mult_a1" : np.array(super_a1_mult),
                          "super_mult_a2" : np.array(super_a2_mult),
                          "layer_distance" : layer_dis,
                          "scell_shift_x" : scs_x,
                          "scell_shift_y" : scs_y,}
    self.supercell_info_list.append(curr_supercell_info)

def _check_primvecs_in_layers(self):
    """Check the primitive cell vectors in different layers"""
    # Get the reference vector of primitive cell
    ref_primcell_vecs = self.primcell_info_list[0]["prim_vecs"]
    ref_p_a1 = ref_primcell_vecs[0]
    ref_p_a2 = ref_primcell_vecs[1]

```

```

# Check each primitive cell vectors
for i, primcell_info in enumerate(self.primcell_info_list):
    layer_i = i + 1
    prim_vecs = primcell_info["prim_vecs"]
    p_a1 = prim_vecs[0]
    p_a2 = prim_vecs[1]
    theta_1 = self.calc_vectors_angle_wsign(ref_p_a1, p_a1)
    theta_2 = self.calc_vectors_angle_wsign(ref_p_a2, p_a2)
    # Discuss the different cases:
    if theta_1 * theta_2 < 0.0:
        print("[warning] Layer %d: the chirality of this layer's primitive cell vectors do not agree with the 1st
        ↪ layer's. Be careful when pick the super_a1,2_mult vector of this layer." %layer_i)
    if not self.float_eq(abs(theta_1), abs(theta_2)):
        print("[warning] Layer %d: the angle between primitive cell vectors a1,a2 in this layer do not agree with
        ↪ the 1st layer's. Be careful when pick the super_a1,2_mult vector of this layer." %layer_i)

def twist_layers(self, start_z=0.0, super_a3_z=20.0):
    """Generate the twisted layers atoms fractional coordinates."""
    # Check the layers firstly
    self._check_primvecs_in_layers()
    # For each twist layers
    supercell_shift_z = start_z
    for i, supercell_info in enumerate(self.supercell_info_list):
        primcell_info = self.primcell_info_list[i]
        # Read in necessary supercell information
        super_mult_a1 = supercell_info["super_mult_a1"]
        super_mult_a2 = supercell_info["super_mult_a2"]
        layer_dis = supercell_info["layer_distance"]
        scell_shift_x = supercell_info["scell_shift_x"]
        scell_shift_y = supercell_info["scell_shift_y"]
        primitive_vecs = primcell_info["prim_vecs"]
        a3_z = primcell_info["a3_z"]
        quantities_list = primcell_info["quantities"]
        atom_coord_list = primcell_info["atom_coords"]
        # Generate the supercell
        supercell_vecs, supercell_quantities_list, \
            supercell_atom_coord_list, frac_z_range = \
            self.gen_supercell(super_mult_a1, super_mult_a2, primitive_vecs,
                               a3_z, quantities_list, atom_coord_list, super_a3_z,
                               scell_shift_x, scell_shift_y, supercell_shift_z)
        # Update the supercell shift z
        curr_max_atom_z = frac_z_range[1]
        if curr_max_atom_z > 1.0: # Not use supercell_shift_z to avoid mistake near the bound.
            self._exit("[error] Twisting layers: coordinate z is out of range, pls reduce the layer distance or
            ↪ increase the cell length of z.")
        supercell_shift_z = curr_max_atom_z + (layer_dis / super_a3_z)
        # Record the current rotated supercell info
        self.supercell_info_list[i]["supercell_vecs"] = supercell_vecs
        self.supercell_info_list[i]["supercell_quantities_list"] = \
            supercell_quantities_list
        self.supercell_info_list[i]["supercell_atom_coord_list"] = \
            supercell_atom_coord_list

def calc_layers_twist_angles(self):
    """Calculate the layers twisted angles"""
    twisted_angles = []
    ref_vec_a1 = self.supercell_info_list[0]["supercell_vecs"][0]
    for supercell_info in self.supercell_info_list:
        supercell_vec_a1 = supercell_info["supercell_vecs"][0]
        theta = self.calc_vectors_angle(ref_vec_a1, supercell_vec_a1)
        theta = self.angle_pi2degree(theta)
        twisted_angles.append(theta)
    return twisted_angles

def calc_layers_strain(self):

```

```

    """calculate the strain added in each layer (volume changes)"""
    layers_vol_changes = []
    ref_svec = self.supercell_info_list[0]["supercell_vecs"]
    ref_area = self.abs_cross_a2d(ref_svec[0][:2], ref_svec[1][:2])
    for scell_info in self.supercell_info_list:
        svec = scell_info["supercell_vecs"]
        area = self.abs_cross_a2d(svec[0][:2], svec[1][:2])
        vol_change = (ref_area / area) - 1
        layers_vol_changes.append(vol_change)
    return layers_vol_changes

# +-----+
# | POSCAR Output |
# +-----+
def _find_element_in_2dlist(self, list_2d, targ_ele):
    """Find the element index in a 2d list"""
    res_indeies = []
    for i, list_1d in enumerate(list_2d):
        for j, curr_ele in enumerate(list_1d):
            if targ_ele == curr_ele:
                res_indeies.append([i, j])
    return res_indeies

def _combine_poscars(self):
    """Combine the elements list in different layers' POSCAR"""
    # Get symbol list list
    elements_list_list = [
        self.primcell_info_list[i]["elements"]
        for i in range(len(self.primcell_info_list))
    ]
    # Get atoms number list list
    quantities_list_list = [
        self.supercell_info_list[i]["supercell_quantities_list"]
        for i in range(len(self.supercell_info_list))
    ]
    # Get atoms coordinates
    atom_coords_list_list = [
        self.supercell_info_list[i]["supercell_atom_coord_list"]
        for i in range(len(self.supercell_info_list))
    ]
    # Output the orginzed POSCAR info
    org_elements_list = []
    org_quantities_list = []
    org_atom_coords_list = []
    for poscar_i, elements_list in enumerate(elements_list_list):
        for sym_i, sym in enumerate(elements_list):
            # If this symbol has already be recorded
            if sym in org_elements_list:
                continue
            # Find if there are any similar elements in other poscars
            sym_indeies = self._find_element_in_2dlist(elements_list_list, sym)
            sym_num = 0
            for row_i, col_i in sym_indeies:
                curr_sym_num = quantities_list_list[row_i][col_i]
                sym_num += curr_sym_num
                start_i = sum(quantities_list_list[row_i][:col_i])
                org_atom_coords_list += \
                    atom_coords_list_list[row_i][start_i:start_i+curr_sym_num]
            # Record the symbol and number to the list
            org_elements_list.append(sym)
            org_quantities_list.append(sym_num)
    return org_elements_list, org_quantities_list, org_atom_coords_list

def write_res_to_poscar(self, filename=DEFAULT_OUT_POSCAR):
    """Write the output POSCAR"""

```

```

# Generate POSCAR
out_poscar_lines = []
# 1st comment line
out_poscar_lines.append("Generated by Twist2D")
out_poscar_lines.append("    1.0")
# Cell vectors
supercell_vecs = self.supercell_info_list[0]["supercell_vecs"]
for vec in supercell_vecs:
    out_poscar_lines.append(
        "    %13s    %13s    %13s" %("%4.8f"%vec[0],
                                     "%4.8f"%vec[1],
                                     "%4.8f"%vec[2]))

# Atoms symbol list
org_elements_list, org_quantities_list, org_atom_coords_list = \
    self._combine_poscars()
org_elements_list_str = ' '.join(org_elements_list)
out_poscar_lines.append("    %s" %org_elements_list_str)
# Atoms number list
org_quantities_list = [str(val) for val in org_quantities_list]
org_quantities_list_str = ' '.join(org_quantities_list)
out_poscar_lines.append("    %s" %org_quantities_list_str)
# Direct line
out_poscar_lines.append("Direct")
# Atoms coords
for coord in org_atom_coords_list:
    out_poscar_lines.append(
        "    % .8f    % .8f    % .8f" %coord[0], coord[1], coord[2]))
# Add \n in each end of line
for line_i, _ in enumerate(out_poscar_lines):
    out_poscar_lines[line_i] += '\n'
# Write the POSCAR to file
print("[do] Write the twisted structure to %s ..." %filename)
with open(filename, 'w') as fwp:
    fwp.writelines(out_poscar_lines)

#####
### Special System ###
#####
class TwistBGL(Twist2D):
    def _check_angle(self, angle):
        # Get the vector of primitive cell
        for i, primcell_info in enumerate(self.primcell_info_list):
            layer_i = i + 1
            prim_vecs = primcell_info["prim_vecs"]
            p_a1 = prim_vecs[0]
            p_a2 = prim_vecs[1]
            # Calculate the angle
            phi = self.calc_vectors_angle(p_a1, p_a2)
            phi = self.angle_pi2degree(phi)
            # Compare to the target angle
            if not self.float_eq(phi, angle, prec=1E-3):
                self._exit("[error] Layer %d: the primitive cell vectors' angle must be 60 degree, current is %f."
                           % (layer_i, phi))

def add_graphenelike_layers(self, m, n, layer_dis, scs_x, scs_y, prim_poscar):
    """Write in the graphene like supercell vectors"""
    # 1st layer
    super_a1_mult = [m, n]
    super_a2_mult = [-n, m+n]
    self.add_layer(super_a1_mult, super_a2_mult, layer_dis, scs_x, scs_y, prim_poscar)
    # 2nd layer
    super_a1_mult = [n, m]
    super_a2_mult = [-m, n+m]
    self.add_layer(super_a1_mult, super_a2_mult, layer_dis, scs_x, scs_y, prim_poscar)
    # Check if the primitive vector angle is 60 degree
    self._check_angle(60)

```

```

def gen_TBGL(self, m, n,
              prim_poscar=DEFAULT_IN_POSCAR,
              poscar_out=DEFAULT_OUT_POSCAR,
              start_z=0.1, super_a3_z=20.0, layer_dis=2.0,
              scs_x=0.0, scs_y=0.0):
    """Generate the twisted bilayer graphene(TBG) system."""
    self.add_graphenelike_layers(m, n, layer_dis, scs_x, scs_y, prim_poscar)
    self.twist_layers(start_z, super_a3_z)
    # Update the out POSCAR name
    if poscar_out == DEFAULT_OUT_POSCAR:
        poscar_out = 'POSCAR.T2D-%dx%d.vasp' %(m, n)
    # Save the data to out POSCAR
    self.write_res_to_poscar(poscar_out)

```

(改编脚本) 快速计算投影能带

► 代码清单 A.74.1 (python, 651 行)

```

import numpy as np
import math
import pymatgen
import sys
import argparse
import logging
import os.path

from pymatgen.io.vasp.outputs import Procar, Vasprun
from pymatgen.io.vasp.inputs import Kpoints
from pymatgen.core import Structure
from pymatgen.electronic_structure.core import Spin, Orbital

import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
from matplotlib.gridspec import GridSpec

class SaneFormatter(argparse.RawTextHelpFormatter,
                   argparse.ArgumentDefaultsHelpFormatter):
    pass

parser = argparse.ArgumentParser(prog='plot_fatbands.py',description=("Plot projected band structure (fatbands)
↳ from a VASP calculation.\n" +
    'Author: Marco Cappelletti. Inspired by Kevin Waters (kwaters4.github.io) and sumo-bandplot.\n'
    ↳ +
    'By default it assumes that:\n' +
    '\tthe current directory contains KPOINTS and vasprun.xml from the band structure calculation\n'
    ↳ +
    '\tthe directory ../dos contains vasprun.xml from the DOS calculation\n' +
    '\tthe parent directory (../) contains the POSCAR file\n' +
    '\tthe grandparent directory (../..) contains the POTCAR file\n')
    ,formatter_class=SaneFormatter)
parser.add_argument('-B','--vasprun-file-bands', type=str, help='Path of the vasprun.xml file of the band
↳ calculation', default='vasprun.xml')
parser.add_argument('-K','--KPOINTS-file', type=str, help='Path of the KPOINTS file with the band path',
↳ default='KPOINTS')
parser.add_argument('-C','--POSCAR-file', type=str, help='Path of the POSCAR file', default='POSCAR')
parser.add_argument('-O','--PROCAR-file', type=str, help='Path of the PROCAR file from the band calculation',
↳ default='PROCAR')
parser.add_argument('-P','--POTCAR-file', type=str, help='Path of the POTCAR file', default='POTCAR')

```

```

parser.add_argument('-D','--vasprun-file-dos', type=str, help='Path of the vasprun.xml file of the dos
↳ calculation', default='../dos/vasprun.xml')
parser.add_argument('-p','--project', help='Band projections. Accepts 1-5 arguments. Nomenclature:\n'
'\t- E/n: all orbitals of element with symbol E (H, C, N, ...) or atom index n (1, 2,
↳ ...) \n'
'\t- E/n.o: o-orbital of element with symbol E or atom index n
↳ (s, px, py, pz, dxy, ...) \n'
'\t- E/n.s.pz: s+pz orbitals of element with symbol E or atom
↳ index n \n'
'\t- X.s: s orbitals of all elements (literally X, see
↳ example) \n'
'\t- 0.s.pz+N.pz: sum of 0(s,pz) and N(pz) \n'
'You are free to mix anything you want. There is no
↳ restriction. \n'
'Examples (a number represents an atom index, starting from 1,
↳ as in PROCAR): \n'
'\t-p N 0.s.pz+H.s C+X.pz =>'
' color 1: N(all orb.), color 2: 0(s+pz)+H(s), color 2: C(all
↳ orb.)+(all atoms pz) \n'
'\t-p 1+2+3 4.px.py.pz+5 H.s =>'
' color 1: 1(all orb.)+2(all orb.)+3(all orb.), color 2:
↳ 4(px+py+pz)+5+(px), color 3: H(s) \n'
, nargs='+', default=['X.px', 'X.py', 'X.s.pz'])
parser.add_argument('-t','--projection-type', type=str, help=f'Type of projection. \n'
'\t- blend: plot projections into the additive (r,g,b) space,
↳ or into the subtractive (c,m,y) space. \n'
'\t- stack: plot stacked colored circles, with radius
↳ depending on contribution. \n',
choices=['blend','stack'], default='stack')
parser.add_argument('--split', help='Split projections in different plots.', action='store_true')
parser.add_argument('-n','--normalization', type=str, help=f'Normalization of the projection. \n'
'\t- all: with respect to all contributions. \n'
'\t- selection: with respect to selection only. \n', choices=['all','selection'],
↳ default='selection')
parser.add_argument('-l','--max-l', type=int, default=1, choices=[1,2,3],
help='Maximum value of l (angular momentum) for the projection. \n'
Increases computational costs, so increase it only if necessary.')
parser.add_argument('-N','--no-projection', help='Do not perform any projection', action='store_true')
parser.add_argument('-m','--emin', type=float, help='Minimum of energy in the plot. If none, it chooses the lower
↳ limit', default=None)
parser.add_argument('-M','--emax', type=float, help='Maximum of energy in the plot. If none, it chooses the upper
↳ limit', default=None)
parser.add_argument('-s','--scale', type=float, help='DOS scale factor', default=1.0)
parser.add_argument('-H','--height', type=float, help='Height of the plot in inches', default=3.5)
parser.add_argument('-W','--width', type=float, help='Width of the plot in inches', default=3.3)
parser.add_argument('-r','--ratio', type=float, help='Bandplot - dosplot width ratio', default=3.0)
parser.add_argument('--blw','--band-lw', type=float, help='Linewidth of non-projected bands', default=2.0)
parser.add_argument('--bs','--band-size', type=float, help='Circle size of bands, when projection is on.',
↳ default=2.0)
parser.add_argument('--ba','--band-alpha', type=float, help='alpha value (transparency) of bands', default=1.0)
parser.add_argument('--dlw','--dos-lw', type=float, help='Linewidth of DOS', default=1.25)
parser.add_argument('--flw','--Fermi-lw', type=float, help='Linewidth of Fermi level. Set it to 0 to remove it',
↳ default=1.0)
parser.add_argument('--vlw','--vlines-lw', type=float, help='Linewidth of vertical lines. Set it to 0 to remove
↳ them', default=1.0)
parser.add_argument('--vla','--vlines-alpha', type=float, help='alpha value (transparency) of vertical lines.',
↳ default=1.0)
parser.add_argument('--glw','--grid-lw', type=float, help='Linewidth of grid. Set it to 0 to remove them',
↳ default=1.0)
parser.add_argument('--gla','--grid-alpha', type=float, help='alpha value (transparency) of grid lines.',
↳ default=1.0)
parser.add_argument('-f','--font-size', type=float, help='Fontsize', default=7)
parser.add_argument('--c-mode','--color-mode', type=str,
help='Color mode for blended fatbands. rgb (additive) = red, green, blue; cmY
↳ (subtractive) = cyan, magenta, yellow.',
choices=['rgb','cmY'], default='rgb')
parser.add_argument('--cmap','--colormap', type=str,
help='Color scheme for stacked fatbands.',

```

```

        choices=['rgb','saturated','normal','pale','dark','alt','darkalt','accent','plt']
        ↪ , 'sumo'], default='rgb')
parser.add_argument('--cord','--color-order', nargs='+', type=int, help='If you like to change the color order.\
        E.g. --cmap rgbop --cord 3 2 1 4 5 means blue,\
        ↪ green, red, orange, purple.\
        Note: list of 5 integers regardless the number of\
        ↪ projections! And first color is 1!!',\
        default=[1,2,3,4,5])
parser.add_argument('--custc','--custom-colors', nargs='+', help='Custom colors, as in matplotlib. Examples:\n'
        '\t--custc red green blue\n'
        '\t--custc brown lime indigo', default='None')
parser.add_argument('-o','--output-file', type=str, help='Path and name of the output file, excluding the format',
        ↪ default='fatbands')
parser.add_argument('--format', type=str, help='Output file format', choices=['pdf','png'], default='pdf')
parser.add_argument('--redo','--readlog', help='Rerun the last command, if plot_fatbands.log file is present.\
        ↪ This overrides every other argument!', action='store_true')

args = parser.parse_args()
redo = args.redo

if redo is True:
    if os.path.isfile(f'plot_{args.output_file}.log') is False:
        raise ValueError(f'I cannot rerun: plot_fatbands_{args.output_file}.log does not exist')

    with open(f'plot_{args.output_file}.log') as f:
        argstr = f.readline()

    args = parser.parse_args(argstr.split()[1:])

logging.basicConfig(
    filename=f'plot_{args.output_file}.log',
    level=logging.INFO,
    filemode="w",
    format="%(message)s",
)

if redo is True:
    logging.info(argstr)
else:
    logging.info(" ".join(sys.argv[1:]))

scale = args.scale
no_proj = args.no_projection
N_proj = len(args.project)

# Set colors -----

if args.custc == 'None':
    match args.cmap:
        case 'rgb':
            colors = [[1,0,0], [0,1,0], [0,0,1], [1, 0.50, 0.14], [0.58,0.40,0.74]]
            color_names = {0: 'red', 1: 'green', 2: 'blue', 3: 'orange', 4: 'purple'}
        case 'saturated':
            colors = [mcolors.BASE_COLORS['r'], mcolors.BASE_COLORS['g'], mcolors.BASE_COLORS['b'],
                    mcolors.TABLEAU_COLORS['tab:orange'], mcolors.BASE_COLORS['m']]
            color_names = {0: 'red', 1: 'green', 2: 'blue', 3: 'orange', 4: 'magenta'}
        case 'pale':
            colors = [mcolors.TABLEAU_COLORS['tab:red'], mcolors.TABLEAU_COLORS['tab:green'],
                    ↪ mcolors.TABLEAU_COLORS['tab:blue'],
                    mcolors.TABLEAU_COLORS['tab:orange'], mcolors.TABLEAU_COLORS['tab:purple']]
            color_names = {0: 'red', 1: 'green', 2: 'blue', 3: 'orange', 4: 'purple'}
        case 'normal':
            colors = [plt.cm.Set1.colors[0], plt.cm.Set1.colors[2], plt.cm.Set1.colors[1], plt.cm.Set1.colors[4],
                    ↪ plt.cm.Set1.colors[3]]

```

```

        color_names = {0: 'red', 1: 'green', 2: 'blue', 3: 'orange', 4: 'purple'}
    case 'dark':
        colors = [mcolors.CSS4_COLORS['darkred'], mcolors.CSS4_COLORS['darkgreen'],
        ↪ mcolors.CSS4_COLORS['darkblue'],
            mcolors.CSS4_COLORS['chocolate'], mcolors.CSS4_COLORS['purple']]
        color_names = {0: 'red', 1: 'green', 2: 'blue', 3: 'orange', 4: 'purple'}
    case 'alt':
        colors = [plt.cm.Set2.colors[i] for i in range(5)]
        color_names = {0: 'teal', 1: 'orange', 2: 'blue', 3: 'pink', 4: 'green'}
    case 'darkalt':
        colors = [plt.cm.Dark2.colors[i] for i in range(5)]
        color_names = {0: 'teal', 1: 'orange', 2: 'blue', 3: 'pink', 4: 'green'}
    case 'accent':
        colors = [plt.cm.Accent.colors[i] for i in range(5)]
        color_names = {0: 'green', 1: 'lilac', 2: 'orange', 3: 'yellow', 4: 'blue'}
    case 'plt':
        colors = [plt.cm.tab10.colors[i] for i in range(5)]
        color_names = {0: 'blue', 1: 'orange', 2: 'green', 3: 'red', 4: 'purple'}
    case 'sumo':
        colors = ["#3952A3", "#FAA41A", "#67BC47", "#6ECCDD", "#ED2025"]
        color_names = {0: 'blue', 1: 'orange', 2: 'green', 3: 'red', 4: 'purple'}

    if args.projection_type == 'blend':
        colors = np.array([[1,0,0], [0,1,0], [0,0,1]])
        color_names = {0: 'red', 1: 'green', 2: 'blue'}
        if args.c_mode == 'cmy':
            colors = 1 - colors
            color_names = {0: 'cyan', 1: 'magenta', 2: 'yellow'}

    else:
        colors = [ mcolors.CSS4_COLORS[cval] for cval in args.custc ]

    color_order = {0: args.cord[0]-1, 1: args.cord[1]-1, 2: args.cord[2]-1, 3: args.cord[3]-1, 4: args.cord[4]-1}

    # -----

def CheckInput():
    # accepts only 1-3 entries for the projection
    if N_proj < 1 or N_proj > 3:
        if args.split is False:
            raise ValueError('Only 1, 2 or 3 components allowed for the non-split projection')
        elif N_proj < 2 or N_proj > 5:
            raise ValueError('2-5 components allowed for the split projection')
    elif N_proj == 1 and args.normalization == 'selection':
        print('\tWARNING: you selected 1 projection and \'selection\' normalization. This does not make sense.
        ↪ Normalization is changed to \'all\'')
        logging.info('WARNING: normalization is changed to \'all\'')
        args.normalization='all'

    if args.split is True and args.projection_type == 'blend':
        raise ValueError('Only stacked projection allowed for split')

    if args.split is True and no_proj is True:
        raise ValueError('Cannot split non-projected bands!')

def plotcircles(ax, KPOINTS, band_energies, contrib, index):

    if args.split is False and args.projection_type == 'stack':
        sizes = args.bs**2 * contrib[:, :, index:3].sum(axis=2)**2
    elif args.split is True:
        sizes = args.bs**2 * contrib**2

    for bidx, be in enumerate(band_energies):

```

```

if args.projection_type == 'blend':
    ax.scatter(KPOINTS,be,s=args.bs/2,color=colors[color_order[index]],alpha=contrib[bidx,:,index]**2,edgec
    ↪ ecolor='none')
elif args.projection_type == 'stack':
    ax.plot(KPOINTS,be,lw=args.blw/4,color='k',alpha=args.ba,zorder=0)
    ax.scatter(KPOINTS,be,s=sizes[bidx],color=colors[color_order[index]],edgecolor='none')

def CalculateProjectionsAndPlot():

    # calculates contributions for bands and DOS projections
    el_orbs = []
    el_orbs_labels = []

    # read the projections
    # 1-5 components, e.g. 'N.s.pz', 'N.s+0.s.pz', 'N+0.s', 'N+0', 'X', 'X.s', 'X.s+N.px.py.pz', ...
    for component in args.project:
        element_components = component.split("+") # split elements, e.g. 'N.s+0.s.pz' becomes ['N.s', '0.s.pz']
        color_component = [] # list of components for one color
        label_text = [] # more readable, user-friendly text for the legend
        for element_component in element_components: # e.g. for element in the list ['N.s', '0.s.pz'] (example
            ↪ above)
            splits = element_component.split(".") # e.g. 'N.s.pz' converted to ['N', 's', 'pz']
            element = splits[0] # first element is the atom symbol, e.g. 'N'
            if len(splits) == 1: # if splits has one element, then no orbitals are specified
                ↪ (e.g. 'N') => plot all orbitals (= 'all')
                orbitals = 'all'
            else:
                orbitals = splits[1:] # e.g. ['s', 'pz']
                color_component.append([element,orbitals])

        if element == 'X': # 'X' represents all atoms
            if len(splits) == 1: # if input is only 'X'
                label_text.append('s+p_x+p_y+p_z') # this makes no sense to choose, it's the projection
                ↪ of all atoms and orbitals, but who am I to judge
            else:
                label_text.append(".".join(orbitals).replace('p','p_')) # more readable, e.g. 'X.s.pz'
                ↪ becomes 's+p_z'
        else:
            if len(splits) == 1: # all orbitals, only the element: 'N'; or the index:
                ↪ '2'
                if element.isnumeric() is True:
                    label_text.append(atom_labels[int(element)-1]+element)
                else:
                    label_text.append(element)
            else:
                if element.isnumeric() is True:
                    label_text.append(f"{atom_labels[int(element)-1]+element}({'.'.join(orbitals).replace('p'
                    ↪ ', 'p_'))}")
                else:
                    label_text.append(f"{element}({'.'.join(orbitals).replace('p','p_'))}") # e.g. '0.s.pz'
                    ↪ becomes '0(s+p_z)'
        el_orbs.append(color_component)
        el_orbs_labels.append(".".join(label_text)) # e.g. '0.s.pz+H.s' becomes '0(s+p_z)+H(s)'

    logging.info(f'Calculating contributions for orbitals with angular momentum up to l = {args.max_l}. Ignoring
    ↪ the rest if l < 3')
    if args.normalization == 'selection':
        logging.info(f'Normalization mode is {args.normalization}: selected contributions sum to 1. Other
        ↪ contributions (if any) will not be visible in the plot!')
    else:
        logging.info(f'Normalization mode is {args.normalization}: selected contributions are normalized with
        ↪ respect to all contributions.')

```

```
print(f'\tColormap: {args.cmap}')
print(f'\tBands are projected into {N_proj} contributions:')
logging.info(f'Bands are projected into {N_proj} contributions:')
for color_idx, color_contrib in enumerate(el_orbs_labels):
    print(f'\t\t - color {color_names[color_order[color_idx]]:\t{color_contrib}')
    logging.info(f'\t-color {color_names[color_order[color_idx]]:\t{color_contrib}')

print(f'\tNormalization mode is: \'{args.normalization}\')
print(f'\tProjection type is: \'{args.projection_type}\')


logging.info('\n-----')
logging.info('Contribution table for high-symmetry points (only for plotted bands)')

logging.info(f'\nband n' + '\t' + '\t'.join(labels))

# as in VASP - 修改轨道名称映射
orbital_values = { 's': 0,
                  'py': 1, 'pz': 2, 'px': 3,
                  'dxy': 4, 'dyz': 5, 'dz2': 6, 'dxz': 7, 'x2-y2': 8 } # 修改这里: x2_y2 -> x2-y2

# 动态检测 PROCAR 的实际轨道数量
try:
    # 获取第一个原子的轨道数量作为参考
    actual_orbital_count = len(data[Spin.up][0][0][0])
    print(f" 检测到 PROCAR 文件包含 {actual_orbital_count} 个轨道")
    # 使用实际轨道数量或理论最大值中的较小值
    max_l_index = min((args.max_l + 1)**2, actual_orbital_count)
except:
    max_l_index = min((args.max_l + 1)**2, 9) # 安全回退

# contributions to the band per each band, k-point, and color: contrib_bands[band][k-point][color]
contrib_bands = np.zeros((max_band_to_plot-min_band_to_plot+1, len(bands.kpoints), N_proj))

# contributions to the DOS per each color: contrib_dos[color][energy]
contrib_dos = np.zeros((N_proj, len(dosrun.pdos[0][Orbitals.s][Spin.up])))

# obtain contributions
# sum over all bands to plot
for b in range(max_band_to_plot-min_band_to_plot+1):
    # sum over all k-points
    for k in range(len(bands.kpoints)):
        for color_idx, color_contrib in enumerate(el_orbs): # color_idx: 1-5. color_contrib: list of
            ↪ [elements, orbitals] for each color
            for element_contrib in color_contrib:
                element = element_contrib[0]          # e.g. 'X', 'N'
                orbitals = element_contrib[1]         # e.g. 'all', 's', ['s', 'pz']
                if element == 'X':                    # if all atoms, get all indexes
                    element_indexes = range(len(atom_labels))
                elif element.isnumeric() is True:
                    element_indexes = [int(element)-1]
                else:                                  # else, get the indexes with label = atom symbol (e.g.
                    ↪ 'C')
                    element_indexes = [i for i, x in enumerate(atom_labels) if x == element]

            if orbitals == 'all':
                orbital_indexes = range(0, max_l_index) # sum all orbitals if 'all'
            else:
                orbital_indexes = [orbital_values[o] for o in orbitals if o in orbital_values]

            for i in element_indexes:
                for j in orbital_indexes:
                    # 更安全的边界检查 - 关键修改
                    try:
```

```

        # 先检查所有索引是否在有效范围内
        if (i < len(data[Spin.up][k][b+min_band_to_plot]) and
            j < len(data[Spin.up][k][b+min_band_to_plot][i])):
            contrib_bands[b,k,color_idx] += data[Spin.up][k][b+min_band_to_plot][i][j]**2

    if k == 0 and b == 0: # needs to be done just once
        try:
            # DOS 部分也添加边界检查
            if (i < len(dosrun.pdos) and
                j < len(dosrun.pdos[i])):
                contrib_dos[color_idx] +=
                    ↪ np.array(dosrun.pdos[i][Orbital(j)][Spin.up])
        except (IndexError, KeyError):
            pass # 如果出错就跳过
    except (IndexError, KeyError):
        pass # 如果任何索引访问出错, 跳过

# normalization
if args.normalization == 'selection':
    if np.sum(contrib_bands[b,k,:]) != 0:
        contrib_bands[b,k,:] = contrib_bands[b,k,:]/np.sum(contrib_bands[b,k,:])
    elif k in labels_kpt_num+1: # I don't know why, VASP prints all contribution to zero at those
        ↪ points
        contrib_bands[b,k,:] = contrib_bands[b,k-1,:]

    # print contributions at high-symmetry point to log file
elif args.normalization == 'all':
    tot = 0.0
    for i in range(len(atom_labels)):
        for j in range(0, min(max_l_index, 9)): # 限制最大为 9 个轨道
            try:
                if (i < len(data[Spin.up][k][b+min_band_to_plot]) and
                    j < len(data[Spin.up][k][b+min_band_to_plot][i])):
                    tot += data[Spin.up][k][b+min_band_to_plot][i][j]**2
            except (IndexError, KeyError):
                pass
        if tot != 0:
            contrib_bands[b,k,:] = contrib_bands[b,k,:]/tot
if b+min_band_to_plot < 9:
    b_str = ' ' + str(b+min_band_to_plot+1)
elif b+min_band_to_plot < 99:
    b_str = ' ' + str(b+min_band_to_plot+1)
else:
    b_str = str(b+min_band_to_plot+1)
logging.info(f'{b_str}: ' + '\t'
            + '\t'.join([' ' + ', '.join(f'{x:.2f}' for x in contrib_bands[b, lkn, :])
                        + ''] for lkn in labels_kpt_num))

# Plotting -----

bands_to_plot = bands.bands[Spin.up][min_band_to_plot:max_band_to_plot+1] - bands.efermi

if args.split is False:
    if args.projection_type == 'stack':
        for proj_n in range(N_proj):
            plotcircles(ax_bands,
                        KPOINTS,
                        bands_to_plot,
                        contrib_bands,
                        proj_n)

            ax_DOS.plot(contrib_dos[proj_n], dosrun.tdos.energies - dosrun.efermi, \
                        c=colors[color_order[proj_n]], label = f'${el_orbs_labels[proj_n]}$', linewidth = args.dlw)
    elif args.projection_type == 'blend':
        for proj_n in range(N_proj):
            plotcircles(ax_bands,
                        KPOINTS,

```

```

        bands_to_plot,
        contrib_bands,
        proj_n)

    ax_DOS.plot(contrib_dos[0], dosrun.tdos.energies - dosrun.efermi, \
        c=colors[color_order[0]], label = f'${el_orbs_labels[0]}$', linewidth = args.dlw)
    ax_DOS.plot(contrib_dos[1], dosrun.tdos.energies - dosrun.efermi, \
        c=colors[color_order[1]], label = f'${el_orbs_labels[1]}$', linewidth = args.dlw)
    if len(el_orbs) == 3:
        ax_DOS.plot(contrib_dos[2], dosrun.tdos.energies - dosrun.efermi, \
            c=colors[color_order[2]], label = f'${el_orbs_labels[2]}$', linewidth = args.dlw)

else:
    for proj_n in range(N_proj):
        ax = ax_bands[proj_n]
        plotcircles(ax,
            KPOINTS,
            bands_to_plot,
            contrib_bands[:, :, proj_n],
            proj_n)

        # plot DOS
        ax_DOS.plot(contrib_dos[proj_n], dosrun.tdos.energies - dosrun.efermi, \
            c=colors[color_order[proj_n]], label = f'${el_orbs_labels[proj_n]}$', linewidth = args.dlw)

# -----

#----- Program starts -----

if __name__ == "__main__":
    print('--- plot_fatbands.py -----')
    print(f'\tPlotting fatbands into {args.output_file}.{args.format}. Size: {args.width} x {args.height}'
        ↪ ' inches.')
    if redo is True:
        print(f'\tRedoing previous run: {argstr}', end='')
    print(f'\tAdditional data is printed in plot_{args.output_file}.log')
    logging.info(f'Plotting fatbands into {args.output_file}.{args.format}. Size: {args.width} x {args.height}'
        ↪ ' inches.\n')

    CheckInput()

    # Read files -----

    # Load Structure
    structure = Structure.from_file(args.POSCAR_file)
    # 修改原子标签获取方式 - 兼容新版 pymatgen
    try:
        atom_labels = [site.species_string for site in structure]
    except AttributeError:
        atom_labels = structure.labels

    # Load Band Structure Calculations
    bands =
    ↪ Vasprun(args.vasprun_file_bands, parse_potcar_file=args.POTCAR_file).get_band_structure(args.KPOINTS_file,
    ↪ line_mode = True)
    # Read KPOINTS file with path
    kpts = Kpoints.from_file(args.KPOINTS_file)

```

```

if no_proj is False:
    # projected bands
    data = Procar(args.PROCAR_file).data

# density of states
dosrun = Vasprun(args.vasprun_file_dos, parse_potcar_file=args.POTCAR_file)

# -----

# Interpret k-points -----

# k-point labels
n_labels = len(kpts.labels)
labels = []
labels.append(kpts.labels[0])
for label_idx in range(1, n_labels, 2):
    labels.append(kpts.labels[label_idx])

# get kpoint number for each high symmetry point
labels_kpt_num = np.zeros(len(labels), dtype=int)
lab_idx = 0
for idx, bkpt in enumerate(bands.kpoints):
    if bkpt.label is not None and idx-1 not in labels_kpt_num: # usually they are duplicated, I just take the
        ↪ first
        labels_kpt_num[lab_idx] = idx
        lab_idx += 1

# Number of points between kpoints, found in the KPOINTS file
step = kpts.num_kpts

# -----

# Initialize plot -----

# general options for plot
font = {'family': 'serif', 'size': args.font_size}
plt.rc('font', **font)

# if no split is requested: one band plot and one DOS plot
if args.split is False:
    # set up 2 graph with aspect ratio args.ratio/1
    # plot 1: bands diagram
    # plot 2: DOS
    gs = GridSpec(1, 2, width_ratios=[args.ratio, 1], wspace=0.1)
    fig = plt.figure(figsize=(args.width, args.height))
    ax_bands = plt.subplot(gs[0])
    ax_DOS = plt.subplot(gs[1]) #, sharey=ax1)

# if split is requested: N band plots (N=number of projections, 1-3) and one DOS plot
else:
    print('\tWARNING: split is requested, remember to change plot width!')
    width_ratios = [ args.ratio/N_proj for proj_n in range(N_proj) ]
    width_ratios.append(args.ratio/(2*N_proj))
    gs = GridSpec(1, N_proj+1, width_ratios=width_ratios, wspace=0.1)
    fig = plt.figure(figsize=(args.width, args.height))
    ax_bands = [ plt.subplot(gs[proj_n]) for proj_n in range(N_proj) ]
    ax_DOS = plt.subplot(gs[N_proj])

# -----

# Set both fermi levels equal to the band fermi level
bands.efermi = dosrun.efermi #= 0

# set y limits for the plot
emin = args.emin
emax = args.emax

```

```

# if either is not defined, get the min and/or the max of energy from bands.bands.keys()
if emin is None and emax is None:
    emin=1000
    emax=-1000
    for spin in bands.bands.keys():
        for b in range(bands.nb_bands):
            emin = min(emin, min(bands.bands[spin][b]))
            emax = max(emax, max(bands.bands[spin][b]))
    emin = emin - bands.efermi
    emax = emax - bands.efermi
elif emin is None:
    emin=1000
    for spin in bands.bands.keys():
        for b in range(bands.nb_bands):
            emin = min(emin, min(bands.bands[spin][b]))
    emin = emin - bands.efermi
elif emax is None:
    emax=-1000
    for spin in bands.bands.keys():
        for b in range(bands.nb_bands):
            emax = max(emax, max(bands.bands[spin][b]))
    emax = emax - bands.efermi

# set first and last bands to plot, so that it calculates the projections only on those
min_band_to_plot = 1000
max_band_to_plot = -1

for spin in bands.bands.keys():
    min_band_to_plot = min(min_band_to_plot, np.min(np.where(bands.bands[spin] >= emin + bands.efermi)[0]))
    max_band_to_plot = max(max_band_to_plot, np.max(np.where(bands.bands[spin] <= emax + bands.efermi)[0]))

print(f'\tPlotting bands from {emin} to {emax} eV, band numbers from {min_band_to_plot+1} to
↪ {max_band_to_plot+1}')

reciprocal = bands.lattice_rec.matrix/(2*math.pi)

# Empty lists used for caculating the distances between K-Points
# unchanged from Kevin Waters's script
KPOINTS = [0.0]
DIST = 0.0
# Create list with distances between Kpoints (Individual), corrects the spacing
for k in range(len(bands.kpoints)-1):
    Dist = np.subtract(bands.kpoints[k+1].frac_coords, bands.kpoints[k].frac_coords)
    DIST += np.linalg.norm(np.dot(reciprocal, Dist))
    KPOINTS.append(DIST)

TICKS = [0.0]
for i in range(step, len(KPOINTS)+step, step):
    TICKS.append(KPOINTS[i-1])
# set y-axis limit
if args.split is False:
    ax_bands.set_ylabel(r"$E - E_f$ (eV)", labelpad=1) #labelpad might work bad
    ax_bands.set_ylim(emin, emax)
    ax_bands.grid(lw=args.glw, alpha=0.5)
    ax_bands.set_axisbelow(True)
    ax_bands.hlines(y=0, xmin=0, xmax=len(bands.kpoints), color="k", lw=args.flw)
    for i in range(step, len(KPOINTS)+step, step):
        ax_bands.vlines(KPOINTS[i-1], emin, emax, "k", lw=args.vlw, alpha=args.vla)
    ax_bands.set_xticks(TICKS)
    ax_bands.set_xticklabels(labels)
    ax_bands.tick_params(axis='x', which='both', length=0, pad=5)
    ax_bands.set_xlim(0, KPOINTS[-1])
else:
    ax_bands[0].set_ylabel(r"$E - E_f$ (eV)", labelpad=1) #labelpad might work bad
    for axis in ax_bands:
        axis.set_ylim(emin, emax)
        axis.grid(lw=args.glw, alpha=0.5)
        axis.set_axisbelow(True)

```

```

axis.hlines(y=0, xmin=0, xmax=len(bands.kpoints), color="k", lw=args.flw)
for i in range(step, len(KPOINTS)+step, step):
    axis.vlines(KPOINTS[i-1], emin, emax, "k", lw=args.vlw, alpha=args.vla)
axis.set_xticks(TICKS)
axis.set_xticklabels(labels)
axis.tick_params(axis='x', which='both', length=0, pad=5)
axis.set_xlim(0, KPOINTS[-1])
if axis != ax_bands[0]:
    axis.set_yticks([])
    axis.set_yticklabels([])

ax_DOS.set_ylim(emin, emax)

if no_proj is False:
    CalculateProjectionsAndPlot()
    DOSlabel='total'
    ax_DOS.legend(fancybox=False, shadow=False, prop={'size':
        ↪ args.font_size-1}, labelspace=0.15, borderpad=0.20, handlelength=1.2, framealpha=0.6)
else:
    print('\tNo projection requested. Plotting normal bands.')
    for b in range(min_band_to_plot, max_band_to_plot+1):
        ax_bands.plot(KPOINTS, [e - bands.fermi for e in bands.bands[Spin.up][b]], lw=args.blw, color='k')
    DOSlabel=None

ax_DOS.fill_betweenx(dosrun.tdos.energies - dosrun.fermi,
    0, dosrun.tdos.densities[Spin.up],
    color = (0.7, 0.7, 0.7),
    facecolor = (0.7, 0.7, 0.7))
ax_DOS.plot(dosrun.tdos.densities[Spin.up],
    dosrun.tdos.energies - dosrun.fermi,
    color = (0.6, 0.6, 0.6),
    label = DOSlabel, lw=args.dlw)

# scaling factor for the x axis limit, if the peaks are too high
maxdos = max(dosrun.tdos.densities[Spin.up])/scale

ax_DOS.set_yticklabels([])
ax_DOS.grid(lw=args.glw, alpha=args.gla)
ax_DOS.tick_params('y', length=0, width=1, which='major')
ax_DOS.set_xticks([])
ax_DOS.set_xlim(0, maxdos)
ax_DOS.hlines(y=0, xmin=0, xmax=maxdos, color="k", lw=args.flw)
ax_DOS.set_xlabel("DOS", labelpad=5)

# Plotting
# -----
plt.savefig(f"{args.output_file}.{args.format}", format=args.format, bbox_inches='tight', dpi=400)

print('\tFile saved.')
print('-----')
```

► 代码清单 A.74.2 (python, 621 行)

```

import numpy as np
import math
import pymatgen
import sys
import argparse
import logging
```

```

import time

from pymatgen.io.vasp.outputs import Procar, Vasprun
from pymatgen.io.vasp.inputs import Kpoints
from pymatgen.core import Structure
from pymatgen.electronic_structure.core import Spin, Orbital

import matplotlib.pyplot as plt
from matplotlib.collections import LineCollection
from matplotlib.gridspec import GridSpec

# 添加 tqdm 导入
try:
    from tqdm import tqdm
except ImportError:
    # 如果 tqdm 不可用, 创建空实现
    def tqdm(iterable, **kwargs):
        return iterable

class SaneFormatter(argparse.RawTextHelpFormatter,
                    argparse.ArgumentDefaultsHelpFormatter):
    pass

parser = argparse.ArgumentParser(prog='plot_fatbands.py',description=("Plot projected band structure (fatbands)
↪ from a VASP calculation.\n" +
    'Author: Marco Cappelletti. Heavily inspired by Kevin Waters
    ↪ (kwaters4.github.io) and sumo-bandplot.\n' +
    'By default it assumes that:\n' +
    '\tthe current directory contains KPOINTS, vasprun.xml from the band
    ↪ calculations\n' +
    '\tthe directory ../dos contains vasprun.xml from the dos calculation\n' +
    '\tthe parent directory (..) contains the POSCAR file\n' +
    '\tthe parent parent directory (../..) contains the POTCAR file\n')
    ,formatter_class=SaneFormatter)

parser.add_argument('-b','--vasprun-file-bands', type=str, help='Path of the vasprun.xml file of the band
↪ calculation', default='vasprun.xml')
parser.add_argument('-K','--KPOINTS-file', type=str, help='Path of the KPOINTS file with the band path',
↪ default='KPOINTS')
parser.add_argument('-c','--POSCAR-file', type=str, help='Path of the POSCAR file', default='POSCAR')
parser.add_argument('-O','--PROCAR-file', type=str, help='Path of the PROCAR file from the band calculation',
↪ default='PROCAR')
parser.add_argument('-w','--POTCAR-file', type=str, help='Path of the POTCAR file', default='POTCAR')
parser.add_argument('-d','--vasprun-file-dos', type=str, help='Path of the vasprun.xml file of the dos
↪ calculation', default='../dos/vasprun.xml')
parser.add_argument('-p','--project', help='Band projection to rgb. Either 2 (red and green) or 3 arguments (red,
↪ green, blue). Nomenclature:\n'
    '\t- E: all orbitals of element E (H, C, N, O, ...)\n'
    '\t- E.o: o-orbital of element E (s, px, py, pz, dxy, ...)\n'
    '\t- E.s.pz: s+pz orbitals of element E\n'
    '\t- X.s: s orbitals of all elements\n'
    '\t- O.s.pz+N.pz: sum of O(s,pz) and N(pz)\n', nargs='+',
    ↪ default=['X.px', 'X.py', 'X.s.pz'])
parser.add_argument('-n','--normalization', type=str, help=f'Normalization of the projection.\n'
    '\t-\tall: with respect to all contributions.\n'
    '\t-\tselection: with respect to selection only\n',
    ↪ choices=['all','selection'], default='selection')
parser.add_argument('-l','--max-l',type=int,default=2,choices=[1,2,3],help='Maximum value of l (angular momentum)
↪ for the projection. Increases computational costs, so increase it only if necessary.')
parser.add_argument('-e','--emin', type=float, help='Minimum of energy in the plot. If none, it chooses the lower
↪ limit', default=None)
parser.add_argument('-E','--emax', type=float, help='Maximum of energy in the plot. If none, it chooses the upper
↪ limit', default=None)
parser.add_argument('-s','--scale', type=float, help='DOS scale factor', default=2.0)
parser.add_argument('-H','--height', type=float, help='Height of the plot in inches', default=3.5)
parser.add_argument('-W','--width', type=float, help='Width of the plot in inches', default=3.3)
parser.add_argument('-r','--ratio', type=float, help='Bandplot - dosplot width ratio', default=3.0)
parser.add_argument('-f','--font-size', type=float, help='Fontsize', default=8)

```

```

parser.add_argument('-o', '--output-file', type=str, help='Path and name of the output file, excluding the format',
↪ default='fatbands')
parser.add_argument('--format', type=str, help='Output file format', choices=['pdf', 'png'], default='pdf')
# 添加轨道气泡图参数
parser.add_argument('--element', type=str, help='Element for orbital bubble plot (e.g., Fe)', default=None)
parser.add_argument('--orbital-type', type=str, choices=['d', 'p'], help='Type of orbital to plot (d or p)',
↪ default='d')
parser.add_argument('--bubble-min', type=float, help='Minimum bubble size', default=0.05)
parser.add_argument('--bubble-max', type=float, help='Maximum bubble size', default=0.3)
parser.add_argument('--bubble-scale', type=float, help='Bubble size scaling factor', default=100)
parser.add_argument('--bubble-density', type=int, help='Bubble density (1: every point, 2: every second point,
↪ etc.)', default=5)
parser.add_argument('--bubble-threshold', type=float, help='Minimum weight to plot bubble', default=0.05)

args = parser.parse_args()
scale=args.scale

logging.basicConfig(
    filename="plot_fatbands.log",
    level=logging.INFO,
    filemode="w",
    format="%(message)s",
)

logging.info(" ".join(sys.argv[:]))

# plot colored line. Function written by Kevin Waters
def rgbline(ax, k, e, red, green, blue, KPOINTS, alpha=1.):
    #creation of segments based on
    #http://nbviewer.ipython.org/urls/raw.github.com/dpsanders/matplotlib-examples/master/colorline.ipynb
    pts = np.array([KPOINTS, e]).T.reshape(-1, 1, 2)
    seg = np.concatenate([pts[:-1], pts[1:]], axis=1)
    nseg = len(KPOINTS) - 1
    a = np.ones(nseg, np.float64)*alpha
    lc = LineCollection(seg, colors=list(zip(red,green,blue,a)), linewidth = 2)
    ax.add_collection(lc)

if __name__ == "__main__":
    # Load Structure
    structure = Structure.from_file(args.POSCAR_file)
    atom_labels = structure.labels
    # Load Band Structure Calculations
    bands =
    ↪ Vasprun(args.vasprun_file_bands,parse_potcar_file=args.POTCAR_file).get_band_structure(args.KPOINTS_file,
    ↪ line_mode = True)
    # Read KPOINTS file with path
    kpts = Kpoints.from_file(args.KPOINTS_file)

    # projected bands - get both spin up and spin down data
    procar = Procar(args.PROCAR_file)
    data_up = procar.data[Spin.up]
    data_down = procar.data[Spin.down] if Spin.down in procar.data else None

    # density of states
    dosrun = Vasprun(args.vasprun_file_dos,parse_potcar_file=args.POTCAR_file)

    # k-point labels
    n_labels = len(kpts.labels)
    labels = []
    labels.append(kpts.labels[0])
    for label_idx in range(1,n_labels,2):
        labels.append(kpts.labels[label_idx])

    # Number of points between kpoints, found in the KPOINTS file
    step = kpts.num_kpts

```

```

# general options for plot
font = {'family': 'serif', 'size': args.font_size}
plt.rc('font', **font)

# set up 2 graph with aspect ratio args.ratio/1
# plot 1: bands diagram
# plot 2: DOS
gs = GridSpec(1, 2, width_ratios=[args.ratio,1], wspace=0.1)
fig = plt.figure(figsize=(args.width, args.height))
ax1 = plt.subplot(gs[0])
ax2 = plt.subplot(gs[1]) #, sharey=ax1)

# Set both fermi levels equal to the band fermi level
bands.efermi = dosrun.efermi #= 0

# set y limits for the plot
emin = args.emin
emax = args.emax

# if either is not defined
if emin is None and emax is None:
    emin=100
    emax=-100
    for spin in bands.bands.keys():
        for b in range(bands.nb_bands):
            emin = min(emin, min(bands.bands[spin][b]))
            emax = max(emax, max(bands.bands[spin][b]))
elif emin is None:
    emin=100
    for spin in bands.bands.keys():
        for b in range(bands.nb_bands):
            emin = min(emin, min(bands.bands[spin][b]))
elif emax is None:
    emax=-100
    for spin in bands.bands.keys():
        for b in range(bands.nb_bands):
            emax = max(emax, max(bands.bands[spin][b]))

ax1.set_ylim(emin, emax)
ax2.set_ylim(emin, emax)

# 检测是否需要绘制投影能带
draw_projected_bands = True
if args.element:
    print(f"Orbital bubble plot enabled for {args.element} {args.orbital_type}-orbitals")
    draw_projected_bands = False # 当指定元素和轨道类型时, 不绘制投影能带

# accepts only 2 or 3 entries for the projection
if len(args.project) != 2 and len(args.project) != 3 and draw_projected_bands:
    raise ValueError('Either 2 or 3 components for the projection')

# 定义轨道类型
orbital_values = {
    's': 0,
    'py': 1, 'pz': 2, 'px': 3,
    'dxy': 4, 'dyz': 5, 'dz2': 6, 'dxz': 7, 'dx2_y2': 8,
    'f_3': 9, 'f_2': 10, 'f_1': 11, 'f0': 12, 'f1': 13, 'f2': 14, 'f3': 15
}

max_l_index = (args.max_l + 1)**2 # 1 -> 4, 2 -> 9, 3 -> 16. So to have range(0,max_l_index) =
↳ [0,1,...,max_l_index-1]

# 定义 p 轨道
p_orbitals = ['px', 'py', 'pz']
p_orbital_indices = [orbital_values[o] for o in p_orbitals]

# 定义 d 轨道
d_orbitals = ['dxy', 'dyz', 'dz2', 'dxz', 'dx2_y2']
d_orbital_indices = [orbital_values[o] for o in d_orbitals]

```

```

# 定义轨道的形状和颜色 (空心)
orbital_markers = {
    'dxy': ('s', 'black'),      # 黑色正方形 (空心)
    'dxz': ('o', 'red'),       # 红色圆形 (空心)
    'dyz': ('^', 'green'),     # 绿色正三角形 (空心)
    'dz2': ('v', 'blue'),      # 蓝色倒三角形 (空心)
    'dx2_y2': ('D', 'cyan'),   # 天蓝色菱形 (空心)
    'px': ('o', 'red'),        # 红色圆形 (空心)
    'py': ('s', 'green'),      # 绿色正方形 (空心)
    'pz': ('^', 'blue')        # 蓝色正三角形 (空心)
}

# 根据轨道类型选择要绘制的轨道
if args.orbital_type == 'd':
    orbitals_to_plot = d_orbitals
    orbital_indices = d_orbital_indices
elif args.orbital_type == 'p':
    orbitals_to_plot = p_orbitals
    orbital_indices = p_orbital_indices
else:
    orbitals_to_plot = []
    orbital_indices = []

# 只有当需要绘制投影能带时才计算投影贡献
if draw_projected_bands:
    # calculates contributions for bands and DOS projections
    el_orbs = []
    el_orbs_labels = []
    for component in args.project:      # either 2 or 3 components, e.g. 'N.s.pz', 'N.s+0.s.pz', 'N.s',
        ↪ 'N+0', 'X', 'X.s', 'X.s+N.px.py.pz', ...
        element_components = component.split("+")    # split elements, e.g. 'N.s+0.s.pz' becomes ['N.s',
        ↪ '0.s.pz']
        color_component = []
        label_text = []
        for element_component in element_components:
            splits = element_component.split(".")    # e.g. 'N.s.pz' converted to ['N', 's', 'pz']
            element = splits[0]                      # first element is the atom symbol
            if len(splits) == 1:                      # if no orbital is specified (e.g. 'N'), then plot all
                ↪ orbitals (= 'all')
                orbitals = 'all'
            else:
                orbitals = splits[1:]                 # e.g. ['s', 'pz']
                color_component.append([element, orbitals])

        # write text for legend
        if element == 'X':                          # 'X' represents all atoms
            if len(splits) == 1:
                label_text.append('s+p_x+p_y+p_z')    # this makes no sense, it's the projection of all
                ↪ atoms and orbitals, put who am I to judge
            else:
                label_text.append("+".join(orbitals).replace('p', 'p_'))    # e.g. 'X.s.pz' becomes 's+p_z'
        else:
            if len(splits) == 1:                      # all orbitals, only the element: 'N'
                label_text.append(element)
            else:
                label_text.append(f"{element}({','.join(orbitals).replace('p', 'p_')})")    # e.g. '0.s.pz'
                ↪ becomes '0(s+p_z)'
        el_orbs.append(color_component)
        el_orbs_labels.append("+".join(label_text))    # e.g. '0.s.pz+H.s' becomes '0(s+p_z)+H(s)'

    color_values = { 0: 'red', 1: 'green', 2: 'blue' }
    print('projections:')
    for color_idx, color_contrib in enumerate(el_orbs_labels):
        print(f'color {color_values[color_idx]}: {color_contrib}')

# contributions to the band per each band, k-point, and color: contrib_bands[spin][band][k-point][color]

```

```

contrib_bands = {
    Spin.up: np.zeros((bands.nb_bands, len(bands.kpoints), 3))
}
if data_down is not None:
    contrib_bands[Spin.down] = np.zeros((bands.nb_bands, len(bands.kpoints), 3))

# 检测自旋极化 DOS 计算
has_spin_down_dos = (Spin.down in dosrun.pdos[0][Orbital.s]) # 检测是否存在自旋向下分量

# 初始化 contrib_dos, 分别存储自旋向上和自旋向下 (如果存在)
contrib_dos = {
    Spin.up: np.zeros((len(el_orbs), len(dosrun.tdos.energies))),
}
if has_spin_down_dos:
    contrib_dos[Spin.down] = np.zeros((len(el_orbs), len(dosrun.tdos.energies)))
else:
    contrib_dos[Spin.down] = None

# 添加进度条 - 计算投影贡献
print("\nCalculating projections:")
for spin in [Spin.up, Spin.down]:
    if spin not in contrib_bands:
        continue

    # 创建进度条
    spin_name = "up" if spin == Spin.up else "down"
    band_range = tqdm(range(bands.nb_bands),
                        desc=f"Processing {spin_name} bands",
                        total=bands.nb_bands,
                        leave=True)

    # sum over all bands
    for b in band_range:
        # sum over all k-points
        for k in range(len(bands.kpoints)):
            for color_idx, color_contrib in enumerate(el_orbs): # color_idx: up to either 2 or 3.
                ↪ color_contrib: list of [elements, orbitals] for each color
                for element_contrib in color_contrib:
                    element = element_contrib[0] # e.g. 'X', 'N'
                    orbitals = element_contrib[1] # e.g. 'all', 's', ['s', 'pz']
                    if element == 'X': # if all atoms, get all indexes
                        element_indexes = range(len(atom_labels))
                    else: # else, get the indexes with label = atom symbol
                        ↪ (e.g. 'C')
                        element_indexes = [i for i, x in enumerate(atom_labels) if x == element]

                    if orbitals == 'all':
                        orbital_indexes = range(0, max_l_index) # sum all orbitals if 'all'
                    else:
                        orbital_indexes = [orbital_values[o] for o in orbitals]

                    for i in element_indexes:
                        for j in orbital_indexes:
                            if spin == Spin.up:
                                contrib_bands[spin][b, k, color_idx] += data_up[k][b][i][j]**2
                            else: # spin down
                                contrib_bands[spin][b, k, color_idx] += data_down[k][b][i][j]**2

                # Only need to add to DOS once (using spin up)
                if spin == Spin.up and k == 0 and b == 0:
                    contrib_dos[Spin.up][color_idx] +=
                    ↪ np.array(dosrun.pdos[i][Orbital(j)][Spin.up])
                if has_spin_down_dos:
                    contrib_dos[Spin.down][color_idx] +=
                    ↪ np.array(dosrun.pdos[i][Orbital(j)][Spin.down])

# normalization
if args.normalization == 'selection':

```

```

        if np.sum(contrib_bands[spin][b,k,:]) != 0:
            contrib_bands[spin][b,k,:] =
                ↪ contrib_bands[spin][b,k,:]/np.sum(contrib_bands[spin][b,k,:])
    elif args.normalization == 'all':
        tot = 0.0
        for i in range(len(atom_labels)):
            for j in range(0,max_l_index):
                if spin == Spin.up:
                    tot += data_up[k][b][i][j]**2
                else:
                    tot += data_down[k][b][i][j]**2
        if tot != 0:
            contrib_bands[spin][b,k,:] = contrib_bands[spin][b,k,:]/tot

# 添加进度条 - 计算轨道权重
weights = {}
if args.element:
    print(f"\nCalculating {args.orbital_type}-orbital weights for {args.element}:")
    weights = {
        Spin.up: {orb: np.zeros((bands.nb_bands, len(bands.kpoints))) for orb in orbitals_to_plot},
        Spin.down: {orb: np.zeros((bands.nb_bands, len(bands.kpoints))) for orb in orbitals_to_plot}
    }

# 获取特定元素的原子索引
element_indexes = [i for i, x in enumerate(atom_labels) if x == args.element]
if not element_indexes:
    raise ValueError(f"No atoms of element {args.element} found in the structure")

for spin in [Spin.up, Spin.down]:
    if spin == Spin.down and data_down is None:
        continue

    # 创建进度条
    spin_name = "up" if spin == Spin.up else "down"
    band_range = tqdm(range(bands.nb_bands),
                        desc=f"Processing {spin_name} weights",
                        total=bands.nb_bands,
                        leave=True)

    for b in band_range:
        for k in range(len(bands.kpoints)):
            # 计算该元素所有指定轨道的总贡献
            total = 0.0
            for i in element_indexes:
                for j in orbital_indices:
                    if spin == Spin.up:
                        total += data_up[k][b][i][j]**2
                    else:
                        total += data_down[k][b][i][j]**2

            # 计算每个轨道的权重
            for orb in orbitals_to_plot:
                j = orbital_values[orb]
                orb_value = 0.0
                for i in element_indexes:
                    if spin == Spin.up:
                        orb_value += data_up[k][b][i][j]**2
                    else:
                        orb_value += data_down[k][b][i][j]**2

                if total > 1e-6: # 避免除以零
                    weight = orb_value / total
                else:
                    weight = 0.0

                weights[spin][orb][b, k] = weight

reciprocal = bands.lattice_rec.matrix/(2*math.pi)

```

```

# unchanged from Kevin Waters
# Empty lists used for caculating the distances between K-Points
KPOINTS = [0.0]
DIST = 0.0
# Create list with distances between Kpoints (Individual), corrects the spacing
for k in range(len(bands.kpoints)-1):
    Dist = np.subtract(bands.kpoints[k+1].frac_coords, bands.kpoints[k].frac_coords)
    DIST += np.linalg.norm(np.dot(reciprocal, Dist))
    KPOINTS.append(DIST)

# 只有当需要绘制投影能带时才绘制彩色能带
if draw_projected_bands:
    print("Plotting projected bands...")
    for spin in [Spin.up, Spin.down]:
        if spin not in contrib_bands:
            continue

        for b in range(bands.nb_bands):
            rgbline(ax1,
                    range(len(bands.kpoints)),
                    [e - bands.efermi for e in bands.bands[spin][b]],
                    contrib_bands[spin][b, :, 0],
                    contrib_bands[spin][b, :, 1],
                    contrib_bands[spin][b, :, 2] if len(args.project) == 3 else np.zeros(len(bands.kpoints)),
                    ↪ # 如果只有两个颜色, 第三个分量设为零
                    KPOINTS=KPOINTS)
else:
    # 绘制简单的能带图 (灰色)
    print("Plotting simple bands...")
    for spin in [Spin.up, Spin.down]:
        if spin not in bands.bands:
            continue

        for b in range(bands.nb_bands):
            ax1.plot(KPOINTS, [e - bands.efermi for e in bands.bands[spin][b]],
                    color='gray', alpha=0.5, linewidth=0.8)

# 优化后的轨道气泡图绘制 (空心气泡)
if args.element:
    print(f"Plotting {args.orbital_type}-orbital bubbles...")
    # 计算气泡大小范围
    min_size = args.bubble_min * args.bubble_scale
    max_size = args.bubble_max * args.bubble_scale

    # 为每个轨道绘制气泡 (空心)
    for spin in [Spin.up, Spin.down]:
        if spin not in weights:
            continue

        for orb in orbitals_to_plot:
            marker, color = orbital_markers[orb]
            # 收集所有点一次性绘制
            x_vals = []
            y_vals = []
            sizes = []

            # 使用步长减少点数
            k_step = args.bubble_density
            b_step = args.bubble_density

            # 创建进度条
            total_points = (bands.nb_bands // b_step) * (len(bands.kpoints) // k_step)
            if total_points > 0:
                progress = tqdm(total=total_points,
                               desc=f"{orb} bubbles",
                               unit="point")

            else:
                progress = None

```

```

    for b in range(0, bands.nb_bands, b_step):
        for k in range(0, len(bands.kpoints), k_step):
            weight = weights[spin][orb][b, k]

            # 只绘制权重大于阈值的点
            if weight > args.bubble_threshold:
                # 计算气泡大小 (与权重成正比)
                size = min_size + weight * (max_size - min_size)

                # 获取能量值
                energy = bands.bands[spin][b][k] - bands.efermi

                # 添加点
                x_vals.append(KPOINTS[k])
                y_vals.append(energy)
                sizes.append(size)

            if progress:
                progress.update(1)

# 一次性绘制该轨道的所有点 (空心)
if x_vals: # 确保列表不为空
    ax1.scatter(
        x_vals,
        y_vals,
        s=sizes,
        marker=marker,
        facecolors='none', # 空心
        edgecolors=color, # 边框颜色
        linewidths=0.4, # 边框线宽
        alpha=0.4, # 稍微降低透明度
        zorder=10 # 确保气泡在能带上方
    )

    if progress:
        progress.close()

# 创建图例 (只包含轨道气泡)
legend_handles = []
for orb in orbitals_to_plot:
    marker, color = orbital_markers[orb]
    legend_handles.append(
        plt.Line2D([0], [0],
            marker=marker,
            color=color,
            markerfacecolor='none', # 空心
            markeredgecolor=color,
            markersize=8,
            linestyle='None',
            label=f"{orb}")
    )

# 在右侧的 ax2 上绘制图例 (DOS 位置)
ax2.clear() # 清除原有内容
ax2.axis('off') # 关闭坐标轴

# 创建图例并放置在右侧中央位置
ax2.legend(handles=legend_handles,
    loc='center', # 中心位置
    bbox_to_anchor=(0.57, 0.81), # 精确中心位置
    fancybox=False,
    shadow=False,
    prop={'size': args.font_size},
    framealpha=0.8,
    title=f"{args.element} {args.orbital_type}-orbitals",
    title_fontsize=args.font_size)

# 调整标题位置
ax1.set_title(f"{args.element} {args.orbital_type}-orbitals",

```

```

        fontsize=args.font_size, pad=10,y=1.05)
# 调整右侧子图位置, 使图例更靠近能带图
plt.subplots_adjust(right=0.85) # 增加右侧空白区域

# style
ax1.set_ylabel(r"$E - E_f$ (eV)",labelpad=-2) #labelpad might work bad
ax1.grid()

# fermi level line at 0
ax1.hlines(y=0, xmin=0, xmax=len(bands.kpoints), color="k", lw=2)

TICKS = [0.0]
for i in range(step,len(KPOINTS)+step,step):
    ax1.vlines(KPOINTS[i-1], emin, emax, "k")
    TICKS.append(KPOINTS[i-1])
ax1.set_xticks(TICKS)
ax1.set_xticklabels(labels)
ax1.tick_params(axis='x', which='both', length=0, pad=5)
ax1.set_xlim(0, KPOINTS[-1])

# 只有当需要绘制投影能带时才绘制 DOS
if draw_projected_bands:
    print("Plotting DOS...")
    # 绘制总 DOS
    if has_spin_down_dos:
        # 自旋向上 (正方向)
        ax2.plot(dosrun.tdos.densities[Spin.up], dosrun.tdos.energies - dosrun.efermi,
                color=(0.6, 0.6, 0.6), label="total up", lw=1)
        # 自旋向下 (负方向)
        ax2.plot(-dosrun.tdos.densities[Spin.down], dosrun.tdos.energies - dosrun.efermi,
                color=(0.6, 0.6, 0.6), label="total down", lw=1)
        # 填充区域
        ax2.fill_betweenx(dosrun.tdos.energies - dosrun.efermi,
                        0, dosrun.tdos.densities[Spin.up],
                        color=(0.7, 0.7, 0.7), facecolor=(0.7, 0.7, 0.7))
        ax2.fill_betweenx(dosrun.tdos.energies - dosrun.efermi,
                        0, -dosrun.tdos.densities[Spin.down],
                        color=(0.7, 0.7, 0.7), facecolor=(0.7, 0.7, 0.7))
        # 计算最大 DOS 值 (考虑两个自旋方向)
        maxdos = max(np.max(dosrun.tdos.densities[Spin.up]),
                    np.max(dosrun.tdos.densities[Spin.down])) / scale
    else:
        # 非自旋极化计算
        ax2.plot(dosrun.tdos.densities[Spin.up], dosrun.tdos.energies - dosrun.efermi,
                color=(0.6, 0.6, 0.6), label="total", lw=1)
        ax2.fill_betweenx(dosrun.tdos.energies - dosrun.efermi,
                        0, dosrun.tdos.densities[Spin.up],
                        color=(0.7, 0.7, 0.7), facecolor=(0.7, 0.7, 0.7))
        maxdos = np.max(dosrun.tdos.densities[Spin.up]) / scale

# 绘制投影 DOS (自旋向上)
ax2.plot(contrib_dos[Spin.up][0], dosrun.tdos.energies - dosrun.efermi,
        c=(1.0, 0.0, 0.0), label=f'${el_orbs_labels[0]}$', linewidth=1)
ax2.plot(contrib_dos[Spin.up][1], dosrun.tdos.energies - dosrun.efermi,
        c=(0.0, 1.0, 0.0), label=f'${el_orbs_labels[1]}$', linewidth=1)

# 如果有自旋向下分量, 绘制负方向的投影
if has_spin_down_dos:
    ax2.plot(-contrib_dos[Spin.down][0], dosrun.tdos.energies - dosrun.efermi,
            c=(1.0, 0.0, 0.0), linewidth=1)
    ax2.plot(-contrib_dos[Spin.down][1], dosrun.tdos.energies - dosrun.efermi,
            c=(0.0, 1.0, 0.0), linewidth=1)

# 如果有第三个颜色分量
if len(el_orbs) == 3:
    ax2.plot(contrib_dos[Spin.up][2], dosrun.tdos.energies - dosrun.efermi,
            c=(0.0, 0.0, 1.0), label=f'${el_orbs_labels[2]}$', linewidth=1)
    if has_spin_down_dos:

```

```

        ax2.plot(-contrib_dos[Spin.down][2], dosrun.tdos.energies - dosrun.efermi,
                  c=(0.0, 0.0, 1.0), linewidth=1)

# 调整 x 轴范围
if has_spin_down_dos:
    ax2.set_xlim(-maxdos, maxdos)
else:
    ax2.set_xlim(0, maxdos)

# 添加 y=0 的水平线
ax2.hlines(y=0, xmin=ax2.get_xlim()[0], xmax=ax2.get_xlim()[1], color="k", lw=2)

# 添加 x=0 的垂直线 (仅当有自旋向下分量时)
if has_spin_down_dos:
    ax2.vlines(x=0, ymin=emin, ymax=emax, color="k", lw=1, linestyle='--', alpha=0.5)

ax2.set_yticklabels([])
ax2.grid()
ax2.set_xticks([])
ax2.set_xlabel("DOS")
ax2.legend(fancybox=False, shadow=False, prop={'size': args.font_size-1},
           labelspace=0.15, borderpad=0.20, handlelength=1.2,
           framealpha=0.6, loc='upper right')

# Plotting
# -----
print("Saving plot...")
plt.savefig(f"{args.output_file}.{args.format}", format=args.format, bbox_inches='tight')
print("Done!")

```

(原创脚本)python 解线性方程组

► 代码清单 A.75.1 (python, 25 行)

```

import numpy as np

# 系数矩阵 A
A = np.array([
    [18, 18, 18, 1],
    [-6, -6, 18, 1],
    [-6, 2, 2, 1],
    [-2, -6, 2, 1]
])

# 从用户输入右端向量 b
print(" 请输入右端常数向量 b 的 4 个数 (以空格分隔, 例如: 1 2 3 4) :")
b = np.array(list(map(float, input().split())))

# 检查维度是否正确
if len(b) != 4:
    raise ValueError(" 必须输入 4 个数! ")

# 求解线性方程组 Ax = b
x = np.linalg.solve(A, b)

# 输出结果
print("\n 方程组的解为: ")
for i, val in enumerate(x, start=1):
    print(f"x{i} = {val:.6f}")

```

(改编脚本) 快速绘制轨道投影能带

► 代码清单 A.76.3 (shell, 5 行)

```
0.0 0.0 0.0 g
0.5 0.0 0.0 m
0.3333333 0.3333333 0.0 k
0.0 0.0 0.0 g
-0.3333333 -0.3333333 0.0 k
```

► 代码清单 A.76.5 (python, 27 行)

```
# Python 3 version
with open("k-points", 'r+') as file:
    lines = file.readlines()

line_count = len(lines)
mesh = 20
print("k-points along high symmetry lines")
print((line_count - 1) * mesh + 1)
print("Reciprocal")

# Convert each line to a list of floats
for i in range(line_count):
    lines[i] = lines[i].strip().split()
    lines[i][0] = float(lines[i][0])
    lines[i][1] = float(lines[i][1])
    lines[i][2] = float(lines[i][2])

# Generate the k-point mesh
for i in range(line_count - 1):
    for j in range(mesh):
        # Calculate the k-point coordinates for the current position along the path
        for k in range(3):
            print(lines[i][k] + (lines[i + 1][k] - lines[i][k]) * j / (mesh + 0.0), end=' ')
            print(1.0)

# Print the last k-point
print(lines[line_count - 1][0], lines[line_count - 1][1], lines[line_count - 1][2], "1.0")
```

► 代码清单 A.76.6 (shell, 12 行)

```
totalk=`awk '{if(NR==2)print $4}' PROCAR`
totalb=`awk '{if(NR==2)print $8}' PROCAR`
Ef=`awk '{if(NR==6)print $4}' DOSCAR`
l=`awk '/^band/{print $6}' PROCAR |wc -l`
# if ispin = 2 without of soc, we should use the following line, else comment it with #
l=$((l/2))
awk '/^band/{print $5 - '$Ef'}' PROCAR |head -$l > bnd-up.dat
awk '/^band/{print $5 - '$Ef'}' PROCAR |tail -$l > bnd-dn.dat
echo $totalk $totalb >note-bnd
grep -A3 reciprocal OUTCAR |tail -3 |awk '{print $4,$5,$6}' >>note-bnd
cat k-points >>note-bnd
grep mesh kp.py |head -1 |cut -d= -f 2 >>note-bnd
```

► 代码清单 A.76.7 (shell, 10 行)

```
#ion      s      py      pz      px      dxy      dyz      dz2      dxz      x2-y2      tot
#1        2      3      4      5      6      7      8      9      10
grep ^tot PROCAR|awk '{print $3}' >py
grep ^tot PROCAR|awk '{print $4}' >pz
grep ^tot PROCAR|awk '{print $5}' >px
grep ^tot PROCAR|awk '{print $6}' >xy
grep ^tot PROCAR|awk '{print $7}' >yz
grep ^tot PROCAR|awk '{print $8}' >z2
grep ^tot PROCAR|awk '{print $9}' >xz
grep ^tot PROCAR|awk '{print $10}' >x2
```

► 代码清单 A.76.8 (python, 118 行)

```
import numpy as np
from numpy import *
import matplotlib as mpl
import os
from matplotlib import pyplot as plt
from matplotlib import rc
from matplotlib import cm
import sys
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.colors as colors
rc('text', usetex=False)
plt.rc('font', family='DejaVu Sans', size=20)

max_val = 2
min_val = -3
step = (max_val - min_val)/10
mode = 2
spin = 'True'

fn = 'bnd.pdf'
m = open("note-bnd", 'r+')
ml = m.readlines()
ml = [ml[i].strip().split() for i in range(len(ml))]
knum = int(ml[0][0])
bnum = int(ml[0][1])

base = [ml[1], ml[2], ml[3]]
base = np.array(base)
dkvec = []
for k in range(4, len(ml)-1):
    ml[k] = [float(ml[k][j]) for j in range(3)] # kvec to float
for k in range(5, len(ml)-1):
    dkvec.append((np.array(ml[k]) - np.array(ml[k-1])).tolist()) # distance of kvec

def r(m, n):
    global rfina1
    global r1
    global r2
    global r3
    r1 = 0
    r2 = 0
    r3 = 0
    r1 = [float(m[0]) * float(n[0][i]) for i in range(3)]
    r2 = [float(m[1]) * float(n[1][i]) for i in range(3)]
    r3 = [float(m[2]) * float(n[2][i]) for i in range(3)]
    x = r1[0] + r2[0] + r3[0]
    y = r1[1] + r2[1] + r3[1]
```

```

    z = r1[2] + r2[2] + r3[2]
    rfinal = np.sqrt(x**2 + y**2 + z**2)

sum_k = 0
k = [0 for i in range(len(ml)-6)]
for i in range(len(ml)-6):
    r(dkvec[i], base)
    k[i] = rfinal
    sum_k = k[i] + sum_k
mesh = int(ml[len(ml)-1][0])

if mode == 1:
    x = []
else:
    x = [0]
a0 = 0
for i in range(len(ml)-6):
    for j in range(mesh):
        a0 = a0 + k[i]/(mesh + 0.0)
    x.append(a0)

# 定义高对称点标签 - 这是原始代码中的定义
klabel = ['Γ', 'M', 'K', 'Γ', 'K']

plt.figure(figsize=(6, 10))

##### spin up #####
with open("bnd-up.dat", 'r') as file:
    lines = file.readlines()
    data_up = np.array([float(line.strip()) for line in lines])
    data_up.resize(knum, bnum)
    bands_up = np.transpose(data_up)

plt.ylim(min_val, max_val)
plt.xlim(0, sum_k)

# 绘制 spin up 能带 (蓝色)
for i in range(bnum):
    plt.plot(x, bands_up[i], 'b', linewidth=1.5)

##### spin down #####
if spin == 'True':
    data_dn = np.loadtxt('bnd-dn.dat')
    data_dn.resize(knum, bnum)
    bands_dn = np.transpose(data_dn)

    # 绘制 spin down 能带 (红色)
    for i in range(bnum):
        plt.plot(x, bands_dn[i], 'r', linewidth=1.5)

    plt.plot([0, sum_k], [0, 0], 'r-.')

hline = 0
xlab_val = [0]
for i in range(len(ml)-6):
    hline = hline + k[i]
    xlab_val.append(hline)

hline = 0
for i in range(len(ml)-7):
    hline = hline + k[i]
    plt.plot([hline, hline], [min_val, max_val + step], 'b-.', linewidth=1)

plt.ylabel('Energy (eV)', fontsize=25)
plt.xticks(xlab_val, klabel)
plt.tight_layout()
plt.yticks(np.arange(min_val, max_val + step, step))
ax = plt.gca()
ax.tick_params(labelsize=20)

```

```
plt.savefig(fn, dpi=300, bbox_inches='tight')
plt.show()
```

► 代码清单 A.76.9 (python, 185 行)

```
# -*- coding: utf-8 -*-

import numpy as np
import matplotlib.pyplot as plt
from matplotlib import rc
rc('text',usetex=False)
plt.rc('font', family='DejaVu Sans', size=20)

#####
min_val = -2
max_val = 1
step = (max_val - min_val)/10
fonts = 15
spin = 'both' # 修改为 'both' 表示同时绘制两个自旋通道
#####
orbs=['px','py','pz','z2','x2','xy','yz','xz']
kp = ['Γ','M','K','Γ','K']

# 根据 spin 参数设置文件和输出名
if spin == 'up':
    bndfiles = ['bnd-up.dat']
    orb_prefixes = [''] # 不加后缀
    figname = "Project_upbnd.pdf"
    colors = ['blue'] # spin up 用蓝色

elif spin == 'dn':
    bndfiles = ['bnd-dn.dat']
    orb_prefixes = [''] # 不加后缀
    figname = "Project_dnbnd.pdf"
    colors = ['red'] # spin down 用红色

elif spin == 'both': # 同时绘制两个自旋通道
    bndfiles = ['bnd-up.dat', 'bnd-dn.dat']
    orb_prefixes = ['', ''] # 轨道文件名相同
    figname = "Project_bothbnd.pdf"
    colors = ['blue', 'red'] # up 蓝色, down 红色

elif spin == 'no':
    bndfiles = ['bnd-up.dat']
    orb_prefixes = ['']
    figname = "Project_nospinbnd.pdf"
    colors = ['black']

m=open("note-bnd",'r+')
ml=m.readlines()
ml=[ml[i].strip().split() for i in range(len(ml))]
knum=int(ml[0][0])
bnum=int(ml[0][1])

base=[ml[1],ml[2],ml[3]]
base=np.array(base)
dkvec=[]
for k in range(4,len(ml)-1):
    ml[k]=[float(ml[k][j]) for j in range(3)]
for k in range(5,len(ml)-1):
    dkvec.append((np.array(ml[k])-np.array(ml[k-1])).tolist())
def r(m,n):
    global rfinal
```

```

global r1
global r2
global r3
r1=0
r2=0
r3=0
r1=[float(m[0])*float(n[0][i]) for i in range(3)]
r2=[float(m[1])*float(n[1][i]) for i in range(3)]
r3=[float(m[2])*float(n[2][i]) for i in range(3)]
x=r1[0]+r2[0]+r3[0]
y=r1[1]+r2[1]+r3[1]
z=r1[2]+r2[2]+r3[2]
rfinal=np.sqrt(x**2+y**2+z**2)
sum_k=0
k=[0 for i in range(len(ml)-6)]
for i in range(len(ml)-6):
    r(dkvec[i],base)
    k[i]=rfinal
    sum_k=k[i]+sum_k
mesh=int(ml[len(ml)-1][0])
x=[0]
a0=0
for i in range(len(ml)-6):
    for j in range(mesh):
        a0=a0+k[i]/(mesh+0.0)
    x.append(a0)

plt.figure(figsize=(20,16))

def plt_pro(i, orb_name, col, lab):
    plt.subplot(2,4,i)

    # 设置坐标轴范围
    plt.ylim(min_val, max_val)
    plt.xlim(0, sum_k)

    # 绘制费米面和对称线
    plt.plot([0,sum_k],[0,0], 'r-.', linewidth=1)
    hline=0
    xlab_val = [0]
    for i in range(len(ml)-6):
        hline=hline+k[i]
        xlab_val.append(hline)
    hline=0
    for i in range(len(ml)-7):
        hline=hline+k[i]
        plt.plot([hline,hline],[min_val,max_val+step], 'b-.', linewidth=1)

    # 处理每个自旋通道
    for spin_idx, (bndfile, color) in enumerate(zip(bndfiles, colors)):
        # 读取能带数据
        a = np.loadtxt(bndfile)
        a.resize(knum, bnum)
        b = np.transpose(a)

        # 绘制能带线
        for j in range(bnum):
            if spin == 'both':
                # 两个自旋通道用不同的线型和颜色
                if spin_idx == 0: # spin up
                    plt.plot(x, b[j], '--', color=color, linewidth=1, alpha=0.6)
                else: # spin down
                    plt.plot(x, b[j], '-', color=color, linewidth=1, alpha=0.6)
            else:
                plt.plot(x, b[j], 'k--', linewidth=1)

        # 读取轨道投影数据
        a2 = np.loadtxt(orb_name)
        lens = len(a2) // 2

```

```

if spin == 'both':
    # 对于两个自旋通道, 需要选择正确的部分
    if spin_idx == 0: # spin up
        a2_up = a2[0:lens]
        a2_up.resize(knum, bnum)
        b2_up = np.transpose(a2_up)
        for j in range(bnum):
            plt.scatter(x, b[j], c=color, s=b2_up[j]*300, lw=0, alpha=0.5)
    else: # spin down
        a2_dn = a2[lens:]
        a2_dn.resize(knum, bnum)
        b2_dn = np.transpose(a2_dn)
        for j in range(bnum):
            plt.scatter(x, b[j], c=color, s=b2_dn[j]*300, lw=0, alpha=0.5, marker='^')
else:
    # 单个自旋通道
    if spin == 'up':
        a2 = a2[0:lens]
    elif spin == 'dn':
        a2 = a2[lens:]

    a2.resize(knum, bnum)
    b2 = np.transpose(a2)
    for j in range(bnum):
        plt.scatter(x, b[j], c=col, s=b2[j]*300, lw=0, alpha=0.7)

# 设置图例
if spin == 'both':
    from matplotlib.lines import Line2D
    legend_elements = [Line2D([0], [0], color='blue', linestyle='--', lw=2, label='Spin Up'),
                       Line2D([0], [0], color='red', linestyle='-', lw=2, label='Spin Down'),
                       plt.scatter([], [], c=col, s=100, label=lab)]
    plt.legend(handles=legend_elements, frameon=False, fontsize=15, loc='upper right')
else:
    ax = plt.scatter([], [], c=col, s=100, label=lab)
    plt.legend([ax], [lab], frameon=False, fontsize=20, loc='upper right')

# 设置坐标轴
plt.ylabel('Energy (eV)')
plt.xticks(xlab_val, kp)
plt.tight_layout()
plt.yticks(np.arange(min_val, max_val+step, step))
plt.subplots_adjust(left=0.15, right=0.9, top=0.9, bottom=0.1)

# 绘制所有轨道的投影
plt_pro(1, orbs[0], "gray", r'$p_x$')
plt_pro(2, orbs[1], "gray", r'$p_y$')
plt_pro(3, orbs[2], "gray", r'$p_z$')
plt_pro(4, orbs[3], "cyan", r'$d_{z^2}$')
plt_pro(5, orbs[4], "red", r'$d_{x^2-y^2}$')
plt_pro(6, orbs[5], "red", r'$d_{xy}$')
plt_pro(7, orbs[6], "blue", r'$d_{yz}$')
plt_pro(8, orbs[7], "blue", r'$d_{xz}$')

plt.subplots_adjust(wspace=0.25, hspace=0.1)
plt.savefig(filename, format='pdf', bbox_inches='tight', dpi=300)
plt.show()

```

► 代码清单 A.76.10 (python, 194 行)

```
# -*- coding: utf-8 -*-
```

```

import numpy as np
import matplotlib.pyplot as plt
from matplotlib import rc
rc('text',usetex=False)
plt.rc('font', family='DejaVu Sans', size=20)

#####
min=-2
max=1
step = (max - min)/10
fonts = 15
#####
orbs=['px','py','pz','z2','x2','xy','yz','xz']
bndfile_up = 'bnd-up.dat'
bndfile_dn = 'bnd-dn.dat'
figname = "Project_both_spins.pdf"

# 设置颜色
col='cyan'
kp = ['Γ', 'M', 'K', 'Γ', 'K']

# 读取 k 点信息
m=open("note-bnd",'r+')
ml=m.readlines()
ml=[ml[i].strip().split() for i in range(len(ml))]
knum=int(ml[0][0])
bnum=int(ml[0][1])

base=[ml[1],ml[2],ml[3]]
base=np.array(base)
dkvec=[]
for k in range(4,len(ml)-1):
    ml[k]=[float(ml[k][j]) for j in range(3)]
for k in range(5,len(ml)-1):
    dkvec.append((np.array(ml[k])-np.array(ml[k-1])).tolist())

def r(m,n):
    global rfinal
    global r1
    global r2
    global r3
    r1=0
    r2=0
    r3=0
    r1=[float(m[0])*float(n[0][i]) for i in range(3)]
    r2=[float(m[1])*float(n[1][i]) for i in range(3)]
    r3=[float(m[2])*float(n[2][i]) for i in range(3)]
    x=r1[0]+r2[0]+r3[0]
    y=r1[1]+r2[1]+r3[1]
    z=r1[2]+r2[2]+r3[2]
    rfinal=np.sqrt(x**2+y**2+z**2)

sum=0
k=[0 for i in range(len(ml)-6)]
for i in range(len(ml)-6):
    r(dkvec[i],base)
    k[i]=rfinal
    sum=k[i]+sum

mesh=int(ml[len(ml)-1][0])
x=[0]
a0=0
for i in range(len(ml)-6):
    for j in range(mesh):
        a0=a0+k[i]/(mesh+0.0)
        x.append(a0)

plt.figure(figsize=(20, 20))

```

```

def plt_pro(row_offset, i, orb, col, lab, spin):
    """ 绘制单个轨道的投影能带

    Parameters:
    -----
    row_offset : int
        行偏移量, 0 表示 spin up, 4 表示 spin down
    i : int
        轨道索引 (1-8)
    orb : str
        轨道数据文件名
    col : str
        颜色
    lab : str
        轨道标签
    spin : str
        自旋类型 'up' 或 'dn'
    """
    # 计算子图位置: 2 行 4 列, 加上行偏移
    plt.subplot(8, 4, row_offset * 4 + i)

    # 选择对应的能带文件
    if spin == 'up':
        bndfile = bndfile_up
        spin_color = 'red' # spin up 用红色
        spin_label = '(↑)'
    else:
        bndfile = bndfile_dn
        spin_color = 'blue' # spin down 用蓝色
        spin_label = '(↓)'

    # 读取能带数据
    a = np.loadtxt(bndfile)
    a.resize(knum, bnum)
    b = np.transpose(a)

    # 设置坐标范围
    plt.ylim(min, max)
    plt.xlim(0, sum)

    # 绘制费米能级线
    plt.plot([0, sum], [0, 0], 'r-.', linewidth=1)

    # 绘制高对称点竖线
    hline = 0
    xlab_val = [0]
    for j in range(len(ml)-6):
        hline += k[j]
        xlab_val.append(hline)

    hline = 0
    for j in range(len(ml)-7):
        hline += k[j]
        plt.plot([hline, hline], [min, max+step], 'b-.', linewidth=1)

    # 绘制能带
    for j in range(bnum):
        plt.plot(x, b[j], 'k--', linewidth=1)

    # 读取轨道投影数据
    a2 = np.loadtxt(orb)
    lens = len(a2)//2

    if spin == 'up':
        a2 = a2[0:lens] # 前半部分是 spin up
    else:
        a2 = a2[lens:] # 后半部分是 spin down

    a2.resize(knum, bnum)

```

```

b2 = np.transpose(a2)

# 绘制轨道投影散点
for j in range(bnum):
    # 能带线
    plt.plot(x, b[j], 'k--', linewidth=0.5, alpha=0.3)
    # 轨道投影散点
    ax = plt.scatter(x, b[j], c=col, s=b2[j]*300, lw=0, alpha=0.7)

# 设置标签
if row_offset == 0: # spin up 行
    if i == 1: # 第一列添加 y 轴标签
        plt.ylabel('Energy (eV)\nSpin Up')
        plt.title(f'{lab} {spin_label}', fontsize=16)
    else: # spin down 行
        if i == 1: # 第一列添加 y 轴标签
            plt.ylabel('Energy (eV)\nSpin Down')
            plt.title(f'{lab} {spin_label}', fontsize=16)

# 设置 x 轴刻度
if row_offset == 7: # 最后一行显示 x 轴标签
    plt.xticks(xlab_val, kp)
    plt.xlabel('k-path')
else:
    plt.xticks(xlab_val, [])

# 设置 y 轴刻度
if i == 1: # 第一列显示 y 轴刻度
    plt.yticks(np.arange(min, max+step, step))
else:
    plt.yticks([])

plt.tight_layout()
ax = plt.gca()

# 绘制 spin up 的 8 个轨道
spin_up_orbs = orbs
for i, (orb, col_orb, lab) in enumerate(zip(orbs,
                                            ["gray", "gray", "gray", "cyan", "red", "red", "blue", "blue"],
                                            [r'$p_x$', r'$p_y$', r'$p_z$', r'$d_{z^2}$',
                                              r'$d_{x^2-y^2}$', r'$d_{xy}$', r'$d_{yz}$', r'$d_{xz}$']), 1):
    plt_pro(0, i, orb, col_orb, lab, 'up')

# 绘制 spin down 的 8 个轨道
spin_dn_orbs = orbs
for i, (orb, col_orb, lab) in enumerate(zip(orbs,
                                            ["gray", "gray", "gray", "cyan", "red", "red", "blue", "blue"],
                                            [r'$p_x$', r'$p_y$', r'$p_z$', r'$d_{z^2}$',
                                              r'$d_{x^2-y^2}$', r'$d_{xy}$', r'$d_{yz}$', r'$d_{xz}$']), 1):
    plt_pro(4, i, orb, col_orb, lab, 'dn')

plt.subplots_adjust(left=0.05, right=0.95, top=0.95, bottom=0.05, hspace=0.1, wspace=0.1)
plt.savefig(figname, format='pdf', bbox_inches='tight', dpi=300)
plt.show()

```

▶ 代码清单 A.76.11 (python, 127 行)

```

# -*- coding: utf-8 -*-

import numpy as np
import matplotlib.pyplot as plt
from matplotlib import rc
rc('text',usetex=False)

```

```

plt.rc('font', family='DejaVu Sans', size=20)

#####
min=-2
max=1
step = (max - min)/10
fonts = 15
spin = 'dn' #spin = 'up/dn/no' default value: 'up'
#####
orbs=['px','py','pz','z2','x2','xy','yz','xz']
bndfile = 'bnd-up.dat'
figname = "Project_upbnd.pdf"

if spin == 'dn':
    bndfile = 'bnd-dn.dat'
    figname ="Project_dnbnd.pdf"

if spin == 'no':
    bndfile = 'bnd-up.dat'
    figname ="Project_nospinbnd.pdf"

col='cyan'
kp = ['Γ','M','K','Γ','K']
m=open("note-bnd",'r+')
ml=m.readlines()
ml=[ml[i].strip().split() for i in range(len(ml))]
knum=int(ml[0][0])
bnum=int(ml[0][1])

base=[ml[1],ml[2],ml[3]]
base=np.array(base)
dkvec=[]
for k in range(4,len(ml)-1):
    ml[k]=[float(ml[k][j]) for j in range(3)]
for k in range(5,len(ml)-1):
    dkvec.append((np.array(ml[k])-np.array(ml[k-1])).tolist())
def r(m,n):
    global rfinal
    global r1
    global r2
    global r3
    r1=0
    r2=0
    r3=0
    r1=[float(m[0])*float(n[0][i]) for i in range(3)]
    r2=[float(m[1])*float(n[1][i]) for i in range(3)]
    r3=[float(m[2])*float(n[2][i]) for i in range(3)]
    x=r1[0]+r2[0]+r3[0]
    y=r1[1]+r2[1]+r3[1]
    z=r1[2]+r2[2]+r3[2]
    rfinal=np.sqrt(x**2+y**2+z**2)
sum=0
k=[0 for i in range(len(ml)-6)]
for i in range(len(ml)-6):
    r(dkvec[i],base)
    k[i]=rfinal
    sum=k[i]+sum
mesh=int(ml[len(ml)-1][0])
x=[0]
a0=0
for i in range(len(ml)-6):
    for j in range(mesh):
        a0=a0+k[i]/(mesh+0.0)
        x.append(a0)

plt.figure(figsize=(20,16))

def plt_pro(i,orb,col,lab):
    plt.subplot(2,4,i)

```

```

#####up#####
a = np.loadtxt(bndfile)
# file=open(bndfile,'r+')
# lines=file.readlines()
# a=[float(lines[i].strip()) for i in range(len(lines))]
# a=np.array(a)
a.resize(knum,bnum)
b=np.transpose(a)
plt.ylim(min, max)
plt.xlim(0,sum)
plt.plot([0,sum],[0,0], 'r-.',linewidth=1 )
hline=0
xlab_val = [0]
for i in range(len(ml)-6):
    hline=hline+k[i]
    xlab_val.append(hline)
hline=0
for i in range(len(ml)-7):
    hline=hline+k[i]
    plt.plot([hline,hline],[min,max+step], 'b-.', linewidth=1)
for i in range(bnum):
    plt.plot(x,b[i], 'k--',linewidth = 1)
a2 = np.loadtxt(orb)
lens = len(a2)//2
if spin == 'up':
    a2 = a2[0:lens]
else:
    a2 = a2[lens:]
a2.resize(knum,bnum)
b2=np.transpose(a2)
for i in range(bnum):
    ax=plt.scatter(x,b[i],c=col, s=b2[i]*300,lw=0, alpha=0.7)
    plt.legend([ax],[lab],frameon= False,fontsize = 20,loc='upper right')
###Fermi-and-high-sym-line###

plt.ylabel('Energy (eV)')

plt.xticks(xlab_val, kp)
plt.tight_layout()
plt.yticks(np.arange(min,max+step,step))
ax = plt.gca()
plt.subplots_adjust(left=0.15, right=0.9, top=0.9, bottom=0.1)
plt_pro(1,orbs[0],"gray",r'$p_x$')
plt_pro(2,orbs[1],"gray",r'$p_y$')
plt_pro(3,orbs[2],"gray",r'$p_z$')
plt_pro(4,orbs[3],"cyan",r'$d_{z^2}$')
plt_pro(5,orbs[4],"red",r'$d_{x^2-y^2}$')
plt_pro(6,orbs[5],"red",r'$d_{xy}$')
plt_pro(7,orbs[6],"blue",r'$d_{yz}$')
plt_pro(8,orbs[7],"blue",r'$d_{xz}$')
plt.subplots_adjust(wspace =0.25, hspace =0.1)
plt.savefig(figname,format='pdf', bbox_inches='tight', dpi=300)

```

(原创脚本)Pyprocar 绘制能带图

► 代码清单 A.77.1 (python, 87 行)

```

import pyprocar as ppc
import matplotlib.pyplot as plt
import os

```

```

from matplotlib.lines import Line2D

# 设置全局字体为 Times New Roman
plt.rcParams.update({
    'font.family': 'Times New Roman',
    'mathtext.fontset': 'stix',
    'font.size': 12,
    'axes.titlesize': 14,      # 标题字体大小
    'axes.labelsize': 12,     # 坐标轴标签字体大小
    'xtick.labelsize': 10,    # x 轴刻度字体大小
    'ytick.labelsize': 10,    # y 轴刻度字体大小
    'legend.fontsize': 10,    # 图例字体大小
})

# 保存图片文件, 相对路径
save_dir = r"./"

# 定义绘图颜色
spin_up_color = '#D62728'    # 深红色
spin_down_color = '#1F77B4'  # 蓝色

# 定义文件名
filename = "band.png"
filepath = os.path.join(save_dir, filename) # 组合路径和文件名

# 图例控制: 创建自定义句柄
custom_up = Line2D([0], [0], color=spin_up_color, lw=1.5, label='Spin up')
custom_down = Line2D([0], [0], color=spin_down_color, lw=1.5, label='Spin down')

fig, ax = plt.subplots(figsize=(6,6))

# 绘制 spin up
ppc.bandsplot(
    code='vasp',
    dirname='.',
    mode='plain',
    spins=[0],
    color=spin_up_color,
    elimit=[-2,2],
    fermi=-1.420514,
    ax=ax,
    show=False
)

# 在绘制后设置线宽
for line in ax.lines:
    if line.get_color() == spin_up_color: # 识别 spin up 的线
        line.set_linewidth(1.5)

# 绘制 spin down
ppc.bandsplot(
    code='vasp',
    dirname='.',
    mode='plain',
    spins=[1],
    color=spin_down_color,
    elimit=[-2,2],
    fermi=-1.420514,
    ax=ax,
    show=False
)

# 在绘制后设置线宽
for line in ax.lines:
    if line.get_color() == spin_down_color:
        line.set_linewidth(1.5)

# 确保所有线都是实线
for line in ax.lines:

```

```

        line.set_linestyle('--')

plt.title("00 Band Structure")
plt.xlabel("K-path")
plt.ylabel("Energy (eV)")
plt.axhline(0, color='gray', linestyle='--', linewidth=0.8)
plt.legend(handles=[custom_up, custom_down], loc="best")
# 保存为透明背景的图片
plt.savefig(filepath,
            dpi=300,
            bbox_inches='tight',
            transparent=True, # 关键参数: 设置透明背景
            pad_inches=0.1,
            )
plt.show()

```

MLWFS 绘制轨道

► 代码清单 A.78.4 (shell, 6 行)

```

wannier_plot = .true.
wannier_plot_supercell=3
wannier_plot_format = cube
wannier_plot_mode = crystal
wannier_plot_radius = 2.5
wannier_plot_scale = 1.0

```

(原创脚本) 绘制精美的投影能带图

► 代码清单 A.79.1 (python, 424 行)

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
##### 作者: 马睿骁 浙江理工大学
import os
import re
import argparse
from typing import List, Tuple, Optional

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D

# =====
# POSCAR
# =====
def read_poscar_elements_counts(poscar_path: str) -> Tuple[List[str], List[int]]:
    with open(poscar_path, "r", encoding="utf-8", errors="ignore") as f:
        lines = [ln.strip() for ln in f.readlines() if ln.strip()]
    if len(lines) < 8:
        raise RuntimeError(f"POSCAR too short: {poscar_path}")

```

```

elements = lines[5].split()
counts = [int(x) for x in lines[6].split()]
return elements, counts

def element_to_atom_indices_lbased(elements: List[str], counts: List[int], pick: Optional[List[str]]) ->
↳ np.ndarray:
    total = sum(counts)
    if not pick:
        return np.arange(1, total + 1, dtype=int)

    pick_set = set(pick)
    out = []
    s = 1
    for el, n in zip(elements, counts):
        if el in pick_set:
            out.extend(range(s, s + n))
            s += n
    if not out:
        raise RuntimeError(f"Selected elements {pick} not found in POSCAR elements={elements}")
    return np.array(out, dtype=int)

# =====
# OUTCAR: E-fermi
# =====
def read_fermi_from_outcar(outcar_path: str) -> Optional[float]:
    if not os.path.exists(outcar_path):
        return None
    pat = re.compile(r"E-fermi\s*:\s*([-\\d\\.]+)")
    fermi = None
    with open(outcar_path, "r", encoding="utf-8", errors="ignore") as f:
        for ln in f:
            m = pat.search(ln)
            if m:
                fermi = float(m.group(1))
    return fermi

# =====
# KPOINTS (VASPKIT)
# =====
def pretty_label(lbl: str) -> str:
    s = lbl.strip()
    if s.upper() == "GAMMA":
        return r"$\Gamma$"
    if s in ("K'", "K' ", "Kp", "KP"):
        return r"$K'$"
    return rf"${s}$"

def parse_vaskit_kpoints_nodes(kpoints_path: str) -> List[str]:
    coord_pat = re.compile(
        r"^\s*([-\\d\\.]+)\s+([-\\d\\.]+)\s+([-\\d\\.]+)\s+([A-Za-z0-9\\-\\+'' ]+)\s*$"
    )
    labels_all = []
    with open(kpoints_path, "r", encoding="utf-8", errors="ignore") as f:
        for ln in f:
            m = coord_pat.match(ln.strip())
            if m:
                labels_all.append(m.group(4).strip())

    if len(labels_all) < 2:
        raise RuntimeError("Failed to parse high-symmetry labels from KPOINTS.")

    compressed = []
    for lab in labels_all:
        if not compressed or lab != compressed[-1]:
            compressed.append(lab)

```

```

cleaned = []
i = 0
while i < len(compressed):
    cleaned.append(compressed[i])
    if i + 2 < len(compressed) and compressed[i + 1] == compressed[i + 2]:
        i += 2
    else:
        i += 1
return cleaned

def make_kticks_knames(nodes: List[str], nk: int) -> Tuple[List[str], List[int]]:
    nseg = len(nodes) - 1
    kticks = [int(round(i * (nk - 1) / nseg)) for i in range(len(nodes))]
    kticks[0] = 0
    kticks[-1] = nk - 1
    knames = [pretty_label(x) for x in nodes]
    return knames, kticks

# =====
# Orbitals mapping (PROCAR lm decomposed header order)
# ion s py pz px dxy dyz dz2 dxz x2-y2 tot
# =====
ORBITAL_INDEX = {
    "s": 0,
    "py": 1, "pz": 2, "px": 3,
    "dxy": 4, "dyz": 5, "dz2": 6, "dxz": 7, "x2-y2": 8,
}
def normalize_orb(s: str) -> str:
    t = s.strip().lower().replace(" ", "").replace("^", "").replace("_", "")
    if t in ("dx2-y2", "dx2y2", "x2y2", "x2-y2"):
        return "x2-y2"
    return t

# =====
# Robust PROCAR parser: auto-detect nspin even without "spin component"
# =====
def parse_procar_lm_decomposed_autospin(procar_path: str, force_nspin: Optional[int] = None):
    """
    Returns:
        kpoints: (nk,3)
        energies: (nk, nb, nspin)
        weights: (nk, nb, nion, 9, nspin)
        nk, nb, nion, nspin
    Works for:
        - with 'spin component 1/2'
        - without 'spin component', but PROCAR contains two full k-point cycles (k=1..nk) -> nspin=2
    """
    with open(procar_path, "r", encoding="utf-8", errors="ignore") as f:
        lines = f.readlines()

    header_text = "".join(lines[:50])
    m = re.search(r"#\s*of\s*k-points:\s*([0-9]+)\s*\#\s*of\s*bands:\s*([0-9]+)\s*\#\s*of\s*ions:\s*([0-9]+)",
                  "".join(lines))
    if not m:
        raise RuntimeError("Failed to parse PROCAR header '# of k-points/bands/ions'.")

    nk = int(m.group(1)); nb = int(m.group(2)); nion = int(m.group(3))
    norb = 9

    # detect number of cycles of "k-point 1"
    kpt1_pat = re.compile(r"^\s*k-point\s+1\s*:", re.IGNORECASE)
    kpt1_count = sum(1 for ln in lines if kpt1_pat.match(ln))

    # prefer force if user specifies
    if force_nspin is not None:

```

```

    nspin = int(force_nspin)
else:
    # typical: 1 cycle -> 1 spin; 2 cycles -> 2 spins
    nspin = 2 if kpt1_count >= 2 else 1

# allocate arrays
kpoints = np.zeros((nk, 3), dtype=float)
energies = np.full((nk, nb, nspin), np.nan, dtype=float)
weights = np.zeros((nk, nb, nion, norb, nspin), dtype=float)

# regex
spin_pat = re.compile(r"spin\s+component\s+([12])", re.IGNORECASE)
kpt_pat = re.compile(r"^\s*k-point\s+(\d+)\s*:\s*([-d\.Ee+])\s+([-d\.Ee+])\s+([-d\.Ee+])")
band_pat = re.compile(r"^\s*band\s+(\d+)\s*\#\s*energy\s+([-d\.Ee+])")
ion_header_pat = re.compile(r"^\s*ion\s+s\s+py\s+pz\s+px\s+dx\s+dy\s+dz\s+dxz\s+xy2\s+y2\s+tot",
↪ re.IGNORECASE)

cur_spin = 0
cur_k = -1
cur_b = -1

# For "no spin component" case, we auto-switch spin when k-point index resets
seen_k_cycle = 0 # 0 for first cycle, 1 for second
last_k_index = None

i = 0
while i < len(lines):
    ln = lines[i]

    # explicit spin tags (if present)
    ms = spin_pat.search(ln)
    if ms:
        cur_spin = int(ms.group(1)) - 1
        i += 1
        continue

    mk = kpt_pat.match(ln)
    if mk:
        k_index_lb = int(mk.group(1)) # 1..nk

        # auto spin switching if no explicit spin tags:
        if force_nspin is None and ("spin component" not in header_text.lower()):
            if last_k_index is not None and k_index_lb == 1 and last_k_index != 1:
                # new cycle begins
                seen_k_cycle += 1
                cur_spin = min(seen_k_cycle, nspin - 1)

        last_k_index = k_index_lb
        cur_k = k_index_lb - 1
        if cur_spin == 0:
            # only need store kpoints once
            kpoints[cur_k, :] = [float(mk.group(2)), float(mk.group(3)), float(mk.group(4))]
            i += 1
            continue

    mb = band_pat.match(ln)
    if mb:
        cur_b = int(mb.group(1)) - 1
        energies[cur_k, cur_b, cur_spin] = float(mb.group(2))
        i += 1
        continue

    if ion_header_pat.match(ln):
        # read nion rows next
        for j in range(nion):
            row = lines[i + 1 + j].split()
            ion_id = int(row[0]) - 1
            vals = [float(x) for x in row[1:1+norb]]
            weights[cur_k, cur_b, ion_id, :, cur_spin] = vals

```

```

        # skip: header + nion lines + "tot" line
        i = i + 1 + nion + 1
        continue

    i += 1

# sanity
if np.isnan(energies).any():
    nan_pos = np.argwhere(np.isnan(energies))
    raise RuntimeError(f"PROCAR parse incomplete: energies has NaNs, e.g. {nan_pos[:8].tolist()}")

return kpoints, energies, weights, nk, nb, nion, nspin, kptl_count

# =====
# Main plotting
# =====
def main():
    ap = argparse.ArgumentParser("2-spin fatband (3-color bubbles) from VASP5.4.4 PROCAR lm decomposed.")
    ap.add_argument("--dirname", default=".")
    ap.add_argument("--elements", default=None, help="filter elements, e.g. 'Mo' or 'Ru,Br'. default all.")
    ap.add_argument("--orbitals", default="dz2,dxy,dx2-y2",
                    help="exactly 3 orbitals, e.g. 'dxy,dyz,dx2-y2'")
    ap.add_argument("--fermi", type=float, default=None)
    ap.add_argument("--emin", type=float, default=-1.2)
    ap.add_argument("--emax", type=float, default=1.2)
    ap.add_argument("--out", default="fatband_2spin_like_target.png")
    ap.add_argument("--dpi", type=int, default=400)

    # visual knobs
    ap.add_argument("--plain_color", default="0.82")
    ap.add_argument("--plain_lw", type=float, default=1.1)
    ap.add_argument("--bubble_scale", type=float, default=220.0)
    ap.add_argument("--bubble_pow", type=float, default=1.0)
    ap.add_argument("--bubble_clip", type=float, default=1.0)

    # force spin if you know it's ISPIN=2 but PROCAR is weird
    ap.add_argument("--force_nspin", type=int, default=None, help="set to 2 to force 2-spin parsing")

    args = ap.parse_args()
    d = os.path.abspath(args.dirname)

    procar = os.path.join(d, "PROCAR")
    poscar = os.path.join(d, "POSCAR")
    kpoints = os.path.join(d, "KPOINTS")
    outcar = os.path.join(d, "OUTCAR")

    for p in [procar, poscar, kpoints]:
        if not os.path.exists(p):
            raise FileNotFoundError(f"Missing file: {p}")

    # atoms selection
    elements, counts = read_poscar_elements_counts(poscar)
    pick = None
    if args.elements:
        pick = [x.strip() for x in args.elements.split(",") if x.strip()]
    sel_ions_1b = element_to_atom_indices_1based(elements, counts, pick)
    sel_ions_0b = sel_ions_1b - 1

    # fermi
    fermi = args.fermi
    if fermi is None:
        f0 = read_fermi_from_outcar(outcar)
        fermi = f0 if f0 is not None else 0.0

    # parse PROCAR (auto spin)
    kpts, E, W, nk, nb, nion, nspin, kptl_count = parse_procar_lm_decomposed_autospin(
        procar, force_nspin=args.force_nspin
    )

```

```

# kpath labels
nodes = parse_vaskit_kpoints_nodes(kpoints)
knames, kticks = make_kticks_knames(nodes, nk)

# orbitals (must be 3)
orb_names = [x.strip() for x in args.orbitals.split(",") if x.strip()]
if len(orb_names) != 3:
    raise RuntimeError(f"--orbitals must give exactly 3 orbitals. Got: {orb_names}")

oidx = []
for n in orb_names:
    key = normalize_orb(n)
    if key not in ORBITAL_INDEX:
        raise RuntimeError(f"Unknown orbital '{n}'. Allowed: {sorted(ORBITAL_INDEX.keys())}")
    oidx.append(ORBITAL_INDEX[key])

# weights sum over selected ions -> (nk, nb, nspin)
def orb_weight(orb_i: int) -> np.ndarray:
    return W[:, :, sel_ions_0b, orb_i, :].sum(axis=2)

wA = orb_weight(oidx[0])
wB = orb_weight(oidx[1])
wC = orb_weight(oidx[2])

def bubble_size(w):
    ww = np.clip(w, 0.0, args.bubble_clip)
    ww = np.power(ww, args.bubble_pow)
    return args.bubble_scale * ww

sA, sB, sC = bubble_size(wA), bubble_size(wB), bubble_size(wC)

# x index along k
x = np.arange(nk, dtype=float)
XX = np.repeat(x[:, None], nb, axis=1).ravel()

# colors
C_BLUE = "#1f4aa8"
C_RED = "#e53935"
C_ORNG = "#f2a400"
colors = [C_BLUE, C_RED, C_ORNG]

# legend labels
legend_map = {
    "dz2": r"$d_{z^2}$",
    "dxy": r"$d_{xy}$",
    "x2-y2": r"$d_{x^2-y^2}$",
    "dxz": r"$d_{xz}$",
    "dyz": r"$d_{yz}$",
    "px": r"$p_x$",
    "py": r"$p_y$",
    "pz": r"$p_z$",
    "s": r"$s$",
}
leg_labels = [legend_map.get(normalize_orb(n), rf"${n}$") for n in orb_names]

# ---- plot
fig, ax = plt.subplots(figsize=(3.3, 3.3), dpi=140)

# base bands: plot ALL spins (overlay, same style)
for sp in range(nspin):
    ax.plot(x, (E[:, :, sp] - fermi), color=args.plain_color, lw=args.plain_lw, zorder=1)

# scatters: plot ALL spins
def draw_scatter(S, color, z):
    for sp in range(nspin):
        YY = (E[:, :, sp] - fermi).ravel()
        SS = S[:, :, sp].ravel()
        ax.scatter(XX, YY, s=SS, c=color, marker="o", linewidths=0, alpha=1.0, zorder=z)

```

```

draw_scatter(sA, colors[0], z=4)
draw_scatter(sB, colors[1], z=5)
draw_scatter(sC, colors[2], z=3)

# dashed references
ax.axhline(0.0, color="0.55", ls="--", lw=0.8, zorder=0)
for t in kticks[1:-1]:
    ax.axvline(t, color="0.55", ls="--", lw=0.8, zorder=0)

# axes formatting
ax.set_xlim(0, nk - 1)
ax.set_ylim(args.emin, args.emax)
ax.set_yticks([-1.2, -0.6, 0.0, 0.6, 1.2])
ax.set_ylabel("Energy (eV)", fontsize=12)

ax.set_xticks(kticks)
ax.set_xticklabels(knames, fontsize=12)

ax.tick_params(axis="y", labelsize=11, width=1.0, length=4)
ax.tick_params(axis="x", width=1.0, length=4)

for spn in ax.spines.values():
    spn.set_linewidth(1.0)

ax.grid(False)

# top legend
proxies = [
    Line2D([0], [0], marker="o", color="none",
           markerfacecolor=colors[i], markeredgecolor="none", markersize=5.2)
    for i in range(3)
]
ax.legend(
    proxies, leg_labels,
    loc="upper center",
    bbox_to_anchor=(0.5, 1.10),
    ncol=3,
    frameon=False,
    handletextpad=0.25,
    columnspacing=0.8,
    fontsize=12,
)

plt.tight_layout(pad=0.6)
fig.savefig(args.out, dpi=args.dpi, bbox_inches="tight")

# guaranteed stdout
print("[OK] saved:", args.out)
print(f"Parsed PROCAR: nk={nk}, nb={nb}, nion={nion}, detected nspin={nspin} (k-point 1 count={kpt1_count})")
print("Fermi used:", fermi)
print("Selected ions (1-based) count:", len(sel_ions_1b), "example:", sel_ions_1b[:10].tolist())
print("Orbitals:", orb_names, "-> indices:", oidx)
print("K-path nodes:", nodes)
print("K ticks:", kticks)
print("K names:", knames)

if __name__ == "__main__":
    main()

```

► 代码清单 A.79.3 (python, 487 行)

```
#!/usr/bin/env python3
```

```

# -*- coding: utf-8 -*-
##### 作者: 马睿骁 浙江理工大学
import os
import re
import argparse
from typing import List, Tuple, Optional, Dict

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
from matplotlib.colors import to_rgba

# =====
# POSCAR
# =====
def read_poscar_elements_counts(poscar_path: str) -> Tuple[List[str], List[int]]:
    with open(poscar_path, "r", encoding="utf-8", errors="ignore") as f:
        lines = [ln.strip() for ln in f.readlines() if ln.strip()]
    if len(lines) < 8:
        raise RuntimeError(f"POSCAR too short: {poscar_path}")
    elements = lines[5].split()
    counts = [int(x) for x in lines[6].split()]
    return elements, counts

def element_to_atom_indices_lbased(elements: List[str], counts: List[int], pick: Optional[List[str]]) ->
↳ np.ndarray:
    total = sum(counts)
    if not pick:
        return np.arange(1, total + 1, dtype=int)
    pick_set = set(pick)
    out = []
    s = 1
    for el, n in zip(elements, counts):
        if el in pick_set:
            out.extend(range(s, s + n))
            s += n
    if not out:
        raise RuntimeError(f"Selected elements {pick} not found in POSCAR elements={elements}")
    return np.array(out, dtype=int)

# =====
# OUTCAR: E-fermi
# =====
def read_fermi_from_outcar(outcar_path: str) -> Optional[float]:
    if not os.path.exists(outcar_path):
        return None
    pat = re.compile(r"E-fermi\s*:\s*([-\d\.]+)")
    fermi = None
    with open(outcar_path, "r", encoding="utf-8", errors="ignore") as f:
        for ln in f:
            m = pat.search(ln)
            if m:
                fermi = float(m.group(1))
    return fermi

# =====
# KPOINTS (VASPKIT)
# =====
def pretty_label(lbl: str) -> str:
    s = lbl.strip()
    if s.upper() == "GAMMA":
        return r"$\Gamma$"
    if s in ("K'", "K'", "Kp", "KP"):
        return r"$K'$"
    return rf"${s}$"

```

```

def parse_vaskit_kpoints_nodes(kpoints_path: str) -> List[str]:
    coord_pat = re.compile(
        r"^\s*([-\d\.]+)\s+([-\d\.]+)\s+([-\d\.]+)\s+([A-Za-z0-9\-\+\']+)\s*$"
    )
    labels_all = []
    with open(kpoints_path, "r", encoding="utf-8", errors="ignore") as f:
        for ln in f:
            m = coord_pat.match(ln.strip())
            if m:
                labels_all.append(m.group(4).strip())
    if len(labels_all) < 2:
        raise RuntimeError("Failed to parse high-symmetry labels from KPOINTS.")

    compressed = []
    for lab in labels_all:
        if not compressed or lab != compressed[-1]:
            compressed.append(lab)

    cleaned = []
    i = 0
    while i < len(compressed):
        cleaned.append(compressed[i])
        if i + 2 < len(compressed) and compressed[i + 1] == compressed[i + 2]:
            i += 2
        else:
            i += 1
    return cleaned

def make_kticks_knames(nodes: List[str], nk: int) -> Tuple[List[str], List[int]]:
    nseg = len(nodes) - 1
    kticks = [int(round(i * (nk - 1) / nseg)) for i in range(len(nodes))]
    kticks[0] = 0
    kticks[-1] = nk - 1
    knames = [pretty_label(x) for x in nodes]
    return knames, kticks

# =====
# Orbitals (ONLY s/p/d allowed)
# =====
ORBITAL_INDEX = {
    "s": 0,
    "py": 1, "pz": 2, "px": 3,
    "dxy": 4, "dyz": 5, "dz2": 6, "dxz": 7, "x2-y2": 8,
}

def normalize_orb(s: str) -> str:
    t = s.strip().lower().replace(" ", "").replace("^", "").replace("_", "")
    if t in ("dx2-y2", "dx2y2", "x2y2", "x2-y2"):
        return "x2-y2"
    return t

def expand_orbitals(orb_str: str) -> List[str]:
    raw = [x for x in re.split(r"[,\+\;\s]+", orb_str) if x.strip()]
    out = []
    for tok in raw:
        k = normalize_orb(tok)
        if k == "s":
            out.append("s")
        elif k == "p":
            out += ["px", "py", "pz"]
        elif k == "d":
            out += ["dxy", "dyz", "dz2", "dxz", "x2-y2"]
        else:
            if k not in ORBITAL_INDEX:
                raise RuntimeError(

```

```

        f"Illegal orbital token '{tok}'. Only s/p/d orbitals are allowed: "
        f"{sorted(ORBITAL_INDEX.keys())} and groups s/p/d."
    )
    out.append(k)

seen = set()
uniq = []
for x in out:
    if x not in seen:
        uniq.append(x)
        seen.add(x)
return uniq

def latex_label(orb: str) -> str:
    if orb == "s":
        return r"$s$"
    if orb in ("px", "py", "pz"):
        return rf"$p_{{{{orb[-1]}}}}$"
    if orb == "dz2":
        return r"$d_{z^2}$"
    if orb == "dxy":
        return r"$d_{xy}$"
    if orb == "dyz":
        return r"$d_{yz}$"
    if orb == "dxz":
        return r"$d_{xz}$"
    if orb == "x2-y2":
        return r"$d_{x^2-y^2}$"
    return rf"${orb}$"

# =====
# Robust PROCAR parser: auto-detect nspin
# =====
def parse_procar_lm_decomposed_autospin(procar_path: str, force_nspin: Optional[int] = None):
    with open(procar_path, "r", encoding="utf-8", errors="ignore") as f:
        lines = f.readlines()

    m = re.search(
        r"#\s*of\s*k-points:\s*([0-9]+)\s*\#\s*of\s*bands:\s*([0-9]+)\s*\#\s*of\s*ions:\s*([0-9]+)",
        "".join(lines)
    )
    if not m:
        raise RuntimeError("Failed to parse PROCAR header '# of k-points/bands/ions'.")

    nk = int(m.group(1)); nb = int(m.group(2)); nion = int(m.group(3))
    norb = 9

    header_text = "".join(lines[:80]).lower()
    kpt1_pat = re.compile(r"^\s*k-point\s+l\s*:", re.IGNORECASE)
    kpt1_count = sum(1 for ln in lines if kpt1_pat.match(ln))

    if force_nspin is not None:
        nspin = int(force_nspin)
    else:
        if "spin component" in header_text:
            nspin = 2
        else:
            nspin = 2 if kpt1_count >= 2 else 1

    kpoints = np.zeros((nk, 3), dtype=float)
    energies = np.full((nk, nb, nspin), np.nan, dtype=float)
    weights = np.zeros((nk, nb, nion, norb, nspin), dtype=float)

    spin_pat = re.compile(r"spin\s+component\s+([12])", re.IGNORECASE)
    kpt_pat = re.compile(r"^\s*k-point\s+(\d+)\s*\:\s*([-\d\.Ee+])\s+([-\d\.Ee+])\s+([-\d\.Ee+])")
    band_pat = re.compile(r"^\s*band\s+(\d+)\s*\#\s*energy\s+([-\d\.Ee+])")
    ion_header_pat = re.compile(
        r"^\s*ion\s+s\s+py\s+pz\s+px\s+dxy\s+dyz\s+dz2\s+dxz\s+x2-y2\s+tot",

```

```

        re.IGNORECASE
    )

    cur_spin = 0
    cur_k = -1
    cur_b = -1

    seen_k_cycle = 0
    last_k_index = None

    i = 0
    while i < len(lines):
        ln = lines[i]

        ms = spin_pat.search(ln)
        if ms:
            cur_spin = int(ms.group(1)) - 1
            i += 1
            continue

        mk = kpt_pat.match(ln)
        if mk:
            k_index_lb = int(mk.group(1))

            if force_nspin is None and ("spin component" not in header_text):
                if last_k_index is not None and k_index_lb == 1 and last_k_index != 1:
                    seen_k_cycle += 1
                cur_spin = min(seen_k_cycle, nspin - 1)

            last_k_index = k_index_lb
            cur_k = k_index_lb - 1

            if cur_spin == 0:
                kpoints[cur_k, :] = [float(mk.group(2)), float(mk.group(3)), float(mk.group(4))]
                i += 1
                continue

        mb = band_pat.match(ln)
        if mb:
            cur_b = int(mb.group(1)) - 1
            energies[cur_k, cur_b, cur_spin] = float(mb.group(2))
            i += 1
            continue

        if ion_header_pat.match(ln):
            for j in range(nion):
                row = lines[i + 1 + j].split()
                ion_id = int(row[0]) - 1
                vals = [float(x) for x in row[1:1+norb]]
                weights[cur_k, cur_b, ion_id, :, cur_spin] = vals
                i = i + 1 + nion + 1
            continue

        i += 1

    if np.isnan(energies).any():
        nan_pos = np.argwhere(np.isnan(energies))
        raise RuntimeError(f"PROCAR parse incomplete: energies has NaNs, e.g. {nan_pos[:8].tolist()}")

    return kpoints, energies, weights, nk, nb, nion, nspin, kpt1_count

# =====
# Colors: fixed by orbital + optional explicit list
# =====
def parse_fixed_colors(s: Optional[str]) -> Dict[str, str]:
    if not s:
        return {}
    out = {}

```

```

for part in s.split(","):
    part = part.strip()
    if not part:
        continue
    if "=" not in part:
        raise RuntimeError(f"Bad --fixed_colors token '{part}', expect key=#RRGGBB")
    k, v = part.split("=", 1)
    k = normalize_orb(k)
    v = v.strip()
    # validate color string now (raise if invalid)
    to_rgba(v)
    out[k] = v
return out

def parse_colors_list(s: Optional[str]) -> Optional[List[str]]:
    """
    --colors "#2f4aa8,#e53935,#f2a400"
    """
    if not s:
        return None
    cols = [c.strip() for c in s.split(",") if c.strip()]
    if not cols:
        return None
    for c in cols:
        to_rgba(c) # validate
    return cols

def auto_color_list(n: int) -> List[str]:
    if n <= 10:
        cmap = plt.get_cmap("tab10")
        return [cmap(i) for i in range(n)]
    cmap = plt.get_cmap("tab20")
    return [cmap(i % 20) for i in range(n)]

def assign_colors(orbs: List[str], fixed: Dict[str, str], colors_list: Optional[List[str]]) -> List:
    """
    Priority:
    1) fixed_colors per-orbital
    2) --colors list by order (cycled)
    3) auto colors
    """
    n = len(orbs)
    if colors_list:
        cols = colors_list
        out = []
        for i, o in enumerate(orbs):
            out.append(fixed.get(o, cols[i % len(cols)]))
        return out

    auto = auto_color_list(n)
    return [fixed.get(o, auto[i]) for i, o in enumerate(orbs)]

# =====
# Main
# =====
def main():
    ap = argparse.ArgumentParser("Fatband (s/p/d only) any number of orbitals, custom colors supported.")
    ap.add_argument("--dirname", default=".")
    ap.add_argument("--elements", default=None, help="e.g. 'Mo' or 'Ru,Br'. default all atoms.")
    ap.add_argument("--orbitals", required=True,
        help="Any number. Tokens can be s,p,d or explicit: px,py,pz,dxy,dyz,dz2,dxz,x2-y2. "
        "Example: 'd' or 'dxy dyz dx2-y2' or 'p dz2'.")
    ap.add_argument("--fixed_colors", default=None,
        help="Fixed colors by orbital: 'dz2=#2f4aa8,dxy=#e53935,x2-y2=#f2a400'")
    ap.add_argument("--colors", default=None,
        help="Optional ordered color list: '#2f4aa8,#e53935,#f2a400' (cycled if shorter than
        ↪ orbitals)")

```

```

ap.add_argument("--fermi", type=float, default=None)
ap.add_argument("--emin", type=float, default=-1.2)
ap.add_argument("--emax", type=float, default=1.2)
ap.add_argument("--out", default="fatband_anyorb_spd.png")
ap.add_argument("--dpi", type=int, default=400)

ap.add_argument("--plain_color", default="0.82")
ap.add_argument("--plain_lw", type=float, default=1.1)
ap.add_argument("--bubble_scale", type=float, default=150.0)
ap.add_argument("--bubble_pow", type=float, default=1.0)
ap.add_argument("--bubble_clip", type=float, default=1.0)

ap.add_argument("--force_nspin", type=int, default=None, help="force 2 if needed")

args = ap.parse_args()
d = os.path.abspath(args.dirname)

procar = os.path.join(d, "PROCAR")
poscar = os.path.join(d, "POSCAR")
kpoints = os.path.join(d, "KPOINTS")
outcar = os.path.join(d, "OUTCAR")

for p in [procar, poscar, kpoints]:
    if not os.path.exists(p):
        raise FileNotFoundError(f"Missing file: {p}")

# orbitals expand
orbs = expand_orbitals(args.orbitals)
if len(orbs) == 0:
    raise RuntimeError("No valid orbitals parsed from --orbitals")

# atoms selection
elements, counts = read_poscar_elements_counts(poscar)
pick = None
if args.elements:
    pick = [x.strip() for x in args.elements.split(",") if x.strip()]
sel_ions_lb = element_to_atom_indices_lbased(elements, counts, pick)
sel_ions_0b = sel_ions_lb - 1

# fermi
fermi = args.fermi
if fermi is None:
    f0 = read_fermi_from_outcar(outcar)
    fermi = f0 if f0 is not None else 0.0

# parse PROCAR
kpts, E, W, nk, nb, nion, nspin, kptl_count = parse_procar_lm_decomposed_autospin(
    procar, force_nspin=args.force_nspin
)

# kpath labels
nodes = parse_vaskit_kpoints_nodes(kpoints)
knames, kticks = make_kticks_knames(nodes, nk)

# colors
fixed = parse_fixed_colors(args.fixed_colors)
colors_list = parse_colors_list(args.colors)
colors = assign_colors(orbs, fixed, colors_list)

# bubble mapping
def bubble_size(w):
    ww = np.clip(w, 0.0, args.bubble_clip)
    ww = np.power(ww, args.bubble_pow)
    return args.bubble_scale * ww

x = np.arange(nk, dtype=float)
XX = np.repeat(x[:, None], nb, axis=1).ravel()

# compute sizes for each orbital

```

```

sizes_list = []
for o in orbs:
    oi = ORBITAL_INDEX[o]
    w = W[:, :, sel_ions_0b, oi, :].sum(axis=2) # (nk,nb,nspin)
    sizes_list.append(bubble_size(w))

# ---- plot
fig, ax = plt.subplots(figsize=(3.3, 3.3), dpi=140)

# base bands: all spins
for sp in range(nspin):
    ax.plot(x, (E[:, :, sp] - fermi), color=args.plain_color, lw=args.plain_lw, zorder=1)

# scatters
for i, o in enumerate(orbs):
    S = sizes_list[i]
    col = colors[i]
    z = 3 + i
    for sp in range(nspin):
        YY = (E[:, :, sp] - fermi).ravel()
        SS = S[:, :, sp].ravel()
        ax.scatter(XX, YY, s=SS, c=[col], marker="o", linewidths=0, alpha=1.0, zorder=z)

# dashed refs
ax.axhline(0.0, color="0.55", ls="--", lw=0.8, zorder=0)
for t in kticks[1:-1]:
    ax.axvline(t, color="0.55", ls="--", lw=0.8, zorder=0)

# axes formatting
ax.set_xlim(0, nk - 1)
ax.set_ylim(args.emin, args.emax)
ax.set_yticks([-1.2, -0.6, 0.0, 0.6, 1.2])
ax.set_ylabel("Energy (eV)", fontsize=12)
ax.set_xticks(kticks)
ax.set_xticklabels(knames, fontsize=12)
ax.tick_params(axis="y", labelsize=11, width=1.0, length=4)
ax.tick_params(axis="x", width=1.0, length=4)
for spn in ax.spines.values():
    spn.set_linewidth(1.0)
ax.grid(False)

# legend
proxies = [
    Line2D([0], [0], marker="o", color="none",
           markerfacecolor=colors[i], markeredgecolor="none", markersize=5.2)
    for i in range(len(orbs))
]
labels = [latex_label(o) for o in orbs]
ncol = min(len(orbs), 5)
ax.legend(
    proxies, labels,
    loc="upper center",
    bbox_to_anchor=(0.5, 1.16),
    ncol=ncol,
    frameon=False,
    handletextpad=0.25,
    columnspacing=0.8,
    fontsize=12,
)

plt.tight_layout(pad=0.6)
fig.savefig(args.out, dpi=args.dpi, bbox_inches="tight")

print("[OK] saved:", args.out)
print(f"Parsed PROCAR: nk={nk}, nb={nb}, nion={nion}, detected nspin={nspin} (k-point 1 count={kpt1_count})")
print("Fermi used:", fermi)
print("Selected ions (1-based) count:", len(sel_ions_1b), "example:", sel_ions_1b[:10].tolist())
print("Orbitals expanded:", orbs)
print("Colors used:", colors)

```

```

print("K-path nodes:", nodes)

if __name__ == "__main__":
    main()

```

► 代码清单 A.79.5 (python, 481 行)

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import os
import re
import argparse
from typing import List, Tuple, Optional, Dict

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
from matplotlib.colors import to_rgba

def set_global_font_times():
    plt.rcParams["font.family"] = "Times New Roman"
    plt.rcParams["axes.unicode_minus"] = False

    # Make mathtext (legend labels like  $x_{xy}$ ) use Times New Roman too
    plt.rcParams["mathtext.fontset"] = "custom"
    plt.rcParams["mathtext.rm"] = "Times New Roman"
    plt.rcParams["mathtext.it"] = "Times New Roman:italic"
    plt.rcParams["mathtext.bf"] = "Times New Roman:bold"

def read_poscar_elements_counts(poscar_path: str) -> Tuple[List[str], List[int]]:
    with open(poscar_path, "r", encoding="utf-8", errors="ignore") as f:
        lines = [ln.strip() for ln in f.readlines() if ln.strip()]
    if len(lines) < 8:
        raise RuntimeError(f"POSCAR too short: {poscar_path}")
    elements = lines[5].split()
    counts = [int(x) for x in lines[6].split()]
    return elements, counts

def element_to_atom_indices_lbased(elements: List[str], counts: List[int], pick: Optional[List[str]]) ->
↳ np.ndarray:
    total = sum(counts)
    if not pick:
        return np.arange(1, total + 1, dtype=int)
    pick_set = set(pick)
    out = []
    s = 1
    for el, n in zip(elements, counts):
        if el in pick_set:
            out.extend(range(s, s + n))
            s += n
    if not out:
        raise RuntimeError(f"Selected elements {pick} not found in POSCAR elements={elements}")
    return np.array(out, dtype=int)

def read_fermi_from_outcar(outcar_path: str) -> Optional[float]:
    if not os.path.exists(outcar_path):
        return None

```

```

pat = re.compile(r"E-fermi\s*:\s*([-\\d\\.]+)")
fermi = None
with open(outcar_path, "r", encoding="utf-8", errors="ignore") as f:
    for ln in f:
        m = pat.search(ln)
        if m:
            fermi = float(m.group(1))
return fermi

# ---- plain non-italic HS labels (Times) ----
def plain_label(lbl: str) -> str:
    s = lbl.strip()
    if s.upper() == "GAMMA":
        return "Γ"
    if s in ("K'", "K' ", "Kp", "KP"):
        return "K'"
    return s

def parse_vaskit_kpoints_nodes(kpoints_path: str) -> List[str]:
    coord_pat = re.compile(
        r"^s*([-\\d\\.]+)s+([-\\d\\.]+)s+([-\\d\\.]+)s+([A-Za-z0-9\\-\\+'' ]+)s*$"
    )
    labels_all = []
    with open(kpoints_path, "r", encoding="utf-8", errors="ignore") as f:
        for ln in f:
            m = coord_pat.match(ln.strip())
            if m:
                labels_all.append(m.group(4).strip())
    if len(labels_all) < 2:
        raise RuntimeError("Failed to parse high-symmetry labels from KPOINTS.")

    compressed = []
    for lab in labels_all:
        if not compressed or lab != compressed[-1]:
            compressed.append(lab)

    cleaned = []
    i = 0
    while i < len(compressed):
        cleaned.append(compressed[i])
        if i + 2 < len(compressed) and compressed[i + 1] == compressed[i + 2]:
            i += 2
        else:
            i += 1
    return cleaned

def make_kticks_knames(nodes: List[str], nk: int) -> Tuple[List[str], List[int]]:
    nseg = len(nodes) - 1
    kticks = [int(round(i * (nk - 1) / nseg)) for i in range(len(nodes))]
    kticks[0] = 0
    kticks[-1] = nk - 1
    knames = [plain_label(x) for x in nodes]
    return knames, kticks

ORBITAL_INDEX = {
    "s": 0,
    "py": 1, "pz": 2, "px": 3,
    "dxy": 4, "dyz": 5, "dz2": 6, "dxz": 7, "x2-y2": 8,
}

def normalize_orb(s: str) -> str:
    t = s.strip().lower().replace(" ", "").replace("^", "").replace("_", "")
    if t in ("dx2-y2", "dx2y2", "x2y2", "x2-y2"):
        return "x2-y2"

```

```

return t

def expand_orbitals(orb_str: str) -> List[str]:
    raw = [x for x in re.split(r"[,\+;\s]+", orb_str) if x.strip()]
    out = []
    for tok in raw:
        k = normalize_orb(tok)
        if k == "s":
            out.append("s")
        elif k == "p":
            out += ["px", "py", "pz"]
        elif k == "d":
            out += ["dxy", "dyz", "dz2", "dxz", "x2-y2"]
        else:
            if k not in ORBITAL_INDEX:
                raise RuntimeError(
                    f"Illegal orbital token '{tok}'. Only s/p/d are allowed."
                )
            out.append(k)
    seen = set()
    uniq = []
    for x in out:
        if x not in seen:
            uniq.append(x)
            seen.add(x)
    return uniq

def latex_label(orb: str) -> str:
    # 图例仍用 mathtext (显示 d_{xy}), 但字体我们强制 Times New Roman
    if orb == "s":
        return r"$s$"
    if orb in ("px", "py", "pz"):
        return rf"$p_{{\{orb[-1]\}}}$"
    if orb == "dz2":
        return r"$d_{z^2}$"
    if orb == "dxy":
        return r"$d_{xy}$"
    if orb == "dyz":
        return r"$d_{yz}$"
    if orb == "dxz":
        return r"$d_{xz}$"
    if orb == "x2-y2":
        return r"$d_{x^2-y^2}$"
    return rf"${orb}$"

def parse_procar_lm_decomposed_autospin(procar_path: str, force_nspin: Optional[int] = None):
    with open(procar_path, "r", encoding="utf-8", errors="ignore") as f:
        lines = f.readlines()

    m = re.search(
        r"#\s*of\s*k-points:\s*([0-9]+)\s*\#\s*of\s*bands:\s*([0-9]+)\s*\#\s*of\s*ions:\s*([0-9]+)",
        "".join(lines)
    )
    if not m:
        raise RuntimeError("Failed to parse PROCAR header.")

    nk = int(m.group(1)); nb = int(m.group(2)); nion = int(m.group(3))
    norb = 9

    header_text = "".join(lines[:80]).lower()
    kpt1_pat = re.compile(r"^\s*k-point\s+l\s*:", re.IGNORECASE)
    kpt1_count = sum(1 for ln in lines if kpt1_pat.match(ln))

    if force_nspin is not None:
        nspin = int(force_nspin)
    else:

```

```

        if "spin component" in header_text:
            nspin = 2
        else:
            nspin = 2 if kpt1_count >= 2 else 1

kpoints = np.zeros((nk, 3), dtype=float)
energies = np.full((nk, nb, nspin), np.nan, dtype=float)
weights = np.zeros((nk, nb, nion, norb, nspin), dtype=float)

spin_pat = re.compile(r"spin\s+component\s+([12])", re.IGNORECASE)
kpt_pat = re.compile(r"^\s*k-point\s+(\d+)\s*:\s*([-\d\.Ee+])\s+([-\d\.Ee+])\s+([-\d\.Ee+])")
band_pat = re.compile(r"^\s*band\s+(\d+)\s*\#\s*energy\s+([-\d\.Ee+])")
ion_header_pat = re.compile(
    r"^\s*ion\s+s\s+py\s+pz\s+px\s+dxy\s+dyz\s+dz2\s+dxz\s+x2-y2\s+tot",
    re.IGNORECASE
)

cur_spin = 0
cur_k = -1
cur_b = -1

seen_k_cycle = 0
last_k_index = None

i = 0
while i < len(lines):
    ln = lines[i]

    ms = spin_pat.search(ln)
    if ms:
        cur_spin = int(ms.group(1)) - 1
        i += 1
        continue

    mk = kpt_pat.match(ln)
    if mk:
        k_index_lb = int(mk.group(1))

        if force_nspin is None and ("spin component" not in header_text):
            if last_k_index is not None and k_index_lb == 1 and last_k_index != 1:
                seen_k_cycle += 1
            cur_spin = min(seen_k_cycle, nspin - 1)

        last_k_index = k_index_lb
        cur_k = k_index_lb - 1

        if cur_spin == 0:
            kpoints[cur_k, :] = [float(mk.group(2)), float(mk.group(3)), float(mk.group(4))]
            i += 1
            continue

    mb = band_pat.match(ln)
    if mb:
        cur_b = int(mb.group(1)) - 1
        energies[cur_k, cur_b, cur_spin] = float(mb.group(2))
        i += 1
        continue

    if ion_header_pat.match(ln):
        for j in range(nion):
            row = lines[i + 1 + j].split()
            ion_id = int(row[0]) - 1
            vals = [float(x) for x in row[1:1+norb]]
            weights[cur_k, cur_b, ion_id, :, cur_spin] = vals
            i = i + 1 + nion + 1
            continue

    i += 1

```

```

if np.isnan(energies).any():
    nan_pos = np.argwhere(np.isnan(energies))
    raise RuntimeError(f"PROCAR parse incomplete: energies has NaNs, e.g. {nan_pos[:8].tolist()}")

return kpoints, energies, weights, nk, nb, nion, nspin, kpt1_count

def parse_fixed_colors(s: Optional[str]) -> Dict[str, str]:
    if not s:
        return {}
    out = {}
    for part in s.split(","):
        part = part.strip()
        if not part:
            continue
        if "=" not in part:
            raise RuntimeError(f"Bad --fixed_colors token '{part}', expect key=#RRGGBB")
        k, v = part.split("=", 1)
        k = normalize_orb(k)
        v = v.strip()
        to_rgba(v)
        out[k] = v
    return out

def parse_colors_list(s: Optional[str]) -> Optional[List[str]]:
    if not s:
        return None
    cols = [c.strip() for c in s.split(",") if c.strip()]
    if not cols:
        return None
    for c in cols:
        to_rgba(c)
    return cols

def auto_color_list(n: int) -> List[str]:
    cmap = plt.get_cmap("tab10") if n <= 10 else plt.get_cmap("tab20")
    return [cmap(i % cmap.N) for i in range(n)]

def assign_colors(orbs: List[str], fixed: Dict[str, str], colors_list: Optional[List[str]]) -> List:
    auto = auto_color_list(len(orbs))
    out = []
    for i, o in enumerate(orbs):
        if colors_list:
            out.append(fixed.get(o, colors_list[i % len(colors_list)]))
        else:
            out.append(fixed.get(o, auto[i]))
    return out

def bold_ticks(ax):
    for t in ax.get_xticklabels():
        t.set_fontweight("bold")
        t.set_fontfamily("Times New Roman")
        t.set_fontstyle("normal")
    for t in ax.get_yticklabels():
        t.set_fontweight("bold")
        t.set_fontfamily("Times New Roman")
        t.set_fontstyle("normal")

def main():
    set_global_font_times()

    ap = argparse.ArgumentParser("Fatband (s/p/d only).")
    ap.add_argument("--dirname", default=".")
    ap.add_argument("--elements", default=None)

```

```

ap.add_argument("--orbitals", required=True)
ap.add_argument("--fixed_colors", default=None)
ap.add_argument("--colors", default=None)

ap.add_argument("--fermi", type=float, default=None)
ap.add_argument("--emin", type=float, default=-1.2)
ap.add_argument("--emax", type=float, default=1.2)
ap.add_argument("--out", default="fatband.png")
ap.add_argument("--dpi", type=int, default=400)

ap.add_argument("--plain_color", default="0.82")
ap.add_argument("--plain_lw", type=float, default=1.1)
ap.add_argument("--bubble_scale", type=float, default=220.0)
ap.add_argument("--bubble_pow", type=float, default=1.0)
ap.add_argument("--bubble_clip", type=float, default=1.0)
ap.add_argument("--width_scale", type=float, default=1.0)

ap.add_argument("--force_nspin", type=int, default=None)

ap.add_argument("--legend_y", type=float, default=1.14, help="legend bbox_to_anchor y (closer to top)")
ap.add_argument("--legend_fontsize", type=float, default=20)
ap.add_argument("--legend_markersize", type=float, default=10.0)

args = ap.parse_args()
d = os.path.abspath(args.dirname)

procar = os.path.join(d, "PROCAR")
poscar = os.path.join(d, "POSCAR")
kpoints = os.path.join(d, "KPOINTS")
outcar = os.path.join(d, "OUTCAR")
for p in [procar, poscar, kpoints]:
    if not os.path.exists(p):
        raise FileNotFoundError(f"Missing file: {p}")

orbs = expand_orbitals(args.orbitals)
if not orbs:
    raise RuntimeError("No valid orbitals parsed from --orbitals")

elements, counts = read_poscar_elements_counts(poscar)
pick = None
if args.elements:
    pick = [x.strip() for x in args.elements.split(",") if x.strip()]
sel_ions_0b = element_to_atom_indices_lbased(elements, counts, pick) - 1

fermi = args.fermi
if fermi is None:
    f0 = read_fermi_from_outcar(outcar)
    fermi = f0 if f0 is not None else 0.0

_, E, W, nk, nb, _, nspin, _ = parse_procar_lm_decomposed_autospin(procar, force_nspin=args.force_nspin)

nodes = parse_vaskit_kpoints_nodes(kpoints)
knames, kticks = make_kticks_knames(nodes, nk)

fixed = parse_fixed_colors(args.fixed_colors)
colors_list = parse_colors_list(args.colors)
colors = assign_colors(orbs, fixed, colors_list)

def bubble_size(w):
    ww = np.clip(w, 0.0, args.bubble_clip)
    ww = np.power(ww, args.bubble_pow)
    return args.width_scale * args.bubble_scale * ww

x = np.arange(nk, dtype=float)
XX = np.repeat(x[:, None], nb, axis=1).ravel()

sizes_list = []
for o in orbs:
    oi = ORBITAL_INDEX[o]

```

```

        w = W[:, :, sel_ions_0b, oi, :].sum(axis=2)
        sizes_list.append(bubble_size(w))

scale = 0.7

fig, ax = plt.subplots(figsize=(6.0, 6.0), dpi=140)

for sp in range(nspin):
    ax.plot(x, (E[:, :, sp] - fermi), color=args.plain_color, lw=args.plain_lw, zorder=1)

for i, o in enumerate(orbs):
    S = sizes_list[i]
    col = colors[i]
    z = 3 + i
    for sp in range(nspin):
        YY = (E[:, :, sp] - fermi).ravel()
        SS = S[:, :, sp].ravel()
        ax.scatter(XX, YY, s=SS, c=[col], marker="o", linewidths=0, alpha=1.0, zorder=z)

ax.axhline(0.0, color="0.55", ls="--", lw=1.2, zorder=0)
for t in kticks[1:-1]:
    ax.axvline(t, color="0.55", ls="--", lw=1.2, zorder=0)

ax.set_xlim(0, nk - 1)
ax.set_ylim(args.emin, args.emax)
ax.set_yticks([-1.2, -0.6, 0.0, 0.6, 1.2])

ax.set_ylabel("Energy (eV)", fontsize=34 * scale, fontweight="bold", fontfamily="Times New Roman")
ax.set_xticks(kticks)
ax.set_xticklabels(knames, fontsize=32 * scale, fontweight="bold", fontfamily="Times New Roman")

# [OK] 只保留下/左刻度线, 朝内, 并把刻度标签与轴线距离设为 pad=14
ax.tick_params(axis="both", which="both", direction="in", top=False, right=False)
ax.tick_params(axis="y", pad=10, labelsize=25 * scale, width=3.0, length=8)
ax.tick_params(axis="x", pad=10, width=3.0, length=8)

bold_ticks(ax)

for spn in ax.spines.values():
    spn.set_linewidth(3.0)
ax.grid(False)

proxies = [
    Line2D([0], [0], marker="o", color="none",
           markerfacecolor=colors[i], markeredgcolor="none",
           markersize=args.legend_markersize)
    for i in range(len(orbs))
]
labels = [latex_label(o) for o in orbs]
n_items = len(orbs)
ncol = min(n_items, 5)
nrows = int(np.ceil(n_items / ncol))

legend_y = args.legend_y + 0.03 * (nrows - 1)

leg = ax.legend(
    proxies, labels,
    loc="upper center",
    bbox_to_anchor=(0.5, legend_y),
    ncol=ncol,
    frameon=False,
    handletextpad=0.25,
    columnspacing=0.9,
    fontsize=args.legend_fontsize,
)

for txt in leg.get_texts():
    txt.set_fontweight("normal")
    txt.set_fontfamily("Times New Roman")

```

```

top_margin = 0.88 - 0.06 * (nrows - 1)
top_margin = max(0.70, top_margin)
plt.subplots_adjust(top=top_margin)

plt.tight_layout(pad=0.6)
fig.savefig(args.out, dpi=args.dpi, bbox_inches="tight")

print("[OK] saved:", args.out)

if __name__ == "__main__":
    main()

```

► 代码清单 A.79.8 (python, 522 行)

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import os
import re
import argparse
from typing import List, Tuple, Optional, Dict

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
from matplotlib.colors import to_rgba

def set_global_font_times():
    plt.rcParams["font.family"] = "Times New Roman"
    plt.rcParams["axes.unicode_minus"] = False

    # Make mathtext (legend labels like  $\$d_{xy}\$$ ) use Times New Roman too
    plt.rcParams["mathtext.fontset"] = "custom"
    plt.rcParams["mathtext.rm"] = "Times New Roman"
    plt.rcParams["mathtext.it"] = "Times New Roman:italic"
    plt.rcParams["mathtext.bf"] = "Times New Roman:bold"

def read_poscar_elements_counts(poscar_path: str) -> Tuple[List[str], List[int]]:
    with open(poscar_path, "r", encoding="utf-8", errors="ignore") as f:
        lines = [ln.strip() for ln in f.readlines() if ln.strip()]
    if len(lines) < 8:
        raise RuntimeError(f"POSCAR too short: {poscar_path}")
    elements = lines[5].split()
    counts = [int(x) for x in lines[6].split()]
    return elements, counts

def element_to_atom_indices_lbased(elements: List[str], counts: List[int], pick: Optional[List[str]]) ->
↳ np.ndarray:
    total = sum(counts)
    if not pick:
        return np.arange(1, total + 1, dtype=int)
    pick_set = set(pick)
    out = []
    s = 1
    for el, n in zip(elements, counts):
        if el in pick_set:
            out.extend(range(s, s + n))
        s += n

```

```

    if not out:
        raise RuntimeError(f"Selected elements {pick} not found in POSCAR elements={elements}")
    return np.array(out, dtype=int)

def read_fermi_from_outcar(outcar_path: str) -> Optional[float]:
    if not os.path.exists(outcar_path):
        return None
    pat = re.compile(r"E-fermi\s*:\s*([-\d\.]+)")
    fermi = None
    with open(outcar_path, "r", encoding="utf-8", errors="ignore") as f:
        for ln in f:
            m = pat.search(ln)
            if m:
                fermi = float(m.group(1))
    return fermi

# ---- plain non-italic HS labels (Times) ----
def plain_label(lbl: str) -> str:
    s = lbl.strip()
    if s.upper() == "GAMMA":
        return "Γ"
    if s in ("K'", "K' ", "Kp", "KP"):
        return "K'"
    return s

def parse_vaskit_kpoints_nodes(kpoints_path: str) -> List[str]:
    coord_pat = re.compile(
        r"^\s*([-\d\.]+)\s+([-\d\.]+)\s+([-\d\.]+)\s+([A-Za-z0-9\-\+\']+)\s*$"
    )
    labels_all = []
    with open(kpoints_path, "r", encoding="utf-8", errors="ignore") as f:
        for ln in f:
            m = coord_pat.match(ln.strip())
            if m:
                labels_all.append(m.group(4).strip())
    if len(labels_all) < 2:
        raise RuntimeError("Failed to parse high-symmetry labels from KPOINTS.")

    compressed = []
    for lab in labels_all:
        if not compressed or lab != compressed[-1]:
            compressed.append(lab)

    cleaned = []
    i = 0
    while i < len(compressed):
        cleaned.append(compressed[i])
        if i + 2 < len(compressed) and compressed[i + 1] == compressed[i + 2]:
            i += 2
        else:
            i += 1
    return cleaned

def make_kticks_knames(nodes: List[str], nk: int) -> Tuple[List[str], List[int]]:
    nseg = len(nodes) - 1
    kticks = [int(round(i * (nk - 1) / nseg)) for i in range(len(nodes))]
    kticks[0] = 0
    kticks[-1] = nk - 1
    knames = [plain_label(x) for x in nodes]
    return knames, kticks

ORBITAL_INDEX = {
    "s": 0,
    "py": 1, "pz": 2, "px": 3,

```

```

    "dxy": 4, "dyz": 5, "dz2": 6, "dxz": 7, "x2-y2": 8,
}

def normalize_orb(s: str) -> str:
    t = s.strip().lower().replace(" ", "").replace("^", "").replace("_", "")
    if t in ("dx2-y2", "dx2y2", "x2y2", "x2-y2"):
        return "x2-y2"
    return t

def expand_orbitals(orb_str: str) -> List[str]:
    raw = [x for x in re.split(r"[,\+;\s]+", orb_str) if x.strip()]
    out = []
    for tok in raw:
        k = normalize_orb(tok)
        if k == "s":
            out.append("s")
        elif k == "p":
            out += ["px", "py", "pz"]
        elif k == "d":
            out += ["dxy", "dyz", "dz2", "dxz", "x2-y2"]
        else:
            if k not in ORBITAL_INDEX:
                raise RuntimeError(
                    f"Illegal orbital token '{tok}'. Only s/p/d are allowed."
                )
            out.append(k)
    seen = set()
    uniq = []
    for x in out:
        if x not in seen:
            uniq.append(x)
            seen.add(x)
    return uniq

def latex_label(orb: str) -> str:
    # 图例仍用 mathtext (显示 d_{xy}), 但字体我们强制 Times New Roman
    if orb == "s":
        return r"$s$"
    if orb in ("px", "py", "pz"):
        return rf"$p_{{{{orb[-1]}}}}$"
    if orb == "dz2":
        return r"$d_{{z^2}}$"
    if orb == "dxy":
        return r"$d_{{xy}}$"
    if orb == "dyz":
        return r"$d_{{yz}}$"
    if orb == "dxz":
        return r"$d_{{xz}}$"
    if orb == "x2-y2":
        return r"$d_{{x^2-y^2}}$"
    return rf"${orb}$"

def parse_procar_lm_decomposed_autospin(procar_path: str, force_nspin: Optional[int] = None):
    """
    Robust PROCAR parser for "lm decomposed" format.

    Handles:
    - Fortran D/d exponents in numbers (converted to E/e).
    - k-point coordinates that may not be cleanly whitespace-separated.
    - Extra blocks after the (ion ... tot) table (e.g., spin projections in LSORBIT/noncollinear),
      which are safely ignored.

    Returns:
    kpoints: (nk, 3)
    energies: (nk, nb, nspin)

```

```

    weights: (nk, nb, nion, norb, nspin), norb=9 (s,py,pz,px,dxy,dyz,dz2,dxz,x2-y2)
"""
with open(procar_path, "r", encoding="utf-8", errors="ignore") as f:
    lines = f.readlines()

header = "".join(lines[:200])
m = re.search(
    r"#\s*of\s*k-points:\s*([0-9]+)\s*\#\s*of\s*bands:\s*([0-9]+)\s*\#\s*of\s*ions:\s*([0-9]+)",
    header, re.IGNORECASE
)
if not m:
    raise RuntimeError("Failed to parse PROCAR header (# of k-points / bands / ions).")

nk = int(m.group(1)); nb = int(m.group(2)); nion = int(m.group(3))
norb = 9

header_text = header.lower()

# Determine nspin
kpt1_pat = re.compile(r"^\s*k-point\s+l\s*:", re.IGNORECASE)
kpt1_count = sum(1 for ln in lines if kpt1_pat.match(ln))

if force_nspin is not None:
    nspin = int(force_nspin)
else:
    if "spin component" in header_text:
        nspin = 2
    else:
        nspin = 2 if kpt1_count >= 2 else 1

kpoints = np.zeros((nk, 3), dtype=float)
energies = np.full((nk, nb, nspin), np.nan, dtype=float)
weights = np.zeros((nk, nb, nion, norb, nspin), dtype=float)

spin_pat = re.compile(r"spin\s+component\s+([12])", re.IGNORECASE)
ion_header_pat = re.compile(
    r"^\s*ion\s+s\s+py\s+pz\s+px\s+dxy\s+dyz\s+dz2\s+dxz\s+x2-y2\s+tot",
    re.IGNORECASE
)

# Generic number finder: supports D/d and E/e exponents and catches adjacent signs
num_pat = re.compile(r"[+-]?[d+](?:\.\d*)?(?:[EeDd][+-]?[d+]?")

def fnum(s: str) -> float:
    return float(s.replace("D", "E").replace("d", "e"))

cur_spin = 0
cur_k = -1
cur_b = -1

# For collinear ISPIN=2 without explicit "spin component" blocks
seen_k_cycle = 0
last_k_index = None

i = 0
while i < len(lines):
    ln = lines[i]

    ms = spin_pat.search(ln)
    if ms:
        cur_spin = int(ms.group(1)) - 1
        i += 1
        continue

    # k-point line
    if ln.lstrip().lower().startswith("k-point"):
        # Extract k-point index (first integer after 'k-point')
        mki = re.search(r"^\s*k-point\s+(\d+)", ln, re.IGNORECASE)
        if mki:

```

```

        k_index_lb = int(mki.group(1))

        if force_nspin is None and ("spin component" not in header_text):
            if last_k_index is not None and k_index_lb == 1 and last_k_index != 1:
                seen_k_cycle += 1
            cur_spin = min(seen_k_cycle, nspin - 1)
            last_k_index = k_index_lb

        cur_k = k_index_lb - 1

        # Prefer parsing coords after ':' to avoid grabbing the k-point index
        after = ln.split(":", 1)[1] if ":" in ln else ln
        nums = num_pat.findall(after)
        if len(nums) >= 3 and cur_spin == 0:
            kpoints[cur_k, :] = [fnum(nums[0]), fnum(nums[1]), fnum(nums[2])]
        i += 1
        continue

# band energy line
if ln.lstrip().lower().startswith("band") and "energy" in ln.lower():
    mb = re.search(r"^\s*band\s+(\d+)", ln, re.IGNORECASE)
    if mb:
        cur_b = int(mb.group(1)) - 1
        after = re.split(r"energy", ln, flags=re.IGNORECASE, maxsplit=1)[1]
        nums = num_pat.findall(after)
        if nums:
            energies[cur_k, cur_b, cur_spin] = fnum(nums[0])
        i += 1
        continue

# orbital weight table header
if ion_header_pat.match(ln):
    # Read exactly nion lines after header. Ignore the 'tot' line and any subsequent projection blocks.
    for j in range(nion):
        row = lines[i + 1 + j].split()
        if len(row) < 11:
            continue
        ion_idx_lb = int(row[0]) - 1
        vals = [float(x) for x in row[1:10]]
        weights[cur_k, cur_b, ion_idx_lb, :, cur_spin] = vals
        i += 1 + nion
    continue

i += 1

nan_pos = np.argwhere(np.isnan(energies))
if nan_pos.size > 0:
    ex = nan_pos[0].tolist()
    raise RuntimeError(
        "PROCAR parse incomplete: energies has NaNs (some 'band ... energy ...' lines were not parsed). "
        f"Examples [k_idx0, b_idx0, spin]: {ex}. "
        "If this persists, check that PROCAR is complete (not truncated) and belongs to this KPOINTS."
    )

return kpoints, energies, weights, nk, nb, nion, nspin, norb

def parse_fixed_colors(s: Optional[str]) -> Dict[str, str]:
    if not s:
        return {}
    out = {}
    for part in s.split(","):
        part = part.strip()
        if not part:
            continue
        if "=" not in part:
            raise RuntimeError(f"Bad --fixed_colors token '{part}', expect key=#RRGGBB")
        k, v = part.split("=", 1)
        k = normalize_orb(k)
        v = v.strip()

```

```

        to_rgba(v)
        out[k] = v
    return out

def parse_colors_list(s: Optional[str]) -> Optional[List[str]]:
    if not s:
        return None
    cols = [c.strip() for c in s.split(",") if c.strip()]
    if not cols:
        return None
    for c in cols:
        to_rgba(c)
    return cols

def auto_color_list(n: int) -> List[str]:
    cmap = plt.get_cmap("tab10") if n <= 10 else plt.get_cmap("tab20")
    return [cmap(i % cmap.N) for i in range(n)]

def assign_colors(orbs: List[str], fixed: Dict[str, str], colors_list: Optional[List[str]]) -> List:
    auto = auto_color_list(len(orbs))
    out = []
    for i, o in enumerate(orbs):
        if colors_list:
            out.append(fixed.get(o, colors_list[i % len(colors_list)]))
        else:
            out.append(fixed.get(o, auto[i]))
    return out

def bold_ticks(ax):
    for t in ax.get_xticklabels():
        t.set_fontweight("bold")
        t.set_fontfamily("Times New Roman")
        t.set_fontstyle("normal")
    for t in ax.get_yticklabels():
        t.set_fontweight("bold")
        t.set_fontfamily("Times New Roman")
        t.set_fontstyle("normal")

def main():
    set_global_font_times()

    ap = argparse.ArgumentParser("Fatband (s/p/d only).")
    ap.add_argument("--dirname", default=".")
    ap.add_argument("--elements", default=None)
    ap.add_argument("--orbitals", required=True)
    ap.add_argument("--fixed_colors", default=None)
    ap.add_argument("--colors", default=None)

    ap.add_argument("--fermi", type=float, default=None)
    ap.add_argument("--emin", type=float, default=-1.2)
    ap.add_argument("--emax", type=float, default=1.2)
    ap.add_argument("--out", default="fatband.png")
    ap.add_argument("--dpi", type=int, default=400)

    ap.add_argument("--plain_color", default="0.82")
    ap.add_argument("--plain_lw", type=float, default=1.1)
    ap.add_argument("--bubble_scale", type=float, default=220.0)
    ap.add_argument("--bubble_pow", type=float, default=1.0)
    ap.add_argument("--bubble_clip", type=float, default=1.0)
    ap.add_argument("--width_scale", type=float, default=1.0)

    ap.add_argument("--force_nspin", type=int, default=None)

    ap.add_argument("--legend_y", type=float, default=1.14, help="legend bbox_to_anchor y (closer to top)")

```

```

ap.add_argument("--legend_fontsize", type=float, default=20)
ap.add_argument("--legend_markersize", type=float, default=10.0)

args = ap.parse_args()
d = os.path.abspath(args.dirname)

procar = os.path.join(d, "PROCAR")
poscar = os.path.join(d, "POSCAR")
kpoints = os.path.join(d, "KPOINTS")
outcar = os.path.join(d, "OUTCAR")
for p in [procar, poscar, kpoints]:
    if not os.path.exists(p):
        raise FileNotFoundError(f"Missing file: {p}")

orbs = expand_orbitals(args.orbitals)
if not orbs:
    raise RuntimeError("No valid orbitals parsed from --orbitals")

elements, counts = read_poscar_elements_counts(poscar)
pick = None
if args.elements:
    pick = [x.strip() for x in args.elements.split(",") if x.strip()]
sel_ions_0b = element_to_atom_indices_lbased(elements, counts, pick) - 1

fermi = args.fermi
if fermi is None:
    f0 = read_fermi_from_outcar(outcar)
    fermi = f0 if f0 is not None else 0.0

_, E, W, nk, nb, _, nspin, _ = parse_procar_lm_decomposed_autospin(procar, force_nspin=args.force_nspin)

nodes = parse_vaskit_kpoints_nodes(kpoints)
knames, kticks = make_kticks_knames(nodes, nk)

fixed = parse_fixed_colors(args.fixed_colors)
colors_list = parse_colors_list(args.colors)
colors = assign_colors(orbs, fixed, colors_list)

def bubble_size(w):
    ww = np.clip(w, 0.0, args.bubble_clip)
    ww = np.power(ww, args.bubble_pow)
    return args.width_scale * args.bubble_scale * ww

x = np.arange(nk, dtype=float)
XX = np.repeat(x[:, None], nb, axis=1).ravel()

sizes_list = []
for o in orbs:
    oi = ORBITAL_INDEX[o]
    w = W[:, :, sel_ions_0b, oi, :].sum(axis=2)
    sizes_list.append(bubble_size(w))

scale = 0.7

fig, ax = plt.subplots(figsize=(6.0, 6.0), dpi=140)

for sp in range(nspin):
    ax.plot(x, (E[:, :, sp] - fermi), color=args.plain_color, lw=args.plain_lw, zorder=1)

for i, o in enumerate(orbs):
    S = sizes_list[i]
    col = colors[i]
    z = 3 + i
    for sp in range(nspin):
        YY = (E[:, :, sp] - fermi).ravel()
        SS = S[:, :, sp].ravel()
        ax.scatter(XX, YY, s=SS, c=[col], marker="o", linewidths=0, alpha=1.0, zorder=z)

ax.axhline(0.0, color="0.55", ls="--", lw=1.2, zorder=0)

```

```

for t in kticks[1:-1]:
    ax.axvline(t, color="0.55", ls="--", lw=1.2, zorder=0)

ax.set_xlim(0, nk - 1)
ax.set_ylim(args.emin, args.emax)
ax.set_yticks([-1.2, -0.6, 0.0, 0.6, 1.2])

ax.set_ylabel("Energy (eV)", fontsize=34 * scale, fontweight="bold", fontfamily="Times New Roman")
ax.set_xticks(kticks)
ax.set_xticklabels(knames, fontsize=32 * scale, fontweight="bold", fontfamily="Times New Roman")

# [OK] 只保留下/左刻度线, 朝内, 并把刻度标签与轴线距离设为 pad=14
ax.tick_params(axis="both", which="both", direction="in", top=False, right=False)
ax.tick_params(axis="y", pad=10, labelsize=25 * scale, width=3.0, length=8)
ax.tick_params(axis="x", pad=10, width=3.0, length=8)

bold_ticks(ax)

for spn in ax.spines.values():
    spn.set_linewidth(3.0)
ax.grid(False)

proxies = [
    Line2D([0], [0], marker="o", color="none",
           markerfacecolor=colors[i], markeredgcolor="none",
           markersize=args.legend_markersize)
    for i in range(len(orbs))
]
labels = [latex_label(o) for o in orbs]
n_items = len(orbs)
ncol = min(n_items, 5)
nrows = int(np.ceil(n_items / ncol))

legend_y = args.legend_y + 0.03 * (nrows - 1)

leg = ax.legend(
    proxies, labels,
    loc="upper center",
    bbox_to_anchor=(0.5, legend_y),
    ncol=ncol,
    frameon=False,
    handletextpad=0.25,
    columnspacing=0.9,
    fontsize=args.legend_fontsize,
)

for txt in leg.get_texts():
    txt.set_fontweight("normal")
    txt.set_fontfamily("Times New Roman")

top_margin = 0.88 - 0.06 * (nrows - 1)
top_margin = max(0.70, top_margin)
plt.subplots_adjust(top=top_margin)

plt.tight_layout(pad=0.6)
fig.savefig(args.out, dpi=args.dpi, bbox_inches="tight")

print("[OK] saved:", args.out)

if __name__ == "__main__":
    main()

```

绘制 3DMAE 的流程 (原创脚本与流程)

► 代码清单 A.80.1 (python, 3 行)

```
2                                ! 1 for preprocess, 2 for postprocess
0 360 24                        ! 0<= azimuth angle phi in spherical coordinate system <=360 with a increment of 30
↪ degrees, 12 points in total
0 180 13                        ! 0<= polar angle theta in spherical coordinate system <=180 with a increment of 15
↪ degrees, 13 points in total
```

► 代码清单 A.80.2 (python, 37 行)

```
#General
PREC = ACCURATE
ISTART= 0
ICHARG= 1
ENCUT = 500
EDIFF = 1E-6
EDIFFG = -0.01
LREAL = .F.
NPAR = 4
NSW= 0
IBRION = -1
ISIF = 2
NELM=300
ISMEAR = 0
SIGMA = 0.05
LCHARG = .F.
LWAVE = .F.
POTIM = 0.1

#Magnetic properties

ISPIN=2

MAGMOM = 0 0 0 0 0 4 24*0.0

NELMIN = 6
LORBIT = 11
ISYM = 0
LSORBIT = .True.
LMAXMIX = 4

LDAU = .TRUE.
LDAUTYPE = 2
LDAUL = 2 2 -1 -1
LDAUU = 0.5 2.0 0.0 0.0
LDAUJ = 0.0 0.0 0.0 0.0
LDAUPRINT = 2
```

► 代码清单 A.80.3 (python, 32 行)

```
#!/bin/bash
#PBS -l nodes=1:ppn=64
```

```
#PBS -l walltime=640:01:00
#PBS -j oe
#PBS -q sugon
#PBS -N mae

cd $PBS_0_WORKDIR

echo job id is $PBS_JOBID | tee pbslog
echo run nodes is following: | tee -a pbslog
cat $PBS_NODEFILE | tee -a pbslog

echo begin time is `date` | tee -a pbslog
id=`echo $PBS_JOBID|awk -F. '{print $1}'`
NP=`cat $PBS_NODEFILE|wc -l`

source /public/software/profile.d/compiler_intel-compiler-2021.3.0.sh
source /public/software/profile.d/mpi_intelmpi-2021.3.0.sh
RUN_1=/public/home/cssong/app/vasp/intel_2021.3.0/5.4.4/vasp_std
RUN_2=/public/home/cssong/app/vasp/intel_2021.3.0/5.4.4/vasp_ncl
cat $PBS_NODEFILE | uniq -c | awk '{ printf("%s:%s\n", $2, $1); }' >> $PBS_JOBID-$PBS_JOBCOOKIE.hosts

shopt -s extglob
for i in +([0-9])?(.+([0-9]))_+([0-9])?(.+([0-9]))
do

    cd $i
    mpirun -np $NP ${RUN_2}>&log

cd ..
done
```

```
import numpy as np
from scipy.interpolate import griddata

# ===== 参数设置 =====
input_file = "MAE.dat"      # 输入文件名 (确保文件在当前目录)
output_prefix = "MAE_3D"    # 输出文件前缀
nphi = 500                  # 极角网格点数 (0 到  $\pi$ )
ntheta = 500                # 方位角网格点数 (0 到  $2\pi$ )
# =====

# ----- 1. 读取数据 -----
print(" 正在读取数据...")
data = np.loadtxt(input_file, skiprows=1) # 跳过标题行
phi_deg = data[:, 0]      # 第一列: 极角  $\phi$  (单位: 度)
theta_deg = data[:, 1]    # 第二列: 方位角  $\theta$  (单位: 度)
R = data[:, 2]            # 第三列: R 值

# ----- 2. 角度转换 -----
phi = np.deg2rad(phi_deg) # 转换为弧度 [0,  $\pi$ ]
theta = np.deg2rad(theta_deg) # 转换为弧度 [0,  $2\pi$ ]

# ----- 3. 生成新网格 -----
print(" 生成插值网格...")
phinew = np.linspace(0, np.pi, nphi)      # 极角范围 [0,  $\pi$ ]
thetanew = np.linspace(0, 2*np.pi, ntheta) # 方位角范围 [0,  $2\pi$ ]
xx, yy = np.meshgrid(phinew, thetanew, indexing='ij') # 创建网格

# ----- 4. 插值计算 -----
print(" 正在进行立方插值...")
points = np.column_stack((phi, theta))      # 原始数据点坐标
```

```

xi = np.column_stack((xx.ravel(), yy.ravel())) # 新网格点坐标

# 使用立方插值计算 Rnew, 并处理可能的 NaN 值
Rnew = griddata(points, R, xi, method='cubic')
Rnew = Rnew.reshape(xx.shape) # 转换为矩阵形状
Rnew = np.nan_to_num(Rnew, nan=np.nanmean(Rnew)) # 用均值填充缺失值

# ----- 5. 转换为笛卡尔坐标 -----
print(" 坐标转换中...")
x = Rnew * np.sin(xx) * np.cos(yy)
y = Rnew * np.sin(xx) * np.sin(yy)
z = Rnew * np.cos(xx)

# 生成镜像数据 (关于 xy 平面的对称)
x_mirror = x
y_mirror = y
z_mirror = -z

# 合并原数据和镜像数据 (沿极角  $\phi$  方向堆叠)
x_combined = np.vstack((x, x_mirror))
y_combined = np.vstack((y, y_mirror))
z_combined = np.vstack((z, z_mirror))
R_combined = np.vstack((Rnew, Rnew))

# ----- 6. 保存为 Origin 矩阵文件 -----
print(" 保存数据文件...")
np.savetxt(f"{output_prefix}_X.txt", x_combined, delimiter='\t', fmt='%.6f')
np.savetxt(f"{output_prefix}_Y.txt", y_combined, delimiter='\t', fmt='%.6f')
np.savetxt(f"{output_prefix}_Z.txt", z_combined, delimiter='\t', fmt='%.6f')
np.savetxt(f"{output_prefix}_R.txt", R_combined, delimiter='\t', fmt='%.6f')

print(" 处理完成! 请在当前目录查看生成的文件。")

```

► 代码清单 A.80.5 (python, 84 行)

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import griddata
from matplotlib.colors import Normalize, LinearSegmentedColormap

# ===== 数据加载 =====
data = np.loadtxt('MAE.dat', skiprows=1)
phi = data[:, 0] * np.pi / 180
theta = data[:, 1] * np.pi / 180
R = data[:, 2]

# ===== 网格生成 =====
nphi, ntheta = 500, 500
phinew = np.linspace(0, np.pi, nphi)
thetanew = np.linspace(0, 2*np.pi, ntheta)
xx, yy = np.meshgrid(phinew, thetanew)

# ===== 镜像对称插值 =====
phi_mirror = np.pi - xx
theta_mirror = (yy + np.pi) % (2*np.pi)
R_upper = griddata((phi, theta), R, (xx, yy), method='cubic')
R_lower = griddata((phi, theta), R, (phi_mirror, theta_mirror), method='cubic')
Rnew = np.where(xx > np.pi/2, R_lower, R_upper)

# ===== 坐标转换 (放大 4 倍) =====
scale_factor = 4.0
x = scale_factor * Rnew * np.sin(xx) * np.cos(yy)
y = scale_factor * Rnew * np.sin(xx) * np.sin(yy)

```

```

z = scale_factor * Rnew * np.cos(xx)

# ===== 柔和渐变色 (橙-黄-绿) =====
cmap_soft = LinearSegmentedColormap.from_list(
    "OrangeYellowGreen", ["#FFA500", "#FFFF66", "#008000"]
)

# ===== 可视化 =====
fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')

# 统一颜色映射
norm = Normalize(vmin=np.min(Rnew), vmax=np.max(Rnew))
colors = cmap_soft(norm(Rnew))

surf = ax.plot_surface(
    x, y, z,
    facecolors=colors,
    rcount=300, ccount=300,
    antialiased=True
)

# ===== 视角和比例 =====
ax.view_init(elev=30, azim=-50)

# 自动保持 x,y,z 方向比例一致
ax.set_box_aspect([np.ptp(x), np.ptp(y), np.ptp(z)])

# ===== 设置坐标范围 (比图形大 1.5 倍) =====
margin = 1.5
ax.set_xlim([np.min(x) * margin, np.max(x) * margin])
ax.set_ylim([np.min(y) * margin, np.max(y) * margin])
ax.set_zlim([np.min(z) * margin, np.max(z) * margin])

# ===== 坐标轴样式: 只保留网格, 去掉刻度数字和标签 =====
ax.tick_params(labelbottom=False, labelleft=False, labelright=False, labeltop=False)
ax.set_xlabel("")
ax.set_ylabel("")
ax.set_zlabel("")
ax.grid(True)

# 网格线颜色调浅
ax.xaxis._axinfo['grid']['color'] = (0.8, 0.8, 0.8, 1)
ax.yaxis._axinfo['grid']['color'] = (0.8, 0.8, 0.8, 1)
ax.zaxis._axinfo['grid']['color'] = (0.8, 0.8, 0.8, 1)

# ===== 颜色条 (柔和渐变) =====
mappable = plt.cm.ScalarMappable(norm=norm, cmap=cmap_soft)
mappable.set_array([])
cbar = fig.colorbar(mappable, shrink=0.5, aspect=10)
cbar.set_label("") # 去掉标签

# ===== 保存 & 显示 =====
plt.tight_layout()
plt.savefig("MAE_3D_orange_yellow_green_scaled4_margin15.png", dpi=600, bbox_inches="tight", facecolor="white")
plt.show()

```

(原创脚本)vaspberry 计算贝里曲率与后处理

► 代码清单 A.82.2 (shell, 1 行)

```
vaspberry -kx 12 -ky 12 -ii 1 -if 18
```

► 代码清单 A.82.3 (shell, 1 行)

```
vaspberry -kx 12 -ky 12 -ii 18 -if 18
```

► 代码清单 A.82.4 (shell, 1 行)

```
vaspberry -kx 12 -ky 12 -ii 1 -if 18 -s 1
```

► 代码清单 A.82.5 (python, 55 行)

```
import numpy as np
import matplotlib.pyplot as plt

# =====
# Parameters
# =====
nkx, nky = 21, 21
nkx, nky = nkx * 3, nky * 3

# =====
# Load data
# =====
data = np.loadtxt("BERRYCURV.dat", skiprows=20)

# Extract reciprocal coordinates + Berry curvature
kx = data[:, 4] # 倒数第五列
ky = data[:, 5] # 倒数第四列
berry = data[:, 3] # 贝里曲率

# Read B1, B2 from header (line 9 and 10, 0-based index 8,9)
with open("BERRYCURV.dat", "r") as f:
    lines = f.readlines()

B1 = np.array([float(x) for x in lines[8].split()[-3:-1]])
B2 = np.array([float(x) for x in lines[9].split()[-3:-1]])

# Convert to Cartesian-like coords used in your original script
kx_n = B1[0] * kx + B2[0] * ky
ky_n = B1[1] * kx + B2[1] * ky

# Reshape to grid
kx_grid = kx_n.reshape(nkx, nky)
ky_grid = ky_n.reshape(nkx, nky)
berry_grid = berry.reshape(nkx, nky)

# =====
# Plot: ONLY contourf + colorbar
```

```

# =====
fig, ax = plt.subplots(figsize=(10, 8))

# 颜色: 反转红蓝渐变 (如需反转 RdBu, 可使用 RdBu_r)
level = 100
cf = ax.contourf(kx_grid, ky_grid, berry_grid, level, cmap="RdBu_r")

# 去掉主图所有边框/坐标轴/刻度/标题
ax.set_axis_off()
ax.set_aspect("equal", adjustable="box")

# 只保留 colour bar (需要的话可删掉 label 这行)
cbar = fig.colorbar(cf, ax=ax, fraction=0.046, pad=0.04)
cbar.set_label(r"Berry Curvature  $\langle A^2 \rangle$ ", fontsize=12)

# 保存图片 (tight 以尽量减少白边, 同时包含 colorbar)
fig.savefig("Berry.png", dpi=600, bbox_inches="tight", pad_inches=0)
plt.close(fig)

```

► 代码清单 A.82.6 (python, 57 行)

```

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap

# =====
# Parameters
# =====
nkx, nky = 21, 21
nkx, nky = nkx * 3, nky * 3

# =====
# Load data
# =====
data = np.loadtxt('BERRYCURV.dat', skiprows=20)

# colormap: blue-white-red
colors = ['blue', 'white', 'red']
cmap = LinearSegmentedColormap.from_list('custom_red_white_blue', colors, N=256)

# Extract reciprocal coordinates + Berry curvature
kx = data[:, 4] # 倒数第五列
ky = data[:, 5] # 倒数第四列
berry = data[:, 3] # 贝里曲率

# Read B1, B2 from header (line 9 and 10, 0-based index 8,9)
with open('BERRYCURV.dat', 'r') as f:
    lines = f.readlines()
B1 = np.array([float(x) for x in lines[8].split()[-3:-1]])
B2 = np.array([float(x) for x in lines[9].split()[-3:-1]])

# Convert to Cartesian-like coords used in your original script
kx_n = B1[0] * kx + B2[0] * ky
ky_n = B1[1] * kx + B2[1] * ky

# Reshape to grid
kx_grid = kx_n.reshape(nkx, nky)
ky_grid = ky_n.reshape(nkx, nky)
berry_grid = berry.reshape(nkx, nky)

# =====
# Plot: ONLY contourf + colorbar
# =====

```

```

fig, ax = plt.subplots(figsize=(10, 8))

contour = ax.contourf(kx_grid, ky_grid, berry_grid, levels=100, cmap=cmap)

# Remove everything from main axes: frame/spines/ticks/labels
ax.set_axis_off()
ax.set_aspect('equal', adjustable='box')

# Keep colorbar (with label; if you also want to remove label, set label="" below)
cbar = fig.colorbar(contour, ax=ax, fraction=0.046, pad=0.04)
cbar.set_label(r'Berry Curvature  $\mathcal{A}^2$ ', fontsize=12)

# Save: tight to avoid extra margins, but keep colorbar included
fig.savefig("Berry.png", dpi=600, bbox_inches='tight', pad_inches=0)
plt.close(fig)

```

► 代码清单 A.82.7 (shell, 1 行)

```

vaspberry -kx 21 -ky 21 -ii 1 -s 2 -if 44

```

► 代码清单 A.82.8 (python, 24 行)

```

from pythtb import W90
from pythtb import TBModel, Lattice, WFArray, Mesh
import matplotlib.pyplot as plt
import numpy as np

nso = W90(".", "wannier90")

# hard coded fermi level in eV
fermi_ev = -1.4862

# all pair distances between the orbitals
print("Shells:\n", nso.shells())

# plot hopping terms as a function of distance on a log scale
(dist, ham) = nso.dist_hop()
fig, ax = plt.subplots()
ax.scatter(dist, np.abs(ham))
ax.hlines(
    5e-6, xmin=0, xmax=max(dist), colors="r", linestyle="dashed", label="Cutoff = 1meV"
)
ax.legend()
ax.set_xlabel("Distance (A)")
ax.set_ylabel(r" $\mathcal{H}$  (eV)")
ax.set_yscale("log")

```

► 代码清单 A.82.9 (python, 12 行)

```

# get tb model in which some small terms are ignored
my_model = nso.model(
    zero_energy=fermi_ev,

```

```

        min_hopping_norm=5e-6,
    )

    (w90_kpt, w90_evals, w90_k_dist, w90_k_nodes, w90_k_labels) = nso.bands_w90(
        return_k_dist=True, return_k_nodes=True
    )

    print("k-point labels:", w90_k_labels)
    print("k-point nodes (fractional):\n", w90_k_nodes)

```

► 代码清单 A.82.10 (python, 19 行)

```

k_vec, k_dist, k_node_dist = my_model.k_path(w90_k_nodes, nk=500, report=False)

int_evals = my_model.solve_ham(w90_kpt)

fig, ax = plt.subplots(figsize=(10, 6))

ax.plot(w90_k_dist, w90_evals[:, 0] - fermi_ev, "k-", zorder=0, label="Wannier90")
ax.plot(w90_k_dist, w90_evals[:, 1:] - fermi_ev, "k-", zorder=0)
ax.plot(w90_k_dist, int_evals[:, 0], "r--", zorder=1, label="TBModel")
ax.plot(w90_k_dist, int_evals[:, 1:], "r--", zorder=1)

# set x-ticks at k-point nodes
ax.set_xticks(k_node_dist)
for n in range(len(w90_k_nodes)):
    ax.axvline(x=k_node_dist[n], linewidth=0.5, color="k", zorder=1)
ax.set_xticklabels(w90_k_labels, size=12)
ax.set_xlim(k_node_dist[0], k_node_dist[-1])
ax.set_ylabel("Band energy (eV)")
plt.show()

```

► 代码清单 A.82.11 (python, 8 行)

```

my_model.plot_bands(
    w90_k_nodes,
    k_node_labels=w90_k_labels,
    nk=501,
    scat_size=2,
    proj_orb_idx=[1],
    cmap="plasma",
)

```

► 代码清单 A.82.12 (python, 228 行)

```

import numpy as np
import matplotlib.pyplot as plt

# =====
# User settings
# =====
HR_FILE = "wannier90_hr.dat"

```

```

# 手动设置费米能级 (单位 eV, 必须与 hr/TB 的能量参考一致)
EF = -1.4862

# 0K 阶跃占据: KBT 取很小; 若想平滑 (模拟有限温度) 可取 0.01 eV 等
KBT = 1e-6 # eV

# "first three conduction bands": 从 CB_START_1BASED 开始取 3 条
# 体系 VBM=43 时通常 CBM=44
CB_START_1BASED = 44
N_CB = 3

# 数值参数
N_PER_SEG = 3
KZ_FRAC = 0.0
DK_CART = 1e-4
OUT_PREFIX = "hr_kubo_EFmanual_VBsum_onecurve"

# reciprocal vectors from your wannier90.wout (Ang^-1), x,y components
b1 = np.array([0.955608, 0.551736], float) # 1/Å
b2 = np.array([0.000011, 1.103456], float) # 1/Å

# =====
# Read wannier90_hr.dat
# =====
def read_hr(fname):
    with open(fname, "r") as f:
        lines = f.readlines()

    nwan = int(lines[1].split()[0])
    nrpt = int(lines[2].split()[0])

    deg = []
    idx = 3
    while len(deg) < nrpt:
        deg += [int(x) for x in lines[idx].split()]
        idx += 1
    deg = np.array(deg[:nrpt], dtype=int)

    data = []
    for L in lines[idx:]:
        sp = L.split()
        if len(sp) < 7:
            continue
        R1,R2,R3 = map(int, sp[0:3])
        i = int(sp[3]) - 1
        j = int(sp[4]) - 1
        re = float(sp[5]); im = float(sp[6])
        data.append((R1,R2,R3,i,j,re+1j*im))
    return nwan, nrpt, deg, data

nwan, nrpt, deg, data = read_hr(HR_FILE)

# CB band indices (0-based)
cb0 = CB_START_1BASED - 1
CBANDS = list(range(cb0, cb0 + N_CB))
if cb0 < 0 or cb0 + N_CB > nwan:
    raise ValueError(f"CB_START_1BASED/N_CB out of range. Valid bands: 1..{nwan}")

# unique R points in order
R_list = [(int(r[0]), int(r[1]), int(r[2])) for r in data]
seen=set(); Runiq=[]
for R in R_list:
    if R not in seen:
        Runiq.append(R); seen.add(R)
Runiq = Runiq[:nrpt]

HR = {R: np.zeros((nwan,nwan), complex) for R in Runiq}
for (R1,R2,R3,i,j,amp) in data:

```

```

R=(int(R1),int(R2),int(R3))
HR[R][int(i),int(j)] = complex(amp)

for idxR,R in enumerate(Runiq):
    HR[R] = HR[R] / deg[idxR]

# =====
# reduced <-> cart mapping
# =====
M = np.stack([b1,b2], axis=1)      # 2x2
Minv = np.linalg.inv(M)

def red_to_cart2(kred2): return M @ np.asarray(kred2, float)
def cart2_to_red(kcart2): return Minv @ np.asarray(kcart2, float)

# =====
# H(k) from hr (Wannier90)
#  $H(k) = \sum_R H(R)/\deg(R) * \exp(i 2\pi k \cdot R)$ , k in reduced coords
# =====
def H_of_kred(kred):
    kred = np.asarray(kred, float)
    if kred.size < 3:
        kred = np.array([kred[0], kred[1], 0.0], float)
    kred = kred.copy()
    kred[2] = KZ_FRAC
    k1,k2,k3 = kred
    Hk = np.zeros((nwan,nwan), complex)
    for (R1,R2,R3), HRmat in HR.items():
        phase = np.exp(2j*np.pi*(k1*R1 + k2*R2 + k3*R3))
        Hk += HRmat * phase
    return 0.5*(Hk + Hk.conj().T)

# =====
# Fermi-Dirac (for occupied sum)
# =====
def fermi_dirac(E):
    x = (E - EF) / max(KBT, 1e-12)
    x = np.clip(x, -200, 200)
    return 1.0 / (1.0 + np.exp(x))

# =====
# One-k evaluation: eigen + velocities + all-band  $\Omega_{mn}$ 
#  $\Omega_{mn} = -2 \text{Im} \sum_{m \neq n} \langle n | v_x | m \rangle \langle m | v_y | n \rangle / (E_m - E_n)^2$ 
# =====
def omega_all_bands_at_k(kred):
    kred = np.asarray(kred, float)
    if kred.size < 3:
        kred = np.array([kred[0], kred[1], 0.0], float)
    kred = kred.copy(); kred[2] = KZ_FRAC

    H0 = H_of_kred(kred)
    E, U = np.linalg.eigh(H0)

    # velocities by Cartesian finite diff
    kcart2 = red_to_cart2(kred[:2])

    kx_p = kcart2 + np.array([DK_CART, 0.0])
    kx_m = kcart2 - np.array([DK_CART, 0.0])
    kred_p = kred.copy(); kred_p[:2] = cart2_to_red(kx_p)
    kred_m = kred.copy(); kred_m[:2] = cart2_to_red(kx_m)
    vx = (H_of_kred(kred_p) - H_of_kred(kred_m)) / (2.0 * DK_CART)

    ky_p = kcart2 + np.array([0.0, DK_CART])
    ky_m = kcart2 - np.array([0.0, DK_CART])
    kred_p = kred.copy(); kred_p[:2] = cart2_to_red(ky_p)
    kred_m = kred.copy(); kred_m[:2] = cart2_to_red(ky_m)
    vy = (H_of_kred(kred_p) - H_of_kred(kred_m)) / (2.0 * DK_CART)

    # eigenbasis

```

```

Udag = U.conj().T
Vx = Udag @ vx @ U
Vy = Udag @ vy @ U

dE = (E[None,:] - E[:,None])
denom = dE*dE
np.fill_diagonal(denom, np.inf)

P = Vx * Vy.T
omega = -2.0 * np.imag(np.sum(P / denom, axis=1))
omega = np.real(omega)
return E, omega

# =====
# k-path:  $\Gamma$ -K'-M-K- $\Gamma$  (reduced)
# =====
G = np.array([0.0,0.0,0.0])
Kp = np.array([2/3,-1/3,0.0]) # K'
Mpt= np.array([1/2,0.0,0.0])
K = np.array([1/3,1/3,0.0])

nodes=[("G",G),("Kp",Kp),("M",Mpt),("K",K),("G",G)]

def interp(k0,k1,npt):
    t=np.linspace(0,1,npt)
    return (1-t)[:None]*k0[None,:] + t[:None]*k1[None,:]

k_list=[]; tick=[0]; lab=[nodes[0][0]]; pos=0
for i in range(len(nodes)-1):
    _,p0=nodes[i]
    name1,p1=nodes[i+1]
    seg=interp(p0,p1,N_PER_SEG)
    if i>0: seg=seg[1:]
    k_list.append(seg)
    pos += len(seg)
    tick.append(pos-1); lab.append(name1)

k_path=np.vstack(k_list)
x=np.arange(len(k_path))

# =====
# Compute band-group Omega along path
# =====
Omega_VB = np.zeros(len(k_path), float) # occupied sum (valence bands)
Omega_CB3 = np.zeros(len(k_path), float) # sum of first three CB (optional)

for i, kred in enumerate(k_path):
    E, omega_all = omega_all_bands_at_k(kred)

    f = fermi_dirac(E) # occupied weights
    Omega_VB[i] = np.sum(f * omega_all)
    Omega_CB3[i] = np.sum(omega_all[CBANDS])

    if i % 50 == 0:
        print(f"[INFO] {i}/{len(k_path)} Omega_VB={Omega_VB[i]: .6f}")

# =====
# Save
# =====
out_dat = OUT_PREFIX + ".dat"
header = "idx k1 k2 EF(eV) Omega_VB(occ_sum) Omega_CB3(sum_first3CB)"
np.savetxt(out_dat,
            np.column_stack([x, k_path[:,0], k_path[:,1], np.full_like(x, EF, float), Omega_VB, Omega_CB3]),
            fmt="%.10f", header=header)
print("[INFO] saved", out_dat)

# =====
# Plot: ONE curve (literature-style single line)
# Here: Valence bands (occupied sum)

```

```

# =====
plt.figure(figsize=(8.0,3.6), dpi=220)
plt.plot(x, Omega_VB, lw=2.0)
plt.axhline(0, lw=0.8)
for tp in tick:
    plt.axvline(tp, lw=0.6)
plt.xticks(tick, lab)
plt.xlabel("k-path: G-K'-M-K-G")
plt.ylabel(r"$\Omega_{xy}$ (valence bands, occupied sum)")
plt.tight_layout()
out_png = OUT_PREFIX + ".png"
plt.savefig(out_png, bbox_inches="tight")
plt.show()
print("[INFO] saved", out_png)

```

► 代码清单 A.82.14 (python, 44 行)

```

# ===== Cell: build k-distance x-axis + print high-symmetry k coords =====
import numpy as np
import pandas as pd

# 1) Print high-symmetry points (reduced + Cartesian)
rows = []
for name, kred in nodes:
    kred = np.asarray(kred, float)
    k_cart = red_to_cart2(kred[:2]) # 2D cart (Å-1)
    rows.append({
        "label": name,
        "k_red_1": kred[0], "k_red_2": kred[1], "k_red_3": kred[2] if kred.size > 2 else 0.0,
        "k_cart_x(1/Å)": k_cart[0], "k_cart_y(1/Å)": k_cart[1],
        "|k_cart|(1/Å)": float(np.linalg.norm(k_cart)),
    })

df_kpts = pd.DataFrame(rows)
display(df_kpts)

# 2) Build cumulative k-path distance as x-axis (Å-1)
kcart_path = np.array([red_to_cart2(k[:2]) for k in k_path]) # (N,2)
dk = np.linalg.norm(np.diff(kcart_path, axis=0), axis=1) # (N-1,)
x_kdist = np.concatenate([[0.0], np.cumsum(dk)]) # (N,)

# 3) Get tick positions (x-axis locations) for each high-symmetry node
# This must match how you built k_path: first point of first seg kept;
# later segments drop the first point (seg[1:]).
tick_idx = [0]
pos = 0
for i in range(len(nodes)-1):
    seg_len = N_PER_SEG if i == 0 else (N_PER_SEG - 1)
    pos += seg_len
    tick_idx.append(pos - 1)

tick_pos = x_kdist[tick_idx]
tick_lab = [name for name, _ in nodes]

print("tick_idx =", tick_idx)
print("tick_pos (1/Å) =", tick_pos)
print("tick_lab =", tick_lab)

# Now, in your plot cell, use:
# plt.plot(x_kdist, Omega_VB) or plt.plot(x_kdist, Omega_VBM)
# plt.xticks(tick_pos, tick_lab)

```

► 代码清单 A.82.15 (python, 61 行)

```
# ===== Next cell: plot 1D Berry-curvature curve with high-symmetry x-axis + write final.dat =====
import numpy as np
import matplotlib.pyplot as plt

# ---- Safety checks: pick which Omega you already computed ----
# Prefer Omega_VBM if exists, else Omega_VB
if "Omega_VBM" in globals():
    Omega_y = np.asarray(Omega_VBM, float)
    y_label = r"$\Omega_{xy}$ (VBM single band)"
elif "Omega_VB" in globals():
    Omega_y = np.asarray(Omega_VB, float)
    y_label = r"$\Omega_{xy}$ (occupied sum)"
else:
    raise NameError("Neither Omega_VBM nor Omega_VB found. Please run the Berry-curvature calculation cell
    ↪ first.")

# ---- Build k-distance x-axis ( $\text{\AA}^{-1}$ ) ----
kcart_path = np.array([red_to_cart2(k[:2]) for k in k_path]) # (N,2)
dk = np.linalg.norm(np.diff(kcart_path, axis=0), axis=1) # (N-1,)
x_kdist = np.concatenate([[0.0], np.cumsum(dk)]) # (N,)

# ---- High-symmetry ticks (must match your k_path construction) ----
tick_idx = [0]
pos = 0
for i in range(len(nodes)-1):
    seg_len = N_PER_SEG if i == 0 else (N_PER_SEG - 1) # because later segments drop first point
    pos += seg_len
    tick_idx.append(pos - 1)

tick_pos = x_kdist[tick_idx]
tick_lab = [name for name, _ in nodes]

# ---- Save final.dat ----
# Columns:
# 1) x_kdist ( $\text{\AA}^{-1}$ )
# 2) Omega_y
# 3) k1 (reduced)
# 4) k2 (reduced)
# 5) idx (0-based point index)
final_dat = "final.dat"
header = "x_kdist(1/A) Omega_xy k1_red k2_red idx"
out = np.column_stack([x_kdist, Omega_y, k_path[:,0], k_path[:,1], np.arange(len(k_path))])
np.savetxt(final_dat, out, fmt="%.10f", header=header)
print("[INFO] saved", final_dat)

# ---- Plot ----
plt.figure(figsize=(8.0, 3.6), dpi=220)
plt.plot(x_kdist, Omega_y, lw=2.0)
plt.axhline(0, lw=0.8)

for tp in tick_pos:
    plt.axvline(tp, lw=0.6)

plt.xticks(tick_pos, tick_lab)
plt.xlabel("k-path: " + "-".join(tick_lab) + r" (cumulative |dk|, 1/\text{\AA})")
plt.ylabel(y_label)
plt.tight_layout()
plt.show()

print("\n[For your plotting]")
print("x-axis = column 1 : x_kdist(1/A)")
print("y-axis = column 2 : Omega_xy")
```


► 代码清单 A.82.16 (python, 27 行)

```
# =====
# Output high-symmetry x-coordinates (VASPKIT-like klabels)
# x-axis coordinate = cumulative |dk| along the path (in 1/Å)
# =====

# 1) compute cumulative distance axis x_kdist (same curve, but physical distance)
kcart_path = np.array([red_to_cart2(k[:2]) for k in k_path]) # (N,2) in 1/Å
dk = np.linalg.norm(np.diff(kcart_path, axis=0), axis=1)
x_kdist = np.concatenate([[0.0], np.cumsum(dk)]) # (N,)

# 2) high-symmetry points positions on x-axis: use your existing tick indices
# tick already stores indices on the discrete path
klabels_lines = []
klabels_lines.append("# K-Label    Coordinate of high-symmetry k-point in band-structure plots")
for name, idx in zip(lab, tick):
    # optional: map "G" -> "GAMMA" to match VASPKIT style
    out_name = "GAMMA" if name in ["G", "Γ", "Gamma", "GAMMA"] else name
    klabels_lines.append(f"{out_name:>10s}    {x_kdist[idx]:.3f}")

# 3) print and save
print("\n".join(klabels_lines))

with open("klabels_berry.dat", "w", encoding="utf-8") as f:
    f.write("\n".join(klabels_lines) + "\n")

print("[INFO] saved klabels_berry.dat")
print("[INFO] x_kdist (1/Å) stored in variable: x_kdist")
```

► 代码清单 A.82.17 (python, 176 行)

```
# ===== Cell: keep top/right spines (box), but NO top/right ticks =====
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.ticker import FixedLocator

# -----
# 0) Pick y-data
# -----
if "Omega_VBM" in globals():
    y0 = np.asarray(Omega_VBM, float)
    ysrc = "Omega_VBM"
elif "Omega_VB" in globals():
    y0 = np.asarray(Omega_VB, float)
    ysrc = "Omega_VB"
else:
    raise NameError("Omega_VBM or Omega_VB not found. Please run the Berry-curvature calculation cell first.")

x0 = np.asarray(x, float) if "x" in globals() else np.arange(len(y0), dtype=float)

if "tick" not in globals() or "lab" not in globals():
    raise NameError("tick/lab not found. Please run the k-path construction cell first.")

tick_idx = list(tick)
tick_lab = list(lab)

def _rename(lbl):
    if lbl in ["G", "Γ", "Gamma", "GAMMA"]:
        return "Γ"
    if lbl in ["Kp", "K'", "K-"]:
        return "K-"
```

```

        if lbl in ["K", "K+"]:
            return "K+"
        return lbl

tick_lab2 = [_rename(s) for s in tick_lab]

# -----
# 1) Units: Å^2 -> Bohr^2
# -----
CONVERT_A2_TO_BOHR2 = True
ANG_PER_BOHR = 0.529177210903
A2_TO_BOHR2 = (1.0 / ANG_PER_BOHR) ** 2

if CONVERT_A2_TO_BOHR2:
    y = y0 * A2_TO_BOHR2
    y_label = r"$\Omega_z\;(\mathrm{Bohr}^2)$"
    y_unit = "Bohr^2"
else:
    y = y0.copy()
    y_label = r"$\Omega_z\;(\mathrm{\AA}^2)$"
    y_unit = "Å^2"

# -----
# 2) Gentle smoothing (Hermite) with extrema at K- and K+
# -----
Nsub = 40
m = np.zeros_like(y)
m[1:-1] = (y[2:] - y[:-2]) / (x0[2:] - x0[:-2])

idx_Km = None
idx_Kp = None
for lbl, idx in zip(tick_lab2, tick_idx):
    if lbl == "K-":
        idx_Km = idx
    if lbl == "K+":
        idx_Kp = idx

if idx_Km is not None:
    m[idx_Km] = 0.0
if idx_Kp is not None:
    m[idx_Kp] = 0.0
m[0] = 0.0
m[-1] = 0.0

def hermite_segment(xa, xb, ya, yb, ma, mb, t):
    h = xb - xa
    t2 = t * t
    t3 = t2 * t
    h00 = 2*t3 - 3*t2 + 1
    h10 = t3 - 2*t2 + t
    h01 = -2*t3 + 3*t2
    h11 = t3 - t2
    return h00*ya + h10*h*ma + h01*yb + h11*h*mb

xd_list = [x0[0]]
yd_list = [y[0]]
for i in range(len(x0) - 1):
    xa, xb = x0[i], x0[i+1]
    ya, yb = y[i], y[i+1]
    ma, mb = m[i], m[i+1]
    tt = np.linspace(0.0, 1.0, Nsub+1)[1:]
    xd = xa + (xb - xa) * tt
    yd = hermite_segment(xa, xb, ya, yb, ma, mb, tt)
    xd_list.append(xd)
    yd_list.append(yd)

x_smooth = np.concatenate([np.atleast_1d(v) for v in xd_list])
y_smooth = np.concatenate([np.atleast_1d(v) for v in yd_list])

```

```

# -----
# 3) Plot styling
# -----
W_cm, H_cm = 12.9, 4.66
fig = plt.figure(figsize=(W_cm/2.54, H_cm/2.54), dpi=600)

plt.rcParams.update({
    "font.family": "Times New Roman",
    "mathtext.fontset": "stix",
    "axes.linewidth": 1.0,
})

ax = plt.gca()

# curve: light blue
ax.plot(
    x_smooth, y_smooth,
    lw=1.8,
    color="#8ecae6",
    solid_capstyle="round",
    solid_joinstyle="round"
)

# dashed guides: high-symmetry + y=0, thin 25%, light red
guide_lw = 0.8 * 0.75
guide_color = "#f4a3a3"
guide_ls = (0, (4, 3))

for tp in tick_idx:
    ax.axvline(tp, lw=guide_lw, color=guide_color, linestyle=guide_ls, zorder=0)
ax.axhline(0.0, lw=guide_lw, color=guide_color, linestyle=guide_ls, zorder=0)

# x ticks at high-symmetry points
ax.set_xticks(tick_idx)
ax.set_xticklabels(tick_lab2, fontsize=10)

# y range + 4 y-ticks
ax.set_ylim(-14, 10)
ax.yaxis.set_major_locator(FixedLocator([-14, -6, 2, 10]))

ax.set_ylabel(y_label, fontsize=12)
ax.set_xlabel(r"$k$-path", fontsize=11)

# Keep top/right spines (box), but disable top/right ticks
ax.tick_params(direction="in", length=4.0, width=1.0, labelsize=10,
               top=False, right=False, bottom=True, left=True)

# Ensure spines are visible (do NOT hide them)
ax.spines["top"].set_visible(True)
ax.spines["right"].set_visible(True)

plt.tight_layout(pad=0.5)

out_png = "berrycurve_pub_v4.png"
out_pdf = "berrycurve_pub_v4.pdf"
plt.savefig(out_png, bbox_inches="tight")
plt.savefig(out_pdf, bbox_inches="tight")
plt.show()

# -----
# 4) Print K- / K+ values
# -----
def _val_at(idx):
    return float(y[idx])

print("[INFO] saved:", out_png, "and", out_pdf)
print("[INFO] Omega source:", ysrc, "| units:", y_unit)

if idx_Km is not None:

```

```

    print(f"[K-] idx={idx_Km}, Omega={_val_at(idx_Km): .10f} {y_unit}")
else:
    print("[K-] not found; available:", tick_lab2)

if idx_Kp is not None:
    print(f"[K+] idx={idx_Kp}, Omega={_val_at(idx_Kp): .10f} {y_unit}")
else:
    print("[K+] not found; available:", tick_lab2)

```

► 代码清单 A.82.18 (python, 54 行)

```

# ===== Cell: output conduction-band Berry curvature along the same k-path =====
import numpy as np
import matplotlib.pyplot as plt

# 1) sanity: require CB result already computed in the main cell
if "Omega_CB3" not in globals():
    raise NameError("Omega_CB3 not found. Please run the main calculation cell (the loop over k_path) first.")
if "k_path" not in globals():
    raise NameError("k_path not found. Please run the k-path construction cell first.")
if "x" not in globals():
    x = np.arange(len(Omega_CB3), dtype=float)

# 2) build a clean output file name
try:
    ncb = int(N_CB)
except Exception:
    ncb = None

cb_prefix = f"{OUT_PREFIX}_CB_first{ncb}" if ncb is not None else f"{OUT_PREFIX}_CB"
cb_dat = cb_prefix + ".dat"
cb_png = cb_prefix + ".png"

# 3) save CB curve data
# columns:
# 1) idx (x-axis index)
# 2) k1_red
# 3) k2_red
# 4) Omega_CB (sum over first N_CB conduction bands)
header = "idx k1_red k2_red Omega_CB(sum_first_NCB)"
np.savetxt(
    cb_dat,
    np.column_stack([x, k_path[:, 0], k_path[:, 1], Omega_CB3]),
    fmt="%.10f",
    header=header
)
print("[INFO] saved", cb_dat)

# 4) plot CB curve (same x-axis ticks as your VB plot)
plt.figure(figsize=(8.0, 3.6), dpi=220)
plt.plot(x, Omega_CB3, lw=2.0)
plt.axhline(0, lw=0.8)
for tp in tick:
    plt.axvline(tp, lw=0.6)
plt.xticks(tick, lab)
plt.xlabel("k-path: G-K'-M-K-G")
plt.ylabel(r"$\Omega_{xy}$ (first $N_{\mathrm{CB}}$ conduction bands sum)")
plt.tight_layout()
plt.savefig(cb_png, bbox_inches="tight")
plt.show()
print("[INFO] saved", cb_png)

print("\n[For plotting from file]")

```

```
print("x-axis = column 1 (idx)")
print("y-axis = column 4 (Omega_CB)")
```

► 代码清单 A.82.19 (python, 176 行)

上个代码用的是 hermite 插值, 然后单位转换, 转换后数值太大了, 现在不进行单位转换
改用未经单位转换的数据

```
# ===== Cell: CB Berry curvature (Hermite + publication style, NO unit conversion) =====
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.ticker import FixedLocator

# -----
# 0) Sanity check
# -----
if "Omega_CB3" not in globals():
    raise NameError("Omega_CB3 not found. Run main calculation cell first.")
if "k_path" not in globals():
    raise NameError("k_path not found.")
if "tick" not in globals() or "lab" not in globals():
    raise NameError("tick/lab not found.")
if "x" not in globals():
    x = np.arange(len(Omega_CB3), dtype=float)

y = np.asarray(Omega_CB3, float)
x0 = np.asarray(x, float)

tick_idx = list(tick)
tick_lab = list(lab)

def _rename(lbl):
    if lbl in ["G", "Γ", "Gamma", "GAMMA"]:
        return "Γ"
    if lbl in ["Kp", "K'", "K-"]:
        return "K-"
    if lbl in ["K", "K+"]:
        return "K+"
    return lbl

tick_lab2 = [_rename(s) for s in tick_lab]

print("max raw =", np.max(y))
print("min raw =", np.min(y))

# -----
# 1) Hermite smoothing
# -----
Nsub = 40
m = np.zeros_like(y)
m[1:-1] = (y[2:] - y[:-2]) / (x0[2:] - x0[:-2])

# enforce extrema at K- and K+
idx_Km = None
idx_Kp = None
for lbl, idx in zip(tick_lab2, tick_idx):
    if lbl == "K-":
        idx_Km = idx
    if lbl == "K+":
        idx_Kp = idx

if idx_Km is not None:
    m[idx_Km] = 0.0
```

```

if idx_Kp is not None:
    m[idx_Kp] = 0.0

m[0] = 0.0
m[-1] = 0.0

def hermite_segment(xa, xb, ya, yb, ma, mb, t):
    h = xb - xa
    t2 = t*t
    t3 = t2*t
    h00 = 2*t3 - 3*t2 + 1
    h10 = t3 - 2*t2 + t
    h01 = -2*t3 + 3*t2
    h11 = t3 - t2
    return h00*ya + h10*h*ma + h01*yb + h11*h*mb

xd_list = [x0[0]]
yd_list = [y[0]]

for i in range(len(x0)-1):
    xa, xb = x0[i], x0[i+1]
    ya, yb = y[i], y[i+1]
    ma, mb = m[i], m[i+1]
    tt = np.linspace(0, 1, Nsub+1)[1:]
    xd = xa + (xb - xa) * tt
    yd = hermite_segment(xa, xb, ya, yb, ma, mb, tt)
    xd_list.append(xd)
    yd_list.append(yd)

x_smooth = np.concatenate([np.atleast_1d(v) for v in xd_list])
y_smooth = np.concatenate([np.atleast_1d(v) for v in yd_list])

print("max smooth =", np.max(y_smooth))

# -----
# 2) Save data
# -----
header = "idx k1_red k2_red Omega_CB_A2"
np.savetxt(
    "CB_berrycurve.dat",
    np.column_stack([x0, k_path[:,0], k_path[:,1], y]),
    fmt="%.10f",
    header=header
)
print("[INFO] saved CB_berrycurve.dat")

# -----
# 3) Publication-style plot
# -----
W_cm, H_cm = 12.9, 4.66
fig = plt.figure(figsize=(W_cm/2.54, H_cm/2.54), dpi=600)

plt.rcParams.update({
    "font.family": "Times New Roman",
    "mathtext.fontset": "stix",
    "axes.linewidth": 1.0,
})

ax = plt.gca()

# curve
ax.plot(
    x_smooth, y_smooth,
    lw=1.8,
    color="#8ecae6",
    solid_capstyle="round",
    solid_joinstyle="round"
)

```

```

# dashed guides
guide_lw = 0.8 * 0.75
guide_color = "#f4a3a3"
guide_ls = (0, (4, 3))

for tp in tick_idx:
    ax.axvline(tp, lw=guide_lw, color=guide_color, linestyle=guide_ls, zorder=0)

ax.axhline(0.0, lw=guide_lw, color=guide_color, linestyle=guide_ls, zorder=0)

# x ticks
ax.set_xticks(tick_idx)
ax.set_xticklabels(tick_lab2, fontsize=10)

# y range 自动根据数据留冗余
ymax = np.max(y_smooth)
ymin = np.min(y_smooth)
margin = 0.1 * (ymax - ymin)

ax.set_ylim(ymin - margin, ymax + margin)

ax.set_ylabel(r"$\Omega_z\;(\mathrm{AA}^2)$", fontsize=12)
ax.set_xlabel(r"$k$-path", fontsize=11)

# box but no top/right ticks
ax.tick_params(direction="in", length=4.0, width=1.0,
                top=False, right=False, bottom=True, left=True)

ax.spines["top"].set_visible(True)
ax.spines["right"].set_visible(True)

plt.tight_layout(pad=0.5)

plt.savefig("CB_berrycurve_pub.png", bbox_inches="tight")
plt.savefig("CB_berrycurve_pub.pdf", bbox_inches="tight")
plt.show()

print("[INF0] saved CB_berrycurve_pub.png/pdf")

# -----
# 4) Print K- / K+
# -----
def _val_at(idx):
    return float(y[idx])

if idx_Km is not None:
    print(f"[K-] Omega = {_val_at(idx_Km): .10f} Å²")
if idx_Kp is not None:
    print(f"[K+] Omega = {_val_at(idx_Kp): .10f} Å²")

```

(原创脚本)blender 绘制斯格明子

► 代码清单 A.83.1 (shell, 9 行)

```

#!/bin/bash -x
mkdir result
for i in {0,5,10,15,20,25}
do
for j in {0,100,200,300,400,500}
do

```

```
cp B$i/T$j/*600000.ovf ./result/B$i-T$j-final.ovf
done
done
```

► 代码清单 A.83.2 (python, 127 行)

```
import numpy as np
import os
import matplotlib.pyplot as plt
from scipy.interpolate import griddata
from pylab import *
import zhshen as zs

# 设置文件路径
path_top = "/public/home/cssong/song/lmr/9_single_layer/19_ReIrGe2S6/TcIrGeSe/5_spirit/2_spirit/2_spirit/result/"
fig_path = path_top + "0-figs-Mz"

FILE1 = path_top + "POSCAR"
repeat_x = 300
repeat_y = 300
repeat_z = 1

# 读取 POSCAR 文件
mag_atom_posi = zs.read_poscar(FILE1)[4]
base_vec = zs.read_poscar(FILE1)[2]

# 生成超胞原子坐标
super_posi = []
count = 0
for i in range(int(repeat_z)):
    for j in range(int(repeat_y)):
        for k in range(int(repeat_x)):
            for m in range(len(mag_atom_posi)):
                super_posi.append([])
                super_posi[count].append(float(mag_atom_posi[m][0]) + float(k)*float(base_vec[0][0]) +
                    ↪ float(j)*float(base_vec[1][0]) + float(i)*float(base_vec[2][0]))
                super_posi[count].append(float(mag_atom_posi[m][1]) + float(k)*float(base_vec[0][1]) +
                    ↪ float(j)*float(base_vec[1][1]) + float(i)*float(base_vec[2][1]))
                super_posi[count].append(float(mag_atom_posi[m][2]) + float(k)*float(base_vec[0][2]) +
                    ↪ float(j)*float(base_vec[1][2]) + float(i)*float(base_vec[2][2]))
                count += 1

super_posi = np.array(super_posi)
x1 = super_posi[:,0]
y1 = super_posi[:,1]
z1 = super_posi[:,2]

# 确保图片保存文件夹存在
try:
    if not os.path.exists(fig_path):
        os.makedirs(fig_path)
        print(" Directory created: " + fig_path)
    else:
        print(" " + fig_path + " exists!!!")
except BaseException as msg:
    print(" 新建目录失败: " + str(msg))

# 自旋文件参数
j1 = ['0','5','10','15','20']

#j1 = ['25']
j2 = ['0','100','200','300','400','500']
spirit_or_vampire = 1 # 1 for spirit
```

```

# 遍历所有文件
for i in range(len(j1)):
    for j in range(len(j2)):
        FILE2 = path_top + "B" + str(j1[i]) + "-" + "T" + str(j2[j]) + "-final.ovf"
        fig_name = fig_path + '/' + "B" + str(j1[i]) + "-" + "T" + str(j2[j]) + "-final.png"

        # 检查文件是否存在
        if not os.path.exists(FILE2):
            print(f"  文件不存在: {FILE2}")
            continue

        data2 = np.loadtxt(FILE2)

        if spirit_or_vampire == 1: # spirit 格式
            spin_x1 = data2[:,0]
            spin_y1 = data2[:,1]
            spin_z1 = data2[:,2]

        # 提取顶层的自旋信息
        up_z = np.max(z1)
        x = []
        y = []
        z = []
        spin_x = []
        spin_y = []
        spin_z = []

        for ii in range(len(z1)):
            if abs(z1[ii] - up_z) < 3.0:
                x.append(x1[ii])
                y.append(y1[ii])
                spin_x.append(spin_x1[ii])
                spin_y.append(spin_y1[ii])
                spin_z.append(spin_z1[ii])

        x = np.array(x)
        y = np.array(y)
        spin_z = np.array(spin_z)

        # 网格化
        nx, ny = 300, 300
        xi = np.linspace(x.min(), x.max(), nx)
        yi = np.linspace(y.min(), y.max(), ny)
        zi = griddata((x, y), spin_z, (xi[None,:], yi[:,None]))

        # 创建图形, 设置分辨率和大小
        fig = plt.figure(figsize=(4, 4), dpi=300, facecolor='white', edgecolor='white')
        ax = fig.add_subplot(111)

        # 设置白色背景
        fig.patch.set_facecolor('white')
        ax.patch.set_facecolor('white')

        # 绘制填充等高线图
        levels = np.linspace(-1.0, 1.0, 150) # 自旋分量范围
        cf = ax.contourf(xi, yi, zi, levels, cmap='RdBu', extend='both')

        # 移除边框、坐标轴、刻度
        ax.set_frame_on(False) # 移除边框
        ax.axis('off') # 移除坐标轴

        # 移除所有空白边距
        plt.subplots_adjust(top=1, bottom=0, right=1, left=0, hspace=0, wspace=0)
        plt.margins(0, 0)

        # 保存图片, 不包含空白边距
        fig.savefig(fig_name, bbox_inches='tight', pad_inches=0, facecolor='white', edgecolor='white')
        plt.close(fig) # 关闭图形, 释放内存

```

```

        print(f"    已生成: {fig_name}")

print("    所有图片生成完成!")

```

► 代码清单 A.83.3 (python, 162 行)

```

import os
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import colors
from matplotlib.colors import Normalize
from scipy.interpolate import griddata
from tqdm import tqdm
from scipy.interpolate import CloughTocher2DInterpolator

# 参数设置
#discrete_num = 500                # 控制色块数 (对于清晰度更重要, 但是也大幅影响出图速度, 并且由于数据本身的连续性,
↳ 太大不一定会有肉眼提升)
#save_dpi = discrete_num/5        # 导出图像分辨率, 由于figsize = (5,5) 因此取 discrete_num 的 1/5
↳ 可以保证精度同时让图像最小

input_dir = "."                  # ovf 文件所在目录
output_dir = "."                # 输出图像目录
#is_plot_inplain = 0            # 0: 只根据 Sz 设置颜色, 1: Sx, Sy 决定颜色 Sz 决定明暗
# cmap = 'bwr'                  # 如果只根据 Sz, 控制 Sz 映射颜色 (默认 bwr: 红色 +z 方向)

#is_colorbar = 0                # 是否加上 colorbar

# ===== 晶格与原子信息 (手动设置) =====
repeat_x = 300                  # 扩胞尺寸
repeat_y = 300
repeat_z = 1

mag_atom_posi = [
    [0.0000027711196385, 0.0000078820326969, 0.3339943794051239],
    [0.5000027711196385, 0.0000078820326969, 0.3339943794051239],
    [0.2500027711196385, 0.5000078820326969, 0.3339943794051239],
    [0.7500027711196385, 0.5000078820326969, 0.3339943794051239]
]

base_vec = [
    [13.1502104227330392, -0.0001278456871606, 0.0013066726293966], # a 向量
    [-0.0001009508214391, 11.3881951329254409, -0.0008005358804461], # b 向量
    [0.0016905845507726, -0.0009144982993895, 17.5548063557554279] # c 向量
]

# repeat_x = 50                # 扩胞尺寸
# repeat_y = 50
# repeat_z = 1

# mag_atom_posi = [
#     [0.0, 0.0, 0.0] # 原胞中原子的相对坐标
# ]

# base_vec = [
#     [1.0, 0.0, 0.0], # a 向量
#     [0.0, 1.0, 0.0], # b 向量
#     [0.0, 0.0, 1.0] # c 向量
# ]
# =====

```

```

# 自动扩胞生成所有原子绝对坐标
def generate_supercell_positions(mag_atom_posi, base_vec, repeat_x, repeat_y, repeat_z):
    base_vec = np.array(base_vec) # shape: (3, 3)
    pos_list = []

    # Step 1: 将分数坐标转换为绝对坐标
    mag_atom_abs = []
    for atom in mag_atom_posi:
        frac = np.array(atom) # shape: (3,)
        abs_pos = frac @ base_vec # 点乘得到绝对坐标
        mag_atom_abs.append(abs_pos)

    # Step 2: 对每个晶胞进行平移复制
    for i in range(repeat_z):
        for j in range(repeat_y):
            for k in range(repeat_x):
                shift = k * base_vec[0] + j * base_vec[1] + i * base_vec[2]
                for abs_pos in mag_atom_abs:
                    new_pos = abs_pos + shift
                    pos_list.append(new_pos)

    pos_array = np.array(pos_list)
    return pos_array[:, 0], pos_array[:, 1], pos_array[:, 2] # 返回 x, y, z 坐标

# 绘图主函数
def plot_inplain(x, y, spin_x, spin_y, spin_z, fig_path, xi, yi, dpi=100):
    XI, YI = np.meshgrid(xi, yi)

    interpolator_x = CloughTocher2DInterpolator(np.column_stack([x, y]), spin_x)
    interpolator_y = CloughTocher2DInterpolator(np.column_stack([x, y]), spin_y)
    interpolator_z = CloughTocher2DInterpolator(np.column_stack([x, y]), spin_z)

    sx = interpolator_x(XI, YI)
    sx = np.clip(sx, -1.0, 1.0)
    sy = interpolator_y(XI, YI)
    sy = np.clip(sy, -1.0, 1.0)
    sz = interpolator_z(XI, YI)
    sz = np.clip(sz, -1.0, 1.0)

    hue = (np.arctan2(sy, sx) + np.pi) / (2 * np.pi)
    value = Normalize(vmin=-1, vmax=1)(sz)
    value = np.clip(value, 0, 1)

    hsv = np.zeros((sz.shape[0], sz.shape[1], 3))
    hsv[..., 0] = hue
    hsv[..., 1] = 1 - np.abs(1 - 2 * value)
    hsv[..., 2] = value
    rgb = colors.hsv_to_rgb(hsv)

    plt.imshow(rgb, extent=(x.min(), x.max(), y.min(), y.max()), origin='lower', aspect='auto')
    #plt.colorbar(format='%5.2f')
    plt.axis('equal')
    plt.axis('off')
    plt.savefig(fig_path, dpi=dpi, bbox_inches='tight', pad_inches=0, transparent = True)
    plt.close()

def plot(x, y, spin_x, spin_y, spin_z, fig_path, xi, yi, cmap, dpi=100):
    XI, YI = np.meshgrid(xi, yi)
    interpolator_z = CloughTocher2DInterpolator(np.column_stack([x, y]), spin_z)
    sz = interpolator_z(XI, YI)
    sz = np.clip(sz, -1.0, 1.0)

    plt.figure(figsize=(5, 5))
    plt.imshow(sz, extent=(x.min(), x.max(), y.min(), y.max()), origin='lower', aspect='auto', cmap = cmap,
        ↪ vmin=-1, vmax=1)
    if is_colorbar:
        plt.colorbar(format='%5.2f')
    plt.axis('equal')
    plt.axis('off')

```

```

plt.savefig(fig_path, dpi=dpi, bbox_inches='tight', pad_inches=0, transparent = True)
plt.close()

# 主执行函数
def main():
    # 创建输出目录
    os.makedirs(output_dir, exist_ok=True)

    # 获取文件列表
    files = sorted(f for f in os.listdir(input_dir) if f.endswith(".ovf"))
    if not files:
        print("[!] 未找到 ovf 文件")
        return

    # 生成原子坐标与插值网格
    x, y, z = generate_supercell_positions(mag_atom_posi, base_vec, repeat_x, repeat_y, repeat_z)
    #xi = np.linspace(x.min(), x.max(), discrete_num)
    #yi = np.linspace(y.min(), y.max(), discrete_num)

    # 批量绘图
    for file in tqdm(files, desc=" 处理文件"):
        file_path = os.path.join(input_dir, file)
        if os.path.isdir(file_path):
            continue
        data = np.loadtxt(file_path)

        spin_x, spin_y, spin_z = data[:, 0], data[:, 1], data[:, 2]

        spin_data = np.column_stack((x,y,z,spin_x, spin_y, spin_z))

        fig_name = os.path.splitext(file)[0] + ".csv"
        fig_path = os.path.join(output_dir, fig_name)
        np.savetxt(fig_path, spin_data, delimiter=',')

    print("[OK] 自旋数据已生成, 保存在: ", output_dir)

# 执行
if __name__ == "__main__":
    main()

```

► 代码清单 A.83.4 (python, 69 行)

```

import cv2
import os

# =====
# 路径设置
# =====
path = './' # 原图文件夹
savepath = './resize/'

os.makedirs(savepath, exist_ok=True)

# =====
# 找到所有 png
# =====
png_files = sorted([f for f in os.listdir(path) if f.endswith('.png')])

if len(png_files) == 0:
    raise FileNotFoundError(" 当前文件夹下没有找到 .png 文件")

# =====

```

```

# 用第一张图手动框选 ROI
# =====
first_file = png_files[0]
first_path = os.path.join(path, first_file)

img0 = cv2.imread(first_path)
if img0 is None:
    raise ValueError(f" 无法读取图片: {first_path}")

print(f" 请在弹出的窗口中框选需要保留的区域: {first_file}")
print(" 操作说明: ")
print("1. 鼠标左键拖动框选")
print("2. 按 Enter 或 Space 确认")
print("3. 按 c 取消重选")

x, y, w, h = cv2.selectROI("Select ROI", img0, showCrosshair=True, fromCenter=False)
cv2.destroyAllWindows()

if w == 0 or h == 0:
    raise ValueError(" 未选中有效区域, 程序终止。")

print(f" 选中的 ROI: x={x}, y={y}, w={w}, h={h}")
print(f" 对应裁剪写法: img[{y}:{y+h}, {x}:{x+w}]")

# =====
# 批量裁剪
# =====
for file in png_files:
    png_name = os.path.join(path, file)
    img = cv2.imread(png_name)

    if img is None:
        print(f" 跳过无法读取的文件: {file}")
        continue

    # 防止不同图片尺寸略有不同导致越界
    H, W = img.shape[:2]
    x0 = max(0, x)
    y0 = max(0, y)
    x1 = min(W, x + w)
    y1 = min(H, y + h)

    cropped = img[y0:y1, x0:x1]

    savepng_name = os.path.join(savepath, file)
    cv2.imwrite(savepng_name, cropped)
    print(f" 已保存: {savepng_name}")

print('结束')

```

▶ 代码清单 A.83.5 (shell, 1 行)

ROI: x=912, y=602, w=79, h=68

▶ 代码清单 A.83.6 (python, 42 行)

```

import cv2
import numpy as np

```

```

png_file = "B15-T500-final.png"
csv_file = "2026-03-05_08-36-18_Image-00_Spins_600000.csv"

# 当前 ROI
x, y, w, h = 912, 602, 79, 68

# 读图片尺寸
img = cv2.imread(png_file)
if img is None:
    raise ValueError(f" 无法读取图片: {png_file}")
H, W = img.shape[:2]

# 读 csv
data = np.loadtxt(csv_file, delimiter=',', skiprows=1)

xmin, xmax = data[:, 0].min(), data[:, 0].max()
ymin, ymax = data[:, 1].min(), data[:, 1].max()
zmin, zmax = data[:, 2].min(), data[:, 2].max()

# 像素 -> 实际坐标
x_range = (
    float(xmin + (x / W) * (xmax - xmin)),
    float(xmin + ((x + w) / W) * (xmax - xmin))
)

y_range = (
    float(ymin + ((H - (y + h)) / H) * (ymax - ymin)),
    float(ymin + ((H - y) / H) * (ymax - ymin))
)

z_range = (
    float(zmin - 1.0),
    float(zmax + 1.0)
)

print(" 可直接填写到 filter.py 中: ")
print(f"x_range={x_range},")
print(f"y_range={y_range},")
print(f"z_range={z_range},")

```

► 代码清单 A.83.7 (python, 42 行)

```

import cv2
import numpy as np

png_file = "B15-T500-final.png"
csv_file = "2026-03-05_08-36-18_Image-00_Spins_600000.csv"

# 当前 ROI
x, y, w, h = 912, 602, 79, 68

# 读图片尺寸
img = cv2.imread(png_file)
if img is None:
    raise ValueError(f" 无法读取图片: {png_file}")
H, W = img.shape[:2]

# 读 csv
data = np.loadtxt(csv_file, delimiter=',', skiprows=1)

xmin, xmax = data[:, 0].min(), data[:, 0].max()
ymin, ymax = data[:, 1].min(), data[:, 1].max()

```

```

zmin, zmax = data[:, 2].min(), data[:, 2].max()

# 像素 -> 实际坐标
x_range = (
    float(xmin + (x / W) * (xmax - xmin)),
    float(xmin + ((x + w) / W) * (xmax - xmin))
)

y_range = (
    float(ymin + ((H - (y + h)) / H) * (ymax - ymin)),
    float(ymin + ((H - y) / H) * (ymax - ymin))
)

z_range = (
    float(zmin - 1.0),
    float(zmax + 1.0)
)

print(" 可直接填写到 下方 cell 中: ")
print(f"x_range={x_range},")
print(f"y_range={y_range},")
print(f"z_range={z_range},")

```

► 代码清单 A.83.8 (shell, 4 行)

可直接填写到 filter.py 中:

```

x_range=(2995.7427548963037, 3255.24496400857),
y_range=(1506.399353766607, 1699.6781734408135),
z_range=(4.623772823642497, 7.254555104183036),

```

► 代码清单 A.83.9 (python, 48 行)

```

import numpy as np

def filter_csv_by_xyz(file_path, x_range, y_range, z_range, output_path=None):
    """
    根据给定的 x, y, z 范围筛选 CSV 文件中的点, 并保存更新后的数据,
    同时打印筛选前后 xyz 的范围。
    """
    data = np.loadtxt(file_path, delimiter=",")

    # 筛选前范围
    print(" 筛选前范围: ")
    print(f"x: [{data[:,0].min():.6e}, {data[:,0].max():.6e}]")
    print(f"y: [{data[:,1].min():.6e}, {data[:,1].max():.6e}]")
    print(f"z: [{data[:,2].min():.6e}, {data[:,2].max():.6e}]")

    # 筛选
    mask = (
        (data[:, 0] >= x_range[0]) & (data[:, 0] <= x_range[1]) &
        (data[:, 1] >= y_range[0]) & (data[:, 1] <= y_range[1]) &
        (data[:, 2] >= z_range[0]) & (data[:, 2] <= z_range[1])
    )
    filtered_data = data[mask]

    # 筛选后范围
    if filtered_data.size == 0:
        print(" 警告: 筛选后没有点满足条件! ")

```

```

else:
    print(" 筛选后范围: ")
    print(f"x: [{filtered_data[:,0].min():.6e}, {filtered_data[:,0].max():.6e}]")
    print(f"y: [{filtered_data[:,1].min():.6e}, {filtered_data[:,1].max():.6e}]")
    print(f"z: [{filtered_data[:,2].min():.6e}, {filtered_data[:,2].max():.6e}]")

# 默认覆盖原文件
if output_path is None:
    output_path = file_path

np.savetxt(output_path, filtered_data, delimiter=",", fmt="%.18e")
print(f" 已筛选并保存至 {output_path}, 剩余 {filtered_data.shape[0]} 个点。")

return filtered_data.shape[0]

filter_csv_by_xyz(
    "2026-03-05_08-36-18_Image-00_Spins_600000.csv",    ## 源文件名
    x_range=(2995.7427548963037, 3255.24496400857),
    y_range=(1506.399353766607, 1699.6781734408135),
    z_range=(4.623772823642497, 7.254555104183036),
    output_path="data_filtered.csv"    ## 新文件名
)

```

► 代码清单 A.83.10 (python, 44 行)

```

import os
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import CloughTocher2DInterpolator

input_csv = "data_filtered.csv"
output_dir = "./result_fig"
output_name = "data_filtered_sz.png"

discrete_num = 500
save_dpi = 300
cmap = "bwr"

os.makedirs(output_dir, exist_ok=True)

data = np.loadtxt(input_csv, delimiter=",")
x = data[:, 0]
y = data[:, 1]
sz = data[:, 5]

xi = np.linspace(x.min(), x.max(), discrete_num)
yi = np.linspace(y.min(), y.max(), discrete_num)
XI, YI = np.meshgrid(xi, yi)

interp_sz = CloughTocher2DInterpolator(np.column_stack([x, y]), sz)
SZ = interp_sz(XI, YI)
SZ = np.clip(SZ, -1, 1)

plt.figure(figsize=(6, 6))
plt.imshow(
    SZ,
    extent=(x.min(), x.max(), y.min(), y.max()),
    origin="lower",
    cmap=cmap,
    vmin=-1,
    vmax=1,
    aspect="equal"
)

```

```
plt.axis("off")
plt.savefig(os.path.join(output_dir, output_name), dpi=save_dpi,
            bbox_inches="tight", pad_inches=0, transparent=True)
plt.close()

print(" 完成")
```

► 代码清单 A.83.11 (python, 44 行)

```
import os
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import CloughTocher2DInterpolator

input_csv = "data_filtered.csv"
output_dir = "./result_fig"
output_name = "data_filtered_sz.png"

discrete_num = 500
save_dpi = 300
cmap = "bwr"

os.makedirs(output_dir, exist_ok=True)

data = np.loadtxt(input_csv, delimiter=",")
x = data[:, 0]
y = data[:, 1]
sz = data[:, 5]

xi = np.linspace(x.min(), x.max(), discrete_num)
yi = np.linspace(y.min(), y.max(), discrete_num)
XI, YI = np.meshgrid(xi, yi)

interp_sz = CloughTocher2DInterpolator(np.column_stack([x, y]), sz)
SZ = interp_sz(XI, YI)
SZ = np.clip(SZ, -1, 1)

plt.figure(figsize=(6, 6))
plt.imshow(
    SZ,
    extent=(x.min(), x.max(), y.min(), y.max()),
    origin="lower",
    cmap=cmap,
    vmin=-1,
    vmax=1,
    aspect="equal"
)
plt.axis("off")
plt.savefig(os.path.join(output_dir, output_name), dpi=save_dpi,
            bbox_inches="tight", pad_inches=0, transparent=True)
plt.close()

print(" 完成")
```

► 代码清单 A.83.12 (python, 44 行)

```
import os
import numpy as np
```

```

import matplotlib.pyplot as plt
from scipy.interpolate import CloughTocher2DInterpolator

input_csv = "data_filtered.csv"
output_dir = "./result_fig"
output_name = "data_filtered_sz.png"

discrete_num = 500
save_dpi = 300
cmap = "bwr"

os.makedirs(output_dir, exist_ok=True)

data = np.loadtxt(input_csv, delimiter=",")
x = data[:, 0]
y = data[:, 1]
sz = data[:, 5]

xi = np.linspace(x.min(), x.max(), discrete_num)
yi = np.linspace(y.min(), y.max(), discrete_num)
XI, YI = np.meshgrid(xi, yi)

interp_sz = CloughTocher2DInterpolator(np.column_stack([x, y]), sz)
SZ = interp_sz(XI, YI)
SZ = np.clip(SZ, -1, 1)

plt.figure(figsize=(6, 6))
plt.imshow(
    SZ,
    extent=(x.min(), x.max(), y.min(), y.max()),
    origin="lower",
    cmap=cmap,
    vmin=-1,
    vmax=1,
    aspect="equal"
)
plt.axis("off")
plt.savefig(os.path.join(output_dir, output_name), dpi=save_dpi,
            bbox_inches="tight", pad_inches=0, transparent=True)
plt.close()

print(" 完成")

```

(绘图教程) VESTA 到 Blender 的二维材料 3D 论文原子结构绘图教程

► 代码清单 A.84.1 (text, 1 行)

<center>

► 代码清单 A.84.2 (text, 1 行)

</center>

► 代码清单 A.84.3 (text, 9 行)

```
POSCAR/CONTCAR
↓
VESTA 结构检查与显示优化
↓
导出模型文件
↓
Blender 三维渲染
↓
得到论文级原子结构示意图
```

► 代码清单 A.84.4 (text, 2 行)

```
VESTA 负责晶体结构准确性
Blender 负责三维表现、光照和渲染效果
```

► 代码清单 A.84.5 (text, 3 行)

```
POSCAR
CONTCAR
CIF
```

► 代码清单 A.84.6 (text, 1 行)

```
CONTCAR
```

► 代码清单 A.84.7 (text, 11 行)

```
导入晶体结构
→
检查原子位置
→
设置键连接
→
调整原子显示比例
→
扩胞
→
导出模型
```

► 代码清单 A.84.8 (text, 1 行)

File → Open

► 代码清单 A.84.9 (text, 1 行)

POSCAR/CONTCAR/CIF

► 代码清单 A.84.10 (text, 5 行)

Edit
→
Edit Data
→
Unit Cell

► 代码清单 A.84.12 (text, 1 行)

3×3×1 supercell

► 代码清单 A.84.13 (text, 2 行)

Cr-Se
Ru-Br

► 代码清单 A.84.14 (text, 1 行)

vdW 界面弱相互作用

► 代码清单 A.84.15 (text, 5 行)

VESTA
↓
VRML (.wrl)
↓
Blender

▶ 代码清单 A.84.16 (text, 5 行)

```
File  
→  
Export Data  
→  
VRML
```

▶ 代码清单 A.84.17 (text, 1 行)

```
structure.wrl
```

▶ 代码清单 A.84.18 (text, 5 行)

```
Edit  
→  
Preferences  
→  
Add-ons
```

▶ 代码清单 A.84.19 (text, 1 行)

```
VRML
```

▶ 代码清单 A.84.20 (text, 5 行)

```
File  
→  
Import  
→  
VRML
```

▶ 代码清单 A.84.21 (text, 1 行)

```
structure.wrl
```

► 代码清单 A.84.22 (text, 5 行)

添加 Camera
→
调整视角
→
Ctrl + Alt + NumPad 0

► 代码清单 A.84.23 (text, 2 行)

Lens:
35-70 mm

► 代码清单 A.84.24 (text, 1 行)

52 mm → 35 mm

► 代码清单 A.84.25 (text, 1 行)

2400 × 1200

► 代码清单 A.84.26 (text, 1 行)

3000 × 1400

► 代码清单 A.84.27 (text, 1 行)

Camera Lens

► 代码清单 A.84.28 (text, 1 行)

52 mm → 35 mm

▶ 代码清单 A.84.29 (text, 1 行)

Ctrl + Alt + NumPad 0

▶ 代码清单 A.84.30 (text, 5 行)

```
View  
→  
Align View  
→  
Align Active Camera to View
```

▶ 代码清单 A.84.31 (text, 2 行)

View Transform:
Standard

▶ 代码清单 A.84.32 (text, 2 行)

Exposure:
+0.2~0.4

▶ 代码清单 A.84.33 (text, 2 行)

Atom radius:
1.2-1.4 倍

▶ 代码清单 A.84.34 (text, 2 行)

Bond radius:
1.1-1.2 倍

▶ 代码清单 A.84.35 (text, 1 行)

Metallic ≈ 0

► 代码清单 A.84.36 (text, 2 行)

Roughness:
0.2-0.4

► 代码清单 A.84.37 (text, 5 行)

Metallic:
0

Roughness:
0.2-0.4

► 代码清单 A.84.38 (text, 8 行)

Key Light
主光

Fill Light
补光

Rim Light
轮廓光

► 代码清单 A.84.39 (text, 21 行)

1. DFT 得到 POSCAR/CONTCAR
2. VESTA 打开结构
3. 检查元素、晶格和层结构
4. 扩胞
5. 设置 bond
6. 导出 VRML
7. Blender 导入
8. 调整 Camera
9. 调整材质和灯光
10. 高分辨率渲染
11. Illustrator 排版

► 代码清单 A.84.40 (text, 8 行)

结构准确性:
VESTA 保证

视觉表现:
Blender 完成

论文排版:
Illustrator 完成

► 代码清单 A.84.41 (text, 1 行)

VESTA + Blender + Illustrator

复现文献中材料结构的心得

► 代码清单 A.85.1 (python, 196 行)

```
#!/usr/bin/env python3
"""
2H-VSeTe 单层结构 POSCAR 生成脚本
基于文献《Predicted 2D ferromagnetic Janus VSeTe monolayer》的结构参数
Nanoscale, 2020, 12, 22735-22742
"""

import numpy as np
import math

def create_2H_VSeTe_structure():
    """
    创建 2H-VSeTe 单层结构的精确几何参数
    基于文献提供的结构参数和图片描述:
    - 晶格常数:  $a = b = 3.486 \text{ \AA}$ 
    - 键长:  $V-Se = 2.514 \text{ \AA}$ ,  $V-Te = 2.728 \text{ \AA}$ 
    - 垂直距离:  $Se-V = 1.507 \text{ \AA}$ ,  $Te-V = 1.840 \text{ \AA}$ 
    - 真空层厚度:  $16 \text{ \AA}$ 
    - 根据图片 (b), Se 和 Te 在 x 和 y 方向坐标相同, 只有 z 坐标不同
    """

    # 文献中的精确结构参数
    a = 3.486 # 晶格常数 ( $\text{\AA}$ )
    c_vacuum = 16.0 # 真空层厚度 ( $\text{\AA}$ )

    # 六方晶系的晶格矢量
    lattice_vectors = np.array([
        [a, 0.0, 0.0], # a 方向
        [-a/2, a * math.sqrt(3)/2, 0.0], # b 方向 ( $120^\circ$  夹角)
        [0.0, 0.0, c_vacuum] # c 方向 (真空层)
    ])

    # 原子种类和数量 (原胞)
    elements = ['V', 'Se', 'Te']
```

```

num_atoms = [1, 1, 1]

# 精确分数坐标计算
# V 原子位于六方晶系原点, 居中放置
v_position = np.array([0.000000, 0.000000, 0.500000])

# Se 和 Te 原子位置 (根据图片 b, 它们在 x 和 y 方向坐标相同)
# 使用相同的 x,y 分数坐标
x_pos = 1.0/3.0
y_pos = 2.0/3.0

# Se 原子位置 (上层)
z_se = 0.5 + 1.507 / c_vacuum # 基于 Se-V 垂直距离 1.507 Å
se_position = np.array([x_pos, y_pos, z_se])

# Te 原子位置 (下层)
z_te = 0.5 - 1.840 / c_vacuum # 基于 Te-V 垂直距离 1.840 Å
te_position = np.array([x_pos, y_pos, z_te])

# 合并所有原子位置
positions = np.array([v_position, se_position, te_position])

return lattice_vectors, elements, num_atoms, positions

def validate_structure_parameters(lattice_vectors, positions):
    """
    验证生成的结构参数是否符合文献值
    """
    print(" 正在进行结构参数验证...")

    # 转换为直角坐标进行距离计算
    def fractional_to_cartesian(frac_pos, lattice):
        return np.dot(frac_pos, lattice)

    v_cart = fractional_to_cartesian(positions[0], lattice_vectors)
    se_cart = fractional_to_cartesian(positions[1], lattice_vectors)
    te_cart = fractional_to_cartesian(positions[2], lattice_vectors)

    # 计算实际键长
    v_se_distance = np.linalg.norm(se_cart - v_cart)
    v_te_distance = np.linalg.norm(te_cart - v_cart)

    # 计算垂直距离
    v_se_z = abs(se_cart[2] - v_cart[2])
    v_te_z = abs(te_cart[2] - v_cart[2])

    # 计算水平距离 (V 到 Se/Te 投影点的距离)
    v_se_horizontal = np.linalg.norm(se_cart[:2] - v_cart[:2])
    v_te_horizontal = np.linalg.norm(te_cart[:2] - v_cart[:2])

    print(" 结构参数验证结果:")
    print(f" 晶格常数 a = {np.linalg.norm(lattice_vectors[0]):.6f} Å, b = {np.linalg.norm(lattice_vectors[1]):.6f} Å")
    print(f" 真空层厚度 = {lattice_vectors[2,2]:.1f} Å")
    print(f" V-Se 键长 = {v_se_distance:.3f} Å (文献值: 2.514 Å)")
    print(f" V-Te 键长 = {v_te_distance:.3f} Å (文献值: 2.728 Å)")
    print(f" Se-V 垂直距离 = {v_se_z:.3f} Å (文献值: 1.507 Å)")
    print(f" Te-V 垂直距离 = {v_te_z:.3f} Å (文献值: 1.840 Å)")
    print(f" V-Se 水平距离 = {v_se_horizontal:.3f} Å")
    print(f" V-Te 水平距离 = {v_te_horizontal:.3f} Å")

    # 检查 Se 和 Te 的 x,y 坐标是否相同
    se_xy = positions[1][:2]
    te_xy = positions[2][:2]
    xy_same = np.allclose(se_xy, te_xy, atol=1e-6)
    print(f" Se 和 Te 的 x,y 坐标是否相同: {xy_same}")
    if xy_same:
        print(f" Se/Te 共同 x,y 坐标: ({se_xy[0]:.6f}, {se_xy[1]:.6f})")

```

```

# 检查偏差
tolerances = [0.001, 0.01, 0.01, 0.001, 0.001]
targets = [3.486, 2.514, 2.728, 1.507, 1.840]
actuals = [np.linalg.norm(lattice_vectors[0]), v_se_distance, v_te_distance, v_se_z, v_te_z]

all_valid = True
for i, (target, actual, tol) in enumerate(zip(targets, actuals, tolerances)):
    deviation = abs(actual - target)
    if deviation > tol:
        print(f" 警告: 参数 {i+1} 偏差 {deviation:.3f} Å > 容限 {tol} Å")
        all_valid = False

return all_valid

def write_POSCAR(filename, lattice_vectors, elements, num_atoms, positions):
    """
    写入标准 VASP 格式的 POSCAR 文件
    """
    with open(filename, 'w') as f:
        # 文件头
        f.write("2H-VSeTe Janus Monolayer - Generated from literature parameters\n")
        f.write("1.0\n")

        # 晶格矢量
        for vector in lattice_vectors:
            f.write(f"      {vector[0]:16.10f}      {vector[1]:16.10f}      {vector[2]:16.10f}\n")

        # 元素行
        f.write("      " + "      ".join(elements) + "\n")

        # 原子数量行
        f.write("      " + "      ".join(map(str, num_atoms)) + "\n")

        # 坐标类型
        f.write("Direct\n")

        # 原子坐标
        for pos in positions:
            f.write(f"      {pos[0]:16.10f}      {pos[1]:16.10f}      {pos[2]:16.10f}\n")

def main():
    """
    主函数: 生成 2H-VSeTe POSCAR 文件
    """
    print("=" * 70)
    print("2H-VSeTe 单层结构 POSCAR 生成器")
    print(" 基于文献: Nanoscale, 2020, 12, 22735-22742")
    print(" 和图片 (b) 的结构信息")
    print("=" * 70)

    # 生成 2H-VSeTe 结构
    print(" 正在生成 2H-VSeTe 单层结构...")
    print(" 注意: 根据图片 (b), Se 和 Te 在 x 和 y 方向具有相同坐标")
    lattice_vectors, elements, num_atoms, positions = create_2H_VSeTe_structure()

    # 验证结构参数
    print("\n" + "-" * 70)
    is_valid = validate_structure_parameters(lattice_vectors, positions)

    # 写入 POSCAR 文件
    output_file = "POSCAR_2H_VSeTe"
    write_POSCAR(output_file, lattice_vectors, elements, num_atoms, positions)

    print("\n" + "-" * 70)
    print(f"✓ POSCAR 文件已成功生成: {output_file}")
    print(f"✓ 结构类型: 2H-VSeTe Janus 单层")
    print(f"✓ 原胞包含: {sum(num_atoms)} 个原子 ({' + '.join([f'{n}{e}' for e, n in zip(elements, num_atoms)])}")
    print(f"✓ 晶格常数: a = b = {np.linalg.norm(lattice_vectors[0]):.3f} Å")
    print(f"✓ 真空层厚度: {lattice_vectors[2,2]:.1f} Å")

```

```
print(f"✓ 结构特征: Se 和 Te 在 x,y 平面投影重合, 只有 z 坐标不同")

# 结构说明
print("\n" + "=" * 70)
print(" 修改后的结构说明")
print("=" * 70)

print(" 根据图片 (b) 的信息, 我们对结构进行了以下调整:")
print("1. Se 和 Te 原子现在具有相同的 x 和 y 分数坐标")
print("2. 只有 z 坐标不同, 形成垂直堆叠的 Janus 结构")
print("3. V 原子位于中间层, Se 在上层, Te 在下层")
print("4. 这种排列保持了文献报道的键长和垂直距离")

print("\n 几何关系:")
print(" • V 原子: 位于晶格中心 (0,0,0.5)")
print(" • Se 原子: 位于 (1/3,2/3,z_se), z_se > 0.5")
print(" • Te 原子: 位于 (1/3,2/3,z_te), z_te < 0.5")
print(" • Se 和 Te 在 xy 平面投影重合, 形成垂直 Janus 结构")

print("\n" + "=" * 70)
print("POSCAR 文件已准备就绪, 可直接用于 VASP 计算")
print("=" * 70)

if __name__ == "__main__":
    main()
```